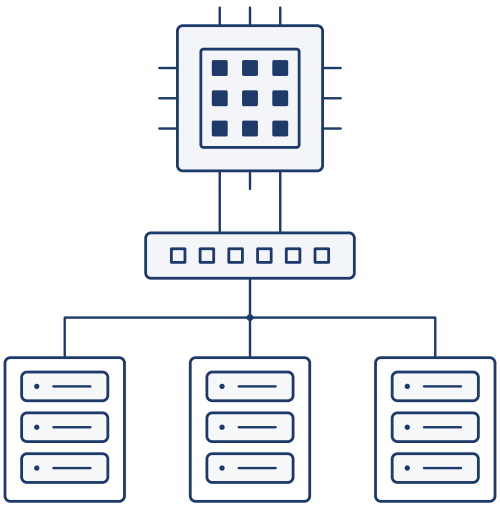
# 深入理解 AI Infra

量化分析与系统设计



李博杰　著

v1.0 • 2026 年 9 月 23 日

# 前言

我的上一本著作《深入理解 AI Agent》讨论 Agent 的架构设计与工程实践。在与读者交流、解答问题的过程中，我愈发意识到，要开发好基于模型的应用，还需要理解这类应用赖以运行的基础设施。一方面是模型本身：模型怎样处理输入、生成输出，并根据上下文与环境交互。另一方面是模型的运行系统：参数和上下文状态存在哪里，计算怎样执行，多个加速器怎样协作，以及模型调用怎样与工具程序衔接。本书由此而来，讨论支撑模型训练与推理的基础设施——AI Infra。

要理解应用开发者为什么也需要这些底层知识，可以看一个熟悉的类比。大多数软件工程师不需要亲自开发操作系统、编译器和芯片，但仍然需要学习操作系统、编译原理和计算机体系结构。这些知识帮助我们理解程序所依赖的抽象，以及抽象背后的实现。申请一块内存、读取一个文件、调用一个函数，看起来只是简单的操作，却各有资源与时间代价。理解这些代价，才能解释程序为什么慢，以及怎样改进。基于模型开发应用，同样需要这样的基础。选择多大的模型、保留多长的上下文、允许多少条请求同时执行、把任务放在本地还是云端，都会改变系统需要完成的工作。

## 编程抽象上移：从操作系统到模型上下文

我在微软亚洲研究院与中科大联合培养读博期间，做的是计算机系统研究，很自然地习惯了从操作系统、编译器和硬件的分工来理解应用。这一领域的两大顶级会议 SOSP（操作系统原理研讨会）和 OSDI（操作系统设计与实现），名字里都包括 “OS”，也就是操作系统。传统操作系统需要支撑多种多样的应用，编译器和硬件则为事先未知的程序提供通用能力。以往我们做系统优化时，总要考虑可编程性与性能的权衡取舍，为了极致性能牺牲可编程性往往不是好的选择。

但如今 LLM 就是最重要的应用，在 AI 推理和训练系统中，从算子执行到分布式调度都可以为特定的模型和加速器架构优化，从而为跨层联合优化打开了新的空间。我认为，**编程抽象从操作系统到模型上下文的迁移，是计算机系统领域数十年来最重要的变化之一。**从某种意义上说，**模型成了 LLM 时代的操作系统，而 AI Infra 成了 LLM 时代的计算机体系结构**。《计算机体系结构：量化研究方法》是我在体系结构领域的入门书，而目前 AI Infra 领域还缺少这样一本从硬件约束和模型架构出发，量化推导系统设计的书。这就是我写作本书的动机。

## 从数量级估算到系统设计

早年做数据中心加速时，无论面对的是模型推理、网络处理，还是存储访问，我都会先做一遍数量级估算：一项任务要完成多少计算，读写多少数据？需要驻留的数据，显存装得下吗？存储和互联带宽能否满足数据供给需求？计算、读写和必须串行执行的步骤，分别给出了怎样的耗时下界？Jeff Dean 所倡导的 back-of-the-envelope estimation，也就是用纸笔做粗略估算，体现的正是这种习惯。几步计算往往就能帮助我们判断一个方向是否值得继续，以及应该先解决哪个问题。

推动我动笔的还有一个更直接的原因。在与 AI Infra 从业者交流时，我发现，不少人熟悉模型和框架，却还没有形成对容量、带宽、算力和延迟的数量级直觉。面对一个设计，能够说出所用的技术，与能够判断它是否可行、性能大致能达到什么水平，是两种不同的能力。我希望这本书能帮助读者建立后一种能力。

在数量级估算上，不仅人容易犯错，AI 也一样。最近，我尝试让 GPT-6 Astra、Claude Fable 5.1 等模型帮忙估算模型推理或训练系统，或者新芯片架构的性能。这些模型能列出公式，算出看似精确的数字，却仍会漏掉决定结果的基本约束：有时只算权重读取，忘了注意力还要读取 KV 缓存；有时估算了读取时间，却没有检查权重、KV 缓存和运行时工作区合计是否放得进显存；有时按峰值算力推算速度，却没有检查存储带宽能否

持续供给数据；有时把工作量平均分给多张卡，却遗漏卡间通信；还有时算出了很高的吞吐量，却没有考虑串行依赖与通信往返的延迟。漏掉这些约束中的任何一项，结论都可能偏离几倍甚至几个数量级。

因此，我希望贯穿本书的方法是：**从约束推导设计。**先明确任务与质量要求，再列出计算、存储、通信和依赖关系，检查显存等存储资源的容量是否足够、存储和互联带宽是否满足计算的数据需求、哪些依赖造成的等待无法消除，然后讨论模型怎样分工、状态放在哪里、执行如何组织。把已知的基本约束逐一纳入，才能排除明显不可能的方案；实际系统中的开销和变化，则要继续用测量来校正。估算不必一开始就精确，但必须知道自己算进了什么，还有什么没有算进去。

## 从算子加速到万卡互联

约束是设计之源。2016 年，我在微软亚洲研究院实习时，和团队一起探索用 FPGA 加速微软 Bing 搜索排序所用的深度神经网络。模型权重反复从片外内存搬进芯片，限制了计算速度。我想到，能否把模型拆到多块 FPGA，让各部分留在各自的片上存储中，只通过高速网络传递中间结果？我兴奋地把想法告诉徐宁仪老师，他说，这叫“模型并行”。计算放在哪里、数据怎样流动、专用化能省下什么，已经是我当时反复思考的问题。那几年，导师张霖涛博士反复叮嘱：优化一定要做到物理所允许的极限。后来我在 KV-Direct 论文中写下 “close to the physical limits of the underlying hardware”，也由此养成了一个习惯：先按第一性原理算出硬件允许的上限，再看系统离上限还有多远。这个习惯贯穿了我后来的工作，也贯穿本书。

2019 年加入华为后，我参与了深度学习框架 MindSpore 的自动算子生成项目 AKG。这项工作延续了此前的软硬件协同思路：把模型计算转换为适合昇腾 NPU 执行的程序，由编译器安排数据分块、存储和执行顺序。我还记得，为了融合 softmax 算子，我到处寻找合适的算法。读到 NVIDIA 在 2018 年发表的在线 softmax 研究后，我终于找到了实现融合的办法。再后来我才意识到，自己当时竟然摸到了后来注意力算法 FlashAttention 的一条核心思路：把在线归约与分块、融合结合起来，减少中间结果的存储和读写。

到了 2020 年，我加入 Unified Bus（UB，统一互联）项目，开始研究更大规模的加速器协作：怎样让成千上万个处理器高效地共同完成一项计算？

最早在华为内部推动这件事时，我们遇到了不少来自其他部门的质疑。当时有人问：“现在模型最多就八卡训练，你搞一个万卡的，什么时候才能有一万张卡啊？”那时候，整个公司都没有一万张卡。对习惯了单机多卡训练的人来说，为万卡规模设计互联，确实离眼前的需求很远。

我们之所以作出这一判断，是因为看到了另一组证据。2019 年，谭焜博士把 AI 算力需求的发展趋势画成散点图，看到需求的增长远快于单芯片能力的提升，于是推动了面向上万张卡的高性能互联研究。2020 年我加入项目，GPT-3 论文的发表又为这一方向提供了新的论据。如果模型训练需要的计算量继续这样增长，就必须让更多加速器协同工作；互联架构的研究和实现需要时间，等需求完全显现以后再开始，很可能已经来不及。

如今，UB 已应用于昇腾 910C 和 950 系统。截至本书写作时，基于 UB 的 NPU AI 训练集群架构是国内唯一支持万卡以上规模的架构。今天回头看，万卡训练的必要性已经不难理解。系统设计需要同时看清眼前的负载和正在变化的条件。八张卡能够完成当时熟悉的任务，是一种实际经验；更大的计算需求会推动更大规模的协作，是对未来的判断。要说服别人，就需要把这一判断所依赖的趋势、资源和代价讲清楚。

从模型推理加速到 AKG，再到 UB，这些工作跨越了不同的尺度，我反复遇到的却是相通的问题：计算需要的数据在哪里，数据搬移的开销是否压过了必要的计算？如果修改系统和应用的抽象，让系统掌握更多来自应用的信息，是否可以降低数据搬移的开销？这些工作都始于某个比例的变化：片外存储的读取速度满足不了计算需求，促使我们用 FPGA 加速 Bing 排序；算力需求的增长快于单芯片能力的提升，促使我们推动万卡互联。

2023 年离开华为创业以后，这些问题又以另一种方式出现在我面前。我们做的 Agent 需要与人实时语音交

互。用户说完一句话，要等多久才能听到回答？一通电话持续半个小时，服务成本又是多少？

2024 年，我们在 GPT-4o 发布之前就推出了实时语音交互。后来与基于 GPT-4o 的实时语音 API 比较，我们的运行成本约为调用它的百分之一。我还记得，2023 年底第一个语音电话演示的延迟高达 5 秒。我们把整条交互管线拆开，先估算模型一次推理需要多少计算、读取多少数据，再不断改进；随后继续优化网络传输、数据库访问等等。延迟就这样从 5 秒降到 2.5 秒、1 秒，后来做到约 500—600 毫秒。

这段经历让我认识到，应用开发与基础设施之间有很直接的联系。同样是实时语音，延迟相差几倍，产品体验就可能完全不同；成本相差一个数量级，能够支持的商业模式也会随之改变。

## 我希望这本书讲清楚什么

应用与基础设施之间的这种联系，在模型研发内部同样存在。在优秀的基础模型团队中，我越来越常看到一种跨越分工的工作方式：**最懂 Infra 的人做算法，最懂算法的人做数据，最懂数据的人做 Infra。**这句话表达的是一种相互理解的深度。算法设计者知道硬件的容量、带宽和通信限制，才能在模型结构中利用这些条件；数据工作者理解模型怎样学习，才能判断哪些样本和训练任务最有价值；Infra 工程师了解数据的组织、长度和使用过程，才能把系统资源真正用在有效的训练与推理上。

近年来的模型设计中已经出现了这种跨层配合。DeepSeek V4 和 V4.1 是很好的例子。V4 在模型架构中就充分考虑了 Infra 的效率：将局部窗口、上下文压缩和稀疏选择结合起来，减少长上下文需要保存的状态和反复读取的数据。显存容量、存储带宽与计算之间的约束，直接影响了模型怎样表示和使用上下文。

V4.1 Flash 更进一步，用非对称的因果编码器—解码器（Causal Encoder-Decoder，CED）架构重新划分理解与生成的职责，针对 Agent 输入多、输出相对少的负载重新安排计算投入，说明系统效率的要求能够推动模型架构本身的创新。

我希望读者也能建立这种跨层理解：看到一种模型结构时，能想到它要求加速器做哪些工作；看到一种硬件能力时，能判断怎样改变模型与执行方式，才能真正把它用起来。

贯穿这些分析的一条线索，是**数据搬移**。计算单元需要取得权重和中间结果，多个加速器需要交换各自算出的数据，远端服务需要接收输入并返回结果。我们看到的是模型在生成答案，底层发生的却是一次次读取、计算、保存和交接。数据放在哪里、重复使用多少次、需要经过哪些接口、后续工作是否必须等待，都会影响执行效率。

因此，本书反复追问五个问题：**搬什么、搬多少、搬几次、经过哪里、谁必须等它。**从芯片上的存储层次，到超节点（通过高带宽互联紧密协作的一组加速器）和数据中心网络，再到终端、边缘与云（端边云）之间的任务分工，这五个问题都能帮助我们找到需要计算的量。保留数据可以减少重复计算，却会占用容量；扩大并行规模可以分担工作，却会增加通信；减少传输可能需要更多本地计算，也可能改变结果质量。

要回答这五个问题，就需要把每个量都算出来，这也是本书采用“量化分析与系统设计”作为副标题的原因。《计算机体系结构：量化研究方法》为这种分析方式提供了很好的示范。本书也希望把前面所说的估算习惯落实到每一层设计中：从模型的工作量出发，对照资源约束，解释为什么选择一种执行方式，以及条件改变以后应该如何重新选择。

## 全书结构

十二章按照“理解工作需求—认识执行资源—组织完整系统”的顺序展开，如图 0-1 所示。第一部分是模型与负载（第 1—3 章），建立分析方法，说明计算量、数据量和任务依赖从哪里来。第二部分是芯片与系统（第

4—7 章），从单加速器执行走向多加速器协作，说明资源怎样承担这些工作。第三部分是推理与训练系统（第 8—12 章），研究如何组织请求、模型状态和训练过程，并延伸到任务运行环境与端边云部署。

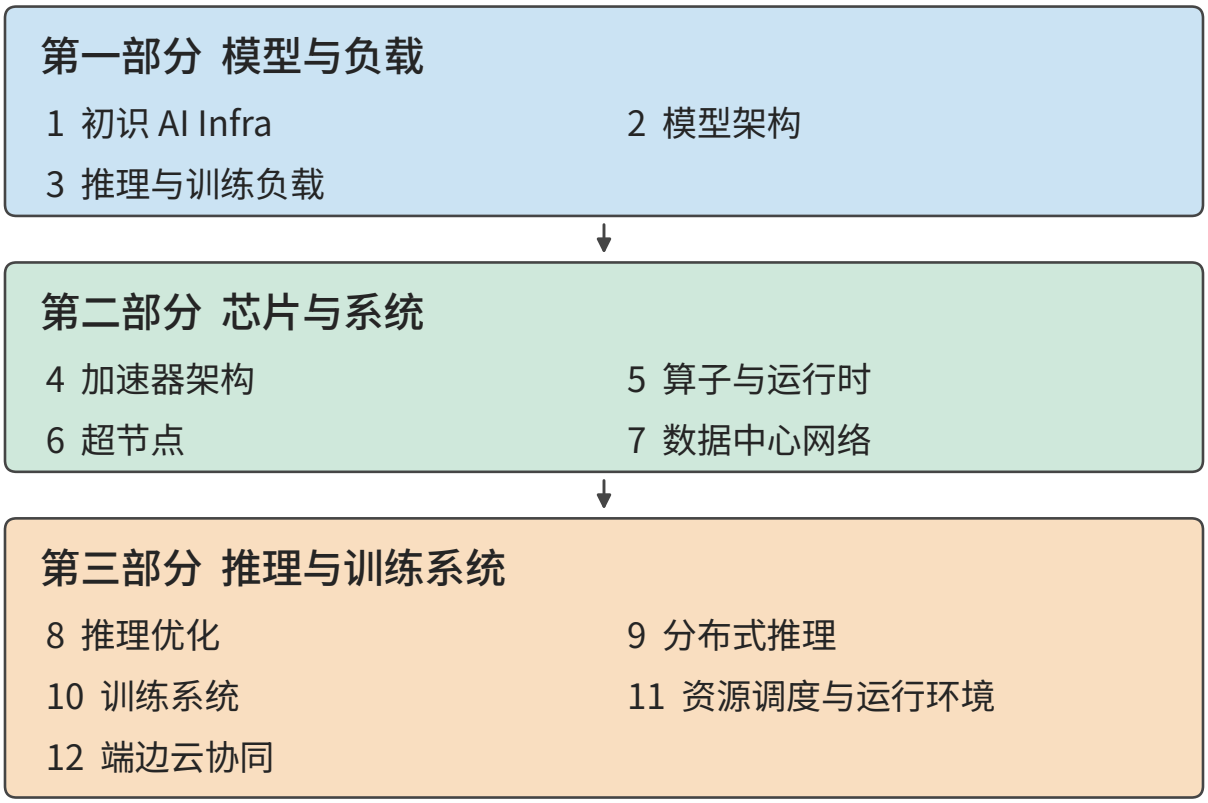



图 0-1　全书结构与阅读顺序。前一部分为后一部分提供分析依据；进入服务与部署之后，还要根据任务质量、完成时间和成本，重新检查模型与资源选择。

下面是每章要回答的主要问题。你可以先在这里找到自己关心的问题，再看看前面的章节为它提供了哪些基础。

**第一部分：模型与负载**

- **第 1 章初识 AI Infra**：如何认识系统全景，并用几个关键数字估算一次模型执行？
- **第 2 章模型架构**：模型的计算、参数和上下文状态如何形成资源需求？
- **第 3 章推理与训练负载**：请求到达、多轮调用、多模态输入和训练过程如何改变资源需求与等待时间？

**第二部分：芯片与系统**

- **第 4 章加速器架构**：芯片的计算、存储和数据通路如何配合，适合什么负载？
- **第 5 章算子与运行时**：如何组织算子与执行过程，减少重复读写、提交和等待？
- **第 6 章超节点**：模型如何在多个加速器间分工，协作规模应当多大？
- **第 7 章数据中心网络**：数据如何跨加速器传输，交接规则、拥塞和故障如何影响计算？

**第三部分：推理与训练系统**

- **第 8 章推理优化**：如何安排批处理、请求和缓存，提高给定加速器组合的服务效率？
- **第 9 章分布式推理**：计算与状态应当放在哪里，如何组织分工、共享和扩缩容？
- **第 10 章训练系统**：如何安排训练状态、通信与恢复，在期限内取得有效训练进展？
- **第 11 章资源调度与运行环境**：如何把模型服务、工具环境和共享资源组织成完整的任务系统？
- **第 12 章端边云协同**：如何结合实际传输与交互要求，选择任务在端、边、云的部署位置？

随着阅读推进，对同一个问题的分析会不断加入新的条件：最初只考虑权重容量，后来还要计算上下文状态；最初比较单次执行，后来还要面对并发、交接和故障恢复。我希望你能跟着这些变化，回头检查自己在前面作出的判断。这也是我把模型、芯片、网络和服务系统放在一本书里讲的原因：它们共同决定了一项任务怎样完成。

## 如何阅读本书

如果你希望从头建立系统认识，我建议先读第 1—3 章，学会估算一次执行，弄清模型与负载提出了哪些需求，再依次阅读后面两部分。不同背景的读者，也可以在这一共同基础上选择自己的重点。

- **模型与应用开发者**可以重点阅读第 8、9 章的推理服务，以及第 11、12 章的任务环境和部署。遇到容量、算子或通信问题时，再回到第 4—7 章追踪原因。
- **系统与网络工程师**可以重点阅读第 5—7 章的执行与加速器协作，再看第 9、10 章，检查这些机制如何影响分布式推理和训练。
- **芯片与体系结构工程师**可以重点阅读第 4—7 章，并结合后续章节的任务算例，检查自己熟悉的硬件指标如何转化为实际的服务能力。

无论选择哪条阅读路径，我都尤其建议你在看答案之前，先自己估一下。拿一张纸，写下数据量、处理能力和必须等待的步骤，通常就能发现一些值得追问的地方。如果结果与书中不同，先检查双方是否使用了相同的条件；如果条件相同，就继续寻找漏算或重复计算的工作。这种来回检查，往往比直接记住一个结论更有收获。

书中的练习分为核心与延伸，核心练习帮助你完成本章的主要推导，延伸练习可按兴趣和项目需要选择。配套实验、计算工具与参考资料整理在本书的开源仓库中：

**配套开源仓库：** https://github.com/bojieli/ai-infra-book

仓库中有三类材料，建议与正文配合使用：

- **实验与计算。**按章节组织的实验提供运行说明、输入条件和结果记录；配套计算工具可以帮助你复算书中的数字，或换一组条件，观察结论怎样变化。
- **论文与技术资料。**相关论文、开源软件和官方技术文档的索引与来源记录，便于沿着书中的问题继续阅读，并核对具体论据。
- **芯片与模型参数表。**汇集调研中整理的硬件规格与模型配置，包括算力、存储容量与带宽、互联能力，以及模型层数、维度、混合专家模型的专家配置和上下文状态等估算所需的信息。参数表与来源、计算工具配合使用，帮助你检查数字的适用条件，再将它们代入自己的分析。

可以从仓库首页进入实验、量化计算和参考资料目录，按各项目的说明运行。需要特定加速器的实验会注明环境和加速器条件；没有相应加速器时，也可以先分析已有记录，再改变输入重新计算。模型和硬件不断更新，这些材料也会随书稿继续补充和修订。

读这本书时，你也可以带着自己的模型、机器和业务问题。把书里的输入替换掉，看看结论是否改变，再决定下一步值得测什么。我希望这些例子能够成为你分析自己问题的起点。

## 前置知识

本书面向有编程经验、希望理解模型执行与系统设计的读者。下面几项基础会帮助你跟上书中的推导和实验。

- **编程与工具。**能读懂和修改简单的 Python 程序，熟悉命令行和基本的依赖安装。基础练习从手算、小脚本和已有实验记录开始，使用特定加速器的实验会另行说明条件。
- **数学。**理解向量、矩阵乘法和基本代数，会做单位换算。平均值与概率的基本知识有助于分析负载和等待；导数与梯度的直觉有助于阅读训练章节。
- **计算机系统。**了解进程、内存、文件与网络通信的基本用途。模型结构、并行方式和专用硬件机制会随问题逐步介绍。

如果你已经调用过模型 API，或者在自己的机器上运行过一个模型，可以把这些经验带进书里的算例：为什么长上下文变慢，为什么增加并发后每个人等得更久，为什么换了加速器却没有得到预期的加速？这些都是很好的阅读起点。

不必等到熟悉所有层次才开始阅读。做系统研究和工程实践时，我一直需要围绕具体问题，学习相邻层次的知识。遇到陌生的概念，可以先抓住它要解决的问题，再回来看实现细节。

我希望你读完后，面对一个新模型、一款新加速器或一种新的部署要求，能够画出执行过程，算出几个关键数字，指出最需要验证的假设，并根据观察修改自己的选择。你的结论可以与书中不同；能够说明差异来自哪里，本身就是理解系统的表现。

## 致谢

首先，感谢 GPT-6 Astra 模型。很久以前，我就想写这样一本书，把这些年对计算机系统和 AI Infra 的思考整理下来，却一直没有找到足够的时间。前段时间，我终于把自己的思考蒸馏成了一份提纲。GPT-6 Astra 发布后，我让 Agent 连续工作了一周，帮助我调研模型相关的论文、梳理开源软件的进展、做实验，再逐步写成书。目前你看到的版本还只是初稿。我还在不断蒸馏自己的想法，修改书中的内容。

感谢这些年在网络、系统和 AI Infra 领域与我一起做研究、合作发表论文的各位合作者。也感谢在微软、华为工作期间，在可编程网卡加速、算子生成、Unified Bus 大规模网络互联等项目中给予我指导、与我并肩工作的领导、专家和同事。这里无法一一列举大家的名字，但从提出假设、理论分析、实验验证，到真正做成生产级的系统，我从大家身上学到了太多。这本书里的许多思考，都离不开那些一起讨论、推导、实验和解决问题的日子。

最后，要感谢我的太太孟佳颖。和写《深入理解 AI Agent》时一样，她始终支持我把想做的事情做完。这几天，她还把自己的 Codex token 额度让给了我，让我有充足的 token 推进这本书的写作。

# 目录

**前言** **I**
编程抽象上移：从操作系统到模型上下文 . . . I
从数量级估算到系统设计 . . . I
从算子加速到万卡互联 . . . II
我希望这本书讲清楚什么 . . . III
全书结构 . . . III
如何阅读本书 . . . V
前置知识 . . . V
致谢 . . . VI

**第 1 章 初识 AI Infra** **1**
1.1 AI Infra 为什么重要 . . . 1
1.2 系统设计的关键数字 . . . 6
1.3 用几个数字估算一次模型执行 . . . 11
1.4 需求如何推动架构设计 . . . 17
谬误与陷阱 . . . 20
本章数字速查 . . . 21
本章小结 . . . 21

**第 2 章 模型架构** **23**
2.1 计算图与前向执行 . . . 23
2.2 自回归生成的资源消耗计算 . . . 31
2.3 上下文状态表示与访问机制 . . . 34
2.4 条件计算与专家权重复用 . . . 47
2.5 网络形状与计算组织 . . . 50
2.6 整个模型的资源需求与比较 . . . 52
谬误与陷阱 . . . 64
本章练习与复现 . . . 65
模型矩阵查阅表 . . . 65
本章小结 . . . 73

**第 3 章 推理与训练负载** **74**
3.1 从单次请求到持续到达的负载 . . . 74
3.2 多轮交互与 Agent 任务 . . . 80
3.3 多模态与实时交互 . . . 87
3.4 训练的计算与状态需求 . . . 92
3.5 强化学习（RL） . . . 96
3.6 从负载需求到训练与服务预算 . . . 100
谬误与陷阱 . . . 106
代码任务轨迹 . . . 107
本章小结 . . . 107

**第 4 章 加速器架构** **108**
4.1 从模型计算认识加速器 . . . 108
4.2 计算单元 . . . 117
4.3 存储层次 . . . 123
4.4 数据搬移 . . . 128
4.5 多芯片封装与互联 . . . 133
4.6 模型需求如何推动架构演进 . . . 138
4.7 专用架构 . . . 145
4.8 根据架构分析性能、选择加速器 . . . 149
本章小结 . . . 156

**第 5 章 算子与运行时** **157**
5.1 加速器执行过程中的提交、搬移与同步 . . . 157
5.2 单个算子的分块、布局与实际访存 . . . 164
5.3 相邻算子的融合、缓冲与流水执行 . . . 172
5.4 编译器：表达、变换与选择 . . . 179
5.5 运行时：提交、重放与动态形状 . . . 185
5.6 从局部优化到完整请求 . . . 193
常见误区 . . . 195
习题与配套实验 . . . 196
历史与进一步阅读 . . . 198
本章小结 . . . 198

**第 6 章 超节点** **199**
6.1 从单卡到多卡 . . . 199
6.2 六种并行方式 . . . 203
6.3 组合并行与模型规模 . . . 215
6.4 集合通信的实现与执行代价 . . . 219
6.5 超节点的物理组织 . . . 226
6.6 内存池 . . . 238
6.7 从并行方案到超节点规模 . . . 241
本章小结 . . . 251

**第 7 章 数据中心网络** **253**
7.1 从超节点走向集群 . . . 253
7.2 跨节点流量与物理路径 . . . 258
7.3 远程访问的路径与并发 . . . 269
7.4 数据交接、顺序与状态管理 . . . 280
7.5 共享网络中的拥塞与可靠性 . . . 288
7.6 从通信改进到任务完成 . . . 298
本章小结 . . . 307

**第 8 章 推理优化** **309**
8.1 推理请求的执行过程与资源需求 . . . 309
8.2 批处理与请求调度 . . . 312
8.3 KV 缓存的分配、复用与释放 . . . 319

8.4 压缩与卸载 . . . 328
8.5 推测解码 . . . 334
8.6 请求负载下的性能与配置选择 . . . 339
历史与延伸阅读 . . . 344
本章小结 . . . 345

**第 9 章 分布式推理 346**
9.1 计算与状态的分布方式 . . . 346
9.2 Prefill–Decode 分离与资源配比 . . . 350
9.3 Attention–FFN 分离与异构执行 . . . 357
9.4 专家放置、复制与负载均衡 . . . 364
9.5 KV 的分布、共享与请求路由 . . . 372
9.6 服务启动、扩缩容与故障恢复 . . . 382
9.7 部署方案的综合比较 . . . 386
练习 . . . 390
本章小结 . . . 391

**第 10 章 训练系统 392**
10.1 根据训练任务估算容量与时间需求 . . . 392
10.2 训练状态的分片、重计算与卸载 . . . 398
10.3 训练步的流水调度与通信重叠 . . . 406
10.4 数据输入、checkpoint 与故障恢复 . . . 417
10.5 强化学习训练 . . . 424
10.6 从系统方案到完成期限与硬件选择 . . . 432
常见误区 . . . 436
习题与实验 . . . 436
本章小结 . . . 438

**第 11 章 资源调度与运行环境 439**
11.1 根据任务执行过程计算资源需求 . . . 440
11.2 运行环境的创建、保留与释放 . . . 444
11.3 共享资源池的分配与调度 . . . 455
11.4 模型服务的选择与调用成本 . . . 465
11.5 完整任务的恢复、成本计算与扩容 . . . 471
练习与实验 . . . 479
本章小结 . . . 480

**第 12 章 端边云协同 481**
12.1 从模型运行时间到完整交互时间 . . . 481
12.2 跨设备分工 . . . 490
12.3 广域网传输中的额外开销与等待 . . . 496
12.4 无线变化下的交互与恢复 . . . 504
12.5 从端边云分工到完整部署 . . . 508
练习 . . . 515
本章小结 . . . 517

# 第 1 章 初识 AI Infra

前言介绍了本书的写作缘起、章节安排和阅读方法。本章起沿着模型的执行过程，认识支撑这一过程的系统。**AI Infra** 是支撑 AI 训练和推理的基础设施，包括计算设备、存储与互联，以及组织这些资源的软件。训练通过样本调整模型参数；推理使用训练得到的参数处理输入、产生输出。

本章先说明应用如何通过代码、模型与上下文共同表达任务，再沿一次请求的处理过程认识系统，用容量、算力和带宽回答具体的设计问题：数据能否放下，回答要等多久，增加加速器或改变数据表示能带来多少收益。最后回顾几个真实架构的形成过程，说明需求如何推动系统设计。

## 1.1 AI Infra 为什么重要

### 1.1.1 编程抽象上移：从操作系统到模型上下文

以文档问答服务为例。开发者先规定文档读取、检索和结果展示的程序流程，再把用户问题与检索到的材料交给语言模型生成答案。**上下文**（context）是本次模型调用可以使用的输入内容，包括任务指令、参考材料和此前的交互结果。保持模型参数不变，改变指令、示例或参考材料，就能改变模型处理的任务及回答方式。

若让该服务检查代码，可以把代码、测试要求和可调用的工具一同提供给模型，由模型提出修改方案或发起工具调用；应用程序执行测试，把结果加入下一次调用的上下文，再由模型继续判断。工具是供应用调用的程序功能，例如读取文件或运行测试。模型负责选择下一步做什么，宿主程序负责检查权限、执行操作和保存结果。完成一次任务可能需要多次模型调用和工具执行。

**抽象**是把复杂实现封装在明确的接口之后，让开发者用较少的概念组织程序。**可编程性**是开发者通过某种接口改变系统行为的能力。传统应用主要用程序代码表达行为，编译器将代码转换为可执行指令，操作系统管理程序运行所需的资源。模型驱动的应用增加了另一个重要入口：通过选择模型、组织上下文和提供工具来表达任务。一部分原来需要逐条编写规则才能实现的行为，现在由模型根据上下文决定；代码继续组织调用流程，工具程序继续通过操作系统访问文件、网络和设备。[1]

图 1-1 对照了这两种组织方式。右侧的 Agent 负责组织上下文、调用模型，并执行模型选择的工具。其中，**模型接口**规定应用如何提交输入、取得输出。接口背后的运行系统计算模型输出，并保存后续步骤需要复用的上下文状态；加速器是适合并行执行大量模型运算的处理器，内存保存参数与运行状态，互联在设备之间传递数据。开发者从上层改变任务，底层要把这些变化落实为具体运算与数据访问。

可编程性的变化也改变了模型服务的运行方式。在 AI 系统中，同一个模型为不同应用处理请求时，使用的是同一组参数。系统可以将多条请求合并计算，共用一次读入的参数，提高加速器利用率。这种方式称为批处理，每批共同执行的请求数称为 batch size（批次大小）。与此同时，长文档、长回答和多轮工具调用会改变计算量、状态占用和等待时间。同一个模型、同一套加速器，处理不同任务时，性能可能相差甚远。

这种性能差异的背后，是模型与硬件之间的相互影响。从模型设计来看，压缩保存的状态、改变注意力机制或只使用部分模型参数，可以改变硬件需要完成的工作；从硬件条件来看，存储容量、数据传输速度和设备连接方式，又会影响哪些模型结构和执行方法更合适。理解这种双向关系，才能同时讨论回答质量、响应时间和成本，并解释为什么适合一个应用的方案，用于另一个应用时还需重新评估。

[1] 相关研究见抽象边界调研；基础模型承接多种下游任务的讨论见 Stanford CRFM。

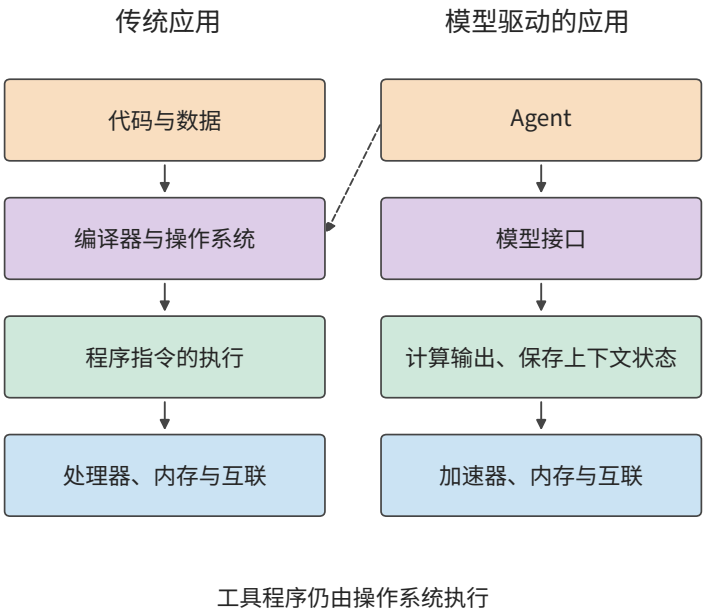



图 1-1　应用行为的表达方式与执行路径。左侧通过代码组织程序，右侧通过上下文调用模型；实线表示向下的执行依赖，虚线表示工具执行仍需传统操作系统。加速器、内存与互联分别承担模型计算、数据存储和设备间传输的职责。

在以模型训练和推理为主要应用的系统中，各层可以围绕同一个完整的执行过程共同设计。模型结构给出计算和数据依赖，运行时掌握状态何时产生、何时使用，硬件承担相应的运算和传输。利用这些信息重新组织执行，可以消除通用实现中的部分开销。同一模型又能根据不同上下文处理多种任务，既能支持多样的应用行为，又能让底层针对模型执行做专门优化。**上层可编程性的变化，为跨层联合优化打开了新的空间。**本书用量化分析研究这些优化机会，解释原有的设计取舍如何改变，新的实现又如何提高训练和推理的性能。

### 1.1.2 从任务到硬件的六层分工

一次请求从提出到完成，要依次经过系统的多个层次。用户提交一段代码，请模型找出错误。应用把代码和问题交给模型服务，服务选定执行请求的加速器，加速器读入权重与输入、完成计算，再把生成的回答交回应用。若多个加速器共同执行，还要在加速器之间传递中间结果。沿这条路径，就能逐步找到等待发生在哪里、哪份数据需要搬移。

华为半导体首席科学家廖恒博士提出“十八层宝塔”，用来描述从应用、模型与软件系统，到芯片、制造工艺和底层物理的多个层次。他指出，大多数人只了解其中的一层或几层，因而错失了许多跨层联合优化的机会。[2] 图 1-2 借鉴这一方法，将本书内容划分为六个层次：

最上面是**应用与任务**。对话、代码生成、语音交互，以及能够调用工具推进任务的智能体（Agent），都在这一层提出具体要求：要完成什么任务，多快返回结果，成本上限是多少。训练也有自己的任务目标，例如在给定时间内完成一次模型更新或一轮预训练。

下一层是**模型与负载**。模型结构规定要做哪些计算、保留哪些数据；实际负载则决定这些工作何时到来、持续多久。一句简短的提问和一篇长文档，即使交给同一个模型，也会形成不同的计算与存储需求。模型结构在第 2 章展开，训练与推理负载在第 3 章展开。

**训练与推理系统**负责把这些需求组织成可执行的工作。这一层接收请求或作业，组织 batch，管理运行状态，

[2]“十八层宝塔”由华为半导体首席科学家廖恒博士在 2026 年 7 月的一次公开长访谈中提出，用来描述从应用到制造工艺的层层依赖。

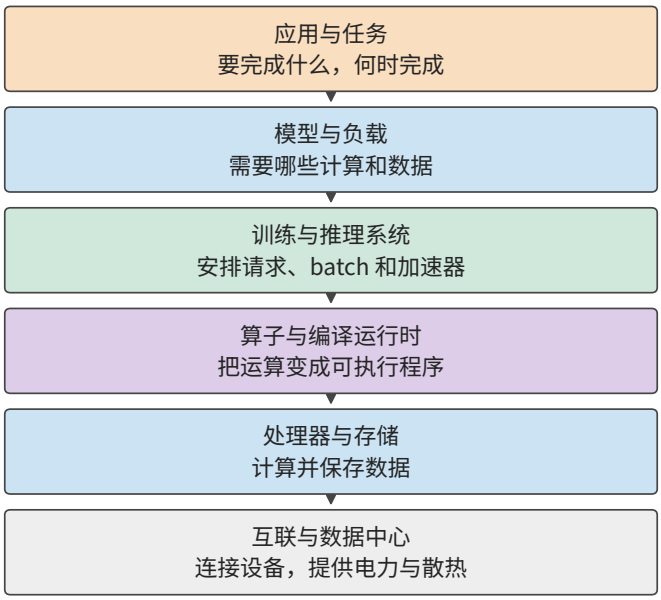



图 1-2　从应用到硬件的六层分工。

并决定如何使用一个或多个加速器。在推理中，它要协调正在生成的请求和新进入的请求；在训练中，它还要组织参数更新、通信和恢复。第 8—10 章讨论这一层。

再向下是**算子与编译运行时**。模型中的矩阵乘法、归一化（按统计量调整数值尺度）等操作，需要转换为处理器能够执行的程序。本书把矩阵乘法等基本运算称为算子；算子库提供这些运算的实现，编译器把程序转换为加速器可执行的形式，运行时负责提交程序和管理执行中的资源。三者共同决定如何分块、如何使用临时存储、何时提交工作。相同的数学表达式，可以有不同的数据访问和执行方式，第 5 章会具体分析这些差异。这里还有理解分布式并行的一条线索：矩阵在一张卡内可以分块执行，模型在多张卡上也可以按样本、序列、特征、层或专家（混合专家模型中按输入逐个选用的子网络）分工。两者都要先确定每块由谁计算，再安排输入到达与结果汇合。区别在于块之间跨越的是片内存储、显存还是网络。第 6 章将沿这条线索解释各种并行策略，而非将其视为互不相关的技巧。

**处理器与存储**负责执行计算、保存数据。CPU 是执行通用程序的中央处理器；GPU 是擅长并行处理大量相似运算的图形处理器，如今也广泛用于模型计算；NPU 是面向神经网络运算设计的处理器。主存存放 CPU 使用的数据，显存存放 GPU 使用的数据，片上存储则位于处理器芯片内部，让计算单元能就近读取数据。计算单元先读取输入，再执行运算，最后写回结果。第 4 章将从计算能力与数据读取速度的关系出发，解释加速器的设计。

最下面是**互联与数据中心**。设备之间通过互联交换数据，服务器和更大的协作组通过网络连接，供电与散热决定资源能够如何部署。第 6、7 章讨论超节点与数据中心网络，第 12 章进一步把执行位置扩展到端、边和云。

六个层次说明了主要分工，但有些能力贯穿多个层次。资源调度、运行环境、观测和计费需要结合多个层次的信息来组织；工艺、封装、供电和散热则决定设备能够如何制造和部署。网络也并非只在最下方发挥作用：需要跨设备协作的模型、算子和服务，都要经过具体互联。

沿这六层向下看，任务逐步变成具体的计算与数据访问；向上看，加速器的可用存储容量、算力与带宽决定系统能运行多大的模型、同时处理多少请求，以及能多快作出响应。这些层次之间的接口，也是联合优化的切入点。模型改变状态表示，会改变存储和通信需求；编译器将相邻算子融合，会改变中间结果的存放位置；运行

时让加速器直接交接，会改变主机参与的次数。

### 1.1.3 一次生成请求的处理过程

**推理实例**是能够独立完成模型请求的一组执行资源，可以由一个或多个加速器组成。每次请求交由其中一个实例处理。图 1-3 展示了请求分配与实例内部执行的分工：服务路由器是选择请求执行位置的软件；路由器选定实例后，由实例内部安排加速器执行模型。先分清谁负责执行，才能知道一份权重应放在哪里、一份中间结果应传给谁。

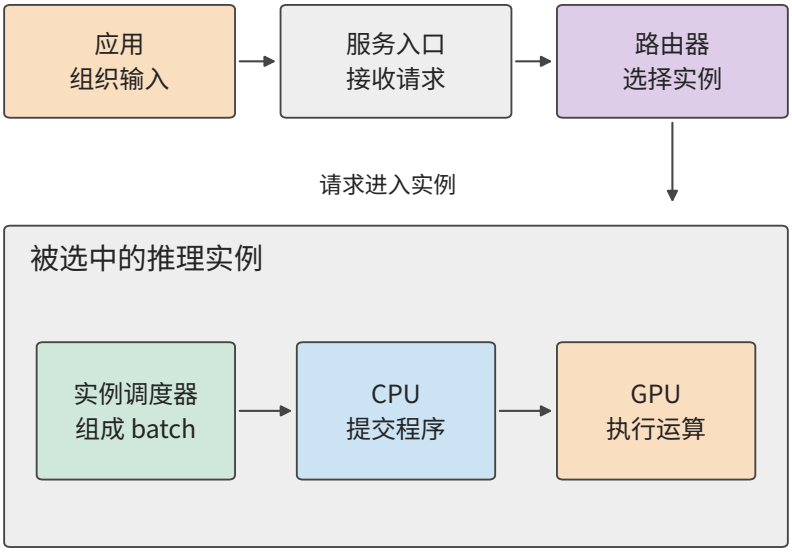



图 1-3　一次请求先由路由器选择推理实例，再由实例内部的调度器安排执行。灰色外框圈出同一个实例，实线表示请求和工作提交的方向。

上下文包括用户输入、已有对话、检索结果和此前生成的内容。应用把这些信息组织成请求，交给模型服务入口。入口核验访问权限、检查配额和请求内容后，路由器依据目标模型、实例负载等条件选择一个实例；实例可以运行在一张卡上，也可以分布在多张卡上。选定实例之后，实例内部再安排这些卡协作。

文本随后转换成 token 序列。词元（token）是模型处理文本的基本单位，可能对应一个字、词的一部分或其他片段，具体文本的 token 数由分词结果确定。文本转换可以由实例或共享前端完成。实例调度器将请求放入队列，再编入 batch，并为其状态分配空间。CPU 上的运行时随后向 GPU 提交计算任务，GPU 执行相应算子。[3]

模型先处理输入，这一阶段称为预填充（prefill）；随后逐步生成新 token，这一阶段称为解码（decode）。后续步骤会复用此前保存的状态，其中的 KV 缓存（KV cache）保存上下文 token 产生的键和值两类向量，供后续位置的注意力计算使用。键和值是注意力机制使用的数据表示，第 2 章将结合具体运算说明键和值如何产生。生成循环由实例内部的调度器持续推进，前一步输出成为下一步输入。产生的 token 经输出处理转换为文本，沿连接流式返回；如果输出要求调用工具，应用运行工具，把结果加入上下文，再发起后续调用。

上述每步计算反复使用的数据，首先是模型权重。模型权重是训练得到、用于把输入变换成输出的数值参数。权重在实例启动或模型切换时从存储加载，随后持续存放在显存中（称为驻留），供多次请求使用。生成时，GPU 计算单元从显存读取当前层的权重和上下文状态，完成运算，再将结果交给下一层。同一份数据由此经历

[3]请求图是通用职责示意；调度与状态管理的职责划分可参照 v0.26.0 的 vLLM Scheduler 文档；1.3.3 节的实测使用 vLLM 0.23.0。路由策略、tokenizer 位置以及 prefill / decode 是否分离属于部署选择。

两种频率不同的搬移：加载把模型送到加速器，执行则反复把所需数据送到计算单元。在多卡实例中，一张卡算出的中间结果还要传给其他卡，后续计算才能继续。中间张量是运算产生并交给后续运算的多维数值数组，矩阵就是二维张量。**外部请求可能只有一小段文字，内部搬移的数据却包括大得多的权重、状态和中间张量。**

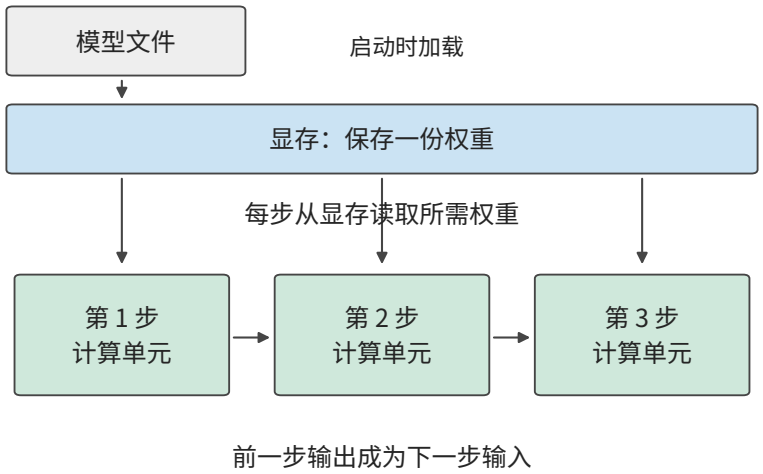



图 1-4　权重加载一次，生成时逐步读取。蓝框表示显存中持续保存的同一份权重，三个绿框表示先后发生的计算；箭头表示读取或结果依赖，步骤间距不代表耗时。

图 1-5 展示这些软件功能对应的硬件。数据中心包含入口与 CPU 服务、共享存储，以及由多个超节点构成的加速器资源池。超节点是一组通过高带宽互联紧密协作的加速器，可以分布在多个服务器或计算托盘（机柜内装有 CPU 与加速器的可插拔单元）中。

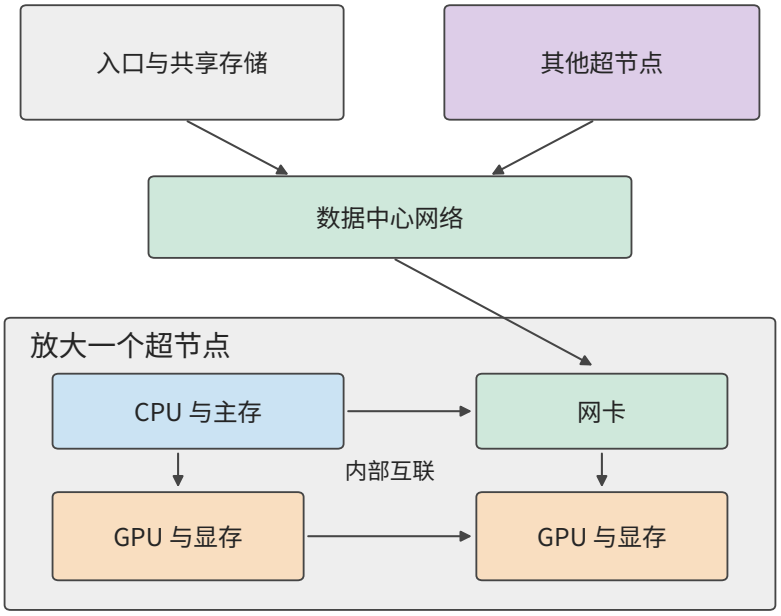



图 1-5　物理连接的两层视图。上方用数据中心网络连接服务与超节点，下方放大一个超节点，显示主机、网卡、GPU 与显存；线条展示数据路径，设备数量用于示意。

在服务器或计算托盘内，CPU 连接主存，负责主机程序和执行控制；GPU 连接各自的显存，执行模型计算、保存运行所需的数据。HBM（高带宽内存）是加速器常用的一类显存，通过宽接口持续提供数据。CPU 与 GPU 可以通过 PCIe（高速外设互联）连接，GPU 之间也可以通过独立的高速互联直接交换数据。网卡把本机

连接到网络。在支持直接内存访问的系统中，数据可以由传输硬件直接写入指定内存区域，CPU 负责发起和管理传输；第 7 章会进一步解释这种交接。

超节点内部的互联通常称为 **scale-up**（纵向扩展），重点是让一组加速器以较低的通信开销共同计算；多个超节点再通过网卡与交换网络相连，称为 **scale-out**（横向扩展），用来扩大集群范围。两者都要考虑带宽、延迟、拥塞与故障，但协作范围和代价不同。以 NVIDIA 的 GB200 NVL72 为例，官方设计把 36 个 Grace CPU 和 72 个 Blackwell GPU 组织在一个机柜中，72 个 GPU 通过 NVIDIA 的 GPU 高速互联 NVLink 直接协作，形成同一个互联域，集群网络则负责对外连接。这种组织方式把紧密协作的范围从单机扩大到了机柜，模型可以在更多加速器之间分配权重和状态，频繁的交接则由机柜内互联完成。[4]

AI 网络与传统数据中心网络由此形成了分工。服务入口传递请求，共享存储提供模型文件，加速器互联传递计算中的中间结果。三种流量的传输频率、数据量和依赖关系不同：模型文件可以提前加载，当前层的部分结果却必须及时到达，下一层才能继续。物理连接因此直接影响模型的执行时间。

> **练习 1-1〔延伸〕：一次请求中的权重读取、状态增长与数据传递**
> 沿图 1-3 至图 1-5，补画 KV 状态随输出增加的过程，再分别标出模型文件加载、一步权重读取、KV 追加和卡间结果传递。若回答从 100 个 token 增至 1000 个，哪些数据只加载一次，哪些访问会增加？若把一个实例复制成两个实例，权重容量与请求路由又怎样改变？

一次模型执行既受单卡能力影响，也受加速器连接方式影响。单卡的算力和存储带宽决定本地计算与读写的耗时，多卡协作则增加数据交换和等待。本章先估算单卡执行，后续章节再把同样的方法用于超节点和网络。

先介绍一组贯穿全书的真实模型。DeepSeek V4 和 V4.1 Flash 提供了一条可以持续追踪的线索。V4 已经通过上下文压缩与稀疏访问改变存储和读取需求，V4.1 Flash 则进一步重新划分模型内部的职责。传统的 decoder-only（仅解码器）模型通常让输入 token 经过完整主干，逐层建立生成所需的状态；V4.1 Flash 采用**因果编码器—解码器（Causal Encoder-Decoder，CED）**架构，由编码器形成上下文表示，解码器从这些表示取得全局信息。大量输入 token 因此无需经过解码器主体的计算；生成新 token 时，模型仍依次经过编码器与解码器。输入与输出的执行路径并不对称，模型因此能针对 Agent 输入多、输出相对少的负载重新安排计算投入。[5]

这是一项架构层面的选择，也会改变系统需要保存和传输什么。本书将围绕同一类会话展开讨论：Agent 等待工具返回，恢复此前上下文，处理新增输入，再继续生成。第 2 章解释 CED 的结构与状态，第 3 章分解输入和生成负载，第 4—7 章追踪加速器执行和数据路径，第 8、9 章讨论如何缓存状态、分配请求，最后比较完整任务的执行效果。开源模型 Qwen3 提供基础算例，用于建立计算方法；V4 / V4.1 用于检验模型与系统如何共同改变。

## 1.2 系统设计的关键数字

### 1.2.1 用数量级判断方案

估算单卡执行之前，先要知道各类基本操作大致耗时多少。Jeff Dean 在 2009 年的一次演讲中列出了一张后来被广泛引用的表，标题是 “Numbers Everyone Should Know”。表中既有缓存、内存的访问时间，也有磁盘和网络操作的时间。这张表体现了一种实用的工作方式：在写程序或搭建系统之前，先估算主要操作需要多少时间和资源。[6]

下面完整列出原幻灯片的 12 项操作和数值。所有时间统一为 ns（纳秒，十亿分之一秒）；1 μs（微秒）等于

[4] 具体产品例子见 NVIDIA GB200 NVL72、硬件指南与网络指南。图 1-5 使用一般化连接。

[5] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

[6] Jeff Dean，LADIS 2009 演讲，本地 PDF，“Numbers Everyone Should Know” 及纸笔估算部分。寻道算例设连续执行 20 次随机读取。

1000 ns，1 ms（毫秒）等于 1000 μs。数据单位中，1 byte（字节）等于 8 bit（比特）；bit/s 表示每秒传送的比特数，Gbit/s 中的 G 表示十亿。容量单位 KB、MB、GB、TB 分别表示 $10^3$、$10^6$、$10^9$、$10^{12}$ 字节；KiB、MiB、GiB 分别表示 $2^{10}$、$2^{20}$、$2^{30}$ 字节。这组数字对应 2009 年的设备环境，用于理解不同操作之间的数量级关系。

缓存保存近期可能再次使用的数据；一级缓存通常比二级缓存更小、更靠近计算核心。分支预测是处理器对接下来执行哪条指令路径的预判，预测错误时要重新安排执行。线程是程序中独立调度和执行指令的单位；互斥锁限制多个线程同时访问同一份共享数据。磁盘寻道是机械磁盘移动磁头、定位目标磁道的过程。Zippy 是 Google 当时使用的压缩库。这些操作分别涉及计算、同步、存储与网络，耗时可能相差多个数量级。

| 操作 | 时间（ns） |
|---|---|
| 访问一级缓存 | 0.5 |
| 分支预测错误后的恢复 | 5 |
| 访问二级缓存 | 7 |
| 对互斥锁加锁或解锁 | 25 |
| 访问主存 | 100 |
| 用 Zippy 压缩 1K 字节 | 3,000 |
| 经 1 Gbit/s 网络发送 2K 字节 | 20,000 |
| 从主存连续读取 1 MB | 250,000 |
| 同一数据中心内往返 | 500,000 |
| 磁盘寻道 | 10,000,000 |
| 从磁盘连续读取 1 MB | 20,000,000 |
| 数据包从加州到荷兰再返回 | 150,000,000 |

取其中三个历史数字：主存访问约 100 ns，同一数据中心内的一次往返约 500,000 ns，磁盘寻道约 10,000,000 ns。换算单位后即 0.1 μs、0.5 ms 和 10 ms，如图 1-6 所示。

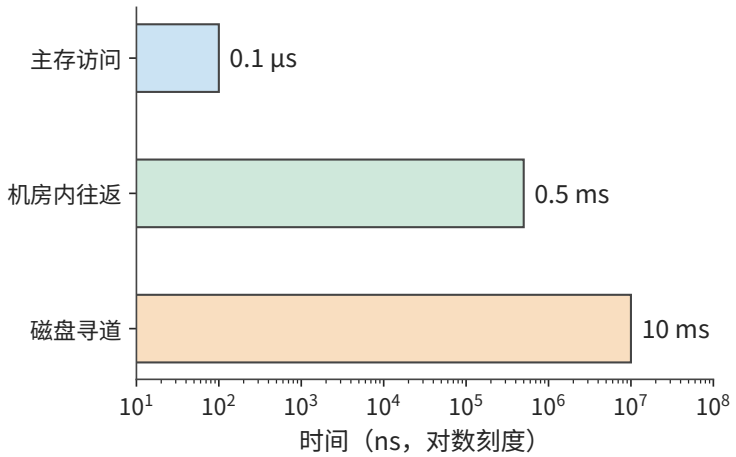



图 1-6　Jeff Dean 2009 年演讲中的三种操作延迟。横轴为对数刻度，每相邻数量级相差十倍；先识别操作，再比较它们在串行等待中的代价。

按这组数字，一次磁盘寻道所需的时间约为一次主存访问的十万倍。若请求需要依次等待多次访问，这些延迟就会累加，延长请求的完成时间。若请求必须先找到一块数据，才能确定下一次访问的位置，处理器在两次访问之间即使只做很少的计算，也要等待整个存储访问过程。

例如，设请求串行执行 20 次寻道，每次 10 ms，全部 CPU 计算为 1 ms，总时间就是 201 ms。把计算速度提高一倍，只能节省 0.5 ms；把相关数据连续放置，将寻道次数从 20 次减为 10 次，则节省 100 ms。两种优

化分别作用于计算与等待，收益的差距来自它们在原执行时间中的占比。

把时间分解后，就能算出优化方向：某一部分在原总时间中占比越大，缩短这一部分对整体的改善越大。在前面的寻道例子中，计算只占总时间的 1/201；即使这一部分完全消失，总时间也只减少约 0.5%。

将上述关系推广，便得到总加速比的计算公式。若原执行时间中有比例 $f$ 的部分能够加速 $s$ 倍，其余工作与执行次序保持不变，则总加速比为：

$$\text{Speedup} = \frac{1}{(1-f) + f/s}.$$

这就是 Amdahl（阿姆达尔）定律。即使把这部分加速到耗时可以忽略，总加速比也不超过 $1/(1-f)$。当寻道次数减少时，被缩短的是占比最大的等待部分，因此这项优化对总时间的作用远大于 CPU 加速。

**例：同样使用 AI 编程工具，为什么不同团队的整体加速比不同？**假设完成一项开发任务，需要依次经历沟通、编码、评审和审批等环节，AI 编程工具让编码速度提高到原来的 5 倍，其他环节的耗时保持不变。在协调层级较多的大公司团队里，编码可能只占原总耗时的 20%—30%，其余时间花在开会、跨团队沟通和审批上。代入公式，整体加速比只有约 1.19—1.32 倍。例如，原来需要 100 小时，其中 20 小时用于编码；编码缩短到 4 小时，其他环节仍需 80 小时，总耗时就是 84 小时。

在沟通成本较低的创业团队里，假设编码占原总耗时的 70%—80%，同样的编码加速就能让整体加速比达到约 2.27—2.78 倍。以编码占 80% 为例，原来的 100 小时变成 $20 + 80/5 = 36$ 小时。

**Amdahl 定律适用于任何只加速部分环节的完整流程。** CPU、模型推理和软件开发都可以用同一个问题来分析：被加速的部分原来占多少时间，剩余时间花在哪里？想进一步缩短交付时间，就要继续改善沟通、评审和审批等环节。局部速度的提升，最终要放回完整任务中衡量。

这张表体现的方法可以直接用于模型执行：先确定各项工作使用什么资源，再比较所需时间。

### 1.2.2 分析 AI 系统需要哪些关键数字

设某任务同时需要保存 $M$ 字节，执行 $F$ 次浮点运算，并通过某个存储接口读写 $R$ 字节。加速器提供容量 $M_{\text{cap}}$、计算吞吐 $\Pi$ 和接口带宽 $\beta$。这三类需求必须分别与加速器提供的资源比较：

$$M \le M_{\text{cap}}, \qquad T_{\text{compute}} = \frac{F}{\Pi}, \qquad T_{\text{memory}} = \frac{R}{\beta}.$$

容量决定能否同时保存所需数据；后两项把工作量换算为时间。一份数据可以保存一次、读取多次，所以 $M$ 与 $R$ 要分别计算。若 $\Pi$ 和 $\beta$ 取峰值，得到的就是完成指定计算与读写所需的时间下界。

峰值由芯片的计算单元数量、工作频率和存储接口决定，所以这个下界就是硬件的物理极限，软件无论怎样组织都不可能更快。下界与实际耗时 $T$ 之比，反映硬件能力被用掉了多少。计算方面的比值 $F/(\Pi T)$ 称为 **MFU**（model FLOPs utilization，模型 FLOPs 利用率），带宽方面的比值 $R/(\beta T)$ 称为 **MBU**（memory bandwidth utilization，带宽利用率），两者都不超过 1。全书反复使用这两个比值，回答的是同一个问题：设计离硬件的物理极限还有多远。

除了容量、计算吞吐和带宽，还需要认识操作延迟。下面分别说明这四类数字如何进入估算。

**容量**回答能同时放下多少数据。例如，一张加速卡有多少显存，决定了能否容纳模型权重、上下文状态和运行时的临时工作区。容量以 byte（字节）计量。

**计算吞吐**回答单位时间内能完成多少运算。常见的 FLOP 表示一次浮点运算，FLOPs 表示运算总次数，FLOP/s 表示每秒的浮点运算数。GFLOPs、TFLOPs 分别表示十亿、万亿次浮点运算，GFLOP/s、TFLOP/s 则表示相应的每秒速率。矩阵计算中一次乘法加一次加法通常计为两次运算。矩阵峰值对应特定的输入精度、累加精度和稀疏条件。稠密计算按完整矩阵执行；稀疏计算利用零元素或规定的稀疏结构跳过部分运算。

**带宽**回答单位时间内能传送多少数据。显存带宽描述显存与芯片之间的数据搬移能力，卡间链路和网络带宽描述其他路径的能力。估算哪一段传输，就使用该段接口的带宽。

**延迟**回答发起一次操作以后需要等多久。高带宽接口也可能有不可忽略的启动或往返时间。传输大数据块时，耗时主要取决于数据量与带宽的比值；请求较小且必须逐次等待时，启动和往返时间就可能占据主要部分。

以一款真实加速器为例。H100 是 NVIDIA 的一代 GPU，SXM 指本例采用的模块形态。BF16 是每个数占 16 位（2 字节）的浮点格式，FP32 是每个数占 32 位（4 字节）的浮点格式；矩阵乘法可以读取 BF16 输入，用 FP32 保存累加结果。H100 SXM 的显存容量为 80 GB，HBM 带宽为 3.35 TB/s，BF16 输入、FP32 累加的稠密矩阵峰值约为 989.4 TFLOP/s。[7]

将 H100 与 NVIDIA 的 A100 80GB SXM、GeForce RTX 4090 放在一起，就得到一张 GPU 资源速查表。三者分别采用 Hopper、Ampere 和 Ada 架构。表中的容量使用厂商名义 GB，带宽使用十进制 TB/s；矩阵算力统一采用 BF16 输入、FP32 累加、稠密计算的峰值。最后一行由 1 GB 除以显存带宽得到。[8]

| 资源或操作 | RTX 4090 | A100 80GB SXM | H100 SXM |
|---|---|---|---|
| 显存容量（GB） | 24 | 80 | 80 |
| 显存带宽（TB/s） | 1.008 | 2.039 | 3.35 |
| BF16 矩阵峰值（TFLOP/s） | 165.2 | 312 | 989.4 |
| 按带宽峰值读取 1 GB（ms） | 0.99 | 0.49 | 0.30 |

这张表提供了把工作量换算成时间所需的硬件性能数据。例如，读写量保持不变而带宽增加一倍，$R/\beta$ 减半；计算量保持不变而算力增加一倍，$F/\Pi$ 减半。

### 1.2.3 参数存储量与链路带宽的换算

这四类指标要与具体工作对应起来，才能用于估算。第一步是把模型参数换成存储字节数，判断加速器能否放下；第二步再用字节数除以带宽，估算搬移这些数据需要多久。

以真实模型 **DeepSeek-R1-Distill-Llama-70B** 为例。该模型基于 Llama 架构，是稠密模型（每个 token 都使用全部参数计算），公开权重包含约 705.54 亿个参数。名称中的 B 表示十亿，70B 是参数规模的约数；后文公式中的变量 $B$ 表示 batch 中的请求数。用 $N$ 表示参数数量，$b_W$ 表示每个权重的存储字节数。BF16 每个权重占 2 字节，公开权重文件中的总字节数为[9]

$$\begin{aligned} M_W &= N b_W \\ &= 70{,}553{,}706{,}496 \times 2 \text{ bytes} \\ &\approx 141.11 \text{ GB}. \end{aligned}$$

[7] NVIDIA，H100 架构白皮书与规格页面快照。本章采用 SXM 形态与 BF16 稠密矩阵运算规格；原始输入版本和校验值见配图来源清单。

[8] 数字取自固定 GPU 规格与逐项出处，对应 `rtx4090`、`a100-80gb-sxm`、`h100-sxm`；各精度、累加方式与稠密条件分别核对。GB 与 TB 在本表均为十进制单位。

[9] 参数与 BF16 字节数来自 DeepSeek-R1-Distill-Llama-70B 公开权重索引及固定版本配置。分组量化的权重总量采用逐项计算记录中的全模型汇总，每组 128 个参数、scale 占 2 字节。RTX 4090 的 24 GB 名义规格见官方页面存档。本节用规格标称 GB 作统一容量预算。

本节统一使用十进制 GB，即 1 GB $= 10^9$ bytes；二进制单位 GiB 则为 $2^{30}$ bytes。因此，141.11 GB 约为 131.42 GiB。加速器的可用存储容量与权重大小换成相同单位后，才能相减。

141.11 GB 权重超过一张 H100 SXM 的名义 80 GB 显存。将权重均分到两张卡上，每张约占 70.55 GB，各剩约 9.45 GB。**切分**即让不同加速器分别保存并计算模型的一部分。两张卡之间需要交换中间结果，第 6 章将讨论具体的切分与通信方法。

还可以改变权重的存储精度。**量化（quantization）**用较少的位数表示数值，例如用 8 位整数近似表示原来的 16 位浮点权重，并用 scale（缩放因子）将整数还原为相应范围内的近似值。每个量化后的权重从 2 字节减为 1 字节，内存占用因而明显下降。

本书采用的分组量化方案每 128 个权重共享一个 scale，同时让部分参数继续使用 BF16。按真实模型配置计算，8 比特方案的全部权重及量化附加数据约占 73.73 GB，可以放入一张 H100 SXM，留下约 6.27 GB。图 1-7 将两种办法放在相同的容量刻度上：BF16 权重分到两张卡，或者量化后放入一张卡。

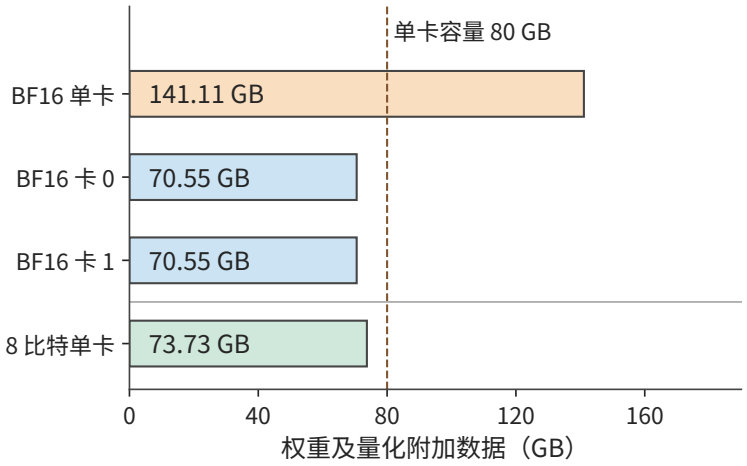



图 1-7　DeepSeek-R1-Distill-Llama-70B：BF16 权重共 141.11 GB，两卡均分后每卡约 70.55 GB；分组 8 比特量化后共 73.73 GB。各条使用相同尺度，虚线表示一张 H100 SXM 的名义 80 GB 容量。柱长只计权重及相应量化附加数据。

**KV 缓存如何增加单请求的显存需求。**生成文本时，显存还要保存 KV 缓存，供后续计算直接复用；算子执行还需要临时工作区。因此，容量检查应写成

$$M_W + M_{\text{state}} + M_{\text{work}} \le M_{\text{cap}}.$$

其中 $M_W$ 是权重与量化附加数据，$M_{\text{state}}$ 是请求的上下文状态，$M_{\text{work}}$ 是工作区，$M_{\text{cap}}$ 是加速器的可用存储容量。以该模型的一条 8192 个 token 的请求为例，BF16 KV 缓存为 2.5 GiB，约 2.68 GB；再预留 2 GiB、约 2.15 GB 给工作区，8 比特方案总共约占 $73.73 + 2.68 + 2.15 = 78.56$ GB，符合这一容量预算。第 2 章将从模型层数与注意力结构推导 KV 大小，并计算更长上下文和更多并发请求的内存需求。

换成名义显存为 24 GB 的 RTX 4090，同一份 73.73 GB 权重就超过了三张卡合计的 72 GB。按相同的名义容量预算，仅保存权重也至少需要四张；还要逐卡检查 KV 和工作区。模型与精度相同，单卡容量不同，所需加速器数量就会改变。

网络传输也要做同样的单位换算。网络带宽常以 bit/s 计量，8 bit 为 1 byte，所以一张 400 Gbit/s 的 ConnectX-7 网卡，线速（链路的标称传输速率）等于 50 GB/s。按此线速传输 1 GB 数据需要 20 ms。每次传输还需要启动时间，协议开销和共用链路的其他流量也会影响应用实际获得的带宽。沿传输路径计入这些因素，便能更细致地估算通信时间。

**练习 1-2〔核心〕：不同权重精度下的显存需求与多卡分配**
DeepSeek-R1-Distill-Llama-70B 的 BF16 权重占 141.11 GB，8 比特量化权重占 73.73 GB。部署在 H100 SXM 上，每张卡有 80 GB 显存。为每张卡预留 5 GB 状态与工作区，分别判断单卡部署和将权重均分到两张卡的方案能否容纳模型。再考虑两张显存容量不同的卡：一张 48 GB 的 RTX A6000 和一张 80 GB 的 H100 SXM，分别判断两种精度下能否完成部署，并为可行的方案给出一种权重分配方式。最后将 400 Gbit/s 换成 GB/s，求传输 1 GB 的理想时间。

## 1.3 用几个数字估算一次模型执行

### 1.3.1 生成一个 token，先检查什么

**例题 1-1：70B 模型的单步生成如何接近 10 ms 目标？**给定一张 H100 SXM，能否让上一节的 DeepSeek-R1-Distill-Llama-70B 在 10 ms 内生成一个新 token？先考虑每参数一字节存储，再比较算力翻倍、带宽翻倍和批内权重复用。

**解：先检查显存容量，再计算运算量和读取量。**

为便于手算，将该模型的参数量近似取为 $N = 70 \times 10^9$，先按每个参数一字节估算主要权重的数据量，读入后转换为 BF16 参加矩阵运算。单个请求每步处理一个新 token，每步从显存读取一遍这些权重。

**估算条件：权重读取与矩阵计算充分重叠。**容量采用上一节计入量化附加数据的 73.73 GB 结果；本节先将主要权重的读取量近似为 70 GB。时间模型只计主要权重矩阵的乘加和一次完整权重读取，采用资源表中的 BF16 矩阵峰值与 HBM 带宽；读取和计算按充分重叠估算。

单请求的容量预算上一节已经检查。生成速度方面，权重加载到显存之后，每一步仍需将参与计算的权重送到计算单元；按上述近似，每步读取约 70 GB。

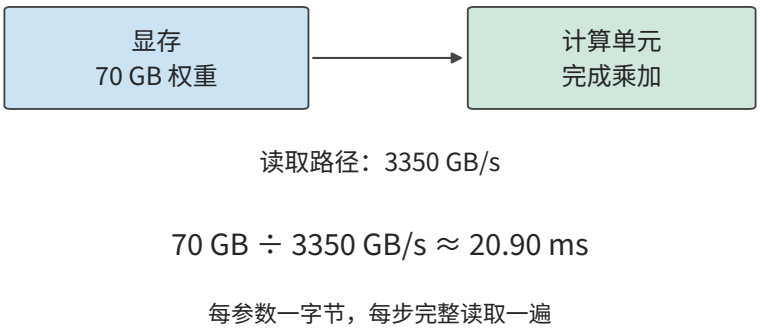



图 1-8　本例按每参数一字节，将主要权重读取量近似取为 70 GB。权重驻留显存，计算单元每步沿同一接口读取一遍；用读取量除以接口带宽，得到 20.90 ms 的读取下界。

这 20.90 ms 的读取开销会在每个生成步骤重复出现。对单个请求，下一步的输入由当前输出确定，后续步骤要沿这条依赖逐次推进。

计算量也可以先做近似。一次矩阵与向量的乘法中，每个权重通常参与一次乘法，所得乘积再累加到结果中；按乘法与加法分别计数，大约对应两次浮点运算。因此，本例的主要矩阵运算量近似为：

$$\begin{aligned}\text{每步矩阵运算量} &\approx 2 \times \text{参数数} \\ &\approx 2 \times 70 \times 10^9 \text{ FLOPs} \\ &= 140 \text{ GFLOPs}.\end{aligned}$$

$2N$ 的来源是：在投影（用权重矩阵对特征向量做的线性变换）中，每个权重与一个 token 特征向量中的对应分量相乘，再将乘积累加到输出中。参数矩阵中的权重越多，这部分工作量就越大；一批中参与计算的 token 数增加时，同一权重矩阵处理更多 token 的特征向量，运算量随之增加。第 2 章将展开真实模型，在这部分线性工作之外加入随上下文长度变化的注意力交互。

把 batch size 记为 $B$，并让这 $B$ 个请求共同使用一次读入的权重，就得到这一教学模型的三个量：

$$M_W = b_W N, \qquad R_W = b_W N, \qquad F \approx 2BN.$$

前两个式子在这一步恰好相等，是因为权重保存一份、读取一遍。执行下一步生成时，驻留量仍是 $M_W$，却会再增加一次 $R_W$ 的读取。$B$ 增大则增加本次运算量：同一份权重要与更多请求的当前输入 token 向量做乘加运算。第 2 章将进一步加入各请求独立的上下文状态。

### 1.3.2 计算与读取分别需要多久

把 $F = 140$ GFLOPs 与 $R_W = 70$ GB 分别除以相应的资源能力，得到单请求的两项时间下界。

$$\begin{aligned}\text{纯权重读取时间} &\geq \frac{70 \text{ GB}}{3350 \text{ GB/s}} \approx 20.90 \text{ ms}, \\ \text{矩阵计算时间} &\geq \frac{140 \text{ GFLOPs}}{989400 \text{ GFLOP/s}} \approx 0.1415 \text{ ms}.\end{aligned}$$

权重读取的时间下界约为矩阵计算的 148 倍。按标称算力，矩阵运算耗时极短；显存把权重送到计算单元却需要长得多的时间。完成当前步骤之前，这些数据必须全部读入。[10]

原因在于，本例每读取一个权重只做一次乘加。计算单元迅速处理完已读入的数据后，又要等待后续数据。提高算力只能缩短乘加耗时，无法加快显存读取。

这两项时间应当相加，还是取其中较大者，取决于读取与计算能否重叠。若数据分块到达，计算当前块时可以继续读取下一块；在充分重叠的简化模型中，耗时下界为：

$$T_{\text{step}} \geq T_{\text{lower}} = \max\left(\frac{F}{\Pi}, \frac{R_W}{\beta}\right).$$

进入稳定阶段后，每块数据都要经过读取和计算，较慢的一项决定处理速度。本例中，读取权重远慢于矩阵计算，因此优化应首先针对这 20.90 ms 的读取时间。

**讨论：算力、带宽与批内复用分别能缩短多少生成时间？** 仅权重读取就至少需要 20.90 ms，已经超过 10 ms 的目标。虽然显存能容纳这些数据，读取速度仍然达不到要求。先分别比较增加算力与增加带宽的效果。

[10] 70B 近似预算复算及配套场景；单位与逐卡容量复算。1.3 的时间估算图读取这组固定的 70B 近似结果；1.2.3 的容量图使用真实权重索引与分组量化结果。

把计算能力翻倍，矩阵计算时间从约 0.1415 ms 变为 0.0707 ms，纯权重读取仍需要约 20.90 ms，决定执行时间的读取下界保持不变。把 HBM 带宽翻倍，读取时间才会降为约 10.45 ms，当前下界随之下降。两种改动都提升了一项硬件能力，但对任务耗时的影响不同。

另一种办法是减少需要读取的数据。运算量不变时，把权重减半，读取下界也降到约 10.45 ms。带宽翻倍让读取速度加倍，权重减半则让待读取的数据减少一半，两种办法在这条算式中取得相同结果。第 2 章和第 5 章将讨论低位宽表示的格式和转换过程。

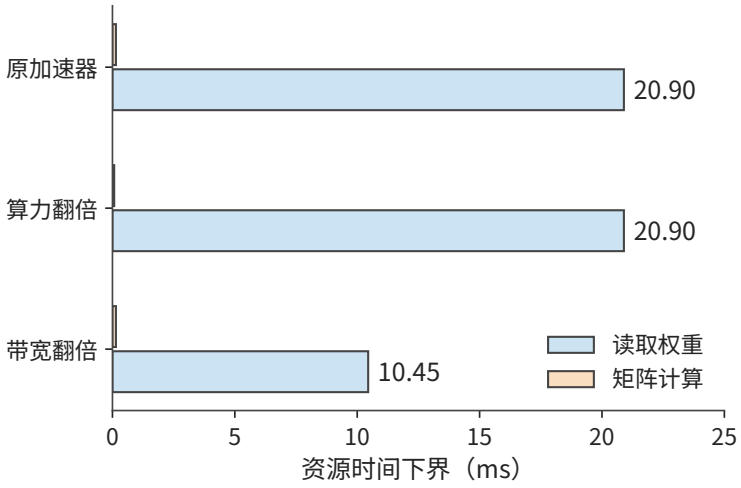



图 1-9　保持运算量与读取量不变，分别把算力或带宽翻倍。蓝条是权重读取下界，橙条是矩阵计算下界；较长的读取项决定这组条件下的优化方向。各柱分别表示计算或数据读取的时间下界，完整执行仍须满足依赖关系。

接着改变请求组织。假设 8 个请求同时处理并共享一次权重读取，整批的权重读取仍是 70 GB，矩阵运算量则约为单请求的 8 倍。这时矩阵计算的时间下界约为 1.13 ms，读取下界仍为 20.90 ms。若这一批产生 8 个输出，每个输出分摊的权重读取时间约为 2.61 ms。

整批执行产生八个输出，吞吐因而从单请求模型的约 47.9 token/s 提高到约 383 token/s，而每个请求仍须等待整批执行完成。**批内复用增加同一时间内生成的输出数，单请求延迟则取决于它经历的执行与等待。**

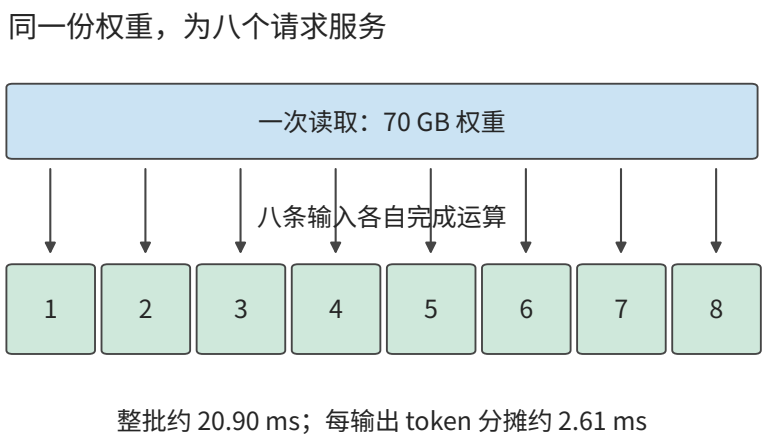



图 1-10　一批八个请求共享一次权重读取，各产生一个输出。整批读取仍需约 20.90 ms，除以八得到每输出分摊的服务时间；每个请求经历整批执行。“每输出 token 分摊”是整批耗时除以输出 token 数，用于换算吞吐，不是单个请求的响应延迟。

batch 增大到一定程度后，计算时间将追平权重读取时间。令 $2BN/\Pi = b_W N/\beta$，得到转折点

$$B_* = \frac{b_W \Pi}{2\beta}.$$

本例取 $b_W = 1$、$\Pi = 989.4 \times 10^{12}$ FLOP/s、$\beta = 3.35 \times 10^{12}$ bytes/s，得到 $B_* \approx 147.7$。在这一只计主要矩阵运算与权重读取的模型中，batch 较小时，增加请求可以分摊权重读取开销；超过约 148 后，计算耗时超过权重读取耗时，继续增加 batch 会近似按比例增加整批时间。该转折点只比较矩阵运算与权重读取，没有计入各请求的上下文状态：按 1.2.3 节的容量预算，一张 H100 SXM 容纳 73.73 GB 权重后只剩约 6.27 GB，无法容纳 148 条请求的 KV。

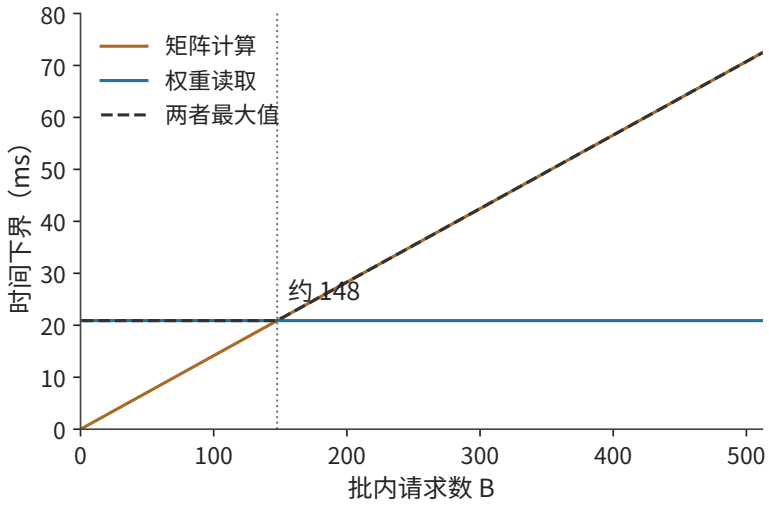



图 1-11　批内请求增加时，矩阵运算量按 $2BN$ 增长，权重读取保持每批 70 GB。约 148 个请求处两项下界相等，随后计算耗时决定整体速度。

把每批输出数 $B$ 除以上图中的时间下界，便得到这组题设对应的吞吐上界。转折前，共享的读取由更多输出分摊；转折后，计算时间与输出数一起增长，曲线逐渐趋于平坦。

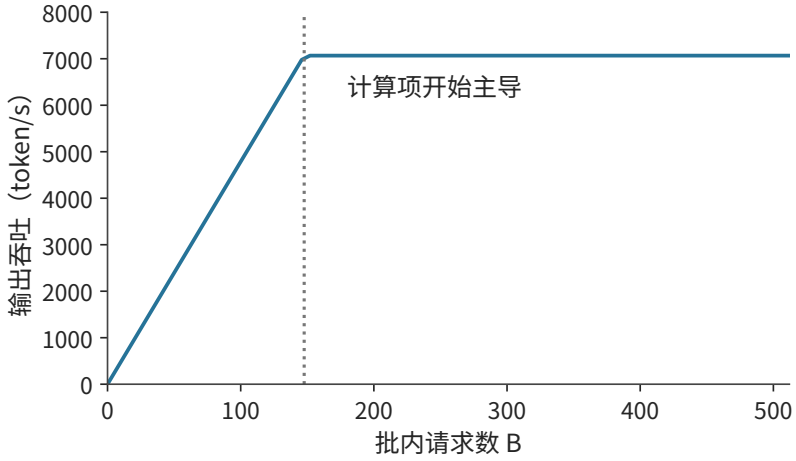



图 1-12　上述批处理模型的理想输出吞吐率。每批输出数除以时间下界得到曲线；竖虚线与前图对应同一个约 148 请求的转折点。

同一转折也可以用算术强度 $I = F/R_W = 2B/b_W$ 描述，即每读取一字节数据所完成的运算次数。当 $I$ 小于加速器的算力与带宽之比 $\Pi/\beta$ 时，读取所需时间更长；超过该比值后，计算所需时间更长。这就是第 4 章

Roofline（屋顶线）模型的出发点。[11]

> **练习 1-3〔核心〕：带宽与 batch size 如何影响单步生成时限**
> 采用例题 1-1 的模型与加速器，将有效带宽设为标称值的 70%，有效计算吞吐率设为峰值的 50%。分别取 $B = 1, 16, 64$，计算整批执行时间的下界、平均每个输出 token 分摊的执行时间，以及理想吞吐率。目标是每请求每步不超过 10 ms：哪些 batch size 仅根据时间下界就可以判定无法满足目标？再把权重读取量减半、运算量保持不变，重新判断。求计算时间与读取时间相等时的 batch size $B_*$，并解释为什么每个输出分摊的执行时间减少，并不意味着每个请求的单步延迟缩短。

### 1.3.3 用实测检验估算

前面的模型预测了两个趋势：batch 较小时，吞吐随 batch 增加；整批时间则受到一次权重读取的限制。真实程序中，每条新增请求还会增加上下文访问与计算。图 1-13、1-14 给出 Qwen3-8B 在 RTX PRO 6000 Blackwell Workstation Edition 上使用 BF16 权重的测量结果。这张卡有 96 GB 显存、1.792 TB/s 显存带宽，BF16 输入、FP32 累加的稠密矩阵峰值为 503.8 TFLOP/s。vLLM 是加州大学伯克利分校等机构的研究者在 2023 年提出的开源推理服务系统，负责组织模型请求并执行推理，最初重点解决的是 KV 缓存浪费显存、限制 batch size 的问题。本次采用 0.23 版本的 eager 模式，即按程序运行顺序逐项向加速器提交工作。每请求输入 2048 个 token、生成 256 个 token，每档 batch 测量三次。[12] 吞吐表示单位时间内产生的输出数；**每输出 token 时间**（time per output token，TPOT）在这里表示客户端观察到的平均输出间隔。

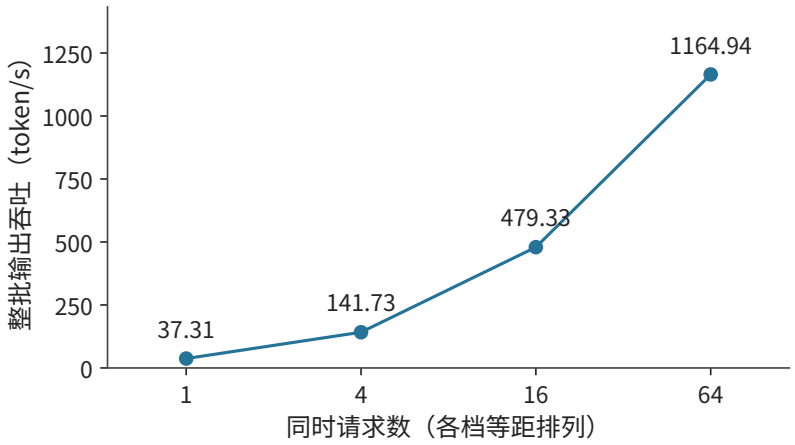



图 1-13　Qwen3-8B 实测的整批输出吞吐。四档请求数等距排列，纵轴从零开始；模型、精度、输入输出长度和计时范围保持一致。

从 1 个请求增到 64 个请求，整批吞吐约提高 31 倍，单请求输出间隔则从约 26.5 ms 增至 40.5 ms。两幅图合起来描述批处理的取舍：加速器在同样长的时间内完成更多请求的工作，每个请求完成一步生成所需的时间却更长。

图 1-13 的吞吐按整批输出数除以处理这批请求的总时间计算，其中包括输入处理时间，三次测量取中位数。TPOT 则先对每个请求计算首末输出事件之间的时间，再除以其间的 255 个输出间隔，然后取同批请求的中位数，最后取三次测量的中位数。前者描述整个实例的输出速度，后者描述单个用户经历的生成节奏。第 3 章将进一步分析请求到达、排队和少量慢请求对这些指标的影响。

这一趋势可以用权重与上下文状态的区别来解释。多个请求共同使用模型权重，但各自保存上下文状态。并发增加时，一次矩阵运算会同时处理更多请求的当前输入 token 向量，权重得到更多复用；与此同时，注意力

[11] Williams、Waterman、Patterson，Roofline 论文。

[12] 练习 1-4 记录说明、结构化分析结果及后续补测。正文和图 1-13、图 1-14 采用最初记录的短输入组。

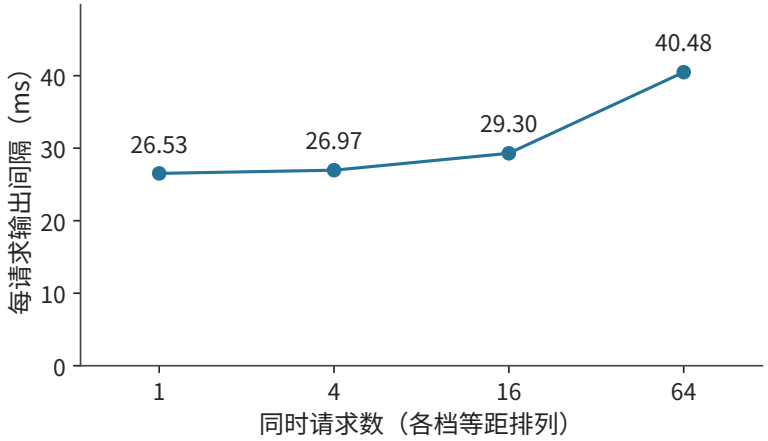



图 1-14　同一组实测中的每请求输出间隔。吞吐增加的同时，单个请求的平均间隔也在增大；两张图分别说明加速器的输出速度和用户的等待时间。

计算和上下文访问也随请求数增加。

在基本模型中补上这部分随请求数增加的工作，可先把一步耗时写成

$$T_{\text{step}} \approx \max\left(\frac{BF_1}{\Pi}, \frac{R_W + BR_{\text{state},1}}{\beta}\right),$$

其中 $F_1$ 是每请求的运算量，$R_{\text{state},1}$ 是每请求的状态读写量。batch 增加时，权重读取开销由更多输出分摊，但各请求的状态读写总量也随之增加。下一章将根据实际模型计算这些工作量，并预测上下文变长后，计算时间与读取时间相等的转折点会移到哪里。

**练习 1-4〔核心〕：用实测吞吐与输出间隔检验批处理模型**

根据图 1-13、1-14 标出的四档并发请求测量值，计算各档并发相对于单请求的吞吐率之比，以及 TPOT 的增长比例。假设增加并发后仍只受共享权重读取限制，计算与上下文访问都不增加耗时，吞吐率应怎样变化？对照实际结果，提出两个可以通过测量区分的解释，并设计一次只改变上下文长度的测量来判断它们。指出测量前需要保持不变的模型、输出长度与计时起止点。

回看这次估算，全程都在回答五个问题：**搬什么、搬多少、搬几次、经过哪里、谁在等待。**模型结构决定需要哪些权重，位宽和参数量决定存储大小，批内复用改变了读取次数，显存接口提供了带宽，执行依赖决定了等待。这五个问题把模型结构、数据路径和执行顺序连成一份可以逐项计算的预算。

### 1.3.4 实测为何达不到极限：模型漏项还是系统开销

做系统优化，常见的做法是与旧实现比较：新算子比旧的快了三成，就算成功。可是不知道极限在哪里，就无法判断这三成是已经逼近上限，还是离上限仍有一个数量级。正确的起点是先按第一性原理算出硬件允许的上限，再看实测离上限有多远。以本章的实测为例：按 1.792 TB/s 的显存带宽，一步 decode 读取权重与旧 KV 至少需要 8.62 ms，而 batch 为 1 时实测的输出间隔是 26.5 ms，MBU 只有 33%。这样的差距有两种来源，处理方式截然不同。第一种是理论模型漏了项：上面的模型只计入权重与 KV 的读取，没有计入注意力之外的向量运算和采样；补上这些项，下界上移，差距随之缩小，修正的是模型而不是系统。第二种是实现本身的开销：本次实验以 eager 模式逐个提交 kernel，每次提交都要经过主机上的 Python 代码、驱动和 PCIe，加速器在两次 kernel 之间空转；客户端逐步记录日志，又占用了主机时间。这些开销与硬件无关，是可以去掉的。要分清两种来源，只能靠每次只改变一个条件的测量。同一台卡上关闭逐步日志后补测，输出间隔降到 12.1 ms；只按加速

器上的事件计时，一步模型执行为 10.7 ms，与 8.62 ms 的下界只差两成。[13] 可见三倍的差距大部分是主机一侧的系统开销。后续各章凡是实测与下界不符，都按这一顺序检查：先问模型有没有漏项，再问实现有没有开销。

## 1.4 需求如何推动架构设计

前面的估算把硬件当作给定条件；反过来，这些硬件本身也是为应对具体需求而设计的。推动架构变化的因素主要有两类：需求变化，如业务规模扩大或出现新的负载；硬件变化，如某类器件的性能提升快于其他器件。两类变化最终都体现为比例的改变。计算量与数据读取量、带宽与处理能力、需求增长与单芯片能力，这些比例一旦改变，原先合理的分工就可能不再合算。本节先分析三个案例：Google 在 2013 年面对的语音推理需求、微软在 2015—2016 年扩展 Azure 云网络的需求，以及华为在大模型发展过程中组织多设备计算与存储的需求；再说明这类变化如何在 AI 系统中逐级传递。

### 1.4.1 TPU

TPU 是 Google 为神经网络计算设计的张量处理器，其第一代产品记为 TPU v1。介绍这代产品的论文回顾了一次需求判断。Google 很早就讨论过在数据中心使用 GPU、可在部署后配置硬件逻辑的 FPGA，或专用芯片，但在一些早期应用中，既有数据中心的空闲资源已经足以满足需求。2013 年，一项语音搜索使用量预测改变了讨论：如果每位用户每天使用三分钟语音搜索，满足这项计算需求可能需要让数据中心规模翻倍。Google 因而推进专用推理芯片设计，TPU v1 于 2015 年进入数据中心部署，架构论文于 2017 年发表。[14]

这一判断可以用简单的算式表示：把每天的请求量记为 $Q$、每请求需要的计算量记为 $F_1$，每日总需求就是 $QF_1$。请求量增加，现有设备的空闲能力最终会用尽；此时可以增加通用服务器，也可以开发专用处理器。后者有前期开发成本，却可能降低每请求的设备时间和能耗；服务规模越大，累计节省的成本就越容易抵偿前期投入。

专用处理器可以为反复出现的矩阵运算配置更多计算单元，再用片上缓冲和数据通路为计算单元提供输入。响应时间与功耗要求决定了应如何分配计算、存储和通信资源。这样，应用规模的变化就落实到了芯片设计上。

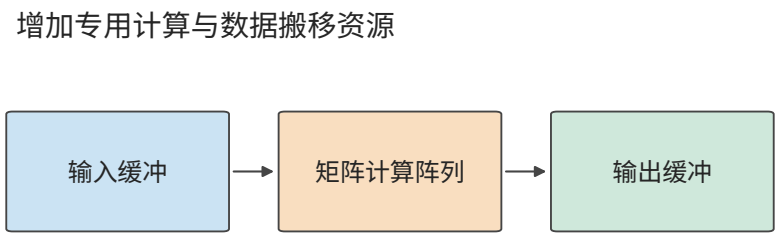



图 1-15　专用处理器围绕反复出现的矩阵运算组织计算阵列和输入输出缓冲。缓冲是临时保存待计算或已算完数据的存储区域；三个方框及其箭头展示数据搬移方向。

第 4 章将介绍 TPU 的矩阵阵列、缓冲和数据通路。每日计算需求 $QF_1$ 接近数据中心的空闲算力时，专用处理器便可能比通用处理器更合算。

[13] 补测记录与加速器事件计时。补测同时关闭了前缀缓存并重新构造了输入，逐步日志的影响无法与这些条件完全分开；加速器事件只覆盖模型执行，不含采样与输出。

[14] Jouppi 等，TPU v1 论文，第 2 节关于起源、架构与实现的说明。三分钟语音搜索是论文记载的历史需求预测。

### 1.4.2 SmartNIC

微软 Azure 在 2015 年设计新一代 40 Gbit/s 云服务器时，面临另一类增长：网络带宽提高，虚拟网络策略也日趋复杂。**SmartNIC（智能网卡）**是能够在网卡上执行部分数据处理的设备。微软从 2015 年末起在新部署的 Azure 服务器中安装基于 FPGA 的 SmartNIC，2016 年向客户提供 Accelerated Networking（加速网络）服务。FPGA 的硬件逻辑可以重新配置，网络处理因此能随策略更新而改变。[15]

这项设计要同时满足两个要求：既把更多 CPU 核留给租户的应用，又能快速更新网络隔离、转发和访问控制规则。云服务器上的网络处理占用的正是这些 CPU 核：CPU 除了运行应用，还要处理收发的数据包。虚拟机是在一台物理主机上划分出的独立计算环境。多个虚拟机共用物理网络时，平台需要识别数据的归属、查找转发规则、完成封装，并维持隔离。

先估算这部分工作占用多少 CPU。40 Gbit/s 链路在最小帧条件下每秒约需处理六千万个包。若单核每秒处理一千万至两千万个包，持续转发所需核数约为

$$n_{\text{core}} = \frac{\lambda_{\text{packet}}}{\mu_{\text{core}}} \approx 3\text{—}6.$$

其中 $\lambda_{\text{packet}}$ 为包到达率，$\mu_{\text{core}}$ 为单核处理率。链路带宽提高使 $\lambda_{\text{packet}}$ 随之增大，$\mu_{\text{core}}$ 却没有同步提高，所需核数便随两者之比增加。这些核反复完成相同的包处理流程；将相应工作移到网卡，就可以把 CPU 时间留给应用。[16]

Azure 的做法是让主机软件管理复杂策略，把适合重复执行的包处理规则交给 FPGA。数据经过网卡时就完成这些处理，CPU 可以把更多时间用于客户应用。设计的关键是兼顾软件更新能力与硬件处理速度。

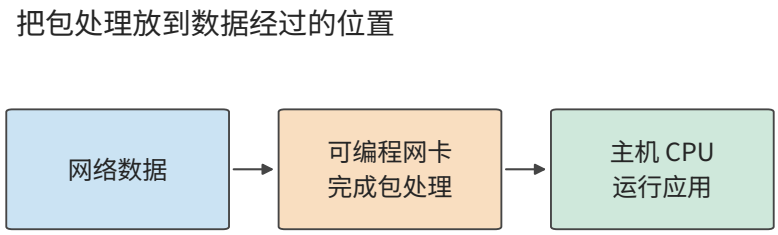



图 1-16　可编程网卡在数据进入主机前完成指定的包处理。图中实线跟踪数据；网卡承担的处理减少主机 CPU 的辅助工作。

将包处理移到网卡后，部分操作可以在数据到达时直接完成，CPU 的工作随之减少。如果网卡还需要主机内存中的数据，就要通过 PCIe 读取；此时，返回延迟和可同时发出的请求数决定了读取速度。第 7 章将用 KV-Direct（让可编程网卡直接处理键值存储请求的系统）分析这种访问。键值存储按唯一的键查找、读取或更新对应的值。

[15] Firestone 等，微软，Azure Accelerated Networking: SmartNICs in the Public Cloud，NSDI 2018；原论文文本。摘要给出 2015 年末部署、2016 年向客户提供服务的时间，第 3 节说明减少 CPU 占用、保持可编程性与支持更高带宽的设计目标。

[16] 笔者博士论文，第 4.2.1 节简单转发基线；核数预算复算。约六千万包每秒来自 40 Gbit/s 与每帧占用 84 字节线时的计算，3 至 6 核对应简单转发基线。

### 1.4.3 Unified Bus

SmartNIC 改变了单台服务器内部的分工。若问题是单个加速器的资源总量不足，就需要进一步考虑加速器之间如何协作。当单个加速器容纳不下模型和运行状态时，最直接的办法之一是增加加速器。但“总容量变大”与“任务能够高效执行”之间，还隔着工作划分、数据交换和同步。

例如，两张卡的显存总量可能足够保存权重，但执行计算的加速器必须能够访问所需的数据。若一步计算依赖另一张卡的结果，就要等待交接；若多张卡共同完成一个操作，还要组织相应的协作。加速器数增加带来了资源，也增加了连接这些资源的工作。

华为面向昇腾等异构计算设备推进的 **UB（Unified Bus，统一互联）**，把问题扩大到多设备协作。笔者于 2020—2023 年参与这一项目，相关研究在 2019 年已经启动，早于 OpenAI 于 2020 年发布自回归语言模型 GPT-3；GPT-3 展现出大模型的能力后，业界更广泛地认识到多设备协同的需求，项目投入随之扩大。模型训练所需的算力增长远快于单个加速器能力的提升，两者之比不断扩大，协同工作的加速器只能越来越多。

这一时期的核心需求是让规模不断增长的模型使用多张加速卡，并使计算设备更方便地访问其他设备上的内存和数据。数据一旦跨过主机边界，软件就要改用消息传递接口、重新安排缓冲区，再经过网卡驱动和协议栈，每多一层抽象就多一段时间。UB 让设备直接访问其他设备的内存，把这些抽象层去掉，使一次远程访问的耗时逼近线路时延决定的下限，上层也能更灵活地组织资源。统一的访问机制使设备能直接使用更大范围内的资源，拓扑则决定这些访问的距离与带宽。模型分工与互联设计由此紧密相连。第 6.5.5 节从超节点规模的角度讨论 UB 的组织方式，第 7.3 节和第 7.4 节沿着一次远程访问的路径推导它的延迟、请求速率和连接状态，并算出去掉的各层抽象原来各占多少时间。

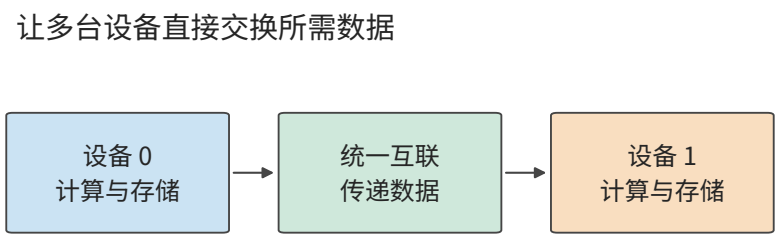



图 1-17　统一互联连接不同设备的计算与存储资源。模型分工确定要交换什么，互联负责把数据送到后续使用它的设备。

以本章的两卡模型部署为例：权重均分可以满足逐卡容量约束，但每个阶段还要等待输入数据和前序结果。增加加速器改变了 $M_{\text{cap}}$ 和可用算力，也引入新的通信量与依赖。因此，第一步先检查每个加速器能否容纳所需数据，第二步计算各加速器的运算量与读写量，第三步根据执行依赖计算总时间。模型再大、加速器再多，这一分析顺序仍然成立。

### 1.4.4 变化的逐级传递

三个案例中的设计一经部署，又会带来新的约束。FPGA 部署到 Azure 服务器以后，云平台需要负责 FPGA 逻辑的更新和故障处理[17]；UB 把大量加速器连成一体以后，每次同步要等待的设备增多，可能出错的部件也增多。在 AI 系统中，这种逐级传递仍在继续，下表按本书章节顺序列出主要环节。

[17] Firestone 等，微软，Azure Accelerated Networking: SmartNICs in the Public Cloud，NSDI 2018；原论文文本。摘要给出 2015 年末部署、2016 年向客户提供服务的时间，第 3 节说明减少 CPU 占用、保持可编程性与支持更高带宽的设计目标。

| 变化 | 不再成立的假设 | 章节 |
|---|---|---|
| 神经网络的执行时间集中在矩阵运算 | 通用处理器能以合理的面积与功耗完成主要计算 | 第 4、5 章 |
| 模型权重超过单卡显存 | 一个模型可以放在一张卡上 | 第 6 章 |
| 训练与推理跨越多台服务器同步执行 | 网络流量来自相互独立的数据流，可以统计复用 | 第 7 章 |
| decode 每读一遍权重只完成少量计算 | 峰值算力决定执行速度 | 第 8 章 |
| 上下文变长，KV 的规模接近甚至超过权重 | 上下文状态较小，可以随时重算 | 第 9 章 |
| 训练规模达到上千张卡 | 故障是偶发事件 | 第 10 章 |
| Agent 与 RL 负载成批创建工具环境 | 容器中的程序相互独立，各自决定运行节奏 | 第 11 章 |
| 端侧设备能够运行模型 | 计算集中到云端总能缩短完成时间 | 第 12 章 |

与通用操作系统和云平台相比，AI 系统有一个有利条件：负载种类少，而且事先已知。通用平台要运行事先未知的程序，只能依靠统一的抽象、严格的隔离和统计复用；AI 系统围绕少数模型和确定的执行过程设计，可以提前掌握第 1.1.1 节所说的计算与数据依赖，并据此打破原有的抽象边界。例如，第 11 章的 RL 沙箱平台根据训练框架提供的抢占信息，主动暂停相应的沙箱。

AI 系统中的许多单项技术并非新发明：第 8 章的 KV 分页管理借用操作系统的分页，第 11 章的沙箱放置采用 2001 年提出的 power-of-k-choices 随机选择算法，虚拟机内存回收沿用 2002 年提出的 balloon 机制。[18]比例改变以后，原先不划算或用不上的方法又变得合算；一项技术是否适用，仍需按当前的比例重新计算。

从语音推理到云网络，再到大模型的多设备执行，需求与硬件的变化决定了首先要解决哪个瓶颈，也重新划分了硬件、软件与互联之间的工作。对照开篇的六层图，这些设计都重新组织了计算与数据搬移。三条线索贯穿后续各章：比例的变化解释为什么需要新的设计，数据搬移说明新设计改变了什么，第 1.3.4 节的物理极限衡量设计达到了什么程度。本书的主线由此展开：**数据搬移塑造了 AI Infra 的架构。**

## 谬误与陷阱

**误区：峰值算力翻倍，执行速度就会翻倍。**先检查当前执行是否受计算能力限制。例题 1-1 中，权重读取的时间下界远大于矩阵计算的时间下界，提高矩阵峰值算力几乎不改变执行时间的下界；若优化的是完整任务的一部分，还要用 Amdahl 定律检查收益的上限。

**误区：显存能容纳权重，就能支持目标并发数。**权重只是常驻数据的一部分。上下文状态、工作区和运行时预留的空间共同占用显存，多加速器还必须逐卡核算。

**误区：比旧实现快了三成，优化就算成功。**不知道硬件允许的极限，就无法判断这三成是逼近了上限，还是离上限仍有一个数量级。应当先算出极限，再按第 1.3.4 节的顺序查明差距来自模型漏项还是系统开销。

**误区：在上一代场景中成熟的系统，可以直接用于新的负载。**每种系统设计都以设计时的比例为前提。通用容器平台假定请求来自相互独立的租户，程序自行决定运行节奏；RL 训练中的沙箱却成批创建，节奏由 GPU 一侧决定，照搬原有设计会让昂贵的加速器空等（第 11 章）。沿用一种设计之前，应先检查它所依赖的比例是否仍然成立。

[18]power-of-k-choices 见 M. Mitzenmacher, The Power of Two Choices in Randomized Load Balancing, IEEE TPDS, 2001；balloon 机制见 C. A. Waldspurger, Memory Resource Management in VMware ESX Server, OSDI 2002。两篇文献均由 DSec 论文引用，相应机制见第 11.2.2 节与第 11.3.2 节。

**误区：吞吐率提高，每个用户的等待时间就会缩短。**批内复用减少每个输出分摊的读取开销，但每个请求仍须经历排队和整批执行。吞吐与响应时间应同时报告。

## 本章数字速查

本表用第 1.3.3 节实测采用的 Qwen3-8B 汇总模型侧的关键数字。prefill 一次处理已有输入，并得到首个输出；decode 每次把一个新 token 送入模型，利用已经保存的 KV 继续生成。本表固定 BF16 权重、激活和 KV，并发为 1；prefill 的输入为 2048 个 token，decode 则在已有 2048 个 token 的上下文后执行一步。[19]

| 模型需求 | 数值 | 如何使用 |
|---|---|---|
| 完整 BF16 权重 | 16.38 GB | 判断模型加载所需的显存 |
| 每个上下文 token 的全模型 KV | 144 KiB | 上下文长度乘以此数，得到单请求 KV 容量 |
| 2048 个 token 的上下文的 KV | 288 MiB | 约 0.302 GB，随独立请求数增加 |
| 2048 个 token 的 prefill 的矩阵运算 | 29.69 TFLOPs | 除以相应矩阵算力，得到计算时间下界 |
| 一步 decode 的矩阵运算 | 16.34 GFLOPs | 输入为一个 token，另访问已有上下文 |
| 一步 decode 的主要权重读取 | 15.14 GB | 权重读一次；嵌入层（把 token 编号映射为向量的查找表）只读取当前 token 对应的一行 |
| 一步 decode 的旧 KV 读取 | 0.302 GB | 每份上下文 KV 读一次时的字节数 |

**用计算量与读取量估算执行时间下界。**这里采用图 1-13、1-14 实测所用的 RTX PRO 6000 Blackwell Workstation Edition，先单独计算矩阵运算和数据读取：按 503.8 TFLOP/s 矩阵峰值换算，prefill 的矩阵运算约需 58.9 ms，decode 约需 0.0324 ms；按 1.792 TB/s 显存带宽，一步 decode 的主要权重与旧 KV 读取约需 8.62 ms。实测 batch 为 1 时的输出间隔约 26.5 ms，约为该读取下界的三倍，差距的来源见第 1.3.4 节。这里分别估算计算与读取所需的最短时间，完整执行还要沿算子顺序加入其他运算、实际访存和启动时间。第 2—5 章将逐项建立这些计算。

这组数字解释了两阶段的差别：prefill 在一次调用中处理大量输入，矩阵工作量较大；单请求 decode 每次只处理一个 token，却仍要读取大量权重和上下文。提高计算吞吐、提高带宽和增加批内复用，分别作用于不同的时间项。

## 本章小结

应用用模型与上下文表达行为，许多任务因此共享相近的底层计算。掌握模型中的计算依赖和状态驻留时间，就能统筹考虑原来分属不同层次的设计选择：改变数据表示、合并执行步骤、复用准备工作，或缩短加速器间数据传递和同步的时间。后续各章将量化这些改动的收益，说明哪些设计取舍因此改变，再根据新的约束选择模型与系统方案。

分析系统时，先检查数据能否放下，再用运算量和读写量估算耗时，最后根据执行顺序分析重叠与等待。按峰值算出的下界就是硬件的物理极限，下界与实测之比就是利用率；两者的差距不是模型漏了项，就是系统有可以去掉的开销。在 70B 算例中，权重读取比矩阵计算慢约 148 倍，因此增加带宽、减少读取和提高复用更有助于缩短时间。批内共享可以提高吞吐，而每个请求的响应时间还要包括整批执行和排队。

[19] 速查数字复算源码根据固定 Qwen3-8B 配置统计 prefill 与 decode 的矩阵运算，读取量来自批内权重复用计算。权重占用包含完整嵌入表，逐步生成时只读取当前 token 对应的嵌入行；读写时间采用每份主要数据访问一次的理想条件。

本章核心练习为 1-2、1-3、1-4，分别练习单位换算与可行性判断、条件改变后的资源估算，以及预测与实测结果的比较。下一章将根据真实模型配置，细化 $2N$ 计算量与权重读取量的估算，为这些计算提供具体的矩阵尺寸与状态大小。

# 第 2 章 模型架构

第 1 章从存储容量、运算量和数据读取量三个方面分析了一次模型执行，并借助硬件的关键数字估算时间下界。这样的估算可以迅速排除一些方案，却还留下一个问题：所代入的计算量和数据量从哪里来？“一个 8B 模型”（参数量后缀 B 表示十亿，8B 即约 80 亿个参数）只说明参数数量的大致范围，没有说明每个 token 使用哪些参数、上下文保存在什么地方，也没有说明不同工作之间的先后关系。

本章从一个 token 的数值表示开始，逐步分析它经过网络各层后，如何得到下一个 token 的分数，再计算这一过程所需的存储容量 $M$、运算量 $F$ 和读写量 $R$。计算图用节点表示运算，用边表示运算之间传递的数据。先认识图中的运算和数据，才能知道资源预算中的每个数字来自哪里。

这张计算图也提供了优化实现所需的信息。哪些权重在不同请求间保持不变，哪些状态后续生成会反复用到，哪些结果只供紧接着的运算使用，都可以从模型结构中找到。后续章节设计存储、编译和调度时，将利用数据用途上的这些差异，决定哪些数据可以共享、哪些需要长期保存、哪些只需暂存。

本章选择五个典型模型：DeepSeek V4.1 Flash、Qwen3-8B、Qwen3.6-35B-A3B（名称中的 35B 指总参数约 350 亿，A3B 指每个 token 实际激活的参数约 30 亿；下文简称 Qwen3.6）、DeepSeek V4-Flash 与 Kimi K3。2026 年 9 月发布的 V4.1 Flash 用 CED 架构分开输入处理与生成，并结合跨层 KV 共享、分层索引与 Engram（按 n-gram，即连续 n 个 token 组成的片段，查表取得附加表示的模块），为理解模型架构如何改变系统需求提供了新的案例。

推导从 Qwen3-8B 开始：它约含 81.9 亿参数，通过 36 层网络处理文本。理解这一稠密模型的执行过程后，再逐步改变上下文的保存方式、每个 token 使用的参数集合和层间连接，最后把五个模型放在相同输入条件下比较。

开始推导前，先约定本章的计数口径。统计运算量时，一次乘法加一次加法计为 2 FLOPs。存储量采用 GB 和 GiB：1 GB $= 10^9$ bytes，1 GiB $= 2^{30}$ bytes。BF16 每个元素占 2 字节，因此将张量的元素数乘以 2，就得到它的存储大小。

## 2.1 计算图与前向执行

张量是按一个或多个维度排列的数值数组：向量只有一个维度，矩阵有两个维度。模型用向量表示每个 token 的特征，把多个 token 的向量按行排成矩阵。理解模型资源需求，首先要把计算节点和数据依赖分开。一个节点执行矩阵乘法或其他变换，一条边表示后续节点需要前面的结果。节点的运算量决定计算需求，边上的张量决定传递的数据量，边的方向则表明哪些运算必须先后执行。理解模型的执行过程，需要同时考虑这三个方面。

本章的计算图先描述数学运算和数据依赖。实际执行时，框架会把这些运算交给加速器程序；一个运算可能拆成多个程序，多个运算也可能合在一起执行。计算图中表示张量的一条边，只说明后续运算需要这份数据，不表示它一定要在 CPU 与 GPU 之间复制。第 4 章说明数据放在哪里，第 5 章再解释程序如何提交、数据如何传入传出。

### 2.1.1 序列依赖与因果计算图

要计算执行时间，先要知道哪些运算能够同时开始。考虑四个 token、三层网络，横轴是 token 在序列中的位置索引，纵轴是层数。本章用 “词元” 指词表中的一个条目，用 token 指它在序列中的一次出现；同一个词元在序列中出现多次时，位置索引用来区分它们。循环神经网络（RNN）把前一 token 位置的状态传给后一 token

位置，token 状态满足 $h_t = f(x_t, h_{t-1})$。其中 $x_t$ 是当前输入，$h_t$ 是处理到当前 token 位置时保存的状态，$f$ 是用模型权重做的变换。当前结果依赖前一 token 位置，所以同一层按位置顺序执行。即使四个输入已经全部到达，第 $t$ 个 token 仍要等待第 $t-1$ 个 token。

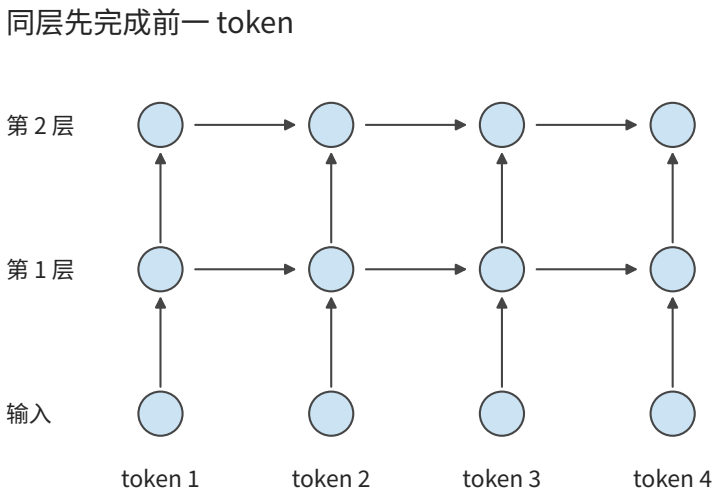



图 2-1　循环神经网络的依赖。相同颜色的圆点表示各 token 位置的计算，箭头指向需要前一结果的节点；同层 token 位置沿横向逐次推进。

Transformer 用注意力机制让一个 token 读取其他位置的信息；“因果” 表示当前 token 位置只能使用自身及此前位置的信息。因果 Transformer 的依赖形式不同。第 $l$ 层计算某个位置时，通过注意力读取上一层中该位置及其之前各 token 位置的表示。只要输入 token 已知，并且上一层的相应表示已经完成，同一层不同位置的计算就不需要再等待本层前一个 token 的输出。因果掩码用来规定各 token 位置允许访问哪些信息，屏蔽当前 token 位置之后的信息。按查询与被查询 token 位置排列，掩码呈三角形；掩码限制信息来源，却不要求所有已知 token 逐个执行。网络深度方向的依赖仍然存在：下一层使用这一层的结果。

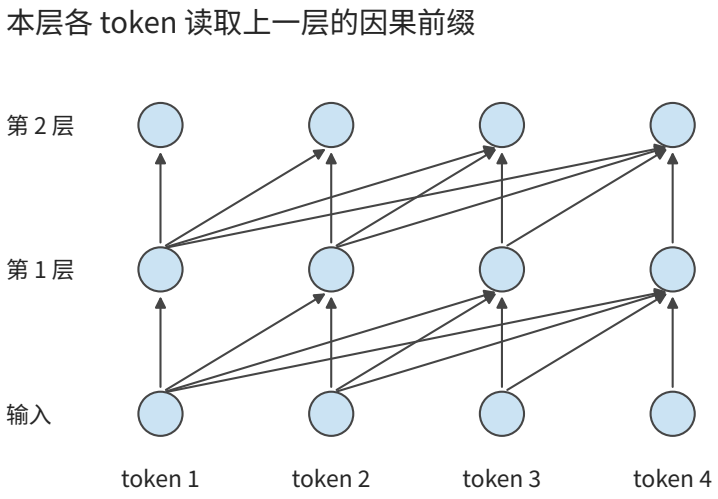



图 2-2　因果 Transformer 的层间依赖。各查询读取上一层中允许访问的位置；上一层完成后，同一层的已知输入 token 可以并行计算。

沿图中的箭头追踪，便能区分两种等待：RNN 在同层等待前一 token 位置，因果 Transformer 在下一层等待所需的上一层结果。已知输入能否并行，取决于这些依赖，而非 “序列” 这一名称。生成新的 token 又增加了另一条依赖：下一次调用的输入由本次输出确定。

处理已知输入与逐步生成是两类不同的工作。编码器把已知输入转换为表示，解码器根据允许访问的信息

逐步产生输出。现代因果语言模型常用 decoder-only 结构，本章先用这一结构建立基础，再讨论 V4.1 Flash 如何通过 CED 架构重新划分输入处理与生成的工作。处理已知输入时，同层 token 位置可以并行；生成时，第一个输出确定后才能成为下一次输入。因此，输入处理沿网络深度推进，生成还增加了跨调用的串行依赖。[1]

> **练习 2-1〔延伸〕：RNN 与 Transformer 的依赖如何限制并行计算**
> 对于含四个已知输入 token 的序列，分别画出三层 RNN 和三层因果 Transformer 的计算依赖图。再增加一个待生成的输出位置，指出计算该位置之前必须等待哪个节点完成。设每层每 token 的计算耗时为一个单位、并行资源充足，分别求处理全部已知输入时的最长依赖链长度；解释增加计算单元可以缩短哪些等待、不能缩短哪些等待。

### 2.1.2 模型配置与张量结构

先把一个计算节点展开。设本次对 $m$ 个输入 token 执行投影，每个 token 用包含 $k$ 个特征的向量表示。把这些向量逐行排成输入矩阵 $X$，就得到 $m$ 行、$k$ 列的矩阵。设输出宽度为 $n$，则有：

$$X \in \mathbb{R}^{m\times k}, \qquad W \in \mathbb{R}^{k\times n}, \qquad Y = XW \in \mathbb{R}^{m\times n}.$$

每个输出元素是长度为 $k$ 的点积，输出共有 $mn$ 个元素，因此

$$N_W = kn, \qquad F = 2mkn, \qquad M_W = b_W kn.$$

$N_W$ 是权重参数的数量，$b_W$ 是每个参数占用的字节数。行数 $m$ 翻倍，运算量翻倍，权重大小保持不变；输出宽度 $n$ 翻倍，权重数量与运算量都翻倍。这两个变化分别对应“同一模型处理更多输入”与“模型的一层变得更宽”。下面的模型表都可以用这条规则逐行计算。

先只看 Qwen3-8B 的一条数据路径。一次**前向执行**是使用当前权重从输入算到输出的过程；隐藏向量是网络内部传递的特征表示，其维度即每个 token 的向量含多少个数。**前馈网络**（feed-forward network，FFN）对每个 token 分别做特征变换；**注意力**则在 token 之间选择并汇总信息。这两类运算交替组成网络的一层。

Qwen3-8B 包含 $L = 36$ 层，隐藏维度为 $d = 4096$，FFN 中间维度为 $f = 12288$；注意力把特征分成多组并分别建立 token 之间的关系，每一组称为一个头。这里有 32 个查询头和 8 个供查询使用的 KV 头，每头 128 维；第 2.1.3 节将沿查询、键和值的路径解释它们。词表包含 151936 个 token，词嵌入是一张把 token 编号映射为向量的查找表，输出头则把末层向量变成每个候选 token 的分数。两者各自保存一份权重。各层参数与这两份词表矩阵相加，模型共有约 81.9 亿个参数。[2]

一次前向执行从 token ID 开始。词嵌入为每个 ID 查出一个 4096 维向量，向量依次经过 36 层注意力与 FFN，最后经过归一化和词表投影产生 logits，即尚未转换为概率的候选 token 分数。归一化使逐层计算的数值保持在适当范围内。词嵌入按 ID 选择行，一个 token 取出一条向量，整个词表则作为常驻数据供不同 token 查询。词表输出头将隐藏向量与词表矩阵相乘；只生成下一个 token 时，通常只需要最后一个输入 token 的 logits。

设同时处理 $B$ 条等长请求，每条本次输入 $P$ 个已知 token。投影和 FFN 的输入可以表示为 $X \in \mathbb{R}^{BP\times 4096}$，每一行是一个 token 在当前层的表示，共有 $m = BP$ 行。已有上下文 token 的键和值从缓存读取，不计入这里的 $m$。每条请求只补入一个 token 时，$m = B$。第一种情形容易形成行数较大的矩阵，第二种在小 batch 下接近矩阵与向量的乘法。权重相同，输入形状却明显不同。

[1] 序列依赖计算记录。

[2] Qwen3-8B 配置与参数计算，参数总数为 8,190,735,360。

下面比较两种输入形状：一条请求一次处理 8192 个输入 token，以及一条已有 8192 个上下文 token 的请求再处理一个新 token。用 $B$ 表示请求数，$S$ 表示调用前已有的上下文长度，$P$ 表示本次输入长度，$d$ 表示隐藏维度。前者取 $B=1, S=0, P=8192$，后者取 $B=1, S=8192, P=1$。

| 配置项 | Qwen3-8B 参数 | 对资源需求的直接影响 |
|---|---|---|
| 层数 $L$ | 36 | 重复的计算、逐层状态与深度依赖 |
| 隐藏维度 $d$ | 4096 | 子层输入输出与投影尺寸 |
| FFN 中间维度 $f$ | 12288 | 三个 FFN 矩阵的权重与计算 |
| $Q$ / KV 头数 | 32 / 8 | 查询向量与上下文表示的宽度 |
| 头维度 $d_h$ | 128 | 每个头的点积与状态宽度 |
| 词表大小 $V$ | 151936 | 嵌入和输出头规模 |

**例题 2-1：从矩阵尺寸求一层的参数量与运算量。** Qwen3-8B 的查询和输出投影宽度为 4096，键和值投影宽度为 1024，FFN 中间维度为 12288。求四个注意力投影与三个 FFN 矩阵的参数量，以及为本次输入的 $m$ 个 token 完成这些投影和 FFN 计算所需的运算量。每个 token 在这一层都有一个 4096 维向量，作为输入矩阵的一行；$m$ 统计本次实际参与计算的 token 数。同一个词元出现两次，就算作两个 token。若有 $B$ 条等长请求、每条本次处理 $P$ 个 token，则 $m=BP$。例如，一条请求处理 128 个输入 token 时 $m=128$；八条请求各推进一个 token 时 $m=8$。

解：查询与输出各使用一个 $4096\times4096$ 矩阵，键和值各使用一个 $4096\times1024$ 矩阵，三个 FFN 矩阵各有 $4096\times12288$ 个参数。因此，

$$
\begin{aligned}
N_{\text{proj}} &= 2\times4096^2 + 2\times4096\times1024 = 41{,}943{,}040,\\
N_{\text{FFN}} &= 3\times4096\times12288 = 150{,}994{,}944,\\
F_{\text{linear}} &= 2m(N_{\text{proj}}+N_{\text{FFN}}) = 385{,}875{,}968m.
\end{aligned}
$$

此处尚未使用上下文长度：这些投影对每个 token 的特征向量做相同的变换。下一节将介绍查询与上下文之间的注意力计算，其运算量与查询—键配对的数量有关。将两类工作分开，就能看出“增加本次参与计算的 token 数”和“让每个 token 读取更长的上下文”的代价为什么不同。

| 符号 | 含义与使用范围 |
|---|---|
| $B, P, S$ | 请求数、本次每请求输入 token 数、已有上下文长度 |
| $m=BP$ | 本次参与投影的 token 总数，也就是输入矩阵行数；普通 decode 时 $P=1$，故 $m=B$ |
| $d, f$ | 主干隐藏宽度、前馈网络中间宽度 |
| $n_Q, n_{\text{KV}}, d_h$ | 查询头数、KV 头数、每头维度 |
| $N_{\text{pair}}$ | 每层所有请求合计的有效因果查询—键配对数，不含头数 |
| $t_e$ | 分派给专家 $e$ 的 token 数（见第 2.4 节），每个 token 的特征向量占该专家输入矩阵的一行；$k_{\text{top}}$ 为每个 token 选中的专家数，路由过程中不丢弃分派给专家的 token 时 $\sum_e t_e = mk_{\text{top}}$ |
| $m_{\text{out}}$ | 实际执行词表投影的 token 数；只用每条请求的末尾 token 预测下一个 token 时为 $B$，它不等于本次生成的全部输出长度 |

\(S\)：请求开始时，已有 KV 缓存的 token 数。
\(P\)：本次 prefill 新处理的输入 token 数。
\(H=S+P\)：prefill 完成后的 KV 缓存 token 数；字节大小是 \(c_{\mathrm{KV}}H\)。
\(j\)：decode 的步骤编号，从 0 开始；第 \(j+1\) 步开始时，已有 \(H+j\) 个缓存 token。
\(G\)：最终返回的输出 token 数。
\(n_d=G-1\)：decode 的总步数，因为第一个输出 token 在 prefill 阶段产生。

计算整个模型的资源需求时，先按表格中每项操作的矩阵尺寸求一次调用的运算量，再乘这一类层的数量。词表头另外按实际输出 token 数 $m_{\text{out}}$ 计算；生成下一 token 时，每条请求只需最后一个 token 的分数。归一化、激活与查表操作各有自己的运算方式，后文沿数据流分别说明。

### 2.1.3 注意力计算与位置编码

先跟踪单个 token 的计算。模型从该 token 的隐藏向量产生三种表示：**查询** $Q$ 用来提出“当前需要什么信息”，**键** $K$ 用来与查询计算匹配分数，**值** $V$ 是被选中后汇总到输出的信息。三者都由同一个 token 的输入经过不同权重矩阵产生。线性投影就是这样的矩阵乘法：把输入特征变换到另一组坐标。

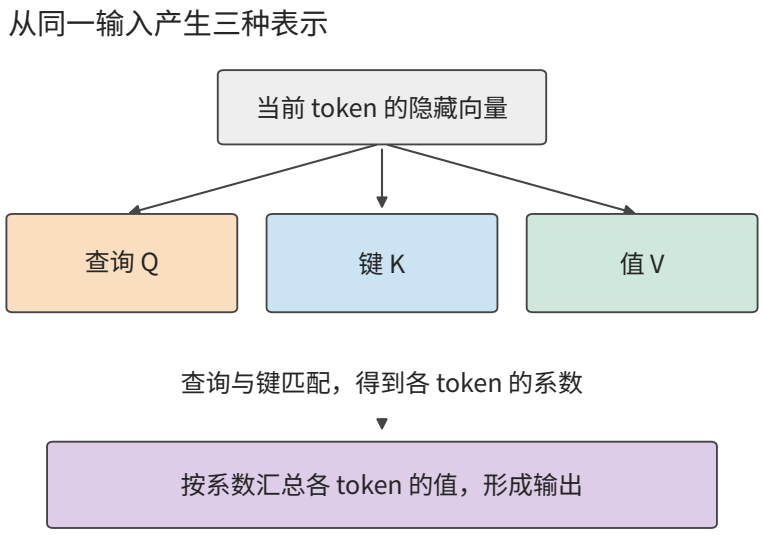



图 2-3　同一个 token 的输入经不同投影产生查询、键和值。注意力先用查询与键计算 token 之间的关系，再用这些关系汇总值。

采用数学右乘记号，输入 $X$ 分别乘三个权重矩阵，得到 $Q$、$K$、$V$。$Q$ 的总宽度为 $32 \times 128 = 4096$，$K$ 和 $V$ 的总宽度各为 $8 \times 128 = 1024$。因此，$W_q$ 为 $[4096, 4096]$，$W_k$ 和 $W_v$ 各为 $[4096, 1024]$。代码中的线性层常以 $(d_{\text{out}}, d_{\text{in}})$ 保存权重；换成数学右乘记号后，输入宽度在前、输出宽度在后，便可逐项对照下面的表格。

上述投影产生了 $Q$、$K$、$V$，但尚未让当前 token 位置与上下文交换信息。接下来将 $Q$ 拆成 32 个头，把 $K$、$V$ 拆成 8 个头。每组四个 $Q$ 头共用一个 KV 头。对一个 $Q$ 头，查询向量与允许访问位置的 $K$ 做点积，将点积除以头宽的平方根，避免分数的数值尺度随头宽增大而过度增长，然后屏蔽未来 token 位置。Softmax 把允许位置的分数 $s_j$ 变成正数并归一化，系数为 $a_j = e^{s_j} / \sum_i e^{s_i}$；这些系数之和等于一，决定各上下文 token 贡献多少信息。这里的“注意力权重”是本次输入算出的系数，区别于训练后保存的模型参数。随后用这些权重对 $V$ 加权求和。各 $Q$ 头的结果拼接后，经 $[4096, 4096]$ 的输出投影 $W_o$ 转回隐藏向量。

用矩阵记号写出一个头的计算：

$$A = \text{Softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_h}} + \mathcal{M}\right), \qquad O = AV.$$

$A$ 是各查询对上下文 token 的权重，$\mathcal{M}$ 是因果掩码，$d_h$ 是头维度。点积决定哪些位置与当前查询相关，Softmax 将分数转成加权系数，再用这些系数汇总值向量。Qwen3 在点积前还要做**查询与键归一化（QK Norm）**

和**旋转位置编码（Rotary Position Embedding，RoPE）**：前者调整各头查询与键的数值尺度，后者通过随位置变化的旋转，使点积包含相对位置信息。

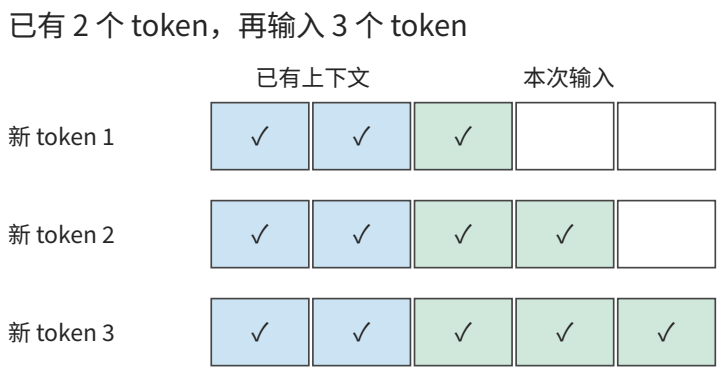



图 2-4　两段上下文形成长方形加三角形。蓝色为三个新 token 分别读取两个旧 token，绿色为新输入内部的因果访问，空白为被屏蔽的未来 token 位置。每个有色格表示一次查询—键配对；横轴对应被读取的 token，纵轴对应新输入的查询 token。

下面计算点积的数量。输入 $P$ 个新 token、已有 $S$ 个上下文 token 时，第 $i$ 个新 token 可访问 $S+i$ 个 token，其中 $i=1,\dots,P$。将各查询可以访问的上下文 token 数相加，就得到一个长方形加一个三角形：

$$N_{\text{pair}} = B\left(PS + \frac{P(P+1)}{2}\right)$$

对每一对查询与上下文 token，一个 $Q$ 头的 $QK^{\mathsf{T}}$ 点积约需 $2d_h$ FLOPs，随后 $AV$ 的贡献又约需 $2d_h$ FLOPs。合计所有 $Q$ 头，$QK^{\mathsf{T}}$ 与 $AV$ 的有效矩阵运算量为：

$$F_{\text{attn}} = 4n_Q d_h N_{\text{pair}} = 16384 N_{\text{pair}}$$

在此前没有缓存上下文的 prefill 中，序列长度加倍时，投影的输入行数变为两倍，全注意力的查询—键配对数则约为原来的四倍，因为新加入的 token 也要与此前的上下文交互。计算这些查询—键配对有两种常见路径：完整矩形乘法先产生所有分数再屏蔽上三角，因果分块则跳过上三角。分块还可以把评分、归一化和 $V$ 汇总接在一起，让分数在片上用完即释放。第 5 章将沿这条数据流解释如何减少中间结果的显存读写。

因此，保持 $BP$ 不变只能保持投影行数不变，不能保持上下文交互不变。例如，把一条长输入拆成两条彼此独立的短输入，原来跨越分界的那些查询—键配对就消失了。相反，仅在同一请求前面增加上下文，会增加每个新 token 的点积数，输入投影的行数却保持不变。

### 2.1.4 前馈网络与前向计算量

注意力负责不同 token 之间的信息交互，FFN 则对每个 token 的特征执行非线性变换。SwiGLU 是一种带门控的前馈结构：一条分支变换输入特征，另一条分支为它生成逐元素的调节量。SwiGLU 采用 SiLU 激活函数 $\text{SiLU}(x) = x/(1+e^{-x})$，用非线性变换改变特征间的组合方式。Qwen3-8B 使用这种结构：输入分别经过 gate 与 up 两条升维投影，gate 分支经过 SiLU 后，再与 up 分支逐元素相乘，最后通过 down 投影降回隐藏维度：

$$\mathrm{FFN}(X) = \left[\mathrm{SiLU}(XW_{\mathrm{gate}}) \odot (XW_{\mathrm{up}})\right] W_{\mathrm{down}}$$

式中 $\odot$ 表示对应元素相乘；gate 是门控分支，up 是升维分支，down 是降维投影。三者产生的中间向量称为激活。gate 与 up 将每行从 $d$ 维升到 $f$ 维，down 再降回 $d$ 维。三个矩阵因此合计 $3df$ 个参数，处理 $m$ 个 token 的特征向量需 $6mdf$ FLOPs。Qwen3-8B 取 $d = 4096, f = 12288$，FFN 约有 1.51 亿个参数，BF16 权重占 288 MiB。升维后较宽的激活也需要临时空间，但只需在相应计算期间保留。

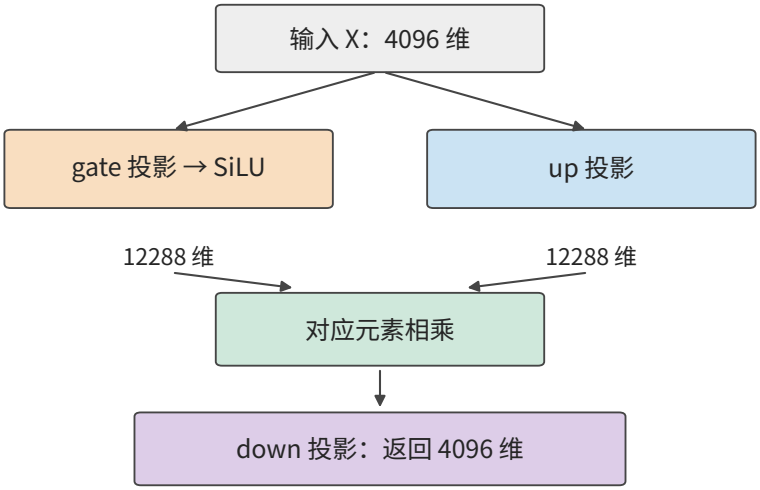



图 2-5　SwiGLU 的两条升维分支。gate 经 SiLU 后调节 up 的对应元素，乘积再由 down 投影降回主干维度。

把两类投影放在一起比较：注意力的 QKV 与输出投影合计约有 4194 万个参数。加上 FFN 的约 1.51 亿个参数后，FFN 占这些主要投影参数的约 78%，因此短上下文的计算中，三个 FFN 矩阵占据较大份额。随着输入变长，FFN 的运算量随 token 数线性增长，全注意力的运算量则随 $P(P+1)/2$ 增长。两者增长速度不同，因此各部分计算量的占比会随上下文长度变化。

除矩阵变换外，一层之内还有子层之间的连接。残差连接把子层的输入保留下来，在该子层完成变换后，与输出逐元素相加。这样，一条路径执行新变换，另一条路径直接传递已有表示。下面把注意力、前馈和两次残差连接连成完整的一层。

**表 2-1　Qwen3-8B 的单层模块与输出接口**

注意力和前馈各行按 36 个主干层累计；词嵌入根据 token 编号读取词表中对应的行，不再按一次词表矩阵乘法（GEMM，通用矩阵乘法）计算。注意力的 $K$、$V$ 只各有 1024 维，而查询仍有 4096 维。

**模型入口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词嵌入查行 | $m$ 个 ID → $m \times 4096$ | 查表 $151936 \times 4096$ | 0；查行访问另计 | 1 |

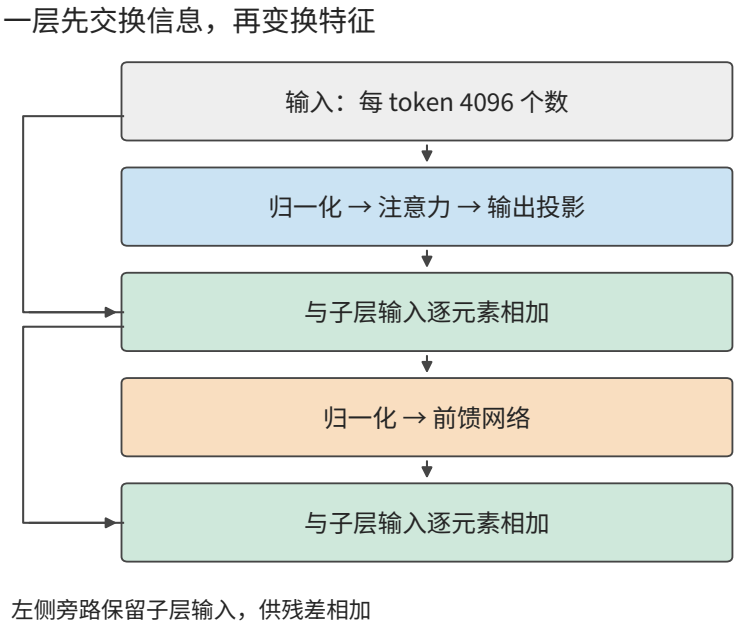



图 2-6　Qwen3-8B 一层的骨架。先完成注意力子层，再完成前馈子层；左侧的残差连接保留子层输入，再与变换结果相加。

**注意力**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| $Q$ 投影：形成查询 | $m \times 4096 \to m \times 4096$ | $4096 \times 4096$ | $2 \times m \times 4096 \times 4096$ | 36 |
| $K$、$V$ 投影：形成可复用上下文 | $m \times 4096 \to m \times 1024$ | $4096 \times 1024$，共 2 个 | $2 \times 2 \times m \times 4096 \times 1024$ | 36 |
| $QK^{\mathsf{T}}$ 与 $AV$：按内容汇总上下文 | 32 个查询头，每头 128 维 | 无新增投影权重 | $4 \times 32 \times 128 \times N_{\text{pair}}$ | 36 |
| 输出投影：合并各头 | $m \times 4096 \to m \times 4096$ | $4096 \times 4096$ | $2 \times m \times 4096 \times 4096$ | 36 |

单次点积用的是一个Q头与他对应的一个K头，俩边都是128维

**前馈与连接**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| SwiGLU：逐 token 特征变换 | $m \times 4096 \to m \times 12288 \to m \times 4096$ | $4096 \times 12288$ 两个；$12288 \times 4096$ 一个 | $6 \times m \times 4096 \times 12288$ | 36 |
| 归一化、RoPE、激活与残差 | 保持或逐元素变换上述张量 | 归一化向量等 | 按元素计算 | 36 |

**输出接口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词表头：转为 token 分数（整模型末端一次） | $m_{\text{out}} \times 4096 \to m_{\text{out}} \times 151936$ | $4096 \times 151936$ | $2 \times m_{\text{out}} \times 4096 \times 151936$ | 1 |

表中有两种增长方式：投影与 FFN 的工作随 $m$ 增长，上下文交互随 $N_{\text{pair}}$ 增长。FFN 的三次大投影解释

了它为什么在短上下文中占有较大的计算份额；上下文变长后，上下文交互的占比逐渐提高。

将每层的线性变换与上下文交互相加，再加入词表头，得到 Qwen3-8B 的主要矩阵运算量：

$$F_{\text{matrix}} = 36\left(385{,}875{,}968m + 16{,}384N_{\text{pair}}\right) + 2m_{\text{out}} \times 4096 \times 151936.$$

式中第一项随输入行数增长，第二项随查询—键配对数增长，最后一项随实际执行词表投影的 token 数增长。对包含 8192 个 token 的输入，按有效因果查询—键配对计算注意力，词表头只处理最后一个 token，共需 133.6 TFLOPs；已有 8192 个上下文 token、再补入一个 token 时，共需 20.0 GFLOPs。[3]

prefill 同时处理 8192 个输入 token，每个 token 都要经过投影和 FFN。decode 只对新 token 执行这些运算，旧 token 的 $K$、$V$ 则从缓存读取。因此，缓存省去了旧 token 的投影和 FFN 重算，当前查询与上下文的注意力计算仍要执行。下一节计算这份缓存的容量和访问量。

> **练习 2-2〔核心〕：输入行数相同，注意力计算量为何不同**
> 使用表 2-1，取 $B = 4, S = 4096, P = 1024$，分别计算投影与 FFN、有效因果注意力，以及对每条序列的最后一个 token 执行词表投影所需的 FLOPs。保持 $BP = 4096$，改为 $B = 1, S = 4096, P = 4096$，先预测哪些项相同，再计算差异。最后以一条含 8K 输入的请求为例，比较完整处理输入与复用前缀两种方式。复用时保留前 6144 个 token 的 KV，只处理新增的 2048 个 token。求后一种方式相对于完整处理输入，节省了多大比例的矩阵运算量。

## 2.2 自回归生成的资源消耗计算

要计算生成完整回答所需的资源，除一次前向包含的运算外，还需要知道模型调用多少次，以及状态保存多久。同一份权重在一条请求中可能使用数百次，KV 则一边读取一边增长。最终保存的字节数与累计访问的字节数因此可以相差几个数量级。

### 2.2.1 Prefill、Decode 与状态复用

在 prefill 阶段，模型收到一段已知输入，可以同时计算多个 token 的表示。由最后一个输入 token 的 logits 选择首个输出 token 后，进入 decode 阶段：下一次调用把该 token 作为新输入，得到再下一个输出，如此逐步扩展。两者使用同一个模型，但前者有多个已知输入 token，后者在普通串行生成中每条请求只增加一个 token。

如果每生成一个 token 都把整个前缀重新送入模型，旧 token 的投影与 FFN 会重复执行。因果模型中，旧 token 不能看到后来加入的 token；在权重、token 位置索引与相关计算条件不变时，旧 token 产生的 $K$、$V$ 可以保留。后续查询读取这些上下文 $K$、$V$，并补入当前 token 位置的新 $K$、$V$，就避免了旧 token 的大量重算。

缓存把“重新计算旧 token 的表示”变成“保存并访问旧 token 的状态”。未来 token 位置会产生自己的 $Q$，用它对旧 token 的 $K$ 评分，再根据评分得到的权重汇总 $V$，所以长期保存的是 $K$ 和 $V$。旧查询已经完成了自己的任务，下一步使用新的查询。第 8 章将进一步讨论如何在不同请求之间共享这些状态。

Qwen3-8B 的 BF16 KV 每个上下文 token 占：

[3] 逐层记录。

$$\begin{aligned} c_{\mathrm{KV}} &= 2Ln_{\mathrm{KV}}d_h b_{\mathrm{KV}} \\ &= 2 \times 36 \times 8 \times 128 \times 2 \text{ bytes} \\ &= 144 \text{ KiB}. \end{aligned}$$

第一个 2 分别对应键和值；$L$ 层各保存 $n_{\mathrm{KV}}$ 组宽度为 $d_h$ 的向量，每元素占 $b_{\mathrm{KV}}$ 字节。因此，保存 $H$ 个上下文 token 需 $M_{\mathrm{KV}} = c_{\mathrm{KV}}H$。Qwen3-8B 的 $H = 8192$ 对应 1.125 GiB，再处理一个新输入 token 就追加 144 KiB。层数或 KV 头数减半，增长速度也减半；batch 增加时，每条独立请求各增加一份上下文。[4]

### 2.2.2 内存占用与数据访问量

KV 缓存保存已经算出的结果，省去后续生成中的重复计算。为了分析节省的计算与增加的存储开销，按用途和生命周期把数据分为权重、上下文状态和临时数据三类。

| 数据类别 | 存储与复用 | 本章计算的主要项目 |
|---|---|---|
| 权重 | 通常在多次调用间保持不变 | 常驻字节、被访问的权重集合、批内复用 |
| 上下文状态 | 随请求保留，持续追加或更新 | 实际占用的容量、每步读写量及更新所需的计算 |
| 临时数据 | 在相应算子或阶段中产生，使用后释放 | 张量尺寸、经过相应存储接口的数据量、需要同时保存的部分 |

首先是权重。Qwen3-8B 全部 BF16 权重约占 16.38 GB，即 15.26 GiB。模型加载后，这份数据供多个请求反复使用。一次前向中，嵌入按 token ID 查行，层内投影与 FFN 使用对应矩阵，末端词表头计算输出分数。因此，访问哪些权重由计算图中的具体操作决定。

对 $[m, d] \times [d, f]$ 的矩阵乘，输入的每一行都使用同一个权重矩阵。将权重块读入片上存储后，可以依次用于多行输入，让一次读取的数据参与更多乘加运算。处理行数 $m$ 增加，计算量随之增加，整份矩阵的容量保持不变。第 5 章将进一步解释片上容量与分块如何决定一份权重能够复用多少次。

上下文状态已在第 2.2.1 节计算；临时数据占用多少内存，则取决于哪些张量同时存在。前一层释放缓冲后，后一层可以复用同一块空间；算子融合（把相邻的几个运算合成一个程序执行）还可以让部分中间值只保存在寄存器或片上存储中。因此，计算读写量时要数清每次访问，计算内存峰值时则要找出同一时刻仍需保存的张量。第 5 章将结合具体程序分析。

存储量取决于某一时刻需要保留哪些数据，访问量则取决于执行过程中读写了多少次数据。例如，一步 decode 若将整段旧上下文读一遍，逻辑读取量为 $c_{\mathrm{KV}}H$；连续执行多步，就要逐步累计。确定这些访问实际经过哪一级存储后，才能将相应的 $R$ 除以该接口的带宽。

**例题 2-2：增加 batch 后，上下文读取量何时超过权重读取量？** Qwen3-8B 每批将所需共享权重读取一遍，数据量约为 15.14 GB，嵌入按请求查行另计。每请求的 8K BF16 上下文为 1.125 GiB，假定各请求独立、旧上下文各读一次。

解：每批共享权重读取量记为 $R_W$，各请求的上下文独立，则上下文读取量为 $Bc_{\mathrm{KV}}H$。两者相等时

[4] 缓存累计计算。

$$B_* = \frac{R_W}{c_{\mathrm{KV}} H}.$$

以 $R_W = 15{,}136{,}811{,}008$ 字节、$c_{\mathrm{KV}} = 147456$ 字节代入，$H = 8192$ 时最小整数 batch 为 13；$H = 2048$ 时为 51。上下文长度缩短到原来的四分之一，上下文读取量超过权重读取量所需的请求数就约增至四倍。上下文越长，批内权重复用的收益就越早受到限制，因为每个新增请求都会增加一份独立上下文的读取。[5]

沿第 1 章的数据路径看，加载把权重从存储送到 GPU 显存，执行再把所需权重送到计算单元。KV 和层间激活也可在加速器内部保存与使用。相同的逻辑访问因此可以留在片上、经过 HBM，或跨加速器传输；第 4 章将分别分析这些路径的存储容量与传输带宽。

### 2.2.3 生成完整回答的计算量与状态读写量

完整回答会重复执行许多步，同一份上下文也随之读取许多次。下面固定一条普通串行生成请求：$B = 1$，已恢复前缀为 $S$，本次输入 $P \geq 1$ 个 token，最终返回 $G \geq 1$ 个 token。令 $H = S + P$。prefill 处理新输入并产生首个输出，随后执行 $n_d = G - 1$ 次 decode。最后返回的 token 尚未重新送入模型，所以请求结束时并不自动为它保存 KV。

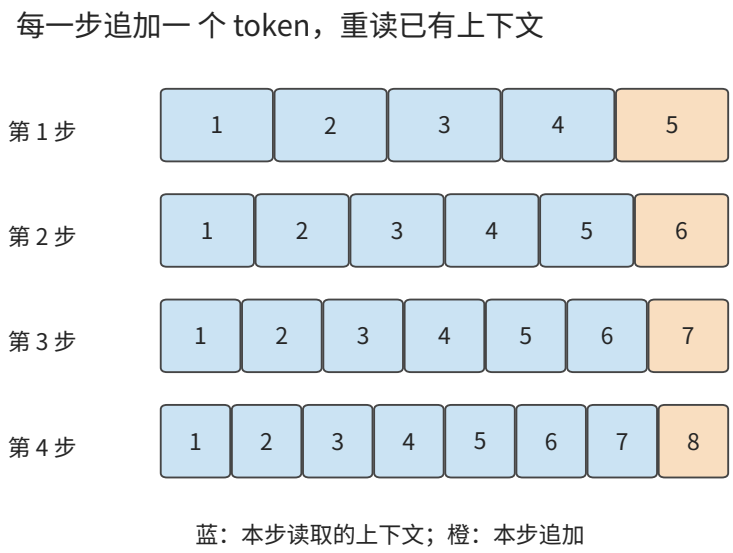



图 2-7　四步生成的上下文读写。每一步读取蓝色的已有位置，再追加一个橙色新 token；本图初始上下文为 4 个 token，四步后共保留 8 个。

图 2-7 中的每一行都比上一行多一个已填色方块：先前追加的位置现在也成为上下文。第 $j$ 次 decode（$j$ 从 0 开始）执行前，上下文长度为 $H + j$。因此，若每次 decode 将此前的上下文各读取一遍，累计读取量为：

Decoder number

$$R_{\mathrm{old}} = c_{\mathrm{KV}} \sum_{j=0}^{n_d-1} (H + j) = c_{\mathrm{KV}} \left[ n_d H + \frac{n_d(n_d - 1)}{2} \right]$$

decode 新增的状态大小为 $c_{\mathrm{KV}} n_d$；若保留整个上下文，最终有效 KV 为 $c_{\mathrm{KV}}(H + n_d)$。本次请求的全部新增 KV 还应包含 prefill 写入的 $c_{\mathrm{KV}} P$。已有前缀的恢复与传输单独核算。求和中的每一项都对应一次 decode 开始时已有的上下文：早出现的位置参与更多后续查询，晚出现的位置参与较少查询。

[5] 跨章核对。

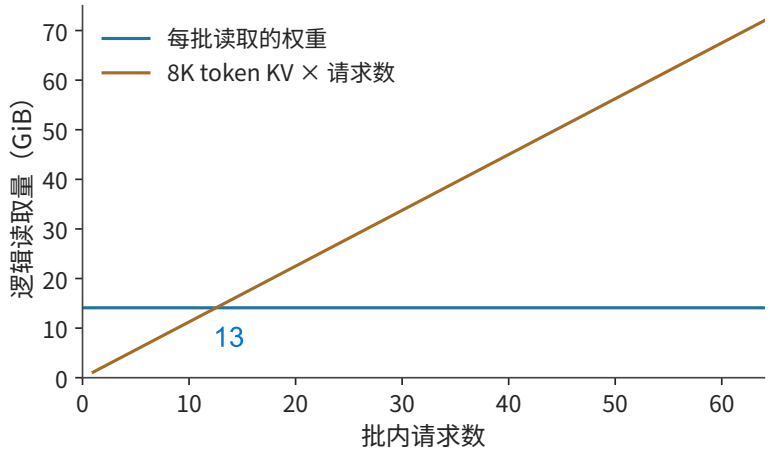



图 2-8　同一 batch 的主要投影权重可共享读取，各请求则各有一份上下文。图中固定每请求 8K 上下文，分别累计共享权重与独立 KV 的逻辑读取量。权重项按整个 decode batch 读取一次计量；KV 项累加各独立请求在本步读取的上下文状态。

**例题 2-3：逐步生成为何使 KV 累计读取量远超存储量？**求请求结束时保存的 KV 大小，以及生成期间读取已有 KV 的总量。

解：取 $S = 0$、$P = 8192$、$G = 1025$，则 $n_d = 1024$，全过程累计读取上下文 token 的 KV 达 8,912,384 次。每个 token 的 KV 占 144 KiB，因此累计读取约 1224 GiB。生成期间只新增 144 MiB KV，结束时共保存约 1.27 GiB。累计读取量是最终存储量的近千倍，因为已经进入上下文的 token，后面的每次 decode 都要再读一遍；本例中绝大部分上下文是 prefill 写入的 8192 个输入 token。

这组调用也决定了总运算量。线性投影和 FFN 在每次 decode 中处理一行，上下文交互则随 $H + j$ 增长。设每步固定的矩阵运算量为 $F_0$、与每个上下文 token 交互所需的运算量为 $a$，便有

Fdecode,once=Fmatrix=Fdecode,total/Nd

$$F_{\text{decode,total}} = n_d F_0 + a\left[n_d H + \frac{n_d(n_d-1)}{2}\right].$$

输出变长时，各步固定部分的累计工作量随 $n_d$ 线性增长，生成过程中新增的上下文还会使一部分累计运算量按生成步数的平方增长。返回一个 token 时 $G = 1$、$n_d = 0$，只有 prefill；这一特殊情形也说明为什么完整回答包含 $G - 1$ 次后续 decode。

图 2-7 沿生成步数累加读取，图 2-8 沿请求数累加读取。多条独立请求的状态容量与上下文访问分别相加，批内权重则共同使用。长度不同时，逐请求代入各自的 $H$ 与 $n_d$：输出越长，已有上下文被读取的次数就越多；输入越长，第一次 decode 要访问的上下文就越多。对实际对话，可先根据每轮输入长度、缓存命中情况和返回的 token 数，还原这条调用链；已保存的 Chat 长度核算给出了具体代入过程。

> **练习 2-3〔延伸〕：输出变长后，KV 容量与累计读取量如何增长**
> 取 Qwen3-8B、$B = 1, S = 0, P = 8192$，分别返回 $G = 513$ 和 $G = 1025$ 个 token。求后续 decode 次数、最终 KV 大小和累计旧上下文读取量。解释输出数量接近翻倍时，生成阶段新增的 KV 容量、最终 KV 总容量与累计读取量为何按不同的比例增长。再将 KV 位宽减半，指出这三项中哪些随之减半。

## 2.3 上下文状态表示与访问机制

模型继续生成时，需要利用前面已经读过的内容。先比较两种记录办法：一种逐页留下记录，要查哪一页就取哪一页；另一种只保留一张定长摘要，每读到新内容，就改写这张摘要。前一种记录随历史增长，后一种大小

固定，但不能保证找回每个 token 的全部细节。

对应到模型，一类机制为每个 token 位置保存键和值，有限状态递推则把历史贡献汇入固定大小的状态。逐 token 记录与递推摘要都以数值向量或矩阵保存。模型处理上下文后保存下来、供后续计算复用的这些表示，称为上下文状态。上下文长度按 token 数计量，状态容量按实际保存的字节数计量。

比较这些机制时，先看历史留下了什么，再看新一步如何取得所需信息：可以用更少的维度表示每个 token，也可以只直接访问近期位置、将多个位置汇总为较少条目，或先筛选再读取。另一类机制持续更新固定大小的状态，查询从中取得结果。这些安排分别改变表示宽度、条目数量和访问方式；节省的存储与读取，要同新增的选择、压缩和更新工作一起计算。

### 2.3.1 多头注意力与 KV 共享

逐 token 记录最直接的节省办法，是减少每个 token 保存的 $K$、$V$ 份数。多头注意力让不同的查询头分别学习上下文中的关系。在多头注意力（Multi-Head Attention，MHA）中，每个 $Q$ 头对应自己的 $K$、$V$ 头；多查询注意力（Multi-Query Attention，MQA）让所有 $Q$ 头共用一组 $K$、$V$；分组查询注意力（Grouped-Query Attention，GQA）则把 $Q$ 头分组，每组共享一组 $K$、$V$。共享后，每个上下文 token 需要保存的 $K$、$V$ 组数减少，上下文 token 的数量保持不变。

以只有四个查询头的单层为例。MHA 为四个查询头各存一份 $K$、$V$；GQA 可以让前两个查询头共享第一份 $K$、$V$，后两个共享第二份；MQA 则让四个查询头共用一份。若上下文长度和头宽相同，需要保存的 KV 份数依次为四、二、一。因此，GQA 在此例中把上下文存储量减半。四个查询头仍可给同一段上下文打出四组不同的分数，这正是“共享上下文表示”与“合并查询”之间的区别。

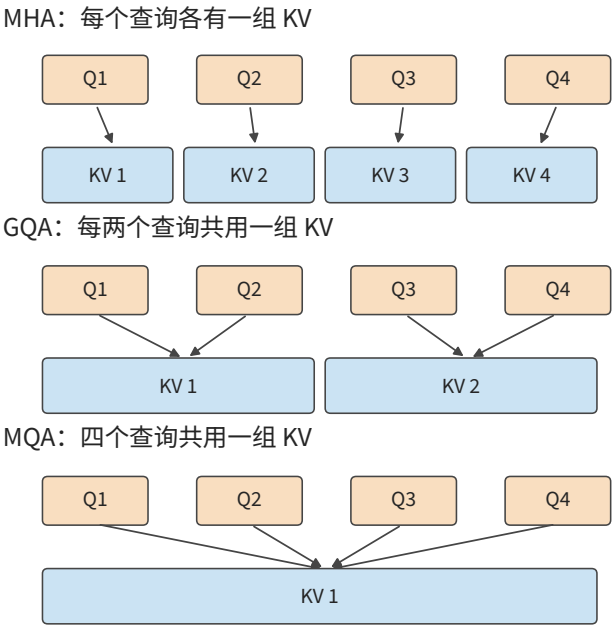



图 2-9　固定四个查询头，分别为每头、每组和全部查询保存 KV。连线表示使用关系；共享上下文后，各查询仍分别产生自己的评分与输出。

Qwen3-8B 采用这种共享方式，让每四个查询头共用一组 KV，总计 32 个查询头、8 个 KV 头。如果给每个查询头单独保存 KV，容量会是当前配置的四倍；如果全部查询共享一组，则会是当前配置的八分之一。实际 GQA 配置保存 8192 个 token 需要 1.125 GiB。本节末的容量图固定上下文长度与头宽，只改变 KV 组数，因此柱高之比直接呈现共享比例。

共享 $K$、$V$ 改变了被查询的数据，但 32 个 $Q$ 头仍分别计算分数和加权输出。因此，KV 投影和存储量按组数减少，$QK^{\mathsf{T}}$ 与 $AV$ 交互仍按查询头数累计。一次读入的 $K$、$V$ 还可以供同组的多个查询头使用，实际计算中就复用了同一份数据。

可以直接由组数预测资源变化。将 $n_{\mathrm{KV}}$ 减半，键值投影参数和上下文状态容量都减半；查询头数 $n_Q$ 不变，查询与上下文交互的运算量仍按原来的头数累计。

共享 K、V 减少了各查询头独立编码上下文的自由度，可能损失细粒度的信息区分与提取能力。损失大小取决于共享程度和训练适应情况；GQA 保留多组上下文表示，通常能在 KV 存储量与模型质量之间取得较好的折中。

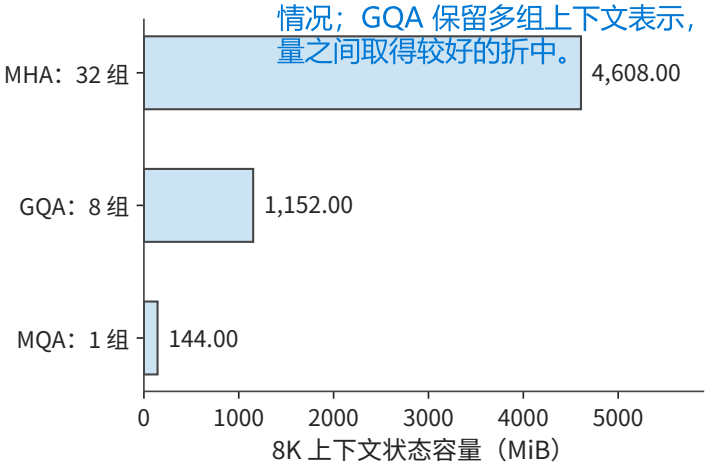



图 2-10　固定 Qwen 层数、头宽、8192 个上下文 token 与 BF16，只改变 KV 组数。实际模型采用 GQA；其余两柱是用于分析共享比例的结构变体。

KV 共享也说明，基础设施的限制会影响模型设计。MQA 的原论文直接把增量推理中的内存带宽开销作为动机；GQA 则在 MHA 的质量与 MQA 的速度之间寻找折中。[6] 减少 KV 头数既改变模型可表达的注意力关系，也减少服务端必须保存和读取的状态。

### 2.3.2 潜变量压缩与 MLA 执行路径

GQA、MQA 靠减少 KV 组数节省空间，多头潜变量注意力（Multi-head Latent Attention，MLA）则靠降低上下文表示的维度节省空间。可以先把输入投影为低维潜变量 $c$，再通过上投影得到用于注意力的 $K$、$V$。模型在训练中学习这些低维表示与上投影，生成时将上下文以 $c$ 的形式保存。

c=X*Wd

Wk原=Wd*Uk

只保存潜变量的依据如下。设键的上投影为 $U_k$，上下文潜变量按行组成矩阵 $c$，展开键为 $K = cU_k$。矩阵乘法的结合律给出

用一份更短的表示c替代整套 K、V 缓存，原来不同token需要存不同的计算出的K值（K=xk），这只需存贮一个共同的Uk以及各自Token的Ck

$$qK^{\mathsf{T}} = q(cU_k)^{\mathsf{T}} = (qU_k^{\mathsf{T}})c^{\mathsf{T}}.$$

左边先为每个上下文 token 恢复键，右边先变换当前查询。对单个新查询，右边只需做一次查询变换，随后直接与潜变量点积。值向量也可在加权汇总后再上投影：先汇总低维向量，再恢复输出维度。这样，原来针对每个上下文 token 执行的升维操作，改为针对当前查询和汇总结果执行。

本章采用 Kimi K3 的 **Gated MLA（门控多头潜变量注意力）**作实例；门控用学习得到的系数调节注意力输出。紧凑路径保存 512 维潜变量和参与 $QK^{\mathsf{T}}$ 点积的 64 维额外分支。Kimi K3 的全部 MLA 层采用 NoPE（No Position Encoding），查询和键都不施加 RoPE，位置信息由 KDA 层（第 2.3.4 节）的递推门控与衰减提供；参考实现仍投影并缓存这 64 维分支，只是不对它做旋转，所以每个 token 保存 $512 + 64$ 维。24 个 MLA 层、8192 个 token 的上下文、BF16 下，存储量为 $24 \times 8192 \times (512 + 64) \times 2$，即 216 MiB。

[6] MQA 原文摘要；GQA 原文摘要；Llama 3 报告第 6.1 节。容量示例按题设的服务状态推导。

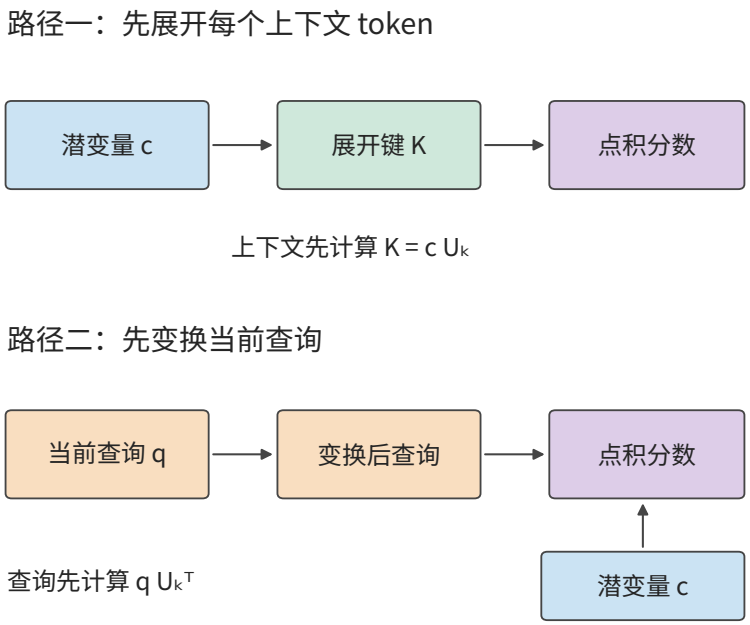



图 2-11　两种计算路径改变上投影的位置。展开路径先恢复上下文键，紧凑路径先变换当前查询，再直接读取潜变量；结合律保证对应点积可以按这两种次序计算。

展开缓存将潜变量先恢复为各头的 $K$、$V$，再保存这些向量；同样的 8192 个 token 需要约 11.25 GiB。紧凑缓存则保存上投影之前的潜变量。两者的容量差异因此来自缓存位于线性变换的哪一侧：紧凑缓存每步变换当前查询，展开缓存直接读取各头的上下文向量。[7]

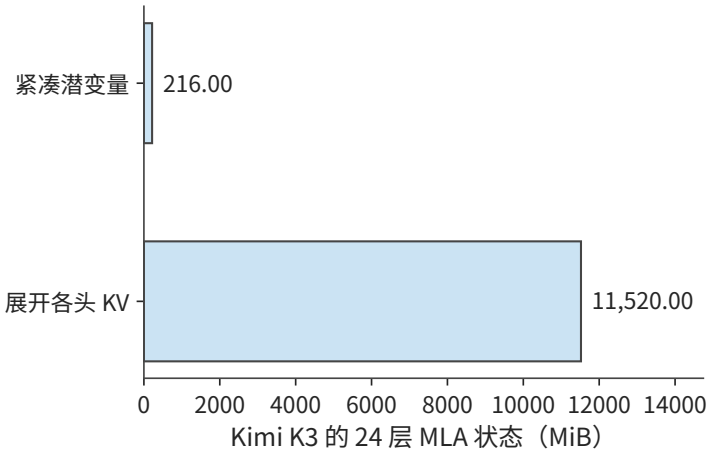



图 2-12　Kimi K3 的 24 层 MLA 在相同 8K 上下文、BF16 下的两种存储量。紧凑路径保存上投影前的表示，展开路径保存各头的键和值。

这一变换减小了每个上下文 token 的表示宽度，但仍需为每个 token 保存一个潜变量向量。因此，存储量和点积运算量仍随上下文长度增长。上下文越长，紧凑表示节省的读取量越可观；上下文越短，查询变换本身的开销占比越高。

[7]Kimi K3 MLA 展开计算。

因此平均每增加一个 token：

$$
g_{\mathrm{V4}}=21\times\frac{584+68}{4}+20\times\frac{584}{128}=\boxed{3514.25\ \mathrm{B/token}}
$$

8K 时：

$$
3514.25\times 8192=\boxed{27.457\ \mathrm{MiB}}
$$



### 2.3.3 局部窗口、上下文压缩与稀疏索引

假设模型已经读过一段长文本，现在要处理一个新 token。模型可以直接读取最近一段位置，也可以读取由多个历史 token 汇成的条目，还可以先找出与当前查询相关的条目，再读取这些条目的主注意力状态。这三种动作分别引出局部窗口、上下文压缩与稀疏选择。三者可以组合使用，但要分别计量：窗口限制近期范围，压缩减少长期条目，索引选择当前要读哪些条目。筛选本身也要读取索引表示并计算分数，不能只统计最后选中的主注意力状态。

先区分三个对象。**局部滑动窗口注意力（Sliding-Window Attention，SWA）**直接读取最近的一段位置；窗口长 128 时，较早位置退出这一局部窗口。**全局主 KV** 是供主注意力读取的长期表示，可能已将多个位置合并；“全局”说明它覆盖较长历史，不表示每次都读取全部条目。**索引 K** 是用于检索的另一份较小表示：索引器先用它给条目打分，再让主注意力读取选中的主 KV。索引分数用于选择缓存条目，主注意力另用自己的查询计算最终汇总权重。

模型通过训练学会把一段输入汇总为共享表示。一条 512 维主 KV 记录兼作注意力中的键和值，供多个查询头共享，容量按一份记录计算。SWA 保存近期细节，全局分支提供较早信息，两者共同供当前查询使用。

DeepSeek V4-Flash 把这套机制组合在同一个模型中，有 43 个主干层、4096 隐藏维度、64 个注意力头，每头维度 512。$Q$ 先从 4096 维投影到 1024 维，再得到 $64\times512$ 维查询；共享 KV 从 4096 维投影到 512 维；输出采用分 8 组的低秩投影，即先投影到较窄的中间维度，再投影到目标维度。各投影的输入、输出维度分别确定矩阵形状。

DeepSeek V4-Flash 使用两种压缩注意力：**压缩稀疏注意力（Compressed Sparse Attention，CSA）**先压缩上下文，再通过索引选取相关条目；**高压缩注意力（Heavily Compressed Attention，HCA）**把更多位置汇成较少的粗粒度条目，供查询访问。两者都用于减少长上下文的存储与访问量，区别在于压缩程度和是否进一步筛选条目。其主干包含 2 个纯窗口层、21 个 CSA 层和 20 个 HCA 层，窗口长度为 128。CSA 按压缩比 4 保存上下文，并维护索引；HCA 按压缩比 128 保存更粗的上下文。采用 BF16 作为参考存储格式，CSA 每个完成的压缩条目含 512 维主注意力状态，另有 128 维索引条目。已有 $S$ 个 token 时，已生成的条目数按 $\lfloor S/4\rfloor$ 计算；尚未完成的压缩块还需独立缓冲。

64（条目）*128（维度）

512*2048

128*2048

压缩比描述输出条目数，不一定等于一个条目的全部输入范围。V4 的 CSA 每四个新 token 生成一条记录，但采用相邻块重叠：除首块的边界处理外，一个条目用两组不同的投影汇总相邻两块、共八个 token 的信息；每个 token 还使用学习得到的逐通道权重。HCA 每 128 个 token 生成一条记录，不采用这种相邻块重叠，也不使用 top-k 索引（只取索引分数最高的 k 个条目），而是读取所有已完成的粗粒度条目。V4.1 的 CSA2（第二版 CSA）才取消了 V4 CSA 的重叠压缩。

一次 CSA 查询的主注意力最多读取窗口加 512 个压缩条目，但索引打分仍扫描已保存的压缩索引条目。在 $S=8192$ 时，一个 CSA 层保留 2048 个压缩条目：主注意力状态约 2 MiB、索引约 0.5 MiB，窗口约 0.125 MiB；主注意力选中的压缩条目合计最多约 0.5 MiB，索引扫描却仍需处理那 2048 个索引条目。HCA 则有 64 个已生成的条目，主注意力状态约 0.0625 MiB，并访问窗口状态与这组粗粒度压缩条目。

将 43 层相加，按上述 BF16 参考格式，在 8192 个 token 的上下文下，窗口、压缩上下文和索引共 59.125 MiB；另有约 11.641 MiB 的 FP32 压缩器缓冲，合计约 70.8 MiB。最后一次查询的主注意力（含窗口）与索引读取量合计 27.625 MiB。状态分解逐项列出了这些数据，因此能看出选中的条目数、保存的状态大小和每步读取量分别代表什么。

压缩还引入一种与查询不同的更新节奏。新 token 到达时，先更新当前压缩块；累计处理的 token 数达到压缩比要求后，再生成一个长期条目。CSA 每四个 token 完成一块，HCA 每 128 个 token 完成一块。因此，在处理同一个输入 token 时，主注意力访问多少条目、索引扫描多少条目、压缩器是否完成一块，需要分别计算。

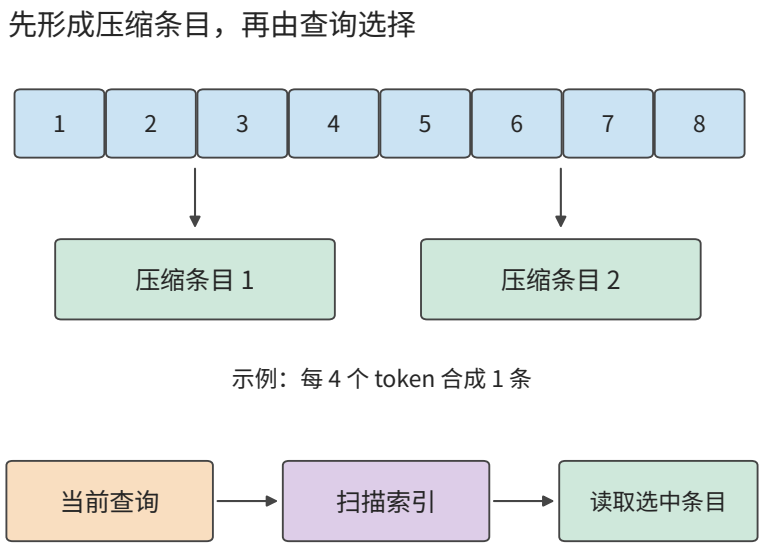



图 2-13　上下文压缩与索引选择的先后关系。上方只示意八个 token 产生两条记录的数量关系，省略 V4 CSA 的重叠投影与加权汇总，下方展示一次查询先扫描索引再读取主注意力状态的过程。

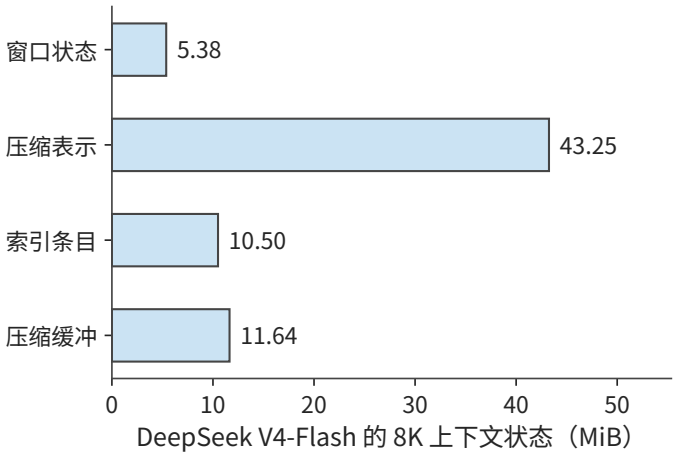



图 2-14　DeepSeek V4-Flash 在 8K 上下文下的四项状态。窗口、压缩上下文和索引保存已处理的上下文信息；压缩缓冲保存尚在汇总或更新中的数据，单独计入容量。

处理到第 128 个 token 时，两种压缩块可能同时完成。[8]

> **练习 2-4〔延伸〕：KV 头共享与上下文压缩如何改变存储和访问量**　4*128*4096*2
>
> 先把 Qwen3-8B 的 KV 头数从 8 改为 4，保持 32 个查询头不变，分别计算 KV 容量、KV 投影参数量，以及查询与上下文交互所需运算量的变化。再对 DeepSeek V4-Flash 的 CSA 层，把上下文从 8192 增至 16384，按压缩比 4、每次最多选择 512 个压缩条目，计算保存条目数、索引扫描条目数和选中条目上限。解释三者为何不按同一比例增长。

### 2.3.4 线性注意力与有限状态递推

第 2.3.1 至 2.3.3 节的机制仍能区分保存下来的各个上下文条目。有限状态递推采用另一种过程：每来一个新 token，就把它的贡献合入已有状态，当前查询直接使用更新后的状态；下一步继续改写这份状态。这对应第

[8]缓存后缀续算记录。

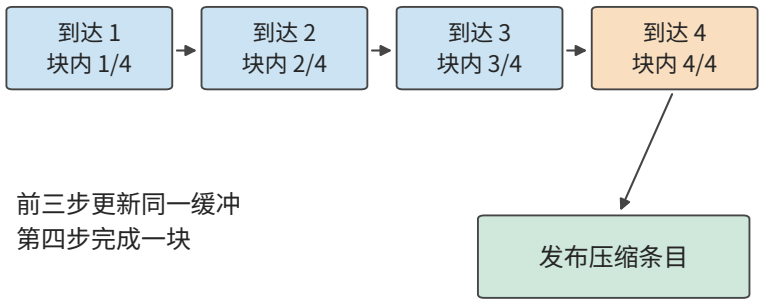



图 2-15　压缩比为四时的一个压缩块的生成过程。前几次输入更新同一块内缓冲，第四个 token 到来后形成可供后续查询使用的压缩条目。

2.3 节开头不断改写定长摘要的图景。这类线性注意力重新组织查询、键和值的计算次序，不再显式算出所有查询—键配对的注意力分数。在特征维度固定时，状态矩阵大小不变，处理整段序列的运算量随序列长度线性增长。

构造状态的基本运算是外积，即把一个键向量与一个值向量的每对元素相乘，组成二维的关联表。考虑由键值外积累加得到的矩阵 $\mathcal{S}_t$。每个新 token 到来时，先读取旧状态，再写入本位置的贡献，当前查询直接从状态矩阵读出结果：

$$\mathcal{S}_t = \mathcal{S}_{t-1} + k_t v_t^{\top}, \qquad o_t = q_t^{\top} \mathcal{S}_t.$$

若键宽为 $d_k$、值宽为 $d_v$，状态始终有 $d_k d_v$ 个元素。新增 token 改变的是矩阵内容，而不是矩阵大小。逐 token 的上下文列表因此变成了一组累积的键值关联；查询从这组关联中取得输出。

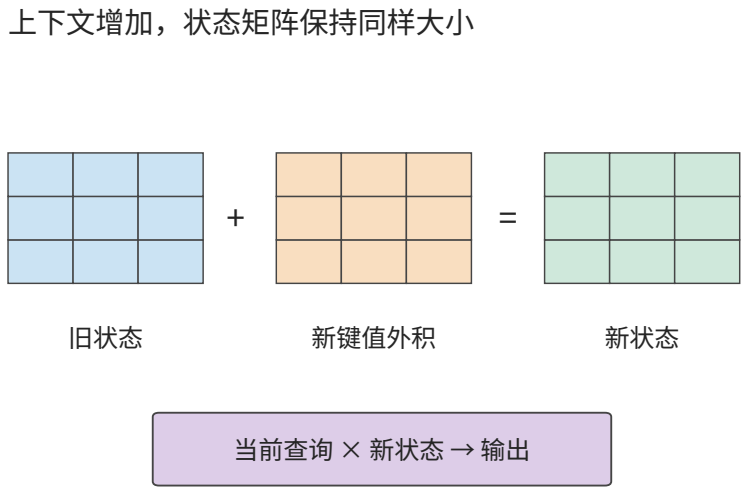



图 2-16　固定矩阵的递推更新。新键和值的外积加入旧状态，矩阵形状保持不变；查询使用更新后的状态计算输出。本图采用简单累加递推。

实际的递推注意力在简单累加之外，还加入门控与修正。**Kimi Delta Attention（KDA）**是 Kimi 使用的一

种门控递推注意力，用固定大小的关联矩阵保存上下文信息。KDA 使用门控与 delta 更新；delta 在这里指依据当前预测误差写入修正量。每次更新不只写入新信息，还用当前键检验既有状态的预测，据此写入修正；旧信息以汇总后的关联形式保存，当前键决定修正落在哪里，衰减门决定保留多少旧信息。KDA 更新的是一张固定大小的关联矩阵，而非向逐 token 保存的列表追加条目。

Kimi K3 每个 KDA 层有 96 个头，每头递推矩阵为 $128 \times 128$。按 FP32 保存，单层为 6 MiB，69 层为 414 MiB。一次 decode 读取旧矩阵、融合新信息并写回，理想情况下，各读写一次的总量就是 828 MiB。这一读写量不随上下文长度增长：短上下文时是一笔显著的固定开销，长上下文时则避免了扫描量随上下文 token 数持续增加。

prefill 与 decode 的实现方式也不同。decode 使用单步递推；一段已知输入可以用块式算法并行计算，再在块之间传递状态。块内中间量和块间状态会占用临时空间，因此仅凭最终递推状态的大小，还无法预测 prefill 期间的内存占用峰值。第 5 章根据缓冲区从分配到释放的过程分析这一问题。[9]

### 2.3.5 混合注意力的状态构成

实际模型常把有限状态层与全局注意力层混合使用。这样既利用递推结构的上下文压缩，也保留部分直接访问全局上下文的能力。整个模型的状态由多个部分组成：有的大小固定，有的随上下文长度增长。

Kimi K3 共 93 层，其中 69 个 KDA 层、24 个 MLA 层。8192 个 token 的上下文、紧凑 BF16 MLA、FP32 递推状态及 BF16 短卷积槽位下（短卷积沿序列方向混合相邻几个位置的输入，槽位保存下一步更新要用的近期输入），三项分别为 216 MiB、414 MiB 与 19.4 MiB，合计约 649.4 MiB。若使用参考展开缓存，MLA 一项就变为 11.25 GiB，整体随之改变。[10]

Qwen3.6-35B-A3B 同样需要分别计算线性注意力层和完整注意力层的状态占用。其 30 个线性注意力层加 10 个完整注意力层，每请求每个上下文 token 的全局 KV 为 20 KiB，另有 60 MiB 的固定 FP32 递推状态，以及 1.875 MiB 的 BF16 短卷积槽位。上下文为 8192 个 token 时，三项合计 221.9 MiB。这些状态来自注意力分支。随后的混合专家（Mixture of Experts，MoE）分支提供多个前馈子网络，由路由器为每个 token 选择其中一部分来执行，决定同一主干中实际使用哪些权重。这些供选择的子网络称为路由专家，所有 token 都执行的一支称为共享专家。注意力与 MoE 两类机制共同组成完整模型。

这五条状态曲线的形状来自不同表示。Qwen3-8B 每增加一个 token，所有 GQA 层都追加 $K$、$V$；Kimi K3 只有 MLA 部分追加上下文，KDA 部分保持固定矩阵；DeepSeek V4-Flash 则在压缩块完成时增加条目。V4.1 Flash 进一步让多层读取同一份全局缓存，Qwen3.6 则将递推层与全注意力层组合。对长上下文，增长速度决定新增容量；对短上下文，递推矩阵等固定部分决定起始占用。

将固定递推状态与逐 token 上下文相加，就得到混合结构的容量模型：

$$M_{\text{state}}(H) = M_{\text{fixed}} + c_{\text{state}} H, \qquad H_* = \frac{M_{\text{fixed}}}{c_{\text{state}}}.$$

$c_{\text{state}}$ 为每个上下文 token 增加的状态字节数，$H_*$ 表示两部分占用相等的上下文长度。Qwen3.6 的固定部分为 61.875 MiB，每 token 增加 20 KiB，因此 $H_* = 3168$；Kimi K3 的紧凑表示每 token 增加 27 KiB，固定部分约为 433.4 MiB，交点约在 16.4K 个上下文 token。低于交点时，缩短上下文只能节省较小一部分空间；远高于交点时，减小每个 token 的表示宽度能更有效地节省容量。DeepSeek V4-Flash 则按完成的压缩块增加条目，曲线呈阶梯增长。

[9] 块式状态与缓冲记录。

[10] 混合状态计算。

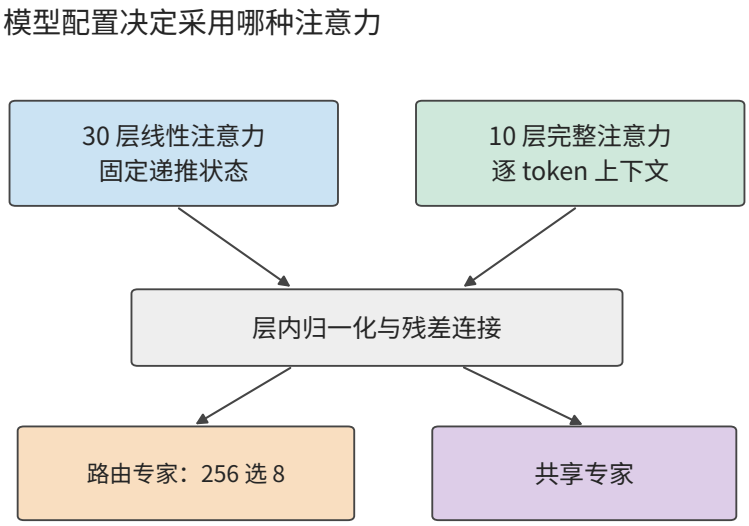



图 2-17　Qwen3.6 的两类层。配置决定每层采用线性或完整注意力，随后都执行路由专家和共享专家；上方两个分支分别表示线性注意力和完整注意力，模型分别有 30 层与 10 层。

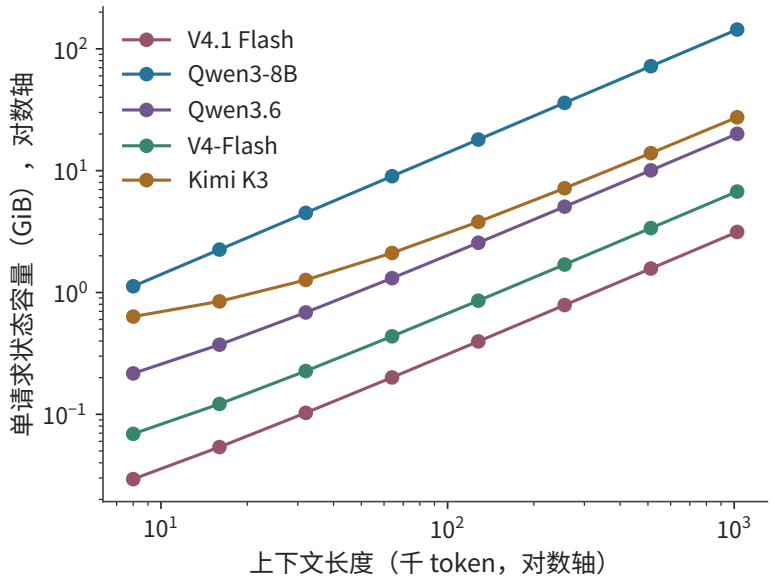



图 2-18　上下文增长时的状态容量。横纵轴均为对数刻度；五个模型分别通过逐 token 追加、递推、压缩和跨层共享组织状态。纵轴为单请求的状态容量，横轴按上下文 token 数计量。

这些状态支持不同的信息访问方式：全局 KV 允许查询直接选择旧 token，递推矩阵靠逐步更新保留上下文关系，压缩索引则在较少条目中寻找相关内容。第 2.6.3 节将结合具体检索任务比较资源需求。[11]

**练习 2-5〔核心〕：上下文多长时，KV 容量会超过固定状态？**

对 Qwen3.6，取固定状态 61.875 MiB、每个上下文 token 20 KiB；对 Kimi K3 的紧凑表示，递推状态占 414 MiB，卷积状态占 20,348,928 字节，每个上下文 token 另占 27 KiB。分别求随上下文增长的状态与固定状态容量相等时的上下文长度，以及 32K、128K 上下文的状态总容量。若只有 256 MiB 状态预算，两个模型各自最多能保存多少个上下文 token？说明容量预算小于固定状态大小时，为什么缩短上下文仍无法容纳模

[11] 完整模型实验。

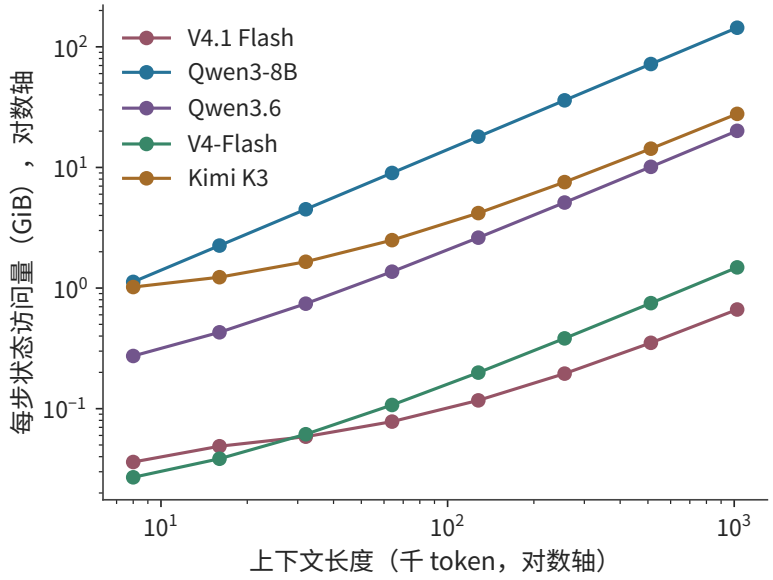



图 2-19　同一组模型每步计入的状态访问。逐 token 上下文计读取，递推矩阵计一读一写；这些是指定路径的逻辑数据量，用于比较容量与访问的不同增长方式。每步指为一个请求生成下一个 token 的一次 decode；纵轴不包含模型权重读取。

型状态。

### 2.3.6 每个 token 存多少，每次 decode 读多少

比较模型的上下文成本，需要同时给出三个量：全局历史每增加一个 token 的平均存储增量、长度为 $N$ 时实际保存的状态，以及一次 decode 查询需要读取的历史。全局 KV 会随上下文增长；滑动窗口只保留最近一段；递推状态的大小固定，却仍要在每一步读写。把三者都除以上下文长度，会掩盖它们不同的增长规律。

下面考虑一条请求，当前查询可访问的上下文共 $N = 8192$ 个 token，包含查询 token 位置自身。表中按各行指定的精度和缓存路径计算全局历史容量，以及一次查询的逻辑读取量：每层选中的 K/V 或共享潜变量读取一次，供各查询头的 QK 评分与 PV 计算（即前文的 $AV$，用注意力权重汇总值）复用，再加上局部 SWA 窗口与索引扫描。递推矩阵、卷积和压缩器的状态更新在表后分别讨论。

| 模型与缓存路径 | 全局历史平均增长（B/token） | 8K 全局历史（MiB） | 每次 decode 注意力 KV + 索引读（MiB） |
|---|---|---|---|
| Qwen3-8B，BF16 GQA | 147,456 | 1,152 | 1,152 |
| Qwen3-32B，BF16 GQA | 262,144 | 2,048 | 2,048 |
| Qwen3-30B-A3B，BF16 GQA | 98,304 | 768 | 768 |
| Qwen3-235B-A22B，BF16 GQA | 192,512 | 1,504 | 1,504 |
| Qwen3.5-397B-A17B，BF16 全注意力部分 | 30,720 | 240 | 240 |
| Qwen3.6，BF16 全注意力部分 | 20,480 | 160 | 160 |
| Kimi K3，BF16 紧凑 MLA 路径 | 27,648 | 216 | 216 |
| DeepSeek V4-Flash，生产混合格式 | 3,514.25 | 27.455 | 12.556 |
| DeepSeek V4.1 Flash，生产混合格式 | 890 | 6.953 | 11.375 |

Kimi K3 的两种 MLA 表示展示了缓存路径对容量的影响：表中的紧凑路径每个 token 增加 27,648 B，本书采用的 Hugging Face 参考实现将 K/V 展开，每个 token 增加 1,474,560 B。跨模型缓存计算列出了两种路径的精确字节数。

两代 Flash 的行采用生产混合格式：V4-Flash 的主记录为 FP8/BF16 混合、每条 584 B，索引为 MXFP4（每 32 个 4 位值共用一个 scale 的格式）；V4.1 Flash 的格式见下文。第 2.3.3 节的 V4-Flash 数字则按 BF16 参考格式计算，同一 8K 上下文下，全局历史为 53.75 MiB（另有 5.375 MiB 窗口），一次查询读取 27.625 MiB。表中 V4-Flash 的 27.455 MiB 是生产格式下的全局历史驻留量，与 BF16 参考格式的读取量 27.625 MiB 只是数值接近。

混合注意力还要加固定状态。在本章 FP32 递推、BF16 短卷积的条件下，Qwen3.5 的递推矩阵与卷积槽合计 184.219 MiB，Qwen3.6 为 61.875 MiB，Kimi K3 为 433.406 MiB。这些固定状态与表中的上下文缓存共同决定当前的内存占用；一次 decode 还要读写递推矩阵并更新卷积槽。

除了逐层各自保存，状态还可以在层之间共享。长期以来，decoder-only 是通用生成式语言模型的主流架构：输入处理与逐 token 生成使用同一套主干层，各层通常根据自己的隐藏状态产生 KV。V4.1 Flash 对这种分工做了重要调整，将上下文表示的构建与后续查询分开，使输入处理只需较少的主干计算。技术报告将这一设计称为 CED，并说明其受到 YoCo 跨层复用 KV 的思路启发。[12]

这里的“编码器负责理解、解码器负责生成”，具体指两部分在上下文表示与生成计算中的不同职责。编码器遵守因果约束，并非传统序列到序列模型中可以双向访问整个输入的编码器；生成新 token 时也仍然需要编码器参与。这种不对称体现在信息在哪里产生、各阶段要执行哪些层，而不是把输入与输出交给两个互不参与对方计算的模型。

理解这套结构时，可以先把“生成全局表示”和“查询全局表示”分开。V4.1 Flash 的 CED 把 40 层文本主干按顺序分成前 20 层因果编码器和后 20 层解码器。两部分属于同一个自回归模型，编码器中的每个 token 只访问当前 token 位置及其之前的信息。

[12] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

编码器先处理输入。解码器需要的全局 KV 由编码器末层表示投影生成，不必让每个输入 token 都先经过全部解码器层。解码器每层的局部 SWA 则仍依赖该层自己的输入表示，因此要把提示末尾最多 128 个 token 的编码器输出送入解码器，构建局部状态，下文称为末尾窗口重放。开始逐 token 生成后，新 token 依次经过编码器和解码器的全部 40 层。第 3 章再计算节省了哪些输入工作，第 8 章区分编码器和解码器的两种重放。

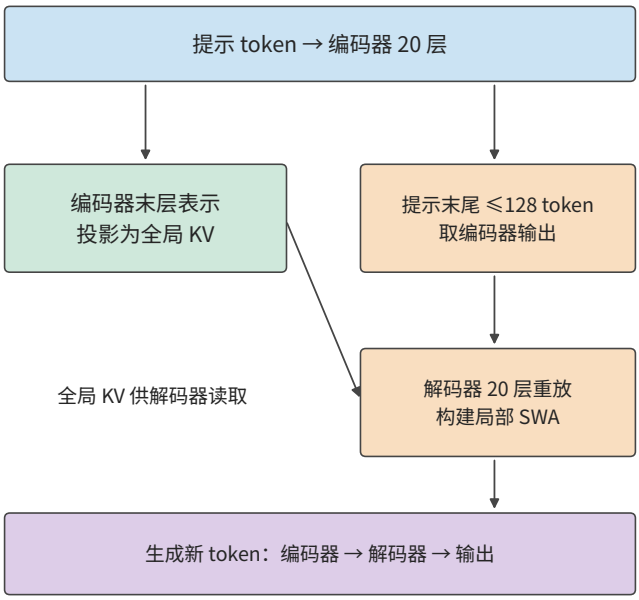



图 2-20　CED 把全局 KV 的生成与解码器局部状态的构建分开。大部分提示 token 只执行编码器主体及全局 KV 投影；末尾窗口的编码器输出还经过解码器，近似构建局部 SWA。生成的新 token 仍通过全部 40 层。

长上下文的成本还取决于同一段信息在多少层中重复保存。V4.1 的技术报告直接把 HBM、SSD 与主机内存容量，以及 KV 迁移带宽列为设计约束。V4.1 没有继续提高 V4 的序列压缩比：V4 使用 4:1 的 CSA 和 128:1 的 HCA，V4.1 取消 HCA，编码器的全局条目按 2:1 压缩，解码器按 1:1 保留。V4.1 缩小缓存主要靠跨层共享与更低精度，同时保留了更细的历史 token 粒度。[13]

共享也改变了各层能够独立选择的信息。复用层不再生成自己的全局 K/V，但每层仍保留独立的查询 Q 和局部 SWA，因而可以对共享条目计算不同的注意力权重，形成新的输出。缓存共享减少了一类逐层独立表示，却没有把所有层的计算变成相同操作。

把上述共享关系代入容量计算。2026 年 9 月发布的 DeepSeek V4.1 Flash 有 40 层文本主干，但只有 4 层拥有独立的全局 KV：按从零开始的编号，第 2、8、14 层每两个 token 生成一条压缩记录，第 20 层每个 token 生成一条记录，其余全局注意力层共享这些缓存。主记录的 512 个值采用 FP4（每个值占 4 位的浮点格式），每 16 个值另有一个字节的 scale，因此每条记录占 $512/2 + 512/16 = 288$ 字节；索引记录占 $128/2 + 128/32 = 68$ 字节。全局 KV 的平均存储增量为

$$\left(\frac{3}{2} + 1\right)(288 + 68) = 890 \text{ B/token}.$$

作为对照，旧 V4-Flash 有 21 层 4:1 压缩和 20 层 128:1 压缩。按官方 FlashMLA（DeepSeek 开源的 MLA 注意力计算库）的格式（主记录 584 B、索引记录 68 B），其全局 KV 的平均存储增量为

[13] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

到 8192 时：已经凑齐 4096 对。
来第 8193 个 token 时：新的一对只有第一个，三个压缩器暂时不新增完整记录。只有第 20 层照常新增一条，共 356 B。
来第 8194 个 token 时：与 8193 凑成一对，三个压缩器各新增一条；加上第 20 层照常新增的一条，共 4 条 = 1424 B。
来第 8195 个 token 时：又开始下一对，等待 8196，因此仍然只新增一条。

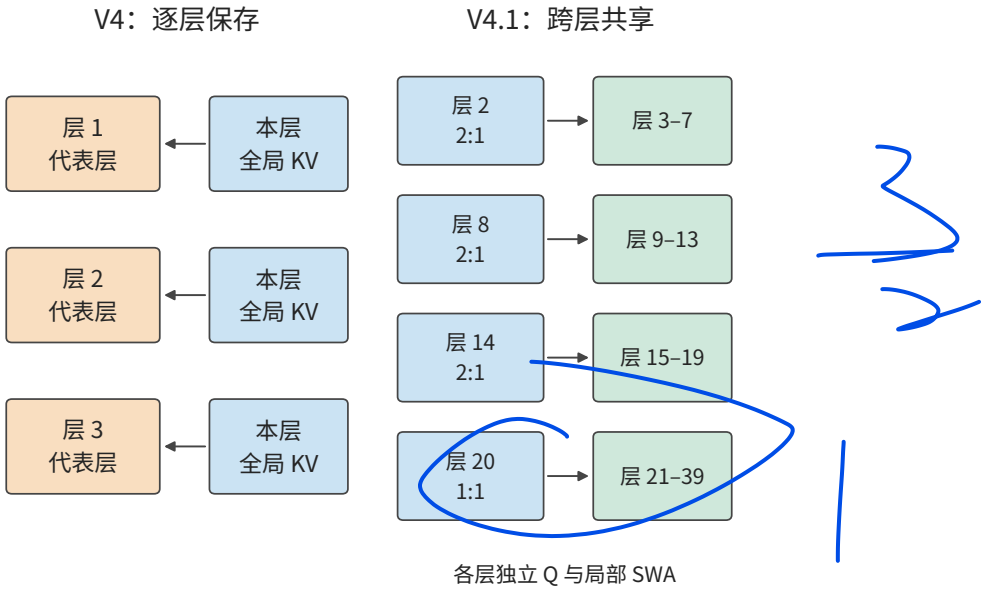



图 2-21　V4 与 V4.1 的全局 KV 保存方式。左侧以三个代表层示意逐层保存；右侧显示 V4.1 四个独立保存全局 KV 的层及共享关系。实线表示数据读取；各使用层仍有自己的 Q 与局部 SWA，图中省略其他计算。

$$21\frac{584+68}{4}+20\frac{584}{128}=3514.25\ \text{B/token}.$$

两者的全局 KV 平均增量相差约 3.95 倍，对应官方图中取整后的 3,514 与 890 B/token。除全局上下文缓存外，V4.1 的局部窗口每层最多保存 128 条记录，每条 FP8（每个值占 1 字节的浮点格式）+scale 记录占 528 B，40 层合计 2.578 MiB。全局历史随上下文增长，局部窗口则在填满后保持固定容量。

沿生成过程逐步观察，还能看出全局缓存的增长节奏。对 V4.1，全局缓存从偶数长度增长到下一个奇数长度只增加 356 B，即第 20 层每 token 一条的记录；到下一个偶数长度时，第 2、8、14 层的三个 2:1 压缩器各再完成一条记录，本步合计增加 $4\times356=1{,}424$ B。窗口填满后，后续写入会覆盖旧槽位，容量不再增长，但仍然产生写入：40 层 SWA 每步合计写入 $40\times528=21{,}120$ B，压缩器更新另计。

共享同一份全局缓存的各层仍需读取数据，部分层还会重新索引；8K 场景下，按表中的生产混合格式，V4-Flash 与 V4.1 Flash 的注意力 KV（含局部 SWA）与索引逻辑读取量分别为 12.556 和 11.375 MiB，跨层共享主要减少了重复保存的空间。

缓存复用与条目选择还可以分别安排。V4.1 的 CSA2 把层分成三种模式。**Full**（完整）层完整执行“生成全局 KV、索引、选取和注意力”的流程。**Reindex**（重索引）层共享 KV，用自己的索引查询重新选取 top-k；**Reuse**（复用）层连同所选缓存条目一起复用。V4.1 的 38 个全局注意力层中，有 4 个 Full、4 个 Reindex 和 30 个 Reuse；另外两个层仅使用 SWA。解码器的首个 Full 层扫描全局，并选出最多 $2048\times8=16{,}384$ 个候选条目；技术报告第 2.3.2 节将这一集合称为**候选池（candidate pool）**，其中的候选是可供注意力检索的缓存条目。后续 Reindex 层只在池内各自选取最多 512 个条目。[14] 后续层的搜索量因此有了上限，但初始全局扫描仍随上下文增长。

例如，某次查询面对 131,072 个全局缓存条目，将它们按每块 8 个分成 16,384 块。首个解码器 Full 层先为

[14] DeepSeek V4.1 技术报告，第 2.3.2 节 Hierarchical Sparse Indexer 使用 candidate pool 和 candidate positions；Figure 5 展示共享候选池与后续 top-k 的区别。

所有缓存条目评分，以块内最高分作为块分数，再选出 2,048 块，形成 16,384 个候选条目。该层仍从全局分数选取 top-512；后续 Reindex 层在候选内用新的索引查询重算分数，各自选出 512 个缓存条目。候选池限定后续层的搜索范围，各层的 top-512 决定本层最终读取哪些条目。

直接用之前计算出的索引进行复用

候选池让后续层的搜索量有了上限，也使首轮筛选影响后续选择：某个缓存条目一旦落在候选池之外，本次查询的后续 Reindex 层便无法再选入该条目。模型训练需要适应这一搜索范围，第 10 章将说明 V4.1 如何把候选限制纳入后训练。

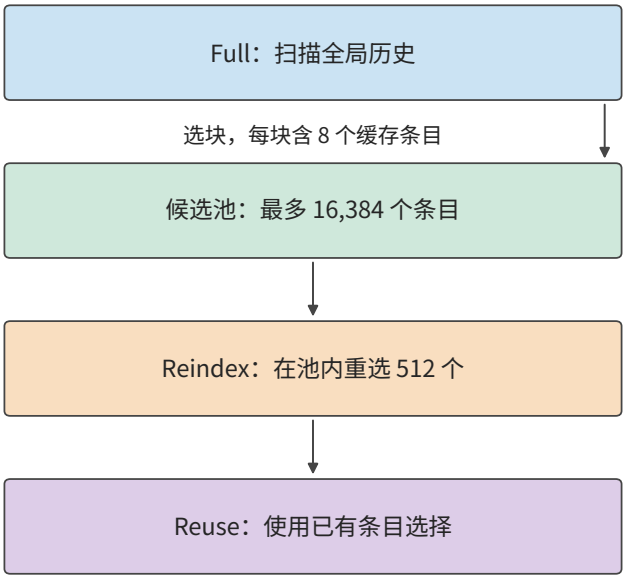



图 2-22　V4.1 解码器的分层缓存条目选择。首个 Full 层先扫描全局，选出最多 16,384 个候选条目；后续 Reindex 在候选内重新选 512 个，Reuse 使用已有选择。示意条目数量不按比例；各层独立 Q 与局部 SWA 未画出。

## 2.4 条件计算与专家权重复用

第 2.3 节通过共享、压缩和递推减少上下文状态。另一个主要数据来源是模型权重：每个 token 是否必须使用模型中的全部参数？本节从 FFN 出发分析这一问题。

稠密 FFN 对每个 token 使用同一组矩阵；MoE 把参数分配给多个专家，由路由器为每个 token 选择少数专家。这样，“模型保存多少参数”与“一个 token 激活多少参数”就可以分别调整，batch 内各 token 的选择也会影响实际访问的权重集合。

### 2.4.1 稠密前馈网络与专家路由

典型的 MoE 分支先从输入计算路由分数，按 top-k 选择路由分数最高的 k 个专家，再把 token 送入选中专家，最后按路由权重合并输出。有共享专家时，还会有一条所有 token 都参与的分支。专家本身通常仍由 gate、up、down 三个矩阵构成，因此可以按第 2.1.4 节的 SwiGLU 计算逐个展开。

以 Qwen3.6-35B-A3B 为例。其文本主干包含 40 层，隐藏维度 2048；每层 256 个路由专家，每 token 选 8 个，专家中间维度为 512，另有共享专家。一个路由专家包含 $3 \times 2048 \times 512 = 3,145,728$ 个参数，以 BF16 保存为 6 MiB。每层全部路由专家占 1.5 GiB，一个 token 选中的八个专家占 48 MiB。

设专家 $e$ 实际收到 $t_e$ 个 token，其 gate/up 是 $[t_e, 2048] \times [2048, 512]$，down 是 $[t_e, 512] \times [512, 2048]$，主

要矩阵运算量为 $6t_e \times 2048 \times 512$。将各专家的运算量相加，就得到全部路由专家的矩阵运算量。路由器先决定各个 $t_e$，选中专家执行三次投影，随后收集并加权合并结果；共享分支对全部输入行执行。

用 $E$ 表示路由专家总数、$k_{\text{top}}$ 表示每 token 选中的专家数，单个专家有 $N_e$ 个参数，则每层保存 $EN_e$ 个路由专家参数，单 token 执行的专家矩阵工作约为 $2k_{\text{top}}N_e$。增大 $E$ 扩充了可选择的参数集合；只有实际选中数或专家尺寸增加，才会直接增加这部分单 token 运算。Qwen3.6 的 A3B 名称体现激活规模，全部文本参数约为 346.6 亿，BF16 权重约为 69.321 GB。

### 2.4.2 实际运算量与批内权重读取量

第 2.4.1 节的 $2k_{\text{top}}N_e$ 只统计单个 token 的专家工作；一批 token 同时路由时，还要看它们落到多少个不同的专家上。

**例题 2-4：相同专家计算量，权重读取为何相差 32 倍？** 对 Qwen3.6 的同一层，比较均匀分派和集中分派。

解：即使激活参数相同，batch 内的权重读取也可能不同。假设同时为 $B = 64$ 条请求各执行一步 decode，每条请求本次输入一个 token，因此 $m = B = 64$。每个 token 选择 8 个专家，总共有 512 次 token 到专家的分派。若均匀分到 256 个专家，每个专家平均接收 2 个 token，当前 batch 会访问全部路由专家；若所有 token 都集中到同一组 8 个专家，则每个专家接收 64 个 token，当前 batch 只访问 8 份专家权重。

设本批实际访问的不同专家数为 $U$。若路由过程中不丢弃任何分派给专家的 token，则有 $\sum_e t_e = Bk_{\text{top}}$，因此

$$F_{\text{expert}} = 2N_e \sum_e t_e = 2N_e B k_{\text{top}}, \qquad R_{\text{expert}} = b_W N_e U.$$

第一式按分派的总行数累计，第二式按实际读入的不同权重累计。例中两种路由方式都产生 512 次 token 到专家的分派；一个 token 发给八个专家，会在八个专家的输入矩阵中各占一行。这 512 行来自 64 个原始 token，矩阵运算量相同；但 $U$ 分别为 256 和 8，每层读取量分别为 1.5 GiB 和 48 MiB，相差 32 倍。40 层所有路由专家合计保存 60 GiB，集中路由每批选中的权重合计只有 1.875 GiB。

| $B = 64$ 的路由情形 | 每层访问的不同专家 | 每专家 token 数 | 每层路由专家理想读取 |
|---|---|---|---|
| 均匀分派 | 256 | 2 | 1.5 GiB |
| 集中分派到同一组 8 个专家 | 8 | 64 | 48 MiB |

读取量的差别来自每份权重供多少输入行使用：均匀分派时每个专家处理两个 token，集中分派时每个专家处理 64 个 token。同一份权重参与的计算量增至原来的 32 倍，理想权重访问量便降至原来的三十二分之一。与此同时，原来要执行许多输入只有两行的矩阵乘法，现在只需执行少数输入为 64 行的矩阵乘法；权重复用和计算效率因此都会受到影响。[15]

把各层累加成整模型再来比较：Qwen3.6 在已有 8192 个 token 上下文、$B = 1$ 的 decode 中约需 7.33 GFLOPs，小于 Qwen3-8B 的约 20.0 GFLOPs。整个模型的计算量差异来自几项结构上的变化：2048 维主干缩小投影，30 个线性层使用递推状态，MoE 每次只执行选中的专家及共享分支。专家数量增加使总参数量增大，而每次调用的工作量取决于实际执行了哪些运算。

稀疏激活让总参数量与每个 token 实际使用的参数量可以分别调整，模型就能用较少的计算访问较大的参数集合。服务时，路由结果决定每份专家权重供多少 token 复用，例题 2-4 展示的正是这两种访问方式的差别。

[15] 均匀路由；集中路由。

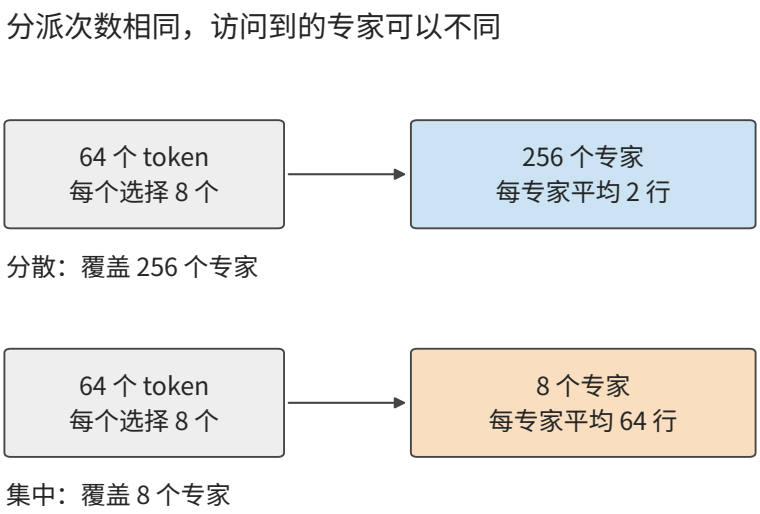



图 2-23　64 个 token 各选八个专家，总计 512 次分派。分散时可覆盖 256 个专家，集中时只访问八个；每专家处理的行数随之改变。

### 2.4.3 共享专家与潜空间计算

上述差距由专家接收的输入行数决定。还要考虑专家本身的尺寸：共享专家处理全部输入，潜空间专家则先缩小输入维度。两者都会改变单个专家需要多少权重、执行多少运算。

DeepSeek V4-Flash 每层有 256 个路由专家，每 token 选 6 个，另有 1 个共享专家；路由专家中间维度为 2048。单专家的 gate/up 为 $[t_e, 4096] \times [4096, 2048]$，down 将宽度从 2048 变回 4096；三个投影合计包含 25,165,824 个参数。前三层采用哈希路由，后续采用评分路由：哈希路由按 token 确定专家，评分路由根据当前表示选择专家。

共享专家处理本次全部 $m$ 个 token 的特征向量，路由专家 $e$ 只处理分派给它的 $t_e$ 个 token。若共享专家有 $N_s$ 个参数，一层的专家矩阵运算量为 $2mN_s + 2\sum_e t_e N_e$。普通单步 decode 中，每条请求输入一个新 token，此时 $m = B$；prefill 则要累计各请求本次处理的全部 token。DeepSeek V4-Flash 每 token 选择六个路由专家并执行一个共享专家，运算量正是这两部分之和；容量则需要保存全部 256 个路由专家和共享分支。

Kimi K3 每个 MoE 层有 896 个路由专家，每个 token 选择其中 16 个。输入先从 7168 维主干空间投影到 3584 维潜空间，再经过中间维度为 3072 的专家变换，合并后投影回主干空间。共享专家直接处理主干表示。降维减少了路由专家的矩阵尺寸，同时增加了进入和离开潜空间的两次投影。[16]

如果专家分布在不同的卡上，就要把输入 token 发送到专家所在的卡，专家算完后再把结果合并。通信量因此取决于专家放在哪里、输入保存在哪里。第 6 章讨论多卡划分，第 9 章介绍 CPU / GPU 放置，以及将注意力（Attention）与 FFN 交给不同资源的 AF 分离。

> **练习 2-6〔延伸〕：专家数量与分派方式如何影响计算和权重读取**
> 对 Qwen3.6 的一层，取 $B = 64$、256 个路由专家、每 token 选 8 个。分别考虑均匀分派和集中分派到同一组 8 个专家的情况，写出各专家接收的 token 数 $t_e$，再计算 FLOPs、被访问的专家数量与 BF16 权重读取量。再将专家总数减半、中间维度加倍、每个 token 选中的专家数减半，判断专家参数量与实际执行的专家矩阵运算量是否变化。最后说明如何在上述计算中加入共享专家的参数量与运算量；对于采用潜空间专家的模型，再说明如何计入进入和离开潜空间的投影。

[16] Kimi K3 专家计算。

## 2.5 网络形状与计算组织

上下文状态与专家选择改变了主要数据集合，网络的深度、宽度和连接方式则进一步决定这些工作使用多大的矩阵、按什么顺序执行。参数量相近的模型也可以有不同的计算图；矩阵 FLOPs 相近的模型，缓冲区的占用时间和串行步骤也可能不同。

### 2.5.1 深度、宽度与专家粒度

先忽略词表和小向量参数。若 FFN 宽度与隐藏维度保持固定比例，稠密主干的参数量大致正比于 $Ld^2$。把隐藏维度减半、层数增至四倍，可以保持这部分参数量相近，但逐层依赖链的长度也增至原来的四倍。每个矩阵更小，但计算要经过更多层；每层还要保存相应的上下文状态。层数与宽度由此在相近参数预算下改变了计算粒度和依赖深度。

专家结构也可以作类似调整，同时保持参数量和运算量不变。单层路由专家的总参数为 $3Edf$，单 token 专家运算量为 $6k_{\text{top}}df$。将专家数 $E$ 减半、中间维度 $f$ 加倍、选中数 $k_{\text{top}}$ 减半，这两个量都不变。Qwen3-235B-A22B 的 128 个宽度 1536 的专家、每 token 选 8 个，可以与 64 个宽度 3072 的专家、每 token 选 4 个作这一对照。路由输出维度与每专家接收行数仍会改变，从而改变分组矩阵乘法（把多个专家的矩阵乘法组织成一次执行）的形状。

用算例核对基线与两个变体。在 16 个 token 的算例中，三种专家粒度的专家矩阵运算量保持相同，基线与 64 专家变体均约为 454 GFLOPs。专家数减半、单专家宽度加倍时，同样多的权重分成更少的大矩阵；专家数加倍、宽度减半时，则成为更多的小矩阵。路由器要为每个专家计算路由分数，因此其参数也随专家数变化。专家粒度计算将路由与专家矩阵分别累计，说明专家参数总量相同时，矩阵形状为何仍会不同。

矩阵和状态的形状也决定多卡分工的最小单位。128 个专家可分成八组，每组 16 个；四个 KV 头按完整头分工时只能形成四份，八卡执行就需要复制部分 KV 或调整分组。第 5 章从矩阵分块分析执行效率，第 6 章将进一步把这些矩阵和状态分配到各张卡。

### 2.5.2 残差连接与跨层状态

除了层数和宽度，还要看各层如何连接。残差连接把子层输入直接加到变换结果上，让原始信息和梯度有一条跨层直通的路径。普通残差计算 $x_{\ell+1} = x_\ell + f_\ell(x_\ell)$，其中 $f_\ell$ 是第 $\ell$ 个子层的变换，因此子层计算期间要保留输入 $x_\ell$，等变换结果产生后才能相加。若后续多个层还会引用同一表示，就必须将它保存到最后一次使用结束。连接方式由此直接决定哪些张量需要同时保存。

普通残差把不同深度的信息不断相加到同一条通路中。为了让层间信息有更多组合方式，同时保持深层网络中的信号传播稳定，DeepSeek V4-Flash 使用 **mHC（Manifold-Constrained Hyper-Connections，流形约束超连接）**。mHC 把一条残差通路扩展成多条，并约束通路之间的混合系数；这里的“流形约束”，具体是把残差混合矩阵的元素限制为非负，且各行与各列的和接近 1。[17]

该模型保存 4 路残差状态，每一路都是当前 token 的一份 4096 维中间表示。保留四路表示，是为了让后续层学习如何组合不同通路的信息。每个子层先将四路表示混合为 4096 维子层输入，执行注意力或 FFN，再把结果写回残差通路。因此，主体矩阵保持 4096 维输入，新增工作集中在进入和离开子层的混合与汇合。四路状态让层间可以传递更多组表示，混合操作将这些表示与原有子层连接起来。

处理 8192 个输入 token 时，mHC 的混合投影约需 555 GFLOPs，非矩阵运算约需 149 GFLOPs。残差混合矩阵先经 **Sinkhorn 迭代**归一化，即交替将每行、每列除以各自的和，使行和与列和逐渐接近 1；这使多层混

[17] DeepSeek V4 技术报告第 2.2 节说明 mHC 的多路表示与稳定传播，第 2.3 节说明 CSA / HCA；Kimi K3 技术报告说明 Attention Residuals 沿深度选择信息的动机与块式实现。

合中的数值尺度更稳定。归一化后的系数再用于组合各条残差通路；末端汇合这些通路，形成送入词表头的表示。因此，多路残差不仅增加要保存的表示，还增加每层的混合与归一化计算。[18]

Kimi K3 希望当前层能够按需要选取不同深度的信息，而不只是接收逐层相加后的单一混合结果。为此，Kimi K3 使用 **Attention Residuals（AttnRes，注意力残差连接）**，对较浅层的表示计算权重，再加权求和，形成当前层的输入。这种注意力沿网络深度选择信息，前面的序列注意力则在不同 token 之间选择信息。为减少保存和读取所有层输出的开销，块式实现把若干层组织成一块，保留供后续块选择的表示。[19]

后续层还要使用这些块表示，本次前向中要继续保存，前向结束后即可释放。上下文 KV 则要保留到后续生成步骤。两者需要保存的时间范围不同：AttnRes 增加一次前向中的临时存储，KV 随请求跨多次调用保留。[20]

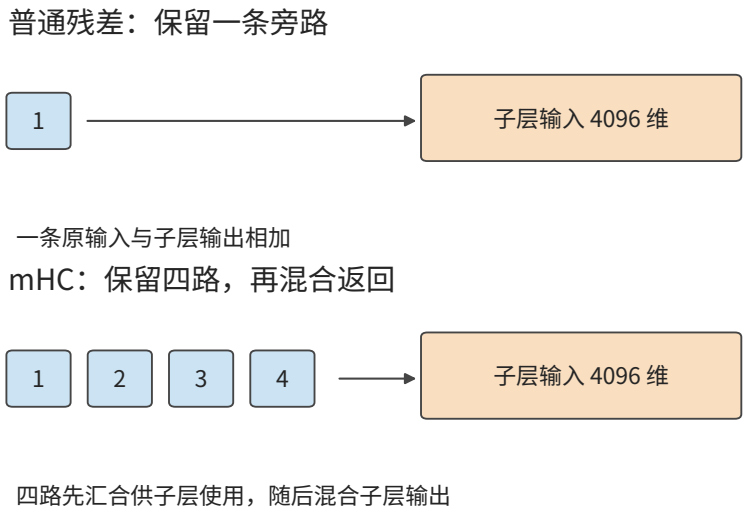



图 2-24　普通残差与四路 mHC 的连接范围。普通残差保留一条子层输入旁路；mHC 汇合多路状态供子层计算，再混合输出。两种图中的子层输入均为 4096 维。

### 2.5.3 多 token 预测与辅助计算

前两节改变的是主干内部的形状与连接；另一类设计在主干末端附加预测模块，改变一次前向能给出多少个 token。普通自回归主干根据当前上下文预测下一个 token。**多 token 预测（Multi-Token Prediction，MTP）**在此基础上增加后续多个 token 的预测目标或辅助预测模块。训练时，这些额外目标促使当前表示包含对更远后续内容有用的信息；推理时，辅助模块可以先提出多个候选 token，再由主模型验证，以减少生成同样数量 token 所需的串行轮数。

**推测解码（speculative decoding）**先用较低开销生成候选序列，再让目标模型验证这些候选。用于推测解码时，MTP 根据当前主干特征预先生成后续 token，再交给目标模型验证。如果多个 token 通过验证，一轮就能产生多个有效输出；否则从最后一个通过验证的位置继续生成。因此，加速效果取决于草稿生成、目标模型验证各需多久，以及每轮有多少 token 通过验证。第 8 章将计算这三者的关系。

[18] mHC 运算量。

[19] DeepSeek V4 技术报告第 2.2 节说明 mHC 的多路表示与稳定传播，第 2.3 节说明 CSA / HCA；Kimi K3 技术报告说明 Attention Residuals 沿深度选择信息的动机与块式实现。

[20] 逐层调度记录。

以 DeepSeek V4-Flash 为例计算一次辅助预测。其 MTP 辅助块接收主干特征和 token ID，预测后续位置。对单个 token，即 $B = 1, P = 1$，一次辅助计算约需 1.80 GFLOPs，其中共享词表头约需 1.06 GFLOPs。共享词表头占这次计算的一半以上：虽不增加独立权重，每次调用仍要执行词表投影。[21]

辅助预测使一次前向执行能够提供多个候选，经验证和采样后形成有效输出。第 3 章将把编码、生成、迭代及实时输出组织成完整负载。

设一次普通 decode 耗时为 $T_d$，一轮推测解码的草稿和验证合计耗时为 $T_s + T_v$，平均得到 $\bar{g}$ 个有效输出。每输出分摊时间为 $(T_s + T_v)/\bar{g}$，有收益的条件是

$$\bar{g} > \frac{T_s + T_v}{T_d}.$$

例如，普通 decode 为 20 ms，草稿与验证合计为 30 ms，平均每轮得到两个有效输出时，每输出分摊 15 ms；若只能得到一个，则需要 30 ms。多预测几个位置会增加单轮工作量，只有采用的输出足够多，才能抵消新增的计算开销。

## 2.6 整个模型的资源需求与比较

第 2.3 至 2.5 节介绍了三类结构变化：上下文表示决定状态大小和访问方式，专家选择决定每次使用哪些权重，网络深度与连接方式决定计算顺序。本节将这些部分合起来，比较完整模型在相同输入条件下的计算量与存储需求，再用具体任务检查模型输出。

### 2.6.1 各组件的资源需求与全模型汇总

**五种模型的注意力结构与专家配置。**本章选择一个稠密基线和四个结构不同的 MoE 模型，用相同的矩阵规则比较其资源需求。表 2-A 中参数量的后缀 T 表示万亿。

[21] DeepSeek V4-Flash MTP 计算。

**表 2-A　五个模型的总体架构（文本部分）**

| 架构参数 | DeepSeek V4.1 Flash | Qwen3-8B | Qwen3.6-35B-A3B | DeepSeek V4-Flash | Kimi K3 |
|---|---|---|---|---|---|
| 逻辑参数总数，约 | 551.57B + 196.93B Engram | 8.19B | 34.66B | 284.33B | 2.78T |
| 主干层数 $L$ | 40（20 + 20） | 36 | 40 | 43 | 93 |
| 主干隐藏维度 $d$ | 5120 | 4096 | 2048 | 4096 | 7168 |
| 注意力组成 | 2 SWA + 38 CSA2 | 36 GQA | 30 线性 + 10 全注意力 | 2 窗口 + 21 CSA + 20 HCA | 69 KDA + 24 MLA |
| 前馈组成 | 40 层 MoE | 稠密 SwiGLU | 40 层 MoE | 43 层 MoE | 首层稠密 + 92 层 MoE |
| 每 MoE 层路由专家数 $E$ | 384 | — | 256 | 256 | 896 |
| 每 token 选择数 $k$ | 6 | — | 8 | 6 | 16 |
| 路由专家计算宽度 $d_e$ | 5120 | — | 2048 | 4096 | 3584 |
| 稠密 FFN / 路由专家中间宽度 $f$ | 2304 | 12288 | 512 | 2048 | 3072 |
| 共享专家中间宽度 | 2304 | — | 512 | 2048 | 合计 6144 |
| 跨层连接 | Single-Pass mHC，4 路 | 普通残差 | 普通残差 | mHC，4 路 | AttnRes |

表中列出五个模型的文本部分。V4.1 Flash 的约 551.57B 主干参数与 196.93B Engram 参数分别列出；后者包含约 196.61B 查表参数及其投影、门控参数，两者共同计入后文的完整文本权重。V4.1 的 Single-Pass mHC 是 mHC 的改进：每层沿用上一层算出的输入混合系数，使残差状态只需读取一遍。Kimi K3 将不同的前馈结构用于不同层：首层是中间维度为 33792 的稠密 FFN，后续层则使用中间维度为 3072 的路由专家。稠密层让每个 token 经过同一组宽矩阵，MoE 层则让每个 token 使用多组较窄的专家矩阵。

V4.1 Flash 的执行阶段如下：40 层分成 20 层因果编码器和 20 层解码器，只有第 2、8、14、20 层保存独立全局 KV，第 2、8、14、20、24、28、32、36 层运行索引器，层号从零开始。Full 层生成缓存和索引，Reindex 层重算索引，Reuse 层复用已有选择；每层仍保存自己的局部窗口。因此，层数不再等于独立全局缓存的份数，输入长度也不再等于每层实际处理的查询 token 数。

表中还有三个关系。Qwen3.6 的总参数比 Qwen3-8B 多，主干宽度却只有一半：它把大量参数放进可选择的专家，每个 token 只调用其中八个。Qwen3-8B 与 DeepSeek V4-Flash 的主干同为 4096 维，DeepSeek V4-Flash 通过每层 256 个专家形成更大的参数集合。Kimi K3 又将专家计算移到 3584 维潜空间，使专家输入不必与主干采用相同的宽度。这些差异说明，“保存多大的模型” 和 “一个 token 做多少工作” 需要分别计算。表 2-1、2-2、2-4、2-5、2-6 将逐个展开这五个模型，DeepSeek-V3 的表 2-3 用作 MLA 的历史参照。

有了一类层的参数、一次调用的运算和一条请求的状态，整模型只需把这些项按各类层的实际数量加权求和：

$$M_W = b_W \sum_g L_g N_g + M_{\text{embedding}} + M_{\text{head}}, \qquad F_{\text{model}} = \sum_g L_g F_g + F_{\text{head}}.$$

$g$ 表示模型层的类别，同一类层采用相同的注意力、前馈网络与连接结构；$L_g$ 为第 $g$ 类层的数量。MoE 的 $N_g$ 包含全部专家，$F_g$ 则使用各专家实际接收的行数。表 2-1 已建立 Qwen3-8B 的稠密基线；章末查阅表按同

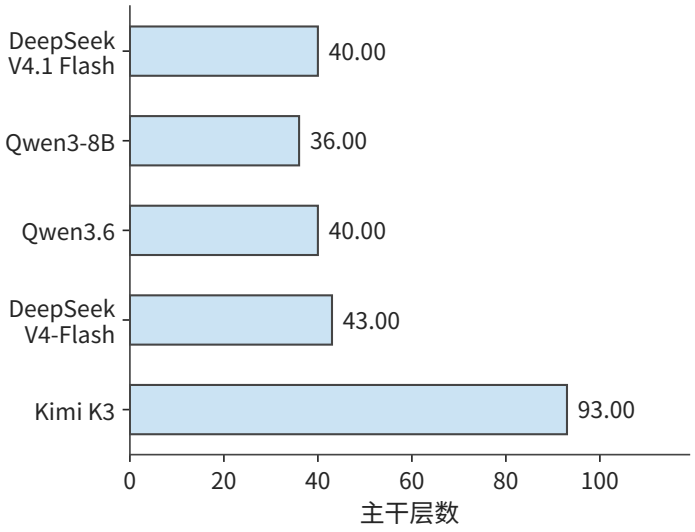



图 2-25　五模型的主干层数，使用同一线性尺度。层数决定同类工作沿网络深度重复多少次。

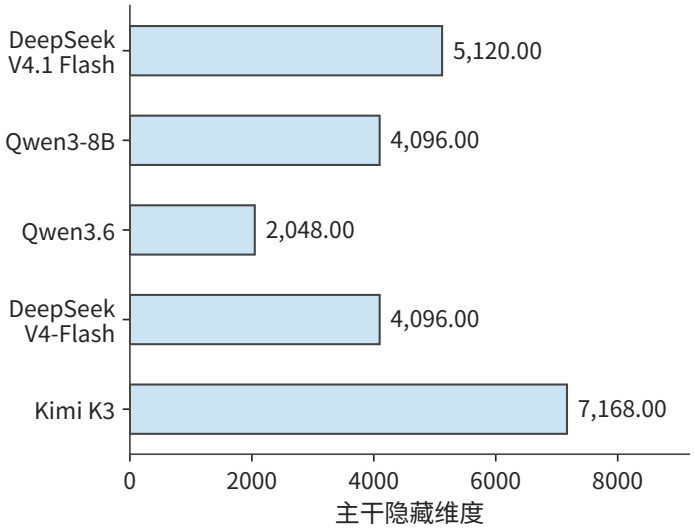



图 2-26　同一线性尺度下的主干隐藏维度。每个 token 的特征宽度决定投影的输入或输出尺寸。

样方式展开 Qwen3.6、DeepSeek-V3、DeepSeek V4-Flash、Kimi K3 和 DeepSeek V4.1 Flash，方便读者查明总量中的每一项来自哪个矩阵。

阅读时可以按数据流走一遍：输入进入哪类注意力层，随后经过哪类前馈网络，最终如何得到词表分数；累加时还要加入嵌入查行、末端归一化和词表头。每张表按注意力、前馈与连接、输出接口分组；“层 / 调用数”给出整模型累计倍数，“矩阵 FLOPs”仍是该行对应操作执行一次的浮点运算量。

将同样的累加规则用于其他模型，可以逐项列出各模型的计算分支。下表按模型列出要累计的上下文状态、权重与附加计算，以及合计时最容易遗漏的部分；MTP、DSpark（V4.1 的推测解码模块）与视觉模块按本次执行范围单独加入。

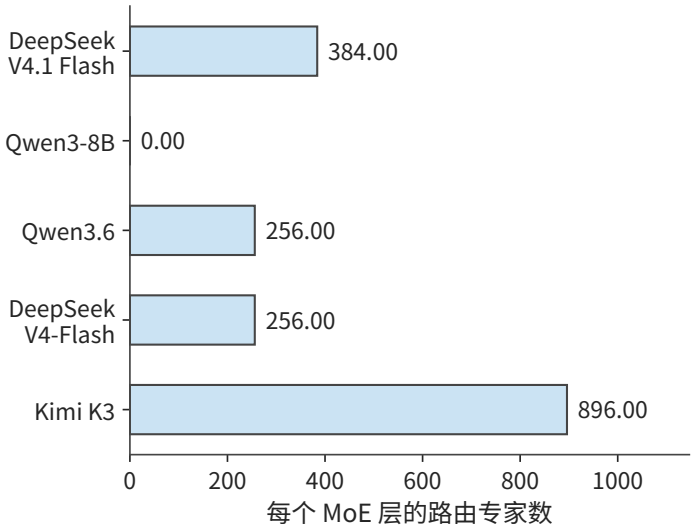



图 2-27　每个 MoE 层保存的路由专家数量。Qwen3-8B 使用稠密前馈层，其路由专家数为零；每个 token 实际选中数见表 2-A。

| 模型分支 | 上下文状态 | 权重与附加计算 | 合计时最容易遗漏的部分 |
|---|---|---|---|
| DeepSeek V4.1 Flash | 4 份共享全局 KV / 索引 + 各层 SWA + 压缩缓冲与 token 历史 | CED、CSA2、Engram、专家、Single-Pass mHC | 编码器全输入、解码器末尾重放与全局投影的不同位置数 |
| Qwen3-8B | GQA KV | 稠密 SwiGLU、输出头 | 非矩阵算术、输出头位置范围 |
| Qwen3.6 | 全局 KV + 递推与卷积状态 | 路由 / 共享专家 | 线性层保存固定大小的递推状态 |
| DeepSeek-V3 | MLA 潜变量或展开 KV，随执行路径变化 | 前三层稠密 FFN、后续 MoE 与共享专家 | 上投影执行位置、两种缓存路径的区别 |
| DeepSeek V4-Flash | 窗口 + 压缩 + 索引 + 缓冲 | 专家、压缩器、mHC | 索引扫描、块边界更新、共享分支 |
| Kimi K3 | MLA + KDA + 卷积 | 潜空间专家、AttnRes | 展开路径、跨层临时状态、首层差异 |

先依次比较完整权重、单步计算量和上下文状态大小，再通过各模型的分项表说明差异分别来自哪些矩阵和状态。

五个模型的逐矩阵展开见本章末的模型矩阵查阅表。下面用分项合计解释资源差异。

将矩阵尺寸相乘并按层数累计，DeepSeek V4-Flash 主干约有 2843 亿个逻辑参数，统一 BF16 表示约为 568.7 GB。

Kimi K3 文本主干约有 2.78 万亿个参数，绝大多数位于 92 个 MoE 层的路由专家中。每个 token 在每层只选择 16 个专家，因此完整模型的存储容量与单 token 的计算量取决于不同的因素：前者由全部专家决定，后者由选中的专家及共同执行的注意力、共享专家和投影决定。[22]

下面将各模型的参数划分为互不重叠的六类：嵌入与输出、注意力投影、稠密或共享 FFN、路由专家、Engram、其余组件。将各类参数相加即为模型总参数量，再按每个参数 2 字节换算，就得到图 2-28 的权重容量。这种分

[22] Kimi K3 模型配置说明；Kimi K3 前向计算。

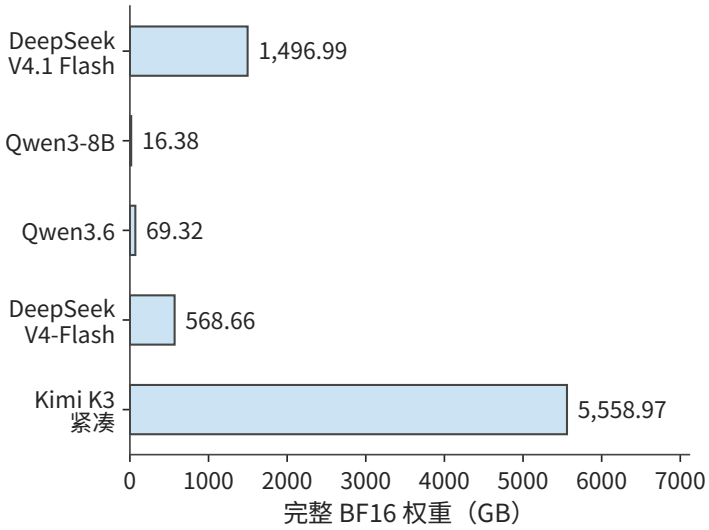



图 2-28　五模型完整权重统一用 BF16 表示时的容量，使用同一线性尺度。这里累计全部参数，包括每次仅选中部分的路由专家。

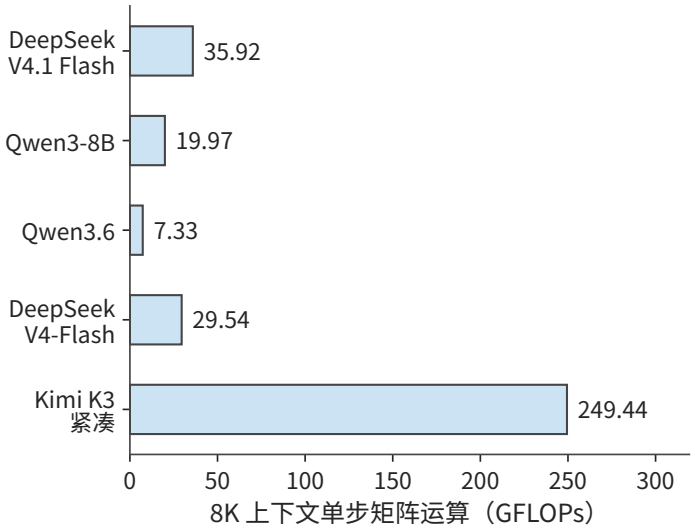



图 2-29　相同单请求、已有 8K 上下文时的单步矩阵运算量。按各模型在表 2-C 说明的执行路径累计，Kimi K3 采用紧凑 MLA。

解能直观展示参数的分布，也是解释下一张表的依据。

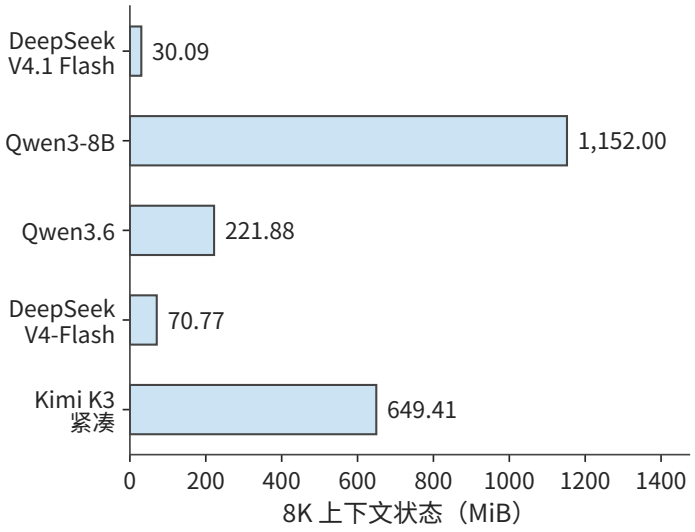



图 2-30　相同 8K 上下文下的每请求状态。上下文表示采用 BF16，递推矩阵和压缩缓冲采用相应实现的精度；各项合计为一条请求保存 8192 个上下文 token 的容量。

**表 2-B　参数由哪些部分构成（单位：十亿个逻辑参数）**

| 模型 | 嵌入与输出头 | 注意力投影 | 稠密 / 共享 FFN | 路由专家 | Engram | 其他 |
|---|---|---|---|---|---|---|
| DeepSeek V4.1 Flash | 1.324 | 5.126 | 1.416 | 543.582 | 196.929 | 0.118 |
| Qwen3-8B | 1.245 | 1.510 | 5.436 | 0.000 | 0.000 | $< 0.001$ |
| Qwen3.6 | 1.017 | 1.283 | 0.126 | 32.212 | 0.000 | 0.022 |
| DeepSeek V4-Flash | 1.059 | 5.085 | 1.082 | 277.025 | 0.000 | 0.081 |
| Kimi K3（紧凑 MLA） | 2.349 | 36.180 | 12.882 | 2722.741 | 0.000 | 5.333 |

其他包含路由器、潜空间进出投影、归一化、卷积和连接等参数；注意力投影包括 DeepSeek V4-Flash 的压缩与索引投影。Qwen3-8B 的主要参数集中在处理每个 token 时都要执行的稠密 FFN，Qwen3.6、V4-Flash 和 Kimi K3 则将大部分参数放进路由专家。V4.1 Flash 还保存了较大的 Engram 表，表中单列它的查表、投影与门控参数；查表参数增加了完整模型容量，却不意味着每个 token 都要遍历整个表。Qwen3.6 的路由专家占约 93%，DeepSeek V4-Flash 与 Kimi K3 约为 97%—98%。因此，三者的总容量主要由专家集合决定，单 token 的专家运算量则取决于每层选中的专家。

**表 2-C　相同输入条件下的 prefill 计算与选中专家权重**

| 模型 | 8K prefill（TFLOPs） | 单 token 选中路由专家的 BF16 权重（GiB） |
|---|---|---|
| DeepSeek V4.1 Flash | 143.95 | 15.820 |
| Qwen3-8B | 133.59 | — |
| Qwen3.6 | 52.09 | 1.875 |
| DeepSeek V4-Flash | 231.13 | 12.094 |
| Kimi K3（紧凑 MLA） | 1863.46 | 90.562 |

两种调用均取 $B=1$，prefill 为 $S=0, P=8192$，decode 为 $S=8192, P=1$，词表头均只处理末尾 token；图 2-29 的单步矩阵运算量按同一组条件累计。图 2-30 的状态是保存 8192 个上下文 token 所需的空间：上下文表示采用 BF16，递推状态和压缩缓冲按相应实现采用 FP32。V4.1 Flash 还计入参考实现的 int64（64 位整数）token 历史缓存，供 Engram 构造跨调用 n-gram；生产 FP4 全局缓存与 FP8 窗口的容量见第 2.3.6 节。表中最后一列将一个 token 在各层选中的路由专家权重相加，再按 BF16 换算为字节数。把这张表与图 2-28 至图 2-30 对照，就能分别比较完整模型占用的空间、一次调用的运算量，以及请求上下文占用的空间。

各模型按自己的执行路径累计：V4.1 Flash 采用论文 CED + 末尾窗口重放及候选集内索引，Qwen3-8B 为有效因果注意力，Qwen3.6 为逐项执行的矩形注意力实现（eager）与块式线性分支，DeepSeek V4-Flash 为有效主注意力及参考索引，Kimi K3 为紧凑 MLA 与块式 KDA。对照图 2-28 至图 2-30：Qwen3.6 保存的权重约为 Qwen3-8B 的四倍，单步矩阵运算量却更少；DeepSeek V4-Flash 权重继续扩大，8K 状态反而因压缩与选择缩小；Kimi K3 拥有更多专家，隐藏维度更大、层数也更多，因此需要保存更多权重，完成更多运算。这些差异说明，总参数量、单次运算量和上下文状态大小需要根据各自对应的结构分别计算。

**由上下文状态大小推算单步读取量。** Qwen3-8B 每步使用 1152 MiB 旧 KV；DeepSeek V4-Flash 按图 2-30 采用的 BF16 参考格式，主注意力读取与索引扫描合计 27.625 MiB，并更新压缩器；换成第 2.3.6 节的生产混合格式则为 12.556 MiB。Kimi K3 紧凑 MLA 使用 216 MiB 上下文状态，414 MiB 递推矩阵还要一读一写，读写量合计 828 MiB；Qwen3.6 对应 160 MiB 全局注意力的 KV 缓存和 120 MiB 递推矩阵读写。递推状态每步都要读出并更新，上下文表示则要供当前查询使用。因此，即使状态大小固定，每步也仍有相应的读写开销。

**量化编码与元数据共同决定权重容量。** 统一按 BF16 计算时，每个参数占 2 字节，权重大小可以直接反映参数量的差异。实际发布的模型 checkpoint（检查点，保存参数的文件集合）使用各自的存储格式，DeepSeek V4-Flash 主干约为 156.02 GB，Kimi K3 文本约为 1559.97 GB。文件大小由低位宽矩阵、采用较高精度的参数以及格式元数据共同决定；统一精度反映参数规模，文件大小则反映这些参数实际占多少字节。模型加载后，还需要为格式转换和计算分配空间。第 5 章将分析存储格式与运行时实现如何影响显存占用。

比较两张表时，还要注意 V4.1 Flash 各阶段的分工。V4.1 的完整文本权重包含 Engram；prefill 的全部矩阵计算则由编码器、解码器全局 KV 投影、末尾窗口重放和输出头共同组成。生成新 token 时，两部分网络都需要执行，因此不能把 prefill 节省的比例直接用于 decode。

表 2-B 的参数组成解释了权重容量，表 2-C 则展示这些参数如何参与一次调用。二者结合起来，可以区分“保存整个模型需要多少空间”与“处理一个 token 要使用多少资源”。[23]

表 2-C 中 V4.1 Flash 的 8K prefill 只需 143.95 TFLOPs。表 2-D 把这笔计算拆到各执行阶段，并与参考全层前向对照。

**表 2-D　V4.1 Flash 的 8K 输入如何分配到执行阶段（TFLOPs）**

| 阶段 | 参考全层前向 | CED + 末尾窗口重放 |
|---|---|---|
| 20 层编码器 | 142.139 | 141.727 |
| 解码器共享全局 KV 与索引键投影 | 0.044 | 0.044 |
| 20 层解码器的查询与前馈等计算 | 140.202 | 2.175 |
| 末尾 token 词表头 | 0.001 | 0.001 |
| 合计 | 282.386 | 143.947 |

参考路径执行全部 8192 个 token 的 40 层，并按公开实现先计算矩形索引分数再屏蔽；CED 路径的编码器处理全部输入，解码器全局 KV 与索引键仍为全部输入生成，解码器主体只处理末尾 128 个 token，后续索

[23] 五模型比较数据；表格生成程序；五模型统一计算；Qwen3.6 末尾 token 输出头计算。

引限制在候选集合内。表中分别包含各路径的全部文本矩阵工作，差异同时来自执行位置数与索引算法。

两条路径均包含注意力、路由评分、路由及共享专家、Engram 投影、Single-Pass mHC 投影和词表头。把解码器的全局投影与查询计算分开，就能看出 CED 如何缩短长提示经过的计算链。[24]

**上下文从 8K 增至 1M 时的运算量与状态增长。**长文档问答和多轮任务会放大上下文访问的成本。下面保持单请求、一次新增一个 token 和末尾 token 词表头不变。8K 场景有 8192 个历史 token；1M 场景有 1,048,575 个历史 token，加上当前查询正好为 1,048,576 个可见位置。

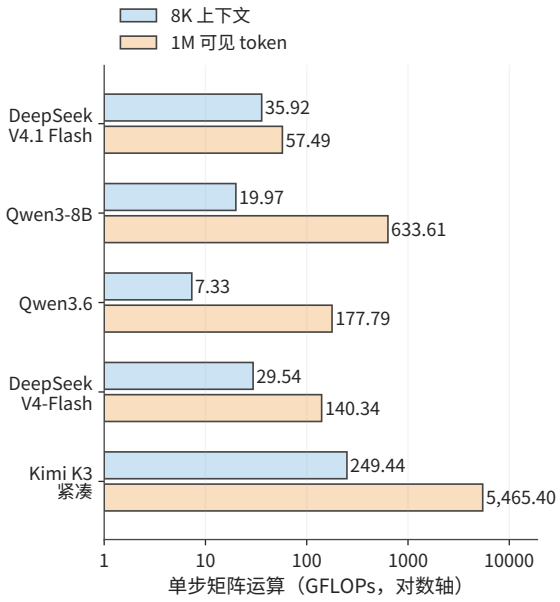



图 2-31　8K ／ 1M 上下文下再处理一个 token 的矩阵运算量。沿用表 2-C 的执行路径，横轴为对数刻度；1M 包含当前查询。

表 2-E 单独累计当前查询的注意力评分与值汇总；V4-Flash 与 V4.1 Flash 再加入索引点积。状态列列出调用前的上下文和固定状态，精度与图 2-30 一致。Qwen3-8B 与 Qwen3.6 的发布配置分别支持 40,960 和 262,144 个位置，表中两者的 1M 行按同一结构外推，用于比较增长规律。[25]

**表 2-E　长上下文下的交互运算与状态容量**

| 模型 | 8K 上下文交互（GFLOPs） | 1M 上下文交互（GFLOPs） | 8K 状态（GiB） | 1M 状态（GiB） |
|---|---|---|---|---|
| DeepSeek V4.1 Flash | 3.66 | 25.23 | 0.029 | 3.138 |
| Qwen3-8B | 4.83 | 618.48 | 1.125 | 144.000 |
| Qwen3.6 | 1.34 | 171.80 | 0.217 | 20.060 |
| DeepSeek V4-Flash | 3.00 | 113.80 | 0.069 | 6.735 |
| Kimi K3（紧凑 MLA） | 41.08 | 5257.04 | 0.634 | 27.423 |

V4.1 Flash 的全局 KV 只有四份独立来源，增加上下文时不会为所有 40 层各增一份。主注意力每层最多使用 128 个局部 token 和 512 个全局条目；解码器首个 Full 层生成候选集合，后续四个 Reindex 层各扫描至多 16384 个候选条目。编码器的三个索引器和解码器首个索引器仍需扫描增长的全局历史，因此计算量继续上

[24]V4.1 Flash 完整文本矩阵计算实现；CED 8K 输入、参考全层输入、8K decode、1M decode。计算与复核记录。

[25]8K / 200K / 1M 比较数据；统一计算实现。计算检查 8K 数值与图 2-29、图 2-30 一致，并核对压缩块边界、候选上限和逐层运算。

升；跨层共享和分层索引改变了增长幅度。1M 下，其上下文交互为 25.23 GFLOPs，完整单步矩阵工作为 57.49 GFLOPs。

V4-Flash 则沿用 CSA 与 HCA 的两种压缩粒度：CSA 的主注意力选择上限保持不变，但索引扫描随历史增长；HCA 会读取全部已完成的粗粒度条目。1M 下，其上下文交互为 113.80 GFLOPs，完整单步矩阵工作为 140.34 GFLOPs。较短上下文时，V4.1 Flash 更大的主干会增加投影和专家计算；历史足够长时，较少的索引器与分层候选选择才会抵消这部分新增工作。

Kimi K3 的 69 个 KDA 层保持固定递推状态，24 个 MLA 层继续访问随上下文增长的历史。紧凑 MLA 减少了缓存容量，却没有取消查询与历史 token 之间的计算；1M 下，这部分交互达到 5257.04 GFLOPs，整模型达到 5465.40 GFLOPs。评估长上下文设计，应分别检查保存了多少状态、访问哪些位置，以及为索引和更新状态增加了多少工作。

统一计算还保留了 200K 场景：V4.1 Flash 的完整单步矩阵工作为 40.21 GFLOPs，V4-Flash 为 50.48 GFLOPs。到 1M 时，两者的计算量差异进一步扩大，所以正文选用 1M 展示长上下文下的设计价值。

### 2.6.2 参数规模、数值精度与容量约束

把分项计算得到的模型权重和每条请求状态放进同一张显存容量表，就可以判断一张卡能同时容纳多少条请求，以及量化权重能释放多少空间。

对一张卡，先从总容量中扣除权重、工作区和其他固定预留空间，剩余空间才能放请求状态。在请求等长、各自独立保存 KV 的简化情形下，可写为：

$$B_{\max} = \left\lfloor \frac{M_{\text{device}} - M_{\text{weight}} - M_{\text{workspace}}}{M_{\text{state/request}}} \right\rfloor$$

分子是扣除固定驻留数据后可用于请求的空间，分母是每条请求的状态大小。每增加一条请求，都要为它分配相应的状态空间；剩余空间无法容纳完整的一份状态时，就无法再增加请求，因此结果要向下取整。递推与卷积状态同样属于每请求数据，与 KV 一起计入分母。

以一张 24 GB 显存的 RTX 4090 运行 Qwen3-8B BF16 为例，权重占 16,381,470,720 字节，工作区预留 2 GiB，每条请求的 8K 上下文占 1.125 GiB。可容纳请求数为

$$B_{\max} = \left\lfloor \frac{24 \times 10^9 - 16{,}381{,}470{,}720 - 2 \times 2^{30}}{1.125 \times 2^{30}} \right\rfloor = 4.$$

上下文变为 16K 时，每条请求占 2.25 GiB，同一空间只能放两条；变为 4K 时，则能放九条。请求数按整数跳变，是因为剩余空间必须容纳一条请求的完整状态，容纳一部分并无意义。

保留多少历史，由此成为既影响应用、也影响系统的决定。应用多保留一段材料，模型就能继续参考它；服务却要为每条请求分配更多状态空间。第 2.3 节的 GQA、MLA 和递推表示进一步改变了保存这些信息的方式。模型结构、上下文组织与并发容量可以放在同一份预算里比较，再用实际任务测试，确定哪些方案能达到所需的回答质量。

量化能释放多少空间，以更大的 DeepSeek-R1-Distill-Llama-70B 为例。第 1.2.3 节已给出该模型的 BF16 权重 141.107 GB、8-bit 分组量化权重 73.726 GB，以及一条含 8192 个 token 请求的 2.5 GiB BF16 KV。同一分组方案下，让嵌入、输出头和归一化等部分继续使用较高精度，并计入 scale 及打包时尾部填充的空间后，4-bit 方案约为 39.500 GB。

4-bit 方案的 39.500 GB 包括低位宽矩阵、仍采用高精度的参数，以及 scale 与打包开销，比把所有参数一律按半字节折算的结果更大。换到一张显存 80 GB（十进制）的 H100 SXM、固定预留 2 GiB 时，8-bit 方案可放 1 条 8K 请求，4-bit 方案可放 14 条。上下文从 8K 增至 32K 时，每请求 KV 增至四倍，4-bit 方案可同时容纳的请求数降到 3 条。压缩权重释放的空间可以保存更多请求的上下文，但每条上下文越长，能同时容纳的请求就越少。[26]

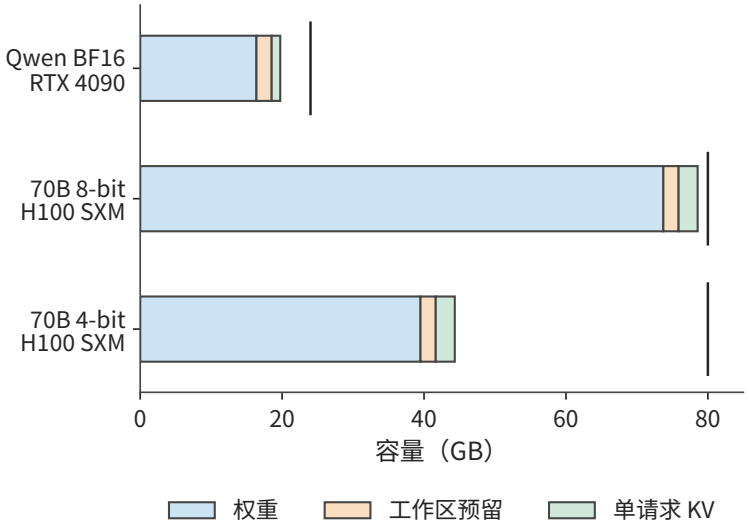



图 2-32　权重、固定预留和一条请求 KV 的逐项容量。短竖线标出 RTX 4090 的 24 GB 与 H100 SXM 的 80 GB 容量；KV 采用 BF16、上下文长度 8192，固定预留为 2 GiB。“工作区预留”指执行所需的临时空间，“单请求 KV”指一条 8192 个 token 的上下文的缓存。

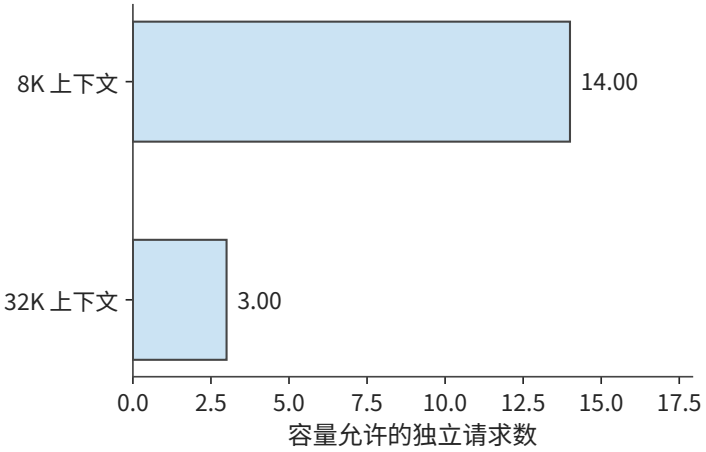



图 2-33　70B 的 4-bit 方案在同一张 80 GB 的 H100 SXM 上，上下文长度从 8K 增至 32K 时可同时容纳的独立请求数。每请求状态增加，使剩余空间容纳的请求数减少。

同样方法也能分析更大的部署。235B 的 BF16 权重约 470 GB，超过八张 RTX 4090 合计的 192 GB，而一台 HGX 服务器的八张 H100 SXM 合计为 640 GB。后者扣除权重后约剩 170 GB，要容纳各请求的状态、工作区，以及切分到多卡后需要在几张卡上重复保存的数据，例如每张卡各存一份的归一化参数与路由器，以及 4 个 KV 头分到八张卡时，每个头在两张卡上各存一份的 KV。第 6 章将根据模型在各卡上的分工，计算每张卡的这几项预算。

容量决定一个实例至少需要多少张卡，请求长度与并发数则决定实例要处理多少工作。更低位宽释放空间，

[26] 8K 容量表；32K 对照。

较短上下文允许更多请求同时驻留，批内复用则改变这些请求的执行成本。因此，选择部署方案时需要依次考虑容量、访问量与运算量，第 3 章将进一步加入请求的到达与时限。

**练习 2-7〔核心〕：由显存容量决定可容纳请求数**

使用显存为 24 GB 的 RTX 4090，工作区预留 2 GiB，Qwen3-8B BF16 权重采用表中的精确值。分别求 4K、8K、16K 上下文下最多可同时容纳多少条请求；在 8K 情形下，将工作区预留增至 3 GiB，重新计算。再使用表中的 70B 4-bit 权重 39.500 GB，换成显存为 48 GB 的 RTX 6000 Ada、预留仍为 2 GiB，分别求 8K 与 32K 上下文的请求数。先比较权重、工作区与 KV 的容量需求，再用剩余显存计算可容纳的请求数。

反过来，给定加速器和服务目标，也可以求留给模型的空间。设每卡可用容量为 $C$，固定预留工作区为 $M_0$，每条请求的 KV 为 $K$，并发为 $B$，每参数占 $b_W$ 字节。忽略其他驻留对象，单卡容量给出的参数上界为

$$N_{\max} = \frac{C - M_0 - BK}{b_W}.$$

先确定服务所需的状态大小，再计算留给权重的空间。取 RTX 4090 的 24 GB 容量、2 GiB 工作区、4 条各 8192 个 token 的请求，每请求 KV 沿用 Qwen3-8B 的 1.125 GiB。BF16 每参数 2 字节，剩余空间最多容纳约 85.1 亿参数。[27] 图 2-34 先扣除服务状态，再划出权重预算。

改变模型结构时，权重预算与状态预算会一起变化。增加层数或 KV 头数会增大每请求的 $K$，留给权重的空间随之减少；跨卡部署则扩大总容量，同时增加通信。Llama 3 报告中的 405B BF16 推理使用两台、共 16 张 H100，其所需容量超出了单台服务器能提供的显存。[28] 模型规模、状态结构与卡数因此需要一起选择。

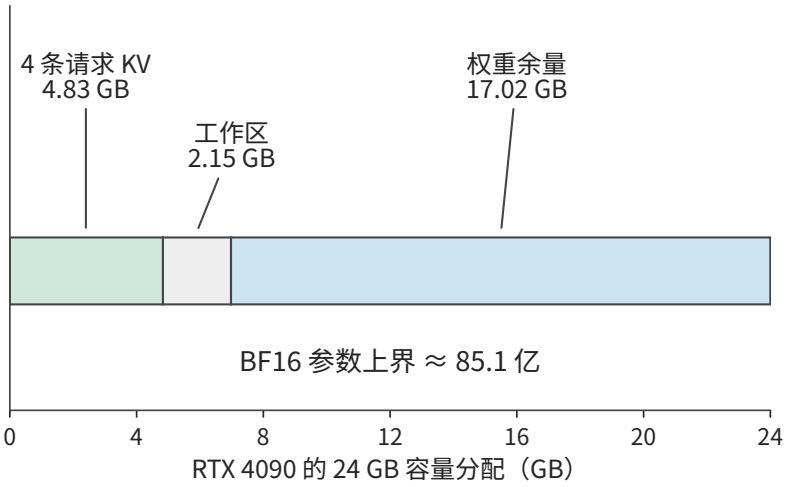



图 2-34　从加速器反推权重预算。RTX 4090 的 24 GB 中先为 4 条 8K 请求的 KV 和 2 GiB 工作区预留空间，余量给出 BF16 参数上界；图中容量均为十进制 GB。

### 2.6.3 不同请求条件下的模型比较

最后把容量、调用链和任务质量合在一起。先根据每张卡的容量排除容量不足的配置，再沿每个模型的实际计算方式累计请求工作，最后检查模型能否完成目标任务。

比较完整模型时，可以从两种输入出发。固定 $S, P, G, B$，得到相同调用长度下的资源需求，便于观察结构

[27] 本章条件比较的逐项复算与计算程序。

[28] MQA 原文摘要；GQA 原文摘要；Llama 3 报告第 6.1 节。容量示例按题设的服务状态推导。

差异；固定文本和任务，则先经各模型的分词器得到输入长度，再检查输出。这两种对照分别回答“同样多的位置如何计算”和“同一个任务需要如何执行”。

先比较相同输入、输出长度下的资源需求。取 $S=0, P=128, G=4, B=1$，模型执行一次 prefill 和三次 decode，最终保存 131 个已处理位置。将这些调用的矩阵运算量相加，得到下表。[29]

| 模型 | 完整请求矩阵运算量（TFLOPs） | 本次计算采用的实现方式 |
|---|---|---|
| DeepSeek V4.1 Flash | 4.14 | CED + 末尾窗口重放，候选集内索引 |
| Qwen3-8B | 1.83 | 有效因果注意力 |
| Qwen3.6 | 0.66 | 矩形全注意力与块式线性分支 |
| DeepSeek V4-Flash | 3.40 | 有效主注意力与参考索引 |
| Kimi K3（紧凑 MLA） | 27.16 | 紧凑 MLA 与块式 KDA；decode 采用递推 KDA |

这张表与表 2-C 保持同一执行路径，Kimi K3 均使用紧凑 MLA，把上投影移到查询和汇总一侧。V4.1 Flash 的输入恰好为 128 个 token，全部落在解码器重放窗口内，因此这次短请求没有长提示那种跳过大量解码器位置的收益。完整请求沿同一条选定路径累计 prefill 与三次 decode，上下文从 128 增到 131；这一路径的矩阵形状与状态表示共同决定各项运算量。

从一次调用扩展到完整请求之后，输入和输出的作用更加清楚。增加 $P$，主要增加一次输入处理的工作量，以及后续 decode 开始时的上下文长度；增加 $G$，则反复执行投影、专家与输出头，并多次读取上下文。长输入、短输出与短输入、长输出由此对各类资源提出不同的需求，第 3 章将分析这两类请求持续到达时的系统负载。

**练习 2-8〔延伸〕：前缀复用与输入改写如何改变模型计算量**

一轮输入总长 1443 个 token，其中前 1392 个命中缓存，返回 128 个 token。求 $S, P, G, n_d$ 的值、请求结束时的上下文长度，以及本轮新增 KV 的大小。相对于完全重算，复用缓存后投影行数与有效因果查询—键配对数分别减少多少？若在第 501 个输入 token 修改了内容，重新求最长可复用前缀与新输入长度。

再看第二种对照，即固定文本。此时不同分词器会产生不同的输入长度。例如，在八道短语检索题中，Qwen3-8B 与 DeepSeek V4-Flash 都正确找到了目标短语，但短文在 Qwen 中对应 523 或 524 个 token，在 DeepSeek V4-Flash 中对应 501 个；较长文本分别对应 2121 或 2122 个和 2036 个。使用相同文本可以保持任务内容一致，模型各自的 token 数则决定了实际输入形状。

因此，比较模型需要同时回答两个问题：能否完成同一任务，以及完成任务需要多少资源。先按相同标准判断答案，再将各自的输入长度代入模型表，才能把结构差异与实际用途联系起来。[30]

**练习 2-9〔延伸〕：根据容量、计算需求与任务质量选择模型**

使用图 2-28 至图 2-30 与表 2-C，为每个模型分别列出完整权重、8K 状态、单步 decode 与 8K prefill 的需求。部署在显存为 80 GB 的单张 H100 SXM 上，预留 2 GiB 工作区，并采用 BF16 权重。先判断哪些模型能完整放入单卡；再把问题改为多卡部署，说明还需从后续章节获得哪些时间与通信参数。将输出 token 数 $G$ 改为 1，重新计算 prefill 与 decode 的调用次数。最后为一个短语检索任务写出质量标准，说明同 token 数比较与同文本比较分别回答什么问题。

本章从计算图出发，逐步算出了完整请求的资源需求。注意力决定如何保存和访问上下文，MoE 决定每次选择哪些权重，网络深度与连接方式决定计算顺序和临时存储。下一章将加入请求长度分布、到达时间、工具

[29] 五模型完整请求与逐调用计算；运行 `python3 calculations/reproduce_ch02.py`。五个模型统一使用本章表 2-C 的路径；Kimi K3 使用紧凑 MLA，旧的展开路径与 V4-Pro 对照保留在历史请求记录。

[30] 配对检索实验。Qwen 使用 BF16 / vLLM；DeepSeek V4-Flash 使用 MXFP4 专家、FP8 KV / SGLang，并将部分权重放在 CPU 内存。

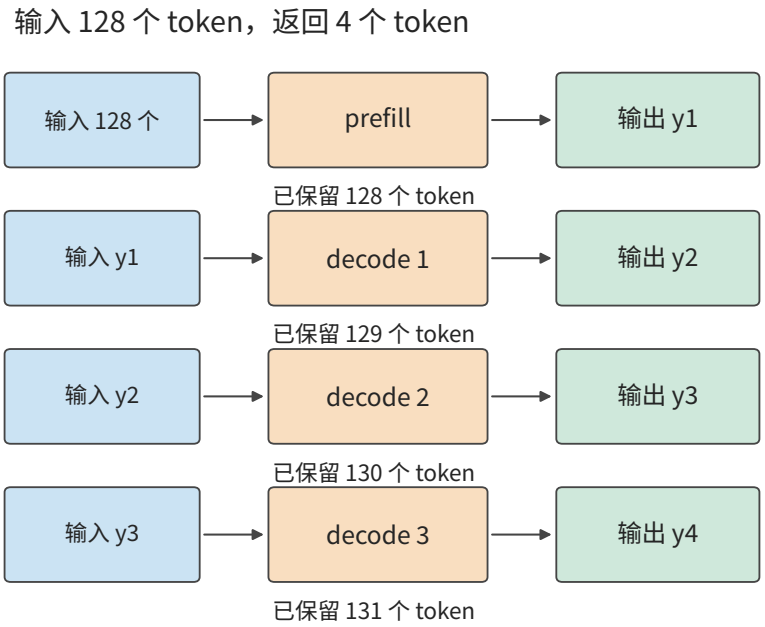



图 2-35　生成四个输出的调用顺序。prefill 处理 128 个输入并产生首输出，随后三次 decode 各将前一输出送回模型；最终保留 131 个 token。

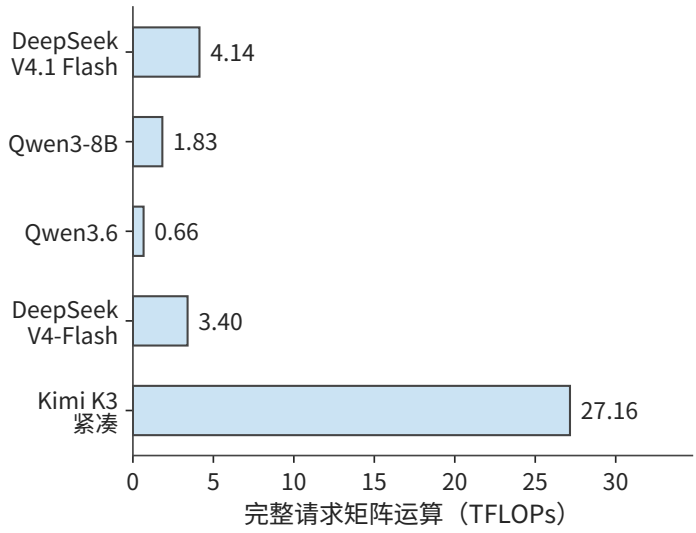



图 2-36　输入 128 个 token、输出四个 token 时，各模型处理同一请求所需的矩阵运算量。模型执行范围与缓存路径见本节题设；这是逐调用累计的计算量。

调用、训练与强化学习（RL），分析这些计算如何随时间累积成系统负载。

## 谬误与陷阱

**误区：参数更多，每个 token 的计算量就一定更大。**总参数决定完整模型的容量；MoE 的选中专家、注意力机制的类型和调用形状决定当前工作。对照表 2-A、表 2-B 与图 2-29，分别解释参数量和计算量由哪些因素决定。

**误区：KV 容量降至四分之一，注意力计算量也会同比下降。** GQA 共享上下文表示，查询头仍分别评分；

MLA 改变上下文宽度，也改变上投影的执行位置。先找被改变的矩阵与状态，再计算其余工作。

**误区：状态大小固定，访问开销就可以忽略。**固定递推矩阵仍需更新；prefill 还可能产生临时块状态。容量、累计访问与峰值工作区分别回答不同问题。

**误区：输入 token 数相同，就能公平比较模型能力。**输入和输出的 token 数适合比较资源结构；同文本任务还需固定评分、质量与执行条件，并允许不同分词器产生不同长度。

## 本章练习与复现

练习 2-2、2-5、2-7 为核心；其余练习用于分析特殊情况，并将方法应用于其他模型。先独立完成预测与推导，再用配套计算核对。计算命令在仓库根目录运行，完整变体参见第 2 章扩写资料。

| 对应内容 | 本地复算入口 |
|---|---|
| 序列依赖与缓存等价性 | `python3 calculations/calc.py sequence-dependencies --format md` |
| Qwen3.6 前向与专家 | `python3 calculations/calc.py qwen36-forward --format md` |
| DeepSeek V4.1 Flash 完整矩阵计算 | `python3 calculations/calc.py v41-forward --tokens 8192 --execution ced --format md` |
| DeepSeek V4.1 Flash 参考全层路径 | `python3 calculations/calc.py v41-forward --tokens 8192 --execution reference --format md` |
| 五模型比较与完整请求 | `python3 calculations/reproduce_ch02.py` |
| DeepSeek V4-Flash 状态分解 | `python3 calculations/calc.py state --model deepseek-v4-flash --length 8192` |
| Kimi K3 状态与块式执行 | `python3 calculations/calc.py k3-kda-chunk --tokens 8192` |
| 专家粒度 | `python3 calculations/calc.py qwen235-expert-granularity --format md` |
| 单次 MTP | `python3 calculations/calc.py v4-mtp-forward --format md` |
| 已知轨迹字段代入 | `python3 calculations/calc.py trace-resource-bridge --format md` |
| 五模型请求比较 | `python3 calculations/calc.py request-model-comparison --format md` |

本章配图、计算数据与重建方法见配图目录。章末资料介绍了各模型的结构与设计。

## 模型矩阵查阅表

这组表用于从整模型合计追溯到具体矩阵。查阅时先确定模型与执行分支，再用单次矩阵 FLOPs 乘层数；路由专家还要按实际接收的输入行数累计。表 2-1 的 Qwen3-8B 基线见第 2.1.4 节。

**表 2-2　Qwen3.6-35B-A3B：混合注意力与 MoE 各模块的资源消耗计算**

主干共 40 层：10 层选完整注意力分支，30 层选线性注意力分支；两类层随后都执行 MoE。每 token 从 256 个路由专家中选 8 个，另有一个共享专家。线性分支的 QKV 输出拆为 2048、2048、4096 维；其 $Q$、$K$ 会

按值头的分组关系复用。

**模型入口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词嵌入查行 | $m$ 个 ID → $m \times 2048$ | 查表 $248320 \times 2048$ | 0；查行访问另计 | 1 |

**完整注意力分支**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 完整注意力：$Q$ 与输出门控 | $m \times 2048 \to m \times 8192$ | $2048 \times 8192$ | $2 \times m \times 2048 \times 8192$ | 10 |
| 完整注意力：$K$、$V$ | $m \times 2048 \to m \times 512$ | $2048 \times 512$，共 2 个 | $2 \times 2 \times m \times 2048 \times 512$ | 10 |
| 完整注意力：上下文交互 | 16 个 $Q$ 头、2 个 KV 头，头宽 256 | 无新增权重 | $4 \times 16 \times 256 \times N_{\text{pair}}$ | 10 |
| 完整注意力：输出投影 | $m \times 4096 \to m \times 2048$ | $4096 \times 2048$ | $2 \times m \times 4096 \times 2048$ | 10 |

**线性注意力分支**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 线性注意力：QKV 联合投影 | $m \times 2048 \to m \times 8192$ | $2048 \times 8192$ | $2 \times m \times 2048 \times 8192$ | 30 |
| 线性注意力：输出门控 $z$ | $m \times 2048 \to m \times 4096$ | $2048 \times 4096$ | $2 \times m \times 2048 \times 4096$ | 30 |
| 线性注意力：$a$、$b$ 控制量 | $m \times 2048 \to m \times 32$ | $2048 \times 32$，共 2 个 | $2 \times 2 \times m \times 2048 \times 32$ | 30 |
| 线性注意力：卷积与 delta 递推 | $Q, K$ 各 $16 \times 128$；$V$ 为 $32 \times 128$ | 卷积：8192 通道、核宽 4 | 递推为 $O(m \times 32 \times 128^2)$；块式算法另计 | 30 |
| 线性注意力：输出投影 | $m \times 4096 \to m \times 2048$ | $4096 \times 2048$ | $2 \times m \times 4096 \times 2048$ | 30 |

**各层共同的专家分支**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 每层：路由评分 | $m \times 2048 \to m \times 256$ | $2048 \times 256$ | $2 \times m \times 2048 \times 256$ | 40 |
| 路由专家 $e$：在选中分支变换 | $t_e \times 2048 \to t_e \times 512 \to t_e \times 2048$ | $2048 \times 512$ 两个；$512 \times 2048$ 一个 | $6 \times t_e \times 2048 \times 512$ | 40 |
| 共享专家：所有 token 的共同变换 | $m \times 2048 \to m \times 512 \to m \times 2048$ | $2048 \times 512$ 两个；$512 \times 2048$ 一个 | $6 \times m \times 2048 \times 512$ | 40 |
| 共享专家门控 | $m \times 2048 \to m \times 1$ | $2048 \times 1$ | $2 \times m \times 2048 \times 1$ | 40 |

**输出接口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词表头（末端一次） | $m_{\text{out}} \times 2048 \to m_{\text{out}} \times 248320$ | $2048 \times 248320$ | $2 \times m_{\text{out}} \times 2048 \times 248320$ | 1 |

累计时，用各行公式乘最后一列的层数；路由专家部分还需对各专家实际接收的行数 $t_e$ 求和。末端归一化执行一次，层内归一化、激活、位置处理和状态更新沿相应分支执行。

读这张表时，先根据这一层使用线性注意力还是完整注意力，选择对应的一组表格行，再加 MoE 的共同部分。线性注意力决定上下文如何保存，MoE 决定当前 token 使用哪些权重；两种机制可以同时存在。A3B 描述名义激活规模，全部专家仍共同构成常驻模型。[31]

**表 2-3　DeepSeek-V3：MLA 与稠密／专家前馈网络**

DeepSeek-V3 共 61 层，前三层使用稠密 FFN，后 58 层每 token 选择 8 个路由专家并执行一个共享专家。下面按展开路径计算 MLA：每个 token 的潜变量经上投影形成各头的 $K$、$V$，64 维位置表示与 $K$ 拼接。查询与键共同参与位置相关的评分，$V$ 则用于汇总内容。

**模型入口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词嵌入查行 | $m$ 个 ID $\to m \times 7168$ | 查表 $129280 \times 7168$ | 0；查行访问另计 | 1 |

**MLA 注意力**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 查询下投影：形成低秩查询 | $m \times 7168 \to m \times 1536$ | $7168 \times 1536$ | $2 \times m \times 7168 \times 1536$ | 61 |
| 查询上投影：展开 128 个头 | $m \times 1536 \to m \times 24576$ | $1536 \times 24576$ | $2 \times m \times 1536 \times 24576$ | 61 |
| KV 下投影：512 维潜变量及 64 维位置分支 | $m \times 7168 \to m \times 576$ | $7168 \times 576$ | $2 \times m \times 7168 \times 576$ | 61 |
| KV 上投影：展开每头 $K$、$V$ | $m \times 512 \to m \times 32768$ | $512 \times 32768$ | $2 \times m \times 512 \times 32768$ | 61 |
| 展开 MLA：评分与值汇总 | 128 头；Q/K 宽 192，$V$ 宽 128 | 无新增权重 | $2 \times 128 \times (192 + 128) N_{\text{pair}}$ | 61 |
| 注意力输出投影 | $m \times 16384 \to m \times 7168$ | $16384 \times 7168$ | $2 \times m \times 16384 \times 7168$ | 61 |

[31] Qwen3.6 前向记录。

**前馈分支**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 前三层：稠密前馈网络 | $m \times 7168 \to m \times 18432 \to m \times 7168$ | $7168 \times 18432$ 两个；$18432 \times 7168$ 一个 | $6 \times m \times 7168 \times 18432$ | 3 |
| 后 58 层：路由评分 | $m \times 7168 \to m \times 256$ | $7168 \times 256$ | $2 \times m \times 7168 \times 256$ | 58 |
| 路由专家 $e$ | $t_e \times 7168 \to t_e \times 2048 \to t_e \times 7168$ | $7168 \times 2048$ 两个；$2048 \times 7168$ 一个 | $6 \times t_e \times 7168 \times 2048$ | 58 |
| 共享专家 | $m \times 7168 \to m \times 2048 \to m \times 7168$ | $7168 \times 2048$ 两个；$2048 \times 7168$ 一个 | $6 \times m \times 7168 \times 2048$ | 58 |

**输出接口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词表头（末端一次） | $m_{\text{out}} \times 7168 \to m_{\text{out}} \times 129280$ | $7168 \times 129280$ | $2 \times m_{\text{out}} \times 7168 \times 129280$ | 1 |

累计时，用各行公式乘最后一列的层数；路由专家部分还需对各专家实际接收的行数 $t_e$ 求和。末端归一化执行一次，层内归一化、激活、位置处理和状态更新沿相应分支执行。

V3 将两种节省资源的方法组合起来：MLA 用低秩潜变量表示上下文，MoE 让一个 token 只执行部分专家。DeepSeek V4-Flash 进一步改变上下文的组织方式，用窗口保存近期位置，再用不同压缩比例保存长期上下文。下面的模块表将这几类上下文分别展开。[32]

**表 2-4　DeepSeek V4-Flash：选择性上下文访问与专家计算**

43 层包含 2 个纯窗口层、21 个 CSA 层和 20 个 HCA 层；每 token 选 6 个路由专家，另有一个共享专家。$A$ 是该层跨全部请求的主注意力有效查询—键配对数，$I$ 是索引实际扫描的查询—键配对数，两者均不含头数。分组输出投影的八组处理八份不同切片。CSA 的压缩投影先产生包含重叠分支的 1024 / 256 维表示，再分别形成 512 维的主注意力上下文表示和 128 维的索引条目。

**模型入口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词嵌入查行 | $m$ 个 ID $\to m \times 4096$ | 查表 $129280 \times 4096$ | 0；查行访问另计 | 1 |

[32] V3 前向计算。

**所有层的注意力投影**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 所有层：查询下投影 | $m \times 4096 \to m \times 1024$ | $4096 \times 1024$ | $2 \times m \times 4096 \times 1024$ | 43 |
| 所有层：查询上投影 | $m \times 1024 \to m \times 32768$ | $1024 \times 32768$ | $2 \times m \times 1024 \times 32768$ | 43 |
| 所有层：共享 KV | $m \times 4096 \to m \times 512$ | $4096 \times 512$ | $2 \times m \times 4096 \times 512$ | 43 |
| 主注意力：窗口及所选压缩上下文 | 64 头，每头 512 维 | 无新增权重 | $4 \times 64 \times 512 \times A$ | 43 |
| 所有层：分组输出低秩投影 | $m \times 4096 \to m \times 1024$ | $4096 \times 1024$，共 8 个 | $8 \times 2 \times m \times 4096 \times 1024$ | 43 |
| 所有层：拼接后输出投影 | $m \times 8192 \to m \times 4096$ | $8192 \times 4096$ | $2 \times m \times 8192 \times 4096$ | 43 |

**CSA 专有工作**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| CSA：主注意力的上下文压缩与门控 | $m \times 4096 \to m \times 1024$ | $4096 \times 1024$，共 2 个 | $2 \times 2 \times m \times 4096 \times 1024$ | 21 |
| CSA：索引查询 | $m \times 1024 \to m \times 8192$ | $1024 \times 8192$ | $2 \times m \times 1024 \times 8192$ | 21 |
| CSA：索引头权重 | $m \times 4096 \to m \times 64$ | $4096 \times 64$ | $2 \times m \times 4096 \times 64$ | 21 |
| CSA：索引压缩内容及门控 | $m \times 4096 \to m \times 256$ | $4096 \times 256$，共 2 个 | $2 \times 2 \times m \times 4096 \times 256$ | 21 |
| CSA：索引扫描及选择 | 64 个索引头，每头 128 维 | 无新增权重 | 点积 $2 \times 64 \times 128 \times I$；加权与 top-k 另计 | 21 |

**HCA 专有工作**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| HCA：主注意力的上下文压缩与门控 | $m \times 4096 \to m \times 512$ | $4096 \times 512$，共 2 个 | $2 \times 2 \times m \times 4096 \times 512$ | 20 |

**各层专家与连接**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 路由评分 | $m \times 4096 \to m \times 256$ | $4096 \times 256$ | $2 \times m \times 4096 \times 256$ | 43 |
| 路由专家 $e$ | $t_e \times 4096 \to t_e \times 2048 \to t_e \times 4096$ | $4096 \times 2048$ 两个；$2048 \times 4096$ 一个 | $6 \times t_e \times 4096 \times 2048$ | 43 |
| 共享专家 | $m \times 4096 \to m \times 2048 \to m \times 4096$ | $4096 \times 2048$ 两个；$2048 \times 4096$ 一个 | $6 \times m \times 4096 \times 2048$ | 43 |
| mHC：残差汇合与混合 | $m \times 4 \times 4096 \leftrightarrow m \times 4096$ | 混合投影与归一化参数 | 混合矩阵、标量与特殊函数独立累计 | 43 |

**输出接口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词表头（末端一次） | $m_{\text{out}} \times 4096 \to m_{\text{out}} \times 129280$ | $4096 \times 129280$ | $2 \times m_{\text{out}} \times 4096 \times 129280$ | 1 |

累计时，用各行公式乘最后一列的层数；路由专家部分还需对各专家实际接收的行数 $t_e$ 求和。末端归一化执行一次，层内归一化、激活、位置处理和状态更新沿相应分支执行。

这张表说明，“每步只选少量上下文”不等于总工作量固定：主注意力使用 $A$，索引扫描使用 $I$，压缩投影则对新输入持续执行。这三类工作量的增长规律不同。[33]

**表 2-5　Kimi K3：KDA、紧凑 MLA 与潜空间专家**

93 层包含 69 个 KDA 层、24 个 MLA 层；首层使用稠密 FFN，其余层从 896 个路由专家中选 16 个，并执行合计宽度 $2 \times 3072 = 6144$ 的共享分支。表内 MLA 采用直接对潜变量计算注意力的紧凑路径，96 路小投影分别使用每头参数。共享专家直接处理 7168 维主干向量，路由专家才进入 3584 维潜空间。

**模型入口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词嵌入查行 | $m$ 个 ID $\to m \times 7168$ | 查表 $163840 \times 7168$ | 0；查行访问另计 | 1 |

**KDA 分支**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| KDA：$Q$、$K$、$V$ | $m \times 7168 \to m \times 12288$ | $7168 \times 12288$，共 3 个 | $3 \times 2 \times m \times 7168 \times 12288$ | 69 |
| KDA：衰减低秩投影 $a$ | $m \times 7168 \to m \times 128$ | $7168 \times 128$ | $2 \times m \times 7168 \times 128$ | 69 |
| KDA：衰减低秩投影 $b$ | $m \times 128 \to m \times 12288$ | $128 \times 12288$ | $2 \times m \times 128 \times 12288$ | 69 |
| KDA：更新门控 | $m \times 7168 \to m \times 96$ | $7168 \times 96$ | $2 \times m \times 7168 \times 96$ | 69 |
| KDA：输出门控 | $m \times 7168 \to m \times 12288$ | $7168 \times 12288$ | $2 \times m \times 7168 \times 12288$ | 69 |
| KDA：有限状态更新 | 96 个 $128 \times 128$ 状态矩阵 | 短卷积及衰减参数 | 递推 $O(m \times 96 \times 128^2)$；块式实现另计 | 69 |
| KDA：输出投影 | $m \times 12288 \to m \times 7168$ | $12288 \times 7168$ | $2 \times m \times 12288 \times 7168$ | 69 |

[33] DeepSeek V4-Flash 完整前向记录。

**MLA 分支**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| MLA：查询下投影 | $m \times 7168 \to m \times 1536$ | $7168 \times 1536$ | $2 \times m \times 7168 \times 1536$ | 24 |
| MLA：查询上投影 | $m \times 1536 \to m \times 18432$ | $1536 \times 18432$ | $2 \times m \times 1536 \times 18432$ | 24 |
| MLA：潜变量及额外分支 | $m \times 7168 \to m \times 576$ | $7168 \times 576$ | $2 \times m \times 7168 \times 576$ | 24 |
| MLA：将键的上投影与查询变换合并 | $m \times 128 \to m \times 512$ | $128 \times 512$，共 96 个 | $96 \times 2 \times m \times 128 \times 512$ | 24 |
| 紧凑 MLA：潜空间评分与汇总 | 96 头；评分宽 576，汇总宽 512 | 使用紧凑的上下文表示 | $2 \times 96 \times (576 + 512) N_{\text{pair}}$ | 24 |
| MLA：汇总后恢复值表示 | $m \times 512 \to m \times 128$ | $512 \times 128$，共 96 个 | $96 \times 2 \times m \times 512 \times 128$ | 24 |
| MLA：输出门控 | $m \times 7168 \to m \times 12288$ | $7168 \times 12288$ | $2 \times m \times 7168 \times 12288$ | 24 |
| MLA：输出投影 | $m \times 12288 \to m \times 7168$ | $12288 \times 7168$ | $2 \times m \times 12288 \times 7168$ | 24 |

**前馈与专家分支**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 首层：稠密前馈网络 | $m \times 7168 \to m \times 33792 \to m \times 7168$ | $7168 \times 33792$ 两个；$33792 \times 7168$ 一个 | $6 \times m \times 7168 \times 33792$ | 1 |
| 其余层：路由评分 | $m \times 7168 \to m \times 896$ | $7168 \times 896$ | $2 \times m \times 7168 \times 896$ | 92 |
| 进入专家潜空间 | $m \times 7168 \to m \times 3584$ | $7168 \times 3584$ | $2 \times m \times 7168 \times 3584$ | 92 |
| 路由专家 $e$：潜空间变换 | $t_e \times 3584 \to t_e \times 3072 \to t_e \times 3584$ | $3584 \times 3072$ 两个；$3072 \times 3584$ 一个 | $6 \times t_e \times 3584 \times 3072$ | 92 |
| 合并后离开潜空间 | $m \times 3584 \to m \times 7168$ | $3584 \times 7168$ | $2 \times m \times 3584 \times 7168$ | 92 |
| 共享专家：两个专家宽度合并 | $m \times 7168 \to m \times 6144 \to m \times 7168$ | $7168 \times 6144$ 两个；$6144 \times 7168$ 一个 | $6 \times m \times 7168 \times 6144$ | 92 |

**深度连接**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| AttnRes：深度方向混合 | 可供选择的块表示 → 当前主干表示 | 深度混合参数 | 按每层参与混合的块数计，不作为上下文 KV | 按参与混合的块数累计 |

**输出接口**

| 模块及作用 | 输入 → 输出（行数 × 宽度） | 权重矩阵（输入宽 × 输出宽） | 矩阵 FLOPs | 层 / 调用数 |
|---|---|---|---|---|
| 词表头（末端一次） | $m_{\text{out}} \times 7168 \to m_{\text{out}} \times 163840$ | $7168 \times 163840$ | $2 \times m_{\text{out}} \times 7168 \times 163840$ | 1 |

累计时，用各行公式乘最后一列的层数；路由专家部分还需对各专家实际接收的行数 $t_e$ 求和。末端归一化执行一次，层内归一化、激活、位置处理和状态更新沿相应分支执行。

阅读 Kimi K3 的表时，需要分别看注意力和前馈网络的层数：69 层采用 KDA，24 层采用 MLA；第一层采用稠密 FFN，后 92 层采用潜空间 MoE。共享专家保持 7168 维主干输入，路由专家先经过降维投影，处理

3584 维输入。表中的层数、宽度与 $t_e$ 因而分别控制重复次数、单次尺寸与实际调用量。[34]

**表 2-6　DeepSeek V4.1 Flash：CED、共享上下文与 Engram**

本表覆盖普通文本生成的全部矩阵投影。输入行数 $m_l$ 表示第 $l$ 层本次处理的查询 token 数；$u_l$ 表示需要生成全局 KV 的位置数，$n_{\mathrm{cmp},l}$ 表示本次新完成的压缩条目数。参考全层 prefill 中 $m_l = P$；CED 中编码器 $m_l = P$、解码器 $m_l = \min(P, 128)$，但解码器全局 KV 来源层仍有 $u_{20} = P$。所有行数还需乘请求数 $B$。

| 模块及作用 | 权重矩阵（输入宽 × 输出宽） | 执行行数 | 层 / 调用数 |
|---|---|---|---|
| 注意力 Single-Pass mHC 投影 | $20480 \times 24$ | $m_l$ | 40 |
| FFN Single-Pass mHC 投影 | $20480 \times 24$ | $m_l$ | 40 |
| 查询下投影 | $5120 \times 1280$ | $m_l$ | 40 |
| 查询上投影 | $1280 \times 32768$ | $m_l$ | 40 |
| 局部 SWA KV 投影 | $5120 \times 512$ | $m_l$ | 40 |
| 分组输出投影 | $4096 \times 1024$，共 8 组 | $m_l$ | 40 |
| 输出拼接投影 | $8192 \times 5120$ | $m_l$ | 40 |
| 路由评分 | $5120 \times 384$ | $m_l$ | 40 |
| 共享专家 gate | $5120 \times 2304$ | $m_l$ | 40 |
| 共享专家 up | $5120 \times 2304$ | $m_l$ | 40 |
| 共享专家 down | $2304 \times 5120$ | $m_l$ | 40 |
| 路由专家 gate | $5120 \times 2304$ | $\sum_e t_e = 6m_l$ | 40 |
| 路由专家 up | $5120 \times 2304$ | $\sum_e t_e = 6m_l$ | 40 |
| 路由专家 down | $2304 \times 5120$ | $\sum_e t_e = 6m_l$ | 40 |
| Engram 查询结果的 KV 投影 | $6144 \times 25600$ | $m_l$ | 2 |
| 全局 KV 投影 | $5120 \times 512$ | $u_l$ | 4 |
| 2:1 压缩门控 | $5120 \times 512$ | $u_l$ | 3 |
| 全局索引键投影 | $512 \times 128$ | $n_{\mathrm{cmp},l}$ | 4 |
| 索引查询投影 | $1280 \times 4096$ | $m_l$ | 8 |
| 索引头权重投影 | $5120 \times 32$ | $m_l$ | 8 |
| 末尾 token 词表头 | $5120 \times 129280$ | 末尾 token 一次 | 1 |

每个二维投影的矩阵运算量为输入行数乘输入宽、输出宽，再乘 2；分组输出投影只在各组内部运算，不能把组间不存在的连接计入。路由专家的三行按各专家接收的行数相加，共享专家处理全部查询 token 位置。

注意力交互没有新增权重：每个查询有 64 个头、每头 512 维，QK 与 PV 共同按实际访问位置累计。索引器有 32 个头、每头 128 维，Full、Reindex、Reuse 的扫描范围和是否执行索引各不相同。压缩器仅在相应块完成后产生索引键，不能把键投影的行数写成每次输入的位置数。

Engram 首先从 n-gram 对应的 24 个桶各取一个 256 维向量，拼成 6144 维输入，再经表中的投影形成四路键和一个共享值；归一化点积与门控将其写入四路残差。查表不计作矩阵乘法，读取载荷、门控和加权归约分别列在计算记录中。查表地址只由 token 序列决定，与激活无关，因此可以在该层执行之前预取，第 6.7.4 节比较表放在主机内存、HBM 或 ROM 的代价。最后的 mHC 加权汇合与 RMSNorm（均方根归一化）不引入另

[34] 完整前向记录。

一份词表权重。[35]

## 本章小结

设计模型时，也可以先从加速器容量中扣除服务所需的状态和工作区，再比较剩余空间能够容纳的模型规模与结构。MQA、GQA 与稀疏激活说明，提高系统效率也是选择模型结构的重要依据；在容量允许的范围内，还要比较质量、时延与成本，才能确定设计方案。

本章把第 1 章的三个资源量展开为可计算的对象。矩阵尺寸决定权重容量与线性运算量，查询—键配对数决定注意力交互的运算量，缓存的驻留时间与模型调用次数决定状态的存储量和读取量，专家分派决定每批使用多少份权重。面对新条件，先指出它改变了哪一个对象，再重新代入相应公式：增加 batch、延长上下文、扩大专家集合和增加选中专家数，会作用于不同的项。

读完本章，应能根据新的模型配置，列出一次前向、一次 decode 和完整生成各自的资源需求，并解释长度或 batch 改变时哪些项增长。第 3 章将进一步考虑请求的到达过程与工具调用的依赖关系，使这些需求成为随时间变化的负载。

[35] V4.1 Flash 完整文本矩阵计算实现；CED 8K 输入、参考全层输入、8K decode、1M decode。计算与复核记录。

# 第 3 章 推理与训练负载

假设某服务平均每秒收到四个请求，每个请求输入 4,608 个 token、输出 1,152 个 token。第 2 章已经说明如何按输入和输出长度计算模型工作量。把平均工作量乘以每秒请求数，是否就能确定需要多少资源?

两种服务的平均请求率和平均输入、输出长度可以完全相同：一种始终均匀接收请求，另一种前一分钟集中处理长文档，后一分钟集中生成长回答。前者各阶段的资源需求较为稳定，后者则在不同时段分别需要更多输入处理能力或生成能力。到达顺序之外，任务内部的往返与等待同样影响负载。Agent 在模型与工具之间往返，等待工具结果时模型暂停生成，上下文状态却仍要保留，供后续调用使用。训练中的序列长度、有效标签（参与损失计算的 token）和更新次数，也会影响运算量和状态需要驻留的时间。

第 2 章给出了单次调用的容量、运算量和读取量。本章再加入三个问题：工作何时到达，哪些步骤必须等待，什么结果才算任务完成。由此依次建立排队、状态驻留时间、关键路径和成功任务成本的模型，再综合计算训练和长期服务的成本。

## 3.1 从单次请求到持续到达的负载

### 3.1.1 Prefill 与 Decode

负载是一定时间内到达系统、等待和执行的工作集合。先按时间顺序分析单个请求。恢复 $S$ 个上下文 token，处理 $P$ 个新输入，返回 $G$ 个 token，调用链由一次 prefill 和 $n_d = G - 1$ 次 decode 构成。把后续 decode 从 $j = 0$ 开始编号，第 $j$ 次开始时，已有上下文为 $S + P + j$。因此，每次调用的资源需求可以用第 2 章的方法算出，调用之间的顺序则由生成依赖确定。[1]

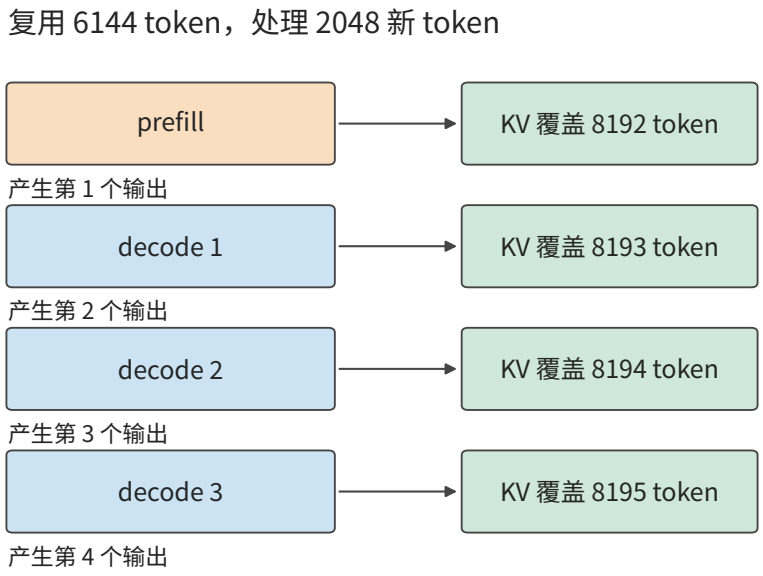



图 3-1　恢复 6144 个上下文 token，处理 2048 个新输入并生成 4 个输出。首输出来自 prefill，后续 3 次 decode 各追加一个 token；纵向表示调用次序，间距用于示意。

[1] 第 2 章模型架构，§2.2 的请求调用约定和 Qwen3-8B KV 计数；四模型请求计算。

两个阶段使用同一套权重，对资源的需求却不同。以一组权重矩阵为例，prefill 的输入包含 $BP$ 行，单步 decode 只有 $B$ 行。输入 token 较多时，一次读入的权重可以参与更多乘加运算；低并发 decode 则可能只为少数几个 token 读取整组权重。这就是“prefill 通常受算力限制、decode 通常受内存带宽限制”这种说法的由来。

单个请求之外，不同请求之间也能形成新的复用机会。多个 decode 同时执行时，共同使用权重，各自读取上下文；一条长输入到来时，prefill 可以同时处理较多已知 token，它们的特征向量组成行数更大的输入矩阵。服务系统需要在这些不同形状的工作之间分配时间。仅把总 token 数相加，会丢失这些 token 分别属于哪个阶段、何时能够执行的信息。

> **练习 3-1〔延伸〕：相同输出总量下，请求数量如何改变模型调用次数**
> 分别考虑一条请求输出 1024 个 token，以及八条请求各输出 128 个 token。每条请求的输入均为 1024 个 token。求两种情况下的 prefill 次数、后续 decode 总步数，以及各请求结束时的上下文长度。假设八条请求同步生成，比较每批共享一次权重时的读取轮数，再解释总输出相同为什么不足以判断完成时间。

### 3.1.2 任务目标与评价指标

模型请求与用户任务通常不是一一对应的。回答问题可能只需一次生成，修复函数却可能需要读文件、改代码、运行测试、根据错误再修改。评估系统时，先写出什么算完成，再决定统计什么。

先在时间线上标出三个事件：请求到达 $t_a$、首个 token 返回 $t_1$、最后一个 token 返回 $t_G$。由此得到

$$T_{\mathrm{TTFT}} = t_1 - t_a, \qquad T_{\mathrm{request}} = t_G - t_a, \qquad \overline{T}_{\mathrm{token}} = \frac{t_G - t_1}{G - 1} \quad (G > 1).$$

TTFT（time to first token）是从请求到达到首个 token 返回的时间，本书也称首响应。首响应衡量多久开始输出，完整请求延迟衡量多久生成完，平均 token 间隔衡量输出节奏。把某一段时间缩短，只会直接改变包含这段时间的指标。例如，减少请求开始执行前的排队会改善 TTFT，却不会自动缩短已经开始生成后的 token 间隔。

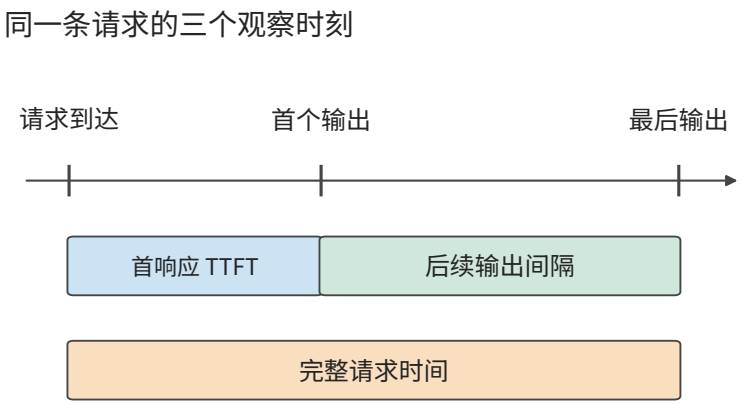



图 3-2　请求到达、首输出与末输出决定三个计时区间。首响应包含开始生成前的等待，输出间隔描述生成过程，完整请求时间从到达累计到结束。

首个 token 到达客户端之前，请求要依次经过排队、输入准备、加速器计算和结果返回等阶段。输入最初保存在主机还是加速器上，决定数据传输发生在哪里；计算完成后，结果还要经过主机处理和连接发送。记录同一请求在各阶段的开始和结束时间，就能看出 TTFT 主要消耗在哪些环节。第 5 章将展开加速器提交、传输与完成的具体时序。

客户端按块接收多个 token 时，可以直接记录每块的到达时间与 token 数，再统计输出节奏。reasoning 模型还会多出一个观察点：它在给出可见回答前先生成中间思考内容（称为思考 token），首个内部 token 出现时，用户未必已看到答案。因此，内部生成、用户可见输出和任务完成分别对应不同的时间点。语音的首次播放还要再经过解码与设备缓冲，第 3.3.3 节再讨论。

| 要回答的问题 | 需要的指标或记录 |
|---|---|
| 多久开始回应? | TTFT；reasoning 另记首个有用输出 |
| 输出是否连续? | 逐 token / 逐块间隔及其分布 |
| 多久把事情做完? | 完整请求时间、完整任务时间 |
| 每秒完成多少合格任务? | 在质量与时限要求下的有效吞吐 |
| 得到一个成功结果花多少? | 全部尝试的模型、工具和环境成本 / 成功任务数 |
| 等待期间还占着什么? | 状态所属的请求与模型版本、大小、存放位置和驻留时间 |

只看平均值会掩盖较慢的一部分请求。p95 是第 95 百分位数，用来描述所选指标中较慢的一部分观测。本章采用最近秩法：把 $n$ 个观测从小到大排列，取第 $\lceil 0.95n \rceil$ 个。四条请求的 p95 是最大值，480 条请求的 p95 是第 456 个。将请求完成、排队与首响应的耗时分别排序，就能看出长等待发生在哪一段；完整请求的 p95 则直接由各请求的全程耗时排序得到。

除响应时间外，等待期间的状态占用也需要度量。状态驻留得越久，同样的容量能同时服务的任务就越少。令任务状态大小为 $M(t)$，状态占用空间对时间的积分为

$$A_M = \int_{t_{\text{start}}}^{t_{\text{end}}} M(t)\,\mathrm{d}t.$$

将各时间段占用的空间乘以持续时间再相加，就得到占用空间随时间变化的曲线下面积。其单位为 byte · s：1 GiB 状态持续占用空间 10 秒，对应 10 GiB · s。状态没有变大，但这段时间里这块空间不能供其他请求使用。若任务以平均每秒 $\lambda$ 个的速度持续到达，每个任务都占用 $M_0$ 字节、平均等待 $\tau$ 秒，稳定情况下仅这些等待任务的平均占用就约为 $\lambda M_0 \tau$。工具等待由此也会成为容量问题。

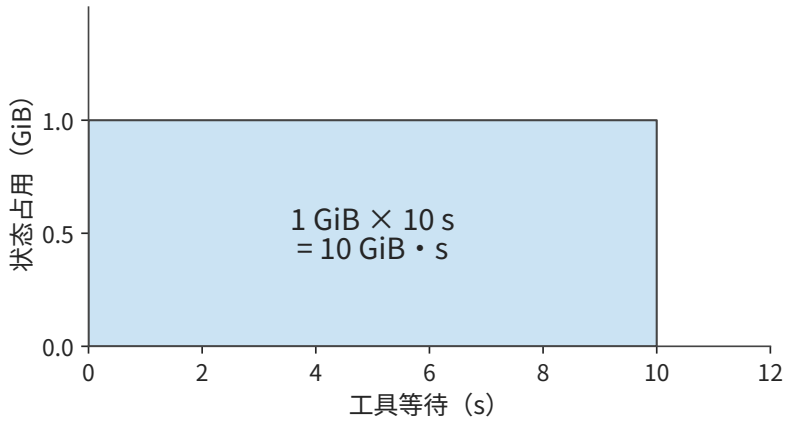



图 3-3　1 GiB 状态持续占用空间 10 秒，对应面积为 10 GiB • s。横轴为状态驻留时间，纵轴为占用空间；面积描述这段等待期间的累计存储占用。

时间与状态之外，还要核算完成任务的代价。成功任务成本把所有尝试的成本相加，再除以成功任务数。这样，失败生成、工具测试和环境运行都进入分子，分母对应用户得到的合格结果。batch 零成功时，记录其总消

耗与失败原因；取得成功结果后，再用全部尝试的总成本除以成功任务数。[2]

### 3.1.3 请求分布

前一分钟加速器有空闲，后一分钟突然到达许多需要长时间生成的请求，后到的请求仍然要排队：先前空闲的计算时间无法留到现在使用。TTFT 和输出间隔描述单个请求的体验；多个请求陆续到达时，这些指标还取决于各请求是否争用同一时段的处理能力。

负载描述应保留到达时间、输入与输出长度的联合分布。长请求既增加自身服务时间，也可能延长状态驻留；短请求提前完成，则可能让后续请求较早开始。因此，即使平均工作量相同，等待时间、完成时间和状态占用峰值也可能不同。下面保持章首两分钟负载的请求总量与平均长度不变，先计算到达顺序如何改变积压，再观察真实系统。

实际请求往往来自不同客户。某个客户主要提交长文档，另一个主要生成长回答；即使各自的请求形状稳定，各客户请求到达速率的变化也会改变整体负载。ServeGen 对生产服务的研究观察到了这种现象，并用客户组成与时间变化重建请求分布。[3]

**例题 3-1：平均处理能力足够，为什么仍会积压？** decode 由两张 RTX PRO 6000 Blackwell Workstation Edition 承担，判断它们能否持续处理章首的负载。

解：定义两类请求：长输入类输入 8192、输出 256 个 token；长输出类输入 1024、输出 2048 个 token。两分钟内各到达 240 条，合计平均每秒四条。一种负载中，两类请求始终各占一半；另一种前一分钟两类占比为 9 : 1，后一分钟为 1 : 9。每条请求的首输出由 prefill 产生，后续 decode 步数为输出数减一，于是有：

| 时间窗 | 新输入 token/s | 后续 decode 步/s |
|---|---|---|
| 均匀混合 | 18,432 | 4,604 |
| 时段变化，前一分钟 | 29,900.8 | 1,736.8 |
| 时段变化，后一分钟 | 6,963.2 | 7,471.2 |

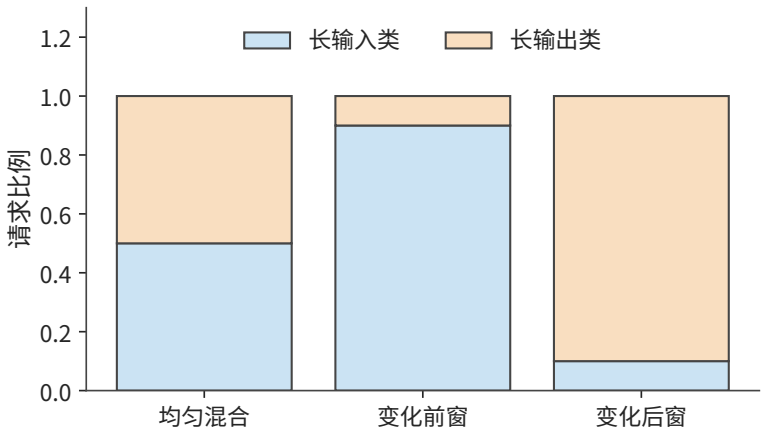



图 3-4　两分钟总量相同的三个时间窗中的请求组成。长输入类为 8192 输入、256 输出，长输出类为 1024 输入、2048 输出；两类混合比例改变阶段需求。各类请求的输入、输出长度均以 token 为单位。

[2] 2023—2026 年 token 成本调研，§2、§4、§11；API 价格、生产成本与成功任务成本分开。

[3] ServeGen，NSDI 2026，§2–7；资料与源码阅读笔记。

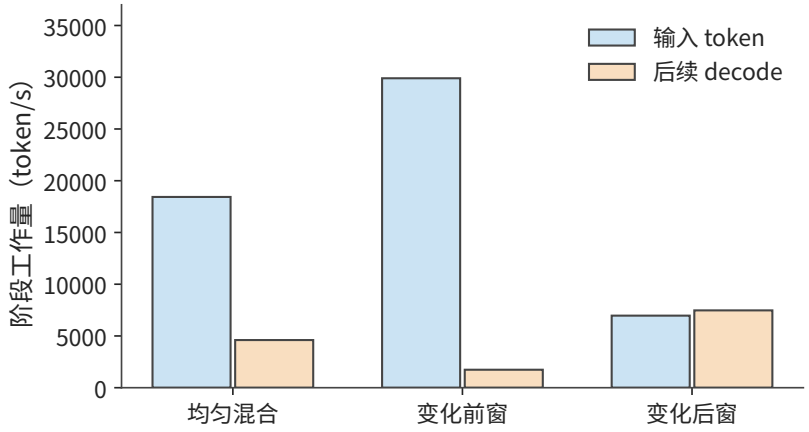



图 3-5　三个时间窗中，请求组成对应的输入与后续生成需求。输入数按 token 计，后续生成按每请求的一次 decode 步计；每请求首输出已计入 prefill。输入项统计每秒新处理的输入 token；decode 项统计所有请求每秒需要执行的后续 decode 步，每步推进一个 token。

**队列假设：窗口内均匀到达、处理率固定。**使用流体队列模型，把离散的生成工作近似为连续流量；每个窗口内均匀到达，处理率固定，初始队列为空。单位是一条请求的一次后续 decode 步，prefill 能力单独核算。

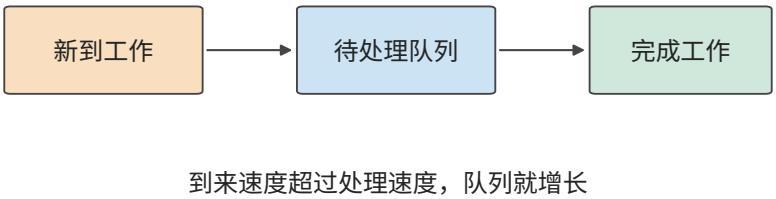



图 3-6　工作先进入队列，再由处理资源完成。到来快于处理时差额留在队列中；处理快于到来时，资源逐步消化已有积压。

以时间窗为步长，积压可按下式逐窗计算：

$$Q_{k+1} = \max(0, Q_k + A_k - \mu_k \Delta t).$$

其中 $Q_k$ 是窗口开始时的待处理工作，$A_k$ 是窗口内新到达的工作，$\mu_k \Delta t$ 是可完成量。到达速度低于处理速度时，积压逐渐减少，直至队列排空；到达速度超过处理速度时，未完成的工作留到下一窗口。

递推式中的处理率 $\mu_k$ 要从硬件参数求出。每秒四个请求的首个输出由 prefill 产生，所以 4,608 个输出中有 4,604 步属于后续 decode。decode 的处理率由显存带宽决定。RTX PRO 6000 Blackwell Workstation Edition 有 96 GB 显存、1792 GB/s 带宽，每张卡各放一份完整的 Qwen3-8B BF16 权重，最多同时执行 64 条序列。每步 decode 至少要把 15.14 GB 共享权重读一遍，再读出每条序列的全部旧 KV，每个上下文 token 占 144 KiB。decode 期间，长输入类的平均上下文为 $8192 + 127 = 8319$ 个 token，长输出类为 $1024 + 1023 = 2047$ 个；均匀混合时按 decode 步加权，平均上下文为 $(255 \times 8319 + 2047 \times 2047)/2302 \approx 2742$ 个 token。64 条序列同时执行时，一步至少读写 41.02 GB，耗时 22.89 ms，所以一张卡每秒最多完成约 2,795.66 步，两张卡约 5,591.33 步。这一处理率是显存带宽允许的物理极限，实测只会更低。三个时间窗都沿用这一处理率。

令 decode 队列初始为空，先把请求的生成工作均匀汇入各窗口。均匀混合和前一分钟的处理能力都足够；后一分钟每秒多到来 $7471.2-5591.33 \approx 1879.87$ 步，60 秒便留下约 112,792 步；停止到达后，按 112792/5591.33 计算，还需约 20.17 秒才能排空。前一分钟未使用的能力无法保存到后一分钟，这正是全局均值掩盖积压的原因。[4]

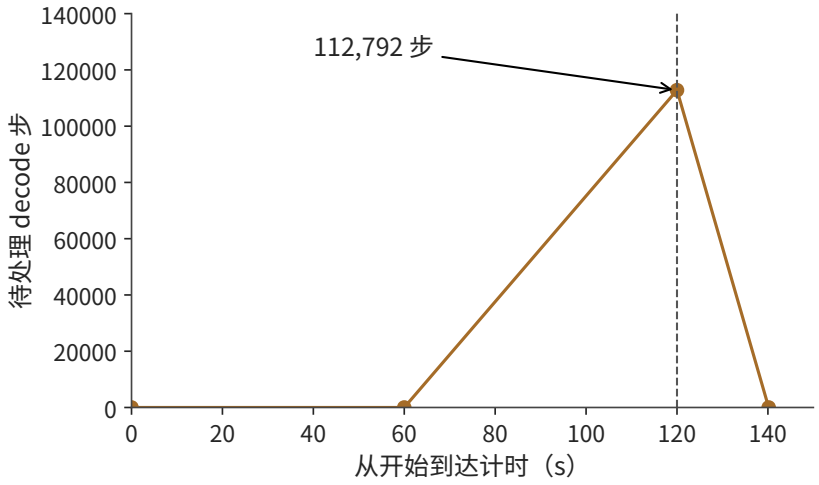



图 3-7　两张 RTX PRO 6000 每秒最多完成约 5,591 个 decode 步时的流体模型。后一分钟积压至 112,792 步，120 秒后停止到达，再经过约 20.17 秒排空。

**KV 容量限制如何影响排队与首响应。**在 Qwen3-8B / vLLM 0.23 上运行这两种到达序列，可以观察到上述差异如何影响用户等待。加速器为一张 96 GB 显存的 RTX PRO 6000 Blackwell Workstation Edition，KV 池是专门留给请求键值上下文的显存区域，本次容量为 24 GiB，同时最多执行 64 条序列；前缀缓存指跨请求复用相同开头的已有 KV，本次关闭这一功能，每条请求生成指定数量的 token。KV 空间不足时，调度器可以暂停某条已开始的请求、释放它占用的 KV 空间，之后再恢复或重算，这种行为称为抢占。两组负载都在 120 秒内到达 480 条请求，全部完成后的结果如下。[5]

| 实际观察 | 均匀混合 | 前 9:1、后 1:9 |
|---|---|---|
| 完整请求 p95 | 约 299.4 s | 约 299.4 s |
| TTFT p95 | 242.9 s | 258.9 s |
| 初次调度等待 p95 | 242.4 s | 258.4 s |
| 最后一条请求完成时距起点的时间 | 419.8 s | 418.7 s |
| 抢占事件 | 62 | 24 |

两次回放都出现了大量积压，这在例题 3-1 的带宽下界中已经可以预见：一张卡每秒最多完成约 2,796 个 decode 步，低于均匀混合时 4,604 步的平均需求，而且同一张卡还要执行 prefill。480 条请求共需 552,480 个后续 decode 步，单是这部分工作就至少需要 197.6 秒，而全部请求在 120 秒内就已到达。

两次回放的完整请求 p95 与最后完成时间接近，TTFT 和初次调度等待却相差约 16 秒。采样记录显示，两次实验的 KV 池占用都达到过 100%，请求在进入执行前经历了长时间等待。输入与输出的时段组合改变了何时释放状态、何时接纳新请求，所以即使整批请求的完成时间接近，用户开始收到响应的时间仍可不同。KV 池填满时发生的抢占，还会改变请求的等待和后续执行次序。

[4]请求分布与资源配置，两分钟固定输入和首 token 修正后的阶段需求；RTX PRO 6000 上 64 条序列的 decode 带宽下界，平均上下文取 2742 个 token。

[5]练习 3-2：真实两分钟回放，完整输入、发送与完成记录、KV 采样及运行范围。

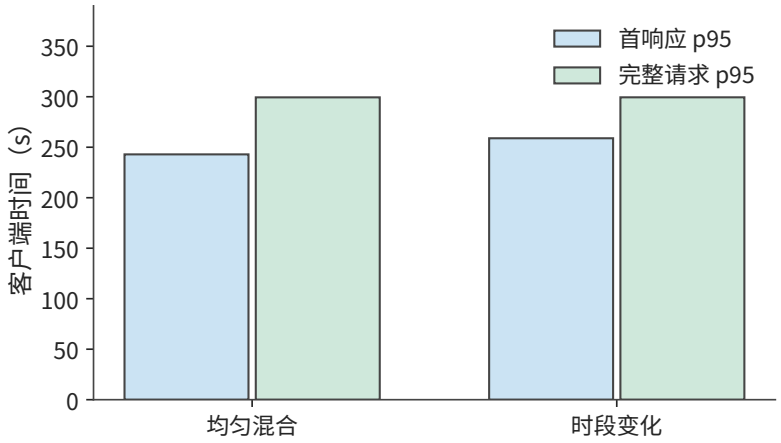



图 3-8　同一实际实例回放两种到达序列的结果。完整请求 p95 接近，首响应 p95 则相差约 16 秒；模型、加速器、KV 池与并发上限均按正文实验条件固定。

排队因此既可能来自处理速度不足，也可能来自已有请求的状态尚未释放：状态池填满后，即使部分计算单元空闲，新请求也要等待已有请求释放空间。[6]

例题 3-1 中的积压递推式还能预测其他突发条件下的积压。若工作到达速率 $\lambda_w$ 连续 $\tau$ 秒高于处理率 $\mu$，初始队列为空，则积压为 $(\lambda_w - \mu)\tau$；停止到达后的排空时间为 $(\lambda_w - \mu)\tau/\mu$。突发持续时间加倍，积压与排空时间都加倍。

**练习 3-2〔核心〕：突发请求与工具等待如何增加积压和状态占用**

采用例题 3-1 中第二分钟的负载强度，即每秒新增 7,471.2 步 decode 工作。将这一突发负载的持续时间改为 30 秒，decode 资源分别取两张和三张 RTX PRO 6000 Blackwell Workstation Edition，每张卡的处理率沿用例题 3-1 求得的约 2,796 步/秒。初始队列为空，求这 30 秒内积压量随时间的变化，以及请求停止到达后清空积压所需的时间。随后考虑另一种负载：每秒有两个任务进入工具等待阶段，每个任务等待 10 秒，期间状态占用 1 GiB 空间。求稳定状态下的平均等待任务数与状态占用；将等待延长到 30 秒后重新计算。

## 3.2 多轮交互与 Agent 任务

### 3.2.1 多轮对话与前缀增长

第 3.1 节把每个请求当作一次独立调用；在对话和 Agent 任务中，同一任务的多次调用前后相连。多轮对话的下一次请求通常带着上下文。第一轮输入系统提示和用户问题；第二轮除了新问题，还可能包含上一轮回答；之后每轮继续累积。若只看用户新写了多少字，就会低估送入模型的输入长度。

设第 $i$ 轮输入长度为 $I_i$，可复用的前缀长度为 $K_i$，则本轮需要重新处理的输入为

$$P_i = I_i - K_i, \qquad S_i = K_i.$$

将 $P_i$ 与 $S_i$ 代入第 2 章的公式，投影和 FFN 只处理新增 token，注意力仍要让新查询访问完整前缀。缓存复用因而减少了重算，却没有把旧上下文从后续计算中删除。

[6] ServeGen 窗口的实际回放。

以一次包含四轮模型调用的代码修复任务为例，计算缓存的收益。四轮累计输入 5,297 个 token，其中命中 4,480 个，未命中 817 个。第二轮输入 1,443 个、命中 1,392 个，只需处理 51 个未缓存 token；但后续 decode 仍然要访问其上下文状态。如果每轮都重新计算全部输入，prefill 的矩阵运算共需 76.196 TFLOPs；复用已命中的前缀后，共需 11.976 TFLOPs。因此，缓存命中既减少了旧输入的重算，也要求系统继续保留旧状态。[7]

修改上下文会改变可复用的前缀长度。末尾追加新内容时，原有前缀保持不变；摘要替换一段旧上下文时，替换位置之后的表示要从新的前缀重新计算；工具定义或模板变化则可能更早改变输入。这些操作决定下一轮有多少 token 命中、多少 token 重新进入 prefill。

上下文策略还会改变未来负载。把新内容持续追加在上下文末尾，有利于复用原有前缀，却会让陈旧内容继续占据上下文；替换前面的内容可以缩短上下文，但其后的缓存可能需要重建。保留、淘汰和重算的代价取决于重建计算量、状态大小与复用间隔（第 8 章）。[8]

用本节的前缀复用关系比较两种编辑方式：在末尾追加 100 个 token，只增加 100 行新输入；若改动已有 2000 个 token 中的第 501 个，其后的表示都会变化，最长可直接复用的前缀只剩 500 个 token。缓存收益不仅取决于改了多少字，还取决于改动发生在哪里。

从应用看，上下文是编排任务的接口；从系统看，上下文的组织方式决定了计算量和状态驻留时间。追加工具返回的内容、替换早期内容、生成多个候选分支，分别改变新增计算、可复用前缀和私有状态。让服务系统掌握这些关系，就能根据复用情况安排缓存，在分支间共享历史，并利用工具等待时间换出状态。因此，负载记录还应包含稳定前缀、改写位置、分支依赖和复用间隔。第 8 章将使用这些信息选择上下文与缓存策略，第 9 章再加入状态位置和传输时间。

### 3.2.2 推理与验证：完成一个任务需要多少成本

前缀缓存减少的是已有输入的重复计算。另一类方法则主动增加生成计算，希望提高答案的正确率。增加推理阶段的计算量，主要有三种方式：延长一条回答中的思考过程，同时生成多份回答并从中选择，或者根据题目难度分配生成次数和长度。延长思考会增加串行生成时间与上下文状态；生成多份回答会增加计算和验证次数；按难度分配则需要决定每道题何时停止。

是否值得增加生成计算，需要比较每个成功任务的成本。设每次尝试的平均成本为 $c$、成功率为 $p$，相同策略下重复尝试的期望次数为 $1/p$，则

$$C_{\text{success}} = \frac{c}{p}.$$

新方法的成本和成功率分别为 $c_2, p_2$，旧方法为 $c_1, p_1$，成本下降要求 $p_2/p_1 > c_2/c_1$。因此，延长思考或增加采样的依据，是正确率的相对提升能否超过成本的相对增长。所有生成、验证和失败尝试的成本都计入 $c$。

**例子：延长生成仍未提高数学题解答成功率。**生成速度只有与任务正确率一起考察，才能反映系统是否有用。一次数学题实验中，对四道题分别生成多份回答，将输出上限从 1024 提高到 4096 个 token，两轮尝试仍未得到通过评分的最终答案。生成已经消耗资源，任务却没有完成；增加输出长度也没有自动带来有效解答。[9]

答案检查应区分接口格式和内容正确性。格式正确的回答可能算错，数值正确的回答也可能缺少任务要求的字段。把两类错误分开，才能判断应调整生成方式、输出约束还是解题过程。[10]

[7] 开启思考模式的真实轨迹复算。

[8] 上下文组织与 Agent 案例，连接作者 AI Agent 书第 2 章与实验记录。

[9] 练习 3-3：1024 预算与4096 预算；理论背景见Test-Time Compute。

[10] 关闭思考模式的诊断，分别记录按预定标准评分的结果、事后从输出中提取答案的结果，以及并发进程的观测数据。

核算成本时，还要考虑价格本身的变化。token 单价降低后，仍需计算完成任务的总成本。用简单假设说明：新系统 token 单价降到原来的 1/10，每任务用量却增加到 20 倍，成功率由 50% 提高到 80%。只计模型成本，并假定重复尝试的分布稳定，每个成功任务的成本比为 $2 \times \frac{0.5}{0.8} = 1.25$。价格下降、质量提高和成功任务成本上升可以同时发生。对带上下文的 Agent，可以沿实际轨迹累加各次尝试，再以通过检查的任务数作分母。[11]

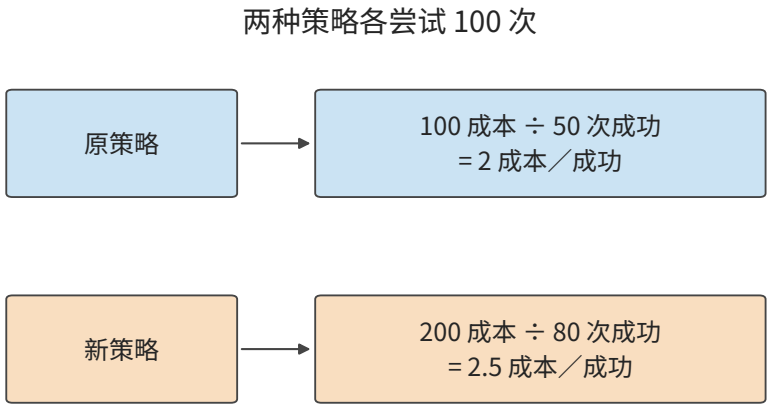
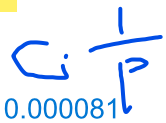



图 3-9　各尝试 100 次的教学比较。所有尝试的成本都进入分子，通过检查的成功任务数进入分母；两种策略的成功任务成本分别为 2 和 2.5 单位。

> **练习 3-3〔延伸〕：成功率提高能否抵消单次尝试成本的增加**
> 旧方法每次尝试成本为 1 单位、成功率为 50%；新方法成本为 2 单位、成功率为 80%。计算平均每完成一个成功任务所需的成本，并判断新方法是否更省。若新方法成本降到 1.5 单位，成功率至少应达到多少，才能使每个成功任务的平均成本不高于旧方法？对需要多次采样的任务，说明生成、验证和失败尝试的开销应怎样计入总成本。
> 沿用第 3.2.4 节末的决策请求算例：决策模型处理一条请求花费 0.000081 美元，转交 LLM 每条花费 0.013880 美元。设转交前后的成功率分别为 $p_1, p_2$，求转交比例为 1% 和 5% 时每条请求的平均费用；$p_2/p_1$ 至少达到多少，转交才能降低每个成功任务的费用？

### 3.2.3 智能体工作负载







反馈还可以改变后续的计算过程。代码 Agent 先生成修改方案，工具写入代码并运行测试，模型再读取测试结果决定下一步。每个循环既有模型计算，也有工具执行；工具返回的日志又成为下一轮输入。因此，减少无关日志能降低下一轮的 prefill 计算量和上下文状态占用，减少无效修改则能直接减少循环次数。

本章的实测任务是修复区间合并函数，检查返回结果是否正确、输入是否保持不变。模型负责决定修改并调用工具，工具负责写入文件和运行测试。下面按四轮内成功完成原定检查的那条轨迹，分开统计模型时间、工具时间和等待期间保存的状态。工具调用明细见章末代码任务轨迹。[12]

检索增强问答也包含类似的处理过程：检索器找到文档，再将文档交给语言模型生成答案。回答质量相同时，检索内容越少，后续 prefill、KV 存储和上下文读取的开销就越小。因此，检索器返回的内容会直接影响模型负载。第 8 章将结合具体任务比较不同检索方案。[13]

[11] 2023—2026 年 token 成本调研，§2、§4、§11；API 价格、生产成本与成功任务成本分开。

[12] 练习 3-4：两条代码 Agent 轨迹，含独立检查与额外别名条件。

[13] 检索与生成算例。

### 3.2.4 分支、工具等待与状态驻留

模型调用与工具执行之间的先后关系可以画成一张依赖图。沿一条串行路径，阶段时间相加；互不依赖的分支可以同时开始，汇合点等待较慢的分支。一张依赖图中，从开始到结束累计耗时最长的路径称为关键路径，它决定任务最早何时能够完成。先用真实代码任务观察模型和工具各占多少时间，再改变依赖关系，预测并行能节省多少。下表比较同一任务在思考模式关闭与开启时的两次运行；思考模式决定模型是否先生成第 3.1.2 节所说的思考 token。

| 观察 | 思考模式关闭 | 思考模式开启 |
|---|---|---|
| 模型调用轮数 | 12 | 4 |
| 总输出 token | 765 | 2,733 |
| 模型调用实际耗时之和 | 13.156 s | 76.294 s |
| 工具执行实际耗时之和 | 0.476 s | 0.078 s |
| 返回值正确性与输入不变检查 | 315/1013 | 1013/1013 |
| 额外检查：返回的子列表无别名 | 1/1013 | 710/1013 |

关闭思考模式时，缓存命中率约为 83.4%，仍未修好代码；开启后，模型时间增加，返回值正确性和调用后输入不变两项原定检查全部通过。额外的无别名检查要求返回子列表与原输入相互独立，开启思考模式的程序通过了其中 710 个用例，另 303 个仍共享了子列表。[14] 两组检查对应两种不同的程序性质，这说明任务的完成标准必须具体到可执行的验证。

开启思考模式后，任务共输出 2,733 个 token，其中 2,553 个在思考结束符（标志思考内容结束的记号）之前，3 个为结束符，177 个在之后。思考占据绝大多数生成工作，工具调用则把模型的决策转化为文件修改与测试操作。沿执行顺序累计模型调用与工具执行的实际耗时，再加上控制和交接时间，就得到用户等待整个任务完成的时间。

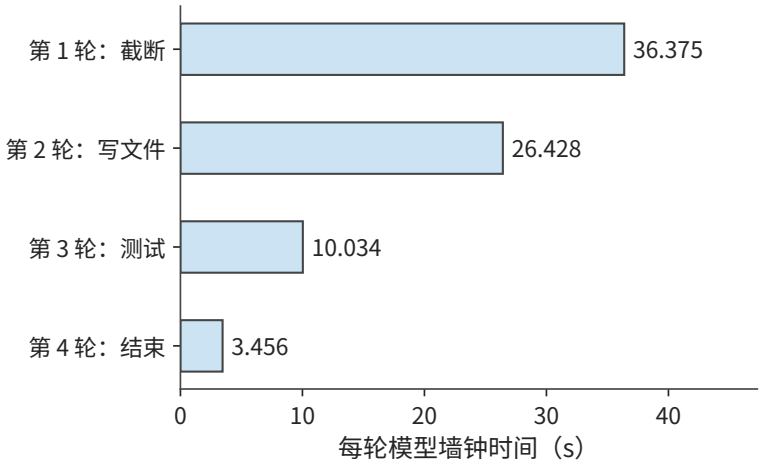



图 3-10　代码任务中四轮模型调用的实际耗时，各轮从自己的起点计时。模型时间合计 76.294 秒，工具合计约 0.078 秒，整任务另含控制与交接时间。

**例题 3-2：加速首轮模型计算能缩短多少代码任务时间？**用第 1 章的 Amdahl 定律分析，假设后续行为与质量不变。

解：以同一条执行轨迹为基准，将首轮耗时 36.375 秒的模型计算缩短一半，其余阶段、输出、工具行为和

[14]练习 3-4：两条代码 Agent 轨迹，含独立检查与额外别名条件。

质量都不变，总时间便从 76.510 秒降到 58.323 秒。此时 $f = 36.375/76.510$，$s = 2$，总加速比约为 1.31。首轮占总时间约 47.5%，将这一段减半节省约 23.8% 的总时间；其余阶段仍合计耗时约 40.1 秒。优化的收益取决于缩短的那一段原来占多大比例。[15]

工具之间的依赖关系也影响总时间。设模型先计算 2 秒，随后调用两个工具，耗时分别为 6 秒和 10 秒，最后再计算 3 秒。若第二个工具依赖第一个的结果，总时间为 $2 + 6 + 10 + 3 = 21$ 秒；若工具相互独立，总时间为 $2 + \max(6, 10) + 3 = 15$ 秒。并行减少了 6 秒，但两个工具的总执行工作仍为 16 秒。

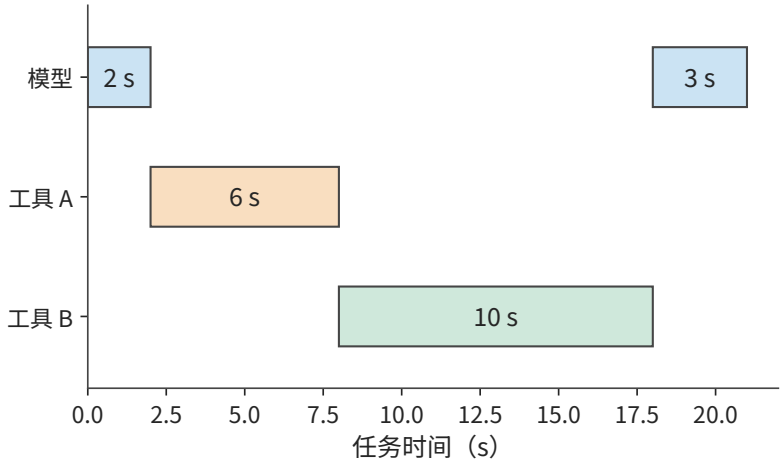



图 3-11　两个工具存在前后依赖时的任务时间线。模型先运行 2 秒，工具 A 用 6 秒，工具 B 用 10 秒，模型最后运行 3 秒，总计 21 秒。

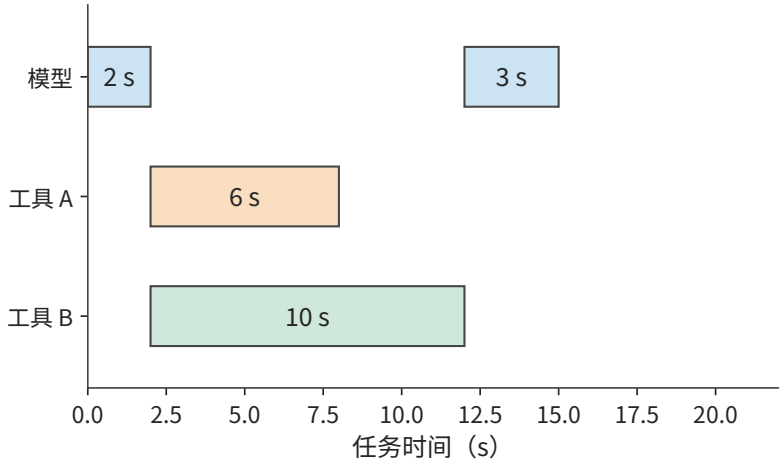



图 3-12　两个工具独立时可以同时开始，模型在较慢的工具 B 完成后继续，任务共 15 秒。与前图使用相同时间尺度；工具工作总量仍为 16 秒。

并行与分支改变的不只是时间，还有状态占用。若分支还要分别生成文本，公共前缀可以共享，分支尾部独立增长。设共享前缀长 $H_0$，第 $i$ 个分支新增 $h_i$ 个 token，每 token 状态为 $c_{\mathrm{KV}}$，则总状态为 $c_{\mathrm{KV}}(H_0 + \sum_i h_i)$。并行缩短了关键路径，也让更多分支的状态同时占用内存。

第 3.2.3 节代码修复任务的实测记录中，工具调用耗时极短，大部分时间都消耗在模型计算上。若工具操作改为耗时 10 秒的编译，模型暂停的同时，下一轮所需上下文仍占据容量；若多个任务同时等待，驻留量按任务累加。工具执行时间越长，状态保留得越久，系统能同时处理的请求数就越少。第 8、9 章将在这条时间线上安

[15] 仅将首轮模型段加速两倍。

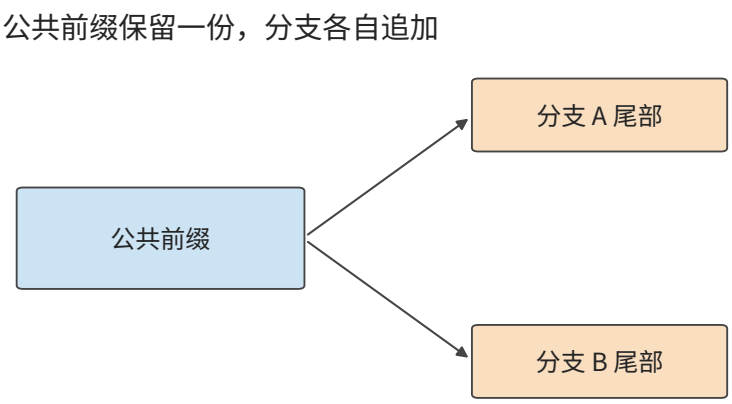



图 3-13　两个生成分支指向同一份公共前缀，并各自保存新增尾部。箭头表示引用关系，共享前缀只计一份容量。

排缓存与交接。

> **练习 3-4〔核心〕：工具并行与局部加速能节省多少任务时间**
> 一次任务先由模型计算 2 秒，然后调用两个相互独立的工具，分别耗时 6 秒和 10 秒。两个工具都完成后，模型再计算 3 秒，任务结束。分别计算两个工具依次执行和同时执行时的任务总耗时。假设仅在等待工具返回期间状态占用 1 GiB 空间，求两种执行方式下状态占用量与占用时长的乘积，单位为 GiB · s。
> 本节的代码任务实测记录显示，任务总耗时为 76.510 秒，其中首轮模型计算耗时 36.375 秒，所有工具执行合计耗时 0.078 秒。保持其余阶段不变，分别计算两种优化能节省多少任务时间：将首轮模型计算速度提高到原来的四倍；将所有工具的执行速度提高到原来的两倍。根据计算结果，解释为什么局部加速倍数不能单独决定一项优化对完整任务的收益。

负载需求还会反过来推动模型结构的变化。把同一条会话换成 DeepSeek V4.1 Flash，可以看到具体例子。设 Agent 已有一段公共上下文，工具随后返回新材料；如果前缀未命中，服务要重新处理较多输入，而本轮输出可能较短。为这样的输入密集场景降低 prefill 成本，比只优化单步生成更有价值。V4.1 因而采用第 2 章介绍的 CED 结构，使用 20 层因果编码器和 20 层解码器：大部分输入只执行编码器主体，解码器的全局 KV 从编码器末层表示投影得到；为了构建解码器的 SWA 状态，把提示末尾最多 128 个 token 的编码器输出重新送入解码器，近似恢复这部分局部状态。输出 token 继续经过全部 40 层。[16]

先比较专家矩阵计算这部分工作，以各 token 经过的专家层数之和衡量工作量。设一次从空状态开始的输入为 $P$ 个 token，则普通 40 层路径为 $40P$，报告中的 CED 调度为 $20P + 20\min(P, 128)$。当 $P = 8192$ 时，两者为 327,680 与 166,400，后者约为前者的 50.8%；当 $P \le 128$ 时，该子项没有减少。

已有前缀时，输入处理的工作量由追加长度与恢复方式共同决定。若编码器 SWA 命中缓存，编码器主体便可以继续处理追加的输入；编码器 SWA 缺失，则先重放前缀末尾窗口。两条路径随后都要构建本轮的解码器 SWA。第 8 章将用 8K 与 128K 上下文的教学算例，比较保存状态和重新执行的成本。

**算例：Jev 为什么一眨眼就给出答案。** 2026 年 9 月，前 OpenAI 研究员 Diogo Almeida 创办的 TypeSafe AI 发布了 Jev 模型。按创始人在发布文章中的说法，他在 OpenAI 参与了让语言模型学会遵循指令、与人对话的方法，这些工作后来成了 ChatGPT 背后的研究。[17] Jev 的官方演示相当吸睛：同一条客服请求，GPT-5.6 Terra 用了 8.566 秒，Jev 只用 0.114 秒，费用也只有前者的 1/171。不少人的第一反应是：这还是 Transformer

[16] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

[17] TypeSafe AI 首页的定价与并排演示；发布博客含创始人自述，并说明并行采样、RLCD 与评测口径；接口文档说明问题在同一状态上并行求值。




图 3-14　同一批 8K 输入中，各 token 经过的专家层数之和。普通全层路径执行 40 层；CED 路径执行 20 层编码器，并为最近 128 个 token 重放 20 层解码器。灰色说明项仍需另外计算；生成阶段执行完整主干。

吗？是不是用了什么了不起的 infra？读过本书就会发现，这其实没有什么秘密，读者也可以轻松实现。

Jev 不生成文字。调用方提交一段状态（例如一张工单）和若干问题，每个问题的选项事先给定，模型返回各选项的概率。回到第 3.1.1 节的调用链，这类请求只有 prefill，没有 decode。状态和所有问题拼在一起做一次前向，每个问题末位置的输出分布就是答案，好比图 3-13 中从同一前缀分出的几条分支，每条只走一步就停下。问题之间互不依赖，请求结束后也不必保留 KV。至于这些概率能否直接当作置信度，靠的是训练：TypeSafe 称其方法为 RLCD，思路与学术界的 RLCR 相同，都在奖励中加入 Brier 分数，让模型给出的概率与实际正确率相符。[18]

下面用本书的条件验证这一点。Jev 只按输入收费，每百万 token 0.042 美元；演示中 Jev 花费 0.000081 美元，折算下来输入约 1929 个 token。把这条请求放到 Qwen3-8B 上，一次前向约 28 TFLOPs，H100 SXM 按 40% 的算力效率计算约需 70 ms，与演示中的 0.114 秒相当。换成 DeepSeek V4.1 Flash 并走 CED 路径，运算量也只多出约四分之一。如果交给普通 LLM，prefill 之后还要逐个生成 token，每一步受内存带宽限制，约需 9 ms。生成 64 个 token，延迟就增至 0.65 秒，是决策请求的 9 倍；由于 decode 可以合批，GPU 时间却只多 28%。图 3-15 把两种执行方式画在同一条时间轴上：一眨眼给出答案，靠的是省掉了串行生成。[19]

价格同样不神秘。按 DeepSeek 推理系统报告的口径，每 GPU 小时租金取 2 美元，则 8B 稠密模型或 V4.1 Flash 每处理一百万个输入 token，算力成本约为 0.02 至 0.025 美元，Jev 的价格是这一成本的 1.7 至 2 倍。也就是说，这一档模型只靠普通的 prefill 就能做到这个价格；反过来，若价格要覆盖成本，Jev 每个 token 的运算量大致不会超过 8B 稠密模型的两倍。演示中 171 倍的费用差距，只有一小部分来自省去生成：在同一模型上，省去 64 至 512 个输出 token，GPU 时间只减少到原来的 1/1.3 至 1/3.4。其余差距来自前沿模型本身的价格，以及推理时生成的大量 token。

实际使用时，决策模型没有把握的请求还要转交 LLM 重新处理。设转交比例为 $e$，每条请求的平均费用为

[18] Beyond Binary Rewards: Training LMs to Reason About Their Uncertainty，§3 的 RLCR 奖励。

[19] 决策请求的执行路径与定价比较，含 V4.1 Flash 全层路径、512 个输出 token 的 LLM 路径与 DeepSeek 价目对照，可用 `python3 calculations/calc.py decision-request --format md` 复算；DeepSeek 推理系统概述的每 GPU 小时 2 美元口径；DeepSeek 模型与价格。

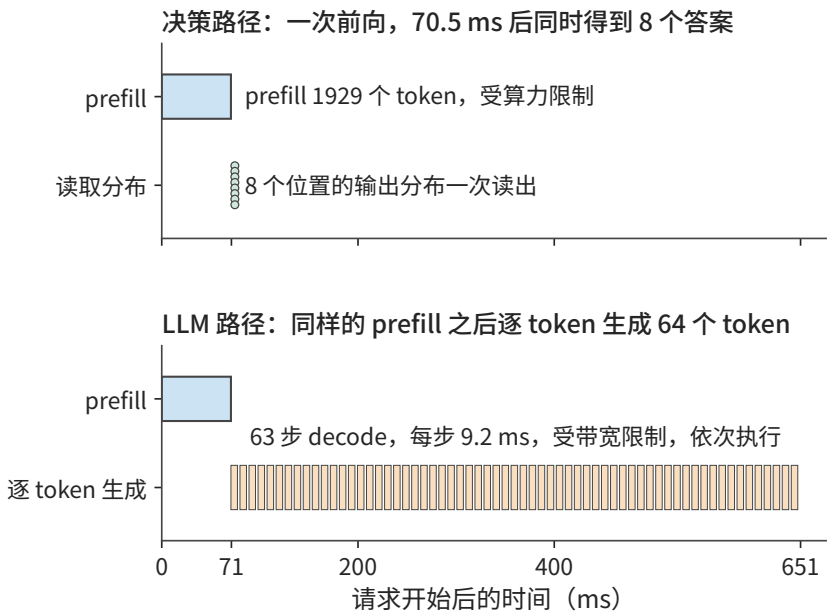



图 3-15　同一条 1929 个 token 的请求在 H100 SXM 上的两种执行方式。上图为决策请求，一次前向后同时读出 8 个问题的答案；下图为普通 LLM，prefill 之后逐个生成 64 个 token。模型为 Qwen3-8B，算力效率取 40%，带宽效率取 50%。

$c_{\text{dec}} + e\,c_{\text{LLM}}$。按演示中的两个费用计算，$e$ 只要超过约 $0.58\%$，转交 LLM 的费用就会超过决策模型本身的费用。因此，第 3.2.2 节成功任务成本中的 $c$ 也要计入转交费用。决策模型是否更省钱，要看有多少请求需要转交，以及接手的 LLM 价格多高。

## 3.3 多模态与实时交互

Agent 的工具结果通常以文本进入下一轮输入。若输入换成图片或连续语音，调用模型之前还需要编码输入，输出端也可能需要持续播放。本节先分析数据在阶段之间如何改变形状，再分析各阶段必须在何时完成。

### 3.3.1 视觉编码、语言生成与状态

图片进入语言模型之前，还要经过预处理、视觉编码和投影。用 E 表示视觉编码，P 表示语言 prefill，D 表示后续 decode，一次视觉问答便有 E → P → D 这条基本依赖。图片大小影响 E 的工作量，编码后的视觉 token 数影响语言部分的工作量；这两项工作应分别计算。

先数视觉 token，再数每个向量的特征维度。设预处理后的图片高、宽为 $H_{\text{img}}, W_{\text{img}}$，将图片分成方形图像块（patch），每块边长为 $p_{\text{patch}}$，每边再合并 $r$ 个 patch，则视觉 token 数为

$$n_v = \frac{H_{\text{img}} W_{\text{img}}}{p_{\text{patch}}^2 r^2}.$$

以具体配置为例。视觉语言模型 Qwen3-VL-4B 的 $640 \times 640$ 图片，按 $16 \times 16$ patch 划分得到 1600 个 patch，再作 $2 \times 2$ 合并，得到 400 个视觉 token。最终特征宽度为 2560，每元素占两字节的 BF16 张量 $[400, 2560]$ 占 $400 \times 2560 \times 2 = 2{,}048{,}000$ 字节。

但编码结果不止这一份。为向语言模型提供不同深度的视觉信息，DeepStack 将视觉编码器中间层的特征送入相应语言层。模型还取三个中间视觉层的这类特征，将它们与最终层特征拼接，完整张量为 $[400, 10240]$，占 8,192,000 bytes，即 7.8125 MiB。四组特征对应同一组 400 个视觉 token：特征宽度变为四倍，语言序列仍增加

400 个视觉 token。视觉编码的矩阵运算量约为 1.310 TFLOPs；图片解码与缩放发生在编码之前，语言计算则在编码之后。[20]

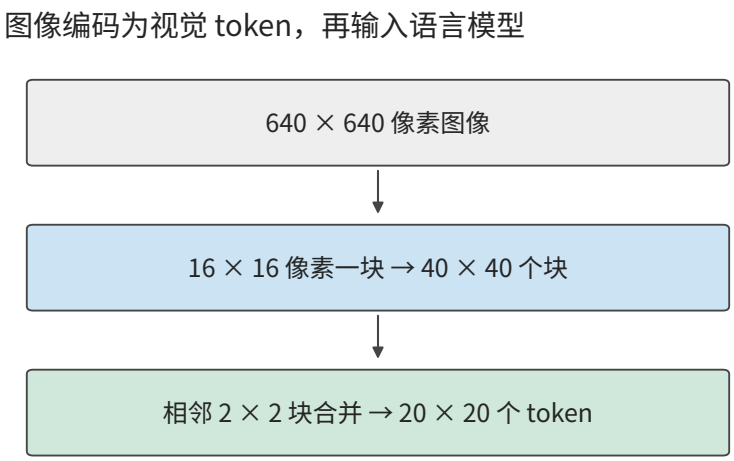



图 3-16　图像从像素网格变为视觉 token。640×640 的方形图片切成 40×40 个块，相邻 2×2 块合并成一个 token，形成 20×20、共 400 个视觉 token。

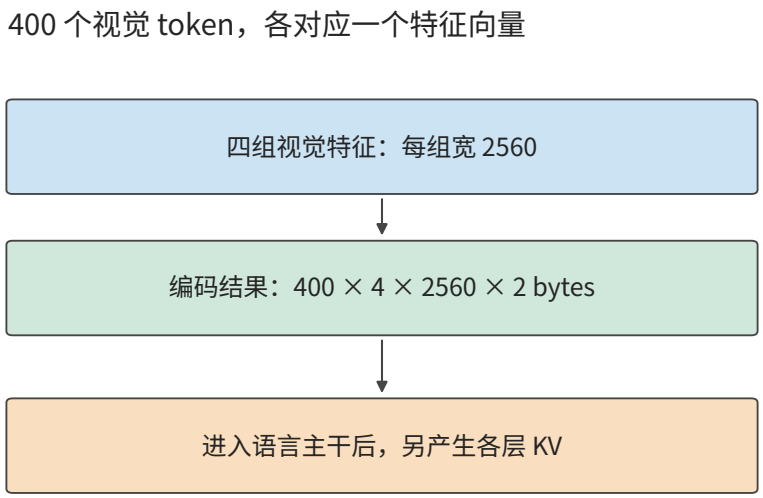



图 3-17　视觉 token 数与每 token 特征宽度分别计量。四组 2560 维 BF16 编码特征占 7.8125 MiB；这些视觉 token 进入语言模型后，另产生各层的 KV 状态。

图 3-16 与图 3-17 中，视觉编码改变了视觉 token 数和特征宽度；进入语言模型后，还会产生另一种表示。编码缓存（encoder cache，EC）保存视觉编码器的输出，语言主干则产生 KV 缓存。该配置的语言 KV 每 token 占 144 KiB，400 个视觉 token 对应 56.25 MiB。因此，同一张教学截图在三个处理阶段对应不同的数据量：原始压缩文件 0.8 MB、完整 BF16 EC 7.8125 MiB、视觉 token 的逻辑 KV 56.25 MiB。[21]

[20] 视觉编码矩阵与张量计数。

[21] 多模态输入的字节、状态与阶段放置，固定 Qwen3-VL-4B 配置、预处理与完整 DeepStack EC。

| 对象 | 本例大小 | 决定它能否复用的条件 |
|---|---|---|
| 压缩截图 | 0.8 MB，给定输入 | 同一文件内容 |
| 完整编码结果 EC | 7.8125 MiB | 图片、预处理、编码器与输出格式一致 |
| 视觉 token 的语言 KV | 56.25 MiB | 还依赖先前上下文、顺序、位置与模型状态 |

图片及编码配置相同时，EC 可以复用。语言 KV 还依赖图片之前的上下文：问题放在图片之前时，更换问题会改变视觉 token 的 KV；问题放在图片之后时，因果注意力使此前视觉 token 的 KV 保持不变。

**多模态输入的编码状态与 KV 容量预算。**设输入包含四张图片与 400 个文本 token，计算这些输入 token 的 EC 与逻辑 KV。四张相同规格图片的 EC 为 31.25 MiB，视觉 token KV 为 225 MiB；再加 400 个文本 token，输入逻辑 KV 为 281.25 MiB。

原图、EC 和 KV 这三种数据表示，对应三个不同的交接位置。发送原图，接收方还要执行视觉编码；发送 EC，接收方从语言 prefill 开始；发送 KV，则交接已经完成的语言上下文。第 8 章安排同卡执行，第 9 章比较阶段交接，第 12 章把相同数据流放到端、边、云链路上。

保持 patch 与合并方式不变，把图片的高度和宽度都加倍，视觉 token 数变为四倍。因此 EC 和视觉 token 的语言 KV 都变为四倍；视觉编码内部 token 之间的交互则还要按注意力结构计算。特征拼接增加的是宽度，图片变大增加的是视觉 token 数，二者在语言主干中产生不同后果。

### 3.3.2 音频与图像生成的计算过程

视觉问答仍通过语言模型逐 token 生成文本。若输出本身是声音或图像，一次迭代处理的数据和迭代次数也会改变。

音频输出还有帧内循环。以 Fish Audio 的语音生成模型 S2 Pro 为例，慢速（slow）路径每次推进一个音频帧，快速（fast）路径在帧内补齐多个码本的编号。码本是离散声音表示的候选向量集合，编解码器（codec）把这些编号还原为波形。每帧先用一次快速路径前向建立状态，再用九次预测补齐码本，因此生成一帧需要十次快速路径前向计算。由此可见，语言 token/s、声学帧/s 与每秒生成的音频时长，衡量的是不同的生成速度。[22][23]

图像生成又换了一种循环。潜在表示（latent）是压缩后的连续图像表示；去噪逐步把含噪表示变为目标图像。扩散 Transformer（DiT）执行每步变换，无分类器引导（CFG）组合带条件与无条件两路预测；变分自编码器（VAE）负责像素与压缩表示之间的转换。以图像生成模型 Qwen-Image-2512 为例，$1024 \times 1024$ 图像形成 4096 个 latent 位置，执行 50 个去噪步，true CFG 路径每步有两支 DiT 前向，总共执行 100 次 DiT 前向计算。去噪矩阵运算量约 $7.830 \times 10^{15}$ FLOPs；文本编码器与 VAE 还有各自的工作。latent 的位置集合在去噪期间保持固定，各步持续更新其内容，累计工作随去噪步数增加。[24]

把每步去噪的运算量记为 $F_{\text{step}}(n_v)$，去噪步数为 $n_s$，每步引导分支数为 $n_b$，总运算量为 $n_s n_b F_{\text{step}}(n_v)$。步数加倍，运算量也加倍；分辨率提高会增加图像潜在表示中的空间位置数，单步内部的矩阵行数和位置间交互也随之增加。文本生成不断追加上下文，每步访问量随之增长；图像去噪反复更新同一组 latent，在固定分辨率下，累计运算量随去噪步数线性增长。

[22] 多模态计算资料。

[23] 生成模型取材与执行路径；Omni 音频、Fish 音频、FLUX 图像、H3 视频、Wan 视频。

[24] Qwen 图像生成请求计数。

### 3.3.3 连续感知、首段响应与打断

工作总量决定平均处理需求，实时播放还要求每块数据都按时到达。令 $A(t)$ 表示截至时刻 $t$ 已到达的可播放音频时长，$P(t)$ 表示已播放的音频时长，缓冲中剩余音频为

$$B_{\text{audio}}(t) = A(t) - P(t).$$

连续播放期间，每经过一秒，已播放音频时长 $P(t)$ 就增加一秒，$A(t)$ 则随数据块到达跳跃增加。两条曲线之间的距离就是缓冲余量；余量降到零而下一块尚未到达，声音就会中断。预缓冲增加初始余量，同时推迟开始播放。

时间要求同时作用于输入和输出。在输入端，截图 Agent 如果只在动作结束后观察屏幕，就可能漏掉中途出现又消失的弹窗。持续观察与交互研究 AOI 针对的正是这类场景：它把屏幕观测与动作执行分开，持续收集图像、音频和事件，再筛选出供模型使用的记录。增加观测频率可以发现更多短暂事件，但保存更多关键帧也会占用上下文；过多关键帧还可能挤占重要信息，使任务效果下降。[25] 因此，持续观测还要配合筛选，决定哪些内容值得保留并交给模型处理。

语音的时间约束更直接。用户停止说话后，系统要采集或确认输入，经过识别 / 编码、推理、语音生成和播放。开始播放后，后续音频还必须及时到达，才能避免中断。

音频块到达应用之后，还要经过解码、排队和播放。两次语音交互测量中，用户停止说话到首块音频到达的时间分别约为 400 ms、370 ms，后续音频块的到达间隔中位数均约为 94 ms。[26] 到达时间说明数据何时可用，播放时钟则决定音频何时播出。

用下面这组参数安排一条时间线，说明两者之间的缓冲如何避免声音中断。输入每 20 ms 形成一块，24 kHz、单声道、每样本 2 bytes，每块数据量为：

$$24{,}000 \times 0.020 \times 1 \times 2 = 960 \text{ bytes}.$$

每块采集结束后，模型处理耗时 12 ms、发送耗时 1 ms，网络传播通常再需 5 ms。第一块在 38 ms 到达，在抖动缓冲区（先暂存到达的音频、吸收到达时间波动的缓冲区）中等待 40 ms 后，于采集开始后的第 78 ms 播放。第三块的传播延迟设为 50 ms，直到 123 ms 才到，错过 118 ms 的播放计划，造成额外 5 ms 的停顿。后续块虽已到达，也要按声音顺序等待。

将缓冲改为 60 ms，这次停顿消失，首播却推迟到 98 ms。增加缓冲可以避免这次到达时间波动造成的播放中断，代价是更晚开始；若音频的平均到达速度长期低于播放速度，有限缓冲终究会耗尽。[27]

播放连续性之外，还要单独检查打断响应。图 3-20 从用户发出打断计时，追踪本地播放何时真正停止；远端生成是否停止需要沿另一条控制路径判断。

打断涉及几项操作：应用发出取消指令，播放设备静音，后端停止生成，并释放缓冲与状态。教学时序在 123 ms 发出打断，控制延迟 5 ms，设备每 10 ms 检查并执行一次控制指令，于是 130 ms 静音，用户感受到的打断延迟为 7 ms。设备静音、后端停止生成和释放缓冲与状态，分别对应停止播放、停止计算和释放存储三个

[25] AOI 论文与关键帧案例。

[26] 两份历史语音记录。

[27] 40 ms 缓冲教学时序与60 ms 缓冲。

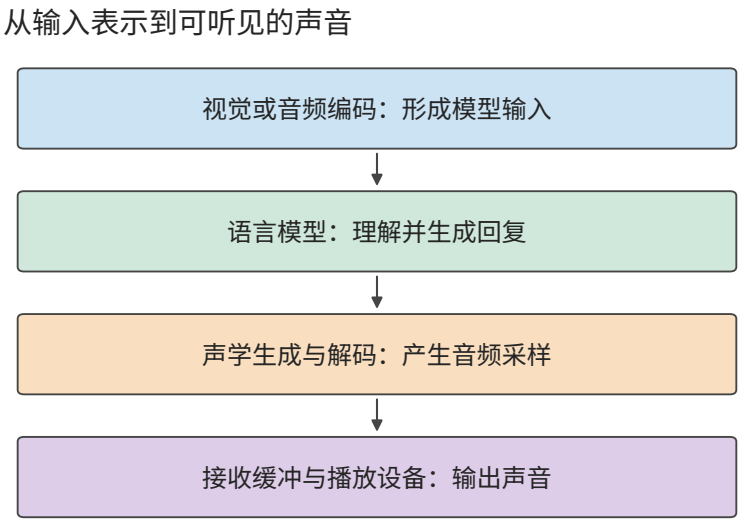



图 3-18　可组合的多模态阶段。编码形成模型输入，语言模型生成回复，声学模块将回复转成音频，接收方的缓冲与播放设备决定何时真正发声。

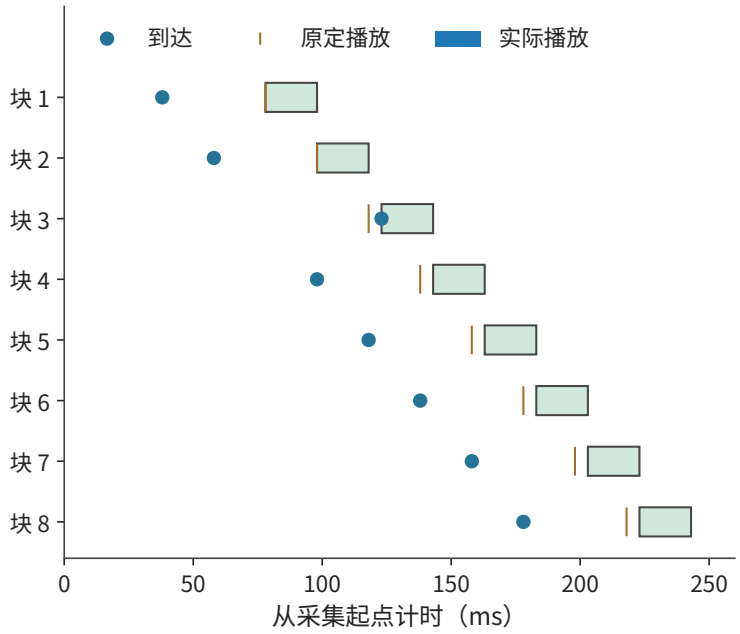



图 3-19　八块音频的教学播放时间线。每块长 20 ms，圆点标到达，短竖线标原定播放时刻，色条标实际播放；第三块晚到 5 ms，后续播放随之顺延。

时刻：后端停止生成决定后续计算何时结束，缓冲释放决定容量何时可供其他任务使用。[28]

并非所有任务都要求连续输出。RAW 是保留相机传感器原始采样信息的图像表示。RAW 图片精修关注的是何时能得到完整成片，无需像语音那样连续输出。精修还需要保留原始图像信息，视觉理解使用的 EC 无法替代原图。由此可见，任务不同，时间要求也不同：有的要尽快开始，有的要持续流畅，有的要尽早全部完成。

[28]打断与静音的教学算例。

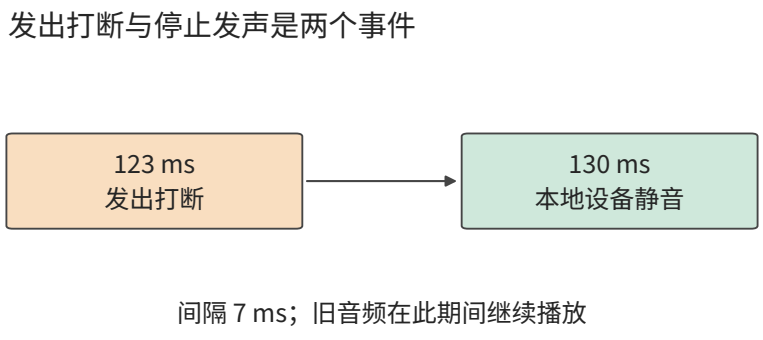



图 3-20　同一教学场景中的本地打断。123 ms 发出操作，130 ms 设备静音；远端计算是否停止属于另一条控制路径。

**练习 3-5〔延伸〕：图像分辨率与音频缓冲如何影响资源需求**

将 Qwen3-VL-4B 输入图片的分辨率从 $640 \times 640$ 改为 $1280 \times 1280$，保持图像块大小、合并方式和特征精度不变，求视觉 token 数、完整编码缓存 EC 的大小，以及这些视觉 token 对应的 KV 大小。再根据本节音频时间线，比较初始缓冲为 40 ms 和 60 ms 时的首次播放时刻与停顿情况。如果每秒收到的音频只能播放 0.9 秒，开始播放时已缓存 0.3 秒音频，求连续播放多久后会耗尽缓冲，并解释增加有限缓冲能解决什么问题。

## 3.4 训练的计算与状态需求

### 3.4.1 前向计算、反向传播与参数更新

推理用权重求输出，训练还要利用输出与目标的差异来调整权重。以最简单的一次乘法为例：输入 x 乘以权重 w 得到 y。后一层传回损失对 y 的敏感度后，要算损失对 w 的敏感度，就还需要当时的 x；要把敏感度传给前一步，则需要 w。因此，前向把结果交给下一层以后，某些输入还要保留，等反向用完才能释放。

对线性层 $Y = XW$，输入 $X$ 为 $[m, k]$，权重 $W$ 为 $[k, n]$。损失 $\mathcal{L}$ 是衡量模型输出与训练目标差异的数值，训练的目标是降低损失；梯度描述损失对某个输入或参数的小变化有多敏感。反向传播从输出端沿依赖逆向传递这些敏感度。设后一层反向传来 $\partial \mathcal{L}/\partial Y$，当前层需要计算两种梯度：

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Y} W^{\mathsf{T}}, \qquad \frac{\partial \mathcal{L}}{\partial W} = X^{\mathsf{T}} \frac{\partial \mathcal{L}}{\partial Y}.$$

前向计算输出，反向分别计算输入梯度和权重梯度。三次矩阵乘法的运算量相同，因此这一线性层的训练矩阵运算量约为前向的三倍。下表将一次乘加计为两个 FLOPs。

| 操作 | 要解决的问题 | 左、右输入的形状 | 输出形状 | 矩阵 FLOPs |
|---|---|---|---|---|
| 前向 $Y = XW$ | 根据输入求输出 | $[m, k]$ 与 $[k, n]$ | $[m, n]$ | $2mkn$ |
| 输入梯度 $\mathrm{d}X = \mathrm{d}Y W^{\mathsf{T}}$ | 将误差传回前一层 | $[m, n]$ 与 $[n, k]$ | $[m, k]$ | $2mkn$ |
| 权重梯度 $\mathrm{d}W = X^{\mathsf{T}}\mathrm{d}Y$ | 求这层权重的调整方向 | $[k, m]$ 与 $[m, n]$ | $[k, n]$ | $2mkn$ |

输入梯度把误差信号传给前一层，权重梯度决定本层参数的调整。观察三组矩阵尺寸：虽然乘法顺序不同，三次运算都含有相同的 $mkn$ 个乘加，因此合计约为 $6mkn$ FLOPs。若所有训练 token 都经过同一组主要参数矩阵，累计便得到常见近似 $F_{\text{train}} \approx 6ND$，其中 $N$ 为参数量，$D$ 为训练 token 总数。[29]

[29] 训练计算量笔记，线性层前反向、序列分布与 DeepSeek V4-Flash 专家分项计算。

结构变化也可以按这一推导处理。冻结某个权重矩阵后，可省去它的权重梯度计算；若更早的层仍需要梯度，则仍要计算输入梯度。MoE 按选中专家累计两类梯度，注意力则按查询—键配对累计。沿反向依赖逐项累加，就得到各训练方法需要执行的计算量。

除计算量外，训练还带来新的状态保存需求。除了计算 $\mathrm{d}W$ 要用的 $X$，对非线性运算求梯度也需要相应的中间结果。参数梯度在反向传播中产生，再由优化器用来计算参数更新量。优化器的状态记录多次更新积累的信息，因此需要保留到下一次更新。每类数据的保留期限取决于最后一次使用它的操作：某层激活通常在该层反向计算用完后释放，优化器动量则需要跨越多次更新持续保留。

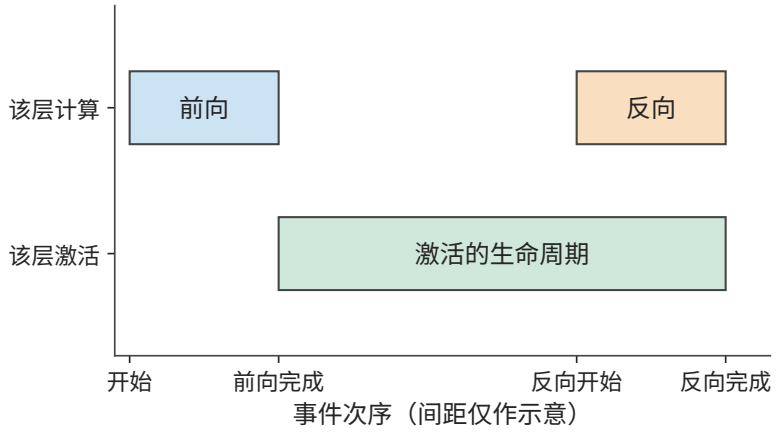



图 3-21　激活的生命周期：一层激活从前向完成后保留到相应反向用完。横轴按事件排列，间距仅表示过程顺序；相应反向计算用完这些激活后，即可释放其占用的空间。

**例题 3-3：梯度与优化器状态为何使训练内存需求成倍增加？**

解：以 Qwen3-8B 的 8,190,735,360 个参数为例，保存 BF16 权重、FP32 梯度、一份独立的 FP32 主权重（master weight），以及 Adam 的两个 FP32 矩估计。Adam 是根据梯度及梯度平方的移动平均调节更新幅度的优化器，两个矩估计分别保存这两种平均量。不做分片（把这些状态切开，分给多张卡保存）时，每参数需要 $2+4+4+4+4=18$ bytes，共 147.433 GB，约 137.308 GiB。尚未计入激活，这些状态就已超过一张 H100 SXM（80 GB）、RTX PRO 6000 Blackwell Workstation Edition（96 GB）乃至 H200（141 GB）的标称显存。BF16 权重用于前向和反向计算，梯度用于计算更新量，FP32 主权重用于高精度更新，Adam 的两个矩估计则累计梯度及其平方的统计量。这些数据合计占用的空间，是推理 BF16 权重的九倍。[30]

明确状态需求之后，再分析数据如何分批进入训练。micro-batch（微批次）是一次前向和反向实际处理的数据子集；完整的训练 batch 可以由多个 micro-batch 组成。micro-batch 与参数更新也要区分。显存无法容纳完整的训练 batch 时，可以分多个 micro-batch 累计梯度，最后再更新一次参数。保持目标、有效标签和归一化一致时，将同一训练 batch 拆成更多 micro-batch，会改变执行顺序与激活的生命周期，而总训练数据量保持不变。第 10 章将据此讨论分片和流水。

训练过程中还要不断准备下一批输入。CPU 读取、解码、分词并组成 batch，再把张量送到加速器。双缓冲可以在计算当前 batch 时准备下一个 batch：一个缓冲供计算读取，另一个接收新数据，交替使用。计算用完已准备的数据后，就必须等待新输入；取回加速器结果时又会等待相应计算完成。第 5 章将结合前向、反向和参数更新的执行顺序，分析数据准备与计算如何重叠。

[30] Qwen3-8B 的 8K 训练矩阵。

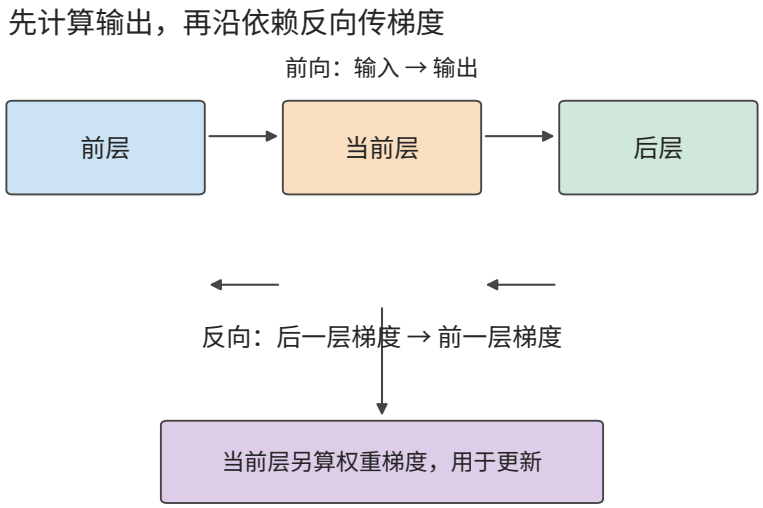



图 3-22　训练沿前向依赖计算输出，再沿反向依赖传递梯度。当前层既向前层传输入梯度，也计算自己的权重梯度，供优化器更新。

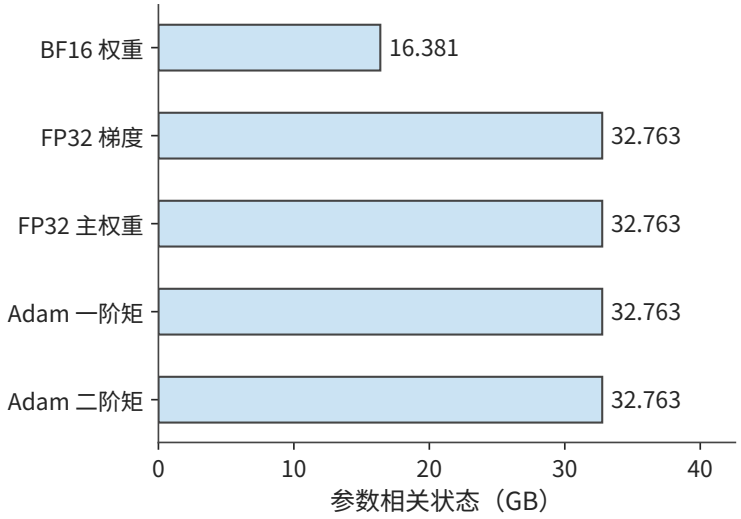



图 3-23　Qwen3-8B 全参数训练的参数相关状态。每参数包括 2 字节计算权重和 4 组 4 字节状态，共 18 字节；激活和工作区的容量需求按各自的生命周期单独计算。

### 3.4.2 预训练、中期训练与 SFT

训练阶段的名称不同，系统仍要回答同样的问题：输入有多少 token，哪些 token 参与损失，多少个 micro-batch 后更新一次参数。预训练通常对连续文本做下一 token 预测；中期训练调整数据组成或上下文长度；监督微调（SFT）使用给定输入与目标回答继续训练模型，常把指令与回答放在一起输入，只对回答计算损失。估算资源时，要把这些差异落实为输入形状、参与监督的 token 数和更新频率。

Qwen3 的训练过程体现了这些变化：通用阶段使用长度为 4096 的序列训练超过 30 万亿 token，随后增加科学、技术、工程与数学（STEM）、代码、reasoning 与合成数据，再训练约 5 万亿 token；长上下文阶段将长度扩至 32768。[31] 数据组成改变学习内容，序列长度还直接改变每个 token 可以访问多少前文，因此需要分别

[31] Qwen3 技术报告，§3.2 的三阶段预训练及表 21 的后训练分支。

观察。

系统需求随长度改变。假设两组独立文档都含 8192 个 token：一组为 $4096+4096$，另一组为 $7168+1024$。参数投影的行数相同，因果注意力的有效查询—键配对却分别为 16,781,312 和 26,218,496。在 Qwen3-8B 一层上，$QK^{\mathsf{T}}$ 与 $AV$ 前向工作约从 0.275 增到 0.430 TFLOPs，增加约 56%。较长文档中的 token 可以看到更多前文，所以在总行数不变时，上下文交互增加；投影与 FFN 仍处理同样的 8192 个 token 表示。整层运算量的增幅取决于这部分新增交互占原有工作量的比例。[32]

SFT 中，模型读取的 token 与参与损失计算的 token 往往不同。一条样本可以包含系统提示、用户问题、工具结果和回答，但训练只对回答部分计算损失。其余 token 仍要经过主干网络，为回答提供上下文。实现还可以只对参与监督的 token 计算词表投影；如果只在计算损失时屏蔽不参与监督的 token，却没有跳过这些 token 的词表投影，这部分矩阵乘法仍会照常执行。

除了只对部分 token 计算损失，微调还可以只训练部分参数，这时需要保存梯度和优化器状态的参数也相应减少。冻结模块的权重保持不变，但梯度仍需穿过它们传到较早的可训练模块，因此仍须计算相应的输入梯度。沿反向图逐层判断是否求输入梯度、是否求权重梯度，就能把训练状态与计算分别算出。

micro-batch 与更新数则控制工作的重复次数。设每个 micro-batch 有 $B_\mu$ 条长度为 $P$ 的序列，累计 $a$ 个 micro-batch 后更新一次，则每次更新读取 $aB_\mu P$ 个输入 token。若每条只有 $P_{\text{label}}$ 个参与监督的 token，损失归一化使用的有效 token 数为 $aB_\mu P_{\text{label}}$。增大累计次数可以扩大每次更新的 batch，同时让一次更新分多次前反向执行；参数状态一直保留，每个 micro-batch 的激活可以在反向完成后释放。

### 3.4.3 从 $6ND$ 估算到逐项计算训练开销

确定训练数据和有效 token 数之后，就可以累计总运算量。先用线性层“训练约为前向三倍”的关系作粗算，再把注意力、词表头和状态更新逐项补回，便能看出粗算与完整模型之间的差距。

以一次完整训练前向和反向为例：Qwen3-8B，$B=1$，$P=8192$，全参数训练，所有输入 token 执行词表头，没有重计算（为少保存激活，在反向时重新执行部分前向）。逐层累计投影、FFN、$QK^{\mathsf{T}}$、$AV$ 和输出头，矩阵运算量如下。[33]

| 矩阵计算 | TFLOPs |
|---|---|
| 前向矩阵 | 143.789 |
| 反向矩阵 | 287.579 |
| 前向加反向 | 431.368 |
| 其中因果注意力 $QK^{\mathsf{T}}$ 与 $AV$ 前反向 | 59.381 |
| 总参数代入 $6ND$ | 402.591 |

把 402.591 TFLOPs 的粗算修正到 431.368 TFLOPs，需要先说明哪些工作漏算了、哪些多算了。查询与上下文之间的矩阵乘法不对应一份模型参数，却在前反向中贡献 59.381 TFLOPs；另一方面，词嵌入等模块虽然包含参数，但并不对每个 token 执行完整矩阵乘法，不能一律按 $6ND$ 计入。两项修正相抵后，矩阵运算总量比粗算高约 7.15%。这属于第 1.3.4 节所说的第一类差距：粗算漏掉了注意力的二次项，又多算了不参与完整矩阵乘法的参数，要改的是模型本身，与实现无关。只有修正后的下界，才能用来衡量系统的利用率。

将有效标签数减到 4096，如果仍对全部 token 计算词表投影，矩阵运算量就不变。如果先取出参与监督的 token 的隐藏向量，再只对它们执行词表投影，总量则降到 416.074 TFLOPs。主干网络仍处理全部输入，节省

[32] 训练计算量笔记，线性层前反向、序列分布与 DeepSeek V4-Flash 专家分项计算。

[33] Qwen3-8B 的 8K 训练矩阵。

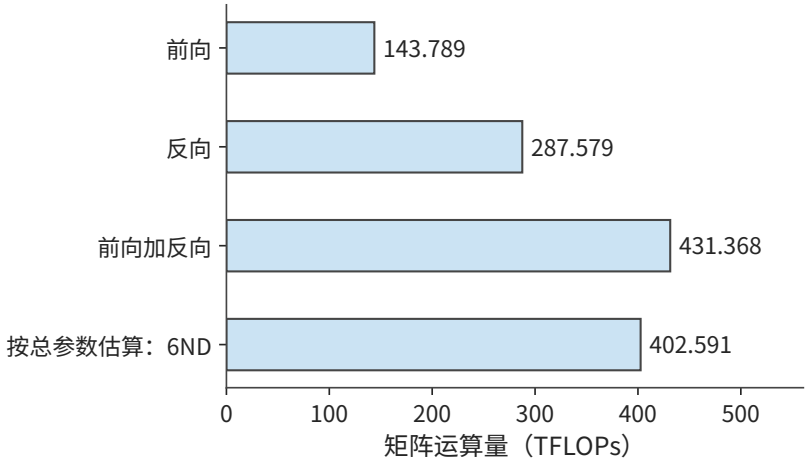



图 3-24　Qwen3-8B、8192 个 token 输入的全参数训练。全部输入 token 执行词表头、没有重计算；按矩阵逐项累计，前向加反向为 431.368 TFLOPs。

的是不参与监督的 token 的词表投影；反向时再把梯度放回对应位置。[34]

归一化、激活、损失与优化器也需要计算和读写。以参数更新为例，即使序列较短，仍要读写权重、梯度和优化器状态；序列越短，这部分成本在总量中的占比往往越高。因此，训练预算既要考虑随 token 数增长的矩阵计算，也要考虑由参数量决定的更新开销。[35]

模型新增的前向操作，也会增加对应的反向计算。上下文压缩需要对投影、池化和状态更新求梯度，mHC 需要向各残差通路传播梯度，MoE 则要对实际选中的专家求梯度。[36] 第 2 章的计算图因而也能用于判断：训练时要保存哪些中间结果，反向又要执行哪些运算。

因此，序列长度变化时可以先判断各项开销的变化趋势：参数投影随输入行数增长，注意力随查询—键配对数增长，优化器更新随参数量和更新次数增长。若采用重计算，容量需求下降，但重做的前向运算还要加回 $F_{\text{train}}$。这些变化都能沿同一张前向—反向图逐项解释。

> **练习 3-6〔延伸〕：一次参数更新需要多少计算、训练状态和监督 token**
> 从 $Y = XW$ 推导输入梯度和权重梯度的尺寸，说明三次矩阵乘法的运算量为何相同。对 Qwen3-8B，按每参数 18 字节计算长期训练状态的总容量，再将梯度改为 BF16，求节省的容量。设每个 micro-batch 含 2 条序列，每条有 4096 个输入 token，其中 1024 个 token 参与监督。累计 8 个 micro-batch 后更新一次参数，求每次更新处理的输入 token 总数和有效监督 token 总数。解释仅屏蔽不参与监督的 token 的损失，与只对参与监督的 token 执行词表投影，在计算量上有何区别。

## 3.5 强化学习（RL）

### 3.5.1 Rollout、奖励与环境验证

推理从外部请求开始，预训练从已有文本开始。强化学习（RL）用行动得到的反馈来改进后续行动。在语言模型中，策略是给定上下文时选择下一个 token 的概率分布。RL 把两条路径连接起来：当前策略先生成回答，再由反馈决定如何更新参数，更新后的策略继续产生下一批数据。生成过程通常称为 rollout。

[34] 标签减半与显式压缩词表头。

[35] 训练非矩阵补算，与8K 变体分别计量。

[36] DeepSeek V4-Flash 在线压缩器、注意力反向、MoE 单层训练、mHC 包装反向、优化器分组。

这条训练路线已有公开实践：DeepSeek-R1 给出了从 reasoning 任务和可验证奖励出发的做法。对代码、数学等容易检查的任务，反馈可以来自规则或测试；对需要实际操作的任务，则要运行工具与环境。奖励是用来评价一次回答或行动效果的数值。奖励若由另一个模型生成，还会产生额外推理。一次“训练迭代”因此可能包含多个模型、CPU 测试、沙箱等待和梯度更新。[37]

更近的模型把 RL 与其他训练阶段组合起来。DeepSeek V4-Flash 先通过 SFT 和 RL 训练领域专家，再以多教师的同策略蒸馏（on-policy distillation，OPD）将多个专家的能力整合到同一个模型中。蒸馏让学生模型学习教师模型给出的输出信息；OPD 由当前学生自己生成轨迹，教师在这些 token 位置提供候选输出的概率分布，学生据此学习。教师前向因此成为生成与更新之间的一项模型工作；规则验证则以测试或程序执行提供反馈，需要另一组计算资源。[38]

沿这一循环分析资源时，先找数据的生产者和消费者：策略生成 token，奖励函数或教师读取回答，学习器读取训练样本并产生新权重，负责生成的模型副本再使用新权重。某一角色变慢，后继角色就要等待；增加教师模型则需要额外的前向计算和权重存储空间。后续的并行安排都必须建立在这张依赖图上。

### 3.5.2 有效样本、策略更新与权重同步

生成回答和计算奖励之后，还要决定哪些样本用于学习。关键是区分已经生成的样本与最终进入更新的样本：前者决定生成开销，后者决定训练 batch，两者的数量未必相同。

设一轮需要保留 $n_{\text{keep}}$ 条样本，生成样本的平均保留比例为 $a$，则所需生成数的期望为

$$n_{\text{generate}} \approx \frac{n_{\text{keep}}}{a}.$$

保留 16 条样本，比例为一半时平均需要生成 32 条，降为四分之一时平均需要 64 条。生成和评分处理全部回答，参数更新只处理保留样本；保留比例下降会增加前两个阶段的工作，而不会按同样比例扩大更新 batch。第 3.5.3 节将采用等长样本，给定具体的生成数量，比较这两种情况。

是否保留一条样本，由训练方法及筛选规则决定。有的方法把失败回答作为负反馈，有的方法排除截断或奖励无效的回答。优势表示某个回答相对于比较基准更好或更差的程度。采用组内相对优势时，还要比较同一道题多份回答的奖励。因此，保留样本数决定更新时处理多少数据，奖励和损失函数则决定这些数据产生何种梯度。

**例子：相同奖励如何消除组内学习信号。**在使用组内相对奖励的策略更新中，如果同组回答得到相同奖励，减去组内平均奖励后，每条回答的优势都为零，相应的策略梯度也为零。回答全部正确就可能出现这种情况。模型完成了生成、评分和更新流程，却没有从这一组回答中获得区分优劣的信号。只有奖励差异提供了非零优势，策略损失才会推动参数沿相应方向调整。[39]

损失的求平均方式也会影响训练。若目标是对整个 batch 的有效 token 求平均，累计各 micro-batch 梯度时就要使用整个 batch 的有效 token 总数作为分母。若先分别计算每个 micro-batch 的均值，再对这些均值求平均，有效 token 较少的 micro-batch 里，token 就可能获得更大权重。第 10 章将用具体训练方法解释这种差别。[40]

[37] DeepSeek-R1 报告。

[38] DeepSeek V4 报告，§5.1 的领域专家与多教师 OPD、§5.2 的教师调度、rollout 与沙箱。

[39] 实验 10-8：固定 verl 最小训练流程。本章用这一实验区分任务质量与参数更新，完整系统的组织方式见第 10 章。

[40] 固定 verl 训练配置的损失归一化与参数更新过程，绑定原配置、源码与导出张量。

更新后还要把策略权重送给生成端。若生成端继续使用旧版本，下一批数据就仍由旧策略产生。同步循环可以等权重就绪后再生成；异步方案允许部分阶段重叠，却要处理样本版本与滞后。本章先明确这一依赖关系，第 10 章再讨论交接、抢占、恢复和资源配比。

### 3.5.3 有效样本数固定时，各阶段需要多少计算

这种区分可以用固定目标量化：每轮都要保留 16 条样本用于更新。如果回答更难通过筛选，就必须先生成更多回答。

**例题 3-4：样本保留比例下降时，为保持更新样本数需要增加多少计算？**

解：以 Qwen3-8B 为例，设有八道题，每题生成四份回答，$P = 1024$，$G = 256$，共保留 16 条样本用于一次策略更新。参考模型（reference model，权重保持固定、用来约束策略偏离程度的模型）与策略模型结构相同，分别保存权重；参考模型对全部 32 份回答各执行一次前向，本例不调用教师模型。各样本长度相同，生成时不提前停止，也不共享前缀。[41]

生成时每条先做一次 prefill，再做 255 次 decode。训练与评分时，轨迹已经确定，可以采用教师强制（teacher forcing）执行前向：每个预测步骤使用轨迹中已经确定的前一个 token，而不是等待模型重新生成它。因此，这些已知 token 可以一同计算。输入长度 $P + G - 1 = 1279$，256 个输出标签位于相应位置。16 条保留样本每轮更新的输入 token 共 20,464，监督输出共 4096。输入 token 决定上下文计算，参与监督的 token 决定损失作用于哪些输出，二者分别进入训练预算。

| 阶段 | 处理样本 | 矩阵运算量（TFLOPs） |
|---|---|---|
| Rollout prefill | 32 | 465.143 |
| Rollout decode，255 步 / 条 | 32 | 129.056 |
| 参考模型，一次前向 | 32 | 634.944 |
| 教师评分，设为零次 | 0 | 0 |
| 策略更新，一次遍历 | 16 | 952.416 |
| 合计 | 保留 16 条 | 2,181.559 |

将一条回答的生成与参考模型前向合计运算量记为 $f_g + f_r$，一次固定更新 batch 的运算量记为 $F_u$，则

$$F_{\text{cycle}}(n) = n(f_g + f_r) + F_u.$$

参考模型读取完整回答，可以一次处理多个已知 token；生成则逐 token 推进。上式虽然把运算量相加，仍需分别记录各阶段的矩阵形状，才能根据对应的执行效率估算时间。

如果样本保留比例从 1/2 降到 1/4，但仍要求得到 16 条保留样本，教学对照改为生成 64 条。生成与参考模型计算翻倍，更新量不变，总矩阵运算量增到 3,410.701 TFLOPs，约增加 56.3%。每个保留样本分摊的矩阵运算量从 136.347 增到 213.169 TFLOPs。增加的 1229.142 TFLOPs 全部来自额外回答的生成与参考模型计算；进入更新的 16 条样本保持不变。[42]

更新完成后，生成副本需要获得新权重。设有 $r$ 个副本，逐个独立发送大小为 $M_W$ 的完整权重，则传输量为 $rM_W$。Qwen3-8B BF16 的一份权重约为 16.381 GB，四个副本共需约 65.526 GB。优化器状态留在学习器，

[41] Qwen 教学 RL batch。

[42] 降低样本保留比例、保持保留样本数。

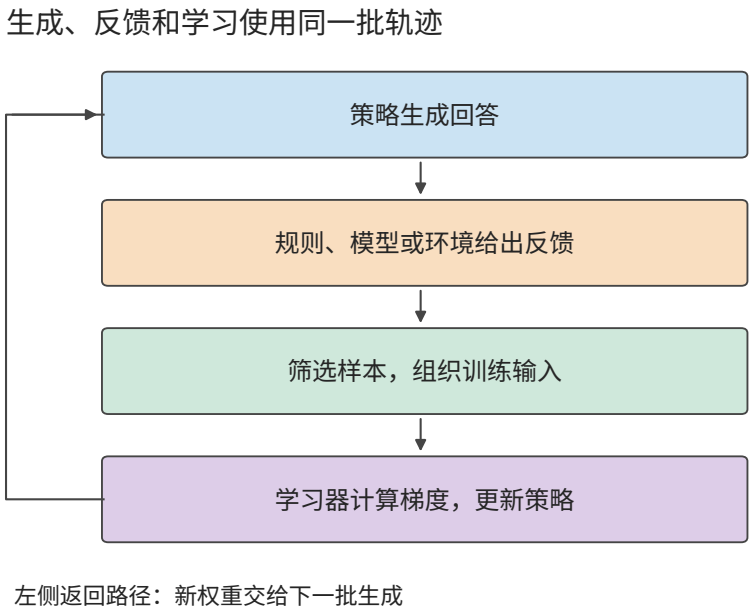



图 3-25　强化学习中的角色和数据流。生成端产生回答，反馈环节评价结果，筛选后送给学习器更新；新权重再用于下一批生成。

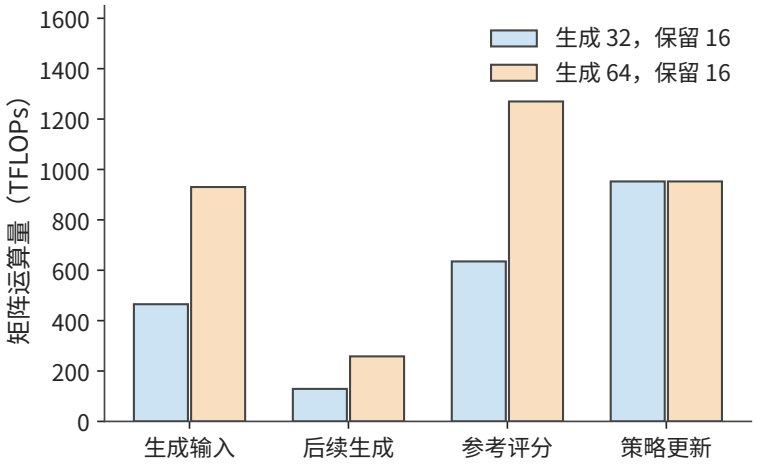



图 3-26　保持 16 条有效样本的目标，生成数由 32 增到 64。生成时的输入处理、后续 decode 和参考模型评分随回答总数增加，策略更新处理的样本数保持相同。

用于下一次更新；生成副本只需要执行前向所需的权重。广播或分层分发则可以让多个副本共用部分传输路径，减少重复发送。

为各阶段分配资源时，既要看运算量，也要看计算方式。生成阶段的 decode 每步通常只处理各请求的一个新 token，在小 batch 下同时处理的 token 较少；评分和训练则能一起处理已知回答中的多个 token，工具验证还需要 CPU 和运行环境。分别估算各阶段的耗时，再按生成、反馈、更新和权重同步的顺序排列，才能得到完整周期。第 4 至 10 章将逐步介绍相应的硬件和调度方法。

**练习 3-7〔核心〕：更新样本数不变时，多生成回答会增加多少 RL 开销**

保持一次更新使用 16 条等长样本。利用例题 3-4 中分别生成 32 条和 64 条回答时的周期总运算量，求每多生成一条回答所增加的生成与参考模型运算量，以及一次更新所需的固定运算量。保持更新样本数不变，据

此预测生成 48 条回答时的周期总运算量。再用与生成模型结构相同的教师模型，对全部 48 条回答各执行一次前向计算，求新增的运算量。若将更新后的 BF16 权重分别发送给四个副本，每个副本独立接收一份，求总传输量。最后画出生成、反馈、更新与同步的依赖，指出哪些阶段可以在不同 batch 之间重叠。

## 3.6 从负载需求到训练与服务预算

### 3.6.1 Scaling Law：模型规模与数据量的关系

单次更新的计算可以逐项求出，训练多少参数、使用多少数据则需要另行决定。设总训练计算预算为 $C$，参数量为 $N$，训练 token 数为 $D$，主要计算近似满足 $C = 6ND$。增加 $N$ 就必须减少 $D$。要在两者之间作选择，就要知道参数与数据分别如何影响质量。Scaling Law（规模规律）用实验拟合模型规模、训练数据和损失之间的关系。下面采用幂律模型，其中参数与数据两项都为正，训练配置和损失评估方式固定：

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}.$$

$L$ 是验证损失，$E$ 是渐近损失，$A/N^{\alpha}$ 描述参数规模不足的影响，$B/D^{\beta}$ 描述数据量不足的影响。这里的 $A, B, \alpha, \beta$ 都是拟合系数。单独增加参数或数据会降低对应项；固定计算预算时，两项却向相反方向变化。

用 $D = C/(6N)$ 消去数据量，再对 $N$ 求导。最优点满足

$$\alpha A N^{-\alpha-1} = \beta B \left(\frac{6}{C}\right)^{\beta} N^{\beta-1}.$$

左侧是增加参数带来的损失下降，右侧是减少数据造成的损失上升；两者相等时，再小幅改变参数量已经没有净收益。整理得到：

$$N_{\text{opt}} = \left(\frac{\alpha A}{\beta B}\right)^{1/(\alpha+\beta)} \left(\frac{C}{6}\right)^{\beta/(\alpha+\beta)}.$$

由此得到的 $N_{\text{opt}}$ 是连续取值的结果，实际设计时可在它附近选择合适的层数和宽度。这一最优解只在固定训练计算预算下最小化验证损失，不含部署后的推理成本，也不考虑数据供给是否充足。模型上线后还要服务大量请求时，最优选择可能偏向更小、训练更充分的配置；第 3.6.2 节将用公开模型的训练记录，检查实际投入与这里的预测相差多少。

不同研究拟合出的分配指数并不一致。Kaplan 等人的早期研究给出约 $N \propto C^{0.73}$，$D \propto C^{0.27}$ 的分配关系；Chinchilla 研究用不同的实验和拟合得出：模型规模与数据量应随预算更接近等比例增长。前一种分配更快增加模型参数，后一种把更多新增预算分给训练数据；曲线及其拟合指数因此直接改变参数与数据的选择。[43]

**例题 3-5：训练规模拟合关系能否预测更大模型的损失？** 从公开训练记录中选定六个较小模型的观测值用于拟合，另将两个较大模型的观测留出。先选定拟合方法，再比较留出模型的预测值与实际观测值。

解：datablations 是研究模型与数据规模变化的公开实验记录，C4 是这些记录使用的一套文本语料。以其中八个模型为例，用六个较小模型拟合曲线，将 $N \geq 2 \times 10^9$ 的两个模型留作检验。两个大模型的实际损失分别约为 2.574、2.337，拟合曲线预测为 2.583、2.363。预测略高于实际值，均方根误差约为 0.0193 nats/token。

[43] Kaplan Scaling Laws，§6；Chinchilla，计算最优分配与拟合方法。

nat 是使用自然对数时的信息量单位，nats/token 表示每个 token 的平均损失。图 3-27 用不同标记区分拟合点与检验点。[44]

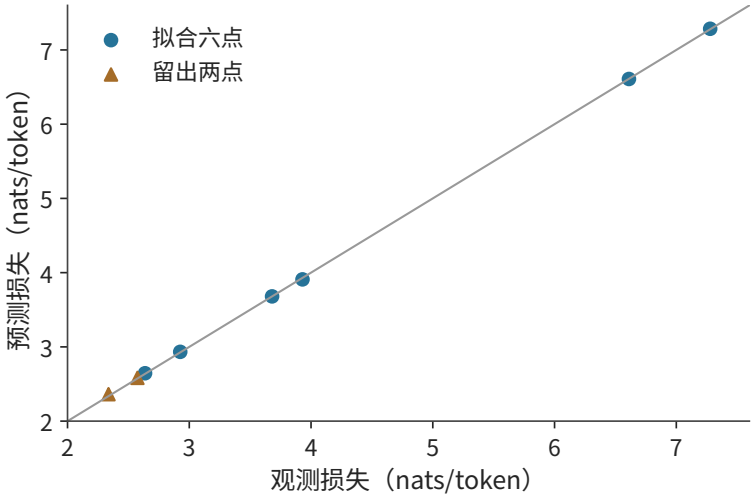



图 3-27　八个公开 C4 观测的预测与实际损失。6 点用于拟合，2 点预先留出作检验；对角线表示预测等于观测，点到线的偏差反映误差。

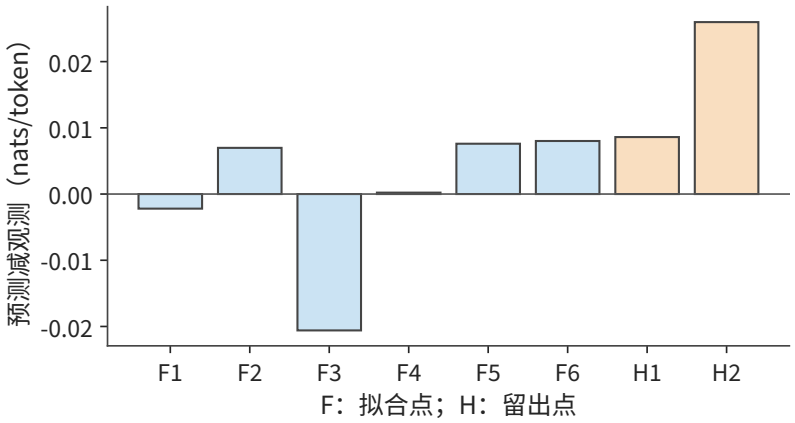



图 3-28　同一组预测误差的放大视图。F1—F6 为拟合点，H1—H2 为留出点；纵轴是预测减观测，保留正负号。

留出检验之所以必要，是因为拟合只保证曲线尽量贴近用于求系数的数据。将较大模型排除在拟合之外，再比较预测与实际损失，才能检查这条关系能否预测新规模。这里的两个误差分别约为 0.009 和 0.026；将它们平方、取平均再开方，便得到均方根误差。

预测出验证损失之后，还要判断模型能否完成实际任务。Llama 3 报告先根据训练计算量预测损失，再建立损失与任务表现的关系。[45] 这样，代码、检索和工具使用等任务的质量要求，才能进一步转化为训练规模与计算量的选择。

拟合和预测都建立在单次训练的观测上，而训练本身也有波动。同样的数据与模型规模，改变随机初始化就可能得到不同的学习曲线；较大模型在训练尚未充分时，也未必优于较小模型。使用多个随机种子重复训练，可以区分规模变化带来的趋势与单次训练的偶然波动。[46]

[44]公开 C4 八点拟合，来源日志、排除项和四项敏感性随报告保存。

[45]Llama 3 报告，训练预算、损失预测与下游任务表现；§3.3.2 与表 4 给出 H100 上 38%–43% 的 BF16 MFU。

[46]练习 3-8：固定文本上的六次小模型实训，保存 checkpoint 与两个 seed 的完整结果。

长期服务会把每次调用的成本累积到训练预算之上。较小模型可以靠更长的训练达到目标质量：前期投入更多，之后每次调用读取的权重更少、执行的运算也更少。Beyond Chinchilla-Optimal 将这种推理需求纳入分析，47 个模型覆盖 150M–6B；其中 150M 模型的训练量最高达到每个参数 10,000 个 token，较大模型最高达到 1,000 个。[47] 训练预算与服务次数共同决定较小模型何时能收回额外训练的成本。

**固定目标损失下的训练投入与累计服务成本。**固定拟合目标损失为 2.9，用矩阵运算量估算训练与服务成本，再统一折算为 H100 SXM 的 GPU 时间（GPU 数量与使用时长的乘积，以 GPU 秒或 GPU 小时计）。请求取 $P = 512, G = 128$，训练运算量为 $6ND$，每个完整请求的运算量为 $2N[P + (G-1)]$。H100 SXM 的 BF16 稠密峰值为 989.4 TFLOP/s；Llama 3 在 H100 上预训练时报告的 BF16 MFU（定义见第 1.2.2 节）为 38%–43%，训练与服务都取 40%，折算后每 GPU 秒完成 395.76 TFLOP。[48] 0.1B 模型需要约 298.6B 训练 token，超过拟合所用数据量的上界，因此成本图用虚线标注这条外推曲线。按这条外推曲线计算，0.1B 模型训练约需 125.7 H100 GPU 小时，比 0.5B 模型多 73.5 GPU 小时；每次调用则少用约 0.00129 GPU 秒。调用约 2.048 亿次时，两者总成本相等。在此之前，0.1B 模型多付的训练成本尚未收回；在此之后，每次服务节省的成本累计超过了前期投入。训练与服务按同一系数折算，所以该交点只取决于运算量，与 MFU 的取值无关。[49]

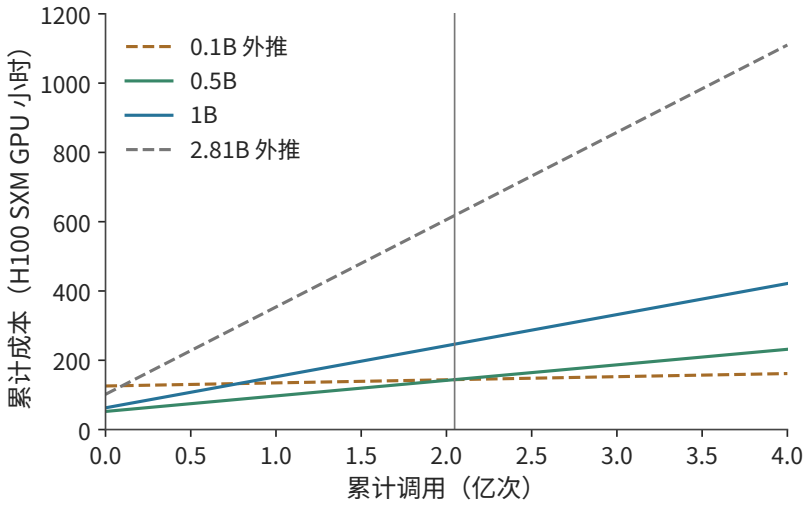



图 3-29　训练与服务累计成本的题设比较。截距是训练投入，斜率是单次调用成本；虚线标出超出拟合参数或数据范围的方案，竖线为约 2.048 亿次的成本交点。图例中的 B 表示十亿个模型参数；纵轴为按 40% MFU 折算的 H100 SXM GPU 小时。

回到参数与数据的分配，还可以直接预测预算增加后的结果。由最优解表达式可得 $N_{\text{opt}} \propto C^{\beta/(\alpha+\beta)}$，再代回计算约束，得到 $D_{\text{opt}} \propto C^{\alpha/(\alpha+\beta)}$。若两个指数相等，预算增至四倍，参数和训练数据各增至两倍。拟合指数的意义由此变得具体：指数决定新增计算应如何分给模型与数据。

**练习 3-8〔延伸〕：如何在模型参数与训练数据之间分配计算预算**

采用损失模型 $L = E + A/N^{\alpha} + B/D^{\beta}$ 和计算预算约束 $C = 6ND$，推导使损失最小的参数量 $N$ 与数据量 $D$ 分别如何随计算预算 $C$ 增长。取 $\alpha = \beta$，当计算预算分别增至原来的四倍和九倍时，最优参数量和数据量应各增至原来的多少倍？再使用例题 3-5 的八个 C4 观测点，用其中六个拟合、另外两个检验，计算两个检验点的预测误差，并解释为什么拟合点上的误差不能替代留出检验的误差。

[47] Beyond Chinchilla-Optimal，ICML 2024 正式版，推理需求、实验范围与长训练外推。

[48] Llama 3 报告，训练预算、损失预测与下游任务表现；§3.3.2 与表 4 给出 H100 上 38%–43% 的 BF16 MFU。

[49] 根据 C4 拟合结果估算生命周期成本，费用按 H100 SXM GPU 秒计。

### 3.6.2 从 Llama 到 Qwen 的训练投入

第 3.6.1 节的拟合说明了模型规模与训练数据之间如何取舍。本节用公开模型的训练记录分析开发者实际采用的训练投入：先比较每个参数对应的训练 token 数，再将算法运算量与具体硬件上的训练时间区分开。

从各报告披露的阶段和规模，可以看到一种趋势：在相近的 7–8B 参数规模下，训练 token 数持续增加。[50]

| 模型与报告范围 | 训练 token | $D/N$ 粗略估算 | $6ND$ 粗略估算 |
|---|---|---|---|
| Llama 1，6.7B | 1T | 149 | $4.02 \times 10^{22}$ FLOPs |
| Llama 2，约 7B | 2T | 286 | $8.40 \times 10^{22}$ FLOPs |
| Llama 3.1，约 8B | 约 15T | 约 1,875 | 约 $7.20 \times 10^{23}$ FLOPs |
| Qwen2.5，约 7B，模型系列披露 | 约 18T | 约 2,571 | 约 $7.56 \times 10^{23}$ FLOPs |
| Qwen3，约 8B，模型系列披露 | 约 36T | 约 4,500 | 约 $1.728 \times 10^{24}$ FLOPs |

$D/N$ 将数据投入换成“每个参数对应多少训练 token”。表中约 7–8B 的稠密模型，$D/N$ 从约 149 增至约 4,500，说明参数量相近的模型，训练数据量也可以相差悬殊。前期训练成本随数据增加，部署后每次调用的权重容量却仍主要由参数数量决定。因此，更充分训练的小模型可能增加一次性投入，同时降低长期服务成本。

这组数字也澄清了第 3.6.1 节的拟合结果与实际训练投入的关系。按 Chinchilla 的等比例分配，约 8B 模型的计算最优数据量约为每参数 20 个 token，即约 160B token；表中实际投入从 1T 增至约 36T，超出这一基准一至两个数量级。两者回答的问题不同：拟合在固定训练计算预算下最小化验证损失，开发者则把上线后的服务成本计入目标，在数据供给充足、预期调用量大时选择更小、训练更充分的模型。第 3.6.1 节引用的 Beyond Chinchilla-Optimal 和第 3.6.3 节的成本交点公式给出了这种选择的定量条件。训练预算最优与全生命周期最优是不同的目标，实际投入超出前者并不表示前者算错了。

更长训练、数据筛选、蒸馏和后训练，都是用更多前期工作提高给定规模模型的任务能力。Llama 3 模型卡的基础模型评测中，Llama 3 8B 与 Llama 2 70B 在覆盖多学科选择题的 MMLU 基准上分别为 66.6 / 69.7，使用维基百科证据的问答基准 TriviaQA-Wiki 为 78.5 / 87.5。[51] 两项差距不同，说明部署方要按自己的任务选择质量要求；达到质量门槛的小模型，才有机会把较小权重带来的容量与读取收益转成服务收益。

分析大模型和 MoE 时，则要分别列出总参数量与激活参数量。DeepSeek-V3 报告为总参数 671B、每 token 激活约 37B、预训练 14.8T；DeepSeek V4-Flash 为 284B / 约 13B、32T，DeepSeek V4-Pro 为 1.6T / 约 49B、33T。总权重影响容量，激活参数只提供计算量的粗略估计。DeepSeek V4-Flash 主干专家的三个投影可单独计算：

$$C_{\text{expert/token}} = 18 n_L h f (k_{\text{routed}} + k_{\text{shared}}) = 18 n_L h f (6 + 1).$$

式中 $n_L$ 为主干层数，$h$ 为隐藏维度，$f$ 为专家中间维度，$k_{\text{routed}}$ 与 $k_{\text{shared}}$ 为每 token 选中的路由专家数与共享专家数；系数 $18 = 3 \times 2 \times 3$，依次对应三个投影、每次乘加计 2 FLOPs、前反向合计为前向的三倍。DeepSeek V4-Flash 的主干专家约 45.4495 GFLOPs/token，按 32T 累计约 $1.45438 \times 10^{24}$ FLOPs；DeepSeek V4-Pro 约 169.2465 GFLOPs/token，按 33T 累计约 $5.58513 \times 10^{24}$ FLOPs。整步训练计算由这些专家矩阵与

[50] 训练投入与 Scaling Law 历史笔记及锁定公开字段的复算。

[51] Meta Llama 3 模型卡，Base pretrained models 表中 Llama 3 8B 与 Llama2 70B 两列。

注意力、路由、MTP、优化器和重计算共同组成。报告的损失权重也不是执行比例，例如 MTP 权重为 0.3，不表示只执行 30% 的辅助计算。[52]

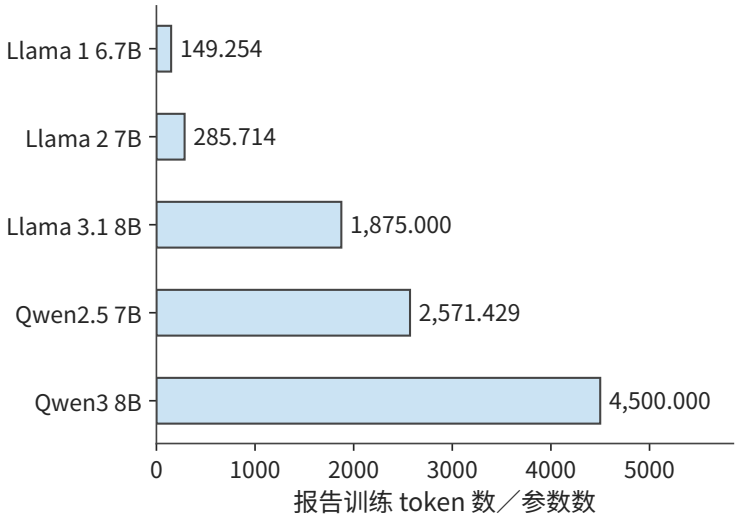



图 3-30　相近参数规模的模型投入不同数量的训练 token。柱值为报告训练数据量除以参数量，Qwen 采用模型系列披露的数据预算。

**练习 3-9〔延伸〕：增加训练数据或专家数量会改变哪些资源需求**

根据 Llama / Qwen 表重算 $D/N$ 与 $6ND$。若参数量保持不变、训练数据量增至四倍，预测训练计算与上线后的纯权重容量分别如何变化。对 MoE，说明总专家数加倍、每 token 选中数不变时，参数容量与专家训练计算能否都按两倍估算。

除运算量外，训练投入还常用 GPU 小时计量。若持续使用 $n_{\mathrm{GPU}}$ 张卡，总 GPU 小时为 $H_{\mathrm{GPU}}$，实际历时为

$$T_{\mathrm{calendar}} = \frac{H_{\mathrm{GPU}}}{n_{\mathrm{GPU}}}.$$

Llama 1 65B 的训练消耗 1,022,362 A100 GPU 小时。持续使用 2048 张卡，对应约 499.2 小时，即 20.80 天；在相同加速器效率下只使用一半卡数，则需要约 41.60 天。GPU 小时体现总投入，卡数决定完成这些工作需要多少实际时间。

DeepSeek-V3 报告预训练 2.664M H800 GPU 小时；14.8T token 只对应预训练阶段；按持续使用 2048 卡计算，约需 54.20 天。后续的上下文扩展与后训练另有用量，但各阶段统计范围不同，不能一并除以预训练 token 数。[53]

用 GPU 小时换算实际历时，要除以同期使用的卡数。对 Llama 3.1 405B，取公开的 30.84M H100 GPU 小时，并假设这项工作全部使用最大规模 16,384 卡，实际历时约为 $30.84 \times 10^6/(16384 \times 24) \approx 78.43$ 天。某个阶段使用的卡数更少时，相同 GPU 小时需要更长的实际时间；训练资源的分配时序因此决定训练完成时间。

上述用量来自三种加速器，GPU 小时不能直接跨设备比较投入规模。按 BF16 稠密峰值算力折算：A100 80GB 为 312 TFLOP/s，H100 与 H800 同为 989.4 TFLOP/s（H800 只是互联带宽更低），一个 H100 或 H800

[52] 训练计算量笔记，线性层前反向、序列分布与 DeepSeek V4-Flash 专家分项计算。

[53] 训练投入与 Scaling Law 历史笔记及锁定公开字段的复算。

GPU 小时约相当于 $989.4/312 \approx 3.17$ 个 A100 GPU 小时。图 3-31 把三组公开用量统一折算为 A100 80GB 等效小时；折算假定各设备的实际利用率相近，用于比较投入规模，不表示效率或成本差异。

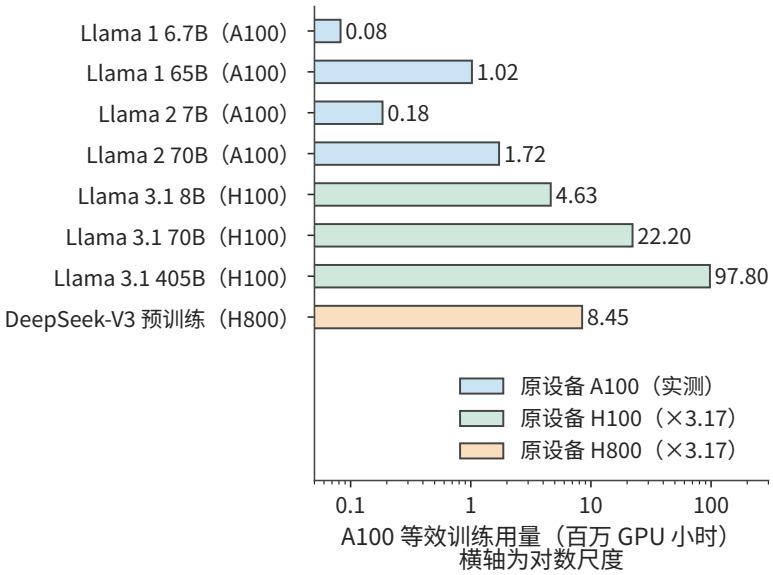



图 3-31　Llama 与 DeepSeek-V3 的公开训练用量统一折算为 A100 80GB 等效 GPU 小时。Llama 1 ／ Llama 2 为 A100 实测小时；H100 与 H800 小时按 BF16 稠密峰值之比 $989.4/312 \approx 3.17$ 折算，DeepSeek-V3 只计预训练阶段。横轴为对数尺度；折算假定各设备实际利用率相近，不表示效率或成本差异。

以上几组用量都属于预训练，后训练的投入同样可以用 GPU 小时比较。Qwen3 技术报告的表 21 把同一个 8B 模型的 RL 与 OPD 两条后训练路线并列，投入分别为 17,920 与 1,800 GPU 小时。两条路线可以二选一，各自包含生成、反馈与学习工作。把 GPU 小时对应到具体阶段和加速器，再结合各阶段的执行过程，就能解释训练投入主要消耗在哪里。

### 3.6.3 满足质量要求的全生命周期成本

现在把第 2 章与本章的分析合在一起。模型结构决定每次调用所需的存储容量、运算量和读取量，负载决定调用次数与等待，训练决定上线前的投入。设模型 $m$ 的前期成本为 $C_0(m)$，未来有 $Q$ 个任务，第 $j$ 类占比为 $w_j$、完成一个任务的平均成本为 $c_j(m)$，则

$$C_{\text{life}}(m) = C_0(m) + Q \sum_j w_j c_j(m).$$

比较之前，各模型都要满足相同质量、时限与完成条件。若任务成功率不同，$c_j$ 中还需包含失败和重试；价格变化、模型寿命、维护与贴现等假设也应在给出预算时说明。专用硬件若需要较长部署周期，模型版本能使用多久也会成为这一选择的条件。

考虑都满足任务质量和时限的两个模型。若较小模型需要额外训练成本 $\Delta C_0$，但每个成功任务能节省 $\Delta c > 0$，累计成本相等的调用量为

$$Q_* = \frac{\Delta C_0}{\Delta c}.$$

若较小模型额外训练 10 万 H100 GPU 小时，即 $3.6 \times 10^8$ GPU 秒，每个任务节省 0.72 H100 GPU 秒，则 $Q_* = 3.6 \times 10^8 / 0.72 = 5 \times 10^8$，即五亿次任务。预期需求低于此数，额外训练成本尚未收回；高于此数，较小模型累计节省更多。容量、延迟、质量和调用量共同决定选择，单看模型大小或 token 单价无法完成这一判断。

**练习 3-10（延伸）：训练完成时间与累计服务成本**

假设完成训练共需 1,022,362 A100 GPU 小时，即 Llama 1 65B 的公开用量。分别持续使用 2048 张和 1024 张 A100，并假设加速器数量改变后总 GPU 小时不变，求两种配置下完成训练所需的天数。再比较两个满足相同质量与时限的模型：小模型多投入 10 万 H100 GPU 小时训练，每个成功任务节省 0.72 H100 GPU 秒。预计完成两亿项和十亿项成功任务时，分别应选择哪个模型？计算总成本差，并说明若实际任务量发生变化，选择会在何处反转。

后续章节将使用本章整理出的三类负载信息。下表列出各类负载需要记录的内容，以及这些信息将用于哪些系统设计问题。

| 负载 | 必须保留的字段 | 后续使用 |
|---|---|---|
| Chat / Agent | 任务检查、模型与版本、每轮输入 / 命中 / 输出、到达、工具依赖、分支与复用间隔 | 第 8 章安排批处理与状态；第 9 章交接与路由；第 11 章管理工具环境 |
| 视觉 / 实时 | 原图字节、E/P/D、EC/KV、帧或块到达、首响应 / 连续播放要求、打断后不再需要的计算 | 第 8、9 章安排编码和交接；第 12 章加入链路与执行位置 |
| 训练 / RL | 数据与质量、长度和标签、micro-batch / 更新、生成样本数 / 保留样本数、奖励计算与模型调用、权重版本、完成期限 | 第 4–7 章分析算力、存储与通信需求；第 10 章安排学习；第 11 章配置环境与资源池 |

这三类例子都说明，描述负载需要记录具体的执行过程。两分钟请求中，各时段的输入输出比例决定积压何时出现；代码 Agent 中，工具依赖决定等待时间，以及状态需要保存多久；RL 中，筛选前后的样本数决定生成与更新阶段各处理多少数据。掌握这些信息，才能为各阶段安排资源。

第 4 章将结合芯片的算力、存储容量与带宽，分析这些负载如何在加速器上执行。本章算出的矩阵尺寸、状态大小和访问次数，将用于判断计算需要多久、数据能否容纳，以及读写能否及时完成。各阶段的依赖关系则决定哪些工作可以重叠，哪些必须等待。

## 谬误与陷阱

**误区：平均请求率和长度相同，负载就相同。**少量超长请求、输入与输出长度的组合、请求到达顺序，都会影响短时间内的资源需求。例题 3-1 中，处理能力足以应付平均到达量，后一分钟却仍会积压。

**误区：token 单价降低，成功任务成本就会降低。**额外思考、未通过检查的回答、工具与环境都应进入分子，合格任务数进入分母。零成功的 batch 只能报告消耗与失败，不能估计成功成本。

**误区：训练 token 总数相同，系统工作量就相同。**序列分布改变注意力查询—键配对的数量，标签与词表头策略改变输出工作，更新次数改变优化器成本；RL 还要为没有采用的回答付出生成与反馈的开销。

**误区：Scaling Law 拟合出的计算最优规模，就是应该采用的训练规模。**计算最优只在固定训练预算、不计推理成本时成立。公开模型的每参数训练 token 数超出这一基准一至两个数量级，是把长期服务成本计入目标后的选择；目标函数不同，不表示拟合结果被推翻。

## 代码任务轨迹

任务是修复区间合并函数，使嵌套区间和端点相接的区间正确合并，同时不修改输入及其嵌套列表。模型可以读取指定文件、改写它、运行固定测试并结束任务，控制器不替模型修改代码。关闭思考模式时，模型在 12 轮内读文件一次、改写五次、运行测试六次，仍未完成；开启后，四轮分别是输出截断、写文件、运行测试和结束。首轮截断产生的等待同样计入任务总时间。[54]

## 本章小结

持续服务的负载既取决于每次请求要做多少工作，也取决于请求何时到达、各步骤之间有哪些依赖。用平均速率可以粗略估算所需资源；分析短时间内的积压和状态驻留时间，则能发现平均值未体现的资源压力。多轮任务还包含工具等待与失败尝试，实时任务要保证数据及时到达，训练则增加反向、参数更新和权重同步。

比较不同方案的资源与成本时，应采用相同的质量要求和任务完成标准。Scaling Law 为训练分配提供可检验的统计模型，长期服务需求又可能改变前期投入的选择。核心练习 3-2、3-4、3-7 分别把这些方法用于持续请求、交互任务和 RL 循环。

[54]练习 3-4：两条代码 Agent 轨迹，含独立检查与额外别名条件。

# 第 4 章 加速器架构

一段计算用时 100 µs，其中矩阵乘法占 60 µs，其余工作占 40 µs。把矩阵单元的速度提高一倍，总时间降为 70 µs，加速比约为 1.4。若矩阵乘法只占 10 µs，同样的改动只能把总时间降到 95 µs。增加的计算能力相同，收益却相差悬殊。通用处理器把芯片面积分配给各类指令；神经网络的执行时间主要花在矩阵运算上以后，这种分配就不再合算，专用矩阵单元应运而生。架构设计要解决的，正是在芯片面积、功耗和存储容量的限制下，选择能最大限度缩短执行时间的方案。

第 2、3 章介绍了模型的计算图，以及请求在不同阶段的计算需求。本章以 Qwen3-8B 的查询投影（Q 投影）为例，即把当前 token 的隐藏向量变换成注意力查询的矩阵乘法，说明加速器如何完成这些计算。先计算运算次数和读写的数据量，再逐步考虑矩阵分块、存储层次和数据依赖，解释实际执行时间为什么与理想估算不同。随后加入注意力和专家计算，分析更复杂的算子如何使用同一套硬件；再集中比较 NVIDIA、昇腾与 Apple 的代际演进，说明模型需求如何推动这些部件共同变化。

本章围绕三个问题展开：任务需要多少计算、存储和传输；计算单元为什么会等待数据或前一步结果；提高某一部分的性能以后，哪一部分会成为新的瓶颈。理解这三个问题，就能够用同一种方法分析 GPU、昇腾、Apple 芯片以及更专用的设计。

## 4.1 从模型计算认识加速器

分析算子时，首先要确定它执行哪些运算、读写哪些数据。本节从这两项需求出发，认识加速器的组成，并初步分析计算单元、存储和接口之间的取舍。

### 4.1.1 矩阵乘法的计算与数据需求

Qwen3-8B 的隐藏维度为 4,096，32 个查询头各有 128 个分量，Q 投影的输出宽度也是 4,096。一次处理 $M$ 个 token 位置时，投影为

$$Y = XW_Q, \qquad X \in \mathbb{R}^{M\times 4096}, \quad W_Q \in \mathbb{R}^{4096\times 4096}, \quad Y \in \mathbb{R}^{M\times 4096}.$$

在投影 $Y = XW$ 中，$M$ 是本次处理的 token 总数，每个 token 的特征向量占 $X$ 的一行；$K$ 是每个输入向量的特征数，也是权重矩阵 $W$ 的行数；$N$ 是每个输出向量的特征数，也是 $W$ 的列数；本例 $K = N = 4096$。这组大写字母描述矩阵形状，与第 1 章容量记号 $M$ 的语境不同。若一次处理 $B$ 条等长请求、每条有 $P$ 个新输入 token，prefill 的 $M = BP$；单步 decode 为每条请求处理一个 token，因此 $M = B$。两个阶段使用同一份权重，完成相同的线性变换，区别在于每次有多少行输入共同使用它。[1]

每个输出元素都需要计算一个长度为 4,096 的内积。将一次乘法和一次加法计为两个浮点操作，矩阵乘法计算量为

$$F = 2MKN = 2M \times 4096^2 = 33\,554\,432M \text{ FLOPs}.$$

[1] Qwen3-8B 固定配置、张量索引与模型实现见模型配置及投影计算结果。

再看数据。输入、权重与输出均为 BF16，每元素两字节；累加器从零开始，以 FP32 保存部分和，每元素四字节。输入与权重各从片外读取一次，最终结果写回一次，总读写量为

$$V = 2MK + 2KN + 2MN = 33\,554\,432 + 16\,384M \text{ bytes}.$$

第一项是固定的 32 MiB 权重，第二项是随行数增长的输入与输出。这里 MiB 为 $2^{20}$ bytes，带宽规格中的 GB/s 使用 $10^9$ bytes/s。将计算量除以访问量，得到算术强度 $I = F/V$。

| 一次 Q 投影 | $M = 1$ | $M = 256$ |
|---|---|---|
| 矩阵乘法计算量 | 约 33.6 MFLOPs | 约 8.59 GFLOPs |
| 权重读取 | 32 MiB | 32 MiB |
| 输入读取与输出写回之和 | 16 KiB | 4 MiB |
| 总访问量 | 约 32 MiB | 36 MiB |
| 算术强度 | 约 1 FLOP/byte | 约 228 FLOPs/byte |

图 4-1 将权重读取分摊到输入行：单行独自承担 32 MiB，256 行则每行分摊 128 KiB。

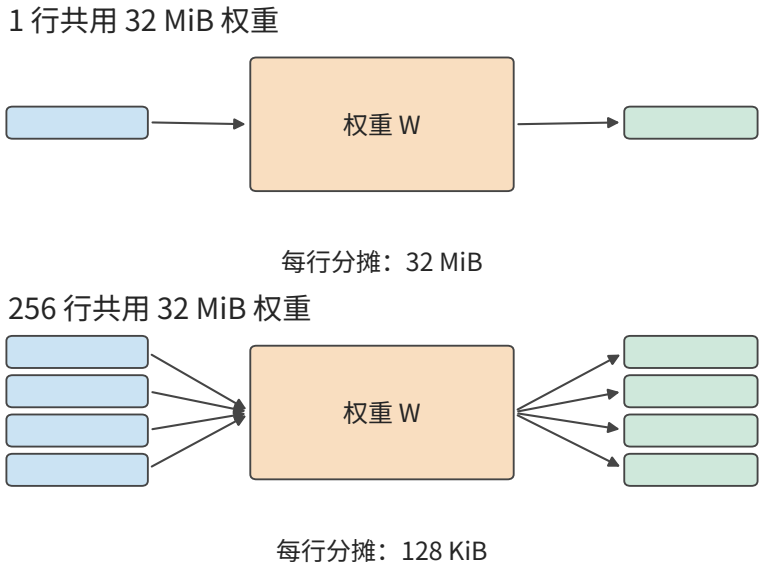



图 4-1　两种调用都使用同一份 32 MiB 权重。上方是一行输入，下方以部分条带示意 256 行；每行分摊的权重读取从 32 MiB 减至 128 KiB。

行数增至原来的 256 倍，运算次数也增至 256 倍，访问量却只增加约八分之一。权重复用使计算量增长快于访问量，算术强度随之提高。[2]

将算术强度的分子、分母同除以 $M$，可以进一步求出复用的极限：

$$I(M) = \frac{2KN}{2KN/M + 2(K+N)}.$$

[2]Q 投影，RTX 4090，M=1；M=256。

随着 $M$ 增长，每行分摊的权重项 $2KN/M$ 不断缩小，输入输出项保持不变。对本例，$I(M)$ 最终趋近 2,048 FLOPs/byte。因此，继续增大 batch 的收益会逐渐减小：每行分摊的权重读取量虽然越来越少，但新增 token 的输入和输出仍然需要读写。

上述计算假定权重只读一次；要在硬件上做到这一点，就需要在使用期间保存操作数和中间结果。

### 4.1.2 加速器的组成与分工

主机（host）通常是运行框架、准备输入并发起任务的 CPU 一侧，加速器负责执行模型计算；在主机—设备（host–device）的编程接口中，加速器一侧通常称为设备。对有独立显存的 GPU，主机内存和显存是两处存储。运行投影前，输入与权重先放到加速器可访问的位置；CPU 随后提交命令，给出算子、数据地址与执行参数。数据已经驻留时，后续调用可以反复引用同一地址。

图 4-2 展示了加速器内部各部分如何协作完成一次矩阵乘法。片外内存保存较大的输入和权重；共享缓存是由多组计算单元共同访问、自动保留近期数据的存储；局部缓冲是分配给当前计算组暂存输入的存储；矩阵单元反复更新部分和；累加存储把部分和留到本块计算结束。向量与通用计算单元负责结果变换、地址计算和流程控制等工作。

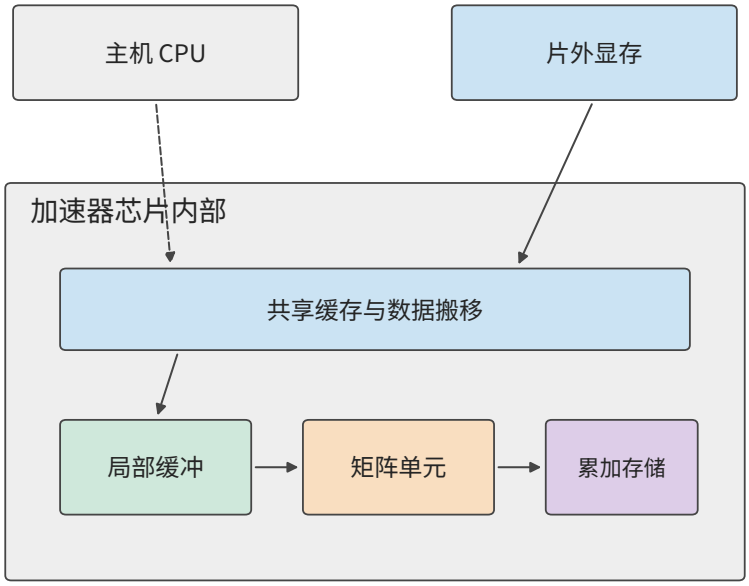



图 4-2　主机、显存与芯片内部的关系。实线表示数据经过缓存、局部缓冲、矩阵单元和累加存储，虚线表示主机提交工作。图按硬件功能分组。

计算一个输出块时，需要沿 K 维依次读入多块输入和权重。每完成一次乘加，部分和仍留在累加存储中，下一块继续更新它。如果每次更新都写回片外，中间结果就要反复写出、读入；把部分和留在片上，这些访问就只剩最后一次写回。数据重复使用的次数越多，留在片上就越有价值。

NVIDIA 把一组可协作执行的线程、通用计算单元、矩阵单元与局部存储组织为流式多处理器（Streaming Multiprocessor，SM），其中矩阵乘加单元称为 Tensor Core。SM 内由线程协作使用、由软件管理的缓冲称为共享内存（shared memory）。线程是程序中分别执行指令的工作单位；一次提交到加速器上运行的函数称为 kernel（内核），一个 kernel 可以由许多线程协作执行。昇腾 DaVinci 架构中的 Scalar、Vector、Cube 和 MTE 则分别指控制单元、向量计算单元、矩阵计算单元与数据搬移单元。Apple 的 Metal 是提交 GPU 工作的编程接口，用线程组组织能够共享局部存储并同步的线程。虽然名称不同，这些部件都要解决几个共同问题：操作数如何到

达执行单元，结果如何保存，下一项工作何时可以开始。[3]

沿图 4-2 中的箭头可以区分两种等待。一种是数据尚未到达，例如输入块仍在传输；另一种是前一项计算尚未结束，例如 Softmax 正在等待查询与键的点积（QK）完成。增加带宽能够缩短第一类等待，改变分块或执行顺序能够改变第二类等待。后续各节将分别展开这两种机制。

这些部件各分到多少资源，并非一次确定。模型与芯片在一代代设计中相互影响。现有加速器的能力影响模型的状态大小、矩阵形状和通信方式，软件通过压缩、融合和并行提高执行效率；当某种负载长期占据较大比重，芯片设计者便会考虑为它增加专用能力。新一代芯片投入使用后，不同模型的运行成本又会随之改变。设计和制造都需要时间，所以硬件回应的是此前已经观察到、并且预计会延续的需求。第 4.6 节将沿三家架构的代际变化，具体展开图 4-3 中的每一段关系。

这种反馈已有公开的实例。DeepSeek V3 报告针对 H800 上的执行瓶颈，提出通信卸载、跨互联的一致操作、低精度累加和量化支持等硬件需求；其中，通信卸载源于部分 SM 被通信占用。[4]

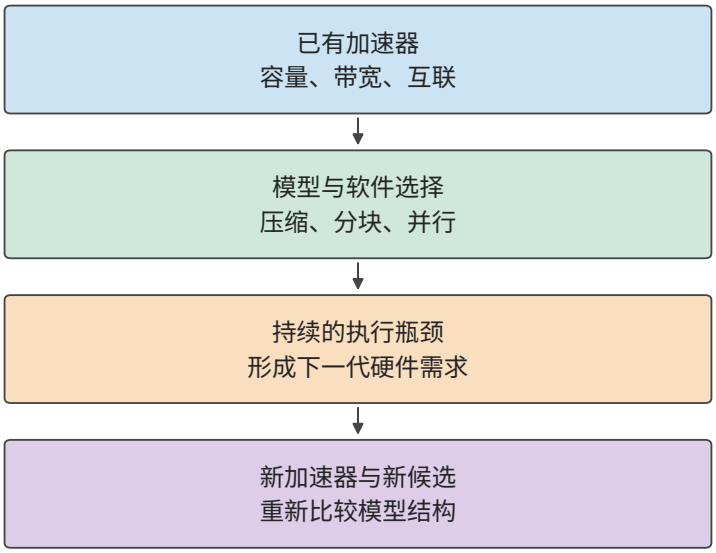



图 4-3　模型与硬件跨代协同。实线沿时间向下：现有加速器影响模型选择，软件运行暴露长期瓶颈，硬件设计回应这些需求，新加速器使更多模型方案成为可能。

### 4.1.3 芯片面积、功耗与封装的限制

计算单元、存储器和接口都占用芯片面积，也都消耗电能。分给某一部分的资源越多，留给其他部分的预算就越少。因此，评价一项局部改进，要看整个任务能缩短多少时间。

将开篇算例推广：以 $f$ 表示串行任务中可加速部分的时间比例，$s$ 表示其加速倍数，根据第 1.2.1 节的 Amdahl 定律，整体加速比为：

$$S = \frac{1}{(1-f) + f/s} = \frac{1}{0.4 + 0.6/2} \approx 1.43.$$

[3]加速器架构与执行比较。

[4]DeepSeek V3 技术报告第 3.5 节，Suggestions on Hardware Design。

继续把矩阵速度提高到原来的四倍，总时间降至 $60/4 + 40 = 55$ µs。第一次翻倍节省 30 µs，第二次只节省 15 µs；即使矩阵计算时间趋近于零，其余工作仍需 40 µs。整体加速上限为 2.5 倍。剩余工作在新总时间中所占比例越来越大，资源投入的重点也随之改变。

早期 TPU 的面积分配反映了计算与存储之间的实际取舍。TPU 是 Google 设计的张量处理器。第一代 TPU v1 具有 $256 \times 256$ 个 8-bit 乘加单元、24 MiB 的统一数据缓冲（Unified Buffer）和 4 MiB 累加存储。TPU v1 论文给出的芯片面积分布中，数据缓冲约占 37%，计算单元约占 30%，控制电路约占 2%。缓冲占用的面积甚至超过计算单元，因为阵列需要反复使用输入、权重和部分和。如果每次都从片外读取，乘加器就会长时间等待数据。[5]

假设为了增加乘加器，设计者缩小了缓冲容量。原来能留在缓冲中的输入块被提前移除，下次使用时要再次加载；重复读取增加的时间，可能抵消新增乘加器带来的收益。反过来，为极少重复使用的数据配置过大的缓冲，也会占用本可用于计算的面积。比较设计方案时，要计算同一任务在改动前后的总时间，把计算加快与重复读取增加的影响一起考虑。

> **思考：继续增加矩阵算力，还是加速其余工作？**在上述 60 µs 加 40 µs 的任务中，以原始矩阵运算速度为基准，把加速比从两倍提高到四倍，和把其余工作从 40 µs 降到 25 µs，哪种改动节省的时间更多？如果两者面积代价不同，应怎样选择？

面积之外，功耗是另一项预算；芯片越大、卡越多，功耗越容易成为决定性的约束。下面用四个推导把功耗变成可以计算的量：数据搬移为什么耗电，各级存储每搬一个字节耗多少能量，一颗芯片能做多大、能接多少内存，以及功率上限如何压低实际运行的频率。

**搬移为什么耗电。**搬移数据的能耗来自给连线充放电。数字电路的功耗分为静态与动态两部分：静态功耗是与翻转无关的漏电，动态功耗产生于信号在 0 与 1 之间的翻转。据何庭波的 LogicFolding 论文，智能手机典型负载中动态功耗约占九成。[6] 一条信号线连同其驱动的门可以看成一个电容 $C$，从 0 充到电压 $V$，电源要提供 $CV^2$ 的能量。设每个时钟周期中发生翻转的信号所占比例为 $\alpha$、时钟频率为 $f$（这里的 $f$ 是频率，不是前文 Amdahl 定律中的比例），动态功耗为

$$P_{\text{dyn}} \approx \alpha C V^2 f.$$

在先进工艺下，$C$ 的主要部分不是晶体管的栅极，而是连线：金属线的电容沿长度分布，信号走得越远，要充放电的电容就越大。图 4-4 上半部分画出这一机制：同一个门驱动一条数百微米的横向连线，要给沿途全部分布电容充电；把电路折叠到上下两层后，原来的横向路径变成几微米的垂直连接。论文报告折叠后典型核心的连线长度缩短 20%，部分关键路径缩短 70%，一个处理模块的时钟缓冲器从 43,600 个减到 19,000 个。

论文中的两个例子可以用这条公式直接手算。第一个是 NPU 的同性能比较：同样完成 29 TOPS（每秒 29 万亿次运算）时，折叠后的 NPU 把电压从 0.85 V 降到 0.55 V。只看电压项，

$$\frac{P_{\text{after}}}{P_{\text{before}}} = \left(\frac{0.55}{0.85}\right)^2 = \frac{121}{289} \approx 0.42.$$

只看频率项，频率降低 63%，$f$ 变为原来的 0.37。电压项与频率项相乘得 $0.42 \times 0.37 \approx 0.16$，远低于论文报告的总功耗比 0.34（下降 66%）。差距来自并行度：前代 NPU 是一个大核心加两个小核心，折叠后是四个大

[5] Jouppi 等，*In-Datacenter Performance Analysis of a Tensor Processing Unit*，ISCA 2017，归档论文。

[6] 何庭波，*Huawei's τ Chip Was Supposed to Melt?*，ChinaXiv:202609.00031v1，2026-09-04，归档论文，§II—V 与图 1—2：动态功耗占比、连线电容、NPU 同性能比较（29 TOPS、0.85→0.55 V、频率降 63%、功耗降 66%）、DSP 功耗降 25% 与投影面积降 40%。

核心（论文 §V），靠更多并行核心在低频下完成同样的 29 TOPS。核心增多，每周期翻转的电容 $\alpha C$ 随之增加，抵消了连线缩短带来的 $C$ 减少。电压、频率与并行度三项合起来才得到 0.34，其中电压项贡献最大，这也是论文认为降压的贡献大于连线缩短的原因。

第二个例子是 DSP（数字信号处理器）：第一代折叠让功耗降到原来的 0.75，投影面积（芯片在平面上占的面积）降到 0.60，于是单位面积的功耗，即功率密度，变为

$$\frac{0.75}{0.60} = 1.25,$$

比原来高 25%，与论文报告的 24% 一致。热量要从芯片面积上散出去，所以总功耗下降，散热压力未必减小。图 4-4 下半部分并列展示这两个例子的数字。

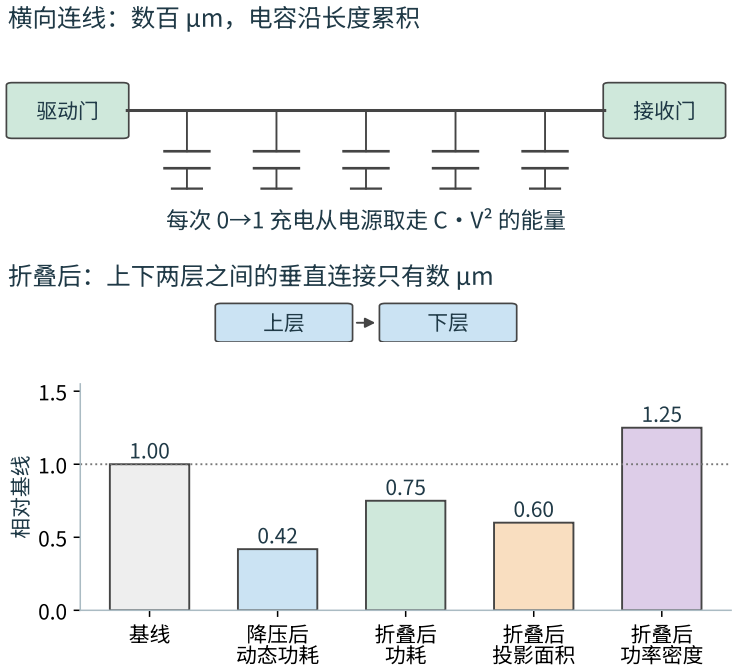



图 4-4　上：同一驱动门经数百微米横向连线传送信号，要给沿线分布电容充电；折叠后改为数微米的垂直连接。下：电压从 0.85 V 降到 0.55 V 使动态功耗降到 0.42；功耗降到 0.75 而投影面积降到 0.60，功率密度反而升到 1.25。

翻转条件（使结论反过来的临界条件）：只有面积比不小于功耗比时，功率密度才不上升；本例面积比 0.60 小于功耗比 0.75，功率密度上升。

**能耗层次：每搬一个字节要花多少能量。**能耗取决于要充放电的电容，电容又随距离增长，所以同一个字节从远近不同的存储层次取来，能耗相差悬殊。下表列出各级存储与链路每字节的能耗：SRAM（静态随机存取存储器）做在芯片上，HBM 与 LPDDR（低功耗 DRAM）都是片外的 DRAM（动态随机存取存储器），NVLink-C2C 是 Grace Hopper 超级芯片中连接 CPU 与 GPU 的芯片间链路。单位 pJ 即皮焦耳（$10^{-12}$ J）。数值来自四份资料，表中一并列出各自的工艺与年份。[7]

[7]Dally，Hardware for Deep Learning，Hot Chips 2023 主题演讲第 12、51—52 页：第 12 页 HFMA 1.5 pJ、HMMA 110 pJ、指令开销约 30 pJ 与开销占比 2000% / 22%（45 nm）；第 51 页能耗表注明取自 Horowitz ISSCC 2014，工艺 45 nm；第 52 页三级存储 5 / 50 / 640 pJ 每 32 位字，未注明工艺；Fine-Grained DRAM，MICRO 2017§1—2，HBM2 3.97 pJ/bit 及其分解，DRAM 能耗模型按 28 nm；NVIDIA Grace Hopper Superchip Architecture In-Depth，2022-11-10，NVLink-C2C 1.3 pJ/bit。

| 层次 | 来源数值 | 每 byte 能耗 | 工艺与年份 |
|---|---|---|---|
| KB 级局部 SRAM | 5 pJ / 32 位字 | 1.25 pJ | 45 nm，Horowitz ISSCC 2014，Dally Hot Chips 2023 转引 |
| MB 级片上 SRAM | 50 pJ / 32 位字 | 12.5 pJ | Dally Hot Chips 2023 第 52 页，工艺未注明 |
| NVLink-C2C 链路 | 1.3 pJ/bit | 10.4 pJ | Grace Hopper 超级芯片，NVIDIA 2022 年 11 月技术博客 |
| HBM2 | 3.97 pJ/bit | 31.76 pJ | 28 nm DRAM 能耗模型，MICRO 2017 |
| LPDDR DRAM | 640 pJ / 32 位字 | 160 pJ | 45 nm，Horowitz ISSCC 2014 |
| 16 位浮点乘加（2 FLOPs） | 1.5 pJ / 次 | 0.75 pJ/FLOP | 45 nm，Dally Hot Chips 2023 |

HBM2 每 bit 的能耗大部分消耗在 DRAM 内部，而非通往 GPU 的连线上。die（裸片）是从晶圆上切下的一块硅片。按 MICRO 2017 论文的分解，DRAM die 内部从存储阵列到基底 die（HBM 堆最底层的 die）的数据通路占 2.24 pJ/bit，行激活（打开存储阵列中的一行）占 1.21 pJ/bit，经中介层（interposer，承载芯片之间连线的一块硅片）送到 GPU 的 I/O 只占 0.3 pJ/bit。三项合计 3.75 pJ/bit，计入 ECC（纠错码）等开销后为 3.92 pJ/bit；该论文正文给出的总数是表中的 3.97 pJ/bit。NVLink-C2C 的 1.3 pJ/bit 则只计链路本身，经它取来的数据还要先从对侧内存读出。

把这张表用到第 4.8.3 节的生成任务上：Qwen3-8B、BF16、单请求、8K 上下文，一步 decode 读取权重 15.137 GB、KV 1.208 GB，合计 16.345 GB，矩阵运算量为 19.97 GFLOPs。按表中 HBM 行与乘加行计算，[8]

$$E_{\text{weights}} = 15.137 \times 10^9 \times 31.76\ \text{pJ} \approx 0.481\ \text{J}, \quad E_{\text{KV}} \approx 0.038\ \text{J}, \quad E_{\text{compute}} = 19.97 \times 10^9 \times 0.75\ \text{pJ} \approx 0.015\ \text{J},$$

合计 0.534 J，其中权重读取占 90%：单请求 decode 的能量几乎全部消耗在搬移上。HBM 行取自 28 nm 工艺的 HBM2 能耗模型，计算行取自 45 nm 逻辑工艺的乘加能耗，两行工艺不同，所以这笔账给出的是搬移与计算两类能量的量级比较。这一步的时间则受带宽限制：按 3.35 TB/s 读完这 16.345 GB 需要 4.9 ms，0.534 J 平摊到这段时间只有约 110 W。图 4-5 下半部分假设同样的 16.345 GB 全部来自某一层次：KB 级 SRAM 只需 0.020 J，MB 级 SRAM 0.204 J，NVLink-C2C 链路 0.170 J，HBM 0.519 J，LPDDR 则为 2.62 J。

带宽只决定一步多快，不改变一步耗多少能量：HBM 带宽提高一倍，这一步的时间减半，0.534 J 不变。降低能量只有两条途径：让字节从更近的层次取来，或者让同一份字节服务更多 token。batch 为 $B$ 时，权重读一次服务 $B$ 个 token，每 token 的能量为 $0.481/B + 0.038 + 0.015$ J；$B = 32$ 时约 0.068 J，是单请求的八分之一。因此，数据复用首先是能耗问题，其次才是带宽问题。第 4.8.1 节按时间求出的转折点由带宽与算力之比决定，这里的转折点由每字节能耗与每 FLOP 能耗之比决定，两者并不相同。

翻转条件：$0.481/B < 0.015$，即 batch 超过 32 后，每 token 的权重搬移能耗才低于计算能耗。

**光罩与封装：一颗芯片能做多大，能接多少内存。**光刻机一次曝光只能覆盖一块光罩（reticle，曝光用的掩

[8]能耗分账结果，读取单请求 8K decode 结果的字节数与 FLOPs，按上表每字节与每 FLOP 能耗相加；运行 `python3 calculations/calc.py energy-ledger --format md` 可以复算。

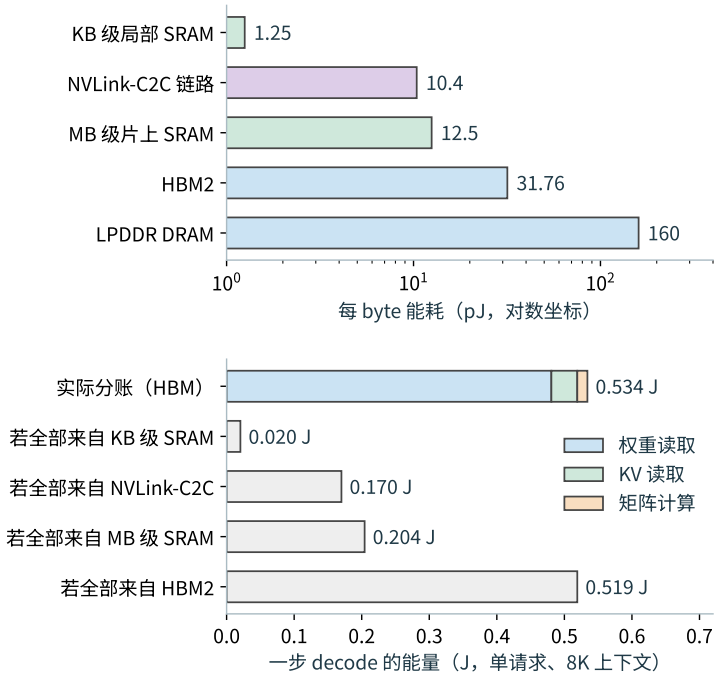



图 4-5　上：各级存储与链路每 byte 能耗，对数坐标。下：Qwen3-8B 单请求 8K decode 一步的能量分账（权重 0.481 J、KV 0.038 J、计算 0.015 J），以及同样 16.345 GB 全部来自某一层次时的能量。

模版）的图形范围，该面积称为光罩上限，单颗 die 不能超过它。H100 的 die 为 814 $\mathrm{mm}^2$，A100 为 826 $\mathrm{mm}^2$，两代都贴近这一上限；Blackwell 数据中心 GPU 的两颗 die 各自做到光罩上限，再用 10 TB/s 的 NV-HBI 链路连成一颗芯片。[9] 芯片分成两颗 die 后，计算放在哪一侧决定有多少访问要经过这条链路，第 4.5.1 节会具体计算。

HBM 把多层 DRAM die 叠在一颗基底 die 上，用硅通孔（TSV，穿过硅片的垂直导线）连接各层，整堆经中介层上的连线接到 GPU。每堆有 1,024 根数据引脚，引脚速率为 $r$ 时，每堆的带宽与容量为

$$R_{\text{stack}} = \frac{1024\, r}{8}, \qquad C_{\text{stack}} = \text{层数} \times \text{每层容量}.$$

以 HBM3E 为例：Micron 的八层堆每堆 24 GB、十二层堆 36 GB；SK hynix 给出的引脚速率 9.6 Gbit/s 代入上式得 $1024 \times 9.6/8 \approx 1229$ GB/s，与其产品页的 1.23 TB/s 一致。图 4-6 画出这种封装：die 居中，HBM 堆沿 die 边缘排列，中介层承载两者之间的连线。用同样的公式反推三代 GPU 的 HBM 配置：

[9] H100 架构白皮书表 4：五堆 HBM3、5120 位接口、2619 MHz DDR、3352 GB/s（本章正文统一采用数据手册取整的 3.35 TB/s，即 3,350 GB/s）、814 $\mathrm{mm}^2$、TSMC 4N 工艺，A100 826 $\mathrm{mm}^2$；Blackwell 技术简报第 7 页两颗光罩上限 die 与 10 TB/s NV-HBI，表 3 的 B200 192 GB / 7.7 TB/s；HGX 平台组件说明表 1，HGX B200 每 GPU 180 GB、最高 8 TB/s；H200 数据手册，141 GB / 4.8 TB/s；Micron HBM3E 产品页，1024 引脚、八层 24 GB、十二层 36 GB；SK hynix HBM 产品页，9.6 Gbit/s 与 1.23 TB/s。

| GPU | 容量 | 带宽 | 堆数 | 每堆容量 | 每堆带宽 | 引脚速率 |
|---|---|---|---|---|---|---|
| H100 SXM5 | 80 GB | 3,350 GB/s | 5，白皮书给出 | 16 GB | 670 GB/s | 5.24 Gbit/s，白皮书给出 2619 MHz DDR（双倍数据速率，每个时钟周期传输两次） |
| H200 SXM | 141 GB | 4,800 GB/s | 6 堆共 144 GB，24 GB 产品启用 141 GB | | 800 GB/s | 6.25 Gbit/s，推算 |
| B200（HGX） | 180 GB | 8,000 GB/s | 8 堆共 192 GB，24 GB 产品启用 180 GB | | 1,000 GB/s | 7.81 Gbit/s，推算 |

表中各代的增长都可以拆到堆数、每堆容量和引脚速率上。H100 的 5 堆构成 5,120 位接口，5,120 位乘以 5.24 Gbit/s 再除以 8，约为 3,350 GB/s。H100 到 H200，容量增长 76%，来自每堆从 16 GB 换成 24 GB、堆数从 5 增到 6；带宽增长 43%，来自多出的一堆和引脚速率从 5.24 Gbit/s 提高到 6.25 Gbit/s。H200 到 B200 又增加两堆：H100 与 H200 都只有一颗 die，分别接 5 堆和 6 堆；B200 的 8 堆沿两颗光罩上限 die 的外侧排列（图 4-6）。堆数由 Blackwell 技术简报中 192 GB、7.7 TB/s 的配置按每堆 24 GB 推得；本书其余各节采用 HGX B200 平台每 GPU 180 GB、8 TB/s 的规格。

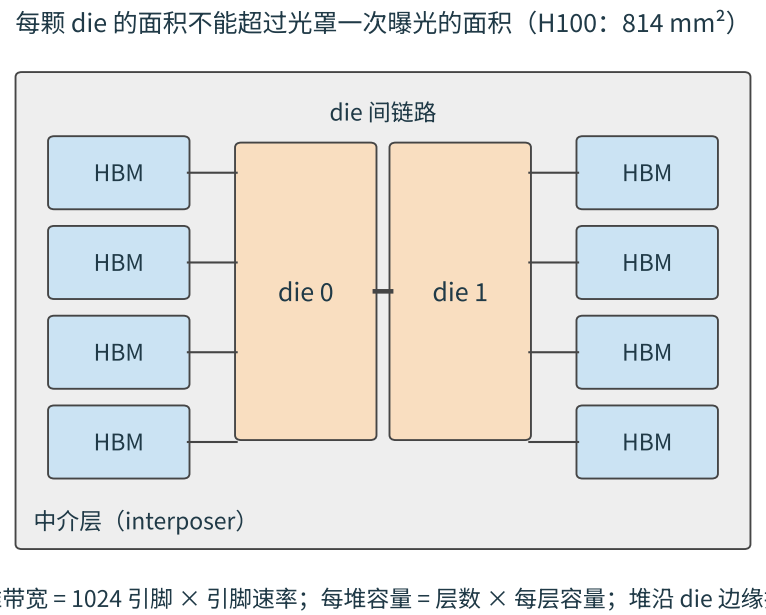



图 4-6　封装俯视示意：两颗达到光罩上限的 die 居中，八堆 HBM 沿两侧边缘排列，中介层承载 die 与 HBM 之间以及 die 之间的连线。每堆带宽由 1,024 根引脚与引脚速率决定，每堆容量由层数与每层容量决定。

翻转条件：所需堆数（目标容量除以每堆容量）超过一颗 die 所接的堆数时，要么增加每堆层数，要么增加 die；换成十二层 36 GB 的堆，六堆就有 216 GB，不必增加堆数。

**功率封顶：峰值算力为什么不能持续。**热设计功耗（thermal design power，TDP）是模块允许的最大持续功率，H100 SXM 为 700 W，B200 为 1,000 W。[10] 这份功率由内存与计算共用。以 H100 为例，按上表 HBM2

[10]H100 数据手册，SXM 最大热设计功耗 700 W；H100 架构白皮书表 4，BF16 稠密 989.4 TFLOPS 与 TDP 700 W；Blackwell 技术简

行的每字节能耗估算，HBM 满带宽的功率为 3.35 TB/s × 31.76 pJ/byte ≈ 106 W，扣除后留给计算的是 594 W；除以 BF16 稠密峰值 989.4 TFLOP/s，得到峰值速率下每 FLOP 的能量预算

$$b = \frac{700\ \text{W} - 106\ \text{W}}{989.4\ \text{TFLOP/s}} \approx 0.60\ \text{pJ/FLOP}.$$

0.60 pJ/FLOP 的预算只有靠矩阵指令与工艺进步才能满足，45 nm 的参考值说明了原因。一条标量半精度乘加指令（HFMA）做 2 FLOPs 耗 1.5 pJ，即 0.75 pJ/FLOP，本身已超过预算；取指、译码与取操作数另有约 30 pJ 的开销，是运算能量的 20 倍。一条 HMMA 矩阵乘加指令耗 110 pJ，完成的 FLOPs 远多于 2，同样的指令开销摊下来只占 22%。[11] H100 采用 TSMC 4N 工艺；从 45 nm 到 4N，每次翻转的电容与电压一代代降低。

设实际每 FLOP 能耗为 $e$。$e \le b$ 时，峰值频率可以持续；$e > b$ 时，功率控制器会降低频率，降到多少由 $P \propto fV^2$ 决定：电压不变时，功率随频率线性下降，持续频率为峰值的 $b/e$；电压随频率同步下降时，$P \propto f^3$，持续频率为峰值的 $(b/e)^{1/3}$。图 4-7 画出这两条曲线：$e$ 为预算的 1.5 倍时，前者降到 0.67，后者只降到 0.87。持续算力随频率等比例下降，所以按峰值算力预测的计算时间比实际短。峰值算力是硬件的物理极限，却不是在任何条件下都能达到：功率封顶时，极限本身随持续频率一起下移，此时衡量利用率应当以持续算力为分母。

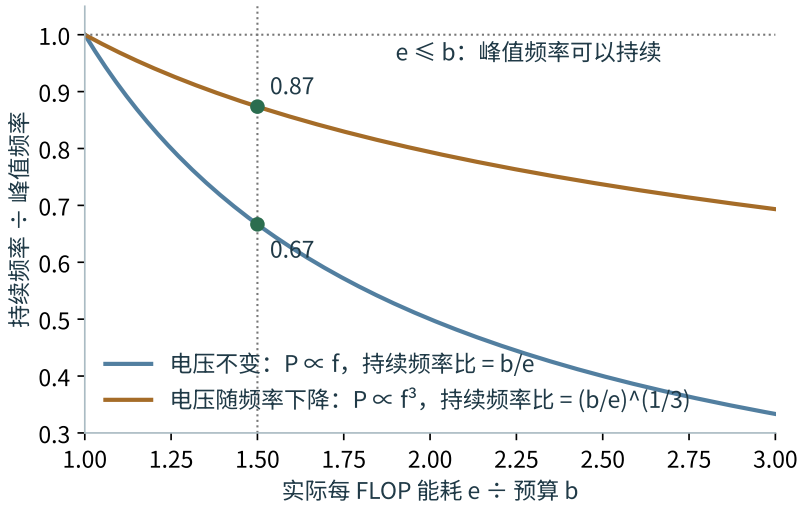



图 4-7　持续频率与峰值频率之比随每 FLOP 能耗 $e$ 与预算 $b$ 之比的变化：电压不变时为 $b/e$，电压随频率下降时为 $(b/e)^{1/3}$。$e/b$ 不超过 1 时峰值频率可以持续；$e/b$ 为 1.5 时两条曲线分别给出 0.67 与 0.87。

翻转条件：每 FLOP 能耗低于 $(\text{TDP} - P_{\text{mem}})/F_{\text{peak}}$ 时，峰值频率可以持续；其中 $P_{\text{mem}}$ 是内存功率，$F_{\text{peak}}$ 是峰值算力，本例这一阈值为 0.60 pJ/FLOP。工艺每前进一代，每 FLOP 能耗下降；TDP 也逐代抬高；两者共同决定这一条件何时满足。

## 4.2 计算单元

第 4.1 节用运算速率表示矩阵单元的性能。本节解释这一速率从何而来：一条指令处理多大的矩阵，为什么小矩阵难以充分利用计算单元，以及矩阵结果还要经过哪些向量与控制操作。

---

报表 3，B200 1000 W。

[11] Dally，Hardware for Deep Learning，Hot Chips 2023 主题演讲第 12、51—52 页：第 12 页 HFMA 1.5 pJ、HMMA 110 pJ、指令开销约 30 pJ 与开销占比 2000% / 22%（45 nm）；第 51 页能耗表注明取自 Horowitz ISSCC 2014，工艺 45 nm；第 52 页三级存储 5 / 50 / 640 pJ 每 32 位字，未注明工艺；Fine-Grained DRAM，MICRO 2017§1—2，HBM2 3.97 pJ/bit 及其分解，DRAM 能耗模型按 28 nm；NVIDIA Grace Hopper Superchip Architecture In-Depth，2022-11-10，NVLink-C2C 1.3 pJ/bit。

### 4.2.1 矩阵单元为什么适合神经网络

大矩阵要拆成许多小块计算，矩阵单元正是执行这些小块的硬件，其支持的矩阵指令形状有限，程序要把大任务组织成相应的小步；一张加速卡上的多个计算单元可以同时处理不同块。

计算一个 $m \times n$ 输出块时，同一行输入参与 $n$ 个输出元素的计算，同一列权重参与 $m$ 个输出元素的计算。矩阵单元让多个乘加单元共享这些操作数：数据在相邻计算单元间传递，部分和留在本地更新。一条指令描述整块工作，控制成本也由大量运算共同分摊。第 5 章将进一步说明，软件如何组织块大小和遍历顺序，让这些小步持续取得所需数据。

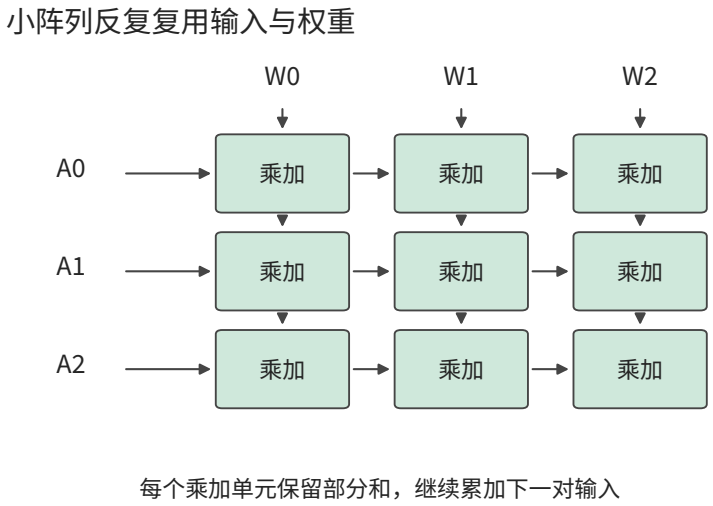



图 4-8　三行三列的乘加阵列示意。输入沿行传递，权重沿列传递，每个乘加单元保留自己的部分和。这里用小阵列解释操作数复用。

以一次 $16 \times 16 \times 16$ 的矩阵操作为例，两个输入块各有 256 个元素，完成 $2 \times 16^3 = 8192$ 次浮点操作。每个输入元素在块内参与 16 次乘法。与每个输出各自取一遍全部操作数相比，这种方式减少了重复搬移和指令开销。

矩阵指令通常按固定大小的块执行。设某种实现的最小计算块为 $m_t \times n_t \times k_t$，矩阵尺寸不足整块的部分用零补齐。有效运算占实际执行运算的比例为

$$\eta_{\text{shape}} = \frac{MNK}{\lceil M/m_t \rceil m_t \lceil N/n_t \rceil n_t \lceil K/k_t \rceil k_t}.$$

若行方向最小为 16，真实矩阵只有一行，其余维度均对齐，则 16 行中只有一行包含有效输入。把矩阵单元数量翻倍，会同时加快有效计算和补零计算；改用适合单行的矩阵向量乘法（GEMV）实现，则可以减少补零造成的无用计算。库如何选择实现，见第 4.8 节的实测。

专家模型让行数问题更加突出。设 64 个 token 各选择八个专家，共有 512 次分派；各专家矩阵尺寸相同。如果分派恰好均匀覆盖 256 个专家，每个专家只处理两个 token 的特征向量；如果全部 token 选择同样的八个专家，每个专家处理 64 个 token 的特征向量。两种情况下，64 个原始 token 都产生 512 次专家分派，在各专家输入矩阵中合计占 512 个有效行。

继续使用行方向按 16 补齐的实现。前一种分派执行 $256 \times 16 = 4096$ 行，其中 512 行有效；后一种执行 $8 \times 64 = 512$ 行，全部有效。两种分派的有效计算量相同，补齐后前一种的执行量却是后一种的八倍。一批 token 选中了哪些专家，既决定需要读取哪些权重，也决定每个专家处理多少行，从而影响矩阵单元的利用率。

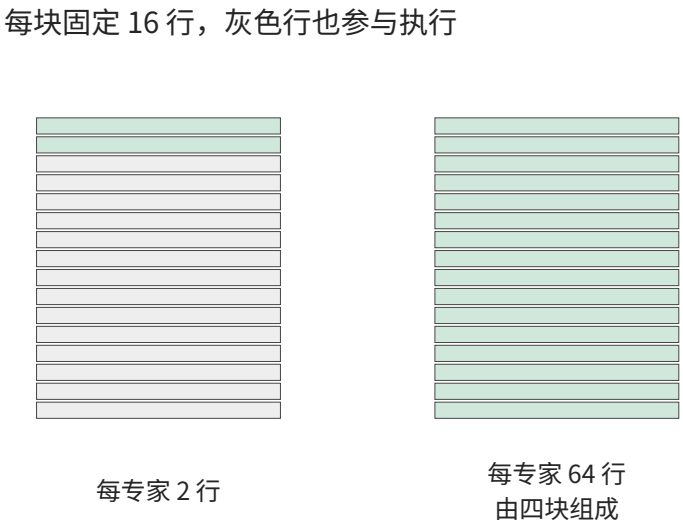



图 4-9　一个专家的 16 行计算块。每专家只有两行时，剩余十四行填零；每专家有 64 行时，可组成四个完整块，图中展示其中一块。

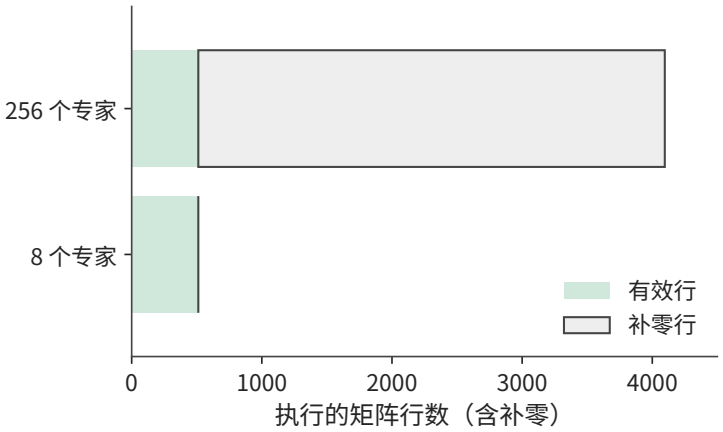



图 4-10　相同 512 行有效输入在全部专家上的执行量。分散到 256 个专家后总计执行 4096 行，集中到八个专家时只执行 512 行。

图 4-9 与图 4-10 中浅灰部分是补零带来的计算，增加乘加器也会加快这些无用操作。减少浅灰部分则要改变每个专家的输入行数或选择更小的计算块。

这解释了为什么库为不同形状准备多种 kernel：大矩阵利用阵列复用，小矩阵需要更合适的执行粒度，多个小任务还可以通过分组减少调度空隙。Tensor Core 与昇腾 Cube 专门执行规则矩阵运算，通用执行单元保留对其他形状的处理能力；两者配合处理实际负载中大小不同的矩阵。[12]

### 4.2.2 向量与控制单元负责哪些工作

除了矩阵乘法，Transformer 层还包含归一化、位置编码和激活函数；专家模型还要做路由。这些步骤包含逐元素运算，也包含多个元素之间的依赖。

以一行有 $n$ 个分数的 Softmax 为例，为保证数值稳定，先求最大值，再计算平移后的指数，最后求和并归一化：

[12] A100 架构白皮书、H100 架构白皮书、Hopper Tuning Guide、Blackwell 技术简报与CUTLASS Blackwell 功能。

$$a = \max_j z_j, \qquad e_j = \exp(z_j - a), \qquad p_j = \frac{e_j}{\sum_k e_k}.$$

$n = 128$ 时，需要计算 128 个指数值，并对 128 个结果归一化；求最大值和求和则要分别合并 128 个输入。逐元素运算可以分配给多个计算单元并行执行；把多个元素合并为较少结果的操作称为归约，例如求和或求最大值；归约需要逐步合并各单元的结果。理想二叉归约经过 $\log_2 128 = 7$ 轮，每轮等待上一轮结果。增加并行计算单元可以缩短每一轮的处理时间，但各轮仍需依次进行。

向量单元承担这类逐元素和归约工作，特殊函数单元处理指数等操作，控制逻辑安排循环、地址、分支和同步。三者共同决定矩阵结果何时能交给后续计算使用。昇腾的 Cube、Vector、Scalar 命名直观地体现了这种分工；昇腾 950 白皮书中的一个 AI 子系统由一个 Cube Core 和两个 Vector Core 组成，向量部分采用以单指令多数据（SIMD，即一条指令对一组数据执行同样操作）为主、单指令多线程（SIMT，即把同一指令分派给一组各有执行状态的线程）为辅的执行方式。[13]

Softmax 的分数随输入变化，需要每次重新求指数；另一些非矩阵运算使用的系数只随位置变化，可以预先计算并反复使用。RoPE 使用固定位置的正弦、余弦系数，对一对分量执行

$$x_0' = x_0 \cos\theta - x_1 \sin\theta, \qquad x_1' = x_0 \sin\theta + x_1 \cos\theta.$$

Qwen3-8B 一层的 32 个 Q 头和 8 个 K 头，每头旋转 128 维，共有 $(32+8)\times 64 = 2560$ 对分量。每对需要四次乘法、两次加减，每个 token 共 15,360 FLOPs。各头在同一位置使用相同系数，因而可以共享一张表：为 1,024 个位置各保存 64 个 cos 和 64 个 sin，BF16 存储只需 $1024 \times 128 \times 2 = 256$ KiB。[14]

预计算把重复的三角函数求值转成系数读取，旋转本身仍按每个 token 执行。压力于是从特殊函数计算转到表访问与向量运算上。

### 4.2.3 NVIDIA、昇腾和 Apple 如何执行注意力计算

注意力把前两节的矩阵计算与向量计算连接起来：$Z = QK^{\mathsf{T}}$ 生成分数，$P = \mathrm{Softmax}(Z)$ 生成概率，$O = PV$ 形成输出。同一块数据必须依次经过这三步；多个块则可以在不同资源上交错推进。

| 工作 | NVIDIA GPU | 昇腾 | Apple GPU |
|---|---|---|---|
| 查询与键的点积 QK、概率与值的乘积 PV | Tensor Core；小矩阵可用通用计算单元 | 矩阵核 Cube（AIC） | Metal GPU 矩阵运算 |
| 指数、归约、缩放 | SM 的通用与特殊函数资源 | 向量核 Vector（AIV） | GPU 线程与相应库实现 |
| 中间状态 | 寄存器、共享内存、累加存储 | 计算单元近旁的零级缓冲（L0）、统一缓冲与交换通路 | GPU 缓冲与线程组存储 |

这一分工决定哪些操作可以同时执行。例如，一个块在处理指数时，矩阵单元可以处理另一个块；但双方若都要从同一个片上接口取操作数，该接口仍需依次提供全部字节。哪些单元能够并行工作、哪些操作共用同一接口，共同决定了流水线稳定运行时的吞吐率。表中的 Apple 路径指 Metal GPU。独立的神经网络加速单元 Neural Engine 与 GPU 内的专用单元在第 4.6.3 节分别介绍。[15]

[13] 昇腾 950 官方架构白皮书，§4.1—4.1.6；早期 DaVinci 与 CANN 分离架构的页级定位见比较笔记。

[14] 从算子利用率到执行中的等待及论文阅读记录。

[15] M2 Pro / Max 官方规格、Apple GPU 架构说明、Metal 存储模式、M5 GPU Neural Accelerator 官方说明。

FlashAttention 系列通过分块减少注意力中间结果的显存访问；第四版进一步研究更强矩阵单元下的计算分工。下面用 FlashAttention-4 对 NVIDIA Blackwell 架构 B200 单个 SM 的分析，计算这种分工对吞吐率的影响。取 $128 \times 128$、头维度为 128 的矩形注意力块，QK 与 PV 合计约 8.39 MFLOPs，指数结果有 16,384 个。按每个 SM 每周期 8,192 FLOPs 的矩阵速率和每周期 16 个结果的指数速率，两者各需 1,024 周期。矩阵操作数的共享内存读取为 96 KiB，按每周期 128 bytes，需要 768 周期。[16]

连续处理多个块时，矩阵单元、共享内存接口和指数单元可以构成流水线；处理每个块时，三者分别占用 1,024、768 和 1,024 周期。若新块到达的速度超过其中任一单元的处理速度，等待队列就会不断增长。因此，在流水线稳定运行后，相邻两块完成的时间间隔至少为

$$\tau \geq \max(T_{\text{matrix}}, T_{\text{smem}}, T_{\text{exp}}).$$

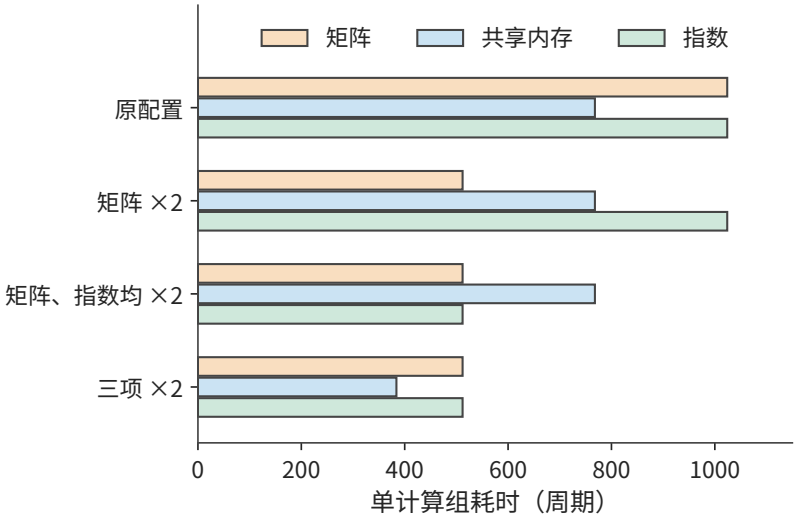



图 4-11　同一注意力块在四种资源配置下的服务周期。矩阵、共享内存和指数三项分别比较，单独增加一种能力后，其他资源可能成为较长的一项。“×2” 表示相应资源的吞吐能力提高一倍，横轴为完成同一计算块所需的时钟周期数。

**例 4-1：注意力流水的吞吐为何受指数运算与共享内存限制?**

**提高矩阵与指数吞吐率后，流水吞吐率能提高多少?** 多个块在独立资源间形成流水。比较两种配置：仅将矩阵吞吐率翻倍；在此基础上再将指数吞吐率翻倍。对每种配置，流水线稳定运行时，相邻两个块最短隔多久完成?

**由最慢单元确定相邻数据块的完成间隔。** 原配置为 $\max(1024, 768, 1024) = 1024$ 周期。只提高矩阵能力，得到 $\max(512, 768, 1024) = 1024$ 周期；再提高指数能力，得到 $\max(512, 768, 512) = 768$ 周期。

**矩阵加速后，瓶颈向指数运算与共享内存转移。** 第一项改动使矩阵单元更早完成，但指数运算的吞吐率保持不变，所以完成时间间隔的下界不变。第二项改动消除了指数的限制，共享内存成为耗时最长的单元，对应的理想吞吐提高到原来的 $1024/768 = 4/3$ 倍。

**共享内存带宽翻倍后，瓶颈转移到哪些单元?** 若共享内存带宽也翻倍，三项变为 512、384、512 周期，下界才降到 512。新的均衡点由矩阵与指数共同决定。

提高一种资源的处理能力，可以减少等待，直到另一种资源成为瓶颈。对单个块，QK、Softmax、PV 仍有先后关系；要达到上述完成时间间隔，需要同时保存不同进度的多个块。第 4.4 节将一并计算所需缓冲与时序。

[16] *FlashAttention-4*，MLSys 2026，论文，§2.2、§3.1.1、公式 1—3 与表 1；单 SM 独立复算。

**实验 4-1 ・核心：注意力分块如何改变运算量与共享内存读取量**

将矩形块改为 $128 \times 256$ 和 $256 \times 128$，分别计算矩阵运算、指数运算和共享内存读取所需的时间。再逐项提高矩阵运算吞吐率、指数运算吞吐率和共享内存带宽，说明采用的提升幅度，并找出每次改动后耗时最长的一项。解释块形状怎样改变数据复用。最后选一种硬件，将三类工作与存储位置标在执行图上。

### 4.2.4 低精度如何改变计算量、数据量与转换开销

矩阵形状决定做多少乘加，数值格式决定每次乘加使用什么表示。分析低精度需要依次回答三个问题：数据占多少空间，执行时经过哪些变换，误差要求允许采用哪些变换。

先看表示。若 $n$ 个权重各存 $b$ bit，每 $g$ 个值共享一个 $s$ bytes 的 scale，总元素数恰好能被分组大小整除时，存储量为

$$V_W = \frac{nb}{8} + \frac{n}{g}s.$$

对第 4.1.1 节的 $4096 \times 4096$ 权重，每个值用 4 bit 保存，每 32 个值共用一个占一字节的 scale：权重数据为 8 MiB，scale 为 0.5 MiB，共 8.5 MiB，约为 BF16 所需 32 MiB 的 1/3.8。若每 16 个值共用一个 scale，总量增为 9 MiB。分组越细，scale 越容易适应该组数值的范围，但需要保存的 scale 也越多。

再看执行。压缩值 $q$ 与 scale $a$ 一起表示近似值 $\hat{w} = aq$。可以先展开成 BF16 再做矩阵乘法，也可以让低精度矩阵指令处理 $q$，随后按 scale 调整结果。图 4-12 并列展示这两种实现：前者先花时间展开权重，并保存高精度副本，后者把缩放和合并放进执行过程。

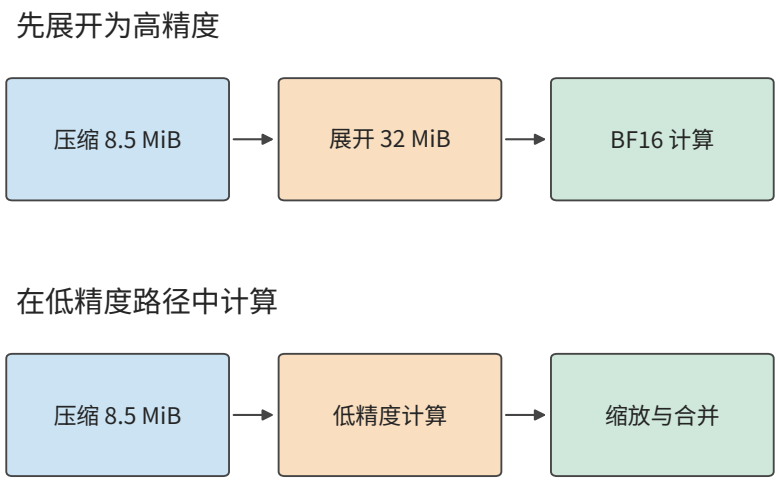



图 4-12　相同压缩权重的两条计算路径。先展开会形成 32 MiB 的 BF16 副本；低精度路径在计算过程中完成缩放和合并。压缩权重及 scale 共 8.5 MiB。

FP4、FP8 与八位整数格式 INT8 已进入实际模型的 kernel：DeepSeek V4-Flash 的一种路由专家实现先将 FP4 权重转换为 FP8，再执行 GEMM。权重以 FP4 保存和读取，减少了传输字节数；转换后，矩阵单元按 FP8 格式计算。若展开后的权重需多次使用，一次转换的时间就能分摊到多次调用；若每次调用都重新展开，转换

时间就成为每次任务的固定部分。低精度能节省多少时间，取决于节省的读取时间与增加的转换时间之差。[17]

配套实验同时测量误差与执行开销，选用 Qwen3-VL-30B-A3B 的一个 $2048 \times 1536$ 专家矩阵，比较两种方法：直接用 INT8 分块计算，以及先展开为 BF16、再完成一次矩阵乘法。实验在 RTX PRO 6000 上运行，输入来自实际专家路由。向量的 L2 范数是各元素平方和的平方根；用输出误差的 L2 范数除以参考输出的 L2 范数衡量局部相对误差，门槛设为 2%。逐行激活缩放的误差约为 2.3%—3.5%；将激活沿 K 维每 128 个元素分别缩放后，误差降为约 1.5%—1.9%。[18]

更细的量化分组改变了点积的累加方式。对一行输入和一列权重，设激活每组 scale 为 $a_g$，权重列 scale 为 $b_j$，点积可以写成

$$y_j \approx \sum_{g=1}^{16} a_g b_j \left( \sum_{k \in g} q_{x,k} q_{w,kj} \right).$$

由于每组的 $a_g$ 不同，计算时先求各组的整数部分和，再分别乘 scale，最后将结果相加。因此，这种实现需要调用 16 次矩阵乘法，并在调用之间完成转换、缩放和合并。输入行数较少时，还要补齐到 kernel 要求的最小行数。另一种方法先将各组展开为 BF16，再通过一次 GEMM 完成整个 K 维的累加。

以真实单 token decode 输入为例，两种方法的总执行时间分别约为 2.81 ms 和 2.52 ms，先展开再计算的方法较快。显存占用则正好相反：内存分配器在后续调用中记录的显存占用峰值分别约为 11.4 MiB 和 35.2 MiB，直接计算的方法节省约三分之二。[19] 更细的量化分组降低了误差，分组调用增加了时间，省去高精度副本减少了显存占用；数值格式与计算方法由此共同决定了误差、执行时间和显存占用。

因此，改进这种实现的方向是把组内计算、缩放和累加融合起来，在降低量化误差的同时减少 kernel 启动开销。

> **实验 4-2 ・延伸：量化权重何时展开，才能减少重复执行成本**
> 从上述分组点积式出发，比较先展开权重再计算与直接分组计算两种方法，分别列出需要保存的数据，包括权重、scale、展开后的权重副本、部分和与输出。计算加载时展开一次、每次调用展开、直接分组计算三种策略在重复使用 $R$ 次后的总时间。再用实际输入的实验记录比较误差与显存占用峰值，解释何时值得多占用内存来保存展开后的权重，以减少重复展开的开销。

## 4.3 存储层次

第 4.2.4 节说明了数值格式如何改变数据大小。本节分析 Q 投影的数据读写，区分三个问题：这些数据能否放下，重复使用时从哪一级读取，独立访问是否足以维持目标带宽。

### 4.3.1 显存与统一内存

模型运行时，权重、KV 缓存与临时工作区同时占用内存。权重占用相对固定，KV 随请求数和上下文长度增长，工作区保存执行中的中间结果。先确定这三项，才能决定加速器能否再接收一个请求。

Qwen3-8B 的 BF16 权重约占 16.4 GB。模型有 36 层，每层有 8 个 KV 头，每头 128 维；K 和 V 各保存一份，每个元素两字节。因此，每个上下文 token 的 KV 占用为

[17] 硬件精度审查、DeepSeek V4-Flash 与 Qwen 逐阶段资源条件、低精度与执行路径调研。硬件演进与成本归因见成本下降调研。

[18] 实验 4-2、真实路由激活、128 元素分组、单专家模型内替换。

[19] 实验 4-2 分配器峰值。

$$m_{\mathrm{KV}} = 36 \times 2 \times 8 \times 128 \times 2 = 147\,456 \text{ bytes} = 144 \text{ KiB}.$$

每条请求保留 $S$ 个位置、同时服务 $B$ 条请求时，容量条件为

$$W_{\mathrm{resident}} + BSm_{\mathrm{KV}} + M_{\mathrm{workspace}} \le C_{\mathrm{available}}.$$

按 $B$ 求解，就得到固定长度下的最大请求数：

$$B_{\max} = \left\lfloor \frac{C_{\mathrm{available}} - W_{\mathrm{resident}} - M_{\mathrm{workspace}}}{Sm_{\mathrm{KV}}} \right\rfloor.$$

以 RTX 4090 的 24 GB 显存为可用容量，留 2 GiB 工作区。扣除权重后，约 5.47 GB 用于 KV。$S = 8192$ 时每条请求需要 1.125 GiB，约 1.21 GB，四条请求的 KV 共约 4.83 GB，可以容纳；增加到五条后，共需约 6.04 GB，超过剩余容量。上下文翻倍后，每条请求 KV 翻倍，最大请求数降为两条。[20]

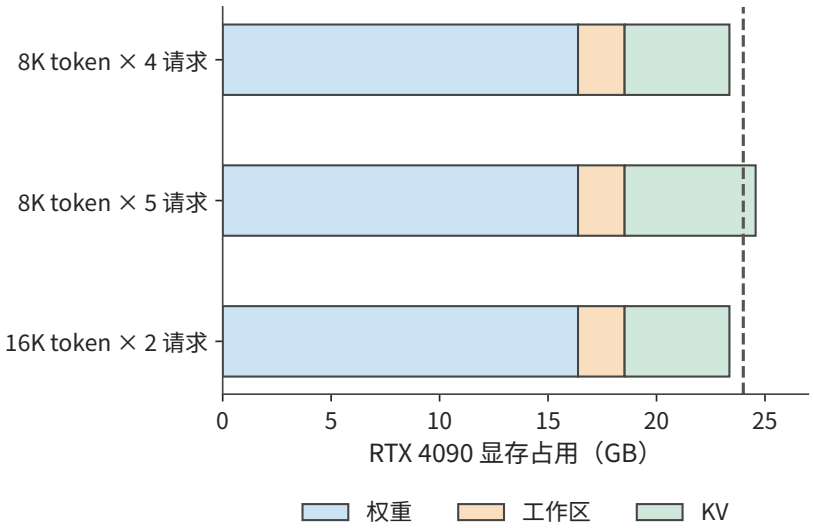



图 4-13　RTX 4090 的 24 GB 显存中的权重、工作区和 KV。8K 四请求与 16K 两请求可以容纳，8K 五请求超过虚线标出的容量上限。

图 4-13 中，权重与工作区的宽度保持不变，变化发生在右侧的 KV 区域。长上下文让每个 KV 块变宽，增加并发则增加块数；二者争用同一份剩余空间。

最大请求数的取整公式揭示了容量的阶梯效应。增加少量内存时，$B_{\max}$ 可能保持不变；只有剩余空间足以容纳一条新请求的全部 KV，才能多接收一条请求。压缩权重会增大公式分子中的剩余空间，因此也能容纳更多请求。对权重较大的模型，减少固定占用可明显提高并发；对长上下文任务，KV 已占较大比例，继续压缩权重的收益会逐渐减小。

降低 KV 位宽也能释放容量。第 2 章的 DeepSeek V4.1-Flash 同时使用全局历史与 SWA 局部窗口。一条主 KV 记录的 512 个值采用 FP4，连同分组 scale 共占 288 bytes；局部窗口的一条记录采用 FP8，连同 scale 共占 528 bytes。主 KV 在参与注意力乘法前反量化，即利用 scale 恢复为计算所用的数值格式。存储格式压缩

[20] Qwen3-8B 配置；存储代际完整结果。

了常驻状态，执行时再恢复操作数。[21]

以上只计了推理时的状态。训练还使用梯度、优化器状态和为反向传播保存的激活。按照第 3 章介绍的张量生命周期计算这些状态的占用：在某一时刻仍需保存的所有张量共同决定内存占用峰值，已经释放的中间结果腾出空间给后续工作。容量规划因而既与每份张量大小有关，也与执行顺序有关。

显存容量最终由内存器件与接口提供。HBM 通过堆叠存储 die 和宽接口提供带宽；图形双倍数据速率内存（GDDR）通常以多颗存储芯片配合高速接口提供显存。两者都是内存器件与接口技术，而统一内存是处理器共享内存的组织方式。Apple 的 CPU 与 GPU 访问共享物理内存，两者可以先后使用同一个缓冲区，减少独立副本；相应地，CPU、GPU 与其他应用也共同消耗这份容量与带宽。[22] 第 4.5 节会沿主机到加速器的路径分析这种组织对传输时间的影响。

> **思考：KV 压缩后能否再接纳一条请求？** 若每条请求 KV 需求从 1.21 GB 降低 10%，RTX 4090 能否容纳第五条请求？先求五条请求的总需求，再比较剩余空间。

### 4.3.2 缓存、片上缓冲与寄存器

算一块输出时，输入中的同一个数往往还要参与邻近输出的计算，尚未算完的部分和也要继续累加。如果每做一次乘加都回到片外取数、写回结果，就会产生大量往返。把常用数据留在计算单元附近，如同把当前要用的材料放在手边的工作台上，可以减少这些搬移；但近处的空间有限。

寄存器保存线程与指令正在使用的值，缓存自动保留最近访问的数据，软件管理的缓冲则由程序明确安排加载和释放。三者都能保存反复使用的数据，但容量、访问方式与管理者各不相同。片上存储还要占用芯片面积，容量通常比片外内存小得多。下面跟踪两块输入和一块部分和，计算所需空间。

考虑 $m = n = 128, k = 64$ 的乘加块。BF16 输入与权重块各为 16 KiB，合计 32 KiB；FP32 输出累加器为 $128 \times 128 \times 4 = 64$ KiB。V100 每个 SM 最多可把 96 KB（即 96 KiB）配置为共享内存，若操作数与累加器都放在其中，恰好用满。[23] 要同时准备下一对操作数，便需要 $2 \times 32 + 64 = 128$ KiB。

若累加器使用独立存储，同样的 96 KiB 就能容纳三组配对的输入块与权重块。局部缓冲的总容量没有增大，但存储分工改变了可以提前准备的输入块数。第 4.6.1 节将用 Blackwell 的专用累加存储说明这种分工。

前例把计算块的部分和留在片上，省去了反复写回。同样的思路也适用于多个计算块共同使用的输入。设四个输出块都要读取同一份 32 KiB 输入，它们共发出 128 KiB 的逻辑请求。L2 是计算单元之间共享的二级缓存。若第一次读取把数据留在 L2，后面三次命中，则片外只需提供 32 KiB，但 L2 仍要向各块提供全部 128 KiB。若四个输出块由同一计算组顺序处理，并将输入留在局部缓冲，后续访问就可以直接从局部缓冲取得数据。

因此，分析同一份数据时，需要区分三个量：数据本身占用的空间、各计算块请求读取的字节数、某一级接口实际传输的字节数。算术强度也随观察层次变化。用 L2 请求量计算的强度要与 L2 带宽一起使用，用 DRAM 流量计算的强度则与片外带宽一起使用。第 4.8 节将用实际计数展示这种差别。

增大计算块通常能增加数据复用，但也需要更大的缓冲。RTX 5090 与 RTX PRO 6000 采用 SM120 架构，每个 SM 的一级数据缓存与共享内存合计 128 KB，其中最多 100 KB 可配置为共享内存，由驻留在该 SM 上的计算组共用：每个计算组需要 32 KiB 时，可以驻留三组；每组扩到 64 KiB 后，只能驻留一组。[24] 一组等待

[21] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

[22] M2 Pro / Max 官方规格、Apple GPU 架构说明、Metal 存储模式、M5 GPU Neural Accelerator 官方说明。

[23] Volta 架构白皮书，Tensor Cores 与混合精度章节。

[24] Blackwell Tuning Guide、CUTLASS Blackwell 功能说明及 SM100 TMEM 示例；SM120（计算能力 12.0）每个 SM 的 128 KB 一级数据缓存与 100 KB 共享内存上限见 CUDA 编程指南的计算能力表。

数据时，另一组可以继续计算。因此，同时运行的组数减少后，计算单元更容易闲置。选择块大小时，要同时考虑每组能减少多少重复读取，以及还能容纳多少组相互独立的计算。

片上缓冲容量也随代际增加。A100 每个 SM 的共享内存容量上限为 164 KB，H100 为 228 KB。[25] 更多计算组驻留在片上，也能发出更多相互独立的读取请求。

### 4.3.3 容量、带宽与延迟分别限制什么

容量是某一时刻能够容纳多少数据，带宽是单位时间传输多少数据，延迟是一次请求从发出到完成经过多久。同时发出足够多的独立请求，则可以在等待一次访问返回时继续处理其他访问。

模型驻留和逐步执行对内存提出的要求不同。按第 4.1.3 节的 HBM 配置表，H100 SXM 为 80 GB、3.35 TB/s，H200 SXM 为 141 GB、4.8 TB/s，容量与带宽的增幅并不相同。Qwen3-235B-A22B 的一组 4-bit 方案中，权重与 scale 约为 123.1 GB，八条请求各保留 8,192 个 token 的 KV 约为 12.6 GB，加 2 GiB 工作区，共约 137.9 GB。H100 的容量缺口接近 58 GB，H200 则有约 3.1 GB 余量。容量增加，首先是让这些数据能够同时驻留。[26]

全部专家的权重都保存在内存中，而每步需要读取哪些权重由路由结果决定。八条请求的专家选择分散时，权重、scale 与 KV 的每步访问约为 75.967 GB；集中选择相同专家时，降至约 24.737 GB。在 H200 上，仅传输这些字节分别需要约 15.8 ms 和 5.2 ms；改用 8 TB/s 的 HGX B200，分别为约 9.5 ms 和 3.1 ms。专家复用把流量减少到约三分之一，加速器升级则把传输一个字节所需的时间减少到原来的约六成，两种改动作用在公式 $T = V/R$ 的不同位置。

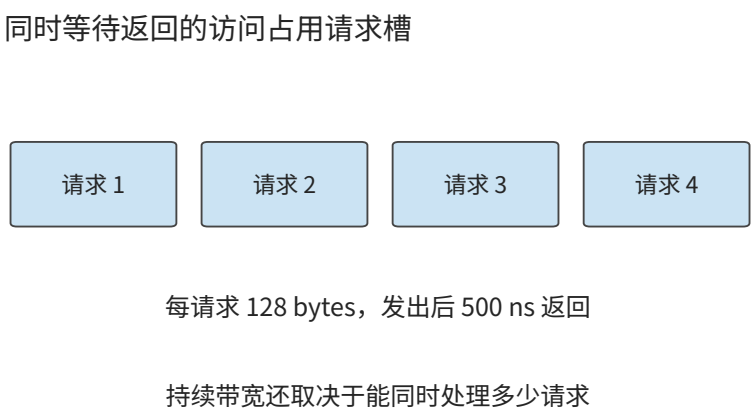



图 4-14　读取请求从发出到返回一直占用请求记录空间。多个独立请求交叠，才能在单次访问等待期间持续利用接口；图中只画四个代表请求。

上述 $V/R$ 计算假定接口可以持续达到给定带宽。要维持这种速度，就必须在一次访问等待返回时继续发出其他请求。将内存接口一次传输的基本单位称为访存事务。设接口中最多同时有 $N_o$ 个在途事务，每个事务带回 $s$ bytes，平均完成延迟为 $L$。一个事务从发出到返回期间占用一个位置；每秒最多完成约 $N_o/L$ 个事务，于是

$$R_{\text{effective}} \le \min\left(R_{\text{interface}}, \frac{N_o s}{L}\right).$$

把式子按 $N_o$ 求解，维持目标吞吐需要

[25] 从负载变化理解芯片演进。

[26] 存储代际比较结果及计算说明。

$$N_o \geq \left\lceil \frac{R_{\text{target}} L}{s} \right\rceil .$$

这就是 Little 定律：平均在途请求数等于请求完成速率乘以平均请求延迟。硬件队列决定能容纳多少在途事务，而程序中相互独立的访问决定能否用满队列。若下一地址依赖上一读取结果，队列即使很深也无法填满；多个独立计算块则能提供互不等待的读取。

**例 4-2：换上带宽更高的显卡，KV 读取也会同比加快吗？**

**比较在途事务数与接口带宽对 KV 读取的限制。**读取一条请求的 8,192 个上下文 token 所对应的 KV，共 1.125 GiB。显存接口取 RTX 4090 的 1,008 GB/s，每事务 128 bytes，完成延迟取 500 ns：内存性能测量工具 Mess 在 H100 上测得的显存空载延迟为 363 ns，接近饱和带宽时升到 699—1,433 ns，500 ns 位于二者之间。[a] 分别计算最多允许 128 和 4,096 个在途事务时的有效读取带宽与耗时，再比较换成 RTX 5090 的 1,792 GB/s 后的结果。

**由在途字节数和访问延迟求有效读取带宽。**维持 1,008 GB/s 需要 $\lceil 1.008 \times 10^{12} \times 500 \times 10^{-9}/128 \rceil = 3938$ 个事务。128 个事务只能提供约 32.8 GB/s，读取时间至少为 36.9 ms；4,096 个事务足以维持 1,008 GB/s，时间降为约 1.20 ms。

**在途事务不足时，更高的接口带宽收益有限。**换成 RTX 5090 后，4,096 个事务仍只能支持约 $4096 \times 128/(500\text{ ns}) = 1.05$ TB/s，时间约为 1.15 ms。接口带宽提高了约 78%，吞吐率却受在途事务数限制，只提高约 4%。

**用满更高带宽或应对更长延迟，需要多少在途事务？**把在途事务数提高到至少 7,000 个，才能维持 1,792 GB/s；若延迟增至 800 ns，该要求又增至 11,200 个。[b]

[a] *Mess*，MICRO 2024，作者接受稿，选读物理页 3—6；访存并发算例及限制。

[b] RTX 4090，128 个在途事务、RTX 4090，4,096 个、RTX 5090，4,096 个、RTX 5090，延迟 800 ns；Mess 的 H100 延迟见其表 I 与图 3(h)。

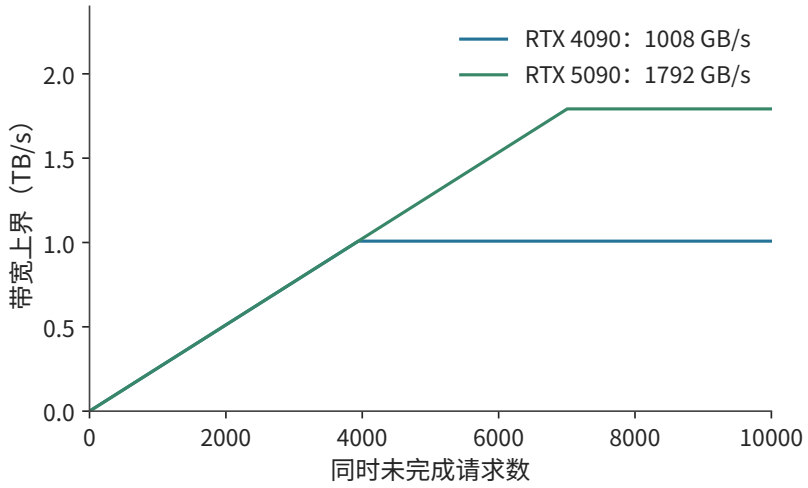



图 4-15　每事务 128 bytes、返回延迟 500 ns 时，增加在途请求数会提高带宽上界，直到碰到 RTX 4090 或 RTX 5090 显存接口本身的速率上限。

图 4-15 的两条曲线在并发较小时重合，是因为限制来自在途字节数。越过各自拐点以后，接口带宽成为新的瓶颈。实际系统中，增加并发访问还会使队列变长，延迟也随负载变化；Mess 用延迟探针与可调背景流量同时观察这两项，得到带宽—延迟曲线。[27] 这条曲线把“同时发起独立请求可以减少等待期间的空闲”与“请求过

[27] *Mess*，MICRO 2024，作者接受稿，选读物理页 3—6；访存并发算例及限制。

多会增加排队时间”放进同一张图中。

分析存储性能时，先根据容量判断数据能否驻留，再根据复用情况计算各级接口的流量，最后结合并发量和延迟判断接口能否持续传输。

> **实验 4-3 ・延伸：内存扩容与带宽提升分别能改善哪些 decode 瓶颈**
> 使用 Qwen3-8B 与 Qwen3-235B-A22B 的存储结果，分别只增加容量、只增加带宽，再改变 batch 和专家分布。先求最大可驻留请求数，再计算每步读取时间，最后求维持目标带宽需要的在途事务数。说明每种方案首先受容量、传输时间还是访存并发数限制。

## 4.4 数据搬移

第 4.1—4.3 节算出了数据的大小与复用次数，也解释了维持带宽所需的并发。本节进一步安排这些数据何时取得、何时使用、何时释放。数据搬移的核心，是让下一块在计算单元使用它之前到达，同时保留当前块直到最后一次使用结束。

### 4.4.1 数据布局与寻址

在矩阵上画出一个小方块，只确定了要取哪些元素。这些元素在内存里未必相邻：若原矩阵按行存放，小方块的一行读完后，还要越过未选中的元素，到原矩阵下一行的对应位置继续读取。数学上的分块可以只改变索引与访问范围，不必先复制出一份独立的小矩阵。硬件需要根据步长找到这些元素。一个元素占 $b$ bytes、行步长为 $s_r$ bytes、列步长为 $s_c$ bytes 的二维视图，元素地址为

$$\mathrm{addr}(i,j) = \mathrm{base} + i s_r + j s_c.$$

矩阵按行连续存储时 $s_c = b$，行步长由完整行宽决定。取子矩阵时，子矩阵每行只占原矩阵一行的一部分，但相邻行起点之间的距离仍等于原矩阵的行步长。这决定了硬件要发起何种访问。

例如，从 BF16、行宽 4,096 的矩阵中取 128 行、每行连续 128 个元素。有效数据为 $128\times128\times2 = 32$ KiB，每行只有 256 bytes，而相邻行起点相差 8,192 bytes。第一行起点到最后一行终点跨越 $127\times8192+256 = 1\,040\,640$ bytes。逐行读取只传输各行的有效区间；如果相邻线程访问相邻元素，它们的请求还可以合并成较少的事务。

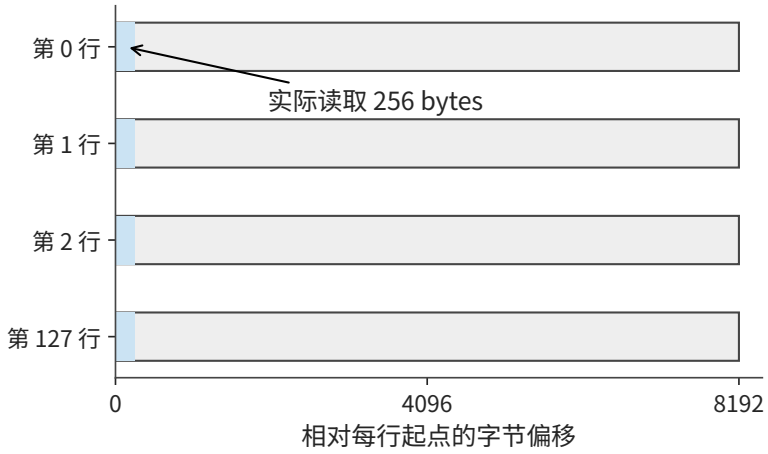



图 4-16　每行前 256 bytes 是实际读取区间，相邻行起点相差 8192 bytes。128 行合计读取 32 KiB，行间灰色区域由步长跳过。

蓝色区域每行只有 256 bytes，下一行的蓝色区域却要到 8,192 bytes 之后才开始。读取整个地址跨度会把灰色空隙也搬走，逐行读取则只取得蓝色部分。由此可见行步长的作用：它告诉搬移单元下一段有效数据从哪里开始。

数据布局从两方面影响访问开销：步长影响地址计算，地址的连续性影响多个访问能否合并为一次事务。相同的 32 KiB，有的排列能够用规则连续请求取得，有的排列要发出大量短请求。第 4.3 节中的每事务字节数 $s$，就取决于这些访问如何合并。

分块又决定同一地址会被请求多少次。DeepSeek V4-Flash 共享专家的一次 w1 计算为 $[32, 4096] \times [4096, 2048]$，FP8 输入只有 128 KiB。输出宽度分成 16 个、各 128 列的块，每个输出块都要遍历完整输入，所以块级输入请求合计为 $16 \times 128$ KiB，即 2 MiB。权重按输出列划分，各块读取自己的部分，权重数据合计为 8 MiB。[28]

输入大小仍是 128 KiB，多出的请求来自输出列方向的分块。若这些输出块共享缓存，重复请求可以直接从片上缓存取得数据；若同一个计算组保留输入并连续计算多个输出块，复用又可以移到局部缓冲。寻址、分块与存储层次在这里连接起来：分块方式决定发出哪些读取请求，数据所在的位置决定这些请求经过哪一级接口。

普通加载指令可以逐步计算地址并发起访问；专门的搬移单元则接收张量形状、步长和起点，重复生成这些地址。这组描述搬移任务的参数称为描述符。NVIDIA Hopper 架构（H100 所属架构）的张量内存加速器（TMA）和昇腾 950 的多维直接内存访问单元（NDDMA）都提供多维张量搬移能力。[29] 对上述子矩阵，起点、128 行、每行 256 bytes、行步长 8,192 bytes 足以描述整次规则搬移。由专门单元负责这些重复的地址计算后，计算线程就可以继续处理其他工作。

### 4.4.2 异步搬移与双缓冲

找到并取出一块数据之后，还要让下一块及时到达：若总等当前块算完才开始读取，计算单元就会反复停顿。准备两个缓冲后，可以一边使用第一个缓冲中的数据，一边把下一块加载到第二个缓冲；当前块用完，两块缓冲交换角色。这就是双缓冲：以额外空间换取加载与计算在时间上的重叠。

设共有 $n$ 块数据，每块加载耗时 $t_l$、计算耗时 $t_c$。若取得一块、算完一块，再开始下一块，总时间为 $n(t_l+t_c)$。加载与计算使用独立资源时，采用双缓冲，第一块先加载、再计算，后续各块以较慢阶段的节奏完成：

$$T_{\text{pipe}} = t_l + t_c + (n-1)\max(t_l, t_c).$$

八块数据每块加载 2 µs、计算 1 µs，串行需 24 µs，流水需 $2 + 1 + 7 \times 2 = 17$ µs。节省的 7 µs 来自重叠，传输字节和乘加次数都保持不变。再把计算加快一倍，总时间仅降到 16.5 µs，因为加载仍然每两微秒才能提供一块。

上述分析把加载看作一个完整阶段。下面将加载拆成传输与等待返回，计算多个请求同时进行时需要多少缓冲。这里用 tick 表示 B200 一个 SM 的时钟周期，并区分**发起间隔**与**完成延迟**：若每隔 64 tick 发出一次读取请求，而每块数据要在请求发出 192 tick 后才可使用，就会有多块同时等待返回。计算开始前，输入必须已经到达。把保存一个输入块的缓冲区称为一个输入槽：从发出读取请求，到该块计算结束，该槽一直被占用。增加槽位可以更早发起后续数据的读取，但也占用更多局部存储。

用注意力的 QK 计算具体推导。两个输入都是 $128 \times 128$，沿头维度切成四个 $k = 32$ 的累加块。每块 Q/K 输入合计 16 KiB，计算量为 1,048,576 FLOPs，四次计算依次更新同一份 FP32 结果，结果保存在独立的 64

[28] DeepSeek V4-Flash 共享专家搬移坐标结果与源级计数说明。

[29] Hopper Tuning Guide、昇腾 950 白皮书、Rubin 官方架构说明。

KiB 累加存储中。矩阵速率取 FlashAttention-4 论文给出的 B200 每个 SM 每周期 8,192 FLOPs；输入传输速率取 256 bytes/tick，传输结束后再等 128 tick 数据才可用。[30][31]

于是每块输入传输耗时 64 tick，发起后 192 tick 就绪，计算再用 128 tick。

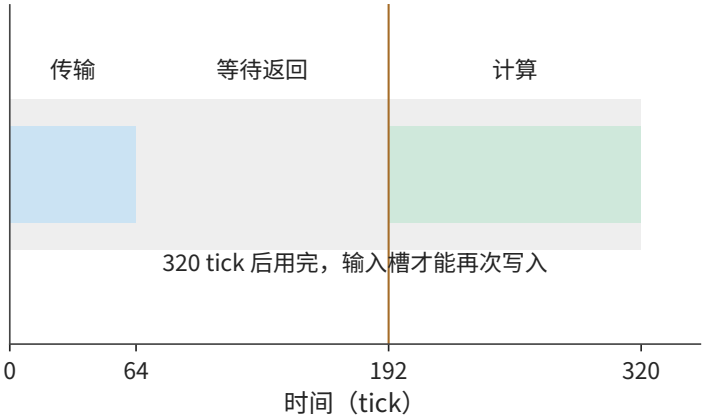



图 4-17　一个输入槽从发起到释放的完整生命周期。传输 64 tick，额外等待 128 tick，数据在 192 tick 就绪，再计算 128 tick，于 320 tick 释放。一个 tick 为 B200 SM 的一个时钟周期。

只有一个输入槽时，该槽经历加载、等待、计算，共 $64 + 128 + 128 = 320$ tick 后才能复用，四块在 1280 tick 结束。

图 4-18 至图 4-20 依次画出一槽、两槽和三槽的执行，并保持时间尺度一致。先沿绿色计算区间寻找空闲，再向上查看下一块输入何时就绪，就能判断等待从何而来。

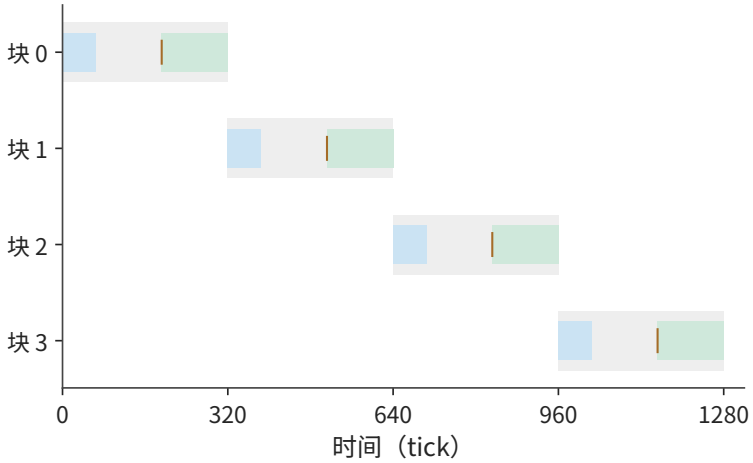



图 4-18　一个输入槽的四块时序。蓝条为传输，橙线为就绪，绿条为计算，浅灰为槽占用；上一块用完后才能再次发起，完成时刻为 1280 tick。一个 tick 为 B200 SM 的一个时钟周期。

两个槽允许在时刻 0 和 64 发出前两块的读取请求。第一块在 192 开始计算、320 结束，释放的槽用于第三块，第三块在 512 就绪；第二块在 448 已算完，因此中间留下 64 tick 空闲。继续按同样次序，第四块在 768 结束。第二个槽让一部分数据加载与计算重叠，但矩阵单元仍会在两块计算之间空闲。

要连续计算四块，最早完成时间是首块等待加四次计算，即

[30] Qwen 注意力输入流水基线、矩阵速率翻倍、建模与独立检查。

[31] *FlashAttention-4*，MLSys 2026，论文，§2.2、§3.1.1、公式 1—3 与表 1；单 SM 独立复算。

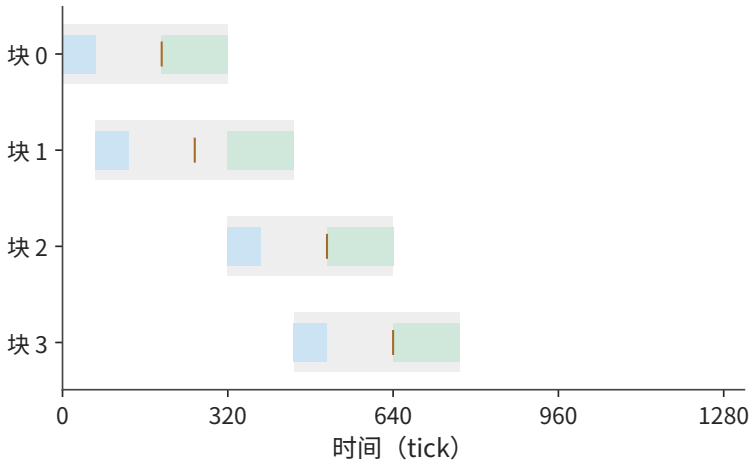



图 4-19　两个输入槽使用相同时间尺度。前两块可提前发起，但第三块到 512 tick 才就绪，第二块在 448 tick 已结束，留下 64 tick 空闲。横轴一个 tick 为 B200 SM 的一个时钟周期；各行对应一个数据块，灰色表示输入槽占用，蓝色表示传输，绿色表示计算，竖标记表示数据就绪。

$$T_{\min} = 192 + 4 \times 128 = 704 \text{ tick}.$$

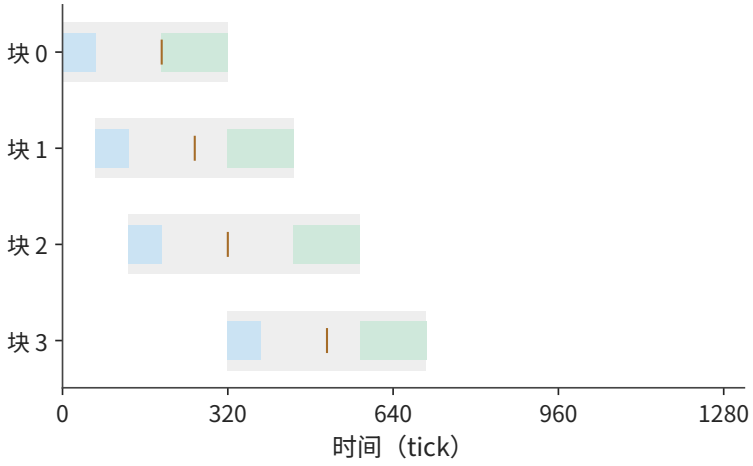



图 4-20　三个输入槽提前发起前三块，第一槽释放后接收第四块。矩阵单元从 192 连续计算到 704 tick，第四槽不再缩短完成时间。横轴一个 tick 为 B200 SM 的一个时钟周期；各行对应一个数据块，灰色表示输入槽占用，蓝色表示传输，绿色表示计算，竖标记表示数据就绪。

三个槽已足以实现这一时间。前三块的读取请求分别在时刻 0、64、128 发出，分别在 192、256、320 就绪；计算从 192 开始依次进行。第一块在 320 释放槽位，立即发出第四块的读取请求，第四块在 512 就绪，早于计划开始计算的时刻 576。矩阵单元因此从 192 到 704 tick 一直连续计算，中途没有空闲。[32]

[32] 三槽流水、计算翻倍变体及设计转折点由教学推导脚本生成，完整时序见推导数据。

| 输入槽数 | 输入缓冲 | 完成时间 | 相对前一项节省 |
|---|---|---|---|
| 1 | 16 KiB | 1280 tick | — |
| 2 | 32 KiB | 768 tick | 512 tick |
| 3 | 48 KiB | 704 tick | 64 tick |
| 4 | 64 KiB | 704 tick | 0 |

这一例子给出了一种寻找缓冲配置的方法：先用首块到达时间与连续计算时间求出目标，再逐个检查每块能否按时到达，最后寻找满足目标的最少槽位。第四个槽虽然让最后一块更早就绪，却没有让计算更早开始；提前取得的数据只是多等待了一段时间。

缓冲配置还随计算速度变化。矩阵速率翻倍后，每块计算只需 64 tick，连续计算的目标变为 $192+4\times64=448$ tick。只有三个槽时，第四块最早在 256 tick 发出读取请求，到 448 tick 才可使用，来不及在原计划的 384 tick 开始计算，完成时间为 512 tick；四个槽则能提前发出全部请求，达到 448 tick。矩阵计算加快以后，原来的加载安排已经来不及准备下一块数据，输入缓冲也需要重新配置。

有了异步搬移，计算线程发起加载后可以继续做其他工作，到要用数据时再等待完成事件。搬移单元向空槽写入数据，完成事件标记数据就绪；计算结束后再释放槽位。NVIDIA 的异步拷贝、昇腾 MTE / NDDMA 与 Metal 的任务依赖机制都用于安排这类交接。[33]

> **思考：输入可用前的额外等待翻倍，三个缓冲槽是否够用？**保持原来的传输和计算速率，将数据传输结束到输入可用的额外等待时间从 128 增至 256 tick。三个槽能否继续实现无间隙计算？分别写出第四块的就绪时刻与所需时刻。

### 4.4.3 单元间的数据传递

输入流水减少了矩阵单元等待数据的时间，但注意力还要把矩阵结果交给向量单元，再将概率送回矩阵单元。若完整保存一个 $128\times128$ FP32 分数矩阵，需要 64 KiB；矩阵单元写一次、向量单元读一次，经过同一存储接口的数据共 128 KiB。若矩阵单元直接把结果传给向量单元，就可以省去这次中间结果写回和重新读取。

向量计算何时开始，还取决于它需要哪些完整结果。普通逐行 Softmax 要先得到一行的全部分数，再求最大值和归一化分母。沿 QK 的内积维度计算一半时，每个分数都只是部分和；按查询行分组则可以先完成一组完整行，把它们交给 Softmax，其余行继续矩阵计算。分组方向因此决定向量单元能否提前开始计算。

将 128 行沿查询方向分成四组、每组 32 行，一组 FP32 分数和 BF16 概率共需 $32\times128\times(4+2)=24$ KiB。下面比较使用一个和两个缓冲区保存交接数据的方案；每个缓冲区从该组 QK 开始占用，到 PV 完成才释放。QK 与 PV 共用矩阵单元，Softmax 使用向量单元，两个方向共用交接接口。使用两个缓冲区时，不同组可以同时执行不同阶段的计算。[34]

本算例在矩形分数块上执行 QK、Softmax 与 PV，按完整行组顺序调度已就绪工作。tick 仍为 B200 SM 的时钟周期，矩阵速率 8,192 FLOPs/tick、交接接口 128 bytes/tick、指数与除法每 tick 16 次，分别取 FlashAttention-4 论文给出的每 SM 矩阵速率、共享内存读取带宽和特殊函数单元速率[35]；向量单元串行执行缩放、减法与求和、比较、指数和除法，普通运算及比较取每 tick 128 次。32 行一组时，QK 与 PV 各用 128 tick；Softmax 的普通运算、比较、指数、除法分别用 96、32、256、256 tick，合计 640 tick。直接交接 16 KiB 分数和 8 KiB 概率分别用 128、64 tick，先写后读的路径则分别用 256、128 tick。各步骤按整数 tick 向上取整。

[33] Hopper Tuning Guide、昇腾 950 白皮书、Rubin 官方架构说明。

[34] 矩阵—向量交接说明、32 行两槽直接路径。

[35] *FlashAttention-4*，MLSys 2026，论文，§2.2、§3.1.1、公式 1—3 与表 1；单 SM 独立复算。

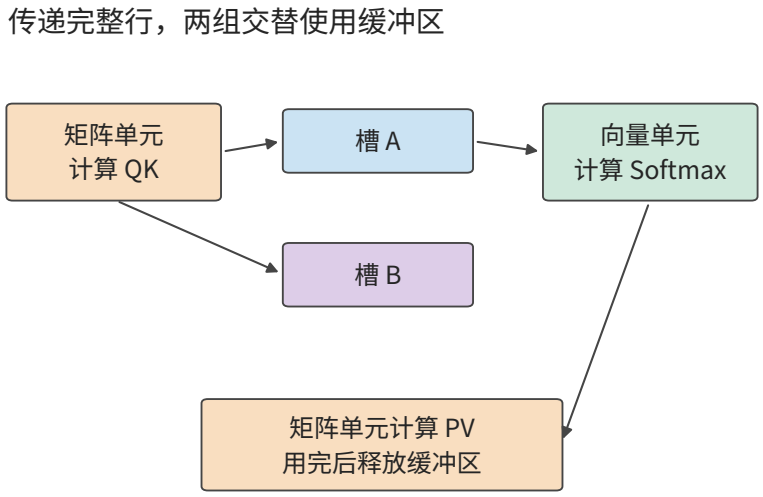



图 4-21　矩阵与向量单元通过完整行组交接。QK 产生分数，Softmax 产生概率，PV 使用概率后释放槽；另一槽容纳相邻行组，使不同组可以重叠推进。

| 交接方式 | 行组大小 | 槽数 | 向量计算开始时刻 | 完成时间 |
|---|---|---|---|---|
| 写入存储后再读取 | 128 | 1 | 1536 tick | 5118 tick |
| 写入存储后再读取 | 32 | 1 | 384 tick | 5120 tick |
| 写入存储后再读取 | 32 | 2 | 384 tick | 3200 tick |
| 直接传递 | 32 | 2 | 256 tick | 3008 tick |
| 直接传递 | 32 | 4 | 256 tick | 3008 tick |

先只改分组。向量单元从 1536 提前到 384 tick 开始计算，但总时间仍约为 5120 tick。原因在于只有一个槽：这一组进入 Softmax 时，槽内结果仍要保留，下一组 QK 无处写入，只能等待前组 PV 结束。提前开始缩短了第一组的等待，却没有让相邻组重叠。

再增加第二个槽，下一组可以使用另一槽推进，总时间降到 3200 tick，减少约 38%。随后改为直接交接，减少每次交接经过存储接口的字节数，总时间进一步降至 3008 tick，节省约 6%。继续增加槽位没有改变耗时最长单元的工作节奏，完成时间保持不变。

分组提前产生完整结果，第二个槽使相邻组重叠，直接通路缩短交接时间。这三项改动分别改变依赖、可同时推进的组数和接口工作量；第 4.6.2 节将这三项改动对应到昇腾的架构演进。

> **实验 4-4 ・核心：维持连续计算需要多少片上缓冲**
> 分别画出输入流水使用一至四个缓冲槽时的时间线，根据数据到达、计算结束和缓冲区释放的时刻，确定维持连续计算所需的最少槽数；再将矩阵速率翻倍，解释最少槽数的变化。随后画出各查询行组经过 QK、Softmax 和 PV 的时间线，区分分组、增加槽位和直接传递各自省下的时间。最后在固定片上容量下，分析每组缓冲的大小如何限制能够同时驻留的组数。

## 4.5 多芯片封装与互联

单个加速器无法满足计算或存储需求时，可以把计算和存储分布到更多芯片。扩展增加了资源，也增加了生成数据与使用数据的单元之间的距离。本节沿封装内、主机与加速器、加速器之间三种连接，分析数据放置和传输粒度如何影响完成时间。

### 4.5.1 封装内的计算芯片、内存与连接

封装可以包含多颗 die，由这些 die 共同提供计算单元和内存接口。Blackwell 数据中心产品与昇腾 910C 都采用双 die 设计：各 die 提供本地计算能力和存储接口，封装内的链路负责跨 die 的数据交换。[36] 容量扩大后，更多模型能够驻留；计算放在哪一侧，则决定多少访问要经过链路。

下面用两款双 die 产品计算。HGX B200 每 GPU 有 180 GB HBM、8 TB/s 带宽，八堆 HBM 平均分到两颗 die 上，每颗 die 本地为 90 GB、4 TB/s，两颗 die 之间的 NV-HBI 为 10 TB/s。昇腾 910C 每颗 die 本地为 64 GB、1.6 TB/s，die 间链路每方向 270 GB/s。两颗 die 各存 32 GiB 权重，某阶段要把这 64 GiB 全部读一遍，计算都放在 die 0 上。die 0 从本地 HBM 读取自己的 32 GiB，同时经链路读取 die 1 上的 32 GiB。跨 die 的数据先从 die 1 的 HBM 读出，再经过链路，速率取两者中的较小者；本地与跨 die 两路使用不同的接口，可以同时进行，所以这一阶段的读取时间由较慢的一路决定。

B200 的 NV-HBI 比每颗 die 的 HBM 还快：技术简报没有注明 10 TB/s 是否为双向合计，即使按双向合计、每方向 5 TB/s 计算，也高于每颗 die 的 4 TB/s。跨 die 读取因而仍按 die 1 的 4 TB/s 进行，本地与跨 die 各需约 8.6 ms，整个阶段约 8.6 ms，与两颗 die 各读本地权重相同：计算放在哪一侧，不改变读取时间。910C 的链路每方向只有本地 HBM 带宽的约 17%，本地读取约 21.5 ms，跨 die 读取却要约 127 ms，整个阶段由链路决定。把处理 die 1 权重的计算移到 die 1，两侧各自读取本地权重，只需约 21.5 ms；链路上传递的内容从 32 GiB 权重变成输入与结果，若共 64 MiB，经 270 GB/s 链路约需 0.25 ms。[37]

两款产品的差别由 die 间链路带宽与每颗 die 本地 HBM 带宽之比决定。B200 的这一比值为 2.5，跨 die 读取与本地读取一样快，两颗 die 可以作为一颗 GPU 使用；910C 的比值约为 0.17，跨 die 读取耗时约为本地的 5.9 倍。由 384 颗昇腾 910C 组成的华为 CloudMatrix384 超节点在 DeepSeek-R1 的 decode 中也以 die 为单位部署专家，每颗 die 恰好放一个专家。比值小于 1 时，应让计算单元在本地反复使用大块权重，只在 die 之间传递较小的输入和结果。

图 4-22 与图 4-23 比较计算单元与权重的相对位置：集中计算路径搬移权重，就地计算路径搬移输入并取回结果。判断如何分配计算任务时，先画出每种方案经过链路的数据，再比较总量。

这种放置还创造了并行机会：两侧本地读取和计算可以分别推进，最终再交接结果。是否需要等两侧都完成，由模型的计算图决定；如果下一步要合并部分和，就在合并点形成同步。第 6 章会从具体的并行切分推导这些交接。

### 4.5.2 CPU 与加速器之间的数据访问

封装内的数据放置决定跨 die 传输；芯片外也有类似问题。图 4-2 展示了主机与加速器之间的数据交接：CPU 准备的数据需要传给 GPU 使用。CPU 准备的输入通常先放在主机内存里，带独立显存的 GPU 要经 PCIe 或平台专用连接取得这些数据。主机到设备称为 H2D，设备到主机称为 D2H。复制引擎通过直接内存访问（DMA）按准备好的地址搬移，CPU 提交复制命令后可以处理其他工作。

同一个张量先经主机链路上传，再由 GPU 从显存读取，需要分别计算传输和读取的时间。以 RTX 4090 为例：该卡经 PCIe 4.0 x16（16 条通道）连接主机，名义带宽为双向合计 64 GB/s，即每方向 32 GB/s；显存带宽为 1,008 GB/s。[38] 设输入为 64 MiB，先忽略启动开销，则

[36] Blackwell 技术简报，10 TB/s NV-HBI；CloudMatrix384 v2，§3.3.1（910C 每 die 64 GB、1.6 TB/s，die 间每方向 270 GB/s）、§4.2 开头（decode 中每颗 die 放一个专家）与 §4.2.2；两款产品的 die 局部性计算见教学推导脚本的 `die_locality` 项；Vera Rubin 平台、UB 与昇腾核对。

[37] Blackwell 技术简报，10 TB/s NV-HBI；CloudMatrix384 v2，§3.3.1（910C 每 die 64 GB、1.6 TB/s，die 间每方向 270 GB/s）、§4.2 开头（decode 中每颗 die 放一个专家）与 §4.2.2；两款产品的 die 局部性计算见教学推导脚本的 `die_locality` 项；Vera Rubin 平台、UB 与昇腾核对。

[38] RTX Blackwell 架构白皮书附录 A 表 3：RTX 4090 为 24 GB GDDR6X、1,008 GB/s、PCIe Gen 4、TGP 450 W，RTX 5090 为 32

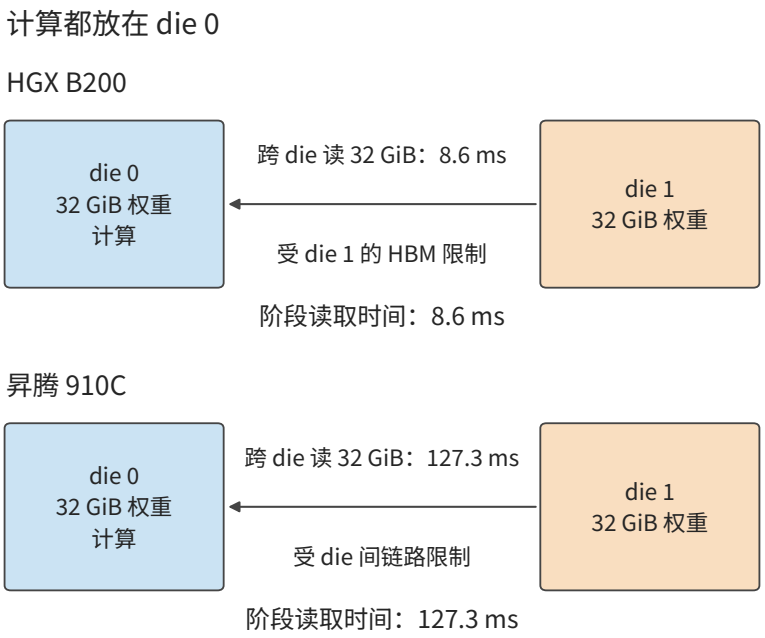



图 4-22　计算都放在 die 0 时，要跨 die 读取 die 1 上的 32 GiB 权重。B200 的跨 die 读取受 die 1 的 HBM 限制，约 8.6 ms，与本地读取相同；昇腾 910C 受 die 间链路限制，约 127 ms。

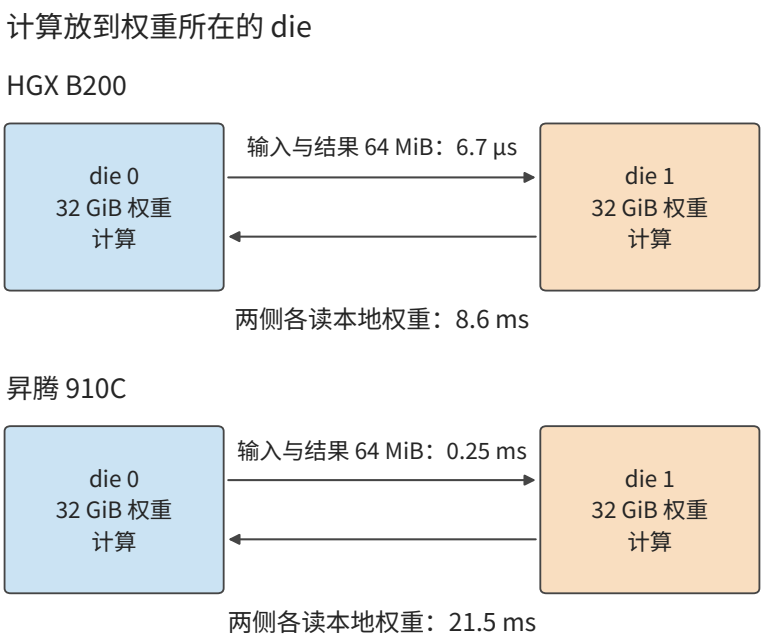



图 4-23　计算放到权重所在的 die 后，链路上只交换 64 MiB 输入与结果，两侧各读本地权重。B200 仍约 8.6 ms；昇腾 910C 从约 127 ms 降到约 21.5 ms。

$$T_{\mathrm{H2D}} = \frac{64\ \mathrm{MiB}}{32\ \mathrm{GB/s}} \approx 2.1\ \mathrm{ms}, \qquad T_{\mathrm{read}} = \frac{64\ \mathrm{MiB}}{1008\ \mathrm{GB/s}} \approx 67\ \mu\mathrm{s}.$$

GB GDDR7、1,792 GB/s、PCIe Gen 5、TGP 575 W；A100 80GB 数据手册，PCIe 4.0 为 64 GB/s（双向合计）。H2D 与显存读取时间见教学推导脚本的 `host_link` 项。

上传时间是一次显存读取时间的 31.5 倍，恰为两条路径的带宽之比。如果只使用一次，主机传输占主要时间；如果上传后在加速器上复用 $R$ 次，每次分摊 $T_{\mathrm{H2D}}/R$。复用 100 次时，平均每次分担的上传时间降至约 21 µs。每次调用只需提交地址和执行参数，整份数据继续留在显存中。

还可以求出至少需要复用多少次：当 $R \geq 32$ 时，每次分摊的上传时间已不大于一次加速器读取。因此，提高主机链路带宽对一次性输入更有价值，保持权重常驻则可以省去重复上传。

复制引擎与计算单元是独立执行资源，可以在 GPU 计算当前 batch 时上传下一个 batch。要形成流水，两个输入缓冲分别保存当前 batch 和下一个 batch，当前 batch 计算结束后再交换缓冲区的用途；这正是第 4.4 节双缓冲在主机链路上的应用。若复制写入与计算读取共同占用内存接口，该接口要承担两部分流量。

并非所有平台都要经过这次复制。Apple 的 CPU 与 GPU 共享物理内存，采用双方可访问的缓冲时，CPU 写好数据并完成同步后，GPU 可以从同一存储读取。与独立显存的做法相比，省去的是单独 H2D 副本及其传输；GPU 随后的读取仍由内存系统提供。[39] 统一内存把数据交接从“复制后使用”变成“同步后使用”。

统一地址空间让 CPU 和 GPU 以同一组虚拟地址引用数据。CUDA 是 NVIDIA GPU 的编程与运行平台；其统一内存机制（Unified Memory）可按平台规则迁移页面，地址相同而物理位置可以变化。[40] 数据与使用它的处理器之间的距离影响访问时间，地址和同步机制则让处理器能够找到数据，并在写入完成后读取。

### 4.5.3 加速器之间的数据交换

主机上传的例子强调了数据驻留和复用。卡之间还会频繁交换激活、专家输入或部分结果，每次传输的数据量不一，除了字节数，还要考虑每次传输的固定开销。一次传输既要提交请求、通知接收方完成状态，也要通过链路传送数据。用固定开销 $\alpha$、有效单向带宽 $R$ 和数据量 $V$ 表示，点对点时间为

$$T_{\mathrm{edge}} = \alpha + \frac{V}{R}.$$

以 HGX H100 服务器为例：机内每张 H100 经 NVLink 与其他 GPU 相连，双向合计 900 GB/s，每方向 $R = 450$ GB/s；A100 SXM 的 NVLink 为每方向 300 GB/s。[41] 固定开销取 $\alpha = 2$ µs，与第 7 章机内 NVLink 每轮启动的取值相同。设传递 $[M, 4096]$ 的 BF16 激活，数据量为 $8192M$ bytes。$M = 1$ 时数据量只有 8 KiB，在 H100 上传输约 0.02 µs，总计约 2.02 µs；$M = 256$ 时数据量为 2 MiB，传输约 4.66 µs，总计约 6.66 µs。

从 A100 换到 H100，带宽提高到 1.5 倍，8 KiB 传输的时间只从 2.03 降至 2.02 µs，2 MiB 传输则从 8.99 降至 6.66 µs，缩短约 26%。若改为在 H100 上把固定开销减半，小数据量的传输时间约减半，大数据量的传输也能节省约 15%。两种优化针对的是公式中不同的主导项。

令两项相等，就得到数据量的转折点：

$$V^{*} = \alpha R = 2 \times 10^{-6} \times 4.5 \times 10^{11} = 900\,000 \text{ bytes} \approx 879 \text{ KiB}.$$

对本例激活，这对应 $M \approx 109.9$，即从 110 行起，传输时间才超过固定开销。链路越快，转折点越大，更多的传输落在固定开销主导的一侧。小于这一规模时，把多次传输合并成一次，能降低每份数据分摊的启动开销；

[39] M2 Pro / Max 官方规格、Apple GPU 架构说明、Metal 存储模式、M5 GPU Neural Accelerator 官方说明。

[40] 主机、DMA 与统一地址的术语见 CUDA Programming Guide。

[41] RTX PRO 6000 产品规格，ECC、MIG 与产品配置；Hopper Tuning Guide，NVLink；GA102 白皮书，3090 NVLink；Blackwell Tuning Guide及 CUTLASS Blackwell 功能，SM100 / SM120。

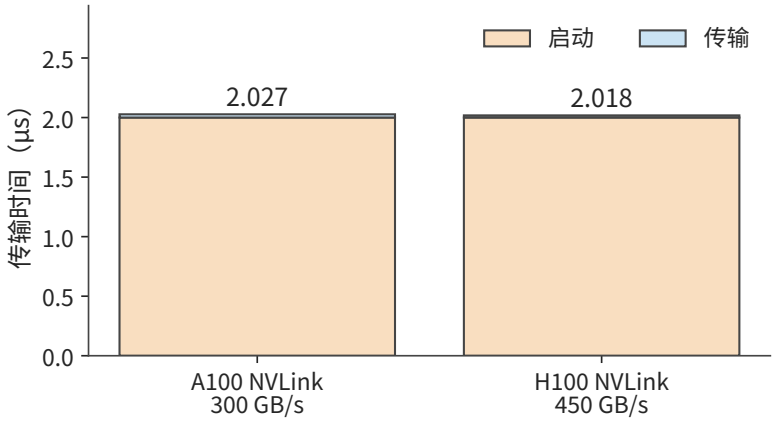



图 4-24　一次 8 KiB 传输的启动与传输时间。固定启动开销 2 μs，NVLink 从 A100 的每方向 300 GB/s 换成 H100 的 450 GB/s，只缩短很薄的蓝色传输项。

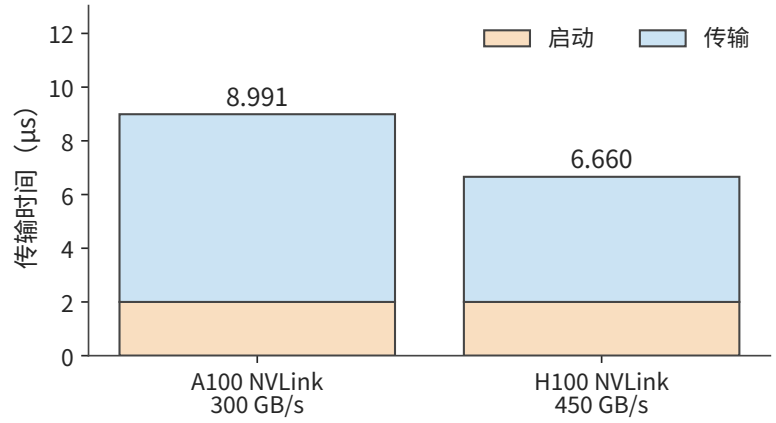



图 4-25　一次 2 MiB 传输在相同启动条件下的时间。蓝色传输项占主要部分，换成 H100 的 NVLink 带来更显著的收益；本图纵轴范围与上一图分别标注。

大于这一规模时，减少字节或提高带宽的收益更明显。把多份已就绪的数据合并发送，会减少启动次数；等待尚未就绪的数据以合并成更大的包，则会增加排队时间。传输的粒度因此也涉及吞吐与响应时间的取舍。

若一次执行有 100 次必须串行的小数据量交换，每次 2 μs，仅固定开销就有 200 μs。跨服务器时固定开销更大：两台 HGX H100 服务器共 16 张卡做一次 16 bytes 的全归约（AllReduce，把各卡的局部贡献合并，并让每张卡取得完整结果），公开实测约 25 μs，[42] 100 次这样的归约便要 2.5 ms。此时，即使减少部分传输数据，总时间也难以明显缩短，而减少交换轮数可以直接缩短依赖链。AllReduce 把多次传输与归约连接起来，第 6 章会分别计算轮数、每条链路的传输字节数，以及传输与计算能够重叠的时间。

NVLink、华为 Unified Bus 等互联提供设备间的传输路径，完成通知让接收方在数据到达后开始计算。[43]

> **思考：合并发送如何改变完成时间与首份数据的响应？**四份 8 KiB 数据依次在 0、1、2、3 μs 就绪。仍取 H100 NVLink 的每方向 450 GB/s、每次启动 2 μs，在同一链路上比较就绪后分别发送与等齐后合并发送：所有数据何时送达？第一份数据又何时可供接收方使用？

[42] 实验 7-3 公开运行记录核验，两台 HGX H100 原始日志：16 rank、NCCL 2.26.2，16 bytes AllReduce 非原位 24.96 μs、原位 24.93 μs。

[43] Blackwell 技术简报，10 TB/s NV-HBI；CloudMatrix384 v2，§3.3.1（910C 每 die 64 GB、1.6 TB/s，die 间每方向 270 GB/s）、§4.2 开头（decode 中每颗 die 放一个专家）与 §4.2.2；两款产品的 die 局部性计算见教学推导脚本的 `die_locality` 项；Vera Rubin 平台、UB 与昇腾核对。

## 4.6 模型需求如何推动架构演进

第 4.2—4.5 节分别分析了计算、存储、搬移和互联。本节把这些部件放回 NVIDIA、昇腾、Apple 的代际变化中，计算新增机制究竟减少了什么工作、改变了哪项资源需求，再解释这些变化如何回应模型负载。

以下逐项比较矩阵单元、数值格式、数据通路和容量的变化；每次保留同一模型工作量，只改变正在考察的硬件或表示方式。硬件参数与复算见配套计算记录。[44]

### 4.6.1 NVIDIA：矩阵、精度与数据路径分别带来多少收益

**Tensor Core 的价值，先用同一块芯片上的两条路径计算。** Volta V100 SXM2 的普通 FP32 峰值为 15.7 TFLOP/s，Tensor Core 的 FP16 输入、FP32 累加峰值为 125 TFLOP/s，HBM2 带宽为 900 GB/s。仍取本章的 $4096 \times 4096$ 投影，输入与权重以 FP16 保存，结果也写为 FP16；普通路径将操作数转换到 FP32 后执行乘加，Tensor Core 路径直接执行混合精度矩阵乘加，即输入用 FP16、累加用 FP32。两条路径读写同样的数据。[45]

一次处理 4,096 行，矩阵计算量为 $2 \times 4096^3 \approx 137.4$ GFLOPs，输入、权重、输出合计 96 MiB。普通 FP32 乘加需要 $137.4/15.7 \approx 8.75$ ms，Tensor Core 需要 $137.4/125 \approx 1.10$ ms，读取与写回则需要约 0.112 ms。矩阵时间降到约八分之一，原因是大量规则乘加交给了专门单元。

再把输入减为一行。两条路径的矩阵时间变成约 2.14 µs 和 0.268 µs，32 MiB 权重连同输入输出的传输却需要约 37.3 µs。两条路径因此都先受数据读取限制。同一项 Tensor Core 改进，对大 batch 的矩阵乘法收益显著，对这次单行调用则帮助不大，要加快的仍是权重读取。

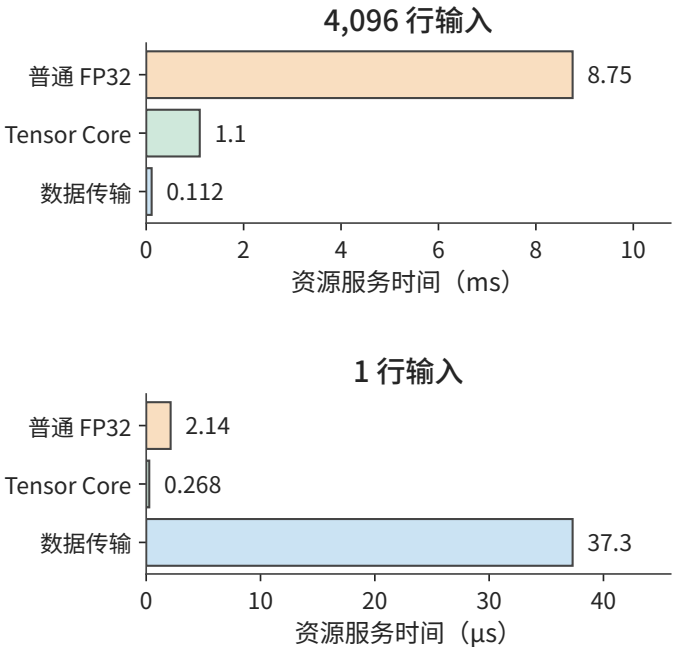



图 4-26　同一 V100 上的 Q 投影。4,096 行时，Tensor Core 大幅缩短矩阵计算；一行时，两条计算路径都短于权重传输。上下两组使用分别标注的时间单位。

**精度演进同时改变数值范围、存储量和执行指令。** Turing 在 Tensor Core 中加入 INT8、INT4，扩展低比

[44] 架构演进量化计算程序与说明、逐项计算结果。参数读取硬件输入表，3090 补充自 GA102 官方白皮书表 9；完整模型配置与张量清单沿用本书 calculations。

[45] Volta 架构白皮书，Tensor Cores 与混合精度章节。

特整数推理；Ampere 增加 BF16、TF32 和 2:4 结构化稀疏；TF32 在矩阵乘法中保留 FP32 的指数范围、缩短尾数，2:4 稀疏则按每四个元素中两个非零值组织计算。Hopper 的 Transformer Engine 将 FP8 矩阵指令与缩放管理结合，Blackwell 进一步支持分块缩放与 FP4。[46][47]

以其中加入的 BF16 为例。FP16 和 BF16 都占两字节，换成 BF16 后，前面的 32 MiB 权重保持不变。浮点数用有效数字乘以二的指数幂表示数值；正规数将有效数字标准化，次正规数则在更小的数值区间逐渐减少有效位数。BF16 使用八位指数，最小正规数约为 $1.18\times10^{-38}$；FP16 使用五位指数，最小正规数约为 $6.10\times10^{-5}$，最小正次正规数约为 $5.96\times10^{-8}$。一个 $10^{-8}$ 的梯度直接舍入到 FP16 会变为零，BF16 仍能表示。这减少了训练时为避免溢出、下溢而调整缩放的需求。代价是有效数字较少：在 1 附近，BF16 相邻数的间隔为 $2^{-7}$，FP16 为 $2^{-10}$。低精度输入配 FP32 累加，分别处理输入表示与长内积累积误差。[48]

再看存储量。沿用第 4.2.4 节每 32 个值共享一字节 scale 的分块格式；MX 是这种微缩放（microscaling）格式的名称。MXFP8 需要 $16+0.5=16.5$ MiB，MXFP4 需要 $8+0.5=8.5$ MiB。以 RTX 4090 的 1,008 GB/s 带宽读取这份 BF16 权重需 33.29 µs；以 RTX 5090 的 1,792 GB/s 读取同一份 BF16 权重需 18.72 µs，读取 MXFP4 权重则需 4.97 µs。前一步来自带宽提高，后一步来自位数与分组表示改变。

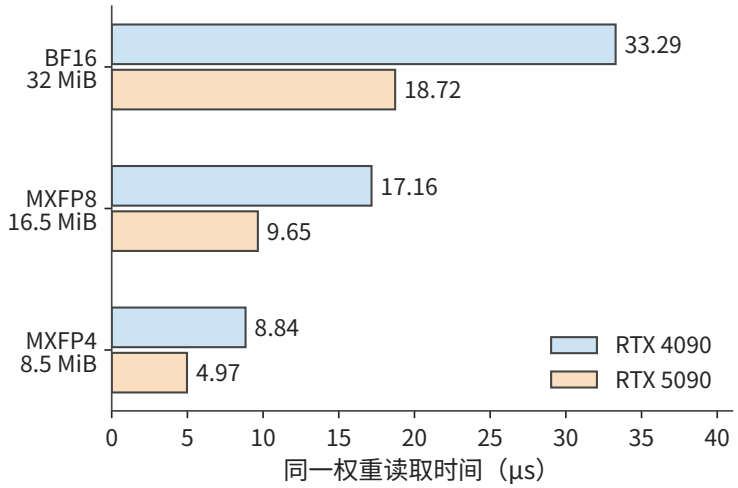



图 4-27　同一个 4096×4096 权重的存储与读取预算。每 32 个低精度值共享一个一字节 scale。位宽从 BF16 降至 MXFP4 后，权重与 scale 合计从 32 MiB 减至 8.5 MiB。

从 33.29 µs 到 4.97 µs，权重读取预算缩小到约 1/6.69。这笔收益由两部分相乘：带宽约提高到 1.78 倍，存储量缩小到约 1/3.76。原生低精度矩阵路径直接处理压缩表示，省去第 4.2.4 节先展开路径中的高精度权重副本。

**矩阵越来越快以后，架构开始减少它前后的准备与等待。** Ampere 的异步拷贝省去全局内存到共享内存的寄存器中转；Hopper 的 TMA 接手多维地址生成与搬移，并配合异步矩阵执行；Blackwell 数据中心 SM100 用张量存储（Tensor Memory，TMEM）保存矩阵累加结果。这三项改动分别改变搬移指令、通用寄存器占用和中间结果存放位置。[49]

第 4.3.2 节的计算块需要 32 KiB 输入和 64 KiB 累加结果。三组输入与一组结果合计 160 KiB，TMEM 独立承担其中的 64 KiB，通用寄存器便可用于其他线程状态。这就是新增专用存储给工作驻留带来的直接变化。

[46] Turing 官方架构白皮书，Turing Tensor Cores 与低精度推理。

[47] A100 架构白皮书、H100 架构白皮书、Hopper Tuning Guide、Blackwell 技术简报与CUTLASS Blackwell 功能。

[48] A100 架构白皮书，BF16 与数值格式章节；数值范围、存储与量化复算。

[49] Blackwell Tuning Guide、CUTLASS Blackwell 功能说明及 SM100 TMEM 示例；SM120（计算能力 12.0）每个 SM 的 128 KB 一级数据缓存与 100 KB 共享内存上限见 CUDA 编程指南的计算能力表。

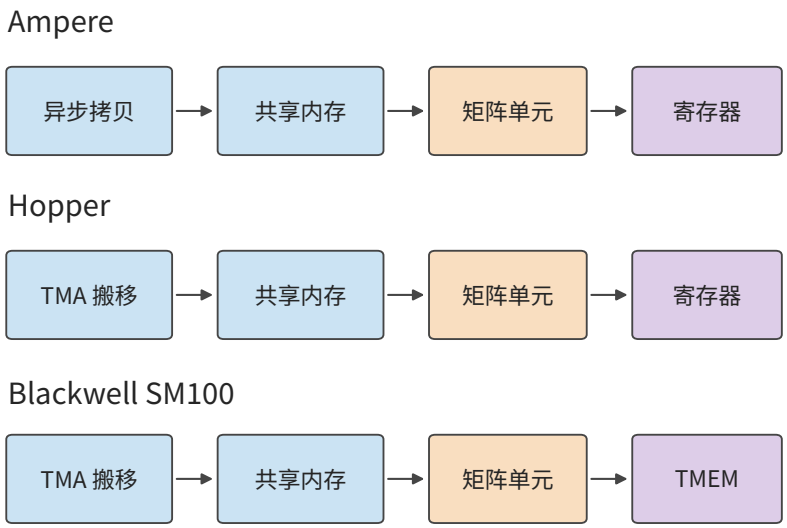



图 4-28　从异步拷贝、TMA 到 TMEM，专门部件承担更多数据准备与累加状态管理。蓝色表示数据存储与搬移，橙色表示矩阵计算，紫色表示累加结果。

再往后一代，Rubin 的 FP32 指数吞吐提高到 Blackwell 的 2 倍，BF16 / FP16 指数吞吐提高到 4 倍，回应了例 4-1 中矩阵加速后暴露的指数瓶颈。Rubin 还允许 TMA 更新专家地址与步长：MoE 切换专家时，矩阵形状可以保持不变，只需切换权重位置，描述符更新由此减少重复的地址准备。[50]

**消费级、工作站和数据中心产品，怎样放进同一张表？**固定 BF16 输入、FP32 累加、稠密矩阵计算，参数如下。RTX 3090、4090、5090 分别采用 Ampere、Ada、Blackwell；A100、H100 分别采用 Ampere、Hopper。[51]

| 型号与产品线 | 容量 GB | 带宽 GB/s | BF16 / FP32 稠密矩阵 TFLOP/s |
|---|---|---|---|
| RTX 3090，消费级 | 24 | 936 | 71.0 |
| RTX 4090，消费级 | 24 | 1,008 | 165.2 |
| RTX 5090，消费级 | 32 | 1,792 | 209.5 |
| RTX PRO 6000 Blackwell，工作站 | 96 | 1,792 | 503.8 |
| A100 80GB SXM，数据中心 | 80 | 2,039 | 312.0 |
| H100 SXM，数据中心 | 80 | 3,350 | 989.4 |
| H200 SXM，数据中心 | 141 | 4,800 | 989.5 |

RTX 3090 与 4090 容量相同，BF16 矩阵速率约提高到 2.33 倍，带宽却只提高约 7.7%。前者主要缩短大矩阵计算，后者才直接缩短单行权重读取。RTX 5090 的容量提高到 32 GB，带宽再提高到约 1.78 倍；新增的 FP4 路径还为能用低精度的模型提供了另一种执行方法。这三代产品分别改变了矩阵处理、权重读取和模型驻留的资源预算。

RTX PRO 6000 与 RTX 5090 带宽同为 1,792 GB/s，工作站卡的主要增量包括 96 GB 带 ECC 的显存、更高的 BF16 / FP32 累加吞吐，以及多实例 GPU（MIG）功能；MIG 将一块 GPU 划分成资源相互隔离的实例。

[50] NVIDIA Rubin GPU 架构说明，MoE 数据搬移、K 维指令吞吐与注意力加速。

[51] 架构演进量化计算程序与说明、逐项计算结果。参数读取硬件输入表，3090 补充自 GA102 官方白皮书表 9；完整模型配置与张量清单沿用本书 calculations。

对相同大小的权重流式读取，两者的带宽预算相同；对大矩阵或超过 32 GB 的常驻数据，结果就不同。SM120 与 SM100 是两种不同的 SM 指令架构。消费级与工作站 Blackwell 采用 SM120；B200 等数据中心 Blackwell 的 SM100 使用前面介绍的 TMEM 和相应矩阵指令，软件按目标架构选择 kernel。[52]

数据中心卡还提供 HBM 与面向多卡的高速互联。A100 SXM 的 NVLink 双向聚合带宽为 600 GB/s，H100 SXM 为 900 GB/s，换成单向发送分别为 300、450 GB/s。按第 4.5 节的方法，传送 64 MiB 数据的链路序列化时间（数据量除以单向带宽）分别约为 224、149 µs。计算规模增加后，张量并行（把同一层内的矩阵分给多张卡计算）每层的通信、训练梯度交换和专家 dispatch 会反复使用这些连接。RTX 4090、5090 没有 NVLink；RTX 3090 支持双卡 NVLink。第 6、7 章会用这些链路参数计算张量切分、梯度交换与专家 dispatch 的时间。[53]

回到计算与带宽两项的比较：H100 相对 A100，BF16 矩阵速率提高到约 3.17 倍，带宽提高到约 1.64 倍，同时增加 FP8、TMA 和新的协作机制。H200 相对 H100，则主要增加容量与带宽，表中的 BF16 矩阵速率基本相同。这两组变化分别对应“计算与执行组织一起改”和“保持计算能力、增强存储”。第 4.8.2 节代入模型工作后，就能判断哪一组增量更有用。

### 4.6.2 昇腾：从 CNN 的卷积展开，到 Transformer 的矩阵与向量交接

早期昇腾 910（后面的代际比较中称作 910A）设计时，ResNet 等卷积神经网络（CNN）占据了大部分训练负载。笔者的设计回顾记录了这一背景；DaVinci 论文进一步给出 Scalar、Vector、Cube 和 MTE 的分工。MTE 包含 img2col、转置和解压模块，其中 img2col 在数据搬移过程中把卷积窗口组织成矩阵输入。先计算这项专用能力在 CNN 中节省了什么，再把负载换成 Transformer。[54]

**卷积展开为何值得交给硬件？**取一个 ResNet 式的 $3\times3$ 卷积，输入为 $56\times56\times64$，输出通道数为 64，步长为 1，通过填充（padding）保持空间尺寸。输入以 FP16 保存，占 $56\times56\times64\times2=401,408$ bytes，约 0.383 MiB。每个输出位置使用九个相邻位置的 64 个通道，展开矩阵为 $3136\times576$，占约 3.445 MiB，是原输入的九倍。

若先在 HBM 中生成展开矩阵，再由矩阵乘法读入，额外发生一次 3.445 MiB 写入和一次读取，共 6.891 MiB。早期 910 的 HBM 带宽为 1.2 TB/s，仅这笔展开矩阵的写读就占约 6.02 µs 的带宽时间。MTE 在片上搬移时完成窗口展开，便省去这份 HBM 中间副本的写读。输入窗口在片上组织后直接送入矩阵计算，HBM 中保留原始输入和权重。[55]

再看计算分工。该卷积需要 $2\times3136\times576\times64\approx231.2$ MFLOPs。DaVinci 论文给出的 Ascend-Max Cube 为每周期 8,192 FLOPs，因此矩阵服务时间为 28,224 周期。输出 ReLU 只处理 200,704 个元素；按 256 bytes 向量宽度，一组可容纳 128 个 FP16 元素，共 1,568 组。卷积的矩阵计算量与后处理相差悬殊，Cube、Vector 和展开硬件各自承担明确的工作。

**换成注意力，资源比例和数据方向一起变化。**$128\times128$、头维度 128 的注意力块，QK 与 PV 合计约 8.39 MFLOPs，在同一 Cube 速率下为 1,024 周期。Softmax 需要计算 16,384 个指数，并执行最大值归约、求和与归一化。按 256 bytes 的向量宽度，一组 FP32 数据有 64 个值，单是指数就要处理 256 组。要让指数阶段不超

[52] RTX PRO 6000 产品规格，ECC、MIG 与产品配置；Hopper Tuning Guide，NVLink；GA102 白皮书，3090 NVLink；Blackwell Tuning Guide及 CUTLASS Blackwell 功能，SM100 / SM120。

[53] RTX PRO 6000 产品规格，ECC、MIG 与产品配置；Hopper Tuning Guide，NVLink；GA102 白皮书，3090 NVLink；Blackwell Tuning Guide及 CUTLASS Blackwell 功能，SM100 / SM120。

[54] 笔者《网络的智能应该放在哪里》，2023 年演讲中关于 2016 年 ResNet 设计背景的回顾；DaVinci 架构论文 §3.2、§3.4、表 5 和 Ascend 910 系统，分别给出 img2col、资源配比、每核带宽与 1.2 TB/s HBM。

[55] DaVinci 架构论文，§3.1—3.4；Ascend C 指南，第 4 章耦合与分离架构；CloudMatrix384 v2，§3.3.1、§4.2.2。

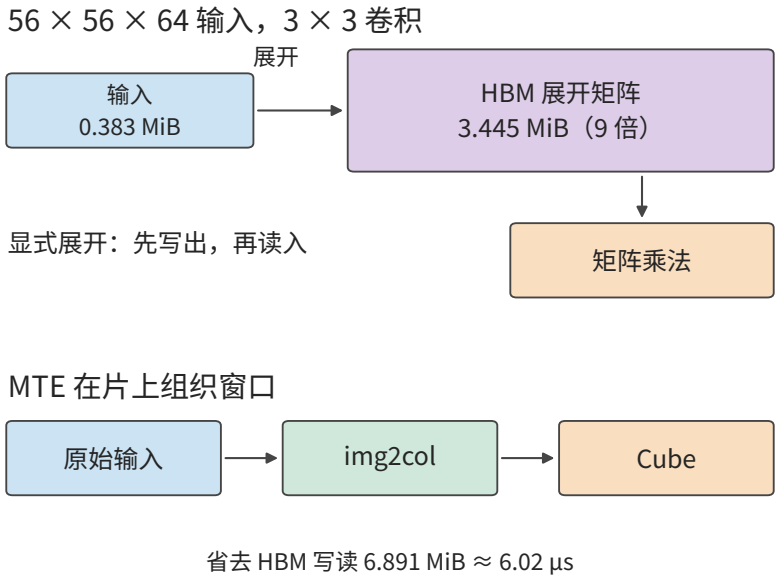



图 4-29　同一个 3×3 卷积的两条准备路径。显式展开形成九倍大小的中间矩阵；MTE 在片上组织窗口，省去 HBM 上 6.891 MiB 的中间结果写读。

过矩阵的 1,024 周期，指数部分需要平均每四周期处理一组，也就是每周期 16 个指数结果。随后执行归约与归一化，形成 PV 的输入。

同一 Cube 上，CNN 卷积的矩阵工作占 28,224 周期，注意力块只占 1,024 周期；向量工作却从简单 ReLU 变成指数、归约与归一化。DaVinci 论文的 BERT 分析中，多数层由 Cube 时间主导。把注意力单独拆开后，可以看到 Softmax 对向量吞吐提出的新需求。[56]

在 CNN 中发挥作用的卷积窗口展开，在 Q、K、V 投影上没有工作可做，因为投影输入已经是矩阵。注意力反而需要把 QK 的结果交给 Vector 做 Softmax，再将概率交回 Cube 做 PV。img2col 节省的卷积展开工作，与这两次矩阵、向量交接所需要的工作不同；负载转变以后，新增硬件资源的重点也随之转移。

**独立调度以后，交接路径怎样影响速度？**910B 所在 Atlas A2 将矩阵核 AIC 与向量核 AIV 分开控制；910C 进一步采用双 die，每颗 die 有 24 个 AIC 和 48 个 AIV。分离控制让不同工作独立推进，AIC 与 AIV 则通过全局地址空间交换结果，实际数据经过缓存层次。CloudMatrix384 在 MLA 中压缩 KV 状态，通过融合与动态分块安排矩阵和向量工作；MTP 一次处理多个预测位置，又改变了这些计算的行数。[57]

用同一个注意力块计算接口工作量。分数与概率各按 FP32 交接，每个张量占 64 KiB。一次交接经过外层交换接口写出、读入，共 128 KiB；QK→Softmax 和 Softmax→PV 两次合计 256 KiB。若两次交接都由同一外层接口承担，要在 1.024 µs 内完成，该接口需要约 256 GB/s。把 DaVinci 论文中每核分配的 94 GB/s 末级缓存（LLC）带宽代入，这笔工作需要约 2.79 µs，已经超过 1 GHz 下 Cube 的 1.024 µs 矩阵服务时间。矩阵计算即使继续加快，外层接口仍需搬完这些字节。

昇腾 950 的下一步改动正针对这条外层接口：在 Cube 的一级缓冲 L1 与 Vector 的统一缓冲（Unified Buffer，昇腾文档简写为 UB，与第 6 章的互联 Unified Bus 并非同一概念）之间增加 CV（Cube–Vector）直接通路。两次各 64 KiB 的中间张量通过直接连接交接，共传递 128 KiB，原来外层接口的 256 KiB 写读由此移出。若要求这条直接通路在 1.024 µs 内完成两次交接，需要约 128 GB/s；若交接格式改为 FP16 / BF16，需求再减半至约 64 GB/s。

[56] DaVinci 架构论文，§3.1—3.4；Ascend C 指南，第 4 章耦合与分离架构；CloudMatrix384 v2，§3.3.1、§4.2.2。

[57] DaVinci 架构论文，§3.1—3.4；Ascend C 指南，第 4 章耦合与分离架构；CloudMatrix384 v2，§3.3.1、§4.2.2。

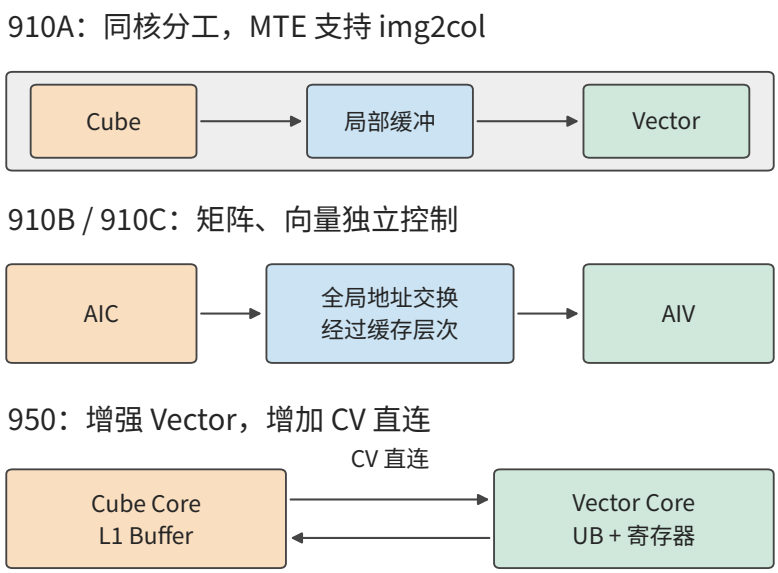



图 4-30　早期同核分工、910B ／ 910C 的独立控制，以及 950 增加的 CV 直接通路。计算单元的配比与结果交换的路径共同决定融合算子的效率。

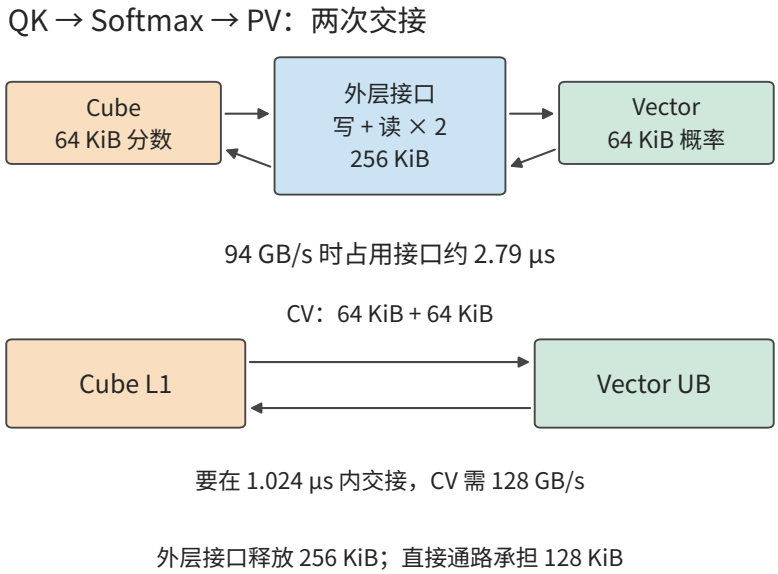



图 4-31　同一注意力块的两次交接。外层交换接口承担写出和读入共 256 KiB；直接 CV 通路传递两个 64 KiB 张量，将这笔流量移出外层接口。在 1.024 µs 内完成交接，对应直接通路的带宽需求为 128 GB/s。

950 同时将单个 Vector Core 的 FP16 / FP32 吞吐提高一倍，在统一缓冲与向量算术逻辑单元（ALU）之间增加寄存器文件，并优化 Softmax 与激活函数 GELU 的执行。对相同普通向量工作 $F_v$，$F_v/P_v$ 降为 $F_v/(2P_v)$；如果原来向量阶段是矩阵阶段的两倍，这项改动能把二者重新配平。按前面的注意力块，指数吞吐需要达到每周期 16 个结果。新增 NDDMA 承担多维地址与布局准备，BufferID 用缓冲区标识组织同步交接。[58]

950 的低精度也可以沿同一工作量理解：同频 FP8 类矩阵计算速率为 FP16 的两倍，MXFP4 为四倍。对

[58]昇腾 950 官方架构白皮书，§4.1—4.1.6；早期 DaVinci 与 CANN 分离架构的页级定位见比较笔记。

固定矩阵 $F$，计算服务时间依次为 $F/P$、$F/(2P)$、$F/(4P)$；但 FP32 分数与概率的两次交接仍是原来的大小。只有同时改变交接路径、格式或布局，接口工作才会下降。矩阵、向量和 CV 通路一起演进，才能把局部加速变成整段注意力的收益。

### 4.6.3 Apple：大容量统一内存，如何换算为模型运行能力

Apple 的 CPU、GPU 和独立 Neural Engine 共享统一内存。统一内存省去了处理器之间的数据复制，也让本地运行的模型能使用较大的内存池。下面先计算统一内存能够容纳的模型，再计算生成一个 token 需要读取的数据量。

先核对容量与带宽的代际关系。M1 Ultra 的代表配置为 128 GB、800 GB/s；M2 Ultra 扩展到 192 GB、800 GB/s；M3 Ultra 的 80 核 GPU 配置有 256 GB 和官方记录的 512 GB，两者均为 819 GB/s。M4 Max 也有 128 GB 配置，带宽为 546 GB/s。[59]

**容量翻倍，哪些时间会改变？**先保持模型、精度和 batch 不变。M3 Ultra 从 256 GB 增至 512 GB，带宽仍为 819 GB/s，同一执行的 $V/R$ 完全相同。M1 Ultra 换成 M3 Ultra，带宽从 800 增至 819 GB/s，读取速度只增加约 2.4%。容量的增量让更多权重、KV 或请求驻留，带宽的增量才直接改变同一批数据的读取时间。

取 Qwen3-8B、BF16、单请求、8,192 位置，本书张量清单给出每步权重与 KV 的接口载荷约 16.345 GB。M1 / M2 Ultra 的读取下界为 16.345 GB/(800 GB/s) ≈ 20.43 ms，M3 Ultra 为 16.345 GB/(819 GB/s) ≈ 19.96 ms，M4 Max 为 16.345 GB/(546 GB/s) ≈ 29.94 ms。同样是 128 GB，M1 Ultra 读取这一步数据比 M4 Max 少用约 9.51 ms。[60]

**容量的主要价值，用大型 MoE 更容易看清。** Qwen3-235B-A22B 采用本书的 4-bit 存储方案，连同 scale 与保留高精度的张量，常驻权重约 123.142 GB。单请求、8K 上下文再加 KV 和 2 GiB 工作区，合计约 126.867 GB。128 GB 整机只余约 1.13 GB；给操作系统和其他应用再预留 8 GB 后，需要约 134.87 GB。256 GB 或 512 GB 的配置则有足够空间容纳这一预算。

MoE 每步只访问选中的专家。相同单请求每步的权重与 KV 接口载荷约 13.697 GB，在 M3 Ultra 上需要约 16.72 ms，而不是将 123.142 GB 全部权重每步读一遍得到的约 150 ms。对 MoE，全部权重决定容量，当前选中的专家决定每步权重读取。大型 MoE 因而能同时表现为“需要大内存”和“每步读取相对较少”。[61]

再代入第 4.3.3 节的八请求路由结果。集中与分散路由的每步读取量分别为 24.737 GB 和 75.967 GB，在 M3 Ultra 上对应 30.20 ms 和 92.76 ms。每步产出八个 token，读取吞吐预算分别为 8/0.03020 ≈ 265 token/s 和 8/0.09276 ≈ 86 token/s。256 GB 与 512 GB 配置可以容纳相同的八请求任务，路由复用决定了这里两组读取预算的差别。

**局部存储和 GPU 专用单元怎样加入这组计算？** M3 引入 Dynamic Caching，按运行需求分配 GPU 局部内存；M4 延续这一组织。这些机制影响工作驻留和片上资源利用，模型容量则由统一内存决定。M5 又在每个 GPU 核内加入 Neural Accelerator，并通过 Metal 4 Tensor API 等接口使用；独立 Neural Engine 继续存在。GPU 内专用单元加速矩阵处理，数据准备与一般运算继续由 GPU 程序组织。[62][63]

---

[59] Apple 官方配置核对、GPU 与容量组合及硬件表。M3 Ultra 512 GB 由官方发布稿及 80 核 GPU 整机测试配置记录交叉确认。M5 Ultra 512 GB / 1,200 GB/s 为 2026 年 8 月公布的规格，512 GB 配置计划于 2026 年 10 月下旬交付。

[60] 架构演进量化计算程序与说明、逐项计算结果。参数读取硬件输入表，3090 补充自 GA102 官方白皮书表 9；完整模型配置与张量清单沿用本书 calculations。

[61] 存储代际比较结果及计算说明。

[62] Apple M3 官方架构介绍、M4 官方发布说明与 M4 Mac mini 规格。

[63] M2 Pro / Max 官方规格、Apple GPU 架构说明、Metal 存储模式、M5 GPU Neural Accelerator 官方说明。

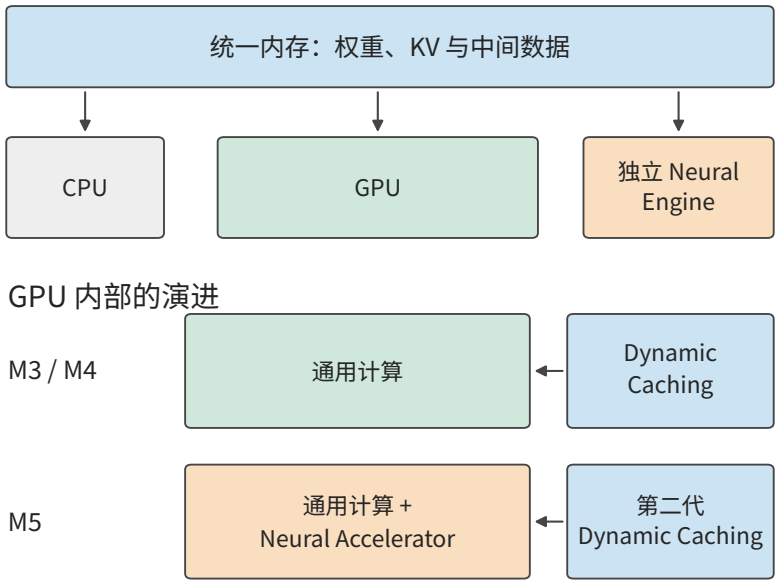



图 4-32　统一内存负责整机的数据容量，Dynamic Caching 管理 GPU 局部资源，M5 的 Neural Accelerator 增加 GPU 内专用计算能力。三者分别进入容量、驻留和计算时间的分析。

内存带宽也在逐代提高：基础 M4 到基础 M5，统一内存带宽从 120 增至 153 GB/s，提高约 27.5%。前面 32 MiB 权重的读取时间从约 279.6 µs 降至 219.3 µs，缩短约 21.6%。Apple 公布 M5 的 GPU AI 峰值超过 M4 的四倍；同一单行投影的权重读取仍由上述带宽决定。增加输入行数以后，权重读取由更多计算共同分摊，新的矩阵单元才更容易成为主要收益来源。这一现象与 V100 上两种行数的结果完全对应。[64]

这些演进把图 4-3 的反馈关系落实为资源变化：大矩阵推动专用乘加，Transformer 增加向量与交接需求，大型 MoE 扩大常驻容量需求。

## 4.7 专用架构

前几节通过复用、流水与局部性提高通用加速器效率。如果负载中有些条件长期保持稳定，还可以把这些条件直接用于硬件设计：固定运算规则、固定通信次序，甚至固定权重。专用化减少某些工作，也把更多资源集中到剩余工作上。

### 4.7.1 TPU

TPU v1 的推理计算以规则矩阵乘法为中心。256 × 256 个乘加单元构成阵列，权重经片上先进先出队列（FIFO，按进入次序取出的缓冲）加载到阵列，输入从 Unified Buffer 送入，结果保存在独立累加存储中，再交给激活单元。一条指令可以描述较大规模的矩阵运算，局部连接反复传递操作数，减少每次乘加所需的控制和远距离访问。[65]

一块 256 × 256 的 8-bit 权重为 64 KiB。若只处理一行输入，完成 $2 \times 256^2 = 131\,072$ 次操作，每字节权重对应两次操作；处理 256 行时，完成约 33.6 M 次操作，每字节权重对应 512 次。权重保持不变，计算量增至原来的 256 倍，正是第 4.1 节投影复用关系在阵列中的实现。

[64] M2 Pro / Max 官方规格、Apple GPU 架构说明、Metal 存储模式、M5 GPU Neural Accelerator 官方说明。

[65] Jouppi 等，*In-Datacenter Performance Analysis of a Tensor Processing Unit*，ISCA 2017，归档论文。

要取得这种收益，阵列需要连续的输入行。权重 FIFO 能容纳四块这样的权重块，权重加载与计算可以通过缓冲衔接；输入不断送入阵列时，独立的累加存储负责保存部分和。大阵列、输入缓冲和权重队列因此互相配合：阵列提供乘加单元，缓冲保证这些单元持续获得所需的操作数。

阵列启动时，操作数还要沿连接传播到各计算位置；结束时，结果要汇集输出。连续计算的块数越多，每块分摊的流水线填充与排空时间越少，小任务则更容易受到这些固定步骤影响。

负载发生变化后，硬件各部分的配比也需要调整。训练增加可写状态、梯度与加速器间交换；长上下文生成增加 KV 占用，并要求每个生成步骤等待前一步结果。Google 的 TPU 8t / 8i 把训练与采样服务分开组织，体现了不同阶段对矩阵、向量、存储和互联的需求。[66] TPU 的共同思路是围绕已知工作安排资源，而具体配比随主要负载变化。

### 4.7.2 Groq、Graphcore 与 Cerebras

将更多数据留在片上，可以减少远距离访问。沿着这一思路，可以把计算和存储分散到许多局部单元，也可以进一步增大芯片，在片上容纳更多这样的单元。二者都让大量数据由附近的计算单元使用，也都需要安排单元之间的交换。

Groq 的张量流处理器（Tensor Streaming Processor，TSP）由编译器预先安排张量的生成、传输和使用时刻。Graphcore 的智能处理单元（Intelligence Processing Unit，IPU）由许多带有局部存储的计算单元（Graphcore 称之为 tile）组成，各计算单元通过计算、同步和数据交换协作完成任务。Cerebras 把大量计算与存储铺在晶圆尺度上，第三代晶圆级引擎 WSE-3 的片上 SRAM 为 44 GB。SRAM 适合直接集成在计算芯片内，用面积换取低延迟访问。[67] 这些设计在不同尺度上实现局部性：尽量缩短通信路径，让反复使用的数据靠近计算。

分布式容量的难点在于各处需求并不相同。某一单元释放了存储空间，其他单元也无法直接把这部分空间当作自己的本地存储；要重新平衡，必须迁移数据或移动计算。路由不均的专家模型尤其容易出现这种情况：总量相同，集中到少数专家的请求会使少数计算单元更忙。

局部存储的不均衡可以靠重新分配任务来缓解，但另有一个问题随服务规模加剧：KV 总量增加。沿用 Qwen3-8B，每条包含 8,192 个 token 的请求，其 BF16 KV 是 1.125 GiB。Groq TSP 每颗芯片有 220 MiB SRAM，[68] 仅存放 BF16 权重就需要 72 颗芯片。KV 另用芯片保存，并可均匀切分：一条请求需要 $\lceil 1152/220 \rceil = 6$ 颗芯片；八条请求共 9 GiB，需要 42 颗；若八条请求的上下文都增至 32,768 个 token，需要 168 颗。

从 6 到 42 再到 168，增加的是可写状态，模型权重并未增加。片上保存权重减少了一类读取，但持续服务还需要为增长的状态提供容量。芯片增多后，分布在各芯片上的 KV 又要跨芯片汇集；容量扩展与通信需求由同一工作负载同时产生。

晶圆尺度缩短了部分原本跨封装的连接，但数据仍沿网络经过多个计算单元。把使用同一状态的计算安排在附近，能够减少经过的链路和同步；把任务均匀摊开则有利于使用更多计算位置。分配计算任务时，既要让数据在本地复用，又要避免少数单元过忙而其余单元空闲，这与多 die 放置的原理一致。

### 4.7.3 固定数据流与固定权重

专用化利用执行中已经确定的条件重新分配硬件资源。固定算子种类，指令和控制就能简化；固定数据流，通信就能提前安排；固定数值格式，存储器和运算单元就能按所需位宽设计；固定权重，则可以为不再变化的参数设计专门的存储结构。原来用于处理多种情况的控制和数据通路，可以相应简化。

[66] Inside the Eighth-Generation TPU: An Architecture Deep Dive。

[67] Groq TSP 论文、IPU Programming Model、Cerebras WSE-3 数据表、SambaNova SN40L 论文。

[68] Groq TSP 论文、IPU Programming Model、Cerebras WSE-3 数据表、SambaNova SN40L 论文。

模型改变后，可重配置设计靠更新连接与调度来适应新的计算。SambaNova 的数据流组织与 FPGA 的可重配置通路，都允许重新配置部分连接和执行顺序；固定权重设计则围绕特定参数组织存储与计算。[69]

固定权重并不等于只能完成一种任务。改变输入，就能让同一个模型执行不同的上层任务。同一组模型权重既可以处理文档问答，也可以根据代码和测试结果提出修改；每次调用使用自己的上下文状态。因而，硬件可以针对共同的模型执行设计权重通路，同时用可写存储保存各条请求的上下文。下面把权重与上下文分别计量，计算这种分工能够释放多少容量和带宽。

笔者的 OpenTallas 项目研究将不可变权重放入掩模 ROM 的架构；掩模 ROM 是一种在制造时就固定存储内容的只读存储器。下面以项目中的 Qwen3-8B 为例，通过公式估算说明“从独立 ROM 读取权重、从 HBM 读写 KV”的设计如何改变各接口的传输量。[70] 每批只读取一次本步计算所需的权重，约 $W = 15.1$ GB；每条请求完整读取 8,192 个 token 的 KV，约 $K = 1.21$ GB。完整驻留权重约 16.4 GB，每步读取其中参与计算的部分。

先看同一接口原来承担多少读取。权重与 KV 共用 HBM 时，每步读取为 $W + BK$。把权重移到独立 ROM 后，HBM 只承担 $BK$。两种 HBM 读取量之比为

$$S_{\mathrm{HBM}} = \frac{W + BK}{BK} = 1 + \frac{W}{BK}.$$

$B = 1$ 时，HBM 读取量从约 16.3 GB 降至 1.21 GB，读取时间降为原来的约 1/13.5。$B = 16$ 时，KV 已为约 19.3 GB，总量从约 34.5 GB 降至 19.3 GB，约为原来的 1/1.8。batch 增大已经把权重摊到更多请求上，因此进一步省去权重读取的相对收益减小。

令 $BK = W$，可求得 KV 读取量与权重读取量相等的转折点，约为 $B = 12.5$；从整数 batch 13 起，KV 读取量超过权重读取量。图 4-33 用水平线与斜线展示了这一转移。

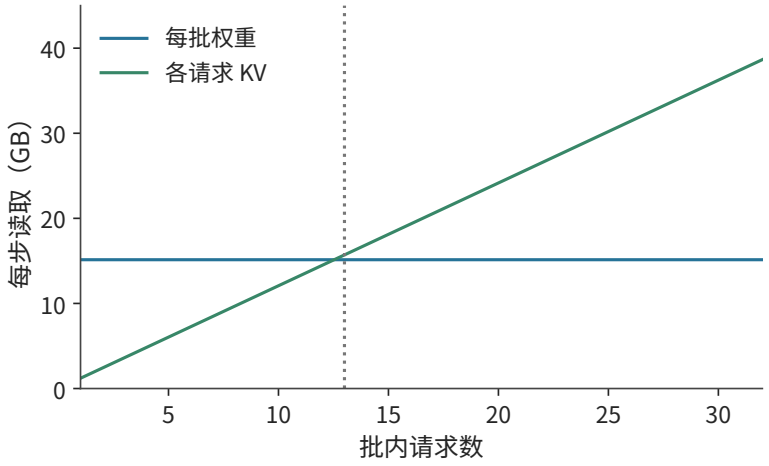



图 4-33　固定权重的同时，各请求仍独立读取 KV。题设每请求保留 8K 上下文，batch 从 13 起 KV 读取超过共享权重读取。

有了独立的 ROM 接口，权重与 KV 就可以同时读取。设两条接口的带宽分别为 $\beta_{\mathrm{ROM}}$、$\beta_{\mathrm{HBM}}$，保持模型、精度、上下文与 batch 不变，两条路径重叠时的存储时间下界为

[69] Groq TSP 论文、IPU Programming Model、Cerebras WSE-3 数据表、SambaNova SN40L 论文。

[70] OpenTallas 案例。

$$t_{\text{memory}} \geq \max\left(\frac{W}{\beta_{\text{ROM}}}, \frac{BK}{\beta_{\text{HBM}}}\right).$$

提高 ROM 带宽会缩短第一项，直到 KV 读取成为较慢的路径。若目标为单用户 10,000 token/s，每 token 只有 100 µs，仅 1.21 GB KV 就要求约 12.1 TB/s 的 HBM 带宽。

独立 ROM 还释放了原来存放权重的 HBM 容量。固定 8K 上下文，以一堆 24 GB 的八层 HBM3E（第 4.1.3 节）作 KV 存储，扣除 2 GiB 工作区后，按每条 1.125 GiB 的 KV 预算，可容纳 18 条请求；原来还需存放完整 BF16 权重时只能容纳 4 条。[71] 上下文增长到 32K，每条请求的 KV 读取增至四倍，前面的权重与 KV 交点便从约 12.5 条降至约 3.13 条。

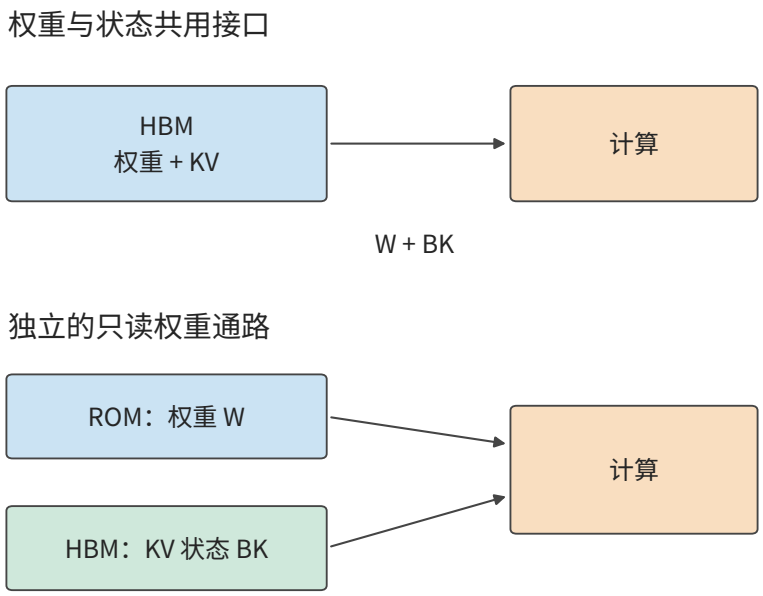



图 4-34　独立只读权重改变两条存储路径。上方权重与 KV 争用 HBM；下方 ROM 提供权重，HBM 存放可写状态。箭头表示读取，KV 还需写入新状态。

同一个 100 µs 时限也可以用来求计算单元数量。工作负载取自该项目的历史执行记录：一组张量操作，共计 15,134,641,792 次。设每条运算通道（lane）在 1 GHz 下每周期完成一次乘加（FMA），100 µs 内所需的运算通道数至少为

$$n_{\text{lane}} \geq \left\lceil \frac{15\,134\,641\,792}{2 \times 10^{9} \times 100 \times 10^{-6}} \right\rceil = 75\,674.$$

256 条运算通道完成这些操作需约 29.6 ms。若分给矩阵计算的时间只有 40 µs，且有效利用率为 60%，需要 315,306 条运算通道，约为最初所需通道数的 4.2 倍。时间预算越紧，留给等待的余量越小，必须配置的算力与存储带宽就越高。

[71] 本章条件比较的逐项复算与计算程序。

## 4.8 根据架构分析性能、选择加速器

第 4.6、4.7 节计算了不同架构如何改变计算、存储与交换工作。本节把这些结果用于设备选择：先合成执行时间模型，再用同一模型比较候选设备，随后用实测校准，最后比较完成任务的成本与能耗。

选择硬件时，还要为后续的切分与调度保留一份可用的资源描述：目标精度与矩阵形状下的有效算力、各级存储容量与带宽、寄存器和共享内存限制，以及设备间的单向带宽、启动延迟和共享出口。第 5 章用其中的局部资源约束 tile（矩阵划分出的数据块）、缓冲和并发，第 6、7 章用互联参数约束跨卡放置与通信组。同一模型的运算和数据需求不变，但这些需求落到哪一级资源、经过哪些路径，会改变执行时间与最合适的切分。

### 4.8.1 从资源预算建立执行时间模型

执行时间来自两种关系：同一硬件单元要依次执行哪些操作，后一项工作要等哪些结果。前者决定各单元需要工作多久，后者决定各步骤的执行顺序。

先看共享资源。同一矩阵单元处理两类各 1 G 次操作的工作，速率分别为 100 和 200 Gops/s，就分别占用 10 ms 和 5 ms，共 15 ms。第 $i$ 类工作在资源 $r$ 上的工作量为 $w_{r,i}$，对应速率为 $p_{r,i}$，则该硬件单元完成这些操作所需的时间为

$$T_r = \sum_i \frac{w_{r,i}}{p_{r,i}}.$$

权重与 KV 共用内存接口时，它们的流量相加；QK 与 PV 共用矩阵单元时，它们的计算时间相加。这些工作即使分布在不同算子中，也消耗同一份硬件能力。

再看独立资源。矩阵和搬移能够同时推进时，总执行时间至少等于耗时最长的单元完成全部操作所需的时间，所以 $T \geq \max_r T_r$。第 4.2 节的稳态分析采用了这一关系：指数运算没有加快时，矩阵单元提前完成后便会空闲，块的完成节奏仍由指数决定。

最后加入依赖。设第一阶段矩阵需 10 µs、内存需 1 µs，第二阶段矩阵需 1 µs、内存需 10 µs；每阶段内部可重叠，第二阶段要等第一阶段完整结束。于是第一阶段用 10 µs，第二阶段也用 10 µs，共 20 µs。只把资源总量汇总，会得到矩阵与内存各 11 µs；差出的 9 µs 来自阶段依赖：第一阶段的矩阵计算与第二阶段的内存访问必须先后进行，无法重叠。[72]

把这些关系用到具体设备上，先要选对速率：矩阵单元的运算速率还与输入和累加精度有关。RTX 4090 的 Tensor Core 执行稠密矩阵运算时的峰值，FP16 输入与 FP16 累加约为 330 TFLOP/s，FP32 累加约为 165 TFLOP/s；本章 BF16 / FP32 投影采用 165.2 TFLOP/s。[73] 对 8.59 GFLOPs，按这两个速率计算，分别需要约 26 和 52 µs。后面算出的读取时间约为 37 µs，因此速率选择会直接改变计算与读取耗时的大小关系。

资源耗时与执行依赖确定以后，还可以求出负载变化时的瓶颈转折点。对本章的 Q 投影，Roofline 模型把这一转折写成算术强度与吞吐之间的关系。

设矩阵峰值为 $P$，片外带宽为 $R$，投影工作量与访问量分别为 $F$、$V$。计算与数据传输所需的时间分别为 $F/P$ 和 $V/R$。计算与访问充分重叠时，较大者决定时间下界，对应 Roofline 模型的吞吐上界为

[72] 各阶段耗时下界的建模说明；实际模型算子范围见Qwen3-8B prefill128 资源结果。

[73] 硬件来源与精度审查及官方基础表。

$$P_{\text{attainable}} \le \min(P, RI), \qquad I = F/V.$$

这里的斜线 $RI$ 描述带宽能够支撑多少运算，水平线 $P$ 描述矩阵单元能够完成多少运算。两者在 $I^* = P/R$ 相交。低于交点时，数据供应较慢；高于交点时，每字节已对应足够多运算，计算速度成为新的瓶颈。这条折线就是硬件允许的上限，任何实测点都落在折线之下；实测吞吐与折线的距离，就是第 1.2.2 节定义的利用率。算术强度高于交点时用 MFU 衡量，低于交点时用 MBU 衡量。

沿用第 4.1 节输入与权重读一次、输出写一次的投影，RTX 4090 在 BF16 输入、FP32 累加时的矩阵运算峰值为 165.2 TFLOP/s，带宽为 1.008 TB/s。将同一组 $F$ 和 $V$ 代入，得到：[74]

| 本次处理的 token 数 $M$ | 矩阵计算时间 | 片外数据传输时间 | 两项耗时的最大值 | 主导资源 |
|---|---|---|---|---|
| $M = 1$ | 约 0.20 µs | 约 33.3 µs | 约 33.3 µs | 内存 |
| $M = 256$ | 约 52 µs | 约 37.4 µs | 约 52 µs | 矩阵 |

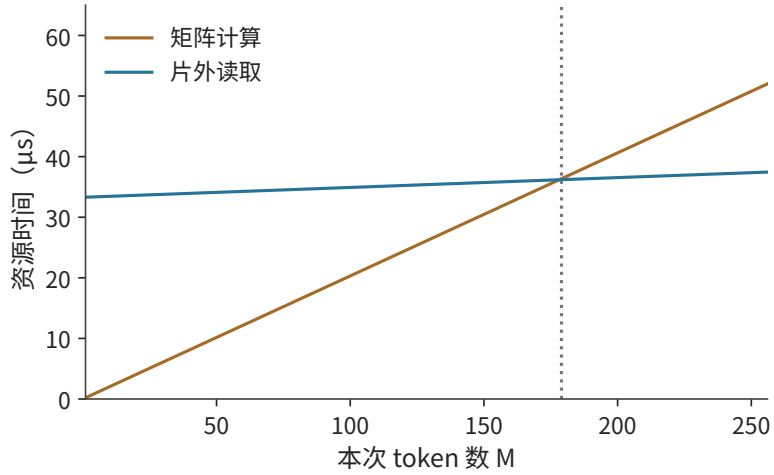



图 4-35　同一 Q 投影的计算与访存耗时随输入行数的变化。计算量按行数增长，片外访问同时包含固定权重和增长的输入输出；从 179 行起计算项较长。

还可以求出转折发生时的输入行数。令 $d = 4096$，第 4.1 节的强度可写成 $I(M) = Md/(d + 2M)$。令它等于 $I^* = P/R$，得到

$$M^* = \frac{I^* d}{d - 2I^*}.$$

代入未经四舍五入的速率数值，$I^* \approx 164$ FLOPs/byte，$M^* \approx 178.1$。因此从整数行数 179 起，矩阵计算时间超过数据传输时间。转折点把“较大 batch 更适合矩阵单元”变成了可以计算的行数阈值。[75]

**实验 4-5 ・核心：batch 如何改变不同架构的性能瓶颈**

推导并画出 Q 投影从 1 到 256 行的算术强度，分别将矩阵计算吞吐率、内存带宽提高到原来的两倍，求计算时间与读取时间相等时的输入行数。随后把注意力与专家矩阵加入资源表，解释上下文长度和各专家输入

[74] Q 投影，RTX 4090，M=1；M=256。

[75] 三槽流水、计算翻倍变体及设计转折点由教学推导脚本生成，完整时序见推导数据。

行数的分布，怎样改变各单元的处理时间及瓶颈所在。选择两种具体加速器，分别用其资源参数估算同一任务的执行时间，并比较瓶颈。

第 4.8.2 节用这一方法从单个投影扩展到整个模型，分别计算 decode 的主要读取与 prefill 的矩阵工作，并检查哪些设备能够容纳所需状态。

### 4.8.2 用同一模型比较候选加速器

先固定一个便于复算的例子：Qwen3-8B、BF16、单请求，已有 8,191 个历史位置，本步追加后为 8,192 个。权重常驻约 16.381 GB，KV 约 1.208 GB，工作区取 2 GiB，总预算约 19.737 GB。一次 decode 的主要权重、KV 接口载荷为 16.345 GB。词嵌入每步读取当前 token 对应的行，输出头读取完整权重，所以常驻权重与每步权重读取量不同。[76]

再计算矩阵工作。Qwen3-8B 有 36 层，隐藏宽度 4,096，前馈宽度 12,288，32 个查询头、8 个 KV 头，头维度为 128。每层 Q、K、V、O 投影与三次前馈投影的矩阵参数数目为

$$W_{\text{layer}} = 2 \times 4096^2 + 2 \times 4096 \times 1024 + 3 \times 4096 \times 12288.$$

每 token 的各层线性运算为 $2 \times 36 W_{\text{layer}} \approx 13.89$ GFLOPs。一次 8K decode 的 QK 与 PV 另需 $4 \times 36 \times 8192 \times 4096 \approx 4.832$ GFLOPs，输出头约 1.245 GFLOPs，共约 19.97 GFLOPs。

再看 prefill：处理 4,096 个新输入 token 时，线性运算按 token 数增加。因果注意力的可见位置对数为 $4096 \times 4097/2$；每对在 32 个头上各做一次 QK 内积与一次 PV 累加，合计 $4 \times 4096$ 次操作。乘以 36 层，有效 QK、PV 运算为 $2 \times 36 \times 4096 \times 4096 \times 4097$。最后一个位置计算输出头，整个 prefill 的矩阵工作约为 61.85 TFLOPs。将这两个阶段分别代入硬件参数表，得到图 4-36。

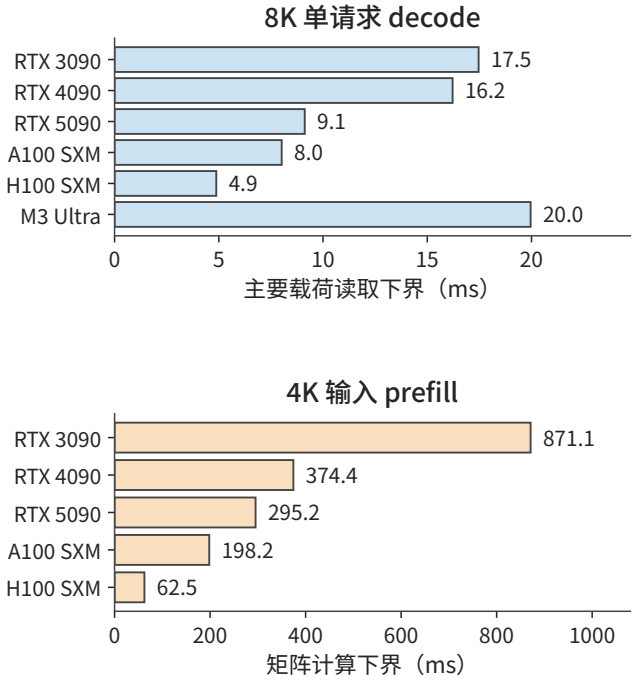



图 4-36　同一 Qwen3-8B 的两种阶段预算。单请求 decode 更直接反映读取带宽，4K prefill 的矩阵预算更直接反映匹配精度的矩阵速率。两图横轴分别标注所计算的时间。

[76] 存储代际比较结果及计算说明。

这组结果给出了几个直接的判断。3090 换 4090，单请求 decode 的读取下界只缩短约 7.1%，4K prefill 的矩阵时间却缩短约 57.0%。A100 换 H100，读取时间缩短约 39.1%，矩阵时间缩短约 68.5%；再换 H200，读取时间继续下降，矩阵时间基本不变。RTX 5090 与 RTX PRO 6000 读取时间相同，但后者的矩阵吞吐和容量更大。对输入较长的文档问答，4090 和 H100 的矩阵增量首先缩短 prefill；对单请求持续生成，5090 和 H200 的带宽增量直接缩短每步读取。

**容量如何改变可选择的设备？** BF16 Qwen3-8B 的单请求预算约 19.737 GB，表中的 24 GB 卡能够容纳；增加到八条 8K 请求，总预算约为 28.193 GB，便超过 3090 / 4090 的 24 GB，进入 5090 的 32 GB 容量范围。

**同样在本地运行，Qwen3-8B 与 Qwen3-235B-A22B 应怎样选？** 先比较单请求生成。Qwen3-8B 使用 BF16、8K 上下文，在 RTX 4090、5090、M3 Ultra 上都能驻留。把表中的每步读取时间换成每秒 token 数，分别得到 $1000/16.22 \approx 61.7$、$1000/9.12 \approx 109.6$ 和 $1000/19.96 \approx 50.1$ token/s 的读取上限。该 8B 稠密模型没有用到 Ultra 的额外容量，5090 的高带宽直接体现为更高的生成预算。

再换成第 4.6.3 节的 Qwen3-235B-A22B，采用 4-bit 权重、单请求和 8K 上下文。该模型需要约 126.867 GB 的模型、KV 与工作区空间，3090、4090、5090 均无法单卡驻留，M3 Ultra 256 GB 和 H200 141 GB 可以。每步读取约 13.697 GB，M3 Ultra 用约 16.72 ms，H200 用约 2.85 ms，对应约 59.8 与 350.4 token/s 的读取上限。Ultra 的大内存让这一 235B 的 MoE 能在一台本地整机上运行，H200 的高带宽则把同一步读取缩短到约六分之一。

多卡运行的张量切分与专家 dispatch 在第 6 章展开。

### 4.8.3 预测与实测为什么有差距

第 4.8.2 节的设备比较给出了容量与主要资源预算。运行库为数学算子选择具体 kernel，缓存改变片外流量，提交和同步也占用时间。测量的用途是识别这些执行步骤，补全第 4.8.1 节的时间模型。

先把 Qwen3-8B 的读取预算换成一次具体的生成任务，放到本书的实测平台 RTX PRO 6000 Blackwell Workstation 上。仍用 BF16、单请求、8K 上下文，每步载荷 16.345 GB，按 1,792 GB/s 的峰值带宽读取需 9.12 ms，对应约 109.6 token/s；取这一上下文附近的 128 个 decode 步骤，合计约 1.17 s。

第 8 章的实验 8-1 在同一张卡上用 vLLM 运行了这一条件：8K 上下文、batch 1，纯 decode 迭代耗时的三轮中位数为 25.83 ms。三轮分别为 26.43、16.18、25.83 ms，其中一轮明显较快，另两轮相差不到 3%；取中位数，可以避免单独一轮的偏离影响下面的比较。[77] 生成速度约为 38.7 token/s，128 步约需 3.31 s；读取下界只占实测时间的 35.3%，实测比下界多出约 16.7 ms。

多出的时间并非来自读取速度：同一实验的 2K 上下文每步少读 0.906 GB KV，按峰值带宽应省约 0.51 ms，实测一轮却是 26.26 ms，并不比 8K 的 25.83 ms 短。batch 增至 64 时，2K 条件每步读取增至 34.46 GB，读取下界为 19.23 ms，实测一轮只增至 29.63 ms，等效带宽约 1.16 TB/s，为峰值的 65%。可见 batch 1 时每步时间主要由一段与读取量无关的固定工作决定：实验以 eager 模式运行，kernel 逐个启动，此外还有向量运算、采样与同步。在这套软硬件上，提高带宽利用率几乎不能加快单请求生成；要缩短每步时间，先要减少这部分固定工作，第 5 章讨论的 kernel 融合与减少启动次数，针对的正是这部分时间。batch 增大后，固定工作由更多 token 分摊，读取才重新成为主要项。

按第 1.2.2 节的定义，35.3% 与 65% 就是这两种条件下的 MBU。第 1.3.4 节所说的两类差距在这里都能看到。下文的计数器记录将说明，模型中的逻辑读取请求并不等于 DRAM 流量，缓存命中的部分不占用显存带

[77] 实验 8-1 batch 扫描及其逐行效率汇总：RTX PRO 6000 Blackwell Workstation，Qwen3-8B BF16，vLLM 0.23.0，eager 模式；每行取每轮纯 decode 迭代的中位数，再取三轮中位数。与读取下界的比值由教学推导脚本的 `measured_decode` 项算出。

宽，这一项要修正的是模型口径；kernel 逐个启动、主机提交与同步留下的间隙，则是可以去掉的实现开销。先修正口径，再去掉开销，两步都不能少。

要把多出的时间拆到具体步骤，还需要计时、流量和 kernel 记录。下面用本书已有的 Q 投影实测，说明如何从这些记录中找到时间差的来源。

配套实验在 M2 Max 与 RTX PRO 6000 Blackwell Workstation 上运行相同的 $K = N = 4096$ 投影，输入、权重、输出为 BF16。为比较不同的权重访问方式，在每个平台上准备 16 份内容相同、地址不同的 32 MiB 权重。复用组始终使用同一地址，轮换组依次使用不同地址。每轮执行 16 次调用，记录包括主机提交与同步在内的整轮耗时，再除以 16，得到该轮每次调用的平均耗时。共运行 11 轮，取这 11 个平均值的中位数。[78]

| 本次处理的 token 数 $M$ | M2 Max：复用 / 轮换 | RTX PRO 6000：复用 / 轮换 |
|---|---|---|
| $M = 1$ | 约 166 / 183 µs | 约 42.9 / 51.9 µs |
| $M = 256$ | 均约 1.835 ms | 约 32.6 / 33.5 µs |

先看 RTX 单行：轮换比复用约慢 21%。一种自然的解释是：反复使用同一份权重，权重就留在了缓存里，减少了 DRAM 读取。该解释可以直接检验：若少读权重是差别来源，复用权重时，从片外读取的字节数应当更少。实验另用 NVIDIA 的 GPU kernel 性能分析工具 Nsight Compute，先按指定方式访问权重，再记录一次待测调用，汇总全部 kernel 的访问计数。

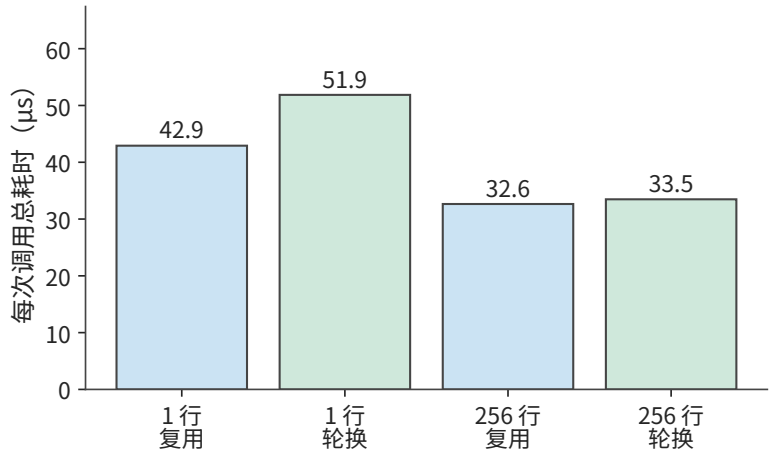



图 4-37　RTX PRO 6000 的投影总耗时。每个条件测十一轮、每轮十六次调用，取每轮平均耗时的中位数；计时包含提交与同步。“复用”表示多次调用读取相同权重地址；“轮换”表示调用之间更换权重地址。

单行在两种权重访问方式下的 DRAM 读取都是约 32 MiB，恰好对应整份权重的规模。两种条件读取了相同规模的片外权重，单行的时间差需要沿提交、执行与等待进一步定位。256 行则是另一种情况：复用同一份权重时只读 256 bytes，轮换不同的权重副本时约读 32.1 MiB，但常规计时都在 33 µs 左右。缓存改变了片外流量，而调用总时间还包含其他计算、传输与等待。

第 4.3 节区分的存储层次在这里变得可见。256 行的两种权重访问方式都有约 147 MiB L2 请求，远大于近乎为零的那次 DRAM 读取。缓存命中只是数据改由片上提供，计算单元的读取请求和局部传输仍然存在。源程序中的同一份权重，分块执行后会由多个计算块反复请求。

片上请求解释了为什么 DRAM 读取减少后仍有数据访问。再看计算。库把一次投影拆成了更多步骤：单行调用使用一个 GEMV kernel，256 行使用 GEMM 加 split-K 归约两个 kernel。split-K 把内积维度上的计算交

[78] 实验 4-6 全部说明与原始记录、投影计时汇总、实际 DRAM / L2 计数。

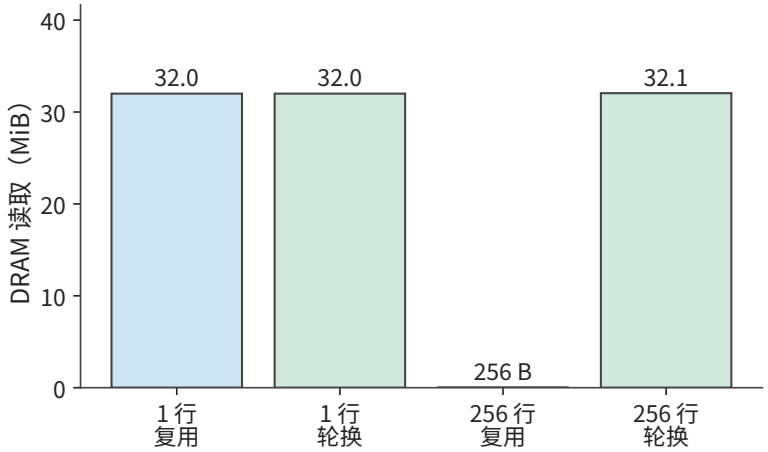



图 4-38　相同四个条件下另行采集的 DRAM 读取计数。单行均约 32 MiB，256 行复用为 256 bytes、轮换约 32.1 MiB。访问计数与常规计时分别测量。“复用”与“轮换”分别表示保持和更换权重地址。

给多个计算块，先形成部分输出，再由归约合并。这样能让更多计算块并行执行，但也要保存和读取部分结果，最后再做一次归约。因而第 4.1 节的一次数学乘法，在这里展开成多个阶段。

再按 RTX PRO 6000 的参数估算输入和权重均从片外读取时的执行时间：BF16 输入、FP32 累加的稠密矩阵运算峰值约 504 TFLOP/s、带宽 1.792 TB/s，两种行数的下界约为 18.7 和 21.1 µs。模型给出矩阵计算与片外读取所需的时间，计数器揭示缓存参与后各级接口的流量，kernel 路径解释额外的合并工作；总耗时还包含提交、同步与执行间隙。[79]

要进一步定位差距，可以把“忙了多久”与“工作时执行得多快”分开。设某硬件单元按理想速率完成全部操作需时 $S_r$，实际处于工作状态的时间为 $A_r$，任务总时间为 $T$，则

$$\frac{S_r}{T} = \frac{S_r}{A_r} \times \frac{A_r}{T}.$$

前一项表示工作时的实际速率与理想速率之比，后一项表示工作时间占总时间的比例。两个任务都耗时 100 µs，理想计算时间也都是 40 µs。执行第一个任务时，该单元只工作了 40 µs，工作时达到理想速率，但其余 60 µs 都在等待；执行第二个任务时，该单元工作了 80 µs，速率只有理想值的一半，等待时间则只有 20 µs。两者整体比值都是 40%，对应的优化方向却不同。

第一个任务更需要提前准备数据、安排更多相互独立的计算，或减少等待前一步结果的时间，类似第 4.4 节两槽到三槽的改动；第二个任务更需要改善执行粒度、指令与局部访问，类似第 4.2 节补齐浪费和分组调用的分析。[80]

**实验 4-6 ・延伸：用 Mac 与 RTX 实测检验执行时间预测**

根据投影在两种平台上的实测总耗时，提出两个能够解释耗时差异的原因，再读取 DRAM、L2 与 kernel 记录，为每个解释列出预期现象和实际现象。随后画出 256 行调用的矩阵、部分结果与归约路径，指出需要哪些阶段时间才能区分计算单元工作时的执行效率与等待时间。将所得结果与假设输入和权重均从片外读取的模型对照。

[79] 配对投影成本与平均功率条件；RTX PRO 6000 的 600 W 取自硬件输入表，每次调用的能耗与 Mac 持平功率由教学推导脚本的 `energy` 项算出，见推导数据。

[80] 从算子利用率到执行中的等待及论文阅读记录。

### 4.8.4 在容量、速度、功耗与成本之间做选择

第 4.8.3 节得到任务完成时间，本节将其换算为成本与能耗。先比较按使用时间计费的方案，再把空闲时间和专用架构的固定投入计入。

设两台整机的小时成本为 $c_A, c_B$，完成同一任务的运行时间为 $t_A, t_B$。按运行时间计费时，每任务成本与 $ct$ 成正比，因此

$$K_A < K_B \quad \Longleftrightarrow \quad \frac{c_A}{c_B} < \frac{t_B}{t_A}.$$

右侧给出了每小时成本可以相差多少倍：如果 A 的速度是 B 的两倍，那么 A 每小时的成本低于 B 的两倍时，每个任务仍然更便宜。把小时成本换成平均功率，同一不等式比较的就是每任务能耗。第 4.8.3 节复用组中，Mac / RTX 时间比分别约为 3.9（单行）和 56（256 行）。RTX 一侧只计显卡，按 RTX PRO 6000 Workstation 的 600 W 功耗上限计算，一次单行调用最多耗能约 25.8 mJ，一次 256 行调用约 19.6 mJ。M2 Max 的官方资料没有给出整机功率；由上式反推，Mac 在单行调用期间的平均功率低于约 155 W 时，每次调用耗能就比 RTX 少，256 行时则要低于约 10.7 W。机器不变，仅改变矩阵尺寸，Mac 要保持能耗优势，允许的功率就从约 155 W 降到约 10.7 W。[81]

长期运行的服务还要计算空闲期间的成本。设一台机器处理请求时每秒生成 $q$ 个有效 token，每小时用于处理请求的时间比例为 $u$，则每小时产出为 $3600uq$，每 token 成本为 $c/(3600uq)$。生成速度翻倍而利用率减半，每小时产出不变，单位成本也不变。

能耗同样由功率与时间共同决定。以第 4.8.2 节的单请求 8K decode 为例，设两张卡都以功耗上限运行、每步用时等于读取下界：RTX 4090 为 450 W、16.22 ms，每步约 7.30 J；RTX 5090 为 575 W、9.12 ms，每步约 5.24 J。[82] 功率增加约 28%，每步能耗反而减少约 28%。一般形式为

$$E = \int_0^T P(t)\, dt.$$

对长期服务，统计一段时间内的总耗电量，再除以这段时间的有效产出，就能把空闲和预热期间的耗电一并计入。缩短运行时间可以减少执行任务时的耗电，提高利用率则能减少每个任务分摊的空闲耗电。第 4.1.3 节的能耗分账给出了一步 decode 耗电的量级估计：单请求、8K 上下文时每 token 0.534 J，其中 0.481 J 用于读取权重；batch 为 32 时，每 token 能耗降到约 0.068 J。这笔分账不含控制、时钟、静态漏电与空闲期间的功耗，所以用总耗电除以有效产出，得到的每 token 能耗会高于这一估计。[83]

专用化另有固定投入。设相对通用方案增加的固定成本为 $F_0$，每个有效输出节省可变成本 $\Delta c$，回本量为

$$V^* = \frac{F_0}{\Delta c}.$$

模型会被更新或替换，因此专用系统能用于该模型的时间有限。设有 $D$ 台机器，每台处理请求时的生成速

[81] 配对投影成本与平均功率条件；RTX PRO 6000 的 600 W 取自硬件输入表，每次调用的能耗与 Mac 持平功率由教学推导脚本的 `energy` 项算出，见推导数据。

[82] RTX Blackwell 架构白皮书附录 A 表 3：RTX 4090 为 24 GB GDDR6X、1,008 GB/s、PCIe Gen 4、TGP 450 W，RTX 5090 为 32 GB GDDR7、1,792 GB/s、PCIe Gen 5、TGP 575 W；A100 80GB 数据手册，PCIe 4.0 为 64 GB/s（双向合计）。H2D 与显存读取时间见教学推导脚本的 `host_link` 项。

[83] 能耗分账结果，读取单请求 8K decode 结果的字节数与 FLOPs，按上表每字节与每 FLOP 能耗相加；运行 `python3 calculations/calc.py energy-ledger --format md` 可以复算。

率为 $q$，利用率为 $u$，模型可用时间为 $L$，总产出为 $DuqL$。这段时间内节省的运行成本达到固定投入后，专用方案才算回本。

**例 4-3：专用化能否在模型寿命内回本？**

增量固定成本为 2,000 万美元，每百万合格输出 token 节省 0.50 美元。部署 100 台，每台处理请求时产出 10,000 token/s，利用率 50%，模型经济寿命一年，每年按 365 天计算。

**比较回本所需输出量与一年可完成的输出量。**回本量为 $20\,000\,000/(0.50/10^6) = 4 \times 10^{13}$ token，即 40 万亿。一年产出为 $100 \times 10\,000 \times 0.5 \times 31\,536\,000 \approx 1.58 \times 10^{13}$ token，即约 15.8 万亿。

**在一年内回本需要多大的实际服务规模？**一年产出约为回本量的 39%。在单机速率与利用率保持不变时，至少需要 254 台运行一年。

**模型寿命减半或利用率下降后的回本规模。**模型半年更换，每台可用时间减半，至少需要 508 台机器维持给定利用率；若利用率从 50% 降至 25%，结果相同。

本例每年约节省 788 万美元，低于 2,000 万美元的固定投入；达到 40 万亿 token 的回本量需要约 2.54 年，超过题设的一年模型寿命。[84]

**实验 4-7 ・延伸：负载改变后，芯片资源改动是否仍有收益**

选择一种架构和一种典型负载，提出一次资源改动。先用工作量、流量和依赖推导时间变化，再计算增加的容量及减少的等待。随后改变 batch 或上下文长度，求原有改动不再节省时间时的临界值。若方案需要额外固定投入，再根据每项任务节省的成本、加速器利用率和模型寿命，求收回固定投入所需的最低服务量。

**实验 4-8 ・延伸：从能耗分账到功率封顶**

沿用第 4.1.3 节的能耗表与 Qwen3-8B 单请求 8K decode 一步的读取字节数。(a) 把 batch 分别取 8、32、128，分别计算每 token 的权重、KV 与计算能耗，求权重项低于计算项的最小 batch。(b) 把上下文改为 32K，求 KV 项等于权重项的 batch，并解释此后继续增大 batch 为什么不再明显降低每 token 能耗。(c) 选择一张卡的 TDP、峰值算力与 HBM 带宽，求峰值速率下每 FLOP 的能量预算 $b$；设实际每 FLOP 能耗为 $1.5b$，按 $P \propto fV^2$ 分别求电压不变与电压随频率下降两种情况下的持续频率比；再把 (a) 中 batch 为 32 时一步的能量（32 乘以每 token 能耗）除以持续频率下这一步的时间，得到平均功率，比较它与 TDP 的差距。

## 本章小结

加速器分析从模型工作量出发：用状态大小筛选容量，用流量与带宽计算读取时间，用矩阵形状、精度与指令速率计算运算时间，再沿依赖安排执行。三家架构的演进说明，模型需求会改变资源配比。访问计数、kernel 记录与计时将这些预算落实为运行速度和任务成本。时间与能耗是两本账：带宽决定一步多快，每字节能耗与复用次数决定一步耗多少焦耳，功率上限又反过来压低可持续的频率。第 5 章继续研究布局、融合与调度，说明软件如何用好这些硬件资源。

[84] OpenTallas 案例。

# 第 5 章 算子与运行时

同一份输入，刚用于一个输出，接着又要用于旁边的输出：若仍留在计算单元附近，下一次就可以直接使用；若已经换走，就要重新搬来。矩阵乘法的程序需要安排这些使用次序，并给输入和部分和留下合适的空间。仅仅改变一次处理多少输出、先算哪一块，就会改变重复搬移的次数，以及能够同时执行的任务数。

本章的矩阵乘算例需要约 103 GFLOPs 的运算，输入、权重和输出总共占 128 MiB，一种分块实现的读写量却达到约 3 GiB。扩大输出块可以让输入多用几次，把读写量减半，但每个块的局部存储需求也从 24 KiB 增至 80 KiB。有了这些数字，就能判断减少搬移节省的时间，能否抵消同时执行的块数变少带来的损失。

第 4 章介绍了加速器的计算能力、存储容量和数据传输带宽。本章讨论程序如何利用这些资源。以一段 FFN 为例，先说明主机如何发起计算、加速器如何完成任务，再分析矩阵乘的循环与分块，以及相邻算子如何共享中间结果。随后讨论编译器和运行时如何实现这些优化，最后计算这些优化能让完整请求节省多少时间。

本章主要使用 Qwen3-8B 的 FFN 作为算例：隐藏宽度 $H = 4096$，FFN 中间宽度 $F = 12288$。先同时处理 $M = 1024$ 个 token，将各 token 的 4096 维特征向量排成输入矩阵 X，计算一支投影 $C = XW$；X、W 和 C 使用 BF16，每个元素占 2 bytes，矩阵乘使用 FP32 累加。M 表示一次调用处理的 token 数，每个 token 对应输入矩阵的一行。这些 token 可以来自 prefill 的一段输入，也可以来自同一批中的多个请求；单请求逐 token decode 时，M 为 1。[1]

| 对象 | 形状 | 大小 | 在计算中的作用 |
|---|---|---|---|
| 输入 X | $[1024, 4096]$ | 8 MiB | 每个 token 的特征向量占一行，投影后得到该 token 的新特征向量 |
| 权重 W | $[4096, 12288]$ | 96 MiB | 由 1024 个 token 的投影计算共享 |
| 投影 C | $[1024, 12288]$ | 24 MiB | 随后做激活等处理 |

本章围绕三个问题展开：哪些工作在重复执行，哪些数据能够复用，哪些计算或等待决定了总时间。回答这些问题时，程序在加速器上的执行单位是 kernel。分析读写量、缓冲容量和 kernel 数，可以了解程序做了多少工作；再结合这些工作的执行顺序，才能计算程序需要多长时间。

## 5.1 加速器执行过程中的提交、搬移与同步

先考虑输入和输出都在 CPU 内存中的矩阵乘。CPU 准备输入，将输入送入 GPU，发起计算，再取回结果。这一过程需要两个处理器协作。理解两者何时独立执行、何时等待对方，是后续融合、流水和图重放的基础。

### 5.1.1 从框架调用到 kernel launch

PyTorch 是表达张量运算并组织模型执行的软件框架。调用其矩阵乘法时，CPU 首先执行框架代码：读取张量形状和数据类型，选择实现，准备地址和参数，再通过运行时与驱动向 GPU 提交任务。向加速器发起一次 kernel 执行称为 **kernel launch**。运行时负责提交任务、管理内存和同步执行；编译器则事先或在首次调用时生成加速器代码。

[1] 固定 Qwen3-8B 配置；模型算子与实现。

这里有三个层次：算子描述数学工作，例如矩阵乘和激活；kernel 在加速器上完成这项工作；launch 是 CPU 发起这次执行的动作。一个矩阵乘可以由多个 kernel 完成，几个逐元素算子也可以由同一个 kernel 完成。因此，既可以优化加速器上的计算，也可以减少主机准备和提交任务的开销。

CUDA 执行一个 kernel 时会启动多个线程。若干线程组成一个线程块（thread block），所有线程块组成一个线程网格（grid）。矩阵乘通常把输出矩阵分成多个 tile，交给不同线程块计算，块内线程协作读取输入并累加结果。加速器将线程块安排到有可用资源的 SM 上。较早完成的块释放资源，后续块便可开始；因此，grid 中的线程块可以分批执行。

kernel launch 采用异步提交。调用返回时，CPU 已完成这次提交，GPU 则按照任务队列和依赖关系执行。CPU 可以立即准备下一项任务。只有当 CPU 要用加速器的结果，或者要覆盖加速器还在使用的数据时，CPU 才需要等相应的加速器操作完成。[2]

这种分工形成一条简单流水：CPU 准备并提交任务，GPU 执行任务。大矩阵计算持续较久，CPU 有时间准备下一次调用；小算子迅速结束，GPU 更容易执行完队列中的任务，停下来等待 CPU 提交下一项。因此，可以从两方面加速：减少加速器等待主机的时间，以及缩短 kernel 本身的执行时间。

### 5.1.2 主机与加速器之间的数据复制

CPU 能够提前提交下一项任务，加速器却只有在输入到达后才能计算。先分析数据的传输过程，就能看清提交任务后、开始计算前还需要完成哪些工作。除了第 4 章介绍的 H2D 与 D2H，有独立显存的 GPU 还常在显存内部复制，常见的复制方向有三种：

| 名称 | 复制方向 | 典型对象 |
|---|---|---|
| H2D | 主机内存 → 显存 | 当前 batch 的输入张量 |
| D2H | 显存 → 主机内存 | CPU 要使用的计算结果 |
| D2D | 显存 → 显存 | 固定输入缓冲或重排结果 |

上述矩阵乘的执行过程是：

> CPU 准备输入 → H2D → GPU 计算 → D2H → CPU 使用结果。

图 5-1 画出这些数据在两侧内存中的位置和复制方向。

运行完整模型时，并非每次调用都要重新传入所有数据。权重加载后可以一直保留在显存中，层间激活可以由下一个 kernel 直接读取，KV 也可以在后续 decode 步中继续使用。如果采样在 GPU 上完成，CPU 只需接收少量 token ID。因此，主机与加速器之间的复制次数，与加速器内部的读取次数并不相同：一份权重可以只传入一次，却在加速器上读取多次。

**例 5-1：主机到 GPU 的输入传输需要多久？** $[8192, 4096]$ 的 BF16 张量占 $8192 \times 4096 \times 2 = 64$ MiB。RTX PRO 6000 经 PCIe Gen5 x16 接到主机，每个方向的标称带宽为 64 GB/s，[3]传输时间为：

$$T_{\mathrm{H2D}} = \frac{64\ \mathrm{MiB}}{64\ \mathrm{GB/s}} \approx 1.05\ \mathrm{ms}.$$

[2]CUDA 执行模型；CUDA Runtime API 的同步语义；CUDA stream 管理与event 管理；PyTorch 锁页内存与异步复制教程。主机复制与缓冲复用另见本章引用的本地计算材料。

[3]RTX PRO 6000 Blackwell 工作站版规格列出系统接口为 PCIe 5.0 x16；NVIDIA H100 规格给出 PCIe Gen5 x16 为 128 GB/s，是收发两个方向的合计，每个方向 64 GB/s。

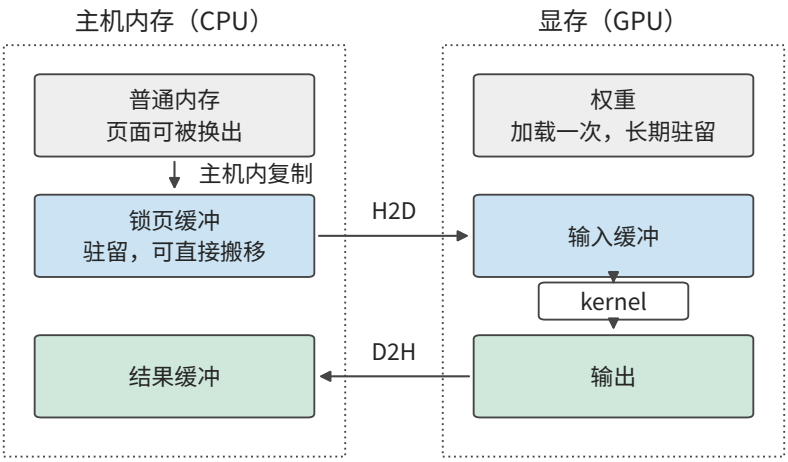



图 5-1：H2D 把输入从主机的锁页缓冲搬进显存，D2H 把结果搬回主机；权重加载一次后留在显存，kernel 直接读取显存中的输入并写出输出。普通内存中的数据要先复制到锁页缓冲，才能由复制引擎直接搬移。

将输入分成两份 32 MiB 张量，每份约需 0.52 ms，总传输量保持不变。这样做的好处是第一份数据可以更早到达：GPU 开始计算前半批时，复制引擎继续传送后半批。第 5.3 节将进一步计算传输与计算重叠后的总时间。[4]

主机内存的类型也影响复制过程。普通内存的页面由操作系统管理；**锁页内存（pinned memory）**在使用期间保持驻留，便于加速器直接搬移。从普通内存传输数据时，运行时常先将数据复制到内部的锁页缓冲：CPU 先复制一遍，再由加速器读取。这就是图 5-1 左侧的主机内复制。若每批都把 64 MiB 数据复制进新建的锁页缓冲，就在 H2D 之前多做了一次主机复制。让 CPU 直接在可复用的锁页缓冲中准备输入，可以省去这次中转和反复分配。[5]

### 5.1.3 stream、event 与完成顺序

数据传到哪里，决定哪个处理器能够使用它；数据何时传完，则决定后续计算何时能够开始。异步调用返回后，复制可能仍在进行，需要用执行顺序保证计算不会读到尚未传完的输入。CUDA 的 **stream（流）**是一串按顺序执行的加速器任务。把 H2D 和读取该输入的 kernel 放在同一 stream 中，计算就排在复制之后；CPU 可以连续提交两项工作。

若复制和计算放在不同 stream 中，就用 **event（事件）**指定跨流的执行顺序。复制流在 H2D 后记录事件，计算流等待事件，再使用这份输入。等待发生在计算流的加速器任务序列中，CPU 仍可继续提交其他工作。

图 5-2 把两条流和两个事件画在同一条时间线上。

事件也可以用来确定缓冲何时能够再次使用。设当前输入从主机缓冲复制到 GPU 的缓冲槽 A。复制完成后，CPU 就可以往主机缓冲写入下一批输入；槽 A 则要等 GPU 用完当前输入后才能覆盖。两块缓冲虽然保存同一批数据，却要在不同的时刻才能重新使用。

[4]流式顺序与缓冲预算；主机搬移与缓冲区占用时间。

[5]CUDA 执行模型；CUDA Runtime API 的同步语义；CUDA stream 管理与event 管理；PyTorch 锁页内存与异步复制教程。主机复制与缓冲复用另见本章引用的本地计算材料。

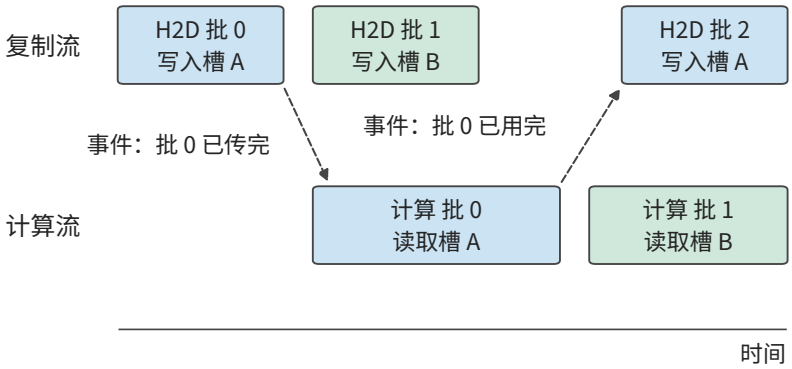



图 5-2：复制流依次传入各批输入，计算流读取它们。计算批 0 要等 “批 0 已传完” 事件；批 2 要重新写入槽 A，必须等 “批 0 已用完” 事件。虚线箭头表示等待事件，不搬移数据。两个槽的交替使用在第 5.3.2 节展开。

| 缓冲 | 最后使用者 | 随后的动作 |
|---|---|---|
| H2D 的主机源缓冲 | 复制引擎读取 | CPU 写入下一批输入 |
| GPU 输入缓冲 | 读取该输入的 kernel | 复制引擎写入下一批输入 |
| D2H 的加速器源缓冲 | 复制引擎读取 | GPU 写入新结果 |
| D2H 的主机目标缓冲 | 复制引擎写入 | CPU 读取当前结果 |

因此，应在最后一次读写之后记录事件，让随后使用这块缓冲的操作等待该事件。CPU 需要 D2H 结果时，可以等待对应事件；其他无关的加速器任务继续运行。第 5.3 节的双缓冲正是把两套这样的使用顺序交错起来。

这套顺序也解释了程序为什么会停住不动。每一次等待都指向一个具体的事件，只要该事件始终未被记录，等待就不会结束；表现出来是程序停住，而不是变慢。上表中四块缓冲各自的最后使用者，就是四个容易漏记的事件：漏掉 GPU 输入缓冲的 “已用完” 事件，复制引擎就一直不能写入下一批。跨流的循环等待也一样：A 流等 B 流记录的事件，B 流的这项任务又排在 A 流尚未完成的工作之后，两条流都不会前进。多卡执行还多一种来源：集合通信要求参与的每张卡都发起同一次调用，只要有一张卡少发起一次，其余的卡就会一直等待这个不会到来的对端（第 6.4 节）。

排查时先看被等的那一侧是否还在前进：仍有结果产出，就只是慢，应回到第 5.1.4 节的时间线逐段计时；完全不动，才是在等一个不会到来的事件，第 10.4.5 节的慢节点属于前一种。为等待设置超时，超时后打印各流尚未完成的任务，就能确定缺的是哪个事件。

### 5.1.4 从提交时间到结果可用时间

把复制、计算和缓冲使用的顺序连起来，才能确定结果何时可用。图 5-3 将这一顺序画成时间线。

**例 5-2：为什么提交耗时、加速器耗时与完整调用耗时不同？** 输入已在主机准备好。CPU 在 0–3 μs 内提交全部任务，第一项加速器工作从 3 μs 开始；H2D、kernel、D2H 分别用 8、20、4 μs，加速器按依赖连续执行。

CPU 提交任务用时 3 μs，kernel 执行用时 20 μs，CPU 到第 35 μs 才能使用结果。若把 kernel 的执行时间从 20 μs 缩短到 5 μs，结果就能在 $3+8+5+4=20$ μs 时使用。kernel 速度提高到原来的四倍，整次执行则从 35 μs 降至 20 μs，加速比约为 1.8；剩余 15 μs 由提交和复制构成。

例 5-2 问到的三种耗时，正对应三种计时位置。测量结果取决于在什么位置开始和结束计时。在异步调用前后读取 CPU 时钟，测得提交所用的时间；若在结束计时前等待结果，则测得从发起到完成的时间。CUDA

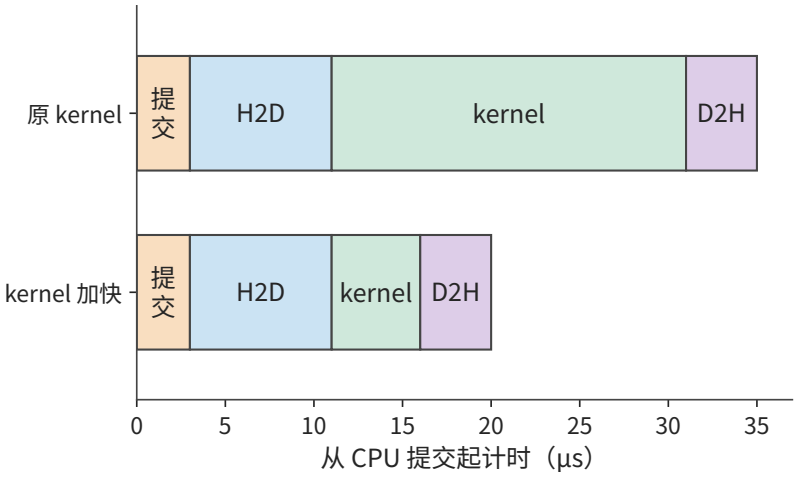



图 5-3：CPU 提交、H2D 输入复制、kernel 执行与 D2H 结果返回依次发生。上方 kernel 执行 20 μs，结果在第 35 μs 可用；下方 kernel 执行 5 μs，结果在第 20 μs 可用。

event 在加速器任务序列中标出起止位置，测得加速器执行这段任务所用的时间。性能分析工具（profiler）把主机调用、加速器计算和复制画在同一条时间线上，提交之后的排队、依赖等待和执行就能逐段看清。

图 5-3 的时间线从输入准备好开始。若程序尚未编译，时间线之前还要加上一次准备工作：首次调用可能需要编译代码或分配工作区。假设首次编译用 100 ms，随后每次执行 1 ms，调用一次共 101 ms，调用 100 次共 200 ms，平均每次 2 ms。调用次数越多，平均分摊的编译时间越少，而每次执行所需的 1 ms 保持不变。第 5.5 节将用同一方法判断分桶（把输入补齐到少数代表形状）和特化（为具体形状生成专门程序）需要重复多少次才值得。

### 5.1.5 一个 kernel 在 SM 上如何执行

图 5-3 把 kernel 画成一段 20 μs 的整体；它能否缩短到 5 μs，取决于这段时间里 SM 上发生了什么。本节固定一个 kernel，沿线程、驻留、访存、矩阵指令和同步五步，推导其在一个 SM 上如何执行。kernel 取第 4.3.2 节的乘加块：输出块 128×128，沿 K 每步 64；BF16 的 A 块与 W 块各 16 KiB，FP32 累加器 64 KiB，三者都放在共享内存中，构成工作集（计算期间必须同时留在局部存储中的数据），共 96 KiB。一个线程块负责一个输出块，含 256 个线程。硬件取 H100 的一个 SM：最多同时驻留 2048 个线程（即 64 个 warp，warp 在下文解释）和 32 个线程块，寄存器 64K 个（每个 32 位），共享内存 228 KB（CUDA 文档的写法，即 233,472 bytes）。kernel 每线程用 128 个寄存器，每条加载指令每线程取 16 bytes，每个 warp 最多挂起 4 条未完成的加载，矩阵指令形状为 16×8×16，显存延迟取 600 ns。[6]

**warp：一起发射的 32 个线程。** SM 不逐个线程发射指令，而是把 32 个线程编成一组，一起发射，这一组称为 **warp**。同一 warp 的 32 个线程执行同一条指令，各自处理不同的数据；一条加载指令因此一次给出 32 个地址。256 个线程的线程块含 8 个 warp。第 5.2.3 节分析共享内存 bank（存储体，共享内存中可独立服务请求的分区）时所说的 32 个 lane，就是一个 warp 中的 32 个线程：它们同时给出的 32 个地址落在多少个 bank 上，决定这条指令要占用共享内存接口几轮。

**驻留与占用率。** 线程块要在 SM 上驻留，即留在 SM 上随时可被调度执行，必须同时分到线程槽（每个线程占一个）、寄存器和共享内存。每类资源能容纳的线程块数，等于资源总量除以每块需求后向下取整；驻留块

[6] SM 驻留、延迟隐藏与 MMA 指令数、累加器放入寄存器、延迟 1000 ns 与每 warp 2 条加载。SM 限制取自 CUDA 编程指南的计算能力表（计算能力 9.0 列）与 Hopper 调优指南，占用率定义取自编程指南的 kernel 编写一节，异步复制、栅栏、warp 分工与 swizzle 取自异步数据复制一节，3.35 TB/s 取自 H100 产品页，132 个 SM 与 BF16 稠密矩阵峰值 989.4 TFLOP/s 取自 H100 架构白皮书（表 3）。每线程寄存器数、显存延迟、每 warp 未完成加载数与加载宽度、MMA 指令形状是本节选定的取值。

数取各类中的最小值：

| 资源 | SM 总量 | 每个线程块的需求 | 可容纳的线程块数 |
|---|---|---|---|
| 线程槽 | 2048 | 256 | 8 |
| 寄存器 | 64K 个 | 256 × 128 = 32K 个 | 2 |
| 共享内存 | 228 KB（233,472 bytes） | 96 KiB 工作集 + 每块预留 1 KiB = 97 KiB | 2 |
| 线程块数上限 | 32 | 1 | 32 |

寄存器与共享内存都只允许 2 块，因此 SM 上驻留 2 个线程块，即 16 个 warp。**占用率（occupancy）**指驻留 warp 数占 SM 最大驻留 warp 数的比例，这里为 16/64 = 25%。图 5-4 把三类资源画成三根横条：两个线程块把寄存器恰好填满，把共享内存填到只剩 34 KiB，线程槽却只用了四分之一。

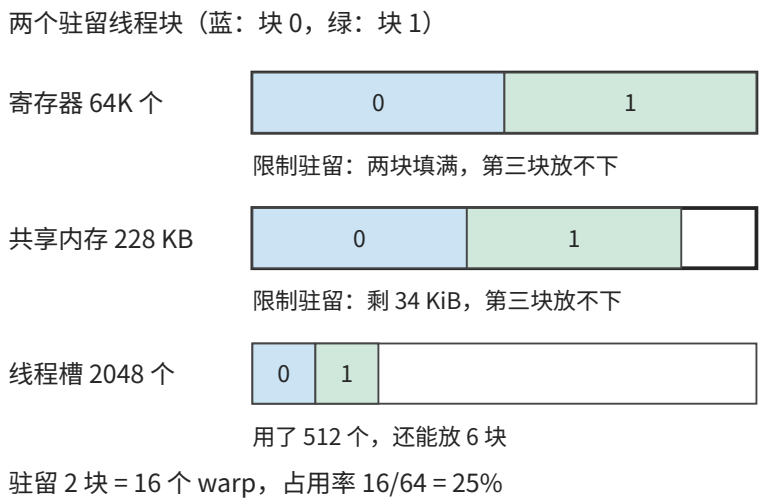



图 5-4：三条分别表示 SM 的寄存器、共享内存与线程槽总量，蓝、绿为两个驻留线程块的占用。寄存器恰好填满，共享内存剩余 34 KiB 放不下第三块的 97 KiB，两者同时限制驻留；线程槽只用了 512 个。每块 256 线程、每线程 128 个寄存器、96 KiB 共享内存，另加每块 1 KiB 预留。

改哪项资源才有用，取决于哪项限制最紧。把 64 KiB 累加器从共享内存挪进寄存器，每个线程要多保存 64 个 FP32 值，即多用 64 个寄存器。每块的共享内存需求随之降到 32 KiB，可容纳 6 块；但若每线程原来只用 64 个寄存器，加上累加器就是 128 个，寄存器仍只允许 2 块。驻留块数不变，限制项只是从两项变成寄存器一项。要让第三块驻留，必须同时压低每线程寄存器和每块共享内存，或者缩小每块的工作集。

**延迟隐藏：驻留的 warp 提供在途请求。**占用率之所以重要，是因为独立的访存请求都来自驻留的 warp：在途请求足够多，一个请求等待返回时，接口仍被其他请求占满，这称为延迟隐藏。第 4.3.3 节的 Little 定律给出维持带宽所需的在途字节数：带宽 × 延迟。取 H100 显存带宽 3.35 TB/s、延迟 600 ns，整卡需要约 2.01 MB 在途，平摊到 132 个 SM，每个 SM 要保持约 15.2 KB 在途。再计算驻留的 warp 能提供多少：一个 warp 的一条加载指令让 32 个线程各取 16 bytes，共 512 bytes；每个 warp 最多挂起 4 条未完成的加载；16 个 warp 合计 32 KiB。

$$B_{\text{need}} = \frac{3.35\ \text{TB/s} \times 600\ \text{ns}}{132} \approx 15.2\ \text{KB}, \qquad B_{\text{have}} = 16 \times 4 \times 512\ \text{bytes} = 32\ \text{KiB}.$$

可用在途字节是需求的 2.15 倍，只驻留 8 个 warp（一个线程块）也已足够。第 5.2.2 节会遇到“大块少读数据却可能变慢”的现象，这里给出它的定量形式：工作集越大，驻留块越少，可用的在途字节越少；一旦低于

需求，有效带宽就降为可用在途字节除以延迟，接口带宽再高也用不上。

**矩阵指令的粒度。**第 4.2.1 节说明矩阵单元按固定形状的块执行。这里一条矩阵乘加指令（MMA）的形状为 16×8×16：A 片 16×16，W 片 16×8，一条指令完成 $2\times16\times8\times16 = 4096$ 次浮点操作。一个 128×128×64 的块要发 $(128/16)\times(128/8)\times(64/16) = 8\times16\times4 = 512$ 条指令，与块的 2,097,152 FLOPs 相符。这 512 条由 8 个 warp 平分：输出块含 $8\times16 = 128$ 个 16×8 的输出片，每个 warp 负责其中 16 片，沿 K 四步共发 64 条。MMA 指令在寄存器中累加，本节的块把累加结果放在共享内存里，每个 K 步前后要在寄存器与共享内存之间搬移这 16 片；上一段把累加器挪进寄存器的方案，省掉的正是这笔搬移。

MMA 有两种发出方式。warp 级指令由一个 warp 单独发出，操作数要先装进该 warp 的寄存器；Hopper 的 warpgroup 级指令由 4 个相邻 warp（合称一个 warpgroup）联合发出，可以直接从共享内存取操作数，并且异步执行：发出指令的 warp 在等待结果的同时可以做其他工作。FlashAttention-3 正是利用这种指令安排计算重叠。[7]

**同步：异步复制与栅栏。**第 4.4.2 节用两个输入槽让加载与计算重叠，前提是知道“槽已填满”和“槽已用完”两个时刻。在 SM 上，这两个时刻由**栅栏（barrier）**给出。栅栏是共享内存中的一个计数器：参与者完成自己的一步后到达栅栏，计数达到预定值时，等在栅栏上的线程被放行。异步复制（发出后由硬件在后台完成、发出者不必等待的复制）完成时同样到达栅栏，并按搬入的字节数计数。每个槽配一对栅栏：“满”栅栏由复制完成触发，计算等它；“空”栅栏由计算结束触发，下一次复制等它。

有了栅栏，warp 可以分工：一个 warp 只发复制（生产者），其余 warp 只做计算（消费者），这种安排称为 **warp specialization**。生产者 warp 发出块 j+1 的复制后不必等它完成，转而等“空”栅栏，准备写入块 j+2；消费者 warp 每等到一次“满”栅栏，就接着计算下一块。

先计算复制和计算各自的耗时。把 H100 的显存带宽 3.35 TB/s 与 BF16 稠密矩阵峰值 989.4 TFLOP/s 平分到 132 个 SM，每个 SM 约得 25.4 GB/s 和 7.50 TFLOP/s，下面按该线程块独占该 SM 的份额计算。一个 K 块的 A、W 共 32 KiB，复制约需 1.29 μs；块乘为 2,097,152 FLOPs，计算约需 0.28 μs。复制比计算慢约 4.6 倍，原因是该块每搬 1 byte 只做 64 次浮点运算，而 H100 的矩阵峰值与显存带宽之比约为 295 FLOP/byte。图 5-5 按这两个时间画出两个槽上的四个 K 块。

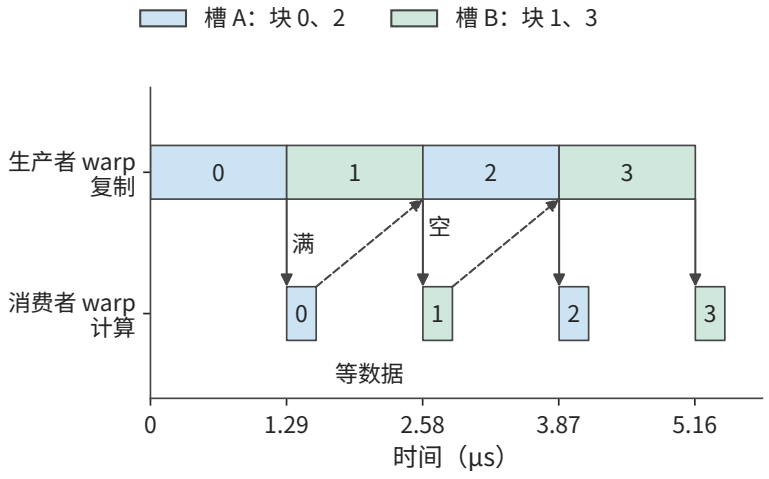



图 5-5：上行是生产者 warp 发出的异步复制，下行是消费者 warp 的计算；蓝、绿分别为槽 A、B。实线箭头为“满”栅栏，复制完成后计算才能开始；虚线箭头为“空”栅栏：块 0 在 1.57 μs 用完槽 A，块 2 的复制到 2.58 μs 才开始，生产者不必等槽。H100 一个 SM 上每块复制 1.29 μs、计算 0.28 μs；复制首尾相接，消费者每算完一块就等下一次“满”栅栏。

这种分工是第 4.2.3 节流水的软件实现。那里的三种资源是矩阵单元、共享内存接口与指数单元；这里生产

[7] FlashAttention、FlashAttention-2、FlashAttention-3。

者 warp 驱动复制，占用共享内存接口，消费者 warp 驱动矩阵与指数运算，栅栏让双方只在数据交接处等待。FlashAttention-3 按此设计：生产者 warpgroup 发 K、V 块的复制，两个消费者 warpgroup 交替做矩阵乘和 Softmax，一个在算指数时，另一个在做矩阵乘。第 5.3.2 节将沿用这里每块复制 1.29 μs、计算 0.28 μs 的取值推导流水总时间，第 5.3.3 节再展开 FlashAttention 的分块。[8]

异步批量复制对数据布局还有一项要求。第 5.2.3 节将用 padding 在每行末尾补一个字来分散 bank；异步批量复制按张量描述符（记录张量形状与步长的结构）整块搬移，不能在行间插入填充，于是改为按固定规则置换块内的地址，这种做法称为 **swizzle**：数据总量不变，同一列的请求仍落到不同 bank。

**对应到昇腾。**本节讨论的线程、驻留、访存、矩阵指令与同步这五步，在昇腾上对应第 4.2.3 节与第 4.6.2 节的另一组部件：

| 本节的机制 | NVIDIA GPU | 昇腾 |
|---|---|---|
| 发射与分工 | warp 是 32 个线程的发射单位，生产者与消费者 warp 在同一 SM 内分工 | AIC、AIV 各是独立的核而不是 warp：矩阵工作在 AIC 上执行，向量工作在 AIV 上执行，各有自己的指令流 |
| 驻留与容量 | 寄存器、共享内存、线程槽决定每个 SM 驻留的线程块数 | L0、L1 与统一缓冲的容量决定每个核一次容纳的块 |
| 搬移与同步 | 异步复制到达栅栏，warp 等栅栏 | MTE / NDDMA 搬移，BufferID 标识缓冲并组织交接 |

上文“在途字节足够”的结论，依赖 600 ns 延迟与每 warp 4 条未完成加载。**翻转条件**：延迟升到 1000 ns、每个 warp 只能挂起 2 条加载时，每个 SM 需要 25.4 KB 在途，16 个 warp 只能提供 16 KiB，覆盖率（可用在途字节与需求之比）降为 0.65；补齐缺口至少要 25 个 warp，即 4 个线程块，而寄存器与共享内存只允许 2 块。此时 25% 的占用率不再足够：要么缩小工作集，换取更多驻留块；要么改用异步批量复制，让一个线程一次请求整块数据，在途字节按块大小计，不再依赖 warp 数。

> **练习 5-1 ・延伸：把输出块缩为 64×64，占用率由哪项资源决定**
> 保持 256 个线程、每线程 128 个寄存器，把输出块改为 64×64，K 步长仍为 64，累加器仍放在共享内存中。求每块的共享内存需求、四类资源各自允许的线程块数与占用率，指出哪项限制最紧。再把每线程寄存器降为 64 个重算一次，说明限制转移到了哪项；最后按 600 ns 延迟与每 warp 4 条加载，检查在途字节是否仍能覆盖需求。

## 5.2 单个算子的分块、布局与实际访存

大矩阵计算可以像切豆腐一样分块：先分成适合处理的小块，再逐块完成。不过，切成什么形状，要同时看计算单元能处理什么，以及其附近能容纳多少数据。第 4 章已经说明，硬件矩阵指令支持有限的形状；整个加速器上的多个计算单元可以同时处理许多块，每份工作内部仍要组织成硬件能够执行的小步。

分块还要让搬来的数据多用几次。沿用第 4 章的工作台比喻，当前输入和尚未算完的部分和要放在计算单元附近，用完才能腾出空间。输出块太小，同一份输入可能要为不同的输出块反复搬移；块太大，又会占用更多局部存储，挤掉其他可以同时执行的任务。下面先跟踪一个输出需要哪些数据，再把多个输出组织成块，分析块大小、访问顺序和局部容量如何共同影响复用。

[8] FlashAttention、FlashAttention-2、FlashAttention-3。

### 5.2.1 矩阵形状与重复读取

以下用 A 表示章首的输入 X，投影写作 $C = AW$，其中 $M = 1024, K = 4096, N = 12288$。A、W、C 采用 BF16，大小分别为 8、96、24 MiB。若 A、W 各读一次，C 写一次，只需传输 128 MiB。要解释一种分块实现为何产生约 3 GiB 的访问，先看单个输出如何算出。

计算 C[i,j] 时，取 A 的第 i 行和 W 的第 j 列，将对应元素相乘，再沿 K 维求和。接着计算右边的 C[i,j+1]，W 换成相邻一列，A 却仍是同一行。图中两次计算的蓝色部分完全相同。

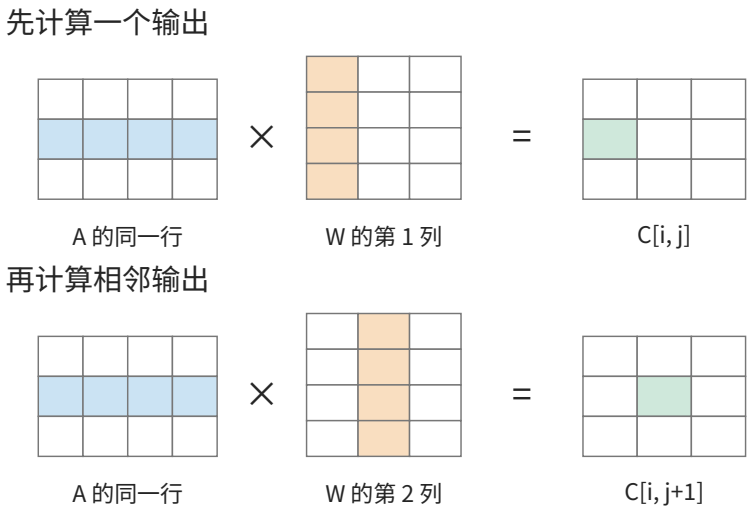



图 5-6：先算一个输出，再算相邻输出。蓝色表示所用的 A 行，橙色表示所用的 W 列，绿色表示本次得到的输出元素。图中矩阵缩小为示意尺寸，实际算例的 K 为 4096。

如果这行 A 在两次使用之间仍留在局部存储或缓存中，就能直接复用；若已被替换，就需要再次向下一级存储请求数据。转到下一行输出时，W 中的元素也会再次使用。同一份数据在数学运算中需要反复使用。程序如何安排这些运算，决定了这份数据要经过存储接口多少次。

本节以工作缓冲和下一级存储之间的接口为计数位置，累计程序每次读入与写出的字节数。下文约 3 GiB 的访问量，也按这一接口计算。[9]

把图中的逐元素计算写成循环，就是：

```
for i in range(M):
    for j in range(N):
        acc = 0.0
        for k in range(K):
            acc += A[i, k] * W[k, j]
        C[i, j] = acc
```

内层 k 循环每执行完一遍，得到一个输出。一次乘加计两个 FLOPs，计算全部输出所需的运算量为：

$$F_{\text{gemm}} = 2MKN \approx 103 \text{ GFLOPs}.$$

[9]分块容量与数据复用，含循环计数、Orojenesis 选读与 bank 映射。

比较的基准是三份 BF16 矩阵各经过接口一次的读写量：

$$V_{\text{once}} = 2(MK + KN + MN) = (8 + 96 + 24)\ \text{MiB} = 128\ \text{MiB}.$$

其中 MiB 为 $2^{20}$ bytes。以各输入读一次、输出写一次为基准，下文据此计算分块增加了多少重复访问。

循环顺序还决定每次访问的地址。W 按行连续存储时，上面的内层循环依次读取 W[k,j]、W[k+1,j]，地址相差 N 个元素。若改成 i、k、j 的顺序，内层便依次读取 W[k,j]、W[k,j+1]，同时将 A[i,k] 用于多个输出。这样一次连续传输取得的数据能立即参与计算。

不过，新顺序也要同时保存一段输出的累加结果。若局部存储容纳不下这些结果，每轮更新都可能增加对下一级存储的读写。接下来用分块限制同时保留的数据量，让输入复用与输出保存相互配合。

### 5.2.2 容量约束下的分块复用

先选 C 的一小块作为当前要完成的输出，为该 $m \times n$ 输出块分配累加缓冲区。每次沿 K 维读入一对输入块：A 块为 $m \times k$，W 块为 $k \times n$。两块相乘，更新同一份输出累加器；换入下一对输入块时，已经得到的部分和继续保留。直到沿 K 完成全部累加，才把结果转为 BF16 并写出。这里的块是软件为复用数据而组织的，其内部还可以分解为多次硬件矩阵指令。分块本身不要求先把原矩阵复制成许多独立的小矩阵，具体搬移由执行程序安排。

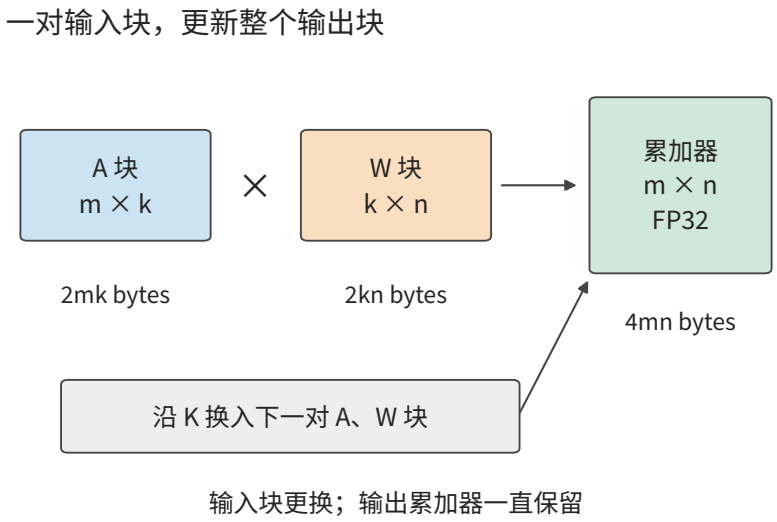



图 5-7：固定当前 m×n 输出块，沿 K 依次搬入对应的 A、W 块，反复更新同一份部分和，全部累加结束后才写回。A、W 输入为 BF16，累加器为 FP32。图示为软件分块，一次块乘可包含多次硬件矩阵指令；尺寸符号表示形状，方框面积不按字节数缩放。

A 块中的每个元素可用于 n 列输出，W 块中的每个元素可用于 m 行输出。扩大输出块，就能让一次读入的数据参与更多乘加；同时也要保存更多输入和输出部分和。

**例 5-3：扩大矩阵输出块能减少多少输入读取？** 图中的 A 输入块、W 权重块和输出累加器分别占 $2mk$、$2kn$ 和 $4mn$ bytes。只使用一组输入缓冲时，局部存储需求为：

$$S = \underbrace{2mk}_{A\ \text{块}} + \underbrace{2kn}_{W\ \text{块}} + \underbrace{4mn}_{\text{累加器}} \quad \text{bytes}.$$

取 $k = 32$，先用 64×64 输出块。A 块和 W 块各占 4 KiB，累加器占 16 KiB，合计 24 KiB。把输出块扩大到 128×128 后，两份输入各占 8 KiB，累加器占 64 KiB，合计增至 80 KiB。输入复用增多，输出累加器却增长得更快。

接着计算全矩阵的读写。假设每个输出块独立读取自己的输入，不考虑不同输出块之间的缓存复用。N 方向有 N/n 个列块，每个列块都要遍历 A；M 方向有 M/m 个行块，每个行块都要遍历 W。最终 C 只写出一次，因此：

$$V = \underbrace{2MK\frac{N}{n}}_{A\ \text{的重复读取}} + \underbrace{2KN\frac{M}{m}}_{W\ \text{的重复读取}} + \underbrace{2MN}_{C\ \text{写出}}.$$

64×64 输出块将输出划为 16 个行块、192 个列块。192 个列块各读一遍 8 MiB 的 A，共 1536 MiB；16 个行块各读一遍 96 MiB 的 W，也是 1536 MiB。加上 24 MiB 输出，总量为 3096 MiB。这就是本节开头约 3 GiB 的来源。

改成 128×128 后，行块数从 16 减到 8，列块数从 192 减到 96，两份输入的读取均减半。输出大小不变，总量降至 $768 + 768 + 24 = 1560$ MiB。图 5-8 把这项收益与占用的局部存储放在一起比较。

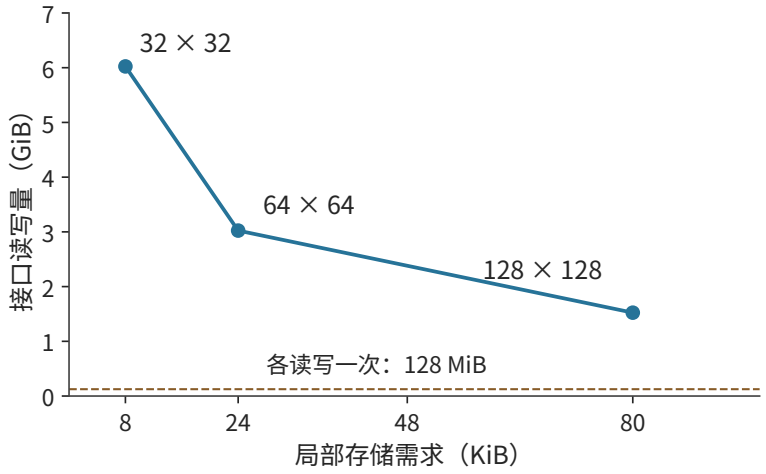



图 5-8：每个点标出输出块形状。扩大输出块能减少跨接口的重复读取，但需要更多局部存储。虚线为三份矩阵各经过一次的 128 MiB；比较条件为 k=32、单组输入缓冲、各输出块独立读取输入。

完整数值如下，算术强度用同一矩阵乘的 FLOPs 除以上述接口字节数计算：

| 输出块 | 局部存储需求 | 工作缓冲与下一级存储之间的访问 | 算术强度 |
|---|---|---|---|
| 32×32 | 8 KiB | 6168 MiB | 15.9 FLOP/byte |
| 64×64 | 24 KiB | 3096 MiB | 31.7 FLOP/byte |
| 128×128 | 80 KiB | 1560 MiB | 63.0 FLOP/byte |

三种输出块的算术强度都远低于第 5.1.5 节给出的 H100 矩阵峰值与显存带宽之比 295 FLOP/byte。只看这一级接口的流量，三种分块都处于带宽受限一侧，矩阵单元的 MFU 上限分别只有 5%、11% 和 21%。扩大输出块能把算术强度推向交点，但单靠这一级分块还越不过交点，其余的复用要靠更靠近计算单元的存储层次。

访问量接近减半，时间是否随之减半，还要看加速器能同时安排多少工作。GPU 上，输入缓冲放在共享内

存中，累加器可以放在共享内存或寄存器中。这里沿用第 5.1.5 节的做法：每个线程块负责一个输出块，输入缓冲与累加器都放在共享内存中。硬件取 RTX PRO 6000 的一个 SM：该卡属于计算能力 12.x，每个 SM 的共享内存上限为 100 KB（即 102,400 bytes），每个线程块另占 1 KiB 预留。[10]

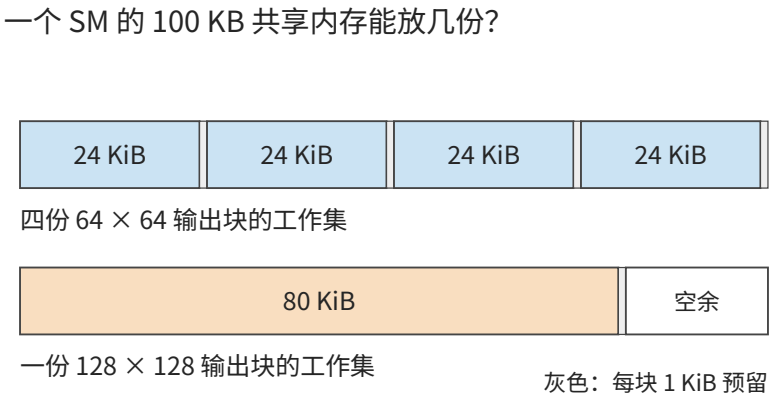



图 5-9：条形总长均表示 RTX PRO 6000 一个 SM 的 100 KB 共享内存。24 KiB 的工作集连同每块 1 KiB 预留，恰好放下四份；80 KiB 的只能放一份。

同时驻留的线程块可以交错发起访存：一些块等待数据时，另一些块继续工作。同时驻留的工作集从四份减为一份，等待访存期间能执行的其他计算也就少了。因此，大块虽然少读数据，实际获得的带宽却可能下降。第 5.1.5 节已经算过这一点：驻留的 warp 提供在途访存请求，在途字节不足时，有效带宽就低于接口带宽。

假设两种实现都受这一层的带宽限制，有效带宽分别为 $B_{64}$ 和 $B_{128}$，则：

$$\frac{T_{128}}{T_{64}} = \frac{1560}{3096}\frac{B_{64}}{B_{128}}.$$

若带宽相同，时间接近减半；若大块只能获得原来四成的带宽，时间比约为 $0.504/0.4 = 1.26$，耗时反而增加约 26%。可以先按容量排除容纳不下的方案，再按访问量缩小比较范围，最后测量实际执行效率。第 5.4 节将把这一过程交给编译器。

### 5.2.3 数据布局：让并行请求分散到不同 bank

上一小节中，大块虽然少读数据，却可能因并行度下降而变慢。并行度不仅取决于驻留多少线程块，也取决于这些线程发出的读取请求能否同时得到服务。先看局部存储的布局。考虑 32×32 的 FP32 暂存块，其所在的局部存储分成 32 个 bank；每个 bank 每轮提供一个 32-bit 字，bank 号就是字地址对 32 取模的结果。第 5.1.5 节已经介绍，一个 warp 的 32 个线程就是 32 个 lane；32 个 lane 各读取同一行中的一个字时，请求分散到 32 个 bank；读一列时，行跨度为 32 个字，所有地址集中到同一 bank，需要 32 轮。

在每行末尾补一个字，行跨度变为 33。同一列相邻行的 bank 号便依次加一，32 次请求分散到 32 个 bank，一轮即可完成。所占空间由 4096 bytes 增至 4224 bytes，只多约 3.1%。有效数据和读取次数都没有变化，但原本需要分 32 轮完成的列读取，现在可以在一轮内完成。

行末补的这个字就是 padding，它不改变计算本身，只让已有的并行请求分散到不同 bank。

padding 用多占的字节换来分散。也可以不增加字节，只改变存放位置：把第 $r$ 行第 $c$ 列的元素放到第 $c \oplus r$ 列，$\oplus$ 为按位异或。行跨度仍是 32 个字，读一列时，32 个 lane 取到的列号是 $c \oplus r$ 随 $r$ 变化的 32 个不同取

[10] RTX PRO 6000 Blackwell 为 SM120 架构（见实验 5-2 的运行环境），即计算能力 12.0；CUDA 编程指南的计算能力表给出该列每个 SM 的共享内存上限 100 KB、每个线程块上限 99 KB，两者之差即每块的 1 KiB 预留。

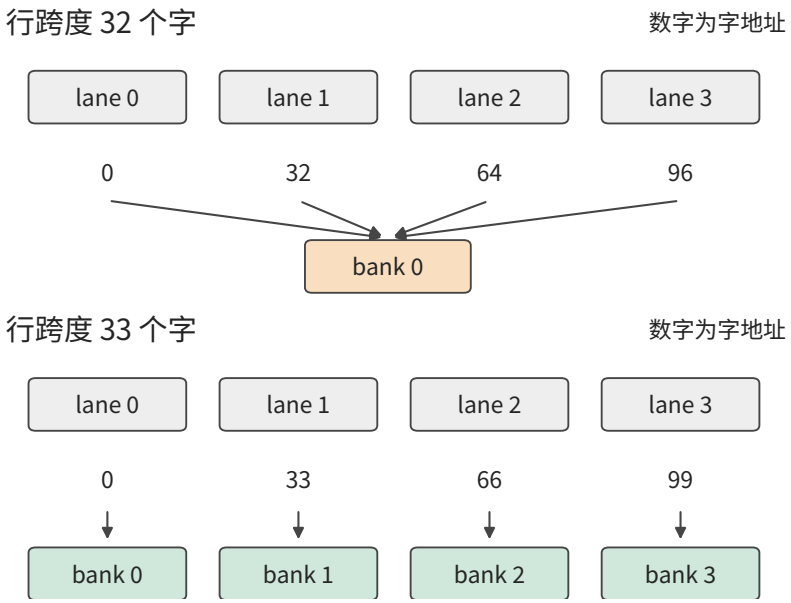



图 5-10：同一列的 32 个不同字由 32 个 lane 同时请求。上半图行跨度为 32 个字，请求集中到同一 bank；下半图补齐为 33 个字，请求分散到 32 个 bank。图中画出前四个请求；假设每个 bank 每轮提供一个 32-bit 字，不计广播。

值，bank 号因此互不相同，同样一轮完成。NVIDIA 的矩阵乘法模板库 CUTLASS 把这种改变存放位置的做法称为 **swizzle**。swizzle 节省了补齐的空间，代价是每次访问都要先按公式算出列号。

### 5.2.4 沿哪个维度切分：并行任务与部分和

padding 解决的是请求能否同时得到服务。如果程序本来就只安排了少量任务，改变地址布局还不够，还要把计算拆开。以 RMSNorm 为例：先根据一行元素的均方值计算共同的缩放因子，再缩放这一行。求缩放因子是一次归约：它来自整行平方和；随后每个元素乘以该因子和对应的可学习参数 gamma。对 $[1024, 4096]$ BF16 输入，一组处理一行，可以独立安排 1024 组；而在 decode 阶段，单请求每次只生成一个 token，同样的分法只产生一组，加速器的大量执行资源就会空闲。

把每行分成八个 512 元素片段，可以增加第一阶段的并行任务。完整归约随之变成三个阶段：每个片段产生局部平方和，合并八个局部和得到行缩放因子，再用该因子归一化输入。前两阶段只需传递很少的数，但最后一阶段要重新读取原始数据。

沿着图 5-11 的箭头计数，就能求出增加并行任务所付出的访存代价。仍取 1024 个 token，每个 token 的特征向量占输入矩阵的一行。第一阶段的并行任务从 1024 组增到 8192 组；8192 个 FP32 局部和占 32 KiB，1024 个 FP32 缩放因子占 4 KiB。一组一行的程序在局部保留输入，读入 8 MiB 输入，每行再读取一份 BF16 gamma，累计 8 MiB，最后写出 8 MiB 结果，总访问量为 $8 + 8 + 8 = 24$ MiB。拆分后，最后的归一化阶段重读 8 MiB 输入；局部和、缩放因子的写出与读取再增加约 0.1 MiB，总量为 32.1 MiB，kernel launch 次数从一次增至三次。[11]

图中每组只处理一行的八分之一，还带来另一项收益：需要保存的临时数据随之减少。以 FP32 保存完整一行需要 16 KiB，保存其中八分之一只需 2 KiB。原来因寄存器不足而写入较远存储的临时数据，可以因此保留在寄存器中。拆分归约的收益来自两处：并行任务变多，每组的临时数据变少；代价则是多读一遍输入，以及阶段之间要传递数据。

[11] RMSNorm 三阶段拆分计算。输入和 gamma 为 BF16，gamma 每行读取；一组一行保留输入，三阶段方案在应用阶段重读输入。

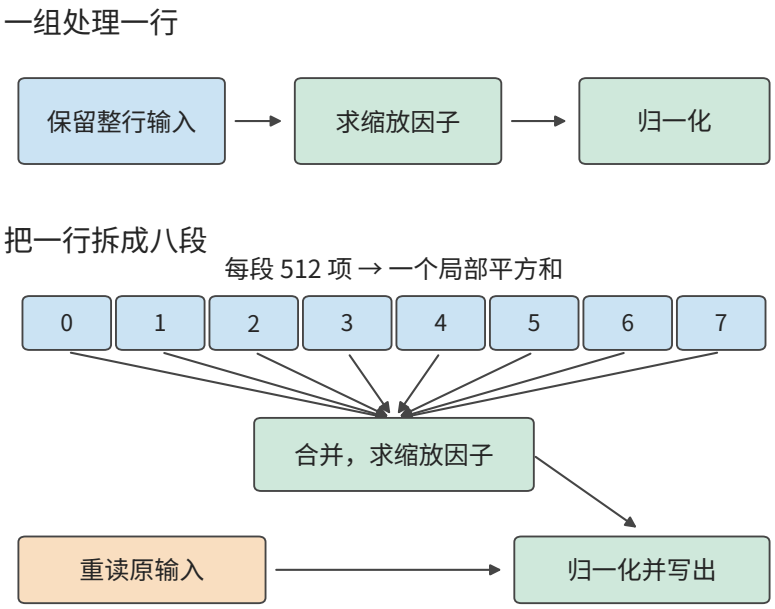



图 5-11：上行由一组处理完整一行，输入保留到归一化结束；下行把一行分为八段，先求局部和，再合并，最后重读输入并归一化。每行 4096 个 BF16 元素；图中仅画一行，推广到 1024 行时，第一阶段任务数由 1024 增至 8192。橙色框表示增加的原输入读取。

RMSNorm 的拆分沿的是归约维：一行的平方和分成八个局部和，必须再合并一次。矩阵乘也有同样的选择。对 $C_{M\times N} = A_{M\times K}W_{K\times N}$，可以沿三个维度中的任何一个把工作分给不同的执行者。图 5-12 画出三种切法各自需要什么输入、得到什么结果。

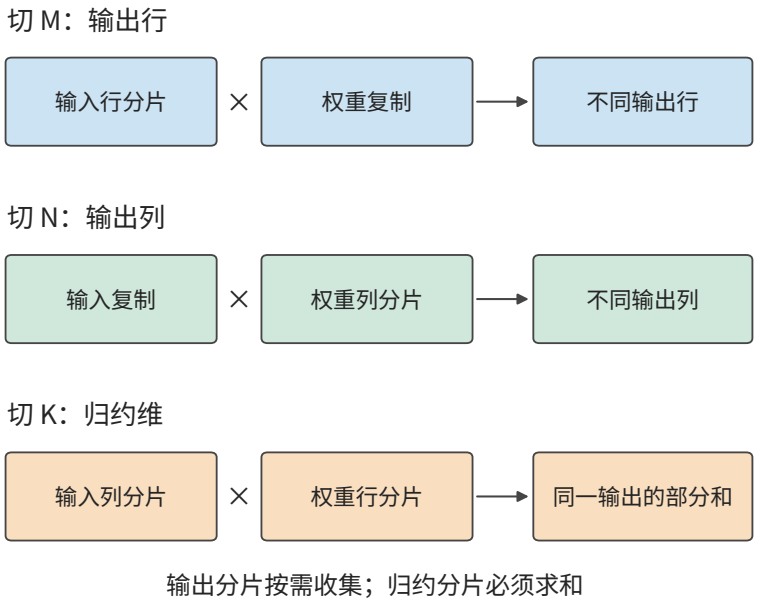



图 5-12：切输出行 $M$ 或输出列 $N$，各执行者得到不同位置的完整结果，需要时再拼起来；切归约维 $K$，各执行者得到同一输出的部分和，必须相加后才能使用。方框表示数据归属，不按矩阵元素数绘制。

前两种切法只要求输入按需复制或分片，输出可以一直保持分片，直到后续算子需要完整数据；第三种切法省下了输入的复制，却像 RMSNorm 的局部和一样，多出一次汇合。第 5.2.2 节的输出块正是同时沿 $M$ 和 $N$ 切分，再沿 $K$ 逐块累加：累加器留在一个线程块内，汇合部分和不需要跨越任何边界。

合并的次序也会改变结果。浮点加法不满足结合律：用 FP32 计算 $1+2^{-24}+2^{-24}$，先加前两项，和仍是 1，再加第三项也仍是 1；先加后两项得到 $2^{-23}$，与 1 相加就得到 $1+2^{-23}$。两种次序的结果相差一个 **ULP**（unit in the last place，同一指数下相邻两个浮点数的间距）。一次舍入丢掉的位无法恢复，之后每一次加法都在已经舍入过的数上继续。

本小节的 RMSNorm 可以算出这一差别有多大。取一行 4096 个 BF16 元素，每个元素的平方在 FP32 中都能精确表示，因此下面几种做法的差别只来自加法的先后。图 5-13 的上半部分比较 1、2、4、8、16、32 段这六种切分算出的平方和，横轴为它们与精确值的距离。一组处理一整行时，累加器逐步增长到 2038 附近，每次加入的一项却只有 0.5 左右，末位反复被舍去，最后比精确值小 175.2 个 ULP。分成八段之后，每个累加器只增长到 255 左右，最后再把八个局部和相加一次，结果与精确值只差 9.2 个 ULP。两条路径相差 166 个 ULP，相对差约 $9.9\times10^{-6}$。可见拆分改变的不只是并行任务数，还有精度。

下半部分固定这八个局部和，只改合并的次序。40,320 种次序只得到三个不同的 FP32 结果，彼此相差一个 ULP。如果用原子加把局部和累加到同一个地址，到达的先后由调度决定；同一份输入、同一个程序，两次运行得到的结果就可能不同。改成先把八个局部和写入缓冲，等它们全部写完，再由一个执行者按固定顺序相加，每次都得到同一个结果。代价有两项：多一次写出与读回；合并要等最慢的那一段算完才能开始，而原子加按到达顺序累加，不必等待全部就绪。

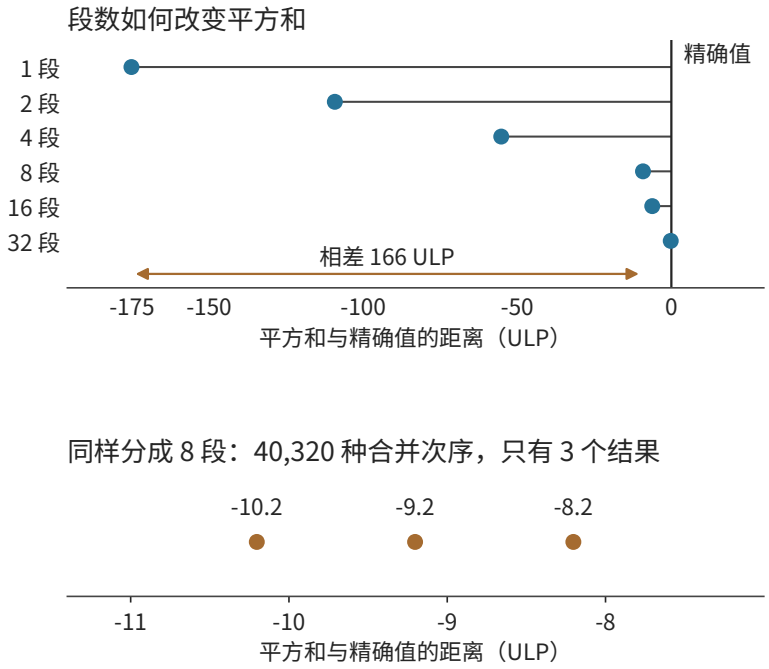



图 5-13：同一行 4096 个 BF16 元素的平方和与精确值的距离，横轴单位为 FP32 的 ULP。上半图比较 1 到 32 段的六种切分；下半图固定八个局部和，列出全部合并次序得到的结果。精确值用有理数算得，只作比较基准。

差别并不止于平方和这一步。按 Qwen3-8B 的 rms_norm_eps 继续计算，两条路径的缩放因子相差 59 个 ULP，这一行的第一个输出元素相差 50 个 ULP，并一起进入下一层。[12]

切分方式因此不只是速度问题。本小节开头已经说明，切成几段取决于能安排出多少并行任务：1024 行时一组一行已足够，decode 只有一行时必须拆开。同一个 kernel 于是会随 batch 大小改变段数，同一行输入在不同 batch 下算出不同的输出。要让输出只由输入决定，段数和合并次序就都不能随 batch 变化，代价是小 batch

[12] 切成几段与合并次序的浮点复算。教学用的这一行由给定公式生成后舍入到 BF16，不是采集到的激活；每个平方在 FP32 中都精确，各条路径的差别只来自加法次序。精确值用有理数算得，只作比较基准；枚举合并次序覆盖原子加可能出现的先后，不代表某个后端的实际分布。

下并行度退回到拆分之前。第 10.5.3 节正是用这一点解释：生成端和训练端加载的是同一份权重，同一个前缀算出的 logprob 却仍然可能不同。

分块还可以越过一张卡的边界。单卡中，这些块的计算由线程块、共享内存和局部归约协作完成；分到多卡后，切输出维的执行者要各自取得输入，切归约维的执行者要把部分和送到一起相加，这些搬移都要经过卡间互联。数学依赖没有改变，变的是搬移数据的距离和代价。

调度则是在分块之后决定执行顺序、放置、缓冲和重叠。例如沿 $K$ 切分后，是等全部输入到齐再算，还是边接收边累加，会改变存储占用和实际等待的时间。模型中的样本、序列、特征、层和专家提供更多分工方向；其中层是计算图上的顺序阶段，专家是由路由选择的分支。第 6.1.4 节将把这些分工方向与这里的三种切法对应起来，第 6.2 节再逐一推导各自的通信。

矩阵分块、bank padding 和归约拆分都在重新安排同一批数学工作。分块让多个输出复用输入，padding 将读取请求分散到不同 bank，拆分归约则增加能够并行执行的任务。分析程序时，既要分别算清这些变化带来多少收益和开销，也要看它们是否改变了浮点结果。配套实验中的连续访问与宽行归约对照提供了对应的执行记录。[13]

## 5.3 相邻算子的融合、缓冲与流水执行

单算子内部可以利用局部存储复用输入。相邻算子之间也有同样的机会：前一个算子的输出，往往立即成为后一个算子的输入。如果后一个算子能直接读取局部存储中的结果，就能省去一次写回和重读。为此，需要确定后一个算子要用哪些数据，以及这些数据何时全部算好。

### 5.3.1 中间张量与融合边界

下面将算例扩展到 FFN 的两支投影。令 $G = XW_g$、$U = XW_u$，激活链计算 $Z = \mathrm{SiLU}(G) \odot U$，再把 Z 交给 down 投影，把结果降回隐藏宽度。G、U、Z 的形状均为 $[1024, 12288]$，以 BF16 保存时各占 24 MiB。

SiLU 和 $\odot$ 都是逐元素运算。Z[i,j] 只依赖 G[i,j] 与 U[i,j]，不同位置之间没有归约。因此，读入同一位置的两个元素后，就能直接算出该位置的最终结果，不必先算出全部 SiLU 结果。

先比较中间结果 $T = \mathrm{SiLU}(G)$ 的去向。分开执行时，完整 T 要写到下一级存储，再由乘法读回；融合后，一小块 SiLU 结果可以直接交给同一个 kernel 中的乘法，用完就释放局部空间。这就像把工作台上刚处理好的材料直接交给下一道工序，省去送回远处再取出的往返。对应到这里，省去的是中间结果的写回与重读，两步运算仍然都要完成。

**例 5-4：激活算子融合如何减少中间张量读写？** 分开执行时，先计算 $T = \mathrm{SiLU}(G)$，读取 G、写出 T，共 48 MiB；再计算 $Z = T \odot U$，读取 T、U，写出 Z，共 72 MiB。总读写量为 120 MiB。将两步融合，每次读取 G、U 的一片，在局部完成 SiLU 和乘法，写出 Z，只需 72 MiB。

节省的 48 MiB 正是 T 的一次写回和一次读取。沿用上一节的带宽模型，取两种程序的有效带宽相同，时间比为 $72/120 = 0.6$，加速约 1.7 倍。实测的激活链从约 14.2 µs 降到 7.7 µs，加速比约为 1.8。理论计算与实测结果都说明了同一个收益来源：省去完整 T 的写回和重读。[14]

接下来将 Z 按固定 scale 量化，使每个元素只占 1 byte。独立转换需读取 24 MiB 的 Z，写出 12 MiB 量化结果，增加 36 MiB。三步完全分开的总量为 156 MiB；融合前两步为 108 MiB；三步全部融合为 $24+24+12 = 60$ MiB。图 5-15 将必需的输入输出读写画在每条柱形的左侧，再依次加上两个中间量的读写。

[13] 实验 5-1、GPU 分块、归约拆分。

[14] 实验 5-2 的原始条件、数据与分析。RTX PRO 6000、共享 GPU、热缓存与图重放；原始中位数为 14.20 / 7.68 µs，仅测激活子链。

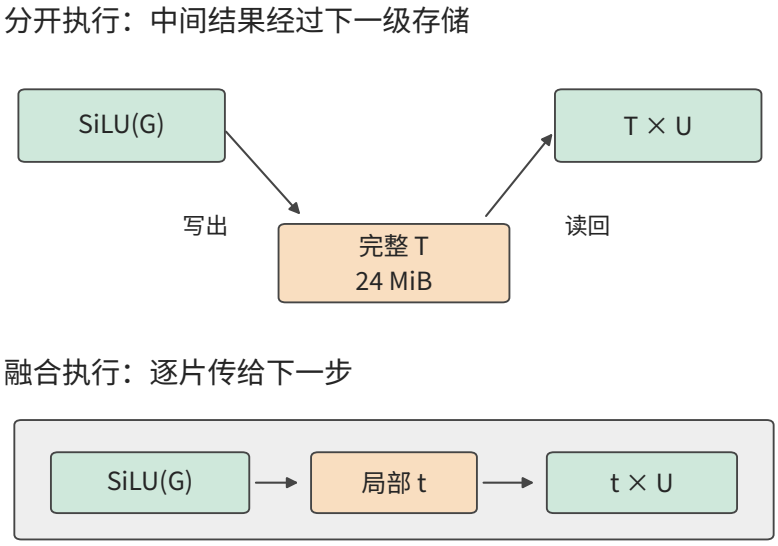



图 5-14：上半图的完整 T 经过一次写出与一次读回；下半图中局部片段 t 直接传给乘法。两种方式仍读取 G、U 并写出 Z，图中省略这些共同的输入输出边。融合后仍保留原有的中间结果舍入规则。

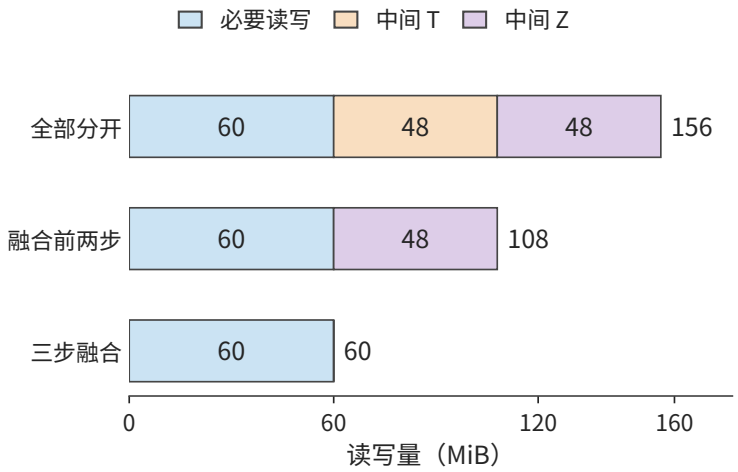



图 5-15：每少保存一个 24 MiB 中间量，就少一次写出和一次读入，共 48 MiB。G、U 为 BF16，最终输出占 1 byte/元素，量化 scale 预先给定。各方案采用相同的运算与舍入顺序。

中间张量的容量与访问量在这里是两回事。设每步先分配输出，完成后释放已用完的输入。独立执行 SiLU 时，G、U、T 同时占用内存，峰值为 72 MiB；三步融合时，G、U 与最终输出同时占用内存，峰值为 60 MiB。读写减少 96 MiB，峰值却只减少 12 MiB，因为同一块存储空间可以反复读写。[15]

逐元素运算的每个输出只依赖对应位置的输入，因此可以在读入后立即计算，减少中间结果的保存。要让这种复用顺利发生，还需要前后两步使用彼此兼容的数据布局，并保证缓冲在最后一次读取结束前不会被覆盖。下面沿着一个数据块的使用过程，分析这两个条件。

### 5.3.2 布局转换、缓冲复用与双缓冲

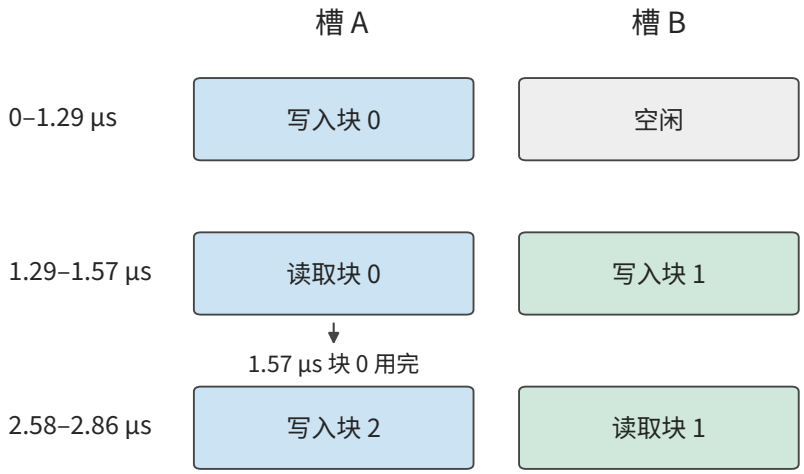
相邻算子有时使用不同布局。前一个算子适合按行写出，后一个算子却需要按另一种块顺序读取。若另建一份 24 MiB 重排结果，就要读原结果、写新结果，共 48 MiB。也可以让前一个算子直接按后续计算所需的布

[15] 固定 scale 融合与张量生命周期。

局写出，省去单独的重排。比较“计算后再重排”与“计算时直接按新布局写出”的总时间，就能判断哪种方式更快。

缓冲还决定两个阶段能否重叠执行。设搬移器把块 0 放入槽 A，计算单元随后读取 A。计算单元读取 A 期间，搬移器把块 1 放入槽 B；计算转向 B 后，就可以把块 2 加载到 A。这样两个槽交替使用，计算单元读当前块时，搬移器就可以写下一块。

注意，槽 A 的“数据已经搬完”与“数据已经用完”是两个时刻。搬移结束后，计算才开始读取；只有最后一次读取结束，才能将下一块数据写入槽 A，覆盖原有内容。沿用第 5.1.5 节 H100 一个 SM 上的数值：每块搬移 1.29 µs、计算 0.28 µs，搬移器和计算单元独立工作。图 5-16 展示三个时段中各槽位的状态，图 5-17 据此画出完整的搬移与计算时间线。




槽 A 早已空出，块 2 等搬移器在 2.58 µs 空闲才写入

图 5-16：块 0 在 1.29–1.57 µs 使用槽 A，此后槽 A 已经空出；块 2 要等搬移器在 2.58 µs 搬完块 1，才开始写入 A。图中抽取三个时段；1.57–2.58 µs 两个槽里都没有可算的数据，计算单元空等。颜色固定表示槽 A、B，文字说明当前读写的是哪个块。

**例 5-5：双缓冲如何重叠数据搬移与计算？**每块搬移 1.29 µs，计算 0.28 µs。搬移器和计算单元独立工作，事件开销记为零。串行执行四块需要 $4 \times (1.29 + 0.28) \approx 6.28$ µs。

采用两个槽后，0–1.29 µs 搬入块 0，1.29–1.57 µs 计算块 0；块 1 同时在 1.29–2.58 µs 搬入。块 0 在 1.57 µs 就已算完，槽 A 随即空出，但搬移器要到 2.58 µs 才搬完块 1、开始写块 2。此后搬移器连续工作，每 1.29 µs 送来一块，计算单元每次只忙 0.28 µs。第四块于 5.44 µs 完成，时间只减少约 13%。

将四块推广到 n 块。首块要先搬移，随后每隔较慢阶段所需的时间完成一块，末块还需完成计算。因此：

$$T_{\text{pipe}} = t_m + (n-1)\max(t_m, t_c) + t_c.$$

其中 $t_m$ 为每块搬移时间，$t_c$ 为每块计算时间。在刚才的例子中，$t_m > t_c$，搬移器从 0 连续忙到 5.16 µs，总时间为 $4 \times 1.29 + 0.28 \approx 5.44$ µs，计算单元只有约两成时间在工作。增加第三个槽无济于事：两个槽已让搬移器没有空闲，更多的槽不能加快搬移；加快计算同样无济于事。两个槽是能重叠起来的最小配置；把槽数加到 $n$、让搬移提前 $n-1$ 块开始，在 CUTLASS 中称为 $n$ 个 **stage**。本例中搬移器已经连续工作，增加的 stage 只会多占容量；只有当每块的搬移时间不稳定，或一次搬移的延迟长于一块的计算时间时，额外的 stage 才能补上空档，而所需容量随 stage 数成比例增长。要继续缩短时间，只能减少每块要搬的字节，例如按第 5.2.2 节扩

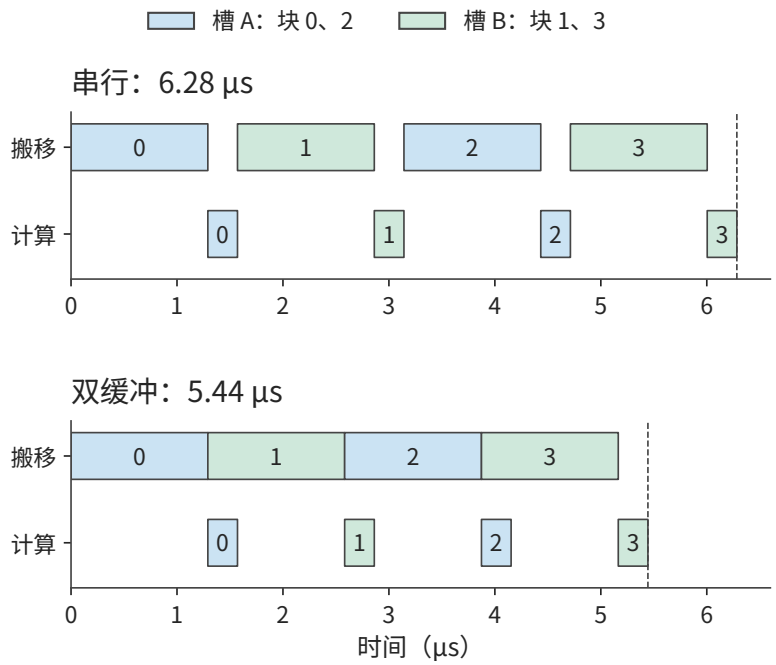



图 5-17：两个输入槽交替复用，让搬移与计算重叠。H100 一个 SM 上每块搬移 1.29 µs、计算 0.28 µs，资源独立，忽略同步开销；同一颜色表示同一槽。重叠只藏住了计算，完成时间由首尾相接的搬移决定。

大输出块，让每个输入字节参与更多乘加，或让同时执行的线程块在 L2 中共用输入块；再就是换用显存带宽更高的卡。

回到本小节开头的重排情形：两个阶段的访问顺序不同时，缓冲要保存更多尚未使用的数据。三个排队的 16 KiB 块占 48 KiB，再加 32 KiB 重排空间，共需 80 KiB。RTX PRO 6000 一个线程块最多可用 99 KB 共享内存，单独驻留时可以容纳；若一个 SM 要同时驻留两个这样的线程块，每块最多只能分到 49 KiB（100 KB 扣除两块各 1 KiB 预留后平分）。扣掉 32 KiB 重排空间，只剩一个 16 KiB 的排队槽：写入第二块之前，搬移器必须等计算单元用完并释放该槽。这种下游来不及处理、使上游暂停的机制称为**背压（backpressure）**。选择共同布局能够减少排队，扩大缓冲则能容纳更多已经写入但尚未使用的数据。[16]

第 5.1 节的主机输入也可以按此组织。复制流传入下一批，计算流处理当前批；计算流等待“复制完成”事件后读取新输入，复制流等待“计算完成”事件后覆盖旧缓冲。这两种事件分别保证输入已准备好和旧数据已用完，时间收益则来自两种资源同时工作。

### 5.3.3 FlashAttention：分块与在线 Softmax

第 5.3.1 节融合的逐元素运算，每个输出只依赖对应位置的输入；注意力中的 Softmax 却要用到整行分数。计算一行注意力输出时，可以先把各位置的分数变成尚未归一化的权重，用这些权重对值求加权和，最后除以权重总和。可以先处理一块，记下它对加权和与权重总和的贡献，再处理下一块；只要能正确合并贡献，就不必一直保留已处理过的全部分数。困难在于 Softmax 要根据分数求指数，后面出现更大的分数时，还要调整前面累计结果所用的数值基准。

FlashAttention 围绕这一逐块计算过程安排存储，减少长序列注意力中间矩阵的显存读写。该方法由斯坦福大学的 Tri Dao 等研究者与纽约州立大学布法罗分校的合作者于 2022 年提出。[17]

读到 FlashAttention 时，笔者想起了自己在 2019 年做 AKG 的一段经历。当时为了融合 Softmax 算子，笔

[16] 流式顺序与缓冲预算；主机搬移与缓冲区占用时间。

[17] FlashAttention、FlashAttention-2、FlashAttention-3。

者四处寻找合适的在线算法，后来找到 NVIDIA 在 2018 年发表的研究，才得以结合 AKG 的分块与融合能力，把前面的矩阵乘法和后面的 Softmax 接在一起计算。笔者那时甚至还不了解注意力机制，后来才意识到，自己偶然摸到了 FlashAttention 的一条核心思路。从这一步走到完整的 FlashAttention，还需要把后续与 V 的加权求和也纳入在线更新，并围绕完整注意力安排片内存储和数据搬移。下面的推导会说明，这一看似局部的算法变化如何消除大块中间矩阵。

回到注意力计算本身。标准注意力先计算 $S = QK^{\mathsf{T}}/\sqrt{d}$，再计算 $P = \mathrm{softmax}(S)$，最后计算 $O = PV$。

把这三个运算分别交给通用算子，可以直接按公式组合程序，算子之间通过中间张量交换结果。掌握完整注意力计算的数据依赖后，便可以采用另一种编译与执行方式：利用 Softmax 的归约关系，跨过独立算子的边界，在小块数据上连续计算。下面先算出原来完整保存中间张量的开销，再推导逐块计算所需保存的状态。

取一个头，序列长度 $L = 8192$，头维度 $d = 128$，对全部 token 计算注意力。Q、K、V、O 使用 BF16，各占 2 MiB；完整 S、P 使用 FP32，各占 256 MiB。Q、K、V 各读一次，O 写一次，共 8 MiB；S、P 各写一次、读一次，却要传输 1 GiB 数据。中间矩阵的大小随序列长度的平方增长，因而造成了大量读写。[18]

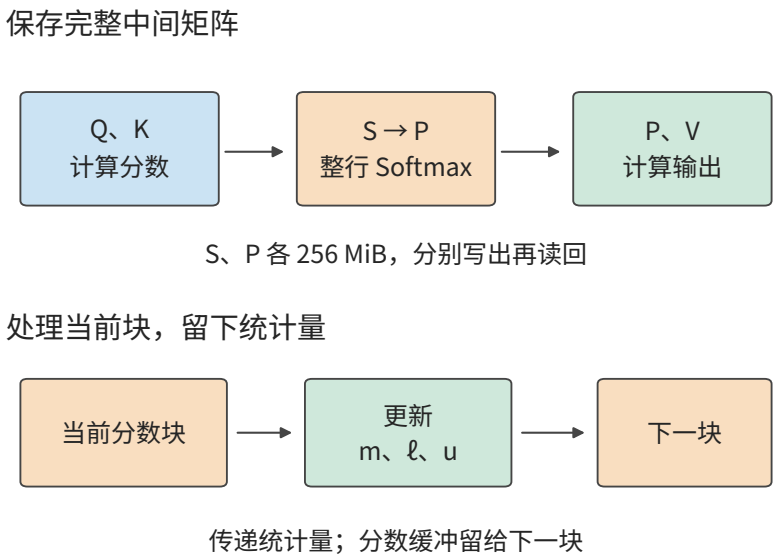



图 5-18：上半图保存完整 S、P，两份 FP32 矩阵各占 256 MiB，写出与读回合计 1 GiB。下半图只传递已处理部分的最大值 m、指数和 ℓ、加权值 u；每处理完一个块，其分数缓冲即可复用。箭头概括处理顺序，完整计算还需读入 Q、K、V。

要省去这 1 GiB 的中间读写，可以沿用第 5.3.1 节的思路；但直接套用逐元素融合会遇到一个障碍：Softmax 的分母是整行指数和。只读到前一块时，既不知道最终分母，也不知道后面是否会出现更大的分数。

要丢弃已经处理过的分数，需要保留三个量：最大分数 m，以 m 为基准的指数和 $\ell$，以及相同指数权重下的值之和 u。这里 u 通常是向量，先用标量值的小例子看清更新过程。

**例 5-6：在线 Softmax 如何合并分块结果并保持归一化？** 分数为 $[0, \ln 2]$，对应值为 $[1, 3]$，每块只含一个元素。第一块得到 $m = 0, \ell = 1, u = 1$。处理第二块后，最大值增至 $\ln 2$，将原来的 $\ell$ 和 u 都乘以 $r = 1/2$；新元素的指数为 1，于是 $\ell' = 1/2 + 1 = 1.5$，$u' = 1/2 + 3 = 3.5$，结果为 $7/3$。一次处理整行时，两个权重正比于 1 和 2，结果同样为 $(1 + 2 \times 3)/3 = 7/3$。

小例子中，第二块把最大值从 0 提高到 $\ln 2$。旧项原来的指数为 1，换到新基准后变成 $1/2$；分母中的旧贡献和分子中的旧贡献必须同时缩小，才能保持它们的相对权重不变。

[18] 注意力分块与在线状态；99 KB 缓冲下的块大小与访问量。容量模型采用单头、无掩码、K/V 复用输入槽和 S/P 复用临时槽；每个线程块 99 KB 的共享内存上限取自 CUDA 编程指南的计算能力表（计算能力 12.x 列）。

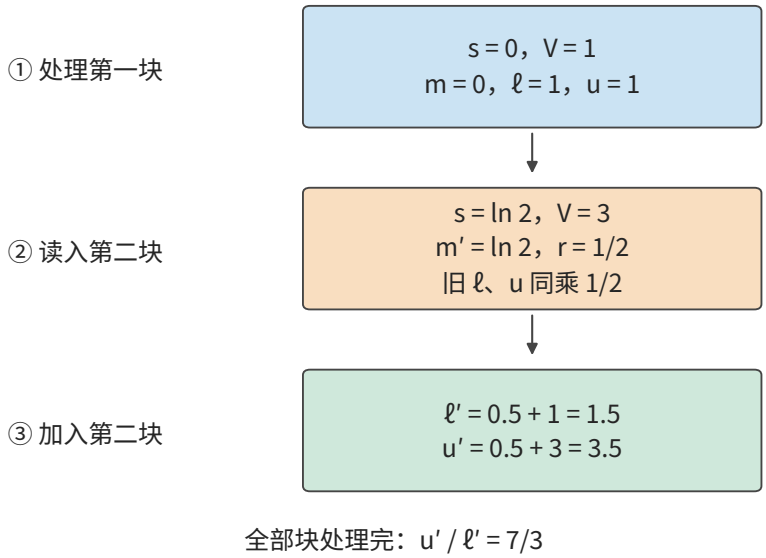



图 5-19：两个块的分数分别为 0、ln 2，值分别为 1、3。最大值增大后，将旧指数和与旧加权值同时乘以 1/2，再加上新块的贡献，最后才做除法。箭头传递的是统计量，旧分数无需保留。m 是已处理分数的最大值，$\ell$ 是以 m 为基准的指数和，u 是对应的值向量加权和，r 是更换最大值基准时的缩放系数。

推广到一行分数，减去共同最大值可避免指数溢出。令 $\ell = \sum_j \exp(s_j - m)$、$u = \sum_j \exp(s_j - m)V_j$，最终输出为 $u/\ell$。新块到来后，把旧状态改到新的最大值基准，再加上新块贡献：

$$m' = \max(m, \max(s)), \qquad r = \exp(m - m'), \qquad p_j = \exp(s_j - m'),$$

$$\ell' = r\ell + \sum_j p_j, \qquad u' = ru + \sum_j p_j V_j.$$

右侧求和只遍历新块中的 j，旧块的全部贡献已经保存在 $\ell$ 与 u 中，无需重新读回旧分数。

FlashAttention 正是利用这一递推关系省去了完整 S 和 P 的存储。每处理完一块，就更新 m、$\ell$ 和 u；该块的分数和概率用完即可丢弃，其缓冲可以留给下一块。在相同注意力范围内，查询与键的匹配、值的加权仍然都要算；改变的是计算顺序与中间结果的保存方式，浮点执行还会有舍入差异。

这样就把存储完整分数矩阵的问题，变成了在快速存储中安排当前块的问题。接下来确定一次处理多少行 Q、多少行 K/V，才能更好地利用这块空间。取 RTX PRO 6000 一个线程块可用的 99 KB 共享内存（101,376 bytes）作快速缓冲，一次保留 a 行 Q 与 FP32 输出累加器，分数块大小为 a×b。K 与 V 按阶段复用同一个 b×d 输入槽，分数和概率复用一个 a×b 槽，另为逐行统计和更新暂存分配三个 FP32 向量。局部存储需求为：

$$S = \underbrace{2ad}_{Q} + \underbrace{2bd}_{K/V\ \text{槽}} + \underbrace{4ab}_{\text{分数/概率}} + \underbrace{4ad}_{\text{输出累加器}} + \underbrace{12a}_{\text{逐行状态}} \le 101376.$$

K/V 块越小，其自身和分数块占用越少，就能容纳更多 Q 行。更多 Q 行共享一次 K/V 扫描，整份 K/V 被重读的次数随之减少。取 b=64，能容纳 a=82 行 Q，需要 $\lceil 8192/82 \rceil = 100$ 次完整扫描。每次读 4 MiB 的 K/V，另有一次 Q 读取和 O 写回，共 $100 \times 4 + 4 = 404$ MiB。

将 b 缩为 1，能容纳 a=128 行 Q，扫描次数降为 64，访问量降至 $64 \times 4 + 4 = 260$ MiB。但每次扫描从 128 个 K/V 块变成 8192 个块，在线状态必须更频繁地更新。

| K/V 块行数 b | 可容纳的 Q 行数 a | K/V 完整扫描次数 | 缓冲与下一级存储之间的访问 | Q 块与 K/V 块之间的状态更新次数 |
|---|---|---|---|---|
| 1 | 128 | 64 | 260 MiB | 524288 |
| 64 | 82 | 100 | 404 MiB | 12800 |
| 128 | 53 | 155 | 624 MiB | 9920 |

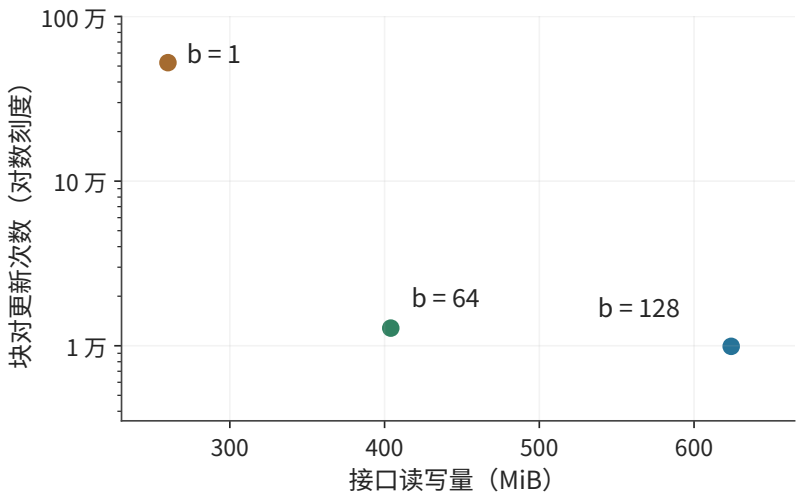



图 5-20：快速缓冲为 RTX PRO 6000 一个线程块的 99 KB 共享内存，序列长 8192、头维度 128，无掩码；每个点对应正文表格的一种 K/V 块大小。横轴为缓冲与下一级存储之间的访问量，纵轴为在线更新次数，采用对数刻度。从 b=64 的点移到 b=1 的点，读取减少，更新次数却约增至 41 倍。b 表示一个 K/V 块中包含的 token 数。

图 5-20 把两项代价放在同一平面上，越靠左表示少读数据，越靠下表示少做更新。从 64 行块缩到 1 行块，少读 144 MiB，却把状态更新从 12800 次增至 524288 次，约为原来的 41 倍。每次更新都要执行循环控制、最大值比较、指数与输出缩放；每次只处理一行 K/V，也使矩阵乘的一个维度过小，难以充分利用矩阵计算单元。较小的 K/V 块能减少读取，却会增加更新次数；较大的块则有利于矩阵计算，并减少循环开销。因此，选择块大小时，需要同时比较这两类开销。

在线归约消除了完整中间矩阵的读写，但块内的矩阵乘、指数计算和统计量更新仍要执行。后续优化便转向这些剩余工作。FlashAttention-2 改进了线程之间的任务分配，减少中间数据交换和非矩阵运算；FlashAttention-3 利用异步执行，让搬移、矩阵乘和 Softmax 相互重叠。省去巨大的中间矩阵之后，这些改进进一步缩短了剩余的计算和搬移时间。[19]

为了检验这些机制在实际程序中的效果，配套实验比较了 PyTorch 中同一个注意力接口的两种实现。这里的**后端（backend）**，指框架在接口内部实际选用的计算程序：调用者给出相同的 Q、K、V，框架可以用不同程序完成注意力计算。

**数学后端（`SDPBackend.MATH`）**按注意力公式组合矩阵乘、Softmax 等通用运算，保存完整的分数和概率中间矩阵。**FlashAttention 后端（`SDPBackend.FLASH_ATTENTION`）**采用前面介绍的分块与在线归约方法，避免保存完整的中间矩阵。实验分别指定这两个后端，比较它们完成同一次注意力计算的时间与新增显存。

实验用 RTX PRO 6000 Blackwell，输入输出为 BF16，一次处理一个序列、一个注意力头，头维度为 128。

[19] FlashAttention、FlashAttention-2、FlashAttention-3。

序列长为 8192 个 token；采用因果注意力，即每个位置只使用自身及此前的位置。预热后，使用 CUDA Graph 将一组 GPU 操作预先记录并重复执行，以免每次由主机提交造成干扰。按每次注意力计算统计，保存完整中间矩阵的数学实现约需 2.6 ms，采用分块与在线归约的 FlashAttention 实现约需 80 µs，后者的加速比约为 33。[20]

再比较一次普通调用的新增显存占用峰值。输入 Q、K、V 预先驻留显存，计量范围为输出与临时工作区：数学实现约为 848 MiB，FlashAttention 实现约为 22 MiB。

这组完整实现的对照说明了分块、在线归约和融合如何配合：分块使当前数据能放入快速存储，在线归约使 Softmax 可以逐块计算，融合省去块内中间结果的写回。三者共同改变了执行时间与中间存储需求。

## 5.4 编译器：表达、变换与选择

前两节通过手工分析选择了循环、块大小、缓冲和算子边界。实际模型会使用多种输入形状和算子组合，每种加速器也有不同的存储层级和执行资源。编译器的任务，是用程序表示这些实现方式，根据依赖关系排除错误的变换，再比较其余实现的执行开销。

### 5.4.1 为什么要比较多种实现

回顾逐元素链：SiLU 对每个数执行非线性变换，再与另一输入对应相乘，最后量化成低位宽表示。两处相邻算子之间，都可以选择分开执行或融合执行，共有四种划分：三步独立、融合前两步、融合后两步、三步融合。如果一条链包含十个算子，就有 $2^9 = 512$ 种相邻边界划分。每一种划分还可以选择块大小、局部布局和流水深度。

这些选择会相互影响。合并两步可以消除中间写回，但也会增加同一时刻需要保存的值；临时数据增多，又会改变能够采用的块大小。因而，编译器要同时表达计算内容、执行顺序和数据存放位置。只记录一个算子名，无法推导这些代价。

编译器可以按三个步骤选择实现。首先，把数学运算表示为循环和数组访问，明确可以变换的循环及其读写关系；其次，根据读写依赖和数值规则检查合法性；最后，结合容量和实测时间比较执行成本。下面先用可读的循环展示前两步，再说明自动算子代码生成系统 AKG 如何通过多面体编译（用线性约束描述循环迭代与数组访问的编译方法）组织整个过程。

### 5.4.2 用循环变换表达分块与融合

第 5.2 节决定了每次处理多大一块，还要决定这些块按什么顺序计算。同样切好的数据，若刚用过就换走，随后又要重新搬回，仍然会产生不必要的访问。循环变换让程序表达这些安排：split 将循环拆成“第几块”和“块内第几个位置”；reorder 调整遍历顺序，让即将再次使用的数据有机会留在局部存储中。再选择在哪一层循环保存或使用中间结果，就能明确它要留下多久。

取 $Y = \mathrm{SiLU}(AW)$，沿用 $M = 1024, K = 4096, N = 12288$，矩阵乘用 FP32 累加，结果舍入为 BF16 后再激活。最直接的程序先生成完整 C，再遍历 C 生成 Y：

```
for i in range(M):
    for j in range(N):
        acc = 0.0
        for k in range(K):
```

[20] 实验 5-3。RTX PRO 6000 热缓存单头对照：2.64 ms / 79.5 µs；新增分配峰值约 848 / 22.2 MiB，包括各后端分配的临时量。

```
            acc += A[i, k] * W[k, j]
        C[i, j] = round_to_bf16(acc)

for i in range(M):
    for j in range(N):
        Y[i, j] = silu(C[i, j])
```

第一项变换是**拆分（split）**。把 i 拆成块号 $i_o$ 与块内坐标 $i_i$，令 $i = 64i_o + i_i$。原来的 1024 行就成为 16 个块，每块 64 行。对 j 同样处理，得到 192 个列块。同时拆分两个输出维度，就形成 64×64 的 tile。

拆分首先改变了索引的表示方式。要连续完成一个 tile，还需要**重排（reorder）**：在外层循环中遍历输出块，在内层循环中遍历块内元素。再把 K 按 32 拆成 128 个归约块。程序先选定一个输出 tile，逐块加载所需 A、W，累加到同一个输出块中。

采用这种循环结构后，就能确定输入块需要保留多久。A、W 的当前块各占 4 KiB，在一次归约块计算后即可替换；4096 个 FP32 累加器占 16 KiB，要一直保留到全部 128 次归约块计算结束。因此，两类缓冲应在不同的循环层分配和释放。

最后调整激活的**计算位置**。一个输出 tile 完成全部 K 归约后，其 4096 个 C 值已经全部算好，可以立即做 SiLU 并写出 Y；其他输出 tile 尚未完成也不影响这一片结果。这样只需在局部保存当前输出块的结果，不再需要完整 24 MiB 的 C，也就省去了 48 MiB 写回与重读。

```
for io in range(ceil_div(M, BM)):
    for jo in range(ceil_div(N, BN)):
        acc = zeros_fp32(BM, BN)
        for ko in range(ceil_div(K, BK)):
            a = load_tile(A, io, ko, BM, BK)
            w = load_tile(W, ko, jo, BK, BN)
            matmul_accumulate(acc, a, w)
        c = round_to_bf16(acc)
        store_tile(Y, io, jo, silu(c))
```

这里 `load_tile` 和 `matmul_accumulate` 分别表示成块读取和矩阵累加；最后一个不足整块的 tile 用掩码标出有效位置。伪代码的缩进清楚地标出了这些先后关系：输入块在 ko 循环内更新，累加器在整个 ko 循环期间始终保留，激活计算在 ko 循环结束后执行。

这些调度动作在编程系统中有对应的表达。Halide 是把“计算什么”和“如何调度”分开表达的编程系统，程序员用 split、tile、reorder 等动作改变执行方式。这里把生成中间结果的算子称为生产者，使用该结果的算子称为消费者。`compute_at` 指定生产者在消费者的哪一层循环内执行。TVM 是面向张量计算的编译系统，也通过调度指定计算位置和缓存读写，让相邻算子在同一块数据上连续计算。[21]

计算位置同时决定复用和重算。把生成中间结果的计算放在最外层，会先算出完整结果，供所有后续计算读取；放到后续计算的分块循环内，就只算当前需要的部分。如果多个输出块需要同一部分中间结果，在每个块内计算就会造成重复。编译器因此要比较两种做法：先保存中间结果供多次读取，或在每次使用前重新计算。

循环中的 `fuse` 还可以把多个迭代维合成一个维度。例如，可以将 16×192 个输出块的二维坐标转换为 0 到 3071 的一维编号，再分配给加速器工作组。这改变了遍历输出块和分配任务的方式；将 SiLU 紧接在矩阵乘的

[21] TVM、TensorIR。

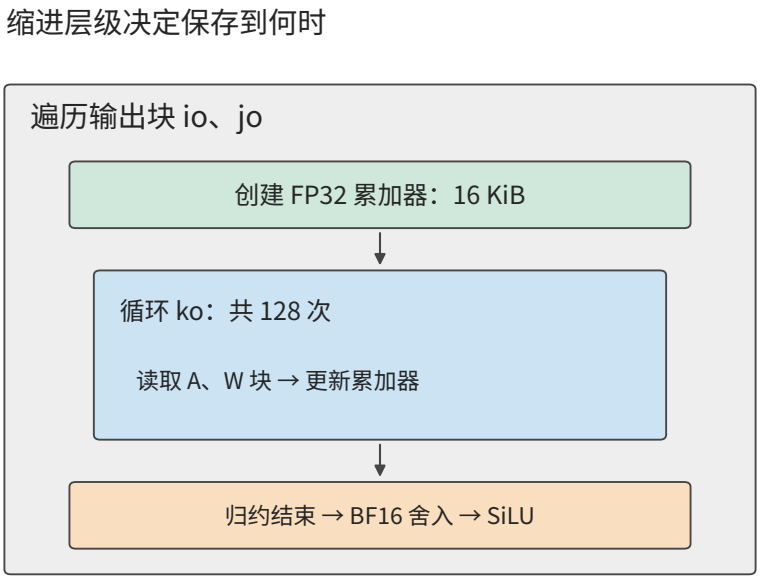



图 5-21：循环层次决定临时数据需要保存多久。外层选输出块，创建 16 KiB 累加器；内层 ko 反复读取 A、W 块，全部归约结束后再舍入并计算激活函数。

tile 后面，则省去了中间矩阵的写回和重读。两者可以在同一份调度中组合使用。

### 5.4.3 依赖与舍入如何限制变换

上一小节把激活移入输出 tile，但仍安排在 ko 循环结束后执行，原因可以用两个数说明。设一个输出的两个部分和为 1 与 $-1$，完整和为 0，SiLU(0)=0。若在每个部分和之后先做 SiLU，再相加，结果为 $\mathrm{SiLU}(1)+\mathrm{SiLU}(-1)\approx 0.4621$。把激活计算移到完整求和之前，已经改变了运算。

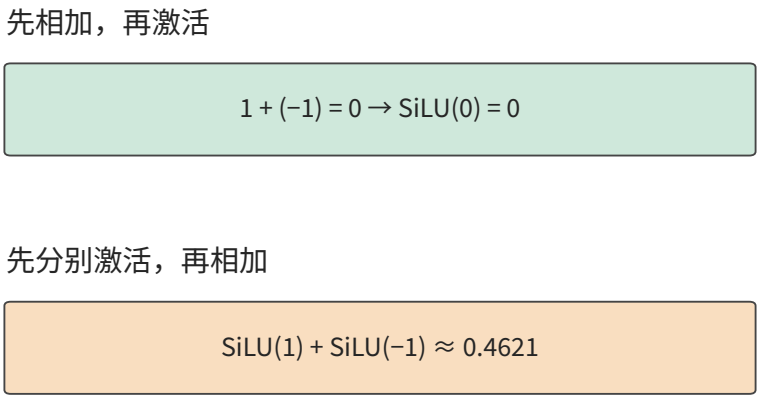



图 5-22：同样两个部分和，先相加得到零，再做 SiLU 仍为零；先对各部分做 SiLU 再相加，得到约 0.4621。两个数说明把激活计算移到求和之前会改变结果。

编译器需要遵守的依赖关系因此有两个层次：同一输出的部分和要按归约规则合并，后续算子要等所需的结果算好。不同输出可以并行计算；同一输出的归约和后续运算则必须按依赖顺序执行。

分块归约同样受这条依赖约束。第 5.3.3 节的 FlashAttention 能逐块完成计算，是因为它保留了 m、$\ell$、u，

并在最大值改变时，同时调整之前累加的指数和与加权值。若只保留每块已经归一化的输出，就丢失了各块分母的相对大小。例如两个块各自只有一个值，分别为 1 和 3，块内输出仍是 1 和 3；仅凭这两个输出，无法判断整行 Softmax 应给它们分配 1:2 还是 2:1 的权重。因此，分块归约必须保存足够的统计量，才能正确合并各块的结果。

浮点舍入进一步限制了变换。原程序将 FP32 累加结果舍入为 BF16，再激活，融合后的程序也要在激活之前执行 `round_to_bf16`。中间值可以从寄存器直接传给激活，但这次舍入仍然存在。数据存放位置与数值格式是两项独立选择。

**例 5-7：整行量化 scale 如何约束分块计算顺序？**一行包含 256 个元素，分成两个块，每块各含 128 个元素，两个块的首个元素分别为 1 和 10，其余为零。量化使用 E4M3FN 格式：一种含符号位、四位指数和三位尾数的八位浮点表示，最大有限值为 448。scale 由整行最大绝对值决定；随后做点积，权重仅第一个元素为 1。因此结果完全取决于第一个元素如何量化。

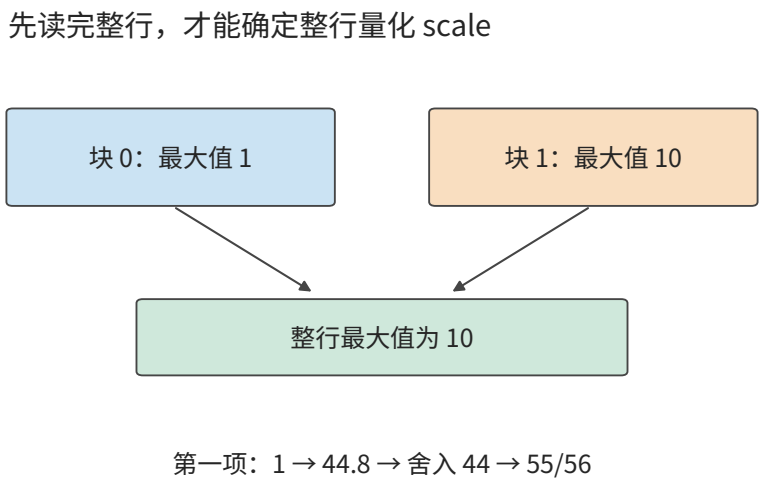



图 5-23：一行分成两个块，后一块中的 10 决定整行 scale。第一项要先按这一 scale 映射，再舍入到格式允许的值，最后反量化。

先读完整行，最大值为 10。第一项缩放为 $1 \times 448/10 = 44.8$，舍入到该格式可表示的 44，反量化结果为 $44 \times 10/448 = 55/56$。若读完第一块就量化，当时最大值为 1，第一项可表示为 448，反量化结果恰好为 1。

读到后一块中的 10 后，即使调整后续计算所用的 scale，第一项也不可能重新经历 44.8→44 的舍入。两种做法的结果相差 1/56，约 1.8%。因此，按整行最大值确定 scale 时，应先读完一行求出最大值，再量化。[22]

这些例子说明了哪些变换可以做，哪些运算顺序必须保留。输出之间可以交换执行顺序，输入可以分块缓存，算好的结果可以直接供后续运算使用；归约的合并方式和量化的舍入位置则属于运算定义，需要在变换中保留。

### 5.4.4 AKG：用多面体编译组织循环与存储

AKG 是面向张量算子的自动代码生成与优化系统，其核心技术是**多面体编译（Polyhedral Compilation）**。该方法把规则循环表示为三类信息：有哪些迭代要执行，每次迭代读写哪些数组元素，以及哪些读写必须保持先后顺序。循环边界和规则下标可以用线性约束描述，这就是“多面体”名称的来源。借助这种表示，编译器能

[22]融合合法性、FP8 舍入和量化投影；RedFuser 后端阅读。444 / 620 MiB 计数包括 A、W 与输出；行 scale 的辅助读写另见配套计算。

够推导迭代之间的依赖，并据此调整执行顺序。[23]

以矩阵乘为例，编译器可以确定不同 i、j 对应不同输出，沿 k 的更新则指向同一个累加结果。选定 64×64×32 的 tile 后，还可以由数组访问推导当前需要的 A、W 区域：A 为 64×32，W 为 32×64。这样既能确定循环的执行顺序，也能确定需要读入哪些数据，以及这些数据要保存多久。

AKG 从张量表达式生成多面体表示，将分块与分层融合结合起来：在外层安排完整算子的执行顺序，在内层安排各块的连续计算和局部缓存。存储管理根据这些区域分配片上空间、插入搬移，代码生成再将计算映射到硬件执行方式，调优阶段选择块形状等参数。前文手工完成的“拆分循环—确定读取区域—保留累加结果—完成归约后激活”，在这里成为相互联系的编译步骤。

将计算和存储一起分析，也能解释融合为什么会增加访存。前面的逐元素链通过融合少读了中间值；若中间结果采用位数更少的表示，又要供后续计算反复读取，保留它反而可以减少读取。

**例 5-8：量化单独执行还是融合进矩阵乘，哪种方式读写更少？**取 $[4096, 4096]$ 的 FP16 输入 A，占 32 MiB，量化为 FP8 后占 16 MiB；FP8 权重为 $[4096, 1536]$，占 6 MiB，FP16 输出占 12 MiB。输出按 128×128 分块，共有 32 个行块、12 个列块。各输出块独立读取所需数据，量化 scale 来自整行最大值。

先看单独执行量化的做法。每处理一行，就用 8 KiB 局部空间保存该行输入，求出 scale 后再量化。全部输入共读入 32 MiB，量化结果共写出 16 MiB。随后，12 个输出列块各读取一遍 FP8 输入，共 192 MiB。因此，与输入有关的读写量为 $32 + 16 + 192 = 240$ MiB。

把量化合并到矩阵乘中后，先读取 32 MiB 原输入求出每行的 scale；12 个列块随后各读一次原来的 32 MiB 输入，并在本地量化。输入相关访问为 $32 + 12 \times 32 = 416$ MiB。虽然省去了 16 MiB 量化结果的写出，但每个列块读取的输入从 16 MiB 增至 32 MiB，12 次读取增加了 192 MiB。

两种方案都由 32 个行块各读取 6 MiB 权重，再写出 12 MiB 输出，共 $32 \times 6 + 12 = 204$ MiB。因此总访问量分别为 $240 + 204 = 444$ MiB 与 $416 + 204 = 620$ MiB，融合后增加约 40%。

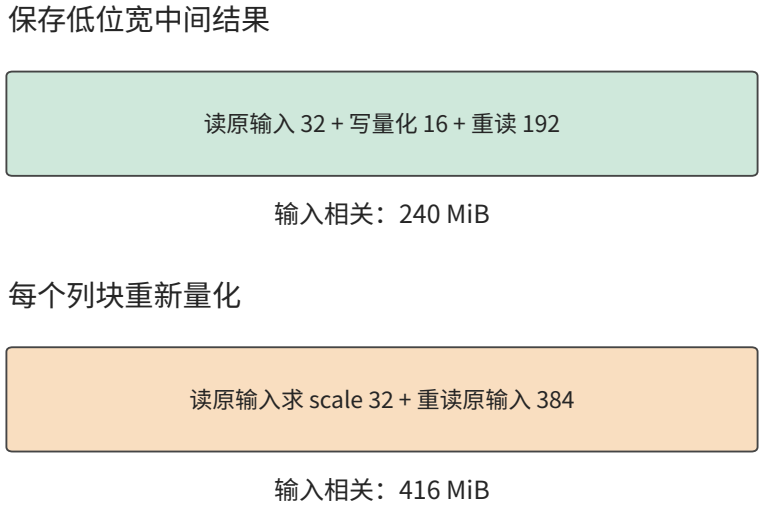



图 5-24：两种方案都先读完整输入以确定每行的 scale。保存 FP8 结果后 12 个列块合计重读 192 MiB；融合方案重读 FP16 输入 384 MiB。权重与输出另有相同的 204 MiB。

[23] Zhao 等，2021，AKG: Automatic Kernel Generation for Neural Processing Units using Polyhedral Transformations。

图 5-24 中，决定差距的是通向 12 个列块的重复读取：每次读取的数据量相差一倍。输出列块数因而直接决定两种做法的访存量差异。每增加一个列块，保留量化结果的方案多读 16 MiB，融合方案多读 32 MiB。将 1536 个输出列放入同一块，可消除这种跨列块重复；相应输出累加器也从 128 列扩大为 1536 列，需要 12 倍空间。因此，选择是否融合量化时，仍要比较第 5.2 节讨论过的局部存储占用与数据复用。[24]

AKG 将融合与分块共同考虑，正是为了处理这样的相互作用。中间结果的计算位置改变后，各输出块读取哪些数据、读取多少次都会改变；缓冲需求随之变化，又影响能够采用的块大小。

### 5.4.5 成本估计与实测选择

例 5-8 已经能够按读写量比较两种实现。面对更多分块与融合组合，编译器也按同样的思路逐步缩小范围：先检查依赖和数值规则，再排除局部存储不足以容纳的实现，最后估计执行开销，决定优先测试哪些实现。第 5.2 节已给出一种估计：用访问量和有效带宽比较两个 tile。编译器还可以加入矩阵指令利用率、寄存器需求和启动成本，确定各实现的测试顺序。

沿用第 5.2 节的矩阵与 RTX PRO 6000 的共享内存，选择过程可以整理成下面的表。该节把输入缓冲和累加器都放在每个 SM 的 100 KB 共享内存里；若把累加器改放寄存器，就要像第 5.1.5 节那样分别检查共享内存与寄存器，两类容量不能互相借用。若把两个候选改成双缓冲，输入占用翻倍，工作集分别变为 32 KiB 和 96 KiB，每个 SM 可驻留的线程块从 4 块、1 块变为 3 块、1 块。

| 选择步骤 | 本章矩阵算例要填的内容 | 排除或比较的依据 |
|---|---|---|
| 固定目标 | 输入形状、精度、误差、调用频数 | 比较同一项数学工作和同一负载 |
| 列出候选 | tile、循环次序、缓冲数、融合范围 | 编译器与硬件是否支持 |
| 检查可行性 | 共享内存、寄存器、线程、对齐和依赖 | 编译资源报告及正确性检查 |
| 估计时间 | 计算量、实际读写量、启动与布局转换 | 各资源服务时间与关键路径 |
| 实测并选择 | 代表形状的时间、波动及完整算子链 | 最小化工作负载成本，并保留相近候选 |

先用资源报告排除容量不足的候选，再测剩余候选的有效带宽、计算时间与完整算子链。若寄存器溢出、矩阵边缘补齐或并发下降抵消了复用收益，就回到候选表改变块形状或缓冲数。第 6 章采用同样过程，只把 tile 的放置扩展到多张卡，并把跨块依赖换算成集合通信（多张卡按固定模式交换或汇总数据的通信操作）。

实测环节可以用配套实验 5-5 的矩阵转置程序说明：同一程序在 RTX PRO 6000 Blackwell 上采用 16×16 块约需 7.9 μs，采用 32×32 块约需 2.9 μs，加速比约为 2.7。块的每个维度增至两倍，内部元素数就增至四倍，存储布局和线程协作也一起改变。[25]

TVM 的自动调优根据硬件实测结果搜索分块、循环顺序等调度方案；让能够调用编译和测试工具的 Agent 修改调度或 kernel 代码，还可以尝试原有调度模板之外的实现。两种搜索都以编译后的程序为评价对象，通过数值检查筛选正确实现，再以执行时间评价调度。评价之所以针对编译后的程序，是因为编译器可以将源码中分开表达的算子融合为一个 kernel。[26] 搜索方法还要解决另一个问题：用什么负载评价这些实现。

**例 5-9：输入形状的调用比例如何改变算子实现的选择？**某个算子有 A、B 两种输入形状，原实现处理它们都需要 10 μs。新实现处理 A 需要 5 μs，处理 B 需要 20 μs：A 的耗时减半，B 的耗时翻倍。若两种形状各出现一次，总时间从 20 μs 增至 25 μs，新实现的耗时增加 25%；若 A 调用 100 次、B 调用一次，总时间从 1010 μs 降至 520 μs，新实现的加速比约为 1.9。

[24] 融合合法性、FP8 舍入和量化投影；RedFuser 后端阅读。444 / 620 MiB 计数包括 A、W 与输出；行 scale 的辅助读写另见配套计算。

[25] 实验 5-5 的生成代码与转置对照。

[26] 实验 5-4 的编译、舍入与执行差异、DeepSeek V4-Flash 专家子链。

令输入形状为 A 的调用占比为 p。原实现的平均执行时间为 10 µs，新实现的平均执行时间为：

$$\overline{T}_{\text{new}} = 5p + 20(1-p) = 20 - 15p \quad \mu\text{s}.$$

要使新实现平均时间更短，需满足 $20 - 15p < 10$，即 $p > 2/3$。图 5-25 的交点说明，只有当 A 占全部调用的比例超过 2/3 时，新实现才更快。

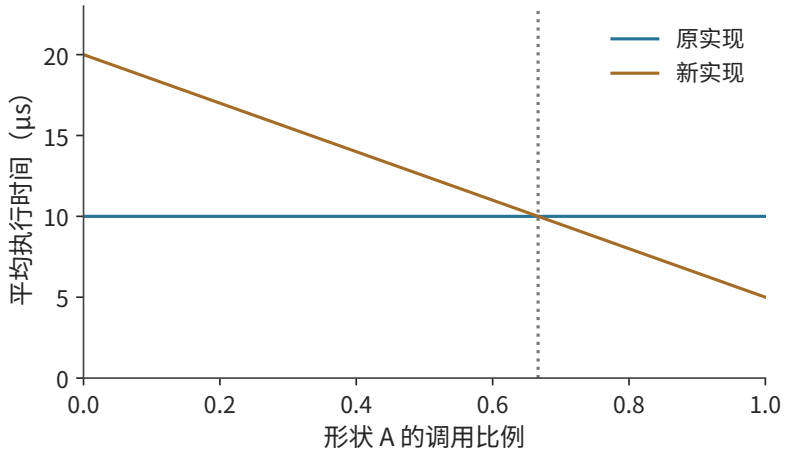



图 5-25：形状 A 占比超过 2/3 时，新实现的总执行时间更短。A、B 原耗时均为 10 µs，新实现分别为 5、20 µs；调用串行，图中比较稳态执行。交点由两种实现的平均时间相等确定。

因此，调优可以按形状分别选择实现，也可以按实际调用频数选择一种综合成本较低的方案。搜索本身消耗的编译与测量时间，则属于准备成本。配套的 Agent 实验保留了代码生成、重复提交和测量记录；下一节将结合调用次数，计算节省的执行时间需要多久才能抵消搜索开销。[27]

## 5.5 运行时：提交、重放与动态形状

编译器选出加速器程序后，运行时还要反复准备输入、选择程序、提交任务并管理缓冲。即使使用同一份 kernel 代码，改变提交方式或复用已有的准备结果，也会改变总时间。本节接着第 5.1 节的时间线，分析这些主机工作。

### 5.5.1 主机准备与加速器计算的重叠

第 5.1 节区分了 CPU 提交时间与加速器执行时间，第 5.3 节又说明两类资源可以重叠工作。把这两点用于模型的一次前向计算：CPU 为下一阶段检查形状、准备参数或元数据，GPU 执行当前阶段。如果下一阶段的准备可以提前，两侧便形成流水；若 CPU 一直等到 GPU 完成才开始准备，加速器会在两个阶段之间停下来。

**例 5-10：加速器加速后，主机为何成为瓶颈？**设有 100 段工作，每段的主机准备和加速器计算各需 20 µs。准备下一段所需的数据均已知，缓冲足够，两类资源独立。严格串行需要 $100 \times (20 + 20) = 4000$ µs；采用流水后，先完成第一段的准备，随后每 20 µs 完成一段，总时间为 $20 + 100 \times 20 = 2020$ µs。

将加速器计算缩短至 5 µs，主机仍每 20 µs 才准备好一段。流水变成 $100 \times 20 + 5 = 2005$ µs，只比原来少 15 µs。CPU 提交任务的速度没有提高，GPU 算得越快，等待下一项任务的时间就越长。再把主机准备时间缩

[27] 实验 5-6 的搜索与测量记录；评测与部署计算。已记录两轮共 12 次 Agent 调用，未产生新的更快实现。

短至 5 µs，两侧才能每 5 µs 完成一段工作，完成时间降为 $5 + 100 \times 5 = 505$ µs。[28]

图 5-26 画出前四段在三种情况下的时间线。

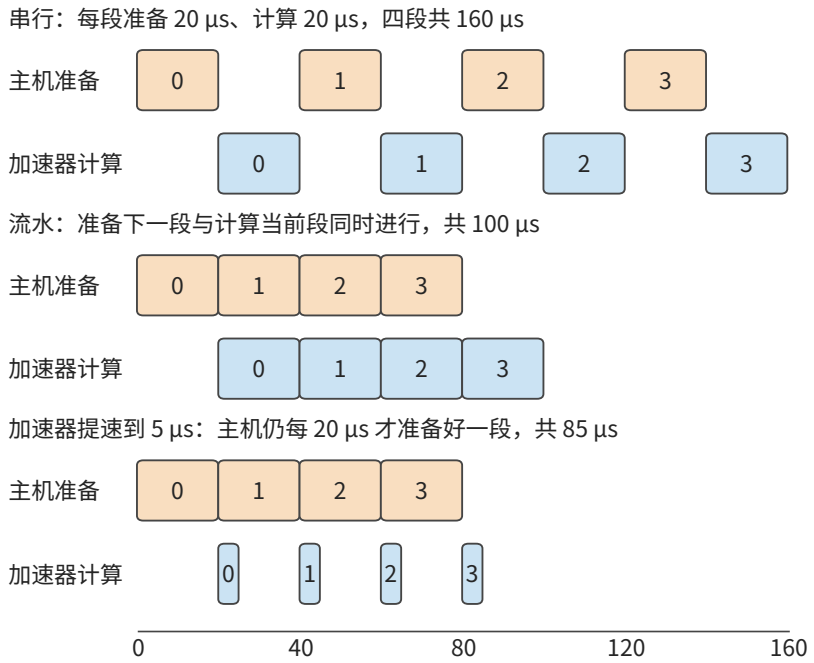



图 5-26：每段主机准备 20 µs。串行执行时两种资源轮流工作；流水后主机准备下一段时加速器计算当前段，每 20 µs 完成一段；加速器提速到 5 µs 后，每段仍要等主机准备好，加速器大部分时间空闲。图中只画前四段，正文算例为 100 段。

这一例子说明了主机优化的两个方向。一是减少准备工作，例如重复使用已经确定的形状和执行计划（根据请求长度、缓存布局等条件事先确定的 kernel 选择、任务划分与工作区安排）；二是把准备移到前一段加速器执行期间，例如异步处理上一轮输出。两者分别减少流水中的工作量和阶段之间的空隙。

在自回归生成中，能否提前准备，取决于所需的数据是否已经算好。下一步计算需要用到上一轮选出的 token；组建 batch、更新语法状态（限定输出格式时，记录已生成内容匹配到语法的哪一步）和判断是否停止，也各自需要相应的结果。运行时可以提前执行不依赖这些结果的工作，减少两轮计算之间的等待。vLLM 等引擎的异步执行与调度演进围绕这一问题展开。[29]

专用加速器把模型执行缩短到远短于主机每轮准备的时间后，稳定流水的步间隔下界便由主机准备决定。此时进一步缩短步间隔，需要把动态 batch（每步都可能加入或移出请求的 batch）准备、地址更新与提交一起优化。

vLLM 在 2024 年 9 月发布的 v0.6.0 就是主机开销占据步间隔的实例。发布前的剖析显示，单张 H100 运行 Llama 3 8B 时，一步之中 API 服务器占 33%，调度占 29%，加速器执行只占 38%：API 服务器与推理引擎运行在同一个 Python 进程里，两者的协程争用 GIL（全局解释器锁），加速器只能空等主机。v0.6.0 把 API 服务器移到单独进程，与引擎之间用消息队列通信，避开了 GIL 争用；又引入多步调度，一次调度后连续执行多步，不再重复准备输入；再把输出处理改为异步，与下一步执行重叠。三项改动合计，Llama 3 8B 的吞吐提高到 v0.5.3 的 2.7 倍，70B 提高到 1.8 倍。[30] 这些改动没有触及任何 kernel，也没有更换硬件。按第 1.3.4 节的判据，主机侧的软件抽象开销一旦占到步间隔的六成，去掉这部分开销，比换一块更快的加速器有效得多。

[28] 主机时间线调研；配置流水与图执行。

[29] 运行方式与反馈、框架演进及token 成本调研第 7—8 节。

[30] vLLM v0.6.0 发布公告归档；收益归因的辨析见 token 成本调研第 8.2 节。公告的吞吐在请求同时到达、`--num-scheduler-steps 10` 的条件下测得，2.7 倍是三项改动的合计收益，不能全部归于 GIL；多步调度在低负载下可能延长首个 token 的等待。

除了同一请求的相邻两轮，不同调用之间也有可以复用的准备工作。两次调用可以分别要求模型解释代码和检查测试，输入内容不同，却仍经过同一组模型层。长度与 batch 落在已有执行计划支持的范围内时，运行时可以复用程序，更新 token、位置索引和状态地址；遇到新的形状或控制路径时，则选择另一份执行计划。应用通过上下文表达变化，运行时利用稳定的执行结构减少重复准备。下面分别计算图重放与形状特化这两种复用的收益。

### 5.5.2 CUDA Graph：提交收益与输入复制开销

上一小节中，GPU 提速后仍在等待主机。模型反复执行相似的任务序列时，其中许多提交工作可以复用：先把加速器任务及其依赖记录成图，再重复提交。使用 CUDA Graph 时，主机每次只需发起一次图重放，不必重新逐个准备和提交 kernel。加速器仍然执行图中的各个节点，减少的是主机的重复工作。图 5-27 用一次 FFN 的六个 kernel 说明两种提交方式的区别。

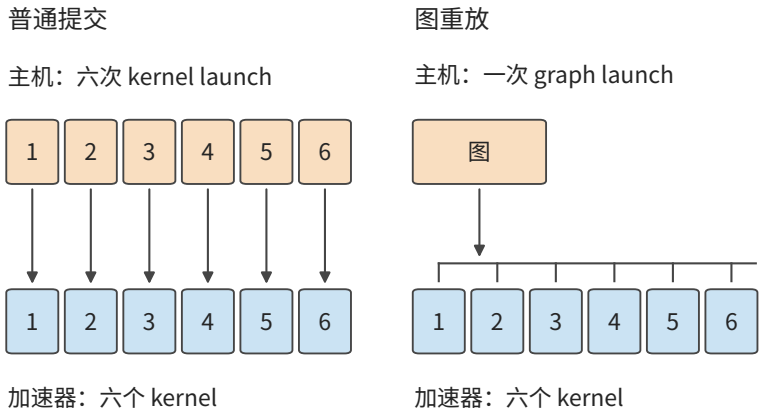



图 5-27：普通提交时，主机为每个 kernel 发起一次 launch；图重放时，主机只发起一次 graph launch，加速器仍然执行图中记录的六个 kernel。要减少 kernel 的数量，靠的是融合，不是图重放。

图 5-28 取 3 次 FFN 执行作对照。普通提交产生 18 次主机 kernel launch 和 18 个加速器 kernel；图重放改为 3 次 graph launch，加速器仍执行 18 个 kernel。将激活链融合后，加速器 kernel 减至 15 个；再使用图重放，主机仍只需 3 次 graph launch。融合将多个算子的计算合并到同一个 kernel 中，图重放减少了主机逐个准备和提交任务的工作。[31]

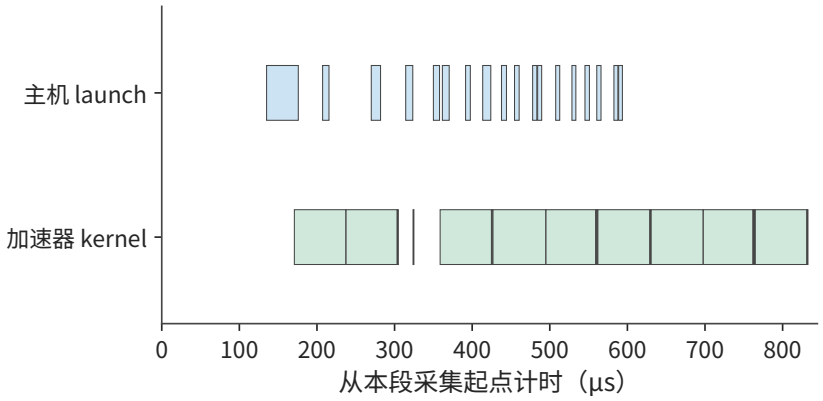



图 5-28：普通提交的 3 次 FFN 实测。上行为主机 kernel launch API，下行为加速器 kernel；横轴从本段标记起点计时。18 次 kernel launch 对应 18 个加速器 kernel。

[31] 实验 5-8与轨迹分析 JSON。

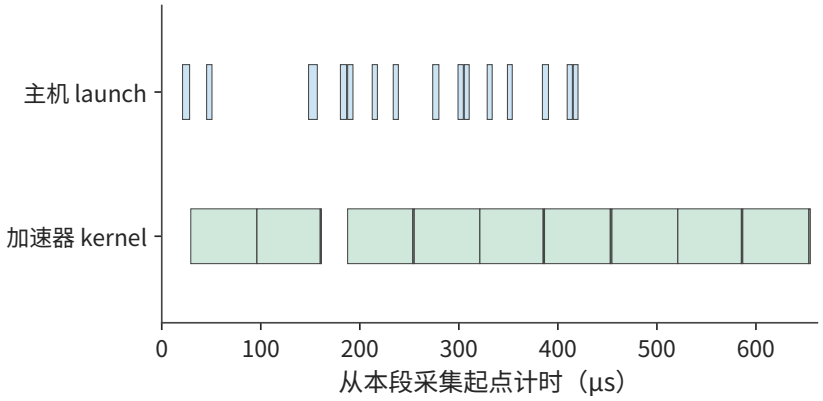



图 5-29：3 次 FFN 融合激活链后，主机 kernel launch 15 次，加速器相应执行 15 个 kernel。数据来自与前图相同的实验中单独标记的采集区间。

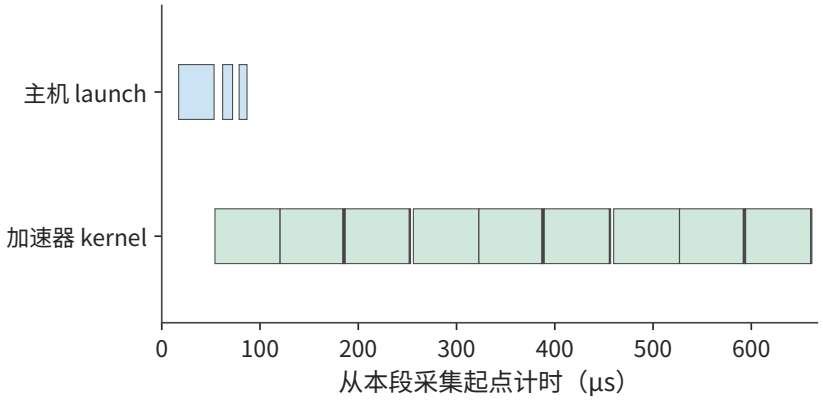



图 5-30：3 次 graph launch 对应 18 个加速器 kernel。图重放减少主机提交次数，加速器仍执行原图各节点；本图按自身采集范围标出真实时间。

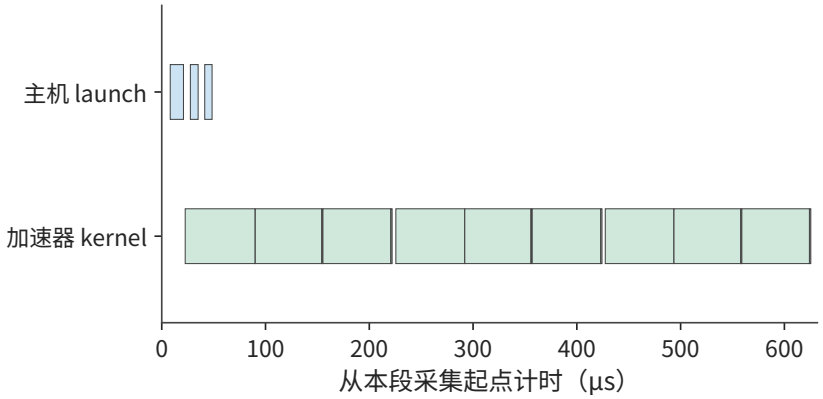



图 5-31：先融合再重放，主机发起 3 次图执行，加速器执行 15 个 kernel。融合减少加速器 kernel 数，图重放减少主机提交次数。

图重放前，还要把新输入放到图所记录的地址。典型实现为图预留固定缓冲区，上游每次将新数据写到这里，图按已记录的地址读取。若上游输出位于另一份张量中，就在图执行前增加一次复制；若上游直接写固定缓冲，这次复制便可省去。

**例 5-11：输入复制何时抵消图重放的收益？**普通执行的加速器计算为 20 µs，加速器等待主机提交任务的时间为 20 µs，共 40 µs。图重放与元数据准备为 5 µs，加速器计算保持 20 µs。图原本可以节省 15 µs。

取图执行所需的 BF16 输入为 $[256, 4096]$，数据量为 2 MiB。复制同时读取源和写入目标，读写量合计为 4

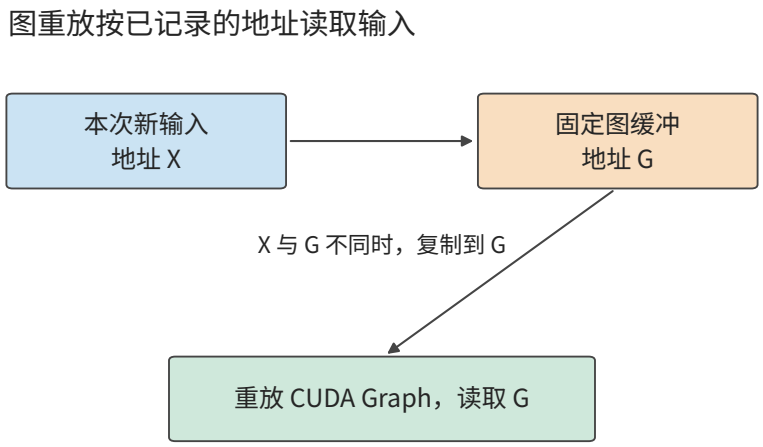



图 5-32：图执行时读取记录的地址 G。新输入位于 X 时先复制到 G；上游直接写 G 时沿用同一缓冲，省去中间复制。

MiB；按 RTX PRO 6000 的显存带宽 1792 GB/s 计（读与写共用这一带宽），复制约需 2.3 µs，图执行的总时间约为 27.3 µs，比普通执行少约 12.7 µs。

保持计算与准备时间不变，把输入扩大至 2048 行，数据量增为 16 MiB。复制读写增为 32 MiB，用时约 18.7 µs，图执行的总时间约为 43.7 µs。这次复制已超过原本节省的 15 µs。

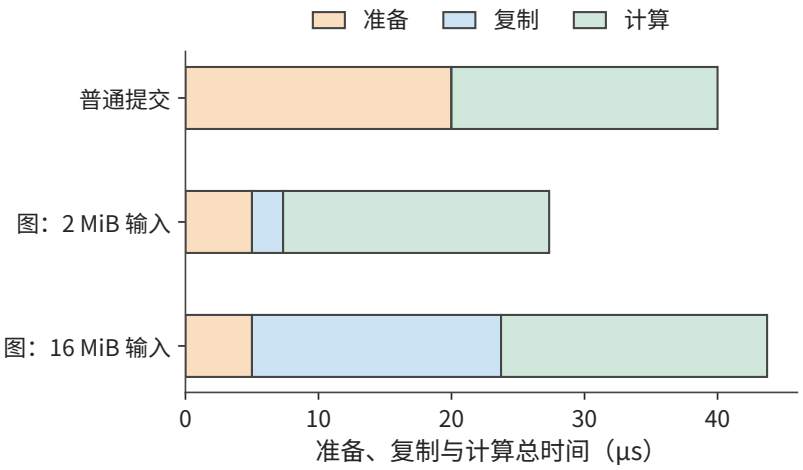



图 5-33：橙色为准备，蓝色为额外输入复制，绿色为加速器计算。普通方式 40 µs；图的 2 MiB 输入约 27 µs，16 MiB 输入约 44 µs。复制按 RTX PRO 6000 的显存带宽 1792 GB/s 计。

图 5-33 中，图重放节省的准备时间是一段固定长度；表示复制时间的条形却随输入增大而变长。两者长度相等时，图重放恰好不再节省时间。令输入数据量为 X，复制带宽为 B，图执行更快的条件为 $2X/B < 15\ \mu$s。代入 B=1792 GB/s 得：

$$X < \frac{15 \times 10^{-6} \times 1.792 \times 10^{12}}{2}\ \text{bytes} \approx 12.8\ \text{MiB}.$$

2 MiB 低于这一临界值，16 MiB 则超过。让前一个算子直接将结果写入图缓冲，或者从输入张量较小的位置开始捕获，都能减少复制开销。因此，选择哪些操作组成一张图时，需要同时比较节省的主机提交时间与增

加的输入复制时间。[32]

图重放复用了提交序列，同一思路还可以用于生成这些任务之前的准备工作。例如，执行计划可以供多个模型层共同使用。推理算子库 FlashInfer 把计划生成（plan）与执行（run）分开：plan 根据本步各请求的长度和 KV 在显存中的存放位置选择 kernel、划分任务、安排工作区，并把这些元数据复制到 GPU；run 按计划执行注意力。同一步的 36 层请求长度相同，KV 的存放方式也相同，可以共用一份计划；若每层都重新 plan，就要做 36 次相同的准备工作。在 RTX PRO 6000 上的两组对照中，改为只 plan 一次后，36 层注意力调用的时间从约 2.0、2.4 ms 降至 0.72、0.75 ms，主机到 GPU 的复制从 144 次降至 4 次，加速器 kernel 数保持不变。反过来，请求长度改变后必须重新 plan，沿用旧计划会按旧长度计算，结果出错。[33]

### 5.5.3 动态形状与编译开销

多个模型层共享计划，利用的是计算结构相同；不同调用共享编译后的程序，还要处理输入形状的变化。输入行数变化时，运行时可以选择三种方式。通用程序根据当前形状计算索引和边界；分桶将输入补齐到少数代表形状，复用这些形状的程序；特化则利用已知的输入条件生成专门的实现，这里指为每个常见形状分别编译程序。越充分地利用已知形状，越有机会简化加速器工作，但也需要编译和保存更多程序。

**例 5-12：重复执行多少次，特化才值得？**取 10 次 FFN 为一组：8 次为 256 行，1 次为 1536 行，1 次为 2048 行，共 $8\times256+1536+2048=5632$ 行。采用 512、2048 两个桶时，实际处理 $8\times512+2048+2048=8192$ 行，多算约 45%。图 5-34 画出每种形状被补齐到哪个桶。

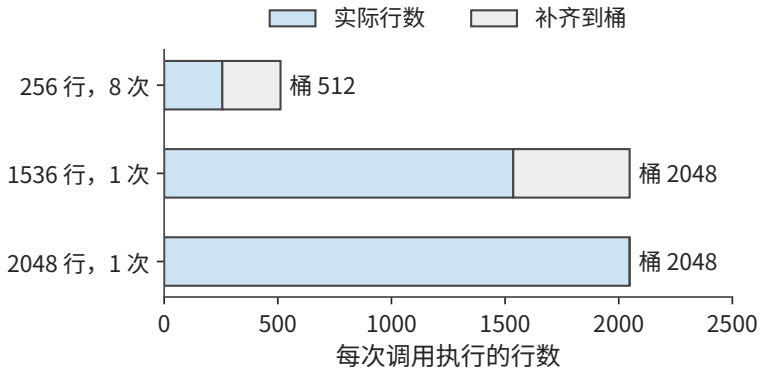



图 5-34：256 行的调用补齐到 512 行的桶，1536 行的调用补齐到 2048 行的桶，2048 行恰好落在桶的边界上。灰色部分是补齐后多算的行；10 次调用合计实际 5632 行，执行 8192 行。

完整 FFN 的三个投影每行共需 $6HF$ FLOPs。在 RTX PRO 6000 上，取通用、分桶、特化实现分别达到 BF16 稠密峰值 503.8 TFLOP/s 的约 20%、40%、50%，即 MFU 为 20%、40%、50%，有效矩阵处理率为 100、200、250 TFLOP/s，则每组时间分别约为 17.0、12.4、6.8 ms。分桶虽然多算 45%，处理率却翻倍，仍比通用实现更快。再设它们的准备时间分别为 100、400、900 ms，得到下表。[34]

| 策略 | 一次性准备 P | 每组执行 T | 形状处理方式 |
|---|---|---|---|
| 通用 | 100 ms | 17.0 ms | 按实际 5632 行执行 |
| 分桶 | 400 ms | 12.4 ms | 补至两个桶，执行 8192 行 |
| 逐形状特化 | 900 ms | 6.8 ms | 三个形状分别生成程序 |

[32]图重放的小输入复制、大输入复制与图执行取舍。复制带宽按源读加目标写的接口流量定义。

[33]FlashInfer 计划复用实测。

[34]形状特化的完整输入与交点。

运行 r 组的总时间为 $T_{\text{total}} = P + rT$。图 5-35 中，每条线的截距表示准备时间，斜率表示每组执行时间。本例中，每组执行越快的方案，直线越平缓，但起点也越高，因为它需要更长的准备时间。

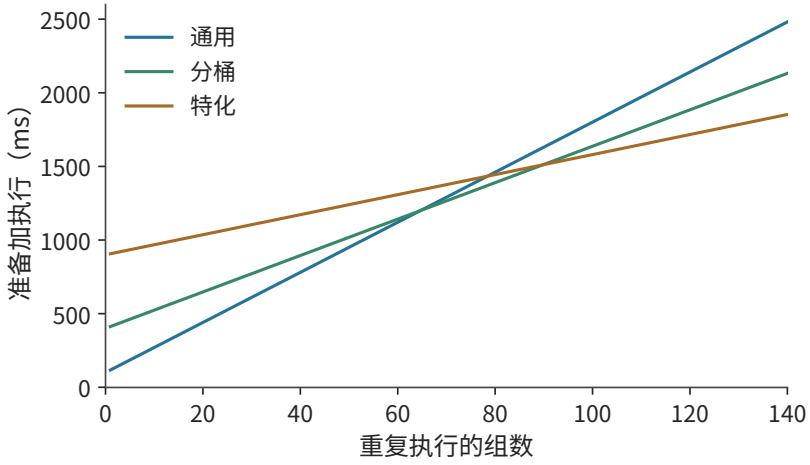



图 5-35：同一组调用反复执行时，最省时的策略随复用次数变化。曲线采用例 5-12 的形状、处理率和准备时间，不同策略适用的整数调用次数范围使用未经四舍五入的数值计算。“通用” 使用同一执行方案处理所有形状；“分桶” 将相近形状归组；“特化” 为具体形状选择专用方案。

分桶比通用方案多用 300 ms 准备，每组却能节省约 4.6 ms，执行约 65 组后就能抵消这部分开销。特化比分桶再多用 500 ms 准备，每组再省约 5.6 ms，需要执行约 90 组。由完整数值求得，1–64 组通用最省，65–89 组分桶最省，90 组起特化最省。

如果实例只执行 70 组，总时间分别约为 1.29、1.27、1.38 s。分桶用时最短，是因为节省的执行时间已经超过增加的准备时间；特化虽然每组执行得更快，但累计节省的时间还不足以抵消额外的准备开销。若三种程序均已缓存，本次准备成本为零，特化从第一组起就用时最少。运行时的选择因而同时依赖形状频数与程序是否已经编译并缓存。

同样可以计算自动调优需要多少次调用才能抵消准备开销。若搜索耗时 10 分钟，每次使用新实现省 5 µs，以累计节省时间抵消搜索时间，需要 $600/(5 \times 10^{-6}) = 1.2 \times 10^{8}$ 次调用。每秒 10,000 次约需 3.3 小时，每秒 100 次约需 14 天。调优耗时越长，就越应优先优化调用频繁、能够长期使用的形状。

第 8 章将结合请求到达情况和动态批处理，进一步讨论如何选择和缓存程序。如果某种形状在短时间内反复出现，保留对应的编译结果可以减少重复准备；如果长期不再使用，就可以删除该缓存，为其他形状腾出空间。

### 5.5.4 Persistent Kernel：按数据块调度任务

图重放和程序缓存减少了主机的重复准备，加速器上还有另一类等待：对于有依赖关系的两个 kernel，后一个通常仍要等前一个全部完成后才能开始。如果后一个算子只需要前一个算子的一块结果，就可以把等待条件缩小到数据块：第一块算好后，立即开始后续运算，同时继续计算其余数据块。

**persistent kernel** 一直驻留在加速器上运行，从队列中取出任务，通过事件确认所需数据是否准备好。这样既减少主机反复 kernel launch 的工作，也让相邻算子能按块重叠执行。Mirage Persistent Kernel / MPK 是采用这种任务调度方式的研究系统。[35]

[35]设备任务组织与 MPK；八块任务计算。投影按 RTX PRO 6000 的 BF16 稠密峰值 503.8 TFLOP/s 推得，激活按 1792 GB/s 显存带宽与每元素 4 bytes 读写（448 G 元素/s）推得，两项取自硬件表；独立资源、任务开销和缓冲条件采用例中设定。

仍以 Qwen3-8B 的 up 投影后接 SiLU 激活为例，取 512 个 token 的特征向量，每 64 个 token 的向量组成一个数据块，共八块。每块投影为 $2 \times 64 \times 4096 \times 12288 \approx 6.44$ GFLOPs，按 RTX PRO 6000 的 BF16 稠密峰值 503.8 TFLOP/s 约需 12.8 µs；每块激活处理 $64 \times 12288$ 个 BF16 元素，每个元素读 2 bytes、写 2 bytes，按 1792 GB/s 的显存带宽约需 1.76 µs。设矩阵与向量资源独立、缓冲充足，每次 kernel launch 需要 5 µs。等全部投影完成再开始激活，共需 $2 \times 5 + 8 \times (12.8 + 1.76) \approx 126$ µs。

现在将计算分为八个投影任务和八个激活任务，每个任务增加 0.5 µs 调度时间、0.2 µs 完成通知时间。每块投影变为 13.5 µs，激活变为 2.46 µs。一次 kernel launch 后，八块投影首尾相接，每块投影完成后立即激活，最后一块激活在投影全部结束后收尾，完成时间为 $5 + 8 \times 13.5 + 2.46 \approx 115$ µs，加速比约为 1.1。

这里较慢的阶段是投影。节省的约 11.0 µs 有两个来源：少了一次 5 µs 的 kernel launch，七块激活约 12.3 µs 隐藏在投影时间内；任务开销又在关键路径上加回 6.3 µs。激活每块只需 1.76 µs，按块提前开始，能隐藏的时间也仅限于此。两个阶段的耗时越接近，按块重叠节省的时间越多；一个阶段远长于另一个时，收益基本只剩少一次 kernel launch。

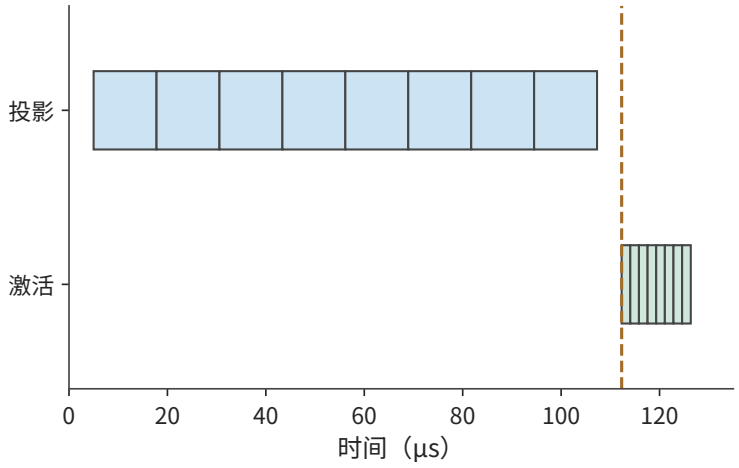



图 5-36：粗粒度执行先完成八块投影，再启动八块激活。按 RTX PRO 6000 的矩阵峰值与显存带宽，每块投影约 12.8 µs、激活约 1.76 µs，两次 kernel launch 各 5 µs；竖线为激活开始。

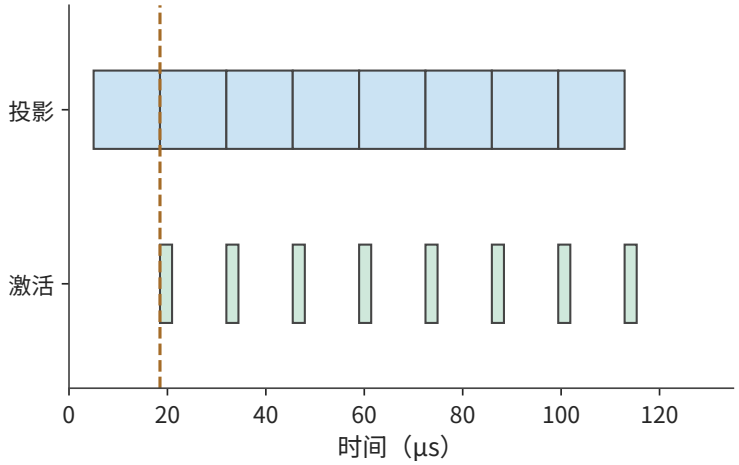



图 5-37：矩阵与向量资源独立、缓冲充足。每个任务另计 0.7 µs 调度与通知，首块投影完成后即可激活；投影首尾相接，激活只在每块投影之后短暂工作，八块流水约 115 µs 结束。

沿着图 5-37 的投影时间线看，八块投影首尾相接，中间没有空隙。这与双缓冲是同一个规律：第一块先完成前一阶段，之后每隔较慢阶段所需的时间完成一块；这里较慢的是投影，正如第 5.3.2 节较慢的是搬移，流水

的节拍由它决定。每个缓冲槽从投影开始一直被占用到激活结束；没有空槽时，投影任务就要等待。因此，任务队列、完成事件和空闲缓冲共同决定下一块何时能够开始。

## 5.6 从局部优化到完整请求

前面每项优化都有明确的作用对象：分块减少输入重读，融合减少中间读写，运行时优化减少准备和等待。但完整请求还包含其他工作，局部节省的时间只占其中一部分。本节先说明如何从局部加速推算请求时间，再分析 Qwen3-8B 的实际请求。

### 5.6.1 热点占比决定加速空间

设请求由串行阶段构成，其他阶段时间保持不变，热点占原时间的比例为 f，热点的加速比为 s。将原请求时间归一化为 1，新时间为 $(1-f)+f/s$，整体加速为：

$$S_{\text{request}} = \frac{1}{(1-f)+f/s}.$$

若热点占 20%，速度提高到原来的两倍后，总时间变为 $0.8+0.2/2=0.9$，加速比约为 1.11。继续把热点加快到十倍，总时间为 0.82；即使热点耗时降到零，其他阶段仍需原总时间的 0.8，整体加速上限为 1.25 倍。

这就是第 1 章介绍的 Amdahl 定律。该定律解释了优化收益逐渐缩小的原因：热点耗时越短，其他阶段在总时间中所占的比例就越高。

### 5.6.2 并行分支与关键路径切换

在 Amdahl 关系中，时间按串行阶段相加。若请求中有两个分支同时执行，缩短一个分支后，还可能需要等待另一个分支；此时应沿依赖图寻找决定完成时间的最长路径。设准备 10 µs 后，同时启动热点 A 和分支 B，分别需 60、40 µs；收尾需 10 µs，并等待两条分支都完成。原请求时间为：

$$T = 10 + \max(60, 40) + 10 = 80\ \mu\text{s}.$$

将 A 加快四倍至 15 µs，B 仍需 40 µs，新时间为 $10+\max(15,40)+10=60$ µs。A 节省 45 µs，请求却只节省 20 µs，因为 B 接替 A 成为关键路径。

A 缩至 40 µs 时，恰好与 B 同时结束；继续只优化 A，完成时间便停在 60 µs。该交点给出了单独优化 A 所能获得的最大收益。若要继续缩短请求，应优化 B，或减少准备和收尾。

共享资源还会改变分支自身的时间。设 A 的新实现大量占用带宽，使并行执行的 B 耗时增至 90 µs，请求就变成 $10+\max(15,90)+10=110$ µs。B 多用的时间超过了 A 节省的计算时间，整个请求反而变慢。[36]

关键路径说明，一个分支节省的时间要放回整张依赖图，才知道请求能快多少。存储上的节省也要这样放回整个执行过程：一份状态少占的容量，未必等于每步少读的字节，因为每步读取多少，取决于有多少层读它、每层读哪些条目。第 2.3.6 节的 V4.1 会话已经给出 8K 条件下的一组数字：V4.1 让 38 个全局注意力层共享 4 份全局 KV，驻留降到 V4-Flash 的约四分之一，逐层逻辑读取却只少约 9%，并且超过了驻留量。共享状态不必每层各存一份副本，但用到它的每一层仍要读取它，容量的节省不会原样变成读取的节省。[37]

[36] 请求关键路径教学计算、争用情景。

[37] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

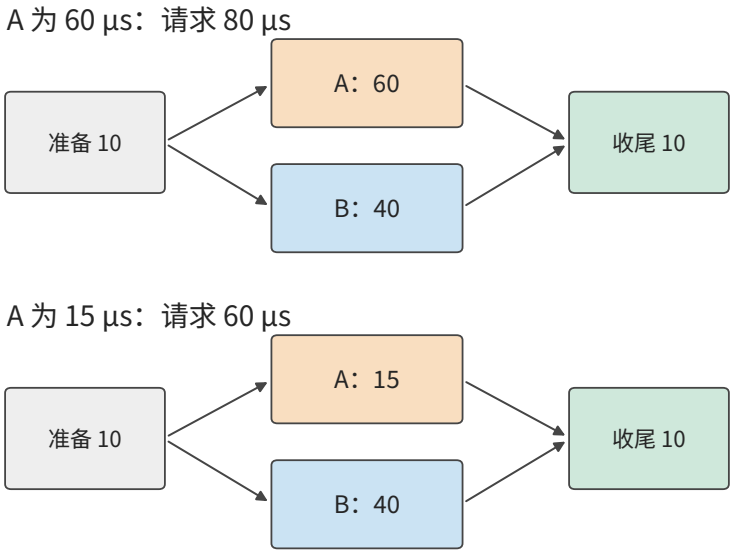



图 5-38：收尾等待 A、B 两条分支。A 从 60 µs 缩短至 15 µs 后，较慢分支由 A 切换为 B，请求从 80 µs 降至 60 µs。准备和收尾各为 10 µs，两条分支使用独立资源。

上下文变长后，仍需执行的索引扫描也会带来更多开销。128K 时，两代模型的全局历史状态与索引的逻辑读取量分别约为 65.085 与 33.422 MiB。V4.1 后续层的索引只访问候选池，初始索引仍需遍历全局。因此，即使后续层索引的访问范围固定，整个 decode 的开销仍会随上下文长度增长。

改变开销的不只是数据从哪里取得，程序是否真正省去了计算也有影响。候选池把后续索引的搜索范围缩小，工作量随之减少。公开参考实现先对完整索引缓存执行点积，再屏蔽候选池之外的位置；论文中的生产实现直接减少后续索引扫描的条目。前者已经完成了池外点积，后者在发起计算时就省去这部分工作。同一选择规则由此对应两种执行成本。

### 5.6.3 综合案例：一次 Qwen3 请求的优化选择

将这一方法用于 RTX PRO 6000 上的 Qwen3-8B。请求输入 7239 个 token，生成 32 个输出 token，并发为 1，采用 BF16 与 eager 执行；这里的 eager 表示由主机按程序顺序逐项提交加速器工作。模型有 36 层，先做一次 prefill 产生首输出，再做 31 次 decode。每层每步执行一次 SwiGLU 的激活部分，即第 5.3.1 节的 $Z = \mathrm{SiLU}(G) \odot U$，共有 $36 \times 32 = 1152$ 次调用，其中 prefill 36 次，decode 1116 次。

两类调用承担不同工作。同一请求的 prefill 在 36 个模型层中分别对 7239 个输入 token 执行该算子，共处理 $36 \times 7239 = 260604$ 个 token 表示；31 步 decode 在每层处理一个新 token，共执行 $36 \times 31 = 1116$ 次单 token 算子调用。这里按“层数 × token 数”累计算子工作，同一个 token 在不同层参与计算会被重复计数。decode 的调用数是 prefill 的 31 倍，处理的行数却小得多。这正是本章反复遇到的形状差异：大矩阵提供较多并行工作，小矩阵的计算时间较短，kernel launch 和任务分配的开销更突出，也更难充分利用加速器。

在同一引擎中，将 36 层 SwiGLU 替换为一个采用不同调度方案的 kernel，执行记录确认 1152 次调用均使用了替换后的 kernel。阶段计时如下。[38]

[38] 实验 5-9 的 kernel 实现、执行记录与 11 对请求。vLLM 0.23，替换后的 kernel 采用 block256 / 4-warps 调度，11 对输出 token 一致。客户端中位数为 816.640 / 815.406 ms，配对时间差的中位数 1.343 ms，首 token 中位数 435.674 / 436.353 ms。单独采集的性能分析记录中，prefill 为 11.218659 / 11.244366 ms，decode 为 2.369803 / 0.920376 ms。原 kernel 耗时 13.588462 ms，替换后 12.164742 ms，采集区间 850.848989 ms；各热点与其他观测 GPU 工作的时间交集为零。

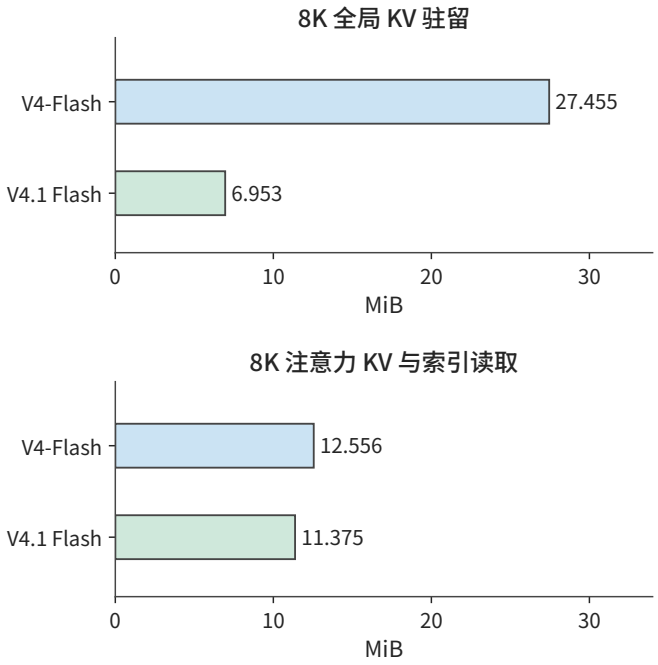



图 5-39：全局驻留与逐层逻辑读取分开比较。上方仅计全局 KV 容量，下方读取包含全局条目、局部窗口和索引；两栏各自采用相同横轴尺度。数值按生产布局计算。

| SwiGLU 阶段 | 调用形状与次数 | 原 kernel | 替换后的 kernel |
|---|---|---|---|
| Prefill | 7239 行，36 次 | 约 11.2 ms | 约 11.2 ms |
| Decode | 1 行，1116 次 | 约 2.4 ms | 约 0.92 ms |

替换后的 kernel 主要缩短了一行输入的 decode 激活时间，加速比约为 2.6，decode 部分合计节省约 1.45 ms。prefill 的激活耗时基本相同，替换后反而多约 0.03 ms。轨迹中的这些激活与其他加速器工作顺序执行。原 kernel 的全部激活计算约为 13.6 ms，占约 851 ms 采集区间的 1.6%；保持其他工作不变，即使省去全部激活计算，也最多节省 13.6 ms。目前只缩短了 decode 部分，全部激活计算从 13.6 ms 降到 12.2 ms，因此整个请求预计减少约 1.4 ms。

在客户端交错测量替换前后的请求，共得到 11 对结果。替换前后的首 token 时间均约 436 ms，完整请求中位数分别约为 817 ms、815 ms。对每轮计算“替换前用时减去替换后用时”，配对时间差的中位数约为 1.3 ms，约占原请求耗时的 0.16%，与约 1.4 ms 的预期相符。图 5-40 列出每轮差值，其中 9 对在替换后耗时缩短，2 对在替换后耗时增加。

decode 的改进发生在首输出之后，因此当前长输入请求的首 token 时间仍由 prefill 中的计算与等待决定。输出越长，一行输入的激活计算重复执行的次数越多，单步节省的时间也就累积得越多。

分析完整请求时，首先统计各种形状的调用次数，再计算每次调用节省的时间，最后根据依赖关系判断这些节省能否缩短整个请求。这样，就能把 kernel 的加速比、各阶段的耗时和用户实际等待的时间联系起来。

## 常见误区

**误区：仅凭 kernel 数量判断执行成本。**图重放将 3 次 FFN 的主机提交从 18 次降到 3 次，加速器仍执行 18 个 kernel；融合才把加速器 kernel 减到 15 个。主机提交和加速器计算应分别沿各自的时间线解释。

**误区：访问量最小的实现一定最快。**注意力的 K/V 块从 64 行缩到 1 行，访问量减少 144 MiB，状态更新

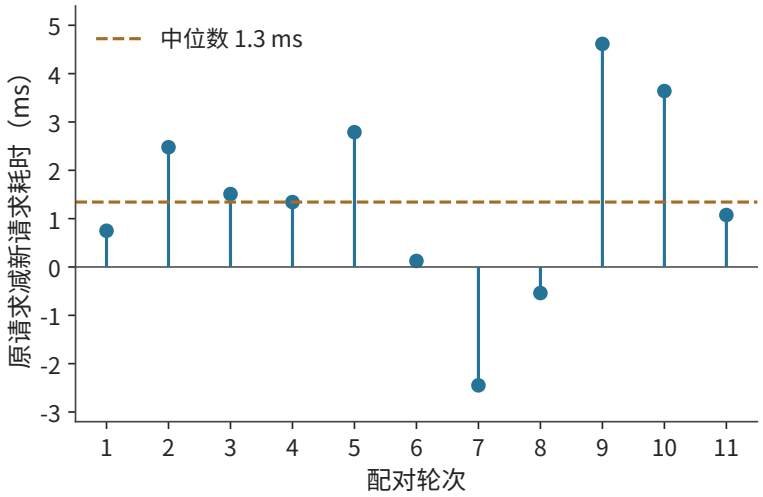



图 5-40：同轮替换前的请求时间减去替换后的请求时间，正值表示替换后更快。RTX PRO 6000 上的 Qwen3-8B，7239 个 token 的输入、强制 32 个 token 的输出，并发 1，BF16、eager，关闭前缀缓存；11 对交错计时，轮内顺序随机，期间有其他驻留服务。虚线为配对时间差的中位数约 1.3 ms。阶段表来自单独采集的性能分析记录。

次数却增至约 41 倍。读取、状态更新与矩阵处理率共同构成执行成本。

**误区：增加缓冲就能提高吞吐率。** H100 一个 SM 上每块搬移 1.29 µs、计算 0.28 µs，两个槽已让搬移器连续工作，四块在 5.44 µs 完成。第三个槽不能让搬移更快，加快计算也不行；要缩短时间，就要减少每块搬移的字节，或者提高带宽。

**误区：稳态执行最快，总耗时就最短。** 执行 70 组调用时，分桶约需 1.27 s，逐形状特化约需 1.38 s。特化每组执行得更快，但节省的时间尚不足以抵消额外的准备开销。

## 习题与配套实验

以下题目沿本章算例逐步改变条件。先完成纸面推导，再用配套代码或执行记录解释差异；实验入口保留原编号，配置、操作步骤和完整记录见对应材料。

**实验 5-1 ·延伸：矩阵块大小如何改变局部存储与读取量**

将例 5-3 的输出块改为 64×128，归约块仍为 32，求局部存储需求、A/W 各自的读取次数与总访问量。与 64×64 方案相比，64×128 方案的有效带宽至少要达到前者的多少，才能使访存时间更短？再比较两种循环次序的执行记录，解释连续访问如何影响处理率。[a]

[a]实验 5-1、GPU 分块、归约拆分。

**实验 5-2 ·核心：激活算子融合能减少多少访存与显存占用**

对 G、U 各 24 MiB 的激活链，分别画出三个算子独立执行和完全融合时，各张量的分配、使用与释放顺序，推导 156 MiB、60 MiB 的访问量，以及两种方案的显存占用峰值。若融合实现每次增加 5 µs 固定开销，接口带宽取 RTX PRO 6000 的显存带宽 1792 GB/s，输入行数至少达到多少时，融合节省的访存时间才能超过这项开销？再结合配套实验结果，解释访问量减少的比例与耗时缩短的比例之间的关系。[a]

[a]实验 5-2 的原始条件、数据与分析。RTX PRO 6000、共享 GPU、热缓存与图重放；原始中位数为 14.20 / 7.68 µs，仅测激活子链。

**实验 5-3 ・延伸：分块归约与预取如何改变注意力计算和访存**

在例 5-6 的输入中再加入一个元素，其分数为 $\ln 4$、对应值为 5，更新 m、$\ell$、u，并与一次处理三个元素比较。保持注意力缓冲的总容量为 99 KB（101,376 bytes），从中额外划出一个 b×d 的 BF16 预取槽，取 b=64，求新的最大 Q 行数、扫描次数与访问量。解释预取缓冲为何会改变数据的重复读取次数。[a]

[a]实验 5-3。RTX PRO 6000 热缓存单头对照：2.64 ms / 79.5 µs；新增分配峰值约 848 / 22.2 MiB，包括各后端分配的临时量。

**实验 5-4 ・延伸：算子融合会改变计算结果还是执行成本**

在第 5.4.2 节的伪代码中，分别将 SiLU 移到 ko 循环内、删除中间 BF16 舍入、将量化计算移入每个输出列块的循环。说明各自改变了数学运算、数值舍入还是执行成本，并给出一个可区分前后结果或访问量的输入。[a]

[a]实验 5-4 的编译、舍入与执行差异、DeepSeek V4-Flash 专家子链。

**实验 5-5 ・延伸：分块循环如何改变访问顺序与临时存储**

用块号与块内坐标写出 i-k-j 的分块循环，标出输入缓存和累加器在程序中的创建、最后使用与释放位置。再对照配套转置程序采用两种块大小时的执行过程，比较访问连续性、同步操作和临时数据量，并据此解释执行时间的差异。[a]

[a]实验 5-5 的生成代码与转置对照。

**实验 5-6 ・延伸：输入形状的调用比例如何影响调优收益**

采用例 5-9 的两种输入形状，令形状 A 的调用占比分别为 1/2、2/3、9/10，计算原实现和新实现的平均时间。若根据形状选择实现，每次多花 1 µs，对 A 使用新实现、对 B 使用原实现，何时优于始终使用原实现？再根据配套搜索记录，统计搜索的总耗时，并计算找到的实现在重复调用中能节省多少执行时间。[a]

[a]实验 5-6 的搜索与测量记录；评测与部署计算。已记录两轮共 12 次 Agent 调用，未产生新的更快实现。

**实验 5-7 ・延伸：特化需要重复执行多少次才有收益**

画出例 5-12 中准备与执行的完整时间线。将每组中小形状的调用次数从 8 次改为 4 次，其他两种形状仍各调用一次。对三种方案两两比较，求各对方案总耗时相等时的重复次数。再假设两个分桶对应的程序已编译并缓存，求准备时间改变后，各方案总耗时最短时对应的重复次数范围。[a]

[a]形状特化的完整输入与交点。

**实验 5-8 ・核心：图重放与算子融合分别减少哪些开销**

阅读图 5-28，分别统计四种方式下的主机提交次数与加速器 kernel 数量。把例 5-11 换到 RTX 4090 上，复制带宽取其显存带宽 1008 GB/s，求输入数据增大到多少时，复制开销恰好抵消图重放节省的时间；再让上游直接写入图缓冲，分别在输入大小为 2 MiB 和 16 MiB 时，比较普通提交与图重放的完成时间。[a]

[a]实验 5-8与轨迹分析 JSON。

**实验 5-9 ・核心：kernel 加速如何改变请求耗时与关键路径**

根据配套的 11 对请求记录，先计算每对请求的耗时差，再取这些差值的中位数。解释它与分别取两组请求耗时的中位数后再相减有何区别。采用本章综合案例的条件，保持 prefill 不变，把 decode 步数从 31 增至 127，假设每步节省的时间不变，估算激活计算总共能节省多少时间。再将图 5-38 中 A 的执行时间逐步缩短，求完整请求时间不再随之缩短时 A 的执行时间。[a]

[a]实验 5-9 的 kernel 实现、执行记录与 11 对请求。vLLM 0.23，替换后的 kernel 采用 block256 / 4-warps 调度，11 对输出 token 一致。客户端中位数为 816.640 / 815.406 ms，配对时间差的中位数 1.343 ms，首 token 中位数 435.674 / 436.353 ms。单独采

> 集的性能分析记录中，prefill 为 11.218659 / 11.244366 ms，decode 为 2.369803 / 0.920376 ms。原 kernel 耗时 13.588462 ms，替换后 12.164742 ms，采集区间 850.848989 ms；各热点与其他观测 GPU 工作的时间交集为零。

## 历史与进一步阅读

Halide 将算法与调度分离，使分块、重排和计算位置成为程序员可组合的动作；TVM 将张量计算、后端生成与自动调优联结起来；AKG 则将多面体调度、分层融合与片上存储安排结合，用于神经处理器代码生成。沿三者的原始材料阅读，可以进一步追踪表达方式如何影响编译器能够自动完成的变换。[39][40]

FPGA 是可配置逻辑器件，高层次综合（HLS）将较高层程序转换为硬件数据通路。笔者早期的 FPGA / HLS 工作，也经历过从接口封装转向代码变换的过程：为了形成所需流水，必须识别原语和读写依赖，改变生成的数据通路。这段经历说明，研究重点从简化编程转向了改进执行方式。

算子编排优化系统 Korch 提供了另一个有启发性的例子：其中一个历史案例里，子图原本由三个 kernel 执行，合计耗时约 91 μs；调整为四个 kernel 后，合计耗时约 69 μs。重新拆分和组合算子缩短了加速器执行时间，足以抵消额外的 kernel launch 和数据传递开销。[41] 这一方向与本章的融合反例一起，说明了优化应比较整个计算过程的开销。

## 本章小结

本章从一次矩阵投影出发，讨论了存储访问、算子融合、编译与运行时，最后分析完整请求。各节反复使用了三个基本判断。

第一，**复用需要空间**。更大的输出 tile 让同一输入参与更多乘加，也需要更多累加存储；保留量化结果让多个列块读取位宽更低的数据，也需要中间存储。判断这部分存储是否值得，应看它能减少多少重读或重算。在 SM 上，这份空间还决定能驻留多少线程块，从而决定有多少在途访存请求可以隐藏延迟。

第二，**数据依赖决定计算顺序**。逐元素运算可以在一个位置的输入准备好后直接完成，归约则要保存部分和或在线统计。编译器根据依赖移动计算位置，运行时通过事件确认数据是否就绪、缓冲区能否释放。融合和流水都以这些依赖关系为基础。

第三，**总时间取决于各项工作的执行顺序**。带宽决定传输一批数据需要多久，较慢阶段决定流水线中相邻两块的完成间隔，复用次数决定后续节省的执行时间能否抵消首次准备时间，关键路径决定局部改进能否缩短请求。因此，既要计算减少了多少工作，也要确定这些工作原本是否影响了最终完成时间。

下一章将这些关系推广到多个加速器。一片结果何时就绪、谁需要读取、要保存多久，将决定通信何时开始以及两端需要多少缓冲。

[39] TVM、TensorIR。

[40] Zhao 等，2021，AKG: Automatic Kernel Generation for Neural Processing Units using Polyhedral Transformations。

[41] Korch 的编排案例：论文 V100 / FP32 子图，三 kernel 为 38.7、24.2、28.2 μs，四 kernel 为 7.6、18.4、7.5、35.7 μs。

# 第 6 章 超节点

上一章把矩阵分成小块，让输入在计算单元附近多用几次。现在把其中一块交给另一张加速卡：这张卡需要哪些输入，算完得到的是完整输出还是部分和，结果又由谁接着使用？数学没有变，但原来经过共享内存的数据，现在可能要穿过加速器互联甚至机房网络。同一种分块方式，由此变成了多卡并行方案。

模型权重一旦超过单卡显存，一张卡运行一个模型的做法就行不通了：Qwen3-235B-A22B 的 BF16 权重约 470 GB，接近单张 H100 显存的六倍。给定八张卡，可以让各卡分别处理请求，也可以合力加速一个请求；切分方式既要容纳模型，又要达到延迟与成本目标。本章从算子的维度与依赖出发，逐步推导各种并行方式产生的通信，最后用给定的模型、八卡硬件和会话期限筛选方案。读完本章，读者应当能够为自己的模型和硬件列出可选的方案，排除不可行的，再根据瓶颈和测量结果做出选择，而不只是记住几个并行方式的缩写。

上一章研究单个加速器内的计算、数据复用与任务执行。多卡并行面对的仍是同一组问题：沿哪些维度切分，每一块放在哪里，各块需要哪些输入、结果在哪里汇合，以及按什么顺序执行。本章把单卡算子的分块与调度扩展到多张卡：先把矩阵和状态分到多张卡，推导由此产生的通信，再把通信放到实际连接上，最后决定一个实例使用多少张卡、同时部署多少个实例。本章的硬件是常见的 HGX H100 八卡服务器，贯穿推导的模型有两个：Qwen3-235B-A22B 的权重要占满整台服务器，用来说明容量、专家分工和 MoE 通信；Qwen3-32B 的权重可以放入一张卡，长上下文的 KV 却迫使模型跨卡，用来比较张量并行的卡数与实例数。其他模型和系统则用来说明结构变化对设计的影响。

## 6.1 从单卡到多卡

### 6.1.1 八张卡的独立部署与协作

第 1 章介绍的推理实例可以占用一张卡，也可以把模型分布在多张卡上。超节点则为实例内的频繁协作提供物理资源：其中的加速器经高带宽、低延迟的互联紧密相连。

推理实例描述模型如何部署和执行，超节点描述加速器如何互联。八卡超节点可以放八个单卡实例，也可以放两个四卡实例。前者的八个请求彼此独立；后者每个请求要在四张卡之间交换中间结果。服务器和机柜是安装与供电的单位；同一实例的各张卡可以都在同一台服务器内，也可以分布在多台服务器上。

用更多的卡，通常出于三种需求：模型权重超过单卡内存，要分散存储；单个请求执行过慢，要把计算分给多张卡；同时到来的请求过多，要增加能独立处理请求的实例。这三种需求分别对应容量、延迟和吞吐三个目标。

### 6.1.2 贯穿算例：Qwen3-235B-A22B 的容量与单步时间

本章的贯穿算例是：在一台 HGX H100 服务器上运行 Qwen3-235B-A22B，模型已处理完输入，缓存中有 8192 个上下文 token，需要继续生成。HGX H100 是常见的八卡服务器：八张 H100 SXM 各有 80 GB HBM，峰值带宽 3350 GB/s，BF16 稠密矩阵运算的峰值为 989.4 TFLOP/s；八张卡经 NVSwitch（组织 NVLink 交换网络的交换芯片）用第四代 NVLink 互联，每张卡每个方向 450 GB/s；每张卡另配一张 400 Gbit/s（50 GB/s）的 ConnectX-7 网卡，用来连接其他服务器。[1]

[1]HGX H100 数据手册：八张 GPU 经 NVSwitch 互联，GPU 间 NVLink 900 GB/s，网络速率至 400 Gbit/s；NVIDIA H100 规格列出 H100 SXM 的 80 GB、3.35 TB/s、NVLink 900 GB/s 与 PCIe Gen5 128 GB/s，后两项为收发两个方向的合计；H100 架构白皮书第 47 页：18 条第四代 NVLink，每条每方向 25 GB/s；ConnectX-7 数据手册：单端口至 400 Gbit/s；DGX H100 用户指南：8 张 H100、640 GB GPU 内存。每卡一张 400 Gbit/s 网卡的配置也见实验 7-3 的公开运行记录。HBM 带宽与 989.4 TFLOP/s 取自硬件

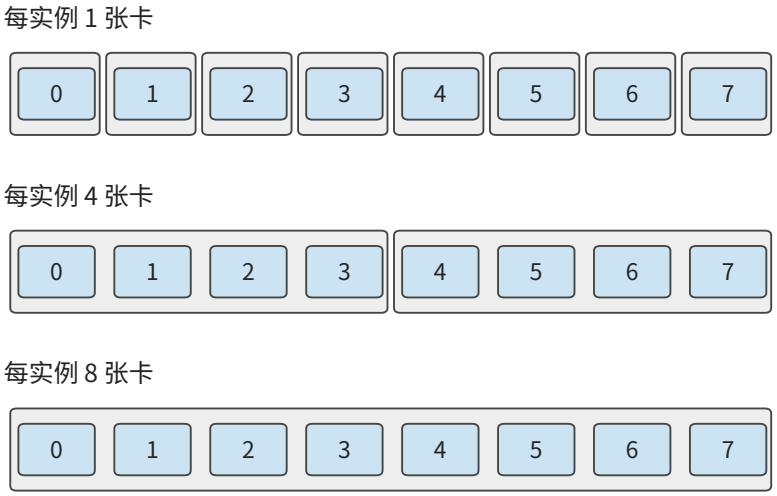



图 6-1：相同八张卡上的三种实例分组。每个外框表示一个独立推理实例，内部连线表示完成请求所需的协作。各实例一次处理一个请求时，三种部署方式分别能同时处理八个、两个和一个请求。

多卡设计首先要确定数据放在哪里。第 $i$ 张卡同时保存的权重 $W_i$、KV 状态 $K_i$、激活 $A_i$ 和其余工作区 $U_i$，应满足

$$W_i + K_i + A_i + U_i \le C_i. \tag{6-1}$$

这里 $C_i$ 是该卡可供任务使用的内存。权重随模型确定，KV 随会话上下文增长，激活和工作区随执行方式变化。这四项在同一时刻共用该卡的物理内存。

**Qwen3-235B-A22B 的模型规模。**模型用 BF16 保存权重与 KV，共 94 层，隐藏维为 4096；每层有 64 个查询头、4 个 KV 头，头维为 128。FFN 由 128 个路由专家组成，每个 token 选其中 8 个，每个专家的中间维为 1536。GB、TB 使用十进制，MiB、GiB 使用二进制。

先求容量。全模型 BF16 权重约为 470.2 GB。每个上下文 token 需要保存每层的 K 和 V，因此全模型每 token 的 KV 为

$$2 \times 94 \times 4 \times 128 \times 2 = 188 \text{ KiB}.$$

8192 个 token 的 KV 共约 1.58 GB。为激活和其余工作区预留 2 GiB，即约 2.15 GB，单卡总需求约为 473.9 GB，接近一张 H100 容量的六倍。

要容纳模型，就必须把这些数据分到八张卡：权重按矩阵的行或列分成八份，每张卡各保存一份；工作区仍然每卡各留一份。KV 按注意力头划分，而该模型只有 4 个 KV 头：八张卡时，每两张卡共用一个 KV 头，各自保存一份，所以每卡保存全部 KV 的四分之一。这种分法如何保证计算结果不变，见第 6.2.2 节。每卡内存需求变为：

---

表的 `h100-sxm` 行。

| 放置 | 权重 | KV | 激活与工作区预留 | 合计 |
|---|---|---|---|---|
| 单卡 | 470.2 GB | 1.58 GB | 2.15 GB | 473.9 GB |
| 八卡中的每卡 | 58.96 GB | 0.39 GB | 2.15 GB | 61.50 GB |

每卡约需 61.50 GB，低于 H100 的 80 GB；整台服务器合计约 492.0 GB，640 GB 中还剩约 148 GB。每卡权重比 470.2 GB 的八分之一略多，因为归一化参数和路由器在每张卡上都有一份副本。每卡剩下的约 18.5 GB 可以保存更多会话：按这种分法，一个 8192 token 的会话在每卡占 0.39 GB，整台服务器最多同时保存 47 个这样的会话。分散存储降低了每卡的内存需求；实例的总内存占用则是各卡占用量之和。[2]

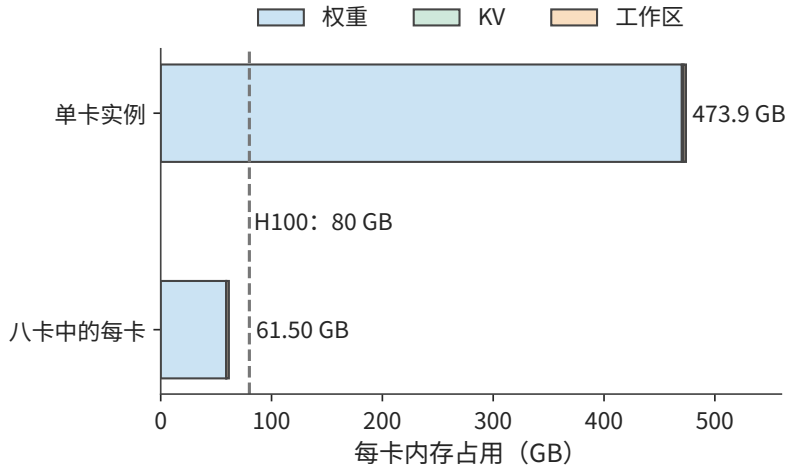



图 6-2：单卡实例与分到八张卡后的每卡内存需求。单卡需要约 473.9 GB，远超 H100 的 80 GB（虚线）；分到八张卡后每卡约 61.50 GB。2 GiB 工作区在每张卡分别预留。

接着求执行时间。内存容量决定能否保存这些数据，访问量决定一次执行需要读取或写入多少数据。以一层的专家计算为例：生成一个 token 时，路由器选中 8 个专家，每个专家有三个矩阵，权重共 $8\times3\times4096\times1536\times2 = 288$ MiB，都要从 HBM 读一遍。按 3350 GB/s 读取一次，约需 90.1 µs；矩阵乘法约有 3.02 亿 FLOPs，按 989.4 TFLOP/s 执行只需约 0.31 µs。权重读取时间比计算时间长两个多数量级，因此，要缩短这次调用的时间，首先应减少每卡读取的权重。整个模型生成一个 token 要读取约 43.1 GB 权重和 1.58 GB KV，若全部由一张卡读取，约需 13.3 ms；按上面的分法均分到八张卡，每卡约读 5.79 GB，约需 1.73 ms。

算子的计算与内存访问重叠执行时，执行时间可以用下式估计：

$$T_{\text{op}} = \max\left(\frac{F}{P}, \frac{V}{B_{\text{HBM}}}\right). \tag{6-2}$$

$F$ 是运算量，$P$ 是矩阵运算性能，$V$ 是 HBM 访问量，单位为字节。计算和内存访问同时进行，耗时较长的一项决定算子的执行时间。后一个算子要等前一个算子的输出，因此依次执行的各段时间要相加。式 (6-2) 就是第 4.8.1 节的 Roofline 模型。按第 1.2.2 节的定义，单个 token 调用的 MFU 不会超过 0.31/90.1，即约 0.3%。这是 batch 为 1 时硬件允许的上限，与实现好坏无关；本章讨论的各种并行方式，都是在这一约束下重新组织权重的读取与复用。第 6.2 节将说明如何把这一层的权重和计算分配到多张卡，第 6.4 节再计算各卡交换结果所需的时间。

[2] 单卡完整 BF16 权重为 470,187,269,120 bytes，8192 个 token 的 KV 为 1,577,058,304 bytes；八卡 TP 时每卡权重为 58,959,617,024 bytes（含每卡各存一份的归一化参数与路由器），每卡 KV 为 394,264,576 bytes，每卡最多保存 47 个 8192 token 的会话。表中 GB 为十进制，GiB / MiB 为二进制，合计先用精确值求和再取近似；2 GiB 包含激活与其余工作区预留。详见Qwen3-235B-A22B 八卡放置及其 JSON，读取固定官方配置与权重索引。单步读取量与运算量见单请求 8K decode与8192 token prefill。

### 6.1.3 Prefill、Decode 与训练对资源的不同要求

第 6.1.2 节的 90.1 µs 与 0.31 µs 对应单 token 调用。若一次处理更多 token，同一份权重就能供多行输入使用，计算与读取时间的比例也会改变。一次调用处理 $m$ 个 token 时，专家的运算量与 $m$ 成正比，被选中专家的权重却可以在这些 token 之间复用。把上例从单 token 改为 8192 个 token 的 prefill，128 个专家全部被选中，一层专家权重的读取量增到 4.5 GiB，按 3350 GB/s 约需 1.44 ms；矩阵运算量则增长到约 2.47 TFLOPs，按 989.4 TFLOP/s 约需 2.50 ms，超过了读取时间。batch 大时，一份权重供许多输入行使用，执行时间的主要限制随之从权重读取转为矩阵计算。

通信也随形状变化。BF16 的隐藏张量为 $m \times 4096$，单 token 占 8 KiB，8192 个 token 占 64 MiB。两次调用经历相同的层数，交换次数相同，每次传输的数据量却相差 8192 倍。小数据量的传输更容易受启动开销影响，大数据量的传输更依赖持续带宽。

Decode 还有时间上的依赖：当前输出 token 决定下一步输入。单个会话必须逐步推进；多个会话则可以同时提供已经就绪的输入。会话越多，权重复用的机会越多，KV 容量和读取量也越大，因而会同时改变计算时间、内存访问时间和排队时间。

训练在前向之后还要反向传播，计算输入梯度和权重梯度，再用梯度更新参数；前向激活要一直保留到反向用完为止。多卡训练因此还要同步梯度、保存更多状态。本章先介绍模型切分及其产生的数据交换，第 10 章再讨论这些操作如何组成完整的训练步。

### 6.1.4 并行方式一览：切分哪个维度

第 6.1.1 节的三种需求都要把工作分给多张卡，分法却各不相同。分法取决于模型处理的数据和模型本身的结构。一层的输入激活有三个维度：一次处理的样本数 $B$，推理时就是同时处理的请求数；每个样本的序列位置数 $S$；每个位置的隐藏特征数 $H$。第 5 章的矩阵乘把 $B$ 和 $S$ 合成了行数 $M$，注意力却要区分同一序列内的不同位置，所以这里把两者分开。模型本身还有两个可以切分的结构：层 $L$，以及 MoE 模型中每层的专家集合 $E$。图 6-3 把这五个维度画在一起，并标出六种常用并行方式各自切在哪里。

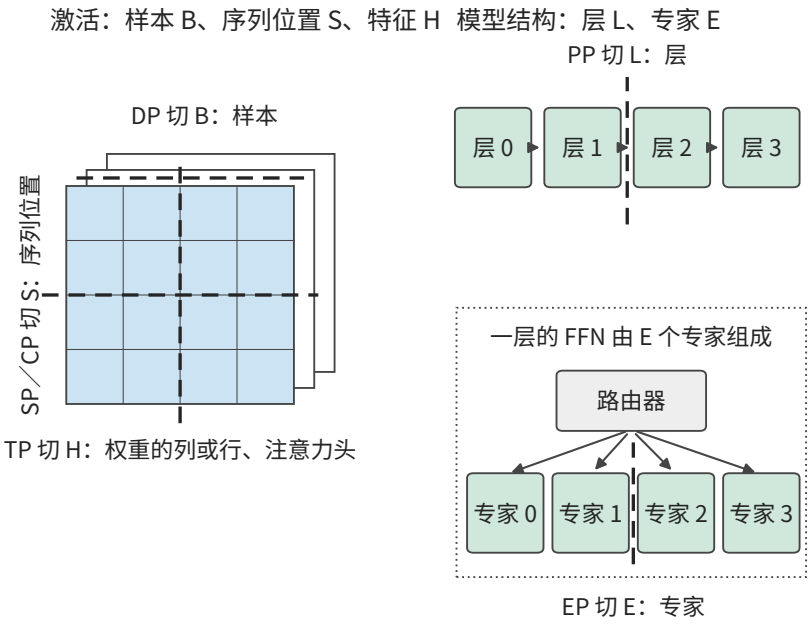



图 6-3：左侧是一层的输入激活，三个维度分别对应数据并行、序列并行与上下文并行、张量并行的切分位置；右侧是模型结构，流水线并行切在层与层之间，专家并行切在同一层的专家之间。粗虚线表示切分位置，不表示数据流向。

按切分的维度，六种并行方式可以概括为下表。表中的“交换”指卡与卡之间必须传递的数据，第 6.2 节各小节将逐一算出其大小。

| 并行方式 | 切分的维度 | 每张卡保存和计算什么 | 卡之间必须交换什么 |
|---|---|---|---|
| 数据并行（DP） | 样本 $B$ | 一份完整模型，不同的样本 | 推理时不交换；训练时汇合各卡的梯度 |
| 张量并行（TP） | 特征 $H$：权重的列或行、注意力头 | 同一层矩阵的一部分 | 输出分片按需收集；部分和必须相加 |
| 序列并行（SP） | 逐 token 算子的序列位置 $S$ | 同一序列在这些算子上的一段位置 | 进入矩阵乘前收集全部位置；部分和求和后重新按位置分片 |
| 上下文并行（CP） | 注意力的序列位置 $S$ | 同一序列一段位置的 Q、K、V | 远端的 K、V，或各段注意力的统计量 |
| 流水线并行（PP） | 层 $L$ | 一段连续的层 | 相邻阶段之间交接激活；训练时再反向交接梯度 |
| 专家并行（EP） | 专家 $E$ | 一部分专家 | 把 token 送到所选专家所在的卡，再把结果送回 |

表中隐含着一项贯穿全章的区别。切样本、序列位置、层或专家时，每张卡得到的是不同数据或不同层的完整结果，需要时收集或交接即可；切特征时，各卡往往只得到同一个输出的一部分贡献，必须把这些部分和相加，结果才能使用。第 5.2.4 节在单卡内区分过输出维与归约维的切分，跨卡时区别相同，只是相加部分和要经过卡间互联。

这些方式不是互斥的。序列并行通常和张量并行使用同一组卡，专家并行可以和注意力部分的数据并行共用同一批卡；流水线并行划分的是有先后依赖的计算图，不是把某个矩阵再切出一个维度。第 6.3 节将说明如何组合这些方式并给每张卡编号。

**实验 6-1 ·延伸：上下文长度与输入行数如何影响容量和执行瓶颈**

（a）求 Qwen3-235B-A22B 在 4096、8192、16384 个上下文 token 时的单请求 KV 容量。

（b）按本节的八卡分法，每卡预留 2 GiB 工作区，求一台 HGX H100 最多能同时保存多少个 32768 token 的会话，权重精确值见注释。

（c）对 1、64、8192 个 token，分别计算一层专家计算的矩阵运算量与被选中专家的权重读取时间（64 个 token 时被选中的专家数用第 6.3.2 节的式（6-8）估计），使用 H100 的参数判断执行时间主要受计算还是内存访问限制。

（d）把 HBM 带宽减半，比较上述三种 token 数下的调用耗时增幅。

## 6.2 六种并行方式

本节按图 6-3 的顺序逐一介绍六种并行方式。为了看清分工，每一小节都先用两张卡说明，再推广到更多卡。

### 6.2.1 数据并行：复制模型，切分样本

**数据并行**（data parallelism，DP）让每张卡保存一份完整模型，各自处理不同的样本。图 6-4 中，两张卡分别处理 batch 的前四个和后四个样本，各自得到自己那部分输出。

推理中的数据并行就是部署多个实例。模型能放进单卡、又有多个独立请求时，最直接的办法是把不同请

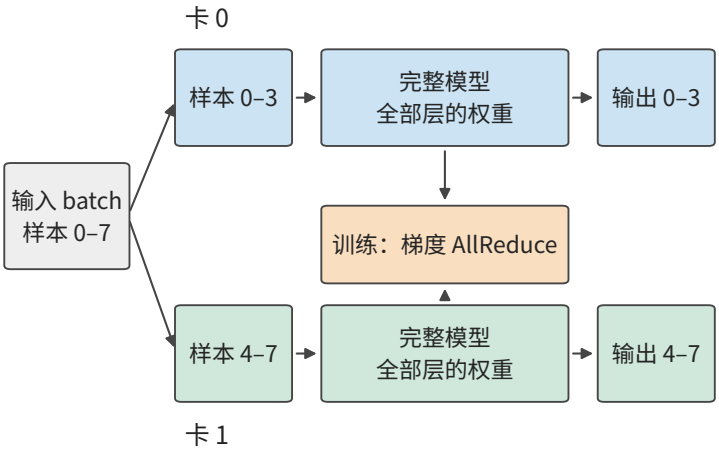



图 6-4：两张卡各保存一份完整模型，分别处理不同样本。推理时两张卡互不等待；训练时两张卡的梯度要先汇合，再各自更新参数副本。

求分给不同实例。每个实例保存自己的权重和会话状态，独立完成输出。若一张卡每秒能处理 $r$ 个请求，八个单卡实例在始终有请求可处理时，每秒就能处理 $8r$ 个请求。

把八张卡合成一个实例后，设单个请求的加速比为 $S(8)$，该实例每秒能处理 $S(8)r$ 个请求。两种部署用的是同样八张卡，吞吐之比为 $S(8)/8$：只有单个请求恰好加速八倍，两者的吞吐才相同。一旦有通信和无法并行的工作，协作实例就是用一部分总吞吐换取更短的单请求时间。

训练的数据并行也在多张卡上放置同一套参数，但各卡的梯度共同决定下一次更新。例如两卡分别处理四个样本，求出各自的平均梯度 $g_0, g_1$，全部八个样本的平均梯度为 $(g_0 + g_1)/2$。两卡各算一半样本，再汇合梯度，就完成了同一次更新。推理实例之间没有这种依赖。

### 6.2.2 张量并行：切分一层内的矩阵

多实例能同时处理更多请求，却不会缩短某个请求在单张卡上的执行时间，也无助于单卡无法容纳的会话。要让两张卡共同承担一个请求，就要把一层内部的计算和状态分开。**张量并行**（tensor parallelism，TP）把同一层内的矩阵分给多张卡，每卡各算一部分，再交换结果。图 6-5 标出一层里的两处切分位置。

**长上下文如何迫使稠密模型跨卡。**稠密的 Qwen3-32B BF16 权重约 65.52 GB，一张 80 GB 的 H100 可以容纳。该模型原生支持 32K token 的上下文，用 YaRN 把位置编码外推四倍后可达 128K。[3] 每个 token 的 KV 为 $2 \times 64 \times 8 \times 128 \times 2 = 256$ KiB，一个 128K 会话要保存约 34.36 GB。单卡部署时，权重、一个 128K 会话的 KV 和 2 GiB 工作区合计约 102.03 GB，超过了 80 GB；分到两张卡，每卡约 52.09 GB，能容纳两个这样的会话；四卡每卡约 27.12 GB，能容纳七个；八卡能容纳十六个。这时采用张量并行首先是为了容量：TP1（TP 后的数字表示张量并行的卡数，TP1 即单卡）连一个 128K 会话都无法容纳。

但有些模型采用了新型注意力机制，可以减少长上下文下的 KV 存储需求。KV 容量由注意力设计决定：层数、KV 头数和头维，以及各层是否都要保存 KV。下表比较三个规模相近的模型：稠密的 Qwen3-32B；同一代的 MoE 模型 Qwen3-30B-A3B，仍用普通的 GQA，48 层，4 个 KV 头，头维 128；以及采用混合注意力的 MoE 模型 Qwen3.6-35B-A3B，40 层中只有 10 层是完整注意力，每层 2 个 KV 头、头维 256，其余 30 层是线性注意力，只保存固定大小的递推状态。

[3] Qwen3 技术报告表 1 列出 Qwen3-32B 为 64 层、64 / 8 个查询 / KV 头、上下文 128K；§3.2 说明预训练序列长 32,768，推理时用 YaRN 与 DCA 把可处理的序列长度提高到四倍。

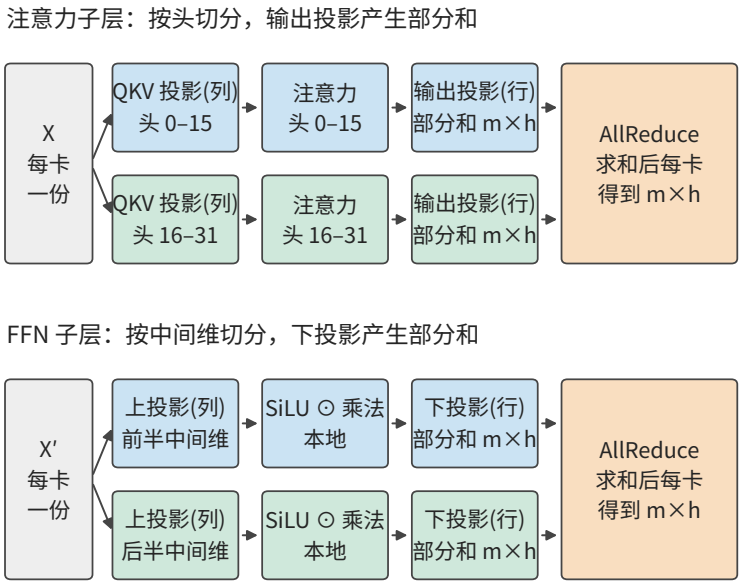



图 6-5：注意力按头分给两张卡，前馈网络按中间维分给两张卡；两处的最后一个投影都只得到部分和，各需要一次 AllReduce 把两卡的部分和相加。m 为一次处理的 token 数，h 为隐藏维。

| 项目 | Qwen3-32B | Qwen3-30B-A3B | Qwen3.6-35B-A3B |
|---|---|---|---|
| 常驻 BF16 权重 | 65.52 GB | 61.06 GB | 69.32 GB |
| 每 token 的 KV | 256 KiB | 96 KiB | 20 KiB（仅 10 个完整注意力层） |
| 每请求的固定状态 | — | — | 62.9 MB 递推状态（FP32）+ 2.0 MB 卷积状态 |
| 一个 128K 请求的状态 | 34.36 GB | 12.88 GB | 2.75 GB |
| 每步权重读取（batch 1，32K 上下文） | 63.97 GB | 6.08 GB | 5.89 GB |
| 每步权重读取（batch 64，32K 上下文） | 63.97 GB | 60.44 GB | 68.30 GB |
| 每步矩阵运算（batch 64，32K 上下文） | 8.49 TFLOPs | 2.04 TFLOPs | 0.73 TFLOPs |
| 每步状态读取（batch 64，32K 上下文） | 549.8 GB | 206.2 GB | 47.1 GB |
| 一张 H100 能容纳的 128K / 32K 请求 | 0 / 1 | 1 / 5 | 3 / 11 |
| 两张 H100 能容纳的 128K / 32K 请求 | 2 / 10 | 7 / 29 | 31 / 117 |

这张表说明了三点。第一，batch 为 1 时，两个 MoE 模型每步只读约十分之一的权重；batch 为 64 时，64 个 token 选中的专家几乎覆盖全部专家，权重读取回到 60～68 GB，与稠密模型的 64 GB 相当。第二，矩阵运算量始终只有稠密模型的 1/12～1/4，这是 MoE 一直保留的优势。第三，上下文长、batch 大时，状态读取成为每步的主要部分：按 3350 GB/s，batch 64、32K 上下文时三个模型读取状态分别约需 164、62 和 14 ms，读取权重只需 18～20 ms。这一项的差距来自注意力设计，而不是 MoE：Qwen3-30B-A3B 同样是 MoE，每 token 仍要保存 96 KiB 的 KV。

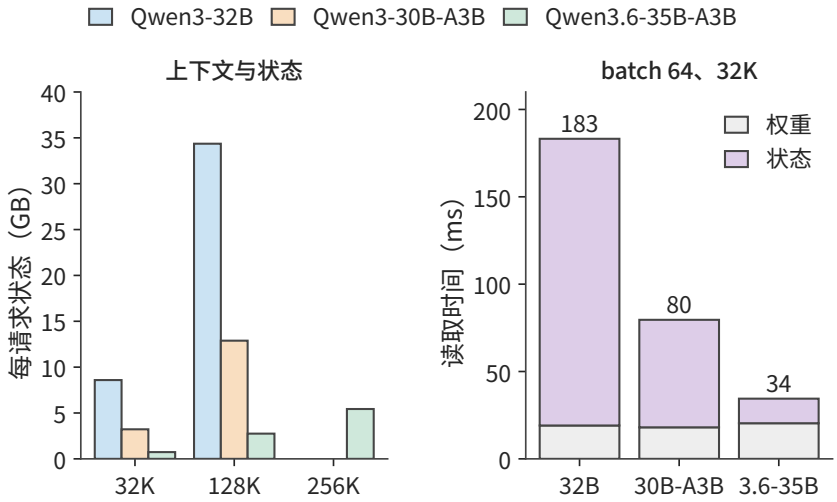



图 6-6：左图为单个请求在 32K、128K 与 256K 上下文下的状态，Qwen3-32B 与 Qwen3-30B-A3B 的上下文上限为 128K。右图为 batch 64、32K 上下文时一次 decode 读取权重和状态的时间，按 H100 的 3350 GB/s 计。三种模型的权重读取相近，状态读取相差十倍以上。

容量随之不同。每卡预留 2 GiB 工作区时，一张 H100 能容纳的 128K 请求分别为 0、1、3 个；两张 H100 时分别为 2、7、31 个。Qwen3.6-35B-A3B 原生支持 262,144 个 token，这样一个请求的状态也只有约 5.43 GB。因此，30B 级别的服务在 128K 上下文下是否需要多卡 TP，主要取决于注意力设计，而不是稠密还是 MoE。DeepSeek V4-Flash 等模型进一步压缩了 KV，第 2.6.1 节在 8K、200K 与 1M 上下文下比较了五个模型的状态与计算，这里不再展开。[4]

回到图 6-5 标出的一层内两处切分：两处的最后一步都只得到部分和，要用 AllReduce 把各卡的部分和相加，再让每张卡都得到总和。Qwen3-32B 每层都要做两次，64 层共 128 次，后文计算执行时间时将反复用到这一次数。要理解为什么一种切法只需拼接、另一种却必须求和，考虑一个最小的矩阵乘。两张卡共同完成时，先要决定各算什么：可以让两张卡分别算出不同的输出元素，也可以让它们各算同一个输出的一部分。前一种结果需要拼接，后一种需要相加。

下面用完整的例子说明这两种切法。输入为 $x = [2, 3]$，权重矩阵为 $W = \begin{bmatrix} 1 & 4 \\ 2 & 5 \end{bmatrix}$，输出是 $xW = [8, 23]$。按权重矩阵的列切分，也就是把输出特征分给不同的卡：卡 0 保存第一列，得到输出的第一个元素 8；卡 1 保存第二列，得到第二个元素 23。两个元素并排拼接，就得到完整输出。

改为按权重矩阵的行切分，也就是把同一个 token 的输入特征分给不同的卡：卡 0 用输入 2 乘第一行，得到 $[2, 8]$；卡 1 用输入 3 乘第二行，得到 $[6, 15]$。两张卡各得到一个含两个元素的向量，但每个向量只包含一行权重的贡献；对应位置相加后，才得到完整输出 $[8, 23]$。

两种切法在 FFN 中可以首尾相接，即图 6-5 下半部分的切法。这里采用第 2 章介绍的 SwiGLU：将输入送入门控投影与上投影，门控结果经过激活函数 SiLU，再与另一支逐元素相乘，最后执行下投影。设本次一起处理 $m$ 个 token，隐藏宽度为 $h$，中间宽度为 $f$。输入为 $X \in \mathbb{R}^{m \times h}$，上投影为 $W_g, W_u \in \mathbb{R}^{h \times f}$，下投影为 $W_d \in \mathbb{R}^{f \times h}$：

$$Y = \left[\mathrm{SiLU}(XW_g) \odot (XW_u)\right] W_d.$$

[4]Qwen3-32B batch 1与batch 64、Qwen3-30B-A3B batch 1与batch 64 均匀路由、Qwen3.6-35B-A3B batch 1与batch 64、Qwen3.6 的 128K 状态与256K 状态。能放下的请求数按 $\lfloor (n \times (80\ \mathrm{GB} - 2\ \mathrm{GiB}) - W)/S \rfloor$ 计，$n$ 为卡数，$W$ 为常驻权重，$S$ 为每请求状态，汇总见第 6 章计算结果的 `dense_moe` 字段。Qwen3.6 的状态读取包括完整注意力层的 KV、线性注意力层的递推状态与卷积状态。

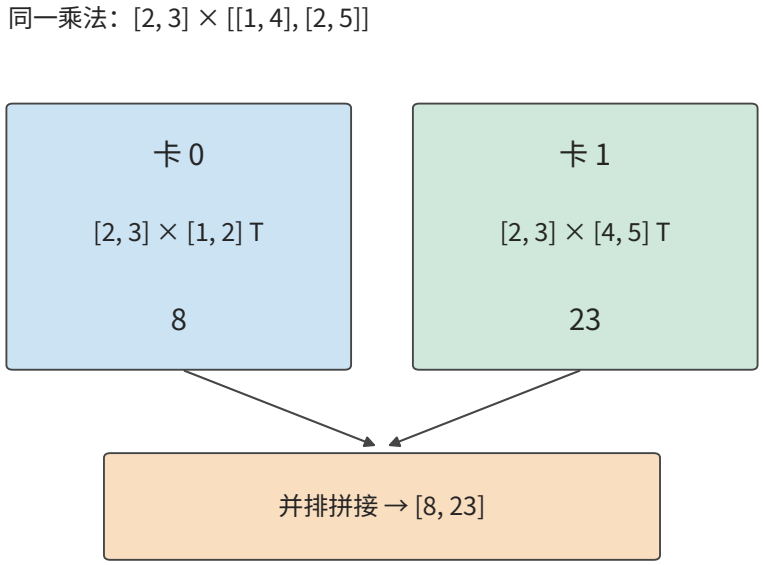



图 6-7：按权重矩阵的列切分时，每张卡使用完整输入，计算不同的输出元素。格内的转置符号 T 表示把方括号中横排的数当作列向量。

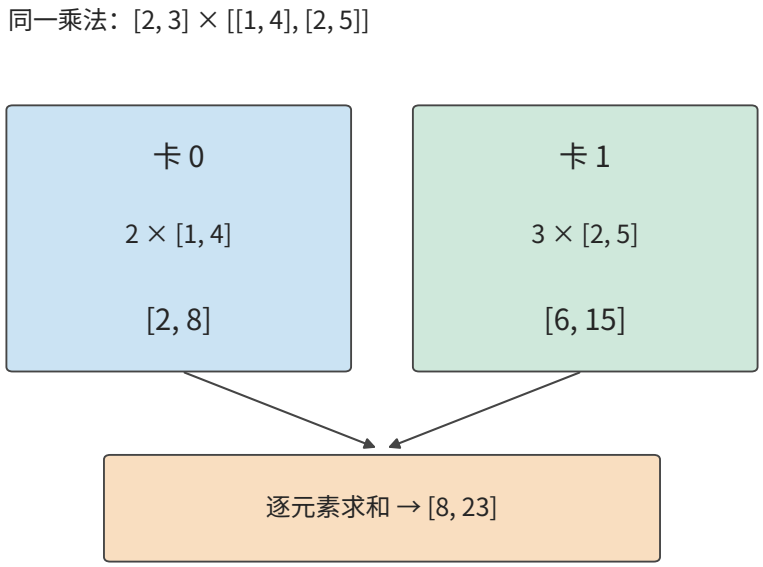



图 6-8：按权重矩阵的行切分时，两卡产生形状相同的输出部分和。对应位置相加，恢复完整乘法的结果。

把中间宽度 $f$ 分成左右两半。卡 0 保存两个上投影矩阵左半部分的列与下投影矩阵上半部分的行，卡 1 则保存两个上投影矩阵右半部分的列与下投影矩阵下半部分的行。两张卡都保存完整输入 $X$，分别得到中间激活 $Z_0, Z_1$。SiLU 和乘法逐元素执行，各卡可以直接处理自己的半段。

下投影沿同一个中间维相接：

$$Y = [Z_0 \ Z_1] \begin{bmatrix} W_{d,0} \\ W_{d,1} \end{bmatrix} = Z_0 W_{d,0} + Z_1 W_{d,1}. \tag{6-3}$$

右侧两个乘积的形状都是 $m \times h$，各自只包含中间维一半分量的贡献，相加后才是下一层要用的输入。

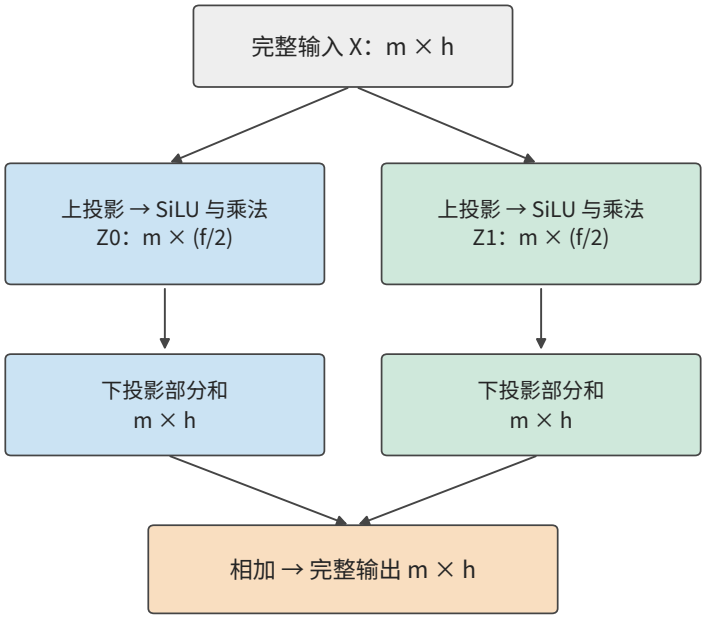



图 6-9：SwiGLU 的两卡切分。上投影按权重矩阵的列划分输出特征，逐元素运算留在本地；下投影按权重矩阵的行划分输入特征。两步均保留全部 $m$ 个 token，切分的是每个 token 的特征维度。每卡中间激活为 $m \times (f/2)$，输出部分和仍为 $m \times h$，最后逐元素求和。

若两张卡都需要完整结果，就执行 AllReduce。若下一个算子可以继续使用分片，则执行 **ReduceScatter（归约后分散）**：先求和，再把求和结果的不同片段留在不同的卡上。要从分片恢复完整数据，则执行 **AllGather（全收集）**，把各卡持有的片段拼起来。

**例题：四卡切分减少多少权重读取，又增加多少通信？**取 Qwen3-32B 的 $h = 5120, f = 25600$，将中间维均分到四卡。

解答：每卡上投影为 $5120 \times 6400$，下投影为 $6400 \times 5120$。三个矩阵共占 187.5 MiB，是原来 750 MiB 的四分之一。按 3350 GB/s，单 token 的读取时间从约 234.8 µs 降到 58.7 µs，每卡少读 562.5 MiB，节省约 176.1 µs。代价是要把四张卡各自的 10 KiB 输出部分和相加。第 6.4 节将用具体算法求出这次汇合的时间。

注意力也可以按头切分：Q、K、V 投影只产生本卡负责的头，各卡独立计算这些头的注意力；输出投影把各头的结果映射回隐藏维，各卡得到的仍是部分和，相加后才是完整输出。因此 Qwen3-32B 每层要对注意力输出和 FFN 输出各做一次归约，64 层共 128 次。词嵌入按词表切分，词表输出投影也按词表分到各卡。

头不能再拆，是分配的最小单位。Qwen3-32B 有 8 个 KV 头，TP8 时每卡一个；TP16 时，共用同一个 KV 头的查询头被分到两张卡，两张卡都要保存该 KV 头的状态。上下文为 128K 时，每个 KV 头的状态占 4 GiB，八卡合计 32 GiB，十六卡合计 64 GiB。Qwen3-235B-A22B 只有 4 个 KV 头，第 6.1.2 节的八卡分法已经让每个 KV 头在两张卡上各存一份。TP 卡数一旦超过 KV 头数，继续增加就会复制 KV。

本章用“实例数”表示独立推理服务的数量，用 $p$ 表示单个实例的 TP 卡数。共有八张卡时实例数为 $8/p$。后文先求单个实例生成一个 token 的时间 $T(p)$，再换算成一组请求的完成时间。

**从单层计算推导完整 decode 步的耗时。**以 Qwen3-32B 的一个 128K 会话为例。各投影权重每步从 HBM 读一次，旧 KV 读一次，并写入一个新 token。注意力投影的 BF16 权重为

$$2\,(2\times 5120\times 8192+2\times 5120\times 1024)=180\ \text{MiB}.$$

其中两个 $5120\times 8192$ 对应 Q 与输出投影，两个 $5120\times 1024$ 对应 K 与 V。每层再加 750 MiB FFN 权重，共 930 MiB。上下文有 131,064 个 token 时，每层读取的 KV 约为 512 MiB。词表有 151936 项，输出投影权重约为 1.56 GB。因此全步 HBM 访问量可写成

$$V(s)=64\times 930\ \text{MiB}+256\ \text{KiB}\times(s+1)+151936\times 5120\times 2\ \text{bytes},\tag{6-4}$$

$s$ 为这一步开始前的上下文长度。代入 131,064，约为 98.32 GB，其中权重 63.97 GB，KV 34.36 GB。按 H100 的 3350 GB/s 读完约需 29.35 ms；同一步的矩阵运算约 0.34 TFLOPs，按 989.4 TFLOP/s 只需约 0.34 ms，所以每层注意力和 FFN 的执行时间都受内存带宽限制。采用 $p$ 路 TP（$p\le 8$）时，权重和 KV 都均分到各卡，这部分时间为 $29.35/p$ ms。归一化、逐元素运算和采样只读写几个隐藏向量，这里不计。[5]计算与归约依次执行，就得到后文各节使用的执行时间模型：

$$T(p)=\frac{T_{\text{local}}}{p}+128\,T_{\text{AR}}(p),\qquad T_{\text{local}}\approx 29.35\ \text{ms}.\tag{6-5}$$

其中 $T_{\text{AR}}(p)$ 是 $p$ 张卡做一次 AllReduce 的时间。式 (6-5) 把增加 TP 卡数的影响分成两项：本地执行时间随卡数增加而下降，通信时间则取决于算法和连接方式。将卡数从 $p$ 翻倍到 $2p$，节省的本地时间为 $T_{\text{local}}/(2p)$。因此，增加卡数能够缩短执行时间的条件是

$$128\,[T_{\text{AR}}(2p)-T_{\text{AR}}(p)]<\frac{T_{\text{local}}}{2p}.\tag{6-6}$$

$p$ 越大，右侧越小。即使每次归约只增加同样一小段时间，累积起来也会抵消本地执行时间的缩短。

卡更多时，线性层还可以同时切分两个矩阵维度，把卡排成行、列两个方向的通信组。以 $[8192,4096]\times[4096,12288]$ 的投影为例，把输出分成 16 片放在 16 张卡上，每张卡起初只有 4 MiB 的输入分片和 6 MiB 的权重分片。排成 4×4 网格时，每张卡要横向收集同一行另外三张卡的输入分片，发送 12 MiB；纵向收集同一列另外三张卡的权重分片，发送 18 MiB。改成 2×8 网格，两个方向分别为 28 MiB 和 6 MiB；改成 8×2 则为 4 MiB 和 42 MiB。两个方向的链路独立且等速时，传输时间由较大的一方决定，因此 4×4 最快。MeshSlice 沿这种布局继续分块，让收集与矩阵乘流水执行；推理系统 Arctic 则在不同执行阶段切换分工，其状态迁移在第 9 章展开。[6]

### 6.2.3 序列并行：逐 token 算子沿序列分片

张量并行让每张卡只保存矩阵的一部分，但层内还有一些算子不做矩阵乘，例如 LayerNorm（层归一化）、Dropout 和残差相加。这些算子逐 token 工作：每个 token 的结果只依赖自身的隐藏向量。**序列并行**（sequence parallelism，SP）让这些算子沿序列位置分片：TP 组内的每张卡只处理一段位置，不再各自保存一份完整激活。

[5]贯穿模型与逐步计算见模型说明、计算脚本和完整结果。Qwen3-32B 形状取自已锁定配置，H100 的 HBM 容量、带宽与矩阵峰值取自硬件表。矩阵算子的时间取内存访问与计算时间的较大值，均按峰值计；通信按层串行，上下文 KV 每步增长。

[6]MeshSlice 的矩阵形状与流水推算、Shift Parallelism 的状态与额外驻留。MeshSlice 的部分重叠为预测，Arctic 的 SP 定义及动态切换边界按固定实现说明。

本书采用大模型训练框架 Megatron 的这一常见定义。[7]

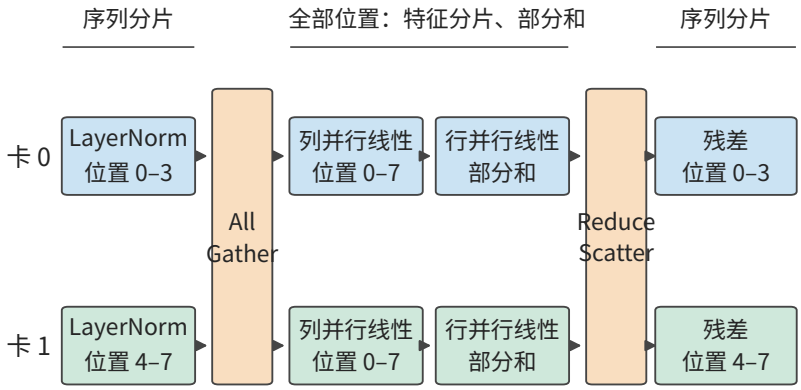



图 6-10：LayerNorm 和残差只用到本 token 的数据，两张卡各处理一半位置；进入列并行线性层前用 AllGather 收集全部位置，行并行线性层的部分和用 ReduceScatter 求和并重新按位置分片。

列并行线性层需要完整的输入行，进入它之前先用 AllGather 收齐各卡的序列分片；行并行线性层产生部分和之后，用 ReduceScatter 一边求和、一边重新按位置分片。AllReduce 本来就等于 ReduceScatter 加 AllGather，序列并行只是把这两半放到了不同的算子边界上，中间的逐 token 算子就不必每张卡都保存一份完整激活。

例如 $[8192, 4096]$ 的 BF16 激活为 64 MiB，四卡各保留连续的 $[2048, 4096]$ 分片时，每卡只占 16 MiB，总计 64 MiB；四卡各存完整张量则总计 256 MiB。这一节省只发生在保持序列分片的区间，不是模型的所有激活与权重都缩小四倍。归约维和输出维的关系仍与图 6-9 相同，变化的只是下一个算子接收哪种布局。配套的实验在 JAX（支持自动求导和编译执行的数值计算框架）中展示了保持分片和重新收集两条 FFN 路径。[8]

### 6.2.4 上下文并行：切分同一序列的注意力

序列并行只处理逐 token 的算子，注意力却要让每个位置看到序列中的其他位置。**上下文并行**（context parallelism，CP）把同一个长序列的位置分给多张卡，每张卡保存自己那段位置的 Q、K、V，并计算这段位置的注意力输出。图 6-11 用八个位置说明两张卡的分工。

本卡的查询仍要用到掩码允许范围内的远端 K、V。可以一次收齐全部 K、V，也可以让 K、V 块在各卡之间环形传递，边到达边用第 5.3.3 节的在线 softmax 累计最大值、指数和与加权输出。环形注意力（Ring Attention）是一种调度和传输的实现方式，上下文并行是工作划分方式，两者不是同义词。[9]

图 6-12 把这种依赖画成八个位置之间的可见关系。

按位置连续切半还会造成计算不均。八个位置的因果注意力共有 $1 + \cdots + 8 = 36$ 个查询—键配对；前四个位置只有 10 对，后四个有 26 对，两张卡的 token 数相同，工作量却相差 2.6 倍。把靠前和靠后的位置搭配起来，例如卡 0 持有位置 $1, 2, 7, 8$，卡 1 持有 $3, 4, 5, 6$，两张卡就各承担 18 对。实际实现常用分块交错的“之”字形布局，但必须保留原始位置和因果掩码，并重新安排通信；远端上下文不能当作 padding 丢弃。

[7] Korthikanti 等，Reducing Activation Recomputation in Large Transformer Models，§3 的 tensor 与 sequence parallelism；Megatron Core 的 Context Parallelism 文档；Liu 等，Ring Attention with Blockwise Transformers for Near-Infinite Context。本节 SP 采用 Megatron 的特定含义，CP 的八位置例子为本书推导。

[8] 八个正式配置通过 FP64 参考检查，配对 TP/SP 输出逐位相同；完整形状与验收见实验 6-2：JAX TP/SP 实际 FFN 路径。实验在 CPU 逻辑设备上执行。

[9] Korthikanti 等，Reducing Activation Recomputation in Large Transformer Models，§3 的 tensor 与 sequence parallelism；Megatron Core 的 Context Parallelism 文档；Liu 等，Ring Attention with Blockwise Transformers for Near-Infinite Context。本节 SP 采用 Megatron 的特定含义，CP 的八位置例子为本书推导。

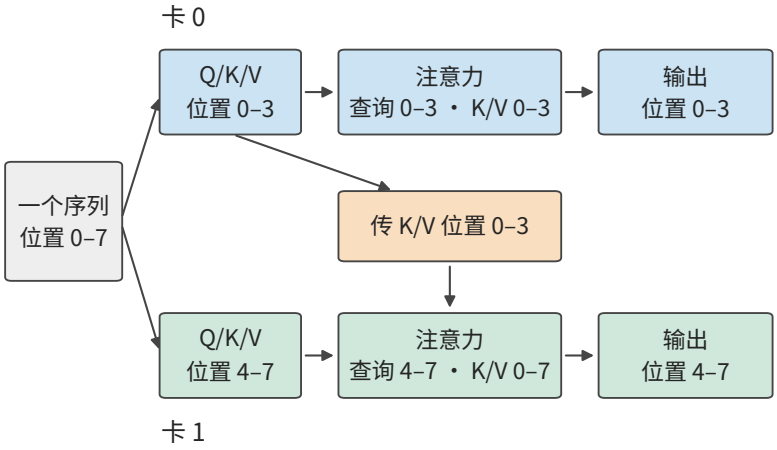



图 6-11：每张卡保存自己那段位置的 Q、K、V。因果注意力下，卡 1 的查询还要用到卡 0 的 K、V，所以 K、V 必须跨卡传递；非因果注意力时两个方向都要传。

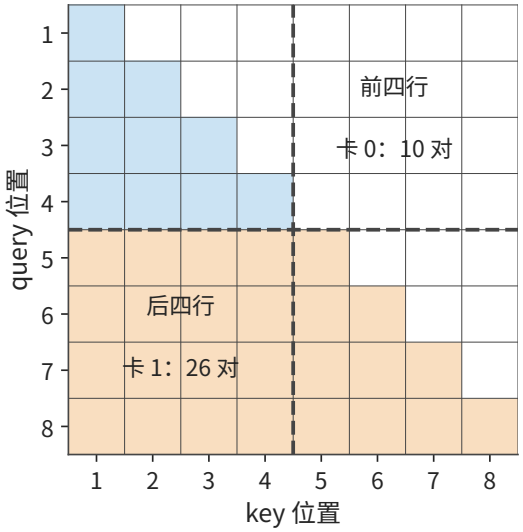



图 6-12：八个 token 的因果注意力，横轴为键的位置、纵轴为查询的位置。实色格是必须计算的查询—键配对；前四行归卡 0，后四行归卡 1。虚线只是设备边界，左下区域的远端依赖并不因此消失。

Decode 时新 token 的查询很少，历史 K、V 却很长。沿历史位置切分后，每张卡先算出本段的最大值 $m_r$、指数和 $l_r$ 与未归一化的加权向量 $u_r$，再合并为

$$m = \max_r m_r, \qquad l = \sum_r e^{m_r - m} l_r, \qquad u = \sum_r e^{m_r - m} u_r, \qquad o = u/l.$$

$o$ 即新 token 在全部历史位置上的注意力输出。这与第 5 章 tile 之间合并 softmax 的做法相同，只是统计量和向量要跨卡传输。不能各卡先分别做完 softmax，再简单平均各卡的输出。上下文并行减少了每张卡保存的上下文状态和本地的注意力计算，代价是 K、V 的交换或统计量的归约；生成时前后 token 之间的依赖并不因此消失。序列并行和上下文并行在不同框架里的名称有交叠，判断一种配置时，应看切了哪些算子、哪些张量、用了哪个通信组，而不只看缩写。

### 6.2.5 流水线并行：按层划分阶段

上述几种方式都在一层内部分工。**流水线并行**（pipeline parallelism，PP）改为按层分工：把模型的层按顺序分成若干阶段，每个阶段由一张或一组卡负责。图 6-13 把 Qwen3-32B 的 64 层分成两个阶段。

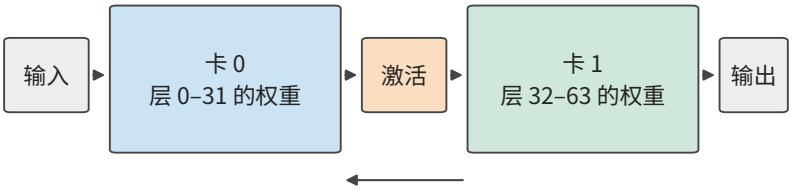



图 6-13：两个阶段各保存自己那些层的权重。相邻阶段之间只交接激活；训练时梯度沿同一边界反向传递。

前一个阶段算完自己负责的层，把激活传给下一个阶段接着算。每个阶段只保存自己那些层的权重，阶段之间只传递中间结果。

送入流水线的输入按 micro-batch 组织：一个 batch 拆成多个 micro-batch，不同阶段就能同时处理不同的 micro-batch。设模型分成 $q$ 个阶段，每个阶段的执行时间（含传递激活）为 $t$，输入是 $b$ 个已经就绪、互不依赖的 micro-batch。考虑四个阶段、每阶段 1 ms 的情形。micro-batch 0 在 0 ms 进入阶段 0，经过四个阶段，在 4 ms 完成；阶段 0 在 1 ms 就能接收 micro-batch 1，后者在 5 ms 完成。其余 micro-batch 依次跟进，每隔 1 ms 完成一个。

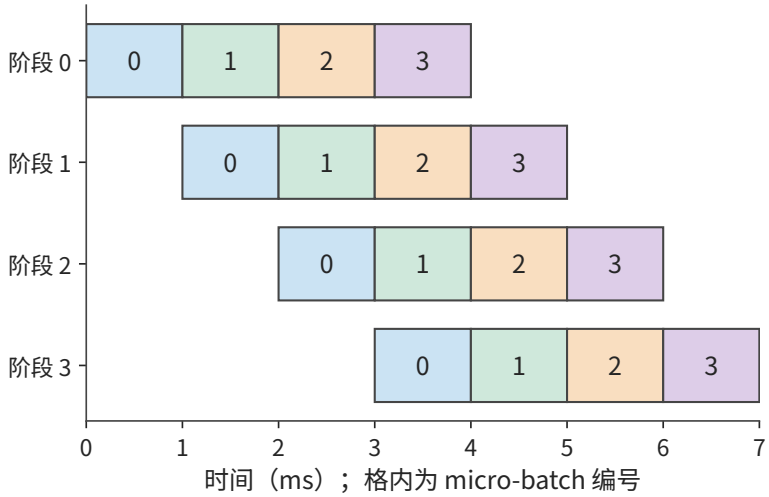



图 6-14：四个等时阶段处理四个独立 micro-batch。每格为 1 ms，同色表示同一 micro-batch。第一项结果在 4 ms 产生，最后一项在 7 ms 产生；左上到右下的空白来自流水填充与排空。

第一个 micro-batch 花费 $qt$，其后每增加一个 micro-batch 只增加 $t$，所以

$$T_{\mathrm{PP}} = (q+b-1)t, \qquad \eta_{\mathrm{PP}} = \frac{qb\,t}{q(q+b-1)t} = \frac{b}{q+b-1}. \tag{6-7}$$

分子是所有阶段实际工作的时间总和，分母是阶段数乘以处理全部 micro-batch 所需的时间。四阶段、四个 micro-batch 的利用率为 4/7，约 57%；增加到 16 个 micro-batch 时为 16/19，约 84%。独立 micro-batch 越多，流水填充与排空造成的空闲时间占比就越小。

各阶段执行时间不均也会让卡空闲。若四个阶段分别需要 1、1、2、1 ms，第一个 micro-batch 在 5 ms 完成，此后最快每 2 ms 完成一个。送到耗时 2 ms 阶段的输入到达速度超过其处理速度，队列逐渐积压；缓冲用满后，上游阶段只能暂停。按各层的实际执行时间划分阶段，可以让各阶段的耗时更接近，减少积压。[10]

自回归会话的下一步输入，要等当前 token 生成后才就绪。四个独立会话可以提供四个 micro-batch，同一会话未来的四个 token 则是一条先后依赖链。流水利用率因此直接取决于同时有多少 micro-batch 可以开始执行。

### 6.2.6 专家并行：切分专家集合

最后一种方式切分的是 MoE 特有的专家集合。稠密模型的每个 token 使用同一组 FFN 权重；MoE 模型准备多个前馈网络作为专家，由路由器计算选择分数，为每个 token 选出要执行的专家。每个 token 只用到被选中的专家，但整个专家集合都要保存在加速器上，供后续 token 选择。MoE 由此增加了一种分工方式：**专家并行**（expert parallelism，EP）把不同专家放到不同的卡上，由各卡处理发给自己的输入。图 6-15 把八个专家分给两张卡。

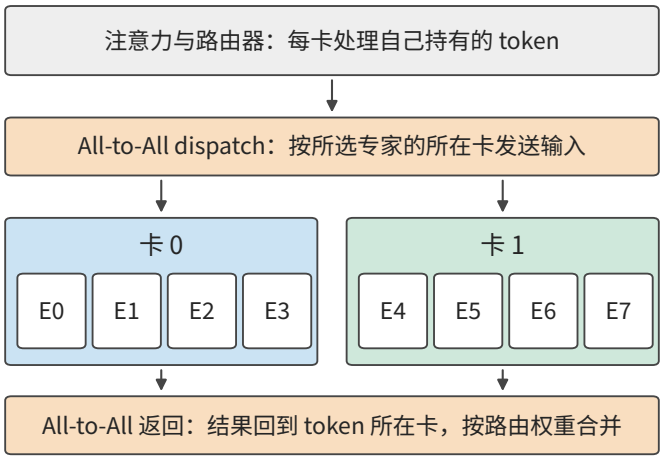



图 6-15：注意力和路由器在 token 所在的卡完成；输入按所选专家的所在卡发送，专家算完后结果送回原卡，按路由权重合并。两次交换都是 All-to-All（全交换），即每张卡向其他各卡分别发送不同的数据。

专家并行首先要解决专家权重放在哪里，然后才是当前 token 如何执行。

先用四张卡、八个专家说明这种分工。卡 0 保存专家 0 和 1，卡 1 保存专家 2 和 3，依次类推。卡 0 当前持有 token A，路由器为 A 选中专家 1 和 6：专家 1 就在本卡，专家 6 在卡 3。卡 0 既要执行本卡的专家 1，也要把 A 的输入送到卡 3，最后收回两份输出，按路由权重合并。

把专家输入送往专家所在卡的过程称为 **dispatch（派发）**，把输出送回原卡并合并的过程称为 **combine（合并）**。继续看 token A：若两个专家的输出分别为 $y_1, y_6$，路由权重为 $a_1, a_6$，完整的 MoE 输出为 $a_1 y_1 + a_6 y_6$。发送方的卡保存 token 的编号和路由信息，接收方的卡执行专家，返回的结果再归到同一个 token 名下。

[10] 有限槽位与背压计算、基础 TP / PP 通信路径。

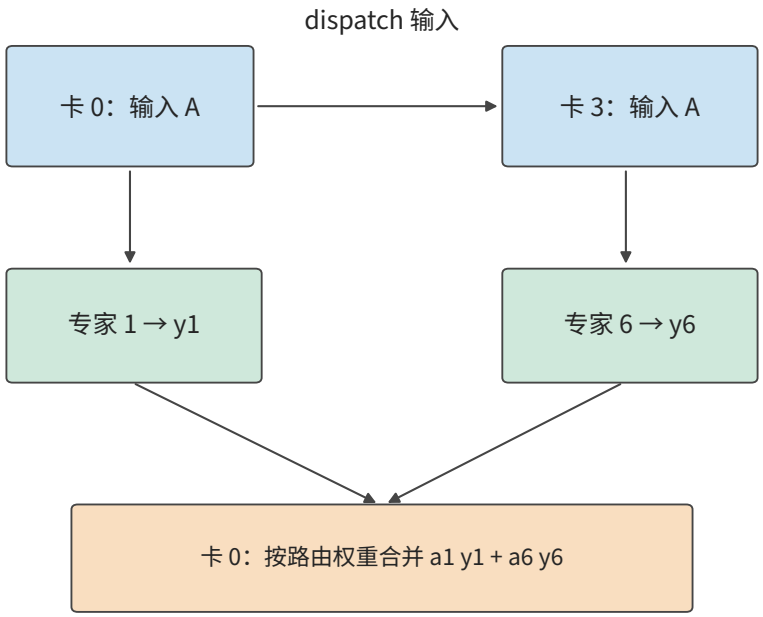



图 6-16：卡 0 持有 token A，选择本地专家 1 与卡 3 上的专家 6。dispatch 沿上方路径发送输入，combine 沿下方送回专家输出并加权合并。每个 token 选中几个专家，就要算几次；专家在哪张卡，决定输入要送到哪里。

多个专家在同一张卡上时，可以只发一份输入。若一个 token 选中的八个专家两两分布在四张远端卡上，4096 维的 BF16 输入为 8 KiB：按专家逐个发送要 64 KiB，按目的卡发送只要 32 KiB，同一张卡上的两个专家共用这份输入。

多个 token 一起处理时，每张卡要向不同的卡发送数量不等的数据，这就是图 6-15 中的 All-to-All。接收方先按专家把收到的输入行重新排列，做矩阵乘，再按 token 编号把输出排回原序。收到输入最多的卡，通常也要做最多的专家计算。

Qwen3-235B-A22B 将这一例子扩大为每层 128 个路由专家，每 token 选 8 个，共 94 层。每个专家包含三次投影，隐藏维 4096，中间维 1536，BF16 权重为 $3 \times 4096 \times 1536 \times 2 = 36$ MiB。单层全部专家占 4.5 GiB，94 层约占 423 GiB。把专家均分到八卡，每卡约为 52.9 GiB，此外每张卡还要保存注意力层、路由器和输入输出层的权重、KV 和工作区。专家集合可以分散到各卡，注意力的头和状态则按注意力自己的分工保存。

**实验 6-2 ・核心：张量切分与 micro-batch 流水如何改变执行时间**

(a) 写出两卡 SwiGLU 的六个权重分片和两个输出部分和的形状，用式 (6-3) 说明为什么需要将两张卡的输出部分和相加。

(b) 对 TP1、TP2、TP4、TP8，计算 Qwen3-32B 一个 128K 会话的本地内存访问时间，暂不计通信开销；求卡数每次翻倍所节省的时间。

(c) 采用第 6.4 节的归约时间，将其代入式 (6-5)，重新计算加速比，解释差距从哪里产生。

(d) 画出四阶段处理八个 micro-batch 的时序，分别考虑四个阶段均耗时 1 ms，以及耗时依次为 1、1、2、1 ms 的情况；比较两种情况下的完成时间，并指出 micro-batch 会在哪个阶段之前积压。

(e) 进阶：对同一个 FFN，比较输出完整张量与保留输出分片两种方式下，相邻算子之间需要传递的数据；也可选择配套二维网格方案，进一步推导通信数据的分块方式。

## 6.3 组合并行与模型规模

六种方式很少单独使用。本节先说明如何组合这些方式并给每张卡编号，再讨论 MoE 特有的两个问题：一批 token 的专家选择如何改变权重读取和各卡负载，以及模型继续增大时组合如何变化。

### 6.3.1 组合方案与设备坐标

张量并行和流水线并行可以组合：例如把八张卡分成四个流水阶段，每个阶段内两张卡做张量并行。每层的两卡归约在阶段内完成，相邻阶段之间传递激活。若阶段内两张卡各自持有完整的 8 KiB 隐藏向量，并分别交给下一阶段对应的卡，两个阶段之间总共要发送 16 KiB；若阶段之间传递的是分片，则由下一阶段按计算需要重新拼合。因此组合两者时，要写清每个阶段输出的是哪些分片，下一阶段每张卡接收哪些分片。

组合多种方式时，先给每张卡标出坐标，再对每个算子写出其输入、权重和输出分片，以及需要通信的卡组。例如数据并行 2、流水线并行 2、张量并行 4 的方案记作 DP2×PP2×TP4，占 $2 \times 2 \times 4 = 16$ 张卡；若在同一 TP4 组内启用序列并行，卡数不变，不再乘一个 SP4。切分确定每份工作归谁，调度确定就绪、执行、通信与回收的顺序，两者合起来才是完整的并行方案。

专家并行也可以和专家内部的张量并行组合。例如四个专家组各拥有 32 个专家，每组再用两张卡切分专家的中间维，每张卡处理 768 维。整个实例仍是八张卡，每张卡用两个编号标识：专家组编号说明该卡负责哪些专家，组内张量并行编号说明负责这些专家矩阵的哪一半。后文沿这一 TP2×EP4 布局推导通信。

专家输入的来源如下：让四个专家组都对同一个请求执行注意力，各自保存该请求的 KV；注意力算完后，每个组都持有完整的专家输入，直接选出本组专家需要的行即可，不必跨组 dispatch 输入；各组的结果最后再归约汇合。

| 专家组 | 卡 | 专家编号 | 每卡专家中间维 | 每卡 KV 头 |
|---|---|---|---|---|
| 0 | 0、1 | 0—31 | 768 | 偶数卡 0、1；奇数卡 2、3 |
| 1 | 2、3 | 32—63 | 768 | 同上 |
| 2 | 4、5 | 64—95 | 768 | 同上 |
| 3 | 6、7 | 96—127 | 768 | 同上 |

数据位置确定后，再跟踪输出如何汇合。组内两张卡各算专家中间维的一半，先在组内相加，得到本组专家的完整贡献；再把四个组的贡献相加。

第二步的求和发生在编号相同的卡之间。例如四个组对某个输出元素的贡献分别为 1、2、3、4，四张偶数卡一组归约得到 10，四张奇数卡另一组也得到 10。此时每张卡都持有相同的完整输出，可以继续算下一层。

省去 dispatch 要付出存储与计算的代价。按这份布局，每张卡的权重约 64.8 GB，八卡共 518.6 GB，比每个参数只保存一份的 470.2 GB 多约 48.4 GB。上下文 8192 个 token 时每卡 KV 为 752 MiB，八卡共 5.875 GiB，是该请求实际 KV 的四倍。每个组都重复执行一遍注意力，换来的是专家输入就在本组。[11]

**例题：注意力复制与专家切分的布局需要多少归约通信？**归约采用 FP32，单个 token 的完整隐藏向量为 $4096 \times 4 = 16$ KiB。用第 6.4.2 节的环形算法在两张卡之间归约，每张卡发送一个完整向量的大小；四张卡归约则发送 1.5 倍。每层有注意力的组内归约、专家的组内归约、四个专家组之间的归约三次汇合，所以每张卡发送

[11] Qwen235 所有权连接练习及独立核对结果。本例采用正文给定的 TP2×EP4 布局。

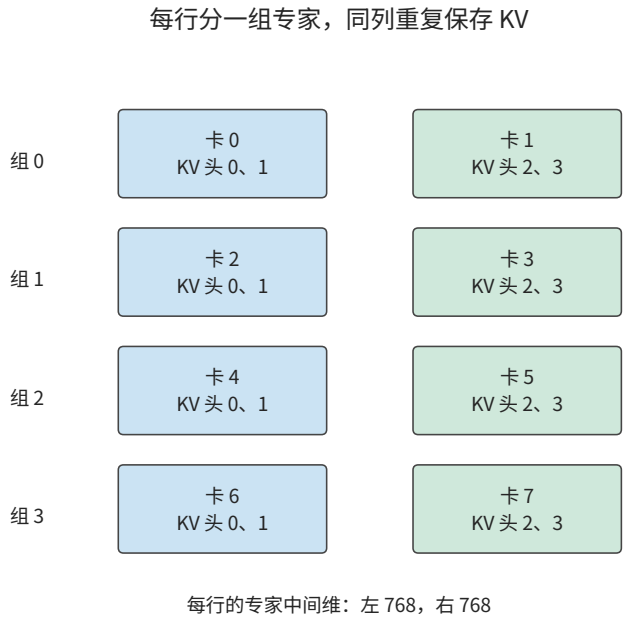



图 6-17：八张卡排成四个专家分组，每组两卡沿专家中间维分工。同一列的注意力头与 KV 重复四份；每组因此都能在本地得到相同的专家输入。

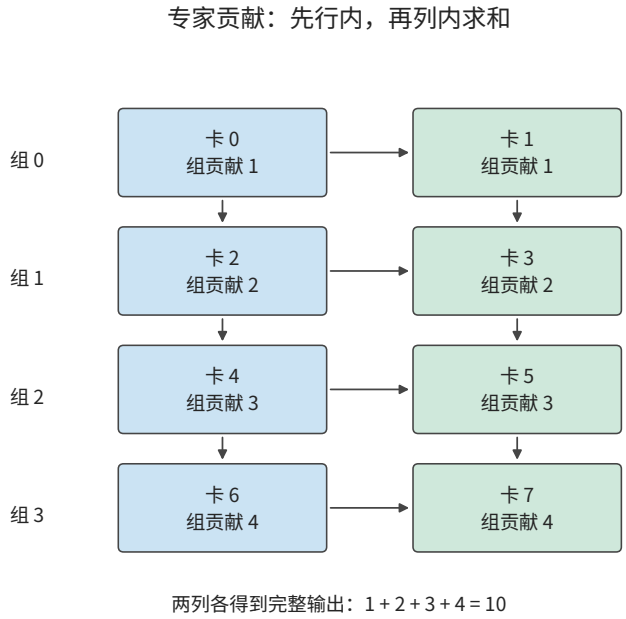



图 6-18：横向箭头表示组内两卡求和，纵向箭头表示四个组求和。格内数字是某个输出元素经组内求和后的值，两列各自都得到相同的完整结果 10。

$$16 + 16 + 24 = 56 \text{ KiB}.$$

八卡合计 448 KiB；8192 个 token 的 prefill 采用同样的分工，每张卡发送 448 MiB。

### 6.3.2 批内复用与负载不均

上一小节确定了专家放在哪些卡上，还没有确定每一步哪些卡最忙，这取决于当前 batch 选中了哪些专家。一个 token 只选八个专家，一个 batch 却可能覆盖整个专家集合。设 64 个 token 各选八个专家，共 512 次 token 到专家的分派。

若这些分派恰好均匀覆盖 128 个专家，每个专家处理四个 token 的特征向量；若所有 token 选择同样的八个专家，每个专家处理 64 个 token 的特征向量。两种分派的有效矩阵运算量相同。假定在一个 batch 内，每个被选中专家的权重只从 HBM 读取一次，前者需要 $128 \times 36$ MiB，即 4.5 GiB；后者需要 $8 \times 36$ MiB，即 288 MiB，读取量相差 16 倍。

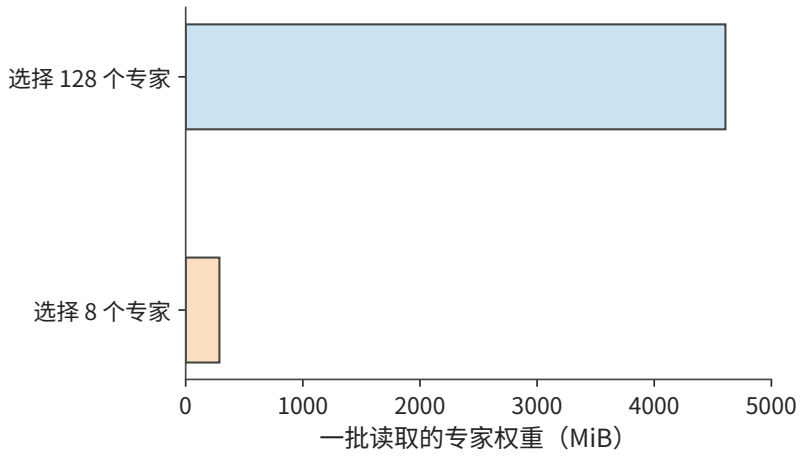



图 6-19：64 个 token、每 token 八个专家，共 512 次分派。均匀覆盖与集中选择的有效计算量相同，被选中专家的权重读取相差 16 倍。每个专家 BF16 权重为 36 MiB，批内读取一次。

活跃专家越少，每个专家的权重就能供更多输入行复用，既减少重复读取，也让矩阵乘的行数变大。但是，这些行由哪张卡执行，同样影响完成时间。在 TP2×EP4 布局中，均匀覆盖时每组承担 $32 \times 4 = 128$ 次分派；若八个活跃专家全都在一个专家组，该组要承担全部 512 次，计算量是均匀时的四倍，其余三组只能等待。

因此，分析执行时间需要同时计算整批的权重读取量，以及负载最重的专家组的计算量。Decode 的小矩阵运算容易受权重读取速度限制，prefill 的大矩阵运算则更容易受负载最重的专家组的计算时间限制。把热点专家分散到不同的 EP 组，既保留了批内复用，又减轻了单个专家组的计算负担。

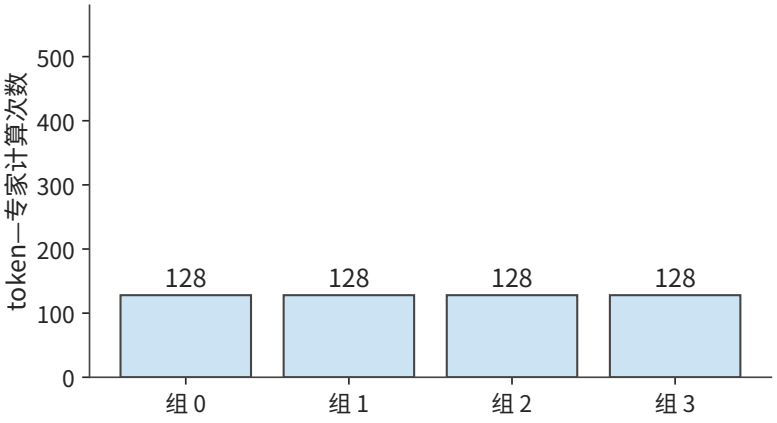



图 6-20：均匀使用 128 个专家时，四个 EP 组各处理 128 次 token—专家计算，合计读取 4.5 GiB 权重。

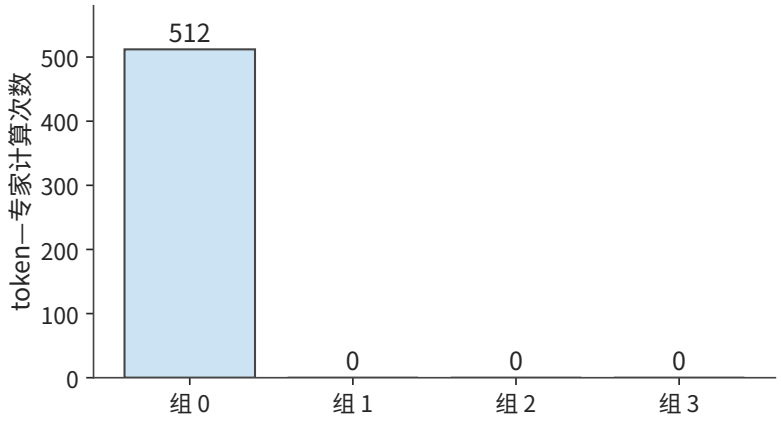



图 6-21：所选八个专家都在组 0：权重读取降到 288 MiB，512 次计算却全部压在同一组。

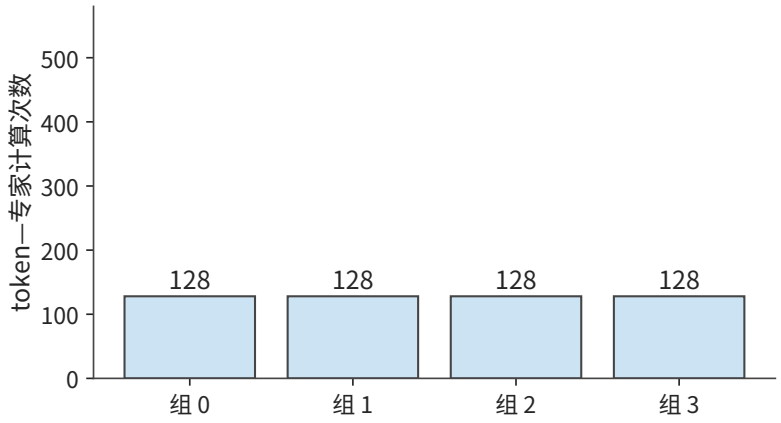



图 6-22：把同样的八个专家分散到四组，将读取量维持在 288 MiB，同时让四组各承担 128 次计算。三图纵轴范围相同。

图中后两种布局读取同样多的权重，完成时间却可能不同，因为参与计算的卡数不同。因此，批内复用应与专家在各卡上的分布一起分析。

除上述均匀覆盖与集中选择两种情况外，还可以计算均匀随机选择专家时的平均结果。设每 token 从 $E$ 个专家中选 $k$ 个，各 token 独立。一个专家被单个 token 跳过的概率为 $1-k/E$，被整批 $m$ 个 token 跳过的概率为 $(1-k/E)^m$。对每个专家定义一个指示变量：该专家在批内被选中时取 1，否则取 0。将这些变量相加，再取期望，可得批内被选中专家数的期望：

$$\mathbb{E}[E_{\text{active}}] = E\left[1-\left(1-\frac{k}{E}\right)^{m}\right]. \tag{6-8}$$

取 $E=128, k=8, m=8$，期望约有 52 个活跃专家，理想读取约 1.8 GiB；八个 token 全部选择同样八个专家时仍只读 288 MiB。随着 batch 增大，被选中的专家数逐渐接近专家总数，继续增加输入行主要提高每份权重的复用。[12]

真实的路由结果由输入和模型共同决定。配套资料中的 DeepSeek V4-Flash 路由记录覆盖四个输入、43 层，

[12] 专家复用与 MoE Serving Tax 阅读。式（6-8）假设各 token 独立、均匀地选择专家；原论文分别以激活 FLOPs 和总参数匹配稠密对照模型。

共约 131 万次专家选择，各专家被选中的频率各不相同。把每个专家收到的 token 数换算成矩阵行数，再按每张卡上的专家集合相加，就能从这份记录得到各卡的计算量分布。逐层热图见配套资料。[13]

### 6.3.3 从数千亿到万亿参数：并行组合如何变化

第 6.3.2 节的复用与负载分析，建立在整个专家集合已经能够存入加速器的前提下。模型继续增大时，首先需要重新检查总存储容量，再决定 TP、PP 和 EP 如何组合。Qwen3-235B-A22B 的全部 BF16 权重约为 470 GB，这里每个参数只计一次；DeepSeek V4-Flash 约 284B 参数，按同样格式约为 568 GB。一台 HGX H100 的八张 H100 共有 640 GB，留给状态和工作区的空间已经不多。DeepSeek V4-Flash 每层有 256 个路由专家，每个 token 选择其中 6 个，另外还执行共享专家，专家集合更大，分配权重时需要同时考虑共享专家和路由专家。[14]

DeepSeek V4-Pro 约 1.6T 参数，即使按每个参数半字节计算，权重也约为 800 GB；Kimi K3 的约 2.8T 参数对应约 1.4 TB。这些模型的存储需求已经超过一台 HGX H100 的总容量。增加 EP 分组数，可以把专家分到更多的卡上；增加专家内部的 TP 卡数，可以减小每卡保存的专家矩阵；增加 PP 阶段数，则可以减少每个阶段保存的模型层数。

专家输入如何表示，也会改变跨卡通信量。Kimi K3 的路由专家在 3584 维的潜空间里工作，主干隐藏向量则是 7168 维。先投影再发送，BF16 向量从 14 KiB 降到 7 KiB；先发送再投影，传的就是未降维的输入。同样的专家计算，投影放在发送前还是接收后，决定了跨卡传输的向量维度。模型每层从 896 个路由专家中选择 16 个，另有两个处理完整主干向量的共享专家，分别承担不同的计算；93 层中，69 个 KDA 层用固定大小的矩阵递推汇总上下文，24 个 MLA 层用低维表示保存上下文。KDA 状态的存储需求主要由其固定维度决定，MLA 状态的存储需求还随会话上下文增长。[15]

模型的变化因此从两个方面影响系统设计：权重增多会扩大所需的总存储容量，计算图不同会改变需要交换的数据。前者确定至少要动用多少资源，后者决定这些资源应如何连接。

> **实验 6-3 ·核心：专家分派与放置如何改变各卡负载和通信量**
>
> (a) 沿图 6-16 写出 token A 的输入发送、两次专家执行和加权合并顺序。
>
> (b) 复算 64 个 token 的两种专家分派，求权重读取量与负载最重的 EP 组的任务数。
>
> (c) 把八个热点专家均分到四个 EP 组，比较总读取量与负载最重的专家组的计算量，解释变化来自哪里。
>
> (d) 采用本节 TP2×EP4 表，求上下文长度为 8192 与 16384 时，每张卡需要保存的 KV 容量，并推导三次结果汇合的发送量。
>
> (e) 进阶：对 DeepSeek V4-Flash、DeepSeek V4-Pro 或 Kimi K3，选择一个具体存储格式，逐项放置路由专家、共享专家、注意力状态与工作区。

## 6.4 集合通信的实现与执行代价

### 6.4.1 根据模型切分方式确定通信需求

第 6.2 节定义了 AllReduce、All-to-All 等集合通信操作，本节讨论其实现与执行代价。先考虑四张卡如何汇总一个四元素向量。每张卡都有四个元素的本地贡献，目标是让每张卡得到完整的逐元素和。可以分两步完

[13] 图形与样本说明见路由热图和配套图说明。记录覆盖 43 层，每题四个实际 decode 输入，包含调度器提前执行的 EOS forward。来源：DeepSeek V4-Flash 路由观测及原始计数分析。

[14] 具体模型的并行推算，Qwen3-235B-A22B 配置，DeepSeek V4 报告，Kimi K3 报告表 1 与 §2.3。DeepSeek-V3 的 671B 为历史对照模型。

[15] 具体模型的并行推算，Qwen3-235B-A22B 配置，DeepSeek V4 报告，Kimi K3 报告表 1 与 §2.3。DeepSeek-V3 的 671B 为历史对照模型。

成：先求和，并让卡 0 留元素 0 的完整和、卡 1 留元素 1 的完整和，依次类推；再交换这些已求和的元素，让四张卡都取得完整向量。

第一步就是 ReduceScatter，第二步就是 AllGather，两步合起来完成一次 AllReduce。两步分别改变数值和数值所在的位置。稠密模型和 MoE 还需要拼接分片、把专家输入发给不同的卡；按通信开始和结束时各卡持有什么数据，可以区分下面四种集合通信。

| 操作 | 开始时各卡持有 | 完成时各卡持有 | 用途 |
|---|---|---|---|
| AllReduce | 同一张量的部分结果 | 完整归约结果 | TP 部分和、训练梯度 |
| ReduceScatter | 同一张量的部分结果 | 归约结果的不同分片 | 分片输出、梯度分片 |
| AllGather | 一个张量的不同分片 | 完整拼接结果 | 恢复完整输入 |
| All-to-All | 发往不同目的地的数据 | 各源卡发给本卡的数据 | 专家输入的 dispatch 与结果回传 |

Broadcast（广播）把某一张指定卡的数据发给全组，Reduce（归约）只把归约结果交给指定的一张卡，两者都不像表中的操作那样让全组持有结果。AllGather 按 rank（通信组内每张卡的编号）拼接数据，不做求和；All-to-All 按目的地重新分配数据，也不会自动按路由权重合并专家输出。MoE 的 combine 因此包括结果回传、恢复 token 顺序和按路由权重求和三步，不能只凭名字把它当成 AllReduce。[16]

操作名称规定了计算什么、结果落在哪些卡上，具体算法则规定数据分几轮、如何传递。下一小节跟踪其中一块沿环传递的过程。

算法决定交换本身要多久，各卡发起通信的时刻则决定每张卡要等多久：全组要等最后一张卡发起之后才能开始交换，先到的卡都在等待该卡完成前面的计算，所以发起时刻相差越大，加快交换本身的收益就越小。第 7.6.1 节用同一组数据展开这一比较。[17]

### 6.4.2 环、树与通信轮次

**环形 AllReduce** 把参与的卡排成一个有向环，每轮每卡向下一卡发送一块，并接收上一卡发来的块。设参与的卡数为 $n$，每卡的完整部分和为 $M$ bytes，把它分成 $n$ 块，每块 $M/n$。

以四卡环 0→1→2→3→0 为例。每卡最初都有块 0、1、2、3 的本地贡献。第一轮，卡 $r$ 发送块 $r$；接收方将收到的块与自己的对应贡献相加。第二轮转发刚刚形成的部分和，第三轮继续转发并加入最后一份贡献。

跟踪块 0：该块从卡 0 发出，经卡 1、卡 2，最后到卡 3。如果四卡对块 0 某元素的贡献为 1、10、100、1000，沿途数值依次为 1、11、111、1111。第三轮结束时，卡 3 拥有块 0 的完整和；同时，卡 0、1、2 分别拥有块 1、2、3 的完整和。

只需三轮，是因为一个块已经含有起点的一份贡献，再经过其余三卡，就包含全部四份。一般地，ReduceScatter 需要 $n-1$ 轮。接下来每卡将自己完成的块向下一卡转发，每轮多获得一块；再经过 $n-1$ 轮，每卡收齐全部块。因此每卡总发送量为 $2(n-1)M/n$，共 $2(n-1)$ 轮。

令每轮固定开销为 $\alpha$，各有向边的有效带宽为 $B$，各卡等本轮发送完成后才进入下一轮，时间模型为

$$T_{\text{ring}} = 2(n-1)\alpha + \frac{2(n-1)M}{nB}. \tag{6-9}$$

[16] NVIDIA，NCCL 集合通信语义、点对点与不等长交换、nccl-tests 带宽口径；阅读快照与哈希见来源记录。

[17] 集合通信路径与诊断，涵盖固定框架源码、custom AllReduce 分派、就绪偏差和测量范围。

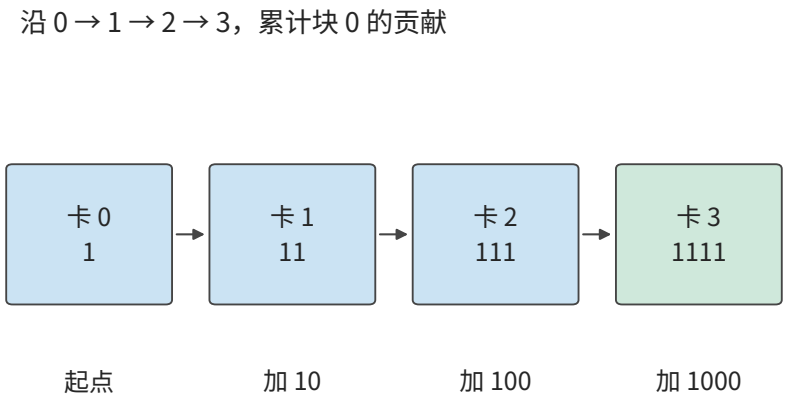



图 6-23：仅跟踪块 0 的一个元素：每经过一张卡，就加入该卡的贡献。三轮后得到 1111，保存在卡 3。其他三个块同时沿环推进。

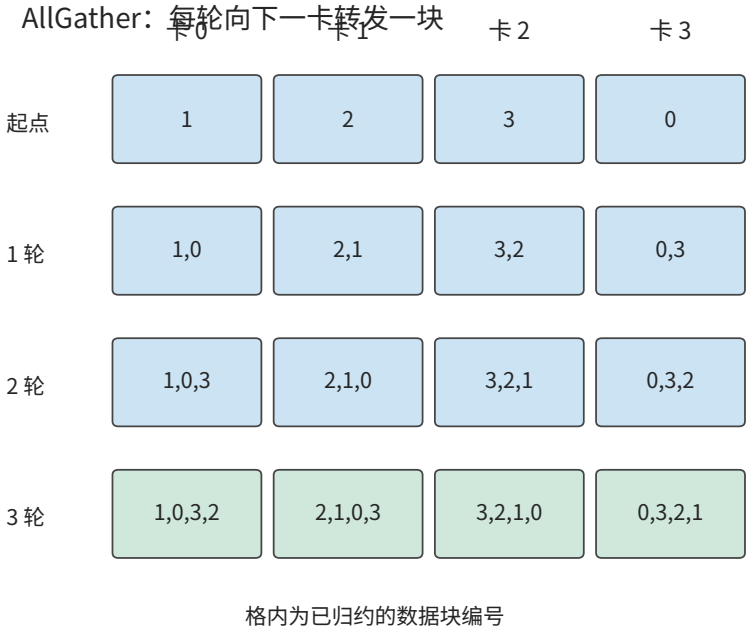



图 6-24：ReduceScatter 结束时，卡 0、1、2、3 分别持有块 1、2、3、0。接下来每轮转发一块，三轮后每卡都拥有四块完整结果。

第一项随卡数增长，第二项逐渐接近 $2M/B$。环形算法把大块数据分成小块，让各卡同时传输；数据量小时，逐轮启动的时间反而占主要部分。

**例题：计入归约通信后，增加 TP 卡数还能加速多少？**仍用 Qwen3-32B 的 128K 会话。每次归约的输入是一个 token 的 BF16 隐藏向量，$M = 10$ KiB；HGX H100 中每张卡经 NVSwitch 的带宽为每方向 $B = 450$ GB/s；每轮固定开销 $\alpha$ 取 MSCCL++ 论文在同样的 H100 八卡服务器上测得的 NVLink 单向延迟 0.822 µs。求 TP2、TP4、TP8 的单次归约以及式（6-5）的时间。

解答：TP8 共十四轮，启动开销合计为 11.51 µs，每卡发送 17.5 KiB，传输约 0.04 µs，单次约 11.55 µs。其

余规模同样代入式（6-9）。TP1 一行只作比较，单卡连一个 128K 会话都无法容纳：

| TP 卡数 | 本地内存访问 | 128 次归约 | 单步时间 |
|---|---|---|---|
| 1 | 29.35 ms | 0 | 29.35 ms |
| 2 | 14.68 ms | 0.21 ms | 14.89 ms |
| 4 | 7.34 ms | 0.64 ms | 7.97 ms |
| 8 | 3.67 ms | 1.48 ms | 5.15 ms |

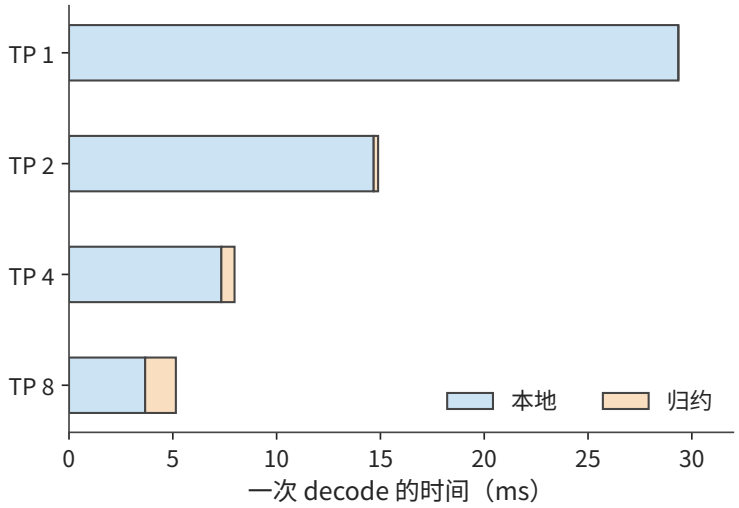



图 6-25：根据式（6-5）和环形归约模型绘制的单步时间。本地内存访问随卡数增加而减少，归约时间则增加；条形总长度为单步执行时间。本地项包括权重与 KV 读取，归约项是卡间集合通信。

图中，蓝色部分随卡数增加而缩短，橙色部分却逐渐变长。两卡到四卡，本地内存访问时间减少约 7.34 ms，通信增加约 0.42 ms，净省约 6.92 ms。四卡到八卡，本地内存访问时间只再减少约 3.67 ms，通信增加约 0.84 ms，净省降至约 2.83 ms。式（6-6）的左右两项逐渐接近，但八卡仍然更快。

这时提高带宽几乎没有收益。八卡一次归约中，11.51 µs 来自启动；即使传输时间降为零，仍要等待这十四轮。把带宽翻倍，整步只节省约 2.5 µs；把每轮固定开销减半，128 次归约却能节省约 0.74 ms。[18]

树形算法从轮数入手。四张卡先两两归约，再由根节点合并结果，归约需要两轮；广播再用两轮，合计四轮。当 $n$ 为二的整数次幂时，未分段二项树共有 $2\log_2 n$ 轮，关键路径上每轮都传输完整张量：

$$T_{\text{tree}} = 2\log_2 n \left(\alpha + \frac{M}{B}\right). \tag{6-10}$$

八卡、10 KiB 时约为 5.07 µs，比环形算法的 11.55 µs 短。数据量增大到 8192 个 token 的 80 MiB，环形算法约需 0.34 ms，树形约需 1.12 ms：减少轮次节省的时间，抵不过在关键路径上反复传递完整大张量增加的时间。令两式相等，交点约在 680 KiB；数据量越过该值，更快的算法就换成另一个。

[18] Ring 逐轮结果、未分段二项树结果，按无争用计算。带宽为 H100 的 18 条 NVLink 4 每方向合计 450 GB/s，见 H100 架构白皮书第 47 页；每轮开销取 C. Hwang 等，MSCCL++: Rethinking GPU Communication Abstractions for AI Inference，ASPLOS 2026，表 1 与表 2：在每节点 8 张 H100、NVLink 4、每卡一张 400 Gbit/s ConnectX-7 的平台上，nvbandwidth 测得 NVLink 延迟 822 ns、吞吐 397.5 GB/s，RDMA perftest 测得 InfiniBand 延迟 3.76 µs、吞吐 48.94 GB/s。

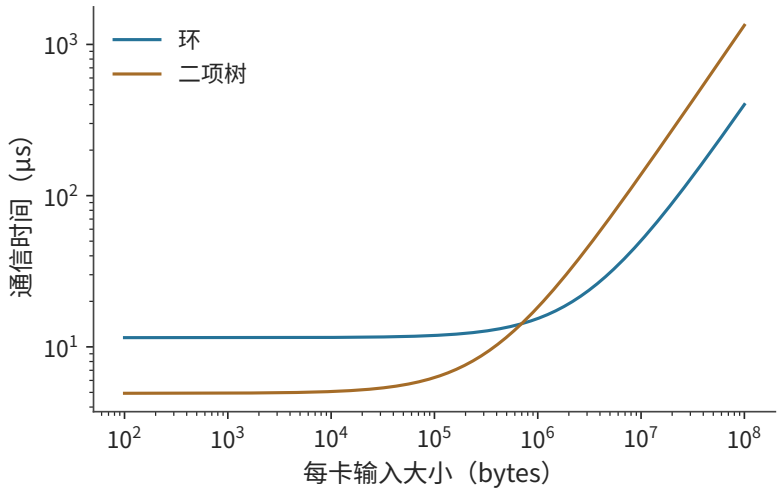



图 6-26：八卡环形算法与未分段二项树的时间模型，每轮 0.822 μs、每方向 450 GB/s。交点约为 680 KiB；10 KiB 的 decode 输入位于主要受启动开销影响的一侧，80 MiB 的 prefill 输入位于主要受数据传输时间影响的一侧。

**实验 6-4 ・延伸：数据量多大时，环形归约比树形归约更快？**

（a）按图 6-23 的发送规则，跟踪块 1 的三轮归约，写出每轮由哪张卡持有该块，以及其中已经累加了哪些卡的贡献。

（b）求四卡和八卡环形归约的每卡发送量，再推导相同数据量下树形归约的耗时。

（c）求八卡部署下两种归约算法耗时相等时的数据量；将 $\alpha$ 减半后重算，解释两种算法耗时相等时的数据量为何变化。

（d）将树形算法归约 10 KiB 数据所需的时间代入本章 Qwen3-32B 算例，求八卡单步时间，比较算法选择与带宽翻倍的收益。

### 6.4.3 通信库与计算的资源竞争

环形和树形算法都要由软件在各卡上实际执行。**通信库**是实现集合通信操作的软件，例如 NVIDIA 集合通信库（NVIDIA Collective Communications Library，NCCL）。昇腾上的对应实现是华为集合通信库（Huawei Collective Communication Library，HCCL）。应用先建立通信组（communicator），给组内每张卡分配一个 rank，再提交操作，说明缓冲区、元素数量、数据类型、归约运算和所在的 CUDA stream。组内各卡的调用必须按接口要求相互匹配，包括调用顺序和数据数量；调用返回通常只表示操作已提交到 stream 中，并不意味着可以复用仍在传输的缓冲区。使用结果的一方要靠同一 stream 的顺序或显式的 event 依赖来等待。集合通信也不能当作任意主机代码都能用的全局同步点。

在 MoE 中，每对卡之间 dispatch 的 token 数随 batch 变化。等长的 All-to-All 表达不了这种不等长的交换，运行时还要自己组织计数、偏移、打包和接收空间。NCCL 可以用成组的点对点 Send / Recv（发送 / 接收）表达任意两卡之间的交换；每个发送必须有对应的接收，需要一起推进的通信要放进同一个组，否则会串行等待对方尚未发起的操作。专门的专家并行通信库进一步结合路由信息实现 dispatch 和 combine。无论选哪种实现，都应记录库的版本与具体接口，不能把固定大小的 All-to-All 测试当作实际的 MoE 通信。[19]

测量 NCCL 时，还要分清它报告的两种带宽：**algbw** 是测试定义的数据量除以时间，**busbw** 则按集合操作的实际传输量换算。对 $n$ 卡 AllReduce，若每卡输入为 $M$，则 $\text{algbw} = M/T$，$\text{busbw} = 2(n-1)M/(nT)$。例如实验 7-3 保存的公开记录中，两台 HGX H100 共 16 个 rank，每卡 64 MiB 的 AllReduce 用时 466.1 μs，algbw 为 143.99 GB/s，busbw 为其 1.875 倍，即 269.98 GB/s。busbw 是归一化指标，不能直接当作每张网卡的实际

[19]NVIDIA，NCCL 集合通信语义、点对点与不等长交换、nccl-tests 带宽口径；阅读快照与哈希见来源记录。

吞吐；分层通信、交换机内归约、链路复用仍要按第 7 章的物理路径逐项计数。数据量小时看延迟，数据量大时看持续带宽，放到模型执行里还要看从数据就绪到用完的整段时间。[20]

第 6.4.2 节比较环和树时，把一次通信当作独立操作。实际执行中，通信常与另一个 micro-batch 的计算重叠，两者会争用加速器资源。归约需要读取部分和、执行加法、写回结果并推进通信。这些工作会占用 SM、HBM 和加速器互联；同一时刻运行的矩阵乘也需要这些资源。通信与计算重叠后，原来独占资源时的速度便会改变。

配套的实验 6-5 测量了这种影响。实验用 Gloo（支持 CPU 等执行环境的集合通信库）在四个 CPU 进程之间做 AllReduce，同时让每个进程做一次矩阵乘，分别记录两者单独运行和同时运行的时间，取五组的中位数：

| 归约数据量 | 通信单独运行 | 计算单独运行 | 同时运行时的通信 | 同时运行时的计算 | 同时运行的整组完成 |
|---|---|---|---|---|---|
| 4 MiB | 2.67 ms | 11.53 ms | 6.14 ms | 11.52 ms | 11.58 ms |
| 64 MiB | 36.92 ms | 11.03 ms | 45.21 ms | 12.89 ms | 45.24 ms |

4 MiB 时，通信与矩阵乘同时运行后慢了一倍多，却仍在矩阵乘结束前完成，整组用时 11.58 ms，与只做矩阵乘几乎相同，通信被计算掩盖了。64 MiB 时，通信从 36.92 ms 增至 45.21 ms，矩阵乘也从 11.03 ms 增至 12.89 ms；两者分开执行共需约 48.0 ms，同时运行只节省约 2.7 ms，而不是矩阵乘的全部 11 ms。评价重叠执行，要比较两者同时运行时的时间，不能把各自单独测得的时间直接相减。[21]

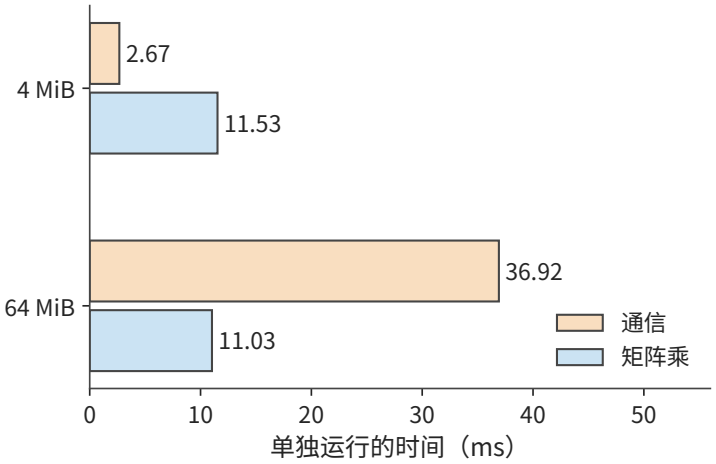



图 6-27：实验 6-5 中通信与矩阵乘单独运行的时间，四个 CPU 进程，五组中位数。4 MiB 通信约 2.67 ms，64 MiB 通信约 36.92 ms，矩阵乘约 11 ms。

GPU 上的道理相同。增加通信线程数和通道数能够加快单独运行的传输，同时也会占用计算所需的执行资源，所以只看单独运行的通信带宽来选择配置，可能让整体变慢。集合通信自动调优系统 AutoCCL 因此在训练过程中记录实际的通信时间，让反馈包含并发干扰；它先选算法、协议与传输实现，再搜索线程、通道和分块。论文报告，在重计算干扰下，AllGather 的带宽从 18.26 GB/s 提高到 32.44 GB/s。搜索和切换本身也要时间：若搜索额外耗时 $S$，此后每次调用节省 $\Delta$，至少要调用 $S/\Delta$ 次才能收回。[22]

[20] NVIDIA，NCCL 集合通信语义、点对点与不等长交换、nccl-tests 带宽口径；阅读快照与哈希见来源记录。

[21] 实验 6-5：四进程实际通信与计算并行。实验在一台 M2 Max 上用 PyTorch CPU 版运行，四个进程经本机 TCP 通信。正式测量分五组，每组把单独通信、单独计算、同时运行三种方式各跑一次，组内顺序随机；表中每项先取组内最慢进程的时间，再取五组的中位数。通信时间从提交异步 AllReduce 算起，到主机收到完成回调为止，可能略晚于数据实际传完。

[22] AutoCCL 采用 16 / 32 张 A40 与 NCCL 2.18.3，八卡 NVLink 为四对连接；其直接搜索目标仍是通信性能，18.26→32.44 GB/s 为论文表 6 的 AllGather 在重计算干扰下的测量。NCCL 2.31.2 跨网络零 CTA 的 AllGather / AlltoAll 路径涉及对称注册窗口、节点内 CE 和节点间 CPU proxy。详见通信调优、计算争用与卸载。

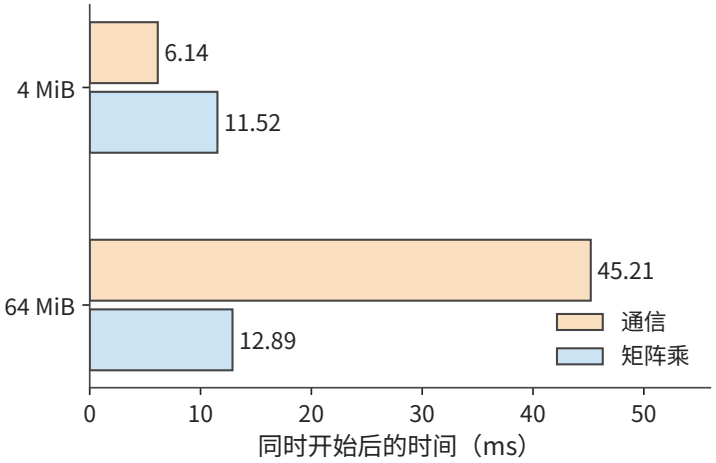



图 6-28：同一实验中两者同时开始。橙色为通信，蓝色为计算；后续工作等待两者都完成。4 MiB 的通信增至 6.14 ms，仍藏在计算之内；64 MiB 的通信增至 45.21 ms，计算也增至 12.89 ms。

### 6.4.4 通信融合、加速器发起与专用卸载

单独加快通信不一定能加快整体执行，因此要区分两种改进：减少通信启动与数据读写次数，以及减少通信对计算资源的占用。在 Qwen3-32B 的八卡算例中，128 次归约的启动开销约为 1.47 ms。围绕这项开销，可以改变三个不同环节：一次启动做多少工作、由谁发起，以及由谁执行搬移和归约。

**融合**让相邻操作在一个 kernel 中衔接。例如归约后还要加残差并做 RMSNorm，分开执行会把结果写回再读入，并经过多次启动；融合后可以在数据仍靠近计算单元时继续处理。归一化必须等归约得到完整输入之后再做：忽略用于避免除零的小常数 epsilon 和可学习缩放参数，$\mathrm{RMSNorm}([1,0]+[0,1])=[1,1]$，分别归一化再相加却得到 $[\sqrt{2},\sqrt{2}]$。融合必须保持这一执行顺序，才能得到相同的计算结果。

**加速器发起**省去主机提交与加速器执行之间的往返：一个 kernel 算出数据后，可以直接发起后续通信，缩短从计算完成到通信开始的等待。**专用卸载**把搬移、归约或同步交给复制引擎、通信处理单元或交换机内的逻辑，把通用计算单元腾出来做矩阵计算。

昇腾的集合通信单元（CCU）将搬移、归约、同步和完成处理集中到通信单元；NCCL 的部分路径用节点内复制引擎（Copy Engine）与节点间 CPU 代理线程（替 GPU 向网卡提交请求的主机线程）承担数据搬移。这些机制改变的是由谁来做这些工作。若计算慢在与通信争用 SM，把 SM 让出来就能缩短计算；若瓶颈在共用的网络出口，出口要传的字节数并没有减少。[23][24]

通信请求由谁发起，决定控制路径要穿越几次 PCIe，也决定占用谁的核。图 6-29 比较三种位置。**CPU 代理线程**：GPU 算完数据后在主机内存里写一个就绪标志，CPU 线程轮询到它，构造请求描述符（描述一次传输的记录）、敲响网卡的门铃（写网卡的一个寄存器，通知它有新请求），网卡再从主机内存读取请求描述符；完成记录也写回主机，由 CPU 转告 GPU。控制路径要穿越 PCIe 三次，每次都是几百纳秒；一个 CPU 线程每秒能处理的请求也有限，大批小请求会在这里排队。**GPU 的 SM**：GPU 直接在自己的内存里构造请求描述符并敲门铃，网卡从 GPU 内存读取请求描述符，完成记录写回 GPU 内存供 SM 轮询。CPU 退出了关键路径，控制路径仍要穿越 PCIe 两次；代价是要分出一部分 SM 来构造请求和轮询完成，这些 SM 不能同时做矩阵计算。NVIDIA 的 GPUDirect Async 和专家并行通信库 DeepEP 的低延迟 kernel 采用的是这条路径。**网卡上的处理**

[23] UB 与昇腾 950 资料核对、UB OS 参考设计§4、950 官方白皮书§4.6。

[24] AutoCCL 采用 16 / 32 张 A40 与 NCCL 2.18.3，八卡 NVLink 为四对连接；其直接搜索目标仍是通信性能，18.26→32.44 GB/s 为论文表 6 的 AllGather 在重计算干扰下的测量。NCCL 2.31.2 跨网络零 CTA 的 AllGather / AlltoAll 路径涉及对称注册窗口、节点内 CE 和节点间 CPU proxy。详见通信调优、计算争用与卸载。

**器**：BlueField 这类网卡内置多核多线程的数据通路处理器，构造请求描述符、敲门铃、取请求描述符都可以留在网卡内部，主机只需为一批操作发一次触发；控制路径不再穿越 PCIe，也不占用 SM 和 CPU 核。三种位置下，发起端的载荷从 GPU 内存搬出、完成记录写回 GPU 内存，各穿越 PCIe 一次，这两项不随发起位置改变；对端网卡读写它所在主机的内存，还要在对端再穿越一次 PCIe。取舍因此是：由谁提供核、控制路径多长、每秒能发起多少请求。第 7.3.3 节将累加每次穿越的时间，第 7.3.4 节再把发起速率纳入吞吐模型。[25]

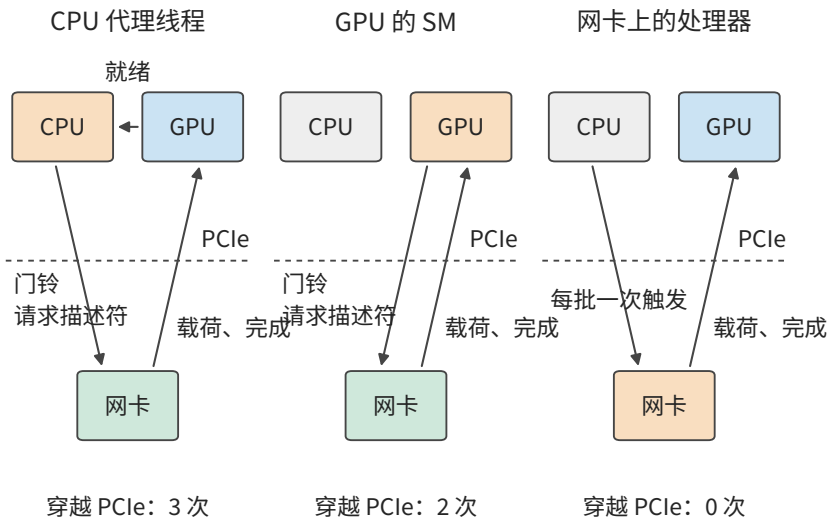



图 6-29：三种发起位置。橙色是构造请求描述符的部件；虚线是 PCIe 边界。CPU 代理线程的控制路径穿越 PCIe 三次，GPU 发起两次，网卡上的处理器零次；发起端的载荷与完成记录在三种方式下都各穿越一次。

回到本节开头的 128 次小数据量归约。框架会按数据量和连接方式选择通信实现。Qwen3-32B 一次处理 1、4、16 个 token 时，归约输入分别为 10、40、160 KiB，vLLM 等框架可以为这些小数据量选用专门的实现。由此，式（6-5）中的 $T_{\mathrm{AR}}$ 有了两个改进方向：减少算法轮次，或缩短每轮的软件与执行开销。[26]

**实验 6-5 ·延伸：通信优化能节省多少时间，何时抵消准备开销？**

（a）用图 6-27 与图 6-28 的实测值，分别求 4 MiB 与 64 MiB 时同时运行比分开运行省下多少时间，指出每种情况下是计算还是通信决定了整体完成时间。

（b）设一次调优需要额外花费 $S$ 的搜索与切换时间，此后每次调用节省 $\Delta$。写出累计节省首次超过 $S$ 所需的最少调用次数；以 64 MiB 实测的重叠收益作为 $\Delta$，求 $S = 1$ s 时的次数。

（c）用两个各含两个元素的向量，比较先归约后执行 RMSNorm 与先分别执行 RMSNorm 后归约的结果，判断两者能否交换顺序。

（d）加速器选做：比较同一形状的独占与并发执行，记录就绪、开始和完成事件，说明共享后哪一段改变最多。

## 6.5 超节点的物理组织

### 6.5.1 直连、交换与分层拓扑

第 6.4 节的算法和通信库决定了每一轮谁发给谁、发多少，但没有涉及数据经过哪些物理连接。集合算法规定逻辑上的发送方和接收方，物理拓扑决定数据实际经过哪些设备与链路。两张卡可以直连，也可以经交换机、

[25] NVIDIA 技术博客 Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async（CPU 代理线程与 GPU 发起两条路径的步骤）；NVIDIA DOCA DPA 文档（把通信代码卸载到 BlueField-3 网卡内处理器的编程模型）；DeepEP README 快照。三种位置的穿越次数按各自的控制路径计，具体实现可能合并或增加步骤；抓取快照见来源记录。

[26] 集合通信路径与诊断，涵盖固定框架源码、custom AllReduce 分派、就绪偏差和测量范围。

其他卡或主机内存中转。多次传输共用一个接口时，要排队使用同一份带宽。

**例题：再加一台服务器，TP16 能否比 TP8 更快？**两台 HGX H100 共 16 张卡，服务器之间由每张卡自己的 ConnectX-7 网卡相连。把 Qwen3-32B 的一个 128K 会话改用 TP16，一个张量并行组横跨两台服务器，每层的 AllReduce 都要经过网络。

解答：本地内存访问方面，权重再减半，但 Qwen3-32B 只有 8 个 KV 头，TP16 时每个 KV 头在两张卡上各存一份，每卡读取的 KV 与 TP8 相同，本地时间只从 3.67 ms 降到 2.48 ms。通信方面，若 16 张卡排成一个环，每轮都要等经过网卡的那一跳；取 MSCCL++ 论文在同一平台上测得的 InfiniBand（服务器之间所用的低时延交换网络）单向延迟 3.76 µs，一次归约要 30 轮，约 113 µs。实际的通信库不会这样做。实验 7-3 保存的 nccl-tests（NCCL 的集合通信性能测试程序）公开记录恰好是两台 HGX H100、16 个 rank 的 AllReduce，10 KiB 的消息取记录中 16 KiB 一档，实测 32.74 µs：

$$T(16) \approx 2.48\ \text{ms} + 128 \times 32.74\ \mu\text{s} \approx 6.67\ \text{ms}.$$

单台服务器内的 TP8 为 5.15 ms，TP16 反而慢约 1.52 ms。增加的卡减少了每卡的权重读取，却换来了更贵的跨服务器同步；即使用上实际通信库的分层算法，每次归约仍要约 33 µs，接近机内环形归约 11.55 µs 的三倍。

要让更多的卡高效协作，就要有足够的端口和交换容量。一张卡若与其他 $N-1$ 张卡都直连，就需要 $N-1$ 条连接。交换机把连接集中起来：卡先接到交换层，再由交换层转发到目的地。一颗交换芯片提供的端口数称为**基数**（radix）。

以 HGX H100 使用的第三代 NVSwitch 为例，一颗芯片有 64 个 NVLink 4 端口，每个端口每方向 25 GB/s。把这颗芯片用作一台**叶交换机**（leaf）：32 个端口向下接 GPU，另外 32 个端口向上接**脊交换机**（spine），GPU 一侧的总带宽和上联总带宽都是 800 GB/s。若改成 48 个下联、16 个上联，接入的 GPU 链路多了，但它们一起向上最多只能送出 400 GB/s，只有下联总带宽的三分之一。端口总数不变时，多接 GPU 就少了上联端口和上联带宽。DGX H100（NVIDIA 以 HGX H100 为基础的整机）的 NVLink Switch System 把多台服务器接入同一个 NVLink 交换网络，正是这样的取舍：每个节点只把全部 NVLink 带宽的一半引出节点，即 2:1 收敛。[27]

把交换层扩展到一颗芯片之外：四颗叶交换芯片各有 32 个下联端口，共 128 个下联端口。再配 32 颗脊交换芯片，每颗各用一条链路连到这四颗叶交换芯片，正好用满每颗叶交换芯片的 32 条上联。把系统分成两部分，所有连接这两部分的链路合起来称为一个**割集**。若某一阶段有 $V_{A\to B}$ bytes 必须从 A 侧穿过割集到 B 侧，而割集在该方向的总带宽为 $B_{A\to B}$，传输至少需要 $V_{A\to B}/B_{A\to B}$。交换机的端口分配决定了割集的总带宽。

端口数决定一共有多少链路，路由决定数据集中到哪些链路。即使每张卡发送的数据量相同，走不同的路径也会形成不同的瓶颈。把 16 张卡连成一个双向物理环，观察 8 MiB 输入的 ReduceScatter 前三轮。递归算法每轮把对端的距离翻倍；Swing 是为环面拓扑设计的集合通信算法，通过改变各轮的对端来分散链路负载。两种算法每张卡每轮同样发送 4、2、1 MiB，但对端不同：

| 前三轮 | 递归路径 | Swing 路径 |
|---|---|---|
| 每次传输经过的物理跳数 | 1、2、4 | 1、1、3 |
| 最大单向链路传输量 | 4、4、4 MiB | 4、2、2 MiB |
| 全网物理链路累计传输量 | 64、64、64 MiB | 64、32、48 MiB |

[27] H100 架构白皮书第 47–48 页：第三代 NVSwitch 每颗提供 64 个第四代 NVLink 端口，每条 NVLink 每方向 25 GB/s；NVLink Switch System 中每个节点以 2:1 收敛引出节点内全部 NVLink 带宽。

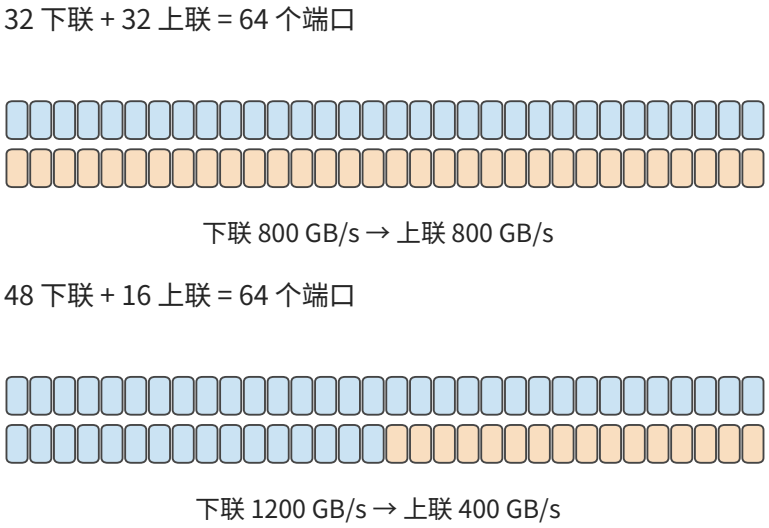



图 6-30：同一颗 64 端口 NVSwitch 的两种分配，每个端口每方向 25 GB/s。上联链路是所有跨交换机流量的共同出口；32 ／ 32 分配提供 800 GB/s 上联带宽，48 ／ 16 分配只提供 400 GB/s。颜色条的格数与端口数一致。

第三轮，递归算法的数据要走四跳，最忙的链路同时承载四份 1 MiB 数据；Swing 的数据走三跳，最忙的链路只承载两份。按 TPU v4 每条 ICI（芯片间互联）链路每方向 50 GB/s 计，前三轮由瓶颈链路决定的传输时间分别约为 252 与 168 µs。传输时间少了三分之一，是因为共享链路上的数据少了。[28]

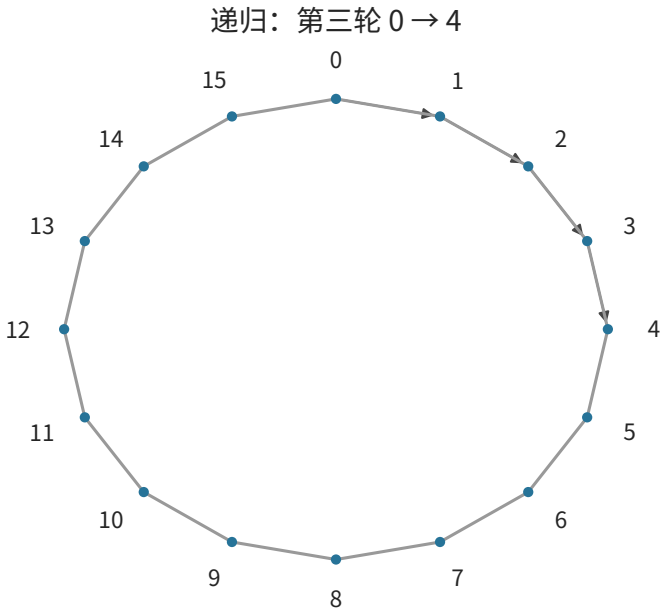



图 6-31：递归算法第三轮，卡 0 发往卡 4。细线为 16 张卡组成的物理环，箭头标出经过的四条链路。

经主机内存中转是同一个问题的另一种形式。非统一内存访问（NUMA）结构把主机内存分在不同 CPU 附近，跨 CPU 读取要经过处理器之间的接口。设卡 0、1 接在 NUMA 节点 A，卡 2、3 接在节点 B，四张卡做

[28] 集合通信的物理路径与作业错峰、96 条消息的物理路径计算、NSDI 2024 选读、Morphlux 版本与阅读记录。TPU v4 的 OCS 基于 MEMS 反射镜，切换需要毫秒级时间，见 TPU v4 论文 §2。

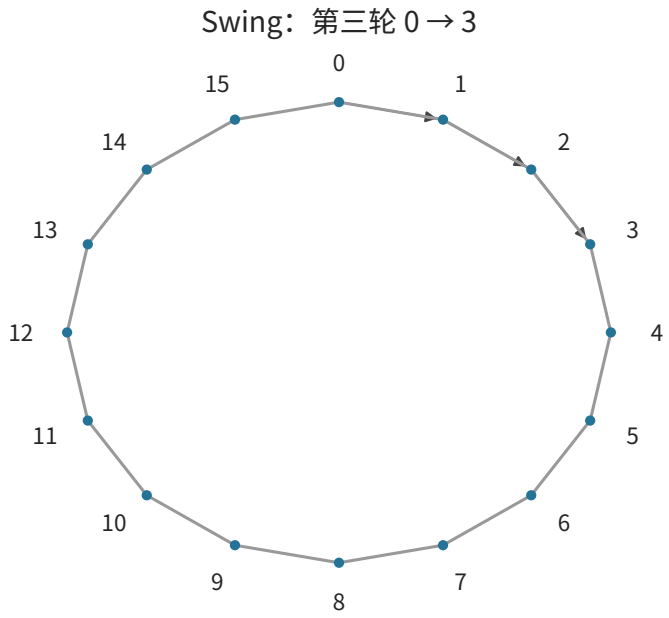



图 6-32：同一物理环上，Swing 第三轮卡 0 发往卡 3，经过三条链路。每张卡这一轮的发送量仍为 1 MiB。

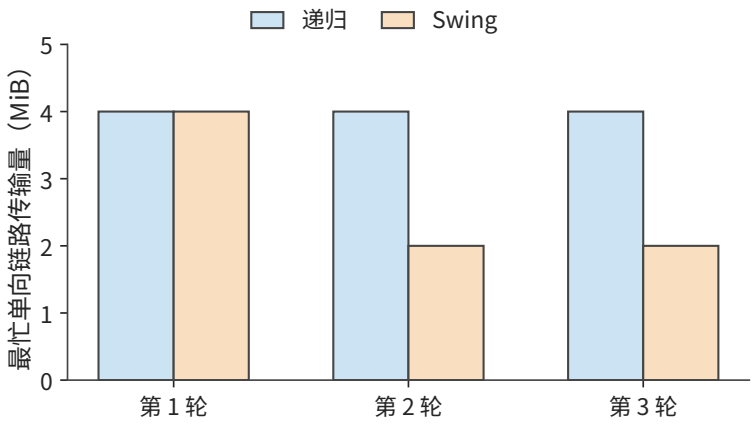



图 6-33：把所有发送方的数据累加到各条有向链路上，取每轮的最大值。递归前三轮均为 4 MiB，Swing 为 4、2、2 MiB。

环形归约共发送 48 MiB。把全部中转缓冲放在 A，CPU 之间每个方向要承担 24 MiB；把缓冲放到发送卡附近，并让同一 NUMA 节点的卡在环中相邻，每个方向降到 12 MiB；若把环改成交替跨越 NUMA 节点，每个方向又回到 24 MiB。DGX H100 的主机就是两路结构：两颗 Xeon 8480C 各带一个 NUMA 节点，两颗 CPU 之间最多有 4 条 UPI（Intel 处理器之间的互联）链路，每条 16 GT/s；每张 GPU 经 PCIe Gen5 x16 连到主机，每方向 64 GB/s。三种方案中每张卡经 PCIe 进出主机的数据量相同，经过 UPI 的数据量却按 2:1:2 变化，跨 CPU 的传输时间也随之变化。缓冲的位置和卡在环中的次序共同决定数据的搬移路径。[29]

**设计案例：直连环面还是交换网络？**上述例子中，数据要么直接走一条链路，要么经过一台交换机。把 64 张卡连成一个超节点时，有两条常见的路线。一条是让每张卡直接和邻居相连：把 64 张卡排成 $4\times4\times4$ 的立方体，每张卡沿三个维度各接两个邻居，每一维首尾相连，这就是**三维环面**（torus）。环面是超级计算机长期使用的一

[29]PCIe 中转与 NUMA 放置、发送端附近缓冲的计算（本章只用其中的字节计数）。HGX H100 主机的 CPU 型号见 DGX H100 用户指南；Xeon 8480C 是 Xeon Platinum 8480+ 的定制版本，后者最多 4 条 UPI 链路、每条 16 GT/s，见 Intel 产品规格与 第四代至强技术概览表 1。Intel 未公开 UPI 每方向的 GB/s，这里不换算传输时间；PCIe Gen5 x16 每方向 64 GB/s 见 H100 架构白皮书第 48 页。

类拓扑，TPU 沿用了它。另一条是让每张卡只接交换芯片，由交换层转发到目的地，NVL72 和 CloudMatrix384 采用这一路线。两条路线给每张卡同样的六个端口，每端口每方向 50 GB/s，这正是 TPU v4 每颗芯片的 ICI 配置；差别在端口的另一端接的是邻居还是交换芯片。图 6-34 画出在这两种连接方式下，一次传输经过的路径。

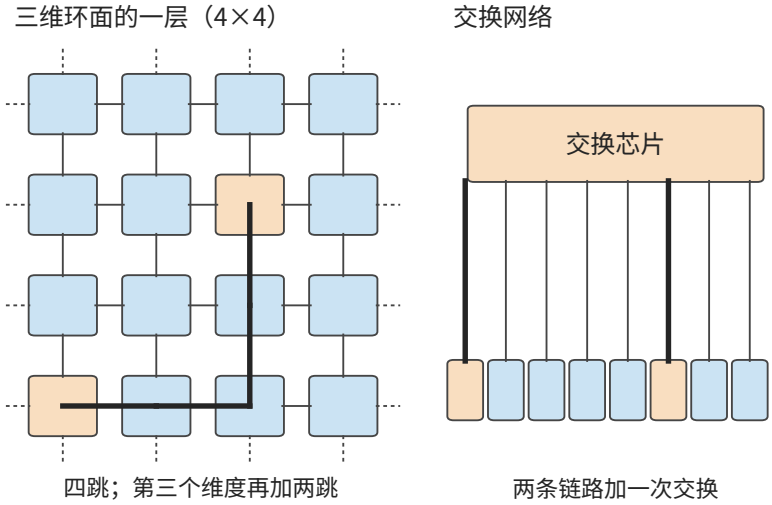



图 6-34：左侧是三维环面的一层，每张卡与四个邻居直连，虚线表示每一维首尾相连；从角上的卡到中心的卡要经过四跳，加上第三个维度最多六跳。右侧是交换网络，任意两张卡之间都是两条链路加一次交换。橙色是相距最远的一对卡，粗线是它们之间传输经过的路径。

先比较**跳数**。环面里最远的两张卡在每个维度上相距两跳，三个维度共六跳；交换网络里任意两卡都是两条链路加一次转发。设数据每经过一颗芯片的路由器或一颗交换芯片要增加 200 ns，这是开放加速器互联标准 UALink 2.0 规范为 128 lane（串行通道）交换芯片设定的空载延迟目标；链路上的传播时间忽略不计。环面上最远的两张卡之间要经过 5 颗中间芯片，交换网络只经过 1 颗交换芯片，环面每次传输多耗时 0.8 µs。单次差距不大，但 decode 每层要做两次归约，Qwen3-32B 的 64 层就是 128 次；若每次都要等最远的卡，环面共多等约 0.10 ms，约为八卡 TP 全部归约时间 1.48 ms 的 7%。卡数增加到 $8^3 = 512$ 时，环面最远变成十二跳；交换网络只需再加一级交换，变成四条链路加三次转发。

再比较**流量模式**。第一种是沿维度的归约：把 AllReduce 拆成沿 x、y、z 三个方向的环形 ReduceScatter 与 AllGather，每一步只在相邻卡之间传输，六条链路同时工作。这时环面把每张卡的全部端口带宽都用上了，和交换网络一样快。TPU 的编译器正是把数据并行和模型并行的分组对应到环面的维度上。第二种是均匀的 All-to-All：每张卡向其他 63 张卡各发一份，MoE 的 dispatch 就是这种模式。环面上每份数据平均要走三跳（每个维度平均一跳）；64 张卡各发 $M$ 字节，全部链路一共要承载 $3 \times 64M$，分摊到 384 条有向链路，每条 $M/2$。交换网络里每张卡的六个端口各分 $M/6$。取 $M = 32$ MiB，环面至少需要约 336 µs，交换网络约 112 µs，相差三倍；512 卡的环面平均六跳，差距变成六倍。图 6-35 把两种模式放在一起。

最后比较**硬件代价**。64 卡环面需要 $64 \times 6/2 = 192$ 条链路，不需要任何交换芯片，路由器做在每颗芯片内；交换网络需要 384 条链路和六颗 64 端口的交换芯片。卡数超过一颗交换芯片的端口数之后，交换网络还要再加一级：CloudMatrix384 为 384 个 NPU 在 48 个节点内各放 7 颗一级交换芯片，再用 7 个子平面各 16 颗二级交换芯片相连，共 448 颗交换芯片，其中的二级交换芯片装在 4 个通信机柜里。环面则在任何规模下每卡都只需六条链路，代价是跳数随规模增加，每卡分到的割集带宽随规模下降，第 6.5.4 节将计算这一点。

三个比较合起来，就是选择拓扑的判断依据：负载以规则的集合通信为主，还是有大量目的地任意的小数据传输和 All-to-All；协作域需要多少张卡；是否愿意用交换芯片和更多线缆换取任意两卡等距。第 6.5.3 至 6.5.5 节将看到 NVIDIA、Google 和华为在这三个问题上各自作了什么选择。[30]

[30] 设计案例每芯片 6 个端口、每端口每方向 50 GB/s 取自 TPU v4：TPU 多代综述表 1 列出 TPU v4 每芯片 6 条 ICI 链路、每条 50

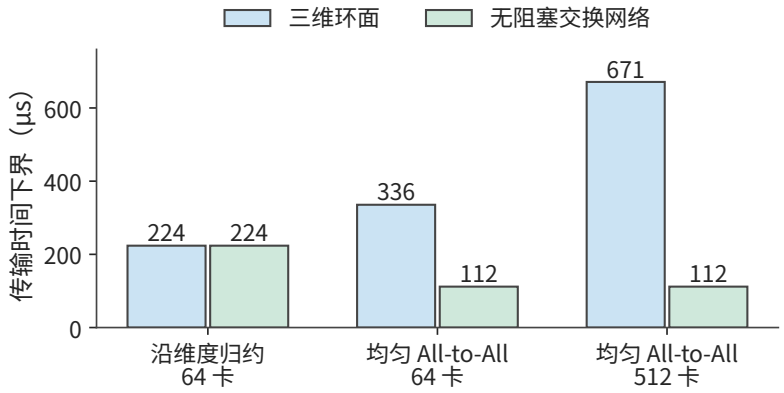



图 6-35：每张卡六个 50 GB/s 端口、每卡发送 32 MiB。沿维度归约时两种拓扑都用满端口带宽；均匀 All-to-All 时环面上的数据平均要走三跳，共享链路承载的数据随之增加，512 卡时增至六跳。图中是链路负载给出的下界，不含启动开销与排队；交换网络按无阻塞计，即交换层不限制各端口同时满速收发。

**实验 6-6 ・延伸：通信拓扑与缓冲位置如何影响传输耗时**

(a) 复算两台 HGX H100 上 TP16 的单步时间，再求跨服务器的一次 AllReduce 降到多少时，TP16 与单台服务器内 TP8 的单步时间恰好相同。

(b) 对 64 端口的 NVSwitch，比较下联 / 上联端口分别为 32 / 32 与 48 / 16 的两种分配，计算所有 GPU 链路同时向上发送时每条链路的平均可用带宽。

(c) 根据图 6-31，累加第三轮经过同一条有向链路的数据量，说明跳数与链路传输量的区别。

(d) 在 NUMA 例中先移动缓冲，再改变环的次序，分别画出新增与消失的跨 CPU 访问。

(e) 把设计案例中的 64 卡改为 $8 \times 8 \times 8$ 环面，求最远跳数、均匀 All-to-All 时每条有向链路的负载，以及两级无阻塞交换网络需要多少颗 64 端口交换芯片；说明哪一项随卡数增长得最快。

### 6.5.2 互联介质、供电与物理规模

第 6.5.1 节把链路看成给定带宽和延迟的连接；在实际系统中建成这些连接，还要解决传输距离、布线和供电问题。把低延迟协作延伸到更远处，连接介质也要随之变化。短距离铜线可以直接传送高速电信号；距离增大后，信号衰减和串扰加重，接收端更难恢复数据与时钟，因此需要更强的均衡或重定时电路。更粗的线缆和更多连接又占用机箱与布线空间。选择连接介质时，需要同时考虑传输距离、信号损耗和布线密度。

光连接把数据调制到光载波上，适合跨越更长距离；同时也增加了光电转换、器件供电和维护的成本。可插拔光模块将转换放在可更换模块中，**共封装光学**（CPO）则把光学引擎移到交换或计算芯片附近，缩短高速电通路。后者提高封装内的连接密度，但光学器件与计算或交换芯片共同封装后，散热、测试和部件更换也需要一并设计。连接介质决定在给定距离和带宽下的实现成本，拓扑决定这些连接如何组成通信路径。[31]

供电能力进一步限制可以安装的卡数。卡按整台服务器安装，设系统预算为 $P_b$，交换机、存储等固定配套功率为 $P_0$，每台服务器装 $g$ 张卡、功率为 $P_s$，则

GB/s，脚注 4 说明按每方向计（TPU v5p 与 Ironwood 为每条 100 GB/s）；TPU v4 论文表 4 同为 6 条 50 GB/s 链路。每次转发 200 ns 取 UALink 2.0 规范 §9.9.2 为 128 lane 交换芯片设定的空载延迟目标，把它同时用于环面芯片内的路由器。32 MiB 为题设的每卡发送量；均匀 All-to-All 的链路负载按维序路由的平均跳数计算。其余系统数据来自 TPU v4 论文 §2（光链路成本、OCS 占比、3D 环面的割集）、NVLink 官方规格（每 GPU 18 条链路、72 GPU 无阻塞）、CloudMatrix384 论文 v2 §3.3.2–3.3.3（一级、二级交换芯片数量与无阻塞设计）。

[31] 端口数量与互联介质分析。端口与功率算例采用题设输入。

$$N \le g \left\lfloor \frac{P_b - P_0}{P_s} \right\rfloor. \tag{6-11}$$

DGX H100 的八张 H100 连同 CPU、NVSwitch、网卡和风扇，系统功率上限为 10.2 kW，平均每张卡 1.275 kW；单张 H100 SXM 本身的功耗上限是 700 W，其余来自服务器内的配套部件。[32] 设一组机柜的供电预算为 120 kW，本例只计服务器，$P_0$ 取 0，最多可以安装 11 台服务器、88 张卡。64 卡（8 台）用 81.6 kW，72 卡（9 台）用 91.8 kW，96 卡（12 台）要 122.4 kW，超过功率预算；64 卡和 72 卡方案还需根据交换端口数量与安装空间确定具体配置。

散热给出另一条上限。风冷机柜靠风扇把热量带进机房空气：ASHRAE 的液冷白皮书指出，一个 40 至 50 kW 的机柜需要多达 5,000 cfm（立方英尺每分钟）的风量，而最好的地板送风口只能提供 1,900 cfm；在 50 kW 机柜里，风扇本身至少消耗 5 kW。本例取 40 kW 作为风冷机柜的阈值，一个机柜最多放 3 台 DGX H100，即 24 张卡、30.6 kW；放 4 台就要 40.8 kW；DGX SuperPOD H100 参考架构给出的示例机柜布置中，每个机柜的功率都超过 40 kW。64 卡的 81.6 kW 与 72 卡的 91.8 kW 若要集中在少数机柜里，都远超这一阈值，要改用液冷，把热量直接交给冷却液而不是机房空气。[33]

如果一个交换单元只能连接 64 个端点，接入第 65 张卡就可能要增加一整套交换设备。卡数逐步增加时，配套交换设备的成本不是平滑上升，而是在跨过这样的整组边界时突然跳升。用式（6-11）求出大致范围之后，再按交换机、托盘、机柜这些安装单位组合，才能得到可以建造的系统。

**实验 6-7 ・延伸：功率预算与整组扩容要求如何限制加速器数量**

（a）分别复算安装 64、72、96 张卡时的总功率，按式（6-11），求功率预算增加到 150 kW 时最多可以安装多少张卡。

（b）若超过 64 卡后要为第二级交换设备预留相当于一台服务器的 10.2 kW，求 120 kW 功率预算下最多可以安装多少张卡。

（c）沿用（b）的功率预算和额外交换设备条件，假设只能按四台服务器（32 张卡）整组增加，求最多可以安装多少张卡，并解释（a）至（c）的上限为何不同。

（d）用前面得到的单实例服务时间比较其中两种加速器配置的服务能力。

### 6.5.3 NVIDIA：从八卡服务器到机柜

端口、距离和功率共同限制了系统规模。在这些条件下，NVIDIA 的几代系统把互联从机箱内直连发展到了机柜级交换网络。DGX-1 用八张 GPU 组成 NVLink 混合立方体网格，部分 GPU 直接相连，其他 GPU 之间的通信需要中转。把频繁通信的计算任务放到相邻的 GPU 上，就能减轻中转和共享链路的压力。

DGX-2 使用 16 张 V100 和 NVSwitch。交换结构把“某对 GPU 是否直接相连”转化为“交换层能同时转发多少数据”，扩大了可用协作范围。DGX A100 又以八卡服务器为基本单元，配备 NVLink / NVSwitch。这些系统采用不同的 GPU 数量与互联结构，以适应芯片接口、机箱空间和应用需求。[34]

GB200 NVL72 将 72 张 GPU 和 36 个 Grace CPU 放进机柜级 NVLink 系统。机柜内更多的 GPU 由此能通过 NVLink 频繁交换数据。回到第 6.5.1 节的例题：Qwen3-32B 的 TP16 横跨两台 HGX H100 时，128 次归

[32] DGX H100 用户指南表 3：系统功率 10.2 kW max；H100 SXM 的 700 W 见硬件表。按服务器计的功率上限见第 6 章计算结果的 `rack` 字段。

[33] ASHRAE TC 9.9，Emergence and Expansion of Liquid Cooling in Mainstream Data Centers，2021，第 14、28 页：白皮书没有给出单一的风冷上限，40 kW 是本例按其风量对比选取的阈值；DGX SuperPOD H100 参考架构第 10 页的示例机柜布置中每机柜功率超过 40 kW。按服务器计的功率上限见第 6 章计算结果的 `rack` 字段。

[34] V100 架构白皮书，NVLink 拓扑与 DGX-1 附录；A100 架构白皮书，NVLink / NVSwitch 与 DGX A100 附录；DGX-2 的 16 卡历史配置亦见ZeRO 论文平台说明。

约经网络约需 4.19 ms，单步约 6.67 ms。若这 16 张卡处在同一个 NVLink 交换域内（DGX H100 的 NVLink Switch System 最多连接 256 张 H100），按每轮 0.822 µs 算，16 卡环形归约共 30 轮，128 次约 3.16 ms，单步约 5.64 ms，仍慢于单台服务器内 TP8 的 5.15 ms。该模型只有 8 个 KV 头，TP16 时每卡读取的 KV 不再减少，环的轮数却多了一倍。对这类模型，更大的互联域的价值不在于把一个会话切得更碎，而在于容纳更多实例，或者容纳超出单台服务器容量的模型。[35]

机柜级互联让更多 GPU 能参与同一个实例。单卡可以容纳的小模型，可以按延迟和吞吐要求选择实例数；权重超过八卡总容量的大模型，则可以用更多 GPU 保存权重，各层的通信走机柜内互联。

### 6.5.4 TPU：拓扑与负载协同

NVIDIA 的路线是用交换层扩大 GPU 的连接范围。Google 的张量处理器 TPU 则沿用第 6.5.1 节设计案例中的环面，让模型切分、编译器的布局和物理通信互相对应。二维环面沿两个维度连接芯片，每一维首尾相连；三维环面再加一个维度，数据可以沿更多方向传输。

TPU v4 以 $4 \times 4 \times 4$ 的 64 芯片电互联单元为基础，再用光路交换机（OCS，通过改变光路连接设备端口）连接多个单元。这一划分同时考虑了网络拓扑和设备安装：64 颗芯片与 16 台 CPU 主机能够放入一个机柜，更大的 $8^3$ 单元包含 512 颗芯片，需要跨柜连接。[36]

用割集带宽可以分析这种规则拓扑扩展后的通信能力。将一个 $k \times k \times k$ 的三维环面沿某一维等分，穿过两个切面的物理链路（即该割集，称为二分链路）共 $2k^2$ 条。$k$ 从 4 增到 8，芯片数由 64 增到 512，增加到原来的八倍，割集链路却只由 32 增到 128，增加到原来的四倍。若每颗芯片都要向另一半发送同样多的数据，平均每颗芯片分到的割集带宽就减半。芯片数与割集带宽增长速度的差别，决定了哪些并行布局适合这种形状。

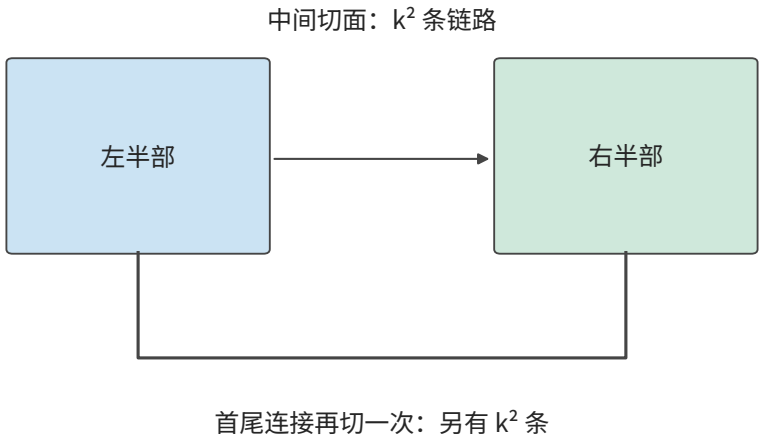



图 6-36：沿一个维度把三维环面网络分成两半，需要切断中间连接和首尾连接。每处包含 $k^2$ 条链路，合计 $2k^2$ 条。

固定拓扑决定了跨分区的带宽，空闲芯片的位置还决定作业能否获得所需的形状。OCS 在更大尺度上重新连接这些电互联单元，为不同作业分配形状合适的芯片子集，这样的子集称为切片。以一个 4×4 的芯片网格为例：若有连续两行空闲，可以放下两个各需要相邻 2×2 芯片的作业；同样八颗空闲芯片若呈棋盘状分布，就找不出一组相邻的 2×2。重新连接可以改变可分配的形状。

[35] GB200 NVL72 官方归档，固定快照中 72 GPU / 36 CPU 与机柜级 NVLink 组织；NVLink Switch System 最多连接 256 张 H100，见 H100 架构白皮书第 48 页。16 卡环形归约的时间见第 6 章计算结果的 `two_servers.nvlink_domain_tp16` 字段。

[36] TPU v4 论文、TPU 多代系统综述，电互联 cube、OCS、切片与故障隔离的设计。

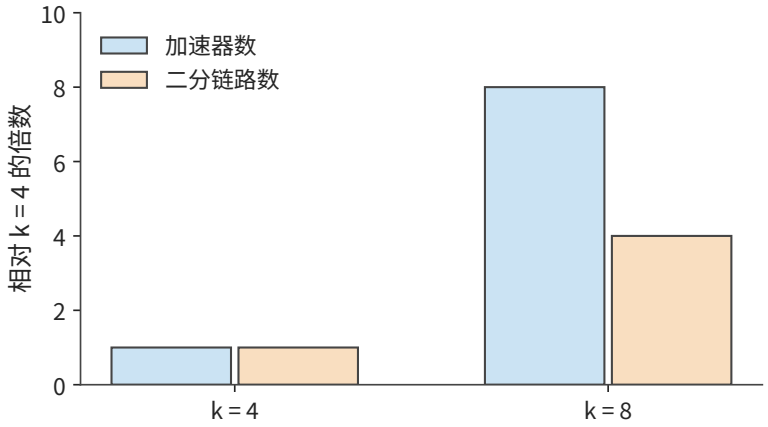



图 6-37：k 从 4 增到 8 时，芯片数从 64 增到 512，二分链路从 32 增到 128。两种增长均以 $k = 4$ 时为基准。k 表示三维环面每个维度的节点数，总节点数为 $k^3$。

重新连接节省的执行时间要超过重连本身的耗时才值得。Morphlux 研究按任务重构光互联拓扑，让连接适合任务的通信需求。设八颗 TPU v4 芯片做 8 MiB 输入的环形 AllReduce，每颗芯片发送 14 MiB。若该切片原来每颗芯片只有一条 50 GB/s 的 ICI 链路可用于该通信组，重连后有三条，传输项就从约 293.6 μs 降到约 97.9 μs，每次节省约 195.7 μs。TPU v4 的 OCS 切换光路需要毫秒级时间：若重连需要 1 ms，重复 6 次就能收回；若需要 100 ms，则要 511 次。作业持续得越久，建立更合适的连接越值得。[37]

### 6.5.5 Unified Bus 与 CloudMatrix

NVIDIA 与 TPU 两种系统主要从加速器互联出发。华为的 UB 进一步把问题扩展到资源访问：连接两台主机之后，一台主机上的设备能否直接使用另一台主机的内存。笔者对 Unified Bus 的早期思考来自主机内总线与主机间网络的分界。同样是读取一块数据，跨过主机边界后，软件往往要改用消息传递接口，重新安排缓冲区，每多一层抽象就多一段时间。UB 希望去掉这些抽象层，扩大资源直接访问的范围，使计算、内存和存储能够按任务重新组合。

本书在两处讨论 UB。本节把 UB 看作一种超节点互联，回答三个与规模有关的问题：控制器接在哪里，片上缓存能容纳多大的互联，建立全部连接需要多长时间。第 7 章则沿着一次远程访问的路径展开：第 7.1.3 节说明统一接口的设计动机，第 7.3.3 节逐阶段推导一次读取的延迟，第 7.3.4 节计算请求速率，第 7.4.2 节推导连接状态的增长和片上缓存的溢出点，第 7.4.3 节讨论按需指定的顺序。两处使用同一组参数和同一个计算脚本，推导结果都与笔者的 UB 实现 OpenURMA 在仿真中测得的数值对照。[38]

这种统一首先要回答两个不同的问题：一次操作要访问谁、读写什么、何时可以使用结果；这次操作的数据如何经过网络并可靠送达。UB 将前一类职责放在**事务层**，后一类职责放在**传输层**。这里的“事务”是一次通信操作，不是数据库中具有提交与回滚语义的事务。

事务层的 Jetty 为应用提供提交、接收和完成通知的端点抽象，保留应用身份与操作所需的上下文；传输层的传输通道（UB 中称为 TP Channel，其中 TP 是 Transport Protocol 的缩写，与张量并行无关）维护报文序号、确认、重传和拥塞控制状态。多个应用端点访问同一远端时，可以共用底层的传输通道，不必让每条应用关系都各带一份完整的可靠传输状态。图 6-38 展示了这一分离。传统远程直接内存访问（RDMA，让网卡直接读

[37] 集合通信的物理路径与作业错峰、96 条消息的物理路径计算、NSDI 2024 选读、Morphlux 版本与阅读记录。TPU v4 的 OCS 基于 MEMS 反射镜，切换需要毫秒级时间，见 TPU v4 论文 §2。

[38] 主机数与连接建立时间的参数和结果见UB 互联计算，运行 `python3 calculations/calc.py ub-fabric --format md` 可以复算；记录尺寸、阶段延迟和缓存条目数取自 OpenURMA 论文表 3、表 7 与 §7.2。目录式缓存一致互联按每个缓存行为每个对端保留 1 bit 的公式计算；NVL72 的 72 张 GPU 是该代产品的公开配置，不是本书推导的结果。

写已授权的远端内存）的可靠连接 QP（queue pair，队列对，由发送队列和接收队列组成的通信端点）把应用端点关系与传输状态绑定得较紧；UB 则允许分别安排两类状态的数量、寿命与共享范围。[39]

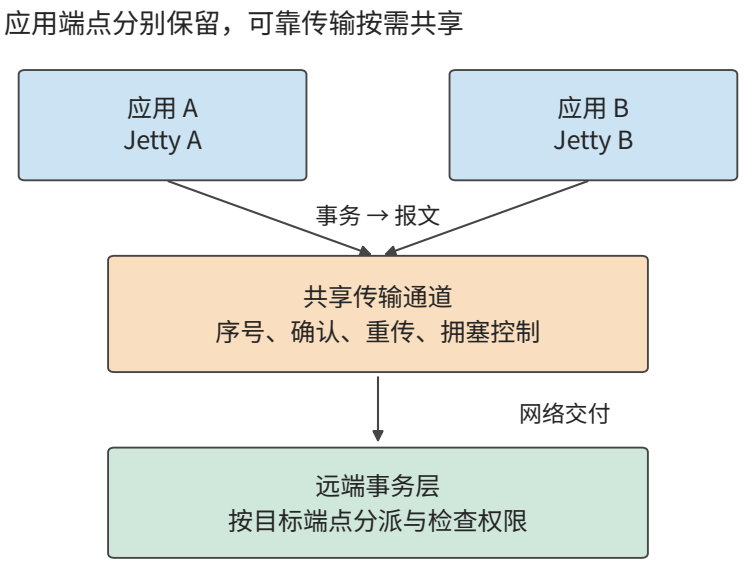



图 6-38：两个本地应用保留各自的 Jetty，经共享传输通道访问远端端点。事务层区分操作及其完成归属，传输层处理报文的可靠交付；共享通道不取消权限检查，也不自动建立跨应用的全局顺序。

另一项关键选择是**内存语义**。单边操作由发起方直接指定已经授权的远端内存地址，远端应用不必为每次操作准备一个匹配的接收请求。UB 支持两种发起方式：Load / Store 由处理器指令发起，Read / Write 由显式提交的请求发起。两种方式都能访问远程内存，但都不意味着跨设备的缓存自动保持一致。第 7.3.3 节比较这两种方式，第 7.4.1 节说明数据何时可见、操作何时算完成。

**控制器的位置。**传统网卡是一个 PCIe 外设，第 6.4.4 节已经跟踪过这条发起路径。处理器发起一次远程读取时，门铃、读描述符、目标端读数据、写回数据、写完成记录一共五次穿越 PCIe，每次都要几百纳秒。UB 把控制器接在处理器的片上总线上：处理器执行一条 Load 指令，请求经片上总线直接到达控制器，返回的数据直接写入寄存器。图 6-39 对比了这两条发起路径。第 7.3.3 节把两条路径逐阶段相加：线路单程时延为 100 ns 时，经 PCIe 外设网卡的一次 64 B 读取约需 2.2 µs，其中五次 PCIe 穿越占约 1.65 µs；经片上总线控制器的读取约需 0.42 µs。节省的时间不是把原有阶段压缩得到的，而是因为处理器与控制器共用一个地址空间，那些阶段不复存在。去掉抽象层，让处理器像访问本地内存一样访问远端内存，正是 UB 收益的来源；剩下的 0.42 µs 主要由线路时延和远端访存决定，已经接近硬件允许的下限。

**互联规模与片上状态容量。**网卡把每个通信关系的上下文缓存在片上，这里取缓存容量为 256 KiB。设互联内有 $H$ 台主机，每台主机有 $A$ 个应用端点，任意两台主机的端点之间都可能通信。连接状态有两种组织方式。第一种是**逐对连接**，以 RoCE（在以太网上承载的 RDMA）的可靠连接为代表：本机的每个端点与其余 $H-1$ 台主机上的每个端点各建立一条连接，每条连接保存 512 B 的上下文。第二种是 UB 的**端点加通道**：本机每个端点保存一份 20 B 的 Jetty 记录和 32 B 的内存段记录，每台远端主机对应一条 56 B 的传输通道。两种方式下每个网卡保存的状态分别为

$$S_{\text{pair}}(H) = 512A^2(H-1) + 32A, \qquad S_{\text{UB}}(H) = 52A + 56(H-1).$$

[39]李博杰，〈Unified Bus 背后的思考〉，Jetty、事务序与 Load/Store 各节；OpenURMA 论文，2026-06-02 修订版（arXiv:2605.28717），§3 设计、§7–§9 状态与延迟、§11 全系统验证、§13 结果汇总。本次整合的来源与适用范围。

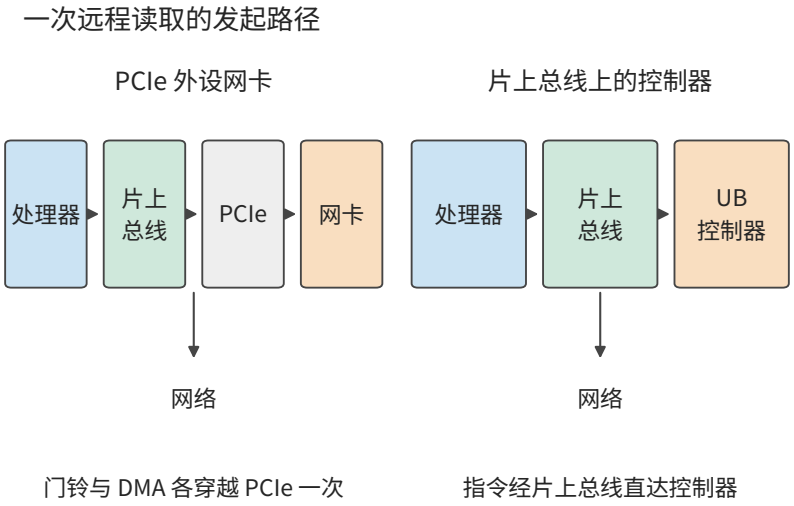



图 6-39：左图的网卡接在 PCIe 之后，门铃、请求描述符、返回数据和完成记录在发起端的处理器与网卡之间各穿越 PCIe 一次，目标端网卡读取数据时再穿越一次；右图的控制器接在片上总线上，处理器的指令直接到达控制器。两条路径从控制器到网络的部分相同。

取 $A = 8$。逐对连接每增加一台主机，状态增加 32 KiB，256 KiB 的缓存只能容纳 8 台主机；端点加通道每增加一台主机只增加 56 B，同样的缓存可以容纳四千多台主机。以 CloudMatrix384 的 192 台主机计，逐对连接需要约 6 MiB，是缓存容量的 24 倍；端点加通道只需约 11 KiB。缓存无法容纳的状态，每次操作都要从主机内存重新读取，代价在第 7.4.2 节计算：逐对连接的每次操作要增加两次 PCIe DMA 读取，约 1 µs。

第三种组织方式是缓存一致的互联，NVLink 属于这一类：加速器可以把对端内存中的数据放进自己的缓存。为了让各个副本保持一致，需要一个目录记录每个缓存行被哪些对端持有，每行至少要为每个对端留 1 bit。设目录跟踪 $W = 2^{20}$ 个缓存行（64 MiB 数据），每行另有 8 B 标签，则目录占用 $W\,((H-1)/8+8)$ B：两台主机时约 8 MiB，1024 台时约 136 MiB。每次写入还要向所有持有副本的对端发送失效消息，消息数随 $H$ 线性增长。这两项开销把缓存一致互联的规模限制在几十个对端，NVL72 的 72 张 GPU 正处在这一范围。UB 不维护跨主机的缓存一致，数据何时对其他设备可见由应用自己安排（第 7.4.1 节），所以三条状态曲线中只有它能延伸到上千台主机。图 6-40 画出了这三条曲线。

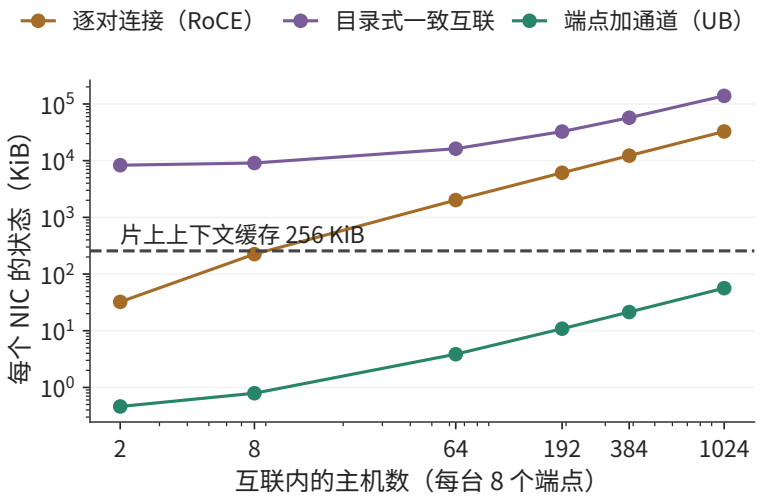



图 6-40：每台主机 8 个端点时，三种组织方式下每个网卡的状态随互联内主机数的变化。虚线是 256 KiB 的片上缓存。逐对连接在 8 台主机时超出缓存，目录式缓存一致互联从两台主机起就远超缓存，端点加通道到上千台主机仍在缓存之内。

**连接建立时间。**状态的数量也决定了作业启动时要执行多少控制操作。逐对连接的每条连接要经过创建和三次状态迁移，共 4 次系统调用，还要与对端交换一次连接编号，需要一次带外往返（经数据通路以外的通道往返一次）。取一次系统调用 5 µs、一次带外往返 500 µs，建立一条连接约需 0.52 ms。$N$ 个本地端点访问 $M$ 个远端端点，需要建立 $NM$ 条连接。端点加通道只需为 $N$ 个端点各执行 1 次系统调用，为 $M$ 台远端主机各执行 1 次系统调用和 1 次往返。$N = M = 1024$ 时，逐对连接要建立 1,048,576 条连接，串行约需 545 s，32 个核并行也要约 17 s；端点加通道只需创建 2048 个对象，并行后约 16 ms。图 6-41 画出了 $N$ 从 1 到 1024 的全过程。作业每次重启或扩缩容都要重复这一过程，所以它直接影响第 6.7.2 节讨论的故障恢复时间。

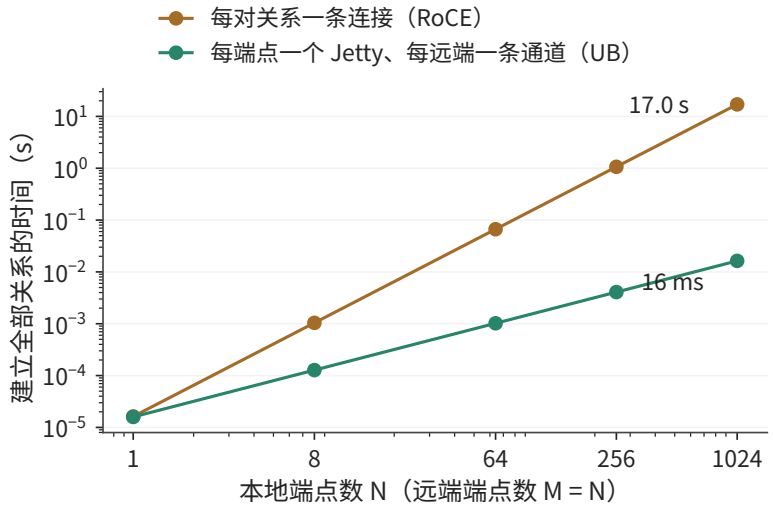



图 6-41：在 N 个本地端点与 N 个远端端点之间建立全部连接所需的时间，32 个核并行执行。逐对连接的时间随 $N^2$ 增长，端点加通道随 N 增长；$N = 1024$ 时二者相差约一千倍。

这些 UB 机制在具体系统上的体现是华为的 CloudMatrix384：该系统将 384 个昇腾 910C NPU 与 192 个鲲鹏（Kunpeng）CPU 通过 UB 交换结构组织起来，超节点外使用 RDMA 网络；业务接入由虚拟私有云（VPC）网络承担，用于隔离和连接租户资源。若 384 个 NPU 两两各铺一条直连，需要 $384 \times 383/2 = 73\,536$ 条连接；交换层通过端口连接和路由转发实现设备间通信。[40]

这些系统都可以采用第 6.3.1 节的 TP2×EP4 并行方案，但分组与物理边界的对应方式不同。张量并行组内的频繁归约适合放在低延迟连接内，不同专家组之间交换专家结果时，则需要足够的跨组带宽。把算法上的通信组映射到物理交换层，才能确定每次汇合占用了哪些端口。

用接口带宽计算传输时间时，要分清单向带宽和双向合计带宽。例如昇腾 950 的 UB 双向合计线速为 2016 GB/s，按 72 条 lane、每条 112 Gbit/s 计，单向为 1008 GB/s。UBoE 是在以太网上承载 UB 通信的方式，UB Link 是 UB 自己的链路方式。两者按预先配置共用同一组串行器 / 解串器（SerDes，把芯片内的并行数据与链路上的串行信号互相转换的电路），所以要把同一组物理通道分配给两种用途。分配方式决定任务实际可用的带宽。[41]

**实验 6-8 ・延伸：用通信需求解释超节点的互联结构**

（a）选择 DGX-1、DGX-2 或 DGX A100，画出一个通信组经过的直连或交换层。

（b）复算 $4^3$ 与 $8^3$ 环面的芯片数和均衡二分链路数，解释为什么每芯片可用的跨分区带宽下降。

（c）将 TP2×EP4 放到两台各有四张卡的主机上，标出跨主机的专家组归约。

（d）把每台主机的端点数改为 64，求 256 KiB 缓存在逐对连接方式下还能容纳几台主机；再把缓存扩大到

[40] CloudMatrix384 论文 v2§3.2、图 2、表 1；论文表中的部分 NPU 测量按 die 计，系统 384 则按 NPU 计。

[41] UB 与昇腾 950 资料核对、UB OS 参考设计§4、950 官方白皮书§4.6。

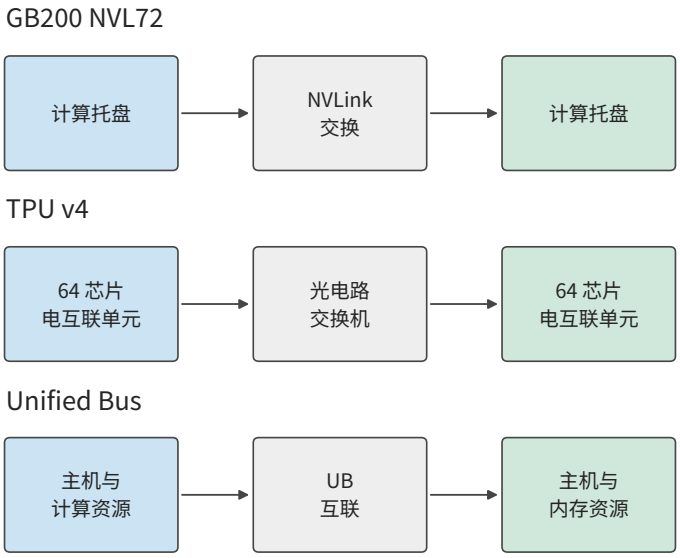



图 6-42：三个公开系统的连接层次。NVIDIA 用 NVLink 连接设备，TPU v4 用 OCS 连接电互联单元，Unified Bus 将主机侧计算和内存资源接入统一互联。

1 MiB，判断 CloudMatrix384 的 192 台主机是否放得下。

(e) 把一次带外往返改为 50 µs，重新计算 $N = M = 1024$ 时两种方式的连接建立时间，说明哪一种对往返时延更敏感。

(f) 选择 UB 或 TPU 的一个设计，说明安装、通信或资源访问中的哪项需求促成了这种系统结构。

## 6.6 内存池

### 6.6.1 局部容量不足与全局空闲

UB 把资源访问扩展到主机之间后，就有了具体用途：让内存不足的节点借用其他节点的空闲内存。第 6.2 节的张量并行、流水线并行和专家并行把计算和权重分到不同的卡上，降低每张卡的内存需求。另一种办法是计算位置不变，只把一部分数据放到别的节点。这就是内存池的基本用途：把分散的空闲容量交给需要它的任务。

考虑四个节点，每个节点用一张 80 GB 的 H100，任务需求分别为 100、60、40、40 GB。总需求 240 GB，小于总容量 320 GB，但任务 0 的需求比所在节点的容量多出 20 GB。其余节点的空闲分别为 20、40、40 GB，合计 100 GB。

把任务 0 整体搬到另一节点，其需求仍超过单节点的 80 GB 容量。若将它的 20 GB 数据放到节点 1 的空闲区，则物理占用变为 80、80、40、40 GB。任务 0 的工作集仍是 100 GB，其中 80 GB 本地、20 GB 远端。这样，任务 0 就能利用其他节点原本空闲的内存。

需要区分借用前后的空闲容量：其他节点原有 100 GB 空闲内存，其中 20 GB 借给任务 0 后，四个节点合计还剩 80 GB。借用没有增加物理内存，只是让任务能够使用其他节点的空闲空间。笔者早期关于 UB 内存池的构思，关注的正是这种服务器之间的不均衡；KV 的保存与复用后来成为它的具体用途。

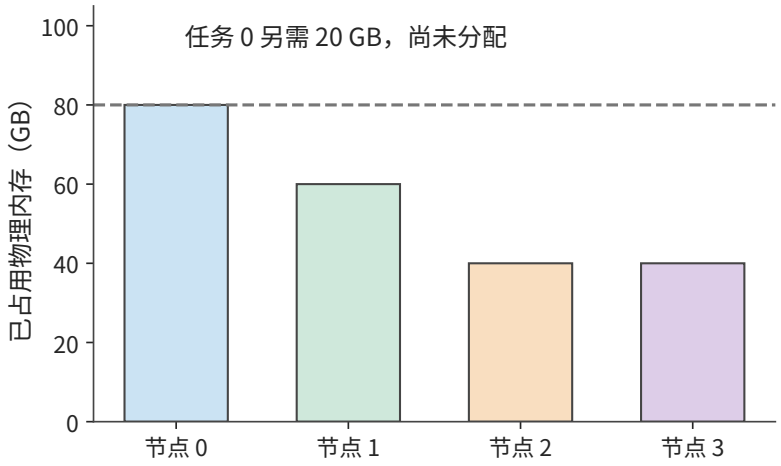



图 6-43：借用前的物理占用。每节点容量为 80 GB，任务 0 的需求为 100 GB，其中 20 GB 尚未找到存储位置。颜色表示数据所属任务。

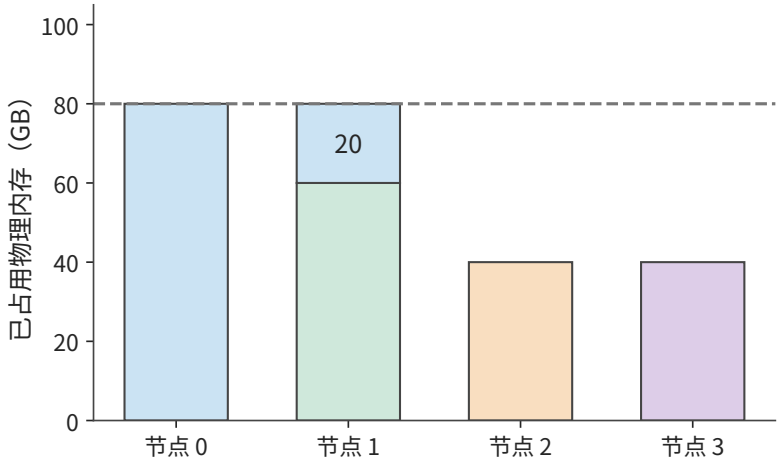



图 6-44：把任务 0 的额外 20 GB 放在节点 1。节点 1 的绿色 60 GB 属于任务 1，蓝色 20 GB 属于任务 0；每个节点都没有超过 80 GB。

### 6.6.2 内存借用与访问边界

图中的 20 GB 已经有了存放位置，下一步是让节点 0 正确访问这段内存。借用首先要明确一段内存由谁提供、由谁使用。提供内存的节点开放一段区域，借用节点建立相应的地址映射，系统负责管理访问权限，并记录这段内存何时可以释放和重新分配。独占借用期间，使用权交给借用者，原持有者等待归还后再使用；共享访问则允许多个使用者按约定的一致性与同步规则访问。[42]

例如节点 1 把一段内存借给节点 0，节点 0 要先写入新 KV，再开始注意力计算。只有确认写入完成、能够读到新的 KV，才能开始计算；节点 1 重新分配这段区域，则要等待借用结束。地址回答“访问哪里”，权限回答“谁能访问”，操作完成与内存释放的顺序回答“什么时候可以使用”。地址、权限和访问顺序三者合在一起，远端内存才能用得正确。

[42]UB 与昇腾 950 资料核对、UB OS 参考设计§4、950 官方白皮书§4.6。

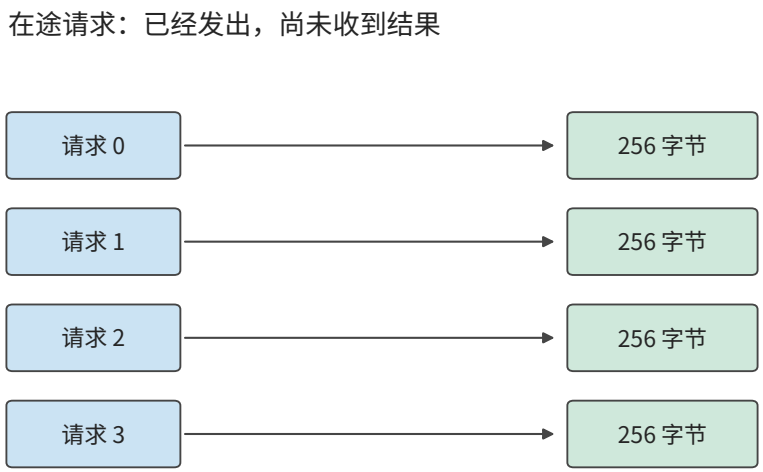



图 6-45：一批读取请求发出后，要经过往返时间 L 才能收到结果。最多有 u 个请求在途、每个返回 q 字节时，在一个往返时间内最多返回 uq 字节。图中四个请求仅用于展示过程；正文算例采用 128 个请求，每个 256 字节，往返时间为 7.52 µs。

权限和完成顺序保证数据正确，图中的等待过程则决定读取速度。请求发出后要过一段时间才有结果；要让链路在等待期间保持忙碌，就要同时发出多个请求。设每个读事务返回 $q$ bytes，从发出到完成需要 $L$，最多允许 $u$ 个事务同时在途（已发出、尚未完成），则每 $L$ 秒最多返回 $uq$ bytes。把路径带宽和在途事务数的限制放在一起，可得

$$B_{\text{window}} = \frac{uq}{L}, \qquad B_{\text{usable}} = \min\left(B_{\text{path}}, \frac{uq}{L}\right). \tag{6-12}$$

设两个节点经 ConnectX-7 网卡的 RDMA 路径相连，路径带宽为 50 GB/s。一次读请求从发出到完成要走一个来回，取 MSCCL++ 在 H100 与 ConnectX-7 平台上测得的 InfiniBand 单向延迟 3.76 µs 的两倍，即 $L = 7.52$ µs。每个读事务取 $q = 256$ bytes，恰好是 Qwen3-32B 一个 KV 头在一层、一个 token 上的 K 向量（128 维 BF16）。若 $u = 128$，每批在途数据为 32 KiB，在途事务数量决定的带宽上限约为 4.36 GB/s。要达到路径的 50 GB/s，至少需要

$$u \geq \left\lceil \frac{50 \times 10^{9} \times 7.52 \times 10^{-6}}{256} \right\rceil = 1469.$$

只提高线速而仍只允许 128 个事务在途，有效读取带宽仍受在途事务数限制；放宽到 4096 个事务，在途事务数决定的上限约为 139 GB/s，超过路径带宽，瓶颈就转到 50 GB/s 的路径本身。路径带宽、事务大小和往返时间通过在途数据量联系起来。

### 6.6.3 共享容量与 KV 缓存

能否用远端内存替代本地内存，还要把第 6.6.2 节求出的读取带宽与应用的读取需求比较。同样是 20 GB 远端数据，读取频率决定所需的平均带宽。每分钟完整读取一次，平均约需 0.33 GB/s；每秒一次需要 20 GB/s；每秒二十次需要 400 GB/s。容量相同，所需的平均读取带宽相差上千倍。

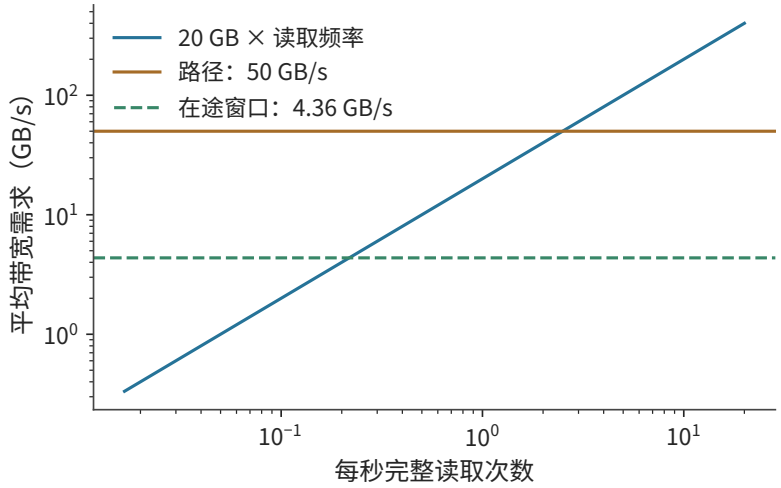



图 6-46：完整读取 20 GB 的频率与平均带宽需求。路径带宽为 50 GB/s；每事务返回 256 bytes、往返 7.52 µs 时，128 个在途事务将有效带宽限制到约 4.36 GB/s。

在途事务数量充足时，经 50 GB/s 的路径完整读取一次约用 0.40 s。每秒发起一次读取，两次读取之间有约 0.60 s 的空闲时间；每 0.05 s 发起一次，上一次尚未结束，新工作已经到来，队列不断增长。若最多只能同时发出 128 个事务，读取变为约 4.59 s，每秒一次也会逐渐积压。[43]

KV 的使用方式决定其适合放在哪里。会话等待用户下一轮输入时，可以长时间不读 KV，这时远端内存主要用来保存状态，会话恢复时再取回；活跃会话每一步都要读上下文状态，反复从远端扫描会持续占用带宽。状态本身有多大则取决于模型的上下文表示：第 9.2.2 节将并列比较同样 8192 个 token 的上下文在 Qwen3-8B 的 GQA 和 DeepSeek-V3 的紧凑 MLA 两种设计下的字节数与取回时间，后者约为前者的一半。

也可以先取回，再在本地重复使用。若同一份 20 GB 快照要读十次，逐次经 50 GB/s 路径读取需要约 4.0 s；先取回一次用 0.40 s，再按 H100 本地 HBM 的 3350 GB/s 扫描十次约用 0.06 s，共约 0.46 s。后者占用 20 GB 本地内存空间，节省了约 3.54 s 的访问时间。因此，对需要反复读取的数据，还要比较远端读取与本地缓存的时间，决定什么时候把数据取回本地。

**实验 6-9 • 延伸：远端内存何时应直接读取，何时应缓存到本地**

(a) 复算四节点借用前后的物理占用，分别计算其他节点在借用前的空闲容量，以及全部节点在借用后的剩余容量。

(b) 每个事务传输 256 bytes，往返时间分别为 7.52 µs 和 15.04 µs。求两种情况下达到 50 GB/s 带宽所需的最少在途事务数。

(c) 比较 128 与 4096 个在途事务时的单次读取时间，假设前一次读取完成后才发起下一次，求连续读取时每秒最多能完成多少次完整读取。

(d) 采用本节远端与本地带宽，求重复读多少次时，先取回本地的总访问时间更少；同时说明所需本地容量。

## 6.7 从并行方案到超节点规模

### 6.7.1 更大的协作组与更多推理实例

本节用前几节的结果，在一台 HGX H100 的八张卡上安排多个 Qwen3-32B 会话。四个会话都已完成 prefill，各自保存了 131,064 个上下文 token 的 KV。每个会话还有一个已经生成、尚未写入 KV 的 token，用作下一步

[43] 第 6 章内存池计算的 `memory_pool` 字段：四个 80 GB 节点的借用前后占用、窗口带宽 $uq/L$、读取时间与重复读取的比较。路径带宽取 ConnectX-7 的 400 Gbit/s，往返时间取 MSCCL++ 表 1 的 InfiniBand 单向延迟 3.76 µs 的两倍，本地带宽取 H100 的 3350 GB/s。

输入。四个会话同时请求再生成八个 token，正好写满 YaRN 支持的 131,072 个位置。一个实例一次服务一个会话，完成这八步后再接下一个会话；请求按实例轮转分配。八张卡都在同一台服务器内，一直分配给这组任务，直到四个会话都完成。

每步时间来自式（6-5）。续写时，式（6-4）的上下文长度依次为 131,064 到 131,071，KV 访问量逐步增加。将八步相加，就得到会话的服务时间。等待中的会话只保存各自的 KV，同一实例中的会话共享模型权重。容量先排除了八个单卡实例：一张卡连一个 128K 会话都无法容纳。按第 6.2.2 节的算法，两卡实例最多能容纳两个会话，四卡实例能容纳七个，八卡实例能容纳十六个；三种部署中每个实例分到的会话分别为一、二、四个，均可容纳。

| 部署方式 | 一个会话的八步 | 四会话完成时刻 | 全部会话完成 |
|---|---|---|---|
| 八个单卡实例 | 无法容纳 | — | — |
| 四个两卡实例 | 119.1 ms | 119.1、119.1、119.1、119.1 ms | 119.1 ms |
| 两个四卡实例 | 63.8 ms | 63.8、63.8、127.6、127.6 ms | 127.6 ms |
| 一个八卡实例 | 41.2 ms | 41.2、82.4、123.5、164.7 ms | 164.7 ms |

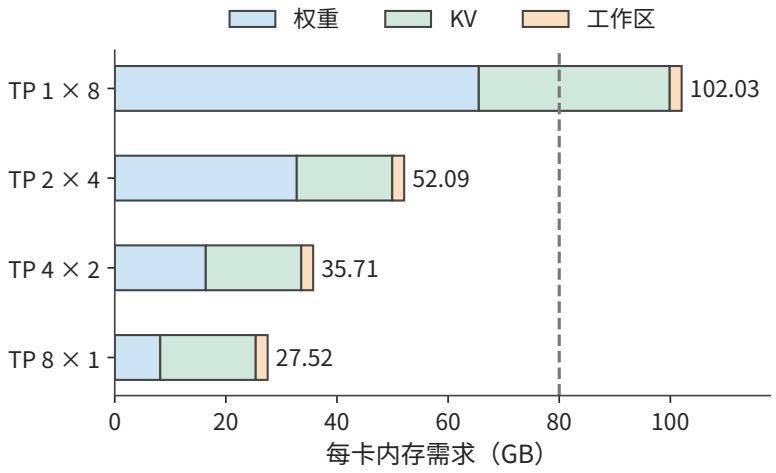



图 6-47：四种部署在 128K 会话下的每卡内存需求，包括权重、实例内全部排队会话的 KV 和 2 GiB 工作区。单卡实例每卡要 102.03 GB，超过 H100 的 80 GB（虚线）；两卡、四卡、八卡实例分别约为 52.09、35.71、27.52 GB。

八卡协作使单个会话的续写时间从约 119 ms 缩短到 41 ms，但四个会话排成一队，所有会话约 165 ms 才完成。采用四个两卡实例时，单个会话的续写时间较长，却能同时推进四个会话，约 119 ms 全部完成。单卡实例本可以开八个，却连一个会话都无法容纳。

这里同时用到了两种并行：模型内部的并行计算和不同请求之间的并发执行。层内矩阵可以切给多张卡，独立会话可以分给多个实例，两者竞争的是同一批八张卡。分清这两层关系，才能按请求的完成期限选择协作组的大小。服务调度给出同时推进的会话数，模型切分给出每步通信量，互联拓扑决定数据交换与同步的耗时，三者共同决定表中的完成时刻。

图中色块的横向长度表示单个会话的执行时间，纵向轨道数表示多少会话可以同时推进。完成期限决定了这两者如何取舍。只有一个会话、要求 50 ms 完成，八卡实例满足目标；四个会话、要求 130 ms 内全部完成，两卡和四卡部署方式都满足，两卡更早完成。单会话期限越短，越需要更大的协作组；独立会话足够多时，则可以利用更多实例。

连接方式改变后，仍沿原式计算。若像第 6.5.1 节那样用两台服务器的 16 张卡组成 TP16，每步约 6.67 ms，

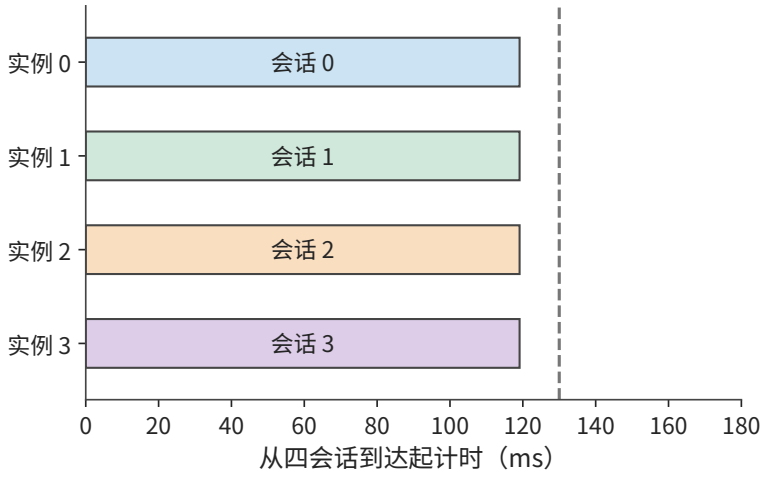



图 6-48：四会话同时推进，每个约 119.1 ms 完成。颜色与相邻配置图中的会话一致。本图使用四个 TP2 实例，每实例两张卡，每个会话续写八个 token；虚线为 130 ms 期限。

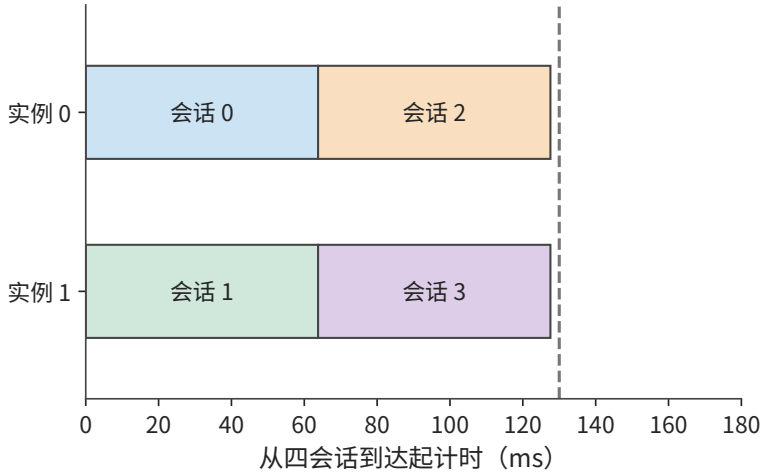



图 6-49：每个实例顺序处理两个会话，分别在约 63.8 ms 和 127.6 ms 完成。本图使用两个 TP4 实例，每实例四张卡，每个会话续写八个 token；虚线为 130 ms 期限。

八步约 53.3 ms，50 ms 的目标就无法达到。第二台服务器更适合运行其他实例；若一定要让一个会话用上更多的卡，就要降低新增的通信开销：改善服务器之间的连接、改变集合通信算法，或者把这些卡放进更紧密的互联域。

第 4 章的固定权重架构会改变上述取舍。权重由独立的 ROM 提供后，原来分摊给多卡的 HBM 权重读取减少，TP 的同步时间在单步耗时中占比上升。减少 TP 卡数能够省去一部分同步，腾出的卡可以组成更多独立实例；每张卡则要容纳更大份额的请求状态。本节的逐卡容量表与会话时间表，可以继续用来比较这两项变化；第 6.7.4 节给出具体数值。

把同样的分析用于状态更紧凑的模型。计算 DeepSeek V4.1 需要多少张卡时，除了每 token 890 B 的逻辑状态容量，还要看状态如何分布在各卡上。该数值对跨层共享的状态只计一份；如果各层放在不同的卡上，后面的层仍要访问生成这些状态的层所保存的数据。可以把相关的层放在相近的卡上，在目标卡上复制状态，或者建立远程访问路径。第一种限制了层的放置，第二种增加存储和更新开销，第三种增加通信依赖，所以要按每张卡实际保存和访问的数据分别计算资源需求。[44]

[44]DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

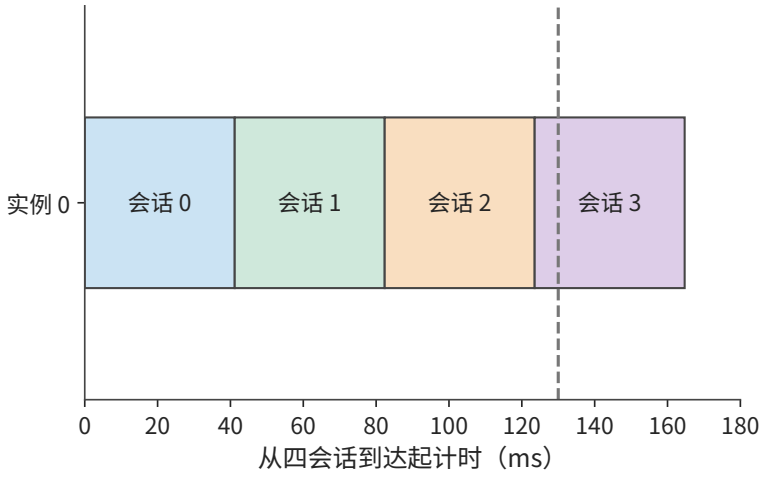



图 6-50：单会话缩短到约 41.2 ms，四会话依次执行，最后一个约 164.7 ms 完成。三图横轴使用同一尺度。本图使用一个 TP8 实例，共八张卡，每个会话续写八个 token；虚线为 130 ms 期限。

### 6.7.2 功率、成本与故障范围如何改变选择

时间线给出了每个会话何时完成，也给出了八张卡需要占用多久，因此可以直接据此计算成本，并比较按时完成同一组请求要占用多少资源。成本以 H100 的 GPU · s 计，即一张卡占用一秒。八张卡从四个会话同时到达时开始计费，直到所有会话完成。完成期限为 130 ms，要求四个会话中至少三个按时完成。

四个两卡实例的总成本约为 $8 \times 0.1191 \approx 0.953$ GPU · s，四个会话全部按时完成，平均每个按时完成的会话约 0.238 GPU · s。两个四卡实例的总成本约为 1.021 GPU · s，四个会话也都按时完成，平均每个约 0.255 GPU · s。一个八卡实例在期限内完成三个会话，刚好达到最低比例，但八张卡要占用到 164.7 ms，总成本约 1.318 GPU · s，平均每个按时完成的会话约 0.439 GPU · s。八个单卡实例无法容纳会话。

以按时完成的会话数为分母，平均成本定义为

$$C_{\text{on time}} = \frac{C_{\text{occupied}}}{N_{\text{on time}}}. \tag{6-13}$$

占用成本按全部工作持续的时间计，其中包括故障停顿与重做的时间；分母只数期限内完成的会话。失败、重做和超时请求的成本都在分子里，所以都分摊到了按时完成的会话上。

以上比较假定各卡一直正常运行。若一张卡发生故障，包含这张卡的实例会暂停，分配给该实例的后续会话也会推迟。沿同一时间线注入一次故障：20 ms 时卡 0 失效，包含它的实例停顿 40 ms，60 ms 恢复后从该会话八步续写的起点重做；其他实例继续执行，不受这次故障影响。

四个两卡实例中，另外三个会话仍在约 119 ms 完成，受影响的会话在 $60 + 119.1 \approx 179.1$ ms 完成，按时比例为 75%。两个四卡实例中，未受影响的实例在约 64 和 128 ms 完成两个会话，受影响的实例在约 124 和 188 ms 完成另外两个会话，按时比例同样是 75%。八卡实例的四个会话依次在约 101、142、184、225 ms 完成，130 ms 内只完成一个。

四个两卡实例继续满足最低比例，成本增为 $8 \times 0.1791 \approx 1.433$ GPU · s，平均每个按时完成的会话约 $1.433/3 \approx 0.478$ GPU · s，是无故障时的 2.0 倍；两个四卡实例约为 0.500 GPU · s。故障被隔离在一个实例内，另外三个会话仍能按时完成；但占用时间延长、按时完成的会话减少，每个会话的成本仍翻了一倍。

故障影响范围取决于实例是否共用设备。两个实例若共用同一电源或交换机，这台共用设备发生故障时，两

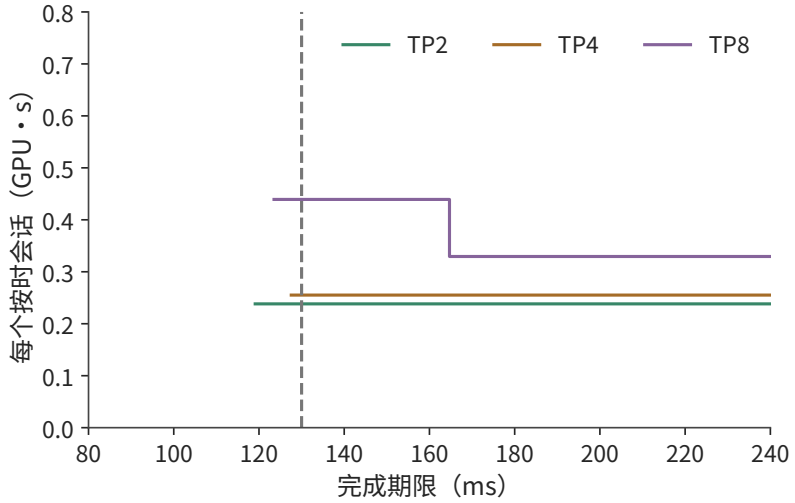



图 6-51：无故障时，按时完成会话的平均成本随期限变化。八卡计费至所有会话结束；曲线从至少三个会话按时完成处开始。TP 数值为每实例使用的卡数；成本以 H100 的 GPU · s 计。TP1 放不下 128K 会话，没有曲线。

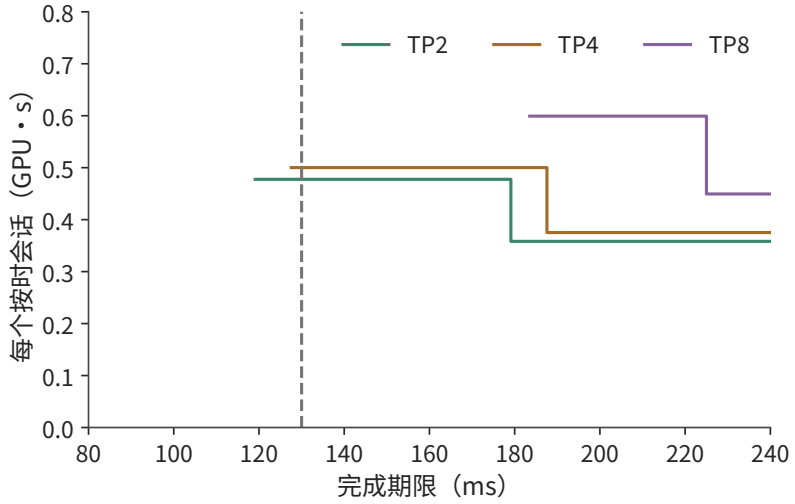



图 6-52：20 ms 时卡 0 故障，受影响实例在 60 ms 从当前会话起点重新执行。采用与无故障图相同的计费方法和纵轴范围。TP 数值为每实例使用的卡数；成本以 H100 的 GPU · s 计。

个实例都会受到影响；拆分实例只有配合资源隔离，才能保留其他实例的进度。一个实例使用的设备越多，请求执行就依赖越多设备正常工作。因此，划分实例时也需要考虑电源和交换机的共用情况。

还可以用功率计算这些部署方式的能耗。按第 6.5.2 节的 DGX H100 系统功率上限 10.2 kW 计，无故障时，四个两卡实例运行约 119.1 ms，消耗约 1.21 kJ；两个四卡实例运行约 127.6 ms，消耗约 1.30 kJ。硬件相同、功率相同时，越早完成任务，消耗的电能就越少。

模型的内存需求会限制可用的部署方式。本例中先被排除的是 TP1：128K 会话的 KV 让单卡需求达到 102.03 GB。上下文缩短到 32K 时，单卡可以容纳一个会话，但八步续写要 173.3 ms，仍超过 130 ms 的期限，这时 TP1 是被时间排除的。先找出内存容量足够的部署方案，再比较这些方案的执行时间和成本。[45]

[45]逐会话时间线、故障与 GPU · s的 `candidates` 与 `candidates_32k` 字段，由计算脚本生成；四组部署条件的筛选见切分选择结果。能耗按 DGX H100 系统功率上限 10.2 kW 计，见第 6 章计算结果。

**实验 6-10 ・核心：按会话完成期限与成本选择并行配置**

（a）从式（6-4）、（6-5）逐步求四种 TP 配置的八步续写时间，再计算四个会话各自的完成时刻。

（b）分别采用 100、130、170 ms 期限，要求至少 75% 会话按时完成，找出满足要求的部署方案并比较成本。

（c）改用两台 HGX H100 上的 TP16，跨服务器的 AllReduce 取实验 7-3 的实测值，重新计算单会话续写时间，判断能否满足 50 ms 的完成期限。

（d）复算故障后的会话完成情况和成本；将停顿从 40 ms 改为 10 ms，解释满足要求的部署方案怎样变化。

（e）把上下文改为 32K，加入功率条件，说明容量、时间和功率各自先排除了哪种部署方案。

### 6.7.3 给定模型与硬件，如何选择切分策略

回到章首的问题。切分策略的选择可以写成有输入、有约束的优化问题：可选方案 $\pi$ 包括并行度、物理放置、通信算法、micro-batch 划分和局部算子实现，先确定目标 $J$，再寻找

$$\pi^* = \arg\min_{\pi \in \mathcal{P}_{\text{feasible}}} J(\pi).$$

可行集 $\mathcal{P}_{\text{feasible}}$ 里的每个方案都要满足每卡容量、模型切分的数据依赖、kernel 支持和服务期限。目标 $J$ 可以是单个请求的延迟，也可以是上一节的每个按时完成会话的成本；若目标是吞吐，就在相同的服务目标（SLO）下比较能持续接纳的请求率。负载和目标不固定，“最优并行度”就没有唯一的答案。

**第一步，记录模型、负载与资源。**模型侧需要层数、矩阵维度、注意力头与 KV 头、专家数和每 token 选中的专家数、精度、共享权重及跨层状态。负载侧需要 prefill 长度、decode 上下文、同时就绪的请求、路由分布、目标延迟与到达过程。硬件侧需要逐卡可用 HBM、不同形状下的有效算力和带宽，以及节点内、节点间每方向的路径、启动开销与共享瓶颈。第 4 章给出硬件参数，第 5 章给出实际形状与局部执行时间；不能把集群的总带宽平均分给每个通信组。

**第二步，按瓶颈列出可选方案，而不是先选一个缩写。**下表列出各种瓶颈下应当考虑哪些方案；它只用来缩小搜索范围，最终仍要比较完成时间。

| 当前约束或瓶颈 | 优先考虑的方案 | 必须补算的代价 |
|---|---|---|
| 权重无法容纳，或小 batch 重复读权重过慢 | TP、PP；MoE 可用 EP | 归约、阶段串行、专家流量与工作区 |
| 独立请求多，完整模型已能容纳 | 更多 DP / 服务副本，较小 TP 组 | 排队、批内复用、每副本 KV 容量 |
| 特定区间的激活重复存储 | 同一 TP 组内启用 SP | 布局边界的收集与归约，kernel 支持 |
| 长上下文的 KV 与注意力占主导 | CP 与 TP 的组合 | K、V 交换、统计量归约、因果注意力的负载不均 |
| 多层权重容量大，跨节点归约昂贵 | 节点内 TP ＋跨节点 PP | micro-batch 是否足够、最慢阶段与流水空档 |
| 专家集合大、路由热点突出 | EP、专家副本、专家分离（路由专家放到独立的专家资源池） | dispatch / combine 两个方向的流量、最忙的卡与专家池的队列 |

**第三步，为每个方案写出数据归属和时间表。**给每张卡标出该卡保存哪些层、头、专家、激活和 KV；算峰值内存时还要加上集合通信缓冲、双缓冲、padding、在途 micro-batch 和副本迁移的空间。然后为每个算子求本地计算与访存时间，为每次跨卡操作求轮数、字节数与共享路径，把两者按依赖关系排到时间线上。同一个 token 的前后层不会因为流水线并行而同时完成，同一个 token 选中的专家结果也不会因为网络空闲而提前就绪。MoE 要代入最忙的专家组而不是平均值，第 9.4 节给出具体的 skew（负载偏斜）模型。

**第四步，在可行方案之间按同一目标排序。**本章的八卡例子固定为稠密的 Qwen3-32B 与 128K 会话，每个

实例一次只处理一个会话；首轮比较的是 TP1、TP2、TP4、TP8，其余卡组成独立实例。专家并行不适用于稠密模型；序列并行只改变一段激活的存储，在这一逐 token、工作区固定的算例里没有额外收益；流水线并行若不交错多个会话，单个会话仍要依次经过全部阶段，所以没有把它当作缩短单会话时间的首选。上下文并行、跨会话流水和连续批处理（在每步迭代的边界重新安排活跃请求）可以进入扩展搜索，但要分别补上上下文归约、阶段排程和 batch 变化的时间模型，不能套用张量并行的 $1/p$ 缩放。

前两节已经给出逐会话时间线和成本。按这些完整数值筛选，可以直接得到下面的选择；改变目标、物理边界或容量后，选择也随之改变。[46]

| 给定条件 | 约束内的选择 | 为什么 |
|---|---|---|
| 四会话、128K、130 ms 内至少完成 75% | 四个 TP2 实例 | TP1 无法容纳；TP2 全部约 119.1 ms 完成，每个按时会话约 0.238 GPU · s，低于 TP4 的 0.255 与 TP8 的 0.439 |
| 单会话、128K、50 ms 期限 | 一个 TP8 实例 | 约 41.2 ms；TP4 约 63.8 ms，已超过期限 |
| 两台 HGX H100 共 16 卡，单会话、128K、50 ms | 仍选一个 TP8 实例 | 跨服务器的 TP16 按实测 AllReduce 约 53.3 ms，超过期限；另一台服务器可以运行其他实例 |
| 四会话、32K、130 ms 内至少完成 75% | 四个 TP2 实例 | TP1 可以容纳，但八步要 173.3 ms；TP2 约 88.3 ms 全部完成，每个按时会话约 0.177 GPU · s |

**第五步，用测量修正估算，不把估算的排名当作结论。**对排在前面的方案，用相同的请求、路由和质量条件做测量，记录峰值内存、各阶段的就绪与完成时刻、各端口的收发量和服务的尾延迟。若实际的矩阵效率、通信争用或排队与估算不同，换上实测参数后重新排序；若两个方案的差距小于测量波动，就都保留，再用更有代表性的负载验证。最后交付的应包括选定的方案、被排除的方案、会让结论翻转的阈值和实测范围。配套脚本可以复算表中的四组条件，但只枚举声明过的方案，不是覆盖任意模型和框架的全局规划器。

不同设计选择影响不同的时间开销。改变 TP 主要改变每卡计算量、内存访问量和参与归约的卡数；改变算法主要改变轮数与发送量；改变拓扑主要改变共享路径与固定等待；增加实例主要改变独立会话的排队顺序。将这些变化代入同一个执行时间模型，就能比较不同设计的性能。

进一步优化时，可以对式（6-5）作敏感性分析。在 HGX H100 八卡、小数据量的算例中，NVLink 带宽翻倍只节省约 2.5 µs，而每轮启动开销减半节省约 0.74 ms；因此，应优先减少每轮通信的启动开销。MoE 若受负载最重的专家组限制，均衡分派直接减少该组的计算量；读取远端 KV 时，若带宽受在途事务数量限制，增加事务并发才能充分利用已有链路带宽。这样就能判断应当优先减少哪项开销、增加哪种资源。

### 6.7.4 超节点扩大与权重、Engram 表、KV 的放置：V4.1 Flash 的综合算例

前三节讨论的是一台服务器内的八张卡如何分给几个实例。本节转而改变超节点本身：先看把超节点放大能换来什么，再按第 4.7.3 节的思路把权重固定进 ROM，最后讨论 Engram 表和 KV 应当放在哪里。算例仍以第 2 章的 DeepSeek V4.1 Flash 为对象，任务是 200K 上下文的 decode。硬件为 256 张 H100 SXM，超节点分别取 8、64、128、256 卡，每个推理实例恰好占用一个超节点。实例内部，注意力按数据并行放置，每张卡只处理并保存自己会话的注意力与 KV；路由专家和 Engram 表按专家并行均分到各卡，每层各做一次 dispatch 和

[46] 切分选择输入、方案筛选脚本、完整结果。使用本章的执行时间模型，按实例内全部排队会话的峰值上下文计算 KV 容量；只在明示的 TP 与实例数组合内排序。

combine。[47]

**超节点大小与每卡会话数。**每张卡上的权重分两部分：注意力、共享专家和输出头每卡各存一份，共 9.5 GB；路由专家 288.8 GB 与 Engram 表 203.1 GB 由实例内的 $S$ 张卡分担。扣除权重和工作区 $U$，其余 HBM 全部留给 KV，每个会话占 $K = 180.7$ MB，于是

$$W_{\text{card}} = 9.5\ \text{GB} + \frac{288.8\ \text{GB} + 203.1\ \text{GB}}{S}, \qquad N_{\text{session}} = \left\lfloor \frac{C - W_{\text{card}} - U}{K} \right\rfloor. \tag{6-14}$$

步时间仍按式（6-2）估算：本卡的权重和本卡全部会话的 KV 各读一遍，读取与计算取时间较长的一项，再加上每层一次 dispatch 与一次 combine，共 80 次 All-to-All。四种超节点的结果见下表。

| 超节点 | 每卡权重 | 每卡会话 | 步时间 | 每卡吞吐 |
|---|---|---|---|---|
| 8 卡 | 71.0 GB | 37 | 14.1 ms | 2,632 token/s |
| 64 卡 | 17.2 GB | 335 | 17.3 ms | 19,420 token/s |
| 128 卡 | 13.4 GB | 356 | 18.4 ms | 19,392 token/s |
| 256 卡 | 11.5 GB | 367 | 18.9 ms | 19,378 token/s |

超节点只有 8 卡时，每张卡保存每层 48 个专家，一步之内几乎全部会被选中，读取专家权重占去了步时间的绝大部分。扩大到 64 卡，每卡分到的专家减为八分之一，专家读取时间降到原来的十分之一以下，腾出的 HBM 又能多容纳九倍的会话，每卡吞吐因此提高 7.4 倍。再往上扩大就没有收益：专家读取已经很小，而 KV 读取、计算和 All-to-All 都随会话数同比例增长。这 7.4 倍主要来自容量。若把 Engram 表移到主机内存，8 卡超节点每卡也能容纳 178 个会话，与 64 卡的差距缩小到 1.9 倍。

单个用户的速度则与超节点大小无关。batch 为 1 时，一个 token 的关键路径是所在卡读一遍 8.2 GB 每卡各存一份的权重，再逐层读取一个专家，共 2.76 ms，即 362 token/s，8 卡与 256 卡完全相同。放大超节点得到的是容量和总吞吐，不是单个会话的速度；要缩短单个会话的时间，只能像第 6.7.1 节那样把注意力也做张量并行。反过来，同样 64 张卡若分散在八台 HGX H100 上，dispatch 和 combine 有八分之七的字节要经过网卡，通信时间从 3.7 ms 增至 29.2 ms，每卡吞吐降到 7,828 token/s。超节点省下的正是这一段。

**把权重固定进 ROM。**图 6-54 画出权重与 KV 的三种放置方式。GPU 把两者都放在 HBM，每一步都要重新读一遍权重；第 4.7.3 节的架构把部署期间不再变化的权重写进掩模 ROM，HBM 只保存 KV；KV 还可以进一步放进片上 SRAM。

后两种方式按 OpenTallas 对 V4.1 Flash 的分析结果计算：两片 N5 工艺的 ROM 晶圆，同样是 200K 上下文、batch 为 1。权重与 KV 各有自己的通路，两项读取和计算可以重叠，集合通信则要等三者都完成才能开始：

$$T_{\text{token}} = \frac{\max(T_{\text{weight}}, T_{\text{KV}}, T_{\text{compute}})}{0.9} + T_{\text{comm}} + T_{\text{fixed}}. \tag{6-15}$$

0.9 是因为两片晶圆分到的层数不可能完全均匀，$T_{\text{fixed}}$ 是每层固定的流水延迟。GPU 一行的权重与 KV 共用 HBM，两项读取时间相加。

[47]固定场景、计算脚本与完整结果。权重按锁定的 V4.1 Flash checkpoint 分片头逐张量累加；KV 与矩阵运算量分别由 `kv-comparison` 与 `v41-forward` 在 200,000 上下文下算出；H100 参数取自硬件表；NVLink 与网卡的每轮启动开销与第 7.6.3、7.6.4 节相同。ROM 晶圆与 58 张 B200 两行取自作者的 OpenTallas 仓库 commit c7093ba 中 DeepSeek-V4.1-Flash 的 roofline 分析，所用分析点保存在摘录中。这些是等硅面积下的分析结果：OpenTallas 尚无流片的芯片，其 ROM 密度与读取带宽在 N5 工艺上也未经测量。Engram 表的行数、每行字节与两个模块的查表行数取自同一 checkpoint 与配置；DeepSeek 的部署把表放在主机内存并用 RDMA 预取，见 V4.1 技术报告§2.4.2 与 §3.1.3；主机内存一次往返与在途标签限制的读取率取第 7.3.4 节 KV-Direct 的测量；ROM 面积按 OpenTallas 的 N5 掩模 ROM 密度换算，表移出 ROM 后的一片晶圆设计取自同一 commit 的 engram-host 分析。WSE-3 的 44 GB 片上 SRAM 见第 4.7.2 节。

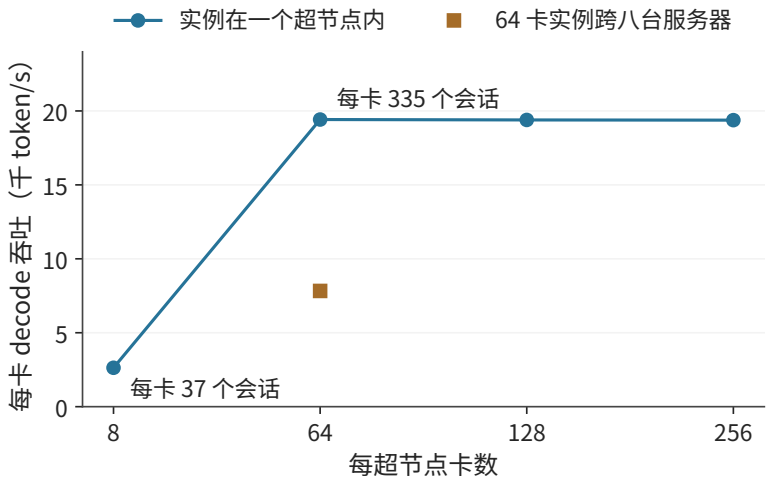



图 6-53：V4.1 Flash 在 256 张 H100 上的每卡 decode 吞吐随超节点大小的变化，服务目标为每 token 50 ms。蓝线：实例都在一个超节点内，8 卡到 64 卡提高约 7.4 倍，此后不再增长；橙点：同样 64 张卡分散在八台服务器上组成专家并行组。

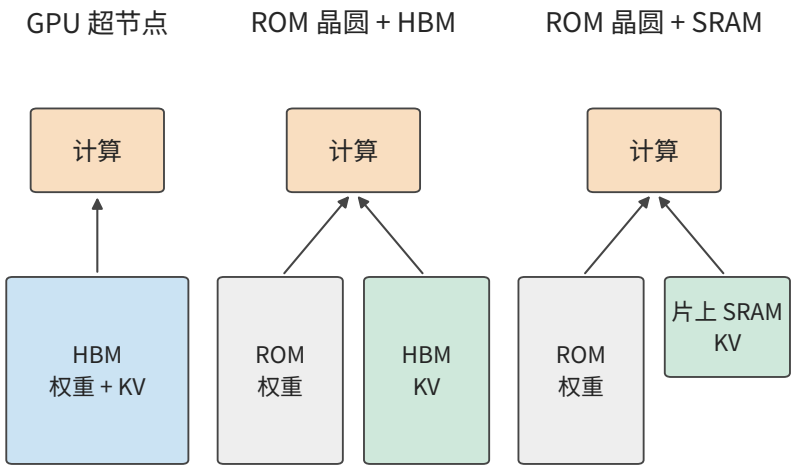



图 6-54：权重与 KV 的三种放置方式。左：GPU 的权重与 KV 共用 HBM；中：权重在制造时写入 ROM，KV 放 HBM；右：权重在 ROM，KV 放片上 SRAM，容量小得多。箭头是每步都要读取的数据，方框高度示意容量。

| 机器 | 存储与计算 | 集合通信 | 单用户速度 | 通信占比 |
|---|---|---|---|---|
| 8 张 H100，权重在 HBM | 2,689 µs | 76 µs | 362 token/s | 3% |
| 58 张 B200 张量并行，权重在 HBM | 165 µs | 562 µs | 1,357 token/s | 76% |
| 两片 ROM 晶圆，权重在 ROM | 77 µs | 159 µs | 4,070 token/s | 65% |

权重离开 HBM 之后，权重读取从 2.7 ms 缩短到 69 µs，但 80 次片上 AllReduce 仍需 159 µs，占每个 token 时间的 65%。换成 58 张 B200 做张量并行，权重读取也能压到 148 µs，通信却增至 562 µs。权重读取一旦不再是最长的一项，单用户速度就由集合通信决定，第 6.5.1 节按跳数和每跳延迟估算通信的方法在这里成了关键。

**Engram 表放在哪里。** V4.1 Flash 有两个 Engram 模块，各带一张 3.84 亿行的表，每行 256 个 FP8 值加 8

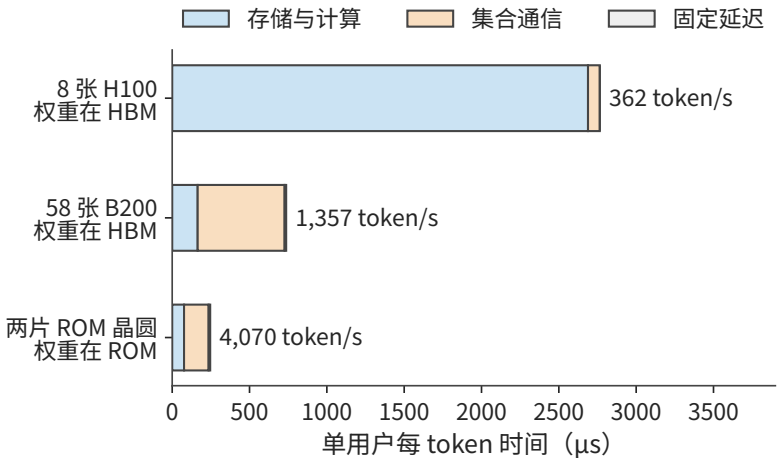



图 6-55：三种机器上 V4.1 Flash 单用户每 token 时间的构成，200K 上下文，batch 为 1。权重在 HBM 时存储读取占绝大部分，权重进 ROM 之后集合通信成为最长的一项。

字节 scale，两张表共 203.1 GB。每生成一个 token，两个模块各查 24 行，共 12.7 KB。这两张表同样是权重，部署后不再改变；要查哪些行只取决于 token 序列，与激活无关，因此每一步开始时就能预取。只要在第一个 Transformer 块算完之前把这些行取回，查表就不占用时间；可用于取回的时间在 H100 上是 69 μs，在 ROM 晶圆上只有 6.1 μs。图 6-56 画出三种放法，代价如下。

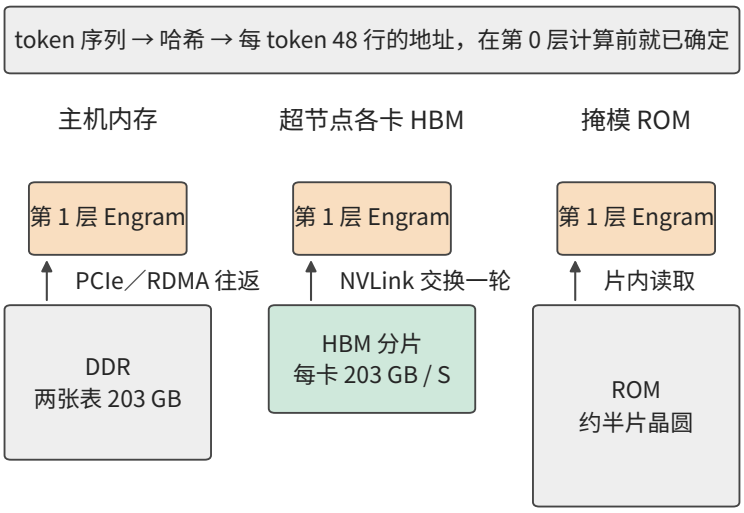



图 6-56：Engram 表的三种放置方式。要查哪些行在第一个 Transformer 块计算之前就已确定，三种放法的差别在于取回这 48 行要走什么路径，以及表占用哪一种存储。

| 放置 | 占用的容量 | 取回 48 行 |
|---|---|---|
| 主机内存，经 PCIe 或 RDMA 预取 | 不占 HBM | 一次往返 1.05 μs |
| 超节点各卡的 HBM 分片 | 8 卡每卡 25.4 GB，相当于 140 个会话；64 卡每卡 3.2 GB，相当于 18 个会话 | 一轮 NVLink 交换 0.83 μs |
| 掩模 ROM | 21,651 $mm^2$，约半片晶圆 | 片内读取 |

在 H100 上，两条取回路径都远短于 69 μs，放在哪里只需看容量。8 卡超节点容纳不下这张表，移到主机内存后每卡的会话数从 37 个增加到 178 个；64 卡起每卡只分到 3.2 GB，放在 HBM 也只减少 5% 的会话。批

量服务时主机内存也不是瓶颈：64 卡实例每卡每步 16,080 次随机读，按第 7.3.4 节实测的每秒 6100 万次计只需 0.26 ms，占步时间的 1.5%。DeepSeek 的部署正是把表放在主机内存，在第一个 Transformer 块计算的同时用 RDMA 预取。

到了 ROM 晶圆上，取舍随之改变。第一个块只有 6.1 µs，一次主机往返就占去六分之一，步时间一旦短于 42 µs 便无法掩盖；把表刻进 ROM 要多用半片晶圆，换来的只是每个 token 读 12.7 KB；放在晶圆边缘的 HBM 只占 1,124 个会话的空间，读取也不经过互联。表移出 ROM 之后，checkpoint 一片晶圆即可容纳，单用户速度反而升到 5,037 token/s。机器越快，这张表越应当放在离计算最近的可写存储里。

**把 KV 放进 SRAM。**片上 SRAM 比 HBM 快得多，但容量先要足够。V4.1 Flash 的 checkpoint 有 510.3 GB，即使用第 4.7.2 节 WSE-3 那样 44 GB 的片上 SRAM，仅权重就需要 12 片晶圆，权重放 SRAM 的方案首先被容量排除。权重进了 ROM，晶圆上剩余的 SRAM 只够保存一个 200K 会话的 KV；KV 改放 HBM，同样两片晶圆可以同时驻留 9,637 个会话。两种设计的单用户速度相同，总吞吐却相差三个数量级。单个会话的速度由关键路径上的存储和通信决定，能同时服务多少会话由可写状态的容量决定，这正是第 6.7.1 节区分的两层并行。

三步用的是同一套方法：先按式（6-1）检查容量，再按式（6-2）逐项估算时间，最后找出关键路径上最长的一项。放大超节点提高的是吞吐，单个会话不会更快；权重固定之后，互联的每跳延迟成了单用户速度的上限；Engram 表这样只读不写的大表，机器越快越要放在离计算近的地方；SRAM 的容量决定能同时服务多少会话。

**实验 6-11 ·延伸：超节点、ROM 与 SRAM 的综合算例**

（a）按式（6-14）复算表中各行的会话数，把服务目标改为每 token 20 ms，求各超节点大小下每卡能服务的会话数。

（b）把实例内的注意力改为 TP8，其余不变，重算 batch 为 1 的单用户速度，与 362 token/s 比较。

（c）把跨服务器的每轮启动开销从 5 µs 改为 2 µs，并给每卡配两张网卡，判断跨服务器的 64 卡实例能否追平超节点内的实例。

（d）把片上每跳延迟减半，按式（6-15）重算 ROM 晶圆一行的单用户速度和通信占比。

（e）把上下文改为 1M，重算 KV 放 SRAM 时能驻留的会话数，以及 KV 放 HBM 时的会话数。

（f）把主机内存的一次往返改为 2 µs，求 H100 和 ROM 晶圆各在多快的单用户速度下，Engram 查表开始占用步时间。

权重、状态或服务需求继续扩大时，单个超节点可能不足以支撑一个实例的运行。下一章沿着本章分析的数据传输路径和依赖关系，加入网卡、数据中心交换网络、远程操作的完成通知与网络拥塞，继续计算扩大协作范围所带来的收益和代价。

## 本章小结

多卡并行延续了单卡算子的分块与调度。沿样本、序列位置、特征、层和专家切分，每张卡持有的数据和依赖各不相同，数据并行、张量并行、序列并行、上下文并行、流水线并行和专家并行就是这些切法的常见组织方式。切分产生交换，调度决定何时发起、何时汇合，算法和物理路径共同决定代价。选方案时先检查容量和执行的合法性，再用完整负载下的延迟、吞吐或成本比较各方案。

并行收益随规模递减：每卡承担的计算量与内存访问量逐渐减少，串行处理的耗时仍然存在，归约轮次和服务器间通信等待继续消耗时间。批内复用、热点分布、在途事务和物理割集又分别改变数据传输速度与最慢阶段的执行时间。分析这些因素时，从数据的产生、持有和下一次使用入手，就能说明每项开销如何影响后续计算。UB 的算例说明：连接状态按端点数相加还是按端点对相乘，决定了片上缓存能容纳多少台主机、建立全部连接要多长时间；控制器接在片上总线上还是 PCIe 之后，决定了每次访问要穿越多少层。

在本章的 128K 续写任务中，单卡无法容纳一个会话，八卡实例的单会话续写时间最短，四个两卡实例则

能更早完成全部四个会话；故障、上下文长度和跨服务器路径又改变可用的部署方案。超节点规模最终服务于明确的任务：提供足够的内存容量，在期限内完成请求，并降低单位请求的资源成本。

# 第 7 章 数据中心网络

上一章把矩阵、层和专家分给不同的卡，并按通信代价选择实例规模。现在模型和总卡数都不变，只把一部分卡移到另一个超节点：原来的切分还合适吗？原本走节点内高速互联的部分和，可能要改走 InfiniBand 或 RoCE；同一次同步因此占用不同的带宽、队列和故障域。要重新算的是切分的放置和执行代价，不是重新学一套并行概念。

模型分到多台服务器之后，训练要继续推进，第一项共同工作就是汇合各副本的梯度。多个数据并行副本处理不同样本后，要汇合梯度，再按同一批样本更新参数。先在超节点内合并各副本的梯度，再跨节点交换，与第 5 章先在局部累加部分和、减少外部搬移的做法相同。局部合并本身也需要时间，只有节省的远端传输和等待多于这些时间，这样安排才划算。通用数据中心网络承载大量相互独立的数据流，拥塞带来的延迟分摊在各条流上；同步归约却要所有卡等待同一批数据，每一轮何时结束取决于最慢的那次传输（第 7.2.1 节）。

本章沿着数据离开一台服务器的路径，研究分到多台服务器上的卡如何共同完成工作。全章围绕两台服务器、每台八张卡展开：先决定梯度如何跨服务器归约，再讨论如何连续提交和处理通信请求，随后说明数据何时可读、缓冲区何时可复用，以及如何处理共享网络中的拥塞，最后把这套分析推广到固定 1024 卡的训练任务，比较不同超节点大小下的通信分层、吞吐与恢复成本。本章还会比较低并发推理中的小数据量传输，说明同一个网络在不同负载下应当优先优化什么。

全章使用三种分析模型。**流量与资源模型**计算数据经过各条链路和接口时需要的时间，找出瓶颈；**并发与吞吐模型**解释充分利用带宽需要同时处理多少请求；**依赖与关键路径模型**用来判断哪些操作可以同时执行，以及缩短哪项操作能够减少总耗时。书中的几个设计案例来自笔者参与的工作：KV-Direct 让可编程网卡直接处理键值存储请求；1Pipe 用网络提供全局操作顺序，简化分布式程序的协作；UB 提供统一的设备互联。笔者参与这些研究和设计时，一直在思考三个问题：如何减少数据搬移，如何在等待一次操作时执行其他工作，如何在简化编程的同时不让系统停摆。

本章用 GB/s 表示十进制带宽，用 KiB、MiB、GiB 表示二进制容量，1 GiB $= 2^{30}$ bytes。

## 7.1 从超节点走向集群

### 7.1.1 存储需求如何引出跨服务器协作

模型需要的存储空间超过单机容量，是跨服务器协作的直接原因之一。以采用混合精度 Adam 训练的 8B 参数模型为例。**混合精度**是在训练中为不同对象选用不同数值精度。若参数、梯度、FP32 主权重以及优化器的一阶矩和二阶矩合计占用 16 bytes / 参数，就需要 128 GB。一张 80 GB 卡无法容纳，两张卡合计 160 GB，扣除这些状态后剩下 32 GB 留给激活和工作区。参数增加到 284B 时，采用相同的数据表示，这些状态共需约 4.5 TB，必须分散到更多的卡上存储和处理。[1]

推理不需要保存梯度和优化器状态，但大模型的权重仍可能需要多台服务器才能容纳。将约 1.6T 参数的 DeepSeek V4-Pro 和约 2.8T 参数的 Kimi K3 按每参数 0.5 byte 存放，权重分别约为 800 GB 和 1.4 TB。一台八卡、每卡 80 GB 的服务器合计 640 GB，仅存放权重就至少分别需要两台和三台服务器。MoE 的路由器为每个 token 只选一部分专家来执行，减少的是这一次前向的计算量；未被选中的专家，权重仍需存放。

[1] 模型规模与状态大小的计算方法见模型资源计算及第 7 章扩写资料。混合精度 Adam 算例按每参数 16 bytes 保存训练状态；万亿参数推理例子按每参数 0.5 byte 计算权重容量。

将权重分开存放之后，程序还要把各部分计算连接起来。按层切分，需要在阶段之间交接激活；在一层内部切分，需要汇合局部计算结果；让多份模型处理不同样本，需要汇合梯度。因此，即使模型相同，采用不同的切分方式，也会产生不同的通信需求。

为了比较这些通信方式，本章用两台 HGX 规格的 H100 服务器组成十六张卡的通信组，每张卡上运行一个通信进程。这些进程共同执行集合通信，下文称为参与者，用 rank 区分。参与者 0 到 7 在服务器 A，8 到 15 在服务器 B。服务器内的八张卡经 NVSwitch 用 NVLink 全连接。每张 H100 的 NVLink 总带宽为 900 GB/s，这是收发两个方向的合计，即每个方向 450 GB/s。每张卡配一张 400 Gbit/s 的网卡（network interface card，NIC），每个方向 50 GB/s，经 PCIe 交换芯片与这张卡相连，不与其他卡共用。两台服务器上编号相同的网卡接到同一台交换机，这样一条从网卡到网卡的路径称为一条 **rail**。交换网络能承载两台服务器之间的双向流量。集合通信每轮的启动时间取 0.83 μs：公开的 nccl-tests 记录显示，在两台这样的服务器上，十六张卡对 16 B 到 128 B 的数据做一次 AllReduce 约需 25 μs；数据量如此之小，传输时间可以忽略，把 25 μs 平均到下一节环形 AllReduce 的 30 轮上，每轮约 0.83 μs。[2]

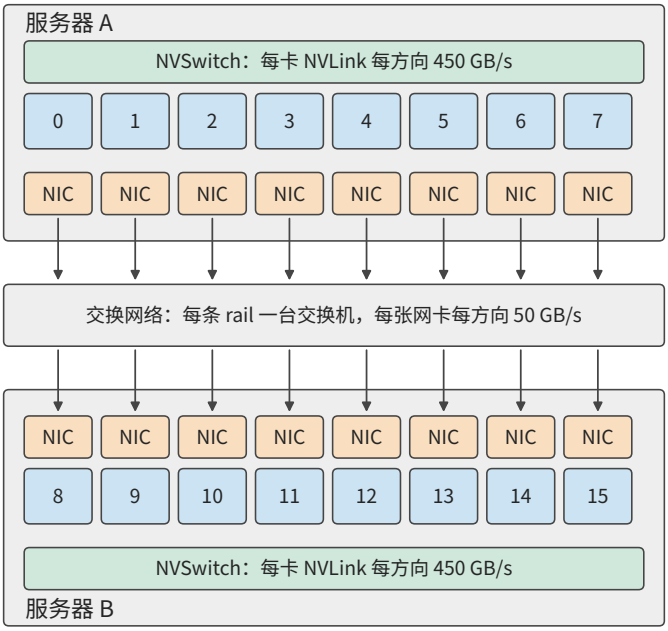



图 7-1：固定两台服务器，每台八张卡。服务器内的卡经 NVLink 与 NVSwitch 互联，每张卡各有一张网卡；跨服务器的数据从发送方的网卡经交换网络到达接收方的网卡，每张网卡每个方向提供 50 GB/s。

图 7-1 中的服务器边界，也是本章反复分析的通信边界。选择 192 MiB 的梯度张量，要求十六个参与者都得到其归约结果。例题 7.6（第 7.6.2 节）的训练步计算 20 ms，梯度在 17 ms 就绪，要让通信完全藏在计算后面，就要在 3 ms 内完成；本章把 3 ms 作为这次通信的预算。这一算例便于逐轮计算，也能说明跨服务器协作的主要困难：本地计算结果必须经过带宽远低于 NVLink 的网卡，才能传给另一台服务器。

网卡比 NVLink 慢得多，所以要先决定哪类通信放在服务器之间。本章沿用第 6.1.4 节的并行方式一览表，不再逐一定义。把频繁的张量并行归约留在较快的互联内，把数据并行的梯度同步或流水线并行的阶段交接放

[2]HGX H100 数据手册：八张 GPU 经 NVSwitch 互联，GPU 间 NVLink 900 GB/s，网络速率至 400 Gbit/s；NVIDIA H100 规格列出 H100 SXM 的 NVLink 900 GB/s 与 PCIe Gen5 128 GB/s，均为收发两个方向的合计；NVLink 规格页按代给出每 GPU 的 NVLink 带宽；ConnectX-7 数据手册：单端口至 400 Gbit/s，主机接口 PCIe Gen5 x16；DGX H200 数据手册：八张 GPU 配八张 400 Gbit/s ConnectX-7。每卡一张 400 Gbit/s 网卡的配置取自 experiments/ch07/07-03 的公开运行记录：两台 HGX 服务器各有 8 张 H100 与 8 张 ConnectX-7 400 Gbit/s InfiniBand 网卡。编号相同的网卡接同一台交换机的 rail 组网见 DGX SuperPOD H100 参考架构。PCIe Gen5 x16 每方向 64 GB/s 高于网卡的 50 GB/s，网卡是这条路径的限制。每轮 0.833 μs 的启动时间由同一份公开记录推得：原始日志中 16 rank 非原位 AllReduce 在 16 B 到 128 B 为 24.93–25.68 μs，取 25 μs，除以十六个参与者环形 AllReduce 的 30 轮；NCCL 在小消息上实际选用的算法未随日志公开，这里只按本章的环模型折算。

到服务器之间，是应当首先计算的一类候选方案；若本地容量不够，上下文并行需要更大范围的上下文，或者专家并行的路由跨越多个节点，通信范围就必须扩大。是否这样放置，要由实际流量和关键路径决定。

跨越超节点边界本身不会改变需要交换的数据量；算法、放置、精度和接收方是否去重决定要传多少字节，路径决定这些字节经过哪个出口、要等多久。只知道网络是 InfiniBand 还是 RoCE，推不出有效带宽、传输延迟、超售比（交换机下联与上联带宽之比）、在途窗口和故障恢复时间。后文用明确给定的网络参数计算，实际部署时换成对应系统的测量值。

> **实验 7-1 ・延伸：八卡服务器之间的模型切分**
> 计算上述两个推理模型的最低服务器数量，再加入每卡 8 GB 的 KV 与工作区。分别画出服务器内张量并行或专家并行、服务器间流水线并行的分工，以及跨服务器张量并行的分工，标出一次 prefill 与一次 decode 需要跨服务器交接的位置。

### 7.1.2 流量与资源模型

确定卡的位置和梯度大小后，再计算数据经过路径上每一段需要多久。一块数据从 GPU 内存出发，经加速器接口到达网卡，再经过交换网络，最后写入接收方的内存。同一块数据要依次经过这些接口和链路，而不同数据块可以在各段上同时传输，形成流水线。持续发送大量数据时，带宽最低的一段决定总吞吐率。例如，一张卡经 NVLink 每个方向能送出 450 GB/s，其网卡每个方向只能发送 50 GB/s，这张卡向另一台服务器的持续发送速率就受网卡的 50 GB/s 限制。

从单条路径扩展到整个集群时，可以先选择要分析的网络边界，再统计所有穿过它的数据。将网络节点分成两组，连接这两组节点的链路就是第 6 章的割集。若一次任务需要沿某个方向跨越割集传送 $V_{\mathcal{C}}$ bytes，而该方向的有效带宽为 $B_{\mathcal{C}}$，传输时间至少为

$$T_{\mathcal{C}} = \frac{V_{\mathcal{C}}}{B_{\mathcal{C}}}.$$

该式给出传输时间的下界：无论如何调度，链路每秒传的字节数都不会超过它的带宽。全双工链路的两个方向分别计算；若多个端口共享内部接口，再对该接口计算合计流量。同一份数据必须依次通过串联的各段链路，因此端到端带宽受最慢的一段限制，不能将各段带宽相加。

割集有多宽，取决于交换网络如何组织。**Clos 网络**由多层交换机构成，通过中间交换层为端点提供多条路径。在分层 Clos 网络中，叶交换机连接服务器，上层交换机提供叶交换机之间的多条路径。以 DGX SuperPOD 参考架构所用的 NVIDIA Quantum QM9700 InfiniBand 交换机为例，其每个端口按 InfiniBand 的 NDR 速率等级运行，为 400 Gbit/s，即每方向 50 GB/s；若一台叶交换机用 48 个端口接服务器、16 个端口接上层，下联总带宽为 2.4 TB/s，上联总带宽为 0.8 TB/s，两者之比 3:1 称为**超售比**。这些合计 0.8 TB/s 的上联链路，就是离开这台叶交换机的流量必须经过的割集。连接在同一叶交换机上的服务器可以直接交换数据；只有跨叶交换机的通信才占用上联带宽。

**例题 7.1：增加卡数的收益何时受跨服务器通信限制？** 把十六个参与者排成一个环做归约（第 7.2.1 节逐轮计算），每台服务器向另一台发送 360 MiB，这些字节全部经过一张网卡，每方向 50 GB/s。原来的卡数下计算时间为 20 ms。增加卡数时计算均匀分摊，跨服务器的传输量和路径不变。求完全串行和完全重叠两种安排的完成时间。

解答：一个方向的传输下界为

$$T_{\mathcal{C}} = \frac{360 \times 2^{20}}{50 \times 10^{9}} \approx 7.5 \text{ ms}.$$

令卡数相对原来的倍数为 $x$，计算时间变为 $20/x$ ms。串行安排需要 $20/x + 7.5$ ms；计算与通信完全重叠时，总时间等于两者中的较大值。卡数翻倍，两个结果分别从约 27.5、20.0 ms 降至 17.5、10.0 ms。增加到四倍，串行安排约为 12.5 ms，完全重叠安排约为 7.5 ms。图 7-2 画出两种安排随卡数变化的曲线。

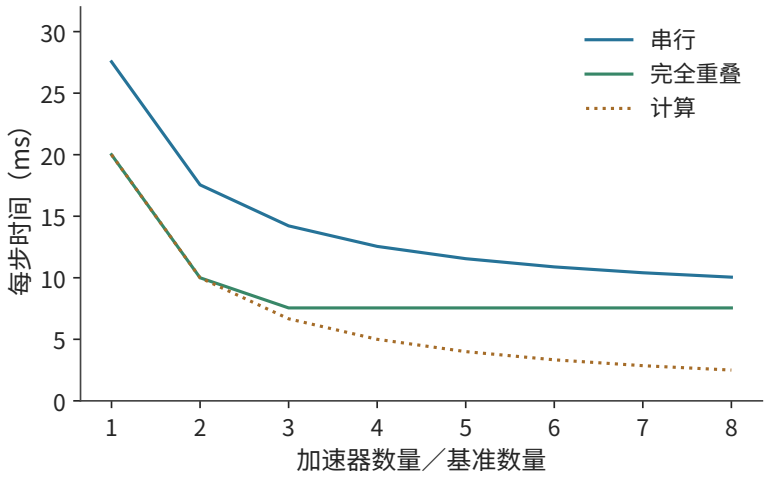



图 7-2：跨服务器传输量固定时增加卡数的效果。每方向发送 360 MiB，经一张 50 GB/s 网卡；计算时间从 20 ms 开始，随卡数增加按反比缩短。曲线分别按计算与通信串行、完全重叠计算，通信项取割集传输时间。

计算时间与传输时间相等时，$20/x = 7.5$，得到 $x \approx 2.6$。越过这一点后，计算时间已经短于传输时间，继续加卡也不能缩短完全重叠安排的时间。要继续缩短时间，就应减少穿过割集的数据量，或让更多网卡同时工作以提高割集的有效带宽。第 7.2 节将说明，重新组织同一次归约怎样同时做到这两点。

这种总任务固定、增加卡数的做法称为**强扩展**；**弱扩展**则随卡数增加而增加任务量。例如，若每个方向的传输量也翻倍而出口不变，通信时间就翻倍。扩展曲线的形状，取决于计算量和跨服务器传输量各自如何随卡数变化。

**实验 7-2 ・延伸：通信量与出口带宽如何限制扩容收益**

沿用例题 7.1，分别考虑每个方向的流量减半，以及让两张网卡同时工作使出口带宽翻倍这两种改动，分别画出串行与完全重叠两种安排下，完成时间随卡数倍数变化的曲线，并求计算耗时与通信耗时相等时的卡数倍数。再令流量随卡数倍数线性增长，求完全重叠安排下使完成时间最短的卡数倍数。

**Clos 网络的层数与半分带宽。**前文把叶交换机的上联当作割集。整个集群的割集有多宽，要从一台交换机的端口数算起。设每台交换机有 $k$ 个端口，每个端口每方向 50 GB/s，QM9700 的 $k = 64$。叶交换机（leaf）用 $d$ 个端口接服务器、$u$ 个端口接上层的脊交换机（spine），$d/u$ 就是超售比。无阻塞时 $d = u = k/2$。两层 Clos 中，每台脊交换机用 $k$ 个端口各接一台叶交换机，每台叶交换机的 $k/2$ 条上联分别接到 $k/2$ 台脊交换机，因此最多有 $k$ 台叶交换机、$k^2/2$ 个端点；三层按 fat-tree（胖树，各层总带宽不收窄的多层 Clos）的 pod 结构（若干叶交换机与一组中间层交换机组成一个 pod，pod 之间经顶层交换机相连）最多接 $k^3/4$ 个端点。$k = 64$ 时，两层接 2048 个端点、用 96 台交换机，正是 DGX SuperPOD 参考架构中 2048 张 GPU 所用的 64 台叶交换机加 32 台脊交换机；三层接 65536 个端点、用 5120 台交换机。[3]

把集群分成端点数相等的两半，连接两半的链路带宽合计称为**半分带宽**（bisection bandwidth）。无阻塞网络的半分带宽等于一半端点的带宽合计：两层 2048 个端点为 $1024 \times 50 = 51.2$ TB/s，三层为 1638.4 TB/s。超

[3]64 端口无阻塞 Clos、3:1 超售：叶交换机 $d$ 下行、$u$ 上联，上层对叶上联不阻塞，三层沿 fat-tree 的 pod 结构；半分带宽按顶层链路的一半计；1024 卡分区按占用整数台叶交换机计割集；超节点出口取1024 卡场景的每卡 50 GB/s 乘卡数再除以超售比，跨域字节取自1024 卡计算。交换机取 DGX SuperPOD H100 参考架构所用的 NVIDIA Quantum QM9700（NDR 400 Gbit/s），其表 3 中 2048 张 GPU 用 64 台叶交换机、32 台脊交换机，与 64 端口两层 Clos 的数目一致。运行 `python3 calculations/calc.py clos-cut --inputs calculations/scenarios/clos-cut-example.json --format md` 可以复算。

售比为 3 时，叶交换机 48 下行、16 上联，两层能接 3072 个端点，只用 80 台交换机；64 台叶交换机共 1024 条上联，半分带宽为其中一半的合计 25.6 TB/s，只有无阻塞网络中同样 3072 个端点所得 76.8 TB/s 的三分之一，每个端点穿过半分割集的份额从 50 GB/s 降到 16.7 GB/s。图 7-3 画出两层 Clos 网络及其半分割集。

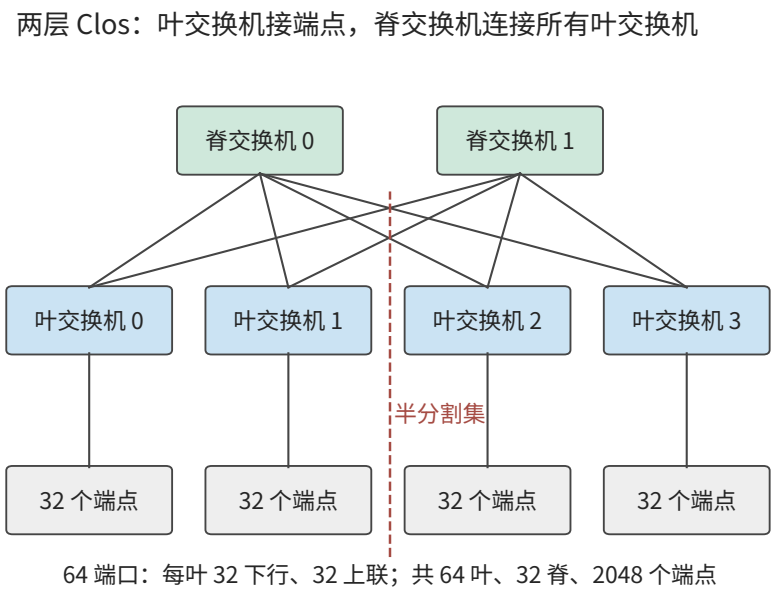



图 7-3：两层 Clos 网络。每台叶交换机一半端口接服务器、一半接脊交换机，任意两台叶交换机之间经任一脊交换机相通；虚线把端点分成两半，穿过它的叶—脊链路就是半分割集。图中画出四台叶交换机和两台脊交换机作代表，64 端口时实际为 64 台叶交换机、32 台脊交换机。

第 7.6.4 节的 1024 卡训练作业占用整数台叶交换机时，其割集就是这些叶交换机的全部上联：

| 超售比 | 每叶下行 / 上联 | 1024 卡占用的叶交换机 | 割集链路数 | 割集带宽 | 每卡份额 |
|---|---|---|---|---|---|
| 1:1 | 32 / 32 | 32 | 1024 | 51.2 TB/s | 50 GB/s |
| 3:1 | 48 / 16 | 22 | 352 | 17.6 TB/s | 17.2 GB/s |

超售比也改变第 7.6.4 节表中“出口随卡数增长”一列的取值。那一列把每超节点出口取为卡数乘 50 GB/s，隐含交换网络无阻塞；超售比为 3 时，出口只剩三分之一，跨域传输时间变为三倍：

| 超节点卡数 | 每节点跨域发送 | 无阻塞出口 | 3:1 超售出口 | 跨域传输时间（无阻塞 → 3:1） |
|---|---|---|---|---|
| 8 | 127 GB | 400 GB/s | 133 GB/s | 0.318 → 0.953 s |
| 64 | 120 GB | 3.2 TB/s | 1.07 TB/s | 37.5 → 112.5 ms |
| 128 | 112 GB | 6.4 TB/s | 2.13 TB/s | 17.5 → 52.5 ms |
| 256 | 96 GB | 12.8 TB/s | 4.27 TB/s | 7.5 → 22.5 ms |

表中是纯传输时间。第 7.6.4 节的跨域项还含 $2(H-1)$ 轮各 0.83 µs 的启动，128 卡超节点时 $H=8$，启动共约 12 µs，对表中的毫秒级时间几乎没有影响。

**讨论：超售的割集何时成为瓶颈？**超售比 $r$ 把每张卡穿过割集的份额降到 $50/r$ GB/s。若一张网卡发出的字节中只有比例 $f$ 要离开本叶交换机，上联不成为瓶颈的条件是 $f \le 1/r$：无阻塞时任何 $f$ 都不受限，$r=3$ 时只有不到三分之一的字节跨叶才不受限。分层归约的跨服务器阶段，配对的两台服务器若不在同一台叶交换机下（在第 7.2.5 节的多轨拓扑里，各服务器编号相同的网卡接同一台叶交换机，每 32 台服务器为一组，这种情

况就是两台服务器不在同一组内），每张网卡的全部字节都要跨叶，$f = 1$，超售的割集直接把这一阶段拉长到 $r$ 倍；两台服务器在同一组内时，第 7.2.5 节的对齐配对让每一对的字节只经过一台叶交换机，$f = 0$，上联不承载这次归约的任何字节。把同一同步组的服务器放进同一个组，就是降低 $f$ 的办法。

> **实验 7-3 ・延伸：交换机端口数与超售比如何决定割集**
> 把交换机端口数改为 128，分别求无阻塞两层与三层 Clos 的端点数、交换机数和半分带宽。保持 64 端口，把超售比改为 2:1，求 1024 卡分区的割集带宽和每卡份额，并求第 7.6.4 节 128 卡超节点的跨域传输时间。最后求超售比为 3 时，分层归约跨服务器阶段的每张网卡需要有多大比例的字节留在本叶交换机内，割集才不再是瓶颈。

### 7.1.3 统一互联

上述下界说明了物理链路的限制。程序还面临另一个问题：同一份数据在机内传输和跨服务器传输时，往往要用不同的访问接口。这和总线与网络长期以来的分工有关：紧耦合的互联把设备接到处理器和内存系统上，适合频繁、细粒度的协作；集群网络连接大量独立的节点，还要处理节点和链路故障。模型跨越服务器之后，同一项计算就同时用到这两类机制。一次本地访存和一次远程传输，在程序里表达的是相似的数据依赖，走的却是不同的提交和完成路径。

笔者参与 UB 早期设计时，核心问题之一就是如何为本地和远程协作提供统一接口。应用希望表达的是“读取这份数据”“把结果交给下一阶段”“等它完成后复用空间”。统一互联为寻址、数据访问和设备间通信提供共同的接口，让 CPU、加速器和其他设备都能参与其中。

理解这种统一，需要区分两件事。接口决定程序如何表达工作，物理路径决定工作经过哪些资源。一次远程读取可以用一条指令发起，但数据仍要经过网卡的 50 GB/s 出口；异步写入可以提前提交，接收方则要等数据就绪后才能开始使用。因此，本章将先求必需的流量，再求充分利用带宽所需的并发请求数，最后分析应用要求的操作顺序。统一接口的收益也在这三个层次上体现：减少搬移，缩短发起路径，简化协作。第 6.5.5 节已经从超节点规模的角度讨论了 UB 的两项设计选择：把控制器接在片上总线上，把连接状态拆成端点记录和传输通道分别保存。本章沿着一次远程访问的路径，逐阶段计算这两项选择带来的效果。

## 7.2 跨节点流量与物理路径

### 7.2.1 数据并行的跨服务器流量

统一接口不能省去必需的传输，但归约算法可以改变哪些数据需要跨服务器。先从数据并行开始：每个参与者算出局部梯度，再把对应元素相加，最后每个参与者都要得到完整的归约结果。算例选 Qwen3-8B 第一层 FFN 中 gate 投影的梯度，形状为 $[12288, 4096]$，按每元素 4 字节的 FP32 存放，每个参与者的输入为

$$M = 12288 \times 4096 \times 4 = 192 \text{ MiB}.$$

第 6.4.2 节推导过环形 AllReduce：先做 ReduceScatter，让每卡持有归约结果的一片，再做 AllGather 把这些分片收集到每卡；张量均分为 $p$ 份，两步各 $p-1$ 轮，每轮向下一个参与者发送一份 $M/p$。因此，每个参与者发送

$$V_{\text{rank}} = 2\frac{p-1}{p}M.$$

十六个参与者各发送 360 MiB，共发送 5760 MiB，执行三十轮。算法决定总发送量，参与者在服务器上的分布则决定其中多少需要跨服务器传输、经过哪些网卡。

**例题 7.2：哪种归约算法能在 3 ms 内完成传输？**使用第 7.1 节的两台服务器，比较不分层的连续环、交错环和分层归约。

**解答：计算各归约方案的跨服务器传输量。**连续环按 0、1、…、15 排列。十六条有向边只有 7→8 和 15→0 跨服务器。每轮每边发送 12 MiB，三十轮每方向各发送 360 MiB，两方向合计 720 MiB；这些字节在服务器 A 只经过参与者 7 的网卡，在服务器 B 只经过参与者 15 的网卡，其余十四张网卡闲置。若改成 0、8、1、9、…、7、15 的交错排列，十六条边全部跨服务器，每轮每张网卡各发送 12 MiB，跨服务器总量增至 5760 MiB。

分层归约先在各服务器内由八个参与者执行 ReduceScatter。七轮之后，每个参与者保存本服务器归约结果的八分之一，即 24 MiB。两台服务器上负责同一分片的两个参与者，再执行一次 AllReduce：各自把 24 MiB 分成两份，用一轮交换并归约其中一份，再用一轮交换得到另一份。每一对在两个方向合计发送 48 MiB，八对共发送 384 MiB；每一对各走自己的一条 rail，八张网卡同时工作。最后各服务器内用七轮 AllGather 得到完整结果。图 7-4 至图 7-6 分别画出三种方案的路径，图 7-7 比较三者跨服务器的字节数。

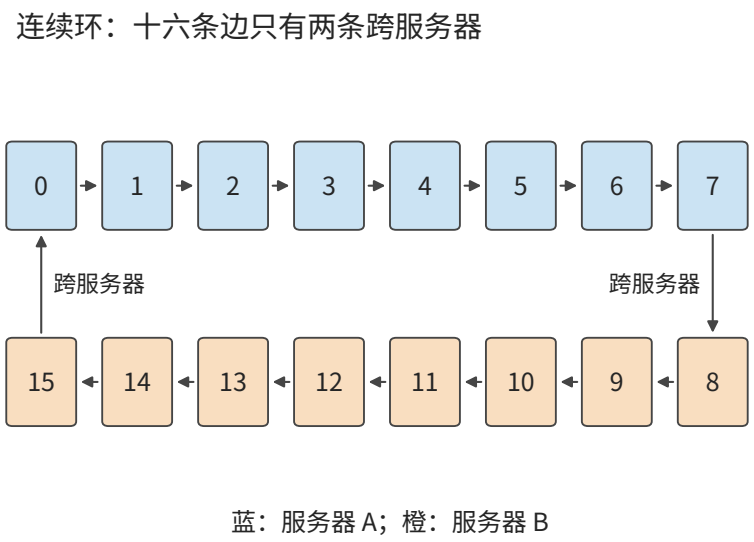



图 7-4：连续环将同一服务器内的参与者排在一起。十六条有向边中只有 7→8 与 15→0 跨服务器，跨服务器流量集中在两张网卡上。颜色表示参与者所在服务器。

**解答：由各轮链路负载计算归约时间。**各轮依次执行，同一轮中的不同传输并行进行。设第 $r$ 轮经过资源 $e$ 的数据量为 $V_{r,e}$，该资源带宽为 $B_e$，每轮启动时间为 $\alpha$，则传输模型为

$$T_{\text{comm}} = \sum_{r} \left( \alpha + \max_{e} \frac{V_{r,e}}{B_e} \right).$$

沿上述三种方案的箭头逐轮计算，就能把流量换算成时间。该式分开了两种关系：一轮之内由最慢的传输决定，各轮之间依次相加。这里只算数据传输和每轮启动的时间。[4]

[4]两级梯度归约计算、连续环、交错环。固定模型形状；各轮以屏障同步，所列时间为无争用条件下的载荷传输与启动时间之和：NVLink 每方向 450 GB/s、每卡一张 NIC 每方向 50 GB/s、每轮启动 0.833 µs。运行 `python3 calculations/calc.py hierarchical-gradient --inputs calculations/scenarios/hierarchical-gradient-example.json --format md` 可以复算。

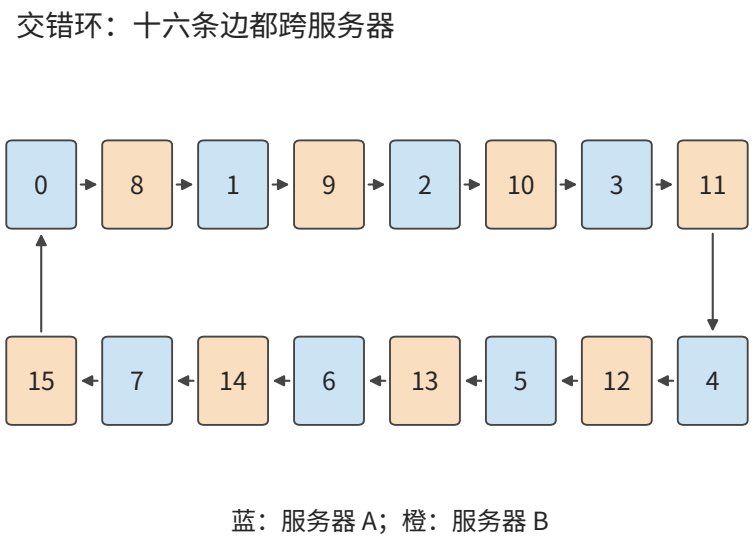



图 7-5：按 A、B 两服务器交替排列参与者，十六条有向边都跨服务器，十六张网卡都在工作。每个参与者的总发送量保持不变。

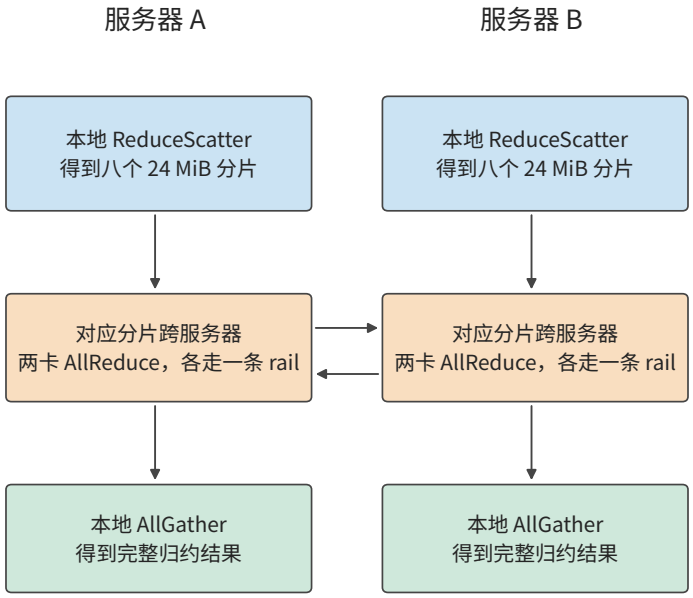



图 7-6：每台服务器先把本机八份贡献归约成分片，再和另一台服务器交换对应的分片，最后在本机收集完整结果。每列从上向下执行，横向箭头表示跨服务器交换，每对参与者各用一条 rail。

连续环每轮在每个方向上经一张网卡发送 12 MiB，需要约 0.25 ms；同一轮的其余十四条边走 NVLink，每条 12 MiB 只需约 0.03 ms，都要等这一条边。因此三十轮约需 7.6 ms。交错环每轮十六条边各经自己的网卡发送 12 MiB，每轮同样约需 0.25 ms，三十轮也约需 7.6 ms：交错环让所有网卡都在工作，却把跨服务器流量增加到八倍，时间没有缩短。

分层归约的两个跨服务器轮次，每张网卡每轮传 12 MiB，共约 0.50 ms。本地 ReduceScatter 和 AllGather 合计十四轮，每轮每个参与者经 NVLink 发送 24 MiB，在 450 GB/s 下合计约 0.78 ms。十六轮启动共约 13 µs，得到

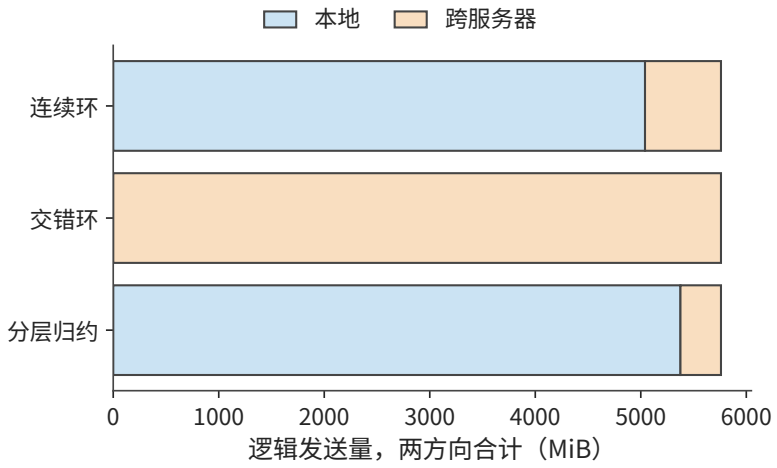



图 7-7：三方案的逻辑发送总量均为 5760 MiB；橙色跨服务器部分分别为 720、5760、384 MiB。两个方向合计。

$$T_{\text{hier}} = 14\frac{24\ \text{MiB}}{450\ \text{GB/s}} + 2\frac{12\ \text{MiB}}{50\ \text{GB/s}} + 16 \times 0.83\ \mu\text{s} \approx 1.3\ \text{ms}.$$

连续环的传输已超过 3 ms 的预算，分层归约则为归约计算、传播和排队留出约 1.7 ms。分层归约使跨服务器传输量减少约 47%，通信时间减少约 83%。两者比例相差悬殊，原因不在字节数，而在路径：连续环三十轮中的每一轮都要等跨服务器的边经网卡传完 12 MiB，其余十四条边走 NVLink，约 0.03 ms 传完后就在等这条边，另外十四张网卡则整轮没有数据可传；分层归约把跨服务器传输压缩成两轮，每轮八张网卡同时各传 12 MiB，割集的有效带宽从 50 GB/s 变成 400 GB/s。

**讨论：本地互联要多快，分层才合算？**把本地带宽写成 $B_L$，网卡仍为每方向 50 GB/s。分层归约的本地十四轮共发送 336 MiB，跨服务器两轮共约 0.5 ms；只要 $B_L$ 不低于网卡带宽，连续环的时间就始终由那一条跨服务器边决定，约为 7.6 ms。令两者相等，$336\ \text{MiB}/B_L + 0.5\ \text{ms} = 7.6\ \text{ms}$，得到 $B_L \approx 50\ \text{GB/s}$：只要本地互联不慢于一张网卡，分层归约就更快，NVLink 的 450 GB/s 远高于该界限。分层归约在这台服务器上没有代价，因为每个参与者的本地发送量（336 MiB）不比连续环的（360 MiB）多，跨服务器阶段又用上了全部八张网卡。

分层归约并不总是免费的。作为对照，考虑一种常见的 PCIe 服务器（下文称对照配置）：每台四张 A100 80GB PCIe，卡与卡之间经 PCIe Gen4 x16 点对点传输，每方向 32 GB/s；每台配一张双端口 ConnectX-7，两个 200 Gbit/s 端口各 25 GB/s，共用这张网卡的 PCIe Gen4 x16 插槽，每方向也只有 32 GB/s。八个参与者的连续环十四轮各发送 24 MiB，本地边受点对点链路限制，跨服务器边受网卡插槽限制，每轮都按 32 GB/s 计，约需 11.0 ms。分层归约的两个跨服务器轮次每方向各要通过插槽传送 96 MiB，共约 6.3 ms，加上六轮本地通信约 9.4 ms，合计约 15.7 ms，反而比连续环慢。令两者相等，本地带宽约为 64 GB/s，约是网卡插槽带宽的两倍。[5] A100 PCIe 的 NVLink 桥接器每方向 300 GB/s，远高于该界限，但一座桥只连两张卡，四卡环上仍有两条边走 PCIe；若四卡环的每条边都有这样的带宽，分层归约降到约 7.3 ms，才快于连续环。差别的来源是共享接口：跨服务器阶段把八个参与者的分片集中到两轮里，这两轮的流量都要经过同一个 32 GB/s 的插槽，本地归约节省的远端字节被更长的远端轮次抵消了一部分；本地链路又不比该插槽快，本地归约增加的时间也就无法补回。

[5] 对照配置的分层归约、连续环、交错环：每台四张 A100 80GB PCIe，卡间 PCIe Gen4 x16 点对点每方向 32 GB/s；一张双端口 ConnectX-7，两个 200 Gbit/s 端口各 25 GB/s，共用 PCIe Gen4 x16 插槽的每方向 32 GB/s；每轮启动 0.833 µs。A100 80GB 数据手册列出 PCIe 4.0 为 64 GB/s、两卡 NVLink 桥接器为 600 GB/s，均为收发两个方向的合计；ConnectX-7 数据手册列出 1 / 2 / 4 端口配置、单卡合计至 400 Gbit/s，主机接口为 PCIe Gen5 x16 / x32；对照配置按这张卡装在 PCIe Gen4 x16 插槽上计算。本地带宽为 300 GB/s 的数字把 `local_bytes_per_second` 改为 300 GB/s 重算得到。

分层归约先在较快的本地互联上完成部分归约，再把结果经网卡发出。它减少跨服务器的字节，更重要的是让每张网卡都有自己要传的分片。跨服务器带宽由多张独立网卡提供时，分层几乎没有代价；跨服务器带宽是共享接口时，只有本地归约增加的时间少于其节省的远端传输时间，分层归约才更快。

**第四种方案：在网归约。**前三种方案都让参与者之间两两传数据，交换机只负责转发。**在网归约**让交换机在转发的同时做加法：NVIDIA 的 SHARP（Scalable Hierarchical Aggregation and Reduction Protocol，可扩展分层聚合与归约协议）在交换芯片里沿聚合树归约来自多个端口的数据，再把结果分发回去，端点不必把同一份数据多次发送。[6] 沿用分层归约的本地 ReduceScatter 与 AllGather，只把跨服务器阶段换成：每个参与者把自己的 24 MiB 分片发给交换机一次，收回归约结果一次，只有一轮。两台服务器时，每张网卡每个方向仍传 24 MiB，跨服务器总量仍是 384 MiB，与分层归约相同；时间从两轮的 0.505 ms 变为一轮的 0.504 ms，只节省一次 0.83 µs 的启动。

差别出现在服务器数增加时。设持有同一分片的服务器有 $S$ 台。环形 AllReduce 让每张网卡发送 $2\frac{S-1}{S}\times 24$ MiB、经过 $2(S-1)$ 轮；在网归约让每张网卡始终只发送一次、接收一次。跨服务器阶段的时间见下表和图 7-8：

| 服务器数 | 环：每网卡发送 | 环：轮数 | 环：时间 | 在网归约：每网卡发送 | 在网归约：时间 |
|---|---|---|---|---|---|
| 2 | 24 MiB | 2 | 0.505 ms | 24 MiB | 0.504 ms |
| 4 | 36 MiB | 6 | 0.760 ms | 24 MiB | 0.504 ms |
| 8 | 42 MiB | 14 | 0.892 ms | 24 MiB | 0.504 ms |
| 16 | 45 MiB | 30 | 0.969 ms | 24 MiB | 0.504 ms |
| 32 | 46.5 MiB | 62 | 1.027 ms | 24 MiB | 0.504 ms |

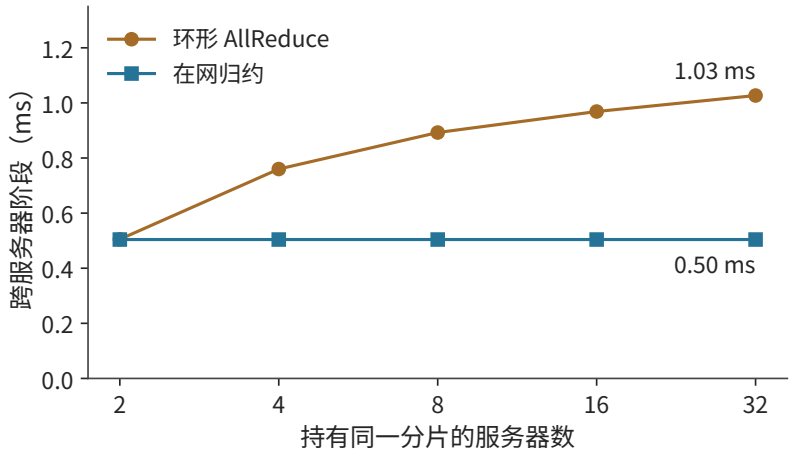



图 7-8：持有同一分片的服务器从 2 台增至 32 台时，跨服务器阶段的时间。环的每网卡发送量趋近两个分片、轮数按 $2(S-1)$ 增长；在网归约保持一个分片、一轮。两台服务器时两者只差一次启动。

**讨论：在网归约何时明显更快？**两台服务器时两者只差一次启动；从 $S=3$ 起，环的字节数是在网归约的 $2(S-1)/S>1$ 倍，还多出 $2S-3$ 轮启动，服务器越多差距越大，32 台时在网归约的时间只需环的 49%。该模型只计网卡的串行发送与启动；交换机归约引擎每秒能归约多少字节、支持哪些数据类型，是要另外给定的输入，流量超过它时瓶颈从网卡转到交换机。SHARP 论文在 128 台主机上测得 8 字节 AllReduce 从 6.01 µs 降到 2.83 µs，4096 字节从 46.93 µs 降到 14.48 µs：小消息节省的主要是轮数带来的启动时间。[7]

[6] 在网归约对照：本地阶段与分层归约相同，跨服务器阶段每张网卡发送分片一次、接收结果一次；$S$ 台服务器的环按每网卡 $2(S-1)/S$ 个分片、$2(S-1)$ 轮计，每轮启动 0.833 µs；交换机归约引擎的吞吐不在模型内。SHARP 的定义取自 NVIDIA SHARP 文档，测量值取自 SHArP 论文摘要。

[7] 在网归约对照：本地阶段与分层归约相同，跨服务器阶段每张网卡发送分片一次、接收结果一次；$S$ 台服务器的环按每网卡 $2(S-1)/S$ 个

**实验 7-4 ・延伸：在网归约何时值得**

把梯度改为 BF16（每参与者 96 MiB），重算两台与八台服务器的环形与在网归约跨服务器时间。设交换机归约引擎每方向的归约吞吐为 200 GB/s，求八条 rail 同时归约时它是否成为瓶颈，并求使在网归约仍快于环的最小引擎吞吐。最后按第 7.6.3 节的 8 KiB 输入，比较 36 层、每层两次归约的总启动时间在两种方案下相差多少。

### 7.2.2 张量并行与流水线并行的通信需求

数据并行通过归约汇合各参与者的梯度。张量并行把一层的计算分给多个参与者，后面的算子往往要等局部结果汇合后才能开始，所以层内通信会直接推迟后续计算。流水线并行按阶段切开模型，把跨服务器的交接集中在阶段边界。两种方式下，数据跨越服务器边界的次数不同。

先计算一次层间交接的数据量。Qwen3-8B 的隐藏宽度为 4096，一个 BF16 隐藏向量是 8 KiB。prefill 中 8192 个 token 的同类张量为 64 MiB，单请求 decode 则为 8 KiB。链路带宽 $B$ 取本章网卡的 50 GB/s，每次传送的启动时间 $\alpha$ 设为 5 µs，传送 $M$ bytes 耗时为

$$T_{\text{msg}} = \alpha + \frac{M}{B}.$$

64 MiB 约需 1.3 ms，8 KiB 约需 5.2 µs。前者的时间主要花在传输数据上，后者约 97% 的时间花在启动上。启动时间与传输时间相等时，$M = \alpha B = 250000$ bytes，约为 244 KiB。数据量远大于该值时，传输时间占主导，提高带宽更有效；远小于该值时，减少传输次数更有效。

上式给出了每次交接的代价；对流水线并行还要看多次交接如何与计算穿插。流水线并行把连续若干层放在同一阶段，阶段之间传递激活。前一阶段处理下一个 micro-batch 时，后一阶段可以处理上一个 micro-batch。取四个前向阶段，每阶段 1 ms。一个 micro-batch 从第一阶段依次走到第四阶段，4 ms 后完成。连续放入八个独立 micro-batch，第一阶段每毫秒将一个 micro-batch 的结果传给下一阶段，最后一个 micro-batch 在第 11 ms 完成。

沿图 7-9 中相同的编号看，一个 micro-batch 必须走完四个阶段；沿同一行看，一个阶段可以连续处理八个 micro-batch。左下和右上的空白，就是流水线启动和结束时的空闲。

一般地，$p$ 个等时阶段、$m$ 个独立 micro-batch、每阶段耗时 $\tau$，总时间与各阶段的利用率分别为

$$T_{\text{pipe}} = (m + p - 1)\tau, \qquad U_{\text{pipe}} = \frac{m}{m + p - 1}.$$

四阶段、八个 micro-batch 时利用率约为 73%。若希望达到 90%，由 $m/(m+3) \geq 0.9$ 得到至少 27 个独立 micro-batch。要让流水线各阶段持续工作，就需要足够多的独立 micro-batch。低并发 decode 中，同一请求的下一个 token 依赖前一个 token 的输出，增加服务器并不会创造新的独立 micro-batch；增加并发请求才会增加流水中的独立工作。

因此两种并行产生不同的等待：张量并行要频繁汇合局部结果，流水线并行在启动和结束时有一部分卡空闲。Prefill 的张量较大，应优先减少跨服务器传输的数据；低并发 decode 逐个生成 token，应优先减少每层通信的启动次数。并发请求数则决定流水线利用率。

---

分片、$2(S-1)$ 轮计，每轮启动 0.833 µs；交换机归约引擎的吞吐不在模型内。SHARP 的定义取自 NVIDIA SHARP 文档，测量值取自 SHArP 论文摘要。

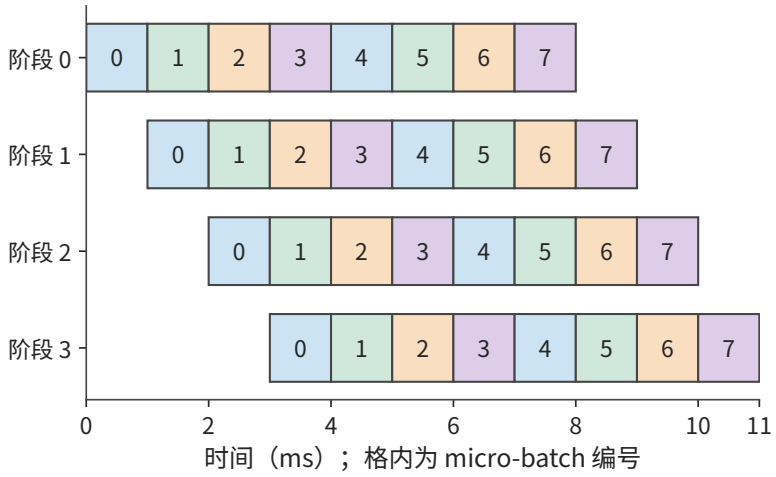



图 7-9：四个前向阶段如何处理八个 micro-batch。每格为 1 ms，数字表示 micro-batch 编号；同一编号沿右下方移动，表示该 micro-batch 依次通过各阶段。每个阶段计算 8 ms，在 11 ms 的总时间中各空闲 3 ms。阶段耗时相同，图中忽略额外交接开销。

### 7.2.3 专家并行的通信负载

张量并行和流水线并行的通信由切分方式确定；MoE 还多一个变数：每个 token 选的专家可能不同，发送目标也随之改变。专家并行把专家放在不同的卡上，token 按路由去相应的专家。取 1024 个 token，每个选八个专家，共 8192 次 dispatch。每份激活 8 KiB，一半跨服务器，dispatch 到远端专家的输入就有 4096 份，共 32 MiB；combine 返回同样大小的结果，再传 32 MiB。[8]

设 token 数为 $n$、每 token 选择数为 $k$、每份激活占 $d$ bytes、跨边界比例为 $f$。若每次 dispatch 单独传送，dispatch 的传输量为 $nkfd$，加上 combine 返回的专家结果共 $2nkfd$。把热门专家放到发送方所在的服务器，降低的是 $f$；改变激活的数据类型或编码，改变的是 $d$。

这 32 MiB 如何分配到接收方，同样影响传输时间。全部发给一张 50 GB/s 的网卡，接收至少需约 0.67 ms（图 7-10）；平均分给接收服务器的八张网卡，每张接收 4 MiB，接收时间降至约 0.08 ms（图 7-11）。对照配置的双端口网卡两个端口共用一个 32 GB/s 的 PCIe 插槽，无论如何分到两个端口，接收 32 MiB 都至少需要约 1.05 ms。将流量均匀分给网卡，消除了单张网卡的瓶颈；端口共用插槽时，共享的插槽随后成为新的瓶颈。

把接收目标分散到多个网卡之后，还要继续向前追踪共同经过的接口。图 7-12 把对照配置的共享插槽单独画出：无论后面如何分到两个端口，这 32 MiB 都必须先通过同一个插槽。

比较这三张路径图，分散目标减少了每张网卡上的数据量；沿最后一张图的共同入口看，32 MiB 仍须全部通过它。计算不规则通信时，也可以先分别统计这些位置的流量。

把从卡 $i$ 发往卡 $j$ 的载荷记为 $V_{ij}$。同一行相加是发送方的发送量，同一列相加是接收方的接收量；把跨越某个割集的元素相加，就是该割集的传输量。把这些量各自除以对应的带宽，再取最大值，就是第 7.1 节的资源模型在不规则交换上的应用。

从该矩阵还能看出 dispatch 与 combine 的热点如何对应。若按 token 到专家逐份发送、返回同样大小的向量，且接收方不做合并，返回矩阵就是发送矩阵的转置：dispatch 的热点接收方，就是 combine 的热点发送方。两次通信的总字节数相同，不代表两段耗时相同：返回的数据要等专家算完才就绪，两个方向的可用带宽、数据格式和归约位置也可能不同。若 dispatch 用 FP8、combine 用 BF16，返回的载荷约为 dispatch 的两倍，量

[8] 模型并行案例、第 6 章正文。本章 EP 算例使用 1024 个 token、每 token 八次 dispatch、每份 8 KiB、跨边界比例二分之一，路由为教学构造。

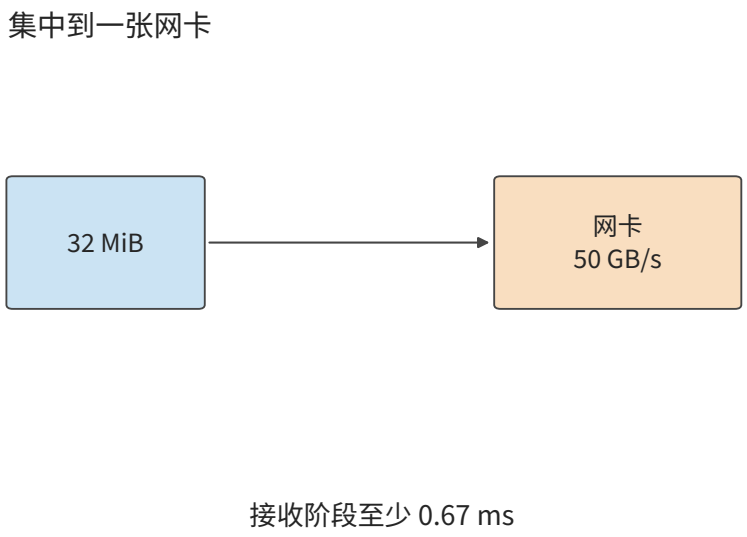



图 7-10：将全部 32 MiB 送入一张 50 GB/s 网卡，接收至少需要约 0.67 ms。

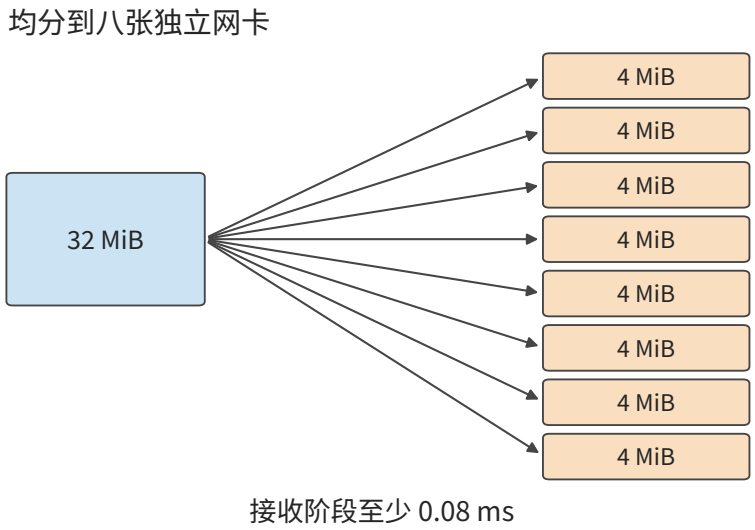



图 7-11：八张独立网卡各接收 4 MiB，接收阶段缩短到约 0.08 ms。

化 scale 等元数据另计。第 9.4 节会把这些流量与计算就绪时间放进同一个算例。

专家放置因此有两项互相联系的目标：让更多 dispatch 留在本地，让远端 dispatch 分散到有余量的接收方。复制热门专家是用额外的容量换少走远端；发往同一张卡上多个专家的输入可以只传一次激活，到了那张卡再分给各个专家。两种办法都减少了相应路径上的传输量。

### 7.2.4 多网卡

专家 dispatch 说明了多入口并行的收益和共享入口的限制。同样的分析也适用于发送方：增加网卡，只有当它确实提供额外的可用带宽、并且有数据分配给它时，才会缩短传输时间。第 7.2.1 节的连续环就是一例：服务器上八张网卡各连一张卡，连续环却只让其中一张有数据可传，分层归约才把分片分给八对参与者，跨服务器阶段从 7.6 ms 降到约 0.5 ms。有网卡不等于用上了网卡，通信算法决定每张网卡分到多少字节。

对照配置说明另一种限制。其双端口网卡有两个 25 GB/s 端口，共用 32 GB/s 的 PCIe 插槽。只用一个端口时，八个参与者的连续环十四轮约需 14.1 ms；把每次要传的数据拆成两段，分别交给两个端口，吞吐只升到

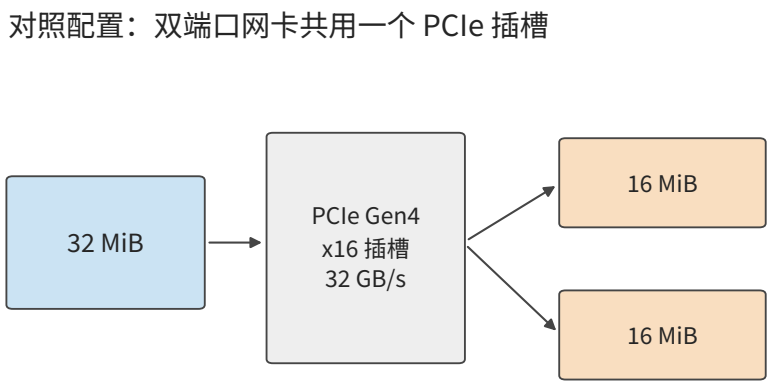



图 7-12：对照配置中，32 MiB 都需要先经过网卡 32 GB/s 的 PCIe 插槽，接收阶段受限于约 1.05 ms。这里各图均只计算接收载荷的传输。

插槽允许的 32 GB/s，时间降到约 11.0 ms，而不是两个端口合计 50 GB/s 对应的约 7.1 ms。即使换成四端口的 ConnectX-7，把第三个端口也用上，插槽仍是同一个 PCIe Gen4 x16，这段传输不能再缩短。共享接口的带宽用尽之后，再加端口也不提高吞吐。[9]

另一种情况是某张 GPU 直连的网卡正忙，相邻 GPU 的网卡却空闲。借用相邻的出口要先把数据经 NVLink 送到相邻 GPU，多了一段本地传输。多网卡通信系统 FuseLink 采用的就是这条路径：借助机内的 GPU 互联，把网络缓冲区重新映射到空闲的网卡上。[10]

下面比较直接发送和经相邻 GPU 中继两种方式。直连网卡每方向 50 GB/s，两张可借用的相邻网卡也各为 50 GB/s；通往相邻 GPU 的 NVLink 每方向 450 GB/s。

沿图 7-13 的分叉计算：直接路径与中继路径可以同时传输，但数据必须依次经过中继路径内部的两段。因此，直接和中继路径合计可提供 $50+\min(450,100)=150$ GB/s。设外部网络与接收路径共同允许的带宽为 $B_D$，实际的发送能力为

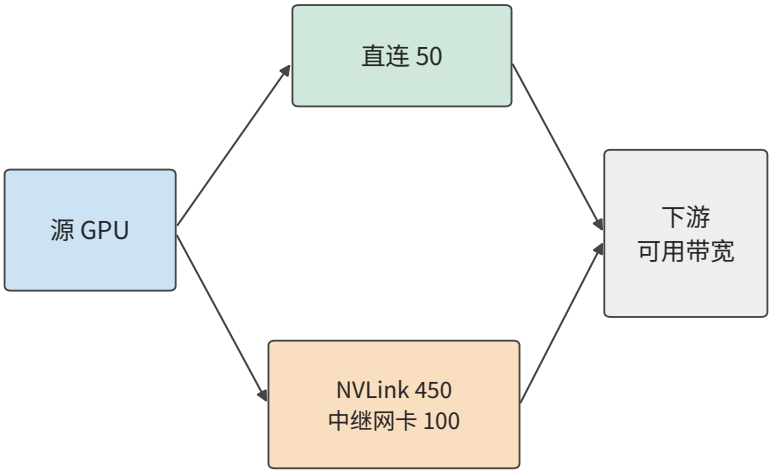
$$B_{\text{eff}} = \min\bigl(B_D,\ 50+\min(450,100)\bigr)\ \text{GB/s},$$

其中 $B_D$ 也以 GB/s 计。$B_D \le 50$ 时直连网卡已能用满下游带宽，中继没有收益；$50 < B_D < 150$ 时，中继收益随下游可用带宽增加；超过 150 后，两张借用网卡的合计带宽成为新的瓶颈。若 NVLink 同时在承担本机归约的流量，留给中继的本地带宽低于 100 GB/s 时，限制中继路径的就换成本地这一段。1.125 GiB 载荷在直接 50 GB/s 路径上约需 24.2 ms，下游支持 100 GB/s 时降至 12.1 ms，支持 150 GB/s 时降至 8.1 ms。

上述按带宽借用网卡的分析，前提是每次传输的数据量足够大。MoE decode 的 dispatch 和 combine 要让每张卡发几千条小消息，这时还要检查网卡每秒能发起多少项请求，也就是第 7.3.4 节的 $1/\delta$。按该节 RoCE 可靠连接实现的 $\delta \approx 18.6$ ns 计，网卡每秒约发起 5400 万项请求；在 50 GB/s 的链路上，只有消息小于约 0.93 KB 时，发起速率才先于带宽成为瓶颈。MoE dispatch 的一条消息是一个 token 的隐藏向量，隐藏维 7168 时

[9] 对照配置下一个端口、两个端口、三个端口的连续环：跨服务器消息按端口条带化，PCIe Gen4 x16 插槽每方向 32 GB/s 不随端口数增加。第 7.6.2 节只剩一张网卡的分层归约见单网卡分层归约。

[10] Ren 等，*Enabling Efficient GPU Communication over Multiple NICs with FuseLink*，OSDI 2025；正式会议稿、通信路径推算。本文中继路径按本章配置取值：网卡每方向 50 GB/s、NVLink 每方向 450 GB/s；插件实现与实验平台见原论文。




单位 GB/s；中继受借用网卡合计 100 限制

图 7-13：借用相邻网卡的路径。直接路径最多传输 50 GB/s；两张中继网卡合计 100 GB/s，数据要先经过 450 GB/s 的 NVLink 到达相邻 GPU。两路在下游汇合，最后还受外部网络和接收方可用带宽限制。箭头表示传输方向。

FP8 约 7 KiB、BF16 约 14 KiB，都远大于该交叉点，所以热点专家所在的卡先用尽的仍是带宽，借用相邻网卡按字节分配即可。只有每条消息不到 1 KB 时，例如逐条发送的控制消息，发起速率才会先于带宽用尽；第 9.4.2 节讨论专家副本放置时，把这种情况下网卡的报文处理能力也计入需要预留余量的资源。[11]

无论是分层归约、专家放置还是多网卡中继，分析步骤相同：画出路径，计算每条链路和接口传输的数据量，再找出耗时最长的一项。

**实验 7-5 ·核心：本地带宽与共享出口如何影响归约算法选择**

完整复算例题 7.2，再改用对照配置（每台四张 A100 80GB PCIe，卡间经 PCIe Gen4 点对点每方向 32 GB/s，双端口 ConnectX-7 的两个 25 GB/s 端口共用 32 GB/s 的 PCIe 插槽）重算连续环和分层归约。在对照配置下把本地带宽从 32 GB/s 提高到 NVLink 桥接器的 300 GB/s（设四张卡之间的每条边都有这样的带宽），解释为什么归约算法的选择会翻转，而在本章配置下不会。再将对照配置每台服务器使用的端口数从一个增为两个、三个，保持 32 GB/s 的插槽不变，画出归约时间随端口数变化的曲线，指出增加到多少个端口后，继续增加已无法缩短时间。

### 7.2.5 多轨拓扑

第 7.2.1 节的分层归约让八对参与者各走一条 rail。DGX SuperPOD 参考架构把计算网络组织成**多轨拓扑**（rail-optimized）：每台服务器的八张网卡分别接到八台不同的叶交换机，所有服务器的第 $i$ 张网卡都接第 $i$ 台叶交换机，这台叶交换机连同接在它上面的所有网卡就是第 $i$ 条 rail；同一组 32 台服务器内，同一条 rail 上的流量经叶交换机一跳到达，不同 rail 之间的流量要经过脊交换机。[12] 图 7-1 里“每条 rail 一台交换机”的交换网络就是这种组织。

[11] 报文速率与带宽的交叉点按第 7.3.4 节 RoCE 可靠连接的启动间隔 18.63 ns 和本章网卡的 50 GB/s 推得，约 931 B；FuseLink 的带宽借用见 [^fuselink]，MoE dispatch 的隐藏维 7168、FP8 dispatch 与 BF16 combine 见 DeepEP README 快照。

[12] 对齐配对、错位配对：两台服务器各八张网卡，第 $i$ 张网卡接第 $i$ 条 rail 的叶交换机，跨服务器阶段为两 rank 的 ReduceScatter 加 AllGather，只计网卡串行发送与每轮 0.833 µs 启动。多轨拓扑、32 台服务器一组内同一 rail 一跳到达、跨 rail 经脊层，取自 DGX SuperPOD H100 参考架构 PDF 第 8、14 页。

分层归约的跨服务器阶段让 rank $i$ 与 rank $i+8$ 配对：rank $i$ 用服务器 A 的第 $i$ 张网卡，rank $i+8$ 用服务器 B 的第 $i$ 张网卡，两张网卡都接在第 $i$ 台叶交换机上。因此八对参与者的流量各自留在一条 rail 内，都只经过一台叶交换机，没有一个字节经过脊交换机。每条 rail 每个方向承载 24 MiB，两轮共 0.505 ms；这 192 MiB 若集中在一张网卡上需要 4.03 ms，八条 rail 同时工作将其缩短到约八分之一。[13] 图 7-14 的上半图画出这种对齐配对，下半图是下文讨论的错位配对。

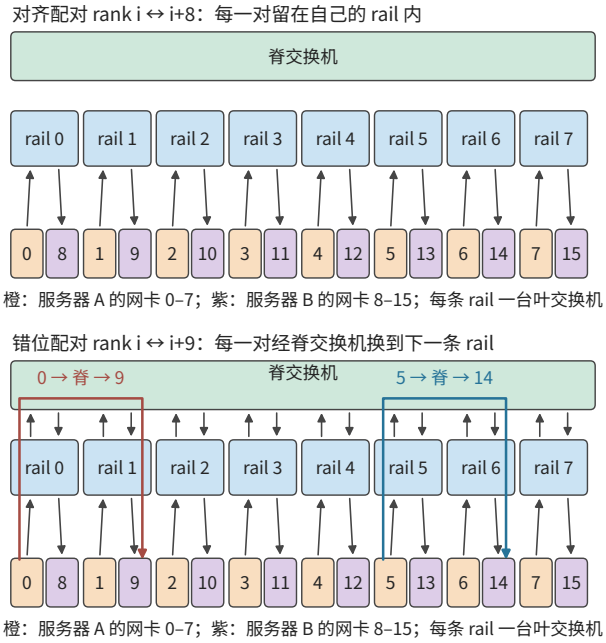



图 7-14：八条 rail 画成八条竖直车道，每条车道里是一台叶交换机和两台服务器上编号相同的网卡。上半图按 rank $i$ 与 rank $i+8$ 配对，每一对的字节留在自己的车道里，只经过一台叶交换机；下半图按 rank $i$ 与 rank $i+9$ 配对，每一对都要从第 $i$ 条 rail 经脊交换机换到第 $i+1$ 条 rail，每个方向 192 MiB 全部经过脊层。

把配对改成 rank $i$ 与 rank $i+9$（rank 7 与 rank 8），rank $i$ 仍用第 $i$ 条 rail，对端却用第 $i+1$ 条 rail。每一对的字节要先进入第 $i$ 台叶交换机，再经脊交换机到第 $i+1$ 台叶交换机，八对无一例外，每个方向 192 MiB 全部经过脊层。每张网卡的发送量没有变，跨服务器阶段的下界仍是 0.505 ms；变的是这些字节占用了脊交换机的链路，要与其他作业的跨 rail 流量共用，还要经过第 7.5.3 节讨论的多路径哈希。配对是否对齐，只取决于两端 rank 在各自服务器内的编号是否相同：$i \bmod 8$ 相等就在一条 rail 内，不相等就经过脊层。交错环（图 7-5）在多轨拓扑上也要付出这一代价：它的十六条边都跨服务器，边 $i \to i+8$ 对齐，边 $i+8 \to i+1$ 不对齐，一半的字节要经过脊层。

把这一规则用于第 7.6.4 节的 128 卡超节点。十六台服务器各八张卡，张量并行组占一台服务器的八张卡，张量并行坐标为 $t$ 的卡在每台服务器里都是第 $t$ 张卡，用第 $t$ 条 rail；同一坐标上的十六个数据并行成员在超节点内归约后，跨超节点交换的就是这一坐标的梯度分片。八个坐标的分片一样大（每张卡 $G = 8$ GB），所以八条 rail 各承载跨超节点发送量的八分之一，即 112 GB 中的 14 GB，没有哪条 rail 更忙。只有当分片大小不同（例如不均匀的张量切分），或者一张网卡故障、其流量经 NVLink 借用相邻网卡（第 7.2.4 节）时，才会出现更忙的 rail。

**实验 7-6 ·延伸：rail 对齐与脊层流量**

在两台服务器上把配对改为 rank $i$ 与 rank $((i+4) \bmod 8)+8$，求经过脊层的字节数。再让服务器 B 的网

[13] 对齐配对、错位配对：两台服务器各八张网卡，第 $i$ 张网卡接第 $i$ 条 rail 的叶交换机，跨服务器阶段为两 rank 的 ReduceScatter 加 AllGather，只计网卡串行发送与每轮 0.833 µs 启动。多轨拓扑、32 台服务器一组内同一 rail 一跳到达、跨 rail 经脊层，取自 DGX SuperPOD H100 参考架构 PDF 第 8、14 页。

卡 3 故障，rank 11 的流量经 NVLink 借用网卡 2 和网卡 4 各一半，求各条 rail 承载的字节与跨服务器阶段的时间。最后在第 7.6.4 节的 128 卡超节点里改用 TP16（一组跨两台服务器），说明每条 rail 承载哪些坐标的分片、负载是否仍然均匀。

## 7.3 远程访问的路径与并发

### 7.3.1 数据传输与完成通知

第 7.2 节只计算了字节经过哪些链路、要传多久，还没有说明这些字节如何交接。分层归约的跨服务器阶段中，卡 0 已经得到本地归约后的分片，需要交给卡 8。一次数据交接包含两项工作：把分片写到卡 8 可以读取的位置，以及让卡 8 知道数据已经就绪。前一项改变数据所在的位置，后一项告诉接收方何时可以开始计算。

第 6.5.5 节的单边读写由发起方直接访问预先授权的远端内存，例如卡 0 把分片写进卡 8 的接收区域：发起端网卡从本地内存读出分片，目标端网卡把它写入卡 8 的内存，对端程序不参与这次搬移。**双边消息**要发送方和接收方配合：接收方预先提交接收请求并提供缓冲区，通信系统把到达的消息与相应的请求配对。已知目标位置的大块分片适合直接读写；来自多个发送方的通知则适合放进消息队列，由队列区分各条通知并交给相应的数据使用方，下文称为消费者。

大块数据与通知由此可以分工：先写分片，再发一个短消息说明分片可以使用。把写入和通知合并成一次提交，还能少发起一次。消费者收到通知后开始归约；写入与通知的先后关系在第 7.4 节展开。

写入和通知完成了数据交接。若还要接收方按指定方式处理数据，就用**远程过程调用**（remote procedure call，RPC）：请求对端执行指定的函数，一次调用包括传参数、远端执行和返回结果。参数的编码方式直接影响这条路径的开销：JSON 是文本形式的数据交换格式；base64 把每 3 个二进制字节编码为 4 个文本字符。将 1 MiB 二进制数据放进 JSON 的 base64 字段，编码载荷约增加三分之一。改用二进制请求，就同时减少发送字节和编码工作。

本书配套的一组跨主机 RPC 测量中，请求大小从约 1.33 MiB 降至 1.00 MiB，客户端 CPU 处理时间中位数从约 9.8 ms 降至 1.1 ms。20 对正式调用中有 11 对整体变快，网络传输和转发等待的耗时波动更大。[14] 这些测量区分了两类耗时：减少编码可以节省本地 CPU 时间；要缩短整个调用，还要减少网络、转发和远端执行中的等待。

**实验 7-7 ・延伸：去掉编码转换能缩短多少远程调用时间**

从配套记录选取同一载荷的成对调用，画出客户端编码、发送、等待、解码和服务端执行的时间线。解释为什么服务端的执行时间已经包含在客户端的等待时间中。计算省去 base64 编码后减少的传输字节数，并分析编码耗时占比如何影响完整调用的加速收益。

### 7.3.2 由产生数据的加速器直接发起通信

第 7.3.1 节从接收方的角度区分了读写、消息与 RPC。发送方一侧的问题是：谁最先知道数据已经就绪，谁负责提交请求。卡 0 的分片在 GPU 上产生。若由 CPU 提交通信，GPU 先报告就绪，CPU 构造请求交给网卡，完成后再把状态交还 GPU。这条控制路径要交接多次，即使载荷从不经过 CPU 内存，交接本身也要花时间。

RDMA 网卡已经能访问授权的远端内存；NVIDIA 的 **GPUDirect RDMA** 进一步让网卡直接访问已注册、已授权的 GPU 内存，载荷不必经主机内存中转。由 CPU 发起通信时，提交请求和处理完成通知仍由 CPU 负责；由加速器发起通信，则让 GPU 自己构造或触发请求并读取完成状态，数据一算完就能发出。URMA 是 UB

[14] RPC 阶段与配对复算、264 次调用原记录。CPU 阶段与请求字节的变化、配对调用变化分别统计。测量路径为 Mac 经 SSH 转发到 Linux 主机，含加密、转发与网络波动；配对完整调用节省中位数约 10.1 ms。

提供的统一远程内存访问接口。GPU 经 NVLink 访问对端内存、设备经 UB 发起异步访问，都是这种直接协作在各自路径上的实现。图 7-15 到图 7-18 依次画出主机 RPC、CPU 提交的 GPUDirect RDMA、GPU 经 NVLink 访问和设备发起的 URMA 访问。

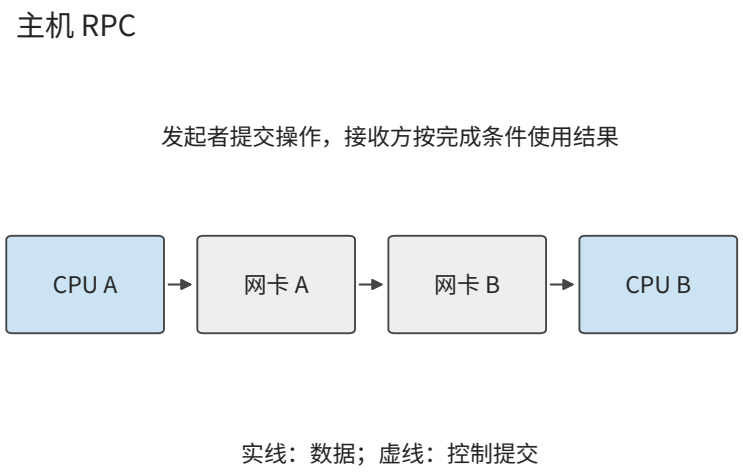



图 7-15：主机 RPC 的参数从 CPU A 所在主机出发，经网卡和网络到达 CPU B 所在主机，由远端执行请求。箭头表示参数数据路径，返回结果沿反方向传送。

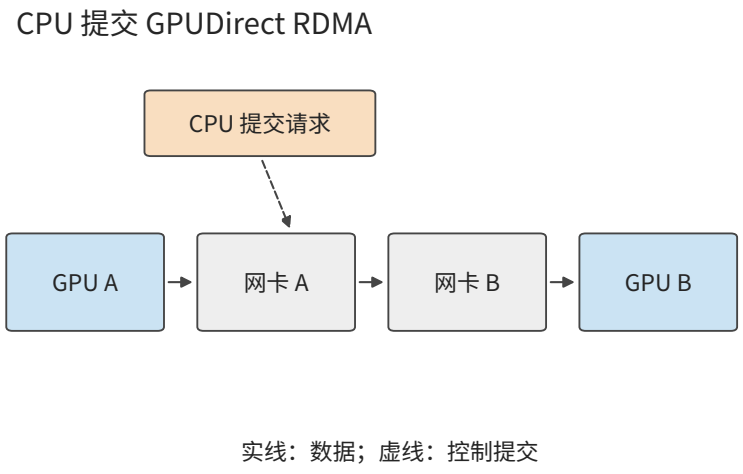



图 7-16：数据从 GPU 内存直接经过网卡到达远端 GPU 内存；虚线表示 CPU 提交请求，数据载荷无需经过 CPU 内存。映射和访问权限预先建立。

数据绕过主机内存之后，还可以进一步改变由谁发起。图 7-17 中 GPU 自己发起对端内存访问，省去了每次都由 CPU 提交的交接。

图 7-18 把同样的发起与完成关系放到 UB 的异步访问接口上。沿实线追踪载荷，沿提交与完成关系追踪控制，便能区分数据传输和请求管理各自的开销。

这一变化可以用第 7.2 节的传输时间公式解释。沿用第 7.2.2 节 50 GB/s 链路的算例，8 KiB 数据的传输时间约为 0.16 µs；若启动时间从 5 µs 缩到 2 µs，总时间就从约 5.2 µs 降到 2.2 µs，节省一半以上。64 MiB 的载荷本身要传约 1.3 ms，同样节省的 3 µs 就微不足道。因此，小数据量的传输应优先减少提交开销，大数据量的传输应优先提高带宽。

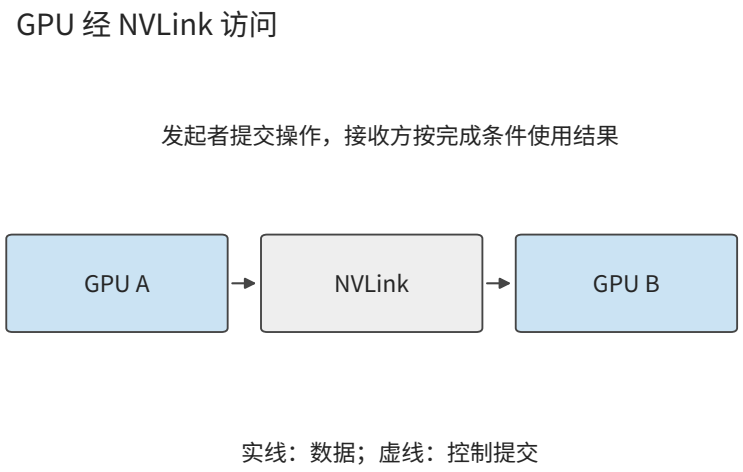



图 7-17：GPU 通过 NVLink 访问对端 GPU 内存，由发起的 GPU 执行已授权的访问。

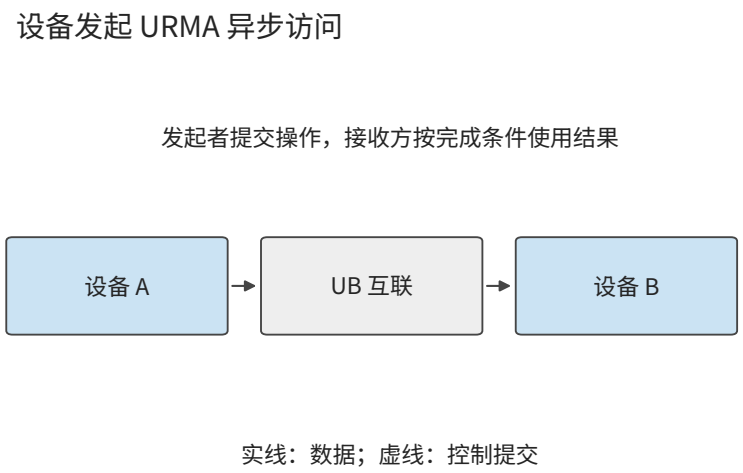



图 7-18：设备通过 URMA 提交异步读写，载荷经过 UB 互联到达目标设备；请求完成后按接口规定检查完成状态和结果。

由主机统一组织通信，好处是能为各种程序提供通用的提交和完成处理机制。模型执行中，数据的生产者、消费者和就绪条件都很明确，因此可以把频繁的发起和完成处理放到设备一侧：系统预先建立地址映射和访问权限，之后由设备连续发起计算与通信，不必每次都经主机中转。收益就是上例中启动时间的缩短。

操作系统负责建立映射、设置权限和分配资源；设备负责已经授权的快速操作；完成通知把操作结果交回运行时。职责这样划分之后，设备上发起通信的线程本身也会限制吞吐。要分析这种限制，先要分清一次请求要等多久、等待期间能否发起其他请求，再计算单位时间内能处理多少项请求。

### 7.3.3 内存语义：Load ／ Store 与 Read ／ Write

要分清这两项，先区分访问的表达方式。远端访问有两种常见表达：处理器的读取 / 写入指令（Load / Store），以及显式的异步读取 / 写入请求（Read / Write）。两种方式都需要记录尚未完成的请求，区别在于由谁管理这些记录。

Load 的结果可以直接成为后续指令的输入。处理器记录这条依赖，并在等待期间执行其他独立指令。异步请求则把目标地址、长度和完成条件放入队列，软件可以连续提交多项工作，在使用结果之前等待相应请求完

成。前者把依赖交给指令执行机制，后者把依赖交给事件和运行时。本书把 Load / Store 称为同步访问，是强调指令依赖由硬件管理，并非指处理器每次只能有一个访存在途；现代处理器可以并行执行多个独立 Load，Store 也可以先进入缓冲区。

| 比较项 | Load / Store | 异步 Read / Write |
|---|---|---|
| 发起与路径 | 处理器指令，经地址映射到可达内存；UB 可由片上控制器承接 | 提交地址、长度等请求；经队列、门铃或设备发起接口推进 |
| 依赖与完成 | 硬件跟踪 Load 的消费者；Store 的远端可见性另需顺序保证 | 运行时等待规定的完成事件，再使用结果或回收缓冲 |
| 常见适用范围 | 细粒度读取、指针访问、访问频繁且需要低提交开销的数据 | 激活、KV、权重等批量搬移及显式流水 |
| 并发限制 | 指令依赖、未完成访存槽位、地址转换与硬件资源 | 请求队列、完成队列、注册内存、软件或设备的提交能力 |
| 故障处理 | 依赖平台的超时、异常及内存故障机制 | 可按请求报告错误；仍须处理超时、部分写入和缓冲回收 |

两种接口都可以采用内存语义，但不能仅凭接口名称推断缓存一致性。单边 Read / Write 不要求远端应用逐次提交接收请求（Receive）来配对，仍需要事先授权和内存有效期；Load / Store 也不保证跨节点共享缓存自动一致。OpenURMA 实现的 Load / Store 路径不维护跨节点的缓存一致，写入、发布、读取之间的依赖由应用自己安排。其省去的 PCIe 往返来自控制器位置和请求路径的改变，把软件接口改名为 Load 并不能得到同样的收益。[15]

**一次远程读取的延迟分解。**把一次 64 B 远程读取的关键路径拆成若干阶段，为每个阶段给定延迟，各阶段之和就是这次读取的总时间。这里比较三条路径：经 PCIe 外设网卡的异步读取，经片上总线控制器的异步读取（UB 的 URMA 接口），以及经片上总线控制器的 Load 指令。给定条件如下：一次片上总线穿越 30 ns；一次 PCIe 门铃写 150 ns，一次 PCIe DMA 读 500 ns、写 250 ns；线路单程 100 ns；目标端内存行命中（要读的数据所在的 DRAM 行已经打开）30 ns；网卡流水线按 322 MHz 时钟的周期数换算，外设网卡 9 个周期，UB 异步路径 25 个周期，Load 路径 8 个周期；接口库提交、构造请求描述符和轮询完成记录属于软件开销。下表按关键路径的先后列出各阶段，空格表示该路径没有这个阶段。

| 阶段 | 类别 | PCIe 外设网卡 | UB 异步读取 | UB Load |
|---|---|---|---|---|
| 接口库提交 | 软件提交 | 50 | 50 | |
| 构造请求描述符 | 软件提交 | 30 | 30 | |
| 门铃写 | PCIe 穿越 | 150 | | |
| 网卡 DMA 读取描述符 | PCIe 穿越 | 500 | | |
| 片上总线提交 | 片上总线 | | 30 | 30 |
| 网卡发送流水线 | 网卡流水线 | 28 | 78 | 25 |
| 线路去程 | 线路 | 100 | 100 | 100 |
| 目标端网卡接收流水线 | 网卡流水线 | 28 | 78 | 25 |
| 目标端网卡 DMA 读主机内存 | PCIe 穿越 | 500 | | |
| 目标端片上总线访存 | 片上总线 | | 30 | 30 |
| 目标端内存行命中 | 内存与完成 | 30 | 30 | 30 |

[15] 李博杰，〈Unified Bus 背后的思考〉，Jetty、事务序与 Load/Store 各节；OpenURMA 论文，2026-06-02 修订版（arXiv:2605.28717），§3 设计、§7–§9 状态与延迟、§10 顺序、§12 传输、§13 结果汇总。本次整合的来源与适用范围。

| 阶段 | 类别 | PCIe 外设网卡 | UB 异步读取 | UB Load |
|---|---|---|---|---|
| 目标端网卡发送响应 | 网卡流水线 | 28 | 78 | 25 |
| 线路回程 | 线路 | 100 | 100 | 100 |
| 网卡接收响应 | 网卡流水线 | 28 | 78 | 25 |
| 响应载荷 DMA 写 | PCIe 穿越 | 250 | | |
| 完成记录 DMA 写 | PCIe 穿越 | 250 | | |
| 片上总线完成 | 片上总线 | | 30 | 30 |
| 完成队列轮询 | 内存与完成 | 70 | 5 | |
| 接口库轮询 | 内存与完成 | 30 | 30 | |
| 序号分配串行化 | 内存与完成 | 50 | | |
| **合计（推导）** | | **2222** | **746** | **419** |
| 仿真测得 | | 2236 | 757 | 500 |

表中网卡流水线的时间已经取整，合计按取整前的值相加。经 PCIe 外设网卡的路径合计 2222 ns，其中五次 PCIe 穿越占 1650 ns；UB 异步路径没有 PCIe 阶段，合计 746 ns；Load 路径连软件提交和轮询也省去了，合计 419 ns。图 7-19 把三条路径按阶段类别画成堆叠条形。笔者的 OpenURMA 实现在两节点周期级仿真中、相同条件下测得 2236、757 和 500 ns，与推导值相差 14、11 和 81 ns。差值来自仿真器在模块之间传递数据的固定开销；Load 路径的阶段最少、总时间最短，这项开销所占的比例也就最大。[16]

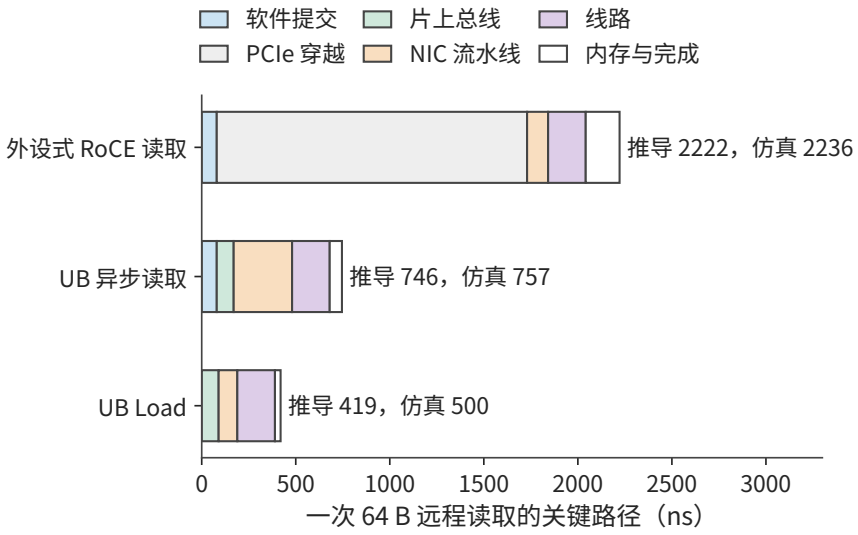



图 7-19：三条路径的关键路径按阶段类别堆叠。经 PCIe 外设网卡的路径中，浅灰色一段是五次 PCIe 穿越；两条片上总线路径没有这一段，Load 路径还省去了软件提交和完成轮询。条形右侧标出推导值和仿真值。

PCIe 外设网卡多出的阶段，不是九处各自可以优化的低效环节，而是三个结构性原因造成的。第一，请求必须先写成主机内存里的描述符，再由网卡读取，因此有接口库提交和构造描述符两个阶段；第二，网卡和处理器各有自己的地址空间，因此有门铃写、描述符 DMA 和目标端 DMA 三个阶段；第三，操作完成后必须跨地址空间通知处理器，因此有响应 DMA、完成记录 DMA 和两次轮询。控制器接到片上总线上以后，第二类阶段合并成一次片上总线穿越；改用 Load 指令以后，处理器自身的依赖跟踪取代了完成通知机制，第一类和第三类阶段随之消失。

[16]阶段延迟、请求速率和状态增长的参数与结果见UB 互联计算，运行 `python3 calculations/calc.py ub-fabric --format md` 可以复算。各阶段延迟取自 OpenURMA 论文表 7 给定的值，记录尺寸取自表 3，仿真对照值取自 §8.1 与 §8.3。推导把所有阶段串行相加，不考虑并发请求之间的重叠。

还有第四条路径：网卡仍是 PCIe 外设，但请求描述符由网卡内的处理器构造并取用，即第 6.4.4 节的第三种发起位置。按同一张表推算，这条路径省去门铃写和网卡 DMA 读取描述符两次穿越，共 650 ns；软件提交与构造请求描述符的 80 ns 改在网卡处理器上完成，按同样取值计；载荷 DMA 写、完成记录 DMA 写和目标侧的 DMA 读仍然保留。合计约 1572 ns，落在外设式的 2222 ns 与片上总线路径的 746 ns 之间。省去的是控制路径上的穿越，省不去的是数据和完成通知必须跨越两个地址空间。这条路径没有做仿真，只按表中取值推导。

线路时延变化时，三条路径的差距也随之变化。每条路径的总时间都等于固定开销加上两倍的线路单程时延 $L$：PCIe 外设网卡为 2022 ns 加 $2L$，UB 异步路径为 546 ns 加 $2L$，Load 路径为 219 ns 加 $2L$。图 7-20 画出 $L$ 从 50 ns 到 500 ns 的情形：三条直线斜率相同、截距不同，绝对差距始终约为 1.8 µs，相对差距则从 $L = 100$ ns 时的 5.3 倍缩小到 $L = 500$ ns 时的 2.5 倍。线路越短，控制器位置带来的相对收益越大，所以这项收益在超节点内部最明显；在跨越数据中心的长链路上，线路时延本身成为总时间的主要部分。这两部分性质不同。$2L$ 由信号传播速度决定，任何路径都省不掉，是一次远程读取的物理下限；截距中高出 Load 路径的部分是抽象层的代价，其中五次 PCIe 穿越就占 1650 ns。推导值与仿真值相差的 14、11 和 81 ns，则属于第 1.3.4 节所说的第一类差距：模型漏计了仿真器模块之间的固定开销，补上这一项差距就消失了，不必再去找实现开销。

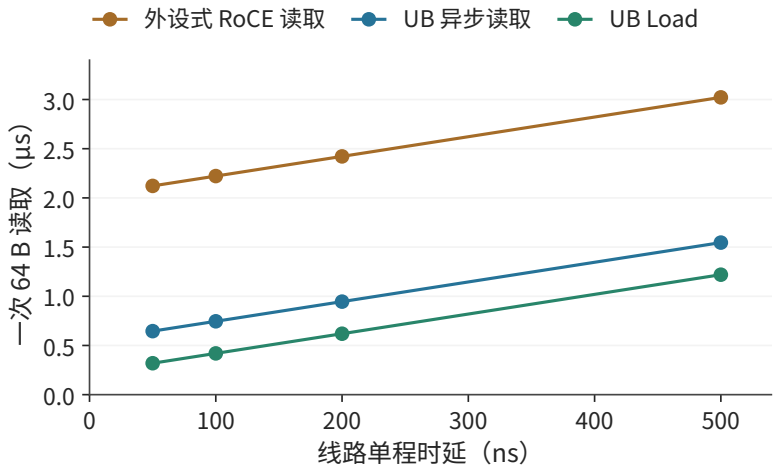



图 7-20：一次 64 B 读取的总时间随线路单程时延的变化。三条直线的斜率都是 2，截距是各自的固定开销；路径之间的差距来自固定开销，不随线路时延变化。

下面用相同的访问延迟比较并发和依赖，再讨论数据复用。前者适用于两类接口，后者是“按需远程访问还是预先搬回本地”的放置选择；直接远程读取不等于 Load，预先搬回也不对应某一种特定接口。

考察八个互不依赖的远端读取，硬件能同时处理八项，每项按上表经 PCIe 外设网卡的路径要等约 2.2 µs，提交开销已计在其中。逐个“发出、等待、使用”要约 17.8 µs；八项一起发出，结果约 2.2 µs 后一起到达。若第一次读取返回的是第二次的地址，第二次又返回第三次的地址，就只能等上一次返回后再发下一次。能同时发起多少次读取，取决于有多少个地址已知且互不依赖的请求。

并行读取利用了不同请求之间的独立性。如果多次请求访问的是同一份数据，还可以利用另一种关系：第一次访问时将数据搬到本地，后续读取反复使用该副本。假设消费者将完整读取一个 144 MiB 的不变 KV 快照，本地有足够空间容纳它。远程读取和搬回都经过本章的网卡，每方向 50 GB/s；本地读取按 H100 的 HBM 带宽 3350 GB/s 计。计入启动与目的端写入，一次远程读取约为 3.02 ms，一次搬回的固定成本约为 3.07 ms，之后每次本地读取约为 0.046 ms。[17]

[17] 不变 KV 快照的远程读取与取回。快照为 Qwen3-8B 的 1024 个 token、BF16 全层 KV，共 144 MiB；远程读取与搬回按 ConnectX-7 每方向 50 GB/s、391 个 256 B 事务在途，本地读写按 H100 SXM 的 HBM 3350 GB/s；一次搬回固定成本约 3.075 ms、每次远程读取约 3.025 ms、本地读约 0.046 ms。10% 访问反例按相同比例缩放每次访问成本，仍按整份快照计算搬回成本。

令复用次数为 $r$，两条路径的时间近似为

$$T_{\text{remote}} = 3.02r \text{ ms}, \qquad T_{\text{stage}} = (3.07 + 0.046r) \text{ ms}.$$

搬回后读取更快的条件是 $r > 3.07/(3.02 - 0.046) \approx 1.03$，即从第二次完整读取开始获益。只读取一次时，直接远程读取略快（3.02 ms 对 3.12 ms）；读取四次时，先搬回本地可将总时间从约 12.1 ms 缩短至 3.3 ms，网络只需传一次快照，传输量从 576 MiB 降至 144 MiB。图 7-21 画出两种使用路径，图 7-22 画出累计耗时的交点。

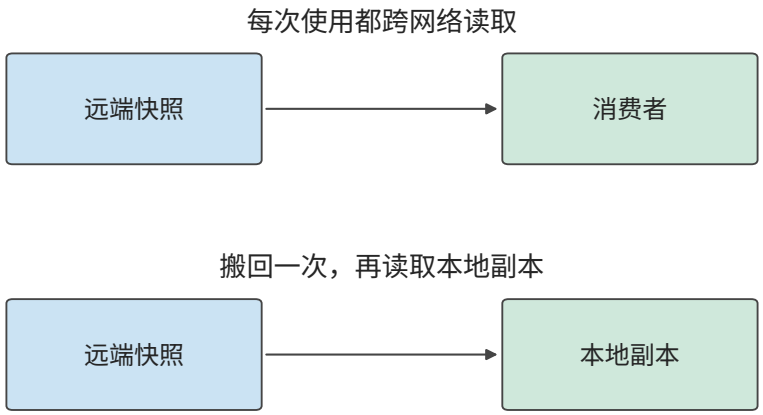



图 7-21：每次使用都访问远端，会重复传送同一份数据；下半图先搬回本地，再多次复用。两种路径处理同一份不变快照。

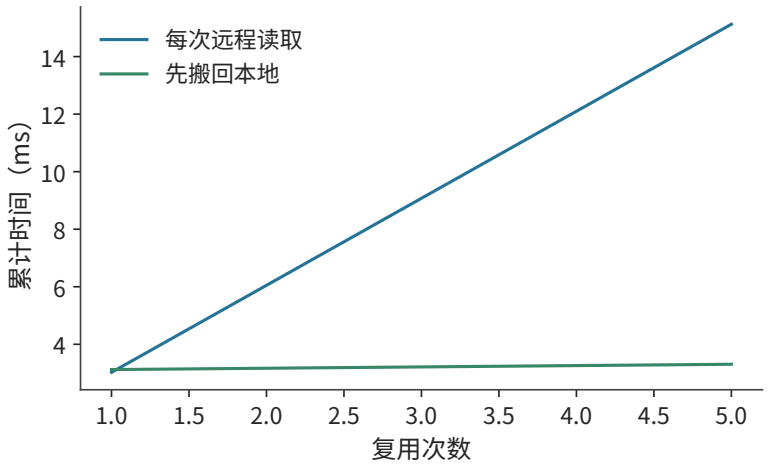



图 7-22：每次读取完整 144 MiB 快照时，两种方法的累计耗时。先搬回本地需要一次固定成本，从第二次读取开始节省时间。

两条曲线的差别来自“先付一次搬移成本，还是每次都跨网络读取”。接下来改变每次读取的范围。若一次调用只访问快照的少量位置，就可以按引用传递，让远端函数按需读取数据。设每次仅访问快照的 10%，而搬回方案仍复制全部快照。此时，按需远程读取的成本变为约 $0.302r$ ms，搬回后每次本地访问约为 0.0046 ms，交点约在 10.3 次，因而从第 11 次读取开始，先搬回本地的方案才会节省总读取时间（图 7-23）。

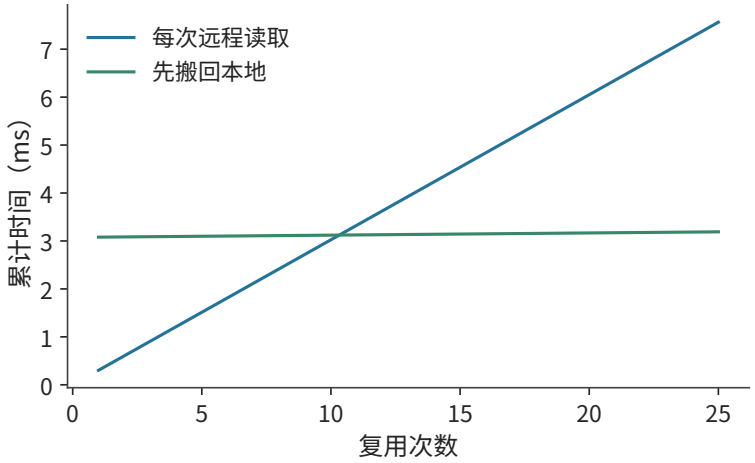



图 7-23：每次远程读取和本地读取都只访问 10%，但搬回时仍复制全部快照；因此需要更多复用才能抵消固定成本。

因此，应根据访问范围和复用次数，决定按需远程读取还是将整个快照搬回本地。URPC 是统一互联上的远程过程调用接口，以函数调用的形式提供远端访问，支持这种方式；按引用传递让被调用方只取实际需要的数据。第 7.4 节将说明何时通知其他设备使用这些对象，以及何时释放对象占用的空间。[18]

### 7.3.4 并发与吞吐模型

第 7.3.3 节用复用减少远端读取的次数；仍要经过网络的请求，则要让链路始终有数据可传。本节分析本章算例中一张网卡每方向 50 GB/s 的路径需要多少个并发请求。**请求槽位**是保存一项未完成请求的地址、长度和状态的记录：提交时分配，完成状态处理完后释放，释放前不能供新请求使用。假设每次远程访问传输 256 B，从发起请求到释放请求槽位需要 2 µs。链路在这 2 µs 内可以传送 100,000 B，相当于 390.6 次访问的数据量。为了让等待期间始终有数据可传，至少要维持 391 个在途事务，即已发出但尚未处理完的请求。图 7-24 画出一个槽位从分配到释放的过程。

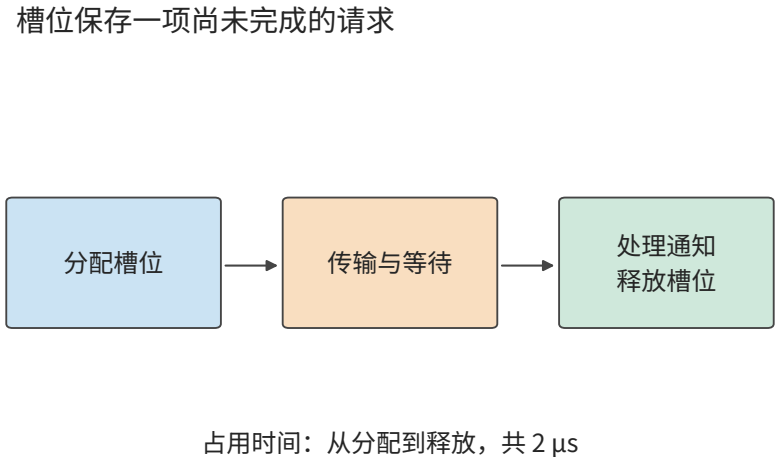



图 7-24：一项请求从分配记录开始占用槽位，经历传输与等待，完成状态被处理后归还记录。这里的 2 µs 覆盖完整占用区间。

[18] UB 规范与操作系统参考设计核对记录。

一般地，每事务载荷为 $m$，槽位占用时间为 $T$，目标带宽为 $B$，所需并发为

$$N \geq \left\lceil \frac{BT}{m} \right\rceil .$$

**带宽—时延积**是目标带宽与等待时间的乘积，表示在等待期间持续利用链路所需的在途数据量。再除以每事务载荷，就得到请求数量。链路越快，等待越长，每次返回的数据越少，系统就需要同时记录更多尚未完成的请求。

**例题 7.3：远程读取的在途槽位不足，为何无法用满出口带宽？**每个事务传输 256 B，占用槽位 2 µs，路径带宽为 50 GB/s。求使用 128 个活跃槽位时的有效读取带宽；再设请求处理单元连续两项事务的最短启动间隔为 18.6 ns（本节后文 RoCE 可靠连接实现的取值），求同时受槽位数量与启动速率限制时的有效带宽上界。

解答：128 个槽位每 2 µs 周转一次，提供的载荷为 $128 \times 256$ B，因此最多约为 16.4 GB/s。若只让一个事务在途，则为 0.128 GB/s。分配了多少槽位与实际同时使用多少槽位，是两个不同的数量。

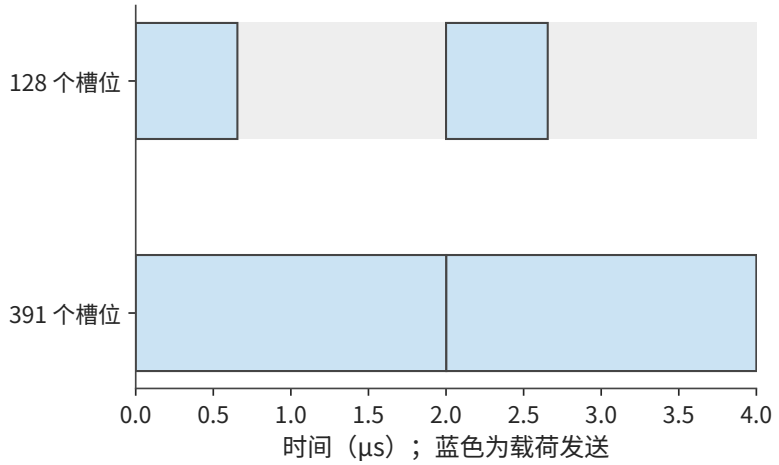



图 7-25：128 个槽位在约 0.66 µs 内全部用完，要等最早的槽位在 2 µs 释放后继续提交。391 个槽位足以覆盖等待。蓝色表示载荷发送，灰色表示链路空闲。

图 7-25 的灰色区间来自槽位尚未释放；即使槽位足够，请求也可能来不及提交。请求处理单元若每 18.6 ns 启动一项事务，每秒最多启动约 5400 万项，256 B 事务只能提供约 13.7 GB/s，比 128 个槽位给出的 16.4 GB/s 还低。即使增加到 391 个槽位，请求处理单元也来不及按这个速度提交请求。将路径带宽、在途请求数与启动速率三项约束合并，得到

$$B_{\text{eff}} \leq \min\left(B_{\text{path}}, \frac{Nm}{T}, \frac{m}{\delta}\right),$$

其中，$T$ 是一项请求占用槽位的时间，$\delta$ 是连续两项请求之间的最短启动间隔。前者决定槽位多久可以复用，后者决定每秒最多能发起多少请求。图 7-26 对比了只考虑槽位与再加上发起速率约束时的有效带宽。[19]

**讨论：远程读取需要多少并发才能达到 50 GB/s？**保持 256 B 事务，启动间隔需缩至 $256/(50 \times 10^9) = 5.12$ ns，比下文 UB 实现的 6.2 ns 还短，并至少维持 391 项在途工作。另一条路是合并相邻访问：每次传输改为 4

[19]远程读取窗口、逐次等待：路径 50 GB/s、每事务 256 B、槽位占用 2 µs。启动间隔 18.6 ns 与 6.2 ns 取自UB 互联计算的请求速率一项（OpenURMA 工具链中 RoCE 可靠连接每项请求 6 个周期、UB 每项 2 个周期，时钟 322 MHz）。公式中的请求等待与服务启动间隔分别定义。

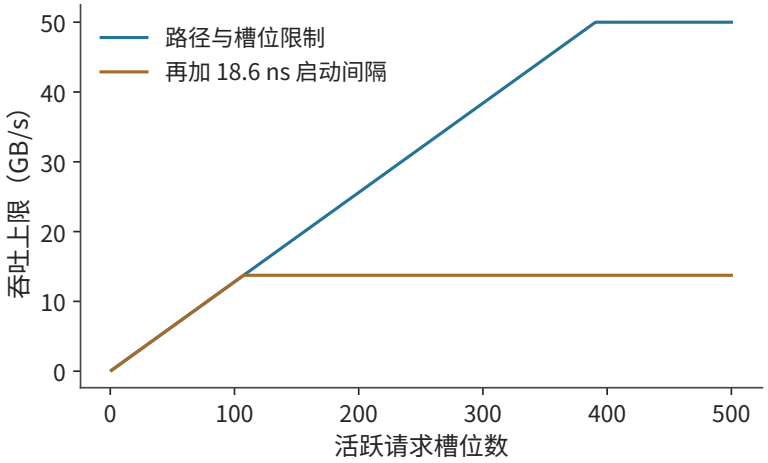



图 7-26：蓝线只考虑链路和槽位，橙线再加入每 18.6 ns 发起一次请求的约束。增加槽位使蓝线升高，橙线仍受约 13.7 GB/s 的提交速率限制。

KiB 后，启动间隔只要不超过 82 ns 就能达到 50 GB/s，18.6 ns 的间隔对应约 220 GB/s，远超链路；在等待时间为 2 µs 时，只需 25 个槽位就能支持 50 GB/s。前者提高事务率，后者让每次请求传输更多数据，降低单位字节的提交开销。

启动间隔 $\delta$ 的一个具体来源见 OpenURMA 的实现：发送流水线中最慢的一级每 2 个时钟周期接受一项请求，在 322 MHz 下 $\delta \approx 6.2$ ns，每秒最多约 1.61 亿项请求。用同一工具链实现的 RoCE 可靠连接对照中，每条连接的序号分配存在先后依赖，每项请求要占 6 个周期，$\delta \approx 18.6$ ns，每秒约 0.54 亿项。仿真用 256 项的连续突发测得的持续速率分别为每微秒 150 项和 54 项：RoCE 与推导的 53.7 项一致，UB 比推导的 161 项低约 7%；突发加长到 1000 项时，UB 升到每微秒约 159 项，与推导值相差约 1%。代入 $m/\delta$：每项 64 B 时两者分别为 10.3 GB/s 和 3.4 GB/s，每项 4 KiB 时为 660 GB/s 和 220 GB/s，后两个数都远高于本章 400 Gbit/s 网卡每方向的 50 GB/s。即小载荷时瓶颈在请求速率，大载荷时瓶颈才在线路。[20]

**读与写为什么受不同限制。**式中的 $N$ 和 $\delta$ 在 PCIe 上有明确的来源。PCIe 是分组交换的总线，每次 DMA 读或写都是一个事务层报文（TLP）。写是 posted 事务：报文带着地址和数据发出，发出即完成，不等回应。读是 non-posted 事务：请求报文带一个标签发出，数据由一个或多个完成报文带回，靠标签对应到原来的请求。发起方同时在途的读请求数受两项限制：接收方为每类事务预先发放的信用，以及发起方能分配的标签数。图 7-27 画出两种事务。

信用与标签的限制在实际平台上就能观察到。笔者在 KV-Direct 中测量过这两项限制。所用的可编程网卡经 PCIe Gen3 x8 访问主机内存，理论带宽 7.87 GB/s；每个 64 B 的 DMA 读写都要带 26 B 的报文头和填充，按报文开销算的上限是 5.6 GB/s，即每秒 8700 万次操作。随机 DMA 读的往返约 1050 ns，要用满链路需要 92 个读同时在途；但主机端为 DMA 读发放的信用只有 84 个，FPGA 的 DMA 引擎又只支持 64 个标签，于是在途读最多 64 个，除以 1050 ns 的往返，上限约为每秒 6100 万次；实测 64 B 随机读每秒 6000 万次，与这个上限基本一致。写不需要回应、不占标签，实测接近报文开销给出的上限。图 7-28 把这几道上限画在一起：读受在途数限制，写受报文处理速率限制，正是式中 $Nm/T$ 与 $m/\delta$ 两项分别起作用的例子。让网卡自己流水式地完成地址计算、访存和结果处理，就是为了在等待一个读结果时继续发出其他请求。这组数字也体现了第 1.3.4 节的检查顺序：先后用链路带宽、报文开销、在途标签三道上限修正模型，直到与实测只差约 1.6%；到这一步，模型已无漏项，余下的差距不必再追究。[21]

同一条 PCIe 链路被两类流量共用时，这一差别会变成不对称的争用。设 GPU 的 PCIe 链路在进入 GPU

[20]阶段延迟、请求速率和状态增长的参数与结果见UB 互联计算，运行 `python3 calculations/calc.py ub-fabric --format md` 可以复算。各阶段延迟取自 OpenURMA 论文表 7 给定的值，记录尺寸取自表 3，仿真对照值取自 §8.1 与 §8.3。推导把所有阶段串行相加，不考虑

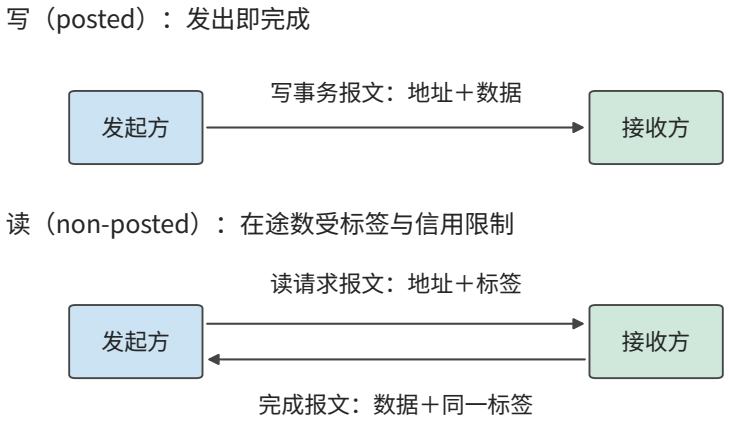



图 7-27：写是 posted 事务，一个报文发出即完成；读要先发带标签的请求报文，再等带同一标签的完成报文把数据送回，在途的读不能超过标签数和信用数。

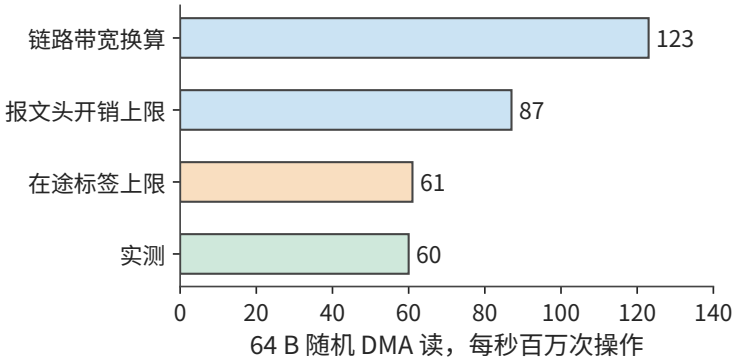



图 7-28：KV-Direct 平台上 64 B 随机 DMA 读的上限与实测。链路带宽换算为每秒 1.23 亿次，报文头开销把上限压到 8700 万次，64 个标签除以 1050 ns 往返只允许 6100 万次；实测 6000 万次，说明在途标签数是这个平台上读的瓶颈。

的方向上同时有两路流量：网卡把远端数据以 posted 写直接送进 GPU 内存；GPU 的复制引擎正在把主机内存的数据复制到 GPU，它发出的是读请求，数据以完成报文回到 GPU。链路一旦饱和，posted 写按协议必须持续推进，而读这一方的在途标签用完后就发不出新请求，只能等完成报文回来，分到的带宽远小于一半。离开 GPU 的方向则不同：GPU 向主机的复制是 GPU 发出的 posted 写，远端读 GPU 内存则要 GPU 回复完成报文，两路都要先从 HBM 取出数据、再经 GPU 的发送通路送上链路，瓶颈在 GPU 内部而不在链路，两路的退化更接近。图 7-29 画出两个方向。判断这类争用时，先分清每一路是 posted 写还是带完成的读，再看它们在链路的哪一侧、经过哪个共用部件汇合。后来的 PCIe 性能测试工具 rPCIeBench 对共享 PCIe 路径的研究也显示，在途工作量与入口竞争会改变带宽分配。[22] 可以用上式判断瓶颈：是链路带宽已用尽，是请求槽位尚未释放，还是请求发起速率太低。

分层归约减少了跨服务器载荷，并发提交让网络连续传输；程序还需要知道何时可以使用这些数据，以及何时可以释放占用的空间。

---

并发请求之间的重叠。

[21] 李博杰等，KV-Direct: High-Performance In-Memory Key-Value Store with Programmable NIC，SOSP 2017，§2.4 与图 3；PCIe 信用与标签的说明另见笔者博士论文第 5 章。数字按原平台（PCIe Gen3 x8、FPGA 网卡）引用，其他平台的信用数、标签数和延迟不同，但读受在途数限制、写受报文速率限制的结构相同。

[22] Hou 等，*Understanding Routable PCIe Performance for Composable Infrastructures*，NSDI 2024，实验使用 PCIe Gen3 平台；PCIe 路径与诊断。

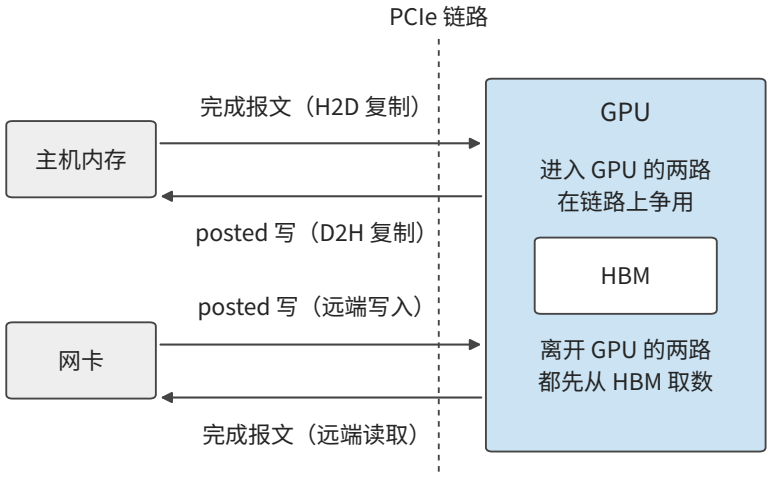



图 7-29：进入 GPU 的方向上，网卡的 posted 写与主机内存返回给复制引擎的完成报文争用链路，前者不等回应，后者受在途标签限制。离开 GPU 的方向上，D2H 复制的 posted 写与远端读取的完成报文都要先从 HBM 取数，共用 GPU 内部的发送通路。

**实验 7-8 ·延伸：用满远程读取带宽需要多少在途事务**

在例题 7.3 中，将槽位的占用时间从 2 µs 增为 4 µs；该时间从请求发出算起，到槽位可以复用为止。分别求每次传输 256 B 和 4 KiB 时，维持目标带宽所需的在途事务数。固定 18.6 ns 启动间隔，求达到 50 GB/s 所需的最小事务大小；再换成 UB 实现的 6.2 ns 重算。最后将独立地址改成指针链，解释完成整条指针链的读取所需时间如何随链长增长。

## 7.4 数据交接、顺序与状态管理

### 7.4.1 数据分片的生成、使用与缓冲区复用

数据送到以后，接收方可能还没开始用，也可能正在使用。这像把装有材料的托盘交给下一道工序：托盘已经送到，对方仍需要时间处理其中的材料，在用完之前不能把托盘中的材料换成下一批。对应到通信，发送完成、数据对接收方可见、接收方消费完成，需要分别判断。第 7.3.4 节把槽位占用时间当作已知条件，本节沿这些事件确定缓冲区何时可以释放。

在分层归约中，卡 0 将本地分片交给卡 8，卡 8 把它与自己的分片相加。发送方要知道源缓冲何时可以改写，接收方要知道目的缓冲何时可以读取，下一次写入还要知道这块目的缓冲何时可以覆盖。三者对应不同的事件。

一份分片从生成到释放，要经过以下过程：生成数据、写入远端、使数据对接收方可见、发出就绪通知、读取并处理数据，最后释放空间。源缓冲能否复用，要看接口的完成语义是否保证传输不再读它；接收方要开始用数据，先要保证写入对它已经可见，并按约定发布和查看就绪通知；目的缓冲能否覆盖，还要等接收方用完其中的数据。网络传输完成本身不能代替“用完了”的通知。

例如，发送在 5 µs 完成，消费者在 8 µs 开始读取、12 µs 结束。发送方在发送完成后就可以复用源缓冲，目的缓冲则要到 12 µs 才能覆盖。如果下一次写入在 6 µs 覆盖目的地址，消费者读到的就是下一份数据；网络传输的每个字节都正确，接收程序却使用了错误的数据。

图 7-30 中两条占用区间结束于不同事件。把“必须等哪个事件发生”画成箭头，就得到描述正确交接的依赖图：节点表示工作，边 $u \to v$ 表示 $v$ 必须在 $u$ 完成后开始。对每个节点，取所有前驱的最晚完成时刻，再加

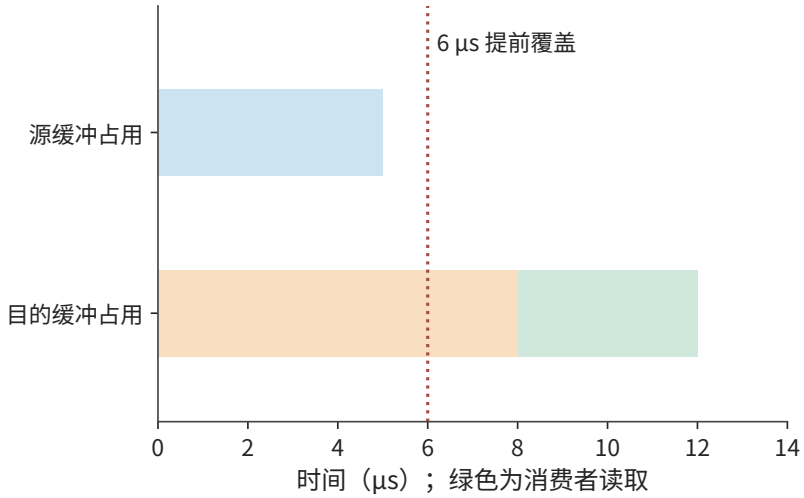



图 7-30：源缓冲占用到 5 µs 发送完成；目的缓冲持续占用到 12 µs 消费者用完。绿色为 8–12 µs 的读取区间。虚线标出 6 µs 提前覆盖目的地址的错误操作。

自身执行时间，就得到其完成时刻；从起点到终点耗时最长的那条链就是关键路径。

这一模型同时解释正确性与性能。缺少“数据可见 → 发出通知”的依赖会让消费者读到未发布的数据；多出“无关传输也必须等待”的边会增加等待。状态管理的任务，是记录哪些节点已经完成、哪些依赖已经满足，以及哪些资源可以归还。

缓冲区的归属也影响复制次数。通信库若为发送另设一块已注册的缓冲区，计算结果就要先复制进去，元数据还要与数据打包，接收方再拆开、搬到连续的区域，这些复制和打包都占用 SM 时间。让计算直接写入已注册的缓冲区可以省去这一步，但本节的事件顺序随之变严：计算写完并对网卡可见之后才能发起发送，发送真正结束之前不能覆盖这块缓冲，接收方也要按同样的规则判断何时可以读、何时可以回收。[23]

### 7.4.2 事务层与传输层分离：Jetty 与共享状态

这些依赖和完成状态需要由系统保存。每增加一条通信关系，是否都要再保存一套相同的传输记录？十六个参与者之间的归约只涉及少量通信关系。系统扩展后，一个进程有多个线程，每个线程又访问多个远端目标，关系数量便按乘积增长。应用需要记录谁提交了请求、完成通知应发送给谁；传输层需要维护序号、确认、重传和拥塞状态。前者标识应用，后者保证数据可靠传输。

**应用端点**是程序提交通信和接收完成通知的逻辑身份；**传输状态**则保存序号、确认、重传与发送速率等通信进度。取 64 个应用端点，每个访问 128 个远端目标，共有 8192 条关系。如果为每条关系单独保存一份 1 KiB 的传输状态，仅传输部分就占 8 MiB。若指向同一目标的关系共享一份传输状态，只需 128 份，共 128 KiB。节省的是为同一目标的不同通信关系分别保存的传输状态。

应用身份仍需保留。每个端点占 256 B，共 16 KiB；每条关系的绑定信息占 64 B，共 512 KiB。加上这些部分，逐关系独占的状态总量约为 8.52 MiB，按目标共享约为 0.64 MiB。传输状态的份数降至原来的 1/64，状态总量降至原来的约 1/13。[24]

设端点数为 $L$、目标数为 $P$，每个端点、每条关系的绑定信息和每份传输状态分别占 $e, r, t$ bytes，则

[23] DeepEP README 快照中为 NVLink 与 RDMA 分别预留的缓冲区大小说明；复制与打包的开销按本节的事件模型推断，不是对某一版本的测量。

[24] 活跃关系与传输状态、八类隔离。模型分别计入端点、关系绑定和传输状态三类容量。

$$S_{\text{private}} = Le + LP(r+t), \qquad S_{\text{shared}} = Le + LPr + Pt.$$

OpenURMA 论文用另一种口径描述这种分离：一个本地接口上有 $L$ 个应用端点、要访问 $P$ 台远端主机时，核心的端点记录和传输上下文按 $O(L+P)$ 增长；如果每条应用关系都独占一条可靠连接，这部分就按 $O(LP)$ 增长。这一结论只针对可以共享的核心硬件状态，软件映射、权限、未完成请求和缓存并不都只占加法量级的空间。上式保留了 $LPr$ 一项，因为本教学方案显式计入了逐关系的绑定记录；本小节末尾再用实现中的记录尺寸重新计算一次。[25]

两个式子直接给出了共享节省的空间：$LPt$ 变成 $Pt$，应用关系的 $LPr$ 仍在。UB 把 Jetty 与传输通道分开（第 6.5.5 节），采用的就是这种设计：同一份传输状态可以服务多条应用关系。[26]

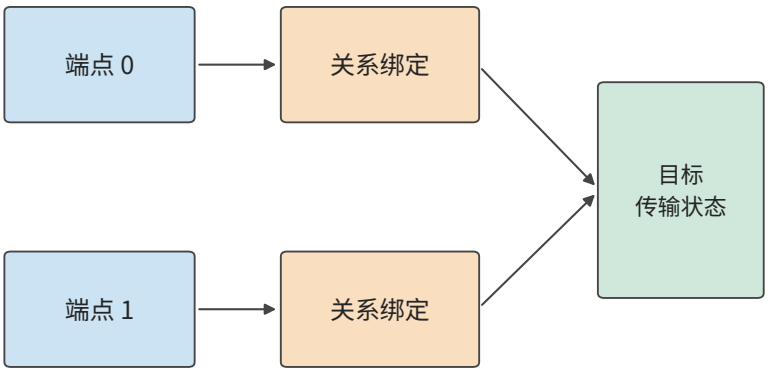



图 7-31：两个应用端点分别保存身份与绑定记录，关系绑定指向同一目标的共享传输状态。共享部分维护传输进度，端点仍能区分各自请求。

图 7-31 中多条关系复用同一份传输状态，节省了存储，也可能争用这条通道的发送窗口、调度机会和恢复资源。分层不要求所有事务排进一条全局有序的队列，应用事务的执行顺序、完成顺序与报文到达顺序仍要分别规定。若实现里用共享的先进先出队列或顺序等待，丢包和长请求就会造成队头阻塞（head-of-line blocking，队首的请求停住时，排在后面的无关请求也只能等待）；让独立的事务继续推进可以缩小影响，但消不掉对共享带宽和窗口的竞争。一条通信关系的大量未完成请求占用共享队列，其他关系的短请求就必须等待。若为不同业务建立 $K$ 组独立的传输状态，总状态变成

$$S(K) = Le + LPr + KPt.$$

按上述配置计算，固定部分为 528 KiB，每增加一组独立传输状态，就增加 128 KiB。1 MiB 的存储空间最多容纳三组独立传输状态；四组需要 1040 KiB，已经超出容量；八组约需 1.52 MiB。图 7-32 画出这几种共

[25] 李博杰，〈Unified Bus 背后的思考〉，Jetty、事务序与 Load/Store 各节；OpenURMA 论文，2026-06-02 修订版（arXiv:2605.28717），§3 设计、§7–§9 状态与延迟、§10 顺序、§12 传输、§13 结果汇总。本次整合的来源与适用范围。

[26] UB 规范与操作系统参考设计核对记录。

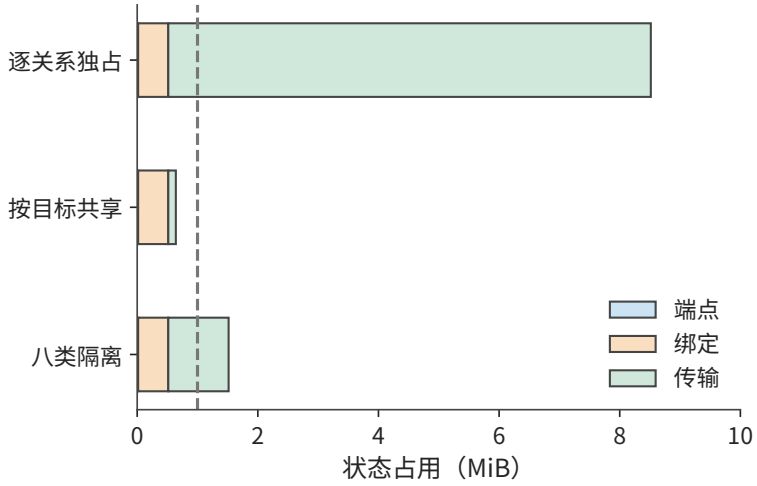



图 7-32：64 个端点访问 128 个目标。蓝色为端点记录，橙色为关系绑定，绿色为传输状态。虚线为 1 MiB 容量；划分八个隔离组时，为各目标保存八份独立的传输状态。

享策略的状态容量。因此，业务隔离需要在存储容量内安排：为关键业务单独分配传输状态，让其他业务共享，可以减少关键业务受到的干扰。

除了在不同关系之间共享，状态还可以分层存储。完整状态放在主存，频繁使用的部分缓存在网卡片上。共享可以减少重复记录，使同样大小的缓存容纳更多通信关系。经常使用的状态留在缓存中，也就减少了从主存读取状态的次数。这样，节省状态空间还可以降低请求处理开销。

**实现中的记录尺寸与状态总量。**前述 256、64 和 1024 B 是本节给定的取值。OpenURMA 实现的状态结构给出了一组可以核对的尺寸：每个 Jetty 20 B，每个已注册的内存段 32 B，每条传输通道 56 B；作为对照的 RoCE 可靠连接实现为每条连接保存 512 B 的 QP 上下文。设本地 $N$ 个端点访问 $M$ 个远端端点，两种组织方式下网卡保存的状态为

$$S_{\text{pair}} = 512NM + 32N, \qquad S_{\text{UB}} = 52N + 56M.$$

与前面的公式对照，这里 $e = 52$、$t = 56$，而 $r = 0$：该实现把关系绑定放在软件映射里，网卡不为每条关系保存硬件状态。$N = M = 1024$ 时，逐对连接需要约 512 MiB，端点加通道只需 108 KiB，相差约 4855 倍；即使把 Jetty 记录补齐规范中的全部字段（48 B），比值仍在三千倍以上。图 7-33 用对数坐标画出两条曲线：一条斜率为 2，一条斜率为 1，二者的比值随 $N$ 线性增长。

**状态总量与片上缓存溢出。**网卡把上下文缓存在片上，容纳不下的部分留在主机内存。取缓存容量 256 KiB，逐对连接在 $512N^2 + 32N > 262144$、即 $N \geq 23$ 时溢出；端点加通道在 $108N > 262144$、即 $N \geq 2428$ 时溢出。溢出以后每次操作都要重新读取上下文，代价取决于上下文放在哪里：PCIe 外设网卡从主机内存读取，发起端和目标端各做一次 PCIe DMA 读，每次操作增加约 1000 ns；片上总线控制器从本地内存读取，两端各做一次片上总线穿越和一次内存访问，增加约 200 ns。图 7-34 把第 7.3.3 节推导的两条路径的延迟画成活跃端点数的函数。训练中的全交换通信通常涉及几十到几千个端点，正好落在这一区间；在这一区间里，逐对连接的每次读取都要付出重新读取上下文的代价。笔者的仿真器按条目数而不是字节数判断溢出（RoCE 512 条，UB 2048 条），因此把 UB 的溢出点放在 1024 个端点；两种口径下结论相同：端点加通道的溢出点晚一个数量级以上，溢出后的代价也小一个数量级。[27]

[27]阶段延迟、请求速率和状态增长的参数与结果见UB 互联计算，运行 `python3 calculations/calc.py ub-fabric --format md` 可以复算。各阶段延迟取自 OpenURMA 论文表 7 给定的值，记录尺寸取自表 3，仿真对照值取自 §8.1 与 §8.3。推导把所有阶段串行相加，不考虑

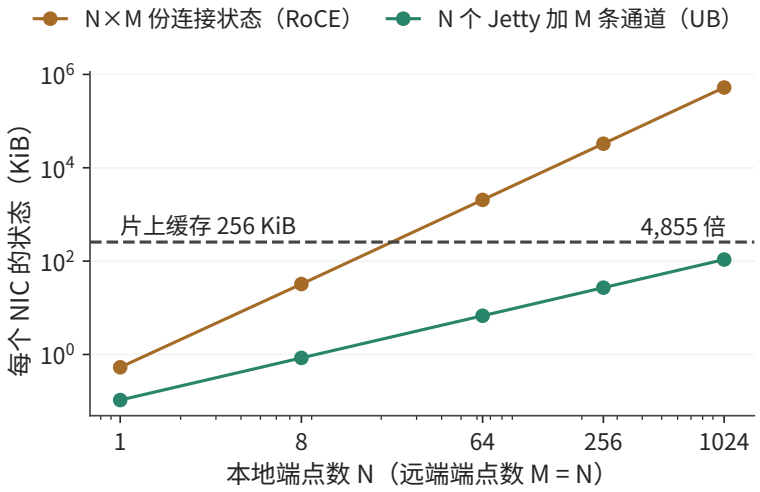



图 7-33：N 个本地端点访问 M = N 个远端端点时，两种组织方式下网卡保存的状态。逐对连接在 N 略大于 20 时就超过 256 KiB 的片上缓存，端点加通道到 N = 1024 时仍不到缓存的一半。

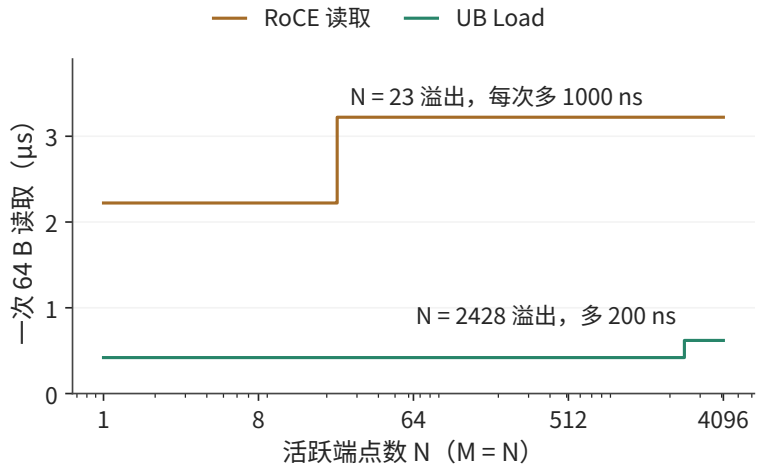



图 7-34：一次 64 B 读取的延迟随活跃端点数的变化，片上缓存 256 KiB。逐对连接在 23 个端点处上升 1000 ns，端点加通道在 2428 个端点处上升 200 ns。横轴为对数坐标。

**实验 7-9 ·延伸：共享传输状态能支持多少隔离组**

保持 64 个端点，只访问一个远端目标，分别计算独占与共享传输状态时所需的总存储空间。再将活跃目标数量改为 128 个，将可用存储空间提高到 2 MiB，求最多容纳多少个完整的隔离组。比较增加端点数和增加目标数对固定状态与传输状态的影响。最后改用实现中的记录尺寸（Jetty 20 B、传输通道 56 B、连接上下文 512 B），求片上缓存为 1 MiB 时两种组织方式各自的溢出端点数。

### 7.4.3 操作依赖与故障隔离

第 7.4.2 节通过独立传输组减少不同业务之间的干扰。同一组操作内部也可能出现不必要的等待，这取决于系统强制哪些操作依次完成。以卡 0 发布分片的过程为例。操作 A 写数据 D，操作 B 发布“D 已就绪”的通知，操作 C 传送另一块无关数据。要正确使用 D，必须保持 A→B 的顺序，C 可以使用另一条独立传输通路。

**例题 7.4：减少顺序约束能让哪些操作更早完成？** A 写入需 20 µs，随后恢复需 80 µs，恢复结束后 D 可见；

并发请求之间的重叠。

B 需 2 µs；C 需 10 µs。比较两种安排：A、B、C 依次完成，以及仅要求 A 完成后才能执行 B。

解答：写入、恢复和通知这一依赖链的耗时是 $20 + 80 + 2 = 102$ µs。串行安排让 C 等到 102 µs 才开始，全组在 112 µs 完成。保留必要依赖时，C 从起点开始，10 µs 完成；发布链仍到 102 µs 才结束。全组提前 10 µs，C 却提前 102 µs。图 7-35 和图 7-36 分别画出这两种安排。

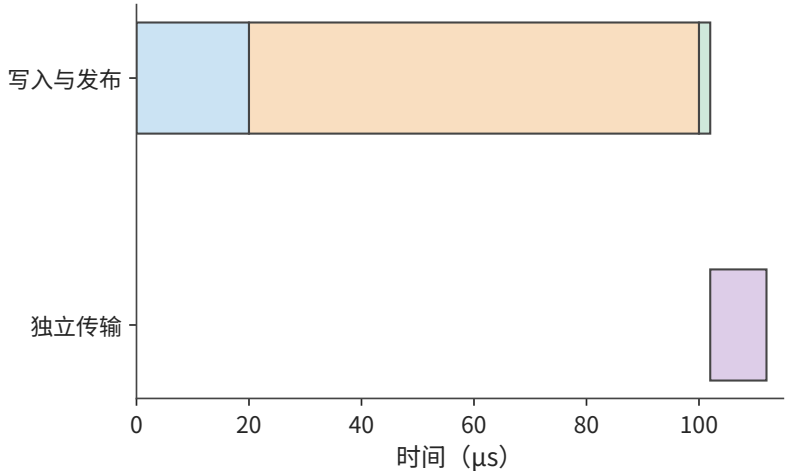



图 7-35：将独立传输也排在发布之后：先写入 20 µs，恢复并使数据可见 80 µs，通知 2 µs，再执行独立传输 10 µs。

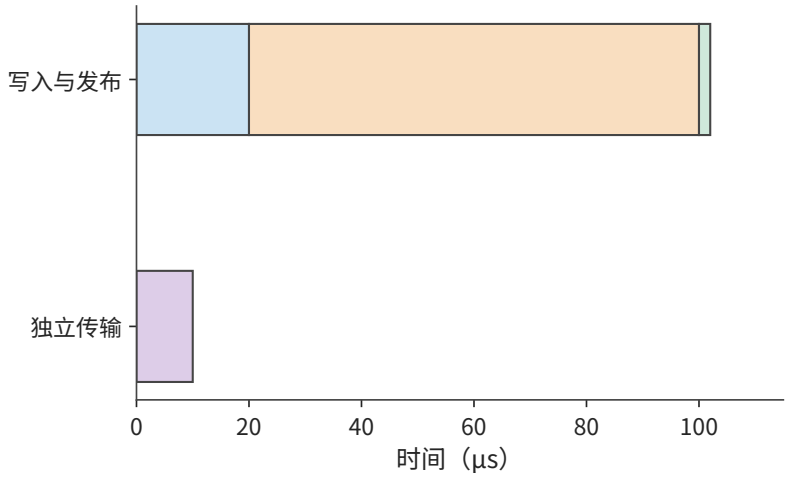



图 7-36：写入、恢复、通知仍依次进行；使用独立资源的传输从 0 µs 开始。蓝色为写入，橙色为恢复，绿色为通知，紫色为独立传输。

恢复时间若增至 200 µs，串行安排中的 C 要到 232 µs 才完成，独立安排中的 C 仍在 10 µs 完成。去掉多余的依赖，就把恢复造成的停顿限制在真正需要这份数据的那条链上。对等着 C 的下游任务来说，这种隔离比全组提前 10 µs 重要得多。[28]

全局顺序让上层容易描述操作的先后关系，也把独立工作放进了同一条等待链。计算图已经明确 A→B 的发布依赖后，接口和运行时可以围绕这条依赖提供顺序保证，让 C 独立推进。应用提供更准确的依赖信息，网络据此只为相关操作保证顺序，既能保证接收方收到通知后读到已经写入的数据，也能减少不必要的等待。

[28] 必要依赖与旧值反例、共享资源变体、远端排序研究与固定 NVSHMEM 实现阅读。目标端排序属于需要新增硬件支持的设计；当前网卡的参照实验用于观察请求与完成时序。发布链的 20 + 80 + 2 µs 与旧值反例的 1–6 µs 是两组独立输入。“全部依次执行”作为串行对照策略。

再假设所有操作共用一个处理单元：从 A 开始写入到 B 完成通知，该单元一直被占用，C 只能随后执行。此时资源使用本身增加了发布链到 C 的顺序边，两种安排都到 112 µs 完成。程序依赖和资源依赖因此要画在同一张图里：减少程序中的顺序约束后，资源约束仍会限制并行程度。

**设计案例：按需指定顺序的硬件开销。**笔者的 OpenURMA 实现把 UB 的四种顺序服务模式和三种执行标签放在同一条发送流水线上实现。周期级仿真显示：不论请求要求哪一种顺序，从提交到第一个报文出线都是 24 个周期；只有当请求明确要求等待前序操作完成时，顺序跟踪器才会让它等待，前序未完成的操作最多四项时，等待不超过 50 个周期，约 155 ns；不要求顺序的请求不承担这笔开销。更重要的是隔离效果：把一个发起方的请求停在等待前序操作的位置，另外四个发起方提交的 8 个不要求顺序的请求仍然在 78 个周期（约 242 ns）内全部发出，被停住的请求没有拖累其他发起方。作为对照，RoCE 可靠连接中同一个 QP 里的请求必须严格按序执行，一个请求等待，后面的请求就全部等待。这正是例题 7.4 里把 C 排在 A、B 之后的那种安排在硬件上的表现。[29]

> **设计背景：全局顺序如何简化复制与协作?**
> 笔者在 1Pipe 研究中探索过由网络提供全局全序的设计。所有参与者按同一顺序观察操作，上层的复制和协作就能使用更简单的顺序模型。进一步处理故障时，顺序与交付需要分别解决：发送方发到一半退出，系统要确定哪些节点已经收到；接收方永久退出，系统要决定哪些工作继续执行。这些问题要求把顺序保证与交付、恢复机制分开，并把应用真正需要的依赖清楚地表达出来。

上述例子要求写入完成后才能发出通知。接收方还必须在正确的时刻读取数据，否则即使通知顺序正确，也可能使用旧值。设 D 初值为 0，在 2 µs 更新为 1；就绪标志在 3 µs 可见。消费者在 1 µs 提前读 D，在 4 µs 读标志。此时它保存的是新标志和旧数据。即使到 5 µs 再按“标志、数据”顺序返回读取结果，保存下来的 D 仍是 0。

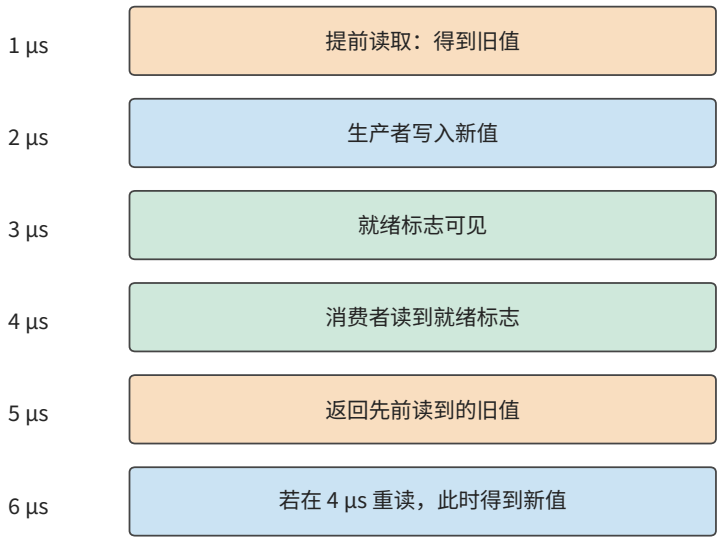



图 7-37：按时间依次列出实际读取、数据更新、就绪通知和结果返回。5 µs 返回的仍是 1 µs 读到的旧值；若在 4 µs 发现冲突并重读，6 µs 得到新值。

图 7-37 中，把表示结果返回时刻的空心点向后移动，并不会移动表示实际取值时刻的实心点。这一反例说明，顺序要求必须落实到读取数据的时刻。消费者可以在观察标志之后再读 D，也可以提前读、检查冲突并重

[29]李博杰，〈Unified Bus 背后的思考〉，Jetty、事务序与 Load/Store 各节；OpenURMA 论文，2026-06-02 修订版（arXiv:2605.28717），§3 设计、§7–§9 状态与延迟、§10 顺序、§12 传输、§13 结果汇总。本次整合的来源与适用范围。

读。若在 4 µs 发现冲突后重读需要 2 µs，就在 6 µs 得到正确数据。在接收方排序是把等待挪到了接收一侧，冲突检查则负责判断哪些提前读到的结果仍然有效。要靠顺序保证来简化编程，就必须保存这些状态并做相应的检查。

### 7.4.4 请求槽位的释放与背压

第 7.3.4 节的请求槽位要等软件处理完完成通知才释放，新请求才能使用它。第 7.4.1 节讨论的数据缓冲区有另外的使用期限：接收方尚未用完数据时，该缓冲区仍需保留。如果传输加快，完成通知却处理得不够及时，就会出现大量“操作已完成、槽位未释放”的请求。

取 16 次操作，每次传 8 KiB，最多每 1 µs 提交一次，提交后 5 µs 完成。若有 16 个槽位，最后一次在 15 µs 提交，20 µs 时全部传完。软件每 20 µs 读取并处理四条完成通知，16 条通知需要四轮，对应的槽位到 80 µs 才全部释放。

若只有八个槽位，最初八次提交迅速占满槽位。第九次提交要等到 20 µs 时软件处理完成通知、释放槽位之后，后续请求继续分批提交，全部传输到 48 µs 才结束。把槽位加倍，传输完成时刻从 48 提前到 20 µs，所有槽位释放完毕仍要到 80 µs。[30]

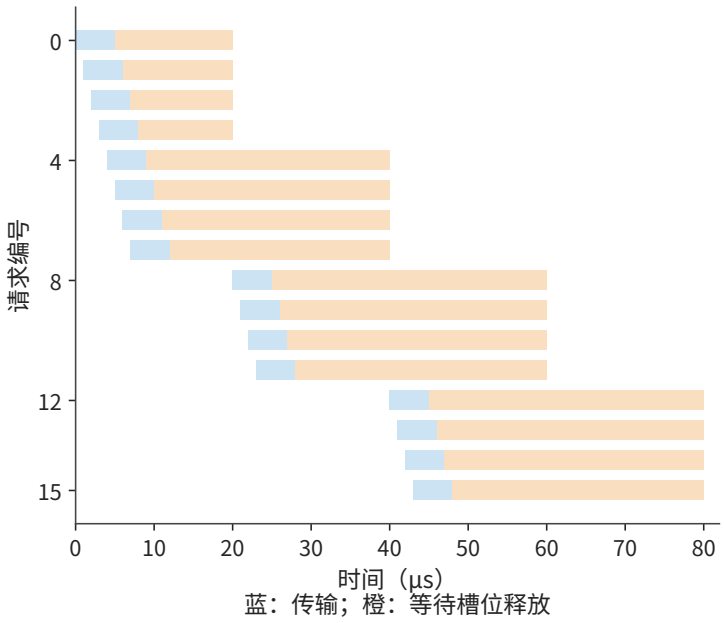



图 7-38：蓝色表示提交到传输完成，橙色表示等待软件处理完成通知，橙色结束才释放槽位。所有请求的时刻取自事件记录。

对比图 7-38 和图 7-39，增加槽位让蓝色传输条带提前了，右侧较长的橙色等待却仍然存在。决定持续吞吐的，是这些槽位最终释放得有多快。用并发与吞吐模型看，处理完成通知的线程每秒只处理 $4/(20\times10^{-6})=200000$ 项；每项 8 KiB，对应约 1.64 GB/s 的持续处理能力。采用这样的操作大小，要达到一张网卡的 50 GB/s，平均每秒需处理约 610 万项，约每 0.164 µs 一项。增加槽位只能容纳短时间内突然增多的请求，持续负载需要提高完成通知的处理速率或扩大每项操作的载荷。

完成通知处理速度不足时，上游提交就要受到约束，这就是第 5 章的背压：请求槽位用满后，端点就暂停或放慢提交，直到有槽位释放；背压靠限制未完成请求数来控制上游，防止未完成的工作无限增长。若缓冲区要等远端程序用完数据才能释放，背压就会跨设备传播；许多工作共享传输状态时，传播范围还会更大。

[30] 完成通知处理与槽位释放、增加槽位。轮询周期、每次轮询处理的完成通知数量与槽位占用时间为教学条件。

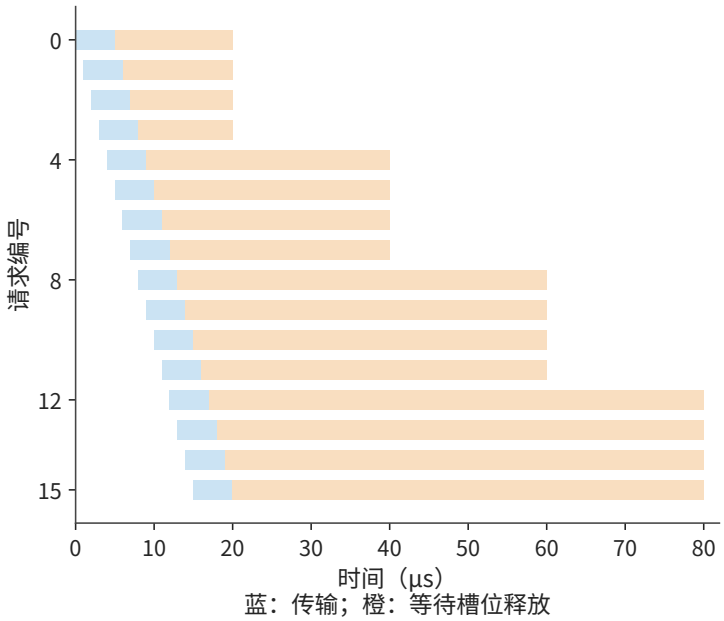



图 7-39：增加槽位后，后续请求更早提交。完成通知仍每 20 μs 批量处理四项，最后一个槽位仍在 80 μs 释放。两图时间轴相同。

发生故障后，也必须先结束旧请求，再释放它们占用的空间。取消请求之后，系统先停止新提交，再等待或隔离旧的在途访问，最后释放空间。否则，迟到的旧写入会覆盖新对象。给每次复用的对象分配新的版本号、撤销访问权、隔离失联端点，都是为了防止旧请求访问已经重新分配的空间。超时触发恢复后，系统要先确认旧请求已经结束，或阻止其继续访问，再复用空间。多个端点共享网络时，同样的到达速度与处理速度之差，会变成交换机里的队列。

> **实验 7-10 ·核心：放宽操作顺序如何减少等待并保持数据正确**
> 将例题 7.4 中独立操作 C 的耗时改为 50 μs，求两种安排的整组操作和独立操作的完成时间。再假设消费者收到通知后需要 30 μs 处理数据，标出目的缓冲区最早可以写入新数据的时刻。对旧值反例分别画出等待后读取、提前读取后重读两条执行路径。

## 7.5 共享网络中的拥塞与可靠性

### 7.5.1 从固定流量到随时间变化的需求

在前几节的配置上再增加一项使用两台服务器的训练作业，并把两项作业放到第 7.2.5 节的多轨拓扑上：每项作业的两台服务器分属不同的组（每组 32 台服务器），同一条 rail 上的跨服务器流量不再一跳到达，要经叶交换机的上联链路走脊层；两项作业在这条 rail 上的流被哈希到同一条 400 Gbit/s 的上联链路，每方向 50 GB/s。分层归约的跨服务器阶段让每张网卡以 50 GB/s 发送，两项作业的高峰重叠时，需求就超过了这条链路的传输能力。

**例题 7.5：平均只用掉 40% 的链路带宽，为什么仍然排队？**两项作业每 100 ms 各有 20 ms 的通信高峰，高峰速率均为 50 GB/s，其他时间不发送。假设队列初始为空，作业按上述时间表发送数据，缓冲区足够大，可以保存所有积压。

解答：两项作业的平均总需求为

$$\bar{\lambda} = 2 \times 50 \times \frac{20}{100} = 20 \text{ GB/s}.$$

与 50 GB/s 相比，平均利用率只有 40%。但两个高峰同时开始时，到达速率比出口速率高出 50 GB/s，20 ms 内积压增长到

$$Q_{\max} = (100 - 50) \text{ GB/s} \times 20 \text{ ms} = 1 \text{ GB}.$$

高峰结束后，还需 1 GB/50 GB/s = 20 ms 排空。两项作业在 20 ms 时停止发送，但出口还要再传输 20 ms 才能发完这些数据。把第二项作业推迟 20 ms，高峰完全错开，任意时刻只有 50 GB/s 到达，队列保持为空。两种安排发送同样多的数据，差别在于两项作业开始发送的相对时刻。图 7-40 画出不同重叠程度下的到达速率，图 7-41 画出对应的积压。[31]

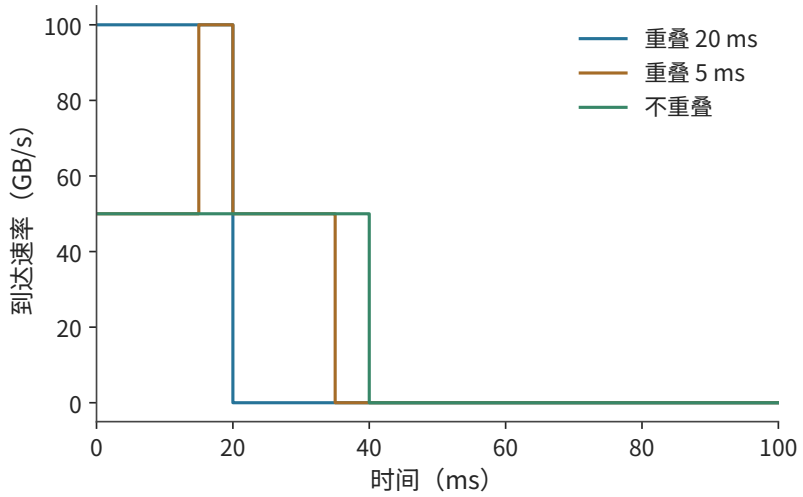



图 7-40：两作业每 100 ms 各发送 20 ms，单作业速率 50 GB/s。不同高峰重叠程度产生不同的总到达速率，共享出口为 50 GB/s。

**讨论：发送时刻偏移多少会使共享出口产生积压？**重叠时间为 $h$ 时，新增积压为 $(100-50)h$。重叠 5 ms 就积压 250 MB。若只给这次突发留出 512 KiB 缓冲，从空队列出发，允许的重叠时间约为 512 KiB/50 GB/s = 10.5 µs。即使发送时刻的误差只造成毫秒级重叠，也远远超过这份缓冲所能容忍的范围。

错开高峰的思路已经有系统实现。CASSINI 利用训练通信的周期性，安排作业的放置和通信时刻来错开高峰，并监测漂移、重新调整。[32] 本书配套实验用两个在 CPU 上运行的分布式数据并行（DDP）训练作业做过一次性错峰的对照：把其中一项推迟 50 ms 后，两者的相位在运行中继续漂移，三轮测量中整组作业的完成时间反而多了约 2% 到 3%。[33] 周期调度要持续维护相位；拥塞控制则根据实时反馈调整偏离预期的发送速率。

[31] 周期需求与队列、错峰、相位漂移。算例的到达速率按给定周期变化。

[32] Rajasekaran 等，*CASSINI: Network-Aware Job Scheduling in Machine Learning Clusters*，NSDI 2024；作业放置与相位调度。论文主实验为 24 台单 A100 40 GB 服务器、50 Gbps 网卡及 2:1 超售逻辑网络；各作业独占训练设备、共享网络。

[33] experiments/ch07/07-08：真实 CPU 训练与一次性错峰。三轮结果计入初始 50 ms 延迟，模型与优化器状态核验一致；这一对照用于分析 CPU 执行顺序。环境为共享 CPU、Gloo loopback，三轮完成时间增加 2.27%–3.32%。

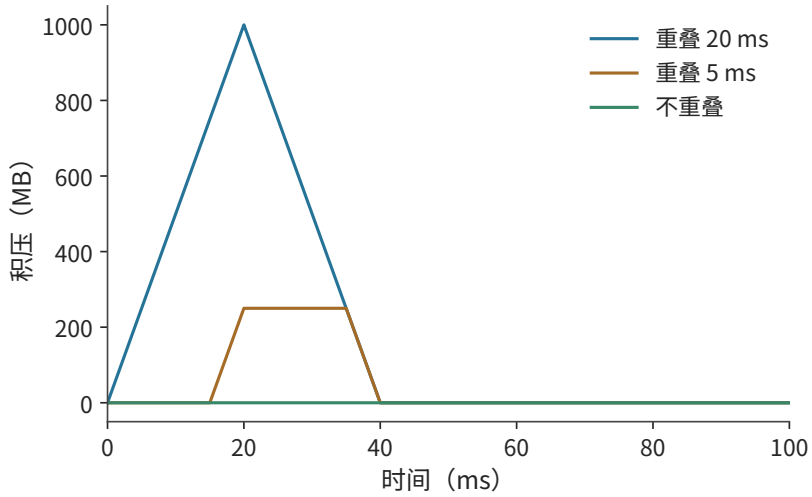



图 7-41：把到达速率与出口速率的差额随时间累积，得到 1000、250、0 MB 三种峰值。此算例按无限缓冲计算积压。

### 7.5.2 反馈延迟与缓冲容量

错峰减少了高峰重叠，但实际发送仍会偏离安排。因此，还需要在队列开始增长时，及时让发送方减速。数据到达的速度超过出口发送的速度时，多出的数据就会在队列中积压。记队列长度为 $Q$、到达速率为 $\lambda(t)$、出口速率为 $B$，队列非空且未满时，

$$\frac{dQ}{dt} = \lambda(t) - B.$$

到达速率高于发送速率，队列增长；到达速率低于发送速率，积压减少。

及时降低到达速率，有两类机制。**链路流控**让相邻的接收方在缓冲不足时暂停上游，防止接收缓冲区溢出。**端到端拥塞控制**把瓶颈信息送回发送方，降低发送速率。前者迅速阻止局部溢出，后者让源头的发送速率适应整条路径的传输能力。

沿用 100 GB/s 到达与 50 GB/s 出口，设缓冲总量为 512 KiB，已经占用 256 KiB。每微秒多到达的 50 KB 数据会占用剩余空间，这些空间只能支撑约 5.2 µs。反馈若到 20 µs 才使发送减速，会新产生 1 MB 超额数据，扣除剩余空间，约 738 KB 被丢弃。[34]

将剩余缓冲记为 $Q_{\text{free}}$，从发现拥塞到降速的时延为 $T_f$，避免这次溢出需要

$$T_f \le \frac{Q_{\text{free}}}{\lambda - B}.$$

因此，在到达和发送速率给定时，可以根据缓冲大小推算允许的反馈时延。剩余空间加倍，可容忍的反馈时延加倍；到达与出口的差额加倍，可容忍的反馈时延减半。在上述速率下，若经过 50 µs 才降速，就会多积压 2.5 MB 数据。

反馈后还要消除积压。只把发送降到 50 GB/s，到达速率恰好等于出口发送速率，已有队列保持不变；降到 40 GB/s 后，每秒有 10 GB 余量用于排空，512 KiB 约需 52 µs。从起点算，队列约在 72 µs 回到零。拥塞

[34] 有限缓冲与反馈延迟。反馈时刻设为 20 µs，反馈后的发送速率按题设调整。

控制既要阻止队列继续增长，也要为排空积压留出余量。图 7-42 画出这条反馈回路，图 7-43 画出缓冲的填满与排空。

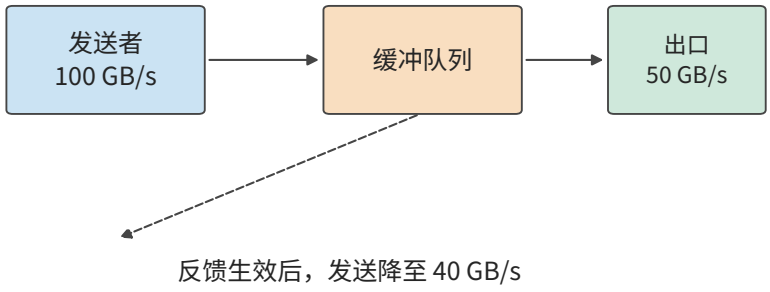



图 7-42：数据沿实线进入队列并由出口发送；虚线表示拥塞反馈返回发送方。反馈到达并生效前，原发送速率继续填充队列。

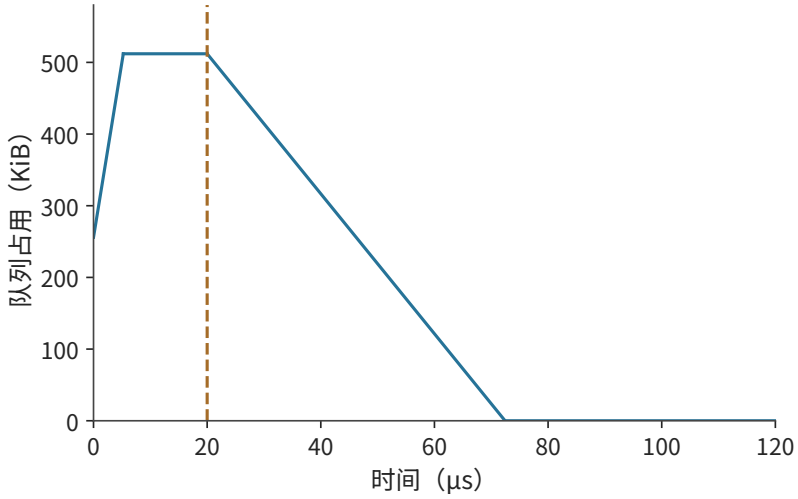



图 7-43：反馈生效之前，缓冲区还会继续填满。到达速率为 100 GB/s，出口为 50 GB/s，512 KiB 缓冲初始占用一半。5.2 μs 时填满，到 20 μs 降速前持续丢弃超额到达的数据；降至 40 GB/s 后，出口以 10 GB/s 的净速率排空积压。反馈时刻与降速幅度为给定条件。

图 7-43 中，反馈生效时刻决定队列满载状态持续多久，降速幅度决定下降段有多陡。各种拥塞控制机制改变队列，靠的都是这两个量。**显式拥塞通知**（ECN）由交换机在报文上打标记，把队列信息带回发送方；**DCQCN** 是利用这种通知调节 RDMA 发送速率的算法；基于时延的方法从往返时间的变化观察排队；UB 的 **C-AQM** 让端点与交换网络协同做主动队列管理（在队列溢出之前就依据队列状态让发送方减速）。理解这些机制，先分清各自缩短了哪一段反馈路径、发送方收到信息后改变了什么。阈值、更新步长和反馈频率共同决定队列响应的速度与幅度。

**多对一汇聚与链路流控。**上述 100 GB/s 到达来自两项作业。集合通信自身也会制造更陡的到达速率。ReduceScatter 若按直接方式实现，每个参与者把第 $j$ 片直接发给持有者 $j$，一轮就完成。$N$ 个参与者若按同一顺序逐个目标发送，每个目标都以网卡的全速 $B$ 发出，那么轮到持有者 $j$ 的时段，另外 $N-1$ 个发送方同时向它发送，通向它的交换机出口以 $(N-1)B$ 的速率收到数据；发送顺序若按 rank 错开，每个目标在任一时刻只收

到一条流。多个发送方同时向同一个出口发送的情形称为 **incast**。把 $\lambda = (N-1)B$ 代入上面的式子，每个发送方 $B = 50$ GB/s，出口也是 50 GB/s，剩余缓冲 1 MiB；$N = 8$ 时到达速率不是一条流的 50 GB/s，而是七条流合计的 350 GB/s：[35]

| $N$ | 发送方数 | 到达速率 | 超额速率 | 允许的反馈时延 |
|---|---|---|---|---|
| 8 | 7 | 350 GB/s | 300 GB/s | 3.50 µs |
| 16 | 15 | 750 GB/s | 700 GB/s | 1.50 µs |
| 64 | 63 | 3150 GB/s | 3100 GB/s | 0.34 µs |

链路流控和端到端拥塞控制这两类机制，在这里对应两段反馈距离。链路流控的一种实现是 IEEE 802.1Qbb 的**基于优先级的流控**（Priority-based Flow Control，PFC）：接收端口的队列超过阈值就向上游相邻端口发暂停帧，按优先级暂停一条链路。它的反馈距离只有一跳：30 m 线缆按 5 ns/m 计，暂停帧上行 150 ns、已在线上的数据下行 150 ns，再加一个 1500 B 报文在 50 GB/s 上串行化的 30 ns，共 0.33 µs。这 0.33 µs 内继续到达的超额数据必须有缓冲容纳，这块预留的空间称为 headroom（缓冲余量）：$N = 8$ 时为 300 GB/s $\times$ 0.33 $\mu$s = 99 KB，$N = 16$ 时 231 KB，$N = 64$ 时 1.02 MB，都不超过 1 MiB；DCQCN 论文按 1500 B 的最大传输单元（MTU）给出的一例是每端口每优先级 22.4 KB。端到端拥塞控制（ECN 标记与 DCQCN 降速）的反馈距离是一个 20 µs 的往返，这 20 µs 内涌入的超额数据为 6.0、14、62 MB，即 5.7、13.4、59.1 MiB，没有一个能放进 1 MiB 缓冲。图 7-44 画出三种 $N$ 下的队列增长与这两段反馈距离。

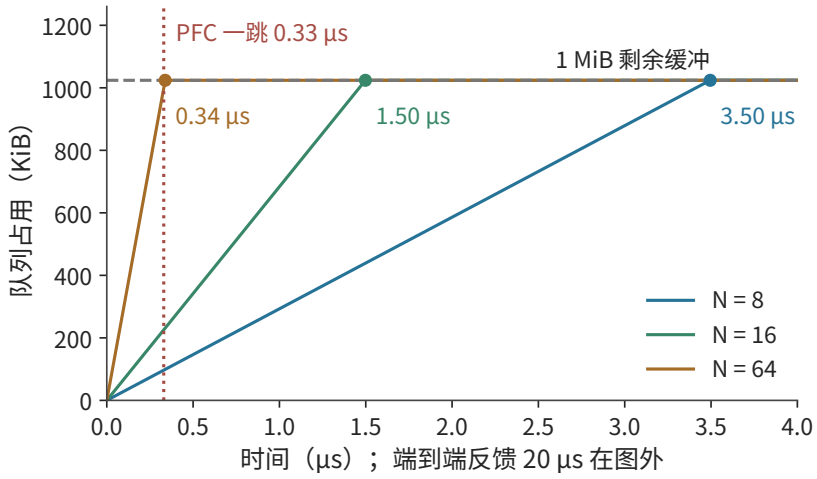



图 7-44：$N = 8$、16、64 时队列以 300、700、3100 GB/s 的超额速率增长，圆点标出 1 MiB 剩余缓冲被填满的时刻 3.50、1.50、0.34 µs。竖直虚线是 PFC 的一跳反馈距离 0.33 µs，三条线在这之前都没有填满；端到端反馈的 20 µs 在图的横轴之外。

两段距离决定了分工：PFC 在 0.33 µs 内挡住相邻一跳，让缓冲在 DCQCN 需要的 20 µs 里不溢出；DCQCN 再把源头的发送速率降到出口能承受的水平，暂停才能解除。只靠端到端反馈不丢包的条件是 $(N-2)\times 50\ \text{GB/s}\times 20\ \mu\text{s} \le Q_{\text{free}}$：1 MiB 缓冲只允许 $N \le 3$，4 MiB 也只到 $N \le 6$。把缓冲放大到 4 MiB，允许的反馈时延变为 13.98、5.99、1.35 µs，仍远小于 20 µs。[36] PFC 的代价留到第 7.5.4 节：暂停会沿上游逐跳传播，形成循环依赖时就是死锁。

[35] incast 反馈、4 MiB 缓冲：$N-1$ 个发送方各 50 GB/s 向 50 GB/s 出口发送，流体模型；一跳反馈距离为 30 m 线缆的往返传播加一个 1500 B 报文的串行化，端到端反馈距离取 20 µs 往返。PFC 按流量类别暂停全双工链路的定义取自 IEEE 802.1Qbb 条目；1500 B MTU 与每端口每优先级 22.4 KB 的 headroom 取自 DCQCN 论文§4。

[36] incast 反馈、4 MiB 缓冲：$N-1$ 个发送方各 50 GB/s 向 50 GB/s 出口发送，流体模型；一跳反馈距离为 30 m 线缆的往返传播加一个 1500 B 报文的串行化，端到端反馈距离取 20 µs 往返。PFC 按流量类别暂停全双工链路的定义取自 IEEE 802.1Qbb 条目；1500 B MTU 与每端口每优先级 22.4 KB 的 headroom 取自 DCQCN 论文§4。

**实验 7-11 ・延伸：缓冲大小与降速幅度如何影响积压消退**

设到达速率为 100 GB/s，出口速率为 50 GB/s，初始积压为 256 KiB，求总缓冲为 512 KiB、1 MiB 时可以容忍的反馈时延。假设反馈在缓冲刚好用满时生效，将降速后的到达速率分别设为 50、45、40 GB/s，求从反馈生效到积压清空所需的时间，并解释其中一个条件为何无法排空。

**实验 7-12 ・延伸：incast 的缓冲与反馈距离**

把出口改为 100 GB/s（两张网卡）而每个发送方仍为 50 GB/s，重算 $N = 8$、16、64 的允许反馈时延与 PFC 的 headroom。再把线缆改为 100 m，求一跳反馈距离，以及 $N = 64$ 时 headroom 是否仍在 1 MiB 内。最后求往返时间为 5 µs 时，只靠端到端反馈就不丢包的最大 $N$。

### 7.5.3 多路径与重传

反馈控制通过降低发送速率缓解拥塞；如果其他路径尚有带宽，还可以把流量分过去。Clos 网络提供多条可选路径。把不同连接分到不同路径，能分散热点链路上的流量；把同一次传输的报文分到多条路径，能同时用上多条链路的带宽。后一种做法也把不同路径的时延差带到了接收方。

取四个 token 的 BF16 隐藏向量，共 32 KiB，拆成八个 4 KiB 报文，轮转到两条独立的 50 GB/s 路径。每个报文串行发送约 82 ns，每条路径发送四个报文约需 0.33 µs。两条路径的传播时延都为 1 µs 时，整份数据约在 1.33 µs 交付；同样八个报文只走一条路径则约需 1.66 µs。两条路径各承担了一半发送工作，并行完成传输。[37]

把第二条路径的传播时延改为 9 µs，这条路径上的最后一个报文在约 9.33 µs 才到达。快路径上提前到达的后续报文要等前面缺失的报文补齐，接收方最多要缓存 12 KiB 乱序数据。总字节数相同，多用一条路径却比单路径慢得多，因为节省的串行发送时间只有约 0.33 µs，远小于新增的 8 µs 路径时延。

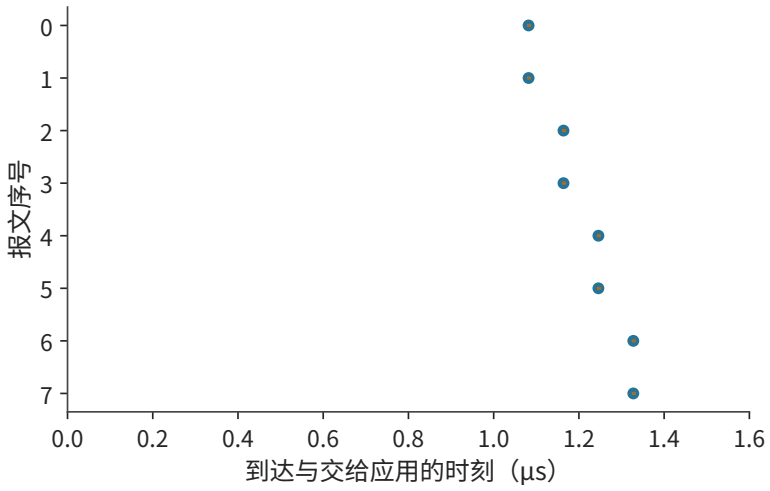



图 7-45：两路径传播时延均为 1 µs，八报文约在 1.33 µs 全部到齐。圆点表示到达；到达后因缺口等待的区间用横线表示。

图 7-45 和图 7-46 中的横线说明，快路径发完并不等于整份数据可以交给应用。两条路径节省了发送时间，却可能增加等待缺失报文的时间。可以直接求出双路径传输更快的条件。单路径需要发送八个报文，双路径每条发送四个。每报文串行发送时间约为 82 ns，双路径节省约 0.33 µs。第二条路径比第一条多出的传播时延若超过这 0.33 µs，平均分配便失去时间优势。在 50 GB/s 的链路上，这一余量只有 30 m 线缆传播时延（约 150 ns）的两倍多，路径之间稍有差别就会抵消它；把一次几十 KiB 的传输均分到时延不同的路径上很少划算，路

[37] 两路径均延迟、路径时延不等、指定丢包恢复。字节数按应用载荷统计。两条路径各 50 GB/s，报文 4 KiB，采用固定载荷与轮转分配，恢复时刻由题设给定；从缺失报文起重发的 32 KiB 数据用于计算额外载荷。

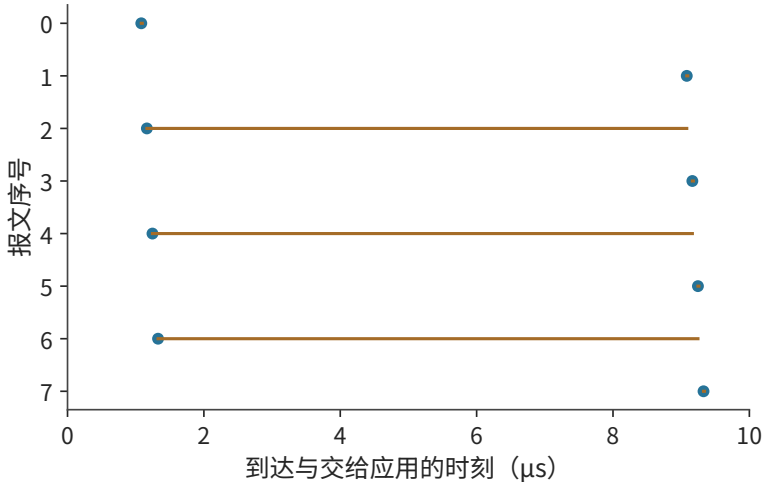



图 7-46：第二条路径传播时延增加到 9 μs。快路径后续报文先到，仍需等待前方缺口；约 9.33 μs 全部可交付。圆点表示报文到达，横线表示到达后等待缺口补齐的时间，纵轴是报文序号。

径选择或不均匀分配应减少慢路径承担的报文，使两条路径的最后一个报文尽量同时到达。只有每条路径分到的串行发送时间远大于路径时延差，多路径才有收益，下文 8 MiB 的逐包喷洒就是这种情形。

传播慢之外，报文丢失是另一种情形，造成的等待也不同。让序号 0 丢失，在它原来发送结束的 20 μs 后重传，整份数据约在 21.2 μs 才能交付：交付要等缺口补齐，不取决于其余报文多早到达。此时其他七个报文都已到达，接收方缓存了 28 KiB 载荷，只需重传缺失的 4 KiB；若从缺失报文开始重发它及其后的所有报文，则要再传输 32 KiB。**选择性重传**用记录接收进度的状态，换来更少的重复传输，图 7-47 画出了这种只补发序号 0 的情形。OpenURMA 的两节点仿真也给出了同样的对照：UB 传输层采用选择性确认，只补发缺失的报文，吞吐随丢包率上升而平缓下降；回退 N 步重传在同样的丢包率下要把整个窗口重发一遍。[38]

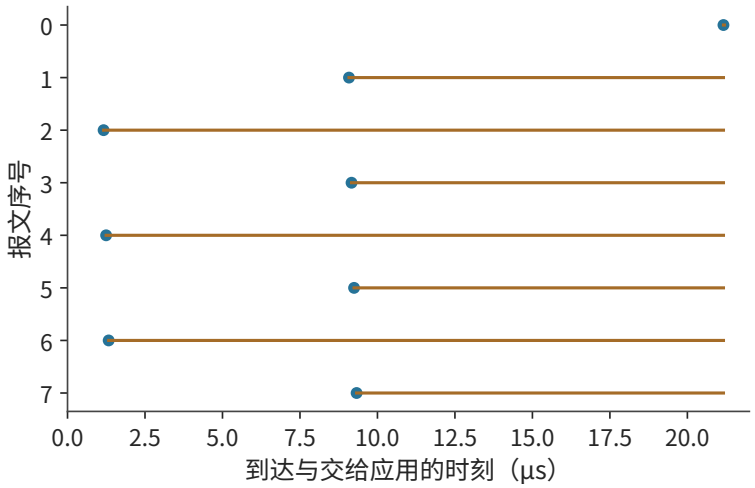



图 7-47：序号 0 丢失后按算例等待时间重传，其他七个报文保留在接收方。只补发缺失报文后，约 21.2 μs 全部可交付。圆点表示报文到达，横线表示到达后等待缺口补齐的时间，纵轴是报文序号。

重传还有一个独立的问题：什么时候认定缺口需要恢复。等太久，真丢包后的停顿就长；慢路径上的报文只是还没到就重发，又会增加重复流量。路径时延的分布决定检测要容忍多大的乱序，接收方记录的状态决定能

[38] 李博杰，〈Unified Bus 背后的思考〉，Jetty、事务序与 Load/Store 各节；OpenURMA 论文，2026-06-02 修订版（arXiv:2605.28717），§3 设计、§7–§9 状态与延迟、§10 顺序、§12 传输、§13 结果汇总。本次整合的来源与适用范围。

缓存多少在缺失报文之后到达的数据。可靠交付与第 7.4 节的应用依赖共同决定哪些已经到达的操作可以继续执行。

**多条路径上的哈希冲突。**前文把同一次传输的报文分到多条路径；本节开头的另一种做法是按连接（流）分配路径，不会乱序，却可能让几条流集中到同一条链路。跨组的流离开叶交换机时，要从该叶的上联中选一条。交换机通常按报文头的地址与端口字段哈希来选择上联，称为**等价多路径**（equal-cost multipath，ECMP）：同一条流的所有报文走同一条路径，不同的流按哈希值分到不同上联。哈希是随机的，两条流落到同一条上联的概率不为零。会彼此冲突的是同一台叶交换机上的跨组流：第 7.2.5 节里错位的配对虽然把八条流送进脊层，但它们从八台不同的叶交换机出发，每台叶上只有一条跨组流，彼此不会在同一台叶的上联上相遇。rail 对齐把每台叶上的跨组流压到最少，正是为了避开这里的冲突。把同一台叶上的 $n$ 条等速流独立、均匀地哈希到该叶的 $m$ 条上联——$k = 64$ 的无阻塞叶有 32 条，3:1 超售叶有 16 条——最忙的一条上联分到的流数记为 $L_{\max}$。没有冲突时，最忙的上联承载 $\lceil n/m \rceil$ 条流；各流公平共享所在链路的带宽、整组要等最慢的流传完时，整组的速度只有无冲突时的 $\lceil n/m \rceil / \mathrm{E}[L_{\max}]$。精确计算最大负载的分布，表中 p99 是 $L_{\max}$ 的第 99 百分位数，即 99% 的哈希结果中最忙上联的流数不超过该值：[39]

| 流数 $n$ | 上联数 $m$ | 无冲突时最忙上联的流数 | $\mathrm{E}[L_{\max}]$ | p99 | 无冲突概率 | 相对无冲突的速度 |
|---|---|---|---|---|---|---|
| 8 | 32 | 1 | 1.66 | 3 | 38.6% | 60.1% |
| 8 | 16 | 1 | 2.06 | 4 | 12.1% | 48.5% |
| 32 | 32 | 1 | 3.53 | 6 | $1.8 \times 10^{-13}$ | 28.3% |
| 32 | 16 | 2 | 4.83 | 8 | 0 | 41.4% |
| 128 | 16 | 8 | 13.36 | 18 | 0 | 59.9% |

一台叶上只有八条跨组流时，无阻塞叶有 38.6% 的概率完全不冲突；一旦冲突，两条流集中在一条上联上，各得一半带宽，最忙的一条上联平均分到 1.66 条流，整组平均只剩无冲突时 60.1% 的速度。超售叶的上联减半，无冲突概率降到 12.1%，速度降到 48.5%。整组 32 台服务器各出一条跨组流、无阻塞叶恰好 32 条上联时最差：流数与上联数相等，几乎必然有若干条上联空着、另一些挤了三四条，整组只剩 28.3%。流数远多于上联数时，随机分配的波动相对变小：32 条流分到 16 条上联为 41.4%，128 条流分到 16 条上联为 59.9%。图 7-48 画出这几组配置的最忙链路负载。**翻转条件**：流数与上联数接近时冲突代价最大，流数远少于或远多于上联数时代价较小；要避开这一代价，一是用 rail 对齐减少同一台叶上的跨组流，二是不再按流分配，改用下文的逐包喷洒。

**逐包喷洒。**把同一条流的报文分到多条路径，就消除了流级哈希的冲突，代价是本节开头算过的乱序。把 8 MiB 的 prefill 激活（1024 个 token 的 BF16 隐藏向量）拆成 2048 个 4 KiB 报文，轮转到八条 50 GB/s 的路径，八条路径的传播时延为 1 到 8 µs。每条路径承担 1 MiB，串行发送 20.97 µs，最慢路径再加 8 µs，全部到达 28.97 µs，约 29.0 µs；同样 8 MiB 走一条路径要 167.8 µs 加 1 µs 传播，共 168.8 µs。喷洒节省的是串行发送时间，多出的是路径时延差；按本节开头的条件，只要最慢路径的时延小于 $168.8 - 21.0 = 147.8$ µs，喷洒就更快，这里的最慢路径只有 8 µs。乱序的代价是接收方要缓存先到的报文：快路径的报文要等慢路径上序号更小的报文，接收方最多同时保留 339 个报文、1.39 MB。[40]

超以太网（Ultra Ethernet，面向 AI 与高性能计算的以太网规范）把这两件事写进了传输层：报文按熵值

[39] 8 流 32 上联、8 流 16 上联、32 流 32 上联、32 流 16 上联、128 流 16 上联：同一台叶交换机上的每条流独立均匀地哈希到该叶的上联，最大负载的分布按截断指数多项式精确计算，相对无冲突的速度为 $\lceil n/m \rceil$ 除以最大负载的期望（$n \geq m$ 时即结果文件中的有效割集）。逐包喷洒沿用本节的报文乱序模型：8 MiB 拆成 4 KiB 报文轮转到八条 50 GB/s 路径，路径时延 1 到 8 µs 为给定输入。喷洒与熵值取自 Ultra Ethernet 规范 v1.0.1§3.6.5.2，回退 N 步的代价取自 UEC 概述第 5 页。

[40] 8 流 32 上联、8 流 16 上联、32 流 32 上联、32 流 16 上联、128 流 16 上联：同一台叶交换机上的每条流独立均匀地哈希到该叶的上联，最大负载的分布按截断指数多项式精确计算，相对无冲突的速度为 $\lceil n/m \rceil$ 除以最大负载的期望（$n \geq m$ 时即结果文件中的有效割集）。逐包喷洒沿用本节的报文乱序模型：8 MiB 拆成 4 KiB 报文轮转到八条 50 GB/s 路径，路径时延 1 到 8 µs 为给定输入。喷洒与熵值取自 Ultra Ethernet 规范 v1.0.1§3.6.5.2，回退 N 步的代价取自 UEC 概述第 5 页。

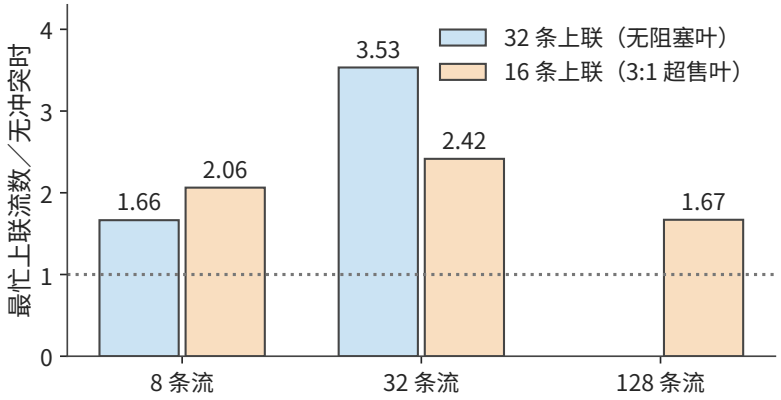



图 7-48：同一台叶交换机上的 $n$ 条跨组流哈希到该叶的 $m$ 条上联时，最忙上联的期望流数与无冲突时最忙上联流数之比，比值的倒数就是整组相对无冲突的速度。无阻塞叶有 32 条上联，3:1 超售叶有 16 条；32 条流分到 32 条上联时比值最大，为 3.53；128 条流分到 16 条上联时降到 1.67。

（entropy，报文头中供交换机哈希选路的字段）喷洒到多条路径，典型配置有 64 到 256 个熵值，反馈显示某条路径拥塞时就减少放到它上面的报文；丢失的报文按选择性确认只补发缺失的那些，而不是回退 N 步重发整个窗口。[41] 这正是图 7-47 的模型：喷洒换来所有链路的带宽，选择性重传把丢包的代价限制在缺失的报文上，接收方为此保存乱序状态。

**实验 7-13 ・延伸：路径时延差与重传如何推迟按序交付**

让第二条路径的传播时延在 1 µs 至 9 µs 之间变化，求均分到两条路径与仅使用第一条路径的完成时间相等时，第二条路径的传播时延。再令两条路径分别承担五个和三个报文，求第一条路径传播时延为 1 µs、第二条为 9 µs 时的完成时间。保持首包丢失，把恢复等待从 20 µs 增为 40 µs，计算首次能够按序交付数据的时刻，以及此前需要缓存的数据量。

**实验 7-14 ・延伸：哈希冲突与喷洒的边界**

求 16 条流分到 32 条上联时最忙链路的期望负载与相对无冲突的速度，与表中同样 $n/m = 1/2$ 的 8 条流分到 16 条上联比较。把喷洒的路径时延改为 1 到 30 µs 均匀间隔，求完成时刻与峰值乱序缓存；再求使喷洒不再快于单路径的最慢路径时延。最后把报文改为 1 KiB，说明峰值乱序缓存的报文数和字节数怎样变化。

### 7.5.4 死锁

第 7.5.3 节中，已经到达的报文要保留到缺口补齐后才能释放空间。端点背压和链路流控还会让这种等待向上游传播：接收方没有空位，上游便不能继续发送。若释放某项资源的动作也需要这项资源，就会形成循环。考虑两项请求分别占用资源 A 和 B；第一项还需要获得 B 才能完成，第二项还需要获得 A 才能完成。二者都在等待对方先释放。

在归约系统里，这种循环可以跨越多个层次：请求占满接收缓冲，完成响应等待发送队列，发送队列又等待远端释放请求所占的空间。把“持有前一项资源、申请后一项资源”画成边，就得到资源依赖图。环上的每项资源都被持有时，所有释放条件就同时停滞。图 7-49 画出这种循环，图 7-50 画出下文为响应预留通路的解法。

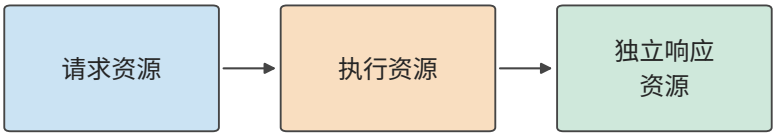
一种解决方法是给资源规定严格申请次序，例如所有工作先申请 A 再申请 B。这样，持有 B 的工作不再回

[41] 8 流 32 上联、8 流 16 上联、32 流 32 上联、32 流 16 上联、128 流 16 上联：同一台叶交换机上的每条流独立均匀地哈希到该叶的上联，最大负载的分布按截断指数多项式精确计算，相对无冲突的速度为 $\lceil n/m \rceil$ 除以最大负载的期望（$n \geq m$ 时即结果文件中的有效割集）。逐包喷洒沿用本节的报文乱序模型：8 MiB 拆成 4 KiB 报文轮转到八条 50 GB/s 路径，路径时延 1 到 8 µs 为给定输入。喷洒与熵值取自 Ultra Ethernet 规范 v1.0.1§3.6.5.2，回退 N 步的代价取自 UEC 概述第 5 页。




双方都需要对方先释放

图 7-49：两项请求分别持有 A、B，各自等待对方持有的资源。箭头表示等待关系，两个请求都无法完成并释放资源。




响应有预留通路，可返回并释放原请求

图 7-50：请求、执行、响应使用独立资源并按顺序申请。响应有预留缓冲和发送机会，能够返回并释放原请求。

头等待 A，环便被打断。另一种方法是给完成响应预留独立缓冲区和发送机会。设接收方八个数据槽已满，发送方等待释放；若响应也要占用这些数据槽，就一起停住。增加独立响应槽，使处理数据后的释放消息可以返回，至少一项请求所占的空间得以释放，数据队列便能继续周转。

虚拟通道用逻辑上分开的队列来划分资源。把请求与响应分入不同通道，再规定资源申请方向，可以建立无环的依赖结构。这里真正发挥作用的是独立容量和使用次序。

内存访问还可能要做地址转换、读页表。原请求等待地址转换，地址转换又要发起一次远程访问，二者若争用已耗尽的同一组槽位，也会形成相同的环。笔者参与 UB 设计时，需要一并分析事务处理、内存访问和链路传输之间的依赖。为完成通知、地址转换和故障恢复预留资源，才能避免系统在拥塞或故障时陷入停滞。

## 7.6 从通信改进到任务完成

### 7.6.1 关键路径从数据就绪开始

第 7.2 至 7.5 节沿一份数据追踪了传输、等待、完成通知和空间释放。要判断这些改进能节省多少训练时间，还要把产生和使用这份数据的计算过程一并计入，并确定通信何时开始。一张卡很早就进入集合通信调用，另一张卡的梯度还没算出来，前者就在调用里等后者。这段等待记在通信调用的耗时里，根源却在前面的计算或数据准备。

考虑四个要做本地归约的参与者，分别在 0、0、0、2 ms 就绪，全部到齐后执行 0.4 ms 的交换。按第 7.4 节的依赖图，交换的开始时刻等于四个就绪时刻中的最大值，全组在 2.4 ms 完成。将交换耗时减半，完成时间变为 2.2 ms；如果四个参与者都在起点准备好，完成时间就变为 0.4 ms。两个优化分别作用于依赖图上的不同节点，图 7-51 至图 7-53 依次画出这三种情形。[42]

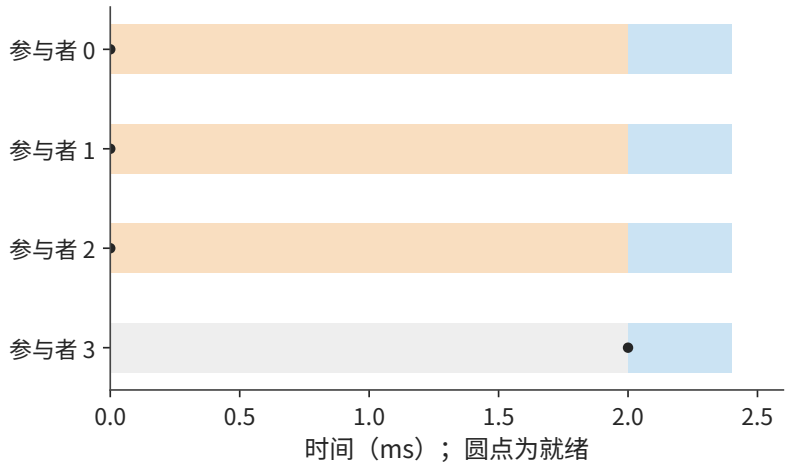



图 7-51：原安排中，前三个参与者等第四个在 2 ms 就绪，再交换 0.4 ms。灰色为尚未就绪，橙色为等待其他参与者，蓝色为交换。

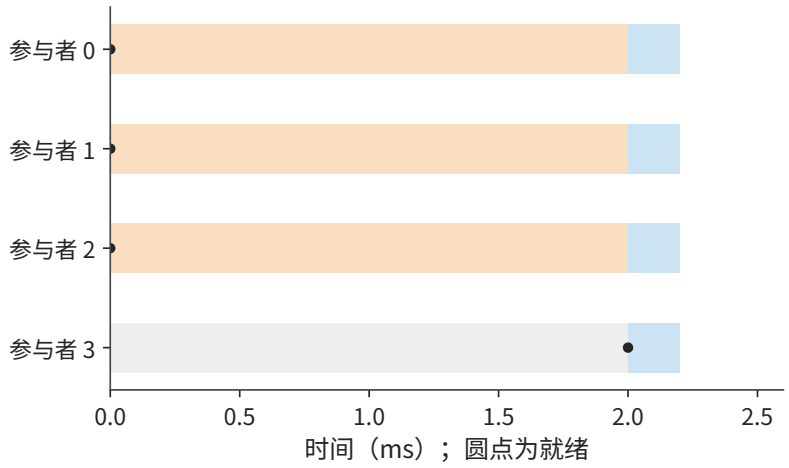



图 7-52：就绪时刻相同，交换从 0.4 ms 减到 0.2 ms，全组从 2.4 ms 提前到 2.2 ms 完成。灰色表示尚未就绪，橙色表示等待其他参与者，蓝色表示交换。

这种就绪偏差在大规模训练系统中同样存在。大规模分布式训练系统 MegaScale 在诊断训练性能时发现，

[42] 就绪偏差教学记录、100 条混合记录、两条恢复记录。p99 取排序后第 $\lceil 0.99N \rceil$ 项。

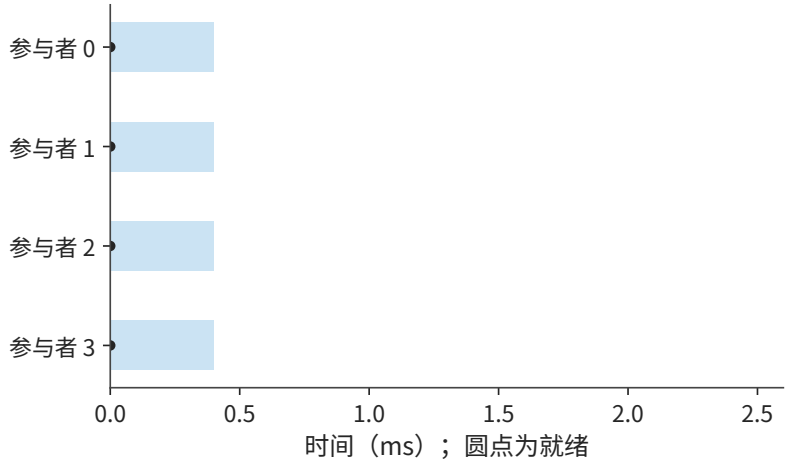



图 7-53：四个参与者在起点同时就绪，交换仍需 0.4 ms，全组在 0.4 ms 完成。三图时间轴相同。蓝色表示交换；各行分别对应一个参与者。

网络带宽保持稳定，但各参与者开始通信的时间差越来越大，ReduceScatter 中的等待也随之增长。进一步分析发现，这一差异与前向计算期间的主机操作有关。[43] 本书配套实验用集合通信库 Gloo 在 CPU 上做的四进程测量也显示同样的关系：4 KiB 输入下，当一个参与者晚到约 25.1 ms 时，全组完成时间的中位数从约 1.6 ms 增至 26.6 ms；最后一个参与者到达之后的尾段仍约为 1.6 ms。[44]

因此，任务时间线应从数据生成时刻画起。把梯度分成多个桶，每个桶的梯度生成后即可开始归约；下一桶仍在计算时，上一桶可以传输。不同桶的通信会竞争网卡和链路，参数更新要等最后一个桶归约完成。分桶让通信提早开始，与后续梯度计算同时进行。

### 7.6.2 分层网络的通信耗时计算

**例题 7.6：归约算法、执行顺序与链路带宽应先优化哪一项？** 沿用前述算例中的 192 MiB 梯度。梯度在计算开始后 20 ms 就绪，归约后执行 2 ms 更新。比较连续环与分层归约，再考察并发不足、提前通信和链路退化。

**解答：先计算归约与计算串行时的训练步耗时。** 计算 → 归约 → 更新形成一条串行链。使用第 7.2 节的传输模型，连续环总时间约为 $20 + 7.6 + 2 = 29.6$ ms，分层归约约为 $20 + 1.3 + 2 = 23.3$ ms。通信时间减少约 83%，整步只减少约 21%。20 ms 计算和 2 ms 更新占据了其余时间。

一般地，不受优化影响的时间为 $T_c$，通信为 $T_n$，通信加速 $s$ 倍，则总加速比为

$$S = \frac{T_c + T_n}{T_c + T_n/s}.$$

接下来考虑第 7.3 节的并发请求数限制。若每张网卡仅有 128 个 256 B 事务在途、槽位周转时间为 2 µs，先不计发起速率，每张网卡的远端吞吐受限于 16.4 GB/s。分层归约跨服务器每张网卡共发送 24 MiB，远端传输便需约 1.5 ms；加上本地通信和启动，总通信约为 2.3 ms，整步约为 24.3 ms。增加到 391 项活跃事务并使发起速率足够，才恢复到约 23.3 ms。换了归约算法之后，还要让请求提交和处理速度足以发挥算法的优势。

再考虑重叠。假设计算开始后 17 ms，需要通信的数据已经准备好，而其他计算还需 3 ms。这 3 ms 的计

[43] Jiang 等，*MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs*，NSDI 2024；正式论文、集合通信诊断。

[44] experiments/ch07/07-10/rank-readiness：四进程 Gloo 就绪偏差。Apple M2 Max、本机 CPU，180 组正式记录；全组完成从最早屏障返回到最晚调用返回，最后到达后尾段仍包含归约、调度与唤醒。正文采用不同样本统计的中位数，逐样本时间线见原记录。

算可以与通信同时执行。两者使用互不争用的硬件资源，更新要等两者都完成。此时

$$T_{\text{step}} = \max(20,\ 17 + T_n) + 2 \text{ ms}.$$

图 7-54 至图 7-59 依次画出六种安排，每张图使用相同时间尺度。灰色计算区间保持不变，蓝色通信区间的长度由传输量和吞吐决定，起点则由数据何时就绪决定。

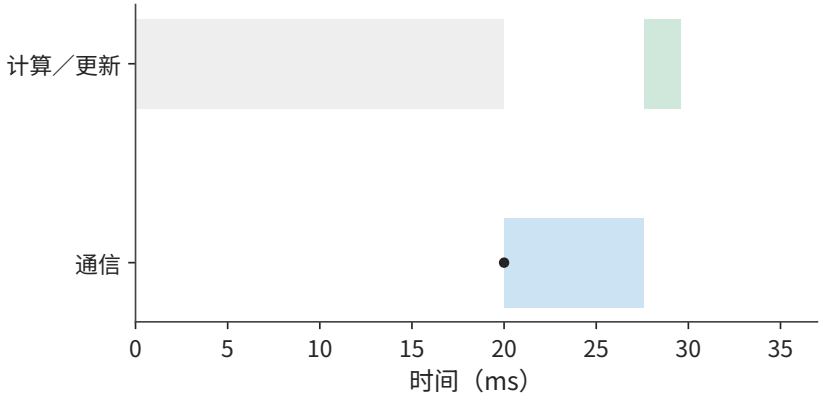



图 7-54：连续环串行安排：计算 20 ms，通信约 7.6 ms，更新 2 ms。灰为计算，蓝为通信，绿为更新；圆点为通信数据就绪。

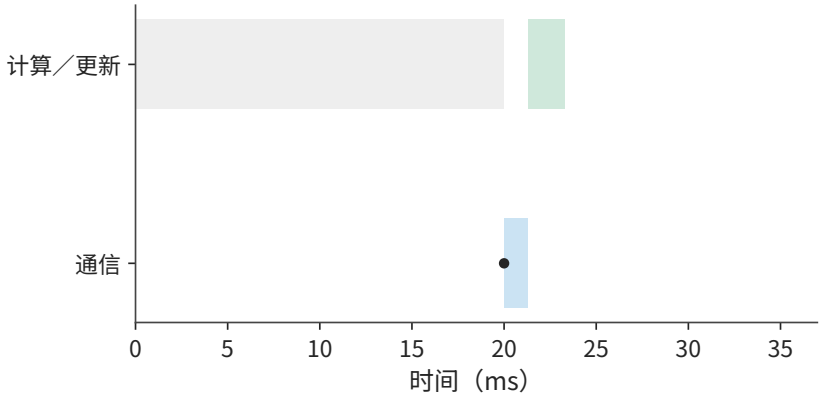



图 7-55：本图改用分层归约，仍按计算、通信、更新串行执行。通信缩短到约 1.3 ms，整步约 23.3 ms。灰色为计算，蓝色为通信，绿色为更新；圆点为通信数据就绪。

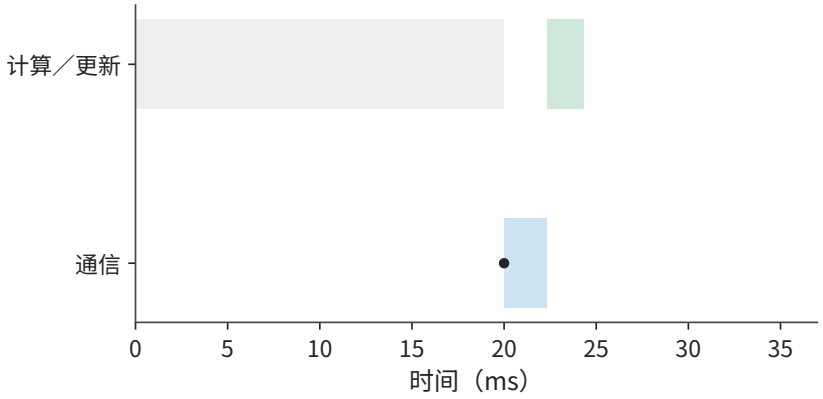



图 7-56：本图采用分层归约，其他计算与更新时间不变。每张网卡 128 个在途事务限制远端吞吐，通信拉长到约 2.3 ms，整步约 24.3 ms。灰色为计算，蓝色为通信，绿色为更新；圆点为通信数据就绪。

数据在 17 ms 就绪并与计算重叠时，连续环整步约为 26.6 ms，分层归约约为 22.0 ms：分层通信约在 18.3

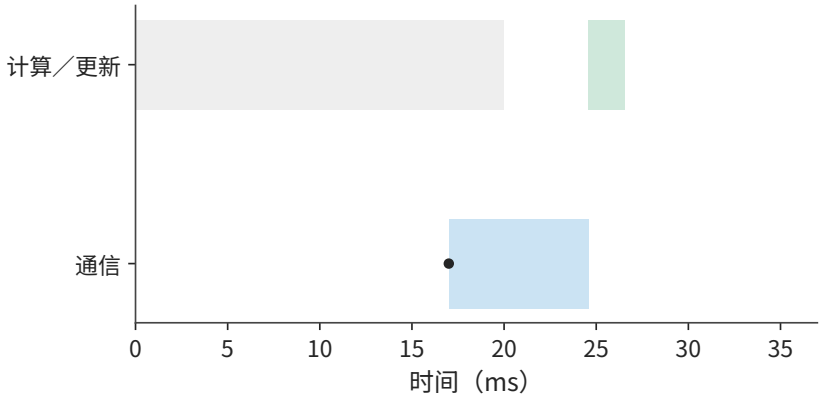



图 7-57：本图把连续环的通信提前到 17 ms 开始，与尚未完成的计算重叠。通信 7.6 ms 超出计算区间，更新等通信结束；整步约 26.6 ms。灰色为计算，蓝色为通信，绿色为更新；圆点为通信数据就绪。

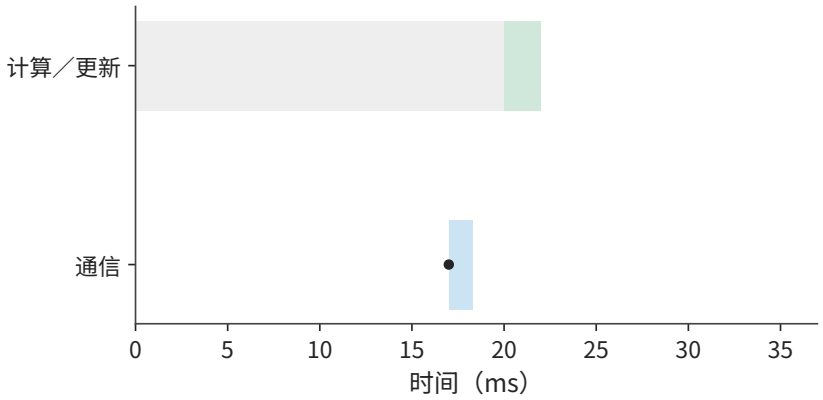



图 7-58：本图把分层归约的通信提前到 17 ms 开始。通信在 18.3 ms 结束，早于计算，更新从 20 ms 开始，整步 22 ms。灰色为计算，蓝色为通信，绿色为更新；圆点为通信数据就绪。

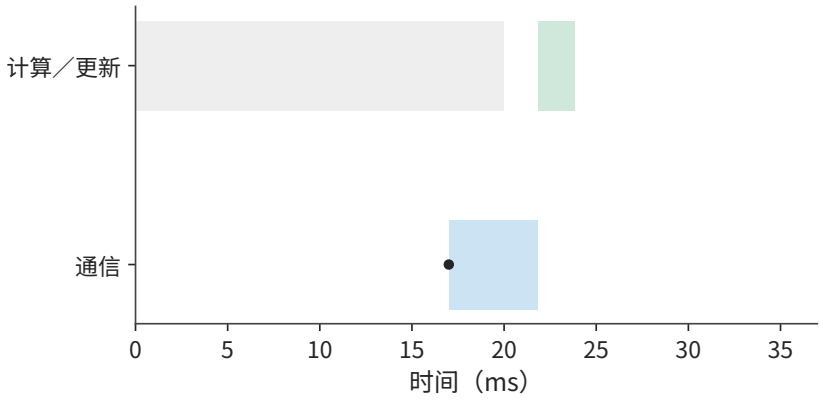



图 7-59：本图从提前通信的分层配置出发，只把可用网卡减为一张。跨服务器两轮各要挤过 96 MiB，通信约 4.8 ms，超出计算区间，整步约 23.8 ms。上述时间线采用同一横轴范围。灰色为计算，蓝色为通信，绿色为更新；圆点为通信数据就绪。

ms 结束，早于 20 ms 的计算结束时刻，更新紧接在计算之后开始。数据最晚在 18.7 ms 就绪，分层通信仍能藏在计算后面。继续缩短通信不再改变这条关键路径，此时需要缩短计算或更新，才能进一步减少总时间。

**讨论：网卡减少后，归约是否重新成为训练步的瓶颈？**若八条 rail 只剩一条可用，每台服务器的八份分片都要经过剩下的这一张网卡，每轮每个方向 96 MiB，两轮共 192 MiB，远端传输从约 0.5 ms 增加到约 4.0 ms，通信约为 4.8 ms。数据在 17 ms 就绪时，通信到 21.8 ms 才结束，整步约为 23.8 ms。通信耗时增加后，其中更多部分要到计算结束后才能完成。

再按顺序比较这些时间线：减少通信量会缩短蓝色条带，并发不足会将其延长，数据提早就绪则让它向左移动。绿色更新要等计算和通信都结束后才能开始。流量、并发与依赖这三个模型也给出了清楚的改进顺序。先用分层归约减少跨服务器传输并让每张网卡都有分片可传，再靠并发提交和处理请求用满链路带宽；到了带宽上限，就找可以提早开始的通信；通信时间被计算完全盖住之后，再优化此时决定完成时间的计算或更新。

### 7.6.3 训练与推理的通信瓶颈

图 7-54 中蓝色条带能够显著缩短，是因为大梯度的数据传输占了大部分通信时间。低并发 decode 中，一个 8 KiB 隐藏向量要在层间反复交接，实际发送数据只需极短时间，每次启动反而占了大部分耗时。取一个由八个参与者组成的环，每轮启动 5 µs，每个参与者有效带宽为 $B$，输入为 $M$ 时一次归约为

$$T_{\mathrm{AR}} = 14 \times 5\ \mu\mathrm{s} + \frac{1.75M}{B}.$$

十四轮启动共 70 µs。链路取本章网卡的 50 GB/s，令载荷项等于启动项，得到 $M = 2$ MB，约为 1.91 MiB。输入大于这一数量级时，数据传输耗时所占的比例增大；8 KiB 远小于它，主要耗时来自启动。

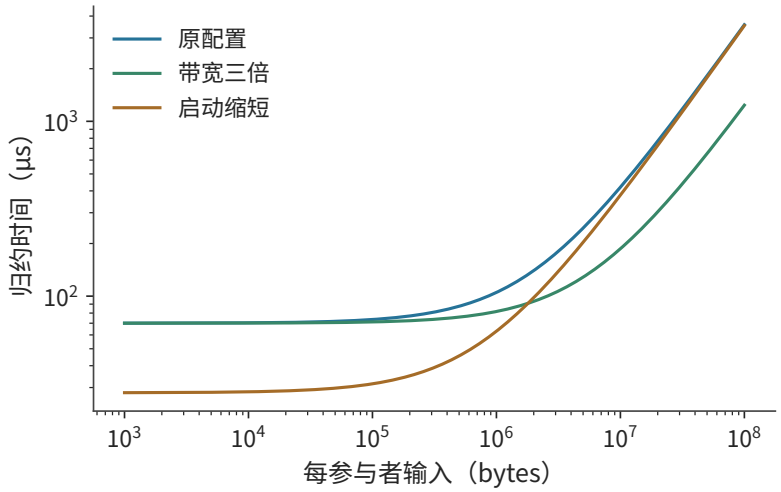



图 7-60：原配置每轮启动 5 µs、带宽 50 GB/s。绿色将带宽增至三倍的 150 GB/s，橙色将每轮启动减至 2 µs；数据量小时主要受启动影响，数据量大时主要受传输影响。

沿图 7-60 的横轴从左向右看，数据量小的一段总时间接近水平，数据量大的一段随载荷增大而上升。提高带宽主要放缓右侧的斜坡，缩短启动时间主要使左侧平坦区的曲线下移。

对 8 MiB 输入，带宽由 50 增至 150 GB/s，一次归约从约 364 µs 降至 168 µs；对 8 KiB 输入，则只从约 70.3 µs 降至 70.1 µs。假设执行一个 36 层模型，每层注意力与 FFN 各有一次这样的归约，72 次串行归约在两种带宽下均约为 5.1 ms，差值约 14 µs。把每轮启动从 5 µs 降到 2 µs，72 次归约却能减少约 3.0 ms。[45]

因此，训练和大 batch prefill 更容易从减少字节、增加带宽和重叠中获益；低并发 decode 更需要减少层内跨服务器同步、缩短发起与完成路径。这也解释了第 7.2 节讨论的流水线并行方案：把一个阶段放在本地，可以减少阶段内部的远程通信；同时处理多少个独立请求，则决定流水线利用率。

平均耗时之外，偶发的长时间等待也会推迟任务完成。取 100 次通信，其中 98 次为 0.4 ms，一次因晚就绪而多等 2 ms，一次因恢复而多等 10 ms。平均约为 0.52 ms；p99 取由小到大排序的第 99 项，得到 2.4 ms。

[45] 网络规划与集合通信案例。小消息算例设每轮启动 5 µs、每个参与者有效带宽为本章网卡的 50 GB/s 及其三倍 150 GB/s，环形模型计载荷传输与启动，数值见本章教学算例的 `small_messages` 项；前述大块归约用由 nccl-tests 记录推得的每轮 0.833 µs。

将正常传输时间减半，p99 降至 2.2 ms；若恢复事件增加为两次，p99 就由包含故障恢复的那次通信决定，变为 10.4 ms。[46]

正常传输、数据准备延迟和故障恢复，都可能成为决定任务完成时间的主要因素。频繁的小数据量传输需要降低每次的启动开销；耗时较长的故障恢复则需要缩小故障影响范围。训练中发生通信故障后，还需要重建通信组并恢复训练状态；第 10 章将进一步说明这一过程。

带宽与启动开销的比较都保持通信模式不变。通信模式本身也可以改变：调整并行方案与专家分布，可以减少交接次数；由加速器直接发起通信，或把发起与完成处理交给卸载单元，可以缩短每次交接。MoE 的路由、dispatch 与 combine 对网络操作提出要求；网络能否高效完成这些操作，又反过来影响专家的粒度和跨节点部署。若模型计算因专用硬件变快，通信时间本身没变，它在关键路径上的占比却会上升。

同样的流量分析也适用于推理中的状态迁移。继续分析贯穿本书的同一个 DeepSeek V4.1 会话，设这条请求要迁移到一个不持有其全局 KV 的实例上。先计算全局历史这一段载荷的传输时间：上下文为 128K，传输走一张以 200 Gbit/s 运行的 ConnectX-7，有效带宽 25 GB/s。V4-Flash 的 439.281 MiB 需要约 18.425 ms，V4.1 的 111.250 MiB 需要约 4.666 ms。状态压缩使这条路径的传输时间减少了约 13.759 ms。[47]

这 13.759 ms 是迁移方案在传输环节可以节省的时间。若请求依次经历排队、缓存查询、传输和局部 SWA 状态的重建，恢复时间就是四段之和；传输缩短多少，总时间就缩短多少。第 9 章会把这条恢复路径与留在原实例等待、从输入重新计算放在一起比较，决定请求该去哪里执行。

### 7.6.4 固定 1024 卡，超节点变大后如何重新选择

前几节用十六个参与者分析了路径，本节把规模扩展到 1024 张卡：保持总卡数不变，超节点大小从 8 改成 64、128、256 卡，考察哪些时间会改变。先固定计算工作量与并行方案，再分别改变网络条件，最后重新比较并行度。[48]

**固定模型与训练工作。**采用 Qwen3-32B 的形状：64 层、隐藏维 5120，参数量取 $P = 32 \times 10^9$。1024 张 H100 SXM 80 GB 按张量并行 8、数据并行 128 组织，记作 TP8×DP128，每个八卡张量并行组都在一个超节点内。每次更新固定处理 $2^{20}$ 个 token，每个数据并行副本处理一个含 8192 个 token 的 micro-batch；全局 batch size、精度和优化器更新的含义都不变。训练状态按每参数 16 bytes 计，每卡 64 GB，另假设 8 GB 的激活与工作区足够，暂不用 ZeRO（Zero Redundancy Optimizer，零冗余优化器，把优化器状态、梯度和参数分片存放到各数据并行副本上）。

梯度按 BF16 传输，每张卡持有的分片为 $G = 2P/8 = 8$ GB。设超节点有 $S$ 张卡，则超节点数 $H = 1024/S$，每个超节点内同一张量并行坐标上有 $q = S/8$ 个数据并行成员。图 7-61 画出 64 卡超节点内的分组：同一行的八张卡是一个模型副本，同一列的卡属于同一个梯度同步组。

**跨超节点的传输量。**节点内的 $q$ 个数据并行成员先做 ReduceScatter，每张卡留下 $G/q$ 的梯度分片；持有同一分片的 $H$ 个超节点之间再做 AllReduce；最后节点内做 AllGather。按环形算法，每张卡跨节点发送 $2(H-1)G/(Hq)$，乘以节点内的 $8q$ 张卡，得到每个超节点每个方向的发送量

$$V_{\text{out}} = 2\frac{H-1}{H}\,8G.$$

[46]就绪偏差教学记录、100 条混合记录、两条恢复记录。p99 取排序后第 $\lceil 0.99N \rceil$ 项。

[47]DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

[48]1024 卡固定场景、完整计算与候选比较、脚本。形状来自 Qwen3-32B 配置，参数总量用 32B 近似。H100 SXM 的 BF16 稠密峰值见硬件表；41% MFU 取自 Llama 3 论文表 4；NVLink Switch System 最多连接 256 张 GPU、全交换带宽 115.2 TB/s 见 Grace Hopper 架构文章快照；第五代 NVLink 每 GPU 1800 GB/s（双向合计）见 NVLink 规格页；1024 卡作业平均 7.9 小时中断一次、中断率与卡数成正比见 Meta 集群可靠性论文图 7。checkpoint 间隔与暂停、固定恢复时间为给定输入。

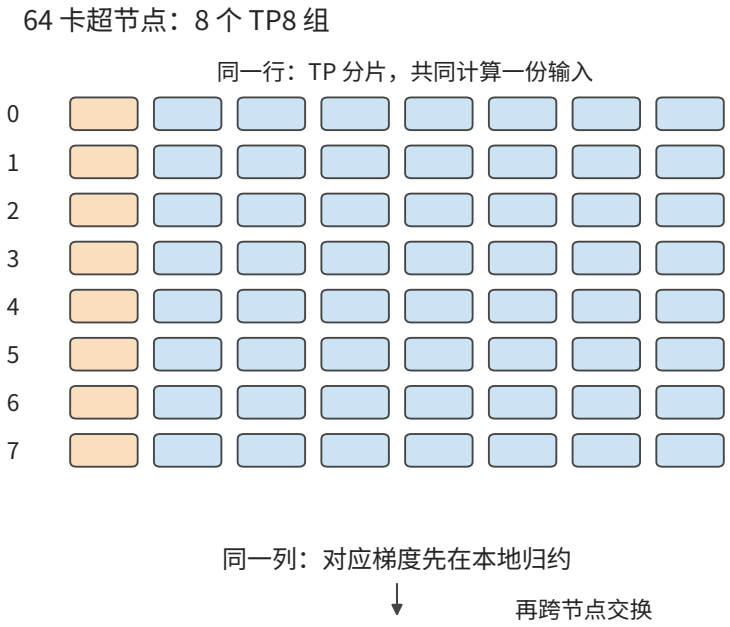



图 7-61：64 卡超节点由八个张量并行组组成。同一行的八张卡处理同一 micro-batch 的不同模型分片；同一列的八张卡先在节点内汇合对应的梯度，再与其他超节点交换同一分片。一个训练作业用上全部的卡，不等于全部的卡组成一个张量并行组。

节点从 8 卡扩大到 128 卡时，$q$ 从 1 增至 16，每卡的跨节点字节减少，但每超节点合计发送量只从 127 GB 降到 112 GB。因此，超节点扩大 16 倍，并不意味着经过出口的字节也减少 16 倍。

节点内按 NVLink 每卡每方向 450 GB/s 计：8 卡就是一台 HGX H100；64 到 256 卡对应 NVLink Switch System，它最多把 256 张 Hopper GPU 连成一个 NVLink 域，全交换带宽 115.2 TB/s，正是 256 张卡各 450 GB/s。跨节点每卡一张 ConnectX-7，每方向 50 GB/s。节点内外每轮启动 $\alpha_L$、$\alpha_R$ 都取第 7.1.1 节的 0.83 µs。记节点内每卡每方向带宽为 $B_L$、每张网卡每方向带宽为 $B_{\text{NIC}}$、整个超节点的可用单向出口为 $B_{\text{out}}$，分层梯度时间为

$$T_{\text{grad}} = 2(q-1)\alpha_L + \frac{2(q-1)G}{qB_L} + 2(H-1)\alpha_R + \max\left(\frac{2(H-1)G}{HqB_{\text{NIC}}}, \frac{V_{\text{out}}}{B_{\text{out}}}\right).$$

式中依次是本地归约与收集、跨节点的启动，以及网卡与共享出口两者中较慢的一项。真实系统还可能受交换网络割集和本地并发通信的影响；本例假设网络能提供所列带宽，各阶段不重叠。

**训练步的总时间。**每卡的有效算力取 H100 SXM 的 BF16 稠密峰值 989.4 TFLOP/s 乘 41%，约 405.7 TFLOP/s；41% 是 Llama 3 405B 在 16384 张 H100 上以 TP8、PP16（流水线并行 16 段）、DP128 训练时报告的 BF16 MFU。按第 1.2.2 节的定义，41% 意味着有 59% 的峰值算力没有转化为模型计算。其中一部分是这类一阶估算没有计入的工作，例如注意力的二次项，以及训练中为节省显存而重复的计算；另一部分是可以减少的等待，例如第 7.6.1 节的就绪偏差和第 7.3.4 节的在途请求不足。计算用 $6P \times 2^{20}/(1024 \times 405.7\ \text{TFLOP/s}) \approx 0.485$ s 估计，忽略注意力的二次项。每层前向与反向合计按四次张量并行 AllReduce 计，8192 个 token 的 BF16 隐藏张量为 80 MiB，64 层合计约 0.086 s；优化器更新另计 0.050 s。于是固定调度下的步时间与吞吐 $\Theta$（每秒处理的 token 数）为

$$T_{\text{step}} = 0.485 + 0.086 + T_{\text{grad}} + 0.050, \qquad \Theta = 2^{20}/T_{\text{step}}.$$

这是把计算、张量并行通信、梯度同步和更新串起来的固定调度估算；真实系统沿梯度就绪的时间线扣掉已经重叠的部分。作为对照，再算一个不分层的连续数据并行环：每个张量并行坐标只有一条跨节点的环边，不把这条边分摊到其他网卡。下表在两种算法中取较快者。

| 每超节点卡数 | 超节点数 | 本地 DP 成员 $q$ | 每节点跨域发送 | 出口随卡数增长：步时间 / 吞吐 | 出口固定 400 GB/s：步时间 / 吞吐 |
|---|---|---|---|---|---|
| 8 | 128 | 1 | 127 GB | 0.939 s / 111.7 万 token/s | 0.939 s / 111.7 万 token/s |
| 64 | 16 | 8 | 120 GB | 0.690 s / 152.0 万 token/s | 0.939 s / 111.7 万 token/s |
| 128 | 8 | 16 | 112 GB | 0.672 s / 156.0 万 token/s | 0.935 s / 112.2 万 token/s |
| 256 | 4 | 32 | 96 GB | 0.663 s / 158.1 万 token/s | 0.896 s / 117.1 万 token/s |

“出口随卡数增长”取 $B_{\text{out}} = S \times 50$ GB/s，即每张卡的 400 Gbit/s 网卡都接入交换网络，意味着为更大的节点配置更多可同时工作的外部端口和足够的交换网络；出口不会随节点变大而自动变宽。“出口固定”则让每个超节点只有 8 条 400 Gbit/s 链路接入 QM9700 交换网络，共 400 GB/s，与一台 HGX 服务器相同。后者的 64 卡方案中，分层归约需约 $31.1 + 300.0 = 331.1$ ms，反而比连续环约 317.7 ms 慢，因而表中选择连续环。这和第 7.2 节关于本地归约是否合算的判断相同。“出口随卡数增长”一列还隐含交换网络无阻塞：第 7.1.2 节算过，3:1 超售把每超节点出口降到三分之一，128 卡节点的跨域项从约 17.5 ms 增至约 52.5 ms，步时间增加约 35 ms；第 7.2.5 节说明了这些跨域字节在多轨拓扑上均匀落在八条 rail 上。

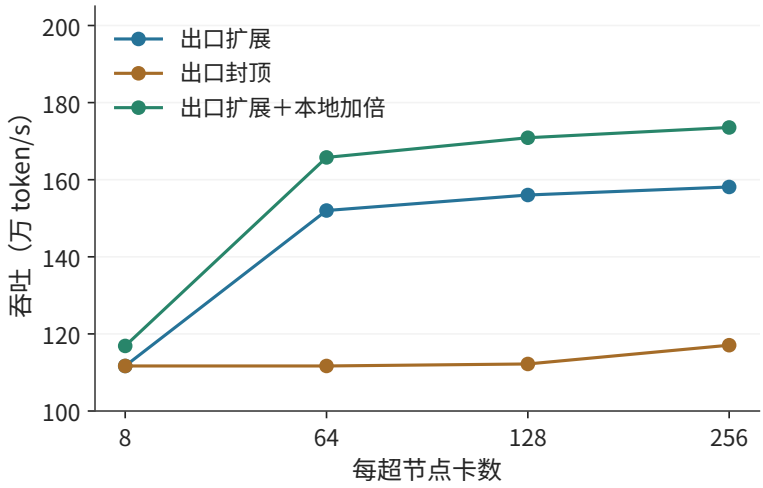



图 7-62：全局 token 数与总卡数固定。蓝线增加每节点外部出口，橙线将出口封顶在 400 GB/s，绿线在出口扩展的基础上将本地带宽从 450 翻倍到 900 GB/s。每点在声明的连续环与分层归约之间选择较快者；纵轴为固定调度估算的正常运行吞吐。

图 7-62 画出三种网络条件下吞吐随超节点大小的变化。从 8 卡增至 64 卡，在出口随卡数增长时吞吐提高约 36%。8 卡超节点只有一台服务器，每张卡要把自己的整份 8 GB 梯度分片经网卡同步，要 0.318 s，占步时间的三分之一；64 卡时先在 NVLink 上把它归约成八分之一，每张卡跨节点只交换这一小片，梯度同步降到约 69 ms。从 128 卡增至 256 卡只提高约 1.3%，因为此时计算与张量并行通信已占步时间的 86%。若内部带宽另从 450 增到 900 GB/s，即第五代 NVLink 的每方向带宽，128 卡节点约为 0.614 s、170.9 万 token/s；这部分收益来自内部带宽的提高，不能归功于节点大小本身。

**并行度的重新比较。**更大的高速域可以容纳更大的张量并行组，但不必填满。对 128 卡超节点，在出口扩展的条件下枚举张量并行 8、16、32、64，对应的数据并行为 128、64、32、16，全局 token 数不变。每个数据并行副本要处理的序列数随之增加，每条序列仍是 8192 个 token，局部矩阵的行数和张量并行的传输量都要重算；假设这些形状都能达到本例的有效算力，步时间分别约为 0.672、0.753、0.942、1.333 s。此时 TP8 仍然最好：张量并行组越大，每个副本要交换的激活越多，节省的本地梯度同步抵不过增加的激活交换。张量并行超过八个 KV 头时还要支持复制 KV 等布局，本例假定 kernel 支持且余量足够。

这次筛选说明，1024 张卡可以一起训练一个模型，但不是把模型的每一层都切到 1024 张卡上。若改为推理，多个较小的实例还能各自处理请求、隔离故障，第 6.7.4 节用同样的方法算过超节点大小对 decode 吞吐的影响。真实训练中局部矩阵效率、内存或重叠能力不同时，张量、上下文、流水线并行的组合可能更好；应像第 6.7.3 节那样扩展方案重算，而不是把本例的 TP8 当作通用答案。

**故障恢复的开销。**上面的吞吐都假设作业从不中断。同步训练中，一张卡故障可能让整个作业停下；数据并行副本也不能像独立的推理实例那样随意丢弃，因为那会改变这次更新用到的样本和梯度。除非系统提供不改变训练含义的局部恢复或弹性方案，否则要回退到一致的 checkpoint。设每隔 $I$ 秒有效计算暂停 $C$ 秒保存 checkpoint，作业中断率为 $\lambda$，恢复用时为 $R$。故障率较低、故障时刻在 checkpoint 间隔内近似均匀分布时，开销比例可以粗略估计为

$$\epsilon \approx C/I + \lambda(I/2 + R), \qquad \Theta_{\text{effective}} \approx \Theta/(1+\epsilon).$$

这里只估计平均有效进度，不保证尾延迟。独立卡故障可按卡数合计中断率，超节点公共设备故障则按实际故障域统计，不能把所有故障都当作独立。中断率取 Meta 研究集群的统计：1024 卡作业平均 7.9 小时中断一次，中断率与卡数成正比，相当于每卡平均约 337 天（约 8090 小时）一次，第 10 章也用这组数据。这组统计已经包含公共设备的故障，本例不再另加超节点公共故障项，因此中断率不随超节点大小变化；任一中断都触发全作业恢复。每 600 秒有效计算暂停 10 秒保存 checkpoint。恢复取固定 60 秒，再加受影响超节点每卡 64 GB 状态经一条 PCIe Gen5 x16（每方向 64 GB/s）读回的时间。128 与 256 卡节点的恢复时间分别为 188、316 秒，附加开销约为 3.4% 与 3.8%，出口扩展条件下有效吞吐约为 150.9 万与 152.3 万 token/s：256 卡节点恢复更慢，多出的开销仍小于其正常吞吐的提高。

中断率来自一个集群的统计，checkpoint 间隔与恢复通道是给定输入。节点变大可能减少公共设备的数量，也可能扩大一次故障的损失和恢复流量；有备用卡、局部恢复或不同故障率时应重新代入。因此，比较方案时要同时给出正常运行的吞吐、恢复模型和目标时间内的有效训练进度，第 10 章再展开 checkpoint 与恢复协议。

### 7.6.5 部署与常见误区

大梯度、小数据量传输和故障恢复分别突出了带宽、启动开销和依赖等待。部署方案的比较也应从这些工作负载出发。InfiniBand、RoCE 和 UB 分别提供具体的连接与通信机制；选择部署方案时，先用本章的三个模型比较各方案的通信路径。GPU 到网卡的连接、共享接口和交换拓扑决定可用带宽；请求提交与完成处理机制决定实际吞吐；应用的同步与恢复机制决定执行顺序。

计算可用带宽时，需要先明确物理端口的连接方式。例如，昇腾 950 中的 UBoE 与 UB Link 链路按预先配置的方式共用 SerDes，一组 SerDes 的用途由端口配置确定。选择端口用途之后，才能计算该方向实际可用的链路带宽。[49] 接口统一后，程序可以用共同的方式访问，端口配置则决定数据经过哪些链路，以及这些链路有多少带宽。

[49] UB 与昇腾 950 原件核对，包含 UB 基础规范 2.0.1、操作系统参考设计 2.0 及昇腾 950 官方白皮书。域、传输模式、双向带宽与 SerDes 复用分别核算。

公开 AllReduce 运行记录可以用来分析这些因素的共同影响。16 GiB 输入、输入与输出分别使用不同缓冲区时，两台与本章算例同规格的八卡 H100 服务器约需 68.7 ms，四台约需 91.0 ms。[50] 增加服务器带来了更多参与者和更长的协作路径。

本章的推导也能澄清四种常见误区。

**误区：发送总量相同，通信耗时就相同。**连续环、交错环、分层归约都发送 5760 MiB，跨服务器部分却分别为 720、5760、384 MiB，用到的网卡分别为一张、八张、八张。传输时间取决于数据经过哪些资源，参与者的排列改变了每张网卡分到的字节。

**误区：链路平均利用率低，就足以应对通信高峰。**两项周期作业平均只需 20 GB/s，通信高峰重叠时却达到 100 GB/s，在 50 GB/s 出口积压 1 GB。缓冲保存的是一段时间内的差额，相位和反馈决定这个差额积累多久。

**误区：按序返回，就一定读到了正确数据。**在数据更新之前读取到的旧值，不会因为延后返回而变成新值。发布与读取之间的依赖必须约束实际读取的时刻，提前执行则需要检查和重读。

**误区：通信加速比就是任务加速比。**分层归约把通信从约 7.6 ms 降至 1.3 ms，串行训练步从约 29.6 ms 降至 23.3 ms；数据提前就绪又把分层方案降至 22.0 ms。同一种通信优化，在不同执行顺序下节省的总时间也不同。

笔者理解统一互联的出发点，是让设备能够直接访问远端数据、发起通信，让上层明确表达依赖，让底层复用相同的传输状态。评价这些设计，需要回答三个问题：少搬了哪些字节，少等了哪段时间，为此需要额外保存哪些请求状态、执行哪些处理。

> **实验 7-15 ・核心：通信优化能缩短多少训练与推理时间**
> 先复算 1024 卡算例，保持全局 token 数不变，将超节点大小、出口上限、本地带宽逐项改变，记录哪种通信算法和张量并行候选更好；再把恢复通道带宽减半，比较有效进度。最后复算例题 7.6，画出连续环、分层归约、在途请求数不足、提前通信与只剩一张网卡的整步时间线。求分层通信恰好被 20 ms 计算覆盖时的数据最晚就绪时刻。再换成 36 层、每层两次 8 KiB 归约，分别考虑将带宽提高到原来的三倍，以及将每次启动开销降至 2 μs，解释两类负载的改进顺序为何不同。

## 本章小结

从单卡算子到超节点，再到数据中心，切分与调度始终围绕同一组数据依赖。分析跨服务器任务，可以先跟踪一份数据：沿算法求它必须交给哪些参与者，再沿拓扑找它经过哪些资源。两台服务器的算例先做本地归约，把跨服务器发送量从 720 MiB 降至 384 MiB，并让八张网卡同时工作，缩短了瓶颈链路上的传输时间。交换网络本身也是一项要计算的资源：超售比决定割集有多宽，rail 是否对齐决定字节走一跳还是经脊层，哈希冲突与 incast 决定有效带宽和缓冲，在网归约则把跨服务器阶段压成一轮。

选定通信路径后，还要连续提交和处理请求。带宽—时延积决定需要多少在途字节，每次请求的数据量和启动间隔决定每秒能提交多少数据，完成通知的处理速度决定请求槽位多久可以复用。共享状态节省容量，隔离状态控制干扰；数据何时可读、何时可以释放，决定这些状态需要保留多久。把一次远程访问逐阶段相加可以看出，控制器接在 PCIe 之后还是片上总线上，决定了每次访问的固定开销；连接状态按端点数相加还是按端点对相乘，决定了它能否留在片上缓存里。

1024 卡算例进一步说明，超节点大小、节点内带宽和外部出口是不同条件。高速域从一台八卡服务器扩到 64 卡，更多归约留在本地，梯度同步从占步时间的三分之一降到一成左右，吞吐提高约 36%，再往上收益迅速

[50] experiments/ch07/07-03 的公开运行记录核验。数据来自官方项目中的用户提交记录。

变小；高速域扩大后也可以重新比较张量并行与数据并行的组合，但出口受限、本地归约的成本或恢复开销都可能改变选择。一个作业用上全部的卡，不等于每种通信都要覆盖全部的卡。

最后，将数据准备、传输、使用和恢复的先后关系画成依赖图。减少数据传输量和请求处理工作量、提早开始独立操作、缩短关键路径，是网络优化转化为任务收益的三种方式。下一章将以这些执行与互联能力建立单实例推理服务，第 9 章进一步组织跨实例的数据交接。

# 第 8 章 推理优化

同一个模型运行在同一台加速器上，单个用户每秒可能只收到几十个 token，所有用户合计每秒却能收到上千个 token。decode 每读一遍权重只做很少的计算，速度受限于显存带宽而不是峰值算力；把多个请求合并计算、共用一次读取的权重，总吞吐就能成倍提高。与此同时，每条请求的上下文长度、排队时间和执行顺序又决定了用户何时能收到答案。本章讨论如何组织这些请求，在给定时间内返回更多正确答案。

推理实例可以使用一张卡，也可以由多张卡、多台服务器协同执行。第 6、7 章已经说明模型计算如何切分，以及加速器如何通信。本章以给定加速器组合为基础，研究批处理、请求调度、内存管理、压缩、卸载和推测解码，比较不同运行配置的执行效率。第 9 章再讨论计算和状态应分布在哪里，如何组织完整副本、阶段服务池和算子分工，并据此确定服务规模。

本章从具体的设计问题出发。示例中的 Qwen3-8B 实例运行在一张 RTX PRO 6000 Blackwell 工作站版显卡上：显存 96 GB，带宽 1792 GB/s，BF16 输入、FP32 累加的稠密峰值为 503.8 TFLOP/s，功耗上限 600 W。[1]除了在 Apple M2 Max 上运行的实验 8-7，本章实验都在这张卡上完成。模型权重采用每元素 2 字节的 BF16 浮点格式，约占 15.3 GiB。KV 缓存保存上下文 token 的键（K）和值（V），供后续注意力计算读取。与实验 8-9 的设置相同，本实例分到 32 GiB 显存，为 KV 缓存及辅助缓冲区预留 12 GiB，其余约 4.7 GiB 用于激活、计算图重放（第 5.5 节）的固定缓冲，以及其他执行工作区。

实例处理两类请求：短对话输入 2048 个 token，长上下文请求输入 8192 个 token，二者都输出 256 个 token。长请求的前 6144 个 token 是相同的系统提示和工具定义。要求每条请求从到达到返回完整的正确答案，耗时不超过 7 秒。一条短请求在这张卡上单独运行约需 6.95 秒（第 8.1.1 节），该时限只留出约 0.05 秒的余量。后文将反复使用这组条件。

| 设计条件 | 短对话 | 长上下文 |
|---|---|---|
| 输入长度 | 2048 token | 8192 token |
| 输出长度 | 256 token | 256 token |
| 公共前缀长度 | 0 | 6144 token |
| 完成时限 | 7 s | 7 s |

首先计算实例能够同时保存多少条请求的 KV 缓存，再计算增大 batch 能为每个输出 token 减少多少权重读取。随后引入分页、前缀共享、压缩、卸载和推测解码，分别分析各自节省了什么、增加了什么。章末将这些分析用于同一个问题：16 条长请求同时到达时，应选择哪种配置？

## 8.1 推理请求的执行过程与资源需求

### 8.1.1 执行资源与请求生命周期

**请求**是一条有独立输入、输出和结束条件的推理工作。本章所说的 batch，指在同一次加速器执行中一起计算的一组输入 token；**调度迭代**是调度器选择本轮工作、提交执行并处理结果的一个周期。请求可以跨越许多迭代，每轮与它合批的其他请求可能不同。

请求到达后，先进入等待队列。调度器将请求加入 batch、分配 KV 空间后，模型才开始处理输入，即 prefill

[1]NVIDIA RTX Blackwell PRO 规格附录 A 表 4，整理在硬件参数表的 rtx-pro6000-blackwell-ws 一行。

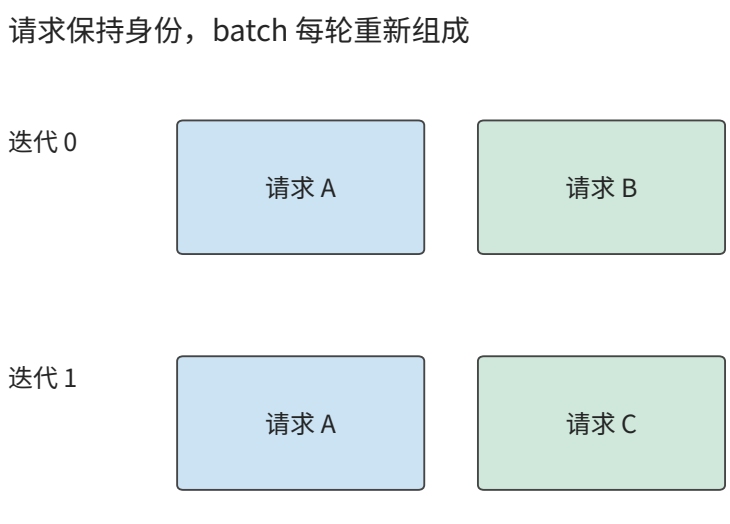



图 8-1：A 跨越两个迭代继续执行。B 在第一轮结束，下一轮由 C 接替其执行位置（每轮为一条活跃请求预留的一行计算）；请求状态仍按各自身份保存。

阶段。由于输入 token 已经全部给定，模型可以同时计算多个 token，并保存各层的键和值。提示的最后一个 token 计算完成后，模型生成第一个输出 token。

随后进入 decode 阶段。模型将刚生成的 token 作为下一次前向计算的输入，读取上下文 KV，再生成下一个 token。同一条请求必须依次完成这些步骤，不同请求的当前步骤则可以一起计算。因此，即使单条请求只能逐个生成 token，实例仍能通过合批提高总吞吐。

在这张卡上，实验 8-1 单独运行一条 2K 输入的短请求：prefill 并返回首个 token 用时约 0.096 秒，此后平均每隔 26.5 ms 返回一个 token。假设这条请求先排队 0.1 秒，用户便在到达后的第 0.196 秒收到第一个 token。总共输出 256 个 token，需要经历 255 个输出间隔，因此请求总耗时为

$$0.1 + 0.096 + 255 \times 0.0265 = 6.95 \text{ s}.$$

这条请求能在 7 秒内完成。如果生成期间插入另一条短请求的 prefill，某个输出间隔就要多等约 0.096 秒，从 26.5 ms 延长到约 123 ms，总耗时增加到 7.05 秒。每个 token 的计算内容没有改变，仅仅调整执行顺序，就可能使请求超时。

图 8-2 把这 6.95 秒画成一条时间线。首个 token 的返回时刻将时间线分成两段：此前的排队和输入处理决定用户何时看到响应，此后的 255 个间隔决定用户何时收到完整答案。后续所有调度优化，都可以理解为改变这条时间线上某一段的长度。

请求完成后，调度器不再为它安排计算，状态管理器则决定如何处理留下的 KV。该请求独有的 KV 随请求结束而释放，公共前缀的 KV 则可以保留下来供后续请求复用，由缓存策略继续管理。第 8.3 节将进一步说明这些缓存何时共享、何时释放。

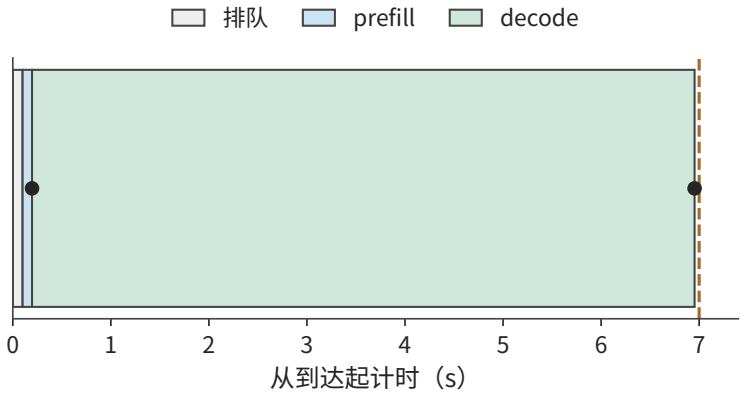



图 8-2：一条输出 256 个 token 的请求。排队 0.1 秒、prefill 0.096 秒，后续 255 个输出间隔各为 26.5 ms（实验 8-1 的 batch 1 实测）。横条按实际时间比例绘制，点标出首、末 token；7 秒虚线是完成时限。

### 8.1.2 权重、KV 与运行时缓冲的存储容量需求

要让多条请求沿各自的时间线向前执行，实例必须同时保存权重、请求状态和计算中间结果。本节的算例只用一张卡，因此只列出这张卡上的内存占用。多卡实例则要按各卡保存的权重分片、KV 和缓冲区分别列式：某张卡上的空闲内存，只有改变数据放置，才能用于另一张卡的计算。

用 $M_w$ 表示常驻权重的容量，$M_{\mathrm{KV}}$ 表示去除重复引用后的物理 KV 块总容量（KV 按固定大小的块分配，多条请求共用的块只计一次），$M_a$ 表示激活与普通工作区的容量，$M_u$ 表示计算图重放等其他缓冲区的容量。这些内存占用之和必须小于或等于加速器可用容量 $C$：

$$M_w + M_{\mathrm{KV}} + M_a + M_u \le C.$$

在这几项内存占用中，权重在请求到达前就已加载到内存，KV 则随着请求到达和生成不断增长。要算出实例能同时处理多少请求，先求每个 token 要保存多少 KV。Qwen3-8B 有 36 层，每层有 8 个 KV 头，每个头的维度为 128。每个 token 在每层都需要保存 K 和 V，BF16 每个元素占 2 bytes，因此每个 token 所需的 KV 容量为

$$k = 2 \times 36 \times 8 \times 128 \times 2 = 147\,456 \text{ bytes} = 144 \text{ KiB}.$$

输入 2048 个 token 后，KV 占 288 MiB；输入 8192 个 token 后，KV 占 1152 MiB。所有请求共用约 15.3 GiB 权重，但每增加一条上下文独立的长请求，就要增加 1.125 GiB KV。16 条长请求仅保存输入的 KV 就需要 18 GiB，超过预留的 12 GiB。[2]

生成输出时，KV 还会继续增长。prefill 产生第一个输出 token，此后将前 255 个输出 token 依次送回模型，生成剩余的 255 个 token。短请求共计算 $2048 + 255 = 2303$ 个 token，长请求共计算 $8192 + 255 = 8447$ 个 token。若每块容纳 16 个 token，分别需要分配能容纳 2304 和 8448 个 token 的空间。

[2]固定模型在 RTX PRO 6000 上的批处理计算及8K 对照。完整 BF16 权重为 16,381,470,720 bytes，一步理想共享矩阵权重读取为 15,136,811,008 bytes；embedding 按请求查行，内存中仍需保存完整的 embedding 矩阵。读取表先用精确字节计算，再舍入各项。

| 独立请求 | prefill 后 KV | 输出 256 个 token 后的 KV 分配量 | 12 GiB 可同时容纳的请求数 |
|---|---|---|---|
| 短对话 | 288 MiB | 324 MiB | 37 |
| 长上下文 | 1152 MiB | 1188 MiB | 10 |

表中最后一列分别由 $\lfloor 12288/324 \rfloor$ 和 $\lfloor 12288/1188 \rfloor$ 得到。如果只按 prefill 结束时的 KV 大小计算，12 GiB 能容纳 42 条短请求；但这些请求都生成 256 个输出 token 后，所需空间会增加到 13,608 MiB。接收请求时就预留生成阶段所需的空间，可以避免执行到一半才发现内存不足。

共享公共前缀可以进一步节省内存。长请求的前 6144 个 token 占 864 MiB KV，其余输入占 288 MiB。如果 16 条请求各自保存前缀，就会保存 16 份相同的 KV；如果引用同一份 KV，节省的空间便可用于各自的后缀。第 8.3 节将说明具体的块映射方式，并计算生成结束时的总容量。

### 8.1.3 从单请求时间到服务目标

把第 8.1.1 节时间线的两段写成可以设定时限的指标。设请求到达时刻为 $t_a$，共输出 $G$ 个 token，第一个与最后一个 token 返回给用户的时刻分别为 $t_1$、$t_G$。于是

$$\mathrm{TTFT} = t_1 - t_a,$$

第 $j$ 个输出间隔为 $\mathrm{ITL}_j = t_{j+1} - t_j$。将首输出之前与之后的时间相加，完整请求时间为

$$T_{\mathrm{request}} = \mathrm{TTFT} + \sum_{j=1}^{G-1} \mathrm{ITL}_j.$$

第 8.1.1 节的例子中，TTFT 为 0.196 秒，后续 255 个输出间隔合计 6.76 秒。较短的 TTFT 让用户尽早看到响应，均匀的输出间隔便于连续阅读，而总耗时决定完整答案何时可用。SLO 将这些要求量化为时限。章首算例要求总耗时不超过 7 秒，调度算例还会比较最长输出间隔。

这些指标针对单条请求；实例整体的产出，还取决于一次同时处理多少请求，即 batch size（下文简称 batch）。吞吐表示单位时间内完成的工作量。实验 8-1 同时处理 16 条 2K 请求时，引擎每轮 decode 约 27.44 ms，每轮共输出 16 个 token，生成吞吐约为 $16/0.02744 \approx 583$ token/s；单条请求每 26.26 ms 输出一个 token 时，该值只有约 38。增加每轮输出数或缩短每轮时间，都能提高吞吐；用户实际等待多久，还取决于请求开始执行前的排队时间。

衡量服务效果还要检查答案是否正确。固定输出长度，便于在相同工作量下比较执行时间；让模型自然停止，则可以观察答案正确率、回答长度和重试次数。第 8.6 节将计算单位时间内正确且按期完成的请求数，同时评价速度与质量。

## 8.2 批处理与请求调度

### 8.2.1 权重复用如何提高批处理效率

考虑矩阵乘法 $Y = XW$，输入矩阵 $X$ 有 $b$ 行。同一个权重元素参与所有行的计算，因此将多行一起处理，可以在读取一次权重后完成更多乘加。假设每个权重元素在批内只读取一次，每次乘加计为两次运算，权重元素占 $s_w$ bytes。只计权重读取时，算术强度为

$$I_{\text{weight}} \approx \frac{2b}{s_w}.$$

BF16 的 $s_w = 2$。把同时执行一步 decode 的请求数从 1 增至 16，每条请求都提供一个新 token，各 token 的特征向量分别占输入矩阵的一行；矩阵便从一行增至 16 行，同样一次权重读取对应的运算量增至原来的 16 倍。不同请求共用权重矩阵，但注意力计算仍要分别读取各自的上下文 KV。

共用的前提是这一批请求用的是同一份权重。**低秩适配（LoRA）**放宽了这一前提：用两个小矩阵的乘积表示权重的调整量，基础权重原样保留，于是不同任务可以共用一个基础模型，各自只附加一份小的 adapter（适配器，即附加在基础模型上的可训练参数模块）。对 Qwen3-8B 的 Q、V 投影取低秩维度 16、BF16，一套 adapter 占 14.6 MiB，100 套合计 1.43 GiB，远小于 14.1 GiB 的矩阵权重。

一批请求若来自不同 adapter，共用的就只剩基础权重这一部分。低秩计算要按 adapter 分组，每组的行数是本批中属于该 adapter 的请求数；adapter 分得越散，每组行数越少，这部分的权重复用就越差。因此，一批里放几种 adapter 和放多少条请求，是两个要分开决定的量。

容量上同样要分开算。100 套 adapter 占 1.43 GiB，而 100 条互不共享的 8K 上下文，KV 合计 112.5 GiB。能容纳多少套 adapter 由小矩阵决定，能同时服务多少条请求仍由 KV 决定；前者宽裕，不等于后者也宽裕。[3]

设每轮只需读取一次的矩阵权重大小为 $D_w$，每条请求有 $L$ 个上下文 token，彼此不共享 KV，每个 token 的 KV 容量为 $k$。一轮的读取总量，以及平均每个输出 token 分摊的读取量，分别为

$$D(b, L) = D_w + bLk, \qquad d(b, L) = \frac{D_w}{b} + Lk.$$

在每个输出 token 的读取量中，第一项是分摊后的权重读取，第二项是本请求的上下文 KV 读取。Qwen3-8B 的 $D_w$ 约为 14.1 GiB，2K 上下文的 KV 为 0.28125 GiB。当 batch 从 1 增加到 16 时，每个输出 token 分摊的权重读取降至约 0.88 GiB；batch 增至 64 时，再降至约 0.22 GiB。此时，每个输出 token 读取的 KV 已经比权重更多。[4]

图 8-3 展示了这一变化。横轴上相邻的刻度表示 batch 加倍，因此权重项每次减半；KV 项保持不变，两项之和逐渐接近 KV 对应的水平线。上下文为 2K 时，batch 从 1 增至 64，每个输出 token 的读取量从约 14.38 GiB 降至 0.50 GiB，约为原来的 1/29。上下文长度增至 8K 后，KV 容量变为原来的四倍，合批后的每个输出 token 仍需读取更多数据。

随着 batch 增加，分摊权重带来的节省逐渐减少。可以用权重读取量与 KV 读取量相等时的 batch 描述这一变化。使整批 KV 读取量达到或超过共享权重读取量的最小 batch 为

$$b_{\text{KV}} = \left\lceil \frac{D_w}{Lk} \right\rceil.$$

用未经四舍五入的权重字节数计算，2K 上下文的结果为 51，8K 上下文为 13。上下文长度增至四倍后，KV 读取在小得多的 batch 下就超过了权重读取。与此同时，更大的 batch 也需要更多内存：64 条 2K 请求的权重和一步 decode 后的 KV 合计约占 35.7 GB。合批减少了每个输出 token 的读取量，却增加了同时驻留的数据量。

[3] MLSys 2024 调研与多 LoRA 计算。

[4] 固定模型在 RTX PRO 6000 上的批处理计算及8K 对照。完整 BF16 权重为 16,381,470,720 bytes，一步理想共享矩阵权重读取为 15,136,811,008 bytes；embedding 按请求查行，内存中仍需保存完整的 embedding 矩阵。读取表先用精确字节计算，再舍入各项。

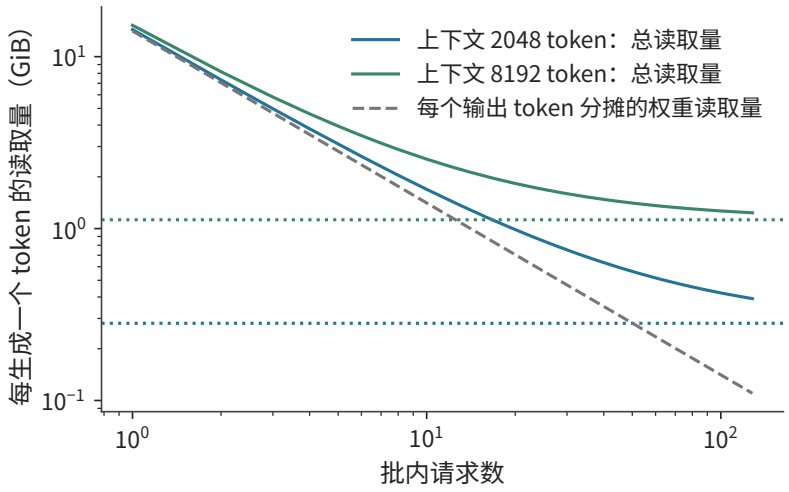



图 8-3：批处理怎样减少每生成一个 token 所需的读取量。横轴为同时生成的请求数，纵轴为整批读取字节数除以本轮输出 token 数。蓝、绿实线分别对应每条请求已有 2048、8192 个 token 的上下文，包含权重与 KV 读取；同色水平点线仅表示该长度下的 KV 读取量。灰色虚线表示批内共用的权重读取量除以请求数。模型为 Qwen3-8B，使用 BF16；每份矩阵权重每轮读取一次，各请求的 KV 独立。两轴均为对数刻度。

再估算这些读取所需的时间。设每条请求的运算量为 $F$，加速器每秒完成 $P$ 次运算，内存带宽为 $\beta$。分别计算运算与读取所需的时间，再取较大者估计一轮执行时间：

$$T_{\text{step}} \approx \max\left(\frac{bF}{P}, \frac{D_w + bLk}{\beta}\right).$$

令两项相等并整理，得到 $b(F/P - Lk/\beta) = D_w/\beta$。每增加一条请求，计算时间增加 $F/P$，KV 读取时间增加 $Lk/\beta$。前者更大时，计算时间会随着 batch 增长而追上读取时间；否则读取始终耗时更长。上述 51 和 13 比较的是两类数据的读取量，这里的方程比较的是计算与内存带宽对执行时间的限制。

两项分别按峰值算力和峰值带宽计算，得到的是这张卡在这组条件下的物理下界。下界与实测一轮时间之比，就是第 1.2.2 节定义的 MFU 和 MBU；第 8.6.3 节将用逐轮记录校准这两个比值，再用于本章其余算例。

落实到请求时间上，还要看更大的 batch 是否延迟了首个 token，以及相邻输出之间是否等得更久。batch 扫描实验同时记录了这两项变化。Qwen3-8B 输入 2K、输出 256 个 token 时，batch 从 1 增至 64，总吞吐从约 37 增至 1165 token/s；但首个 token 的返回时间从约 96 ms 延后到 3.29 秒，平均输出间隔从约 27 增至 40 ms。每轮输出虽然更多，但处理更多输入、运行更大的 batch 也花了更多时间。该实验中每批请求同时提交；在线服务若额外等待 batch 集满，这段时间还要计入首 token 延迟。[5]

**练习 8-1 ·计算：batch 增大时的读取量变化。**取 $D_w = 15,136,811,008$ bytes、$k = 144$ KiB，计算 2K 与 8K 上下文在 batch 为 1、4、16、64 时每个输出 token 的读取量，并分别求两种上下文长度下，KV 读取量首次达到或超过分摊权重读取量的最小整数 batch。再用 12 GiB KV 可用内存、每块 16 个 token 和 256 个输出，分别求两种上下文长度下最多可以接纳多少条独立请求。比较可容纳请求数的上限与两类读取量相等时的 batch，判断容量限制是否会先阻止 batch 继续增大。

如果权重改由足够快的独立存储提供，读取耗时减少，批处理的收益就需要重新计算。沿用第 4 章的 8K 示例，设 $W$ 为整批读取的权重字节数，$B$ 为批内请求数，$K$ 为单请求本步读取的 KV 字节数。传统方案中，每个输出 token 分摊的 HBM 读取量为 $W/B + K$；权重由足够快的独立 ROM 提供时，HBM 每输出一个 token

[5]batch 扫描原始记录。RTX PRO 6000、vLLM 0.23.0、BF16，三轮；正文报告中位数，吞吐覆盖剩余 prefill 和生成，排除加载与预热；8K 条件先建立共享前缀。

仍需读取 $K$。[6] 图 8-4 中两条曲线的距离随 batch 增大而缩小，说明原先依靠 batch 摊薄权重读取的收益正在消失。

不过，batch 仍会影响矩阵计算对计算单元的利用率，以及 batch 内的 KV 读取量。将新的读取量代入上述时间模型，就能求出计算与 KV 读取的交点，再按首 token 延迟和输出间隔选择 batch。

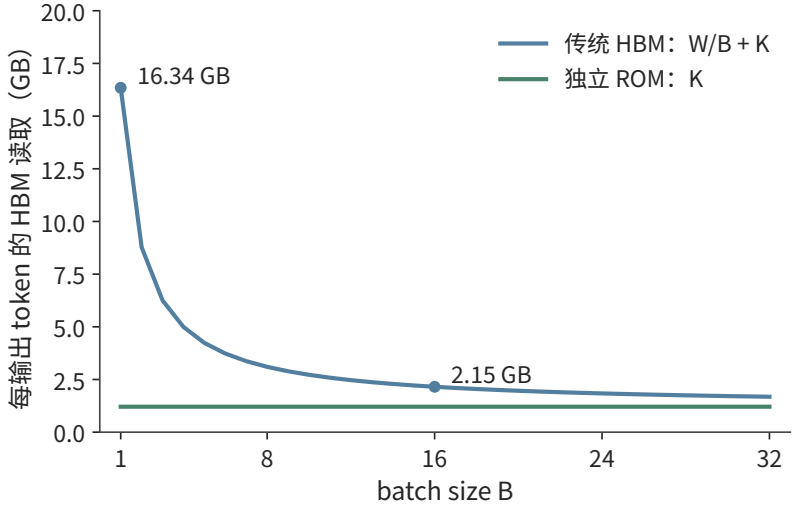



图 8-4：每输出 token 分摊的 HBM 读取量。固定 8K 上下文，传统路径为 W/B+K，独立快速 ROM 路径为 K；纵轴按上述公式计算读取量。W 是整批读取的权重字节数，B 是批内请求数，K 是单请求本步读取的 KV 字节数；W/B+K 为每个输出 token 分摊的读取量。

### 8.2.2 从固定 batch 到连续批处理

两条请求同时开始生成，一条需要八个 token，另一条只需要两个。短请求结束以后，长请求还要继续六步。如果一定要等这两条请求都结束才接下一组，短请求空出的执行位置就会一直闲置。**固定批处理**按整组接收、整组结束；连续批处理则在迭代边界重新安排活跃请求，空出位置后即可接纳新请求。

第 8.2.1 节的 batch 分析假定一轮中有 $b$ 条请求参与计算，实际行数会随请求结束和新请求加入而变化。图 8-1 中的执行位置对应这里的一行计算，与第 7 章记录通信请求的槽位是不同的资源。能否及时补入新请求，决定了下一轮还能有多少行参与计算。

新请求更早进入，也意味着其输入计算更早占用加速器。若新请求带有较长的提示，prefill 就会延长已有请求两次 decode 之间的等待。调度器因此要在两件事之间分配加速器时间：处理新请求的输入，以及让已有请求继续生成。[7]

**算例：连续批处理为何缩短总时间，却延长输出间隔？**最多同时运行两条请求。r0、r1 在零时刻到达，各输入 2048 个 token，分别输出 8、2 个 token；r2 在 20 ms 到达，输入 8192 个 token、输出 2 个 token；r3 在 30 ms 到达，输入 2048 个 token、输出 4 个 token。每轮耗时由实验 8-1 的 batch 1 实测拟合：固定 26.22 ms，每个新 token 加 30.4 µs，每对满足因果关系的查询与键加 3 ns。这组参数复现了 2K 请求单独运行时一轮 decode 的 26.26 ms 和 prefill 的 94.8 ms。固定部分是一轮中与 token 数无关的时间，包括读取整份权重（按 1792 GB/s 的峰值带宽约需 8.4 ms）和逐个提交 kernel 等开销。[8]

[6] 本章条件比较的逐项复算与计算程序。

[7] Orca 原文与分块计算和版本说明。

[8] 固定 batch、连续批处理、分块策略。每轮耗时参数由实验 8-1 的逐轮记录拟合：batch 1 的 2K decode 一轮 26.26 ms、prefill 94.8 ms；最后一个输出 token 在请求结束时尚未写入 KV。

固定 batch 让 r2、r3 等待 r0、r1 整组结束，总时间约 870 ms。连续批处理在 r1 结束后接纳 r2，总时间降到约 766 ms；但 r0 的下一次输出要等 r2 的 8K prefill，最大输出间隔从约 26 ms 增至 376 ms。总时间节省约 12%，一次输出停顿却增长到原来的 14 倍。

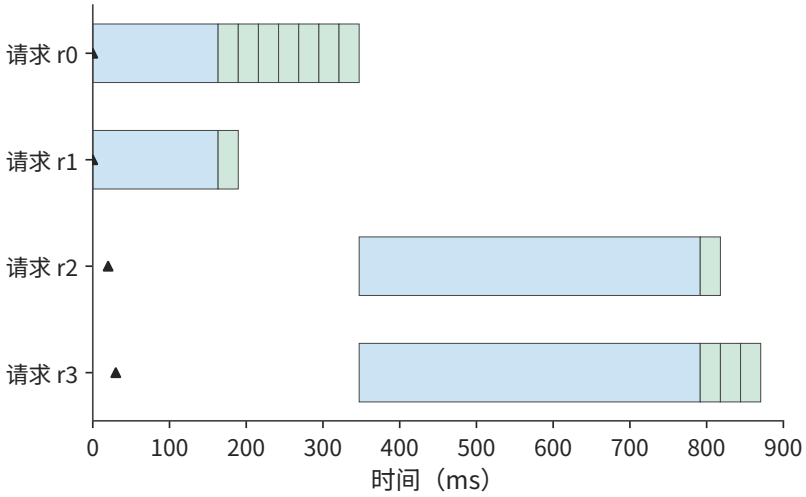



图 8-5：固定 batch 等待整组结束，再接纳 r2、r3。蓝色为 prefill，绿色为 decode，三角为到达时刻。各色带宽度为整次调度迭代耗时，时间按 RTX PRO 6000 上的实测拟合计算。

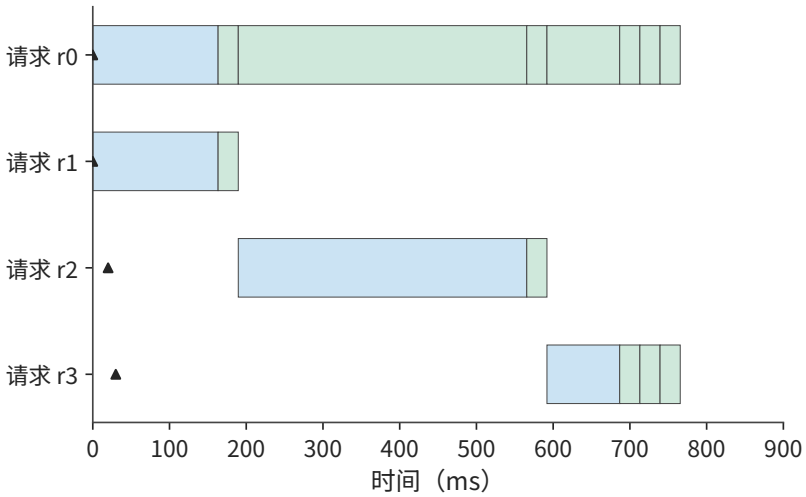



图 8-6：连续批处理在 r1 结束后接纳 r2。每行是一条请求；蓝色为处理输入（prefill），绿色为生成输出（decode），三角形为请求到达时刻，色带宽度为所在调度迭代的耗时。r0 等待 r2 的 8K 输入处理，最长输出间隔增至约 376 ms；全部请求约 766 ms 完成。

进一步把 r2 的 prefill 分成每块 2048 个 token，每轮最多处理 4096 个 token（与实验 8-1 的设置相同），每次先让已有请求继续 decode，再执行一段输入。最大输出间隔降到约 133 ms，所有请求的总完成时间约为 844 ms。分块后，调度器能更频繁地安排已有请求继续生成，使输出间隔更均匀。

### 8.2.3 长 prefill 对生成的干扰与分块执行

一次处理的 prefill 块越大，新请求所需的输入处理轮数越少；块越小，已有请求就越早得到下一次执行机会。调度器通常先安排正在 decode 的请求，再用本轮 token 上限中剩下的部分处理 prefill。这样，长输入就分散到多个输出间隔里计算。

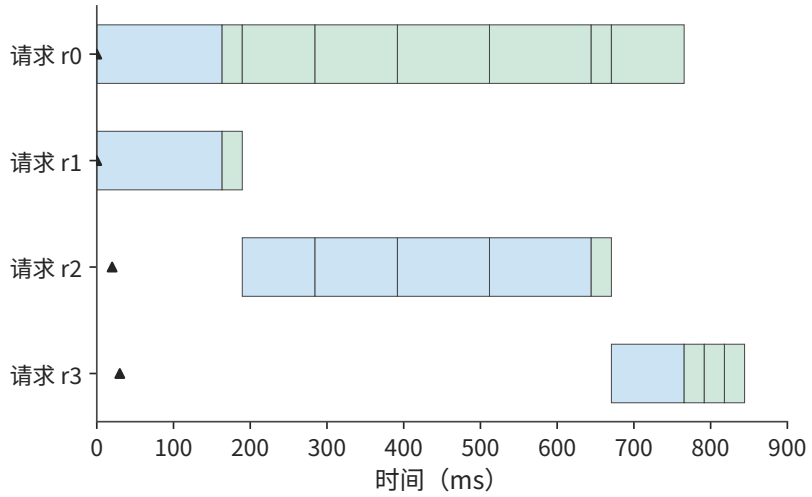



图 8-7：将长输入分块处理，每轮先安排已有请求生成，再处理一块新输入。每行是一条请求；蓝色为输入处理，绿色为输出生成，三角形为到达时刻。分块使最长输出间隔降至约 133 ms，全部请求约 844 ms 完成。横轴与图 8-5、8-6 使用相同时间尺度。

块长相同，注意力的工作量仍会随着上下文增长。设本块包含 $c$ 个新 token，已有上下文长度为 $h$。第一个新 token 关注 $h+1$ 个 token，第二个关注 $h+2$ 个，最后一个关注 $h+c$ 个。把这些配对逐项相加：

$$N_{\text{pair}} = (h+1) + (h+2) + \cdots + (h+c) = ch + \frac{c(c+1)}{2}.$$

$ch$ 来自新 token 对旧上下文的访问，三角形项来自块内的因果注意力。取 $c = 512$，首块的 131,328 个配对全部来自块内；8K 输入的末块已有 7680 个上下文 token，配对数增至 4,063,488，约为首块的 31 倍。

图 8-8 将配对数画成面积：每个新 token 都要访问全部旧上下文，形成左侧矩形；块内只能关注当前 token 位置及其之前的位置，形成右侧三角形。块长固定时，右侧三角形不变，左侧矩形随上下文增长而变宽。这就是末块需要更多注意力计算的原因。

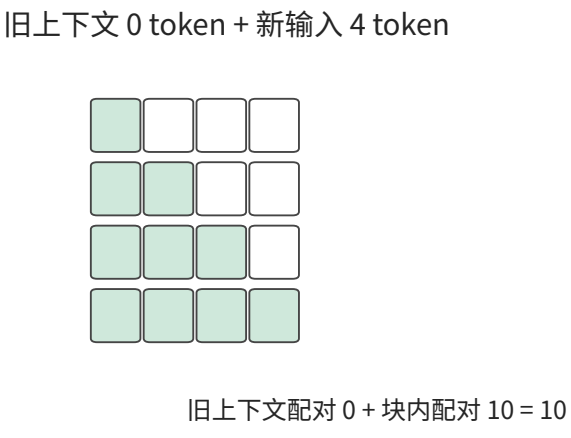



图 8-8：首次处理 4 个 token 时，没有旧上下文，只有新 token 之间的因果注意力配对，形成 $1+2+3+4=10$ 个绿色格。白格表示未来 token，不参与当前查询。

模型还要执行投影和 FFN，这些对各 token 分别做特征变换的计算每块都处理 512 个新 token，工作量基本相同。在已有逐块记录中，末块的模型主干矩阵运算量比首块多约 32%，执行时间中位数从约 25.3 ms 增至

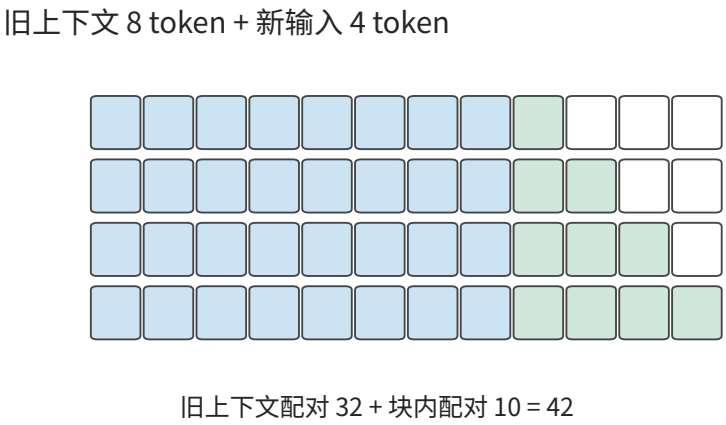



图 8-9：已有 8 个 token 的上下文时，4 个新 token 与旧上下文形成 $4 \times 8 = 32$ 个蓝色格，块内仍为 10 个绿色格。块长相同，总配对从 10 增至 42。

35.1 ms，增加约 39%。整块执行时间因此由两部分共同决定：一部分主要随新增 token 数变化，另一部分还随已有上下文长度增长。[9]

上下文增长解释了同样大小的块为何越来越慢；缩小块长则会带来另一种开销。把一个 256 个 token 的块拆成两个 128 个 token 的块，调度机会增加一次，矩阵行数减半，每个小块都要使用同一份权重。以一层 FFN 的 288 MiB 权重为例，设两次执行分别从显存读取它们，这部分读取就从 288 增至 576 MiB。小块让已有请求更早获得执行机会，也增加了执行次数；调度器应选择能满足输出间隔要求的较大块，使新增启动与读取尽量少。[10]

### 8.2.4 Token 预算、动态形状与执行时间

调度器通常设置每轮处理的 token 数量上限，称为 token 预算；第 8.2.2 节分块算例中每轮最多处理的 4096 个 token 就是一例。实际执行则要把这些 token 的特征向量组成矩阵，并选用 kernel 或预先捕获的 CUDA Graph（第 5.5.2 节）。实际执行的矩阵可能比本轮需要的更大，因此少处理一个 token，并不一定能少做一份计算。

例如，预先捕获 16 行和 32 行两种计算图，将 17 到 32 行的输入都填充到 32 行。把工作从 18 行减为 17 行，仍执行 32 行图；再从 17 行减为 16 行，才能使用较小的图。若为 17 行另捕获一个图，就省去 15 行填充，但捕获该图需要准备时间，重放所用的缓冲区还要长期占用内存。因此，选择 CUDA Graph 的预设形状时，也要计入第 8.1 节的运行时内存开销。

执行形状决定一轮的耗时，token 上限则决定这一轮分给新请求多少工作。增加上限能让长输入更早处理完，但已有请求要等这轮结束才能继续输出。在六条请求的回放记录中，请求每隔 80 ms 到达。每轮 token 上限从 512 增至 8192，后到请求在引擎中排队的时间从约 149 ms 降到不足 0.1 ms，最长输出间隔却从约 32 增至 183 ms。上限越高，一轮完成的输入越多，新请求越早离开队列，已有请求等待下一次输出的时间也越长。[11]

因此，可以分两步确定每轮处理多少 token。先根据允许的输出间隔确定一轮最多执行多久，再结合上下文长度和预设执行形状，换算为可处理的新 token 数。在相同时间内，上下文较长的请求能处理的新 token 较少，上下文较短的请求则可以使用更大的块。确定执行安排后，还要为这些请求分配 KV 空间；下一节将讨论如何

[9]逐块计算和封存测量。11 次请求、每次 16 块，首末块时间中位数 25.30、35.07 ms；CUDA event 区间包括完整模型路径和可能的主机间隙，不含外部 logits / 采样。各块按原始顺序执行，输入 token 与内容逐块变化。

[10]OSDI 2025 调研、NanoFlow 正式稿与资源共享算例。

[11]六请求实测及图执行取舍。实验在共享 GPU 上按固定顺序执行六条合成文本请求。

分配和复用这部分内存。

> **练习 8-2 ·核心·分析：由输出间隔限制确定 prefill 分块大小。**采用第 8.2.2 节的四条请求与执行时间模型，手算 r0、r1 的初次 prefill 和下一步 decode 各需多久，再计算连续批处理接纳 r2 时该轮的耗时。解释最长输出间隔的来源。固定新块长度 $c = 512$，分别计算已有上下文长度 $h = 0$ 与 $h = 7680$ 时的注意力配对数。最后分析配套的六条请求回放记录，在“最长间隔小于 50 ms”和“后到请求尽快接纳”两种目标下分别选择每轮 token 上限，并指出哪一项耗时决定了选择。

## 8.3 KV 缓存的分配、复用与释放

### 8.3.1 预留、碎片与分页分配

第 8.1 节按最大输出长度为每条请求预留空间，保证它能持续生成。如果请求提前结束，预留空间的尾部就一直没有使用。另一种办法是随生成过程逐步扩展内存，但若必须保持空间连续，邻接区域被其他请求占用时，就需要搬移数据或等待。

可以把这种分配比作活页册：阅读顺序由页码确定，各页不必放在相邻位置，只要记得每一页在哪里。序列增长时，在任意空位放入新页并记录其位置，就不必为了连续排列而搬动整册内容。

**分页分配**将序列分成固定大小的块。**逻辑块**按序列中的 token 位置编号，相当于页码；**物理块**是实际保存 KV 的内存区域；每条请求维护的**块表**记录逻辑块对应哪个物理块。序列增长到需要新块时，引擎从空闲池分配一个物理块，并增加映射。注意力计算根据块表找到所需位置的 KV，因此物理块可以分散存放。这里讨论的是 KV 在内存中的分块寻址，多级存储之间的换入换出留到第 8.3.4 节和第 8.4.3 节讨论。

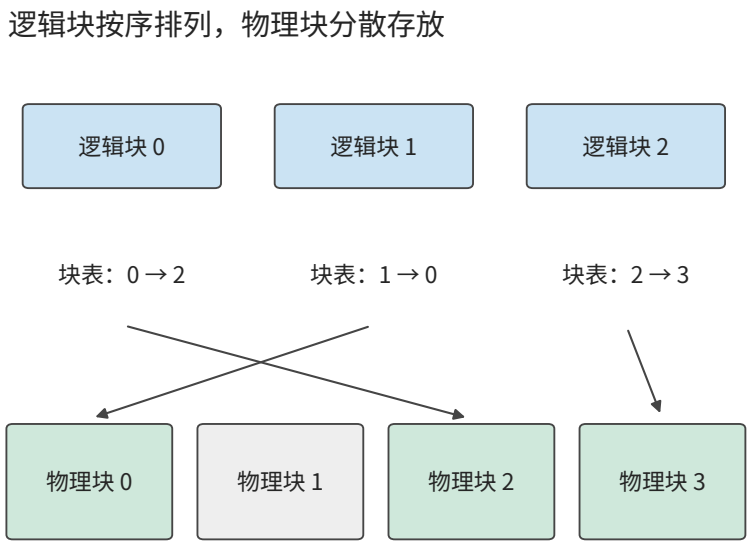



图 8-10：逻辑块 0、1、2 按顺序组成序列，块表分别指向物理块 2、0、3。物理块 1 为空闲，注意力按块表恢复逻辑顺序。

设每块可保存 $p$ 个 token 的 KV，当前长度为 $L$，需要 $\lceil L/p \rceil$ 个块。只有最后一块存在尾部未使用的 KV 槽位，浪费为

$$p\left\lceil \frac{L}{p} \right\rceil - L,$$

最多浪费 $p-1$ 个 token 的 KV 空间。块越小，这一上限越低，块表项和分配次数也越多。2023 年，伯克利等机构的研究者在 vLLM 中提出 PagedAttention，将操作系统分页管理的思路用于 KV 缓存，减少预留空间与内存碎片对服务并发的限制。PagedAttention 将这种映射带入注意力执行；FlashAttention 则在算子内部组织分块计算与中间结果，两者分别作用于状态分配和计算访存。

**算例：KV 分页分配能减少多少预留空间？** 设 A、B、C、D 当前长度分别为 9、13、5、15，每条最大允许 16 个 token。整段预留时，四条请求共预留可保存 64 个 token 的 KV 空间；取 $p=4$，分页分别分配 12、16、8、16，共可保存 52 个 token 的 KV 空间。其中实际保存的 KV 仍对应 42 个 token，未用容量从 22 个 token 减到 10 个 token。

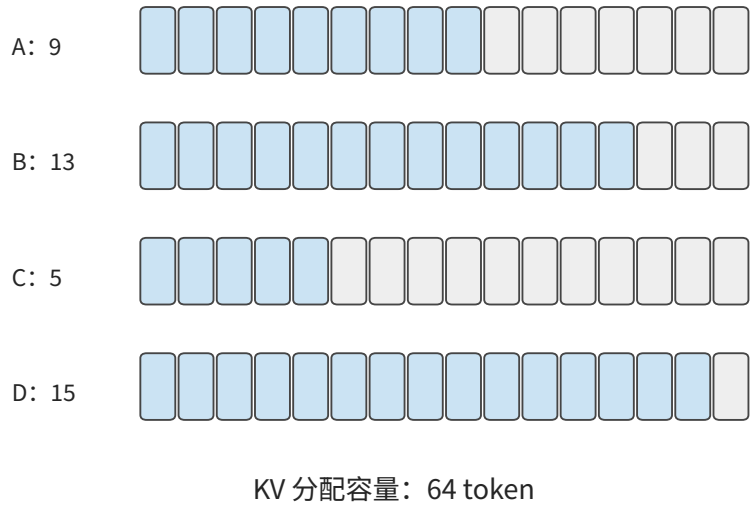



图 8-11：四条请求分别含 9、13、5、15 个 token，每条预留可保存 16 个 token 的 KV 空间，共分配可保存 64 个 token 的 KV 空间。蓝色已用，灰色预留未用。

若 A、B 的前八个 token 相同，还可以让它们共用两个完整块，使总分配量进一步降至 44 个 token。分页减少了尚未使用的预留空间，共享消除了重复保存的上下文，两者节省的是不同部分的内存。

### 8.3.2 分支共享、写时复制与内存释放

前缀共享图中，B 与 A 引用相同的两个物理块。如果 A 先结束，这两个块仍要留给 B；如果 B 要修改其中的内容，又不能影响 A。要实现共享，就需要确定何时释放共享块，以及如何避免不同请求互相覆盖数据。每张块表中指向某个物理块的记录称为一个**引用**，共享块由多个块表引用；**引用计数**记录还有多少使用者需要它。每当一条请求获得引用，引用数加一；请求结束时释放引用。最后一个引用释放后，该块才能回到空闲池；此时还要确认加速器操作不再访问它。这样，同一份共同上下文既能服务多个分支，也能在其中一条分支结束后继续服务其余分支。

写入共享尾块需要取得独立副本。例如，可存 4 个 token 的块中已写入三个共同 token 的 KV，两个分支接下来分别写入不同 token。如果写到原块，两条分支会争用块中的第四个 token。**写时复制**是在修改共享内容之前取得私有副本的机制。此时先复制这三个 token，两条分支分别追加；此前已填满的只读块继续共享。随着分支增长，公共前缀保持一份，各分支分别保存自己新增的后缀。

在已有的四分支实验中，公共前缀占 95 块，每条分支另有 9 个私有块。四张块表共有 $4(95+9)=416$ 次引用，但实际物理块数只有

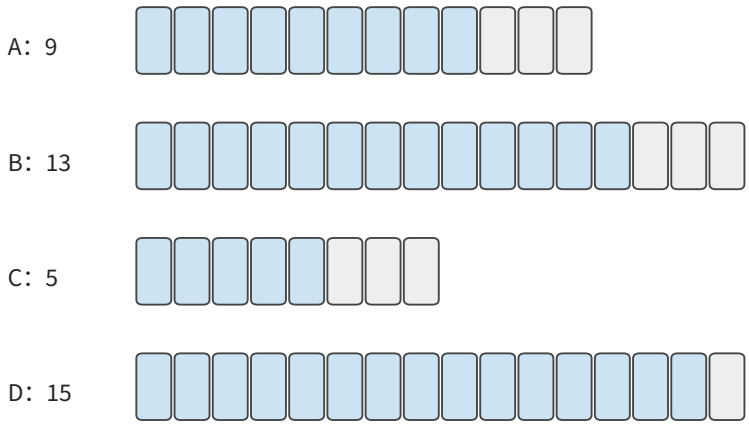



图 8-12：按每块保存 4 个 token 的 KV 分配空间，四条请求分别获得 12、16、8、16 个 token 的容量，合计 52 个。灰色表示块尾未用容量，由 22 个 token 减到 10 个 token。

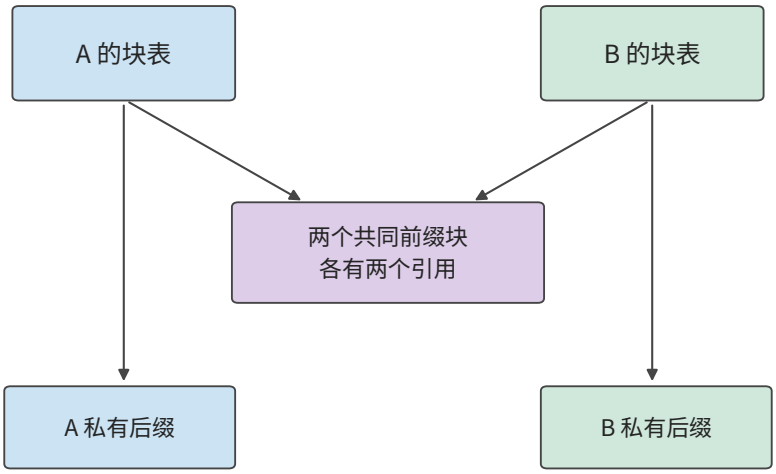



图 8-13：A、B 的两个共同前缀块只保存一次，各自块表都指向它们。A、B 保留各自的私有尾块；四条请求实际分配的 KV 容量进一步降至 44 个 token。

$$95 + 4 \times 9 = 131.$$

独立保存需要 416 块，共享后减少约 69%。这项节省来自对公共前缀的去重，36 个私有尾块仍随分支数增长。[12]

[12] 分页、取消与抢占、四分支共享、冷分支对照。共享分支的输出相同；冷、热分支具有相同的块峰值，但 prefill 量不同。

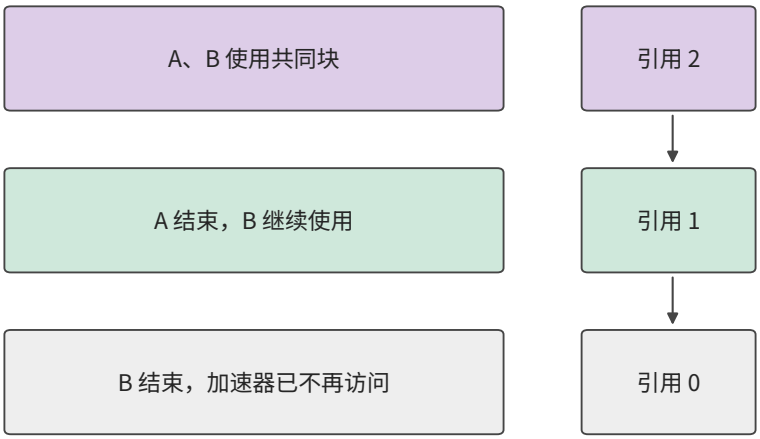



图 8-14：A 结束后引用数从 2 降到 1，B 仍可使用；最后一个引用释放且加速器已用完，块才能回到空闲池。

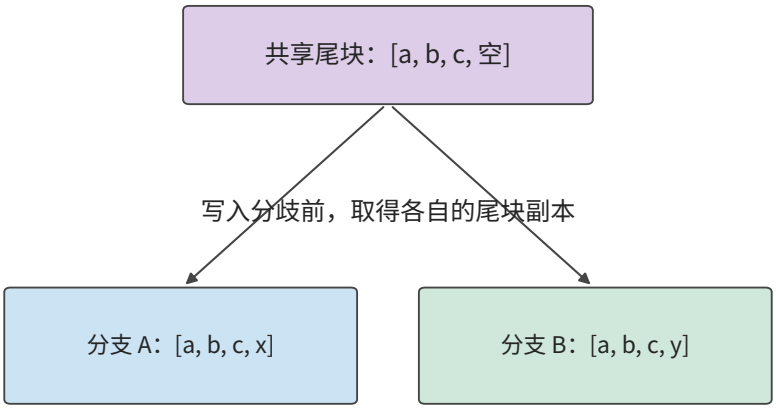



图 8-15：每块 4 个 token 的尾块已有共同的 a、b、c。分支分别追加 x、y，需要不同物理尾块；此前已填满的块继续共享。

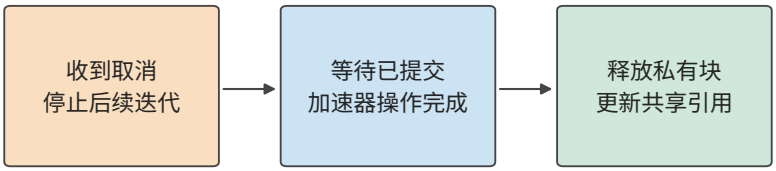
服务系统知道哪些 token 属于共同前缀，哪些 token 会在分支后独立写入，因此可以把模型的复用关系落实为物理块共享。块表负责地址映射，前缀关系决定共享范围，分支写入时才创建私有副本。原来按整条请求独立预留的空间，由此改为按实际产生与复用的状态分配。

**公共前缀共享后的整批 KV 容量。** 16 条长请求共享前 6144 个输入 token，其 KV 共占 864 MiB。每条请求的私有部分包含 $8192 - 6144 + 255 = 2303$ 个 token。按每块 16 个 token 分配，需要容纳 2304 个 token，即 324 MiB。因此，整组请求生成完毕时所需的 KV 空间为

$$M_{\mathrm{KV}} = 864 + 16 \times 324 = 6048 \text{ MiB}.$$

这组请求原来需要 $16 \times 1188 = 19008$ MiB，共享后约为 5.91 GiB，可以放入预留的 12 GiB 内存。一般地，$b$ 条同类长请求占 $864 + 324b$ MiB，因此容量上限从 10 条独立请求增加到 35 条共享请求。

块用完之后，还要正确释放才能交给后续请求。释放内存需要同时满足两个条件：没有使用者继续持有引用，已提交的加速器操作也不再访问该块。用户取消请求后，调度器停止安排新计算，等待已提交的操作完成，随后释放私有块。一次观察中，取消调用约 1.6 ms 就返回了，而对应的 104 个块直到约 31 ms 才释放。后续请求在释放之后使用这些块，便不会覆盖旧请求仍在读取的数据。[13]




取消调用返回 1.6 ms；块释放约 31 ms

图 8-16：取消接口返回表示已接收取消请求。已提交的加速器操作完成后，再释放私有块和相关引用；观测中的 1.6 ms 与 31 ms 对应不同事件。

除了请求结束或取消后的正常释放，内存不足时，引擎还可以抢占一条请求：暂停该请求并释放其 KV，之后再根据已保存的输入和输出重新计算状态。同一组实验使用 1 GiB KV 池时发生了一次抢占，比正常执行多调度 1805 个 token；使用 2 GiB 池时则没有这次重算。抢占暂时把内存让给其他请求，代价是恢复执行时需要重复计算。[14]

> **练习 8-3 ·计算：KV 块大小与前缀共享如何影响容量。**将四条长度 9、13、5、15 的序列分别按 2、4、8 个 token 分页，计算四条序列各自尾块中未使用的 KV 槽位数，并分别求出未使用 KV 槽位和块表项的总数。再按章首算例计算 8、16、32 条长请求在独立与共享状态下的完整容量。保持每条请求的总长度不变，将一个原本私有的完整块改为公共前缀的一部分，求 $b$ 条请求合计可以少分配多少物理块。

### 8.3.3 对话与智能体的前缀复用

第 8.3.2 节计算了同时运行的请求如何共用物理块。这种共享还可以跨越请求的结束时刻：旧请求留下的前缀 KV，能让后来的请求免去同一段输入的计算。如果后续请求的前缀完全相同，并使用相同的模型、位置编码规则、adapter 和状态格式，就能直接使用缓存中的 KV，从前缀末端继续 prefill。章首算例的公共前缀有 6144 个 token，命中缓存后，每条长请求只需处理剩余 2048 个输入 token，再开始生成。

这种复用沿前缀连续发生。假设两条提示前 100 个 token 相同，第 101 个不同，即使后面的文本又相同，它们从第 101 个 token 起所依赖的上下文已经不同。把动态时间戳放在提示最前面，会使后面较长的共同工具

[13]取消观察来自状态实验。记录值约 1.55 ms 与 31.41 ms，计时包含观察器开销。

[14]分页、取消与抢占、四分支共享、冷分支对照。共享分支的输出相同；冷、热分支具有相同的块峰值，但 prefill 量不同。

定义失去复用机会；把稳定的系统提示与工具定义放在前面，再追加本轮变化，就能保留较长的公共前缀。[15]

前缀树把这种结构直接表达出来：从根到分叉点是共同上下文，从分叉点到叶子是私有后缀。图 8-17 的前四轮输入首先共享 206 个 token，后三轮又沿同一条路径延伸。树上的边按新增 token 数标记，叶子给出完整输入长度。

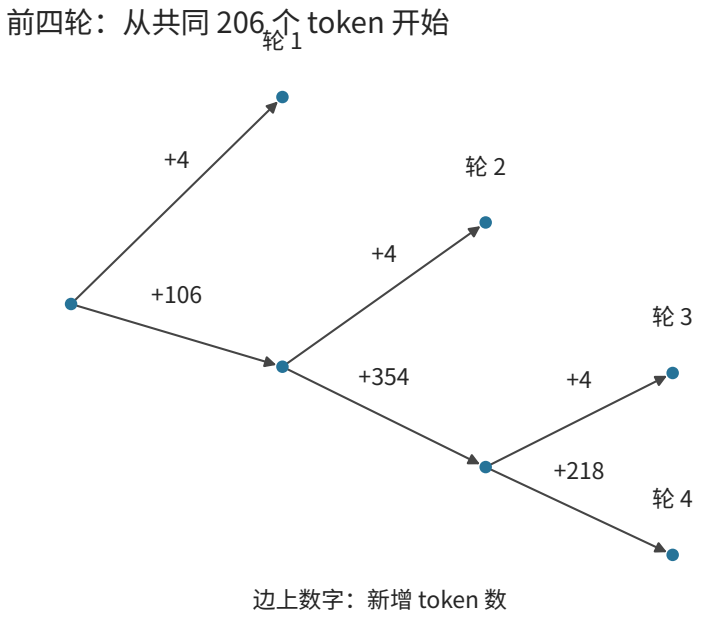



图 8-17：前四轮代码 Agent 输入的压缩前缀树。根部已有 206 个共同 token，边上是新增数量，叶子是输入轮次。分叉表示后续内容不同。

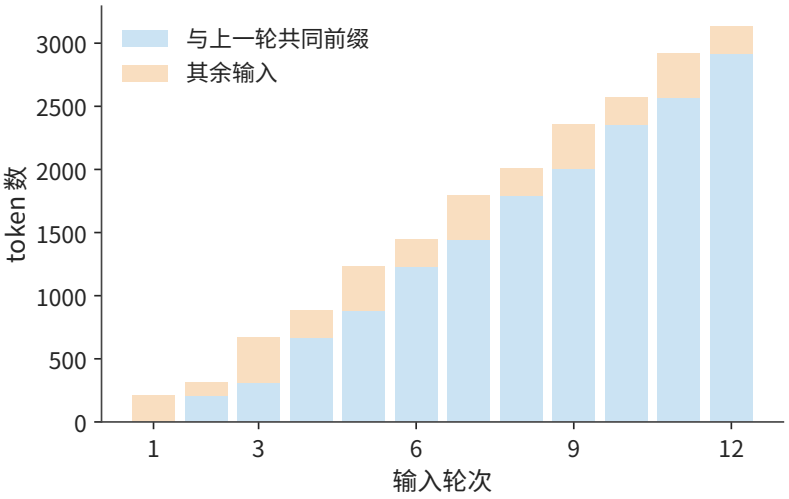



图 8-18：蓝色为与上一轮逐 token 相同的前缀，橙色为其余输入。此图描述输入内容的可复用程度，实际缓存命中还取决于状态是否保留。

每轮在末尾追加内容，可以保留已有的公共前缀，但上下文越长，KV 占用和后续读取量也越大。

**扩展：混合注意力模型如何确定可恢复的前缀位置。**全注意力保留逐 token KV，递推模型通常只保留当前状态。设文本匹配到第 10,752 个 token，递推快照分别保存在第 4096 和第 8192 个 token；系统从 8192 恢复，

[15] 上下文工程案例。

再计算到 10,752，重算 2560 个 token。增加快照可以减少重算的 token 数，但要保存更多状态。以第 2 章的 KDA 为例，一种实现将层内张量分到八张卡（TP=8），此时每卡每份快照约 53.6 MiB，保存 2 份约 107 MiB，保存 32 份约 1714 MiB。快照保存得越密，恢复时需要重算的 token 就越少，保留的状态也越多。[16]

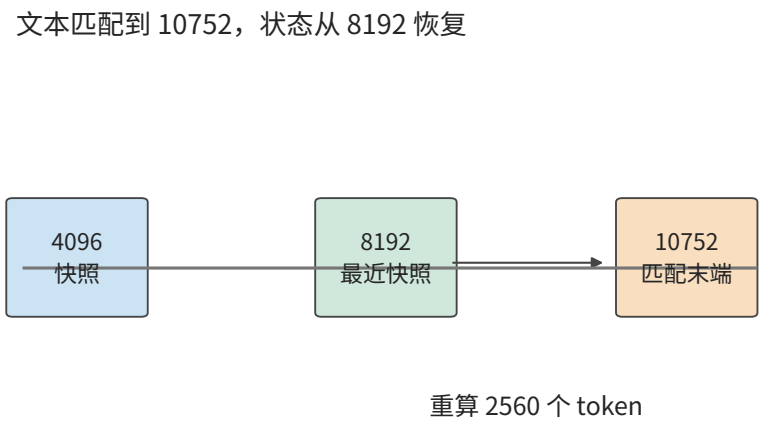



图 8-19：文本匹配到 10752，最近状态快照在 8192。恢复后仍需重算 2560 个 token，才能得到匹配末端的递推状态。图中 10752 和 8192 是从序列起点累计的 token 数。

**上下文编排还可以主动改变复用机会。**假设已有 8K token 的上下文，下一轮追加 1K 工具结果，最多可复用原来的 8K 前缀，只需处理新增部分；如果改写最前面的指令，后面的内容即使文字相同也不能继续复用。另一种方案是把历史总结为 2K，再追加 1K。摘要替换原始历史后，后续计算使用更短的上下文，但多出一次总结与重新处理的开销。因此，组织上下文时，需要同时考虑内存占用、重算时间和后续读取量。

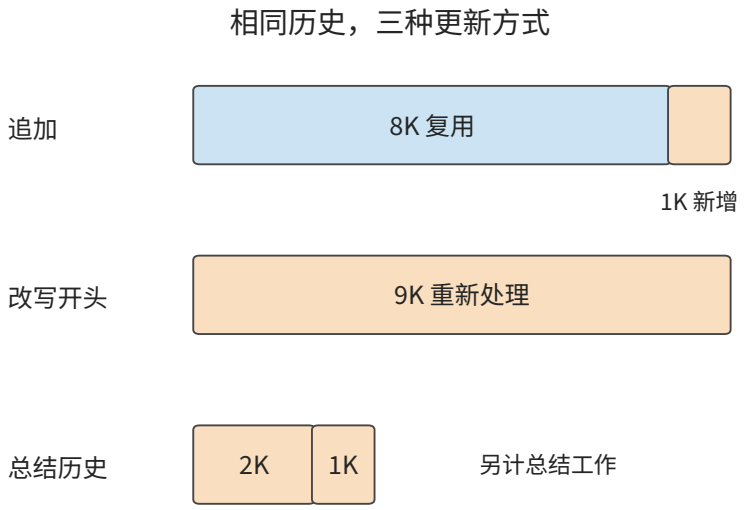



图 8-20：同一历史的三种更新方式。蓝色表示可复用前缀，橙色表示需要重新处理的输入；总结方案先生成摘要，再重建缓存。长度以 K token 示意。

模型侧的前缀复用还涉及更细的状态区分。DeepSeek V4.1 在复用前缀时，对两类局部状态分别处理。第 2 章介绍的 CED 结构包含因果编码器和解码器，两者各有 SWA 的局部状态：**编码器 SWA** 可以在主机 DRAM

[16]混合状态恢复与 checkpoint 计算、MLSys 2025 调研。Kimi K3 的容量按指定存储格式和并行配置计算。

池中短期保留，供下一轮沿已有前缀继续处理新输入；**解码器 SWA** 只为本轮后续生成使用，论文中的部署不把它作为前缀缓存保存。全局 KV 则进入可较长期复用的缓存。因此，生成期间驻留的 40 层 SWA 与跨轮次保留的前缀缓存，需要分别计算容量。[17]

这两类状态是否命中，决定了下一轮续算时编码器要恢复多少前缀。设工具返回后，恢复所需的 token 或输入表示仍可取得，模型版本、token 位置索引与输入保持一致。全局 KV 和编码器 SWA 都命中时，编码器直接处理追加输入；仅全局 KV 命中时，编码器重放已缓存前缀的最近最多 128 个 token，并与未缓存后缀一起执行。重放段重建编码器 SWA，继续使用既有全局 KV；新增后缀产生新的全局 KV 和 SWA。全局 KV 也未命中时，则重算缺失前缀。

随后，每次 prefill 都有共同的一步：取完整提示末尾最多 128 个 token 的编码器输出，经过 20 层解码器，近似构建解码器 SWA，为第一步 decode 准备局部状态。因此，编码器缓存全部命中时，可以省去编码器的前缀恢复，但解码器仍需执行窗口重放。

短窗口重放得到的是近似恢复的状态，误差来自更早的局部依赖被截断。每层虽然只访问最近 128 个 token，多层叠加后，局部状态仍可通过前一层间接依赖更早的输入。从窗口边界重新执行，会改变这部分历史信息。技术报告第 3.2.2 节因此将编码器重建的状态定义为近似状态；这一状态又参与后续计算，使新增后缀的全局 KV 和 SWA 随恢复起点变化。

两类状态的区分最终体现在缓存容量上。在 DeepSeek V4 技术报告采用的工作负载与缓存策略下，SWA 约占长期缓存的一半。V4.1 将这部分移出持久化缓存，再将全局 KV 压到约四分之一，长期存储量因此约为原来的 $1/2 \times 1/4 = 1/8$。短期保留的编码器 SWA 放在 DRAM 中，解码器 SWA 则在本轮生成时驻留加速器。缓存按使用时段分层保存，进一步降低了长期容量需求。

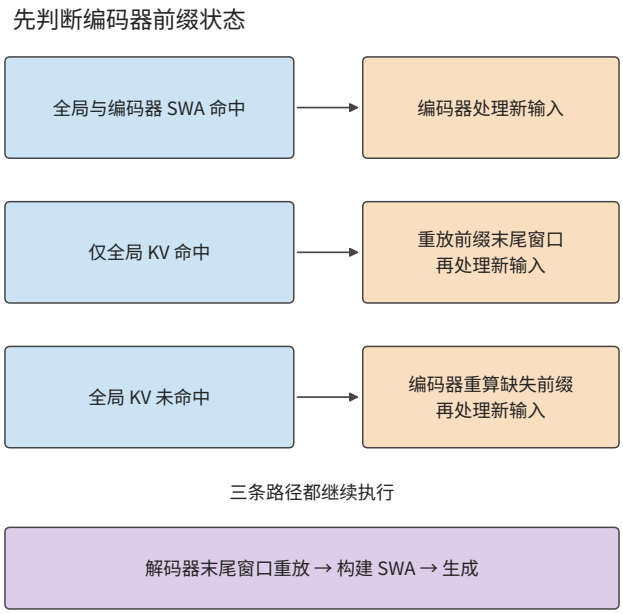



图 8-21：编码器的三条前缀恢复路径与共同的解码器重放。缓存命中状态中的 SWA 专指编码器；所有路径在 prefill 中仍构建解码器 SWA，再开始生成。箭头表示执行先后，框大小不代表耗时。

[17] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

### 8.3.4 缓存准入、淘汰、换出与重计算

请求结束后保留前缀，是为了将来再次使用时省去重算。设前缀再次用到的概率为 $p_h$，重算时间为 $T_r$，从缓存取回并准备好状态的时间为 $T_f$，维护缓存所需时间为 $T_m$。每次命中能节省 $T_r - T_f$；乘以命中概率，再减去维护时间，就得到期望净节省：

$$V_h = p_h(T_r - T_f) - T_m.$$

以章首长请求的 6144 个 token 公共前缀为例，它的 KV 占 864 MiB（0.906 GB）。在 RTX PRO 6000 上重算这段前缀要做 96.5 TFLOP 的矩阵运算，按实验 8-1 中 batch 1 prefill 达到的 BF16 峰值的 62%（第 8.6.3 节）计，约需 307.9 ms。把这段前缀换出到主机内存后，经这张卡的 PCIe Gen5 x16 链路（每方向 64 GB/s）取回约需 14.2 ms；[18]维护开销取换出时写回主机的一次传输，也是 14.2 ms。命中概率为 50% 时平均节省 132.7 ms。要让保留前缀节省时间，需满足

$$p_h > \frac{T_m}{T_r - T_f} = \frac{14.2}{293.7} = 4.8\%.$$

若主机链路换成 PCIe Gen4 x16（每方向 32 GB/s），取回和换出各增至 28.3 ms，每次命中只省 279.6 ms，所需的最低命中概率升至 10.1%。换出释放了加速器空间，但只有前缀足够常用，才能抵消取回和维护所需的时间。

前缀值得保留，并不意味着应该优先保留。空间紧张时，大前缀会占用原本能保存多个小前缀的空间。设 A 为上述 6144 个 token 前缀；每个 B 类前缀含 2048 个 token，占 288 MiB，同样按 PCIe Gen5 取回与换出，重算 94.7 ms、取回和维护各 4.7 ms，命中概率同为 50%，期望节省 40.3 ms。图 8-22 将两者放进同样大小的缓存：保存一个 A 能节省 132.7 ms，保存三个 B 合计节省 120.9 ms。重算时间随前缀长度超线性增长，6144 个 token 的 prefill 是 2048 个 token 的 3.25 倍，因此同一块内存用于保存长前缀的收益更高。

这种比较也可以换算成单位容量的收益：A 每 MiB 约节省 0.154 ms，B 约节省 0.140 ms。按该数值排序，可以比较大小不同的前缀。

上述选择有一个前提：前缀必须保留到下一次使用时。缓存容量不足时，即使下一轮输入保留了相同前缀，也可能因为状态已经淘汰而要重新计算。一次 12 轮 Agent 输入回放包含 19,556 个输入 token。在相同干扰请求下，6 GiB 缓存池命中 16,304 个 token，只需重新处理 3252 个；1 GiB 池则没有命中，需要重新处理全部输入。将相邻两轮之间的等待时间增加 0.2 秒后，命中数量没有改变。这组对照中，增加容量保留了可复用的前缀，单纯延后下一轮请求没有产生同样的效果。[19]

**缓存准入**决定请求结束后是否保留其前缀；淘汰策略决定空间不足时先删除哪个前缀；换出策略决定将状态移到哪一级存储；重计算策略决定重新执行多少输入 token。对于章首的 16 条长请求，保留一份公共前缀只需 864 MiB，却能同时节省重复存储和后续请求的 prefill 计算。

> **练习 8-4 ・核心・分析：有限缓存应保留哪些前缀，才能节省最多时间？**缓存可用空间为 864 MiB。前缀 A 含 6144 个 token、占 864 MiB；三个相互独立的 B 类前缀各含 2048 个 token、占 288 MiB。在 RTX PRO 6000 上，按 BF16 峰值 503.8 TFLOP/s 的 62% 由矩阵运算量求重算时间，按 PCIe Gen5 x16 每方向 64

[18]RTX PRO 6000 Blackwell 工作站版规格：系统接口 PCIe 5.0 x16，带宽为 PCIe Gen4 的两倍；A100 80GB 规格：PCIe 4.0 为 64 GB/s（收发合计），即 x16 每方向 32 GB/s。由此 PCIe Gen5 x16 每方向 64 GB/s。正文按标称值计算，得到的是传输时间的下界。前缀的重算时间按实验 8-1 的效率表计算。

[19]12 轮实际输入的缓存回放。回放采用原任务的输入序列，串行执行请求，每轮生成一个 token。

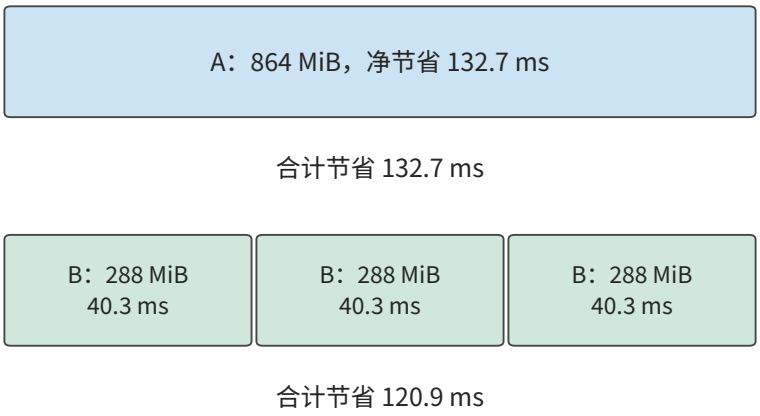



图 8-22：缓存容量均为 864 MiB。A 占 864 MiB，期望净节省 132.7 ms；三个 B 类前缀各占 288 MiB、各节省 40.3 ms，合计 120.9 ms。图中宽度表示容量；时间按 RTX PRO 6000 上的重算时间和 PCIe Gen5 取回时间计算，命中概率均为 50%。

> GB/s 求取回时间，维护时间取一次换出，与取回相同；命中概率均为 50%。比较保留一个 A 前缀与保留三个 B 类前缀的期望净节省时间。保持 B 类前缀的命中概率不变，求 A 的命中概率低于多少时，应改为保留三个 B。再把链路换成 PCIe Gen4 x16（每方向 32 GB/s），重新比较。数据分析部分读取配套 12 轮输入与缓存回放，分别计算共同前缀比例和实际命中比例，解释其他请求占用缓存空间为何会影响实际命中比例。

## 8.4 压缩与卸载

### 8.4.1 权重量化后的内存占用

前缀共享通过去除重复的 KV 节省空间。即使请求之间没有相同前缀，也可以用量化减少权重和 KV 的存储量。量化通常将数值分组编码，每组保存较低位宽的编码值，以及 scale 等元数据；计算时再根据这些信息恢复近似数值。

**Q2_K** 是张量计算库 GGML 系列实现中的一种分组量化格式，主编码使用 2 bit，并为组内的 scale 保存额外信息。以该格式为例，每组有 256 个值，2-bit 编码占 $256 \times 2/8 = 64$ bytes，scale 和最小值等元数据另占 20 bytes。因此，一组共需 84 bytes，平均每值为 $84 \times 8/256 = 2.625$ bit。位宽降低后，元数据在总容量中的比重相应提高，在该格式中约占 24%。[20]

**扩展算例：量化权重加载到内存后，还能容纳多长的上下文？** Qwen3-235B-A22B 的一个 Q2_K 文件变体采用以 2 比特为主的分组量化，并混合使用多个位宽，量化编码与未量化的浮点数据约 64.43 GiB、量化元数据约 15.37 GiB，加上文件头与填充，共约 79.81 GiB。另一个 `UD-Q2_K_XL` 变体约 81.97 GiB。以实验 8-7 所用的 Apple M2 Max 为例，它的统一内存标称 96 GB（十进制），给系统与工作区预留 8 GiB，剩余约 81.41 GiB。[21]

将前一个文件加载到内存后，还剩 1.60 GiB；后一个文件则已经超出可用空间约 0.56 GiB。该模型一条 8K 上下文的 BF16 KV 约占 1.47 GiB，因此前一种方案还能处理一条请求，并剩余约 0.13 GiB。若上下文长度增至 32K，KV 就需要约 5.88 GiB，必须增加内存或改变权重的存放方式。两个文件的大小只相差 2.16 GiB，已

[20] Q2_K 的逐张量布局、8K 文件容量与32K 对照。文件总量 85,691,002,112 bytes，码值 / 浮点载荷 69,178,275,840 bytes，量化元数据 16,506,720,256 bytes，文件头与填充 6,006,016 bytes。上述文件大小按已归档文件头与发布元数据计算。

[21] 固定模型文件与业务预算及235B 分片元数据。精确字节、GB 与 GiB 分别报告。

经足以决定能否容纳请求。

在统一内存系统中，CPU 和 GPU 共用物理内存。将权重文件映射到地址空间后，访问相应数据时，文件页面才需要驻留在物理内存中。如果执行时要访问的数据总量超过内存容量，就会不断换入页面。因此，要解决容量不足的问题，可以减少数据本身的字节数，也可以只保留当前需要的数据，其余数据在使用前搬入。下面先计算 KV 压缩的收益，再分析权重搬入所需的时间。

### 8.4.2 KV 压缩与读取成本

把同样的分组方法用于 KV，可以计算节省的空间能多容纳多少请求。权重由许多请求共享，KV 却持续随输入与输出增长。对本章模型，一条 8K 上下文的 BF16 KV 为 1152 MiB。量化改变每个值的位宽，上下文表示则改变每个 token 要保存多少个值：第 9.2.2 节把同一 8K 上下文的 GQA 与紧凑 MLA 状态并列，后者只有 549 MiB。每 32 个 BF16 值占 64 bytes。**q8_0** 是每 32 个值共用一个 scale 的 8 bit 量化格式。改成 q8_0 时保存 32 bytes 码值和 2 bytes scale，变为原来的 34/64；**q4_0** 采用同样的分组方式、将码值降至 4 bit，保存 16 bytes 码值和 2 bytes scale，变为 18/64。

因此，同一条上下文所需容量为

$$M_{q8} = 1152 \times \frac{34}{64} = 612 \text{ MiB}, \qquad M_{q4} = 1152 \times \frac{18}{64} = 324 \text{ MiB}.$$

与 BF16 相比，q8_0 节省 540 MiB，q4_0 节省 828 MiB。计入 scale 后，两种格式平均每值分别占 8.5 bit 和 4.5 bit。由于每组都要保存 scale，元数据也会随 KV 长度一起增长。[22]

图 8-23 将这一步还原为字节布局。每行都保存相同的 32 个值：编码部分缩短了，scale 仍要留下。因此，q8_0 和 q4_0 的总长度分别是 34 和 18 bytes，而不是只看编码位宽得到的 32 和 16 bytes。

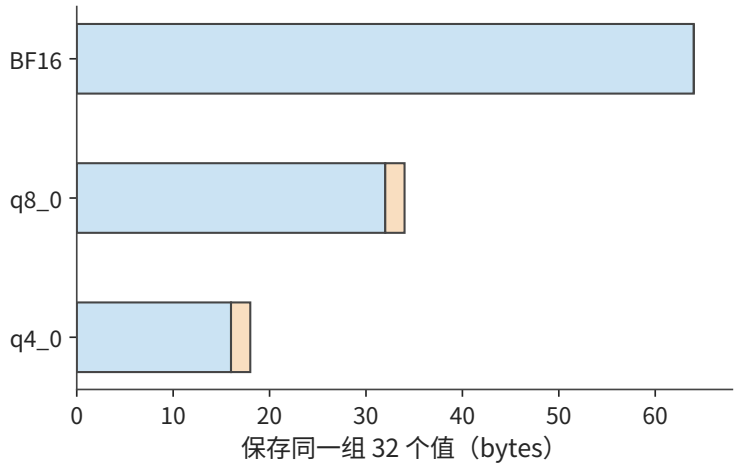



图 8-23：BF16、q8_0、q4_0 保存同一组 32 个数值所需的空间。横条按字节数成比例绘制；橙色为每组 2 bytes 的 scale。把每组总长度乘以组数，就得到整条上下文的 KV 容量。

章首算例中的长请求生成完毕后，BF16 KV 按块分配共占 1188 MiB。改用同一布局的 q8_0 后，需要 $1188 \times 34/64 = 631.125$ MiB，16 条独立长请求合计约 9.86 GiB，可以放入预留的 12 GiB 内存。由此可见，共享和压缩节省空间的方式不同：共享去除重复前缀，压缩减少每个数值所占的字节。

[22]KV 格式和执行成本。资料包含 GGML 分组格式、FP8 scale 粒度及 H100 上的拟合结果。

用压缩 KV 做注意力计算时，读取量减少了，但格式转换需要额外时间。设一次计算少读 $\Delta D$ bytes，有效带宽为 $\beta$，转换增加的串行执行时间为 $T_c$，则净节省时间为

$$\Delta T = \frac{\Delta D}{\beta} - T_c.$$

例如，在 RTX PRO 6000 上将一条 8K 上下文从 BF16 改为 q8_0，少读 540 MiB。按读取受限时的有效带宽约 1.16 TB/s 计（第 8.6.3 节，峰值 1792 GB/s 的 65%），约节省 0.487 ms。若转换增加 0.2 ms，最终节省约 0.287 ms；若转换增加 0.8 ms，反而增加约 0.313 ms。同样的压缩格式是否能加快执行，取决于转换时间是否少于节省的读取时间。

上下文越长，减少读取所节省的时间越多。一项早期评估在 H100 上运行 Llama-3.1-8B，分别用 $6.44 + 4.37 \times 10^{-5} L$ 和 $6.58 + 2.37 \times 10^{-5} L$ ms 拟合 BF16 与 FP8 的输出间隔。FP8 固定项多 0.14 ms，每个上下文 token 的增长项少 $2 \times 10^{-5}$ ms。用固定开销之差除以每增加一个上下文 token 的耗时之差，得到交点约为 7000 个 token；在 2K 时 BF16 约 6.53 ms、FP8 约 6.63 ms，在 16K 时则分别约 7.16 和 6.97 ms。上下文较长时，读取节省的时间超过了新增的固定开销。[23]

本节开头提到，上下文表示决定每个 token 要保存多少个值；混合注意力模型的局部与全局两类状态，也可以按这种方式比较容量。在 8K 上下文基线下，把全局历史与有效 SWA 的容量相加。DeepSeek V4.1 的两项分别为 6.953 MiB 与 2.578 MiB，共 9.531 MiB；V4 的对应两项合计为 30.521 MiB，约为 V4.1 的 3.20 倍。局部窗口占固定空间，因此短上下文下的合计容量差距小于全局历史的 3.95 倍。

### 8.4.3 权重卸载、预取与每步搬移

压缩减少了数据本身的大小。另一种扩展容量的方法是让数据轮流驻留在 GPU 上：只在计算需要时搬入，用完后复用这块空间。权重卸载将部分原本常驻 GPU 的权重移到主存，并在计算相应层之前搬回 GPU。节省的加速器内存可以用于 KV，但每次前向计算都需要重新搬入这些权重。请求逐步生成输出时，搬移也随每轮 decode 重复。

Qwen3-8B 一层 FFN 的三个 BF16 矩阵共占 288 MiB。36 层中每四层卸载一层，共卸载九份 FFN 权重，释放 2592 MiB。GPU 上还要为预取（在计算某层之前，提前把它的权重复制到 GPU）预留缓冲区：一组缓冲占 288 MiB，净省 2304 MiB；两组占 576 MiB，净省 2016 MiB。以一条 8K 输入的 1152 MiB KV 为单位，前者能多容纳两条，后者只能多容纳一条。若按生成完毕时所需的 1188 MiB 预留空间，两种方案均只能增加一条完整请求。[24]

预取必须遵守两个先后关系：权重复制完成后，计算才能读取；计算不再使用这组权重后，缓冲区才能写入下一组。使用一组缓冲时，同一缓冲区依次用于复制、计算和下一次复制；使用两组缓冲时，可以一边计算当前层，一边把后续层的权重复制到另一组缓冲区。这样能够重叠复制与计算，但缓冲区本身也会占用更多内存。

先计算搬移的链路成本。这张卡经 PCIe Gen5 x16 连接主机，每方向标称 64 GB/s。每轮要搬入 2592 MiB，即 2.72 GB，需要

$$T_{\text{copy}} \geq \frac{2.72 \text{ GB}}{64 \text{ GB/s}} \approx 42.5 \text{ ms}.$$

[23] KV 格式和执行成本。资料包含 GGML 分组格式、FP8 scale 粒度及 H100 上的拟合结果。

[24] 权重卸载与预取计算；PCIe Gen5与NVLink-C2C两档链路的复制与预取调度，每层计算时间取 batch 1 一轮 decode 的 26.26 ms 除以 36 层；GH200 架构说明：NVLink-C2C 合计 900 GB/s，每方向 450 GB/s。

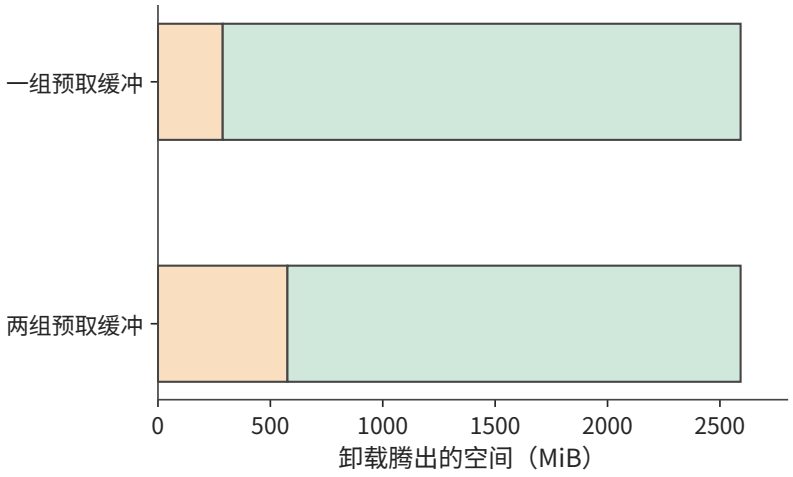



图 8-24：卸载九份 FFN 共腾出 2592 MiB。橙色为留在加速器上的预取缓冲，绿色为可重新分配的净空间；一组缓冲净省 2304 MiB，两组净省 2016 MiB。

这已经超过 batch 1 一轮 decode 的 26.26 ms。即使其余计算都能与复制同时完成，每轮仍至少需要约 42.5 ms。若每轮为四条请求各生成一个 token，总输出吞吐最多约为 $4/0.0425 \approx 94$ token/s，每条请求的输出间隔也至少约为 42.5 ms。生成 256 个输出 token 需要 255 次后续前向计算，仅权重搬移就要约 10.8 秒。因此，经 PCIe 卸载这部分权重，无法满足 7 秒的完成时限。

CPU 与 GPU 之间也有快得多的链路。GH200 用 NVLink-C2C 连接 Grace CPU 与 Hopper GPU，每方向 450 GB/s。同样的 2.72 GB 每轮至少约需 6.0 ms，255 轮合计约 1.54 秒。每轮复制短于一轮 decode 的计算，预取就能用计算掩盖复制。

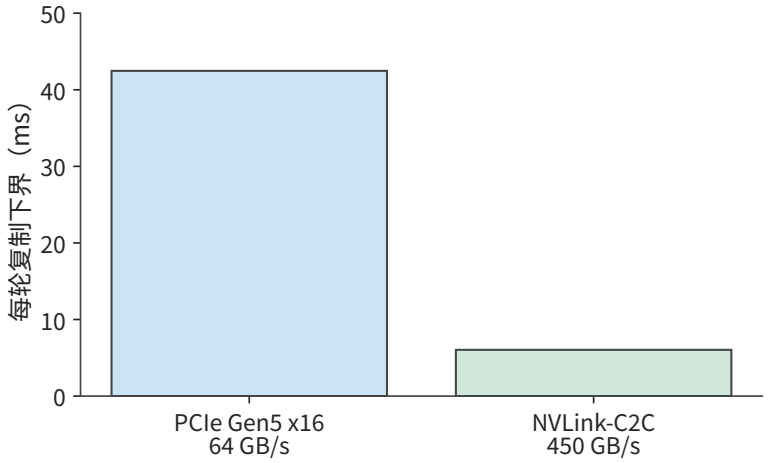



图 8-25：每轮复制量保持为 2592 MiB（2.72 GB）。经 PCIe Gen5 x16（每方向 64 GB/s）至少需要约 42.5 ms，经 GH200 的 NVLink-C2C（每方向 450 GB/s）至少需要约 6.0 ms。

这里起决定作用的是每轮搬移的数据量与有效带宽之比。预取可以让搬移与计算同时进行，但每轮需要传输的字节数仍然相同。

若先压缩权重再搬移，还要加上解压时间。设原始数据为 $S$ 字节，压缩后为 $rS$ 字节，链路带宽为 $B_{\text{link}}$，解压吞吐为 $R_{\text{dec}}$（按输出的原始字节计）。先传完再解压需要 $rS/B_{\text{link}} + S/R_{\text{dec}}$，直接传输需要 $S/B_{\text{link}}$，因此只有当

$$R_{\text{dec}} > \frac{B_{\text{link}}}{1-r}$$

时，压缩后搬移才更快。把 96 MiB 权重压缩到四分之一，经 PCIe Gen5 x16 直接传输约需 1.57 ms，传输压缩数据只需 0.39 ms；要保留这 1.18 ms 的节省，解压器每秒必须输出超过 85.3 GB 的原始数据。链路越快，对解压吞吐的要求就越高。[25]

这一节的卸载中，模型计算始终由 GPU 执行，主存负责保存暂不使用的权重。第 9.3 节将进一步改变计算位置：专家权重已在主存时，可由 CPU 直接计算，只向 GPU 传回较小的激活，届时需要比较每个专家的输入行数、CPU 计算时间与权重传输时间。

**练习 8-5 ·计算：权重卸载节省的容量与增加的搬移。**设卸载九份 FFN 权重，每份占 288 MiB。设每组预取缓冲可以容纳一份 FFN 权重，分别计算使用一组和两组缓冲时净节省的加速器内存，并求这些空间能多容纳多少份 1152 MiB 的输入状态，或多少份 1188 MiB 的完整请求状态。再分别取 PCIe Gen5 x16 的每方向 64 GB/s 与 GH200 NVLink-C2C 的每方向 450 GB/s，求每步复制以及累计 255 步复制的时间下界。若留给所有复制的总时间为 1 秒，求所需的最低单向带宽。

### 8.4.4 质量约束下的容量与速度选择

第 8.4.2 节算出了压缩节省的空间和增加的转换时间，但尚未回答改变数值格式后答案是否相同。这里的执行后端指实际完成注意力计算的 kernel 实现。压缩 KV 改变保存的数值，更换执行后端还可能同时改变 Q 的计算格式、转换过程和归约运算的数值精度。逐项比较张量的存储格式与计算精度，能够找出输出差异的来源。

下面用八道固定任务做这种逐项比较，三幅图按相同顺序排列这些任务。八道任务来自两种长度的文档：每份文档含 128 或 512 条六位数字记录，两种长度各有四份独立文档，实验记录里分别标为 n128-r0 至 n128-r3 和 n512-r0 至 n512-r3，图中依次记为任务 1 至 8。每道任务在两种并发数下各运行两次，用相同任务逐一比较不同数值格式的影响。

一组 Qwen3-8B 实验保持 BF16 权重不变，比较了三种注意力计算方式。32 次自然生成中，使用 BF16 KV 时答对 28 次；改用原 FP8 实现后答对 26 次；保留 FP8 KV、将 Q 恢复为 BF16 后，又答对 28 次。最后一种配置只改变 Q 的精度，结果就发生了变化，说明需要分别考察 KV 的存储格式和 Q 的计算精度。[26]

图 8-28 进一步只恢复查询 Q 的 BF16 精度，KV 仍保持 FP8。按同一列比较三幅图，就能看出改变的是哪些题目的结果，而不只是总正确数。

图中任务 5（实验记录里的 n512-r0，即含 512 条记录的第一份长文档）在 BF16 KV 下四次均错，在 “FP8 KV + BF16 Q” 下四次均对；任务 6（n512-r1）则恰好相反。两种配置都答对 28 次，但答错的任务不同。将同一任务的结果排在一起，就能看出差异发生在哪里，以及重复运行时是否一直如此。

错误还会增加完成任务的时间。第一次回答失败后重试，得到正确答案的总时间应包括首次生成、检查答案和再次生成。已有实验中，两次重试修正了错误，但额外增加了约 2.0 秒生成时间和 92 个 token。如果第一次回答虽然更快，却也更容易出错，完成整个任务反而可能更慢。[27]

比较前述两种 KV 容量方案：16 条独立长请求采用 q8_0 后约占 9.86 GiB，共享 BF16 前缀后约占 5.91 GiB。要判断哪种方案更合适，还需将转换、生成和重试的时间计入请求的总耗时。第 8.6 节的综合例题将完成

[25] 张量压缩与传输。资料包含 LLM.265 的视频引擎实现、专用硬件设计与性能估计。

[26] KV 与 Q 精度对照、固定 FP8 权重的 Q 控制、并发容量记录。正文采用 BF16 权重组。另一组固定 FP8 权重时，BF16 Q 与默认路径分别为 32/32、28/32。

[27] 自然输出与重试成本。实验使用已知答案校验，重试成本包含错误首答的生成成本。

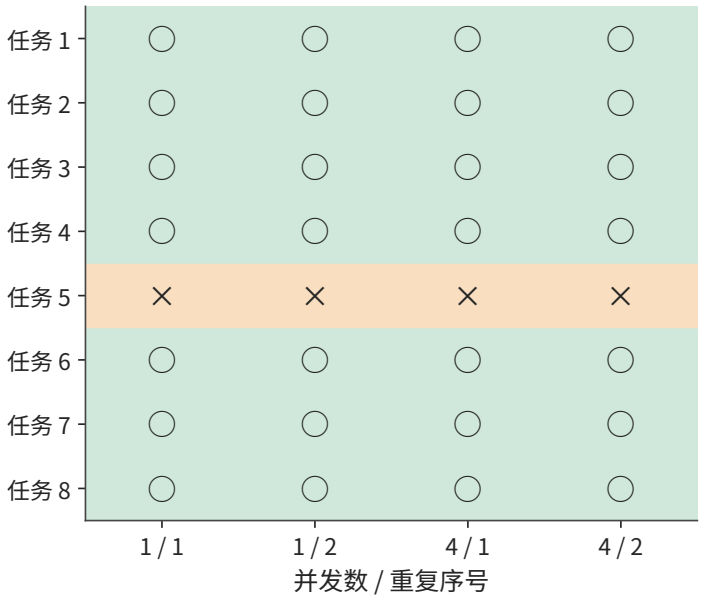



图 8-26：BF16 KV 基线。每行是一道固定任务，四列为并发 1、4 下各两次自然生成。绿色圆圈正确，橙色叉号错误，共 28/32 正确。

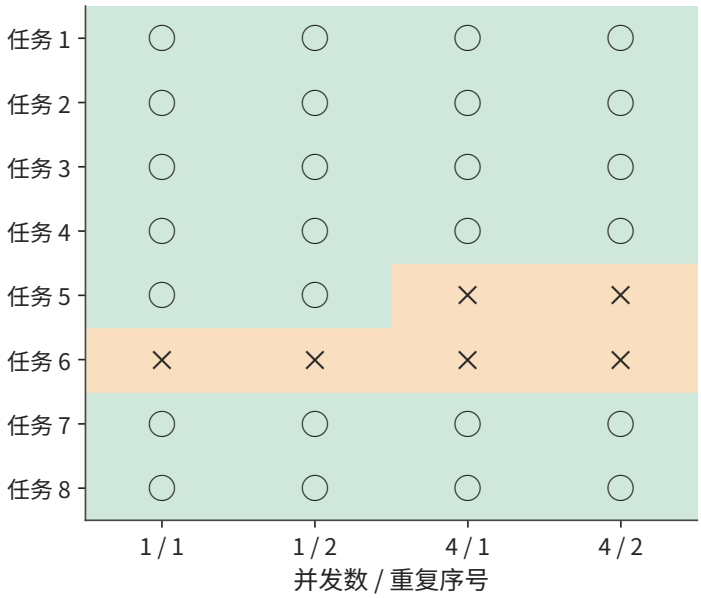



图 8-27：同一任务与运行顺序，原 FP8 实现共 26/32 正确。这一比较包含实现选择对 Q 精度的影响。圆圈表示回答正确，叉号表示回答错误；每行对应同一道题，列表示并发数与重复运行序号。

这一步。

> **练习 8-6 ・数据分析：KV 压缩如何影响容量、答案正确性与重试成本。**设每组包含 32 个值，另用 2 字节保存 scale，复算 8K 输入和完整长请求在 q8_0、q4_0 格式下的 KV 容量。再读取配套三种 KV / Q 精度配置的逐题结果，列出 BF16 KV 与 “FP8 KV + BF16 Q” 中正确性改变的题目。最后读取重试记录，对每个成功任务合计首答与重试时间，并与只计入最后一次成功生成时间的结果比较。

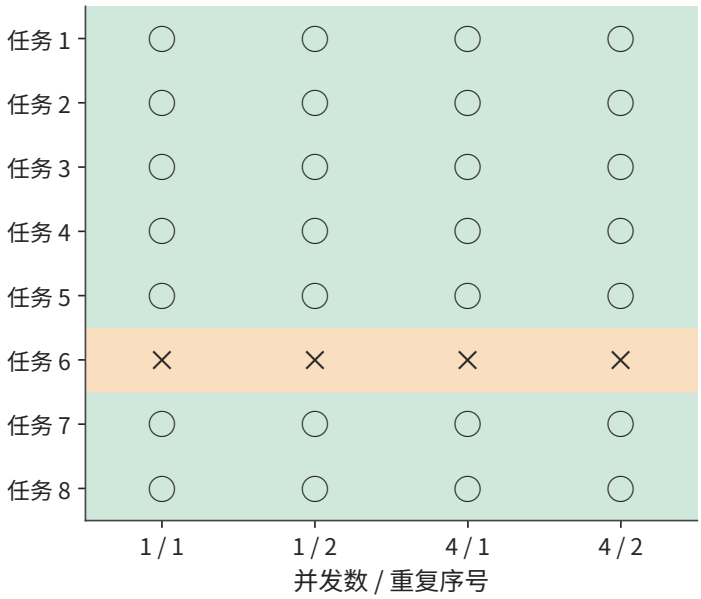



图 8-28：将查询 Q 保持为 BF16，KV 仍用 FP8，共 28/32 正确；答错的题目与 BF16 基线不同。三幅图使用相同任务和列顺序，模型权重均为 BF16。圆圈表示回答正确，叉号表示回答错误；每行对应同一道题，列表示并发数与重复运行序号。

## 8.5 推测解码

### 8.5.1 草稿、验证、回退与正确性

前两节主要改变数据的存放和读取方式，每条请求仍要逐轮生成。若能让一次目标模型计算确定多个输出，生成 256 个 token 就不必再执行同样多的轮次。普通 decode 中，目标模型每次前向计算生成一个 token。第 8.2 节将不同请求合并计算，使一次权重读取用于更多行。推测解码采用另一种办法：先为同一请求生成一段**草稿序列**，再由目标模型同时验证其中的多个 token。若目标模型接受了多个草稿 token，一次计算就能确定多个输出。

先考虑**贪心生成**：每一步都选择目标模型给出的概率最高的 token。假设草稿序列包含四个 token，目标模型分别计算各 token 位置应当输出哪个 token。前两个与草稿相同，第三个不同，那么最终保留前两个草稿 token，并在第三处输出目标模型选出的 token。第四个草稿 token 是在错误的第三个 token 之后生成的，也随之丢弃。下一轮从修正后的序列继续生成。若四个草稿 token 全部匹配，目标模型通常还能在它们之后再生成一个 token。

图 8-29 展示了第三个 token 发生分歧时的处理顺序。目标模型虽然已经计算了后续 token 的验证结果，但第四个 token 依赖错误的第三个草稿 token，不能继续采用。一次验证最终留下两个匹配的草稿 token 和一个修正 token，共三个输出。

随机采样时，需要使最终输出仍服从目标模型的概率分布。设目标分布为 $p(x)$，草稿分布为 $q(x)$。先从 $q$ 中采样得到草稿 token $x$，再以如下概率接受它：

$$\alpha(x) = \min\left(1, \frac{p(x)}{q(x)}\right).$$

采样到 $x$ 并直接接受它的概率为 $q(x)\alpha(x) = \min(p(x), q(x))$。要达到目标概率 $p(x)$，还差 $[p(x) - q(x)]_+$。

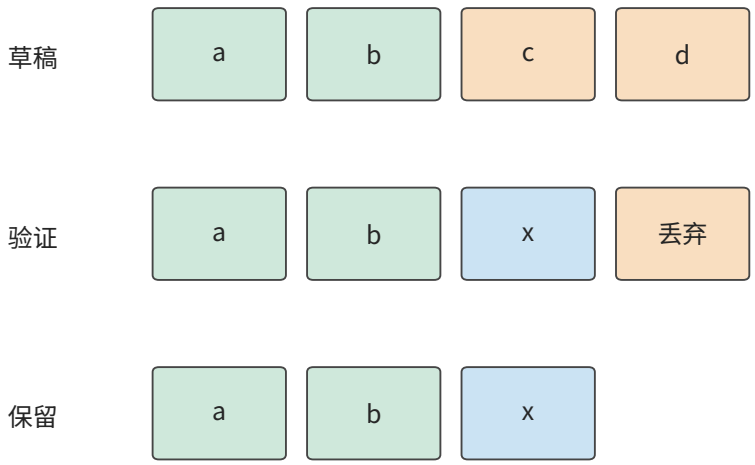



图 8-29：贪心验证的过程。a、b、c、d、x 表示 token；目标模型在第三个 token 选出 x，与草稿 c 不同。第四个 token 及其后的验证结果作废，下一轮从 a、b、x 继续生成。方框表示序列中的 token 位置，不表示执行耗时。

这里 $[z]_+ = \max(z, 0)$，表示只保留正的差额。因此，拒绝草稿 token 后，将这一正差额归一化，再从得到的分布中重新采样。直接接受和拒绝后重新采样两种情况合起来，最终输出 $x$ 的概率就是 $p(x)$。[28]

用只有 A、B 两个符号的例子说明这一过程。目标分布为 $p(A) = 1/4$、$p(B) = 3/4$。若草稿总是生成 A，就以 $1/4$ 的概率接受 A，其余 $3/4$ 的情况拒绝 A、改为输出 B。若草稿总是生成 B，则以 $3/4$ 的概率接受 B，其余情况改为输出 A。两种方法最终都得到目标分布，但草稿的接受概率不同，因而执行效率也不同。

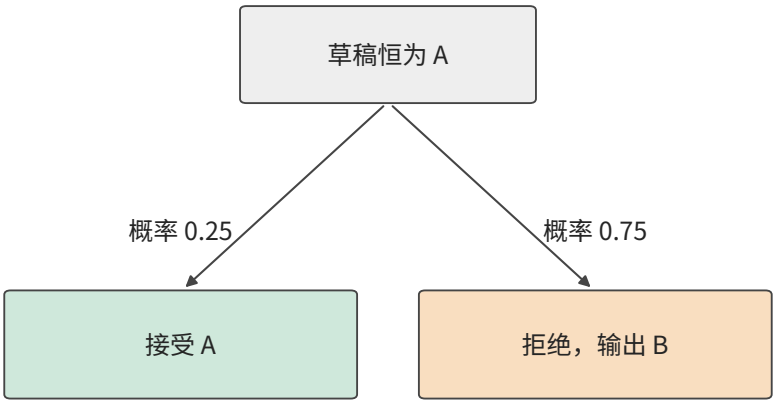



图 8-30：接受 A 的概率为 $1/4$，拒绝后输出 B 的概率为 $3/4$。两条路径合起来给出目标分布。

验证之后，KV 的有效长度调整到已经确定的输出位置，丢弃的草稿 token 对应的 KV 不再参与后续计算。分页器按已经确定的输出长度更新块引用，采样器和语法检查器（按指定输出格式限制可选 token 的组件）也

[28]推测采样原文、推测解码原文、框架计数与预算笔记、调研中的框架覆盖。基础概率算法、草稿路线和具体实现分别引用。

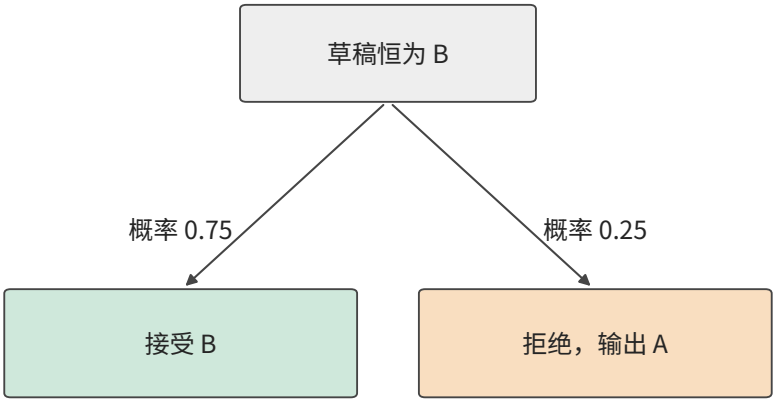



图 8-31：接受 B 的概率为 3/4，拒绝后输出 A 的概率为 1/4。目标分布相同，草稿的接受概率更高。

恢复到对应的状态。下一轮便从修正后的序列继续计算。

### 8.5.2 每轮耗时与输出数量

推测解码既改变每轮耗时，也改变每轮最终输出的 token 数。设第 $r$ 轮耗时为 $T_r$，输出 $N_r$ 个 token，那么连续执行多轮后，平均每个输出 token 的耗时为

$$\bar{t}_{\text{spec}} = \frac{\sum_r T_r}{\sum_r N_r}.$$

例如，两轮各耗时 1.5 ms，分别输出 1 个和 5 个 token，总计 3 ms、6 个 token，平均每个 token 耗时 0.5 ms。如果先算每轮的平均值，再将 1.5 和 0.3 ms/token 等权平均，就会得到 0.9 ms。这相当于让只输出一个 token 的轮次与输出五个 token 的轮次占相同权重。直接用总时间除以总 token 数，才能得到每个 token 的平均耗时。

**算例：草稿接受率如何决定每轮的平均输出数？** 沿用两符号目标分布，并假设各 token 位置独立。每轮草稿生成四个相同符号，最终输出还包括拒绝时的修正 token，或全部接受后的额外 token。设前面的草稿 token 都已接受时，当前位置的接受概率为 $a$。每轮至少输出一个 token；要输出第二个，需接受第一个草稿 token，概率为 $a$；要输出第三个，需接受前两个，概率为 $a^2$。依次相加，平均每轮输出数为

$$\mathbb{E}[N] = 1 + a + a^2 + a^3 + a^4.$$

上式每一项都表示“最终至少输出这么多个 token”的概率。草稿为 AAAA 时，$a = 1/4$，平均每轮输出约 1.33 个 token；为 BBBB 时，$a = 3/4$，平均输出约 3.05 个。在 RTX PRO 6000 上，batch 1、2K 上下文的普通 decode 一轮 26.26 ms（实验 8-1）。验证 5 个位置时，读取的权重和 KV 与普通 decode 相同；多出的 4 行矩阵运算约 65 GFLOP，按 BF16 峰值的 62% 计只需约 0.2 ms，因此一轮验证仍按 26.26 ms 计。实验 8-5 也观察到同样的现象：DFlash（一种前向一次就并行生成整段草稿的草稿网络）每轮生成 7 个草稿 token 并验证 8 个位置，中位耗时 15.2 ms，与同一配置下普通 decode 一步的 16.9 ms 相当。再加上查询草稿的 0.1 ms，一轮共 26.36 ms，两种草稿对应的平均每 token 耗时分别约为 19.8 和 8.64 ms，都快于 26.26 ms/token 的普通 decode。

验证几乎不增加一轮的时间，即使草稿大多被拒绝，每轮也至少得到一个 token，所以接受率低的 AAAA 同样能加速。[29]

接受更多草稿 token 所节省的时间，也可能被查询开销抵消。要优于普通 decode，一轮总耗时须小于平均输出数乘以 26.26 ms。扣除验证所需的 26.26 ms，AAAA 留给草稿查询的时间约为 8.7 ms，BBBB 约为 53.9 ms。图 8-32 把这两个交点画在查询成本轴上。

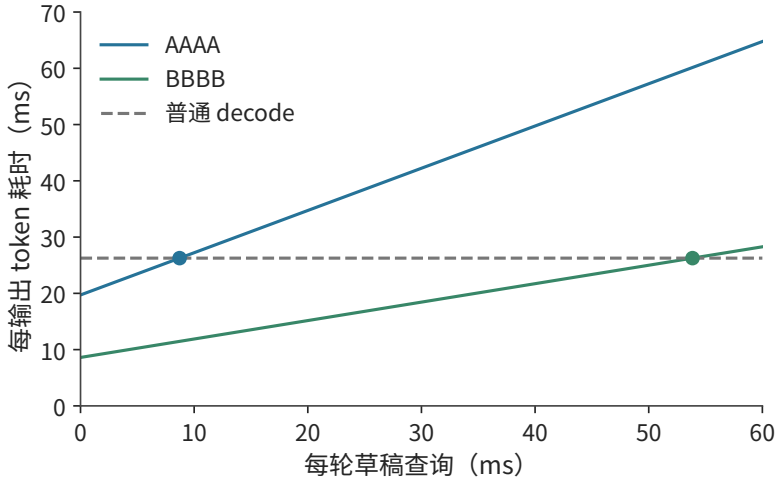



图 8-32：两符号目标分布，每轮生成四个草稿 token。每轮验证 26.26 ms，查询时间沿横轴变化；每个输出 token 的耗时用一轮耗时除以平均输出数。普通 decode 为 26.26 ms/token（RTX PRO 6000，batch 1，2K 上下文）。没有提前停止，额外 token 计入输出数。图中的点标出查询耗时为 0.1 ms 的算例；AAAA、BBBB 分别在查询约 8.7、53.9 ms 处与普通执行相交。

使用草稿前的准备工作也要计入总时间。若建立上下文索引需要 2 秒，而 BBBB 草稿每个输出 token 能节省约 $26.26 - 8.64 \approx 17.6$ ms，生成约 114 个 token 就能抵消这 2 秒开销。输出 256 个 token 的请求用普通 decode 约需 6.72 秒；先用 2 秒建立索引再用 BBBB 草稿，期望约 4.22 秒完成。如果索引可以提前建立并由多个请求共用，这项开销还能由更多输出分摊。

### 8.5.3 草稿来源与串行、并行生成方式

四个草稿 token 能节省多少时间，既取决于其中有多少通过了验证，也取决于生成这段草稿需要多长时间。实际系统可以从上下文中查找草稿，也可以调用额外的网络生成草稿；两种选择会带来不同的准备时间与内存占用。

上下文查找从输入或已有回答中找出重复片段，主要开销是匹配和建立索引。独立小模型逐步生成草稿，需要额外保存自己的权重和 KV。EAGLE 类方法将目标模型的中间表示送入较小的预测网络；DFlash 类方法则并行生成多个草稿 token。模型自带的 MTP 头则利用联合训练得到的预测能力生成草稿。[30]

将每轮分为生成草稿、准备验证、目标验证和确定输出四个阶段，就能逐项比较这些方法。以实验 8-5 的 DFlash 草稿网络为例，其权重约 1.95 GiB（2.10 GB）。按 batch 1 的有效带宽计（26.26 ms 读取 15.44 GB，约 588 GB/s），该网络前向一次约需 3.57 ms。若用它逐个生成四个草稿 token，要前向四次，共 14.28 ms；像 DFlash 那样并行生成整段草稿，只需前向一次，约 3.57 ms。目标验证都需 26.26 ms，若最终每轮都平均输出 3 个 token，串行方式每个 token 约需 $(14.28 + 26.26)/3 \approx 13.5$ ms，并行方式约需 $(3.57 + 26.26)/3 \approx 9.9$ ms。此时，加速来自草稿生成时间的缩短。

[29] 上下文草稿的分布与成本计算及有理数穷举结果。算例采用二符号目标分布；RTX PRO 6000 上的每轮时间见 AAAA、BBBB 与含 2 秒索引的 256 token 请求；普通 decode 一轮 26.26 ms 取自实验 8-1 的逐轮记录，DFlash 的每轮耗时取自实验 8-5的逐步记录。

[30] 推测采样原文、推测解码原文、框架计数与预算笔记、调研中的框架覆盖。基础概率算法、草稿路线和具体实现分别引用。

再设并行生成的草稿接受率较低，平均输出数降为 2 个，平均每个输出 token 的耗时就升至 $(3.57 + 26.26)/2 \approx 14.9$ ms，反而慢于串行方式。更快的草稿生成节省了 10.7 ms，但每轮输出减少，每个输出 token 分摊的时间随之增加，整个请求因而变慢。

### 8.5.4 草稿长度的动态选择与并发竞争

确定草稿来源之后，还需要决定每轮生成多长。草稿越长，一轮能够输出的 token 上限越高。但只有前面的 token 全部接受，后面的才有机会进入最终输出。设前面的草稿 token 都已接受时，第 $j$ 个 token 的条件接受概率为 $a_j$，则长度为 $m$ 的草稿对应的平均输出数为

$$\mathbb{E}[N_m] = 1 + \sum_{j=1}^{m} \prod_{i=1}^{j} a_i.$$

草稿长度从 $m-1$ 增至 $m$ 时，平均多输出 $\Delta N = \prod_{i=1}^{m} a_i$ 个 token。设为此增加 $\Delta T$ 时间；原先一轮耗时为 $T$、平均输出 $N$ 个 token，平均每个输出 token 的耗时为 $T/N$。要使增加草稿长度后平均每个输出 token 的耗时更短，需要满足 $(T + \Delta T)/(N + \Delta N) < T/N$，整理得

$$\frac{\Delta T}{\Delta N} < \frac{T}{N}.$$

也就是说，为新增输出付出的平均时间，必须低于原来的平均每 token 耗时。由于接受概率需要逐项相乘，越靠后的草稿 token 保留下来的概率越低，增加它们就越难获得足够的时间收益。

预设执行形状也会影响新增时间 $\Delta T$。如果本次验证四个或五个 token，都把各 token 的特征向量补齐为八行矩阵，使用预先准备的八行计算图，增加一个 token 所需的额外计算较少；若待验证的 token 从八个增至九个，需要补齐为十六行矩阵并改用相应计算图，计算量便会突然增加。并发请求数也影响推测解码的收益：低负载时，多行验证可以利用闲置算力；高负载时，普通批处理已经充分共享权重，草稿生成和验证还要与其他请求争用加速器和内存。

一组实测比较了不同草稿长度。K7 和 K15 分别表示每轮生成 7 个和 15 个草稿 token 的配置。Qwen3-8B / DFlash 的短提取任务在并发 1 时，普通 decode、K7、K15 的完整耗时中位数约 90.0、30.6、29.4 ms。普通执行改为 K7 节省约 59 ms，从 K7 改为 K15 只再节省约 1.2 ms。逐轮记录中还出现两种块长都只接受一个草稿 token 的轮次：K7 丢弃六个草稿 token，K15 丢弃十四个，最终输出长度相同。更大的验证块增加了计算量，却没有在这一轮带来更多输出。[31]

因此，动态调整草稿长度时，可以使用上述判断条件：对接受概率高、增加验证位置所需时间短的请求，生成更长的草稿；对草稿前几个位置就经常通不过验证，或需要改用更大执行形状的请求，缩短块长。确定草稿长度时，还要给目标模型和草稿模型的 KV 留出空间，再按第 8.3 节的方法决定接收多少请求。

**练习 8-7 ・推导：多长的草稿能使平均每个输出 token 的耗时最短？** 固定草稿接受概率 $a = 3/4$，计算草稿块长为 1、2、4、8 时，每轮平均输出的 token 数。沿用第 8.5.3 节 RTX PRO 6000 上的数据并保留一位小数：一轮验证 26.3 ms，草稿网络逐个生成、每个草稿 token 3.6 ms，即 $T_m = 26.3 + 3.6m$ ms，求每种块长的平均每个输出 token 的耗时。再设草稿按每组 4 个位置执行，不足一组也按整组计时，即 $T_m = 26.3 + 3.6 \times 4\lceil m/4 \rceil$ ms，比较块长 4 与 5，并用边际条件解释选择。

[31] DFlash 同执行器对照及逐请求阶段观测。实验在低并发下，按固定顺序执行给定短题集。并发 1 通过短题的普通、K7、K15 时间中位数分别为 90.02、30.58、29.43 ms；配对输出共 32 对。

## 8.6 请求负载下的性能与配置选择

### 8.6.1 到达率、排队、准入与取消

第 8.2 至 8.5 节讨论了如何改变实例的内存占用和执行方式。在线服务还要处理请求不断到达的情况：一条请求尚未完成，下一条就已经进入队列；一条请求等待工具返回时，模型计算暂停，KV 却仍需保留。调度器因此既要安排当前计算，也要为正在生成或等待继续执行的请求保留状态。

保存这些未完成请求的状态需要多少空间，取决于系统内平均有多少条请求。在稳定服务中，系统内的平均请求数满足 Little 定律

$$\bar{n} = \lambda \bar{T},$$

其中 $\lambda$ 为平均到达率，$\bar{T}$ 为请求在系统中的平均停留时间。把观察期间每条请求的停留时间相加，再除以观察时长，就得到系统内的平均请求数。例如，每秒到达 4 条请求、每条平均停留 0.5 秒时，系统内平均有 2 条请求；平均停留时间增至 2 秒时，就有 8 条。

这些请求有的在排队，有的正在计算，还有的仅保留状态。增大 batch 可以提高输出吞吐，但 KV 读取、计算和其他开销会逐渐限制这种增长。当处理速度无法继续随到达率提高时，新请求就会在队列中累积，等待时间也随之延长。

**准入控制**决定是否开始处理新请求。接收请求之前，先检查可用的 KV 空间和距离截止时间还剩多久。对第 8.1 节的短请求，0.196 秒首输出加 6.76 秒生成已用掉 6.95 秒；再多排队 0.1 秒就会超时。提前识别这种请求，可以拒绝、转交其他实例，或使用更快的执行配置。取消则在请求失去使用价值时停止安排后续迭代，并在加速器操作完成后释放空间。

这些优化会相互影响。KV 压缩让实例能同时接收更多请求，较大的 batch 又让普通 decode 更有效地共享权重，推测解码的相对收益也就随之改变。下面将这些变化应用于同一组请求，计算组合后的执行时间和内存占用。

### 8.6.2 质量与 SLO 约束下的有效吞吐

只看队列长度和完成速度，还不能判断用户是否得到了可用的答案。第 8.4.4 节的质量实验出现过错误答案，排队则会造成正确但迟到的回答。把这两种损失一起计入，才能评价服务效果。设观察窗口长为 $T_{\text{obs}}$，第 $i$ 条请求正确时 $Q_i = 1$，按期完成时 $D_i = 1$。按完成的请求数计量，有效吞吐为

$$R_{\text{good}} = \frac{\sum_i Q_i D_i}{T_{\text{obs}}}.$$

分子只计正确且按期完成的请求。若改用全部完成的请求数作为分子，就得到完成吞吐。两者的差距说明，有多少处理能力花在了错误或迟到的答案上。

一组实测显示了这两项损失的大小。Qwen3-8B 检索实验使用相同的八道题，要求请求在到达后 3 秒内正确完成，比较逐个接收请求、连续批处理和一次提交全部请求三种方式。每次实验提交 16 条请求，全部实验共生成 336 次回答，其中 294 次正确，只有 155 次同时满足时限。其余 139 次正确回答因为太晚返回，仍未满足服务要求。[32]

[32]服务与能量原始记录。同一设备、八道题、七条件各三个 16 请求窗口；同题跨条件输出一致，42 个错误来自同一道题。吞吐表为各窗口

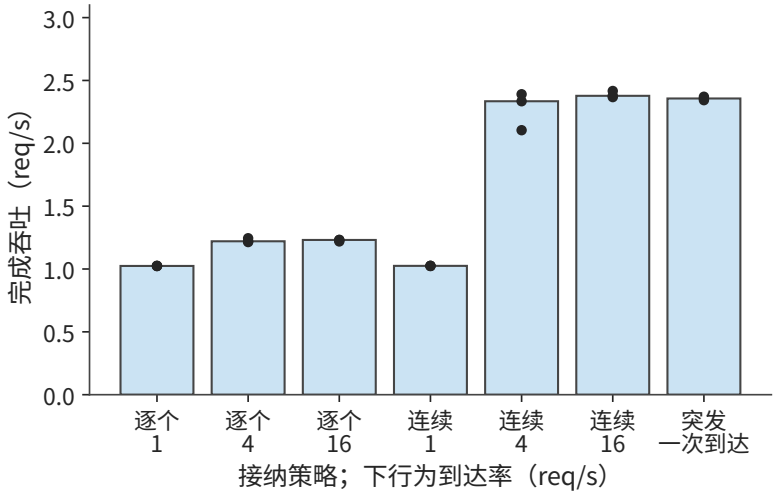



图 8-33：每种接纳与到达条件使用三个 16 请求窗口。柱为全部完成请求的吞吐中位数，圆点为各窗口结果。时间从计划到达起计算。

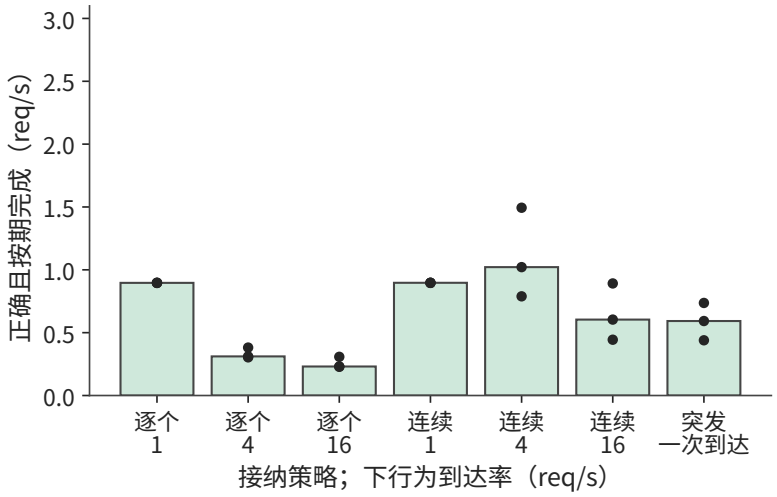



图 8-34：分子仅计正确且在 3 秒内完成的请求。两图采用相同横轴顺序和纵轴范围，差额来自错误或迟到的回答。

到达率为 1/s 时，两种在线策略的有效吞吐均约 0.90 req/s。每批工作较少，逐个接纳也能及时处理大部分正确请求。到达率升至 4/s，逐个接纳的完成吞吐约 1.22 req/s，有效吞吐却降至 0.31 req/s；队列变长，许多正确输出越过了期限。连续批处理把完成吞吐提高到约 2.33 req/s，有效吞吐达到约 1.02 req/s，更多请求在期限内完成。

再把连续模式的到达率推至 16/s，完成吞吐只增至约 2.38 req/s，有效吞吐反而降到约 0.60 req/s，减少约 41%。完成吞吐已基本不再增长，更多请求到达主要增加了排队时间。对这组窗口，4/s 比 16/s 按时返回了更多正确答案。选择服务配置时，要先找出有效吞吐开始下降的到达率，再限制新接收的请求数，避免排队时间过长。

逐个接收请求时，其他请求虽然没有进入引擎，却仍在应用队列中等待。因此，应从请求原定的到达时刻开

指标中位数。每窗口包含 16 条请求，按排序后向上取整的位置计算 p95，得到的就是最大值。NVML 与 RAPL 组件计数窗口略宽于请求窗口，其他服务仍在运行。

始计时，把引擎外的等待也计入。结合本节的几组测量，加速器每秒输出多少 token、单条请求等待多久，以及每秒有多少请求正确且按期完成，需要分别计算。

### 8.6.3 配置、加速器与完整任务成本

比较配置之前，先把实测换算成处理速度。确定批处理、精度和缓存设置后，用完成的工作量除以占用加速器的时间，就得到各阶段的实际处理速度：prefill 速度 $r_P$ 按每秒处理的新输入 token 计，decode 速度 $r_D$ 按每秒完成的 decode 调用计。批内多条请求共享一次执行时，每条请求都算完成了一次调用，占用时间只计一次。一条请求有 $L$ 个新输入 token、输出 $G$ 个 token，它占用加速器的时间（以 GPU 秒计）为

$$t_{\text{GPU}} = \frac{L}{r_P} + \frac{G-1}{r_D}.$$

没有同条件的测量时，可以用规格乘以效率来估计：prefill 时间取矩阵运算量除以峰值算力与算力效率之积，decode 一轮的时间取读取的权重与 KV 字节数除以峰值带宽与带宽效率之积。下表给出实验 8-1 在 RTX PRO 6000 上测得的效率。一轮 decode 取引擎逐轮记录中不含 prefill 的轮次；客户端看到的输出间隔还包括交付时间，略长一些，例如第 8.1.1 节的 26.5 ms。

| 2K 上下文、无共享前缀 | batch 1 | batch 4 | batch 16 | batch 64 |
|---|---|---|---|---|
| 每轮读取的权重与 KV | 15.44 GB | 16.34 GB | 19.97 GB | 34.46 GB |
| 按 1792 GB/s 读取所需时间 | 8.62 ms | 9.12 ms | 11.14 ms | 19.23 ms |
| 实测一轮 decode | 26.26 ms | 26.62 ms | 27.44 ms | 29.63 ms |
| decode 带宽效率 | 33% | 34% | 41% | 65% |
| prefill 算力效率 | 62% | 61% | 61% | 58% |

decode 效率随 batch 上升，并不是因为读取变快了。这组实验关闭了 CUDA Graph，每轮都要逐个提交 kernel：读取量从 15.44 GB 增至 34.46 GB，一轮只从 26.26 ms 增至 29.63 ms，batch 越大，这段几乎固定的时间分摊到的字节越多。prefill 效率则稳定在 BF16 峰值的 58%–62%；8K 输入、前 6K 已缓存时为 52%–58%。

表中的带宽效率就是第 1.2.2 节的 MBU，算力效率就是 MFU。batch 为 1 时，一轮 decode 比 8.62 ms 的读取下界多出约 17.6 ms，而且这段时间基本不随读取量变化。按第 1.3.4 节的判据，这不是模型漏项，而是逐个提交 kernel、主机侧调度与同步造成的开销，用 CUDA Graph 和更好的调度可以去掉；第 5.5.1 节的 vLLM v0.6.0 案例说明了主机侧开销能减少到什么程度。

**综合算例：在内存与时限约束下选择前缀共享、压缩和推测配置。**有效吞吐给出了服务的评价标准，容量和时间分析则能在运行之前排除不合适的配置。采用章首给定的推理实例：RTX PRO 6000 分给本实例 32 GiB，扣除权重和基本工作区后，留给 KV 及辅助缓冲区的空间为 12 GiB。16 条长请求同时到达，每条输入 8192 个 token、输出 256 个 token，前 6144 个输入 token 相同，完成时限为 7 秒。下面比较四种方案，并假设四种方案都能给出正确答案。共享方案在请求到达前已经缓存了公共前缀。推测方案使用实验 8-5 的 DFlash 草稿模型。启用后进程显存峰值增加约 3.1 GiB（从 17,706 MiB 增至 20,894 MiB），其中草稿模型权重约 1.95 GiB。草稿模型还要处理输入，实验 8-5 中输入约 2300 个 token 时，首 token 时间从 129.4 ms 增至 141.6 ms，多出 9.4%。设这组请求平均每轮输出 3 个 token。

这组请求的共享方案，正是实验 8-1 测过的条件：8K 输入、前 6K 已缓存、batch 16。prefill 阶段处理 32,768 个新 token 用时 2.04 s，$r_P \approx 16{,}059$ 新 token/s，达到峰值算力的 58%；此后每轮 decode 用时 27.35

ms，$r_D = 16/0.02735 \approx 585$ 调用/s。每条请求占用 $t_{\text{GPU}} = 2048/16059 + 255/585 \approx 0.563$ GPU 秒，16 条合计约 9.01 GPU 秒。其余方案由这张表和同一条件的实测推出，下表给出整批 16 条请求各阶段的时间。[33]

| 方案 | 状态组织 | 首 token 返回前（整批 prefill） | 首 token 返回后的执行 |
|---|---|---|---|
| A | BF16，独立上下文 | 7.34 s | 255 轮，每轮 29.63 ms |
| B | BF16，共享前缀 | 2.04 s | 255 轮，每轮 27.35 ms |
| C | q8_0，独立上下文 | 7.34 s | 255 轮，每轮约 28.26 ms |
| D | BF16，共享前缀与推测 | 2.23 s | 85 轮，每轮输出 3 个 token，每轮 27.35 ms |

A、C 不共享前缀，每条请求都要完整处理 8192 个 token，整批矩阵运算量为 2137.5 TFLOP，按同一条件 58% 的算力效率计，约需 7.34 s。A 每轮读取 34.46 GB，与 64 条 2K 请求相同，取表中的 29.63 ms；C 的 KV 按 34/64 压缩，每轮读取 25.40 GB，在表中 19.97 GB 与 34.46 GB 两列之间按读取量插值，约 28.26 ms，这里还没有计入格式转换的时间。D 的 prefill 比 B 多 9.4%，约 2.23 s。D 每轮让 16 条请求各验证 8 个位置，矩阵运算约 2.56 TFLOP，按 58% 的效率约需 8.8 ms，短于 B 一轮的 27.35 ms；按第 8.2.1 节取两者中的较大值，每轮仍为 27.35 ms。草稿网络的前向也在同一轮内完成，实验 8-5 中 DFlash 一轮的中位耗时并不长于普通 decode 一步（第 8.5.2 节），因此这里不另加时间。

先检查内存是否足够。A 需 $16 \times 1188 = 19008$ MiB，超出 12 GiB，无法同时接纳这组请求。B 只需一份前缀加 16 份私有状态，共 6048 MiB，约 5.91 GiB。C 的状态按 34/64 压缩，共约 9.86 GiB。D 在 B 的基础上增加 3188 MiB，共 9236 MiB，约 9.02 GiB。B、C、D 都能放入内存，再比较三者完成请求所需的时间。

B 的总耗时为 $2.04 + 255 \times 0.02735 = 9.01$ 秒，与实验 8-1 实测的请求耗时中位数 9.02 秒一致，但超过了 7 秒的时限。在这张卡上，仅 255 轮 decode 就要 6.97 秒，16 条请求一起逐 token 生成 256 个输出，无法按期完成。C 失去了前缀复用，prefill 从 2.04 s 增至 7.34 s，总耗时达到 $7.34 + 255 \times 0.02826 = 14.55$ 秒。D 每轮输出 3 个 token，用 85 轮生成剩余的 $85 \times 3 = 255$ 个 token，总耗时为

$$T_D = 2.23 + 85 \times 0.02735 = 4.56 \text{ s}.$$

内存不足的 A 和超时的 B、C 都已排除，只有 D 能按时返回 16 个正确答案。这一比较依次使用了前缀共享后的容量、压缩格式的大小、每轮耗时和每轮输出数量。在这张卡上，一轮 decode 的时间几乎不随 batch 和上下文变化，只有减少轮数，才能明显缩短生成阶段。

图 8-35 将这两项检查画在同一张平面图上。横向越过虚线，表示 KV 和辅助缓冲区超过 12 GiB；纵向越过虚线，表示完成时间超过 7 秒。只有 D 同时落在两个限制以内。

最后比较成本。每个合格结果占用这张卡的时间，等于整批从到达到全部完成的时间除以合格结果数。D 为 $4.56/16 \approx 0.285$ GPU 秒；按 600 W 的功耗上限计，每个合格结果的能耗不超过约 171 J。B 占用这张卡 9.01 秒，在 7 秒时限下却没有一个合格结果，这些时间和能量全部浪费。

**内存减少或输出缩短时，选择如何变化？** 如果 KV 和辅助缓冲区的可用内存从 12 GiB 减至 6 GiB，D 所需的约 9.02 GiB 无法容纳；B 所需的约 5.91 GiB 虽能容纳，却会超时。这组请求没有可行方案，只能放宽时限、缩短输出，或者改用不占显存的草稿来源（见练习 8-9）。如果内存不变，只把输出缩短到 16 个 token，B

[33] 实验 8-1 的效率表由计算脚本从batch 扫描的逐轮记录导出，峰值取硬件表中 RTX PRO 6000 的 1792 GB/s 与 503.8 TFLOP/s；DFlash 的显存峰值、首 token 时间与每轮耗时取自实验 8-5。各方案的内存与时间由算例检查复算。

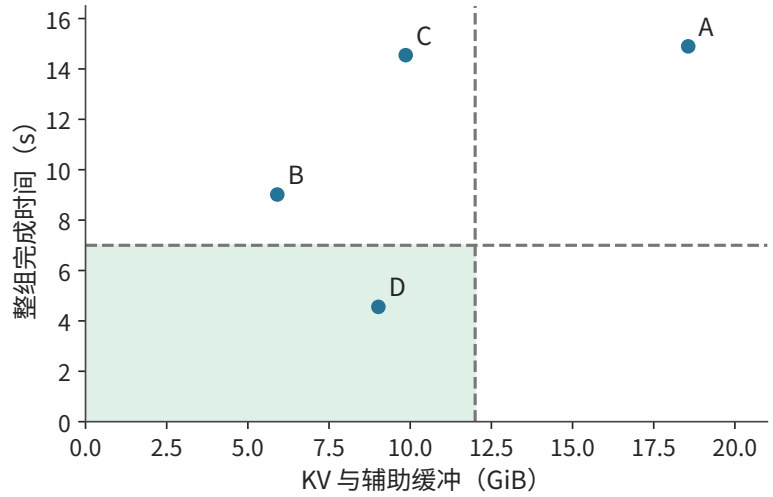



图 8-35：16 条长请求在 RTX PRO 6000 上的配置比较。横轴为 KV 和辅助缓冲区占用，已扣除共同的权重与基本工作区；纵轴为整批请求总耗时，由实验 8-1、8-5 的实测推出。A 内存不足，不能在本实例上执行，图中是它所需的时间。阴影区域同时满足 12 GiB 和 7 秒限制。A 为 BF16 独立上下文，B 为 BF16 共享前缀，C 为 q8_0 独立上下文，D 为 BF16 共享前缀加推测解码。

需要 $2.04 + 15 \times 0.02735 = 2.45$ 秒，D 需要 $2.23 + 5 \times 0.02735 = 2.37$ 秒。两者都能按期完成，D 只快 0.08 秒：草稿模型多做的 0.19 秒 prefill，几乎抵消了少执行 10 轮 decode 节省的时间。

以上分析的都是实例内的执行时间。计算完整 Agent 任务的耗时，还要加上工具执行和外部等待。假设一次任务耗时 100 秒，其中 decode 占 50 秒；将 decode 加速四倍后，总耗时变为 $50 + 50/4 = 62.5$ 秒，整体加速 1.6 倍。如果 decode 原本只占 20 秒，总耗时则变为 $80 + 20/4 = 85$ 秒，整体只加速约 1.18 倍。一般地，设 decode 占原总时间的比例为 $f$，该阶段加速 $s$ 倍，其他阶段耗时不变，则由 Amdahl 定律得到

$$S_{\text{task}} = \frac{1}{(1-f) + f/s}.$$

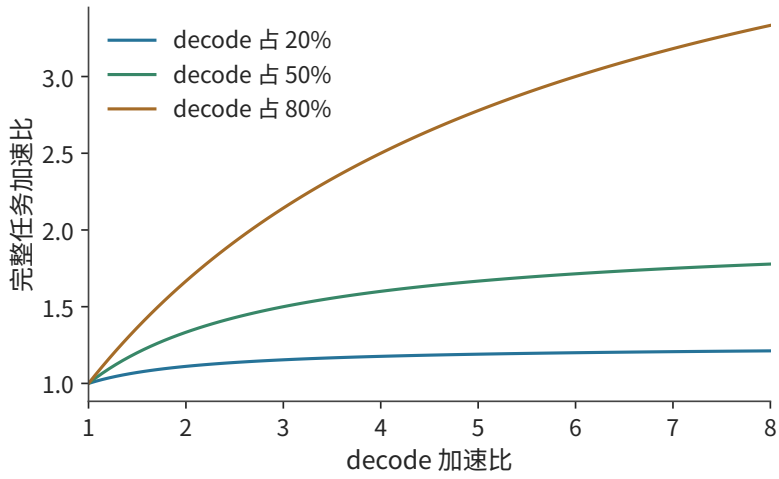



图 8-36：其他阶段时间固定、没有新增准备工作的 Amdahl 加速比曲线。三条曲线分别取 decode 占基线时间 20%、50%、80%。该阶段原有的耗时占比越高，局部加速带来的收益就越大。

图 8-36 中，decode 占比越低，曲线越早变平：即使生成阶段继续加速，工具调用和其他等待仍占据原来的时间。比较加速器占用和能耗时，也要覆盖这些阶段，并按最终得到的合格请求数计算。设观察期间占用加速器

的总时间为 $T_{\mathrm{GPU}}$，总能耗为 $E_{\mathrm{obs}}$，合格结果数为 $N_g$，则每个合格结果平均占用 $T_{\mathrm{GPU}}/N_g$ 的 GPU 时间，消耗 $E_{\mathrm{obs}}/N_g$ 的能量。实验 8-9 在同一张卡上读取了 GPU 与 CPU package（整个处理器封装）的能量计数：连续批处理在到达率 1、4、16 req/s 下，每个合格结果分别消耗 615.0、652.0、1083.7 J；逐个接纳在 4 req/s 下为 1982.3 J。16/s 时连续批处理的完成吞吐与 4/s 相近，迟到的回答却更多，每个合格结果的能耗高出约 66%。即使答案错误或返回太晚，已经消耗的能量也必须计入总成本。[34]

**练习 8-8 ·计算：阶段加速与草稿准备如何改变任务总时间。**一个 100 秒任务由 prefill 20 秒、decode 50 秒、工具及等待 30 秒组成。分别将三个部分的耗时单独减半，求每种情况下的任务总耗时。另从原始任务出发，将 decode 耗时降为原来的四分之一，并增加 8 秒草稿准备时间，求净加速比，并计算准备时间达到多少时这项改动不再节省时间。

**练习 8-9 ·核心·综合设计：在容量与时限内减少每个合格结果占用的 GPU 时间。**沿用综合算例在 RTX PRO 6000 上的容量与时间，以 B 为基础构造方案 E：共享 BF16 前缀，草稿改为从上下文中查找（第 8.5.2 节），索引放在主机内存，不占显存，也不增加 prefill；每条请求平均每轮输出 2 个 token，后续执行 128 轮，整批每轮 27.35 ms 加 0.1 ms 查询。计算 E 的显存占用、总耗时和每个合格结果占用的 GPU 秒，与 B、D 比较；再分别把 KV 和辅助缓冲区的可用内存改为 6 GiB、把输出长度改为 16 个 token，重新选择。开放实验部分使用相同题集和到达轨迹运行两种配置，记录全部请求的到达、首输出、结束、正确性与状态峰值，按本节公式计算有效吞吐。

## 历史与延伸阅读

每轮迭代接纳新请求、分页注意力与前缀树缓存分别推动了请求调度、状态分配和计算复用。Orca、PagedAttention 与 SGLang 的相关设计可沿这些机制阅读；Sarathi-Serve 和 NanoFlow 进一步研究分块、流水及不同资源之间的重叠。[35][36]

第 8.2.1 节的多 adapter 服务可沿 Punica 与 S-LoRA 继续阅读，两者分别管理 adapter 与请求状态，在共用基础模型的同时执行各 adapter 的低秩计算。[37]

混合注意力与递推模型把缓存索引扩展为状态恢复问题，Marconi 等工作讨论恢复机会和缓存价值。权重卸载、张量编码以及统一内存执行，则将加速器容量与链路成本联系起来；配套 M2 Max 记录展示了小模型的真实 KV 分配与输入处理。[38][39][40]

推测采样的接受与修正规则为保持目标分布奠定了基础，后续从上下文查找草稿、依据中间特征预测草稿以及并行生成草稿的方法，主要改变每轮开销与接受的草稿长度。完整 Agent 任务还包含工具和外部阶段，公开速度与任务记录可结合第 8.6 节的阶段分解阅读。[41][42]

[34]服务与能量原始记录。同一设备、八道题、七条件各三个 16 请求窗口；同题跨条件输出一致，42 个错误来自同一道题。吞吐表为各窗口指标中位数。每窗口包含 16 条请求，按排序后向上取整的位置计算 p95，得到的就是最大值。NVML 与 RAPL 组件计数窗口略宽于请求窗口，其他服务仍在运行。

[35]Orca 原文与分块计算和版本说明。

[36]OSDI 2025 调研、NanoFlow 正式稿与资源共享算例。

[37]MLSys 2024 调研与多 LoRA 计算。

[38]混合状态恢复与 checkpoint 计算、MLSys 2025 调研。Kimi K3 的容量按指定存储格式和并行配置计算。

[39]张量压缩与传输。资料包含 LLM.265 的视频引擎实现、专用硬件设计与性能估计。

[40]M2 Max 小模型容量记录及32K 补测。各槽依次执行 prefill。

[41]推测采样原文、推测解码原文、框架计数与预算笔记、调研中的框架覆盖。基础概率算法、草稿路线和具体实现分别引用。

[42]生成实验与 Agent 运行记录。Agent 记录采用非流式调用；阶段时间图采用题设参数。

## 本章小结

本章从单条请求的执行过程出发，逐步分析了内存容量、计算安排和服务效果之间的关系。不同请求共用权重，KV 则随各自的上下文增长。合批能减少每个输出 token 分摊的权重读取，但每条请求仍要读取自己的上下文。这一收益取决于底层资源的性能：权重改由更快的独立存储提供后，合批不再能摊薄权重读取，不同 batch 和推测解码方案都要重新比较。连续批处理及时使用短请求释放的执行位置，分块 prefill 则减少长输入对已有请求生成过程的干扰。

KV 的管理方式决定了实例能同时处理多少请求。分页按需分配空间，共享只保存一份公共前缀，前缀缓存还省去了重复计算。上下文的追加、改写和总结会改变缓存复用以及后续状态存储与读取的开销，比较这些上下文组织方式与其他服务优化方案时，应采用相同的任务质量要求。压缩进一步减少存储量，卸载将部分权重移出加速器，代价是反复搬移。推测解码增加了草稿生成和验证的开销，却能让一次目标模型计算确定多个输出。这些变化都可以通过内存占用、每轮耗时和每轮输出数来比较。

章末的综合算例先检查内存是否足够，再判断能否按期完成，最后比较每个合格结果占用的 GPU 时间。有了前缀共享，原先无法容纳的 16 条长请求就能同时执行；但在 RTX PRO 6000 上，逐 token 生成 256 个输出仍会超时，推测解码减少了 decode 轮数，才让它们按期完成。可用内存减少时，这组请求找不到可行方案；输出缩短时，推测解码的优势几乎消失。选择改变的原因，都能从本章推导的数据量、执行时间和输出数量中找到。

完成本章的配置比较后，应记录三类结果：在给定加速器、精度和请求长度下能同时保存多少条请求的状态，prefill 与 decode 分别以多快的速度处理工作，以及请求延迟如何随 batch 和到达率变化。这些结果已经包含调度、缓存和 kernel 效率的影响。

下一章以这些结果为基础，决定各类加速器承担什么工作、需要多少执行单元，以及 KV 如何传输和共享。多个加速器既可以共同执行一个实例，也可以组成多个完整副本或分别调度的阶段服务池。批处理决定已有资源如何处理请求，部署方式决定这些资源如何分工，两者共同影响完整服务的性能。

# 第 9 章 分布式推理

第 8 章讨论了如何用批处理、请求调度、缓存和推测解码，提高一组给定计算资源的推理效率。这些方法既适用于单卡，也适用于多卡协同执行的实例。本章进一步研究计算与状态放在哪里：不同计算资源分别承担哪些阶段和算子，权重与 KV 如何分布，执行位置之间如何传输数据、分配工作并处理故障。上下文变长以后，状态可能比权重还大：在第 9.2.3 节的算例中，batch size 为 32 时 KV 共占 43.5 GB，约为 Qwen3-8B 权重的 2.7 倍。因此，状态放在哪里、如何交接，与计算如何分工同样重要。

本章用 **P** 表示 prefill 阶段，用 **D** 表示 decode 阶段。**PD 分离**把这两个阶段交给不同资源。**AF 分离**把每层的注意力与前馈网络（包括专家网络）交给不同资源。

围绕计算与状态的放置，本章考察四种基本的组织方式。增加完整副本，可以让不同资源组分别处理请求。将 prefill 与 decode 分开，可以为两个阶段分别配置计算资源；将注意力与专家计算分开，可以利用不同计算资源的计算和存储能力；共享 KV 则让请求使用其他执行位置留下的状态。这些组织方式可能出现在一个推理实例内部，也可能连接多个独立调度的服务池。

本章的设计问题是：一台服务器装有四张 A100 80GB SXM，另一台装有四张 H20 SXM5 96GB，这八张卡如何处理每秒到达的 3.5 个请求？每个请求输入 8192 个 token、输出 1025 个 token，属于带推理过程的请求：模型先生成较长的思考内容，再给出回答。模型采用 Qwen3-8B，权重和 KV 都使用每元素 2 字节的 BF16 格式。每张卡都能容纳完整模型，因此一张卡就是一个执行单元（worker）。两台服务器之间没有 NVLink，只能经 RDMA 网卡通信。后文依次决定计算如何分工、上下文在哪里交接、缓存何时值得保留，以及启动后的积压多久能够消退。第 9.7 节将综合这些结果，为这组资源确定部署方案。作为对照，第 9.2 节还会分析八张 A100 组成的同构集群。

贯穿第 9.3、9.4 节的另一个算例采用 Qwen3-235B-A22B，研究专家权重放在哪里。同样是 512 次专家分派，分派结果不同时，可能需要读取许多专家的权重，也可能反复使用少数热点专家的权重。据此判断专家计算应放在 CPU 还是 GPU 上，再考察热点集中到一张卡后，为什么需要专家副本。两个算例采用相同的分析方法：先确定每个请求需要的工作和状态，再估算各类处理器完成这些工作、存储系统提供所需数据的速度。

## 9.1 计算与状态的分布方式

### 9.1.1 计算资源、推理实例与服务副本

先区分硬件组织与服务组织。GPU、NPU 等加速器和 CPU 提供计算能力与相应的存储资源，进程在这些设备上运行模型和通信逻辑；通信组内的每个进程有一个 rank。推理实例可以由多个这样的进程协同执行。完整服务副本则拥有完成整个模型推理所需的权重和执行能力，可以和其他副本分别处理请求。

图 9-1 至图 9-4 画出四种组织方式。前两种都执行完整模型，区别是八卡协作组共同处理同一批请求，多个完整副本则各自接收请求。后两种进一步改变分工：PD 按生成阶段划分，AF 按层内算子划分。每种组织方式都有自己的调度器管理所负责的计算。

完整副本按请求分工；图 9-3 中，同一请求先后经过两个服务池。沿 P 到 D 的箭头看，交接的是输入处理留下的上下文 KV，两个池仍各自执行相应阶段的完整模型。

图 9-4 将分界移到模型层内部：注意力一侧产生的激活交给前馈一侧，结果再返回。与一次阶段交接相比，这条数据通路要在每一层、每一步生成中反复走过。

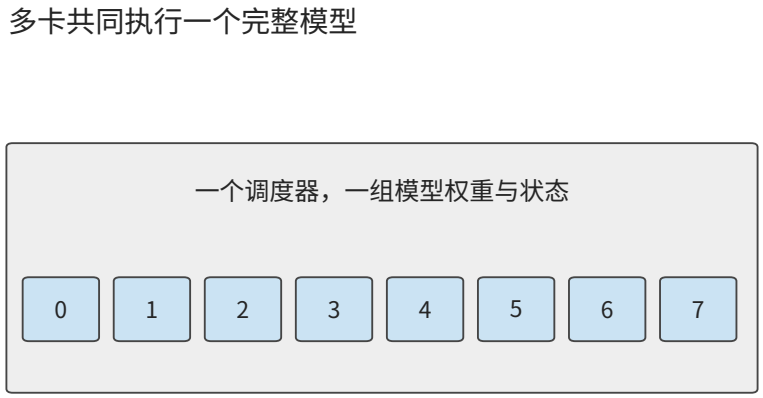



图 9-1：八张卡共同执行一个完整推理实例，接收同一批请求。卡号表示协作组成员。

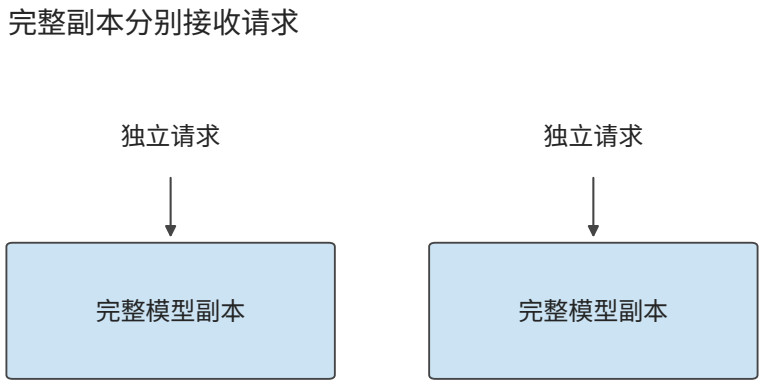



图 9-2：每个副本都能完成整次推理，可以分别接收独立请求。副本自身也可以由多张卡组成。

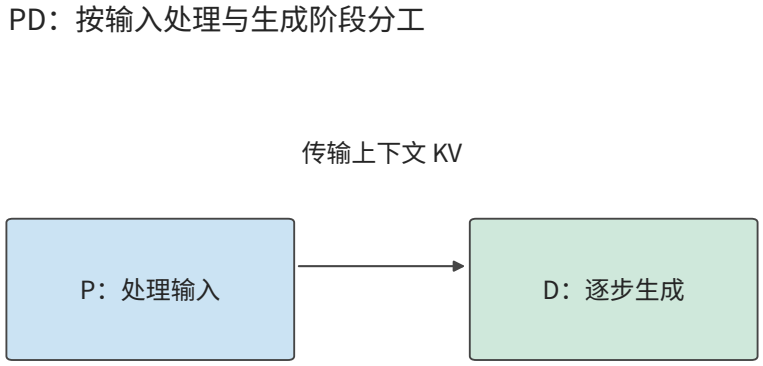



图 9-3：P 处理输入，D 继续生成。两个服务池分别调度，上下文 KV 从 P 交给 D。

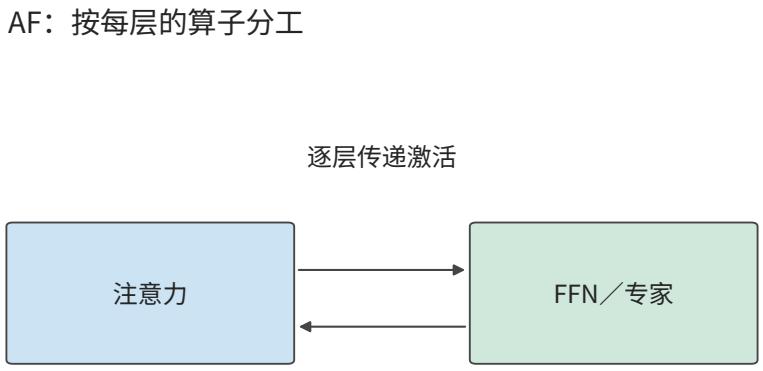



图 9-4：注意力与 FFN ／专家分别执行，隐状态作为激活在两侧往返。每层都需要这次交接。

比较这些部署方式时，先使用第 8 章得到的运行配置和处理速度，再改变计算资源之间的分工。P、D 由同一组执行资源处理的方式称为**共置**。考虑已经调优的共置实例。请求的输入长度为 $L$，总输出长度为 $G$，prefill 计算最后一个输入 token 的 logits，并由此采样首个输出。在本章采用的生成约定下，后续 decode 调用为 $G-1$ 次：最后一个返回的 token 尚未送入模型，所以输出数比后续 decode 调用数多一。

对给定请求分布，记每个请求在资源 $r$ 上消耗的平均服务时间为 $d_r$，同类资源共有 $n_r$ 个可独立工作的单元。到达率与资源占用时间需要满足

$$\lambda d_r < n_r,$$

其中 $\lambda$ 是到达率。若多个操作使用同一资源，应先将它们的资源占用时间相加；若操作分别使用相互独立的资源，则为每种资源单独列出约束。资源占用量按“资源数 × 占用秒数”计算；本章的资源是 GPU，这一乘积记为 GPU 秒。例如，每秒到达四个请求，每个占用 0.5 个 GPU 秒，就需要两张持续工作的 GPU。若恰好只有两张卡，突发到达的额外请求就会一直积压，因为两张卡都忙于处理源源不断的后续请求；只有增加 GPU，才能消化突发。

设每个请求在某类副本上的 prefill 和 decode 分别占用 $d_P$、$d_D$ 秒，两个阶段在该副本上串行执行，那么 $N$ 个相同共置副本的请求率上限为

$$\mu_{\mathrm{co}} = \frac{N}{d_P + d_D}.$$

第 9.2.3 节将从数据表峰值推出开篇两类卡在各阶段的服务时间。一张 A100 处理一次 8192 个 token 的 prefill 约需 0.86 秒，一条请求的 1024 次 decode 合计占用约 1.76 GPU 秒；一张 H20 的 prefill 约需 1.81 秒，decode 只占约 0.88 GPU 秒。每个完整副本都要承担两个阶段，所以一张 A100 每请求占 2.62 GPU 秒，一张 H20 占 2.68 GPU 秒。八张卡合计每秒约完成 3.02 个请求，低于每秒 3.5 个请求的到达率，每秒积压约 0.5 个请求。问题由此变得明确：同样八张卡，重新分工能否补上这 0.5 请求/s 的缺口。

副本可以由多张卡共同构成。例如，Qwen3-235B-A22B 的一种 TP2×EP4 部署方式（TP 将专家矩阵二分，EP 将专家分成四组）用八张卡处理同一批请求：每两张卡分担专家矩阵，四个 EP 组分别持有不同的专家。注意力和 KV 在四个 EP 组间各复制一份，因此一条请求 8192 个 token 的上下文在整个实例内占 5.875 GiB，是

单份逻辑 KV 的四倍。这八张卡共同组成一个推理实例；要增加能独立处理请求的副本，还要另外安排一套完整的权重与状态。[1]

### 9.1.2 计算分工与状态共享

增加完整副本改变请求的分配；PD 分离改变 prefill、decode 的执行位置；AF 分离改变 attention 与 FFN、专家的执行位置；共享 KV 改变上下文状态的保存与读取范围。这些部署方式改变了系统的不同部分，可以逐项引入，也可以组合使用。

| 部署方式 | 希望获得的收益 | 新增的主要约束 |
|---|---|---|
| 增加完整副本 | 处理更多独立请求，分散排队 | 模型复制、缓存分散、启动及保持实例运行 |
| PD 分离 | 分别配置两个阶段的能力 | KV 交接、两端同时占用内存、两个队列 |
| AF 分离 | 让算子使用适合的计算与存储资源 | 逐层激活往返、同步、流水线各阶段的负载均衡 |
| 共享 KV | 跨实例复用上下文，减少重复 prefill | 缓存标识、读取、就绪通知、失效与空间释放 |

选择部署方式，先分析当前负载的资源需求。如果两个阶段的资源需求相近，完整副本已经能够均衡服务，分离可能只会增加交接开销。如果许多请求共享长前缀，跨实例缓存可能比进一步拆分计算更有价值。分别计算资源占用和交接时间，就能判断当前应增加副本、分离阶段还是共享状态。

### 9.1.3 同一请求的执行路径与状态驻留时间

确定分工方式之后，再沿一条请求的时间线考察它经过哪些阶段、留下哪些状态。图 9-5 把一次多轮请求的计算阶段与状态驻留时间放在同一时间线上。带视觉输入时，编码阶段 E 将图像转换为模型使用的特征，EC 保存这些编码结果供复用。P 处理这些编码结果与文本，D 逐步生成输出；工具返回后，新一轮 P 处理新增信息。权重通常跨请求保留，激活主要跨相邻算子传递，KV 随已处理的位置增长。工具等待期间计算可以暂停，但上下文状态仍可能占据内存。

以本章的 Qwen3-8B 为例，一份包含 8192 个 token 的 BF16 上下文 KV 占 1.125 GiB。工具执行 10 秒时，GPU 可以处理其他请求，但这份上下文 KV 仍需占用 1.125 GiB 内存，容量与占用时长的乘积为 11.25 GiB · s；十个这样的会话同时等待，就占 11.25 GiB。计算暂停后，状态仍需保留，这正是共享缓存和换出策略要解决的问题。

这条请求若随后返回 1025 个 token，P 先产生第一个输出，1024 次 decode 再处理前 1024 个生成 token。此时连续 KV 覆盖 $8192+1024=9216$ 个 token，最后一个返回 token 尚未送回模型。下一轮在这 9216 个 token 之后继续，先处理该尚未写入 KV 的末尾 token 和新增输入；恢复生成时从此处继续计算。

[1] 同一 MoE 请求的权重、状态与通信归属。其 TP2×EP4、处理同一批请求和 FP32 传输格式是明确教学条件，用于具体展示通信归属。

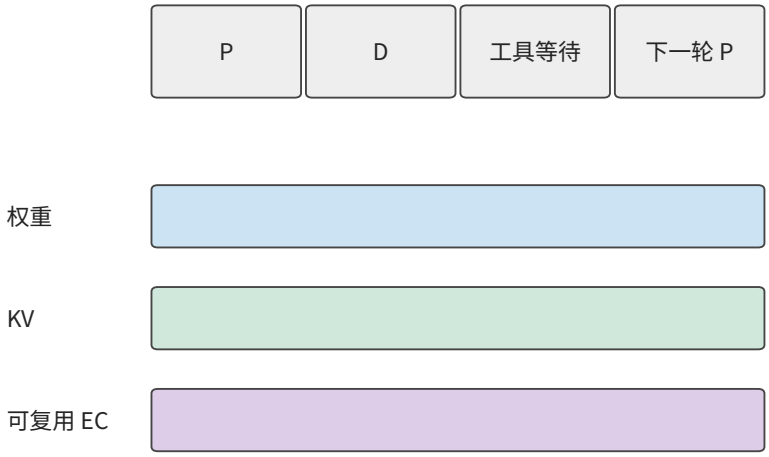



图 9-5：计算暂停期间，状态仍需保留。示意同一请求从 prefill、decode 到工具等待和下一轮的状态驻留时间；横轴按阶段排列，不表示相等时长，EC 仅在有视觉输入时出现。E 为视觉编码，EC 为保存的视觉编码结果，P 为 prefill，D 为 decode。

## 9.2 Prefill–Decode 分离与资源配比

### 9.2.1 Prefill 与 Decode 为什么需要分离

Prefill 可以在一次调用中处理许多新 token，较大的矩阵计算往往能更充分地复用权重。在 decode 阶段，每个请求每步通常只处理一个新 token，需要反复读取权重与上下文状态。两者混合执行时，较长的 prefill 可能推迟已有请求的下一个输出；过度优先 decode 又会延长新请求的等待。

第 8.2 节已经用分块 prefill 缩短了长输入对 decode 的阻塞。本节保持相同模型和请求，进一步改变执行位置。PD 分离把两个阶段放入不同的服务池，使它们拥有独立的队列、批处理和资源配比。DistServe 与 Splitwise 是研究 P、D 分离部署的推理系统。在八张 A100 组成的同构集群里，分离靠独立队列消除两个阶段之间的调度干扰。在 A100 与 H20 组成的异构集群里，每类卡还能只承担自己擅长的阶段。[2]

这种分工像是先由一个人读完材料，再交给另一个人继续处理：接手者需要已有的工作记录，才能继续处理。对本例模型，这份需要交接的记录就是上下文 KV。P 和 D 分别处理同一请求的不同阶段，每个阶段都要执行完整的前向计算。Qwen3-8B 的 P 处理 8192 个输入 token，D 随后调用 1024 次；每次都经过注意力和前馈网络。因此两个池都需要完整的模型权重，KV 则从 P 交给 D。独立批处理带来的收益，首先要抵得过这次新增的交接。

应用通过一次调用请求完整的模型推理，服务内部则可以按阶段安排不同资源。模型的数据依赖确定了阶段之间的切分位置，KV 给出了两阶段之间需要交接的状态，负载比例决定各池需要多少资源。利用这些信息，服务可以在统一调用接口之下，为输入处理和逐步生成分别选择 batch 与计算资源。

### 9.2.2 KV 传输与两端内存占用

P 完成输入处理后，D 需要得到上下文 KV 才能继续生成。GQA 让多个查询头共享一组键和值，为每个上下文 token 保存完整的键和值。设层数为 $n$，上下文长度为 $L$，KV 头数为 $h_{kv}$，每头维度为 $d_h$，每元素为 $b$

[2]DistServe、Splitwise 与阶段放置的已归档材料见本章扩写资料及阶段分工与状态交接研究。

bytes，则逻辑状态大小为

$$V_{KV} = 2nLh_{kv}d_h b.$$

系数 2 分别对应 K 与 V。Qwen3-8B 的固定配置为 36 层、8 个 KV 头、每头 128 维。BF16 状态每个 token 的 KV 占 144 KiB；8192 个 token 占

$$V_{KV} = 8192 \times 144 \text{ KiB} = 1.125 \text{ GiB}.$$

**例 9.1　prefill 与 decode 之间的 KV 交接需要多久？** A100 服务器与 H20 服务器之间经 200 Gbit/s 的 RDMA 网卡相连，单向载荷带宽按 25 GB/s 计，采用整份状态直接交接。纯载荷传输时间为

$$T_{\text{payload}} = \frac{1.125 \times 2^{30}}{25 \times 10^9} \approx 48.3 \text{ ms}.$$

再加一次 5 µs 的启动与同步，总时间仍约 48.3 ms：对这份完整 KV 数据，固定开销只占万分之一左右，提高载荷带宽更有助于缩短等待时间。而在 AF 逐层交接的小数据量传输中，固定开销会占更大的比例。[3]

上述状态大小来自 GQA 的保存方式。MLA（第 2.3.2 节）不保存展开后的键和值，而保存上投影之前的潜变量：每个 token 每层只有一条 $d_c$ 维潜变量和一条 $d_r$ 维位置分支，状态大小为

$$V_{KV,\text{MLA}} = nL(d_c + d_r)b.$$

DeepSeek-V3 的固定配置为 61 层、$d_c = 512$、$d_r = 64$，BF16 下每个 token 占 $61 \times 576 \times 2 = 70{,}272$ bytes，约 68.6 KiB；同样 8192 个 token 的上下文占 549 MiB。沿用例 9.1 的 25 GB/s 链路，纯载荷传输约 23.0 ms；链路提高到 50 GB/s 时为 11.5 ms；同一条链路上，GQA 状态需要 24.2 ms。

| 交接状态 | 每 token | 8192 个 token | 25 GB/s | 50 GB/s |
|---|---|---|---|---|
| Qwen3-8B，BF16 GQA | 144 KiB | 1.125 GiB | 48.3 ms | 24.2 ms |
| DeepSeek-V3，BF16 紧凑 MLA | 68.6 KiB | 549 MiB | 23.0 ms | 11.5 ms |

MLA 的状态更小，是因为缓存放在线性变换的另一侧：GQA 为每个 KV 头保存展开后的 $K$、$V$，MLA 只保存上投影之前的潜变量，按结合律，上投影可移到当前查询一侧执行。若把 DeepSeek-V3 的 128 个头也展开保存，每个 token 要占 $61 \times 128 \times (192 + 128) \times 2$ bytes，约 4.77 MiB，是紧凑表示的 71 倍；与 Qwen3-8B 只有 8 个 KV 头的 GQA 相比，紧凑 MLA 仍只有其 0.48 倍。后文凡是 KV 字节进入公式的地方，都并列给出这两种状态的结果。[4]

[3] Qwen3-8B 的 PD / AF 交接计算，固定官方模型形状、字节与启动假设。生成图读取同名 JSON。25 GB/s 使用十进制单位，1.125 GiB 使用二进制单位；精确载荷为 1207959552 bytes，纯传输耗时为 48.31838208 ms，加上 5 µs 后为 48.32338208 ms。一条请求 1024 步 decode 的 AF 累计交接见对应结果。

[4] 紧凑 MLA 状态的 PD / AF 交接计算，以及 50 GB/s 链路下的 MLA 与 GQA 对照；每 token 字节数与跨模型缓存计算的 deepseek-v3 行一致。$d_c = 512$、$d_r = 64$ 取自 DeepSeek-V2 论文第 12 页，V3 沿用同一注意力配置。精确载荷为 575,668,224 bytes，25 GB/s 下纯传输 23.02672896 ms，临界启动时间为 23003136/71 ns。

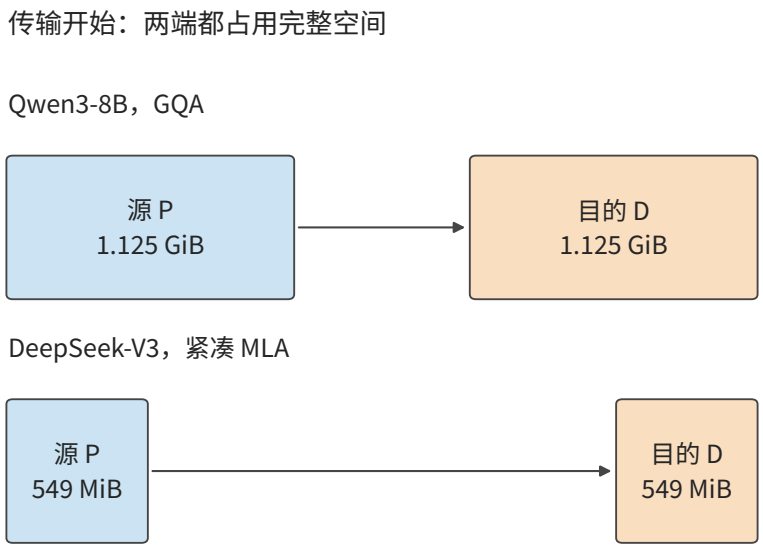



图 9-6：源 P 保留完整状态，目的 D 同时分配同样大小的接收空间。GQA 状态两端各 1.125 GiB，传输过程共占 2.25 GiB；紧凑 MLA 状态两端各 549 MiB，共占 1.07 GiB。方框宽度按字节数比例绘制。

采用整份双缓冲交接时，传输结束前源端保留完整状态，目的端分配完整接收缓冲。图 9-6 至图 9-8 依次画出交接开始、传输完成和释放源端三个时刻。载荷全部写入目的端后，P 发出完成标记，D 看到标记才读取状态（图 9-7）；执行权交给 D 之后，P 才释放源缓冲（图 9-8）。采用分块传输时，目的端确认收到某一块后，源端就可以提前释放这一块，缩短两端重复驻留的时间；目的端靠完成标记确定哪些连续的块已经可用。若经主存中转，状态则依次占用源端、主存和目的端的缓冲，主存到 GPU 的拷贝成为新的依赖边。

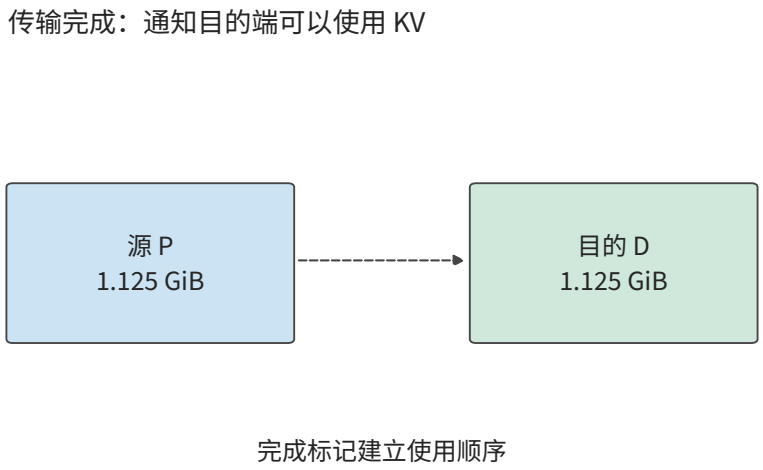



图 9-7：完整的 1.125 GiB 载荷已经写入目的端，P 随后发出完成标记。标记本身不再搬移数据，只规定先写完、后使用的顺序：D 看到标记才读取状态。

请求持续到达时，链路带宽既限制每秒能交接多少份状态，也影响每份状态需要传输多久。每秒 8 次上述交接需要约 9.7 GB/s 的传输带宽；每秒 16 次则需要约 19.3 GB/s。100 GbE（100 Gbit/s 以太网）端口每方向 12.5 GB/s，能满足前者的平均需求，但每个请求仍要等传输完成；后者则不同：即使两池有空闲计算能力，网络队列也会持续增长。前一种情形下每份状态要约 96.6 ms 才能传完，后一种情形每秒还会积累约 6.8 GB 待

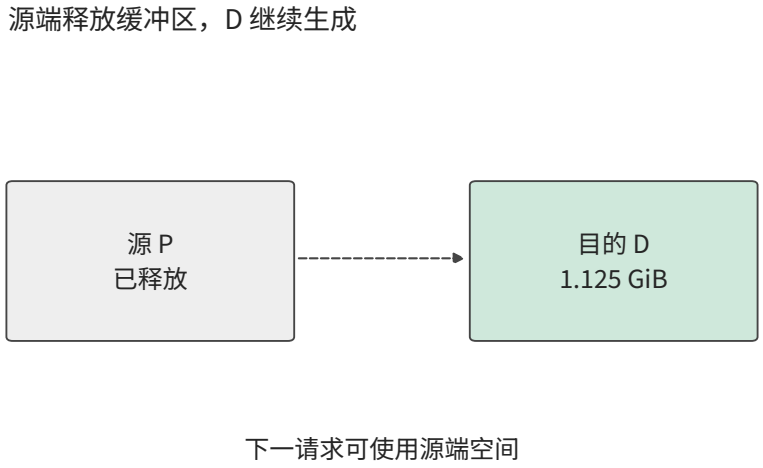



图 9-8：本算例将执行权交给 D 后释放 P 的源缓冲，D 保留上下文并继续生成。源端空间用于后续请求。

传数据。传输时间决定了单个请求至少要等待多久，而数据到达速率与传输速率之差决定了队列增长多快。换成 549 MiB 的紧凑 MLA 状态，每秒 8 次和 16 次交接只需约 4.6 和 9.2 GB/s，同一个 100 GbE 端口能满足两者的平均需求，每份状态约 46.1 ms 传完。队列是否增长由状态字节数与链路带宽共同决定，模型换了上下文表示，同一条链路的结论就会翻转。

### 9.2.3 阶段处理能力与实例数量配比

比较各阶段的处理速度前，应先算出单个请求在各阶段分别需要多少工作。设请求有 $L_{new}$ 个需要 P 新处理的 token、$G-1$ 次 decode 调用。某类 worker 的有效能力分别为 $r_P$ 个新 token/s 与 $r_D$ 次 decode/s，则该 worker 的每请求资源需求为

$$d_P = \frac{L_{new}}{r_P}, \qquad d_D = \frac{G-1}{r_D}.$$

这里的资源占用时间可以直接相加；输入 token/s 与输出 token/s 则要先换算成同一个请求的需求。批内多个请求共享一次 decode 调用时，完成的 decode 步要计入所有请求，但资源占用时间只按这一次批执行计一次。

两类卡的 $r_P$ 与 $r_D$ 可以从数据表推出。第 4 章的 Roofline 模型给出各阶段耗时的下界：计算时间与访存时间中的较大者。下表列出两种卡的峰值。H20 的算力不到 A100 的一半，显存带宽却约为 A100 的两倍，这正是 H20 适合 decode、不适合 prefill 的原因。

| 卡 | BF16 稠密算力 | 显存带宽 | 显存容量 |
|---|---|---|---|
| A100 80GB SXM | 312 TFLOP/s | 2039 GB/s | 80 GB |
| H20 SXM5 96GB | 148 TFLOP/s | 4096 GB/s | 96 GB |

实际 kernel 达不到峰值。第 8.6.3 节用 RTX PRO 6000 上的逐轮记录校准了两项效率：decode 的带宽效率随 batch size 从 batch 1 的 33% 升到 batch 64 的 65%，batch size 为 32 时按两侧实测插值约为 49%；prefill 的算力效率稳定在 58%–62%。本章两项都取为峰值的 50%，decode 一侧与 batch size 32 的插值相当，prefill 一侧比实测低约一成。按此取值，A100 的有效算力和带宽为 156 TFLOP/s、1020 GB/s，H20 为 74 TFLOP/s、

2048 GB/s。

取 50%，就是把第 1.2.2 节的 MFU 和 MBU 都取为 0.5，意味着一半的硬件能力没有转化为模型计算。按第 1.3.4 节的判据，这一半有两类来源：一类是本章一阶模型没有计入的工作，例如 kernel 按 tile 补齐的零行、没有与计算重叠的通信；另一类是可以去掉的主机侧开销，例如逐个提交 kernel、切换前重建通信组和执行图。第 9.4.3 节和第 9.6.2 节各举一例。

把这两组有效能力代入章首的请求。Prefill 处理 8192 个输入 token，需要 133.6 TFLOPs 矩阵运算，却只读一次约 15 GB 权重，受计算能力限制。A100 需要 0.856 秒，每秒约处理 9570 个新 token；H20 需要 1.81 秒，每秒约处理 4540 个。Decode 一步为批内 32 条请求各生成一个 token。取生成过程中的平均上下文 8192+512 = 8704 个 token，这一步读约 15.1 GB 权重和 41.1 GB KV，矩阵运算只有 0.65 TFLOPs，受显存带宽限制。A100 每步需 55.1 ms，每秒约完成 580 次 decode 调用；H20 每步只需 27.4 ms，每秒约完成 1166 次。两种卡的强弱恰好相反：A100 的 prefill 比 H20 快一倍多，H20 的 decode 比 A100 快一倍。batch size 为 32 时，请求生成到最后，KV 共占 43.5 GB，加上权重约 59.9 GB，两种卡都能容纳。[5]

先考虑同构场景：八张 A100 执行同样的模型。共置时每张卡每请求占 $0.856 + 1.764 = 2.62$ GPU 秒，八张合计 3.05 请求/s。分离后两池只能按整数张卡划分：三张做 P 提供 3.50 请求/s，五张做 D 提供 2.83 请求/s，上限为 2.83 请求/s，比共置低 7%。每个请求在分离前后消耗同样的 GPU 秒，所以同构分离不能提高吞吐；整数划分还使两池能力无法恰好相等，多出的能力只能闲置。[6]

同构分离换来的是延迟的稳定。共置且不分块时，一次 8192 个 token 的 prefill 独占 A100 约 0.86 秒，这张卡上所有在途请求都停止生成，相当于错过约 16 个 55 ms 的 decode 步。若这八张卡每秒接收 2.5 个请求，每张卡平均每 3.2 秒就要插入一次这样的 prefill。分离后，D 卡每步稳定在 55 ms；P 卡专门处理输入，首 token 时间是 0.86 秒的 prefill 加上 KV 交接。

第 8.2 节的分块 prefill 是分离之外的另一条路。A100 上一步 decode 的访存需要 55.1 ms，矩阵计算只需 4.2 ms，其余时间算力空闲。按 156 TFLOP/s 计，这段空闲足以处理约 490 个 prefill token。一次 8192 个 token 的输入分成 17 块，捎带在 17 个 decode 步中完成，首 token 时间约 0.94 秒，与分离相当，而在途请求的输出间隔不变。若捎带的计算完全藏在访存时间里，八张 A100 共置的上限升到 4.50 请求/s，远高于分离的 2.83。实际能藏住多少，取决于分块与 decode 同时运行时能否同时占满算力和带宽，要用实测的混合步时间判断。因此在同构集群上，通常先采用分块 prefill；分离的价值在于两个阶段可以分别选择并行方式与 batch size，并彻底隔开彼此的排队。

**例 9.2　异构集群的阶段配比如何决定吞吐上限？**使用 8192 个输入 token、1025 个输出 token。四张 A100 与四张 H20 的阶段能力按上述方法推出，如下表所示；后续改变请求组成时，同样按新的工作量重新推算。

| 卡 | P 能力，新 token/s | D 能力，调用/s | 每请求 P GPU 秒 | 每请求 D GPU 秒 |
|---|---|---|---|---|
| A100 80GB SXM | 9566 | 580 | 0.856 | 1.764 |
| H20 SXM5 96GB | 4538 | 1166 | 1.805 | 0.878 |

共置时，一张 A100 每请求占 2.62 GPU 秒，一张 H20 占 2.68 GPU 秒，八张合计 3.02 请求/s。把四张 A100 分给 P、四张 H20 分给 D，P 池提供 $4/0.856 \approx 4.67$ 请求/s，D 池提供 $4/0.878 \approx 4.55$ 请求/s。每请求交接 1.125 GiB。两台服务器之间按一张 200 Gbit/s 网卡计，25 GB/s 的带宽每秒可交接约 20.7 份，于是总体

[5]阶段能力由 pd-pool 计算从硬件表与 Qwen3-8B 的逐算子前向账推出。A100 80GB SXM 的峰值取自 NVIDIA A100 数据表。NVIDIA 没有公开 H20 数据表，型号与容量取自 AI Enterprise vGPU 文档，BF16 算力与显存带宽取自 MegaScale-Infer第 8 页表 3，同表 A800、H800 两行与 NVIDIA 数据表一致。50% 的校准来自第 8.6.3 节与实验 8-1 的逐轮效率记录。

[6]异构 P、D 整数分配、八张 A100 同构对照、八张 H20 同构对照、前缀命中、129 个输出、4097 个输出。每个结果的 `derived_stage_rates` 字段逐卡列出 prefill 与 decode 的计算量、读取量、Roofline 两项时间和受限资源。

吞吐率上限为

$$\mu_{PD} = \min\left(\mu_P, \mu_D, \frac{B_{net}}{V_{KV}}\right) \approx 4.55 \text{ 请求/s}.$$

其中 $\mu_P$、$\mu_D$ 是 P 池与 D 池的请求率上限，$B_{net}$ 是两池之间交接链路的带宽。

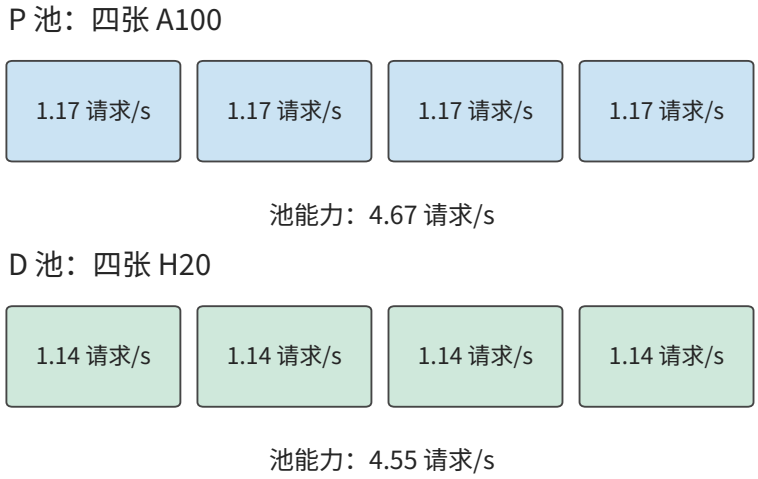



图 9-9：每张 A100 在 P 阶段提供约 1.17 请求/s，每张 H20 在 D 阶段提供约 1.14 请求/s。P 池 4.67 请求/s，D 池 4.55 请求/s，均低于网卡约 20.7 请求/s 的交接能力。

分离后，两类卡都避开了各自较慢的阶段，请求率上限从 3.02 增至 4.55 请求/s，是共置的 1.51 倍，高于每秒 3.5 个请求的到达率。模型仍处理同样的 8192 个输入 token 与 1024 次 decode，差别在于由谁执行。把角色对调，让 H20 做 P、A100 做 D，P 池只剩 2.22 请求/s，吞吐上限不到正确分工的一半。

上文在同构集群上比较过的分块捎带，放到 H20 上效果有限：H20 一步 decode 的空闲算力只够处理约 85 个 prefill token。按理想捎带计，四张 A100 加四张 H20 共置的上限为 4.17 请求/s，仍低于分离的 4.55。异构集群的收益来自硬件本身的差异，分块无法替代。

另一个可以改变的条件是交接状态本身。把交接状态换成紧凑 MLA，$\mu_{PD}$ 的链路项从 20.7 增至 43.4 请求/s，50 GB/s 链路下则从 41.4 增至 86.9 请求/s；上限仍由 D 池的 4.55 请求/s 决定，四张 A100 做 P、四张 H20 做 D 的配比不变。链路成为约束项的条件是 $B_{net} < \mu V_{KV}$：在 4.55 请求/s 下，GQA 状态要求链路至少约 5.5 GB/s，紧凑 MLA 状态只要约 2.6 GB/s。高于此值，再提高带宽不会改变吞吐上限；低于此值，图 9-10 中所有高于 $B_{net}/V_{KV}$ 的格子都会被链路压平到同一个值。[7]

紧凑路径节省的是字节，不是全部工作。按第 2.3.2 节的结合律，上投影移到了查询一侧：DeepSeek-V3 每层每个查询 token 要多做一次 $128 \times 128 \times 512$ 的查询变换和一次同样大小的值恢复，合计 33.5 MFLOPs，61 层为 2.05 GFLOPs，与表 2-3 中 KV 上投影（kv_b_proj）对单个 token 的展开运算量相同。这部分计算量随批内请求数线性增加：一次 decode 调用服务 $B$ 条请求，就多 $2.05B$ GFLOPs。按 H20 的 74 TFLOP/s 有效算力，每条请求每步多 27.7 μs，1024 步合计 28.3 ms。若这部分计算不能藏进访存时间，D 池每请求的 GPU 秒从 0.878 增至约 0.907，D 池能力从 4.55 降到约 4.41 请求/s，仍高于 3.5 的到达率；每条请求每步的额外计算

[7]紧凑 MLA 状态下的 P、D 整数分配、50 GB/s 链路与 GQA 状态 50 GB/s 对照。紧凑路径每步额外的 2,046,820,352 FLOPs 由 MLA 交接结果的 `mla_compact_path` 字段给出，与 V3 单步 decode 算子表的 kv_b_proj 行逐层一致。

超过约 258 µs，才需要第五张 D 卡。紧凑路径改变的是 D 实例的瓶颈位置：每 token 读取从展开状态的 4.77 MiB 降到 68.6 KiB，计算项却随 batch size 增长，于是 D 实例在更小的 batch size 上就从读取受限转为计算受限。例 9.2 的 $r_D$ 是在 batch size 32 下推出的，换成紧凑 MLA 后要按新的转折 batch size 重新推算。

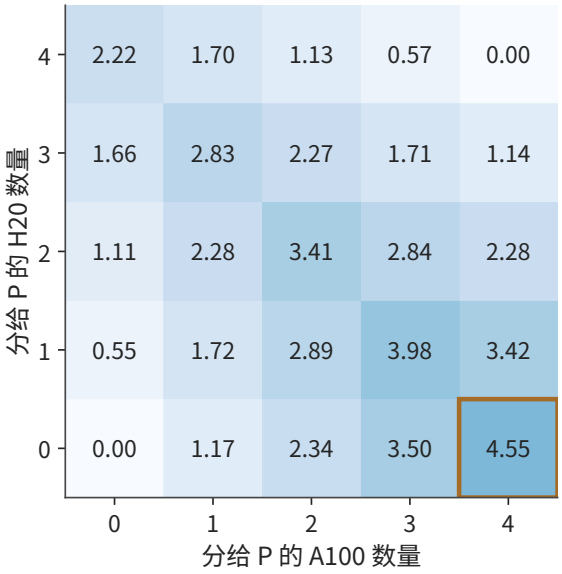



图 9-10：阶段配比如何限制请求率。四张 A100 与四张 H20，阶段能力见例 9.2；横轴为分给 P 的 A100 数，纵轴为分给 P 的 H20 数，其余分给 D。每格取两池能力与 25 GB/s 网卡交接能力的最小值，不含排队；颜色深浅表示可持续的请求率，单位为请求/s，框出的格子为最优配比。

图 9-10 还给出了偏离最优配比后的吞吐率变化。把一张 A100 从 P 调到 D，P 从 4.67 降为 3.50 请求/s，D 从 4.55 增为 5.12 请求/s，结果受 P 限制而降至 3.50。把一张 H20 从 D 调到 P，P 增为 5.22 请求/s，D 却降为 3.42 请求/s，同样变慢。最优配比使两池能力接近相等。改变配比后，一个池即使有富余能力，也无法替另一个池完成工作。

### 9.2.4 不同请求与负载下的收益边界

例 9.2 固定了请求组成，现在只改变其中一个条件：前缀命中。P 已经缓存了前 6144 个 token，只需处理剩下的 2048 个。新 token 减到四分之一，A100 的 prefill 时间从 0.856 秒降到 0.238 秒，一张 A100 的 P 能力从 1.17 增至 4.20 请求/s；若仍给 P 四张 A100，它们合计能处理 16.8 请求/s，D 却仍只能处理 4.55 请求/s。

把两张 A100 调到 D，留下的两张提供 8.41 请求/s 的 P 能力；D 则由两张 A100 与四张 H20 提供 $2/1.764 + 4/0.878 \approx 5.69$ 请求/s。总体吞吐率上限提高到 5.69 请求/s。再调走一张 A100，P 只剩 4.20 请求/s，反而更慢。假定 D 没有缓存这段前缀，因此两种配比仍须交接完整的 1.125 GiB；P 的计算量减少了七成多，但传给 D 的 KV 大小没有变化。

再恢复原始输入，把输出缩短到 129 个 token，即不带推理过程的普通对话。D 调用数从 1024 降到 128，一张 H20 的 D 能力增至约 9.46 请求/s，D 几乎不再是瓶颈。最优配比变成四张 A100 加三张 H20 做 P、一张 H20 做 D，P 池提供 $4 \times 1.168 + 3 \times 0.554 \approx 6.33$ 请求/s。共置此时已有 5.84 请求/s，分离只提高 8.5%。

图 9-11 将三次调整画在同一组计算资源上。每个方块都是原来的一张卡，变化的只是它承担的阶段。前缀命中后 P 的工作减少，可以让出两张 A100；输出变短后 D 的工作减少，三张 H20 转去做 P。

这三种请求最合适的 P 卡数分别是四张、两张和七张。前缀命中减少 P 的新工作，把资源推向 D；输出变短减少 D 的重复工作，把资源推向 P。推理请求的输出越长，D 的比重越大，H20 这类带宽强、算力弱的卡越

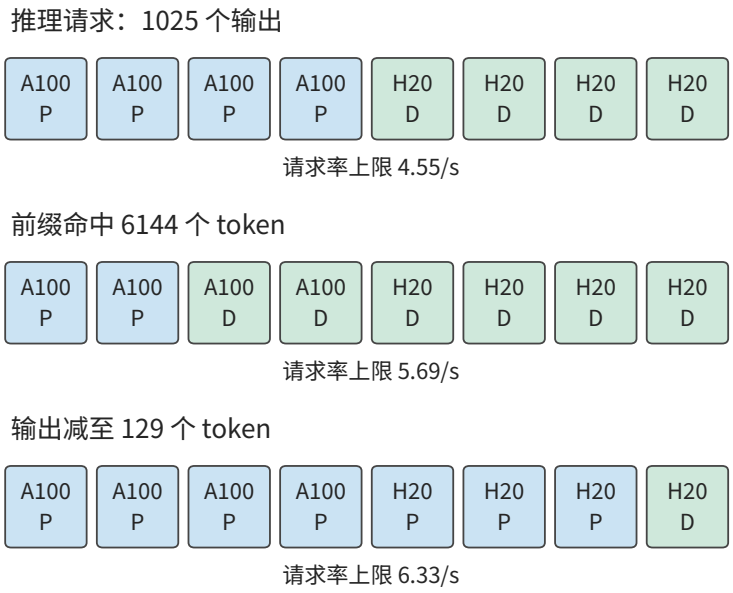



图 9-11：请求组成改变资源分配。四张 A100 与四张 H20，阶段能力按例 9.2 的方法推出；第一行为 8192 输入、1025 输出的推理请求，第二行只减少 P 的新输入，第三行恢复完整输入并将输出减至 129 个 token。方块表示卡，每行下方为对应的吞吐率上限。P 为 prefill 池，D 为 decode 池。

有价值。由此可以把调度分成两个时间尺度：请求到达时选择进哪个队列，负载分布持续变化时调整池的规模。队列分配决定眼前等待，池规模决定积压能否长期消退。

## 9.3 Attention–FFN 分离与异构执行

### 9.3.1 Attention、FFN 与专家的执行边界

AF 指注意力与前馈网络的分工。与 PD 一次划分两个生成阶段不同，AF 在模型层内划分计算：注意力产生隐状态，前馈网络或被选中的专家处理该隐状态，再把结果送回，供后续算子使用。下一层通常要等待该结果，因此小数据量传输的启动与同步会重复多次。

MoE 特别适合这种分工：总专家权重较大，但每个 token 只选择其中一部分。若大多数专家保存在主存，可以将被选中的权重搬给 GPU，也可以让 CPU 使用本地权重完成计算，再传递较小的激活。前一条路径花链路带宽换 GPU 算力，后一条路径花 CPU 算力换较少的链路流量。如何选择取决于本次分派给同一个专家的 token 数：每个 token 的特征向量占该专家输入矩阵的一行，这些行复用同一份专家权重。

### 9.3.2 CPU ／ GPU 协作

上一小节的两条路径交付同一份输出，区别在于跨链路传什么：图 9-12 传权重，图 9-13 传输入和输出。第 8.4.3 节采用第一条路径，将权重暂存主存，使用时搬到 GPU。KTransformers 是采用第二条路径的实际系统：GPU 处理注意力和部分常驻专家，CPU 处理分给自己的专家；把分给 CPU 的工作提交后，GPU 继续执行可以并行的分支，最后同步并合并结果。热点专家、共享专家以及 prefill 阶段可以采用不同的计算资源分工。分支汇合时，下一层必须等两侧都完成后才能开始。[8]

[8]权重卸载与执行位置及实现版本说明。

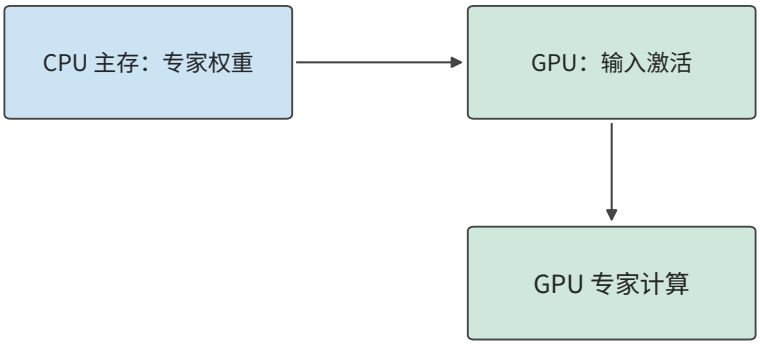



图 9-12：把一份 36 MiB 专家权重从主存送到 GPU，再由 GPU 读取本地输入完成计算。

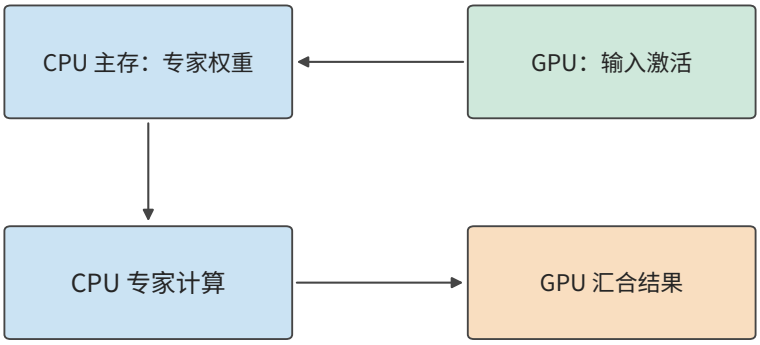



图 9-13：输入从 GPU 交给 CPU，CPU 使用主存权重执行专家，结果回到 GPU。每次 token 到专家的分派，传入和传出的特征向量合计 16 KiB；并行的 GPU 常驻专家分支完成后再汇合。

两条路径的开销都取决于一份专家权重的大小。对 Qwen3-235B-A22B，一个专家包含三个矩阵，参数数量为 $3 \times 4096 \times 1536 = 18874368$。BF16 权重为 36 MiB。

**例 9.3　专家权重复用如何改变 CPU 与 GPU 的执行选择？** 采用 KTransformers 论文的实验机：两颗 Intel Xeon Platinum 8452Y，每颗 36 核、配 1 TB DDR5，经 PCIe 4.0 连接一张 A100 40GB PCIe。论文用 Intel 的内存测试工具 MLC 测得同一 CPU 插槽内的内存带宽为 220 GB/s。在单颗 CPU 上运行 MoE 层时，PyTorch 基于 AVX-512（512 位向量指令集）的 kernel 最高达到 1.8 TFLOP/s；KTransformers 基于 AMX（高级矩阵扩展）矩阵指令的 kernel 最高达到 21.3 TFLOP/s。GPU 取 A100 40GB PCIe 峰值 312 TFLOP/s 与 1555 GB/s 的 50%，即 156 TFLOP/s 与 778 GB/s。PCIe 4.0 x16 的理论带宽为 32 GB/s，交接按 25 GB/s 计，每次启动 5 µs，BF16 激活宽度为 4096。八个专家依次执行，总时间为各专家耗时之和，权重每批读取一次。每个专家只

收到一个 token 的特征向量时，CPU 直接读本地主存里的权重，省去了在较慢链路上搬 36 MiB；随着每个专家的输入行数增加，搬一次权重的开销由更多行的计算分摊，CPU 却更早达到算力上限。

先考虑八个热点专家各处理一个 token：权重合计 288 MiB。在 220 GB/s 主存上读取约 1.37 ms，加激活交接后 CPU 路径约 1.46 ms；搬到 GPU 的路径仅传权重就约 12.1 ms，连同启动与 GPU 计算约 12.5 ms。低复用时，避免大块权重搬移比提高矩阵算力更有价值。

若这八个专家各处理 128 个 token，权重仍是 288 MiB，计算量却增至原来的 128 倍。用 AVX-512 kernel，CPU 路径增至约 22.2 ms，GPU 路径约 12.5 ms，此时在 GPU 上计算更快。图 9-14 画出了中间过程：每个专家的输入从 71 个 token 增到 72 个时，GPU 开始比 CPU 快。换成 AMX kernel，128 个 token 时 CPU 路径只需约 2.57 ms，约为搬权重路径的五分之一，交点推迟到每专家 689 个 token。同一台机器处理同一批请求，CPU 用哪套指令就决定了专家该放在哪一侧。每个专家每多处理一个 token，CPU 增加的计算时间都多于 GPU，而权重搬移的开销不变，所以过了交点 GPU 就占优。[9]

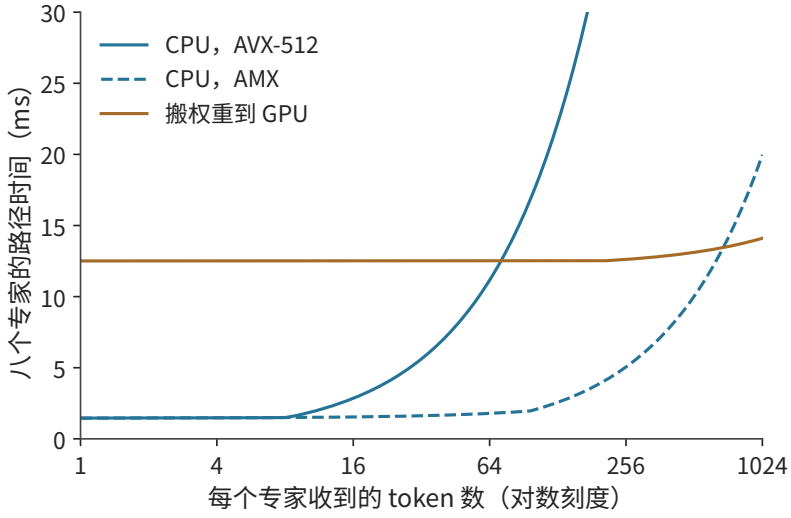



图 9-14：专家复用改变执行位置的选择。八个专家各有 36 MiB BF16 权重。CPU 为单颗 Xeon Platinum 8452Y，AVX-512 与 AMX kernel 分别按论文实测的 1.8 与 21.3 TFLOP/s、同插槽内存 220 GB/s 计；GPU 为 A100 40GB PCIe，按峰值的 50% 计为 156 TFLOP/s、778 GB/s；PCIe 交接 25 GB/s，每次启动 5 µs。横轴为对数刻度，曲线按例 9.3 计算单层专家路径，未含格式转换。

将上述数字写成一般形式。记专家参数数量为 $P_e$、权重字节数为 $W_e$，输入宽度为 $h$、每元素字节数为 $b$。若同一个专家本批收到 $m$ 个 token，其矩阵计算量为 $F_e(m) = 2mP_e$ FLOPs，权重至少要读取一次。设有效 CPU 算力、DRAM 带宽分别为 $C_C, B_D$，GPU 算力、HBM 带宽为 $C_G, B_H$，交接带宽为 $B_L$。假定权重与激活已经转换为执行所需的格式，交接与专家计算串行发生，算子时间取计算与权重读取两项中的较大值，则单个专家在两种执行方式下的耗时分别为

$$T_C(m) = 2\alpha + \frac{2mhb}{B_L} + \max\left(\frac{2mP_e}{C_C}, \frac{W_e}{B_D}\right),$$

$$T_G(m) = \alpha + \frac{W_e}{B_L} + \max\left(\frac{2mP_e}{C_G}, \frac{W_e}{B_H}\right).$$

[9] AVX-512 kernel、每专家 1 个 token、128 个 token，AMX kernel、1 个 token、128 个 token；各结果的 `locality_reuse_regions` 字段给出 71 / 72 与 688 / 689 两个交点。CPU kernel 吞吐见 KTransformers 论文第 4 页与第 6 页，内存带宽与 PCIe 配置见第 10 页；A100 40GB PCIe 峰值见硬件表。

前式将输入传输、CPU 上的专家计算和输出传输三段时间相加；后式先搬一份权重，再执行 GPU 计算。公式中的 $\alpha$ 是每次交接的固定代价。输入激活先到 CPU，输出激活在计算后返回，式中的两次启动和往返载荷正对应这两条依赖边。

硬件变化可以用同一组公式估计。在 NUMA 结构下，线程读另一颗 CPU 上的内存要经过处理器间互联。论文测得这台机器的跨插槽带宽只有 125 GB/s。CPU 路径的时间取权重读取与矩阵计算中较长的一项，图 9-15 把每专家 128 个 token 时的这两项并排画出。用 AVX-512 kernel 时，计算远长于读取，内存放在哪个插槽几乎无关紧要；换成 AMX kernel，计算缩短到略长于同插槽读取，一旦跨插槽读取，读取反而成了较长的一项。就地计算是否划算，由当前的主要瓶颈决定：读取受限时提高主存带宽有用，计算受限时提高矩阵吞吐有用。量化格式同时改变这两项：权重字节数减少，矩阵 kernel 的有效吞吐也随之改变。[10]

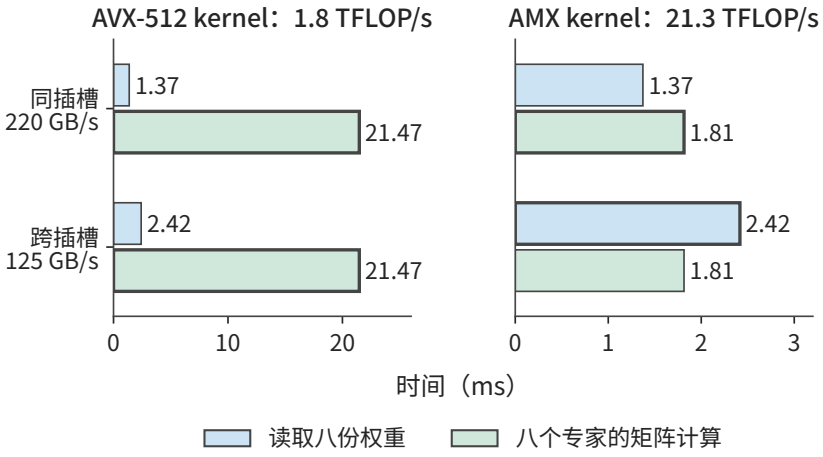



图 9-15：八个 Qwen3-235B-A22B 专家各处理 128 个 token 时，单颗 Xeon Platinum 8452Y 上的两项时间：读取八份 BF16 权重共 288 MiB，矩阵计算共 38.7 GFLOPs。粗框标出较长的一项，即这条路径的耗时下界。左右两图横轴刻度不同；AVX-512 与 AMX kernel 的算力、同插槽与跨插槽带宽均为论文实测值。

### 9.3.3 容量、并发与专家复用对服务能力的影响

能否装下权重和能否及时完成请求，是两个不同的问题。Qwen3-235B-A22B 每层 128 个 BF16 专家共占 4.5 GiB。若每层固定 32 个专家驻留 GPU，单层为 1.125 GiB，94 层合计约 106 GiB。即使把一张 A100 80GB 的全部 80 GB（约 74.5 GiB）显存都给专家，也无法容纳这组常驻专家；还需在多卡间切分，或减少每层驻留数量。把每层驻留数减到 16，这部分权重降到约 53 GiB，但也把更多专家的执行交给了主存一侧。容量选择由此直接改变请求的执行路径。

一个 token 选择八个专家，并不意味着一个 batch 也只访问八个专家。第 6.3.2 节已经算过这批分派：64 个 token 各选八个专家共 512 次分派，均匀覆盖 128 个专家时每专家处理四行，集中到八个专家时每专家处理 64 行，两种情况下专家矩阵的有效计算量都是 $512 \times 2 \times 18874368$，约 19.3 GFLOPs；若批内每份专家权重只读取一次，权重读取则是 4.5 GiB 与 288 MiB，相差 16 倍。在例 9.3 的 220 GB/s 主存上，单看权重读取分别约为 22.0 ms 与 1.37 ms：计算量相同，读取时间却相差一个数量级。

总专家数决定需要安排多少权重，分派次数决定这一批的有效矩阵计算量，批内专家集合决定需要访问哪些权重。图 9-14 已说明，输入行数决定 CPU 与 GPU 哪一侧更快。还要补上另一个量：一批请求究竟会访问多少个专家。知道该数量，才能将单专家的结果扩展到整个 batch。

除完全均匀和完全集中这两种情况外，还可以分析随机路由平均会访问多少个专家。按第 6.3.2 节的推导，

[10] KTransformers 公开实验记录。教程平台为双 6454S + 4090，论文平台为双 8452Y；Expert Deferral 的公开结果包含质量提升与下降两种情况。

每个 token 独立、均匀地从 $E$ 个专家中选择 $K$ 个不同专家时，批内 $M$ 个 token 下的活跃专家数期望值为

$$\mathbb{E}[U] = E\left[1 - \left(1 - \frac{K}{E}\right)^{M}\right].$$

代入 $E = 128, K = 8, M = 64$，期望约 126 个专家，理想权重读取约 4.43 GiB，接近上述均匀覆盖情形。随着 batch size 继续增长，被访问的不同专家数最多到 128 就不再增加，分派次数却继续线性增加，新增的 token 于是越来越多地复用已经读入的权重。

这一分析也说明，配备大容量主存和少量 GPU 的服务器，更适合每个专家输入行数较少的负载。DeepSeek V4-Flash 公开的单张 RTX 5090 配置要配至少 200 GB 主存，Kimi K2 采用 Q4_K_M 分组量化的配置（以 4 比特为主，部分张量采用更高位宽）需要约 600 GB 主存。主存解决了总权重的容纳问题；请求的 batch size 增大后，瓶颈仍可能从权重读取转到 CPU 矩阵计算。图 9-14 的交点正对应这种变化：增加并发，既摊薄读取权重的代价，也逐渐耗尽 CPU 计算能力。[11]

### 9.3.4 跨机 AF、专家分离与 micro-batch 流水

前两小节的分工仍在同一台服务器的 CPU 与 GPU 之间。在 MoE 中，还可以把路由专家放到独立的专家资源池，由保留注意力与 KV 的 worker 发送激活并接收专家结果。这称为**专家分离**（Expert Disaggregation），也常叫 EP 分离；本书用“专家分离”指部署边界，用 EP 指专家并行度。专家分离是本节注意力与专家计算分工的一种形式，具体系统还要说明共享专家、路由器和前后投影分别放在哪一侧。

以第 7 章的两台 HGX H100 服务器为例，考察一层 MoE 在专家分离下如何执行（图 9-16）。服务器 A 的四张卡 A0–A3 执行注意力，保存各自请求的 KV；服务器 B 的四张卡 B0–B3 各持有四分之一的路由专家。一层之内依次发生四步。第一步，A 卡算完注意力，路由器为每个 token 选出 8 个专家。第二步是第 6.2.6 节介绍的 dispatch：A 卡把每个 token 的隐状态按所选专家的所在卡分组，发往对应的 B 卡。四张 A 卡都要向四张 B 卡各发一份数量不等的数据，构成一次 All-to-All。第三步，B 卡收到各来源的输入后，按专家重新排列输入行，执行专家矩阵乘。第四步是 combine：专家输出沿原路送回 token 所在的 A 卡；A 卡收齐一个 token 的 8 份输出，按路由权重加权求和，才能进入下一层的注意力。

图中有两处等待。B 卡通常要等各来源的输入到齐才开始计算，A 卡要等一个 token 的所有专家输出都返回才能求和。一张 B 卡收得多、算得慢，就会同时推迟自身的计算与返回，以及所有等待其结果的 A 卡。第 9.4 节讨论的 skew，正是经过这两处等待影响整层耗时。

**大 EP** 则指一个专家并行组跨越较多的卡。大 EP 让每张卡只保存少量专家，并汇聚多个 attention worker 的输入，使专家获得足够大的矩阵 batch。大 EP 不必采用独立专家池：同一张卡也可以同时执行注意力和自己持有的专家；独立专家池也不必只有一个大 EP 组。DeepSeek-V3/R1 的公开推理报告分别给出 prefill EP32 与 decode EP144（专家并行度分别为 32 与 144），并采用冗余专家（为高负载专家额外放置的副本）和多类负载均衡；这说明大 EP 确有实际用途，但这组规模本身不能证明注意力与专家已经分到两套设备上。[12]

扩大 EP 并不自动扩大每个专家的 batch size。若本批有 $n$ 个 token、每 token 选择 $k$ 个专家、共有 $E$ 个逻辑专家，均匀路由下每专家平均只有 $nk/E$ 行。例如 $n = 1024, k = 8, E = 256$ 时为 32 行；只把同样的 256 个专家从 32 张卡摊到 128 张卡，该平均值仍是 32，减少的只是每卡的专家数。汇聚更多输入才能增大每个专家的 batch，却也会扩大通信范围与同步范围：图 9-16 中的两处等待要覆盖 EP 组里的每一张卡。每卡专家数变少还会让卡间 skew 变大，第 9.4.1 节用同一组 $n, k, E$ 计算其如何随 EP 增大。

[11] DeepSeek V4-Flash 与 Kimi K2 配置记录。表中并发上限取自部署配置。

[12] DeepSeek-V3/R1 推理系统工程报告，大 EP、三类负载均衡及通信精度；DeepEP V2 阅读快照。来源与版本边界。

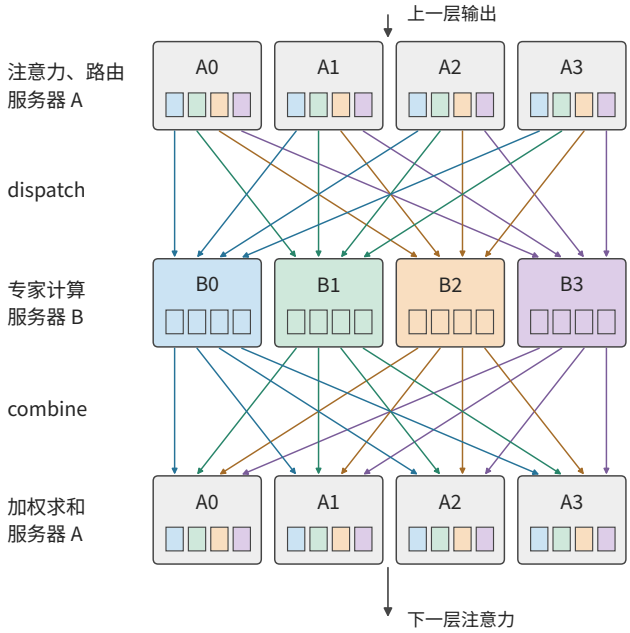



图 9-16：一层 MoE 在两台服务器间的执行顺序，时间从上到下。A0–A3 执行注意力与路由，B0–B3 执行专家。小方块表示一张 A 卡发往一张 B 卡的一组输入，颜色标出它去往哪张 B 卡；dispatch 把各色方块送到同色的 B 卡，combine 再把结果送回原来的 A 卡。两次交换都是 4×4 的 All-to-All。

专家池可以独立增加副本、选择不同算力 / 内存配比，并从多个 attention worker 汇聚任务。代价是原本可以在本地执行的专家输入也要跨池传送，网络成了两池共享的资源。只有多个独立 EP 组各保存一套完整的专家集合，它们才是可以分别接收任务的服务副本；只添加保存部分专家的卡，并不能让每条请求随意绕开繁忙的专家。共享专家池的队列、显存、模型版本和接纳策略要当作一个整体来管理。

把 AF 从同一台服务器扩展到跨服务器后，每次传输的启动时间、链路带宽以及各节点开始执行的时间差都会有更大影响。MegaScale-Infer 是采用注意力与专家计算分离的分布式推理系统。该系统让 micro-batch 在注意力节点与专家节点之间交错执行，形成交替的流水：一个 micro-batch 执行注意力时，另一个 micro-batch 执行专家计算。理想情况下，若两个阶段每个 micro-batch 需要 $t_A, t_F$，忽略交接，处理 $q$ 个独立 micro-batch 的双阶段流水需要 $t_A + t_F + (q-1)\max(t_A, t_F)$。其中首个 micro-batch 花费 $t_A + t_F$，之后每隔较慢阶段的一个服务周期完成一个 micro-batch。跨层时，返回激活将上一层的结束连接到下一层的开始。

MegaScale-Infer 的实验为这种部署提供了实际证据。其 2025 年论文使用 Mixtral-8×22B、DBRX 与 317B 的 Scaled-MoE，权重、激活和 KV 均为 BF16，生产请求的输入 / 输出长度中位数为 571 / 159 个 token。评估约束是 TPOT 不超过 150 ms；在八个节点、每节点八张 80 GB Ampere GPU 的同构集群上，Scaled-MoE 的每 GPU decode 吞吐达到 NVIDIA 推理框架 TensorRT-LLM 的 1.90 倍。这一结果包含分离放置、并行选择、流水和通信库的共同效果，不能当作“只开启 EP 分离”的通用加速比。论文也明确按专家热度配置冗余副本并最小化最忙节点的成本，说明独立专家池仍要处理 skew。[13]

用具体例子计算这条流水：设注意力与专家阶段每个 micro-batch 分别需要 2 ms、3 ms。四个 micro-batch 依次执行共需 20 ms，任一时刻只有一个节点在工作；理想双阶段流水只需 $2 + 3 + 3 \times 3 = 14$ ms，节省 6 ms（图 9-17）。若拆小 micro-batch 后每阶段的实际服务时间变长，或者无法与计算重叠的往返通信超过 6 ms，这项收益便消失。因此，通信和计算效率下降带来的额外耗时合计必须小于 6 ms，流水执行才更快。[14]

[13] Zhu 等，MegaScale-Infer，arXiv:2504.02263v1，§6 的 Load balance、§7.1–7.2 的实验条件与吞吐；CRAFT，MLSys 2026，摘要、§3–5；Demystifying the Mixture of Experts Serving Tax，MLSys 2026，§3–5。原文摘取、阅读范围与数字适用条件见本次研究记录。

[14] 跨机执行与框架演进及AF 扩写依据。

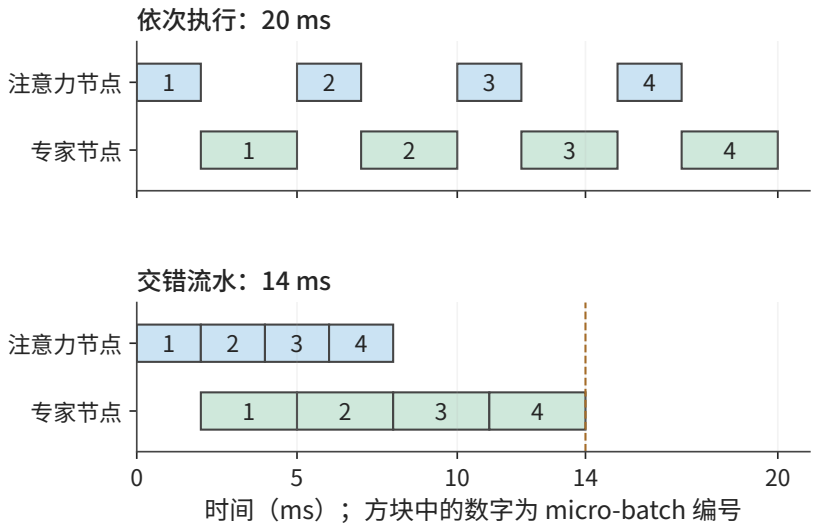



图 9-17：四个 micro-batch 在注意力节点与专家节点间执行，每个 micro-batch 注意力 2 ms、专家 3 ms，忽略交接。上图依次执行，两个节点轮流空闲；下图交错执行，micro-batch 1 做专家计算时 micro-batch 2 做注意力。流水稳定后，专家节点连续工作，每 3 ms 完成一个 micro-batch。

**例 9.4　注意力与前馈网络逐层交接会增加多少通信开销？** 在 Qwen3-8B 的示例部署中，把 36 层稠密前馈网络放在另一侧，每层发送一个完整 BF16 隐状态向量，并返回同样大小的结果。单向传输一次的数据量为 $4096 \times 2 = 8192$ bytes；一步 decode 合计 72 次、576 KiB。沿用例 9.1 的 25 GB/s 网卡，纯载荷约 23.6 µs，72 次 5 µs 启动合计 360 µs，串行交接约 0.384 ms。

这段时间短于例 9.1 中的一次 PD 交接，但两者对应的工作范围不同：PD 每个请求只交接一次，AF 每步 decode 都要交接。8192 个输入 token 的 prefill 若也按整批稠密 AF 交接，激活载荷为 4.5 GiB，约需 193 ms 纯传输，再加启动。PD 的一次搬移随上下文长度增长，AF 的逐层往返还随生成步数反复发生；前者先受大块 KV 搬移的影响，后者更容易受小数据量传输的启动开销和操作间依赖的影响。

AF 的逐层交接不随 KV 表示改变：它传的是 4096 维隐状态，与 KV 头数或潜变量宽度无关，一步 decode 仍是 72 次、576 KiB。改变的是 PD 一侧。令一次 PD 交接与一步 AF 交接的串行时间相等，

$$\frac{V_{KV}}{B} + \alpha = \frac{V_{AF}}{B} + 72\alpha \quad \Longrightarrow \quad \alpha^{*} = \frac{V_{KV} - V_{AF}}{71B},$$

其中 $V_{AF}$ 是一步 AF 交接的总载荷（576 KiB），$B$ 是链路带宽。启动时间低于 $\alpha^{*}$ 时，一步 AF 交接比一次 PD 交接快；高于 $\alpha^{*}$ 时，72 次启动的累计开销已超过整份状态的传输。GQA 状态在 25 GB/s 下 $\alpha^{*} \approx 680$ µs，50 GB/s 下为 340 µs；紧凑 MLA 状态分别为 324 与 162 µs。状态越小、链路越快，留给每次启动的预算越少。5 µs 的启动远低于这四个临界值，单看一步交接，AF 都更快。放到整条请求上比较，结论则不同：1024 步 AF 交接累计约 393 ms，是 GQA 一次 PD 交接 48.3 ms 的 8.1 倍，是紧凑 MLA 一次 PD 交接 23.0 ms 的 17 倍。PD 只交接一次，MLA 把这一次的代价减半；AF 每步都要交接，隐状态宽度不变，没有节省任何字节。输出越长，差距越大。

下一节考察 MoE 中的激活传输：每个输入可能发往多个执行位置，完成时还要等它选中的各个专家都返回结果再汇合。

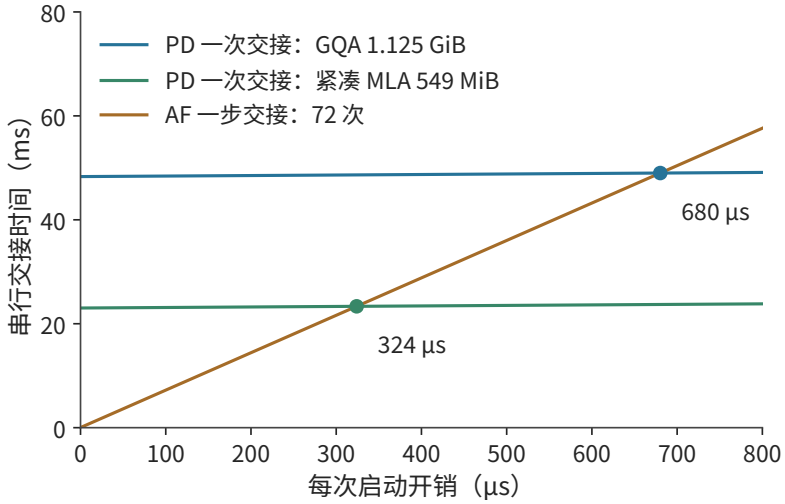



图 9-18：一次 PD 交接与一步 AF 交接的串行时间随每次启动开销的变化，25 GB/s 链路。PD 只启动一次，斜率为 1；AF 一步 72 次启动，斜率为 72。两条 PD 线分别对应 GQA 状态 1.125 GiB 与紧凑 MLA 状态 549 MiB，与 AF 线的交点即临界启动时间 680 与 324 µs；50 GB/s 链路下交点移到 340 与 162 µs。

## 9.4 专家放置、复制与负载均衡

### 9.4.1 大 EP 中的计算与通信 skew

上一节中，集中复用减少了权重读取。但这些热点专家如果都放在同一张卡上，其他卡便无法分担。本节继续跟踪同样的 512 次分派，考察各张卡何时完成计算。即使全系统的总计算量不变，最忙的那张卡也可能让整层推迟完成。对卡 $r$，记矩阵计算量为 $F_r$、需要读取的权重字节数为 $W_r$，有效算力和带宽分别为 $C_r$、$B_r$。该卡完成计算与权重读取的时间至少为 $\max(F_r/C_r, W_r/B_r)$；合并各卡的结果时，必须等耗时最长的卡完成。

把上一节的 512 次分派放到八张卡上，可以看到集中路由的另一面。若各卡恰好分到 64 次任务，每卡处理约 2.42 GFLOPs；若八个热点专家都位于一张卡，该卡处理全部约 19.3 GFLOPs。八张卡取第 7 章的 HGX H100 服务器，每张 H100 SXM 按 BF16 稠密峰值 989.4 TFLOP/s 的 50% 计，即 494.7 TFLOP/s。只看计算，最忙卡的服务时间从约 4.9 µs 增到 39.1 µs，为原来的八倍。

图 9-19 中，横条的长度表示各卡完成计算所需的时间。所有结果合并后才能进入下一层，因此决定完成时刻的是最长的一条，而不是八条的平均长度。

集中路由将需要读取的权重从 4.5 GiB 减少到 288 MiB，却可能把最忙卡的计算量增至原来的八倍。前者有利于读取受限的执行，后者不利于计算受限的执行。均衡策略因此要先确定瓶颈所在，再决定是追求复用还是分散工作。

即使总工作量不变，大 EP 也会放大这种等待。沿用第 9.3.4 节的 $n = 1024, k = 8, E = 256$（DeepSeek-V3 每层也是 256 个路由专家、每 token 选 8 个）：一批共 8192 次分派，平均每个专家 32 行。设其中一个专家较热，收到 128 行，是平均值的 4 倍；其余 8064 行均分给另外 255 个专家，每个约 31.6 行。把 256 个专家按编号均分到 EP 组的各张卡，热点专家所在的卡要处理它自己的 128 行，再加上同卡其他专家的行。EP8 时每卡 32 个专家，热点卡约 1108 行，只比每卡平均的 1024 行多 8%；EP32 时每卡 8 个专家，热点卡约 349 行，比平均的 256 行多 36%；EP256 时每卡只剩这一个专家，热点卡 128 行，是平均 32 行的 4 倍。同一个热点专家，在小 EP 中被同卡的其他专家冲淡，在大 EP 中独占一张卡，专家之间的不均原样变成了卡之间的不均（图 9-21）。

一张卡的行数同时决定三项工作：dispatch 阶段要接收的字节数，combine 阶段要发送的字节数，以及专家

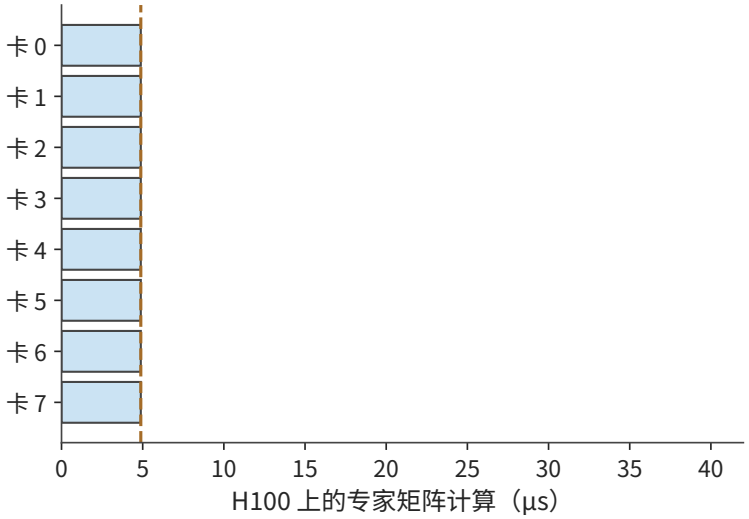



图 9-19：总计 512 次专家分派均分到八张卡，每卡 64 次；虚线表示全部计算结束、可以汇合的时刻。

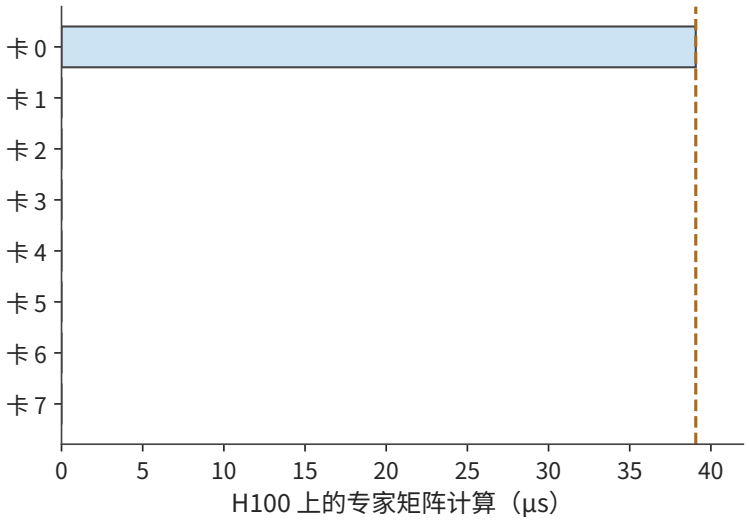



图 9-20：512 次全部落在卡 0，其他卡空闲。相同总计算量，需要等待卡 0 完成；两图使用相同时间尺度。

计算受算力限制时的矩阵时间。按每行 8 KiB、每卡 50 GB/s 网卡计，EP256 的热点卡要接收 1 MiB，约 21 µs，一张平均的卡只收 256 KiB，约 5.2 µs。若 batch 较小、专家计算受权重读取限制，一张卡的矩阵时间主要取决于它要读几份专家权重，与行数关系不大，但两次交换的字节数仍与行数成正比。从 EP8 到 EP256，每卡平均行数缩小到 1/32，最忙卡的行数却只从 1108 降到 128，约为原来的 1/8.7。整层按最忙的卡计时，EP256 中一张平均的卡在这一阶段有四分之三的时间在等待。

没有热点时，EP 增大同样会放大 skew。设每个 token 独立、均匀地随机选择 8 个专家，各专家的行数仍会在 32 附近随机波动。一张卡上的专家越多，这些波动越能相互抵消；卡越多，一批中出现一张特别忙的卡的机会也越大。按固定随机种子模拟 1000 批，EP8 的最忙卡平均只比每卡平均多 4%，EP256 多 52%（图 9-22）。两种效应来自同一个原因：EP 越大，每卡上可以相互抵消的专家越少，要等待的卡却越多。DeepSeek 公开的 decode 部署采用 EP144，每张卡只放 2 个路由专家，位于曲线右端附近；它为此增加 32 个冗余专家，并按负载调整副本，这正是第 9.4.2 节讨论的手段。[15]

[15]EP 规模与最忙卡负载的固定输入、计算脚本、复算结果。热点情形为精确分数，随机情形固定随机种子；只统计有效行数，不含 padding、

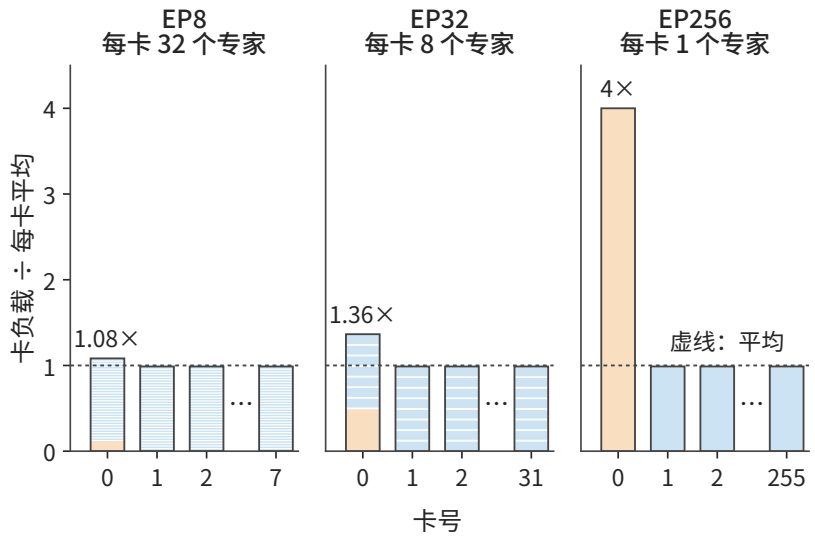



图 9-21：同一个热点专家在三种 EP 规模下造成的卡间 skew。256 个专家，1024 个 token 各选 8 个，专家 0 收到平均值 4 倍的 128 行。每根柱是一张卡，柱内每一段是这张卡上的一个专家，橙色为热点专家；纵轴为该卡行数除以每卡平均行数。每幅图只画卡 0、1、2 和最后一张卡，其余各卡与卡 1 相同。

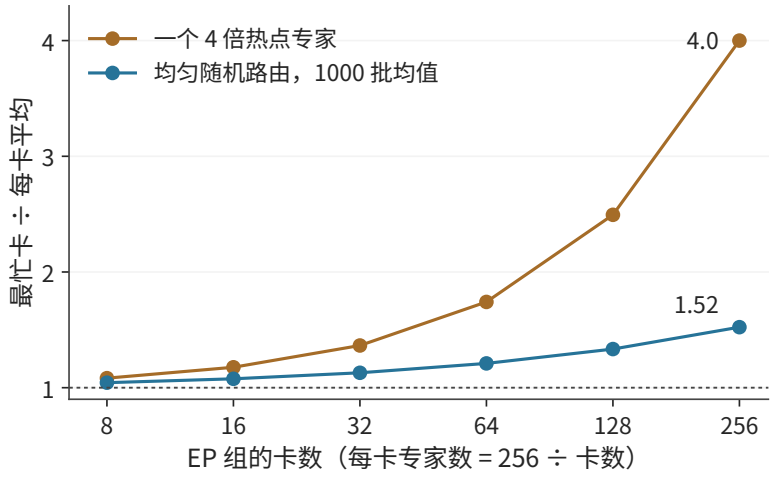



图 9-22：最忙卡行数与每卡平均行数之比随 EP 组卡数的变化。橙线为一个 4 倍热点专家，不含随机波动，可以直接手算；蓝线为均匀随机路由，是固定种子模拟 1000 批的平均值。256 个专家按编号均分，不设副本；横轴为对数刻度。

通信量也取决于 token 最初在哪张卡上。在第 9.1 节的 TP2×EP4 布局中，各 EP 组处理同一批请求，每个 EP 组的成员已经持有相同的隐状态输入，跨 EP 组的 dispatch 可以为零，结果通过部分和归约合并。另一种布局让各张卡持有各自不同的输入 token，这时就要把激活 dispatch 给远端专家，图 9-16 就是这种情形。输入已复制时，通信集中在结果合并；输入分属不同的卡时，执行前还多出一次远端 dispatch。通信量由 token 的起点、专家的位置与结果的归属共同决定。

大 EP 中的 skew 至少要分三个层次观察。逻辑专家的选择频率决定每个专家的有效行数；专家放置与副本选择把这些行数变成每卡负载；token 的来源、目标和网络路径再把每卡负载变成每个方向的流量。专家热度不均未必造成卡间不均，卡间计算均匀也未必让共享链路均匀。第 6.3.2 节的批内复用解释权重读取，本节继续核算逐卡计算、通信与完成时间。

设 $a_{ij}$ 是来源 $i$ 发给专家组 $j$ 的 token—专家分派次数，每份输入与输出分别为 $d_D, d_C$ bytes。在按分派传

权重读取与通信。DeepSeek decode 部署的 EP144、32 个冗余专家与每卡 2 个路由专家见 DeepSeek-V3/R1 推理系统工程报告。

送、无去重与预归约的方案下，

$$V_{ij}^{D} = a_{ij} d_D, \qquad V_{ji}^{C} = a_{ij} d_C, \qquad m_j = \sum_i a_{ij}, \qquad s_{\text{expert}} = \frac{\max_j m_j}{nk/R}.$$

这里 $R$ 是接收专家组数，$\sum_j m_j = nk$。$s_{\text{expert}} = 1$ 表示组间有效任务均衡；同型专家、相同精度且忽略 padding 时，它才近似等于计算量 skew。对通信阶段 $X \in \{D, C\}$，把矩阵按发送方、接收方和物理割集统计，有

$$T_X \geq \max\left( \max_i \frac{\sum_j V_{ij}^{X}}{B_i^{\text{send}}}, \max_j \frac{\sum_i V_{ij}^{X}}{B_j^{\text{recv}}}, \max_{\mathcal{C}} \frac{V_{\mathcal{C}}^{X}}{B_{\mathcal{C}}} \right).$$

其中 $V_{\mathcal{C}}^{X}$、$B_{\mathcal{C}}$ 分别是穿过物理割集 $\mathcal{C}$ 的载荷与链路带宽。这是载荷传输下界，不含启动、排队与数据晚到；同一物理方向的并发流量需要相加，不能把多条流各自除以一份完整带宽后假定同时完成。带宽 skew 在这里指各张卡、各条链路的需求或占用不均，硬件的额定带宽并没有变。

**例 9.5　总通信量相同，为什么热点让两次交换都变慢？** 沿用第 7.2.3 节的 1024 个 token、每 token 选 8 个专家与 8 KiB 向量，放到图 9-16 的两台 HGX H100 服务器上：四个来源是服务器 A 上执行 attention 的四张卡，四个专家组各由服务器 B 上的一张卡执行，全部分派都跨服务器。每张卡经自己的 400 Gbit/s ConnectX-7 网卡收发，每方向 50 GB/s；交换网络在两台服务器之间每方向可承载 $4 \times 50 = 200$ GB/s。四个来源各发送 2048 份，dispatch 总计 64 MiB，combine 也是 64 MiB。均衡时每组接收 2048 份，即 16 MiB；热点分布为 $[5120, 1024, 1024, 1024]$ 时，组 0 接收 40 MiB，另外三组各 8 MiB。两种分布的总通信量和总有效计算量相同，专家 skew 却从 1 变为 2.5。

每份专家计算按 Qwen3 MoE 示例的 $6 \times 4096 \times 1536$ FLOPs 计，每张 H100 的有效算力取 494.7 TFLOP/s。暂不计权重读取、padding、启动与排队，得到下表。最后一列仅适用于三个阶段之间设整批屏障的执行，是三个下界之和；有重叠时须另算关键路径。[16]

| 分布与路径 | dispatch 下界 | 最忙卡计算下界 | combine 下界 | 阶段屏障总下界 |
|---|---|---|---|---|
| 四组均衡，跨服务器 | 0.336 ms | 0.156 ms | 0.336 ms | 0.827 ms |
| 热点组，跨服务器 | 0.839 ms | 0.391 ms | 0.839 ms | 2.068 ms |
| 四组均衡，两池间只剩一条 400 Gbit/s 链路 | 1.342 ms | 0.156 ms | 1.342 ms | 2.841 ms |
| 热点组，dispatch 改为 FP8 | 0.419 ms | 0.391 ms | 0.839 ms | 1.649 ms |
| 热点组，两池同在一台 HGX（NVLink） | 0.093 ms | 0.391 ms | 0.093 ms | 0.577 ms |

跨服务器时，两次交换占阶段屏障总下界的八成，最忙卡的计算只占两成：H100 算一份专家任务约需 76 ns，网卡传一份 8 KiB 激活却要约 164 ns。热点同时延长接收、计算与返回，三段都变为原来的 2.5 倍。把热点分散后，若两池之间只剩一条 400 Gbit/s 链路，仍需每方向至少 1.342 ms，说明增加专家卡不能替代跨池带宽。只把 dispatch 改为 FP8，也不会同比缩短 BF16 combine 或专家计算；表中假设量化不改变计算时间，并忽略尺度元数据，以便分别观察这几项影响。把两池放进同一台 HGX，每方向 450 GB/s 的 NVLink 把两次交

[16] 大 EP 与专家分离 skew 的固定输入、计算脚本、复算结果。网卡与 NVLink 带宽见 HGX H100 数据手册、NVIDIA H100 规格与 ConnectX-7 数据手册，H100 算力取硬件表峰值的 50%；计算输出为所列执行模型的下界，第 9.4.3 节两个 micro-batch 的流水与打平倍数也由同一结果给出。

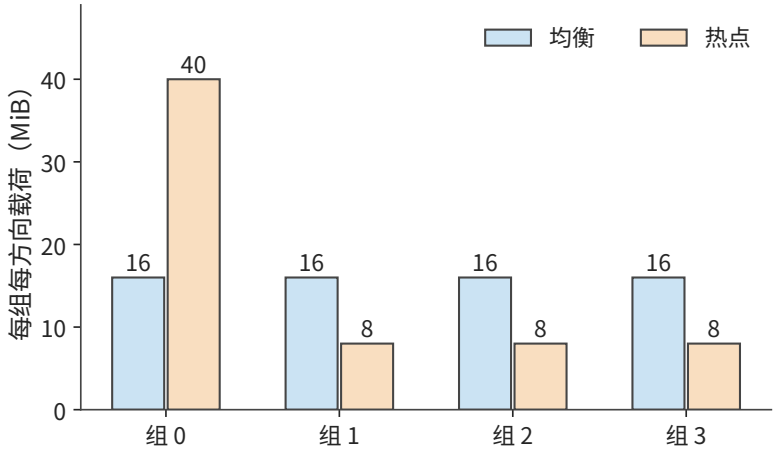



图 9-23：四个专家组在 dispatch 阶段接收、combine 阶段发送的载荷。均衡与热点分布总计均为每方向 64 MiB；热点组在两个阶段都承担 40 MiB。柱高表示字节需求，不表示测得的瞬时带宽。

换都压到 0.093 ms，热点卡 0.391 ms 的计算就成了最长的一段；这时减少热点卡的计算比再提高带宽更有效，下一节的专家副本针对的正是这种情形。

### 9.4.2 专家副本、放置与动态调整

图 9-20 中集中在少数卡上的工作提示了改进方向：把热点专家的副本放到空闲卡上，让这些卡也能处理热点任务。专家复制让同一个逻辑专家拥有多个物理副本，调度器可以把任务分给这些副本。只要副本的权重相同、路由权重用得正确、结果合并无误，复制就不必改变每个 token 选分数最高的 k 个专家这条规则（top-k）；修改专家选择规则会改变模型行为，两者要分开评价。vLLM、SGLang 的专家负载均衡机制（EPLB）根据路由统计，调整各逻辑专家的副本位置和任务分配。

专家副本占用显存，也会挤占 KV 和缓冲。每卡每层增加一个 36 MiB 专家，94 层共需 3384 MiB，约 3.30 GiB；若只为八个最拥挤的层各增加一个副本，则只需 288 MiB。两者相差近 12 倍。CRAFT 是研究专家副本与显存分配的服务系统，要解决的正是各层显存如何分配的问题：把空间花在最能缩短等待的层，而不是机械地给每层同样多的副本。[17]

**例 9.6　热点专家副本需要多少批才能收回复制成本？**八张卡是一台 HGX H100 服务器，一批 128 个 token 都选中卡 0 上的八个热点专家，共 1024 次分派。把其中七个专家各复制一份到卡 1 至卡 7，共 252 MiB；七张接收卡各增加 36 MiB，都能放入 64 MiB 的额外可用空间。每张 H100 按峰值的 50% 计，有效算力 494.7 TFLOP/s、显存带宽 1675 GB/s。卡 0 按 tile 调度，每批要从显存读写约 921 MB，受显存带宽限制；复制后卡 0 只剩 576 次分派，每批从约 0.550 ms 缩短到 0.309 ms，节省约 0.240 ms。七份副本都从卡 0 经 NVLink（每方向 450 GB/s）依次发出，每份另计 5 µs 启动，复制约需 0.622 ms。按未经四舍五入的执行时间，节省时间超过复制时间所需的最少批数满足

$$N\Delta t > T_{copy}, \qquad N_{min} = \left\lfloor \frac{T_{copy}}{\Delta t} \right\rfloor + 1 = 3.$$

在同一台服务器内，副本到第 3 批就收回了复制成本：热点持续 16 批净省约 3.2 ms，持续 64 批净省约 14.8 ms。若热点专家的原副本在另一台服务器上，复制要经 400 Gbit/s 的 ConnectX-7 网卡（每方向 50 GB/s），

[17] MoE Serving Tax、CRAFT 与启动研究笔记。

准备时间增至约 5.32 ms，要到第 23 批才回本：热点只持续 16 批时反而净损失约 1.5 ms，持续 64 批才净省约 10.1 ms。图 9-24 把一次投入与逐批收益画在同一张图上。若接收卡只有 32 MiB 空间，连一份 36 MiB 专家也无法容纳，此时无论热点持续多久，都应先改变放置或释放容量。[18]

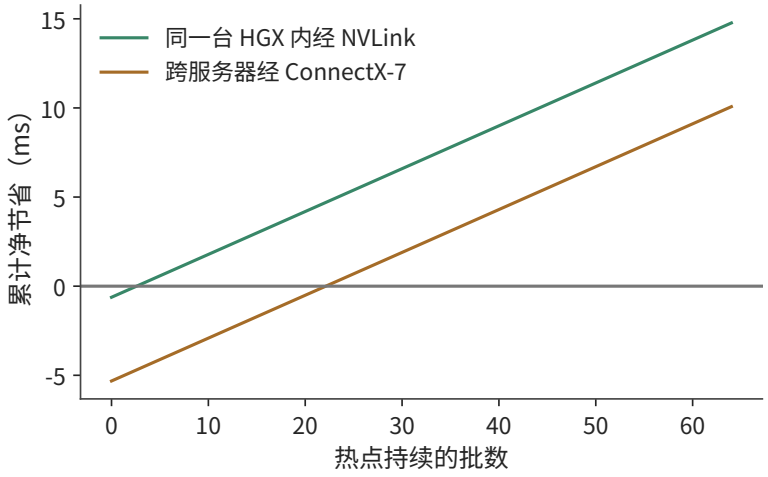



图 9-24：热点持续多久，专家复制才值得。每批节省约 0.240 ms；同一台 HGX H100 内经 NVLink 复制一次约 0.622 ms，第 3 批开始净获益；跨服务器经 ConnectX-7 网卡复制约 5.32 ms，第 23 批开始净获益。曲线使用例 9.6 中未经四舍五入的时间计算；七张接收卡各需额外 36 MiB，假设热点不变。

CRAFT 的实测进一步表明，复制收益与层有关。该研究基于 SGLang v0.4.8，在八个 AWS p4de.24xlarge 节点组成的 A100 80GB 集群上运行 BF16 的 DeepSeek-R1 和 Kimi K2，比较显存预算内按层分配专家副本与既有复制策略的效果。论文报告端到端吞吐平均达到对照的 1.14 倍、最高 1.2 倍。这说明应把副本预算优先给收益大的层，但不表示任意热点复制都能得到这一增幅；其输入按 4096 个 token 分块、输出固定为 256 个 token 等实验条件，以及按有效吞吐（goodput）定义的容量拐点，也不能直接替代线上的 p99 指标。[19]

副本策略要预测的是热点还会持续多久。跨服务器复制时，每次重新复制都需要 5.32 ms，所以频繁跟着热点迁移副本，会推迟收回复制成本的时间。同一台服务器内复制只要 0.622 ms，时间几乎构不成约束，限制副本数量的是每张卡还能释放多少显存。将容量优先分给持续拥挤的层，才能让累计节省的等待时间超过准备耗时。

均衡策略还要沿完整路径选择控制对象。来源侧要平衡 token 数与注意力的就绪时间；专家侧要平衡有效行数、实际 GEMM 时间和接收量；网络侧要检查共享出口、跨机架流量与返回方向。DeepSeek 的公开报告把 prefill、decode 与 EP 负载均衡分别处理，原因就在这里。长上下文 decode 即使每卡请求数相同，注意力的 KV 读取量仍可能不同，最终形成不同的 dispatch 发起时刻。[20]

在独立专家池中，还要控制跨来源的排队。多个 attention worker 可能同时选中同一个热点专家，即使它们各自的小 batch 看起来均衡，汇合后的流量也可能超过该专家的服务能力。若池每秒接纳 $\lambda$ 批，每批平均在专家卡 $j$ 上产生 $\mathbb{E}[F_j]$ FLOPs，则稳定运行至少需要 $\lambda\mathbb{E}[F_j] < C_j$；每条链路 $\ell$ 还需满足 $\lambda\mathbb{E}[V_\ell] < B_\ell$。平均负载满足这些条件只是必要条件，突发和重尾输入仍会造成较长的队列。共享池有时能汇聚 batch、平滑各来源的波动，有时会放大相关的热点，不能直接假定分离会消除 skew。

热点副本应放到同时有计算余量和链路带宽余量的位置。本节的 dispatch 消息是一个 token 的隐状态向量，

[18] 同一台 HGX 内经 NVLink 复制与跨服务器经 ConnectX-7 复制的回本计算。算例固定路由与逐卡额外可用空间，按串行复制计算准备时间。H100 SXM 的 989.4 TFLOP/s 与 3350 GB/s 见硬件表，均取 50%；NVLink 每方向 450 GB/s 见 NVIDIA H100 规格，网卡每方向 50 GB/s 见 ConnectX-7 数据手册。准备耗时分别为 0.62220256 ms 与 5.31982304 ms，每批节省 402784256/1675 ns（约 0.2404682 ms）；最少 3 批与 23 批使用未经四舍五入的数值计算。

[19] Zhu 等，MegaScale-Infer，arXiv:2504.02263v1，§6 的 Load balance、§7.1–7.2 的实验条件与吞吐；CRAFT，MLSys 2026，摘要、§3–5；Demystifying the Mixture of Experts Serving Tax，MLSys 2026，§3–5。原文摘取、阅读范围与数字适用条件见本次研究记录。

[20] DeepSeek-V3/R1 推理系统工程报告，大 EP、三类负载均衡及通信精度；DeepEP V2 阅读快照。来源与版本边界。

BF16 为 8 KiB、FP8 为 4 KiB，远大于第 7.2.4 节在 50 GB/s 网卡上求出的约 0.93 KB 交点，热点卡先用尽的是带宽，而不是网卡发起请求的速率；只有逐条发送不到 1 KB 的控制消息时，才需要为网卡每秒约 5400 万项的请求发起能力一并留出余量。动态选择副本时要计入队列和路径，并保留当前 batch 使用的路由映射直至 combine 完成；迁移或回收旧副本前要排空在途任务。对热点设置接纳额度、限制单个来源的在途量，可以避免它耗尽整个池的缓冲；增加池容量时，也要相应调整接纳额度与背压。丢弃溢出 token、改变 top-k 或替换逻辑专家会改变模型行为，不能当作透明的资源均衡。

### 9.4.3 输入分发、计算与结果合并的重叠

前两小节的核算没有区分 prefill 与 decode，而分派集中对这两个阶段的影响并不相同。《Demystifying the Mixture of Experts Serving Tax》用各阶段的微基准解释这种差异：prefill 的计算不均会让最慢的卡拖住整批，decode 的专家集中却可能通过减少活跃权重和 padding 降低访存成本。这与第 6.3.2 节的复用分析一致。论文的 Mixtral / Qwen2 MoE 使用八张 A100，DeepSeek-V3 使用八张 B200，通信微基准另有 8 / 16 张 H200；这些结果不能合成一个跨集群的端到端加速比。论文提供的证据说明，路由、实际矩阵形状、通信和内存访问要一起测量。[21]

实际矩阵形状的影响主要来自补齐。Grouped GEMM 把多个专家的矩阵乘法放进同一次 kernel 执行，kernel 按 tile 大小补齐各矩阵的行数。例如八个本地专家收到 $[32, 16, 8, 4, 2, 1, 1, 0]$ 个 token，合计 64 次 token 到专家的分派，对应输入矩阵中的 64 个有效行。若每个非空专家补齐到 32 行，需要 224 行；若八个专家都按最大值补齐，需要 256 行。实际执行的行数分别为有效行数的 3.5 倍和 4 倍。这解释了为什么有效任务数相同，矩阵时间仍可能不同：硬件执行的是补齐后的矩阵，模型所需的专家计算只对应其中 64 个有效行；补齐的零行没有对应的真实 token。

补齐影响的是计算一项；三个阶段的总时长还取决于它们能否重叠。以第 9.4.1 节跨服务器的均衡分布为例：dispatch、专家计算与 combine 的下界分别为 0.336、0.156、0.336 ms，完全串行为 0.827 ms。若能均分成两个 micro-batch，各阶段时间减半，理想三阶段流水等于一个 micro-batch 三段之和再加一次最长的一段，即 0.581 ms，节省 0.246 ms。这里最长的一段是通信，流水的节奏由网卡决定。若网卡的 DMA 与同时运行的 GEMM 争用 HBM，使每个 micro-batch 的 dispatch 与 combine 各慢约 49%，同一流水就回到 0.827 ms：通信确实重叠了，整批却没有变快。反过来，专家计算要慢到原来的 3.1 倍，才会抵消同样的收益。[22]

补齐和争用正是 MFU 达不到峰值的两大来源：补齐让矩阵单元去算没有对应 token 的零行，争用让通信和计算互相拖慢。两者都出在实现层，硬件峰值并没有变；选用更贴合实际行数的 tile，错开 DMA 和 GEMM 对 HBM 的访问，就能收回其中一部分。

沿图 9-26 的时间线看，第一个 micro-batch combine 时，第二个 micro-batch 已经开始计算。第一批的 dispatch 负责填充流水，最后一批的 combine 负责收尾，这两段是流水之外的串行时间。

上述流水假设两个 micro-batch 大小均匀。真实的专家分离中，一个 token 的路径仍然是 “dispatch → 所选专家计算 → combine”，**两次通信阶段**夹着专家计算。dispatch 的数据就绪 skew 来自注意力的完成时间和发送队列；专家计算的 skew 又变成 combine 的发送就绪 skew。即使两次交换的字节数完全对称，返回阶段也可能因为数据晚到而拖长。

对 token $t$，令所选专家集合为 $\mathcal{E}(t)$，从当前层起点到专家 $e$ 的排队、分派、计算与返回的累计路径长度为 $L_{t,e}$。其合并完成时刻满足

[21] Zhu 等，MegaScale-Infer，arXiv:2504.02263v1，§6 的 Load balance、§7.1–7.2 的实验条件与吞吐；CRAFT，MLSys 2026，摘要、§3–5；Demystifying the Mixture of Experts Serving Tax，MLSys 2026，§3–5。原文摘取、阅读范围与数字适用条件见本次研究记录。

[22] 专家执行与后端实验记录。

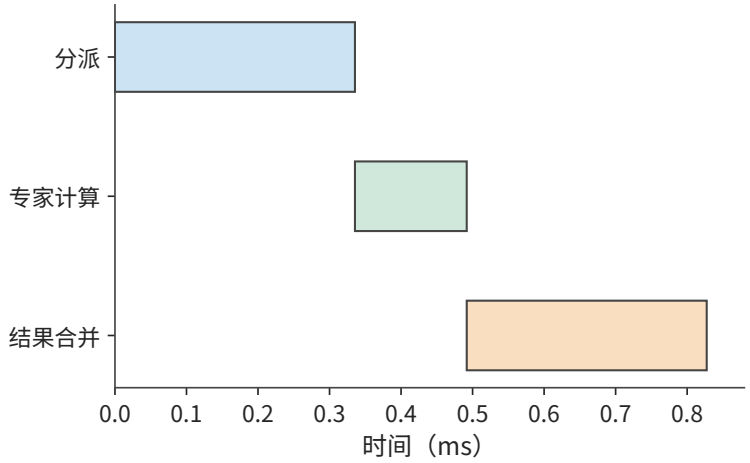



图 9-25：整批依次分派 0.336 ms、计算 0.156 ms、合并 0.336 ms，总时间 0.827 ms；数值取自第 9.4.1 节表中跨服务器的均衡分布。

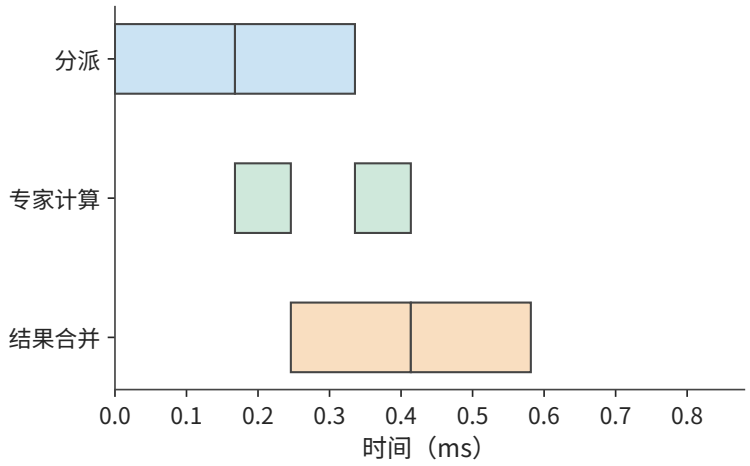



图 9-26：每个 micro-batch 的各阶段时间减半，三条轨道使用独立资源。第一个 micro-batch 计算时可以 dispatch 第二个 micro-batch，总时间降到 0.581 ms。

$$T_t = \max_{e \in \mathcal{E}(t)} L_{t,e} + T_{\mathrm{merge},t}.$$

这里每条路径都包含共享资源上的排队，不能把各阶段单独运行的时间直接代入并假定互不干扰。若某 token 的两条分支分别在 0.5 ms 和 1.4 ms 返回，它要等到 1.4 ms 后才能合并；只把快的那条分支降到 0.3 ms 不会改变它的完成时刻。逐 token 或逐块推进时，没有选中慢专家的 token 可以更早继续；整批屏障则让它们一起等。运行时是否真正支持细粒度推进、后续矩阵是否还要求凑批，决定了等待会传播多远。

这也解释了为什么不能总用 $\max(t_A, t_F)$ 预测专家池的流水周期。该式要求各阶段耗时稳定、资源独立、工作能持续供给；实际周期还受最忙专家、往返链路、返回缓冲和跨层依赖的约束。同一个专家池若为多个层或多个模型服务，必须合计它们占用的资源。第一个 micro-batch 仍要承受完整的往返延迟，提高稳态吞吐并不等于缩短单个 token 的响应时间。

dispatch 与 combine 这两次通信由通信库实现。NCCL 提供通用的集合通信与点对点原语，专用的 EP 通信库则把路由、打包、传输和合并放在一起实现。DeepEP 提供 MoE dispatch / combine 和低精度传输支持；V2 的公开接口使用 ElasticBuffer，底层采用 NCCL Gin（NCCL 由 GPU 发起网络通信的后端），并移除了 V1

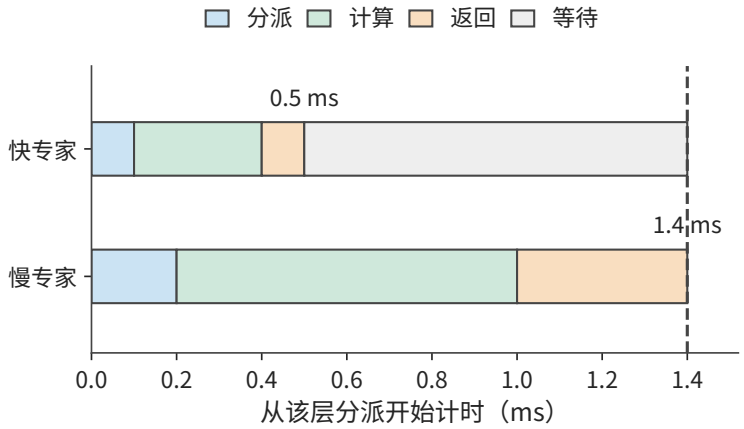



图 9-27：一个 token 选中的两条专家分支，教学时长分别为 0.5 ms 与 1.4 ms。每条都依次经过 dispatch、计算与 combine；同一个 token 要等两份结果，图中忽略本地加权合并的时间。灰色表示快分支的结果到达之后等待的时间。

中不占用 SM、直接经 RDMA 收发的低延迟 EP 通信路径。这说明库名相同也不能省略版本条件。比较实现时，应固定版本、数据形状、精度和拓扑，分别记录发送 / 接收字节数、expert GEMM 行数、padding、每阶段的就绪与完成事件，再统计整个层和整条请求耗时的 p50 / p99。单独测得的通信带宽只能解释其中一部分。[23]

复算本节时，可先保持总分派数不变，只改变专家分布；再固定分布，改变共享出口和 dispatch 精度；最后在跟踪记录中改变一个来源或专家的就绪时间。前三项由配套计算直接重现，最后一项要在执行时间线中观察等待如何传播。这样才能区分流量 skew、计算 skew 和单纯晚到造成的等待，不会把所有慢通信都归咎于网络带宽不足。[24]

## 9.5 KV 的分布、共享与请求路由

### 9.5.1 缓存标识、共享范围与可用状态

前两节靠改变执行位置和任务分配，减少了当前请求的耗时。另一种节省计算的方法，是让后续请求直接使用前一次计算留下的 KV。第 8.3 节已经说明分页、前缀匹配、共享和淘汰的机制。本节接着考虑状态分布在不同执行位置时，如何确认匹配、找到数据并完成传输。前缀缓存的价值来自避免重复执行，但“相同文本”不足以唯一确定所有可复用状态。模型与 adapter 版本、token 序列、位置索引、状态格式以及必要的编码配置都可能参与缓存标识的计算。例如，同样的 token 配上不同的 adapter，投影后会得到不同的 K、V；同样的后缀接在不同的前文后面，注意力看到的上下文也不同。缓存键需要沿前缀链标识出生成这份状态的整个计算。滑动窗口状态或递推状态则还要携带对应的 token 位置索引与更新时刻，才能从同一个状态继续。

确认两份状态可以相互替代之后，还要确定状态存放在哪里、其他实例如何读取。容量相加并不会自动形成共享缓存。两个实例各有 4 GiB 私有的主机内存缓存时，同一份 1.125 GiB 前缀会在两边各存一份，共占 2.25 GiB，而两个实例都无法使用对方那一份。若两者连接到同一个共享存储后端（接收和提供缓存对象的存储服务），一份对象就能服务两边，但每一边在生成前可能还要把它取回本地。SGLang 的分层缓存 HiCache、vLLM 的多级缓存路径和 KV 缓存管理系统 LMCache 提供了不同的组织方式；取回是否经过 CPU，决定下一节的路径时间。[25]

[23] DeepSeek-V3/R1 推理系统工程报告，大 EP、三类负载均衡及通信精度；DeepEP V2 阅读快照。来源与版本边界。

[24] 大 EP 与专家分离 skew 的固定输入、计算脚本、复算结果。网卡与 NVLink 带宽见 HGX H100 数据手册、NVIDIA H100 规格与 ConnectX-7 数据手册，H100 算力取硬件表峰值的 50%；计算输出为所列执行模型的下界，第 9.4.3 节两个 micro-batch 的流水与打平倍数也由同一结果给出。

[25] 缓存层次、路径与路由研究。

状态产生后，需要先保存并发布，其他实例才能查询、取回并使用它。路由器用到的缓存事件通常只有存储位置和缓存标识这类元数据，不带完整的 KV。GPU 上的副本被淘汰时，CPU 上的副本可能还在。**目录**保存缓存标识与位置的映射，**对象**保存实际 KV 字节；恢复时分别确认目录映射和对象数据。事件遗漏、延迟或缓存空间被释放，都可能使原先记录的位置不再可用。




图 9-28：虚线表示按标识查找对象位置。目录用于定位，实际 KV 对象用于恢复计算；路由前还需确认对象版本与可用性。

### 9.5.2 多级存储

第 8.3.4 节在一张卡上比较了保留、换出和重算。放大到整台服务器，同一份 KV 可以存放在四个位置：HBM、主机内存、本地 SSD 和远端存储池，越往下容量越大，离 GPU 也越远。以一台 DGX A100 为例：8 张 A100 80GB、2 TB 主机内存、8 块 3.84 TB U.2 NVMe SSD、8 张 200 Gbit/s 网卡，按每张 GPU 分到的一份计算。SSD 选用 Solidigm D7-P5520 3.84 TB，顺序读、写带宽最高分别为 7.1 GB/s 和 4.2 GB/s。存放的对象仍是 Qwen3-8B 的 8192 个 token 前缀，共 1.125 GiB。[26]

| 层级 | 每张 GPU 的容量 | 到 GPU 的路径 | 取回 8K 前缀 |
|---|---|---|---|
| HBM | ≤ 63.6 GB | 已在 GPU 上 | 0 |
| 主机内存 | 256 GiB | PCIe 4.0 x16，25 GB/s | 48.3 ms |
| 本地 SSD | 3.84 TB | SSD 顺序读，7.1 GB/s | 170 ms |
| 远端存储池 | 随节点数增加 | 网卡 25 GB/s，再经 PCIe | 96.6 ms |
| 对照：在 A100 上重算 | — | 按峰值的 50% 计 | 856 ms |

HBM 一栏是 80 GB 减去 16.4 GB BF16 权重后的上限，尚未扣除激活与工作区。四级的取回时间都远小于 856 ms 的重算时间，因此单就一次取回而言，状态放在哪一级都比重算快。真正的限制来自另外三个方面：缓存能否保存到下一次使用，读取能否与计算重叠，以及写入各级要付出多大代价。

**容量决定缓存能存放多久。**一张 A100 连续对无命中的 8K 请求执行 prefill，每 0.856 s 产生一份 1.125 GiB 的 KV，产生速率 $r \approx 1.41$ GB/s。如果新产生的 KV 全部写入某一级、并按写入先后淘汰，容量为 $C$ 的一级

[26] DGX A100 数据手册（8×A100 80GB、2 TB 主机内存、8×3.84 TB U.2 NVMe、8 张单端口 200 Gbit/s 网卡）与 Solidigm D7-P5520 产品简介（128K 顺序读 / 写最高 7,100 / 4,200 MB/s，5 年内 1 DWPD）。本节的容量、可存放时长、取回与写入时间、逐层预加载层数和 SSD 写入额度见 多级 KV 存储计算，运行 `python3 calculations/calc.py kv-tiers` 复算。主机内存按 2 TiB 均分给 8 张 GPU，SSD 读写按规格上限计。

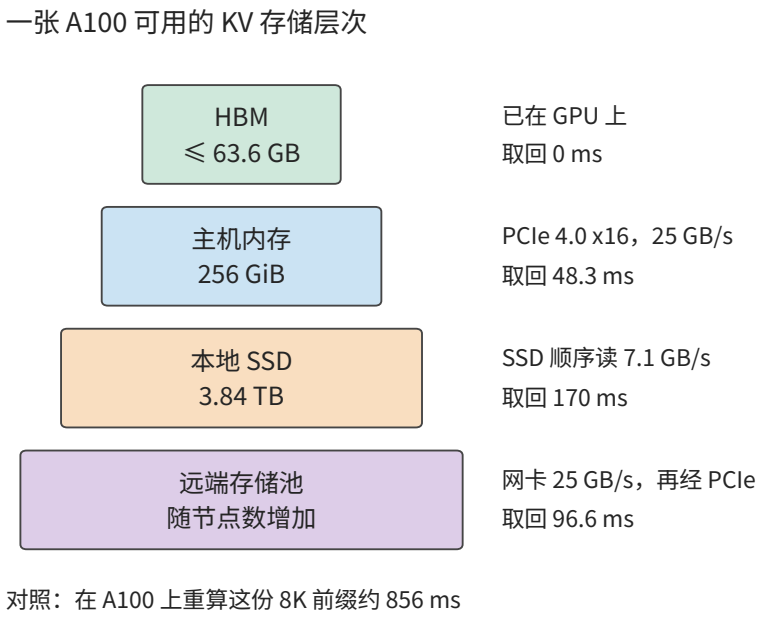



图 9-29：DGX A100 中每张 A100 分到的四级存储。方框宽度只表示容量次序；右侧是这一级到 GPU 的路径，以及取回 1.125 GiB 前缀所需的时间。远端取回先经网卡到主机内存，再经 PCIe 到 GPU，两段串行。

大约能存放最近 $C/r$ 秒内产生的 KV：HBM 约 45 s，主机内存约 195 s，SSD 约 2722 s，即 45 分钟。反过来，设同一前缀两次使用的间隔为 $T$，要让间隔 $T$ 以内再次到达的请求都能命中，这一级的容量至少要满足

$$C \geq rT.$$

阿里云基于线上请求记录（trace）的一项研究统计了两类负载的复用时间：面向个人用户的对话负载中，80% 的复用发生在 10 分钟以内；面向企业的 API 负载中，80% 的复用发生在 10 秒以内。间隔为 10 秒时只需 14.1 GB，HBM 即可容纳；间隔为 10 分钟时需要 846 GB，超过每卡 275 GB 的主机内存，需要加上 SSD 才能覆盖。[27]

$C \geq rT$ 也说明了什么时候不需要 SSD。同一研究在它的对话负载上发现，Llama3-70B 所需的缓存容量约为可用 HBM 的 4 倍；8 卡 A100 服务器配 1 TB 主机内存时，每卡分到 128 GB，已经足够，不必再增加 SSD 或远端层。差别在于 $r$：该研究按每个实例的实际请求率计算，请求也更短，单轮请求平均 973 个 token，多轮请求平均 5953 个；这里则假设 GPU 一直在对 8K 请求执行 prefill。前缀命中越多、每 token 的 KV 越小（例如 MLA），$r$ 就越低。$T$ 取决于由谁发起下一轮：人工对话的间隔以分钟计，程序调用的间隔以秒计。

**容量越大，命中率提高得越慢。**容量只能保留将来会被复用的状态，而有些状态不会再被使用。Mooncake 公开了 Kimi 线上服务一小时的采样 trace，每块 512 个 token。按最近最少使用（LRU，先淘汰最久未被访问的块）的规则淘汰时，缓存从 1000 块增加到 50,000 块，命中率从 30% 升到 50%；容量不受限制时也只有 51%。按 Qwen3-8B 换算，50,000 块约 3.77 TB，与一块 3.84 TB 的 SSD 相当。这份 trace 中超过一半的块从未被再次使用，另一些块却被访问上万次；阿里云的 trace 也同样集中，10% 的块贡献了 77% 的复用。[28]

**只取回一次，还是每一步都远程读取。**共享 KV 有三种典型用法：本地重新 prefill、从远端取回一次后在

[27] Wang 等，KVCache Cache in the Wild（USENIX ATC 2025），第 3.4 节：Trace A 为面向个人用户的对话负载，Trace B 为 API 负载；所需容量的结论针对 GQA 模型，并按每个实例的最大请求率估计。

[28] Mooncake 技术报告第 4 节与表 1（一小时采样 trace，23,608 条请求，平均输入 7590 个 token）；阿里云 trace 的集中程度见上一条注释所引论文。

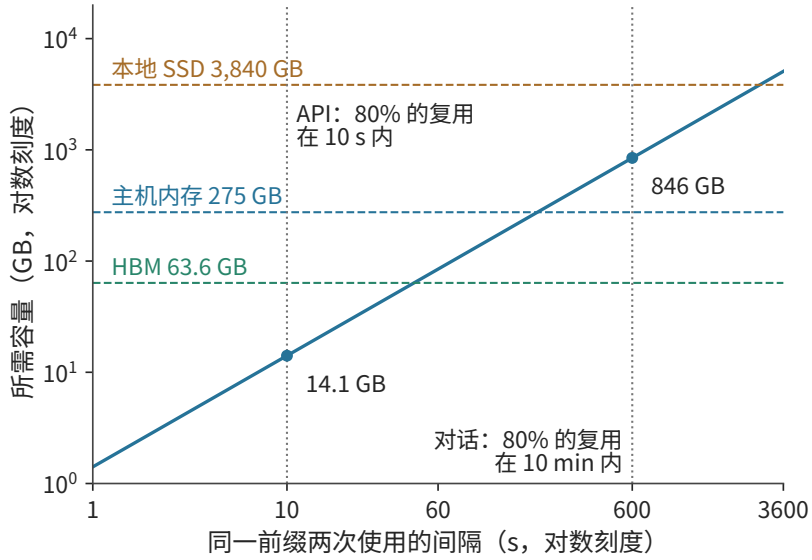



图 9-30：所需容量随复用间隔线性增长。斜线为 $C = rT$，其中 $r \approx 1.41$ GB/s 是一张 A100 连续执行无命中 8K prefill 时产生 KV 的速率；三条虚线是每张 GPU 在三级存储中的容量；两条竖线标出两类负载 80% 复用所在的时间范围。两轴均为对数刻度。

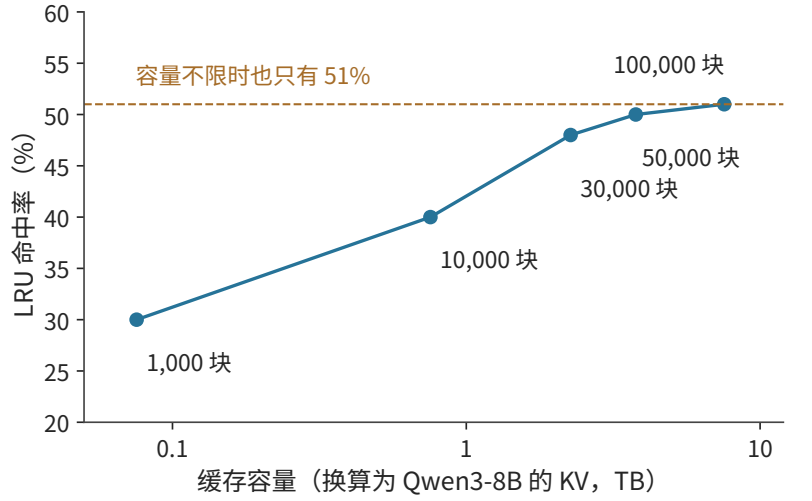



图 9-31：命中率随容量增加而趋于饱和。数据为 Mooncake 一小时采样 trace 在 LRU 淘汰下的命中率，每块 512 个 token；横轴按 Qwen3-8B 每 token 144 KiB 换算为字节。这只是采样得到的一段流量，真实服务所需的容量按流量同比例放大。

本地驻留、每一步 decode 都从远端读取。三者使用同样的上下文，传输频率却完全不同。

沿用 1.125 GiB 前缀和本章的 200 Gbit/s 网卡（25 GB/s），一次载荷取回约 48.3 ms。若 100 次 decode 调用都重新从远端读取这份不变的前缀，累计占用链路约 4.83 s。若这 100 次读取要在一秒内完成，仅这份上下文就要求约 121 GB/s，是给定链路能力的 4.8 倍。把读取与计算并行安排，也无法让 25 GB/s 的链路在一秒内传完这些字节。换成 549 MiB 的紧凑 MLA 前缀，一次取回约 23.0 ms，100 次累计约 2.30 s，一秒内完成仍要约 57.6 GB/s，是链路能力的 2.3 倍：状态减半只把倍数减半，每步都从远端读取仍然不可行。

取回一次并驻留把传输频次从“每步一次”降到“每次复用一次”，代价是占用本地 HBM。设一份状态占 $V$ bytes、在本地保存 $\tau$ 秒，占用的容量与时间之积为 $V\tau$，单位为字节 · 秒。同样保存 10 秒，对象越大，占用越多；同样大小的对象，保存越久，这一乘积越大。第 9.1.3 节的会话在工具执行 10 秒期间，GQA 状态占 11.25 GiB · s，紧凑 MLA 状态占 5.36 GiB · s，同样的本地容量能容纳约两倍的等待会话。因此可以把不活跃的会话

放到远端，会话恢复时取回，在连续 decode 期间留在本地。这样，数据传输主要发生在会话恢复和暂停时。

假定状态将在未来复用 $k$ 次，暂不考虑排队时间，也不考虑其占用空间后对其他缓存内容的挤占。保存后再取回比重新计算更快的条件为

$$T_{write} + kT_{read} < kT_{recompute}.$$

以 Qwen3-8B 的 8192 个 token 前缀为例：在 A100 上重算一次约需 0.856 秒（第 9.2.3 节的 prefill 时间），经 25 GB/s 网卡保存和取回各约 48.3 ms。复用一次时，保存加取回共约 96.6 ms，不到重算的八分之一，只要这份状态还会再使用一次，保存就是值得的。链路变慢后结论会变：带宽介于约 1.41 与 2.82 GB/s 之间时，至少要复用两次才能抵消保存的代价，越接近 1.41 GB/s 所需的次数越多；低于约 1.41 GB/s 时，取回一次就比重算还慢，复用次数越多，损失越大。10 GbE 每方向只有 1.25 GB/s，取回一次约需 966 ms；这时应该重算，或者设法把取回移出关键路径，提前完成。状态大小只进入 $T_{write}$ 与 $T_{read}$，$T_{recompute}$ 由模型的 prefill 代价决定。换成紧凑 MLA 状态，同一个网卡上写入加取回一次只要约 46.1 ms，不等式左边随 KV 字节数减少而缩小。

1.41 GB/s 这一临界带宽恰好就是上文的 KV 产生速率 $r$：取回 $V$ 字节需要 $V/B$，重算需要 $V/r$，只要链路带宽 $B$ 高于 GPU 产生这份 KV 的速率，取回就比重算快。本地 SSD 的写、读带宽都高于 $r$：写入约 288 ms，读出约 170 ms，复用一次合计约 458 ms，仍然少于重算的 856 ms。

**让读取与计算重叠。**命中之后，请求只需计算 256 个新 token，按第 9.2.3 节的约定取 A100 峰值的 50%，约需 30.9 ms。历史 KV 在主机内存中时，如果先整份读入再开始计算，共需 48.3 + 30.9 = 79.2 ms，读取比计算还长。CachedAttention 采用逐层预加载：Transformer 逐层计算，第 $i$ 层只用到第 $i$ 层的 KV，所以 GPU 计算第 $i$ 层时，PCIe 可以同时读取后面的层。Qwen3-8B 每层的历史 KV 为 32 MiB，读取一层需要 1.34 ms，而新 token 计算一层只需 0.857 ms。每层的计算都要等待读取，总时间约 49.2 ms，仍由读取决定。要进一步缩短，就需要在该请求开始执行之前先读入若干层：利用上一个 batch 的执行时间，把前 14 层（448 MiB）读入 HBM 中预留的缓冲区，其余 22 层的读取就能被计算完全掩盖，总时间降到计算本身的 30.9 ms。缓冲区要容纳的正是读取比计算多出的那部分字节：

$$S_{buf} = B\,(T_{read} - T_{new}),$$

其中 $B$ 是链路带宽，$T_{read}$ 是读取全部历史 KV 的时间，$T_{new}$ 是计算新 token 的时间。代入 25 GB/s、48.3 ms 与 30.9 ms，得到约 437 MB，略多于 13 层，按整层向上取整为 14 层。$T_{new} \geq T_{read}$ 时不需要缓冲区：新输入达到约 400 个 token，计算就能掩盖从主机内存的读取；远端存储池的串行路径需要约 800 个，本地 SSD 需要约 1400 个。CachedAttention 在 LLaMA-13B 上的测量中，1K 个历史 token、100 个新 token 时，逐层预加载让 prefill 时间缩短 35%，再设置一个 15 层的缓冲区后，缩短 61%。[29]

**利用排队时间从 SSD 预取。**从本地 SSD 读一层需要 4.73 ms，是计算一层的 5.5 倍，逐层预加载只能掩盖一小部分。请求开始执行时，状态若仍在 SSD 上，即使逐层读取，这一步也需要约 171 ms，而计算本身只需 30.9 ms。可以利用排队时间：请求仍在队列中时，调度器已经知道它需要哪份前缀，可以先把这份前缀从 SSD 读到主机内存。只要排队时间不短于 170 ms，这次读取就不在关键路径上，开始执行时再按上述方式从主机内存逐层读入。这与第 9.5.4 节的 $T_{first} = \max(Q, R) + C$ 是同一个关系：预取让状态就绪时间 $R$ 与排队时间 $Q$ 重叠。

[29]Gao 等，Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention（USENIX ATC 2024），第 3.2–3.3 节给出逐层预加载、异步保存和按队列预取与淘汰；第 4.3.2–4.3.3 节给出预加载缓冲区与命中率的测量。

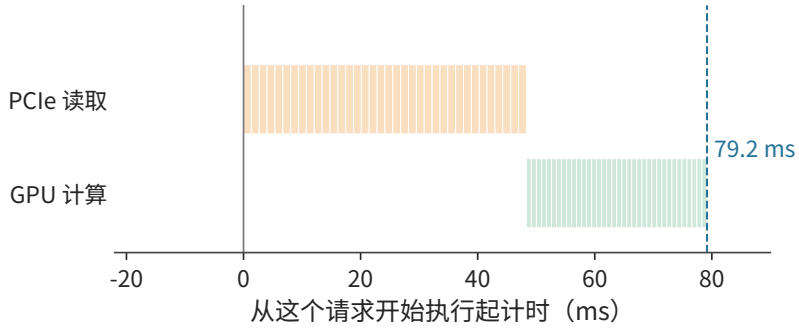



图 9-32：先把 1.125 GiB 历史 KV 整份从主机内存读入，再计算 256 个新 token，共 79.2 ms。橙色为 PCIe 读取，绿色为 GPU 计算，每一小段对应一层。

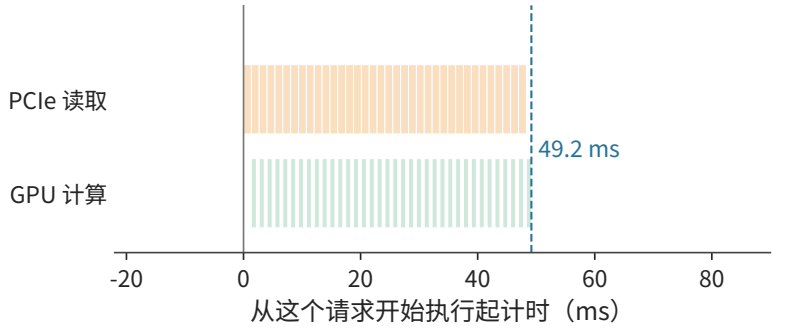



图 9-33：逐层预加载。GPU 计算某一层时，PCIe 读取后面的层；每层读取 1.34 ms、计算 0.857 ms，每层的计算都要等待读取，总时间 49.2 ms 由读取决定。

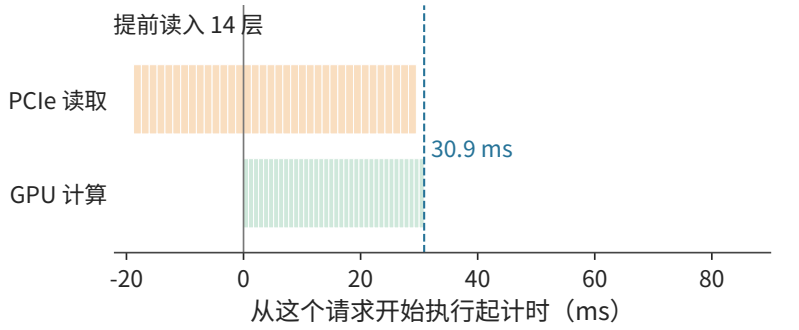



图 9-34：在这个请求开始执行之前，利用上一个 batch 的执行时间先读入前 14 层（448 MiB）；其余 22 层的读取被计算完全掩盖，总时间等于计算本身的 30.9 ms。三幅图的横轴相同。

主机内存能容纳多少份会话，决定了调度器最多能为队列中前多少个请求提前读取：256 GiB 可容纳约 227 份 8K 前缀，所以只需为队列最前面的 227 个请求预取。主机内存不足、需要换出时，也按队列判断：这些请求即将使用的状态不能换出；其余状态中，下次使用最晚的先换出。LRU 和 FIFO 只依据过去的访问，无法利用队列中即将到来的请求。CachedAttention 在 4 张 A100、128 GB 主机内存和 10 TB SSD 上回放 ShareGPT（用户分享的 ChatGPT 对话数据集）中的多轮对话：按队列预取和换出时，总命中率为 86%，其中 99.6% 以上的命中来自主机内存；LRU 与 FIFO 的命中率分别只有 58% 和 48%，来自主机内存的命中都不到 1%，几乎每次命中都要从 SSD 读取。[30]

**SSD 的写入受寿命限制。**写入同样要避开关键路径。prefill 逐层产生的 KV 可以在计算的同时写回，decode

[30]Gao 等，Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention（USENIX ATC 2024），第 3.2–3.3 节给出逐层预加载、异步保存和按队列预取与淘汰；第 4.3.2–4.3.3 节给出预加载缓冲区与命中率的测量。

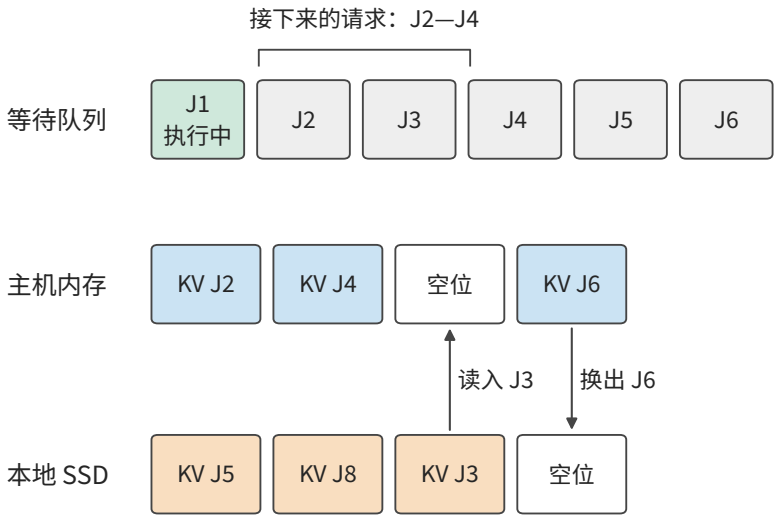



图 9-35：主机内存有四个存放位置，其中一个留作空位，用于接收读入的数据，其余三个留给队列中接下来的 J2—J4。J3 的状态仍在 SSD 上，在它排队期间读入空位；J6 排在这三个请求之后，下次使用最晚，先换出到 SSD。

每步只追加一个 token 的 KV。以 $r \approx 1.41$ GB/s 写入，只占 PCIe 带宽的 5.6%，也只占 SSD 顺序写带宽的 34%。SSD 真正的限制是写入寿命：D7-P5520 标称五年内每天可写满一次（1 DWPD），3.84 TB 的盘平均每秒只能写入 44.4 MB。如果把全部新 KV 都写入 SSD，写入量是这一额度的 31.7 倍，五年的写入寿命在不到两个月内就会耗尽。长期来看，只有约 3.2% 的新 KV 能够写入 SSD。前面两份 trace 中，复用都集中在少数块上，Mooncake trace 中一半以上的块从未被再次使用，因此写入 SSD 之前需要先做准入判断，例如只写入已经被复用过的前缀，或者尚未结束的多轮会话。按这一额度写入时，SSD 中恰好能存放最近一天产生的 KV。

容量与命中之间的这种关系在实验中也能观察到。在一组同卡双引擎实验中，12 次前缀复用机会里，4 GiB 共享 CPU 池完整命中 5 次，8 GiB 池完整命中 12 次。多出的 4 GiB 保留了七次原本会被淘汰的完整前缀；再次请求所需的状态若已经在本地，就不必重复取回。所以共享容量的收益要按“保存多久、避免几次重算、增加几次搬移”来比较。[31]

### 9.5.3 持久化、checkpoint 与部分重算

上一节假定保存下来的状态能完整取回。实例重启后，目录记录、磁盘数据和可连续复用的前缀可能并不一致，恢复过程需要重新确认哪些计算结果已经保存。持久化让状态在实例重启或任务暂停后仍然可用。完整保存减少恢复时的重算，周期性 checkpoint 减少写入但增加恢复工作，部分重算则用已保留的前缀补齐缺的部分。checkpoint 要保存继续计算所需的全部状态：完整 GQA 保存上下文 K、V；DeepSeek V4 的状态还涉及压缩结果、窗口以及未完成的压缩块。恢复时从 checkpoint 所记录的最后一次完整更新继续计算。[32]

持久化的状态按页保存，恢复时也按页读回。先考虑完整的 GQA 页。Qwen3-8B 每个含 16 个 token 的逻辑页为 2.25 MiB。若各层的 K、V 分片分别搬移，会形成许多小的连续段；若按页汇集，传输次数和所需的布局转换都不同。一个逻辑页按 36 层的 K、V 分开后有 72 个 32 KiB 的连续段；按页汇集后则是一个 2.25 MiB 的对象。前者更容易受每秒操作次数的限制，后者把更多时间花在连续载荷的传输上。

重启后的某个请求读取了 64 页，覆盖 1024 个 token，但可复用的连续前缀只有 1008 个 token，即 63 页。多读的那一页占 2.25 MiB，却没有省掉相应的重算。对该请求，读取量为 144 MiB，有效复用量约为 142 MiB；

[31] 共享 CPU KV 池容量对照与真实上下文保留后的 Agent 回放。容量对照采用受控查找任务；真实 Agent 回放中出现了提前结束及任务质量下降。

[32] DeepSeek V4 技术报告的状态持久化与生成服务内容，以及本章的阅读笔记。

更能说明问题的说法是“读入 64 页，用上 63 页”。这里的限制发生在读取之后：只有匹配成功、能连续接到已有上下文上的页，才能替代计算。把磁盘读得更快能缩短读取时间，却改变不了该请求要处理的末页。[33]

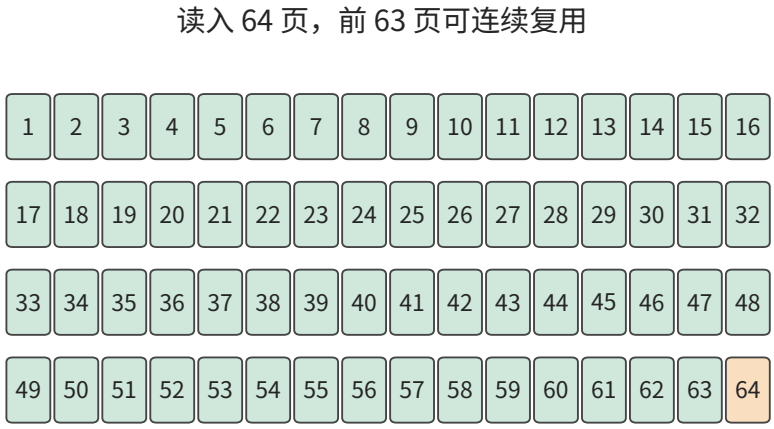



图 9-36：读入的页不一定全部成为可复用前缀。正常重启后的这一请求读入 64 个各含 16 个 token 的页，只复用前 63 页；末页仍需处理。每页 2.25 MiB，该请求的匹配边界为 1008 个 token。

页面缺失时，要比较继续等待和重新计算各需要多久。在 A100 上重算 8192 个 token 的整段前缀约需 0.856 秒（第 9.2.3 节的 prefill 时间）；若为取回一个不可用的对象已经等了 1 秒，这一次等待就已超过整段重算的时间。实际的截断页实验里，无条件等待的请求在 60 秒观察窗口内一直没有完成，改为允许放弃等待后，请求靠重算得到了结果。完成当前请求之后，还要隔离或修复坏页，否则下一个请求还会遇到同样的等待。[34]

### 9.5.4 缓存亲和性与请求路由

确认缓存可以复用后，仍需决定把请求发给哪台机器。缓存所在的机器可能正忙，另一台机器虽然需要取回或重算，却可能先完成。因此路由时要比较的是请求的完成时间。设 GPU 最早可执行时刻为 $Q$，状态从请求到达起的就绪时刻为 $R$，状态可用后的剩余工作为 $C$。在整份状态到齐后才执行、取回可与 GPU 等待重叠的模型中，首 token 时间为

$$T_{first} = \max(Q, R) + C.$$

如果取回要等 GPU 排完队才能开始，排队时间与取回时间就要相加；如果逐层流水，就需要更细的执行图。例如 $Q = 80$ ms、取回耗时 60 ms、剩余计算 10 ms，并行就绪时共需 90 ms；等 GPU 空闲后再启动取回则需 150 ms。计算量和传输字节数都相同，只是依赖关系不同，耗时就差了 60 ms。

**例 9.7　缓存命中与排队等待如何共同决定请求路由？** A、B 是两个 A100 实例，请求带 8192 个 token 的前缀和 256 个 token 的新输入。A 的 HBM 里有这份前缀，但要排队 250 ms；B 只等 20 ms，却没有缓存。按第 9.2.3 节的约定取 A100 峰值的 50%，即 156 TFLOP/s：完整重算 8448 个 token 需要约 138.4 TFLOPs 矩

[33] 重启页读取与实际复用核算，读取量为 144 MiB，有效复用量为 141.75 MiB。读取 64 页、共 1024 个 token，其中 63 页、共 1008 个 token 得到复用。

[34] 截断页的预取策略对照，实验观察窗口为 60 秒。

阵运算，约 887 ms；命中后只算 256 个新 token，约 4.81 TFLOPs、30.9 ms。B 也可以从远端存储取回前缀：查找的固定开销为 10 ms，整份 1.125 GiB 先经网络到主机内存，再经 PCIe 4.0 x16 搬到 GPU，与例 9.3 一样按 25 GB/s 计，约 48.3 ms。

| 路径 | 首 token 时间 |
|---|---|
| A：本地命中 | $250 + 30.9 \approx 281$ ms |
| B：直接重算 | $20 + 887 \approx 907$ ms |
| B：经 50 GbE（6.25 GB/s）远端取回 | 约 282 ms |
| B：经 200 GbE（25 GB/s）远端取回 | 约 137 ms |

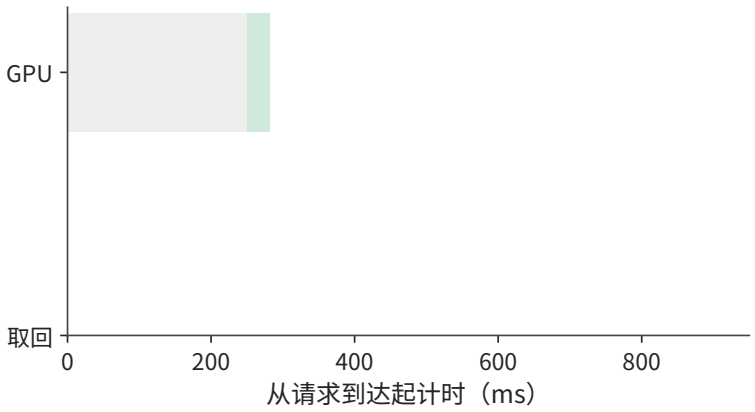



图 9-37：A 本地缓存已命中，GPU 排队 250 ms 后再计算 30.9 ms，首 token 在 281 ms 返回。灰为排队，绿为计算。

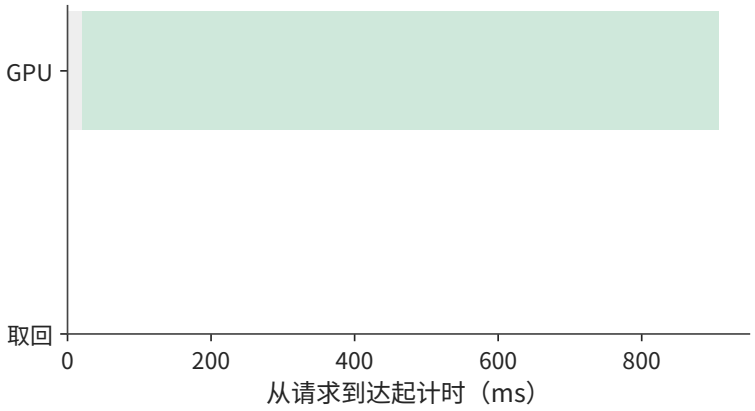



图 9-38：B 在 20 ms 空闲，随后在 A100 上重算 887 ms，首 token 在 907 ms 返回。

第一张路由图中，A 的数据已经在 GPU 上，但计算要等到 250 ms 才能开始。B 在 20 ms 时空闲；直接重算可以立即开始，但 887 ms 的计算比任何一条取回路径都长，远端取回则要继续等数据。提高远端带宽缩短的是橙色条，查找、主存到 GPU 的传输和最后的计算仍然存在。

B 要想比 A 的 281 ms 更早返回，扣除命中后 30.9 ms 的计算，状态必须在 250 ms 内就绪。再扣除查找的 10 ms 与主机内存到 GPU 的约 48.3 ms，远端读取只剩约 191.7 ms。用 1.125 GiB 除以该预算，得到 B 取回与 A 本地命中打平时的带宽，约为 6.30 GB/s：50 GbE 的 6.25 GB/s 略低于此值，200 GbE 则远高于此值。取回与重算打平的带宽只有约 1.48 GB/s，在 A100 上重算 8K 前缀几乎总是最慢的选择。

同一个会话由此可以按状态与资源所在的位置重新安排。应用保留连续的上下文，服务在继续使用本地状态、迁移状态和重新计算之间选择，依据是这三条路径各自何时能让后续的模型执行开始。第 8 章根据内容判

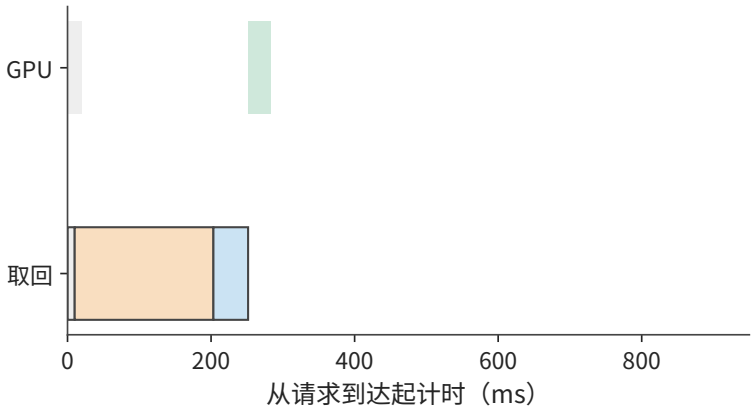



图 9-39：取回先查找 10 ms，再经 50 GbE 以 6.25 GB/s 读取 1.125 GiB 至主存，最后经 PCIe 以 25 GB/s 搬到 GPU。计算要等数据和 GPU 都就绪，首 token 约 282 ms 返回，比 A 慢约 1.6 ms。

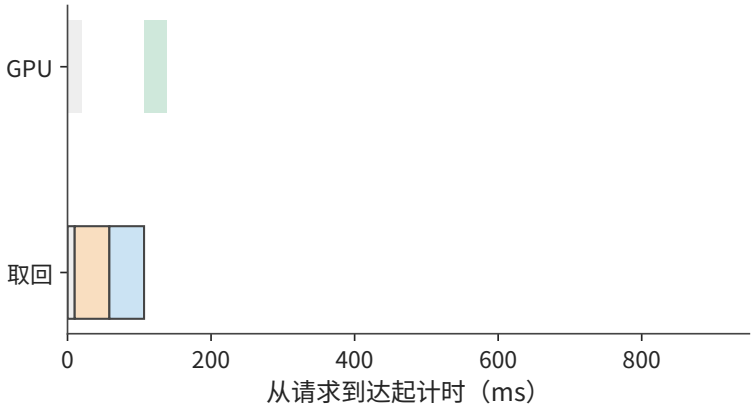



图 9-40：远端链路换成 200 GbE（25 GB/s）后，首 token 约 137 ms 返回。橙为远端读取，蓝为主存到 GPU；四图均从请求到达起计时，横轴相同。

断可复用的前缀，本节进一步把排队时间与链路带宽纳入路由决策。

换成 200 GbE 后，单请求取回明显占优；但每秒 16 次取回需要约 19.3 GB/s，已用去这条 25 GB/s 链路约 77% 的能力。突发请求会增加传输排队时间。因此，按单请求算出所需带宽后，还要计算持续到达时的总流量。[35]

上述比较假定缓存位置已知且数据可用。路由器通常根据缓存事件预测位置，事件延迟或对象被淘汰会让预测失准。为了看清这种误差如何影响响应时间，把 A 的排队改为 80 ms，并设缓存只以概率 $p$ 真正可用。命中时为 110.9 ms，所有层次都失效、只能重算时为 967.2 ms，期望为

$$\mathbb{E}[T_A] = 110.9p + 967.2(1-p) = 967.2 - 856.3p \text{ ms}.$$

平均值要优于 B 直接重算的 907 ms，只需 $p > 7.0\%$。取 $p = 0.9$，平均约 196.5 ms，但 10% 的请求仍是 967 ms，所以该两点分布的 p99 为 967 ms，远达不到 220 ms 的目标。要让 p99 达到 220 ms，该两点时间模型要求命中概率至少 99%；90% 已经大幅改善了均值，却仍让十分之一的请求走 967 ms 的慢路径。[36]

[35]经 50 GbE 取回与经 200 GbE 取回的缓存路由计算，包含整份远端 → 主机内存 →GPU 的串行路径；重算与命中后的计算时间由矩阵 FLOPs 除以 A100 80GB SXM 312 TFLOP/s 的 50% 得到。A100 的 PCIe 4.0 为收发合计 64 GB/s，见 A100 80GB 数据表。

[36]缓存事件与路由判断、缓存失效两点分布。

同时运行两个任务的负载压力实验，更直观地显示了缓存优先与空闲优先之间的取舍。缓存优先时，目标请求约 1.38 秒完成，两个任务也在这时全部结束；把目标请求移到空闲实例后，它的完成时间缩短到约 0.33 秒，两个任务却要到约 1.54 秒才全部完成。目标请求快了约 1.05 秒，全部任务的完成时刻却推迟了约 0.16 秒。做路由决策之前，必须先确定优化的是目标请求的响应时间，还是全部任务的完成时间。[37]

把第 8 章的状态恢复过程计入路由决策后，要同时比较实例何时空闲与状态何时准备好。以 DeepSeek V4.1 的会话状态为例：设实例 A 保留了全局 KV 与编码器的 SWA 状态，但有排队；实例 B 可以立即执行，却要取回全局 KV 并恢复编码器 SWA 状态。A 的优势是能复用状态，B 的优势是计算资源可以立即开始执行；路由器要比较的是两者的预计完成时间，而不是只看缓存命中标志。

设两边后续的新增输入处理、解码器窗口重放与生成耗时相同，准备工作串行执行。B 的全局状态传输按第 7 章的算例为 4.666 ms，编码器 SWA 状态的恢复假设为 8 ms，共需 12.666 ms。A 保留了这两份状态，准备时间就是排队时间：排队 10 ms 时 A 更早开始处理新增输入；排队增到 20 ms 时 B 更早。12.666 ms 由此成为本例中路由选择翻转的排队阈值。[38]

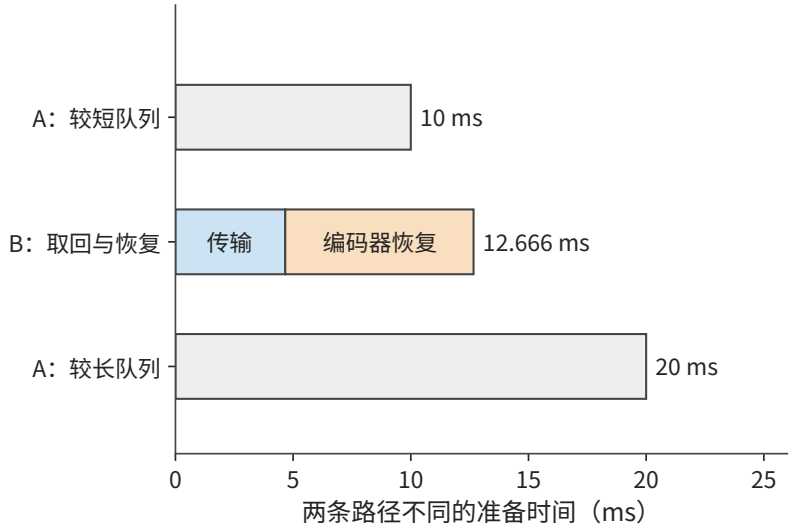



图 9-41：缓存亲和性与空闲执行位置的比较。A 保留全局 KV 与编码器 SWA；B 需要 4.666 ms 全局传输和假设的 8 ms 编码器恢复。两端共同的解码器重放等后续工作略去，只比较不同的串行准备时间；A 的排队时间从 10 ms 增到 20 ms 时，更快的方案由 A 变为 B。灰色为等待队列，其他色块分别为全局状态传输与编码器局部状态恢复。

## 9.6 服务启动、扩缩容与故障恢复

第 9.2 至 9.5 节比较的是请求交给谁、状态放在哪里。本节转向实例本身的变化：新副本启动和预热要多久，运行中改变并行方式时状态如何交接，部分卡故障后如何依靠已保存的状态和输出记录继续服务。

### 9.6.1 启动与预热开销

第 9.5.4 节的路由比较都从已有可用实例开始。扩容或故障恢复时，新实例要先完成启动，而等待中的请求还在不断到达。因此，本节把启动和状态恢复纳入同一条服务时间线。新副本还要完成进程与分词器（tokenizer，将文本转换为 token 编号的组件）初始化、权重读取与分片、编译、KV 容量探测和图捕获（把执行步骤录制为第 5.5.2 节的 CUDA Graph）。这些准备的总时间决定新副本何时开始分担请求。首次试运行触发编译时，编译

[37] 真实路由压力的配对核算，区分目标请求和完整任务对的完成时间，原始条件随结果保存。

[38] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

时间已经包含在试运行里，应按先后依赖计算总时间。启动开销研究 Breaking the Ice 对历史 vLLM 版本的分阶段分析说明了这一问题；把包含关系展开后，启动总时间由从进程创建到可以接收请求的最长依赖路径决定。[39]

假设准备执行图使启动时间增加 $T_s$，但此后每个执行步都能节省 $\delta$，经过 $N$ 步后的净收益为 $N\delta - T_s$。增加 10 秒、每步节省 0.2 ms，要 50000 步才持平，从第 50001 步起累计节省的时间才超过额外的启动时间。若每步为 32 个请求各生成一个 token，50000 步对应约 160 万个输出 token。每执行一个 batch 省一次时间，所以累计收益要按批执行的次数计算。

启动期间新到达的请求还会积压。若原有服务完全不可用、到达率保持 $\lambda$、启动时间为 $T_s$，启动结束时积压约为 $\lambda T_s$。就绪后的服务能力为 $\mu > \lambda$，在理想的连续流量模型中，额外的排空时间为

$$T_{drain} = \frac{\lambda T_s}{\mu - \lambda}.$$

以本章的异构集群为例，启动需要 10 秒，期间每秒到达 3.5 个请求，就绪时已有 35 个请求积压。直接 PD 就绪后每秒约能服务 4.55 个，新请求仍占去 3.5 个，每秒实际只能消化约 1.05 个积压请求，还要约 33 秒才能排空，从开始启动算起是第 43 秒。按理想分块捎带的共置上限 4.17 请求/s，每秒只能消化约 0.67 个，要到第 63 秒；不分块的共置只有 3.02 请求/s，积压会一直增长。三条曲线见图 9-42。

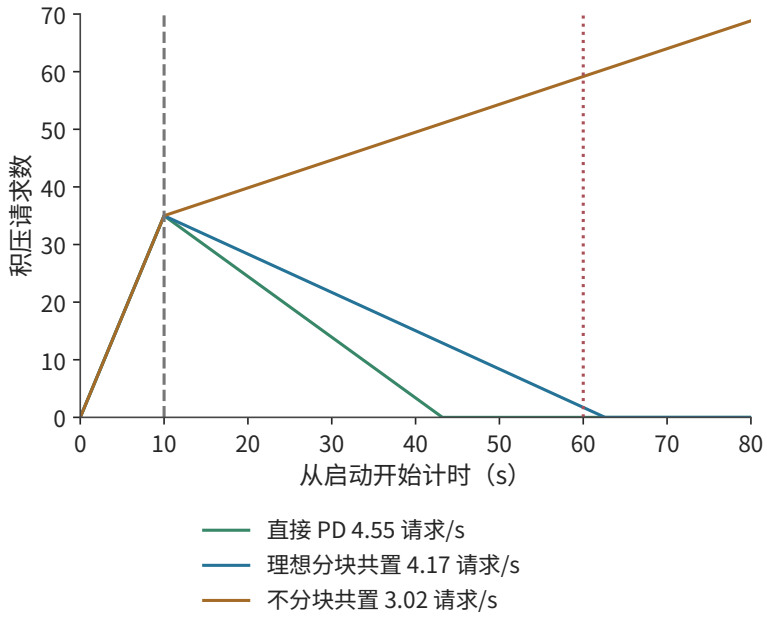



图 9-42：服务余量决定启动积压的消退速度。连续流量模型，每秒到达 3.5 个请求，启动 10 秒后积压 35 个。就绪后服务率分别为直接 PD 的 4.55、理想分块共置的 4.17 和不分块共置的 3.02 请求/s；前两者从开始启动算起在第 43 与第 63 秒排空，后者持续积压。例 9.8 的排空期限为第 60 秒。

积压的请求在就绪后往往一起涌入，这时并发预取还可能改变谁先开始执行。某个 1024 个 token 的请求单独运行时复用了 1008 个 token；八个请求同时到达时，前四个请求各登记了 1024 个 token 的预取，合计占用 4096 个 token，后面的请求因为超过容量限额而没有登记。最先完成的请求恰恰来自后者：该请求没有命中任何缓存，直接开始 prefill；正因为没有等待缓存，它更早进入了执行队列。所以，即使缓存对象存在，请求也要

[39] Breaking the Ice 的研究整理，研究使用 vLLM v0.10.1.1。

先获得预取所需的空间，才能使用这份缓存。预取空间用完后，后面的请求直接重算，而已经开始预取的请求还在等数据。[40]

扩缩容还要考虑模型版本与硬件的对应关系。权重固定在不可修改介质上时，增加同型号加速器可以扩展同版本服务；部署新权重则要配置支持新版本的加速器。路由器按模型版本分派请求，容量规划要为新旧版本并存和加速器替换预留资源。固定版本的服务持续得越久、需求越大，部署投入便能由更多请求分担。

### 9.6.2 运行中重配置与状态交接

启动解决的是新实例何时可用；运行中的迁移还要让目标实例接着源端的进度执行。改变张量并行度、专家并行度或服务池的规模，需要重新安排权重与请求状态。目标卡获得参数只是第一步，还可能要建立通信组、准备新的执行图、转换布局并恢复接收请求。这些准备与数据搬移有先后依赖，共同决定何时能把新请求交给目标。

计划内的迁移可以利用源端仍在运行这一条件：先在后台复制不再改变的上下文，源端继续生成；最后短暂停下，复制完迁移期间新增的状态，再把执行权交给目标。以迁走一张 H20 上的 D worker 为例：批内 32 条请求的平均上下文为 8704 个 token，KV 共约 41.1 GB。源端每秒完成约 1166 次 decode 调用，每次追加一个 token 的 KV（144 KiB），状态每秒只增长约 0.172 GB。目标经本章的 200 Gbit/s 网卡以 25 GB/s 复制，积压以约 24.8 GB/s 减少，约 1.65 秒后完成复制，其间新增的状态约 0.28 GB；只有一个 50 GbE 端口（6.25 GB/s）时要约 6.76 秒。decode 追加 KV 的速率远低于网卡带宽，追赶时间几乎全由初始的 41.1 GB 决定；复制速率一旦降到状态增长速率，积压便不再缩小。最后的交接点同时确定状态版本与执行权，防止目标漏掉源端最后的更新。

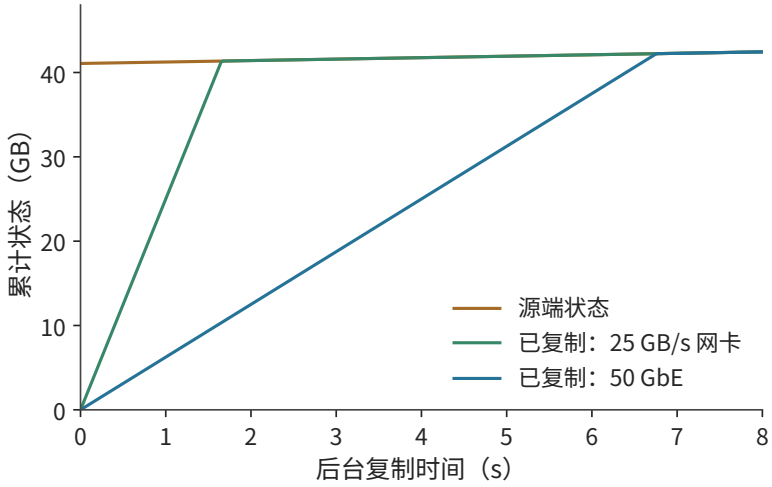



图 9-43：后台复制需要赶上仍在增长的状态。开始时待复制状态为 41.1 GB，源端每秒增长约 0.172 GB；目标经 25 GB/s 的网卡约 1.65 秒赶上，经 6.25 GB/s 的 50 GbE 约 6.76 秒赶上。曲线相交前，垂直距离就是尚未复制的数据量；相交后，目标只需跟随源端的新增状态。

图 9-43 中，源端曲线也在上升，所以追赶速度是两条曲线斜率之差；本例 decode 的增长极慢，源端曲线接近水平。复制能力恰好等于状态增长率时，两条线平行，初始积压便不会缩小。最终交接必须安排在目标已追上源端、两端状态一致的时刻。

迁移之外的另一种运行中调整是切换并行方式：把同一组卡重新组织成 TP 或 SP×TP。这里 TP 切分层内矩阵，SP 按 token 位置分配一部分计算与中间状态。ArcticInference（Snowflake 开源的 vLLM 推理插件）与

[40] HiCache 请求分支观测，使用同一请求 ID 的事件链解释限额和有效命中。原记录的预取限额为 3289 个 token，已登记占用 4096 个 token；八个请求由客户端同时发出，GPU 最多同时运行一个请求。

vLLM 的相关案例展示了这种做法：切换改变每一步参与计算的卡与 token 的分布，也改变所需的权重、KV 布局和执行图。预先保留两种模式的权重与执行图可以缩短切换的等待，但这些额外保存的权重与图会减少 KV 的可用空间。弹性专家并行（Elastic EP，运行中增减专家并行组的卡数）扩大专家组时，新加入的卡要先获得专家权重，随后才能承担分派来的任务；处理完整的请求还需要相应的注意力状态。[41]

目标追上源端的状态更新后，两边的数据才一致。此外，目标还要完成通信组和执行图的准备。下面保持数据量不变，只改变传输带宽。Qwen3-8B 从 TP4 切换到 TP8 的迁移例子需要约 16.4 GB 的网络传输，在本地构建目标张量还要读写约 4.7 GB 数据。把同一份网络载荷放到一个 50 GbE 端口（6.25 GB/s）和本章的 200 Gbit/s 网卡（25 GB/s）上，单看搬移约需 2.63 秒和 0.66 秒，相差四倍。

但若再假设建立通信组、准备执行图与恢复接收请求在传输之后串行占用 9 秒，总时间就约为 11.6 秒和 9.7 秒，只缩短约 17%。若八张卡同在一台 8×A100 服务器内，迁移改走 NVLink，每张 A100 每方向 300 GB/s，发送最多的一张卡要送出约 4.7 GB，只需约 15.7 ms，总时间仍约 9.0 秒。这 9 秒限制了继续提高带宽的收益：即使网络传输趋近于零，总时间也只能趋近 9 秒。若能提前建立通信组并准备好执行图，就能进一步缩短迁移时的等待。[42]

此时搬移本身已接近链路的物理下界，剩下的时间全是软件准备。按第 1.3.4 节的判据，这类差距靠更快的链路解决不了，只能重构软件：提前建立通信组，预先捕获执行图，把进程与调度解耦，让准备工作和传输重叠。

### 9.6.3 部分故障与流式生成恢复

计划内迁移时源端还能提供最后一次更新；故障发生后，源端的数据可能已无法读取，只能依靠提前保存的状态和输出记录。恢复首先要确定用户已经收到哪一段输出。故障后有三种目标：恢复足够的状态继续 decode；根据已经确定的 token 重做 prefill；重新采样一段输出。前两种沿用户已经收到的序列继续生成，第三种重新做随机选择，可能产生不同的后缀。对 RL 的 rollout 来说，这还改变了样本的生成过程。

假设原输入有 8192 个 token，已经向用户返回 1025 个输出，但最近保存的 KV 仅覆盖原输入。恢复时，若这些输出已经可靠记录，只需把前 1024 个输出作为一次 prefill 补入模型，重建覆盖 9216 个 token 的 KV，再处理第 1025 个输出继续生成。若已经有覆盖 9216 个 token 的兼容 KV，就可以省去这 1024 个 token 的重放。

图 9-44 把两种记录的末端分开画出。输出已经返回，不表示对应的 KV 已经保存；恢复时用已记录的 token 补算，就能重建相同的前缀，不必重新采样这些输出。

**预写日志**（write-ahead log，WAL）在对外确认之前可靠保存待恢复的操作或结果。这里的 token 级 WAL 保存已经确定的输出序列，KV 保存的是这段序列已经完成的计算。前者决定恢复后应继续哪条输出，后者决定还要重做多少工作。DeepSeek V4 的生成服务采用这类机制；恢复时还要保持权重版本、token 位置索引和 decode 状态一致。[43]

故障的影响范围由同步组决定。张量并行组或专家并行组丢失一张卡，可能让整个协作实例上的请求停止；其他完整副本则可以继续服务。故障恢复依次经过检测、实例重建与请求恢复；恢复后的处理能力决定积压的请求还要多久才能完成。第 10 章进一步分析这些选择对 RL 样本与训练进展的影响，第 11 章再处理工具和环境的任务恢复。

---

[41] 动态并行与状态条件、专家分派与扩缩容。

[42] 经 50 GbE、经 200 Gbit/s 网卡与经 A100 NVLink的串行迁移计算，按载荷、链路带宽和九项各 1 秒的串行准备时间给出迁移时间的下界。A100 SXM 的 NVLink 为收发合计 600 GB/s，见 A100 80GB 数据表。后台复制的 KV 字节数与 decode 调用速率取自异构 P、D 整数分配中 H20 的 `derived_stage_rates`。

[43] DeepSeek V4 技术报告的状态持久化与生成服务内容，以及本章的阅读笔记。

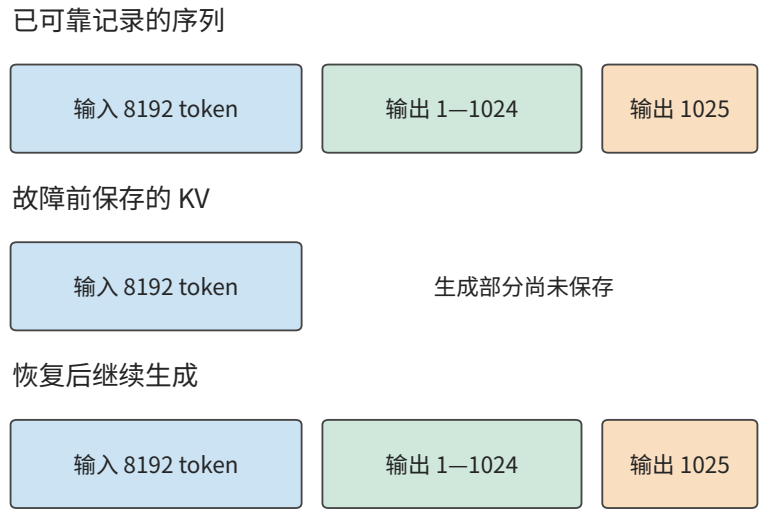



图 9-44：输出记录决定继续哪条序列，KV checkpoint 决定从哪里补算。假定 1025 个输出均已可靠记录，但 KV 只保存了原输入。下方按输入、前 1024 个输出和第 1025 个输出分段示意，宽度不按 token 数量比例绘制。

## 9.7 部署方案的综合比较

### 9.7.1 组合副本、PD、AF 与共享 KV

本节把前几节的结果用于章首的八卡服务，先确定状态传输方式，再比较整个服务能否满足到达率和恢复期限。P 产生的 KV 可以直接交给 D，也可以先进入共享池再由 D 取回；D 新生成的状态若要用于下一轮 P，还需确认这些状态已经保存并发布。直接交接与经池中转读取的可能是相同的内容，占用的链路和缓冲却不同。

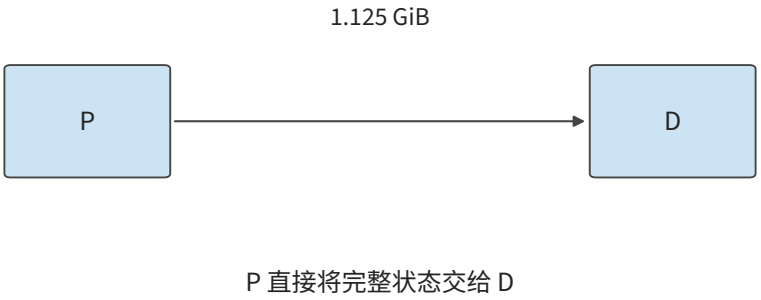



图 9-45：P 直接向 D 交接 1.125 GiB 上下文 KV 缓存，只经过一次直接传输。P 为 prefill，D 为后续逐 token 的 decode。

对本章的 1.125 GiB 上下文 KV 缓存，直接 P→D 交接搬一次；经池中转则在 P→ 池、池 →D 两条边各搬一次，总载荷为 2.25 GiB。假设两条边都为 25 GB/s，且整份写入后才能读取，中转等待约 96.6 ms，比直接交接多约 48.3 ms。

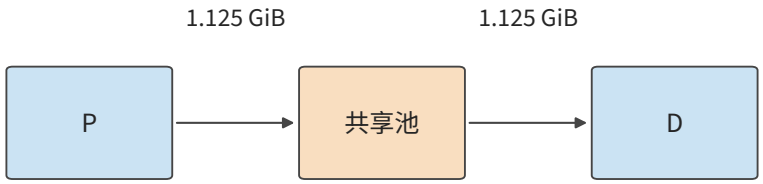



图 9-46：P 先向池写入完整 1.125 GiB 并发布，D 再取回同一对象，共经过写入和取回两次传输。后续实例还可以复用池中对象。P 为 prefill，D 为后续逐 token 的 decode。

若 D 只用一次这份状态，这次中转就没有复用收益。若后续还有另一个实例要继续处理同一会话，就有机会用约 48.3 ms 的读取替代一次约 856 ms 的重算（第 9.2.3 节的 prefill 时间），节省的计算时间足以抵消额外的搬移时间；第 9.7.3 节按例 9.8 的负载核算这项收益。共享池把一次会话留下的计算成果用于后续请求；AF 则改变当前请求每一层的执行位置，两者改变的是时间线上不同的部分。

同样的规则也适用于视觉输入。E 产生的 EC 与语言 KV 使用不同的缓存标识，占用的空间也不同；图像缓存命中可以减少 E 的工作，却不会自动省去后续语言模型的 P 阶段。在 E→P→D 的执行图中，EC 命中使 E 的需求下降，语言前缀命中使 P 的工作减少，资源配比也随各阶段工作量的变化而调整。

### 9.7.2 同质量、同资源约束下的方案比较

**例 9.8　哪种推理部署能持续服务并按期清空启动积压？**采用章首的资源与负载：四张 A100 80GB SXM、四张 H20 SXM5 96GB，每秒到达 3.5 个独立请求，每个请求输入 8192 个 token、输出 1025 个 token。各阶段能力沿用例 9.2，两个阶段在同一张卡上执行时依次占用该卡。两台服务器之间是共享的 25 GB/s 载荷通道，各次搬移占用同一份链路带宽。比较窗口内模型与生成设置相同，请求需要的前缀互不复用。服务从停止状态启动，三种部署方案都在 10 秒后就绪；就绪后的队列按连续流量模型处理。设计目标是持续处理新到达的请求，并在开始启动后的 60 秒内消除启动积压。

容量条件也要检查。每个请求经过 1024 次 decode 后，其 KV 覆盖 9216 个 token，占约 1.27 GiB。一张 H20 按 batch size 32 驻留活动请求，状态共约 40.5 GiB，再预留一份 1.125 GiB 接收缓冲，加上约 15.3 GiB 权重，共约 56.9 GiB，能放入 96 GB（约 89.4 GiB）的显存。换成紧凑 MLA 状态，每份约 618 MiB，32 份加一份 549 MiB 的接收缓冲共约 19.8 GiB。A100 一侧只需同时保存正在计算和正在发送的两份状态，共 2.25 GiB。使用共享池的方案另有 64 GiB 状态容量。尚未执行的请求只保留输入，分配到缓存空间后再开始计算。

下表比较完整副本、直接交接的 PD 和经共享池交接的 PD。共享池按整份写入后再读取，因此每个请求在公共通道上搬两次；池的读写两侧都能达到这条通道的带宽。

| 部署方案 | P、D 的组织 | 每请求通道载荷 | 计算能力，请求/s | 通道能力，请求/s | 启动后的总体吞吐率，请求/s |
|---|---|---|---|---|---|
| 八个完整副本 | 每张卡都执行 P、D | 0 | 3.02 | — | 3.02 |
| 直接 PD | 四张 A100 做 P，四张 H20 做 D | 1.125 GiB | 4.55 | 20.7 | 4.55 |
| PD 加共享池中转 | 相同的 P、D 配比，P→ 池 →D | 2.25 GiB | 4.55 | 10.3 | 4.55 |
| 直接 PD，紧凑 MLA 状态 | 四张 A100 做 P，四张 H20 做 D | 549 MiB | 4.55 | 43.4 | 4.55 |
| PD 加共享池中转，紧凑 MLA 状态 | 相同的 P、D 配比，P→ 池 →D | 1.07 GiB | 4.55 | 21.7 | 4.55 |

完整副本每秒只能完成 3.02 个请求，持续低于每秒 3.5 个的到达率，因此首先排除。两种 PD 方案的通道能力都远高于 4.55 请求/s，均由计算池限制。在每秒 3.5 个请求的到达负载下，直传消耗约 4.2 GB/s，中转消耗约 8.5 GB/s；在 4.55 请求/s 的排空阶段，两者分别需要约 5.5 和 11.0 GB/s。紧凑 MLA 状态下这四个数分别为 2.0、4.0、2.6 和 5.2 GB/s，通道能力翻倍，判断不变。

两种方案的稳态吞吐率相同，但直接 PD 更适合这组请求。后续请求不会复用这批请求的前缀，中转却让通道传输量翻倍，每个请求还多等约 48.3 ms。64 GiB 的共享池最多容纳 56 份完整的 1.125 GiB 前缀，紧凑 MLA 状态下为 119 份；在这一负载下，保留下来的每一份都不会再用到。直接交接完成同样的工作，留出更多链路余量，交接等待也更短。因此本例选择四张 A100 做 P、四张 H20 做 D、P→D 直传。

接着检查这一方案能否按期处理完启动期间积压的请求。按第 9.6.1 节的排空计算，10 秒启动留下的 35 个积压请求，直接 PD 从开始启动算起约 43 秒排空，满足 60 秒目标。完整副本在就绪后仍每秒增加约 0.5 个请求，到第 60 秒时积压已从 35 增至约 59 个；即使按理想分块捎带的 4.17 请求/s 计，共置也要到第 63 秒才能排空，同样错过目标。

反过来，也能求出满足这一期限所需的最低服务率。启动占去 10 秒，剩余 50 秒要消除 35 个请求，同时继续处理每秒到达的 3.5 个新请求，因而要求

$$\mu - 3.5 \geq \frac{35}{50}, \qquad \mu \geq 4.2 \text{ 请求/s}.$$

直接 PD 的 4.55 请求/s 高于这一最低要求，余量约 8%。若实际 kernel 效率只有例 9.2 假设值的九成，D 池能力降到约 4.10 请求/s，稳态仍能处理持续到达的请求，但每秒只能消化约 0.60 个积压请求，排空要到约第 68.5 秒，错过目标。图 9-42 中两条下降线的斜率，正是服务率减去到达率得到的净排空能力。

确认能按期排空之后，再比较成本。计算服务成本时，还要考虑资源的计费方式。设八张卡按小时预留，每小时总成本为八元，实际每秒完成 3.5 个请求，则每小时完成 12600 个请求，成本约为每千请求 0.63 元。若到达率仍是每秒 3.5 个，4.55 请求/s 的能力不会凭空多出请求来处理；多出来的能力用于消化突发和启动积压。

比较满负载时的请求处理量，3.02 与 4.55 请求/s 对应每千请求约 0.74 元与 0.49 元。若业务另设完成时限，而后一方案只有四分之一的请求合格，每秒合格的请求数就降到约 1.14 个，每千个合格请求的成本升到约 1.95 元。一般写成

$$C_{effective} = \frac{\text{统计期间的总成本}}{\text{满足质量与时限的请求或任务数}}.$$

分子随计算资源、主存、网络和保持运行的备用实例变化，分母随实际到达与合格完成数变化。以提前退出的 Agent 为例：它可能少生成 token、少占用资源，但没有完成任务，这次退出不计入已完成的合格任务数。[44]

### 9.7.3 负载变化后的部署调整

现在改变例 9.8 中“前缀不再复用”这一条件。设另一个实例还会把每次留下的 1.125 GiB 上下文 KV 缓存复用一次。在 A100 上重算这 8192 个 token 需要 0.856 秒，取回只要 48.3 ms。先前为中转多付的 48.3 ms 加上后续取回的 48.3 ms，共约 96.6 ms，远低于重算的 856 ms，一次后续复用就净节省约 760 ms。共享池由此获得了具体用途：用更短的读取代替重复计算。

这项收益同时增加了通道的需求。每次原始交接写一次、读一次，后续复用再读一次，共搬 3.375 GiB。每秒到达 3.5 组这样的请求对时，共需约 12.7 GB/s；通道最多支持约 6.9 组请求对/s。紧凑 MLA 状态三次共搬 1.61 GiB，每秒 3.5 组请求对约需 6.0 GB/s，通道最多支持约 14.5 组请求对/s。与只传一次相比，后续复用增加了读取量，占用了原有的带宽余量。共享池省去重算的同时，也降低了通道能支持的请求对吞吐上限。

共享池改变的是传输次数。下面固定 D 没有缓存，只改变 P 要重新计算多少输入，考察资源分配是否也要调整。假定 P 已在本地命中 6144 个 token，D 仍为空。沿第 9.2.4 节计算，完整副本合计约 4.90 请求/s，原来的四张 A100 做 P、四张 H20 做 D 仍为 4.55 请求/s，改成两张 A100 做 P、其余六张卡做 D 则为 5.69 请求/s。若仍采用相同的 10 秒启动与 3.5 请求/s 到达，三者从开始启动算起分别约在第 35.1、43.2 和 26.0 秒排空。前缀命中后，完整副本反而比原来的 PD 配比排空得更早，原配比已不再合适；两张 A100 做 P、其余六张卡做 D 才是这组负载的最佳分工。

再恢复无命中的输入，把输出增至 4097 个 token，即推理过程更长的请求。最佳分工变成两张 A100 做 P、其余六张卡做 D，也只有约 1.26 请求/s。这时无论如何改变 P、D 的数量比例，原来的八张卡都处理不了每秒 3.5 个请求。复制三组这样的八卡部署，合计能力约 3.78 请求/s，刚好能持续处理到达的请求；若还要求相同的启动与排空期限，所需能力至少为 4.2 请求/s，四组提供约 5.04，因此要配置四组。按组扩展原有的计算资源组合时，持续处理到达的请求需要 24 张卡，满足恢复期限需要 32 张。额外计算资源的用途是加快积压的消退。

这也给出了把本章的推导用于新系统的顺序。先把每个请求的工作量换算成各类资源需求；再按整数的资源分配找出最忙的资源池或执行单元；按执行顺序计入状态传输时间与内存占用；最后用服务率减去到达率得到的剩余处理能力，求出突发或启动积压持续的时间。若瓶颈是 D，增加 P 不会减少积压；若瓶颈是共享通道，增加计算副本只会让更多请求等待取回。

PD 的八卡案例和 MoE 的专家案例在这里相接。前者把不同阶段交给更擅长该阶段的计算资源，后者靠批内复用减少权重读取，再靠专家副本分散最忙执行单元上的工作。共享 KV 则保存已完成的计算结果供下一次请求复用；启动与恢复决定这些计算资源何时能提供服务。一次部署选择的依据，是这些部署方式一共改变了多少计算量和等待时间，以及消除积压需要多久。

单个阶段变快也会改变实例配比。设链路充足，每个 P 实例每秒处理 20 个请求，每个 D 实例每秒完成 5 个同样的请求，一个 P 配四个 D 时两池能力相等。若 D 阶段加速四倍，沿用原配比会让 D 池能力增到 80 请求/s，P 池仍是 20；改为一个 P 配一个 D，两池都提供 20 请求/s，并释放三个 D 实例。加速一个阶段的收益，由此体现为完成同样的工作需要的资源变少了。

[44] 例 9.8 的容量、共享通道、启动时间、排空目标和成本，以及服务目标（SLO）的通过比例均为教材假设，用于推导条件变化。单位成本用完整小时成本除以（3600× 有效请求率）。迁移算例的串行准备时间设为 9 秒。

## 练习

练习按复算、改变条件和独立设计展开。前两题建立共同的请求单位，第三至九题分别改变计算、通信与状态条件，第十题完成新工作负载的设计，第十一题把交接状态换成 MLA 重做判断。带“核心”的题目贯穿多个资源层次。

**9-1　三种推理部署的调用次数与状态交接。**对输入 8192 个 token、输出 1025 个 token 的请求，画出完整副本、PD 和共享 KV 三种部署方式，分别列出每请求调用数、KV 产生位置和跨实例交接。再将输出长度改为 1 个 token，指出哪些工作不再需要执行。最后考虑调用一次工具后继续生成的情况，标出最后一个已返回、但尚未生成对应 KV 的 token 位置。

**9-2　链路瓶颈与请求组成如何改变 PD 配比。**复算例 9.2 的 25 种整数分配方案，并将链路的有效带宽设为每秒恰好传输一份 1.125 GiB 快照所需的带宽（约 1.21 GB/s）。求新的总体吞吐率上限；到达率恰等于该上限时，再一次性加入十个请求，求积压随时间的变化。随后复核前缀命中与 129 个输出两个情景。最后把阶段能力的效率假设从峰值的 50% 改为 40%，重新推出两类卡的 prefill 与 decode 能力，判断四张 A100 做 P、四张 H20 做 D 是否仍高于 3.5 请求/s 的到达率。

**9-3　专家 batch size、指令集与权重位宽如何改变 CPU 执行瓶颈。**用例 9.3 中的专家形状，分别按 AVX-512 与 AMX kernel，求 CPU 执行从读取受限转为计算受限时，每个专家接收的 token 数；再把内存带宽换成跨插槽的 125 GB/s，重新计算这两个临界值。分别比较每个专家接收 1 个和 128 个 token 时的执行时间；将权重改为每元素一字节、有效 CPU 算力保持不变，重新求解读取与计算的交点，并解释它的移动方向。

**9-4　PD 与 AF 的传输量、启动次数与通信重叠〔核心〕。**采用 Qwen3-8B，设有效交接带宽为 25 GB/s，分别取 1、5、20 μs 的启动开销，比较一次 1.125 GiB 交接与总字节数相同的 72 次交接。再采用例 9.4 中 AF 的实际载荷大小，计算输入 8192 个 token、输出 1025 个 token 的完整请求所需的累计交接时间。

另考虑四个独立 micro-batch，每个 micro-batch 在注意力侧和前馈侧分别计算 2 ms 和 3 ms。比较全部串行执行与理想双阶段流水的总耗时，再求在流水方案仍快于串行方案的前提下，额外通信和计算效率下降合计最多能增加多少关键路径耗时。

**9-5　异构推理的权重容量与 CPU 处理能力约束。**从已保存的 DeepSeek V4-Flash 或 Kimi K2 配置还原权重存放、GPU 缓冲和主存需求。增加请求并发后，分别预测被访问的专家数量、各专家的任务数，以及负载最重的 NUMA 节点。再构造一组总权重可以容纳、但到达率超过 CPU 计算能力的请求条件，推导积压增长率。

**9-6　专家副本的容量约束与复制成本回收。**设八个专家分别接收 $[32, 16, 8, 4, 2, 1, 1, 0]$ 行输入，比较只计算有效行、将非空专家的输入补齐到 32 行，以及将所有专家的输入补齐到 32 行时的计算量。随后按例 9.6 的条件，求累计节省时间首次超过复制耗时所需的最少批数；再把每卡额外容量改为 32 MiB，解释为什么应先排除不可行的副本方案。如果热点随时间变化，说明如何根据观察窗口内的负载预测复制后的累计收益，并与迁移开销比较。

**9-7　KV 保存、取回与重算的收益条件〔核心〕。**给定一份占 1.125 GiB 的前缀 KV，假设后续请求合计需要使用该前缀 100 次，比较重算一次后驻留、取回一次后驻留，以及每次使用时远程读取三种方式的总耗时。进一步计入写入耗时，以及两次请求之间保留缓存的时间，推导何时保存 KV 比重算更合算；设计一个改变 HBM 可用容量后需要将状态从 HBM 换出的例子。结合已有双引擎记录，解释共享 CPU 池从 4 GiB 增至 8 GiB 时保住了哪些复用机会。

再按第 9.5.2 节 DGX A100 的每卡配置计算：同一前缀两次使用的间隔为 5 分钟时，每张 GPU 至少需要多大缓存容量，需要用到哪几级存储？新输入分别为 128 与 512 个 token 时，从主机内存逐层预加载的总时间是多少，计算开始前至少要读入几层，才能让读取被计算完全掩盖？最后按 D7-P5520 的 1 DWPD 额度，求 SSD 可以长期接纳的新 KV 比例；若前缀命中率为 50%，新 KV 产生速率减半，这一比例变为多少。

**9-8　KV 缺页与版本不兼容时的恢复选择。**解释读取了 1024 个 token 的 KV、却只复用其中 1008 个 token 时，两种计量方式的差异。对缺页、截断页、版本不兼容分别提出继续等待、部分重算与放弃缓存的判断条件。分别说明如何检查请求能否完成、缓存数据是否修复，以及输出是否与不使用缓存时一致。

**9-9　缓存路由的带宽交点与首 token 尾延迟。**推导例 9.7 中 B 的远端取回分别与 A 的本地命中、B 的本地重算耗时相等时的带宽，并计算 $p = 0.5, 0.9, 0.99$ 时，选择 A 路径的首 token 时间均值，以及按逆累积分布定义的 p99。再假设 HBM 副本不可用时，CPU 副本仍可用，且取回过程可以与 80 ms 的排队时间重叠。检查原来的两点分布模型是否仍然适用。

**9-10　长短输出混合负载下的 PD 扩容与排队〔核心〕。**沿用例 9.8，每秒 3.5 个请求中一半输出 1025 个 token，另一半输出 4097 个 token，均输入 8192 个 token，无前缀命中；池内按长期平均工作量分配能力。先按例 9.2 的方法求两类卡在两种输出长度下的 D GPU 秒，得到每请求的平均 D 需求，再枚举将八张卡分配给 P、D 两阶段的所有数量组合。允许复制同样的八卡组合，分别求两种目标下所需的最少组数：持续处理新到达的请求；启动耗时 10 秒，并在开始启动后的第 60 秒前清空积压。最后将长输出请求集中到每分钟的前十秒到达，画出队列变化，并说明平均需求不变时，哪一段时间决定容量选择。

**9-11　MLA 与 GQA 的 PD ／ AF 判断。**把本章交接的状态换成 DeepSeek-V3 的紧凑 MLA 表示（61 层、$d_c = 512$、$d_r = 64$、BF16），保持例 9.2 的阶段能力、例 9.4 的 AF 载荷与 5 µs 启动不变。先求 8192 个 token 的状态字节数，以及 25 与 50 GB/s 下的一次 PD 交接时间；再求 $\mu_{PD}$ 的链路项，并给出链路成为约束项的带宽上限。按 $\alpha^* = (V_{KV} - V_{AF})/(71B)$ 求两种链路下的临界启动时间，说明输出 1025 与 4097 个 token 时，1024 步与 4096 步 AF 累计交接分别是一次 PD 交接的多少倍。最后把每步 2.05 GFLOPs 的查询变换计入 D 池，按 H20 的 74 TFLOP/s 有效算力求 D 池能力的变化，判断四张 H20 是否仍然够用。

**解题示范：CPU 专家执行在多大 batch size 下由读取受限转为计算受限？**单专家 CPU 路径的读取项与计算项相等时，

$$\frac{2mP_e}{C_C} = \frac{2P_e}{B_D}, \qquad m = \frac{C_C}{B_D}.$$

这里 BF16 每参数两字节，每行每参数两次浮点操作，两个系数恰好约去。AVX-512 kernel 的 1.8 TFLOP/s 与 220 GB/s 给出 $m \approx 8.2$：每个专家少于九个 token 时主要等待权重，多于这个数时主要等待计算。AMX kernel 的 21.3 TFLOP/s 把等时点推到约 97 个 token。线程读另一插槽的内存时带宽降到 125 GB/s，两个等时点分别移到约 14 和 170 个 token；单行执行的权重读取时间增加约 76%，AVX-512 kernel 在 128 行时仍由计算项主导。

这里的等时点与图 9-14 的 71 / 72、688 / 689 行边界回答的是不同的问题。前者是 CPU 从带宽受限转为计算受限的分界；后者比较的是包含权重搬移的完整 CPU、GPU 两条路径。CPU 过了等时点已经计算受限，但仍能胜过要先搬权重的 GPU 路径，直到累计的计算代价超过这笔搬移成本。

## 本章小结

分布式推理把一条请求的计算与状态分配到多个位置。PD、AF、专家放置和共享 KV 都要比较局部执行的收益与新增的交接，容量、服务率、启动和恢复共同决定部署是否合适。路由不仅选择空闲计算资源，也决定能够复用哪些状态。同一套推导对不同的上下文表示给出不同的答案：交接状态从 GQA 换成紧凑 MLA 后，PD 交接时间减半，$\mu_{PD}$ 的链路项翻倍，AF 的逐层交接不变，临界启动时间随之缩短。

硬件变化后，先固定请求与部署，找出新的瓶颈，再重新分配各阶段资源。下一章将转向训练，讨论参数与训练状态的持续更新，以及训练进度如何推进。

# 第 10 章 训练系统

一个模型能够在单张加速卡上生成文本，是否意味着它也能在这张卡上训练?

全参数训练会更新模型的全部可训练权重。Qwen3-8B 的 BF16 权重约为 16.4 GB，而用混合精度 Adam 做全参数训练时，系统除模型权重外，还需保存梯度、FP32 主权重，以及优化器跨步保留的梯度历史，合计约为 131 GB。这些训练状态占用的空间约为 BF16 权重的八倍。

反向传播求导时要读取前向阶段保留的激活；执行期间还需要参数收集和格式转换用的缓冲区。设计训练系统，首先要确定这些张量和缓冲区存放在哪里，以及需要保留多久。

把训练状态分到更多卡上，可以减轻单卡的显存压力，但计算往往要等其他卡传来的数据。把状态移到主机内存，可以腾出显存，却又增加了主机与 GPU 之间的传输。训练持续数周时，等待输入、保存 checkpoint 和故障后的重复计算也会影响完成时间。卡数达到上千张以后，故障就不再是偶发事件：按每卡约 337 天故障一次计，1024 卡的作业平均约 7.9 小时中断一次（第 10.4.4 节）。理解这些取舍，需要把三个问题连起来：哪些张量和缓冲区需要同时占用显存，哪些操作决定训练步何时结束，故障后能够恢复到哪一步。

训练计算图明确了各类数据的产生和使用时机，为这些设计提供了依据。前向产生的激活在反向时使用，梯度在参数更新前汇集，优化器状态跨步保留。利用这些已知关系，运行时可以提前取回即将使用的参数、重新计算开销较小的激活，并让已就绪梯度的通信与后续反向计算重叠。模型求导、内存管理与加速器调度由此共同决定一步训练的执行方式。

本章围绕一项任务展开：使用 Qwen3-8B 处理 100B token，完成特定领域的继续训练，期限为 30 天。先估算至少需要多少资源，再比较由 32 张和 48 张 RTX 4090 组成的两套方案。比较从显存需求开始，逐步加入计算、通信、数据准备和故障恢复的耗时，最后确定哪套方案能够按期完成。为了讲清各项机制，文中穿插较小的张量和流水线算例。RL 部分则进一步分析生成、验证和学习如何组成反馈循环。

本章的容量数字采用两套单位：GB、TB 表示十进制容量，GiB、MiB 表示二进制容量。

## 10.1 根据训练任务估算容量与时间需求

### 10.1.1 训练任务、可训练参数与完成目标

系统设计从要完成的工作开始。本章任务采用全参数训练、混合精度 Adam 和 8,192 个 token 的序列，处理 100B 有效 token。每次训练迭代处理 384 条完整长度的序列，即 3,145,728 个有效 token；最后一次训练迭代处理剩余数据，时间预算仍按完整一步安排。比较两套系统方案时，使用相同的训练数据、目标函数和评估要求。

有效 token 指任务中实际需要处理的 token，不包括为对齐长度而补入的填充 token。填充使张量形状更规则，但也占用计算和存储资源。统计任务完成量时，只累计有效 token。与全参数训练相对，LoRA 在原权重旁增加较小的可训练矩阵，其乘积表示权重调整量。LoRA 减少了需要训练的参数：基础权重仍用于前向计算，梯度和优化器状态的大小则主要取决于适配器参数量。

从时间上看，一次训练迭代包括前向传播、反向传播、梯度归约和参数更新。持续训练由多次迭代组成，其间还需要准备输入数据、保存 checkpoint。启动、准备资源和维护也要计入从开始到结束的总时间。本章的设计案例从 30 天中预留 5 天给启动、评估和计划性停顿，留下 25 天用于训练、checkpoint 与故障恢复。平均有效吞吐的要求因此从约 38,600 token/s 提高到约 46,300 token/s。

完成整个任务需要 $\lceil 10^{11}/3\,145\,728\rceil = 31\,790$ 次训练迭代。25 天分摊到每次迭代，只有约 67.9 s。这是贯穿全章的时间预算：每步计算、通信等待、输入等待，以及分摊到这一步的 checkpoint 保存与故障恢复时间，都要算在这 67.9 s 之内。

每步的 batch size 同样需要确定。全局 batch 取 384 条序列而不继续放大，原因在于梯度的信噪比。batch size $B$ 增大时，梯度估计的噪声随之下降；$B$ 足够大以后，再增加样本只能略微减少达到同一损失所需的步数，每步的计算量却成比例增加。发生这一转折的 batch size 称为**临界 batch size**（critical batch size），它可以用**梯度噪声尺度**（gradient noise scale）来估计。设损失对参数的 Hessian（二阶导数矩阵）为 $H$、单样本梯度的协方差为 $\Sigma$、真实梯度为 $G$，梯度噪声尺度定义为

$$B_{\text{noise}} = \frac{\operatorname{tr}(H\Sigma)}{G^{\top}HG}.$$

达到给定损失所需的步数 $S$ 与处理样本数 $E$ 满足经验关系

$$\left(\frac{S}{S_{\min}} - 1\right)\left(\frac{E}{E_{\min}} - 1\right) = 1, \qquad B_{\text{crit}} = \frac{E_{\min}}{S_{\min}},$$

其中 $S_{\min}$ 为达到该损失的最少步数，$E_{\min}$ 为所需的最少样本数。临界 batch size 约等于梯度噪声尺度；按临界 batch size 训练时，步数和样本数都是各自最小值的两倍。[1]

$B_{\text{noise}}$ 随模型与训练阶段变化。本例取二百万个 token：文献在已训练的最大模型上测得，收敛时约为一到二百万 token。把本章的参考运行——48 卡、每步 $384 \times 8192$ 个 token、31,790 步完成 100B token——视为恰好达到目标，由关系式反推 $S_{\min} \approx 19\,434$ 步、$E_{\min} \approx 38.9$B token。当前每步 batch size 约为 $B_{\text{noise}}$ 的 1.57 倍，已经越过 batch size 翻倍、步数近似减半的区间；batch size 再放大，步数最多也只能从 31,790 降到 19,434。下表比较卡数增加时弱扩展与强扩展两种方式的完成时间。

| 卡数 | 弱扩展：每步 token 数 | 弱扩展完成时间 / 天 | 强扩展完成时间 / 天 |
|---|---|---|---|
| 48（参考） | 3,145,728 | 19.4 | 19.4 |
| 96 | 6,291,456 | 15.7 | 9.8 |
| 192 | 12,582,912 | 13.8 | 5.0 |
| 384 | 25,165,824 | 12.8 | 2.6 |
| 1536 | 100,663,296 | 12.1 | 0.8 |

表中的弱扩展让每卡固定累积 8 个 micro-batch，每步仍为 52.8 s，全局 batch 随卡数放大；强扩展固定全局 batch，单卡计算量与卡数成反比，每步 0.58 s 的通信与输入等待不变。两者都不计并行效率的变化。弱扩展的加速上限为 $31\,790/19\,434 \approx 1.64$ 倍：1536 卡把每步 batch size 放大到参考值的 32 倍，完成时间只从 19.4 天降到 12.1 天。强扩展不受 $B_{\text{noise}}$ 限制，但固定等待在每步中的占比随卡数上升，0.8 天中通信与输入等待已占约四分之一。

弱扩展的收益何时翻转，取决于 $B_{\text{noise}}$ 的取值：若噪声尺度为两千万个 token，弱扩展上限升至约 7.36 倍，batch size 随卡数放大才有明显收益。因此扩卡后 batch size 能否随之放大，要先估计 $B_{\text{noise}}$：当前 batch size 远小于该值时，batch size 可以随卡数放大；远大于它时，增加卡数只能按强扩展的方式缩短每步时间。384 条

[1] 临界 batch size 复算与噪声尺度两千万 token 的对照。梯度噪声尺度的定义见大 batch 训练经验模型式 (2.8)；步数—样本关系式 (5.1)、临界 batch size 定义式 (5.2) 与收敛时约一到二百万 token 的区间见缩放定律论文。$B_{\text{noise}}$ 为本例取值，未对本章模型实测；弱扩展与强扩展都不含并行效率变化。

序列正是这样选出的：它在 48 卡的显存与每步预算之内，且已略高于噪声尺度，再放大 batch size 的收益开始递减。[2]

### 10.1.2 参数、梯度、优化器与激活

67.9 s 给出了每步允许花费的时间，但计算开始前还要满足显存要求。设参数数量为 $N$。一次迭代中，前向由输入算出预测和损失，反向沿计算依赖求出损失对各个中间值和参数的梯度，并读取前向留下的激活；优化器再用参数梯度计算下一步权重。

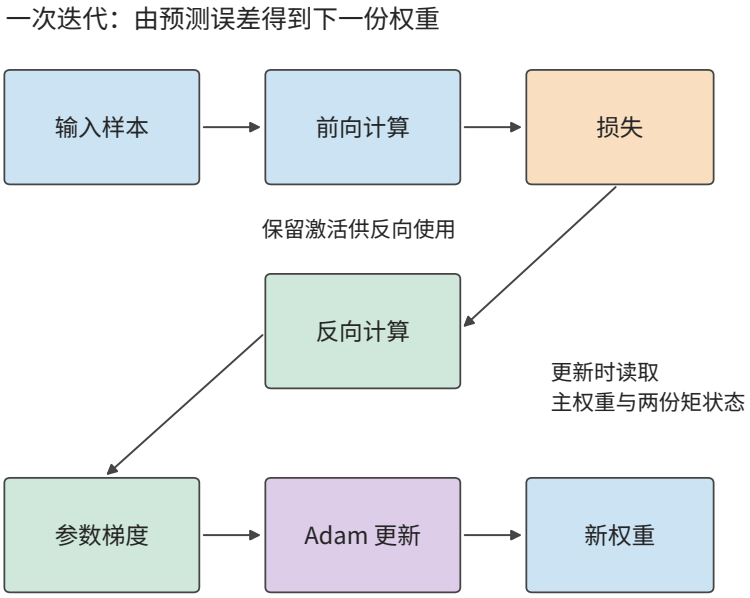



图 10-1：从输入和当前权重出发，前向得到损失，反向得到参数梯度，Adam 生成新权重。前向阶段保留的激活用于反向；更新过程还读取高精度主权重和梯度历史。

具体到本章任务，混合精度 Adam 用 BF16 权重执行矩阵运算，用 FP32 主权重保存优化器计算后的参数值，并保存两份 FP32 矩状态。一阶矩和二阶矩跨迭代保留，分别记录梯度方向和大小的历史；主权重更新后再转换为 BF16 供矩阵运算使用。梯度以 BF16 累积。逐项相加得到每参数 16 bytes。

| 状态 | 每个参数所占字节数 | Qwen3-8B 容量 / GB | 使用时机 |
|---|---|---|---|
| BF16 模型权重 | 2 | 16.4 | 前向与反向矩阵运算 |
| BF16 梯度 | 2 | 16.4 | 反向传播时产生，计算新参数时使用 |
| FP32 主权重 | 4 | 32.8 | 保存新参数，随后生成 BF16 副本 |
| Adam 一阶矩 | 4 | 32.8 | 在训练迭代之间保留梯度历史 |
| Adam 二阶矩 | 4 | 32.8 | 在训练迭代之间保留梯度平方历史 |
| 合计 | 16 | 131.1 | 常驻训练状态 |

[2]临界 batch size 复算与噪声尺度两千万 token 的对照。梯度噪声尺度的定义见大 batch 训练经验模型式 (2.8)；步数—样本关系式 (5.1)、临界 batch size 定义式 (5.2) 与收敛时约一到二百万 token 的区间见缩放定律论文。$B_{\text{noise}}$ 为本例取值，未对本章模型实测；弱扩展与强扩展都不含并行效率变化。

Qwen3-8B 有 $N = 8\,190\,735\,360$ 个参数，因此

$$M_{\text{persistent}} = 16N \approx 131.1 \text{ GB} \approx 122.1 \text{ GiB}.$$

梯度格式是另一项选择。将梯度改为 FP32，每参数增加 2 bytes，总量变为 18 bytes。这一变化来自梯度的数据格式，“混合精度” 一词本身并不规定梯度格式。MoE 每次由路由器选择部分专家子网络执行；各专家同样拥有待更新参数和优化器历史，所以状态容量由全部可训练参数决定。[3]

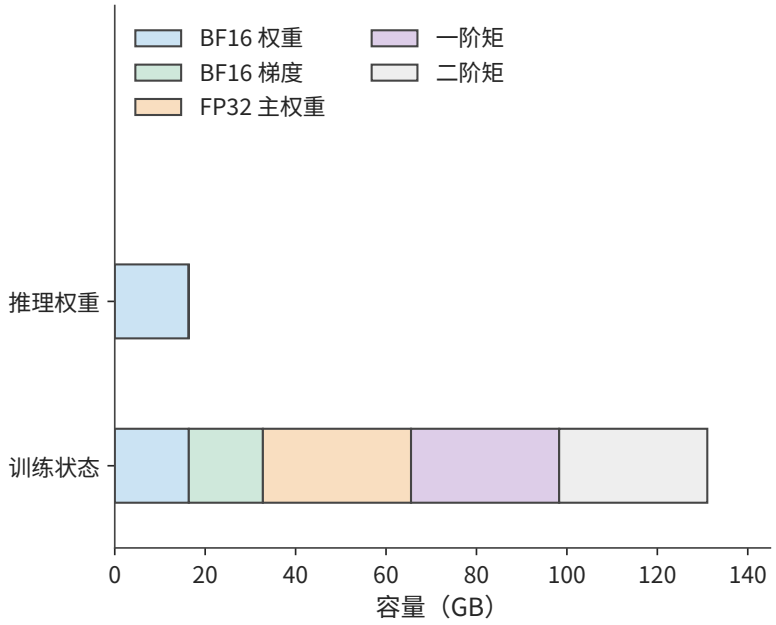



图 10-2：五类训练状态合计相当于八份 BF16 权重的容量。模型为 Qwen3-8B，采用表中数据格式；横轴表示各类状态累计占用的容量。

除了各类数据占多大空间，还要区分各自需要保留多久。前向产生的激活要等到相应的反向计算结束后才能释放，而参数收集缓冲区可能只在某个模块执行期间使用。第 $i$ 张卡的显存峰值取决于同一时刻驻留的训练状态、激活和临时缓冲区：

$$M_{\text{peak},i} = \max_t \left[ M_{\text{persistent},i}(t) + M_{\text{activation},i}(t) + M_{\text{temporary},i}(t) \right].$$

例如，8 GiB 激活与 4 GiB 收集缓冲若同时出现，就要共同占用 12 GiB；若先释放激活再收集参数，同一片空间可以复用。容量优化由此有两个方向：减少张量和缓冲区的大小，或减少需要同时保留的数据。下一节的分片、重计算与卸载分别从这两个方向改变峰值。

上述容量按 BF16 执行矩阵运算计算。换一种浮点格式，状态占用的字节数和数值风险会同时改变。先看 FP16。第 4.6.1 节已比较 FP16 与 BF16 的指数与尾数：FP16 的五位指数把最小正规数定在 $2^{-14} \approx 6.1 \times 10^{-5}$，小于它的值只能用次正规数表示，有效位随数值减小而逐位丢失；小于 $2^{-24} \approx 6 \times 10^{-8}$ 的值则直接变为零。混合精度训练论文给出两组统计：普通话语音识别模型的权重梯度中，约 5% 的值指数小于 $-24$，在 FP16 中会变为零，该文以此说明需要 FP32 主权重；Multibox SSD 检测网络的激活梯度中，大量数值低于 FP16 的可表示范围而变为零，其中落在 $[2^{-27}, 2^{-24})$ 的值对训练仍然重要。

[3]官方参数与 Adam / ZeRO 状态复算；FP32 梯度变体。本章使用规定的五项状态表示，其他优化器与低精度状态需另算。

**损失缩放**（loss scaling）解决这一问题：在反向传播前把损失乘以因子 $S$，梯度随之放大 $S$ 倍，进入 FP16 可表示的范围；参数更新前再除以 $S$。该文训练多种网络时使用的缩放因子在 8 到 32K 之间。因子过小，小梯度仍然下溢；过大，大梯度溢出为无穷大，必须跳过这次更新并调小因子。[4]

BF16 的八位指数与 FP32 相同，两者的表示范围一致，梯度不需要缩放，这是前述状态表采用 BF16 的原因之一。代价是尾数只有七位：矩阵运算输入的舍入误差大于 FP16，长内积的精度要靠 FP32 累加器保证。

继续压低位宽，就要把单一的全局缩放因子细化成分块缩放。FP8 的 E4M3 编码只有四位指数、三位尾数，最大正规数为 448，单一全局因子无法让各层激活同时落入如此窄的范围。DeepSeek-V3 的 FP8 训练按块管理缩放：激活按 $1 \times 128$ 小块（每个 token、每 128 个通道）求 scale，权重按 $128 \times 128$ 块求 scale，矩阵单元每累加 128 个元素就把部分和转为 FP32 继续累加。与 BF16 的对照训练相比，相对损失误差低于 0.25%。[5]

FP8 也改变容量账，但改变的只是送入矩阵单元的那份权重副本：DeepSeek-V3 的 FP8 训练把矩阵运算的操作数转成 FP8，主权重、梯度与优化器状态仍以更高精度保存。这份 FP8 操作数副本每参数 1 byte，每 $128 \times 128$ 块附加一个 FP32 scale，合计每参数约 $1 + 4/16384 \approx 1.0002$ bytes；对 Qwen3-8B，矩阵运算读取的权重从 BF16 的 16.4 GB 降为约 8.2 GB，而前述状态表中每参数 16 bytes 的常驻训练状态并不因此降到 1 byte。

| 矩阵运算的权重格式 | 每参数字节（含 scale） | Qwen3-8B 矩阵运算读取的权重 | 缩放管理 | 主要数值风险 |
|---|---|---|---|---|
| FP16 | 2 | 16.4 GB | 全局损失缩放 | 梯度下溢、缩放溢出 |
| BF16 | 2 | 16.4 GB | 无需 | 尾数较短，由 FP32 累加补偿 |
| FP8（E4M3）操作数副本 | 约 1.0002 | 约 8.2 GB（主权重与优化器状态另存） | 分块缩放与定期高精度累加 | 块内舍入、scale 过期 |

精度越低、缩放粒度越粗，节省的容量越多，数值偏离高精度参考的风险也越大。无论起因是缩放失效、数据异常还是硬件错误，训练偏离都表现为损失曲线上的尖峰（loss spike），常见的处理是回滚到上一份 checkpoint 重新训练。回滚损失多少进度由第 10.4 节的保存周期模型决定，第 10.4.5 节再把回滚的发生率计入该模型。

### 10.1.3 计算需求、MFU 与资源下界

显存决定模型能否开始训练，计算速度决定能否按期完成。同一个模型的状态可以分到多张卡上，但究竟需要多少张卡，还要根据总计算量和期限判断。一条 8,192 个 token 的序列在前向和反向传播中的矩阵计算量约为 $4.31 \times 10^{14}$ FLOPs，平均每 token 约 52.7 GFLOPs。处理 100B token 共需约 $5.27 \times 10^{21}$ FLOPs。这里的工作量由模型矩阵形状和前后向运算累计得到。[6]

先假设这 30 天全部用于训练计算。H100 SXM 的稠密 BF16 矩阵运算峰值为 989 TFLOP/s，达到峰值的 40% 时，一张卡约能处理 7,500 token/s。五张卡合计约 37,600 token/s，低于 30 天所需的约 38,600 token/s；至少需要六张卡才能满足计算需求。相比之下，两张 80 GB 卡已能容纳理想均分的 131.1 GB 常驻训练状态，却需约 77 天完成计算。

[4] 混合精度训练论文（损失缩放机制、8 至 32K 的缩放因子、FP16 可表示范围）、FP8 格式论文（E4M3/E5M2 编码与最大正规数）、DeepSeek-V3 技术报告 §3.3（1×128 激活与 128×128 权重分块缩放、每 128 个元素提升为 FP32 累加、相对损失误差低于 0.25%）。FP16 与 BF16 的指数、尾数比较见第 4.6.1 节。

[5] 混合精度训练论文（损失缩放机制、8 至 32K 的缩放因子、FP16 可表示范围）、FP8 格式论文（E4M3/E5M2 编码与最大正规数）、DeepSeek-V3 技术报告 §3.3（1×128 激活与 128×128 权重分块缩放、每 128 个元素提升为 FP32 累加、相对损失误差低于 0.25%）。FP16 与 BF16 的指数、尾数比较见第 4.6.1 节。

[6] 训练期限与设备下界及日历时间变体。精度、稀疏性与硬件来源在 JSON 中逐项记录。38%—43% 的 BF16 MFU 见 Llama 3 报告 §3.3.2 与表 4；RTX 4090 的 PCIe 4.0、无 NVLink 见 RTX 4090 规格页。

设算法所需的总计算量为 $F$，卡数为 $p$，单卡峰值为 $P$，统计时段内的平均计算效率为 $\eta$，则

$$T = \frac{F}{pP\eta}, \qquad p_{\text{compute}} = \left\lceil \frac{F}{P\eta T_{\text{available}}} \right\rceil .$$

本章按第 1.2.2 节的定义计算 MFU，分母取全部卡的峰值算力。若实际耗时取完整的训练迭代时间，通信等待已经计入其中；若只统计加速器执行计算的时间，得到的就是单卡计算效率。本章的设计案例采用后一种分解：单卡计算效率取 40%，随后把无法与卡内计算重叠的通信时间，以及等待输入数据的时间逐项加上。这样可以直接观察每项系统设计对完成时间的影响。40% 来自 Llama 3 的公开记录：405B 模型在 8,192 至 16,384 张 H100 上预训练，BF16 MFU 为 38%—43%。这组数按整步耗时统计，本身已包含通信等待，本章再单独加上通信，算出的完成时间因此偏长。RTX 4090 没有 NVLink，卡间数据都要经过 PCIe，这部分代价在第 10.3.3 节按实际链路单独计算。图 10-3 至图 10-5 保留 30% 与 50% 两档，用来观察效率偏离时卡数如何变化。[7]

另外六成峰值算力去了哪里，本章各节分别作答：无法与计算重叠的通信（第 10.3.3 节）、流水线气泡（第 10.3.2 节）、受内存带宽限制的参数更新与状态转换（第 10.2.3 节）、重计算（第 10.2.2 节）、掉队者（第 10.4.5 节）和 checkpoint 保存（第 10.4.3 节）。按第 1.3.4 节的判据，前四项是模型必须计入的工作，漏掉任何一项，估出的完成时间都会偏短；后两项和通信中没有重叠的部分则含有可以减少的开销，那才是系统设计的优化空间。

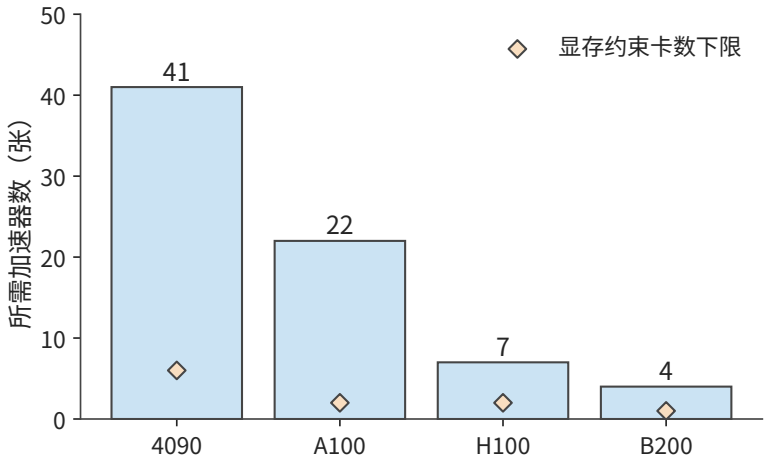



图 10-3：矩阵计算效率为 30% 时所需的卡数。任务为 Qwen3-8B 处理 100B token，序列长 8192，全部 30 天用于执行；蓝柱为计算需求，橙色菱形为训练状态容量下限。A100 为 80 GB SXM，H100 为 SXM。

图 10-3 将显存下限与计算下限放在一起：对本章任务，所需卡数主要由计算决定。下面据此选取两套具体配置，供后续各节逐步比较。RTX 4090 的对应峰值为 165.2 TFLOP/s。按 40% 效率和 30 天计算，需要至少 31 张卡。采用每台主机八张卡的配置后，可以先考虑 32 卡方案；再增加两台主机得到 48 张。两套方案中，每张卡分别累积 12 个和 8 个单序列 micro-batch，共同完成一个包含 384 条序列的全局 batch。增加卡数改变了工作分配，训练目标保持不变。

每次训练迭代约有 $1.66 \times 10^{17}$ FLOPs，使用 32 卡时，每张卡完成所分配的计算约需 78.3 s；使用 48 卡时，约需 52.2 s。相对 67.9 s 的预算，32 卡在加入通信前已经超时，48 卡则留下约 15.7 s。接下来要判断两套方案如何安排状态，以及哪些等待会消耗这 15.7 s。

[7] 训练期限与设备下界及日历时间变体。精度、稀疏性与硬件来源在 JSON 中逐项记录。38%—43% 的 BF16 MFU 见 Llama 3 报告 §3.3.2 与表 4；RTX 4090 的 PCIe 4.0、无 NVLink 见 RTX 4090 规格页。

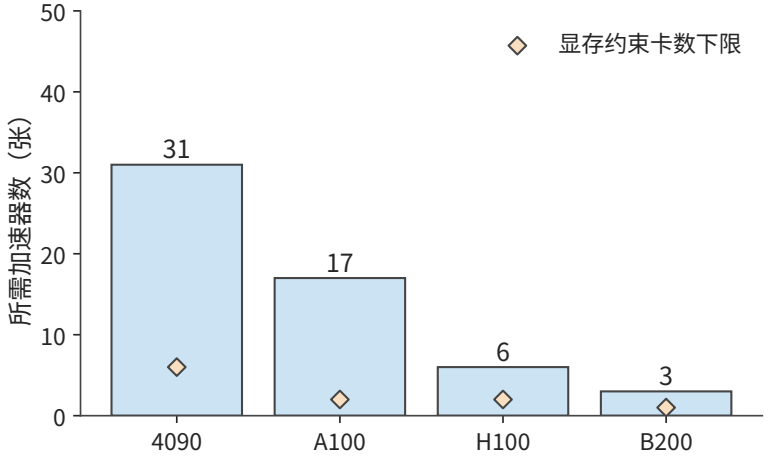



图 10-4：矩阵计算效率提高到 40%，同一任务所需的卡数下降；橙色菱形仍表示状态容量下限。各下限均以卡数计量；容量项是显存约束要求的最少卡数。

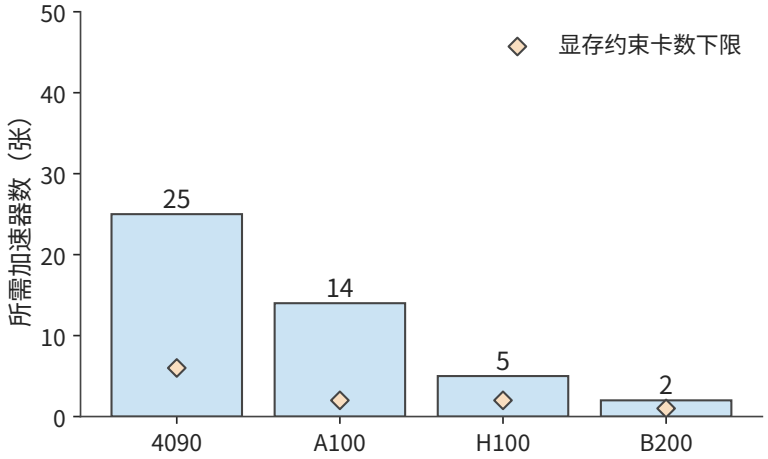



图 10-5：矩阵计算效率为 50% 时所需的卡数。三图使用相同的型号顺序、纵轴范围、任务与 30 天执行期限。各下限均以卡数计量；容量项是显存约束要求的最少卡数。

## 10.2 训练状态的分片、重计算与卸载

### 10.2.1 数据并行与 ZeRO ／ FSDP 状态分片

状态如何分配到多张卡上，由并行方式决定。同步数据并行让各卡处理不同样本，再用归约后的梯度更新各自的参数。各卡得到的新参数相同，因此每张卡通常都保存一份相同的权重与优化器状态；卡越多，复制的状态越多。ZeRO 由微软研究者在 2019 年提出、2020 年发表，用跨卡分片来减少大模型训练中重复保存的状态副本。ZeRO 分阶段划分这些状态的归属：每张卡只负责一部分参数的更新，其他卡通过通信取得执行所需的数据。

完全分片数据并行（Fully Sharded Data Parallel，FSDP）按同样的分片思路组织执行，参数、梯度与优化器状态都分片保存。下面用四张卡逐步改变状态的归属，再分析相应的执行方式。图中每列始终对应同一张卡；“完整”表示该卡保存整份状态，“片”表示只保存该卡负责的四分之一。

前两个阶段仍为每张卡保留完整模型权重。图 10-9 再划分权重的归属，长期驻留的空间进一步下降，代价

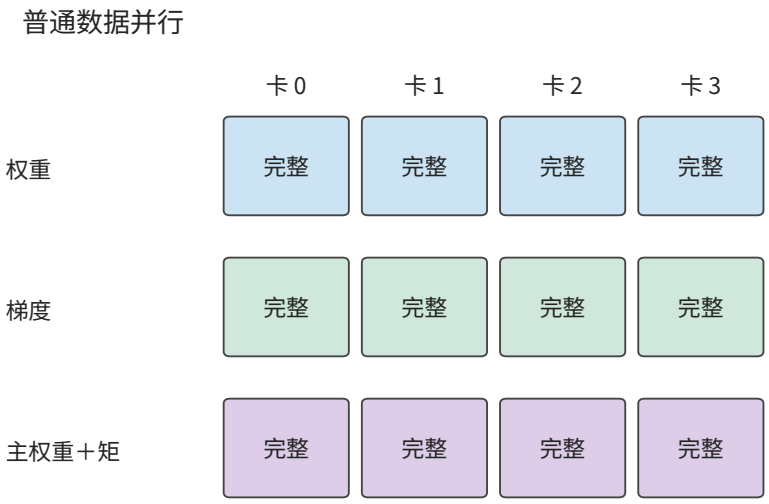



图 10-6：普通 DP：每卡保存完整权重、梯度、主权重和两份矩状态。

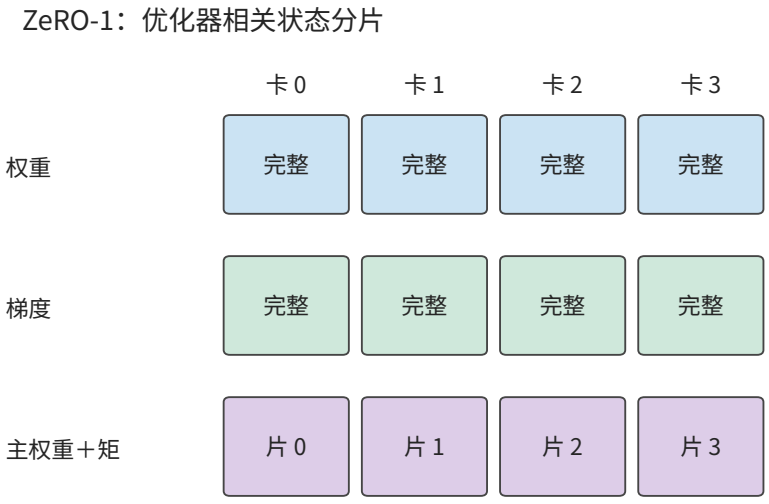



图 10-7：ZeRO-1：主权重和两份矩状态按参数分片，权重与梯度仍完整复制。

是执行每个模块前要通过通信取得所需的权重。

设数据并行组中有 $d$ 张卡。先计算常驻训练状态，普通数据并行和 ZeRO 三个阶段的每卡容量分别为

$$
\begin{aligned}
M_0 &= 16N, \\
M_1 &= 4N + 12N/d, \\
M_2 &= 2N + 14N/d, \\
M_3 &= 16N/d.
\end{aligned}
$$

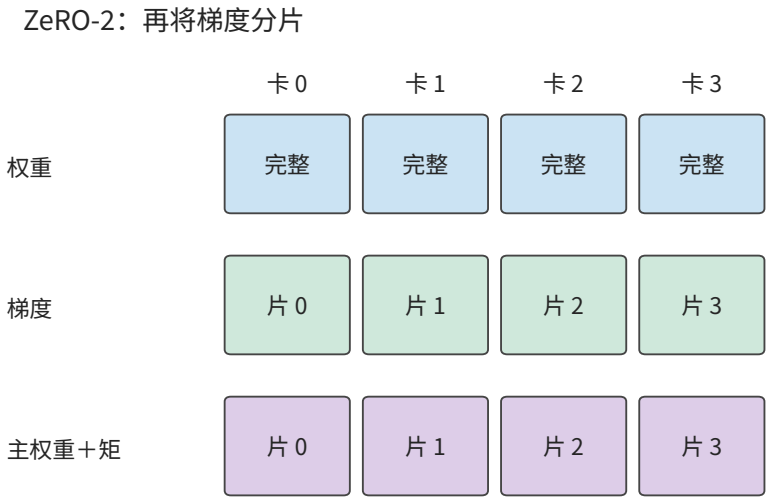



图 10-8：ZeRO-2：进一步划分梯度，每卡只保留归属于自己的梯度分片。

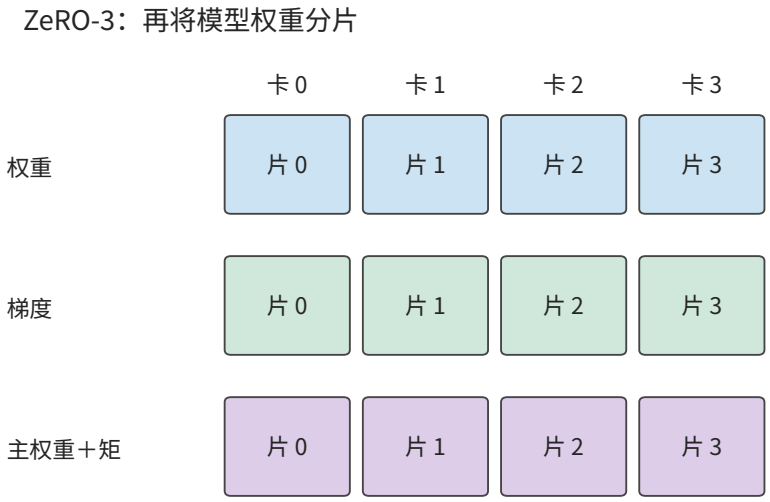



图 10-9：ZeRO-3：权重也分片。执行模块时，各卡通过通信取得所需的完整权重。

第一阶段分片 FP32 主权重和两份 Adam 状态，共 12 bytes/参数，权重与梯度仍各保留一份。第二阶段再把梯度分片，完整复制只剩 BF16 权重。第三阶段连权重也分片。将 $d = 8$ 代入，就得到下面的容量变化。

| 配置 | 每卡常驻训练状态 / GiB | 完整复制的训练状态 |
|---|---|---|
| 普通 DP | 122.1 | 权重、梯度与优化器状态 |
| ZeRO-1 | 42.0 | 权重与梯度 |
| ZeRO-2 | 28.6 | 权重 |
| ZeRO-3 | 15.3 | 常驻训练状态均已分片 |

分片以后，每个模块按以下顺序执行：先通过 AllGather 取得所需参数，执行前向或反向，再通过 ReduceScatter 汇总梯度并分配梯度分片，让负责该分片的卡得到全部梯度贡献，最后各自更新所负责的参数。FSDP 的全分片执行也围绕这条路径组织。若前向后释放完整参数，反向前就要再次收集；若保留完整参数，反向可直接使用，但完整参数会与后续激活同时占用显存。

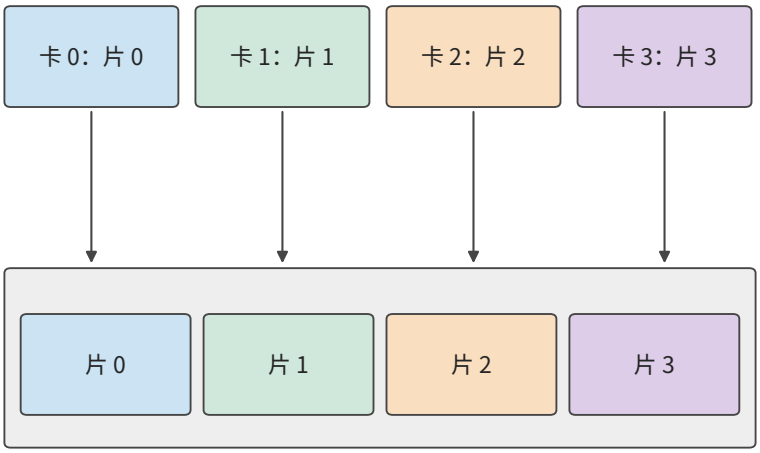



图 10-10：同一模块的权重平时分散在四张卡上；执行时，各卡收集完整权重。图中展开 GPU 0 的收集过程，四种颜色分别表示四份参数分片。完整权重缓冲区用完即可释放，各卡长期保存的原始分片仍然保留。

图 10-10 中，完整权重缓冲区与原有分片同时存在。增加卡数会缩小每份原始分片，完整权重缓冲区的大小却由模块本身决定。这就是常驻状态占用与执行峰值下降幅度不同的原因。

这一差别可以在小实验中直接观察到。一组用 PyTorch 第二代 FSDP 实现（FSDP2）做的 CPU 小模型实验，把参与的进程从两个增加到四个：参数更新后保留的状态从约 18 MiB 减到 9 MiB，执行峰值却只从约 39 MiB 减到 30 MiB。长期保存的部分减半，峰值只减少约 23%，因为执行中仍需约 21 MiB 的其他张量和缓冲区。因此，分析分片后的容量需求，还要考察模块执行时参数与激活各需保存多久。[8]

### 10.2.2 激活的保留与选择性重计算

前向计算留下的激活，像草稿纸上记录的中间步骤：反向计算回到这一层时，还要用到其中一些值。草稿全部保留，取用方便，却占空间；只保留部分步骤，缺失的值就要在使用前重新算出。参数分片已经减少了权重和优化器状态的复制，激活重计算则用额外的计算换取保存这些中间值所需的空间。

考虑矩阵运算 $Y = XW$，其中 $X$ 为输入、$W$ 为权重、$Y$ 为输出。前向计算结束后，$X$ 还要等到求权重梯度时使用。记损失为 $\mathcal{L}$，$dY = \partial\mathcal{L}/\partial Y$ 是后续计算传来的梯度，$dX$ 和 $dW$ 分别是损失对输入与权重的梯度；上标 $\mathsf{T}$ 表示矩阵转置。反向要计算

$$dX = dY W^{\mathsf{T}}, \qquad dW = X^{\mathsf{T}} dY.$$

[8]CPU FSDP2 四配置记录。CPU 小模型四种配置得到的参数与未分片参考结果一致；训练期间分配的内存峰值为 39.16 / 30.14 MiB。

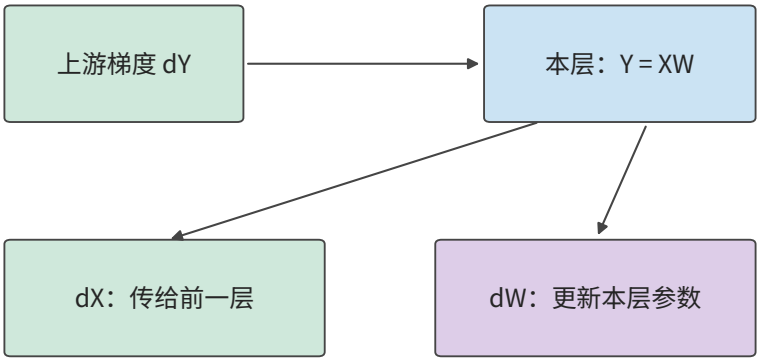



图 10-11：矩阵反向产生两路梯度：dX 交给前一层继续反向，dW 交给本层的归约和参数更新。两路计算都读取上游梯度 dY。

计算参数梯度需要前向输入 $X$。若保留 $X$，就要让它从前向计算时一直驻留到反向计算用完为止；释放 $X$，则要在反向使用前从已保留的输入或中间结果出发，重新执行相应的前向计算。选择哪些值保留、哪些值重算，要看节省多少空间，以及重新计算需要多少工作。

**例：反向传播前重算门控乘积能节省多少激活存储？** Qwen3-8B 的多层感知机（MLP，即本例的前馈子网络）先得到门控结果 $a$ 和上投影结果 $u$，再形成 $h = a \odot u$，最后计算 $Y = hW_{\text{down}}$。下投影的参数梯度需要 $h$。如果已为非线性算子的反向计算保留了 $a, u$，可以在下投影反向前重新相乘，就不必一直保留 $h$。

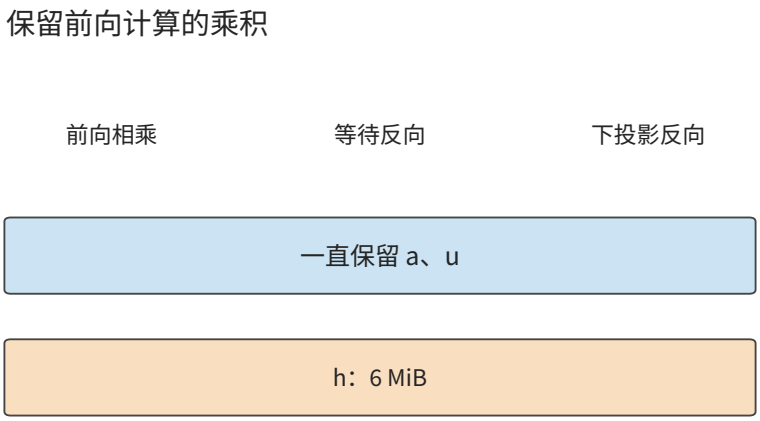



图 10-12：保留乘积 h：蓝色条表示一直保留的 a、u，橙色条表示从前向持续到下投影反向的乘积 h。128 个 token 的 micro-batch 采用 FP32，h 的形状为 [128,12288]，占 6 MiB。

重建后，只需在下投影反向即将开始时生成 h，并保留到这一步计算结束。这块缓冲区的大小如下：取 128 个 token 的 micro-batch 和 FP32 激活，$h$ 的形状为 $[128, 12288]$，占 $128 \times 12288 \times 4 = 6$ MiB；重建需要约 157 万次乘法。同一层的两个归一化加权输出形状均为 $[128, 4096]$，每个占 2 MiB，也可由已保存的归一化结果和权重重建。于是，每层省去 $6 + 2 + 2 = 10$ MiB，每个 micro-batch 在九层组成的流水阶段中可省去 90 MiB，约 94 MB。

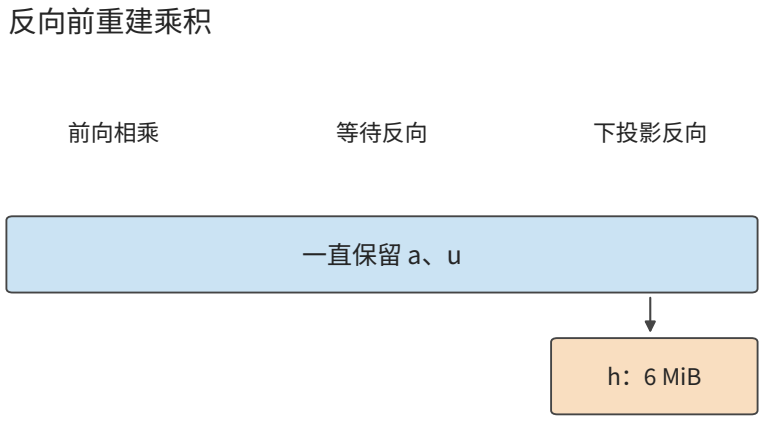



图 10-13：保留 a、u，反向使用前重新相乘得到 h。橙色的 6 MiB 缓冲只在使用前后短暂存在；横轴表示操作顺序。

原方案为这些 GEMM 保存的输入每层共占 15 MiB，其中上述三个乘积占 10 MiB，其余 Q、K、V 和注意力输出占 5 MiB。九层阶段因此从 135 MiB 降到 45 MiB，即从约 142 MB 降到 47 MB。重建按层依次发生，最大的乘积工作区为 6 MiB；上一层用完后，下一层复用这块空间。[9]

若有四个 micro-batch 同时等待反向计算，需要长期保存的数据就能减少 $4 \times 90 = 360$ MiB，约 377 MB，反向时则逐个 micro-batch 执行重建计算。这里的收益来自数据生命周期的差别：等待反向计算的 micro-batch 越多，需要长期保存的数据越多；逐层重建时，却只需反复使用同一块工作区。流水调度若让反向更早开始，等待反向计算的 micro-batch 数量又会下降。因此，重计算与流水调度共同决定需要保存多少数据；新增的计算是否延长每步训练时间，则取决于它在时间线上的位置。

### 10.2.3 CPU ／ GPU 卸载与缓冲区的使用时间

用已保存的结果重建数据的开销较小时，适合重计算。另一种办法是保留数据本身，把它移到容量更大的主机内存，使用前再传回 GPU，这就是**卸载**。选择卸载以后，原先的显存问题就变成了传输能否及时完成的问题。假设一份状态大小为 $V$，链路有效带宽为 $B$，从可以开始传输到算子需要这些数据之间有 $W$ 秒，则超出这段时间的传输耗时为

$$T_{\text{wait}} = \max(0, V/B - W)$$

相应算子需要等待这段时间。传输和计算能重叠多少，取决于数据何时可发、链路何时空闲、相应算子何时需要这些数据。DMA 由设备发起数据搬移，传输期间源数据必须保留。源缓冲在 DMA 结束前保持有效，目标缓冲从开始接收时就占用内存；卸载同时改变两端缓冲区的使用时间。

**例：在 CPU 还是 GPU 上转换梯度，能更快完成卸载？**门控投影的梯度形状为 $[12288, 4096]$，BF16 大小 $G = 96$ MiB，FP32 为 192 MiB。CPUAdam 是在 CPU 上执行 Adam 更新的实现，它使用 FP32 梯度，可以先传 BF16 再在 CPU 转换，也可以先在 GPU 转换再传 FP32。转换都要读 $G$、写 $2G$，共访问 $3G$ bytes。转换受内存带宽限制：GPU 取 RTX 4090 的 1008 GB/s，CPU 取第四代至强单路八通道 DDR5-4800 的 307.2 GB/s。记 CPU、GPU 转换吞吐为 $C_c, C_g$，链路有效带宽为 $B$，串行路径时间为

[9]填满排空事件模型、1F1B 事件模型、GEMM 输入张量的保留关系、选择性重计算。GEMM 输入采用 FP32 参考表示，九层的可重建乘积为 90 MiB，最大逐层重建工作区 6 MiB。流水图采用 save_nonlinear 策略保存的中间结果及其收发缓冲。

$$T_c = G/B + 3G/C_c, \qquad T_g = 3G/C_g + 2G/B.$$

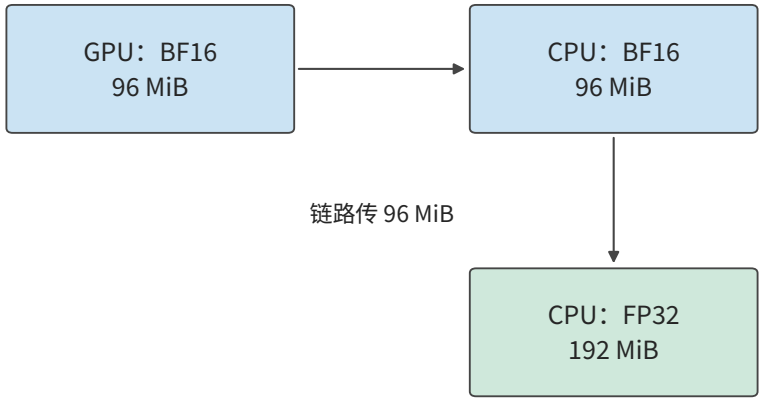



图 10-14：先将 96 MiB BF16 梯度传到 CPU，再在 CPU 转为 192 MiB FP32。箭头表示数据流；转换的输入与输出均位于 CPU 一侧。

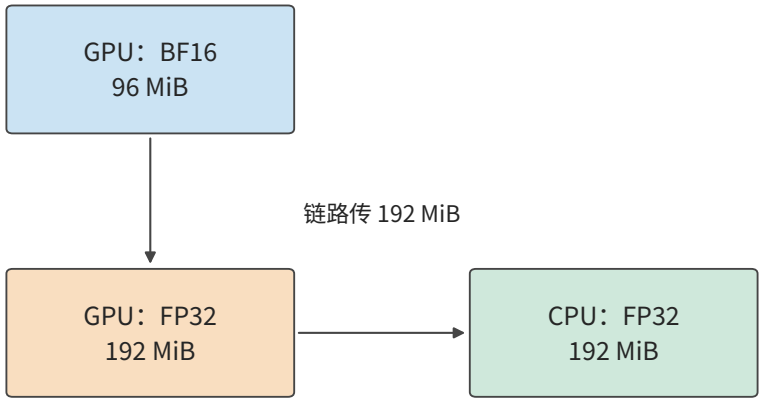



图 10-15：先在 GPU 将 96 MiB BF16 梯度转为 192 MiB FP32，再传到 CPU。转换时输入与输出共存，GPU 峰值为 288 MiB。

图 10-14 与图 10-15 的差别集中在跨越链路的箭头上：GPU 虽然转换更快，却要多传 96 MiB。链路越慢，这部分额外传输越容易抵消转换收益。

RTX 4090 经 PCIe 4.0 x16 连接主机，每方向 32 GB/s。在这条链路上，CPU 路径约为 $3.15 + 0.98 = 4.13$ ms，GPU 路径约为 $0.30 + 6.29 = 6.59$ ms。GPU 路径的转换少约 0.68 ms，传输却多约 3.15 ms，因此更慢。

若链路换成 GH200 中 CPU 与 GPU 之间的 NVLink-C2C，每方向 450 GB/s，多传 96 MiB 只增加约 0.22 ms，两条路径变为约 1.21 ms 和 0.75 ms，GPU 转换获得明显收益。[10]

令两条路径耗时相等，可以求出两种方式快慢互换的带宽分界点：

$$B^* = \frac{1}{3(1/C_c - 1/C_g)} \approx 147 \text{ GB/s}.$$

该阈值把两种相反作用放在同一尺度上：GPU 转换更快，节省了格式转换时间；FP32 数据更大，增加了传输时间。链路越快，后一项代价越小。147 GB/s 高于 PCIe 5.0 x16 的每方向 64 GB/s，所以经 PCIe 连接的卡在本算例中都应先传 BF16、在 CPU 上转换。

显存容量还带来另一项限制。GPU 转换期间原始 96 MiB 与输出 192 MiB 共存，峰值为 288 MiB；先传 BF16 的路径则只需在 GPU 上保留参与这次转换的 96 MiB 梯度。选择 GPU 转换需要额外 192 MiB 缓冲区。SuperOffload 是面向 GH200 这类 CPU 与 GPU 紧耦合芯片的训练卸载系统，它把转换位置和主机上的优化器执行一起安排，利用的正是转换速度、传输数据量和缓冲区使用时间之间的取舍。整步收益则由最后一桶梯度、CPU 上的参数更新和权重回传何时完成决定。

### 10.2.4 组合并行方案中的每卡状态分配与互联拓扑

分片、重计算和卸载分别改变存储量、计算量和传输量。把这些办法组合成完整系统时，还要确定它们作用于哪些卡和通信组。第 6、7 章的并行方法在训练中划分不同的数据和计算：TP 切分层内矩阵，PP 划分层，DP 划分样本，EP 划分专家。第 6.1.4 节的一览表还包括 SP 与 CP：前者让逐 token 算子的激活沿序列分片，通常复用张量并行组；后者把同一条长序列的注意力上下文分到多张卡，保留跨位置的依赖。这些并行方法延续第 5 章“分块—安排输入—汇合结果”的方法，训练再沿相同计算图逆向传播梯度。先确定每份参数由谁更新，再沿前向和反向的依赖关系安排收集、交换与归约，就能同时得到每卡容量和各接口流量。

三种容量优化可以用同一组问题比较。

| 方法 | 减少的显存占用 | 增加的工作 | 决定收益的时间关系 |
|---|---|---|---|
| 状态分片 | 重复保存的权重、梯度或优化器状态 | 参数收集、梯度归约并分片 | 收集能否赶在模块执行前完成 |
| 激活重计算 | 前向到反向之间保存的中间值 | 重建前向值 | 重建是否延迟关键路径上的反向计算 |
| 状态卸载 | 加速器上暂时不用的状态 | 主机与加速器间传输 | 数据能否在使用前传回 |

**例：显存不足时，应增加分片的卡数还是重计算激活？** RTX 4090 标称 24 GB，约合 22.35 GiB；扣除运行时占用，每卡按 22 GiB 可用显存计。Qwen3-8B ZeRO-3 在八卡上每卡保存约 15.3 GiB，留给其他张量和缓冲区约 6.7 GiB。如果激活与临时缓冲区同时需要 10 GiB，总量约 25.3 GiB，超出约 3.3 GiB。此时可以用重计算节省这 3.3 GiB，也可以增至十六卡，将常驻训练状态降到约 7.6 GiB，总量降至约 17.6 GiB。

两种选择对应不同的代价：八卡方案增加重建计算，十六卡方案增加卡数并改变通信组。可以先从图中找出满足显存要求的方案，再比较这些方案的每步耗时。

[10] 梯度转换复算、快链路变体、SuperOffload 阅读与转换案例、CPUAdam 卸载记录。转换路径从 GPU 梯度就绪开始，到 CPU 可读取 FP32 梯度结束；两条路径均使用页锁定主机内存作为缓冲区。GPU 转换吞吐取 RTX 4090 的 1008 GB/s 显存带宽（calculations/configs/hardware.json）；CPU 转换吞吐取至强 Platinum 8480+ 规格的八通道 DDR5-4800，$8 \times 4800$ MT/s $\times$ 8 B = 307.2 GB/s；两者都是按内存带宽计的上限。NVLink-C2C 每方向 450 GB/s 见 GH200 架构说明。

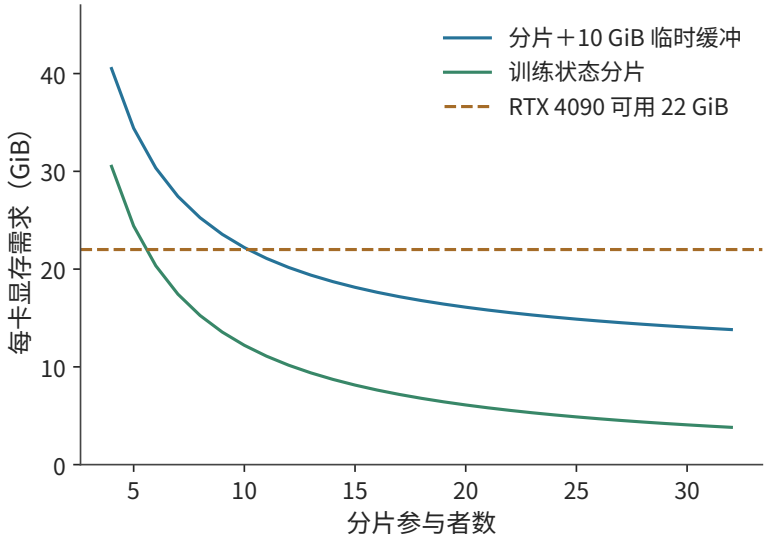



图 10-16：给定附加存储需求时，分片数决定哪些方案满足容量要求。曲线为 $16N/d + 10$ GiB，水平线为 RTX 4090 每卡 22 GiB 可用显存。曲线低于预算的区域可以容纳这些训练状态和缓冲区；纵向距离给出容量余量。

针对本章的训练任务，32 卡与 48 卡方案均采用全组 ZeRO-3，TP=PP=1，每卡 micro-batch 为一条序列。与上例相同，每卡按 22 GiB 可用显存计，并为同时保留的激活、完整模块参数和工作区安排 10 GiB 上限。两套方案常驻训练状态分别约为 3.8、2.5 GiB，总量约为 13.8、12.5 GiB，容量余量约为 8.2、9.5 GiB。两套方案的显存都足够，接下来主要比较完成时间。[11]

## 10.3 训练步的流水调度与通信重叠

容量分析确定显存能够容纳所需数据，时间分析确定数据能否及时到达。本节先明确一次训练迭代所优化的目标函数，再把前向、反向与通信放在同一时间轴上。本章的设计案例采用数据并行；四阶段流水单独作为算例，说明调度会同时改变关键路径和激活的生命周期。

### 10.3.1 micro-batch、梯度累积与参数更新

全局 batch 包含一次参数更新使用的全部样本，可以分成多个 micro-batch 依次执行反向传播，累积梯度后统一更新参数。设 micro-batch $k$ 中有效 token 的索引集合为 $S_k$，有效 token 总数为 $L = \sum_k |S_k|$，token 平均损失为

$$\mathcal{L} = \frac{1}{L} \sum_{k} \sum_{j \in S_k} \ell_j .$$

求和与求导可以交换顺序，因此可以分别对各个 micro-batch 执行反向传播；计算每个 micro-batch 的损失时，都用其损失总和除以同一个 $L$。所有 micro-batch 的梯度累积完成后，执行一次梯度裁剪和一次参数更新。这样划分 micro-batch 只改变执行顺序，不改变各个 token 在损失函数中的权重。

[11] 本章设计案例的输入、推导与复算。32 / 48 卡的容量预留、输入等待和计划停顿为给定输入；单卡计算效率取 Llama 3 的 MFU；模型矩阵工作来自既有 training-deadline 计算，ZeRO-3 通信量由同一份参数表推出。RTX 4090 的 PCIe 4.0、无 NVLink 见 RTX 4090 规格页，卡间不支持 P2P、八卡主机配八张 200 Gbit/s 网卡见 4090 集群配置分析；一步中只有第一次 AllGather 与最后一次 ReduceScatter 无法隐藏见 MegaScale §3.2。逐项数值与独立复算保存在 design-case.json。

**例：按 micro-batch 还是按 token 求平均，为何会改变梯度？**八条回答中，四条各含两个有效 token，四条各含三个，共 20 个。设短回答每 token 的损失为 $\theta/5$，长回答为 $2\theta/5$，则

$$\mathcal{L} = \frac{8\theta/5 + 12(2\theta/5)}{20} = \frac{8\theta}{25}, \qquad \frac{d\mathcal{L}}{d\theta} = 0.32.$$

若先求每条回答的平均损失，再对八条回答取平均，短回答与长回答各占一半权重，梯度变成 $(0.2+0.4)/2 = 0.30$。原先按 token 平均时，长回答占 $12/20 = 60\%$，所以结果更靠近 0.4。若已经使用 20 这一共同分母，累积后又除以八，则梯度变为 0.04。分母决定每个 token 对参数梯度的贡献。[12]

回到设计案例：第 10.1.3 节的 384 条完整长度的序列，在 32 卡上每卡累积 12 个 micro-batch，在 48 卡上每卡累积 8 个。每卡一次只处理一条，每张卡执行的矩阵形状相同；全局有效 token 数和目标函数也相同。参数收集和梯度通信则随执行路径另行排入时间轴。

### 10.3.2 流水线调度、气泡与激活的生命周期

micro-batch 的划分保持了训练目标，却改变了计算的先后顺序。在流水线并行里，顺序还决定每个阶段何时能开始计算，以及有多少份激活在等待反向。流水线并行把模型分成多个阶段，每个 micro-batch 依次前向，再沿反方向传播梯度。**填满排空（fill–drain）调度**先完成所有 micro-batch 的前向传播，再集中执行反向传播。**1F1B（one forward, one backward）**在流水线预热后交替执行一次前向和一次反向，让较早的 micro-batch 更快释放激活。

激活释放的快慢直接决定显存峰值。反向传播要用到前向保存的激活，因此每个 micro-batch 的激活从前向完成起就必须留在显存里，直到对应的反向到达。同一时刻有多少个 micro-batch 在等待反向，显存里就有多少份激活。填满排空要等全部 $m$ 个 micro-batch 都完成前向才开始第一个反向，峰值随 $m$ 线性增长；一旦超出显存容量，这一训练步就无法执行，只能减少同时在流水线中的 micro-batch、缩短序列，或改用代价更高的重计算换取可行性。提前执行反向就是提前结束激活的等待：峰值降低后训练步才能容纳，或者释放的余量可以留给更大的 micro-batch 和更长的序列。显存驻留因此往往不是快慢问题，而是能否运行的问题。

比较任何两种调度，都要同时考察两根轴：一根是**完成时间**，取决于流水线中空闲等待（气泡）的大小；另一根是**显存驻留**，取决于每份激活要保存多久才能等到自己的反向。

先设 $p$ 个阶段完全平衡，每阶段的前向耗时为 $t_f$、反向耗时为 $t_b$，传输和参数更新耗时取零。$m$ 个 micro-batch 填满排空需要

$$T = (m + p - 1)(t_f + t_b), \qquad u = \frac{m}{m + p - 1}.$$

式中的 $m$ 项对应 micro-batch 的实际计算工作，$p-1$ 项来自填充和排空。卡的时间利用率 $u$ 因而随 micro-batch 数量增加而提高。取 $p = 4, m = 8$，得到 $u = 8/11 \approx 72.7\%$。若保持全局 batch size 不变，增加 micro-batch 数量意味着缩小每个 micro-batch；由此减少的流水线空闲时间，需要与更小矩阵的执行时间一起比较。

**例：气泡相同，1F1B 换来了什么？**将 Qwen3-8B 的 36 层分为四个九层阶段，八个 micro-batch 各含一条 128 个 token 的序列。每阶段前向 10 ms、反向 20 ms，边界传输每次 1 ms，正反方向链路独立，参数更新耗时 1 ms。填满排空调度无传输时为 $(8 + 4 - 1) \times 30 = 330$ ms；前向填充经过三条边界增加 3 ms，反向排空再增加 3 ms，参数更新增加 1 ms，总计 337 ms。

[12] 固定 verl 小模型真实闭环、loss 与更新的源码核对。固定 Qwen2.5-0.5B-Instruct 训练配置采用单卡 NO_SHARD、no_sync=False；源码和版本锁定见链接中的记录。0.32 / 0.30 / 0.04 使用真实长度结构与教学标量损失推导。

1F1B 更早开始反向，后续的前向也就要等反向释放计算资源。在阶段 0，前四个 micro-batch 于 40 ms 完成前向，第一份传回的梯度于 106 ms 到达。随后反向执行到 126 ms，第五个 micro-batch 才开始前向。这种交错逐级影响下游的到达时间。阶段 3 在 93 ms 完成第二个 micro-batch 的反向，但第三个前向输入到 95 ms 才到，产生 2 ms 空隙；后续相似的等待继续延长关键路径。按这些依赖关系排定执行顺序后，所有梯度在 346 ms 时计算完毕，加上参数更新共为 347 ms。[13]

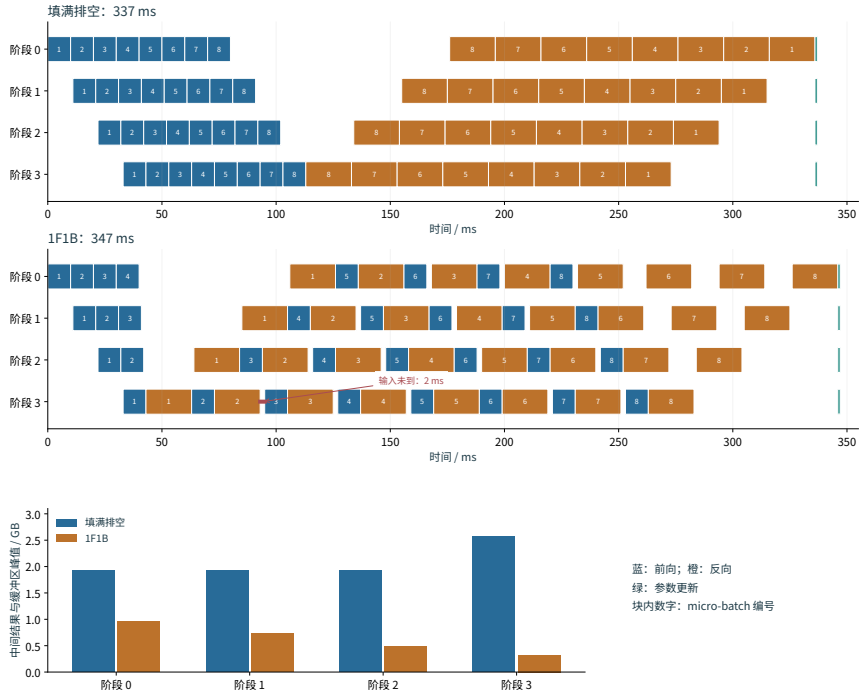



图 10-17：填满排空先完成八个 micro-batch 的前向，再执行反向，最后更新参数。蓝色为前向，橙色为反向，绿色为参数更新。四个阶段共用同一时间刻度，按正文的计算与传输条件共需 337 ms。

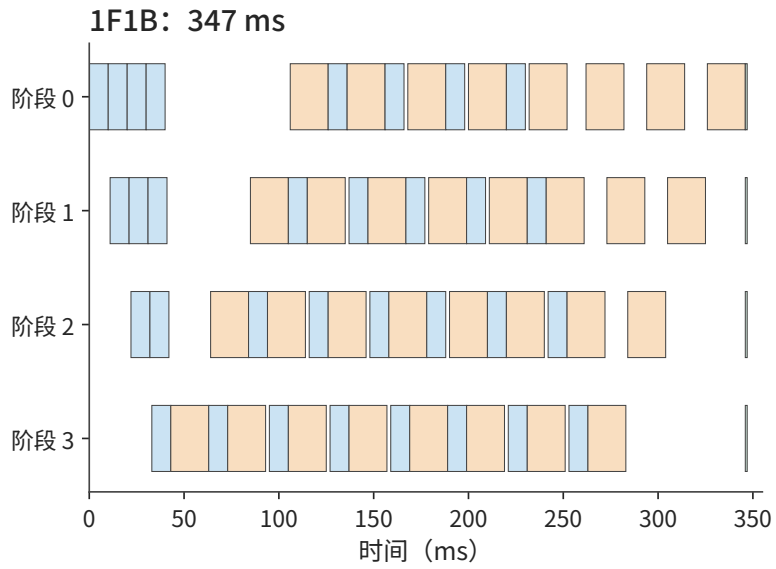



图 10-18：1F1B 在预热后交错前向和反向，本例完成时间为 347 ms。蓝色为前向，橙色为反向，绿色为参数更新；与前图使用相同时间刻度。

两图对比可以看出：两种调度做相同的前后向工作，气泡时间同为 $(p-1)(t_f+t_b)$；1F1B 反而因交错引入

[13]填满排空事件模型、1F1B 事件模型、GEMM 输入张量的保留关系、选择性重计算。GEMM 输入采用 FP32 参考表示，九层的可重建乘积为 90 MiB，最大逐层重建工作区 6 MiB。流水图采用 save_nonlinear 策略保存的中间结果及其收发缓冲。

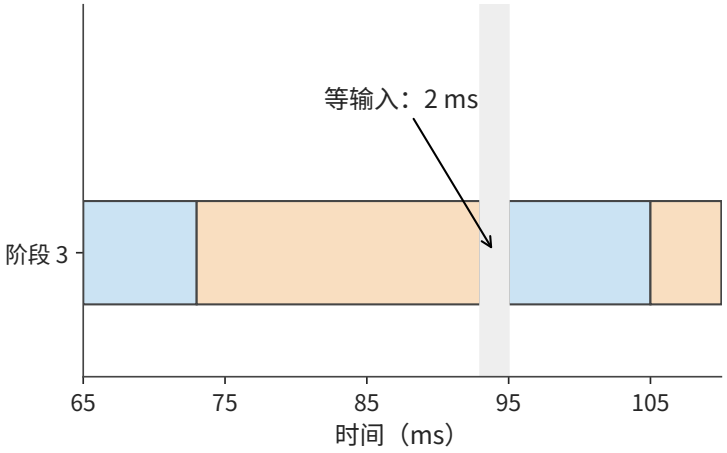



图 10-19：放大 1F1B 的阶段 3：第二个 micro-batch 在 93 ms 结束反向，下一份前向输入于 95 ms 到达，形成 2 ms 等待。

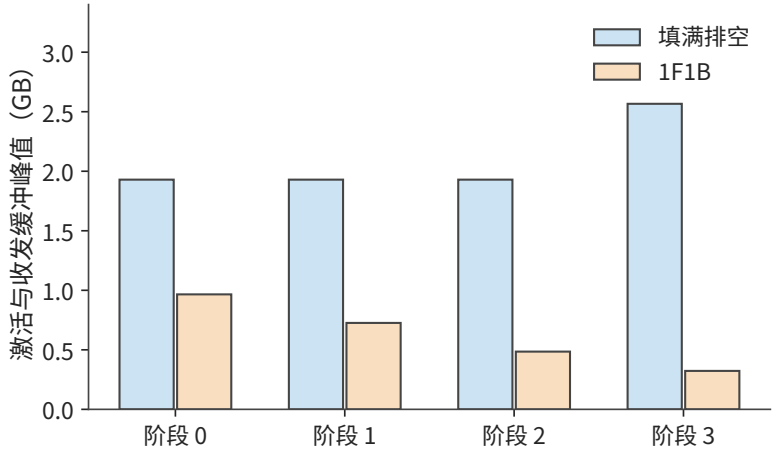



图 10-20：两种调度中各阶段的激活与收发缓冲峰值。提前反向使激活更早释放，最大值由约 2.57 GB 降到 0.97 GB。

的依赖等待（图 10-19）多用约 3% 的时间，收益则体现在图 10-20——中间结果与缓冲区的最大显存占用从约 2.57 GB 降到 0.97 GB，减少约 62%。因此“1F1B 优于填满排空”只在显存这根轴上成立，在时间轴上反而略慢。若只给这些中间结果和缓冲区留 1 GB，只有提前反向的调度能执行这一训练步；容量充裕时，337 ms 的填满排空调度完成得更早。选择哪种调度方式，由“何时释放张量”和“何时得到下一份输入”共同决定。

要缩短完成时间，就必须压缩气泡本身：反向拆得更细、同时流动的 micro-batch 更多，空隙就能填得更满。下面三类调度都建立在上述依赖分析之上，分别用**通信次数**、**激活峰值**和**参数量**换取更小的气泡。三者沿用同一算例（四个阶段、前向 10 ms、反向 20 ms、边界传输 1 ms、参数更新 1 ms），各 micro-batch 在每个阶段上的执行顺序（槽位顺序）由事件模型排定，气泡公式与反向拆分的思路来自已发表的论文。[14]

**交错式 1F1B**（interleaved 1F1B，又称虚拟流水线）把 36 层切成八个块，轮流分给四张卡，每卡得到 $v = 2$ 个不连续的块，合计仍是九层：前四块各五层、后四块各四层，卡 0 执行第 0—4 层与第 20—23 层，其余各卡依次类推。每个 micro-batch 的前向因此跨越边界 $2p - 1 = 7$ 次而不是 3 次，气泡比例则从 $(p-1)/m$ 缩小为

[14]交错式事件模型、零气泡事件模型、DualPipe 事件模型及十六个 micro-batch 的变体（交错式、零气泡、DualPipe）。气泡公式：1F1B 与交错式见 Megatron-LM 论文（$(p-1)/m$ 与 $(1/v)(p-1)/m$），ZB-H1/H2 见零气泡论文表 2，DualPipe 的参数量与激活份数见 DeepSeek-V3 技术报告表 2；1F1B 的稳态定义见 PipeDream。零气泡按论文图 3 上半部分的 ZB-H1 槽位顺序排定；交错式与 DualPipe 的归档文本只给出分块与双向馈入的思路，槽位顺序按各结果文件记录的 declared_schedule_rule 排定；$T_B = T_W = 10$ ms 的拆分比例为本例取值；完成时间由事件模型按依赖关系推出，不是实测。

$$\frac{1}{v}\cdot\frac{p-1}{m}.$$

八个 micro-batch 时完成时间从 347 ms 降到 298 ms，每卡空闲从约 106 ms 降到约 57 ms。代价有两项：传输次数增加一倍以上（全步前向消息从 24 次增到 56 次），以及较早阶段的激活要等待更多块完成前向——阶段 0 的激活与收发缓冲峰值从 921 MiB 升到约 1,329 MiB。

**零气泡**（zero-bubble）调度把反向拆成两段：输入梯度 $dX$（论文记作 B）要传给上一阶段，上一阶段的反向在等它，所以 $dX$ 位于关键路径上，应尽早执行；权重梯度 $dW$（论文记作 W）只交给本层的梯度累积，可以放在本阶段对应 $dX$ 之后的任意时刻，因而推迟到空隙中执行。设每阶段前向、$dX$、$dW$ 的耗时分别为 $T_F$、$T_B$、$T_W$，则 1F1B 的气泡为 $(p-1)(T_F+T_B+T_W)$；ZB-H1 把 $dW$ 填进空隙，气泡降为 $(p-1)(T_F+T_B-T_W)$；ZB-H2 再把更多前向提前，气泡降为 $(p-1)(T_F+T_B-2T_W)$。零气泡论文的两种手工调度按 $T_F=T_B=T_W$ 构造，论文表 2 给出上述一般形式。

本例把 20 ms 的反向对半拆成 $T_B=T_W=10$ ms：1F1B 的气泡为 $3\times30=90$ ms，ZB-H1 的下界为 $3\times(10+10-10)=30$ ms，ZB-H2 的下界为 0。按论文图 3 的 ZB-H1 槽位顺序排定事件模型后，八个 micro-batch 的完成时间为 283 ms，每卡空闲 42 ms；把边界传输取零，空闲恰好是 30 ms 的下界，多出的 12 ms 来自传输。阶段 0 的激活与收发缓冲峰值与 1F1B 相同，都是 921 MiB。代价落在靠后的阶段：阶段 3 的 $dW$ 比 $dX$ 晚三个 micro-batch 执行，事件模型把这段时间里的激活按完整一份保留，阶段 3 的峰值从 1F1B 的 308 MiB 升到约 1,226 MiB。论文按 $dW$ 实际所需的较小激活量计算，其表 2 中 ZB-H1 的峰值与 1F1B 同为 $pM_B$（$M_B$ 为一个 micro-batch 的激活量）。

**DualPipe** 把 micro-batch 分成两半，从流水线两端相向注入，每个阶段同时承载两个方向的块。沿用 DeepSeek-V3 的记号，它的气泡为 $(PP/2-1)(F\&B+B-3W)$，其中 $PP$ 为流水线阶段数，$F\&B$ 为前向与反向重叠执行的时段，$B$ 指完整反向 20 ms，$W$ 指其中的 $dW$ 10 ms。本例不模拟前向与反向的计算重叠，取 $F\&B=F+B=30$ ms，气泡为 $(2-1)\times(30+20-30)=20$ ms，完成时间为 306 ms。代价同样直接：每卡要保存两个方向的参数副本，参数量翻倍，激活保留 $PP+1$ 份；本例各阶段峰值约为 1,228—1,612 MiB。[15]

下表把这五种调度放在同一算例下对照。填满排空虽然没有压缩气泡，仍是容量充裕时的基准，一并列出：

| 调度（八个 micro-batch） | 完成时间 / ms | 每卡空闲 / ms | 阶段 0 峰值 / MiB | 压缩气泡的手段 | 主要代价 |
|---|---|---|---|---|---|
| 填满排空 | 337 | 96 | 1,840（阶段 3 为 2,448） | 无 | 激活驻留最高，容量紧张时不可行 |
| 1F1B | 347 | 106 | 921 | 无（气泡与填满排空相同） | 显存最省，但比填满排空慢约 3% |
| 交错式 1F1B，$v=2$ | 298 | 57 | 1,329 | 每卡层块拆成 $v$ 段 | 前向消息 24 → 56 次 |
| 零气泡（ZB-H1），$T_B=T_W$ | 283 | 42 | 921（阶段 3 为 1,226） | $dW$ 延后填入空隙 | 靠后阶段的激活要保留到 $dW$ 执行 |
| DualPipe | 306 | 65 | 1,228（阶段 1、2 为 1,612） | micro-batch 从两端相向流动 | 每卡参数两份 |

[15] 交错式事件模型、零气泡事件模型、DualPipe 事件模型及十六个 micro-batch 的变体（交错式、零气泡、DualPipe）。气泡公式：1F1B 与交错式见 Megatron-LM 论文（$(p-1)/m$ 与 $(1/v)(p-1)/m$），ZB-H1/H2 见零气泡论文表 2，DualPipe 的参数量与激活份数见 DeepSeek-V3 技术报告表 2；1F1B 的稳态定义见 PipeDream。零气泡按论文图 3 上半部分的 ZB-H1 槽位顺序排定；交错式与 DualPipe 的归档文本只给出分块与双向馈入的思路，槽位顺序按各结果文件记录的 declared_schedule_rule 排定；$T_B=T_W=10$ ms 的拆分比例为本例取值；完成时间由事件模型按依赖关系推出，不是实测。

把表中的数字画到各阶段的时间线上，就能看出气泡在哪里被填掉。下面三幅图与图 10-17、图 10-18 使用相同的时间刻度和颜色。

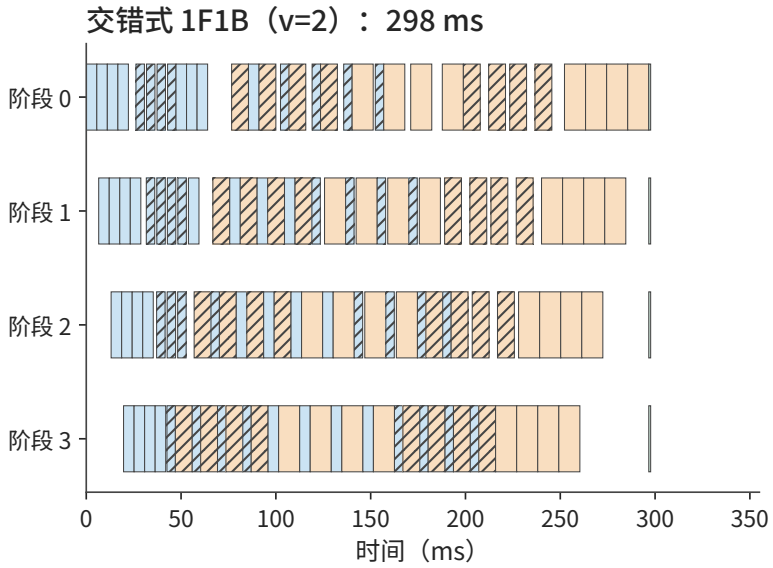



图 10-21：交错式 1F1B（$v = 2$）的各阶段时间线，完成时间 298 ms。蓝色为前向，橙色为反向，绿色为参数更新，斜线块为每卡的第二个层块。每个块约为图 10-18 中的一半长，预热与排空阶段的空隙被另一个层块的计算填上，阶段 3 的第一个前向从 33 ms 提前到约 20 ms；每个 micro-batch 要多穿过四条边界。

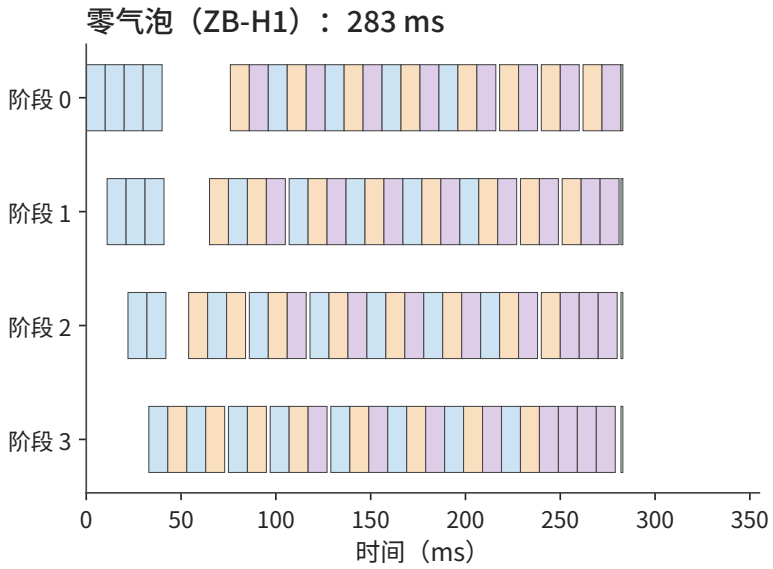



图 10-22：零气泡（ZB-H1）的各阶段时间线，完成时间 283 ms。蓝色为前向，橙色为 $dX$，紫色为 $dW$，绿色为参数更新。稳态中每个阶段按前向、$dX$、$dW$ 轮转，图 10-18 里反向之间的依赖空隙和末尾的排空空隙都被延后的 $dW$ 填上，四个阶段几乎同时结束；阶段 3 的 $dW$ 比 $dX$ 晚三个 micro-batch，那里的激活保留得最久。

没有一种调度在两根轴上同时占优。把五种调度分别按两根轴排序，第一名不是同一种调度：完成时间最短的是零气泡，最大单阶段峰值最低的是 1F1B。三种扩展调度压缩气泡换来的时间，都以另一种资源的增加为代价，所以选择调度时要先确定约束在哪一边：时间还是容量。图 10-24 把两根轴并排画出（未画填满排空），可与上表对照。

micro-batch 翻倍到 16 个时，1F1B、交错式、零气泡与 DualPipe 的完成时间分别为 599、538、523、558 ms，排序不变。压缩气泡是否值得，取决于节省的气泡时间与新增代价的相对大小。$m$ 增大时，气泡比例 $(p-1)/m$ 本身就在下降——16 个 micro-batch 时 1F1B 的气泡占比已从 37.5% 降到 18.8%，拆细带来的相对收益随之缩

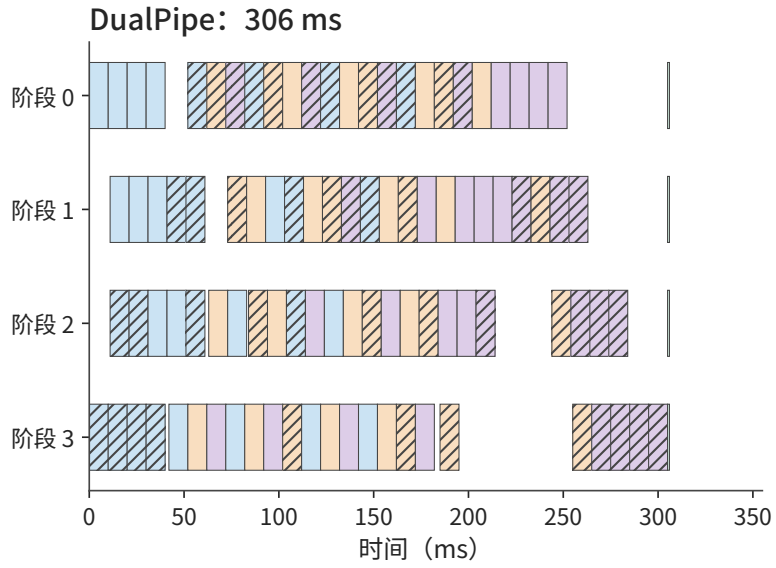



图 10-23：DualPipe 的各阶段时间线，完成时间 306 ms。颜色同图 10-22，斜线块为从阶段 3 进入的另一半 micro-batch（反方向）的计算。每个阶段同时承载两个方向，一个方向的预热和排空空隙由另一个方向的块填上；代价是每卡保存两份参数。

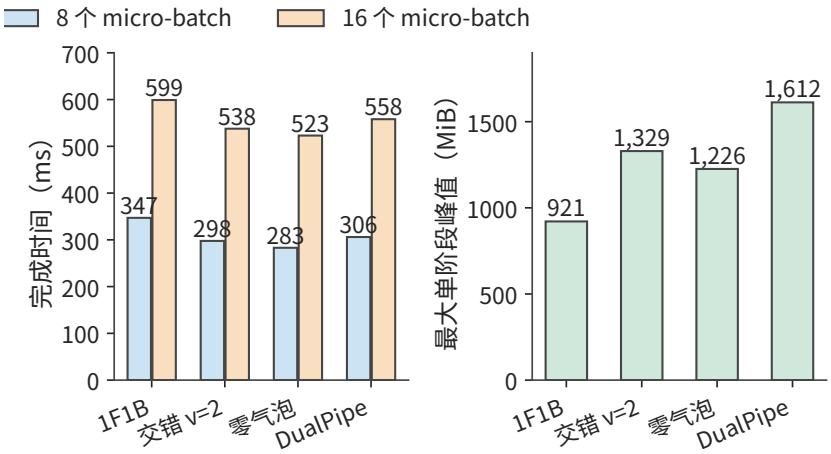



图 10-24：四种流水线调度在八个与十六个 micro-batch 下的完成时间，以及八个 micro-batch 时各调度最大的单阶段激活与收发缓冲峰值。四种调度使用相同的前向 10 ms、反向 20 ms、边界传输 1 ms、参数更新 1 ms 条件。

小，而交错式增加的通信次数、零气泡增加的激活驻留、DualPipe 增加的参数量都不随 $m$ 缩小。第 10.3.3 节的链路争用给出另一条边界：交错式每个 micro-batch 多出的四次跨界传输，在链路已被梯度归约占满时会直接延长关键路径。容量紧张时，选择的顺序反过来：先按第 10.2.4 节排除峰值不可行的调度，再比较完成时间。

### 10.3.3 反向依赖、梯度分桶与通信重叠

上一节的 2 ms 空隙来自输入尚未到达。梯度通信也受同一条规律支配：发送之前要等数据产生，发送时又要等链路空闲。反向传播逐层产生梯度，实现中通常把多个梯度张量合并成一个通信桶，等桶内所有梯度都计算完毕后再发送。桶越大，越可能等候最后生成的梯度；桶越小，需要启动通信的次数就越多。调度要在启动代价与提前发送之间取得平衡。

设归约需要 3 ms，梯度就绪后还有 5 ms 与它无关的计算，归约可以与这段计算同时进行。若共享链路此

前已安排 4 ms 的专家交换，归约只能利用余下 1 ms，另有 2 ms 延伸到计算之后。这部分无法与计算重叠、会延长训练步的时间，称为**通信等待时间**。数据依赖给出最早开始时刻，资源争用把它推迟，后续算子的依赖关系决定这段等待是否延长整个训练步。

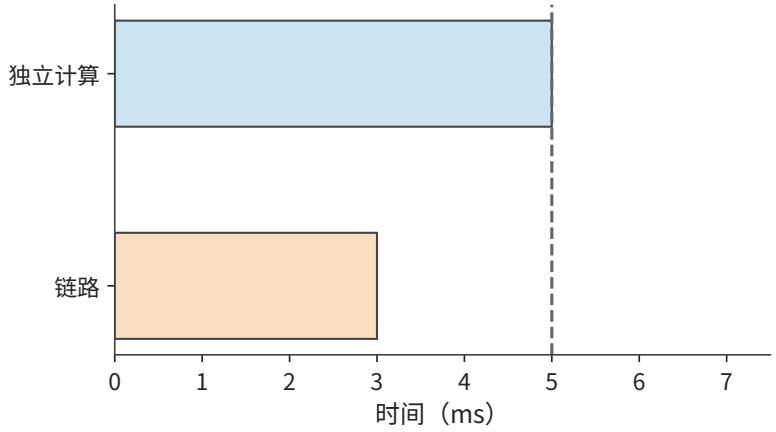



图 10-25：链路空闲时，3 ms 归约（橙色）可以在 5 ms 独立计算（蓝色）结束前完成。虚线标出计算结束。

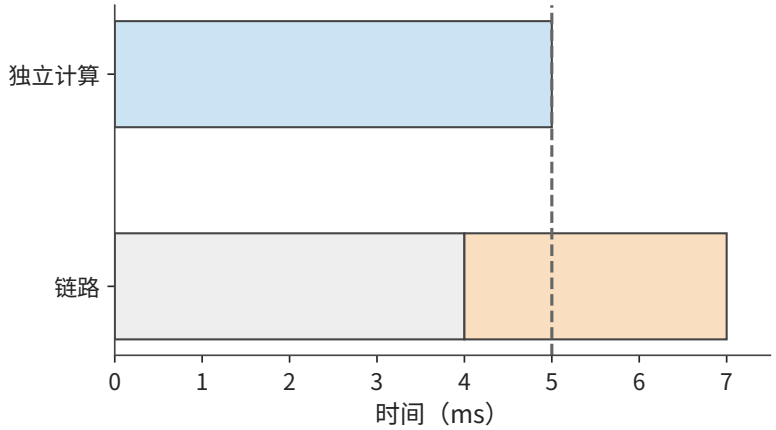



图 10-26：链路先被其他通信占用 4 ms（灰色），归约推迟到 4—7 ms。虚线后多出的 2 ms 延长训练步。

图 10-25 暂时固定了梯度产生的时刻。实际训练还可以改变计算顺序，让某些梯度更早产生，从而把归约在时间轴上提前。矩阵反向中，输入梯度 $dX$ 交给前一层继续反向，参数梯度 $dW$ 交给本层的归约。先算 $dX$ 可以让前一层更早开始反向传播，先算 $dW$ 则可以更早启动本层的梯度通信。两者争用同一计算资源时，改变顺序会重新分配通信窗口。这也是训练调度要同时考虑前后向依赖的原因。

现在用这条规律计算设计案例的通信等待。两套方案都采用全组 ZeRO-3，每个 micro-batch 要做三次集合通信：前向之前用 AllGather 收集一遍 BF16 权重，反向之前再收集一遍，反向之后对 BF16 梯度做一次 ReduceScatter。每次集合通信中，每张卡收发的字节为 $(d-1)/d \times 2N$，48 卡时约 16.0 GB。每卡累积 $m$ 个 micro-batch，一步的通信量为

$$V = 3m \cdot \frac{d-1}{d} \cdot 2N.$$

48 卡时 $m = 8$，$V \approx 385$ GB；32 卡时 $m = 12$，$V \approx 571$ GB。这些字节的传输路径由硬件决定。RTX 4090 没有 NVLink，也不支持卡间 P2P（一张卡经 PCIe 直接读写另一张卡的显存），同一台主机里的两张卡交换数据也要经过主机内存，所以每张卡的全部收发都要穿过各自的 PCIe 4.0 x16，每方向 32 GB/s。按公开的

4090 集群配置分析，每台八卡主机配八张 200 Gbit/s 网卡，每张 25 GB/s；环形集合通信把跨主机的流量分到八张网卡上，网卡不是瓶颈。于是 48 卡每步的链路时间约为 $385/32 \approx 12.0$ s，32 卡约为 17.9 s。

链路时间并不等于通信等待。ZeRO-3 在执行某一层之前预取它的参数，在一层反向结束后立即归约它的梯度。48 卡每个 micro-batch 的链路时间约 1.50 s，而该 micro-batch 的计算约 6.53 s；逐层看，收集一层约 0.39 GB 的参数只需约 12 ms，也远短于这一层的计算。所以大部分通信都能落在计算之内。MegaScale 在数据并行中采用同样的做法，并指出无法隐藏的只剩一步中第一次 AllGather 和最后一次 ReduceScatter。对 Qwen3-8B，这两次通信都作用于约 1.24 GB 的词嵌入：词嵌入是前向的第一层，也是反向最后求出梯度的一层。两次合计约 0.08 s，这就是每步的通信等待。参数更新和卡内的非矩阵运算已包含在单卡计算效率里，两套方案在输入就绪时的每步耗时分别约为 78.4 s 和 52.3 s。[16]

通信大多被计算覆盖，这也给优化设定了收益上限。若一段 1.3 s 的归约中只有 0.2 s 延续到反向计算结束之后，在其余时间轴不变时，即使消除这段通信等待，整步至多缩短 0.2 s。若还差 0.5 s 才能满足目标，就要同时缩短其他关键路径工作。

> **案例：全局 batch 不变时，卡数增至四倍能加速多少？**大规模分布式训练系统 MegaScale 在 175B 模型上保持全局 batch size 为 6144，把卡数从 3072 增至 12288。原每步耗时约 23.7 s，若卡数增至四倍就能带来四倍加速，每步应缩短至约 5.9 s，实际约 6.3 s。总加速约 3.7 倍，对应约 93% 的扩展效率。卡数增加后，每卡的工作减少，跨组同步和负载不均衡造成的等待，在每步耗时中所占的比例随之上升；实际每步比理想值多约 0.4 s，这就是扩展后需要进一步分析的额外耗时。按第 1.3.4 节的顺序，先把跨组同步和负载不均补进模型：卡数越多，每步等待最慢一组的时间占整步的比例越大，这是扩展本身带来的工作；补齐之后仍然解释不了的差额，才归于实现开销。[a]
>
> [a]论文报告的每步耗时为 23.66 / 6.34 s，扩展效率约 93.3%；对应固定全局 batch 的整套系统扩展实验。MegaScale 正式论文及固定任务、历史配置与计量笔记。

### 10.3.4 MoE 与长上下文的调度变化

稠密模型的每次前向都使用各层全部前馈权重，数据并行时的工作划分较为规则。MoE 的路由会让各卡收到数量不同的 token，序列长短不一也会让相同的 token 数对应不同的计算量。因此，分析每步耗时，要看工作如何分到各张卡上，以及哪张卡最后完成。

**专家 dispatch 不均如何延长最忙那张卡的计算时间。**设两张卡上的专家分别接到 96、32 次 token dispatch，每次 dispatch 把一个 token 的特征向量送给一个专家，在该专家输入矩阵中占一行。假定每次 dispatch 的计算时间近似相同。总计 128 次 dispatch，对应 128 个有效行，平均每卡 64 行，但完成时间由 96 行的一侧决定。将分配调整为 64、64 行，专家计算时间就从处理 96 行所需的时间降到处理 64 行所需的时间，缩短三分之一。

训练中的路由均衡机制决定样本如何选择专家；专家副本和拓扑映射则决定这些选择在哪里执行。

上述调整假定 token 可以在专家之间随意搬移。实际的 dispatch 由路由器打分决定，均衡机制只能影响 dispatch 的分布，执行时还需要一条处理不均的明确规则。**容量因子**（capacity factor）$c$ 给出这条规则：每个专家的输入矩阵最多为

$$\text{每专家容量} = \left\lfloor c \times \frac{\text{token 数} \times k}{E} \right\rfloor$$

[16]本章设计案例的输入、推导与复算。32 / 48 卡的容量预留、输入等待和计划停顿为给定输入；单卡计算效率取 Llama 3 的 MFU；模型矩阵工作来自既有 training-deadline 计算，ZeRO-3 通信量由同一份参数表推出。RTX 4090 的 PCIe 4.0、无 NVLink 见 RTX 4090 规格页，卡间不支持 P2P、八卡主机配八张 200 Gbit/s 网卡见 4090 集群配置分析；一步中只有第一次 AllGather 与最后一次 ReduceScatter 无法隐藏见 MegaScale §3.2。逐项数值与独立复算保存在 design-case.json。

行，即平均 dispatch 量的 $c$ 倍，其中 $k$ 为每个 token 选择的专家数，$E$ 为专家数。超出容量的 dispatch 被**丢弃**：该层不处理这些 token，它们的表示经残差连接直接传给下一层；容量未用满的槽位补零，这些行仍占用计算与通信。[17]

代入前例：128 次 dispatch、$E = 2$、$k = 1$，平均每专家 64 行。$c$ 取 1.0、1.25、1.5、2.0 时，每专家容量为 64、80、96、128 行，收到 96 次 dispatch 的专家分别丢弃 32、16、0、0 个 token，两个专家合计补零 32、48、64、128 行。$c = 1.0$ 时总执行量恰好等于 128 次有效 dispatch，却丢弃了其中四分之一；$c = 1.5$ 起不再丢弃，代价是执行行中有三分之一为补零。

再看真实模型的路由规模。Qwen3-235B-A22B 有 $E = 128$ 个专家，每个 token 选择 $k = 8$ 个；8,192 个 token 共 65,536 次 dispatch，平均每专家 512 行。设一半专家收到 1.5 倍于均值的 dispatch，与前例 96/32 同为 3:1 的不均衡，热专家各 768 行、冷专家各 256 行。$c$ 取 1.0、1.25、1.5、2.0 时，每专家容量为 512、640、768、1,024 行，热专家分别丢弃 256、128、0、0 次 dispatch，合计占总 dispatch 的 25%、12.5%、0、0；补零行占全部执行行的 25%、30%、33%、50%。

| 容量因子 $c$ | 每专家容量 / 行 | 丢弃的 dispatch 占比 | 补零行占执行量 |
|---|---|---|---|
| 1.0 | 512 | 25% | 25% |
| 1.25 | 640 | 12.5% | 30% |
| 1.5 | 768 | 0 | 33% |
| 2.0 | 1,024 | 0 | 50% |

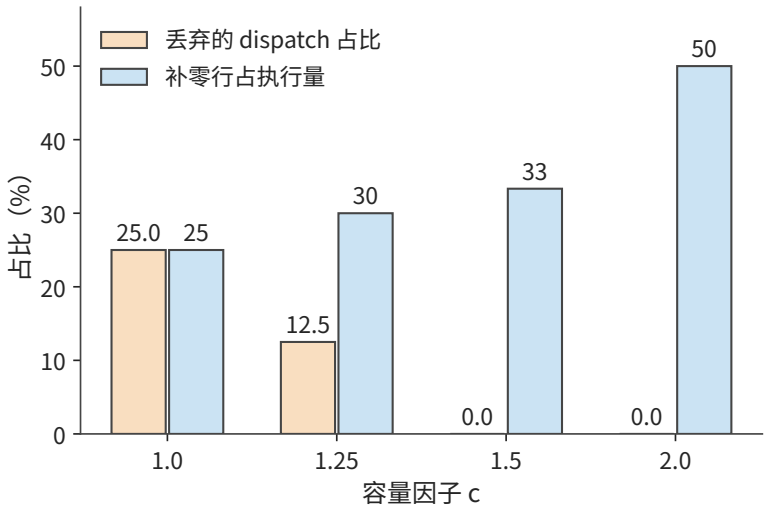



图 10-27：Qwen3-235B-A22B 的路由形状（E=128、k=8、8,192 个 token、一半专家 1.5 倍热）下，容量因子增大时丢弃的 dispatch 占比下降、补零行占比上升。

翻转条件由路由分布决定：上述分布需要 $c = 1.5$ 才把丢弃率降到零；路由越均衡，所需的 $c$ 越接近 1，补零浪费越少。辅助损失（在训练目标中额外加入的负载均衡项）直接惩罚 dispatch 不均，DeepSeek-V3 的无辅助损失方案按专家负载动态调整路由偏置；两者都把分布推向均衡，让容量因子可以取更小的值。第 2.4 节的专家容量公式假设路由过程中不丢弃任何 token dispatch（$\sum_e t_e = mk_{\text{top}}$），对应的就是容量因子足够大、丢弃率为零的情形。[18]

[17]容量因子与丢弃复算。容量规则见 Switch Transformer式 (3)（每专家容量 = 每批 token 数 / 专家数 × 容量因子）与 GShard（容量取 O(N/E)，溢出 token 的表示经残差连接传给下一层）；无辅助损失的偏置均衡见 DeepSeek-V3 技术报告。一半专家 1.5 倍热的分布为本例取值，与正文的 96/32 例子同为 3:1。

[18]容量因子与丢弃复算。容量规则见 Switch Transformer式 (3)（每专家容量 = 每批 token 数 / 专家数 × 容量因子）与 GShard（容量取

均衡专家的计算量以后，数据传输仍可能让卡等待。MoE 的前向 token 交换、反向输入梯度传输和参数梯度归约会争用同一条链路，因此还要利用上一节的通信时间轴分析。取 72 MiB 的 FP32 专家梯度，由持有该专家的四个数据并行成员做环形 AllReduce，每张卡发送 $2 \times 3/4 \times 72 = 108$ MiB。每卡配一张 200 Gbit/s 网卡，每方向 25 GB/s，每次集合调用启动约 0.02 ms。整桶耗时约 4.55 ms，无法在一个 2 ms 空闲时段内完成。若把梯度切成三个 24 MiB 子桶，每桶发送 36 MiB，约 1.53 ms，可分别在三个这样的空闲时段内完成；总通信处理时间却增加到约 4.59 ms。虽然多启动了两次通信，但每个小桶都能在计算结束前传完，因此减少了计算之后的等待。换成 400 Gbit/s 网卡，整桶约需 2.28 ms，仍无法放入 2 ms 空隙，切桶依然必要。[19]

专家负载取决于每张卡上的专家接到了多少次 token dispatch。长上下文还有另一层差别：即使 token 总数相同，需要计算的注意力配对数也可能不同。

**相同 token 总数下，序列长度不均如何增加注意力计算量。**对长度 $s$ 的因果序列，第一个 token 查看一个 token，第二个查看两个，依次相加，得到 $s(s+1)/2$ 个有效注意力配对。两条长度 4096、4096 的序列合计约 1680 万对；改为 7168、1024，总 token 数仍为 8192，配对数却增至约 2620 万，增加约 56%。注意力配对数随序列长度近似按平方增长，所以较长的序列增加了总计算量。[20]

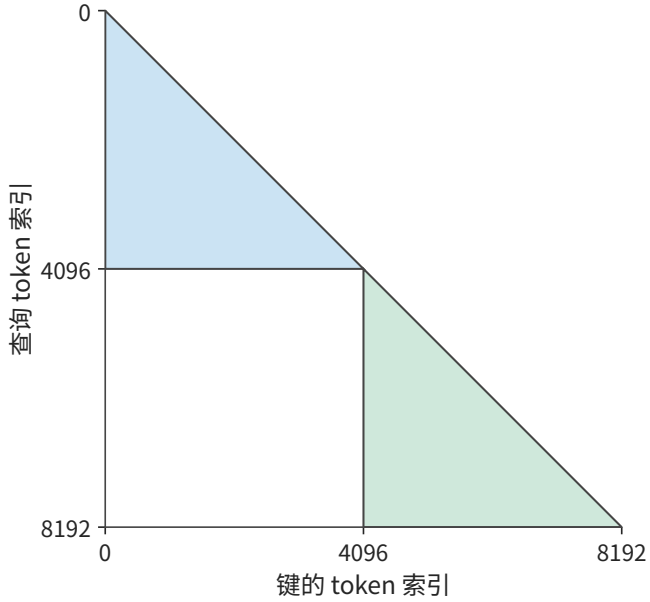



图 10-28：两条 4096 个 token 的序列的因果注意力配对。每个查询 token 读取本序列中不晚于自己的位置，形成两个三角形；两条序列的注意力计算彼此独立，总配对数为 16781312。

图 10-29 中，长三角形的额外面积来自真实的注意力关系，无法靠删除填充 token 消除。打包（把多条较短的序列首尾拼接成一条目标长度的序列）减少填充 token，按长度分组则让各卡承担的三角形面积更接近；上下文并行进一步把长序列的计算分到多张卡，并通过交换 K、V 取得其他卡保存的数据。三者分别改变无效工作、工作分配和数据到达路径。用相同总 token 数比较方案时，保持长度分布、可见性掩码（mask，规定各 token 位置能读取哪些上下文 token）和损失权重不变，就能将调度收益与任务变化分开。

O(N/E)，溢出 token 的表示经残差连接传给下一层）；无辅助损失的偏置均衡见 DeepSeek-V3 技术报告。一半专家 1.5 倍热的分布为本例取值，与正文的 96/32 例子同为 3:1。

[19] DeepSeek-V3 技术报告、专家交换与反向分桶案例。72 MiB 分三桶、四成员环形 AllReduce、0.02 ms 启动和三个 2 ms 空隙为给定输入；每卡一张 200 Gbit/s 网卡的配置见 4090 集群配置分析，400 Gbit/s 见 ConnectX-7 数据手册。专家均衡与路由机制参见 DeepSeek-V3，切块调度参见链接中的案例。

[20] 训练工作与长度分布推算。

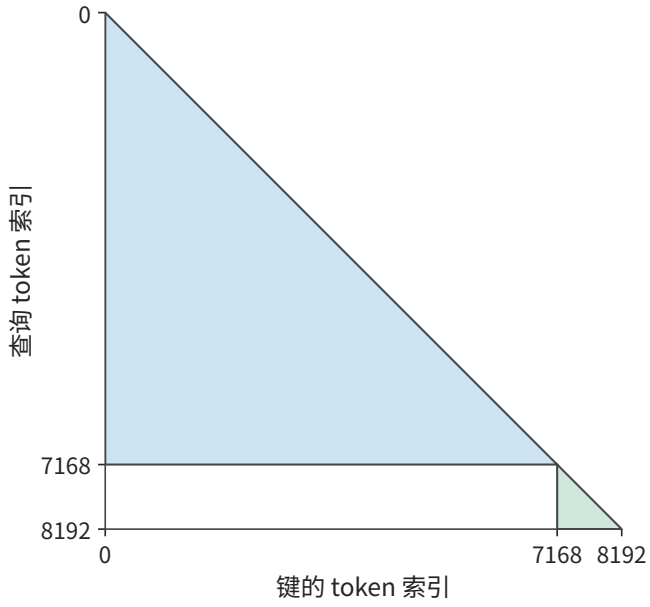



图 10-29：相同 8192 个 token 改分为 7168 与 1024，总因果配对数增至 26218496。较长序列对应三角形增加的面积，超过了较短序列对应三角形减少的面积。

## 10.4 数据输入、checkpoint 与故障恢复

第 10.3 节的每步耗时分析假定输入数据已经准备好，训练也没有因故障中断。持续运行时，数据读取和预处理要跟上计算速度，还要有 checkpoint 保存可恢复的训练状态。本节把二者加入章首的完成时间：输入决定卡能否持续工作，恢复决定已经做过的工作有多少需要重做。

### 10.4.1 数据读取、打包与预取

原始数据经过读取、解码或分词、筛选，再按目标长度拼接成训练序列，放入主机内存缓冲区，最后传到 GPU。主机缓冲区通常使用锁页内存，使数据在异步传输期间保持在物理内存中。各环节的持续处理速度决定输入速度，队列用来缓冲短时间内的速度波动。

GPU 最终收到的数据量小，不意味着此前的准备过程同样快。25 天预算要求约 46,300 token/s，即每秒约 5.7 条 8,192 个 token 的序列。若以每个编号占 8 字节的 int64 格式传输 token ID，数据率只有约 0.37 MB/s。设每个 CPU 数据准备进程每秒准备两条序列，两个数据准备进程只有四条/s，低于需求；三个数据准备进程达到六条/s。传输的数据量虽少，CPU 上的准备过程仍可能拖慢整个集群。

本章的设计案例为输入安排四个这样的数据准备进程，合计八条/s。48 卡输入就绪时每步约 52.3 s，每步处理 384 条，需求约为 7.3 条/s，低于八条/s。数据准备比训练消耗更快，短暂变慢之后可以重新补满预取队列。设计案例再给每步 0.5 s 的平均输入等待时间，表示预取没能消除的等待，32 卡和 48 卡的每步训练时间由此变为约 78.9 s 和 52.8 s。[21]

预取队列让数据准备与 GPU 计算以不同进度运行。正常训练时，提前准备的数据可以缓解输入速度波动造成的等待；保存 checkpoint 时，则必须分清哪些数据已经用于训练。假设数据加载器已经为前 108 个 batch

[21] 本章设计案例的输入、推导与复算。32 / 48 卡的容量预留、输入等待和计划停顿为给定输入；单卡计算效率取 Llama 3 的 MFU；模型矩阵工作来自既有 training-deadline 计算，ZeRO-3 通信量由同一份参数表推出。RTX 4090 的 PCIe 4.0、无 NVLink 见 RTX 4090 规格页，卡间不支持 P2P、八卡主机配八张 200 Gbit/s 网卡见 4090 集群配置分析；一步中只有第一次 AllGather 与最后一次 ReduceScatter 无法隐藏见 MegaScale §3.2。逐项数值与独立复算保存在 design-case.json。

分配了准备任务，而训练只完成第 100 个，第 101—108 个仍在处理或排队。若恢复时直接从第 109 个 batch 开始，就会跳过八个 batch。已分配给数据准备进程的 batch 位置反映预取进度，已用于训练的 batch 位置反映训练进度；两者之间的缓冲保存了尚未训练的数据。

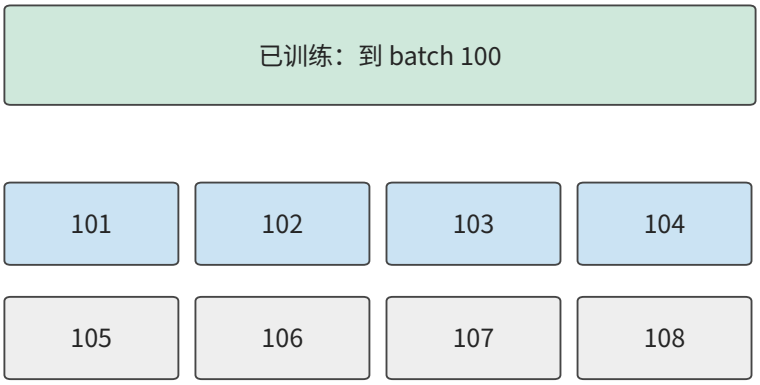



图 10-30：训练已完成 batch 100，预取任务已安排到 108。中间八个 batch 仍需训练，其中一部分已经准备好，另一部分还在处理。虚线框示意尚在处理的 batch。恢复时应保留这些数据或重新准备它们，从 batch 101 继续。

图 10-30 的队列只能暂时填补准备速度的下降；若数据长期准备得更慢，队列终将取空。checkpoint 写入也受同样的约束：保存产生数据的平均速率一旦超过存储能持续接收的速率，待写的快照就会越积越多，有限的暂存缓冲终将被占满，第 10.4.3 节按具体写入速率计算这种积压。对本章的文本任务，输入端的字节量很小，瓶颈在 CPU 上的数据准备；checkpoint 的平均写流量也不大：设计案例采用 1800 s 保存间隔，约 115 GB 的 checkpoint 平均只有约 64 MB/s。保存时的停顿在第 10.4.4 节单独计入。[22]

### 10.4.2 可恢复状态与布局变换

设训练刚完成一次参数更新，此时停机。恢复后，要接着训练，需要知道当时的权重、下一批该读哪些数据，以及优化器已经累积了哪些历史。仅有权重，可以执行前向推理，却不足以确定训练的下一次更新。因此，训练 checkpoint 要保存与同一训练进度对应的全部必要状态，恢复后才能接着训练。

本章采用 Adam 的训练需要保存权重、一阶矩与二阶矩、步数、学习率状态、随机状态和下一批数据的组成。学习率控制更新步长，随机状态决定后续随机采样的序列。权重相同而优化器的矩状态不同，下一次参数更新便可能不同；数据读取位置相同而打包时剩余的 token 不同，下一条训练序列也会改变。第 10.2.2 节的激活重计算解决的是一次前向、反向之内哪些中间值可以重建；checkpoint 要解决的是中断之后整个训练过程如何继续。

一次训练迭代结束后，下一批梯度由反向重新产生。此时保存权重 2 bytes、主权重 4 bytes 和两份 Adam 状态 8 bytes，每参数合计 14 bytes。Qwen3-8B 的这部分 checkpoint 约为 115 GB。在一次训练迭代结束时，把模型状态和已处理到的数据位置一同保存，就能确定恢复后该用哪些参数、从哪一批数据继续训练。

恢复时还可能需要改变并行布局，同一份状态要重新分片。**例：checkpoint 如何从四路张量并行重分片为八路？**门控投影的权重形状为 $[12288, 4096]$，BF16 大小 96 MiB。沿输出维四分，每片 3072 行、24 MiB；八分

[22] 训练数据输入与共享存储模型、输入与保存共存实验。事件模型以小规模参数演示 checkpoint 与输入共用存储时预取队列被取空的过程；本章的设计案例使用四个各准备两条序列/s 的数据准备进程。

后，每片 1536 行、12 MiB。rank 为 $r$ 的目标卡读取旧分片 $\lfloor r/2 \rfloor$，偶数 rank 取前半，奇数 rank 取后半。全局行坐标将旧布局和新布局联系起来。

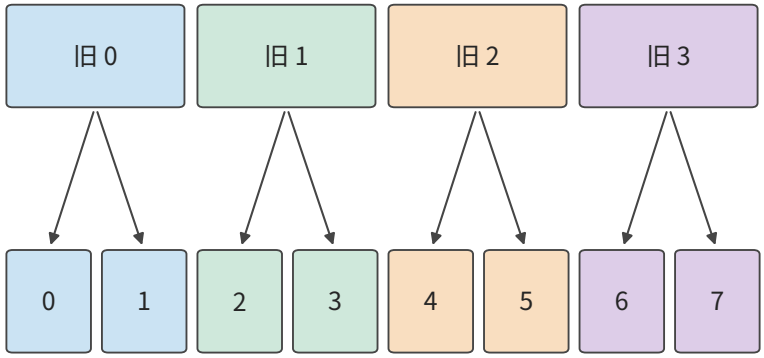



图 10-31：横向位置对应原矩阵的行号，颜色表示旧分片。每份旧分片分成前后两半后，分别存入两个新分片。模型权重的内容和顺序保持相同，改变的是各卡负责的行范围；BF16 权重总量始终为 96 MiB。上方“旧”编号标识原分片，下方编号标识重新分配后的分片。

沿图 10-31 的箭头，恢复程序可以找到每个新分片对应的原始行范围。FP32 主权重与 Adam 两项状态也按相同的行范围拆分，每项为 BF16 权重的两倍。该矩阵在 checkpoint 中的全部状态因此为 $96\times(1+2+2+2) = 672$ MiB。ByteCheckpoint 一类系统用全局形状、偏移和片段长度描述这些张量，使恢复程序按目标布局读取需要的范围。文件分片是存储安排，全局坐标标明每个分片属于哪个张量、位于哪些行列。[23]

权重可以凭原矩阵的行号重新拼接，训练序列也需要保留足够的信息，才能重新组成同样的输入。图 10-30 中已经准备好但尚未用于训练的数据，同样需要在恢复时重新定位。除了 batch 位置，还要保留数据预处理的中间结果。

打包器若保留 2,000 个 token，下一份数据提供 6,192 个，两者组成下一条 8,192 个 token 的序列。如果丢失这 2,000 个 token，就只能从后续数据中取出额外的 token 来补齐下一条序列，从而改变它的前缀、注意力关系和标签。将已用于训练的数据位置、尚未组成完整序列的 token 与参数更新结果一起保存，恢复后才能继续构造同一批数据。[24]

### 10.4.3 同步、异步保存与带宽竞争

本节讨论保存过程如何与训练同时进行，以及一份快照何时才真正可用。

同步保存先生成内容固定的快照，写入完成后继续训练。异步保存将快照复制到独立的内存缓冲区，再由后台写入，让训练较早恢复执行。训练恢复执行时，快照可能还没有写完；只有写入并提交完成后，这份快照才能用于故障恢复。

**例：checkpoint 上传速度如何决定故障后的重做量？**取每份 112 GB 的快照。DGX SuperPOD 参考架构按工作负载给出存储性能档次，NLP 训练对应 Good 档，一个 SU（32 台 DGX 组成的扩展单元）的存储合计写

[23] checkpoint 布局、ByteCheckpoint 与后台保存阅读、逻辑重分片计算、112 GB 时间线。

[24] DCP 恢复记录、真实文本管线恢复。2,000 + 6,192 是说明打包时剩余 token 的教学例。DCP 小模型将两份行分片恢复为三份列分片与完整状态，参数、Adam、随机状态和下一次参数更新结果与不中断训练一致。

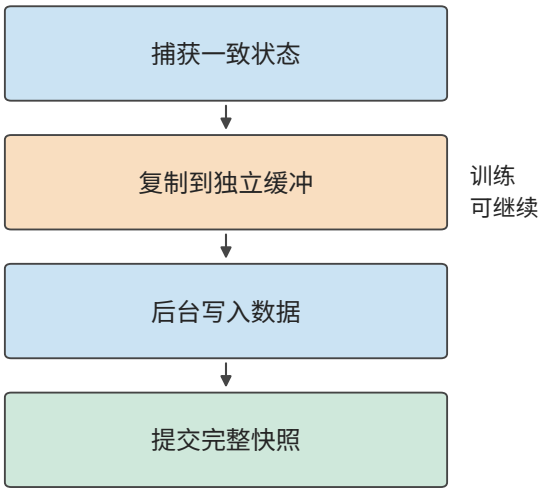



图 10-32：捕获一致的训练状态，复制到独立缓冲后允许训练继续；后台写完数据并提交完整快照后，恢复程序才使用这份 checkpoint。箭头表示先后依赖。

入为 7 GB/s；按此速率，一份快照要写 16 s。每 10 s 产生一份时，待写入数据的产生速率为 11.2 GB/s，队列持续增长。改为每 20 s 一份，平均需求降到 5.6 GB/s，存储能在下一份到达前写完上一份。

在第 20、40 s 捕获两份快照，各花 0.5 s 复制到缓冲区后上传，完成时刻为 36.5、56.5 s。若第 50 s 故障，第二份仍在上传，只能恢复到第 20 s，重做 30 s。换用 Better 档的 20 GB/s 合计写入，每份只需 5.6 s，两份分别在 26.1、46.1 s 完成；同一故障可恢复到第 40 s，只重做 10 s。

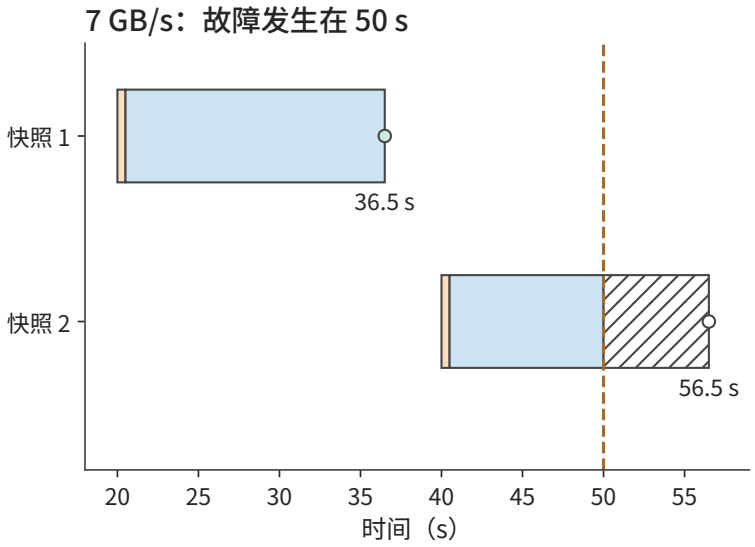



图 10-33：两份 112 GB 快照以 7 GB/s 写入，各先花 0.5 s 复制到缓冲（橙色），随后上传（蓝色）。50 s 故障时第一份已提交，第二份尚未提交；斜线为无故障时剩余上传，空心点为原定提交时刻。

两种方案的前台暂停都是两次 0.5 s，合计 1 s。写入更快，故障后就能恢复到更近的训练进度，少重做 20 s。异步保存的完成点因而直接影响长期进展，后台写入速度影响故障后的重做量，保存时暂停训练的时长则会延长正常运行时间。

写入数据文件之外，实际系统还要提交描述完整快照的元数据。一组 CPU 实验中，保存接口（API）的调

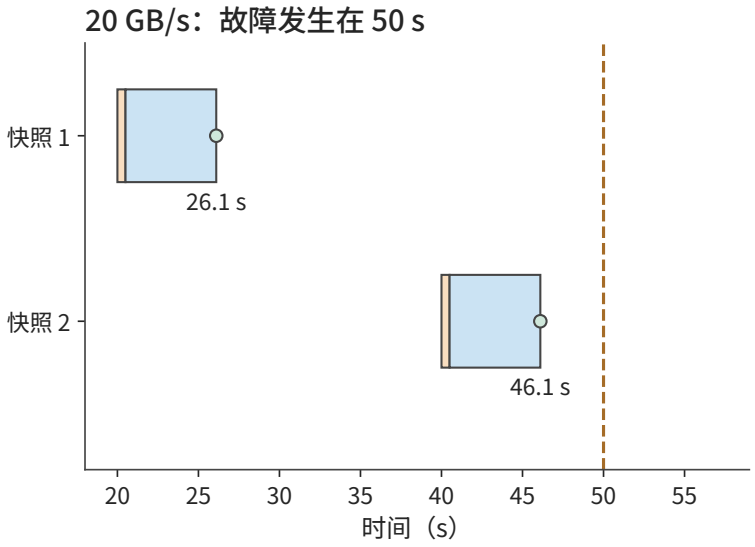



图 10-34：相同快照以 20 GB/s 写入，于 26.1、46.1 s 提交。50 s 故障时可以恢复到 40 s 的训练状态，只需重做 10 s。

用约 7 ms 就返回，恢复元数据约 48 ms 才提交；提交前终止进程，恢复程序选择上一份完整的 checkpoint。数据文件与提交记录共同构成可用快照，恢复程序据此判断哪一份 checkpoint 已经完整保存。[25]

### 10.4.4 故障规模、保存周期与有效训练进度

图 10-33 固定了两次保存的时刻，比较写入速度的影响。还可以改变保存间隔：间隔短，故障时离上一份快照更近；间隔长，正常训练时需要暂停保存的次数更少。这两项代价方向相反，因此存在一个折中点。

设每次同步保存耗时 $c$，保存之间完成 $\tau$ 秒有用训练，作业故障率为 $\lambda$，恢复耗时为 $r$。假设故障发生频率较低且彼此独立，在一阶模型下，单位有用训练时间的附加成本为

$$L(\tau) = \frac{c}{\tau} + \lambda\frac{\tau}{2} + \lambda r.$$

第一项来自每隔 $\tau$ 保存一次。故障落在两次保存之间的任意位置，平均丢失半个间隔，得到第二项。第三项是单位有用训练时间内的预期恢复次数乘以每次恢复时间。前两项一减一增，最优点满足

$$-\frac{c}{\tau^2} + \frac{\lambda}{2} = 0, \qquad \tau^* = \sqrt{\frac{2c}{\lambda}}.$$

**例：卡数与故障率如何决定 checkpoint 保存间隔？** Meta 统计了研究集群上的训练作业：1024 卡作业平均 7.9 小时中断一次，而且中断率与卡数成正比，相当于每卡平均约 337 天出一次故障。任一卡故障均中断作业。约 115 GB 的 checkpoint 以 7 GB/s 保存，$c \approx 16.4$ s。设恢复需 120 s，代入得到最优间隔约 965 s，即约 16 分钟。[26]

图 10-35 中的最低点较平缓，配置时可以选择附近便于使用的间隔。下表比较五、十五和三十分钟，观察偏离最低点后增加的是哪一项成本。

[25] 异步保存与提交前故障实验及无保存 / 同步 / 异步对照。原正常路径 API 为约 7.25 ms、元数据提交约 48.30 ms；故障实验在提交前设置屏障并终止进程。

[26] 保存周期与 Poisson 重试模型。精确数据量为 114,670,295,040 bytes，以 7 GB/s 写入的保存成本约 16.381 s；一阶最优约 965.29 s，含保存期故障的 Poisson 重试模型约 954.40 s。7 GB/s 为 DGX SuperPOD 参考架构表 6 Good 档的单 SU 合计写入；1024 卡作业平均 7.9 小时中断一次、平均无故障时间与卡数成反比见 Meta 集群可靠性论文。本章的选择只需要区分五、十五和三十分钟。

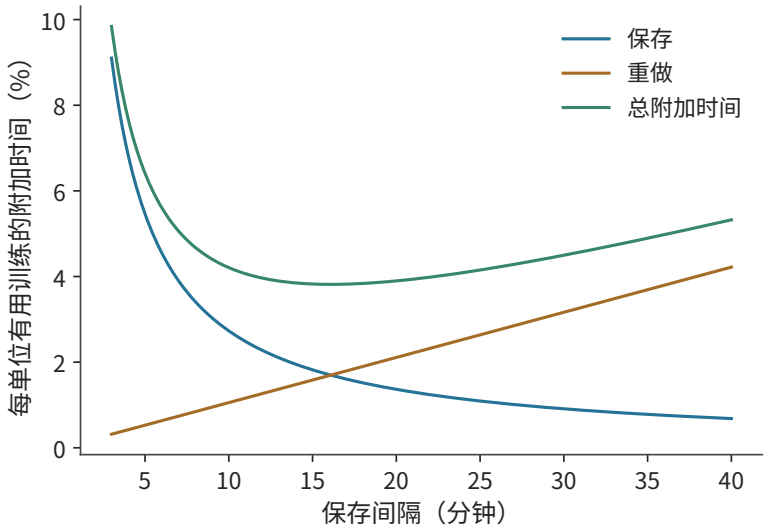



图 10-35：蓝线为保存耗时占比，橙线为故障重做耗时占比，绿线为两者加上恢复耗时后的合计。使用 1024 卡作业平均 7.9 小时中断一次、保存约 16.4 s 和恢复 120 s 的一阶模型。最低点出现在两项随间隔变化的代价相互平衡处。

| 有用训练间隔 | 保存成本 | 重做成本 | 恢复成本 | 合计 |
|---|---|---|---|---|
| 300 s | 5.5% | 0.5% | 0.4% | 6.4% |
| 900 s | 1.8% | 1.6% | 0.4% | 3.8% |
| 1800 s | 0.9% | 3.2% | 0.4% | 4.5% |

将保存间隔从五分钟延长到十五分钟，节省的保存时间多于新增的重做时间；继续延长到三十分钟，新增的重做时间就超过了节省的保存时间。表中合计由未经四舍五入的数值计算。卡数翻倍会使故障率翻倍，最优间隔缩短为原来的 $1/\sqrt{2}$；保存速度翻倍也使最优间隔按同样比例缩短，因为每次保存耗时更短。

本章设计案例使用的卡数较少，沿用每卡平均约 337 天故障一次（单卡平均故障间隔，MTBF）、恢复耗时 120 s 和保存耗时约 16.4 s 的条件，取有用训练间隔 1800 s。32 卡的保存、重做与恢复合计约增加 1.02% 时间，48 卡约增加 1.08%。48 卡方案被卡故障中断的概率更高，但附加比例只多约 0.06 个百分点，远小于缩短单卡计算时间带来的收益。容量、每步训练时间和长期附加成本至此齐备，第 10.6 节将汇总计算完成时间。

大规模训练中，即使一次中断只耽误两三个小时，整组资源的费用也可能达到数万美元。保存周期决定一次故障需要重做多少工作，系统的可靠性则决定这样的损失会发生多少次。

**例：一次训练中断会浪费多少钱？**小米的 MiMo-V2.6 是前沿实验室中首次直播 RL 过程的。公开运行看板显示，Pro 与 Flash 两种尺寸的模型 RL 后训练分别花费约 260 万和 90 万美元，即每小时约 2.06 万和 1.03 万美元。

图 10-36 按日志画出两次运行。Pro 共重启 14 次，中断区间累计约 30.1 h（占总运行时间的 23.6%），折合约 61.9 万美元。Pro 第五次重启前，自上一步完成起已过去 2.76 h，对应约 5.67 万美元；第六次和第八次分别为 2.63 h 与 2.95 h，对应约 5.41 万与 6.07 万美元。这些运行的重启主要涉及三方面：显存双比特错误等硬件故障；容器崩溃、评分服务网络不通等服务故障；以及内存不足，例如推理时 KV 缓存耗尽、训练时专家负载不均导致显存溢出，或序列打包时主机内存不足。此外，一次 infra 错误没有及时被检出，直到完成第 17 步后才从第 15 步重新训练。此前第 16、17 步花掉的约 4 h 随之作废。[27]

[27]总成本与故障原因见 MiMo-V2.6 技术报告 §4.1、§5.5。逐次时间、费率与回滚公告见归档日志，计算过程见案例笔记与复算记录。金额由日志事件区间乘看板固定费率估算，不是报告单列的故障账单；区间内的重做、检测和等待未单独拆分，也未扣除可能复用的采样结果。

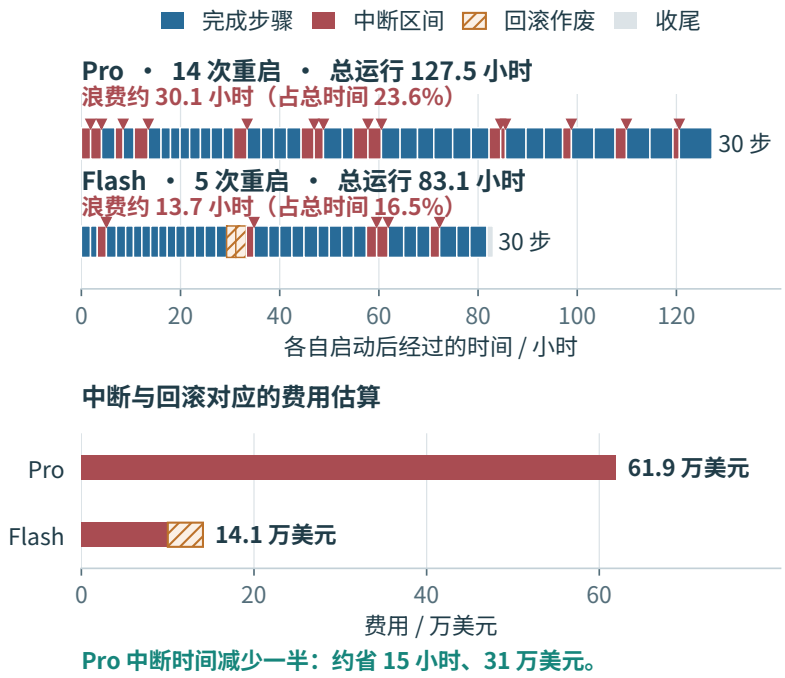



图 10-36：两次运行分别从启动时刻计时。蓝色为完成步骤的区间，红色为上一个事件至重启的区间，三角标记重启；斜线为 Flash 回滚后作废的两步。浪费时间按红色与斜线区间合计估算，括号内为其占总运行时间的比例；下方按公开费率换算费用。节省金额假设训练工作不变，减少的中断时间等量缩短运行。

若能将 Pro 的中断总耗时减少一半，同样的训练工作就能少用约 15 h，按上述费率节省约 31 万美元。这可以来自两方面：一是排除反复触发的故障，例如，MiMo 团队曾调整训练并行方式，降低激活显存，以容纳专家负载峰值。二是更早发现故障、更快恢复，并用较近的 checkpoint 减少重做，缩短恢复时间 $r$ 或丢失的训练间隔。

### 10.4.5 掉队者与慢节点

上一节的模型把故障当作中断作业的事件。还有一类更频繁的退化不会中断作业：某张卡在某一步算得慢。同步数据并行的每一步要等最慢的那张卡算完，这张拖慢整步的卡称为**掉队者**（straggler），步时间因此由各卡的最大值而不是均值决定。设各卡计算时间独立同分布，均值为 $\mu$、标准差为 $\sigma$，$N$ 张卡时步时间的计算部分是 $N$ 个样本的最大值。取 $\mu = 52.2$ s（48 卡方案的单卡计算时间）与每步 0.58 s 的通信与输入等待，$\sigma$ 分别取均值的 2%（1.04 s）与 5%（2.61 s）。[28]

$N$ 个独立正态样本最大值的期望为 $\mu$ 加上若干倍 $\sigma$：$N = 8, 48, 1024$ 时分别约为 1.42、2.23、3.25 倍。代入即得每步的期望耗时：

| 卡数 | $E[\max] - \mu$（倍 $\sigma$） | 每步，$\sigma = 2\%$ / s | 每步，$\sigma = 5\%$ / s |
|---|---|---|---|
| 8 | 1.42 | 54.3 | 56.5 |
| 48 | 2.23 | 55.1 | 58.6 |
| 1024 | 3.25 | 56.2 | 61.3 |

无波动时每步约为 $52.2 + 0.58 \approx 52.8$ s。卡数越多，最大值的期望越大：48 卡、$\sigma = 5\%$ 时每步多约 5.8 s，占去第 10.6 节约 14.4 s 余量的四成；1024 卡时多约 8.5 s。慢卡可以从通信等待中检测出来：每张卡在梯度桶

[28]掉队者最大值模型（σ 为均值的 2%）与（σ 为均值的 5%）。约 0.5% 的机器明显变慢见 MegaScale；1024 卡作业平均无故障 7.9 小时、8 卡作业 47.7 天见 Meta 集群可靠性论文，保存周期部分的每卡故障率即由 1024 卡的 7.9 小时折算。每卡计算时间的标准差与损失尖峰回滚间隔为本例取值；最大值期望按阶次统计的数值积分求得，未模拟相关性、周期性抖动或持续性慢卡。

AllReduce 上的等待，等于步结束时刻减去自己的计算结束时刻；对计算耗时恰为均值的卡，这段等待的期望正是 $E[\max] - \mu$。[29]

发现慢卡以后有三种响应：等待、重分配和驱逐。**等待**：本步按慢卡时间结束。一张慢到 $\mu + 3\sigma$ 的卡让其余各卡多等的时间，取决于其余 $N-1$ 张卡的最大值本来有多大：$\sigma = 2\%$ 时，8 卡多等约 $1.65\sigma$（1.72 s），48 卡多等约 $0.78\sigma$（0.81 s），1024 卡时其余卡的最大值本身已超过 $\mu + 3\sigma$，多等的时间为零。**重分配**：把慢卡的工作均匀摊给其余 $N-1$ 张卡，均值变为 $\mu N/(N-1)$。仍取 $\sigma = 2\%$、不计迁移成本，48 卡每步约 56.2 s，8 卡升到约 61.6 s，都高于同一 $\sigma$ 下等待的期望步时间 55.1 s 与 54.3 s：摊派让每张卡多做 $1/(N-1)$ 的工作，这份增量超过了一张慢卡带来的等待，卡越少差距越大。**驱逐**：把慢卡移出作业，从上一个 checkpoint 重启，代价为期望丢失工作 $\tau/2$ 加恢复时间 $r$。这里取比设计案例的 1800 s 更短的保存间隔 $\tau = 600$ s，$r = 120$ s，代价为 420 s，按 48 卡方案每步 52.8 s 计约为八步。一次 $\mu + 3\sigma$ 的抖动只损失几秒，只有慢卡持续多步时驱逐才值得。实际集群中确实存在慢卡：MegaScale 报告约 0.5% 的机器明显变慢；第 10.4.4 节的 Meta 集群统计则给出作业规模与中断的关系——1024 卡作业的平均无故障时间为 7.9 小时，8 卡作业为 47.7 天。[30]

除硬件故障外，还有一种事件需要回滚 checkpoint：第 10.1.2 节的损失尖峰。损失尖峰不损坏硬件，处理方式却相同，都是丢弃当前参数、回到上一份快照，对保存周期的影响也与硬件故障相同。把损失尖峰导致的回滚看作整个作业同时受到的冲击，平均每隔 $1/\lambda_{\text{spike}}$ 发生一次，上一节的一阶模型修正为

$$L(\tau) = \frac{c}{\tau} + (\lambda_{\text{hw}} + \lambda_{\text{spike}}) \left( \frac{\tau}{2} + r \right), \qquad \tau^* = \sqrt{\frac{2c}{\lambda_{\text{hw}} + \lambda_{\text{spike}}}}.$$

沿用 48 卡的硬件故障率（每卡 MTBF 约 337 天），设损失尖峰平均七天导致一次回滚：同样取 $\tau = 600$ s 时，附加比例从 2.80% 升到 2.87%，最优保存周期从约 4,458 s 缩到约 3,150 s。本章设计案例的 1800 s 间隔处，附加比例从 1.08% 变为约 1.25%，完成时间增加约 0.03 天，不改变第 10.6 节的方案选择。翻转条件由两个发生率的相对大小给出：尖峰越频繁，最优周期越短，$\lambda_{\text{spike}} \gg \lambda_{\text{hw}}$ 时保存周期几乎只由数值稳定性决定；回滚间隔以月计时，对 1.08% 的附加比例几乎没有影响。[31]

## 10.5 强化学习训练

前四节假定训练数据已经给定。RL 让模型参与产生下一批数据：模型生成轨迹，系统计算奖励或验证结果，再将轨迹用于训练，更新后的权重又改变下一轮生成。系统由处理固定数据的训练流水变成反馈循环，新的问题集中在各阶段的处理速度、切换时的状态重叠，以及样本究竟由哪一版策略产生。生成轨迹的推理端与计算损失的训练端还必须对同一条轨迹给出可对齐的概率，这就是本节的**训推一致性**。训推一致性直接影响参数更新是否遵循算法预期，也约束阶段加速与异步执行的收益。

### 10.5.1 一轮回答生成、验证与参数更新

一轮反馈循环的完成过程如下。从四条提示词（prompt）出发，每题生成两条回答，得到八条轨迹。奖励描述这些回答的结果，优势值衡量回答比所选基准好多少，用来确定调整回答概率的方向和幅度。训练端按有

[29] 掉队者最大值模型（σ 为均值的 2%）与（σ 为均值的 5%）。约 0.5% 的机器明显变慢见 MegaScale；1024 卡作业平均无故障 7.9 小时、8 卡作业 47.7 天见 Meta 集群可靠性论文，保存周期部分的每卡故障率即由 1024 卡的 7.9 小时折算。每卡计算时间的标准差与损失尖峰回滚间隔为本例取值；最大值期望按阶次统计的数值积分求得，未模拟相关性、周期性抖动或持续性慢卡。

[30] 掉队者最大值模型（σ 为均值的 2%）与（σ 为均值的 5%）。约 0.5% 的机器明显变慢见 MegaScale；1024 卡作业平均无故障 7.9 小时、8 卡作业 47.7 天见 Meta 集群可靠性论文，保存周期部分的每卡故障率即由 1024 卡的 7.9 小时折算。每卡计算时间的标准差与损失尖峰回滚间隔为本例取值；最大值期望按阶次统计的数值积分求得，未模拟相关性、周期性抖动或持续性慢卡。

[31] 掉队者最大值模型（σ 为均值的 2%）与（σ 为均值的 5%）。约 0.5% 的机器明显变慢见 MegaScale；1024 卡作业平均无故障 7.9 小时、8 卡作业 47.7 天见 Meta 集群可靠性论文，保存周期部分的每卡故障率即由 1024 卡的 7.9 小时折算。每卡计算时间的标准差与损失尖峰回滚间隔为本例取值；最大值期望按阶次统计的数值积分求得，未模拟相关性、周期性抖动或持续性慢卡。

效 token 构造损失并完成一次参数更新。新权重交给生成端后，下一轮样本来自更新后的策略。生成、奖励与学习分别使用不同形式的数据，样本标识将三者连接起来。

这里的数据来源与通常采用固定数据集的 SFT 不同。SFT 在给定答案的真实前缀上预测下一个 token，即教师强制；部署时模型则根据自己生成的前缀继续回答。一旦前面生成错误，后续就可能进入训练数据较少覆盖的上下文，误差随生成累积。这种训练与使用时的前缀分布偏移，是固定数据 SFT 的局限之一。梯度仍可以正确对应当前损失，但训练覆盖的情境与模型实际遇到的情境不同。

针对这种前缀分布偏移，OPD 让当前学生模型自己生成轨迹，再由教师针对学生实际生成的前缀提供监督，例如下一 token 的概率分布。学生因而能在自己容易出错的位置学习，减少前缀分布偏移；教师的逐 token 反馈也提供了比单个终局奖励更细的学习信号。DeepSeek V4 用多教师 OPD 将领域专家的能力合入统一模型，第 3 章已把教师前向列为一项独立的计算工作。[32]

与固定数据的 SFT 相比，OPD 的潜在优势在于：达到目标能力所需的学习样本或更新次数可能减少。它同时增加学生在线生成、教师前向与权重同步的成本，是否减少总 GPU 小时或完成时间仍要实测。SFT 描述的是监督目标，离线（offline）描述的是数据是否预先固定；也可以在线收集数据后使用监督损失。在线（online）表示数据随训练过程持续产生，on-policy 则进一步要求采样分布与所优化的策略一致。异步的在线 RL 可能使用滞后的策略，OPD 也会遇到生成引擎与训练引擎的计算差异。因此，在线采样用于缓解数据分布偏移，而训推一致性还需要对齐生成与训练两端的计算过程。

回到本节开头那一轮的八条轨迹，考察训练端如何按有效 token 构造损失。第 10.3.1 节的八条回答例子来自一次小模型实验的回答长度：回答中的有效 token 共 20 个，包含表示序列结束的 EOS token；输入计算的有效 token 共 366。提示词和回答都参与前向计算，损失则按选中的回答 token 归一化。因此，生成长度影响计算成本；损失掩码（mask）标明每个 token 是否计入损失，决定哪些 token 参与梯度计算。两者的作用不同。[33]

损失之外，同一提示词下的相对奖励还决定优势。例如两条回答奖励均为 1，减去组内均值后都为零；奖励变为 1、0，中心化后为 0.5、−0.5，策略才获得区分两条回答的方向。系统即使完成了前向、反向和优化器调用，也可能遇到第一种优势全为零的 batch。统计训练吞吐量时，应计算每秒实际用于训练的轨迹或 token 数；模型能力是否提高，则用固定的评估任务检验。

> **案例：参数发生变化，模型能力为何没有提高？** 一次固定的小模型实验用强化学习训练框架 verl 和推理引擎 vLLM 协作运行：算术题的精确匹配奖励让同组回答得到相同的奖励，优势和策略梯度都为零，参数变化只来自 AdamW 的权重衰减。AdamW 在 Adam 的梯度更新之外，单独按比例缩小权重；即使当前策略梯度为零，这一步也能改变参数。改用固定符号奖励后，两步训练都出现了非零梯度与权重更新，但算术验证只有 2/4 正确。奖励构造决定学习方向，系统执行负责把这一方向一致地应用到参数。[a]
>
> [a]固定 verl 小模型真实闭环、loss 与更新的源码核对。固定 Qwen2.5-0.5B-Instruct 训练配置采用单卡 NO_SHARD、no_sync=False；源码和版本锁定见链接中的记录。0.32 / 0.30 / 0.04 使用真实长度结构与教学标量损失推导。

### 10.5.2 各阶段的资源分配与权重同步

明确了样本如何形成梯度以后，才能讨论每秒能训练多少样本。RL 的反馈阶段可能执行规则验证或奖励模型（为回答打分的模型），OPD 则需要教师给出监督；两者都应按实际反馈工作的处理能力分配资源。生成、验证和学习像前后相接的三道工序，前一道做得更快，只有后一道能够消化，才会增加最终产出。先把三者的处理能力换算成同一种工作单位。设三者分别为 $r_g, r_v, r_l$，验证后符合策略版本要求的比例为 $q$，在独立资源上稳定流水执行时，每秒可用于训练的样本数上限为

[32]DeepSeek V4 技术报告，§5.1 的领域专家与多教师 OPD；阶段工作划分参见第 3 章。

[33]固定 verl 小模型真实闭环、loss 与更新的源码核对。固定 Qwen2.5-0.5B-Instruct 训练配置采用单卡 NO_SHARD、no_sync=False；源码和版本锁定见链接中的记录。0.32 / 0.30 / 0.04 使用真实长度结构与教学标量损失推导。

$$r_{\text{effective}} = \min\left(r_l, q\min(r_g, r_v)\right).$$

**例：RL 生成、验证与学习三个阶段中，应优先扩容哪一个？**生成每秒提供 12 条等长轨迹，验证处理六条，学习阶段处理八条；验证后四分之一因版本过旧被丢弃。每秒进入学习的只有 $6 \times 0.75 = 4.5$ 条。生成翻倍到 24 条/s，验证瓶颈仍将进入学习的样本限制在 4.5 条/s；验证翻倍到 12 条/s，每秒可送往训练端的轨迹达到九条，此时训练端每秒只能处理八条，成为整个流水线的瓶颈。因此，应把新增资源分配给当前最慢的阶段；该阶段加速后，再判断瓶颈转移到了哪里。

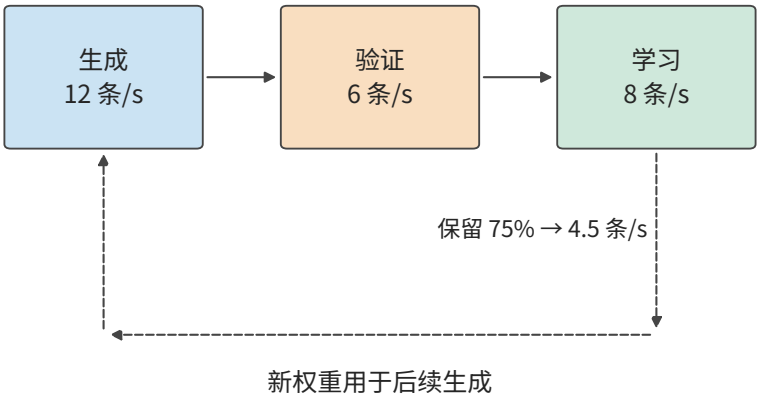



图 10-37：三阶段分别最多处理 12、6、8 条等长轨迹/s。验证后保留 75%，所以只有 4.5 条/s 进入学习。返回箭头表示学习产生的新权重影响后续生成；其同步耗时在后面的阶段切换与异步算例中展开。

图 10-37 先把三道工序画在各自独立的资源上。若改用同一组卡轮流执行，就能减少所需的卡数，但生成和学习之间要切换整套显存里的数据。

**共享加速器**让同一组卡轮流训练和生成，节省重复资源；**独立部署**让两侧独立执行，获得重叠机会，但要在两组卡之间发送权重。共享加速器的关键成本发生在切换：旧状态尚未释放，新状态已经开始加载，两个阶段各自运行时显存都足够，切换时却可能因两套数据同时占用显存而超出容量。

设共享的卡为 H100 SXM，标称 80 GB，约合 74.5 GiB。训练状态占用显存 40 GiB，生成需要约 15.3 GiB BF16 权重与 24 GiB KV 缓存池，另有两阶段共用开销 4 GiB。训练独占时 44 GiB，生成独占时约 43.3 GiB，都低于 74.5 GiB。先恢复生成权重和 KV，再释放训练状态，峰值为 $40 + 15.3 + 24 + 4 \approx 83.3$ GiB，超出一张 H100 的容量。

改变次序，先加载生成所需的权重，暂不分配 KV，峰值为 $40 + 15.3 + 4 \approx 59.3$ GiB。随后释放 40 GiB 训练状态，显存占用降为约 19.3 GiB，再分配 KV，进入约 43.3 GiB 的生成状态。两条路径最终状态相同，峰值差 24 GiB，恰好是一份 KV 池。[34]

独立部署的代价则由权重发送范围决定。Qwen3-235B-A22B 的 BF16 专家权重为 423 GiB，采用 16 路专家并行（EP16）时，每个接收方只需要约 26.4 GiB。逐一向 16 个接收方发送完整专家集合，共发送 $16 \times 423 = 6768$

[34] Qwen3-8B 共享设备峰值、Qwen3-235B-A22B 分片权重同步。40 GiB 为分片 / 卸载后显存占用的给定输入；两个切换峰值精确约为 83.256 / 59.256 GiB，与 H100 SXM 标称 80 GB（74.506 GiB）比较。

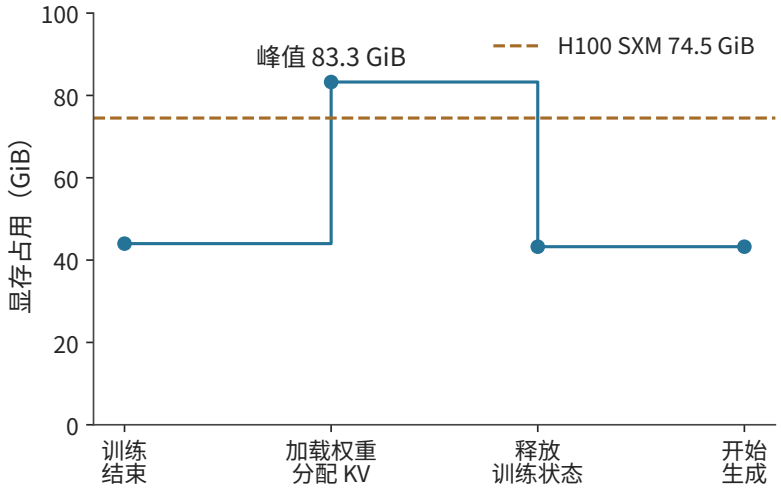



图 10-38：先加载生成权重并分配 KV 池，再释放训练状态，峰值约为 83.3 GiB，超过 H100 SXM 的 74.5 GiB。横轴按操作顺序排列。

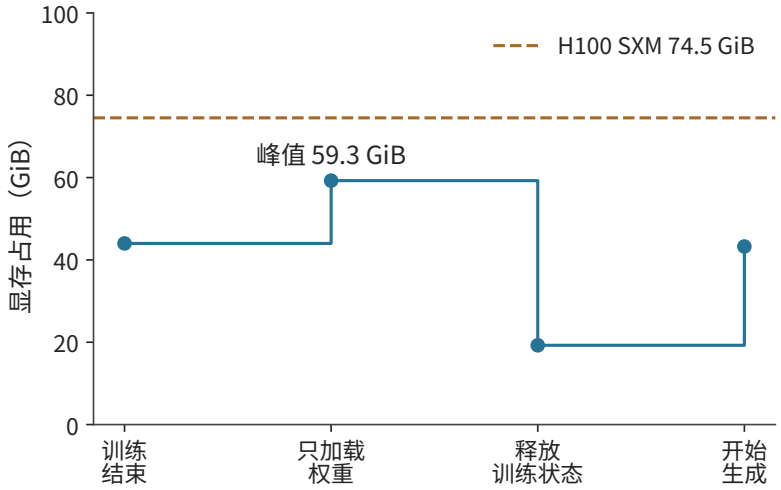



图 10-39：先加载生成权重，再释放训练状态，最后分配 KV 池。峰值降为约 59.3 GiB；两图共用 74.5 GiB 容量线和同一纵轴。

GiB；按各自的分片发送，合计为 423 GiB。先确定每个接收方需要的参数分片，再组织分发，可以同时减少接收缓冲和发送方的发送量。

这里再次用到了本章的两种分析方法：共享加速器时要分析张量的生命周期，独立部署时要分析数据传输和各阶段的处理时间。阶段独立以后，还会产生策略版本差异，下一小节将分析这种版本差异如何影响可用于训练的样本数量。

权重同步中还有一类生命周期不同的权重：第 4.7.3 节的固定权重加速器可以运行 RL 流程中权重长期不变的模型。例如，算法采用固定参考模型时，其权重能够留在专用存储中反复读取；持续更新的策略模型则使用可写权重存储，并将新版本发布给生成端。图 10-40 按这两种生命周期画出权重路径。

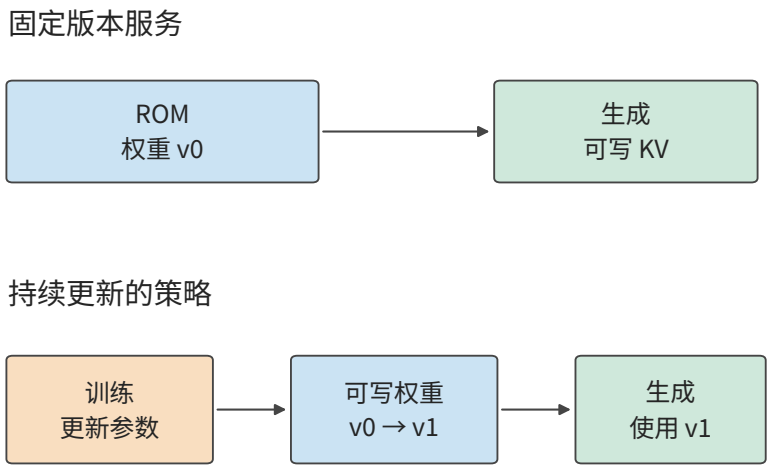



图 10-40：固定版本服务与策略训练的权重生命周期。上方 ROM 重复提供同一版本；下方训练产生新版本并发布给生成端。KV 写入与权重更新使用不同的数据通路。

### 10.5.3 异步训练、策略版本与长轨迹恢复

第 10.5.2 节的独立部署进一步允许不同 batch 交错进行：学习端处理上一批样本时，生成端已经开始生成下一批。同步循环则等待学习和权重同步完成，再产生下一批样本。异步循环让生成与学习重叠，生成端可能仍在使用旧权重。为计算样本对当前参数更新的贡献，需要保留实际采样策略的概率。

权重更新后，模型给出的策略随之变化。下面沿样本流转顺序区分三个策略版本。

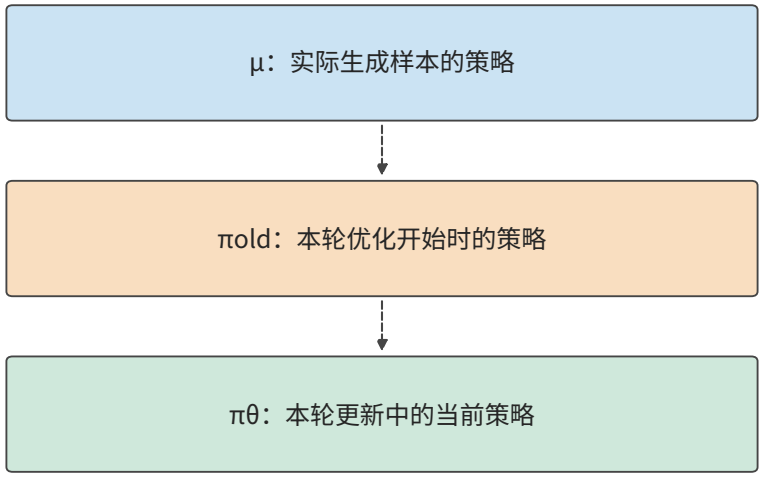



图 10-41：μ 产生训练样本，πold 标识本轮优化的起点，πθ 随本轮更新变化。虚线表示版本演进顺序。三个概率必须针对同一前缀与同一 token 计算。

令实际行为分布为 $\mu$，本轮优化的起点策略为 $\pi_{\text{old}}$，当前优化策略为 $\pi_\theta$。起点策略随优化轮次更新，用 KL 散度做正则时所用的参考模型则保持固定。对同一 token 和前缀，在三个概率有定义且分母非零的位置，概率

比满足

$$\frac{\pi_\theta}{\mu} = \frac{\pi_\theta}{\pi_{\text{old}}}\frac{\pi_{\text{old}}}{\mu}.$$

右边第一项描述本轮参数优化带来的概率变化，第二项描述实际采样分布与本轮起点策略之间的差异，其中既可能有策略版本滞后，也可能有采样配置或跨引擎计算差异。若行为策略和当前策略对该 token 给出的对数概率（logprob）都为 $-3$，总比值为 1；若误用重新计算得到的对数概率代替行为策略原先记录的值，将它写成 $-3.25$，比值就变为 $\exp(0.25) \approx 1.28$。参数没有变化，样本权重却增加了约 28%。保留行为概率，训练端才能区分策略变化与计算差异。[35]

即使每轮都先同步权重，第二项也未必等于 1。生成端通常逐 token decode 并读取 KV 缓存，训练端则把整段轨迹并行输入，重算各 token 位置的概率后再反向传播。不同的注意力实现、归约次序、batch 形状、权重或 KV 精度，都可能让相同权重、相同前缀下的 logprob 不同。第 5.2.4 节已经测量过其中一项：同一行输入按不同段数做归约，平方和相差 166 个 ULP，而切成几段又取决于当时的 batch 大小。两端最先出现差异的是前向计算；反向会把训练前向构造出的损失差异传入梯度。MoE 的离散路由还会放大这种差异，下一小节将展开。

第二项中还有一部分来自采样配置：行为概率按实际采样规则计算。温度缩放、top-k 或 top-p 截断会改变模型原始 Softmax 分布，因此应保存变换后的概率及采样配置。训练端用对应的分词、对话模板、位置编号、注意力掩码和 EOS / 截断位置重建输入，才能对同一事件计算概率比。

在近端策略优化（PPO）一类更新中，概率比决定优势对梯度的权重，也参与裁剪判断：概率比偏离 1 超过设定范围时，裁剪截断它对梯度的贡献。若把实现差异误认为参数更新，样本的贡献就可能被错误地放大或压低，也可能错误地触发裁剪；若使用序列概率比，各 token 位置的 logprob 之差还会相加。因此，较长轨迹会积累更多概率偏差，使更新更容易波动。[36]

重要性加权用概率比调整已采样事件的贡献，因此采样分布必须覆盖目标事件。截断采样排除的 token 无法通过已有样本的概率比补回；分布差异过大时，权重方差也会增大。裁剪可以限制极端权重，但会引入偏差，而且分别裁剪两个因子与裁剪总比值会产生不同的损失。异步执行会增加策略版本的时间差，下面将可用样本比例与执行周期一起计入收益。

**例：异步流水能承受多高的过期样本丢弃率？**每批生成 40 s、学习 16 s、阻塞两侧的权重同步 4 s，同步周期为 60 s。设生成和学习使用独立资源，权重同步仍独占 4 s，则异步稳态周期为 $\max(40, 16) + 4 = 44$ s。

从图 10-42 的同步执行改为图 10-43 的异步执行后，减少的是学习端与生成端轮流等待的时间。但生成得更早也意味着样本可能使用较旧的权重，因此还要扣除被丢弃的样本。

若每批原有 $B$ 条等效轨迹，异步执行后，其中比例为 $q$ 的轨迹保留下来用于训练，则同步和异步执行的有效样本吞吐量分别为 $B/60$、$qB/44$。异步更快要求

$$q > 44/60 = 11/15 \approx 73.3\%.$$

当 $q = 0.8$ 时，有效样本吞吐量提高约 9%；当 $q = 0.7$ 时，虽然每批更早结束，有效样本吞吐量反而减少约 5%。因此，计算异步执行的收益时，还要减去因策略版本过旧而被丢弃的样本。

[35] RL 状态、概率比与版本、DeepSeek V4 技术报告。

[36] RL 状态、概率比与版本、DeepSeek V4 技术报告。

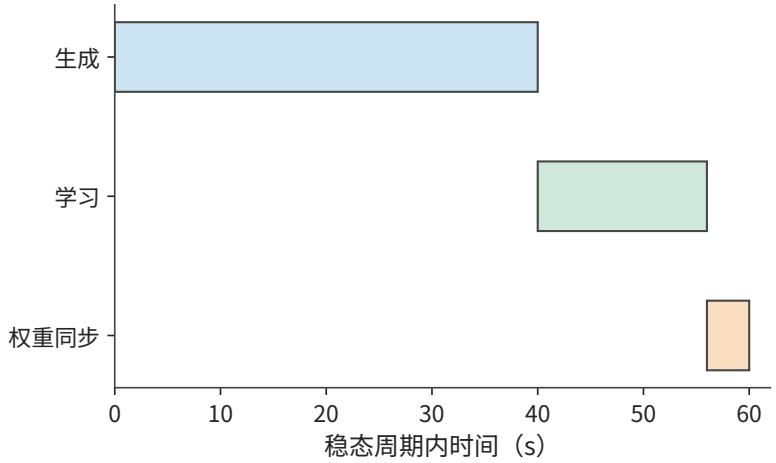



图 10-42：同步循环依次生成本批样本、学习本批样本、同步权重，分别用时 40、16、4 s，共 60 s。

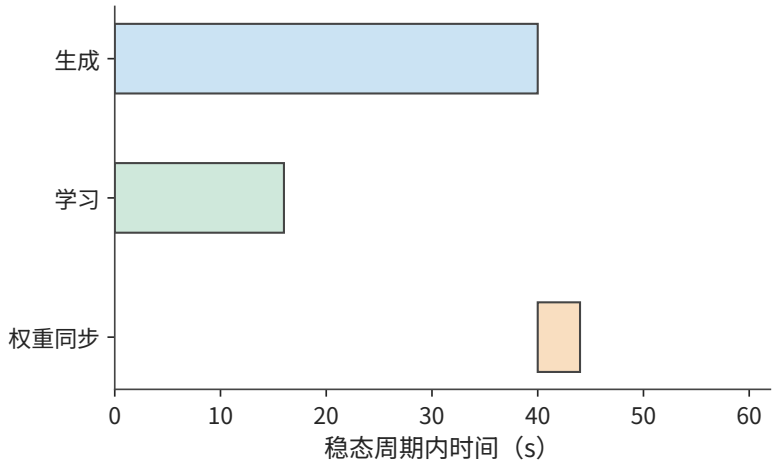



图 10-43：稳态中生成下一批与学习上一批在独立资源上重叠；两者完成后同步权重，周期为 44 s。橙色同步阶段阻塞两侧。

图中的生成阶段画成了一段连续的工作。轨迹越长，这一段越容易跨过参数更新或遇到中断，于是除了筛选完成的样本，还要考虑尚未完成的轨迹如何继续。长轨迹也会延长生成样本与使用样本训练之间的时间差。KV 是给定权重对前缀执行的结果：使用相同权重恢复时，可继续使用已保存的 KV，换权重后则需重新执行前缀来生成对应 KV。Qwen3-8B 的 8K BF16 KV 占 1.125 GiB，保存和取回 KV 缓存需要存储空间与传输时间，重建 KV 缓存则需要重新执行 prefill 计算。恢复方案据此比较取回时间与重建时间。

若总是丢弃中断轨迹并重新采样，长轨迹因执行时间更长而更容易被丢弃。设生成期间的中断率为 $\lambda$，持续 $t$ 的轨迹能不受中断、完整生成的概率为 $e^{-\lambda t}$；随着 $t$ 增长，进入学习的数据比例下降。逐 token 记录生成进度和策略版本，可以在恢复后继续生成同一条轨迹，减少这种由系统中断引入的长度偏移。轨迹在模型与环境两侧的状态如何跨过中断保留下来，见第 11.3.3 节。

让训练与部署使用同一条路径的做法，在实际模型中已经出现。DeepSeek V4.1 把部署时的稀疏访问与恢复方式纳入训练。稀疏注意力从训练之初就使用 64K 序列，后训练加入层级候选限制，使训练和推理使用相同搜索域；量化感知训练（训练时模拟低精度量化的误差）让模型适应 FP4 的主 KV；只重放有限长度的输入来重建解码器 SWA 状态，这一过程也在后训练中模拟。模型由此学习使用部署路径实际提供的表示与局部状态。[37]

[37]DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

### 10.5.4 专家路由重放与训推一致性

MoE 把第 10.5.3 节的数值差异进一步变成离散的路径差异：即使权重相同，生成端与训练端的数值计算差异也可能改变专家选择。由于专家选择是离散的，小幅数值变化就可能让 token 进入另一条计算路径。以选择分数最高的两个专家（top-2）为例，前三个专家分数为 0.500、0.301、0.300 时，选择专家 1、2；第三个分数因计算次序变为 0.302，便改为专家 1、3。很小的分数变化就可能让计算改用另一个专家的权重矩阵，影响可以沿后续层继续传播。

**路由重放**（Routing Replay）记录生成时选中的逻辑专家 ID，训练时按这些 ID 选择专家，再用当前权重计算路由分数、专家输出和梯度。样本、token 位置和层号共同定位这一份记录。这样，生成与训练经过相同的离散专家路径，而数值计算继续反映当前参数。

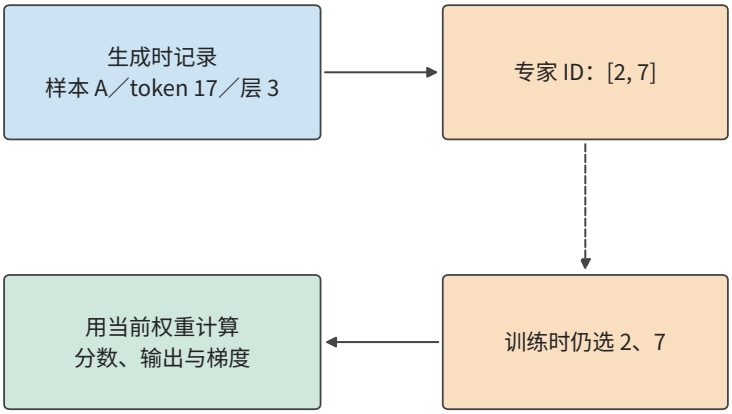



图 10-44：离散专家 ID 连接生成端与训练端，当前权重继续参与数值计算。示意记录为样本 A、token 17、层 3 的 top-2 选择；虚线表示 ID 重放，实线表示当前计算数据流。

先计算这份记录占多大空间。Qwen3-30B-A3B 的 48 层各记录分数最高的八个专家（top-8）的编号。对 8,192 个 token，以每个整数占 2 字节的无符号整数格式 uint16 保存每个专家 ID，大小为

$$8192 \times 48 \times 8 \times 2 = 6 \text{ MiB}.$$

使用每个整数占 4 字节的 int32 格式则为 12 MiB。记录还要随 token 一起移动：序列打包改变 token 的位置，上下文切分改变 token 所在的卡，重计算再次读取同一层的记录。让 ID 与 token 的原始位置共同经过这些变换，训练端才能选中原先那组专家。[38]

> **案例：路由重放对概率误差、训练耗时与奖励的影响。** NVIDIA 公布的路由重放（R3，即 Rollout Routing Replay）验证报告有四组开关对照，各训练 100 步，开启后日志中记录的对数概率误差中位数均下降，每步总耗时中位数略长，训练奖励没有一致提高。重放减少了路由差异，但保存记录和执行重放也增加了耗时。比较两种方案时，分别观察概率误差、训练时间和固定评估任务上的表现。[a]

[38]路由元数据预算、R3 作者公开的日志与分析。四对设置共用 seed42；reward 为训练 batch 指标，logprob 为 R3 作者的日志级统计。

[a]路由元数据预算、R3 作者公开的日志与分析。四对设置共用 seed42；reward 为训练 batch 指标，logprob 为 R3 作者的日志级统计。

同一份权重、同一批 token 下，生成端与训练端的 logprob 差异反映计算路径和采样定义的影响；MoE 的专家 ID 则标明离散路由是否一致。数值是否一致、采样策略滞后当前策略多少，由此共同影响有效训练样本数。

RL 把生成、环境验证和参数更新组织成一个完整的反馈过程。生成端提供采样概率与专家选择，训练端据此组织更新，资源调度再安排各阶段重叠。贯通这些信息，可以同时优化数据交接、执行一致性与资源利用。本节的异步周期和路由重放例子分别量化了节省的等待与增加的记录成本；整个反馈循环最终要用有效样本和训练进展评价。

## 10.6 从系统方案到完成期限与硬件选择

### 10.6.1 汇总容量、每步耗时、停顿与恢复

下面综合计算章首训练任务的完成时间。两套方案都处理 100B token，每步 384 条 8,192 个 token 的序列，共 31,790 次训练迭代。设每步训练时间 $t_s$ 已包含单卡计算、通信等待与输入等待，保存与恢复的附加时间比例为 $L$，计划性停顿为 $T_0$，则本章的一阶完成时间模型为

$$T_{\text{finish}} = T_0 + \left\lceil \frac{D}{b} \right\rceil t_s(1+L).$$

各项来自本章不同层次的分析：分析张量的生命周期，可以确认显存是否足够；分析关键路径，可以求得 $t_s$；checkpoint 模型给出 $L$；计划性停顿为 $T_0 = 5$ 天。将这些结果汇总，就能比较两套方案的完成时间；表中合计由未经四舍五入的数值计算。[39]

| 设计项 | 32 卡方案 | 48 卡方案 |
|---|---|---|
| 八卡主机数 | 4 | 6 |
| 全组 ZeRO-3，每卡单序列累积次数 | 12 | 8 |
| 每卡预计显存占用 / GiB | 13.8 | 12.5 |
| 每卡可用显存 / GiB | 22 | 22 |
| 每步单卡计算 / s | 78.3 | 52.2 |
| 每步 ZeRO-3 链路时间（PCIe 4.0 x16）/ s | 17.9 | 12.0 |
| 每步通信等待 / s | 0.08 | 0.08 |
| 每步等待输入数据的时间 / s | 0.5 | 0.5 |
| 每步训练时间（含 0.5 s 输入等待）/ s | 78.9 | 52.8 |
| 保存、重做与恢复附加时间 | 1.02% | 1.08% |
| 基础训练时间 / 天 | 29.0 | 19.4 |
| 加入保存恢复和 5 天预留 / 天 | 34.3 | 24.6 |

[39]本章设计案例的输入、推导与复算。32 / 48 卡的容量预留、输入等待和计划停顿为给定输入；单卡计算效率取 Llama 3 的 MFU；模型矩阵工作来自既有 training-deadline 计算，ZeRO-3 通信量由同一份参数表推出。RTX 4090 的 PCIe 4.0、无 NVLink 见 RTX 4090 规格页，卡间不支持 P2P、八卡主机配八张 200 Gbit/s 网卡见 4090 集群配置分析；一步中只有第一次 AllGather 与最后一次 ReduceScatter 无法隐藏见 MegaScale §3.2。逐项数值与独立复算保存在 design-case.json。

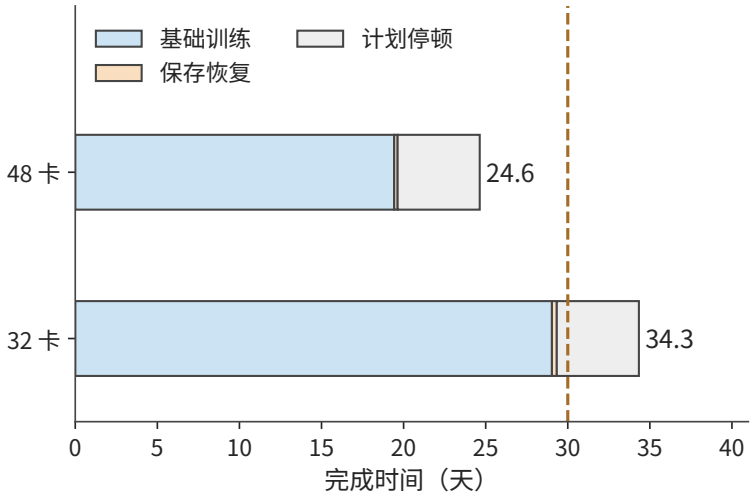



图 10-45：每条横条依次累计基础训练时间、保存与故障恢复的附加时间，以及预留的 5 天计划性停顿。基础训练时间已经包括通信和输入等待；橙色小段为 checkpoint 模型得到的额外耗时。两套方案使用相同任务和全局 batch，虚线标出 30 天期限。

图 10-45 中，两套方案的计划停顿相同，保存与恢复也只占很小一段。主要差别在蓝色的基础训练时间：48 卡把 micro-batch 分给更多的卡并行处理，显著缩短了这一段。

**在这组设计条件下，选择 48 卡方案。**两者都满足容量要求；32 卡方案完成时间超过 30 天，予以排除；48 卡约 24.6 天完成，剩余约 5.4 天。增加卡数的主要价值是减少每卡承担的训练工作，额外故障成本只抵消很小一部分收益。

### 10.6.2 4090 集群的可行范围

图 10-45 给出了约 5.4 天的剩余时间。将这部分余量分摊回每次迭代，就能知道每步还允许增加多少等待，也就得到了可用于实际调优的性能目标。48 卡的每步训练时间必须满足

$$t_s \le \frac{25 \times 86400}{31\,790 \times (1 + 0.0108)} \approx 67.2 \text{ s}.$$

本方案为约 52.8 s，余量约 14.4 s/步。固定单卡计算时间约 52.2 s、输入等待 0.5 s，则总通信等待最多约 14.5 s；按第 10.3.3 节的重叠分析，当前只有约 0.08 s。即使预取完全失效，12.0 s 的链路时间全部暴露，每步约 64.8 s，仍在上限之内。

32 卡则需要提高单卡计算效率。扣除恢复、0.08 s 通信和 0.5 s 输入后，可留给单卡计算约 66.7 s，而 40% 效率下需要约 78.3 s。相同工作量要求单卡计算效率提高到约 $40\% \times 78.3/66.7 \approx 47\%$，已高于 Llama 3 报告的最高 43%。于是，两条改进路线可以直接比较：增加两台主机，或者在原有的卡上把单卡计算效率从 40% 提高到约 47%。

通信预算还要落实到物理接口。第 10.3.3 节的 12.0 s 链路时间以每卡一张 200 Gbit/s 网卡为前提。若一台主机的八张卡只共用一张 200 Gbit/s 网卡，环形集合通信进出这台主机的那条边要承载每卡的全部收发量（第 7 章例题 7.1 的割集），48 卡每步约 385 GB 都要经过这张 25 GB/s 的网卡，链路时间约 15.4 s，超过 14.5 s 的通信上限。每个 micro-batch 的链路时间约 1.92 s，仍短于 6.53 s 的计算，预取有效时暴露的仍只有首尾两次通信；预取一旦失效，每步约 68.1 s，方案超期。每卡配一张网卡时，12.0 s 即使全部暴露也在上限以内。两种配置的差别在于：共用网卡把按期完成押在重叠上，每卡一张网卡则不依赖重叠。

这样，性能目标就对应到了具体的执行过程：每步最多允许 67.2 s，其中计算约需 52.2 s，其余时间留给通信和输入等待。调优时首先缩短超出预算的那段关键路径，如果显存已经足以保存所需张量，就不必再为节省显存而增加额外工作。

### 10.6.3 替换硬件后的瓶颈与选择

更换加速器时，可以继续沿用上一小节的预算，判断计算变快会缩短哪一段，带宽变小又会延长哪一段。新的加速器会同时改变单卡计算、容量和互联。将原有每步耗时分成计算、通信等待等部分，可以判断某项硬件变化有多大价值。设某资源占原每步耗时比例为 $f$，处理能力改为原来的 $r$ 倍，保持其他执行和依赖不变，则

$$\frac{T'}{T} = 1 - f + \frac{f}{r}.$$

这就是根据各项耗时的比例应用 Amdahl 定律。原来 100 ms 中只有 5 ms 通信等待，带宽减半后变为 $95 + 10 = 105$ ms；若原来通信占 50 ms，同样减半后为 $50 + 100 = 150$ ms。前一个任务的耗时增加 5%，后一个增加 50%，因为等待该资源的原有时间占比相差十倍。

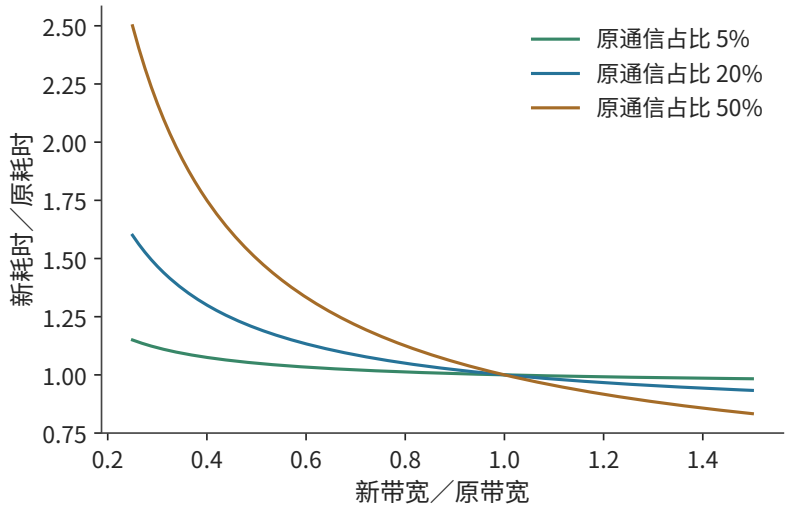



图 10-46：资源能力变化的收益由原有等待时间占比决定。曲线按 $T'/T = 1 - f + f/r$ 计算，固定单卡计算与依赖，通信时间与有效能力成反比。

在本章的 48 卡方案中，暴露的通信只有约 0.08 s，约占每步的 0.14%。PCIe 带宽减半时，每个 micro-batch 的链路时间升到约 3.0 s，仍短于 6.53 s 的计算，暴露部分翻倍到约 0.15 s，每步从约 52.8 s 增到 52.9 s，几乎不变。若单卡计算效率从 40% 降到 30%，单卡计算时间变为约 69.6 s，加上通信与输入约为 70.2 s，超过 67.2 s 的目标。对这套方案而言，决定能否按期完成的是单卡计算效率；带宽只有在预取失效、通信整段暴露时，才成为式中比例 $f$ 较大的那一项。

比较 A100、A800、H20 或其他加速器时，可将其单卡计算时间、接口传输时间和容量代入同样的分析。满足期限的方案再比较加速器租用、能源和准备成本。更强的单项指标是否值得购买，取决于它能减少多少完成时间或卡数。

### 10.6.4 模型规模、数据量与期限的敏感性

前几节保持模型不变，比较了卡数和加速器型号。最后把同一方法扩展到模型规模，考察模型放大后计算量的增长。采用稠密模型的粗略计算量公式 $F = 6ND$，固定数据量 $D = 20$T token、40% MFU 和可用于执行

的期限 $T$，满足期限所需卡数为

$$p \geq \left\lceil \frac{6ND}{P\eta T} \right\rceil .$$

图 10-47 把 90 天期限画成卡数边界。选择某个模型规模后，边界上方的计算资源能够在该效率下完成工作，边界下方需要提高效率或延长期限。

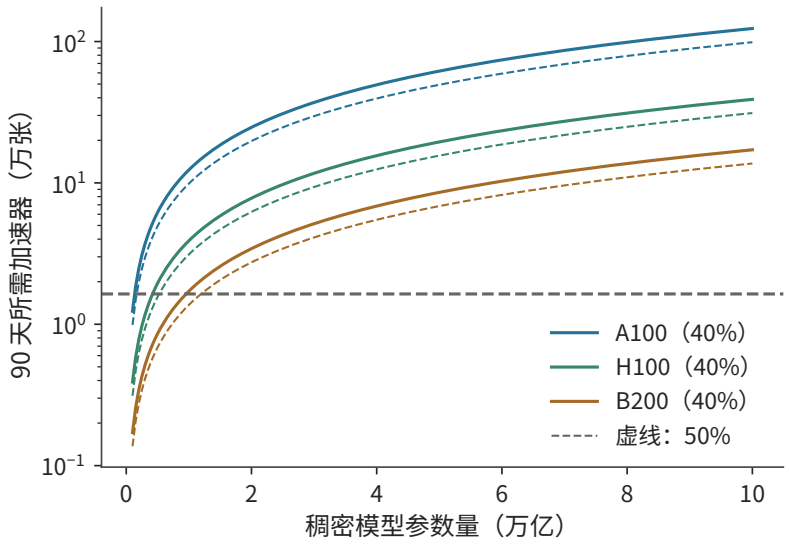



图 10-47：固定 90 天执行期限，稠密模型规模增大要求更多的卡。数据量为 20T token，算法工作为 $6ND$，实线的 MFU 为 40%，虚线为 50%，使用 BF16 稠密矩阵峰值。各线为向上取整前的连续计算边界；横线标出 16,384 张卡。

对 1T 参数，16,384 张 A100、H100、B200 在 40% MFU 下分别约需 679、214、94 天。90 天执行预算下，三者都落在计算边界下方：B200 也超出约 4 天，需增至约 17,147 张。只有 MFU 达到 50% 时，B200 才降到约 75 天，落进期限，余下约 15 天可分配给计划停顿与恢复。MFU 相差 10 个百分点，就决定了 16,384 张 B200 能否在 90 天内训练完 1T 模型。[40]

固定数据量时，模型参数量增至五倍，计算量也增至五倍；若数据量同时取 $D = 20N$，则 $F = 120N^2$，参数量增至五倍，计算量就增至 25 倍。规模问题的答案由模型和数据共同决定。MoE 则分开计算全部可训练参数的状态与被激活路径的执行工作，再将专家交换放入关键路径。

设计完成以后，还要检验长期执行是否达到预期。公开训练日志提供长期执行的观察窗口。若从 checkpoint 继续训练，本次新增 token 等于结束累计值减去恢复起点累计值；用新增工作除以本次时间，得到对应的长期速率。该速率可以直接检验整套系统能否保持设计中的有效训练进度。[41]

回看全章，图 10-10 至图 10-14 解释了数据如何占用显存和经过链路；图 10-17 至图 10-36 说明这些操作如何形成等待、又如何影响长期训练；图 10-45 则把各项耗时汇成了章首任务的完成时间。状态安排、执行顺序和恢复方式由此连成了同一个设计问题。RL 将工作来源改为反馈循环，仍通过同一方法比较资源和有效训练进度。下一章进一步讨论这些作业与阶段如何共享资源池。

[40] Dense 规模与期限、数据量随参数增长的变体。16,384 张 H100、DP=128、8K 序列时 41% 的 BF16 MFU 见 Llama 3 报告表 4。

[41] SmolLM3 原始训练记录核验。SmolLM3 记录含 384、288、192 rank 阶段，最后一次恢复实际运行 14,000 步；终点累计 token 包含此前工作。

## 常见误区

**误区：分片数翻倍，训练显存峰值就减半。**分片只直接缩小分给各卡的常驻训练状态。小模型例中常驻训练状态从 18 MiB 减到 9 MiB，执行期间的其他张量和缓冲区仍占约 21 MiB，峰值从 39 MiB 减到 30 MiB。决定容量需求的是同一时刻的总显存占用。

**误区：通信处理时间更短，训练步耗时就更短。**三个小桶的总处理时间约 4.59 ms，比整桶 4.55 ms 更长，却可以分别利用三个计算空闲时段完成传输。通信完成的时刻决定它是否延长训练步。

**误区：异步保存调用返回，就能恢复全部已有进度。**50 s 故障时，可用恢复点取决于哪份快照已经提交。相同的 1 s 前台暂停可以对应 30 s 或 10 s 重做，区别来自后台写入的完成时刻。

**误区：提高生成吞吐率，就能加快 RL 学习。**生成 12 条/s、验证六条/s、样本保留比例 75% 时，学习端每秒只能收到 4.5 条轨迹。增加生成资源无法消除验证阶段的瓶颈，提高验证能力才会把限制移到训练端。

**误区：卡数翻倍且 batch size 随卡数放大，完成时间就减半。** batch size 超过临界 batch size 后，达到同一损失所需的步数不再明显减少。噪声尺度为二百万个 token 时，每步 batch size 从 315 万放大到约一亿个 token（1536 卡），完成时间只从 19.4 天降到 12.1 天，加速上限为 1.64 倍。batch size 放大受梯度噪声尺度限制；保持 batch size 不变、只增加卡数的强扩展才不受它限制。

## 习题与实验

以下十题按基础计算、机制分析和综合设计递进。计算题所需条件均在正文中；实验部分使用配套记录或自行测量的数据完成。

**习题 10-1 ·基础：训练状态分片与每卡峰值内存**

将本章梯度改为 FP32，计算 Qwen3-8B 常驻训练状态。再求八卡 ZeRO-1、ZeRO-2、ZeRO-3 的每卡容量。设执行中有 6 GiB 激活和两份各 3 GiB 的模块缓冲，分别计算两份缓冲同时占用显存与顺序复用时的峰值。

**习题 10-2 ·基础：按期完成训练需要多高的单卡计算效率**

采用本章的训练任务，将全局 batch size 设为 384 条序列，计算 30 天和 25 天对应的每步预算。给定 32 卡，先按第 10.3.3 节求 ZeRO-3 每步的链路时间与暴露的通信等待，再加上 0.5 s 输入等待；先忽略恢复，求在 25 天内完成训练所需的最低单卡计算效率；再加入本章恢复成本。最后取梯度噪声尺度为二百万个 token，按第 10.1.1 节的关系式求弱扩展的加速上限，以及 96 卡和 1536 卡在弱扩展与强扩展下的完成时间；把噪声尺度改为两千万个 token 后重新计算上限。

**习题 10-3 ·综合设计：在容量、期限与加速器成本之间选择训练方案**

比较 32 卡与 48 卡方案的训练完成时间，以及累计使用的 GPU · 小时。将每步激活与临时缓冲区的峰值需求从 10 GiB 提高到 20 GiB，重新判断容量。若 BF16 权重副本改用第 10.1.2 节的 FP8 分块缩放格式保存，重算两套方案的每卡常驻状态与容量余量。若只能租用四台八卡主机，分别计算需要将任务量减少到多少、将期限延长到多久，或将计算效率提高到多少，才能完成训练。实验部分选一条路线实际测量，用测得的数值替换本章给定的参数。

**习题 10-4 ·分析：micro-batch 数与激活重计算如何改变流水开销**

在四阶段、前向 10 ms、反向 20 ms 的模型中，求 1、4、8、16 个 micro-batch 的理想利用率。沿图 10-19 解释 93—95 ms 的等待来自哪项依赖。再对八个 micro-batch 比较交错式 1F1B（$v = 2$）与零气泡调度（ZB-H1，$T_B = T_W$）：用正文的气泡公式分别计算气泡时间，对照事件模型给出的完成时间 298 ms 与 283 ms，并说明交错式为什么增加跨界传输次数。假设每层每个 micro-batch 的可重建乘积占 10 MiB，原方案中，四个

micro-batch 尚未执行反向传播，每个 micro-batch 都需要保留九层的这些乘积。改为每次只重建一层中一个 micro-batch 所需的乘积，使用后立即释放。计算新方案可以少保留多少数据，以及逐层串行重建所需的工作区大小；再讨论两层同时执行反向传播时工作区怎样变化。

**习题 10-5 ·分析：数据准备进程暂停时，需要预取多少序列**

48 卡每步处理 384 条序列，每步训练时间约 52.8 s，四个数据准备进程每个每秒准备两条序列。若一个数据准备进程暂停 60 s，按训练端的平均消耗速率计算，求维持原训练速度至少需要预存多少条序列，以及四个准备进程全部恢复后，要多久才能把预取队列补回暂停前的数量。再按第 10.4.3 节 7 GB/s 的存储合计写入，求约 115 GB 的 checkpoint 不产生写入积压的最短保存间隔。

**习题 10-6 ·分析：checkpoint 保存频率与故障重做成本**

根据本章的一阶近似模型推导最优保存周期，计算 1024 卡作业分别采用 300、900、1800 s 保存间隔时，保存 checkpoint 与故障重做造成的时间开销。加入每天一次、会同时影响全作业的共同中断，重新计算故障率和最优周期。再把平均七天一次的损失尖峰回滚作为同时影响全作业的冲击，加入 48 卡的模型，重算最优保存周期与 600 s 间隔处的附加时间比例。实验部分在提交前后分别注入故障，比较恢复位置与需要重做的工作。

**习题 10-7 ·综合设计：RL 各阶段配比与有效样本吞吐**

设生成、验证与学习阶段的处理率分别为 12、6、8 条/s，验证后样本的保留比例为 75%，比较只将一个阶段能力翻倍的收益。另采用生成 40 s、学习 16 s、权重同步 4 s 的周期模型。设每批生成的样本数为同一个常数，分别计算异步执行后样本保留比例为 80% 和 70% 时的有效样本吞吐率，并与不丢弃样本的同步执行比较；结果可表示为每秒处理的 batch 比例。实验部分追踪一批样本的标识、奖励、有效损失位置和接收方的权重。

**习题 10-8 ·分析：路由记录与样本对应关系**

设序列含 8K token，模型有 48 层，每层为每个 token 选择 8 个专家。分别计算用 uint16 和 int32 编码专家 ID 所需的记录容量。将两条序列打包后，再沿上下文切成四份，描述专家 ID 如何随 token 重排。构造一个 token 位置错位的例子，说明训练端将如何使用错误专家，并设计定位这一错误的检查。

**习题 10-9 ·分析：通信、计算效率与输入等待如何影响训练期限**

在本章的 48 卡设计案例中，分别将 PCIe 带宽减半（分预取有效与完全失效两种情况）、单卡计算效率降至 30%、输入等待增至 5 s，重新计算训练完成时间，并判断能否满足期限。再给每卡计算时间加上 5% 标准差的独立波动，求 48 卡每步的期望耗时，并判断该方案是否仍满足期限。假设上述退化同时发生，而额外资源只能将其中一项恢复到原值，比较分别恢复各项时可减少的关键路径耗时。再选一份硬件规格或执行记录，把本章的一个给定输入替换为有来源的数值。

**习题 10-10 ·综合设计：满足训练期限需要多少加速器、付出多少成本**

对于参数量分别为 1T、5T、10T 的稠密模型，用 $6ND$ 分别计算两种数据规模下的训练运算量：训练数据固定为 20T token；训练 token 数满足 $D = 20N$。再按 40% 与 50% 两档 MFU，求各情形下在 90 天和 180 天内完成训练所需的最少 A100、H100、B200 卡数。以累计 GPU · 小时比较两个满足期限的方案，并说明准备时间和恢复参数如何改变结论。实验部分选取一份标明恢复起点的公开日志，扣除恢复时已经完成的 token 数，计算观察期间新增训练 token 的平均处理速率。

## 本章小结

训练所需的内存容量由状态的存储格式、各卡之间的分配方式和生命周期共同决定。ZeRO 减少状态复制，重计算减少前向到反向之间需要保存的数据，卸载将暂时不用的训练状态移到主机内存；三者都以额外的计算或传输换取显存空间。执行时，数据就绪与资源空闲共同决定开始时刻，沿依赖关系累积到参数更新结束的延迟决定每步耗时。

估算长期完成时间时，还要加上每步耗时中未包含的数据准备、checkpoint 保存与故障重做时间。checkpoint 既保存优化器状态，也保存已完成训练的数据位置；保存周期平衡正常运行中的写入成本与故障后的重做成本。同步训练还受三类实际因素影响：batch size 能否随卡数放大由临界 batch size 决定，每步耗时由最慢的一张卡决定，损失尖峰的回滚率与硬件故障率一起进入保存周期模型。RL 进一步要求跟踪样本标识、策略版本和行为概率，并对齐生成与训练的数值路径。在线采样使数据跟随策略变化，OPD 让教师监督覆盖学生实际生成的前缀，跨引擎执行还需要保证训推一致性。控制数据分布偏移与执行误差后，便可以根据有效样本吞吐和任务评估，判断阶段重叠如何加快学习。

本章的设计案例中，32 卡与 48 卡 RTX 4090 均能容纳训练状态，但给定相同训练目标和执行效率后，完成时间分别约为 34.3 天和 24.6 天。48 卡满足 30 天期限，每步训练时间上限约为 67.2 s；ZeRO-3 每步 12.0 s 的链路时间绝大部分被计算覆盖，即使完全暴露也不超出这一上限。这一结论来自容量、执行与恢复的连续推导；同一分析模型也给出了效率、通信和故障条件变化时，方案能否按期完成的临界值。

# 第 11 章 资源调度与运行环境

代码修复服务收到用户提交的问题后，先让模型生成修改，再让工具改文件、执行测试，把测试结果交回模型，继续下一轮，直到得到通过测试的补丁。这项需要交付最终结果的工作称为**任务**；一次模型请求或一次工具执行只是其中的一个步骤。

**控制器**保存任务记录并决定下一步调用什么，**执行器**实际运行工具，**运行环境**承载工具所需的文件、进程和内存。模型调用使用推理服务，工具执行使用 CPU，环境中的进程和缓存占用内存。沿着每次调用、返回和等待的过程，可以分别算出这些资源何时被占用、占用多久。

前两章讨论了推理和训练服务的构建。本章研究如何把模型服务、工具环境和共享资源组织成任务运行系统，从一道平台设计题出发，逐步讨论何时创建和释放环境、如何安排任务、如何选择模型，以及如何从失败中恢复。

Agent 与 RL 的工具同样在容器和虚拟机中执行，但承载它们的平台不能照搬通用云平台的设计，原因有三。其一，云平台的请求来自大量相互独立的租户，统计复用可以削平峰谷；一个 RL 作业却会同时申请上万个沙箱。其二，通用容器按各自的节奏运行；沙箱大部分时间在等待模型生成，节奏由 GPU 一侧决定，平台首先要避免昂贵的加速器空等。其三，沙箱中的程序由正在训练的模型生成，训练会促使模型寻找环境中的漏洞，隔离设计必须考虑这类行为。这三点依次在第 11.3.2、11.1.2 和 11.2.1 节展开。

**贯穿设计题：代码修复平台的资源容量、完成期限与任务成本。**本章将围绕以下条件展开讨论。任务均匀到达，每 0.1 秒一项；每项执行三轮，每轮先调用模型，再运行工具。工具使用一个 CPU 核，执行 1 秒，包含读取文件、执行测试、保存结果和清理工作状态。工作目录与工具结果存放在持久存储中，工具进程可以在轮间重建。

| 设计输入 | 给定值 |
|---|---|
| 任务到达率 | 10 项/s |
| 每项任务 | 3 次模型调用、3 次工具执行 |
| 模型服务 | DeepSeek V4-Flash，每个副本 4 张 B200，同时服务 32 个会话 |
| 普通模型服务的完整调用时间 | 每次输出 355 个 token，每个 token 约 25.3 ms，合计约 9.0 s |
| 模型服务并发容量 | 10 个副本（40 张 B200），最多同时处理 320 次调用 |
| 工具执行时间与累计 CPU 时间 | 1 s、1 CPU · 秒/次 |
| 活跃工具环境 | 1 vCPU、2 GiB 内存的 microVM（微型虚拟机） |
| 工具平台 | 一台 EC2 m5d.metal（亚马逊云的裸金属服务器）：48 个 CPU 核、384 GB（约 358 GiB）内存 |
| 业务目标 | 至少 95% 的提交任务在 24 s 内通过测试 |

每个 token 的时间取自本书对这一部署的服务估算，已计入分摊到每个 token 的输入处理与上下文重建。m5d.metal 是 Firecracker（亚马逊开源的轻量虚拟机监控器）论文评测 microVM 时使用的主机，1 vCPU、2 GiB 是 E2B（面向工具执行的环境平台）计价页列出的沙箱规格之一。

先考察一种简单的方案：任务开始时，从预先准备好的环境池中分配一个环境，直到任务结束才释放；清理工作状态后，该环境可以交给下一项任务。后文依次考虑环境创建开销、请求突发、模型选择和失败恢复，在同

一组任务条件下比较各方案的资源需求。[1]

## 11.1 根据任务执行过程计算资源需求

### 11.1.1 一项代码任务的执行与等待

以章首的代码修复任务为例。每轮先由模型用 9 秒生成工具调用，再由工具执行 1 秒。下一轮模型调用需要本轮的测试结果，因此三轮必须依次进行，总时间为 $3 \times (9 + 1) = 30$ 秒。第一轮结束于第 10 秒，第二轮结束于第 20 秒，第三轮结束于第 30 秒。

等待模型给出下一次工具调用时，工具暂时不占用 CPU，但本方案仍保留其环境。执行停下来，并不自动腾出已占的空间：留下的是环境的进程与内存；工作目录和工具结果已经在持久存储中，是否值得销毁环境、下轮再建，第 11.2 节再比较。

在这 30 秒里，工具只占用一个 CPU 核执行三次，每次 1 秒，累计 CPU 时间为 3 秒。这里的 CPU 时间是各个核实际执行时间的总和，记作 CPU · 秒：一个核执行 3 秒，或三个核各执行 1 秒，都是 3 CPU · 秒。环境始终占用 2 GiB 内存，内存占用量与时间的乘积为 $2 \times 30 = 60$ GiB · 秒。

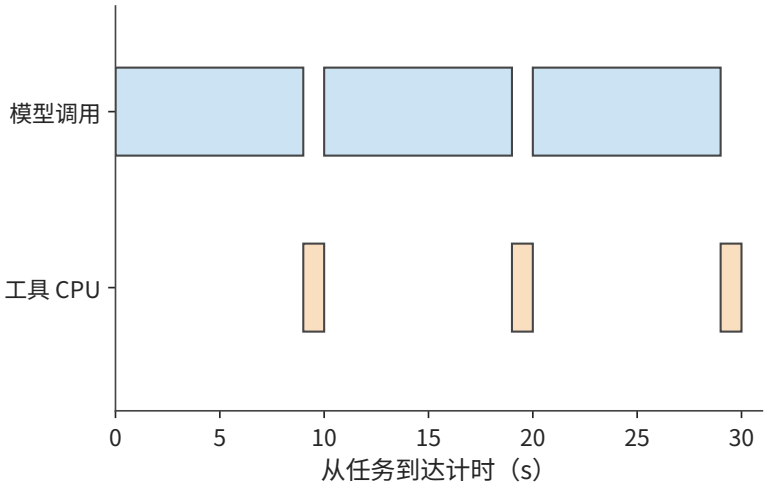



图 11-1：三轮串行任务共需 30 s。蓝色为三次各 9 s 的模型调用，橙色为三次各 1 s 的工具执行；工具累计消耗 3 CPU·秒。

这种等待来自控制器与执行器之间的交接。模型调用结束时，控制器取得完整工具参数，随后把参数发给执行器。执行器修改文件、启动测试、保存结果，再将结果交回控制器。工具必须等模型给出完整参数才能执行，因此在任务的大部分时间里都不占用 CPU。

模型采用流式输出时，可以更早返回部分文本，但工具仍需等待完整的调用参数：收到目标文件、修改内容和操作类型之后，执行器才能执行这次修改。模板下载和运行时初始化只依赖预先确定的环境类型，可以提前放到模型生成期间。第 11.2 节将利用这段时间准备环境。[2]

[1] 贯穿平台与例 11-9 的输入、逐阶段时间表、成本、成本相等的条件与复算见贯穿设计数据及验证程序。模型调用时间取自本书对 4 张 B200 运行 DeepSeek V4-Flash、每会话 200K 上下文的估算：每副本 32 个会话时每个输出 token 25.3 ms，16 个会话时 16.9 ms，均含分摊的输入处理与上下文重建（比较结果、完整数据）；355 个 token 的调用分别约 8.99 s 和 6.01 s，设计取 9 s 和 6 s。B200 每卡时 6.79 美元取自 Runpod 价格页快照。m5d.metal 的 48 核（关闭超线程）与 384 GB 内存取自 Firecracker 论文第 5 节的评测环境；1 vCPU、2 GiB 规格与 CPU、内存单价取自 E2B 计价页快照。热路径准备共 2 秒、0.1 CPU · 秒与 95% 验收通过率为题设，结果保存与工作状态清理包含在工具的 1 秒内；模板容量算例单独说明内容加载。

[2] 调度、模型路由与云端环境研究，包含 ASI、RLBoost、DistRS、SpecBox。ASI 研究覆盖六个月、155,410 张 GPU，其分配比例度量设备归属。token 单价取自 Claude API 计价页快照。

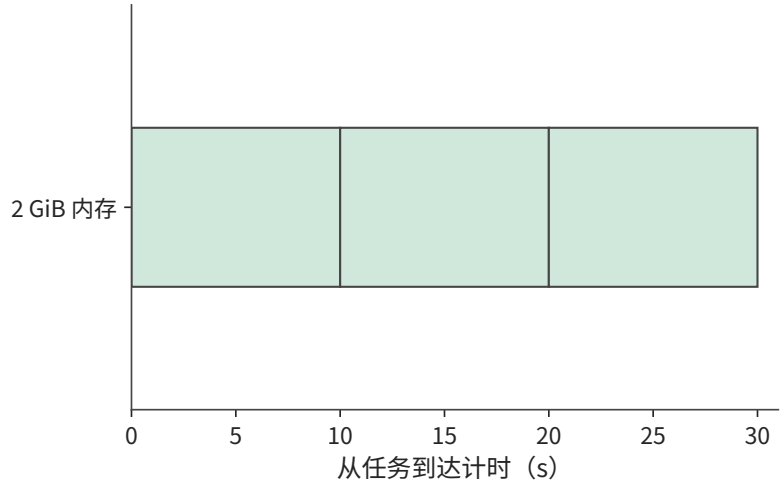



图 11-2：同一任务的环境始终占 2 GiB，持续 30 s，矩形面积为 60 GiB·秒。等待模型时仍保留环境，因而内存占用持续时间长于工具的 CPU 执行时间。

把该例推广到 $R$ 轮。各阶段依次执行时，任务总时间为

$$T_{\text{task}} = T_{\text{queue}} + T_{\text{prepare}} + \sum_{r=1}^{R} (T_{\text{model},r} + T_{\text{tool},r}) + T_{\text{commit}}.$$

上式依次计入排队、环境准备、各轮模型调用与工具执行，以及最终提交结果的时间。模型调用时间包括处理请求和返回结果，工具执行时间包括保存本轮结果，$T_{\text{commit}}$ 则表示最终提交还需花费的时间。如果在模型调用期间准备环境，两者都完成后，工具即可开始执行。原本串行的两个阶段有了重叠，任务总时间和环境占用时间也随之改变。

### 11.1.2 CPU 工作量、环境驻留与并发容量

把单项任务的需求汇总到整个平台，先分别计算任务数、模型调用数和环境创建次数。平台每秒收到 10 项任务，每项需要三次模型调用，因此稳定运行时模型服务每秒接收 30 次调用；工具也每秒执行 30 次。若每项任务只创建一个环境，创建率是每秒 10 个；若每轮重新创建，则是每秒 30 个。任务量虽然相同，每轮重建环境却使创建次数增至三倍。

这些需求都来自同一个关系：

$$\text{资源需求率} = \text{任务到达率} \times \text{每任务的平均资源需求}.$$

每项任务累计使用 3 秒 CPU 时间，所以每秒到达的任务合计需要 30 秒 CPU 时间，平均有 30 个 CPU 核在执行工具；每项任务的累计内存占用为 60 GiB · 秒，所以平台的环境平均需要 600 GiB 内存；每项有三次 9 秒模型调用，所以平均有 $10 \times 3 \times 9 = 270$ 次模型调用正在执行。

**例 11-1：代码修复平台为何先受环境内存容量限制？**将需求除以章首给定的容量，CPU 占用为 $30/48 = 62.5\%$，模型调用的并发容量使用率为 $270/320 \approx 84\%$，内存需求却达到 $600/358 \approx 168\%$。358 GiB 内存最多同时保留 178 个 2 GiB 环境；每个环境要保留 30 秒，平台每秒最多只能开始约 5.9 项任务，其余任务只能排队。此时 CPU 还空着 18 核，增加 CPU 核数也无法解决环境内存不足的问题。

也可以先求环境数量。每项任务占用环境 30 秒，每秒有 10 项任务开始使用环境，因此系统稳定运行时，平均有 300 个环境正在使用。这就是 Little 定律在本题中的形式：

$$\overline{N}_{\text{env}} = \lambda_{\text{env}} \overline{T}_{\text{env}} = 10 \times 30 = 300.$$

每个环境固定占用 2 GiB 内存，300 个环境便需要 600 GiB。当任务的内存占用随阶段变化时，将每项任务的占用面积 $A_M = \int M(t)dt$ 代入 $\lambda_{\text{task}} E[A_M]$，仍可得到平均内存需求。同样，CPU 时间 $W_{\text{cpu}} = \int n_{\text{busy}}(t)dt$ 将各个核的实际执行时间相加。

把每次模型调用从 9 秒延长到 12 秒，工具执行保持不变。任务时间变成 39 秒，平均内存需求升到 780 GiB，同时处理的模型调用增至 360 次，工具仍然只需平均使用 30 个 CPU 核。内存缺口进一步扩大，模型调用并发数也超过了 320 个并发槽，排队的任务更多了。由此可见，模型响应变慢，也会增加工具平台的内存需求，因为环境需要保留得更久。

图 11-3 和图 11-4 将三种资源换算为各自容量的百分比。每次调用 9 秒时，内存柱已经越过容量线；调用变慢后，CPU 柱长不变，模型并发柱也越过了容量线，内存柱则更长。问题因此从“有多少空闲 CPU”转向“能否减少等待期间占用的内存”。

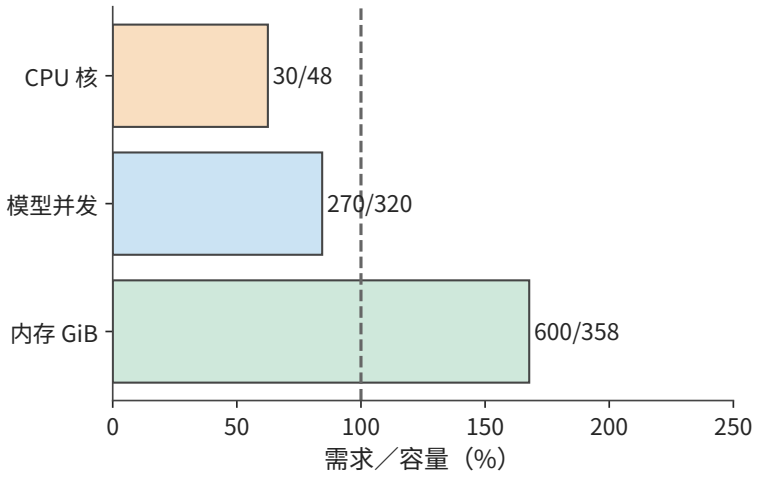



图 11-3：每秒 10 项任务、每次模型调用 9 s 时，三种资源的需求占容量比例。柱末给出需求量／可用容量；虚线为 100%。

生产规模的 Agent 训练平台也是这样。DeepSeek 的 DSec 平台为 RL rollout 和评测提供沙箱，按其一周的运行记录，约九成容器和 microVM 平均只用到所申请 CPU 的 5% 以内；agent 修改的文件、安装的依赖和启动的服务，在模型生成期间却一直保留。沙箱存活时间的中位数约一刻钟，长的超过三小时。既然 CPU 大多闲置，平台便实行**超卖**（overcommit）：各沙箱申请的 CPU 加起来远多于物理核数。DSec 的单个节点曾稳定运行 3,200 个容器或 800 个 microVM。密度再往上提，制约因素有两个：等待期间一直占用的内存，以及挤在同一物理核上的任务相互拖慢。[3]

除了缩短环境保留时间，还可以错开工具执行，减少同时占用内存的进程。一组四进程 CPU 工具实验中，同时启动与错开启动的累计 CPU 时间分别约为 0.29 秒和 0.27 秒，工作量接近。**驻留集大小**（resident set size，RSS）是进程当前驻留在物理内存中的页所占的容量；采样得到的 RSS 峰值从约 320 MiB 降到 80 MiB，整组

[3]DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

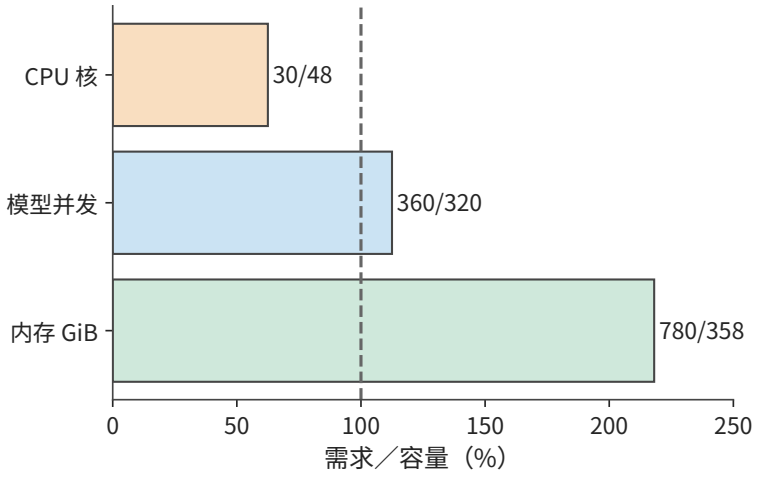



图 11-4：每次模型调用由 9 s 延长到 12 s，CPU 需求仍为 30 核；并发调用增至 360，环境内存增至 780 GiB。虚线为 100% 容量。

完成时间从约 0.10 秒延长到 0.54 秒。错开启动后，同时运行的进程减少，内存峰值随之下降，但整组任务也需要更长时间才能完成。[4]

### 11.1.3 任务完成条件与必须保留的状态

减少等待期间的内存占用，意味着要释放环境中的一部分状态。释放之前，必须先明确丢失这些状态会导致哪些工作重做。在图 11-1 的三轮任务中，前两轮结束时，已经过去了 20 秒，其中工具累计使用的 CPU 时间只有 2 秒。如果丢失工作目录后必须从头执行，就要重新花费这 20 秒，而不仅是重做 2 秒工具计算。能保留哪些状态，决定了故障后哪些阶段需要重来。

| 状态 | 需要记录的信息 | 丢失后的恢复方式 |
|---|---|---|
| 模型上下文和生成进度 | 任务、模型版本、输入前缀、已采用的输出 | 重新提交上下文，重建模型状态 |
| 工作目录与文件 | 仓库版本、文件摘要、补丁版本 | 从持久副本取回或重新应用补丁 |
| 进程与内存 | 环境实例、进程状态、运行时条件 | 恢复快照或重新执行工具 |
| 工具结果 | 工具操作、执行尝试、输入文件版本 | 查询已保存结果或再次执行 |
| 外部操作 | 目标系统中的操作标识与提交状态 | 查询、去重或按业务规则补偿 |

记录这些信息时，任务、工具操作和每一次执行尝试需要各自的标识。同一份测试结果可能被传送两次；同一项测试也可能实际执行两次。前一种情况只需对消息去重，后一种情况已经消耗了两次资源。文件版本不相同时，两次同名测试甚至不是同一个工具操作。

记录每次尝试之后，还要区分尝试失败的原因。测试结果决定控制器的下一步。如果程序运行结果不正确，控制器就需要修复程序；如果工具进程因资源不足而中止，则需要恢复执行。两类失败对应不同的后续工作，完成概率也不同。同样的区分也出现在模型训练中。RL 根据样本的验证结果更新模型，其验证也要区分这两种情况：答案是否正确，用于计算学习反馈；执行故障则先由运行系统处理。

失败与重试还会带来额外的成本。一次提交可以触发多次尝试。平台为每次尝试支付模型、工具和环境成本，用户得到的是最后完成的任务。第 11.5 节将沿这些分支累积支出，再除以通过测试的任务数。

[4]本书 environment-resources 固定计算及工具子进程记录。工具实验包含九组、36 个进程，累计内存占用由采样 RSS 对时间积分得到。

## 11.2 运行环境的创建、保留与释放

### 11.2.1 工具执行路径与隔离边界

第 11.1.3 节区分了需要保留的文件、进程和操作结果，本节说明承载这些状态的环境如何创建和释放。模型输出的工具调用需要由普通程序执行。执行器将调用转换成文件操作、进程启动、浏览器动作或设备任务。运行环境提供代码执行所需的操作系统接口、文件系统、内存、网络以及资源限制。运行环境还需要隔离不同任务，防止相互干扰。

图 11-5 把章首平台分为模型服务和工具环境两条路径。模型服务由 10 个 V4-Flash 副本组成，最多同时处理 320 次调用；环境平台在一台 m5d.metal 的 48 个 CPU 核和约 358 GiB 内存中分配工具资源。控制器协调两者完成任务：先从模型取得参数，再让工具执行并返回结果。

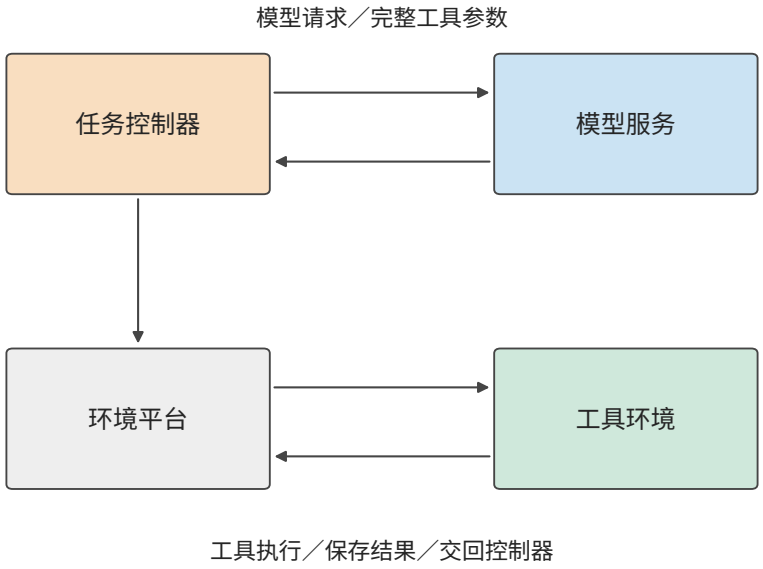



图 11-5：代码任务经过模型服务与工具环境两条路径。控制器从模型取得参数，再向环境平台发起工具操作；执行结果成为下一轮输入。虚线框标出工具环境所在的执行节点。

隔离不同任务有多种实现方式，隔离程度逐级增强。进程通常共享同一个操作系统内核，靠地址空间和权限做基本隔离。容器增加命名空间（让进程看到各自的进程编号、网络等资源视图）、文件系统视图和资源控制，但通常仍共享宿主内核。完整虚拟机运行自己的操作系统内核，并通过虚拟硬件与宿主隔离；microVM 保留虚拟机隔离模型，通过精简虚拟设备和管理功能来降低开销。沙箱是对执行约束的描述，不专指某一种实现。

章首平台选用的正是 microVM 方案。Firecracker 由亚马逊云服务（Amazon Web Services，AWS）为无服务器计算开发，2018 年开源；Firecracker 团队 2020 年在 NSDI 会议上发表的论文讨论了如何在大量短时任务之间兼顾硬件虚拟化隔离、启动速度和内存开销。[5] E2B 使用 Firecracker microVM 运行工具环境，模板可由容器镜像构建。创建时，平台从模板建立环境；运行时，工具通过虚拟机内核读写文件和启动进程。容器镜像决定环境中有哪些文件和依赖，microVM 则决定这些程序运行时如何与宿主系统隔离。[6]

隔离方式要随任务而定：隔离越强、系统功能越完整，启动就越慢，每个环境的资源开销也越大。DSec 为

[5] Agache 等，AWS，Firecracker: Lightweight Virtualization for Serverless Applications，NSDI 2020。

[6] E2B 固定架构、持久化文档（暂停约每 GiB 内存 4 秒、恢复约 1 秒，以及只保存文件系统的暂停）、暂停与快照接口行为核对。E2B 使用 Firecracker。

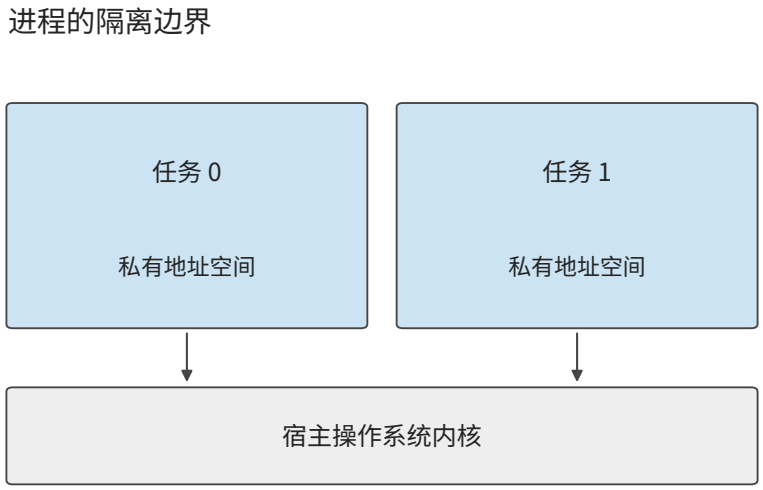



图 11-6：进程拥有独立地址空间，通过权限限制访问；同一节点上的进程共享宿主内核。

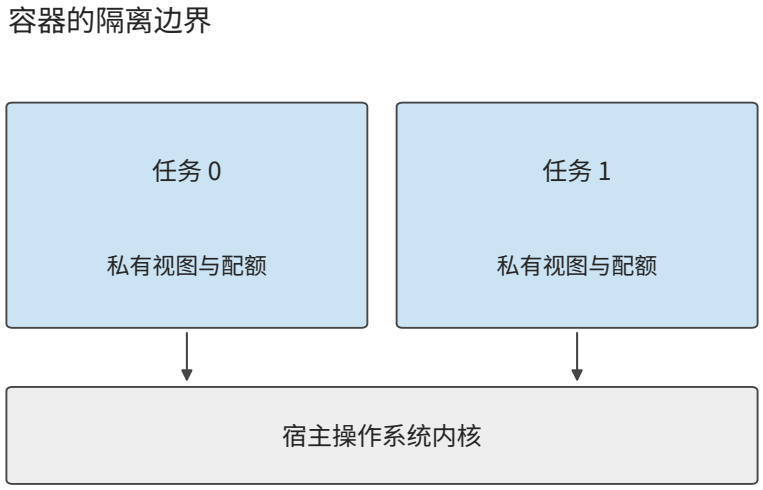



图 11-7：容器为任务建立各自的资源视图和配额，底层仍使用同一宿主内核。

此提供四类后端，由调用方按任务选用。判题、编译等短小的无状态任务，放在预先创建、反复使用的容器里执行，免去每次创建的开销；代码仓库级任务用容器，启动快、密度高；安全相关任务用 microVM；需要完整商用操作系统（如 Android）或图形渲染的任务用完整虚拟机。容器本身也跑在 QEMU 虚拟机里，因为容器共享宿主内核，外面再套一层虚拟机更安全。

隔离还要防范 agent 本身。RL 训练中的 agent 会主动找捷径。DSec 记录到的做法有：向沙箱内平台代理进程的 socket 发送伪造请求，翻查代理日志找残留的答案，改写 `/bin/bash` 以篡改后续命令。也有无意造成的破坏：从根目录递归执行 `grep`，读到 `/proc` 下的某个文件，触发内核缺陷，整台机器随即崩溃；执行 `yes` 命令，平台逐字记录其输出，结果攒下数十 GB。找捷径会让验证结果失真，意外破坏则会殃及同一节点上的其他任务。

相应的防护分三个方向（图 11-9）。沙箱与沙箱之间，靠虚拟化和资源配额隔开。沙箱内的 agent 与平台自己的代理进程之间，用 AppArmor（Linux 按程序限制文件与网络访问的安全模块）规定可以读写哪些文件和 socket，agent 即使以 root 身份运行也无法越权。沙箱与外部网络之间，用 eBPF（在内核中运行经过校验的小

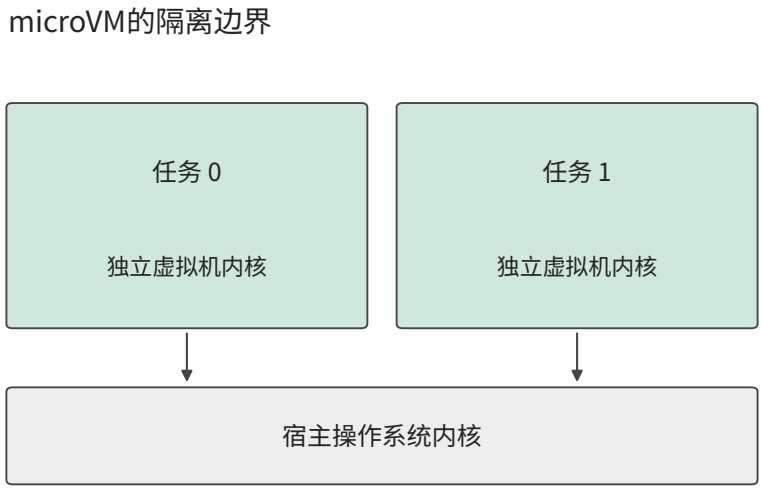



图 11-8：microVM 让每个任务环境运行独立虚拟机内核，通过虚拟硬件访问宿主资源。

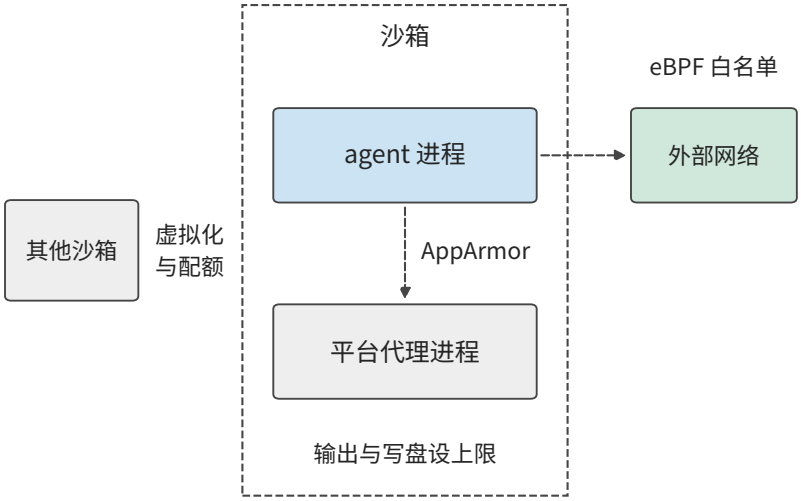



图 11-9：沙箱的三个隔离方向：虚拟化与配额把各沙箱隔开，AppArmor 限定 agent 能访问平台代理的哪些文件和 socket，eBPF 白名单过滤发往外部的流量。输出量和写盘量另设上限。

程序的机制）逐包检查地址、端口和协议，只放行本任务白名单里的软件源。这些手段只能堵住已经发现的捷径；单看最终输出，判断不了 agent 是否按预期完成了任务。[7]

两条路径在同一个任务控制器处汇合：控制器知道模型调用何时开始、工具何时使用环境，以及哪些文件需要跨轮保存。平台据此可以分别安排模型状态与工具状态的保留和释放：模型计算期间可以准备下一轮环境，工具完成后可以保存文件并释放进程。

[7]DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

### 11.2.2 资源分配与环境创建

确定隔离方式以后，平台还需为环境分配资源并加载程序所需的数据。工具平台用 CPU 配额、内存上限以及文件和设备访问权限分配资源。给等待模型的环境增加 CPU 配额，并不会改变其文件和内存状态；销毁该环境会释放内存，但下一次执行前要重建这些内容。有些工具还需要 GPU，例如验证 CUDA 程序时，还要向加速器资源池申请 GPU。设备直通让虚拟机直接操作分配给它的物理设备，可由虚拟机程序 QEMU 配合 Linux 的 VFIO 设备访问接口完成。[8]

创建环境需要取得模板、建立私有状态并准备首次工作所需的数据。最简单的方式是完整复制模板；另一种方式是共享只读内容，只复制需要写入的页；还可以按访问需求加载数据。创建环境时少加载数据，可以缩短启动时间，但程序首次访问这些数据时仍需等待加载。

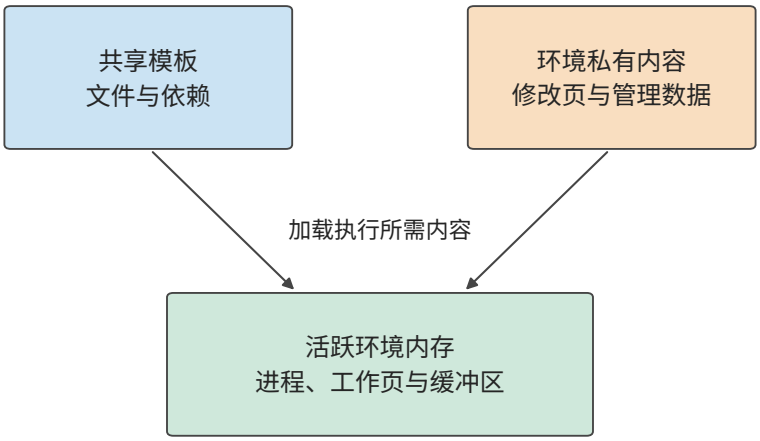



图 11-10：模板包含共享的文件与依赖，私有部分记录各环境的修改；程序运行时还会占用进程内存和缓冲区。加载模板所需的空间与活跃环境的内存占用分别计算。

**例 11-2：按需加载与只读共享能节省多少环境内存？**为分析创建开销，先取章首 300 个环境中的 100 个，比较将模板内容加载到这 100 个环境中所需的内存。每份模板为 2 GiB，工具访问其中 512 MiB，修改 128 MiB，另有 4 MiB 私有管理开销。逐份完整复制时，每个环境需占用 2,052 MiB 本地内存，100 份约为 200 GiB；只加载访问到的内容时，每份为 516 MiB，总量约为 50 GiB。同一组工具任务需要加载到内存的数据因此减少约 150 GiB。[9]

还可以让环境共享只读内容，仅保留 128 MiB 脏页（相对模板被修改过的内存页）和 4 MiB 管理开销。此时每份为 132 MiB，100 份约为 13 GiB。尚未加载到本地内存的只读页在访问时取得。若每个环境再保留 256 MiB 热点只读页，本地总量回升到约 38 GiB，换来较少的远端访问。

四种加载方式依次占用约 200、50、13 和 38 GiB，并另保留一份共享的 2 GiB 模板。章首为每个活跃环境分配的 2 GiB 内存，还包括进程、缓冲区等工具运行所需的空间。共享只读内容可以减少重复存储，将常用数据保留在本地则可以减少远端读取。具体保留多少，要看工具首次访问哪些页，以及此后多久再次访问。

图 11-11 展开每个环境本地保存的内容。少保存这些数据是否会使工具首次读取时等待更久，还需要计算。

[8] UB 操作系统参考设计中的设备虚拟化与交付路径，作为 QEMU / VFIO 的实现例证；这里描述支持设备直通的虚拟机配置。

[9] environment-lifecycle 固定预算区分共享 / 私有容量、串行传输的耗时下界和局部预热实测。本章使用固定容量预算比较四种内容加载方式。

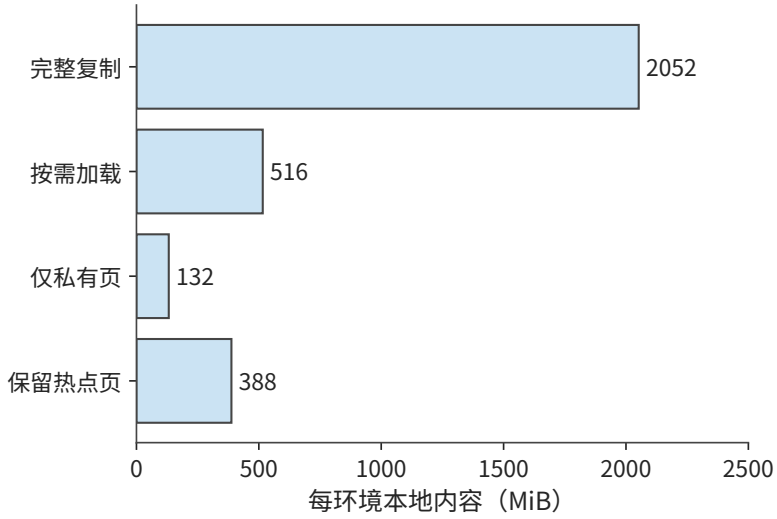



图 11-11：四种方案采用相同的横轴尺度，比较每个环境本地保存的数据。另有一份共享的 2 GiB 模板，不计入各环境的柱长。数值来自例 11-2；管理开销为每环境 4 MiB。

设取回与加载都受同一条 25 GbE 链路限制，速率为 3.125 GB/s（约 2.91 GiB/s）。模板尚未缓存在本机时，需要取回 2 GiB，再将 0.5 GiB 数据加载到内存，合计至少需要 0.86 秒；模板已在缓存中时，只需加载 0.5 GiB，降至 0.17 秒。约 0.69 秒的差额来自省去的模板传输。一般地，这条通路取回与加载数据的时间下界为 $(V_{\text{fetch}} + V_{\text{install}})/B$，其中 $V_{\text{fetch}}$ 和 $V_{\text{install}}$ 分别是取回和加载的数据量，$B$ 是链路带宽。

如果工具首次执行时又访问 1 GiB 尚未加载到内存的页，这条链路便再工作约 0.34 秒。按需加载让创建接口更早返回，尚未访问的数据留到真正需要时再取。E2B 的页与文件读取就是这样组织的：启动过程先提供可运行的环境，随后的访问逐步补齐内容。首次工具执行的完成时间包含环境启动，以及首次访问缺失页时的等待。按需加载将一部分准备工作推迟到了工具执行期间。[10]

实际负载中，工具读取的比例往往更低。DSec 抽查了 C++、Go、Java、JavaScript 和 Python 五类任务的容器镜像，每个 4–12 GB，运行时只读到其中 4%–13%，比例 11-2 假定的 25% 还低。整份复制既推迟启动，传输和写盘的大头又都花在根本不会读的数据上。提前拉取镜像也省不掉这部分工作：数据照样要传输、解压、写盘，只是提早做了。数千个容器同时启动时，按需加载完成整批任务的时间与镜像全部预先缓存在本地时差不多，先整份拉取镜像则要慢约 1.7 倍。[11]

到了虚拟机里，只读内容还多一层重复。microVM 经虚拟块设备读取模板时，宿主机的 **page cache**（内核用来缓存文件数据的内存页）存一份，每台虚拟机自己的 page cache 再存一份。同一节点上许多 microVM 读同一个基础镜像，内存里就堆着许多份相同的数据（图 11-12）。

DSec 让只读镜像层改走 virtio-pmem 设备并开启 DAX（direct access，直接访问）：虚拟机把文件页直接映射到宿主机的缓存页上，不再往自己的内存里复制，同一节点的 microVM 共用一份数据（图 11-13），内存峰值因此下降约四成。这样做也有代价。虚拟机仍须为这段地址范围分配页描述结构，约占设备容量的 1/64；首次访问要同步处理缺页，虚拟机内部的预读也用不上，CPU 峰值随之上升。可写的大磁盘仍走普通块设备，内存另用下面的办法回收。

[10] E2B 固定架构、持久化文档（暂停约每 GiB 内存 4 秒、恢复约 1 秒，以及只保存文件系统的暂停）、暂停与快照接口行为核对。E2B 使用 Firecracker。

[11] DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

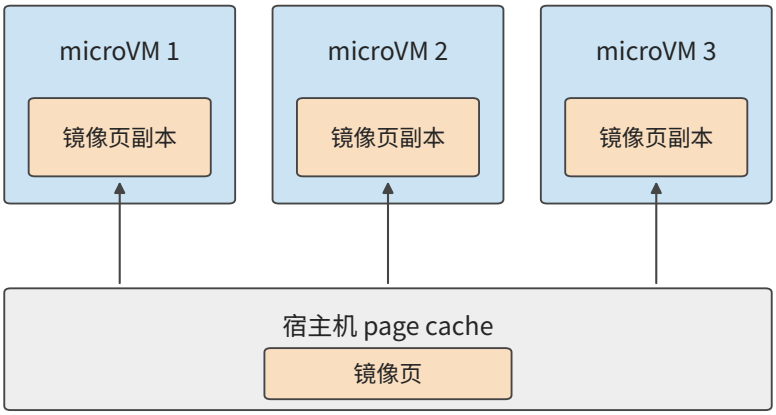



图 11-12：经虚拟块设备读取时，同一份只读镜像数据在宿主机和每台 microVM 的 page cache 里各存一份。

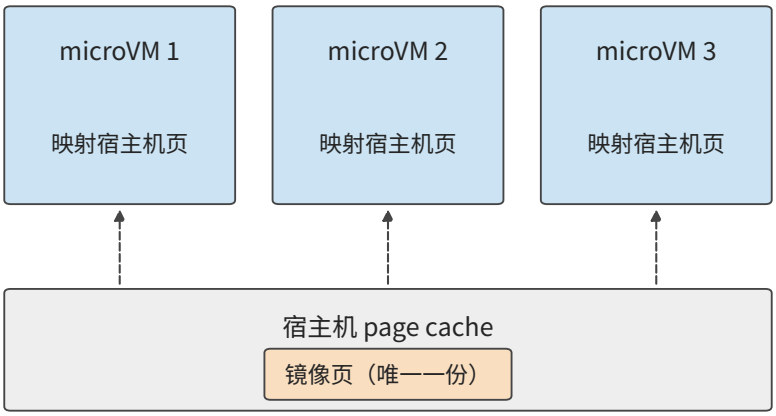



图 11-13：只读层改走 virtio-pmem 并开启 DAX 后，各 microVM 把文件页直接映射到宿主机缓存，内存中只留一份数据。

虚拟机内部释放了内存，宿主机未必收得回来。虚拟机里的程序退出后，内存页只是回到虚拟机自己的空闲链表，宿主机仍当作已分配。虚拟机申请的内存又常常多于实际所需，内部没有回收压力，读过一次的冷文件页就一直留在缓存里。DSec 的做法分两步：一是让虚拟机定期把成块的空闲内存报告给宿主机，由宿主机收回（virtio-balloon 的空闲页报告）；二是用 DAMON（Linux 通过采样统计内存访问情况的机制）找出长时间没被访问的文件页，主动回收，使零散的空闲页拼成可以报告的大块。两步合用，峰值几乎不变，整段运行的累计内存占用则减少约五分之一。

这两种机制正好对应第 11.1.2 节的两个量。峰值决定一台机器装不装得下这批环境，占用面积 $\int M(t)dt$ 决定平均要准备多少内存；共享只读页主要降低峰值，回收冷页主要缩小面积。[12]

[12] DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

### 11.2.3 暂停、恢复与环境重建

第 11.2.2 节减少了每次创建时复制的数据；另一种节省内存的办法是在两次工具执行之间暂时释放整个环境。在章首的三轮任务中，第一轮工具在第 10 秒完成，第二轮工具要到第 19 秒才开始，中间有 9 秒模型调用。始终驻留要占用 $2 \times 9 = 18$ GiB · 秒。按 E2B 文档，暂停一个沙箱时每 GiB 内存约需 4 秒，恢复约需 1 秒，保存和恢复期间环境仍占 2 GiB。暂停 2 GiB 环境需要 8 秒：第 10–18 秒保存状态，第 18–19 秒恢复。下一次工具仍能在第 19 秒开始，但整个间隔都被保存和恢复占满，占用仍是 18 GiB · 秒，没有任何节省。[13]

图 11-14 和图 11-15 的占用面积相同。要在等待期间释放内存，必须先保存下一轮仍需使用的状态，而保存时间随内存容量增长；因此，暂停能节省多少，取决于要保存什么、如何恢复。

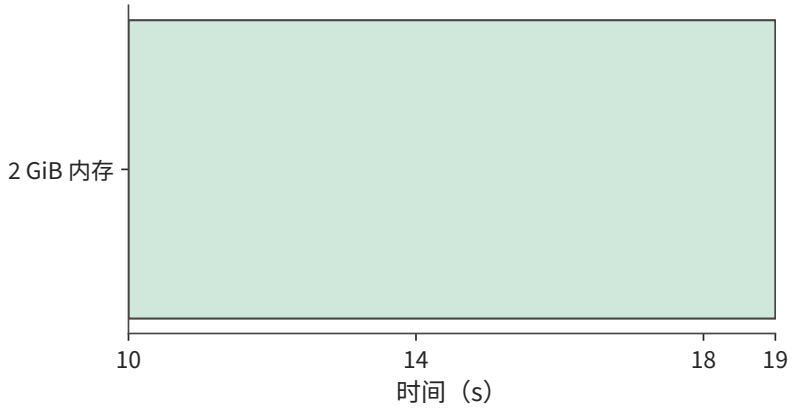



图 11-14：第 10–19 s 等待模型时始终保留 2 GiB 环境，占用 18 GiB・秒。横轴与下一图一致。

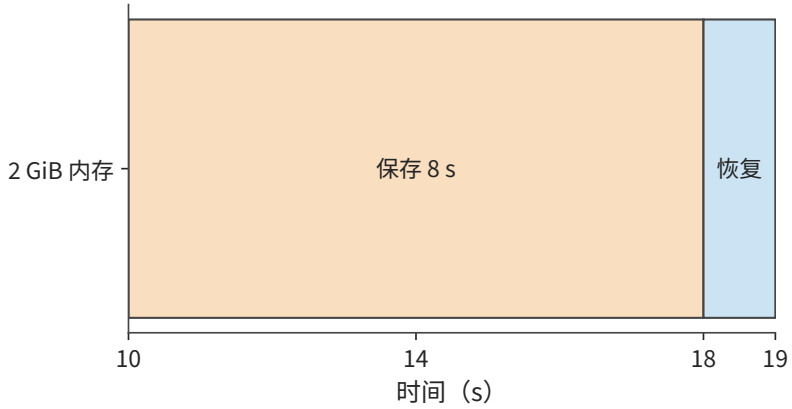



图 11-15：第 10–18 s 按每 GiB 4 s 保存 2 GiB 状态，第 18–19 s 恢复，整个间隔都没有释放内存。有色区间仍占 18 GiB・秒，下一次工具在第 19 s 开始。

恢复已保存状态的方式不止一种。暂停后恢复，是继续同一个任务的执行；快照派生，是从同一状态建立多个分支；从初始环境重新创建，则不继承先前任务的修改。三者都能重新得到可运行的环境，但继承的状态不同。

Agent 从运行快照恢复，会继承先前修改过的文件和进程。RL 验证常常要求每条样本从同一份初始目录出发，因此从干净模板派生。快照中的文件若包含上一次测试输出，恢复时也会带回这份输出。初始模板用于重复试验，运行快照用于延续进度，二者保存的是不同时间点的状态。

回到章首任务中那段 9 秒的模型等待。环境内存更小或间隔更长时，暂停才开始有收益：同样 9 秒的间隔，

[13] E2B 固定架构、持久化文档（暂停约每 GiB 内存 4 秒、恢复约 1 秒，以及只保存文件系统的暂停）、暂停与快照接口行为核对。E2B 使用 Firecracker。

512 MiB 的环境约 2 秒即可保存完毕，可以释放 6 秒；间隔短于保存与恢复之和时，恢复还会推迟下一次工具执行。E2B 另外提供只保存文件系统的暂停，恢复时重新启动系统；第 11.2.4 节的每轮重建环境正是沿这一方向，只保留文件，不保留进程内存。

是否暂停，可以比较成本来决定。设释放的内存为 $M$ GiB，释放后保持空闲的时间为 $G$ 秒，每 GiB · 秒价格为 $p_M$，状态保存、存储和恢复的新增成本为 $C_{\text{transition}}$，暂停节省成本的条件是 $p_M MG > C_{\text{transition}}$。左边随释放时间增长，右边是每次保存和恢复需要支付的成本；令两者相等，就能求出暂停开始节省成本所需的空闲时间。

章首任务的间隔只有 9 秒，上式左边几乎为零；RL 训练中还有长得多的间隔。第 10.4.4 节 MiMo 的 RL 运行，从中断到重启通常要一到三小时。GPU 训练作业被抢占或中断时，rollout 用的沙箱不能销毁，里面保存着未完成轨迹的文件和进程；但训练恢复之前，也不会有工具调用进来。DSec 上一个作业最多可以申请 32K 个沙箱，若每个按章首规格占 2 GiB，合计约 70 TB，超过 DSec 一个集群单元内存总量（约 250 TB）的四分之一。2 GiB 的沙箱保存加恢复仍只要 9 秒左右，释放出来的时间却以小时计：暂停两小时，内存空出来的时间约为保存和恢复期间照常占用时间的 800 倍。

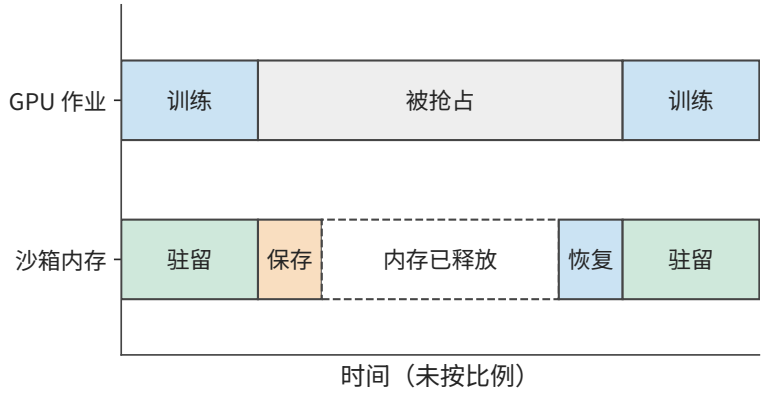



图 11-16：GPU 作业被抢占后，RL 框架暂停相应的沙箱。保存和恢复各占一小段时间，其余时间内存空出；横轴未按比例，抢占通常以小时计，保存和恢复以秒计。

所以在 DSec 上，作业一被抢占，RL 框架就主动暂停该作业的全部沙箱（图 11-16）。容器的做法是先冻结整个进程树，再放开内存换出到磁盘，并主动触发回收；恢复时先对进程的内存映射发出预取提示，再解冻运行。microVM 则把内存和执行状态写成快照，然后结束虚拟机进程；恢复时另起新进程载入快照。两种做法都是把状态从内存挪到下一级存储，用时再搬回来，所以代价随状态大小增长。任何发往已暂停沙箱的请求都会先把它恢复，控制器无须区分沙箱当前是否暂停。[14]

成本比较决定是否暂停；暂停之后，不同状态要用不同方式恢复。快照保存文件和内存，连接由客户端重新建立，外部存储则继续保留已经提交的写入。控制器恢复后先取回操作结果，再接续下一轮，避免把已经完成的修改重新执行一次。[15]

[14] DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

[15] E2B 固定架构、持久化文档（暂停约每 GiB 内存 4 秒、恢复约 1 秒，以及只保存文件系统的暂停）、暂停与快照接口行为核对。E2B 使用 Firecracker。

### 11.2.4 始终驻留、按需创建与提前准备

暂停需要保存进程状态。如果下一轮只需要已经保存的文件和结果，就可以进一步省去进程快照，直接重建环境。对章首代码任务，工作目录和结果已经持久化，每一轮工具都可以启动新进程。E2B 的模板本身就是预先启动过的虚拟机快照，文档给出恢复一个沙箱约需 1 秒。模板已缓存在本机时，从模板启动 microVM，再载入工作目录并初始化工具运行时，设完整准备时间为 2 秒，称为热路径；模板不在本机时，还要先经 25 GbE 取回 2 GiB 模板，增加约 0.69 秒，冷路径约需 2.7 秒。下文默认走热路径。每次准备累计使用 0.1 秒 CPU 时间，从准备开始到工具完成一直占 2 GiB。准备只加载已知的工具运行时和工作目录，文件修改在收到完整的工具调用参数后执行。

已知第一轮模型将在第 9 秒返回，平台可以在第 7 秒开始准备环境，第 9 秒执行工具，第 10 秒保存结果并销毁环境。后两轮分别在第 17 秒和第 27 秒开始准备。三次准备都在模型调用期间完成，任务仍在第 30 秒完成，环境只需在每轮准备和执行的 3 秒内保留。

每项任务的累计内存占用由 60 降到 $3 \times (2+1) \times 2 = 18$ GiB · 秒，平台平均内存占用由 600 降到 180 GiB。代价是每秒创建 30 个环境，准备工作平均额外占用 $30 \times 0.1 = 3$ 个 CPU 核，CPU 需求从 30 增到 33 核。Firecracker 论文报告单台主机每秒最多可创建 150 个 microVM，每秒 30 个仍在这一范围内。平台增加少量环境创建工作，就节省了原本要在等待期间保留的 420 GiB 状态，内存需求从这台主机无法容纳的 600 GiB 降到约一半容量。

图 11-17 和图 11-18 对照两种环境管理方式下的完整任务时间线，显示内存占用如何变化。工具的三个执行时刻没有改变，消失的是长段模型等待期间的内存占用。这里能准确安排准备时刻，是因为下一轮使用什么工具、模型何时返回都已给定。

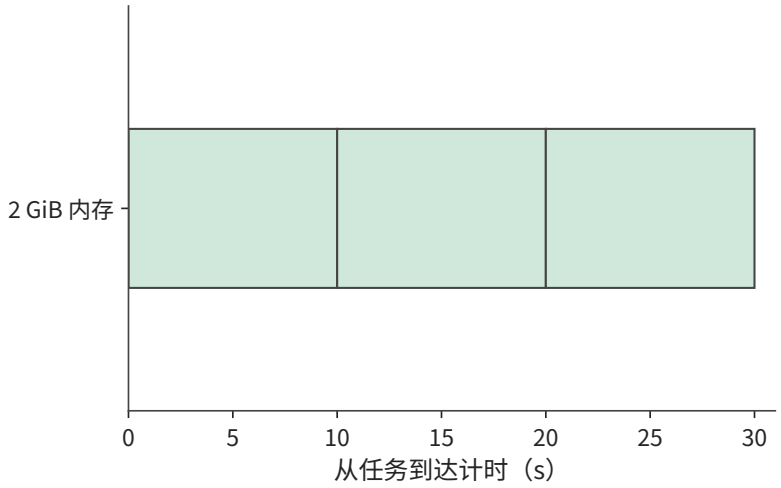



图 11-17：环境从任务开始保留到结束。模型每轮 9 s、工具每轮 1 s，三轮累计占用 60 GiB·秒。与下一图使用相同横轴。

这项安排的前提是准备能在工具执行前完成。若下一个工具的类型要从流式输出中预测，那么调用发出后还要等多久，就取决于准备提前了多少，以及预测是否正确。SpecBox 用流式输出中透露的工具类型和工具切换记录提前准备相应环境，下面按一次调用分析其收益。[16]

设一次准备需要 $P$ 秒，提前量为 $L$ 秒，正确预测所需环境的概率为 $h$。假设正确预测不会争用资源，错误预测可以立即取消，猜错时再按需准备，则发出调用后等待环境就绪的平均时间为

[16]调度、模型路由与云端环境研究，包含 ASI、RLBoost、DistRS、SpecBox。ASI 研究覆盖六个月、155,410 张 GPU，其分配比例度量设备归属。token 单价取自 Claude API 计价页快照。

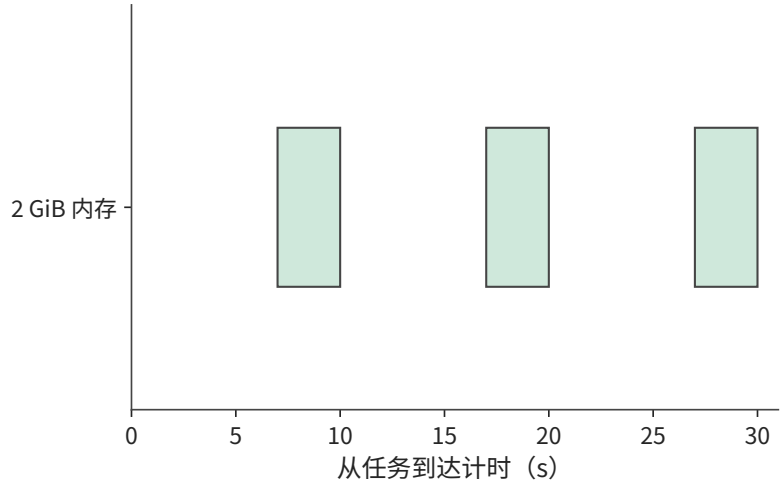



图 11-18：每轮只在准备 2 s 和执行 1 s 期间保留环境，三次合计 18 GiB·秒。工作目录和结果已持久化，三轮仍在第 30 s 完成。

$$E[W] = h\max(P-L,0) + (1-h)P = P - h\min(P,L).$$

正确预测时，准备的前 $\min(P,L)$ 秒与模型调用重叠，工具调用仍需等待剩余的准备工作完成；猜错时，实际需要的环境从头准备。两条路径按命中概率加权，就得到上式。当准备工作已经能在模型返回前完成时，再提早开始准备，只会延长环境就绪后等待调用的时间。

取 $P = 2$ 秒、$h = 0.75$。不提前准备时，每次调用等 2 秒。提前 1 秒时，四次调用平均有三次只等 1 秒，一次因猜错仍等 2 秒，平均等待为 1.25 秒。提前 2 秒时，正确分支不再等待，平均降至 0.5 秒。提前 3 秒仍是 0.5 秒，却让正确分支多保留 1 秒就绪状态。对 2 GiB 环境，每次预测因此增加 $0.75 \times 2 \times 1 = 1.5$ GiB · 秒的平均空闲内存占用。

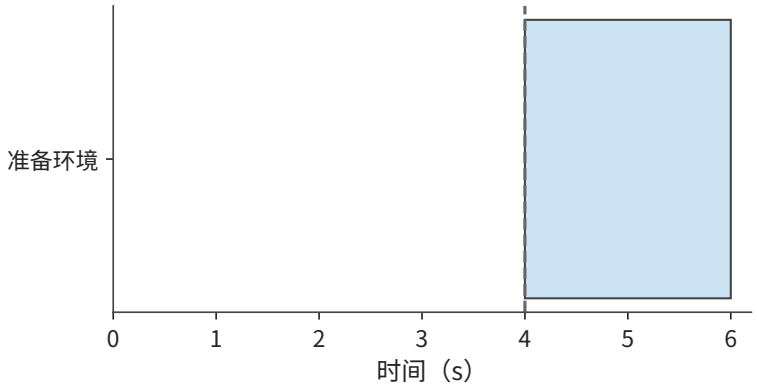



图 11-19：按需创建：第 4 s 调用到达才开始准备，第 6 s 就绪，等待 2 s。蓝色为准备，虚线为调用时刻；以下四图使用相同横轴。

由此可以比较三种减少等待的办法。缓存模板可以省去重复获取数据的开销；保留已经启动的环境，可以在调用到达后立即执行，但需要持续占用内存；每轮重新建立环境，则可以利用模型调用的时间完成准备。章首任务已经知道下一轮所需的运行时，因此在模型调用期间准备下一轮环境；多种工具交替出现时，还需在计算中计入预测命中率。

本地工具重放展示了过早准备的后果：从提前 50 ms 准备改为在模型等待一开始就准备后，累计工具调用

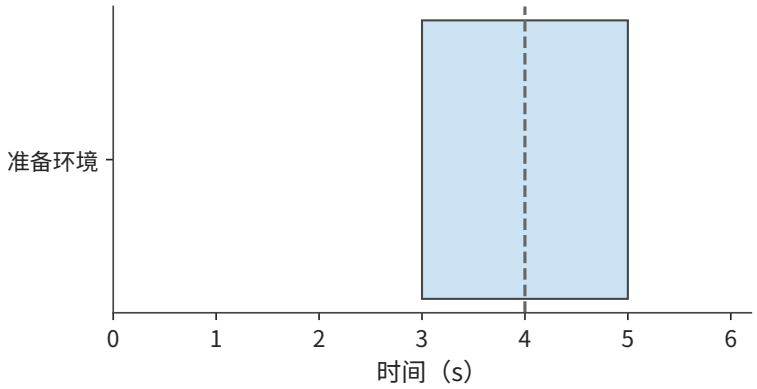



图 11-20：预测正确且提前 1 s：从第 3 s 准备到第 5 s，调用在第 4 s 到达后仍需等 1 s。蓝色为准备环境，虚线标出第 4 s 的调用到达时刻。

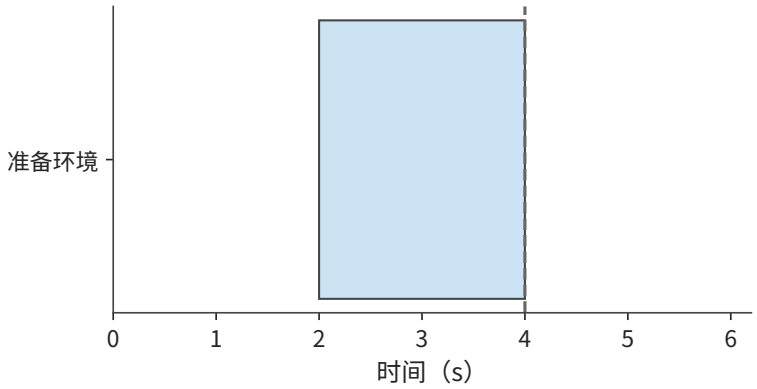



图 11-21：预测正确且提前 2 s：准备恰好在第 4 s 调用到达时完成。蓝色为准备环境，虚线标出第 4 s 的调用到达时刻。

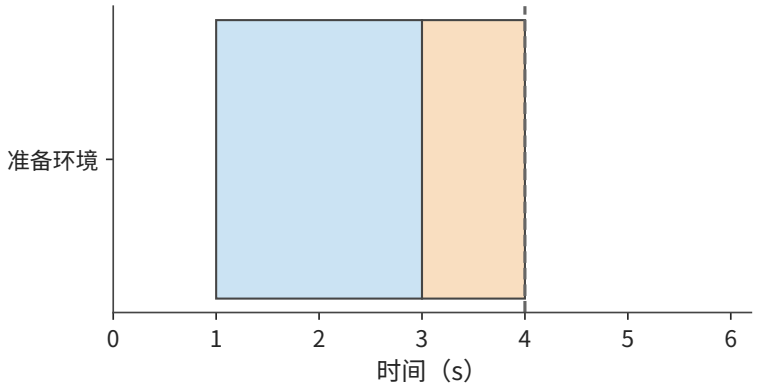



图 11-22：预测正确且提前 3 s：准备在第 3 s 完成，橙色部分表示环境已就绪、等待调用的 1 s 空闲时间。蓝色为准备环境，虚线标出第 4 s 的调用到达时刻。

时间仍约为 0.26–0.28 秒，按采样结果计算的累计内存占用却从约 0.0193 增到 0.467 GiB · 秒，约为原来的 24 倍。增加的内存占用主要发生在环境已经就绪、调用尚未到达的区间。只要能在工具调用到达前准备好环境，就没有必要更早创建。[17]

每轮重建能够成立，靠的是章首的题设：下一轮要用的状态已全部写入持久存储。许多 Agent 任务达不到这一点。模型会安装依赖、启动数据库或后台服务，后续调用要用到这些进程及其内存状态；只保存文件，重建出来的环境与原来的并不相同。DSec 的沙箱在整段交互中一直存在，走另一条路：环境保留，但把每个环境的

[17] 本地工具预热重放。实际调用、预测命中与采样窗口边界在原记录中保存。

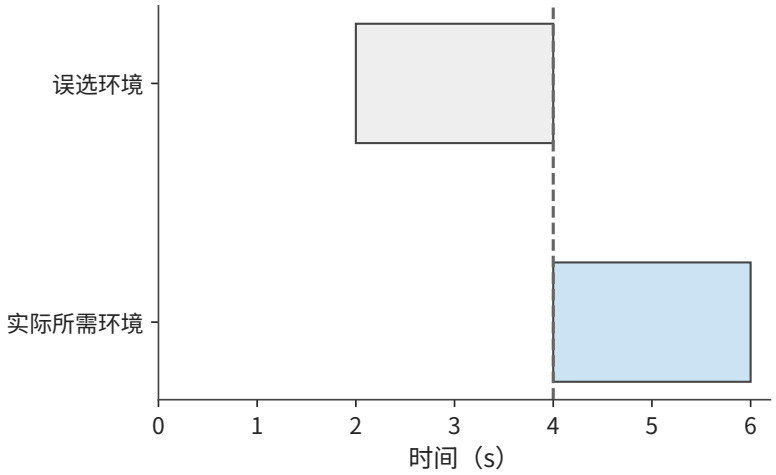



图 11-23：预测错误：第 2–4 s 准备了不需要的环境（灰色）；第 4 s 取消后，再花 2 s 准备实际所需环境。此图采用立即取消、无资源争用的题设。

驻留成本压下来，办法包括第 11.2.2 节的按需加载、只读页共享与冷页回收，以及第 11.2.3 节在长间隔中暂停。选哪条路，要看轮间状态能否完整写入持久存储：能写入，就花少量创建工作换回大量内存；写不进去，就只能保留环境，再设法压缩驻留。[18]

## 11.3 共享资源池的分配与调度

### 11.3.1 异构资源、成组分配与放置约束

算出单个任务的资源需求后，还要判断共享资源池能否同时提供这些资源。多卡作业通常需要特定卡型、每卡显存、配套 CPU 与主机内存，以及适合其通信模式的位置。空闲 GPU 总数满足要求，只是作业能够启动的必要条件之一。

这些资源还必须落在合适的节点上：章首 CPU 工具每次独立使用一个核，多卡训练则必须让一组加速器共同推进，作业中的所有进程才能一起启动。因此，多卡作业能否启动，还取决于空闲资源分布在哪些节点上。

**例 11-3：GPU、CPU 与节点约束如何造成资源碎片？**某作业需要四张 H100 和 16 个 CPU 核，并要求这些资源位于同一节点。节点 1 是一台 DGX H100（8 张 H100 SXM，双路 Xeon Platinum 8480C 共 112 核），已有作业占用了四张 H100 和 104 个核，只剩四张 H100 和 8 个空闲核；节点 2 是一台 DGX A100（8 张 A100，双路 EPYC 7742 共 128 核），空着四张 A100 和 32 个核。整个资源池既有四张空闲 H100，也有足够多的 CPU 核，但没有一个节点满足全部条件。[19]

假设等待节点 1 的原任务结束需要 12 秒，迁移该任务需要 4 秒，新作业在本地执行需要 20 秒。等待后执行共需 32 秒，迁移后执行共需 24 秒；重新安排资源后，新作业提前 8 秒完成。若另一种兼容的跨节点配置准备需 1 秒、执行需 32 秒，则立即分散启动共需 33 秒，反而更晚。三种方案的完成时间分别为

$$T_{\text{wait}} + T_{\text{run,local}}, \qquad T_{\text{move}} + T_{\text{run,local}}, \qquad T_{\text{ready,spread}} + T_{\text{run,spread}}.$$

[18] DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

[19] DGX H100 系统规格（双路 Xeon Platinum 8480C，共 112 核）；DGX A100 系统规格（附录 A 表 10，双路 EPYC 7742，共 128 核）。




图 11-24：整理前：新作业需要同节点四张 H100 与 16 核。节点 1 缺 CPU，节点 2 空着的是 A100。




图 11-25：将占用 8 核的 CPU 任务从节点 1 迁到节点 2 后，节点 1 同时拥有四张空闲 H100 和 16 个空闲核，可以启动新作业。

迁移方案在本例中比等待方案早 8 秒，比分散启动早 9 秒。随着状态增大，迁移耗时也会增长；达到 12 秒时，迁移方案与等待方案同为 32 秒。调度器因此可以把预测的迁移耗时直接与原任务剩余时间比较，选择更早备齐全部资源的方案。

资源碎片在生产集群中同样存在。香港科技大学、阿里巴巴等机构在 2026 年发表的研究分析了阿里巴巴无服务器基础设施（Alibaba Serverless Infrastructure，ASI）的生产 GPU 集群。研究表明，配套 CPU 不足、作业需要成组启动以及网络位置限制，都可能使空闲 GPU 无法使用。加入可抢占的低优先级作业后，研究中 GPU 分配比例从 68% 提高到 93%。这些低优先级作业用的是高优先级作业暂时用不到的加速器；当需要成组启动的作业到来时，调度器收回这些加速器，并通过迁移任务备齐所需资源。[20]

[20]调度、模型路由与云端环境研究，包含 ASI、RLBoost、DistRS、SpecBox。ASI 研究覆盖六个月、155,410 张 GPU，其分配比例度量设备归属。token 单价取自 Claude API 计价页快照。

### 11.3.2 队列、优先级与抢占代价

即使节点配置合适，请求同时到达也可能使资源来不及周转。章首任务均匀到达，每秒需要执行 30 次工具调用，48 个 CPU 核足以处理。当调用集中到达时，即使总工作量相同，排队时间也可能不同。

先考虑单个执行进程，每项工具工作需要 1 秒。十项工作在第 0、1、……、9 秒分别到达时，每项到达即可执行，排队时间都是零。若十项都在第 0 秒到达，执行仍需 10 秒，但各项的排队时间依次为 0、1、……、9 秒，平均为 4.5 秒。总工作量决定执行进程要工作多久，到达时刻和执行顺序决定每个任务先等多久。

图 11-26 和图 11-27 对照这两种到达方式，后者用执行块前方的灰色条形表示等待时间。同样 10 秒的执行工作，既可以没有排队，也可以产生大量等待。因此，从平均资源需求推到实际响应时间，还必须知道请求是否集中到达。

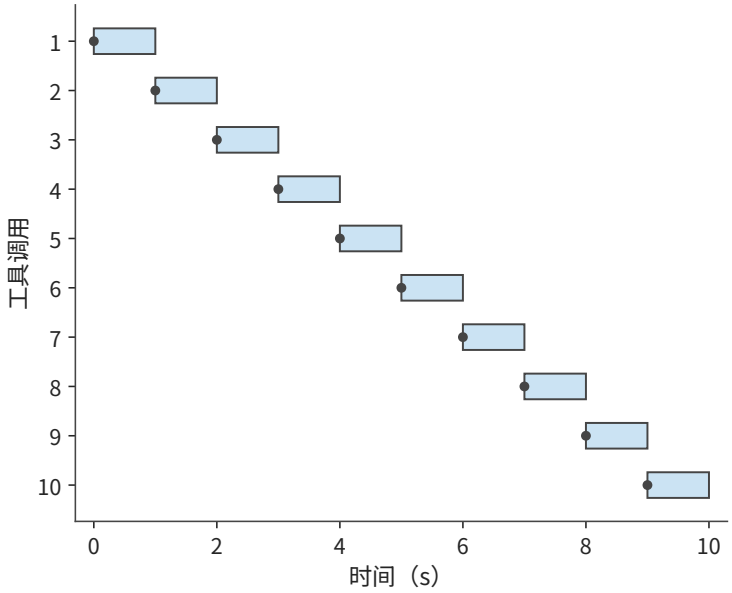



图 11-26：十次调用每隔 1 s 到达，一个执行进程每次处理 1 s。圆点为到达，蓝色为执行，每次均可立即开始。

以代码修复平台的突发工具调用为例。假设平台的 48 个核全部空闲，此时同时收到 80 次各需 1 秒的工具调用，前 48 项立即执行，其余 32 项等待 1 秒；平均等待 0.4 秒，全部在 2 秒内完成。若增至两台 m5d.metal（96 核），这次突发在 1 秒内完成。增加 CPU 核数提高了处理突发请求的能力；任务均匀到达时，平均仍只需 30 个 CPU 核执行工具。

同一批工具调用还需要足够的环境内存。每个环境占 2 GiB，80 个已准备环境需要 160 GiB。始终驻留的方案需要约 600 GiB，本身就超出了 358 GiB 的内存；第 11.2 节的每轮重建环境方案平均只占 180 GiB，剩下的约 178 GiB 足够容纳这 80 个环境。减少等待期间保留的环境，可以腾出内存给新调用，空闲的 CPU 也就能及时开始执行。

突发请求要分到多台节点上，还得决定每个环境放在哪里。DSec 的峰值创建速率超过每秒 5,000 个沙箱，平均到 160 个节点，每个节点每秒约 31 个，与第 11.2.4 节每秒创建 30 个环境的题设相当。调度器掌握的节点负载是定期汇总的，汇总之后新放的请求还没有反映进来；如果每次都挑当前最空闲的节点，同一时刻到达的一批请求就会一齐挤到同一个节点上。

DSec 用三项措施解决这个问题（图 11-28）。第一，每次只随机抽几个节点，从中挑负载最低的，即 power-of-k-choices，同时做出的决定自然就分散了。第二，每个调度实例把自己最近放下、汇总里还没有的沙箱补记到

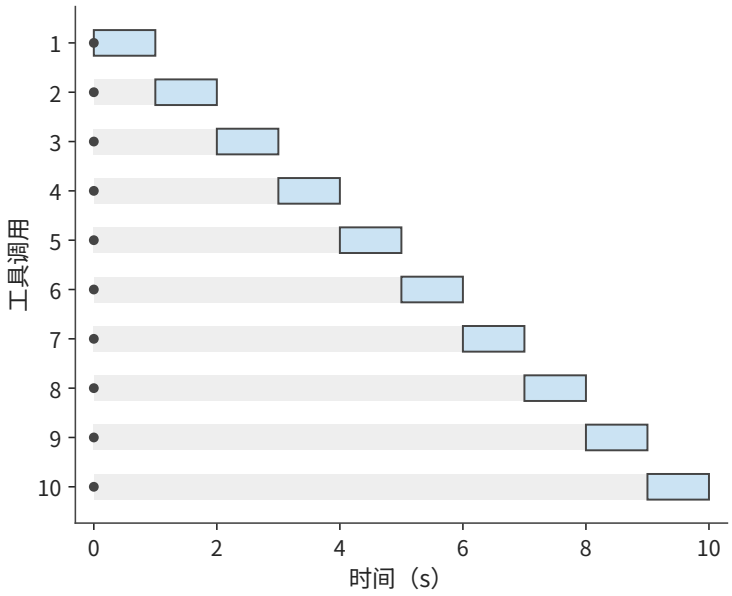



图 11-27：十次调用同时在第 0 s 到达；灰色为等待、蓝色为执行，各行的等待依次为 0–9 s。执行进程仍只工作 10 s，平均排队为 4.5 s。

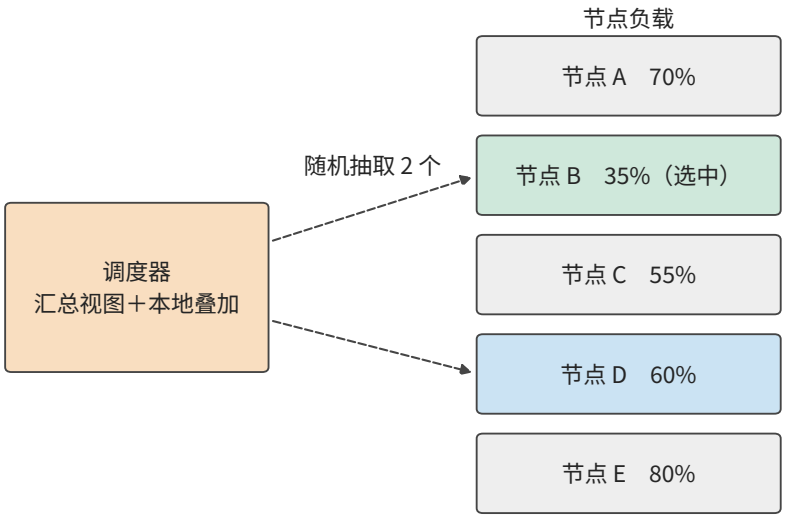



图 11-28：调度器在定期汇总的视图上补记自己刚放下的沙箱，每次随机抽几个节点，选负载较低的一个；节点检查本地容量，不够就拒绝，调度器再换一个节点。

本地视图上。第三，最终是否接收由节点决定，本地资源紧张就拒绝，调度器再换一个节点。全局视图可以有些过时，不超出容量由节点自己的检查来保证；调度器和负载汇总都不需要持久状态，实例可以随时增减或替换。[21]

[21] DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

为突发请求预留容量后，还要决定请求之间的先后顺序。优先级决定谁先使用这份余量。在线任务要求快速开始，离线任务可以等待，平台便可以让离线工作暂借资源，并在在线调用到达时收回。设在线任务最多等 2 秒，而离线作业保存状态需要 30 秒。这组资源即使可以抢占，也要在 30 秒后才交还；因此，平台需要另行预留能在 2 秒内分配给在线任务的资源。

设了优先级，硬件上不一定分得开。超卖使延迟敏感（latency-sensitive，LS）任务与可以等待的尽力而为（best-effort，BE）任务共用 CPU，例如每步都有时限的下棋 agent 与普通代码任务同在一个节点上。把 BE 任务放进 Linux 的 `SCHED_IDLE` 调度类，只要 LS 任务可以运行，BE 任务就让出 CPU。但开启同时多线程（simultaneous multithreading，SMT，即超线程）后，一个物理核有两个硬件线程，共用执行单元和一级缓存。调度器以硬件线程为单位分配，LS 任务和 BE 任务仍可能同时跑在同一物理核的两个硬件线程上（图 11-29）。

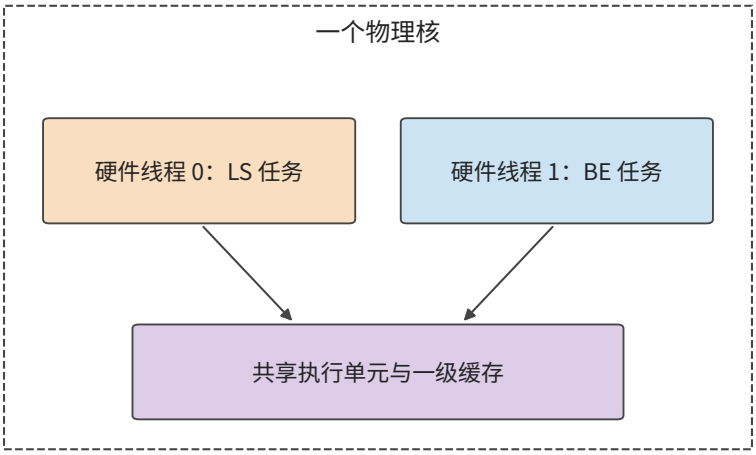



图 11-29：`SCHED_IDLE` 只管同一个硬件线程上谁先运行。LS 任务运行时，BE 任务仍可占着同一物理核的另一个硬件线程，与它争用执行单元。

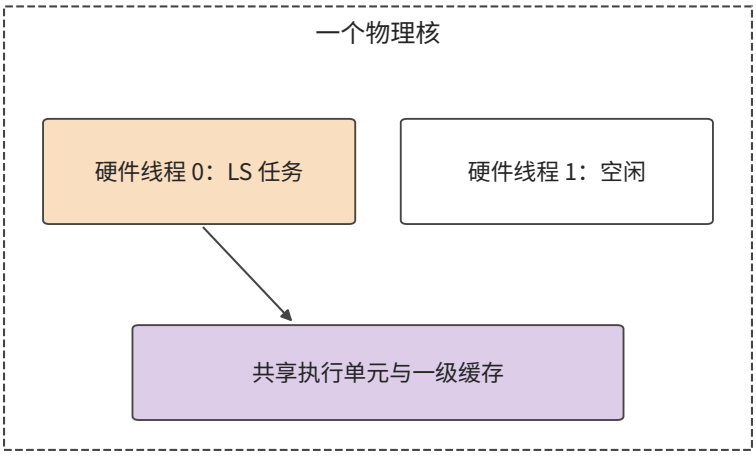



图 11-30：开启 core scheduling 后，同一物理核的两个硬件线程只运行同一组任务；LS 任务运行期间，另一个硬件线程不再跑 BE 任务。

按 DSec 的测量，BE 负载占到节点容量一半时，LS 任务每步耗时增加约 45%，单靠 `SCHED_IDLE` 几乎没有改善。再给 LS 任务开启 Linux 的 core scheduling，不让无关任务与它共用一个物理核（图 11-30），增幅降到约 17%。剩下的干扰来自整机高负载时的降频，以及共用的内存带宽和末级缓存，要隔离这些资源还需另外的机制。章首的 m5d.metal 关闭了超线程，每个核只有一个硬件线程，不存在这层干扰，代价是可调度的逻辑

CPU 少了一半。[22]

一次抢占还会丢失未保存的进度。已执行 20 秒、恢复需 2 秒的作业，从最近的 checkpoint 恢复执行与从头重做之间相差多少，取决于 checkpoint 之后积累的工作。调度器既要考虑新任务能提前多久开始，也要计入保存和恢复状态的开销，以及原作业因此增加的延迟。

抢占说明了让某项任务提前执行的代价。长期运行时，还需要避免同一批任务总被推迟，因此也要明确按什么标准衡量分配是否公平。两项任务各得两张 GPU，看起来份额相同；如果一种卡执行同样工作所需的时间是另一种卡的两倍，完成进度就不同。主导资源公平（dominant resource fairness，DRF）把每个任务在各类资源中占比最高的一类称为主导资源，并依据主导资源份额公平分配资源；Gavel 和 Pollux 等研究则进一步考虑不同加速器的处理能力或训练进度。按资源份额分配，关注的是各任务获得多少资源；按完成进度分配，关注的是各任务推进得多快；限制最长等待时间，则可以避免某些任务长期得不到执行。[23]

从单次工具调用扩大到 RL 训练，资源需求还会随阶段变化。rollout 生成的交互轨迹既包括模型输出，也包括工具或环境的反馈。下一节计算，增加生成这类轨迹的加速器，实际能缩短多少训练时间。

### 11.3.3 RL 生成与训练的动态资源配比

一轮同步的 RL 迭代需要生成样本、完成验证并执行更新。如果每轮所需有效生成量为 $Q$，生成池速率为 $r_g$，简化的生成阶段时间为 $Q/r_g$。增加 rollout 加速器只能缩短可并行的生成部分；每轮仍需完成验证、更新、权重发布和必要的同步。若这些阶段依次执行，则

$$T_{\text{iter}} = T_{\text{generate}} + T_{\text{verify}} + T_{\text{update}} + T_{\text{publish}}.$$

例如生成、验证、更新和发布分别需 40、10、30 和 5 秒，一轮共需 85 秒。若只把生成速率提高一倍，一轮降至 65 秒，整体仅加速约 1.3 倍；即使生成时间降为零，其余阶段仍需 45 秒，整体加速也不超过约 1.9 倍。随着生成时间缩短，验证和更新占总时间的比例逐渐增大，继续增加生成加速器所能节省的时间也越来越少。

图 11-31 中，增加生成加速器只缩短蓝色部分。下一步有两种办法：改变执行顺序，使验证与生成重叠；或者继续增加生成加速器，但先计算新加速器加入前的准备开销。

验证与生成重叠的办法见第 11.3.4 节。增加临时生成加速器时，RLBoost 保持训练组不变，让临时实例在完成权重准备后加入生成池。[24]

**例 11-4：共享的发送出口如何限制多实例的权重分发速度？** Qwen3-8B 的 BF16 完整权重共 16,381,470,720 字节，约 16.38 GB（十进制），要向六个 rollout 实例分别发送。按 RLBoost 的实验配置，训练组的八卡 H100 实例经 200 Gbit/s 前端网卡（接入数据中心通用网络、而非 GPU 间高速互连的网卡）发送，每个两卡 rollout 实例的前端接口为 50 Gbit/s，六路传输共用发送方的出口。向一个接收方传完权重至少需要

$$16.38/(50/8) \approx 2.62 \text{ s},$$

[22] DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

[23] DRF、Gavel、Pollux作为调度机制背景；资源共享与放置补充资源可用性和状态交付的区别。

[24] 权重准备与有效产出调研，记录 RLBoost 论文条件与 PolyRL 固定源码路径。Qwen3-8B 的权重字节数取自 safetensors 索引；200 / 50 Gbit/s 前端网卡与实例价格取自 RLBoost 论文。

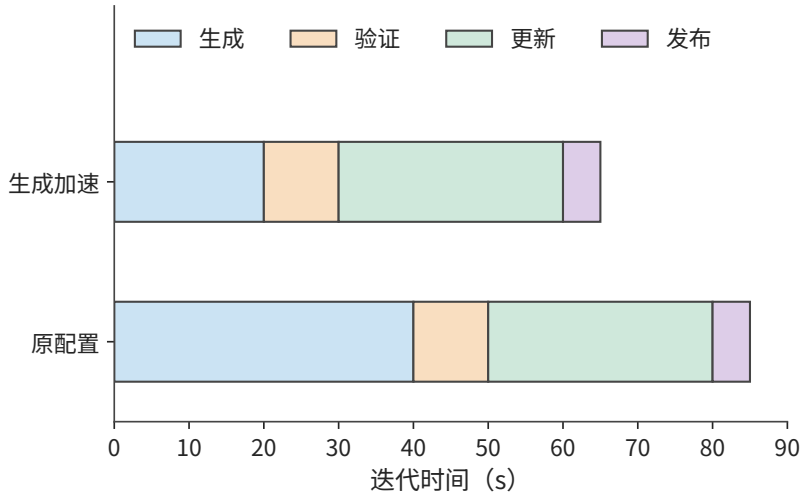



图 11-31：四个阶段依次执行。生成时间由 40 秒降至 20 秒，其余三段仍共需 45 秒，因此每轮总时间由 85 秒降至 65 秒。条形长度按时间比例绘制。

而共享出口要发送约 98.3 GB，向六个实例全部传完权重至少需要

$$T_{\text{all}} \geq \max\left(2.62, \frac{6 \times 16.38}{200/8}\right) \approx 3.93 \text{ s}.$$

六个实例全部收到权重，至少需要 3.93 秒；第一个实例可以更早收到自己的 16.38 GB。随后接收方把权重加载到 GPU 显存并确认版本，调度器再向它分配请求。PolyRL 利用临时资源扩展 RL 生成池，先把完整权重接收到 CPU 缓冲区，再交给采用 TP（见第 6.2.2 节）的 GPU 组。因此，TP=2 只是把实例内部的计算分到两张卡上，每个实例从外部接收的权重仍是 16.38 GB。

图 11-32 标出六份权重共同经过的出口。接收实例增加时，需要发送的总字节数也增加，发送方的带宽却没有随之增加。权重传完后还要加载到 GPU，这些准备时间都要从临时实例可用的时间中扣除。

临时实例分配到资源后，需要先完成准备，剩余时间才能用于生成。设实例从获得资源到开始生成共需 8 秒，可用时间为 20 秒，就只剩 12 秒用于生成；若只能使用 5 秒，则在开始工作之前已经被收回。令可用时间为 $L$、准备与恢复时间为 $T_0$，可工作的窗口就是 $\max(L - T_0, 0)$。实例可用的时间越长，准备开销所占的比例越小，用于实际生成的时间比例就越大。

临时实例提前被收回时，损失的不仅是尚未利用的容量，还有已经生成的内容。保存生成前缀可以把已完成工作带到新实例：新实例读入“原输入 + 已记录的输出”，执行 prefill、建立 KV 缓存，然后继续生成。PolyRL 恢复一组共享采样参数的请求时，会把组内各条响应截到其中最短的已保存长度：两条响应已收到 4,000 和 1,000 个 token 时，各采用 1,000 个 token，共复用 2,000 个 token，较长那条的 3,000 个 token 要重新生成。截到同一长度后，这组请求可以统一安排后续生成。

在一次 Qwen3-8B 4-bit 本地部署的对照中，保留前缀省去了 96 个重复采样 token，请求总耗时却约为 12.8 秒，而从头重做约为 11.6 秒。节省的 token 对应生成阶段，完整路径还包含前缀重建与重新调度。这组对照中，前缀重建与重新调度的额外时间超过了节省的生成时间。[25]

还可以从成本判断增加临时实例是否值得。按 RLBoost 使用的历史价格，训练实例每小时约 84 美元，六个额外实例各约 5.3 美元，总费率约为每小时 116 美元。全部实例全程计费时，处理同一批训练任务的吞吐量

[25] Qwen3-8B 抢占恢复实验，分别记录生成、客户端收到和恢复时采用的 token 数量。

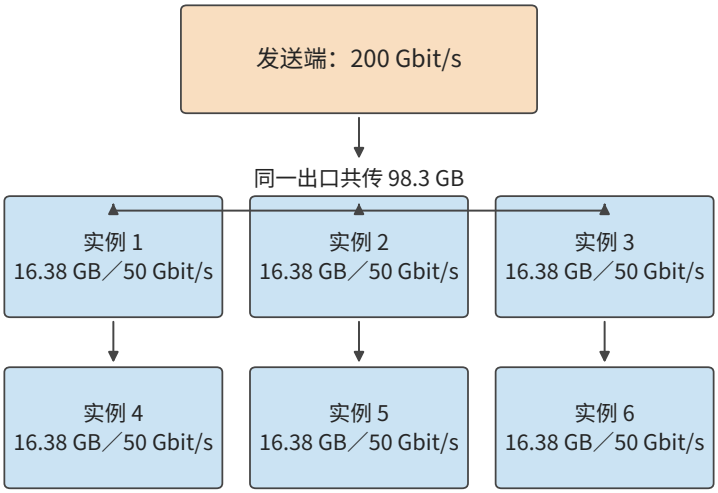



图 11-32：每个接收实例都需要一份完整的 16.38 GB Qwen3-8B 权重，六路传输共用 200 Gbit/s 发送出口。每个接收方为 50 Gbit/s，单份传输至少 2.62 秒，而发送方累计发送 98.3 GB 至少需要 3.93 秒；图中六个接收框各代表一个独立实例。

需达到原来的约 116/84 ≈ 1.38 倍，单位工作成本才下降。若吞吐量只达到原来的 1.2 倍，完成时间缩短，但成本约增至原来的 1.15 倍；达到原来的 1.6 倍，成本则约降至 0.86 倍。[26]

上面保存的是轨迹在模型一侧的进度。Agent rollout 在环境一侧也有状态：沙箱里改过的文件、启动的进程，以及 agent 循环（负责调用模型、下发工具调用、收集结果的控制器）记下的对话和工具结果。GPU 作业被抢占时，这两部分状态的下场并不一样。DSec 早期把 agent 循环同模型服务、RL 框架一起放在可抢占的 GPU 容器里（图 11-33）。作业一被抢占，沙箱还在，agent 循环的状态却随容器一起丢了。恢复时，训练框架先还原自己记录的 rollout 进度，再照命令日志重放：执行过的工具调用直接返回记录下来的结果，不再真正执行，免得同一条命令做两遍，例如往文件里多追加一次内容，或者向外部系统重复提交请求。第 11.1.3 节状态表中“工具结果”一行说的就是这种恢复方式。

从 DeepSeek V4.1 起，agent 循环改在 DSec 的 worker 容器中运行，不再占用可抢占的 GPU 资源，由它和沙箱一起保存完整的 rollout 状态（图 11-34）。GPU 作业恢复后重新连上即可接着做，训练框架也不必再实现重放。这一改动实际上是换了状态的存放位置：状态放在存活时间最长的资源上，存活时间短的资源失效，状态也不会跟着丢。经常被抢占的 GPU 只做丢了可以重算的计算，要跨过中断保留下来的轨迹状态，交给不会被抢占的 CPU 平台。[27]

### 11.3.4 验证 batch、长尾与资源释放

第 11.3.3 节把验证视为生成之后的一整段工作；实际上，样本往往陆续生成，验证可以更早开始；训练更新需要等待的，则是整批验证中的最后一个结果。验证 CUDA 代码时，需要先用 CPU 编译，再用 GPU 执行。安排验证任务时，除了 CPU 容量，还要考虑并发执行对 GPU 性能测量的干扰。DSec 为这类验证单独设

[26] 权重准备与有效产出调研，记录 RLBoost 论文条件与 PolyRL 固定源码路径。Qwen3-8B 的权重字节数取自 safetensors 索引；200 / 50 Gbit/s 前端网卡与实例价格取自 RLBoost 论文。

[27] DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

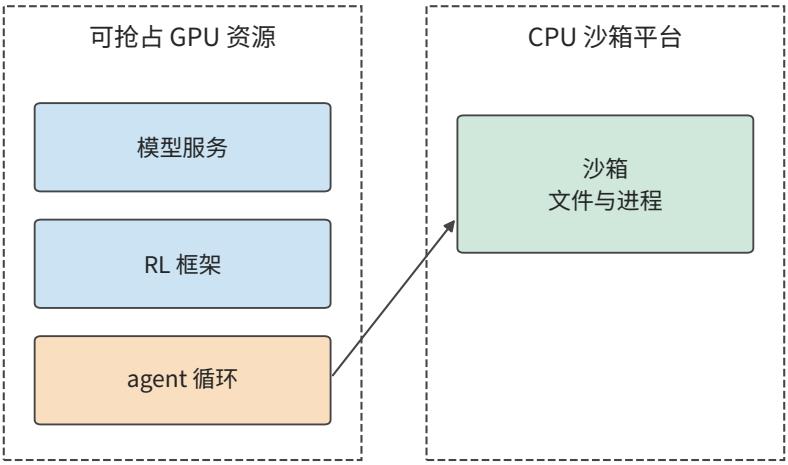



图 11-33：agent 循环与模型服务、RL 框架同在可抢占的 GPU 容器里。一旦抢占，agent 循环的状态随之丢失，沙箱里的文件和进程还在，恢复时靠命令日志把两边重新对上。

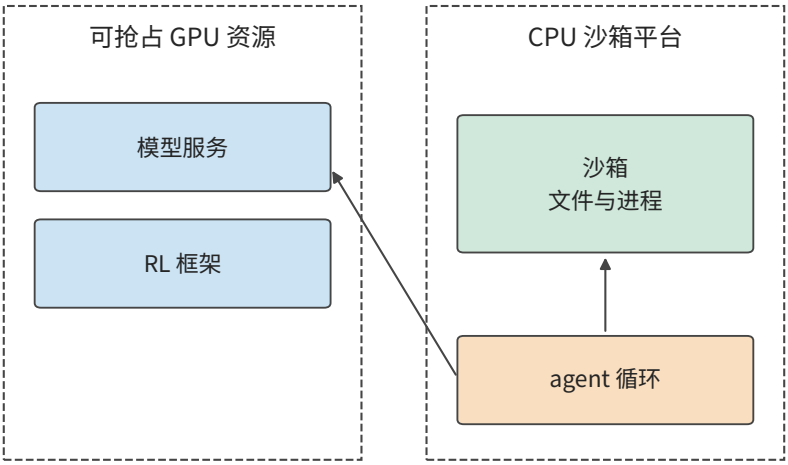



图 11-34：agent 循环移到 CPU 平台的 worker 容器里，和沙箱一起保存完整的 rollout 状态；GPU 作业恢复后重新连上即可接着做。

了 GPU 执行后端，做法有三：用 MIG 把一张 GPU 分成相互隔离的几个实例，几项测量同时进行也互不影响；先在 CPU 上编译，再把编译产物交给 GPU 执行，编译时不占 GPU；预备一批已完成初始化、已导入所需库的 Python 进程，请求一到就直接执行算子。这三项都是为了让 GPU 少花时间在测量以外的工作上。[28]

若样本 $i$ 在 $a_i$ 时刻到达，依次需要 $c_i$ 秒编译和 $g_i$ 秒执行，忽略资源竞争时，整批完成下界为

[28] DeepSeek-AI 与清华大学，DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale，arXiv 2609.22978，2026（PDF）。负载统计见 §2.4、§4；后端与隔离见 §2.2、§3.3、§6.4–6.5；按需加载、内存共享与回收、CPU QoS 见 §5.2–5.3、§8.2、§8.4–8.5；放置与 GPU 验证后端见 §7；暂停与 agent 循环的归属见 §6.2–6.3。沙箱数量、每节点创建速率、暂停收益等换算见案例笔记。

$$T_{\text{batch}} \geq \max_i (a_i + c_i + g_i).$$

执行进程不足会导致排队，启动进程和传输数据也需要时间，这些开销都会进一步推迟整批完成的时刻。尽早提交已经生成的样本，就能让验证与后续生成重叠。图 11-35 和图 11-36 中的两种安排都需要 30 秒的验证处理时间，但完成时刻相差 20 秒；差别只在提交时机。

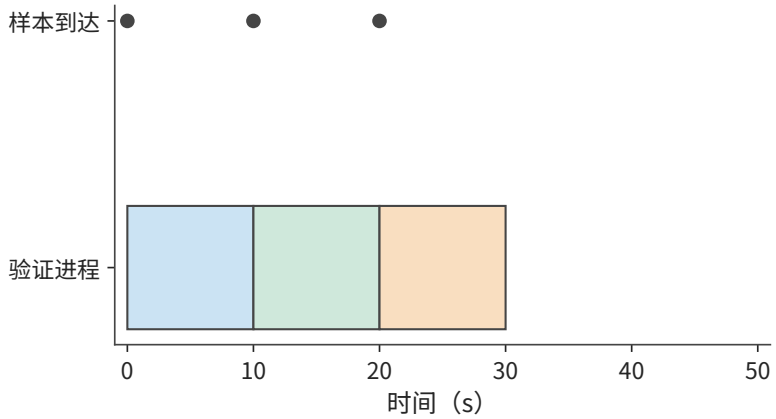



图 11-35：三条样本分别在第 0、10、20 s 到达，立即提交给唯一的验证进程，每条处理 10 s，整批在第 30 s 完成。圆点标到达时刻。

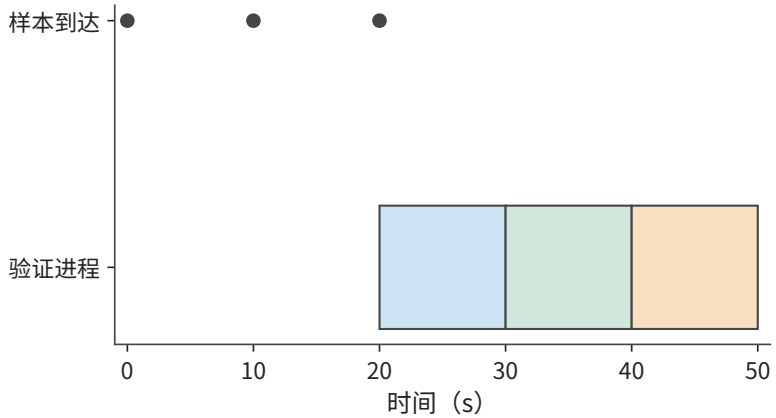



图 11-36：先等三条样本全部在第 20 s 就绪，再依次验证，结束于第 50 s。与前图执行相同的 30 s 验证工作，区别来自提交时机。

逐条提交消除了等待整批生成的空档，但整批完成时间仍可能由最慢的一条验证决定。估计这类长尾任务还要运行多久，需要根据它已经运行的时间更新判断。设已有测量中九条验证各需 1 秒，一条需 100 秒，平均为 10.9 秒。若某条验证已经执行 10 秒仍未结束，用 10.9 减去 10，就会得出只剩 0.9 秒的预测。然而在这一离散分布中，该验证必定是那条 100 秒任务，还需 90 秒。因此，应在已知任务尚未结束的条件下估计剩余时间，即 $E[S - e \mid S > e]$，其中 $S$ 是一项验证的总执行时间，$e$ 是已经执行的时间，$E[\cdot \mid S > e]$ 表示只在尚未结束的样本中取平均。DistRS 的调度算法采用这一类条件剩余时间，而不是始终使用总体平均数。[29]

估计出各条样本的剩余时间后，便可以据此安排其他验证，尽量不延长整批的等待时间。假设 batch 中有一条样本最早要到第 120 秒才能完成。若要安排一项执行时间不超过 100 秒的工作，该工作最迟在第 20 秒启动，便能利用前面的空隙而不推迟整批；若其环境准备需要 5 秒，准备最迟在第 15 秒启动。由整批的完成时刻

[29] DistRS 验证调度与 verl 实现分析。

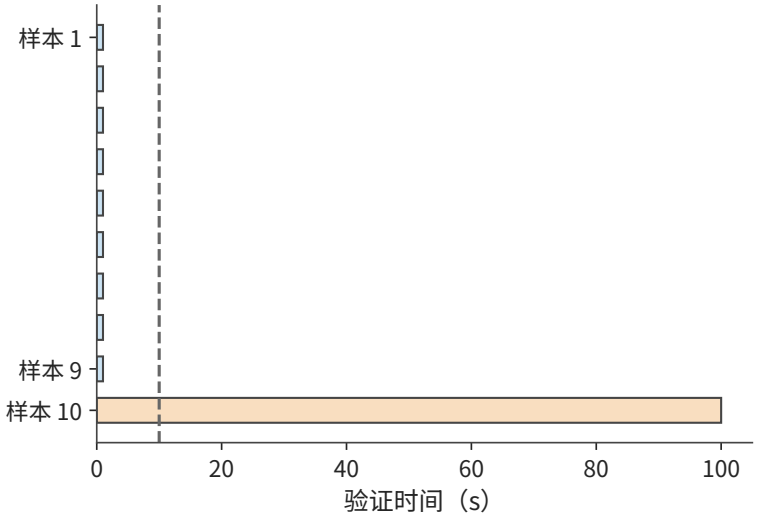



图 11-37：九条验证各用 1 s，一条用 100 s。虚线是已经执行的 10 s；到此时仍未结束的样本只可能来自 100 s 那一类，剩余时间为 90 s。

可以反推出各项工作的最迟开始时间，调度器据此推迟不紧迫的工作，把资源先交给更早到期的 batch。

推迟不紧迫的验证可以减少验证资源占用，但一旦推迟了整批完成，也会让训练组继续等待。假设验证少用 60 H100 · 秒，却使仍持有 64 张 H100（8 台 HGX H100）的训练组多等 2 秒。训练组新增 $64 \times 2 = 128$ H100 · 秒，整个系统净增 68 H100 · 秒。若训练组只有 16 张卡（2 台），新增等待为 32 H100 · 秒，系统反而净省 28 H100 · 秒。验证资源是否值得缩减，因而取决于被阻塞的训练组有多大。

安排好完成时刻以后，还要确认结束或超时的任务确实停止了执行，资源才能交给下一项任务。一次受控验证中，等待方返回超时后，后台线程仍工作约 130 ms；主动终止子进程后，约 0.8 ms 就观察到进程退出。如果超时一到就把同一资源交给下一项任务，只报告超时而不终止后台线程，两项任务就会同时使用资源。对于 CUDA 性能验证，这种竞争还可能改变测得的执行时间和奖励。[30]

## 11.4 模型服务的选择与调用成本

### 11.4.1 任务质量与思考预算

每轮重建环境已经把章首平台内存需求从 600 GiB 降到 180 GiB，但任务完成时间仍为 30 秒，超过 24 秒目标。因此要改变关键路径上的模型调用。把每个 V4-Flash 副本同时服务的会话数从 32 减到 16，每个输出 token 的时间由 25.3 ms 降到 16.9 ms，355 个 token 的完整调用由 9 秒降到 6 秒，三轮便从 30 秒降到 21 秒；模型未变，任务通过率也不变。若环境始终驻留，每项占用也从 60 降到 42 GiB · 秒，整个平台的内存需求从 600 降到 420 GiB，仍超过 358 GiB；如果每轮重建环境，仍只需在各轮准备和执行期间保留环境，累计占用保持在 18 GiB · 秒。模型服务的选择因此会同时影响任务耗时和环境内存需求。[31]

模型选择还决定每次调用的成本构成。一次模型调用的时间和成本，包括处理输入、生成思考 token 和生成

[30] 超时与资源释放实验、双 batch 调度记录。实验使用受控 CPU 进程。

[31] 贯穿平台与例 11-9 的输入、逐阶段时间表、成本、成本相等的条件与复算见贯穿设计数据及验证程序。模型调用时间取自本书对 4 张 B200 运行 DeepSeek V4-Flash、每会话 200K 上下文的估算：每副本 32 个会话时每个输出 token 25.3 ms，16 个会话时 16.9 ms，均含分摊的输入处理与上下文重建（比较结果、完整数据）；355 个 token 的调用分别约 8.99 s 和 6.01 s，设计取 9 s 和 6 s。B200 每卡时 6.79 美元取自 Runpod 价格页快照。m5d.metal 的 48 核（关闭超线程）与 384 GB 内存取自 Firecracker 论文第 5 节的评测环境；1 vCPU、2 GiB 规格与 CPU、内存单价取自 E2B 计价页快照。热路径准备共 2 秒、0.1 CPU · 秒与 95% 验收通过率为题设，结果保存与工作状态清理包含在工具的 1 秒内；模板容量算例单独说明内容加载。

可见输出三部分。思考预算决定允许生成多长的内部推理，实际用量决定每次调用需要支付多少费用。先在输出都能通过测试的条件下，比较两种思考长度设置的单次调用成本，再比较两种服务完成三轮任务所需的成本。

**例 11-5：思考长度缩短到十分之一，总成本减少多少？**设两种思考长度设置都能通过相同测试，每次输入 10,000 个 token，可见输出 200 个 token，实际思考分别为 1,000 和 100 个 token。价格取 Claude Sonnet 5 的标准 API 价格：输入每百万 token 2 美元，生成每百万 token 10 美元，思考 token 按生成计费。那么单次成本（美元）分别是

$$C_{1000} = \frac{10000 \times 2 + (1000 + 200) \times 10}{10^6} = 0.032,$$

$$C_{100} = \frac{10000 \times 2 + (100 + 200) \times 10}{10^6} = 0.023.$$

思考 token 数降为原来的十分之一，总成本却只减少约 28%。原有成本中的 0.020 用于输入，0.002 用于可见输出，这两项保持不变；下降的是从 0.010 变为 0.001 的思考成本。因而，思考占原成本的比例决定了压缩思考能节省多少。[32]

图 11-38 将节省发生的位置单独标出。这一比较假定两种设置都能完成任务；如果缩短思考后需要重试，节省的橙色部分就要与重试新增的整次调用成本一起比较。

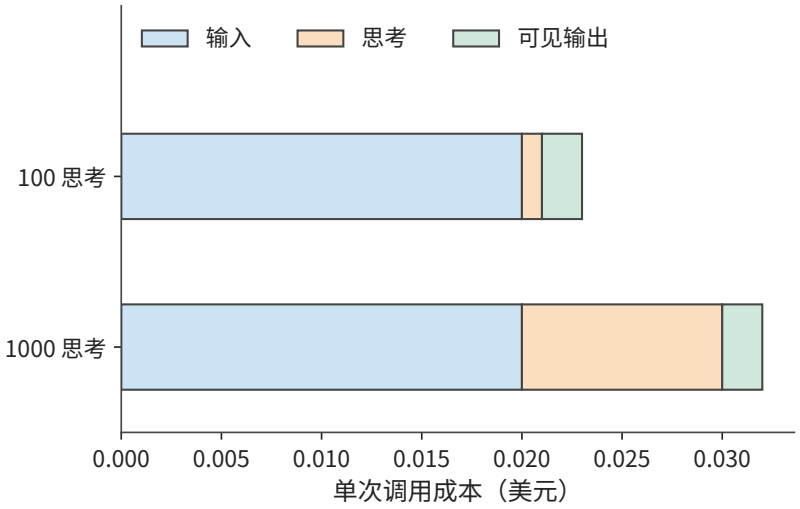



图 11-38：两次调用的输入和可见输出相同，只改变思考长度。橙色成本缩短到原来的十分之一，蓝色和绿色成本保持不变，所以总成本由 0.032 美元降到 0.023 美元，减少约 28%。价格和 token 数见例 11-5。

假设 100 次调用各省 0.009 美元，共节省 0.9 美元；其中 30 次因预算截断而重试，每次重试花费 0.032 美元，就新增 0.96 美元。总成本反而多出 0.06 美元。缩短思考预算可能增加重试次数。因此，比较任务总成本时，要同时计入单次调用的节省和额外重试的开销。[33]

[32] 调度、模型路由与云端环境研究，包含 ASI、RLBoost、DistRS、SpecBox。ASI 研究覆盖六个月、155,410 张 GPU，其分配比例度量设备归属。token 单价取自 Claude API 计价页快照。

[33] 思考预算与质量记录、增加思考预算后的检查。记录分别检查生成是否自然结束，以及输入状态是否满足要求。

### 11.4.2 服务路径、前缀缓存与实际用量

第 11.4.1 节靠减少生成量降低成本；输入一侧的主要节省机会是重复前缀：同一任务的多轮调用通常会重复发送公共提示和此前的对话记录。一项模型请求从控制器进入统一服务入口，再交给选定的模型后端。这里的后端指实际执行模型推理的推理实例或服务提供方。入口负责鉴权、路由和限流，后端负责输入处理与输出生成。章首每秒 30 次调用在这条路径上形成持续需求；每次调用携带的公共提示与任务记录，则决定其中多少输入可以复用。

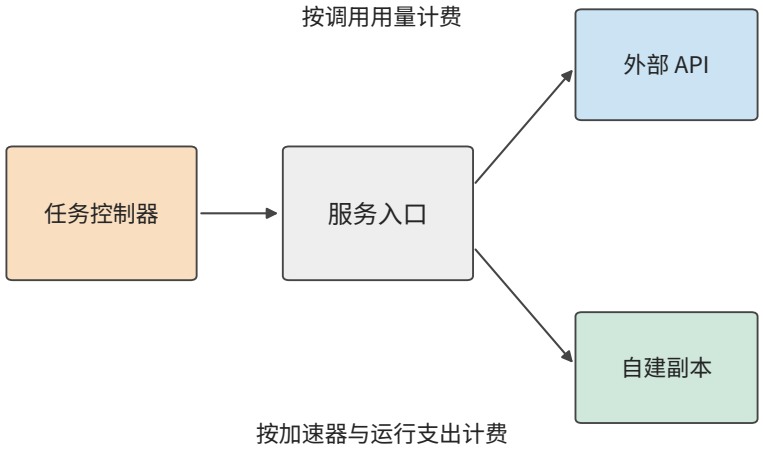



图 11-39：同一任务使用模型服务的两种方式及其计费方法。外部 API 按调用用量收费，自建副本按设备预留及运行支出计价；两条路径都将成本归到任务及尝试。输入拆分为普通处理、缓存创建和缓存读取，生成单独计量。

将一次调用的输入拆为互不重叠的普通输入 $I_u$、缓存创建输入 $I_w$ 和缓存读取输入 $I_h$，计费的生成 token 数为 $O$，对应的每百万 token 价格为 $p_u, p_w, p_h, p_o$，则

$$C_{\text{call}} = \frac{I_u p_u + I_w p_w + I_h p_h + O p_o}{10^6} + C_{\text{storage}} + C_{\text{tool}}.$$

同一个输入 token 在一次调用中只属于普通输入、缓存创建或缓存读取中的一类。将服务返回的用量记录（usage）按这三类整理后，分别乘以对应单价再相加，就得到输入成本。生成 token 数 $O$ 则是思考 token 数与可见输出 token 数之和。

**例 11-6：跨轮缓存公共前缀能节省多少输入费用？** 每轮有 8,000 个 token 公共前缀和 2,000 个 token 不复用的新输入。按 Claude Sonnet 5 的标准价格，普通输入每百万 token 2 美元，5 分钟缓存的首次创建 2.5 美元，缓存读取 0.2 美元。第一次创建，后九次在有效期内全部命中。无缓存时十轮输入成本为 0.20 美元；有缓存时（美元）为

$$\frac{8000 \times 2.5 + 2000 \times 2}{10^6} + 9\frac{8000 \times 0.2 + 2000 \times 2}{10^6} = 0.0744.$$

本例保持十轮生成和工具执行相同，比较的成本差额全部来自输入。第一次使用缓存花费 0.024，比普通处理的 0.020 多出 0.004；此后每轮只花 0.0056，比普通处理少 0.0144。因此第二次访问就已收回首次创建的溢价。共访问 $n$ 次时，使用缓存更省的条件是 $np_u > p_w + (n-1)p_h$。缓存有效期越长，重复读取就越有机会收

回首次创建的额外成本。[34]

缓存节省多少，还取决于命中的是哪些请求。再看两次调用，一次前缀长 1,000 个 token，一次长 9,000 个 token。若只有短前缀命中，请求命中率为 50%，缓存读取的 token 数却只占两次调用前缀总长度的 10%；若只有长前缀命中，请求命中率仍为 50%，读取占比变成 90%。按 token 收费时，后者节省的输入处理费用远多于前者。每次命中的权重，由输入长度决定。

### 11.4.3 排队、限流与模型路由

缓存只有在请求到达保存了相应前缀的后端时才能复用。因此，选择后端既影响命中率，也影响排队时间。为了使用已有的前缀缓存，有时值得多等待一段时间。设运行同一模型的后端 A 排队 200 ms，命中后处理前缀需 50 ms；后端 B 排队 20 ms，未命中处理需 500 ms；共同网络时间为 50 ms，后续生成相同。进入生成前，A 用 300 ms，B 用 570 ms，A 领先 270 ms。缓存命中节省了 450 ms 的处理时间，但增加了 180 ms 的排队时间，净收益恰好为 270 ms。当 A 的排队时间增至 470 ms 时，两者在生成开始前花费的时间就相同了。

章首每秒 30 次模型调用还受到并发容量的约束。每次调用需 9 秒时，平均有 270 次调用正在执行；缩短到 6 秒后，降为 180 次。但快速服务的时间是靠减小每个副本的 batch 换来的：每个副本只同时服务 16 个会话，10 个副本只有 160 个并发槽，无法容纳 180 次调用，需要增至 12 个副本（48 张 B200）。每次调用占用的 GPU 时间也从 $4 \times 9/32 = 1.125$ B200 · 秒增至 $4 \times 6/16 = 1.5$ B200 · 秒。按量 API 则通常分别限制单位时间内的请求数和 token 数；若每次调用的输入或输出增多，token 额度可能先于请求数额度用完。选择模型服务时，需要分别比较请求排队时间、并发调用数和 token 处理能力，找出限制任务完成速度的因素。

**例 11-7：缓存命中与成功率何时能抵消更高的服务单价？**向 Claude Haiku 4.5（记为 A）和 Claude Sonnet 5（记为 B）各提交 1,000 项单次调用任务。每次输入 20,000 个 token，其中公共前缀 19,000、新输入 1,000。前缀缓存已经建立，没有额外的缓存创建、存储、工具和预热成本，任务通过验收的概率与缓存是否命中相互独立。单价取两者的标准 API 价格，思考 token 数与成功概率为题设。[35]

| 参数 | A：Haiku 4.5 | B：Sonnet 5 |
|---|---|---|
| 普通输入单价 / 美元每百万 token | 1 | 2 |
| 缓存读取单价 / 美元每百万 token | 0.1 | 0.2 |
| 生成单价 / 美元每百万 token | 5 | 10 |
| 实际思考 token | 1,800 | 100 |
| 可见输出 token | 200 | 200 |
| 同一验收规则下的成功概率 | 0.80 | 0.98 |

A 的前缀始终命中，每次成本为 0.0129 美元，成功任务平均成本为 $0.0129/0.8 \approx 0.0161$ 美元。B 全命中时每次成本为 0.0088 美元，未命中时为 0.0430 美元。令 B 的请求命中率为 $h$，则

$$C_B(h) = \frac{0.0088h + 0.0430(1-h)}{0.98}.$$

令 B 的平均成本等于 A 的平均成本，得到 $h \approx 79.5\%$。B 的各项 token 单价都是 A 的两倍，却在高命中率下更省，因为实际生成量较少且成功率较高。命中率降至 50% 时，B 的成功任务成本升至约 0.0264 美元，此

[34]调度、模型路由与云端环境研究，包含 ASI、RLBoost、DistRS、SpecBox。ASI 研究覆盖六个月、155,410 张 GPU，其分配比例度量设备归属。token 单价取自 Claude API 计价页快照。

[35]routing-cost 固定结果、交点结果及完整推导。Haiku 4.5 与 Sonnet 5 的单价取自 Claude API 计价页快照；计算采用与缓存命中相互独立的验收通过概率，成功任务数按给定概率计算。

时 A 更便宜。

再要求至少 90% 的提交任务在 6 秒内通过验收。设完整请求时长为：A 10 秒，B 命中 4 秒、未命中 12 秒，均已包含排队和生成。A 不满足期限；B 需要 $0.98h \geq 0.90$，即 $h \geq 91.8\%$。

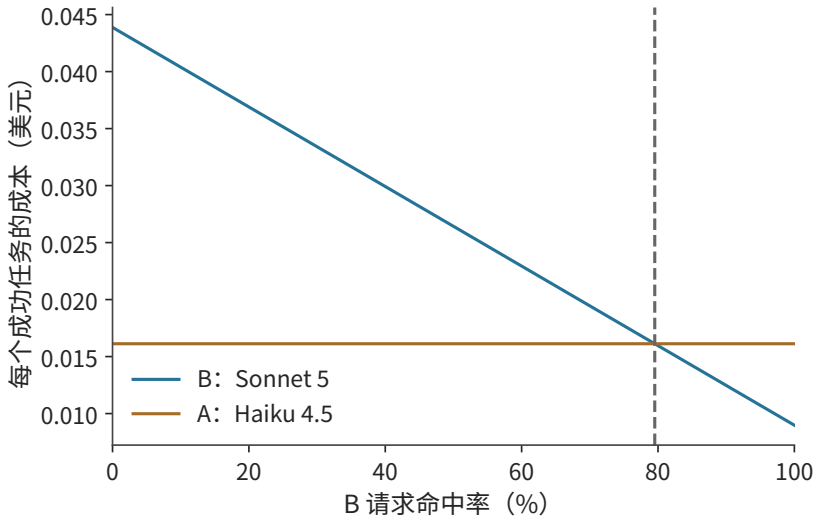



图 11-40：B 的成功任务成本随命中率提高而下降，在约 79.5% 处等于 A。两者的总成本先分别除以各自成功完成的任务数；本图只比较成本，下一图单独加入期限。

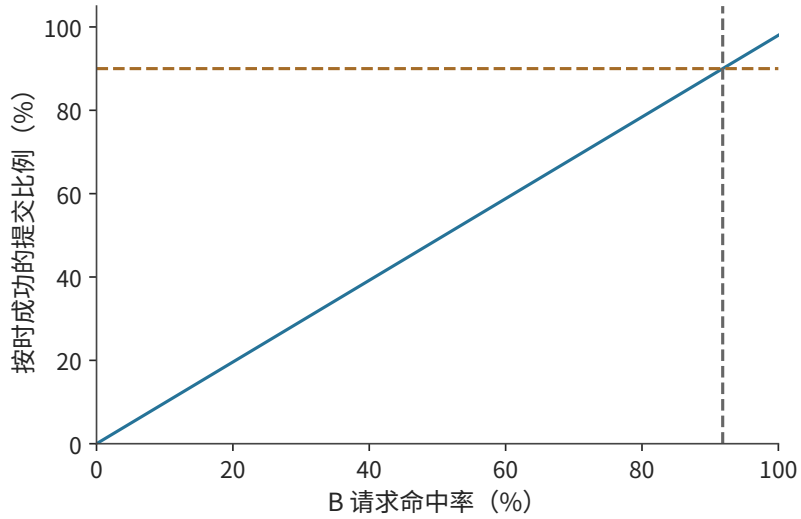



图 11-41：只有 B 命中的 4 s 路径满足 6 s 期限；再乘 98% 的验收通过率，按时成功比例为 0.98h。达到 90% 目标要求 h 至少约 91.8%。

本例每次前缀都是 19,000 个 token，所以请求命中率 $h$ 也等于前缀内部的 token 命中比例；相对于完整的 20,000 个 token 输入，缓存读取占比为 $0.95h$。成本交点约为 79.5%，业务门槛约为 91.8%。从低命中率向右移动，B 先变得便宜，随后才满足按时完成目标。把服务目标放到成本曲线上，就能直接读出最终可选区域。

### 11.4.4 自建服务、按量 API 与订阅的成本比较

路由比较确定了哪些模型服务同时满足质量、成本和期限要求。选定服务以后，还要决定是自建部署、按调用付费，还是购买订阅。模型选择确定每次任务所需的服务，采购方式决定如何为这些服务付费。自建容量先

支付设备与预留成本，再由任务分摊；按量 API 随调用支付；订阅则按月或其他周期支付成本，取得相应产品的使用权限。三种方式的共同问题是：满足相同任务量时，总支出随使用量如何变化。

设一段固定统计窗口中，自建预留与固定支出为 $F$，每项提交任务的新增成本为 $v$；API 每项平均成本为 $c$。若两者成功概率相同且处理能力均满足需求，$N$ 项任务下自建更省的条件为

$$F + Nv < Nc, \qquad N > \frac{F}{c - v} \quad (c > v).$$

若 $c \le v$，API 每项成本已不高于自建新增成本，固定投入又为正，因此 API 在所有使用量下都更省。若两者成功率不同，分别以 $Np_{\text{self}}$ 和 $Np_{\text{api}}$ 为分母，比较的就变成每个成功任务的成本。

下面用同一模型、同一种 GPU 比较两种付费方式，两边 batch size 相同，任务质量也相同，只差计费方式。预留方式按 GPU 云平台 Runpod 公开的 B200 Pod（按小时计费、独占使用的 GPU 实例）价格（每卡时 6.79 美元）租 4 张卡一个月（720 小时），$F = 4 \times 720 \times 6.79 = 19{,}555.2$ 美元；GPU 已经整月付费，每项任务的新增成本 $v$ 为 0。按量方式使用同一平台按秒计费的 B200 Serverless worker（每卡时 8.64 美元），与按量 API 一样随用量付费，并同样以 32 个会话为一个 batch 运行。沿用章首任务，每项三次调用、每次占用 1.125 B200 · 秒，按量成本为 $c = 3 \times 1.125 \times 8.64/3600 = 0.0081$ 美元。

成本相等时的任务量为 $19{,}555.2/0.0081 \approx 241$ 万项。每项任务占用一个会话 27 秒，这组 4 张 B200 一个月最多处理 $32 \times 2{,}592{,}000/27 = 307.2$ 万项，交点对应约 78.6% 的利用率，恰好是两种单价之比 $6.79/8.64$。100 万项时预留仍要 19,555.2 美元，按量只需 8,100 美元；300 万项时按量需要 24,300 美元，预留更省。固定投入在低使用量下难以摊薄，高使用量下则被较低的新增成本抵消。

图 11-42 中，预留成本是一条水平线，高度来自整月租金，右端止于这组 GPU 的处理上限；按量成本从零开始，斜率由每项任务占用的 GPU 时间决定。执行效率提高后，同样 4 张 B200 每月能处理更多任务，预留线向右延长，按量线的斜率随之降低；交点仍对应同一个利用率，即两种单价之比。

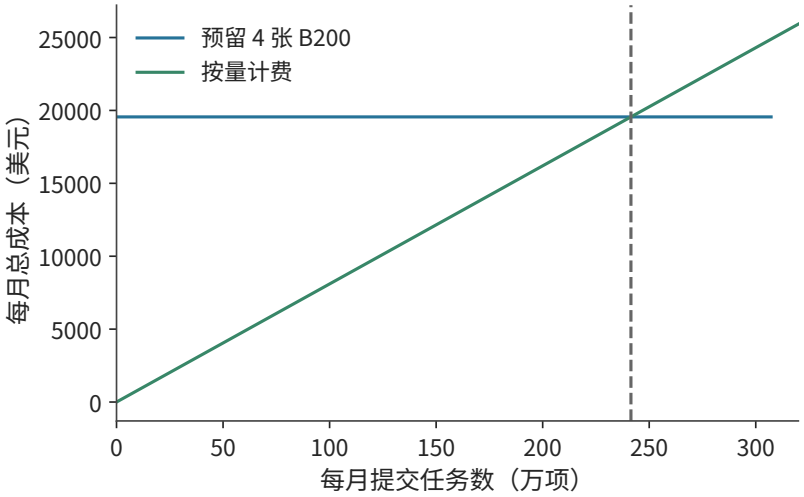



图 11-42：4 张 B200 预留一个月共 19,555.2 美元，按量成本为 0.0081N 美元，N 为提交任务数。两条线在约 241 万项相交，对应预留 GPU 约 78.6% 的利用率；预留线止于每月 307.2 万项的处理上限。

执行效率能提高多少，取决于时间用在哪里。仍以 4 张 B200 运行 DeepSeek V4-Flash、同时服务 32 个会话、每个会话保留 200K token 上下文的部署为例，会话数和 GPU 租价保持不变。每个输出 token 约需 25.3 ms，其中逐 token 的 decode 占 15.9 ms，其余输入处理和上下文重建占 9.4 ms。decode 阶段的实际速度提高到两倍后，每个输出 token 所需时间降为 $15.9/2 + 9.4 \approx 17.4$ ms，总加速约 1.46 倍，每百万输出 token 的 GPU

成本由约 6.0 美元降到 4.1 美元。[36]

原先每个输出 token 所需时间中，约 63% 用于 decode，其余 37% 保持不变，因而两倍 decode 加速只能将总时间缩短约 31%。即使把 decode 时间降到零，其余处理仍需要 9.4 ms。这 9.4 ms 还要按第 1.3.4 节的判据再拆一次：读取 KV 和重建上下文是必需的工作，按带宽算出的时间构成新的下界；调度、拷贝和格式转换则是可以去掉的软件开销，才是优化空间。若要突破这一限制，需进一步优化输入处理和上下文重建；同一组加速器在单位时间内生成更多 token，固定租价才能由更多产出分摊。

章首的代码修复平台每秒调用模型 30 次。快速服务每次调用多占 0.375 B200 · 秒，按 Pod 价格约多 0.00071 美元，平台每秒便多支出约 0.021 美元；这笔额外成本让任务耗时从 30 秒降到 21 秒，满足 24 秒期限。章末将同时计算新增模型费用与节省的环境占用费用，比较各方案的任务总成本。

## 11.5 完整任务的恢复、成本计算与扩容

### 11.5.1 工具失败后的恢复与模型选择

第 11.4 节选择模型时，主要考虑了调用正常完成的情况。实际运行中，还需要处理调用失败、结果丢失和工具异常。如果控制器没有收到测试完成的确认消息，就无法区分两种情况：测试尚未完成，或者测试已经完成但确认消息丢失。

恢复时，控制器先查询已持久化的结果，核对操作标识和输入文件版本。如果结果存在，就可以继续；如果不存在，再判断是否允许重试。幂等操作指以同一操作标识重复调用时，最终业务效果与调用一次相同。只读查询或幂等操作重复执行通常不成问题；外部付款、创建资源或发送消息重复调用，却可能使同一操作实际发生两次，需要目标系统支持操作标识、事务或补偿。

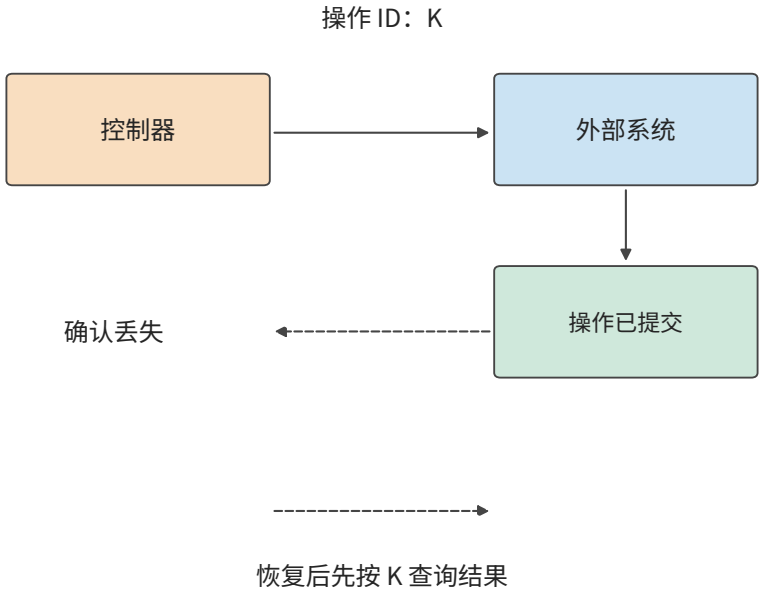



图 11-43：外部系统已执行并提交操作 K，确认却丢失。恢复环境不会撤回外部操作的结果；控制器按同一操作标识查询结果后接续任务。实线表示请求与执行，虚线表示确认及恢复查询。

假设执行一次工具需要 3 CPU · 秒，结果被发送两次。如果接收方按操作标识去重，累计 CPU 时间仍为

[36] B200 Pod 每卡时 6.79 美元、Serverless 每卡时 8.64 美元取自 Runpod 价格页快照。计算结果为 25.317 ms，其中 decode 为 15.918 ms，其余阶段共 9.399 ms。此处逐层固定开销和周期全量重建为给定输入，详见持续 Agent 比较基线、优化审计、时间归因与敏感性。

3 秒；如果控制器因确认丢失而再执行一次，累计 CPU 时间就增至 6 秒。两种情况最后都可能只有一条结果，却消耗不同资源。消息去重解决前一种重复；后一种重复要靠持久化的结果和目标系统的幂等协议来避免。环境外已经提交的操作在快照恢复后仍然存在，恢复流程要先查询其提交状态。[37]

不过，请求正常返回，并不意味着恢复时一定能找到它的记录。传统操作系统也区分“写入完成”和“已经持久化”。以普通文件的缓冲写入为例，`write()` 返回时，数据可能仍在内核的页缓存中，程序可以继续计算，由操作系统在后台写入。若程序必须等数据存好才能继续，就需要调用 `fsync()`，等待文件数据和必要的元数据写入存储设备，并检查是否成功。[38]

Agent 运行时也需要规定清楚：向调用者报告完成时，只保证答案已经生成，还是保证消息、工具结果和恢复位置都已保存。这些约定就是持久性语义。仅对聊天日志调用 `fsync()` 还不够，因为恢复所需的信息可能分别存放在日志、工作目录和外部数据库中；控制器还要核对消息中的操作、目录中的文件和数据库中的结果是否一致。

这里按 ReAct 循环的轮次保存：每轮包括一次模型输出、由此发起的工具调用及其返回结果，可能包含多条消息。控制器将这一轮的记录追加到本地 JSONL 文件，确认整轮保存成功后才进入下一轮；保存失败就立即中止。因此，本地不会在某轮保存失败后，还继续执行并保存后续轮次。这里的“保存成功”还要说明能承受哪种故障：写入本地页缓存、完成本地 `fsync()`，或收到云端持久化确认，提供的保证并不相同。本地文件可以支持进程重启后的恢复，却未必能应对整台机器及其磁盘不可用。

为了减少等待，也可以每轮先保存到本地，再在后台上传该轮新增的记录。假设上传失败后暂不重试，也不中止本地执行：第一轮运行测试、取得失败结果，第二轮修改代码、取得工具确认，第三轮再次测试、取得新结果。三轮记录都已写入本地 JSONL，但第二轮上传失败，第三轮上传成功（图 11-44）。此时本地历史完整，云端却缺少第二轮的模型输出和工具结果。如果本机随后不可用，换一台机器仅凭云端记录恢复，就会遇到这个缺口。若每次上传的是包含此前所有轮次的完整快照，或上传程序必须补齐失败记录才继续，则不能套用这一例子。

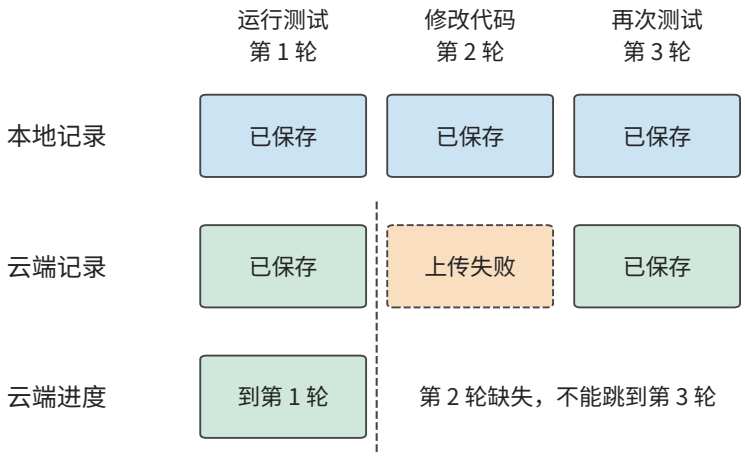



图 11-44：初始状态已在本地和云端保存，每列代表一个 ReAct 轮次。每轮先保存到本地，再异步上传该轮新增记录；本例允许上传失败后继续执行和上传后续轮次。第二轮上传失败后，即使第三轮上传成功，云端历史仍只完整到第一轮。横向排列表示轮次顺序，不表示耗时。

采用这类异步保存方式时，控制器必须分别跟踪本地和云端的保存进度。云端不仅要记录收到的最后一轮，

[37] 任务记录与证据范围、文件队列故障注入、任务退出状态记录。这些记录分别用于说明结果持久化、故障诊断和成本核算。

[38] Linux 手册 write(2) 与 fsync(2)。`write()` 成功不保证数据已写入持久存储，调用者还须检查实际写入字节数与同步错误。新建或重命名文件时，目录项的持久化还可能需要对目录执行 `fsync()`；文件同步本身不提供多文件事务。

还要记下从起点连续保存到了哪一轮。不能因为第三轮上传成功，就把连续保存位置从第一轮改成第三轮；只有补齐第二轮，或用完整的本地副本重建并核对后，才能确认云端历史也完整到第三轮。

删除旧日志时也一样。若要删除前两轮的记录，就必须先保存一份能够恢复到第二轮结束时的完整状态，并将状态与对应轮次一起提交。恢复时若既找不到此前的完整日志，也找不到这份状态，即使剩余日志全部读完，也应返回 `incomplete`，表示没有足够依据确认历史完整。如果接口允许按消息位置恢复，还须确认该位置是否对应一个已保存的完整轮次，并核对工作目录版本；轮内位置若没有单独保存，就不能承诺恢复到那里。已经提交到外部系统的操作，也不会因为消息回退而撤销。[39]

对本章的代码修复任务，可以借用数据库事务的做法，把一轮“模型决策—工具执行—保存结果”作为一个整体提交。这一轮的模型消息、工具结果和相关文件版本都保存成功后，控制器才记下这一轮已提交；重启后，再从最后一轮完整提交的结果接着执行。这也是 **Agent 事务（agent transaction）**研究的一项内容：如何在 Agent 执行中实现原子性、一致性、隔离性和持久性，即数据库中的 ACID 性质。[40] 如果一轮操作还调用了外部服务，就需要对方支持操作标识、幂等接口或补偿处理；仅在本地记下“已提交”，不能保证外部操作也恰好完成一次。

保存得越频繁，故障后通常需要重做的工作越少，但正常执行时的开销也越大。用一个小算例比较：100 个请求依次执行，每个用时 200 ms；每次持久化提交需要 8 ms 的固定开销，另外每写入一个请求的记录需要 2 ms。假设提交期间暂停处理请求，忽略其他开销，结果如下。

| 保存方式 | 保存共用时 | 无故障时总用时 | 故障后最多重做的计算时间 |
|---|---|---|---|
| 每个请求提交一次 | $100(8+2)=1000$ ms | 21 s | 0.2 s |
| 每 10 个请求提交 | $10(8+10\times 2)=280$ ms | 20.28 s | 2 s |

每 10 个请求一起提交，正常执行可节省 0.72 s；但若恰好在提交前发生故障，就要重做这 10 个请求，共 2 s 的计算。这里假定每次提交要么全部成功，要么全部不生效，未提交的请求都需重做，工具也允许安全重试；表中未计入重建环境和重新保存的时间。若改为异步保存，控制器就可以一边处理后续请求，一边写入已有结果。故障后需要重做多少，取决于当时还有多少记录没有存好。因此，比较保存方式时，除了任务完成时间，还应记录已返回但未保存的请求数，并核对重启后实际能恢复到哪里。

这里仍然需要回答全书反复讨论的两个问题：数据放在哪里，谁必须等它。第 11.2 节在模型调用期间释放工具环境，以节省内存。这样做的前提是下一轮所需的状态已经存到环境之外，或可以在丢失后重新计算。若唯一一份结果还在环境内存中，控制器就要等它保存完成再释放环境，否则只能在丢失后重做。因此，调度器安排资源时，也要计入保存状态的等待时间和故障后的重做成本。

确认能从哪里继续后，控制器还可以选择换用更强的模型。若原模型已完成两轮，补丁和测试结果也已完整保存，新模型就能接着求解，无需重做前两轮。这样只需支付下一次模型调用的成本，并可能提高求解成功率。后续修复有多大把握，取决于此前为什么失败。因此，下文计算恢复树中各分支的概率时，都以任务已经到达该节点为条件。

### 11.5.2 成功任务的完整成本

失败后重试或换用模型，都要额外付费，因此只看一次调用的价格，算不出完成一个任务实际花了多少钱。应把成功和失败尝试的支出全部加起来，再除以实际完成的任务数。设统计期间的总支出为 $C_{\mathrm{all}}$，满足质量要求的任务数为 $N_q$，其中按时完成的任务数为 $N_{q,d}$，则每个成功任务和每个按时成功任务的平均成本分别为

[39] Safe to Resume?（2026，预印本）分析恢复状态与继续执行所需依赖不一致的问题。本节的请求缺口与批量提交数字为教学算例，不是论文测量结果。

[40] Agentic Transaction（2026，预印本）提出面向 Agent 的语义原子性、一致性、隔离性与持久性；其中 §2.2.4 讨论已提交状态、证据与恢复元数据的持久保存。本节以代码修复说明提交单元的设计，不据此假定任意工具调用都具备 ACID 保证。

$$\overline{C}_q = \frac{C_{\text{all}}}{N_q}, \qquad \overline{C}_{q,d} = \frac{C_{\text{all}}}{N_{q,d}}.$$

失败尝试也消耗了模型和工具资源，不能从总支出中扣除。例如十项任务共支出 1 美元，五项成功，平台每完成一项实际支付 0.2 美元。若只统计五项成功尝试花费的 0.5 美元，就会得到 0.1 美元，恰好漏掉另一半支出。

这项单位成本也可以在部署前用有限重试树初步估算。对路径 $\pi$，令路径概率为 $p_\pi$，各步骤成本之和为 $c_\pi$，则 $E[C] = \sum_\pi p_\pi c_\pi$。成功概率是所有最终成功路径的概率之和。每个节点的概率都应根据此前经历的执行结果计算；如果两类失败的后续成功率不同，应拆成不同节点。

**例 11-8：局部修复对成功率与超时比例的影响。**另取首次尝试需 10 秒的小例子，比较以下有限恢复策略。首次尝试用 Claude Sonnet 5，输入 2,500、输出 500 个 token，按标准价格花费 0.010 美元，耗时 10 秒；80% 直接成功，12% 进入局部修复，8% 直接升级，即换用能力更强的模型重试。局部修复仍用 Sonnet 5，输入 1,500、输出 300 个 token，新增 0.006 美元和 4 秒，条件成功率 60%，其余升级。升级换用 Claude Opus 5（每百万 token 输入 5 美元、输出 25 美元），输入 3,000、输出 600 个 token，新增 0.030 美元和 8 秒，条件成功率 98%。20 秒期限只用于评价，不主动终止任务。[41]

图 11-45 中，局部修复成功时，任务在第 14 秒结束；修复失败后再升级，就比直接升级多用 4 秒。总成功率提高后仍有一部分成功结果超过期限，原因就在这一额外分支。

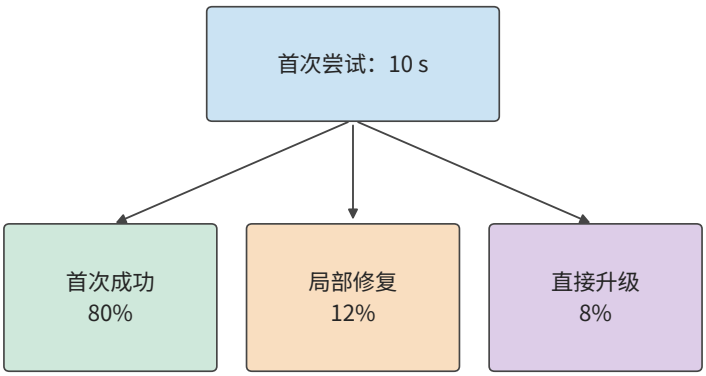



图 11-45：首次尝试在 10 s 后分为成功、局部修复、直接升级三类。框内比例以全部提交为分母；下一图展开修复的条件分支。

六种执行结果如下。每行列出的成本都是整个执行过程的总成本，包含此前各次尝试的支出。

[41] 精确结果：成功概率 0.99744，通过测试且按时完成概率 0.9504，每项期望成本 0.01456 美元，累计 CPU 时间 3.376 秒，驻留 23.008 GiB · 秒。计算结果分别列出资源用量和成本；各节点的给定成本已经是总价，无需再按资源用量重复计费。见retry-paths 有限条件图。各节点成本按 Claude 标准价格与题设 token 数计算，概率按题设给定，重试沿有限执行树展开；超过评价期限的任务仍继续执行到终点。

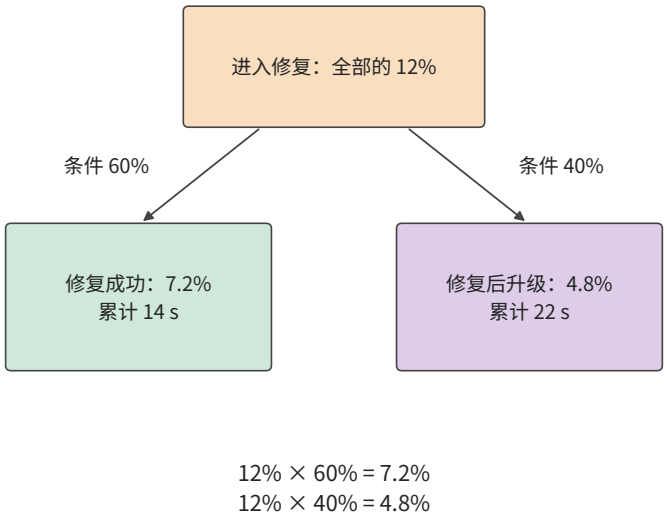



图 11-46：把修复节点放大：进入此处的 12% 中，60% 修复成功，占全部提交 7.2%；40% 转入升级，占全部提交 4.8%。升级再用 8 s，因此后一条路径累计 22 s。

| 执行过程与结果 | 概率 | 完整成本 / 美元 | 时间 / s | 通过测试且按时完成 |
|---|---|---|---|---|
| 首次成功 | 0.80000 | 0.010 | 10 | 是 |
| 首次 → 修复成功 | 0.07200 | 0.016 | 14 | 是 |
| 首次 → 修复 → 升级成功 | 0.04704 | 0.046 | 22 | 否 |
| 首次 → 修复 → 升级失败 | 0.00096 | 0.046 | 22 | 否 |
| 首次 → 升级成功 | 0.07840 | 0.040 | 18 | 是 |
| 首次 → 升级失败 | 0.00160 | 0.040 | 18 | 否 |

可以用 1,000 项提交任务逐步计算。首次尝试花费 10 美元，约 800 项直接成功。120 项进入局部修复，新增成本 $120 \times 0.006 = 0.72$ 美元，其中约 72 项成功；余下约 48 项连同直接升级的 80 项，共有 128 项升级，新增成本 $128 \times 0.030 = 3.84$ 美元。总成本约为 14.6 美元，最终约 997 项成功。

先修复再升级的路径需要 $10 + 4 + 8 = 22$ 秒，超过 20 秒期限。约有 47 项任务通过了测试，但超过了期限，因此按时成功数约为 950。每项通过测试的任务平均花费约 0.0146 美元，每项按时通过测试的任务平均花费约 0.0153 美元；只做首次尝试时，成本为 $10/800 = 0.0125$ 美元，但成功率只有 80%。恢复提高了完成比例，也提高了单位完成成本。业务若要求至少 95% 的提交按时成功，就需要这项额外投入；若只比较单次成本，则会遗漏它带来的完成量。

采用上述恢复策略后，成功比例从 80% 提高到约 99.7%，但约 4.7% 的提交要到第 22 秒才能完成。按 20 秒期限统计时，这部分任务花费了资源，却不能计入按时完成数。如果只缩短评价所用的期限，执行过程不变，只是更少的任务会被算作按时成功；如果限制重试次数，控制器则会少执行一些步骤，支出和成功率也会随之改变。[42]

[42]任务记录与证据范围、文件队列故障注入、任务退出状态记录。这些记录分别用于说明结果持久化、故障诊断和成本核算。

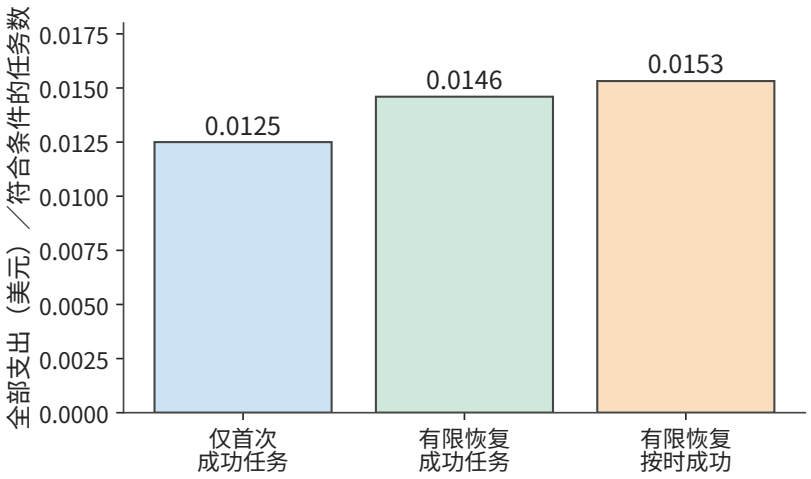



图 11-47：同一批提交任务换用恢复策略后的成本与结果。各柱均以相应策略的全部支出为分子，再分别除以通过测试的任务数或按时通过测试的任务数；只做首次尝试的成功率为 80%，有限恢复策略约为 99.7%，其中约 95.0% 的提交在 20 秒内成功。分支概率、成本和时间见例 11-8，期限用于评价而不强制停止执行。

恢复策略之外，还要考虑模型加速带来的影响。模型变快以后，环境准备可能不再能与模型调用完全重叠，开始影响任务完成时间。假设下一次工具调用必然发生，快速服务还需 6 秒生成，而模板不在本机、走冷路径创建环境约需 2.7 秒；在模型开始生成时创建环境，这 2.7 秒准备时间就完全与模型生成重叠。若模型阶段缩短至 1 秒，同样时机开始创建仍要再等约 1.7 秒。[43] 先只把模型调用加快，保持准备策略不变；再比较更早准备、保留已就绪环境和按需等待三种做法的成本，并计入提前占用的资源，以及提前准备的环境最终没有用上时的开销。

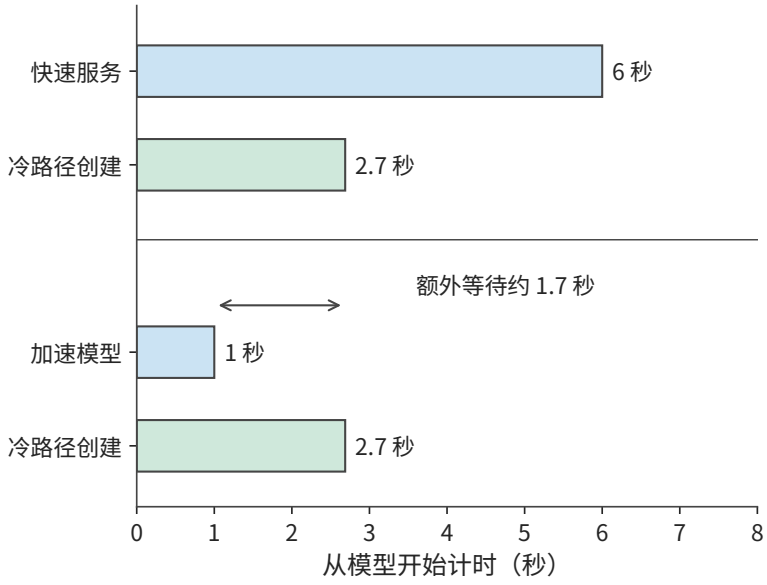



图 11-48：模型加速后，环境创建成为工具开始执行前的等待来源。两种情况都在零时刻开始创建环境；工具须同时等待模型决策与环境就绪。横轴为秒。

恢复时要重建的不只是工具环境，还有模型服务一侧的状态。本章设计题用 V4-Flash 计算时间和成本；模

[43] 本章条件比较的逐项复算与计算程序。

型侧状态的恢复则以全书跟踪的 V4.1 Flash 会话为例，因为其全局 KV 可以取回，局部 SWA 状态则要重放提示末尾的 token 来重建（第 3.2.4 节），两部分的恢复路径不同。这样的会话在等待工具时，同时保留两组状态：操作系统管理进程、文件与环境，模型服务管理全局 KV 和局部 SWA 状态。恢复任务需要分别准备这两组状态，再在工具结果进入下一轮模型调用时汇合；已执行的外部操作通过查询目标系统确认结果。[44] 环境准备与 KV 取回能够并行时，两者中较晚完成的一项决定恢复所需的等待时间。

### 11.5.3 CPU、GPU、内存与服务额度的扩容选择

下面根据章首提出的 24 秒完成期限，选择模型服务与环境管理方案。此前已经得到三项结果：普通服务每次调用 9 秒，三轮需要 30 秒；快速服务每次 6 秒，三轮需要 21 秒；每轮重建环境把每项任务的累计环境内存占用降到 18 GiB · 秒，准备工作额外使用 0.3 秒 CPU 时间。

**例 11-9：如何选择模型服务与环境管理方式，才能按期完成且成本最低？**沿用章首条件：每 0.1 秒到达一项任务，每项执行三轮；工具平台是一台 48 核、约 358 GiB 内存的 m5d.metal，模型服务现有 10 个 V4-Flash 副本（40 张 B200）。两种服务运行同一个模型，最终通过测试的概率均为 95%，且任务能否通过测试与到达时刻无关。模型调用按每次占用的 B200 时间计价，每卡时 6.79 美元：普通服务每次 1.125 B200 · 秒，约 0.00212 美元；快速服务每次 1.5 B200 · 秒，约 0.00283 美元。CPU 与内存按 E2B 公布的沙箱单价计费，每 CPU · 秒 0.000014 美元，每 GiB · 秒 0.0000045 美元，均按实际使用量计算。文件持久化成本已包含在每轮工具执行中；平台共有的固定支出在四种方案中相同。[45]

首先排除“只增加 CPU”。三轮工具共 3 秒，即使将工具加速两倍，普通服务仍需 $27 + 1.5 = 28.5$ 秒；把工具时间降到零，仍有 27 秒模型调用。CPU 扩容无法把这条串行路径缩短到 24 秒以内。

接着比较环境策略。每轮重建环境的准备工作与模型调用重叠，不改变完成时间：普通服务仍为 30 秒，换快速服务后模型总时间降到 18 秒，加工具 3 秒，共 21 秒。因此在快速服务下，始终驻留与每轮重建环境都满足期限，需要进一步比较资源与成本。

图 11-49 先按期限筛选方案：普通模型的最后一轮结束于期限之后，快速模型的三轮则都能在期限内完成。接下来只需在使用快速模型的两种环境策略之间比较成本，同时检查是否超过平台容量。

| 方案 | 完成时间 / s | 累计 CPU 时间 / CPU · 秒 | 每项累计内存占用 / GiB · 秒 | 平均内存 / GiB | 每千项提交成本 / 美元 | 按时通过测试的比例 |
|---|---|---|---|---|---|---|
| 普通服务 + 始终驻留 | 30 | 3 | 60 | 600 | 6.68 | 0 |
| 普通服务 + 每轮重建环境 | 30 | 3.3 | 18 | 180 | 6.49 | 0 |
| 快速服务 + 始终驻留 | 21 | 3 | 42 | 420 | 8.72 | 95% |
| 快速服务 + 每轮重建环境 | 21 | 3.3 | 18 | 180 | 8.61 | 95% |

以最后一行为例，三次模型调用共占 $3 \times 1.5 = 4.5$ B200 · 秒，花费约 0.0084875 美元；CPU 成本为 $3.3 \times$

[44] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

[45] 贯穿平台与例 11-9 的输入、逐阶段时间表、成本、成本相等的条件与复算见贯穿设计数据及验证程序。模型调用时间取自本书对 4 张 B200 运行 DeepSeek V4-Flash、每会话 200K 上下文的估算：每副本 32 个会话时每个输出 token 25.3 ms，16 个会话时 16.9 ms，均含分摊的输入处理与上下文重建（比较结果、完整数据）；355 个 token 的调用分别约 8.99 s 和 6.01 s，设计取 9 s 和 6 s。B200 每卡时 6.79 美元取自 Runpod 价格页快照。m5d.metal 的 48 核（关闭超线程）与 384 GB 内存取自 Firecracker 论文第 5 节的评测环境；1 vCPU、2 GiB 规格与 CPU、内存单价取自 E2B 计价页快照。热路径准备共 2 秒、0.1 CPU · 秒与 95% 验收通过率为题设，结果保存与工作状态清理包含在工具的 1 秒内；模板容量算例单独说明内容加载。

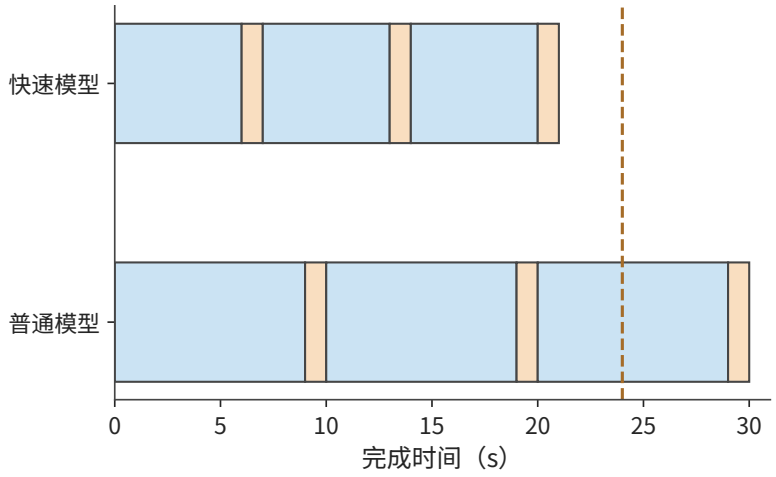



图 11-49：两种方案均在模型调用期间准备环境，工具执行均为每轮 1 秒。模型调用从每轮 9 秒降至 6 秒，三轮结束时刻从第 30 秒移到第 21 秒，从 24 秒期限之后提前到期限之内。

0.000014 = 0.0000462 美元，内存成本为 $18 \times 0.0000045 = 0.000081$ 美元，合计约 0.0086147 美元，即每千项 8.61 美元。模型费用占 98% 以上，因此每轮重建环境的主要收益不在费用，而在容量。每秒进入 10 项任务，工具执行与环境准备平均使用 33 个 CPU 核和 180 GiB 内存，都在 m5d.metal 的容量以内；快速服务 + 始终驻留则需要 420 GiB，这台主机无法容纳。模型服务平均同时执行 180 次调用，快速服务每个副本只容纳 16 个会话，需要 12 个副本，比现有多 2 个。

选中方案的平均需求满足容量要求，还要确认各阶段能否按时安排。任务均匀到达，可以直接写出一个满足容量限制的时间表。快速服务下，以每项任务的到达时刻为起点，分别在第 4–6、11–13、18–20 秒准备环境，在第 6–7、13–14、20–21 秒运行工具；准备工作累计使用 0.1 秒 CPU 时间，均匀分布在两秒内。任务每 0.1 秒到达一项，稳定运行后，每一轮的工具执行各占 10 个核，每一轮的准备工作各占一个核；活跃环境最多约 90 个，共 180 GiB。三个模型调用阶段各平均有 60 次调用正在处理，合计 180 次并发调用，12 个快速副本共有 192 个并发槽。按此时间表执行，既满足各阶段的先后依赖，也没有超过平台容量。

**因此选择快速服务，每轮重建环境，在模型调用期间完成准备，并把模型副本从 10 个增至 12 个。**该方案用现有工具主机即可满足 24 秒要求，是四个方案中唯一既能按期完成、又不超出这台主机容量的方案。每千项按时通过测试的任务平均花费约 $8.61/0.95 \approx 9.07$ 美元；快速服务 + 始终驻留即使另配内存，也要约 $8.72/0.95 \approx 9.18$ 美元，每轮重建环境只让这项成本降低约 1.2%。普通服务 + 每轮重建环境虽然更便宜，但即使通过测试，也已经超过期限。

第 11.5.2 节还说明，增加重试会改变按时完成的比例。这里正常任务在第 21 秒完成，剩余期限只有 3 秒；若最终失败后再加一次 4 秒局部修复，结果将到第 25 秒才出现。因此，本题在三轮后结束任务；未通过测试的任务直接返回失败结果。若业务把期限延长到 25 秒，局部修复才有机会增加按时成功数，届时再将条件成功概率和额外资源放入恢复树。

以上选择建立在模型每轮调用 6 秒的条件下。模型继续加速时，可以释放环境内存的空闲时段也会缩短，原来的环境策略就需要重新比较。保留环境还是重新建立环境，由两种方案成本相等时的等待长度即可判断。设两次工具执行之间需要等待模型 $G$ 秒。保持 2 GiB 环境，驻留成本为 $2G \times 0.0000045$ 美元；销毁后在下一轮模型调用末尾重新准备，内存成本为 $2 \times 2 \times 0.0000045$ 美元，准备所需的 CPU 成本为 $0.1 \times 0.000014$ 美元，共 0.0000194 美元。两者在 $G \approx 2.16$ 秒时相等。等待 6 秒时应重建；等待缩短到 2 秒时，保留环境开始更省，准备也占满全部可重叠窗口。模型继续加速，会改变最合适的环境策略。

还可以用第 1 章的 Amdahl 定律概括局部加速对总时间的影响。令原时间中可加速部分占比为 $f$，将这部

分加速 $s$ 倍，剩余部分保持原时长，则整体加速比为

$$S = \frac{1}{(1-f) + f/s}.$$

章首工具只占 10% 时间，工具加速两倍的整体加速为 $1/(0.9 + 0.1/2) \approx 1.05$；模型调用占 90%，若完整调用加速两倍，则为 $1/(0.1 + 0.9/2) \approx 1.82$。同样加速两倍，模型调用节省的任务时间更多，因为其占执行时间的比例更高。

第 12 章将在相同任务和完成目标下，进一步比较端侧、边缘和云端的执行方式，分析传输时间对任务完成时间的影响。

## 练习与实验

以下练习沿用 11-1 至 11-10 的资料编号。核心题 11-1、11-2 和 11-10 构成从需求到设计的完整练习；实验原始记录与选做运行入口见本章配套。

1. **实验 11-1〔核心〕：从多轮任务推算模型并发、CPU 与环境内存。**平台每秒接收 8 项任务，其中一半执行两轮，另一半执行四轮。每轮模型调用耗时 6 秒，工具占用一个 CPU 核执行 1 秒。求模型调用率、平均并发模型调用数与平均使用的 CPU 核数。若每个环境在任务全程占用 2 GiB，求平台的平均环境内存占用。再假设环境只在每轮 2 秒的准备阶段和 1 秒的工具执行阶段保留，重新计算平均内存占用，以及每秒需要创建的环境数量。
2. **实验 11-2〔核心〕：重建环境与恢复快照如何影响工具等待和内存占用时长？**一个环境中有可重新下载的依赖、未保存的编辑缓冲、已持久化的补丁和已提交的外部写入。逐项说明发生故障后能从哪里恢复，以及哪些内容无法直接恢复。
   再比较两种能够恢复下一轮所需状态的方案：走冷路径完整重建环境需要 2.7 秒；按 E2B 文档，保存 2 GiB 环境的快照约需 8 秒，从快照恢复约需 1 秒。将上一轮工具执行结束记为计时起点。下一次模型调用立即开始，耗时 9 秒，随后工具执行 1 秒。采用重建方案时，在计时起点释放原环境；采用快照方案时，在该起点开始保存快照，保存完成后释放原环境。
   两种方案都在不推迟下一次工具执行的前提下，尽量晚地开始重建或恢复。画出时间线，并计算从计时起点到下一次工具执行完成期间，各方案累计占用环境内存的时长。重建、保存快照、恢复和工具执行期间均计入环境驻留时间。
3. **实验 11-3：提前准备环境能省多少等待，又占用多少内存？**创建环境需要 2 秒，对下一次调用所需环境的预测准确率为 0.6。调用到达后取消错误预测所对应的环境；每个提前准备的环境从准备开始就占用 2 GiB。分别在调用到达前 1、2、4 秒开始准备，计算调用到达后的期望等待时间，以及错误准备造成的期望累计内存占用（以 GiB · 秒计）。预测错误时，在调用到达后重新创建所需环境。若准确率提高到 0.9，哪些收益变大，哪些浪费减少?
4. **实验 11-4：不同作业完成目标下，迁移何时优于等待。**新作业所需的资源正被已有作业占用，等待其释放需要 12 秒。新作业获得资源后，在本地执行需要 20 秒。也可以将已有作业迁至其他节点，在迁移完成后释放本地资源；迁移耗时为未知量 $m$，使已有作业的完成时间比不迁移时推迟 $m + 2$ 秒。先以新作业最早完成为目标，求迁移优于等待的条件；再以新作业的资源等待时间与已有作业的完成延迟之和最小为目标，重新求解。说明为何在两个目标下可以作出不同选择。
5. **实验 11-5：启动耗时如何影响短期实例的有效产出。**每个新实例传输与加载共需 8 秒，之后以每秒 100 个有效 token 的速率工作。三个实例从开始启动到回收的时长分别为 10、20、40 秒。分别求各实例的产出，以及实际工作时间占这段时长的比例。若准备时间降到 4 秒，比较三个实例各自的产出改善，并解释可用时间较短的实例为何更敏感。

6. **实验 11-6：验证进程数量与长尾样本如何决定整批完成时间。**三条样本在 0、6、12 秒到达，验证分别需 10、2、8 秒。先求只有一个执行进程时，按到达顺序处理各条样本的开始和完成时刻；再求使用两个执行进程时，整批最早何时完成。把最后一条样本的验证时间改为 80 秒，判断增加执行进程能否缩短整批完成时间，并解释哪条样本决定了这一结果。
7. **实验 11-7：缓存命中与结果正确性相关时，如何计算成本与按时成功率？**沿用例 11-7 的 B 服务成本，改为命中时成功率 99%、未命中时 90%。推导平均每个成功任务的成本随命中率变化的关系，以及在 6 秒内通过测试的任务比例。求至少让 90% 的任务按时通过测试所需的最低命中率，并与原独立概率模型比较。
8. **实验 11-8：自建与 API 服务的成功任务成本交点。**自建方式按 Runpod 价格预留 4 张 B200 一个月，共 19,555.2 美元，运行 V4-Flash，每项任务不再新增成本，任务成功率为 90%；API 方式使用 Claude Sonnet 5，每项任务调用三次，每次花费与例 11-7 中 B 命中时相同，为 0.0088 美元，成功率为 98%。按一个月作为成本比较期，固定支出全部计入该月。两者均有足够能力满足期限，并处理相同数量的任务，求它们的平均成功任务成本相等时的任务量。再将自建处理上限设为每月 50 万项，讨论交点怎样影响购买决定。
9. **实验 11-9：任务期限与提前停止如何改变重试成本和成功率。**沿用例 11-8，将期限改为 14、18、22 秒，分别求按时通过测试的比例。再设计一种在剩余时间不足以完成下一节点时停止的策略，重新计算期望成本，并解释它与仅改变评价期限的差别。
10. **实验 11-10〔核心〕：到达率提高后如何扩容并选择环境驻留策略。**沿用例 11-9，将到达率增加到每秒 15 项，每次调用只输出 118 个 token，快速服务调用时间降到约 2.0 秒，每项任务仍执行三轮，每轮工具执行仍需 1 秒。工具主机、每个副本的会话数和各项单价保持原值。比较始终驻留与每轮重建环境，决定应增加哪项资源、至少增加多少，并说明环境策略是否需要改变。最后将到达方式改为每秒有 15 项任务同时到达，画出工具执行的时间线，并标出 CPU 并发占用的峰值。

## 本章小结

章首平台平均只用 30 个 CPU 核，环境却需要 600 GiB 内存，超出了 m5d.metal 的 358 GiB。原因是三轮任务中有 27 秒在等待模型，而工具环境始终占用内存。在模型调用期间准备环境，每项任务的累计内存占用就从 60 降到 18 GiB · 秒，能释放大量内存；代价只是准备工作多用 0.3 秒 CPU 时间。

这项改进利用了完整任务的执行关系。环境管理器确定模型调用期间可用于准备的时间后，可以提前创建环境，让进程仅在工具准备和执行期间存在；持久化文件保存各轮之间需要保留的状态。模型服务和环境平台共同安排准备时机，原本贯穿任务全程的资源占用就变成了按阶段分配。

释放内存以后，任务仍需 30 秒，模型调用成为满足 24 秒期限的关键。把每个 V4-Flash 副本的会话数从 32 减到 16，调用从 9 秒降到 6 秒，三轮任务从 30 秒降到 21 秒，平均并发模型调用数也从 270 次降到 180 次；代价是每次调用多占 0.375 B200 · 秒，副本要从 10 个增至 12 个。结合每轮重建环境，一台 48 核、358 GiB 内存的 m5d.metal 即可支持给定任务流。最后在满足质量和期限的方案之间比较成本。

同样的方法也解释了 RL 的资源组织：成组作业先取得配套资源，临时的 rollout 实例先完成权重传输和加载，样本生成后立即提交验证，可以让验证与后续生成同时进行；失败后重试，则需要累加各次尝试的工作量。分析资源需求，要计算任务总共做了多少工作；分析完成时间，还要考虑这些工作必须按什么顺序执行、哪些可以同时进行。结合这两方面，才能判断一项局部改进对整个系统是否有益。

# 第 12 章 端边云协同

一张图片交给云端精修，模型运行 0.3 秒，用户却等待 12.8 秒。将模型加速十倍，只能节省 0.27 秒；把输入文件压缩到原来的一半，却能节省近 6 秒。决定模型放在哪里时，需要比较整个任务的完成时间。远端设备可以加快模型计算，但调用远端服务也会增加传输和等待时间。只有远端节省的计算时间多于新增的传输与等待时间，集中到云端部署才划算；模型推理变快以后，许多交互任务已不满足这一条件。

第 8—11 章介绍了模型服务、资源调度和任务运行环境。本章讨论如何把这些服务和环境部署到不同设备上。**端侧**采集输入并与用户交互；**边缘**是在附近提供计算的工作站或服务节点；**云端**提供远程资源池。终端可以在本地执行，也可以调用附近工作站或云端服务，还可以把模型的不同阶段分配给不同设备。

本章先用图片精修建立时间模型，再用语音解释流水线中的先后依赖，最后为根据截图判断下一步、再调用工具执行操作的截图 Agent 选择部署方案。图片精修需要计算完整成片何时返回，语音需要分析音频能否连续播放，Agent 则要累计每轮通信与执行的开销。三类任务共用一套分析方法：先计算处理和传输各需多久，再确定哪些步骤必须等待，最后加入连接建立、排队和故障恢复等实际开销。

## 12.1 从模型运行时间到完整交互时间

### 12.1.1 图片精修

图片精修需要保留阴影、高光和白平衡等信息。要为后期编辑保留更多原始信息，通常就要传输更大的文件。本例上传一张 30 MB 原图，处理后返回一张 5 MB 成片，以用户收到完整成片为任务结束的标志。[1]

先沿一张图片的路径看数据在何处停留。本例的具体速率和处理时间标在图 12-1 中，随后用通用公式表示这些量之间的关系。

建立模型时，先区分**各步骤所需时间**与**任务完成时间**。文件大小除以发送速率，得到发送所需时间；模型的计算时间则由计算量和处理器的执行速度决定。各步骤依次执行时，总时间等于各步骤耗时之和；部分步骤同时执行时，则要根据它们的先后依赖，计算最后一步何时结束。

把这些关系写成公式之前，先约定本例的简化条件：往返时间（round-trip time，RTT）简化为往返传播所需的时间，排队和协议处理的耗时在第 12.3 节单独讨论。连接已经建立，服务器收齐原图后开始处理，生成完整成片后再向用户发送，上下行以恒定速率传输有效载荷。设输入为 $S_u$ bytes，输出为 $S_d$ bytes，上下行速率为 $B_u, B_d$ bytes/s，RTT 为 $R$，处理时间为 $T_c$。串行完成时间为

$$T_{\text{serial}} = \frac{S_u}{B_u} + R + T_c + \frac{S_d}{B_d}.$$

上传的最后一个字节离开发送方后，还要传播到服务器；成片返回也要传播一次。两次单向传播合计为 $R$。传播时间由路径决定，发送时间由文件大小与速率决定，因此缩小文件只会缩短发送时间。这里的 $B_u, B_d$ 表示有效载荷的传输速率，连接启动和反馈等待将在第 12.3 节单独加入。

**例：图片精修的上传时间如何限制模型加速收益？**上行 20 Mbit/s，下行 100 Mbit/s，$R = 0.1$ s，$T_c = 0.3$ s。本章 MB 与 Mbit 使用十进制；MiB 为 $2^{20}$ bytes。代入得

[1] RAW 图片精修案例与条件预算。算例设原图 30 MB、成片 5 MB、图像处理耗时 0.3 秒。

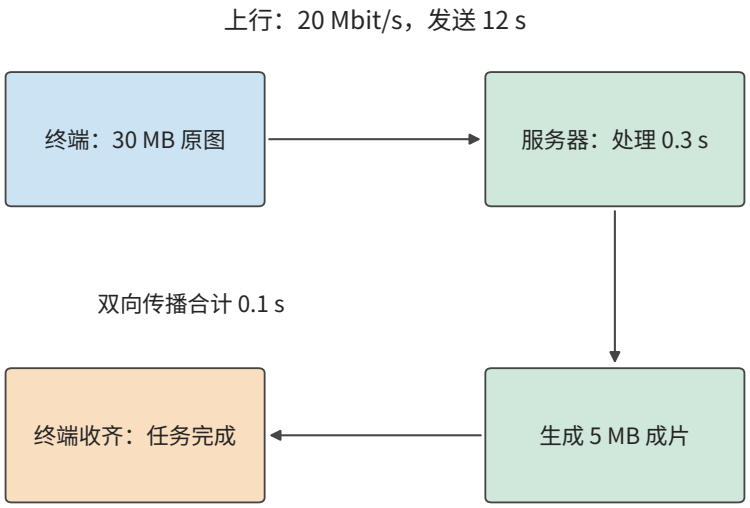



图 12-1：原图经上行到服务器，服务器收齐后处理，再经下行返回完整成片。实线箭头表示数据与处理顺序；连接已建立，双向传播合计 0.1 s。

$$T_{\text{serial}} = \frac{30 \times 8}{20} + 0.1 + 0.3 + \frac{5 \times 8}{100} = 12.8 \text{ s}.$$

上传占 12 秒，约为总时间的 94%。将处理阶段加速十倍，完成时间缩短至 12.53 秒，只节省 0.27 秒。即使处理瞬间完成，上传、传播和下载仍需 12.5 秒。这正是 Amdahl 定律所描述的限制：处理阶段只占总时间的约 2.3%，即使消除这部分耗时，总时间也只能缩短约 2.3%。

模型加速节省的时间有限，另一条路是缩短传输。把上行速率提高至 100 Mbit/s，上传时间降至 2.4 秒，整个任务只需 3.2 秒，速度提高到原来的四倍。另一种办法是保持上行速率不变，压缩输入。假设无损压缩将输入减半，额外的编解码共需 0.15 秒，则发送只需 6 秒，完整任务约为 7.0 秒。压缩之所以值得，是因为增加 0.15 秒编解码，节省了 6 秒传输。

设压缩后的输入大小为 $S'_u$，额外的编解码时间为 $T_e$，压缩节省的总时间为

$$\Delta T = \frac{S_u - S'_u}{B_u} - T_e.$$

令该式的收益为零，就能求出压缩恰好不再省时的上行速率。上行速率为 20 Mbit/s 时，少传 15 MB 可节省 6 秒；为 100 Mbit/s 时节省 1.2 秒；达到 800 Mbit/s 时只节省 0.15 秒，收益降至零。同一种压缩方法的运算量没有改变，改变的是发送这些字节需要多少时间。图 12-2 显示了这种关系。

再考虑各步骤能否重叠。假设原图分成三块后可以逐块独立处理，每块上传需 4 秒、处理需 0.1 秒；三块输出分别为 1、2、2 MB，回传依次需 0.08、0.16、0.16 秒。第一块到达服务器后，其处理和回传可以与第二块上传重叠；第二块同理。最后一块仍要等到第 12 秒才发完，之后再经过处理、回传和传播，完整成片约在 12.4 秒返回用户。与原来的 12.8 秒相比，前两块的处理与回传在后续上传期间就已完成，不再额外增加等待；但总

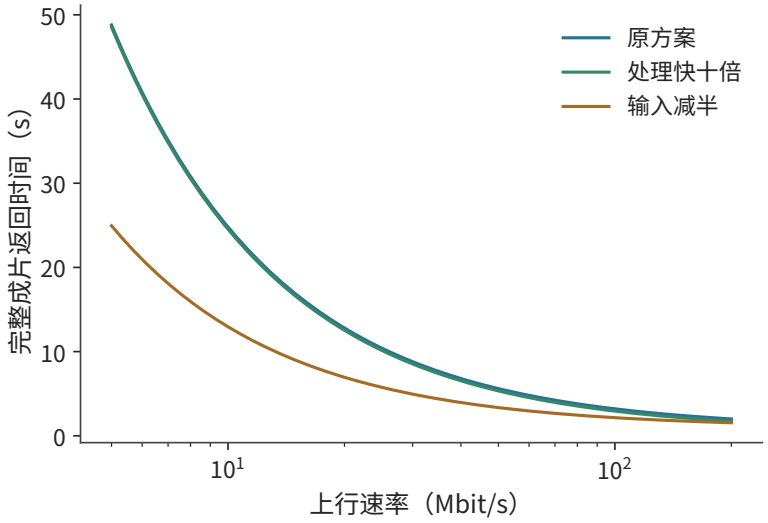



图 12-2：压缩和计算加速对任务完成时间的影响随上行速率而变化。本例采用原图 30 MB、成片 5 MB、下行 100 Mbit/s、RTT 0.1 秒，原处理时间 0.3 秒；压缩方案将输入减半并增加 0.15 秒编解码。三种方案的成片质量相同，连接已建立且各阶段串行。

计 12 秒的上传时间无法缩短。[2]

图 12-3 和图 12-4 用相同的时间轴比较两种安排。

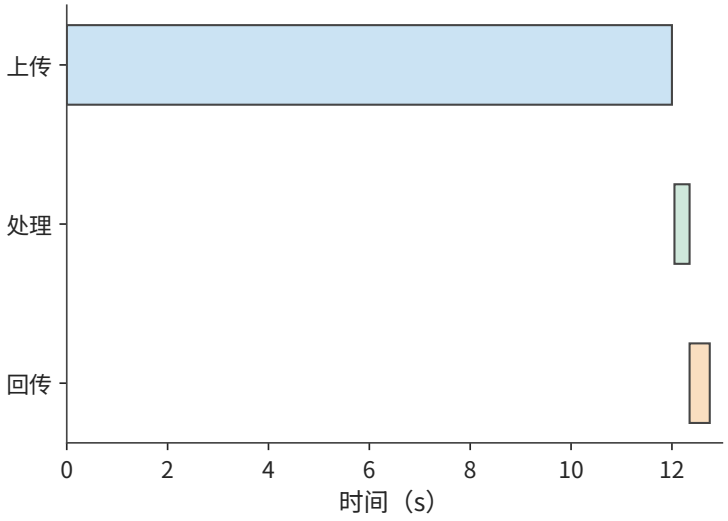



图 12-3：整图串行：30 MB 全部上传并传播到服务器后才处理，随后回传 5 MB。上行 20 Mbit/s、下行 100 Mbit/s、单向传播 0.05 s，任务于 12.8 s 完成。

如果处理需要全局曝光或跨块信息，就要等整张图到齐后才能开始处理，即使分块发送，整个任务仍需 12.8 秒。计算量和数据量决定各步骤所需时间，依赖关系决定各步骤能否同时执行。实时语音服务持续接收、处理和播放音频，更适合用流水线描述这种依赖。

[2]分三块的两种计算分别为 12.8 s 与 12.36 s，正文以约 0.4 s 差额解释重叠收益；额外预览另增编码与传输。来源：整图屏障、独立分块及额外预览。

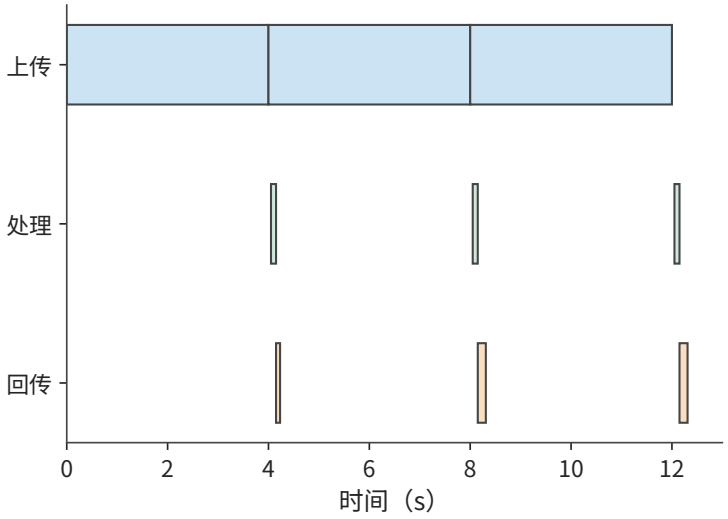



图 12-4：三块各上传 4 s，每块到达后独立处理 0.1 s，随后回传 1、2、2 MB。前两块的处理和回传与后续上传重叠，完整成片于 12.36 s 返回。

### 12.1.2 实时语音

自动语音识别（ASR）将音频转为文本，文本转语音（TTS）将文本转为可播放音频。流式服务可以一边接收输入，一边产生结果。以 TTS 为例，用户可以在整段音频生成完之前开始收听。因此，除了整段合成何时结束，还需要知道首段何时开始播放，以及后续播放是否连续。

要算出这些时刻，先把流水线写成逐块的递推关系。考虑一条输入块与输出块一一对应的音频处理流水线。采集端每 20 ms 产生一块音频，模型处理后将结果发出，播放器按顺序播放。第 $i$ 块输入在 $a_i$ 时刻就绪，模型处理耗时为 $m_i$，发送耗时为 $s_i$，单向传播为 $d_i$。处理器逐块处理音频，网络接口逐块发送结果，则

$$c_i = \max(a_i, c_{i-1}) + m_i, \qquad u_i = \max(c_i, u_{i-1}) + s_i, \qquad r_i = u_i + d_i.$$

其中，$c_i$、$u_i$、$r_i$ 分别表示处理完成、发送完成和到达的时刻。第一个最大值表示，处理本块既要等输入就绪，也要等处理器完成上一块；第二个最大值表示，发送本块既要等计算完成，也要等出口空闲。这两处等待分别来自数据依赖和资源竞争。

**例：网络延迟抖动为何会耗尽音频播放缓冲？**脉冲编码调制（PCM）逐次记录音频幅值采样；24 kHz 表示每秒 24,000 次采样，单声道表示一条采样通道，16-bit 表示每个样本占 2 字节。使用 24 kHz、单声道、16-bit PCM，每块 20 ms、960 bytes。处理一块需 12 ms，发送需 1 ms，通常传播耗时为 5 ms。第一块在 20 ms 采集完，于 38 ms 到达；缓冲 40 ms 后在 78 ms 播放（图 12-5）。第二块在 58 ms 到达，于 98 ms 接着播放。每块的处理时间比采集间隔短 8 ms，发送也只需 1 ms，因此处理和发送都能跟上采集速度。

把第三块的传播时间改为 50 ms，该块在 123 ms 到达，比原定的播放时刻 118 ms 晚了 5 ms。第四块虽然在 98 ms 已经到达，播放器仍须先播第三块，因而后续播放一起推迟（图 12-6）。[3]

把这条播放规则一般化：令每块音频的时长为 $\tau$，初始缓冲为 $J$。首块开始播放的时刻为 $p_0 = r_0 + J$，后续各块的开始时刻满足

[3]音频块时序计算，输入帧与输出音频块一一对应是该模型的简化，用于说明逐块处理与按序播放的依赖。

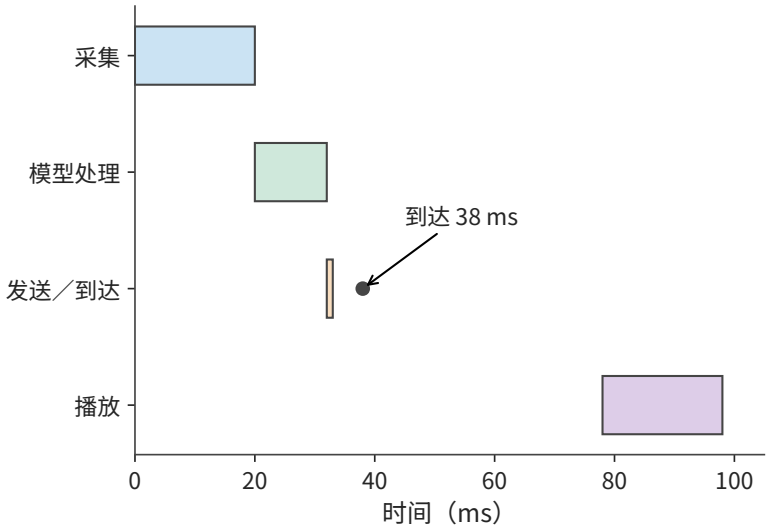



图 12-5：第一块音频从采集开始计时：20 ms 就绪，32 ms 处理完，33 ms 发完，38 ms 到达；初始缓冲 40 ms 后于 78 ms 播放。圆点标出到达，最后一行是播放设备的时钟。

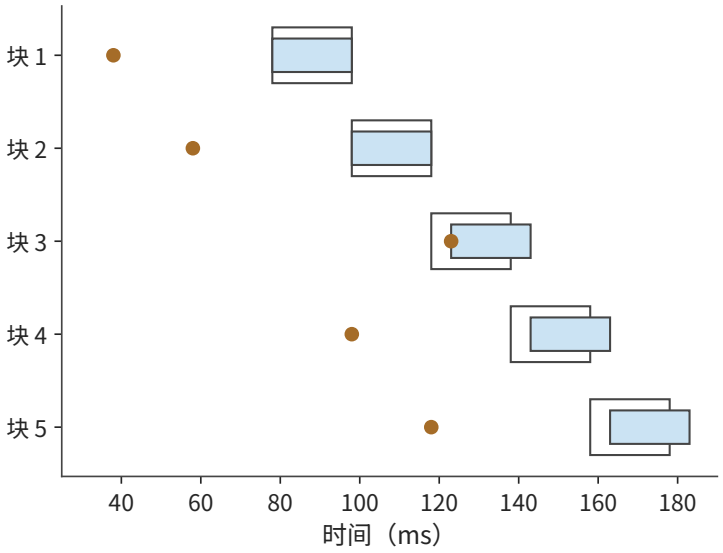



图 12-6：第三块晚到使后续各块的播放时间都推迟 5 ms。每块长 20 ms，模型处理 12 ms、发送 1 ms；通常传播耗时为 5 ms，第三块传播 50 ms，初始缓冲 40 ms。实线段表示实际播放，浅色轮廓表示原定播放区间，圆点表示到达；所有时间均从开始采集第一块起算。图为上述流水线的计算结果，播放器等待缺块而不丢弃。

$$p_i = \max(r_i, p_{i-1} + \tau).$$

这里取最大值，是因为播放器既要等待“本块到达”，也要等待“上一块播完”。当第一项更大时，差额就是新增停顿。把初始缓冲从 40 ms 增至 45 ms，第三块的计划播放时刻也推迟到 123 ms，恰好可以避免晚到 5 ms 造成的播放中断；代价是首段也要晚 5 ms 开始播放。增加初始缓冲会延长首次播放前的等待，但也能容忍更大的到达时间波动，使后续播放更连续。

长期传输速率不足会产生另一种停顿。为便于计算，取 16 kHz、单声道、16-bit PCM，播放所需的数据速率为 256 kbit/s，20 ms 对应 640 bytes。若网络长期只能以 130 kbit/s 传输音频，缓冲每秒净减少 126 kbit。初

始缓冲保存的 60 ms 音频包含 15,360 bits，大约 $15360/(256000-130000)\approx 0.12$ 秒就会耗尽。

把尚未播放的数据画成随时间下降的曲线，就能看出短时抖动与长期速率不足的区别。图 12-7 中，增加缓冲只抬高起点，斜率没有改变。

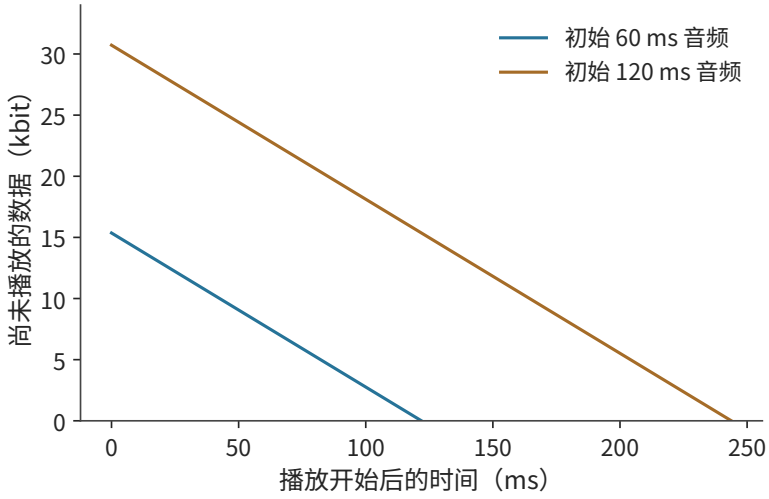



图 12-7：缓冲量等于初始数据量加累计接收量，再减累计播放量。PCM 播放需要 256 kbit/s，接收只有 130 kbit/s，两条直线的下降速率均为 126 kbit/s。初始缓冲分别包含 60 ms 和 120 ms 音频，约在 0.12 秒和 0.24 秒耗尽；图画到各自第一次耗尽为止。

增大缓冲只会按比例推迟耗尽时刻；提高传输速率或降低音频码率，才能让收到的数据足够播放。缓冲可以减轻短时延迟波动对播放的影响，长期传输速率不足则需要增加带宽或减少数据量。

### 12.1.3 Computer Use：通过界面操作完成任务

Computer Use（计算机界面操作）让程序像用户一样通过截图、点击和输入完成任务。程序在观察界面、模型判断、执行操作和再次观察之间循环。这类任务与音频流水线的关键区别是：只有上一轮操作完成、界面更新后，才能取得下一张有效截图。模型还没决定点击哪里时，就无法取得点击后的界面。因此，一轮中的等待会在任务中重复累积。

图 12-8 的返回箭头说明，这一任务难以像音频那样逐块重叠：后续工作必须等待操作产生新的界面。

以需要 30 轮操作的任务为例。使用原有服务时，每轮上传 0.8 MB 截图，上行 6.4 Mbit/s，上传需 1 秒；终端观测、操作与界面更新合计 0.3 秒，远端模型 2.0 秒，往返传播 0.2 秒。连接已建立，控制回复的发送时间取零。一轮为 $1+0.3+2.0+0.2=3.5$ 秒，30 轮共 105 秒。

**例：截图压缩节省的传输时间能否抵消额外操作轮次？**将截图压缩至 0.2 MB，额外编码需 30 ms，上传时间降为 0.25 秒，其余工作仍需 2.5 秒。于是

$$T'_{\text{round}}=0.25+0.03+2.5=2.78\ \text{s}.$$

每轮节省 0.72 秒，30 轮节省 21.6 秒，任务从 105 秒降至约 83 秒。

压缩还可能改变操作轮数。若文字变模糊使 Agent 多执行纠错操作，压缩后的总时间应写为 $2.78N'$，其中 $N'$ 是实际所需轮数。由 $2.78N'<105$ 得 $N'<37.8$，仍能缩短任务时间的最大轮数为 37；38 轮约需 106 秒。

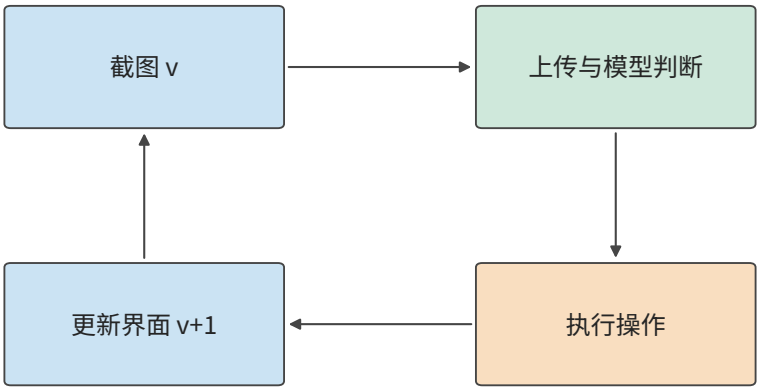



图 12-8：截图 Agent 的一轮执行。箭头表示先后依赖，方框宽度不表示耗时。底部返回箭头要经过操作执行与界面更新，才能取得下一轮截图。

压缩后的图像质量会影响纠错次数。该任务最多允许增加七轮操作；超过这一数量，纠错时间就会抵消压缩节省的时间。

截图版本也直接影响轮数。假设模型读到版本 $v$，随后界面变化，按旧截图作出的操作就可能出错，需要恢复界面并重新截图。把截图版本与动作编号绑定，让终端在界面变化时重新截图，可以避免这类无效工作。[4]

第 12.5 节将继续分析该截图任务：前十轮已完成，剩余二十轮需要选择执行位置，届时保持截图大小、终端工作和原路径参数不变，比较更快的模型服务能缩短多少时间。

### 12.1.4 本地、附近设备与云端的初步比较

第 12.1.1 至 12.1.3 节的分析可以归纳为三个层次，每层对应不同的计算对象：

| 分析层次 | 计算对象 | 已得到的判断 |
|---|---|---|
| 各步骤耗时 | 计算、发送和传播各需多久 | 原图上传占 12 秒，模型占 0.3 秒 |
| 执行依赖 | 哪些工作可以重叠，哪些必须等待 | 独立图片块可重叠，下一轮截图要等上一轮操作完成 |
| 实际执行 | 数据就绪后，计算和发送何时开始 | 音频即使平均传输速率足够，一次晚到仍可造成停顿 |

先用各步骤耗时与依赖关系判断远端执行能否更快。将整个任务交给远端服务时，节省的计算时间必须超过上传、下载、传播、排队和准备增加的时间。多轮任务的准备只需做一次，通信却每轮都会发生；连续播放的音频还要保证数据及时到达。模型放在哪里，会同时改变计算、通信和准备时间。

最短传播时间可以尽早排除距离过远的服务器。光在光纤中约以 $2 \times 10^8$ m/s 传播，单向路径长 10,000 km 时，往返传播至少需要约 100 ms。如果要求 50 ms 内收到远端应答，仅传播时间就已超过时限。使用更近的服务器，可以直接缩短这部分等待。

[4]笔者的材料与技术判断。AOI 与 Latent Bridge 用于说明应观察哪些界面信息，以及设备之间应传递什么数据。

以上比较把端侧当作一个执行位置，只计算端侧与远端的时间差；端侧设备本身能算多快、能容纳什么，尚未展开。第 12.1.5 节先定量分析端侧的执行资源，第 12.2 节再决定传什么：把整项任务交给远端处理时传图片，在端侧编码后传特征，迁移会话时则传状态；传输对象改变后，数据量和所需时间也随之改变。

### 12.1.5 端侧执行资源与本地部署

第 12.5 节比较部署方案时要用到“端侧每轮模型计算 2.9 秒”。要说明这一数值的来源和成立条件，就要把端侧设备当作执行资源定量分析：内存带宽决定每生成一个 token 至少要多久，内存容量决定能容纳什么模型、留下多长上下文，能耗与电池决定能持续多久。

**手机作为执行资源：带宽先给出每秒 token 的上限。**旗舰手机的 SoC 使用 LPDDR5X 内存，这是面向手机等移动设备的低功耗 DRAM 标准。Snapdragon 8 Elite 的产品简介给出最高 5,300 MHz、最大 24 GB 容量，但未给出总线宽度；LPDDR5X 每个时钟周期传两次数据，5,300 MHz 即每引脚 10.6 Gbit/s，略低于 LPDDR5X 器件 10.7 Gbit/s 的最高速率档；JEDEC 标准器件为 x16 单通道。[5] 按 SoC 支持的速率，一条 x16 通道的带宽为 $16 \times 10.6/8 = 21.2$ GB/s；按四条 x16 通道计，合计 84.8 GB/s。[6]

带宽给定后，每步要多久取决于这一步读取多少数据。固定工作量沿用第 4.8.2 节的算例：Qwen3-8B、BF16、单请求、8K 上下文，一步 decode 读取权重 15.14 GB、KV 1.21 GB，合计约 16.345 GB。[7] 只把权重读一遍需要

$$\frac{15.14\ \text{GB}}{84.8\ \text{GB/s}} \approx 0.179\ \text{s},$$

即 decode 上限约 5.6 token/s；计入 KV 后每步约 0.193 s，上限约 5.2 token/s。量化直接减少每步读取的字节数：按第 8.4 节的 q4_0 分组格式，每 32 个 BF16 值从 64 bytes 变为 18 bytes，权重每步读取降至约 $15.14 \times 18/64 \approx 4.26$ GB，每步合计约 5.47 GB、64.4 ms，上限约 15.5 token/s。MELTing Point 在 iPhone 14 Pro 上实测 Zephyr-3B q4_k 为 14.8 token/s、Llama-2 7B q3_k 为 6.0 token/s（q4_k、q3_k 是与第 8.4.1 节 Q2_K 同属 GGML 系列的 4-bit、3-bit 分组量化格式），与这里推导的量级一致。[8]

**容量决定能容纳什么模型、留下多长上下文。**沿用第 2.6.2 节的容量公式：可用内存减去权重与固定预留，剩余部分除以每条请求的状态大小。手机内存取 Micron LPDDR5X 的 6、12、24 GB 三种容量选项，再加旗舰手机常用的 16 GB，系统与工作区预留 4 GB。[9] Qwen3-8B 的 BF16 权重共 16.38 GB，比每步读取的 15.14 GB 多出约 1.24 GB 的词嵌入表：decode 每步只按 token 查一行，整张表却要常驻内存。按常驻口径，只有 24 GB 一档能容纳，扣除预留后剩约 3.62 GB，仅够两条 8K 请求；量化到 q4_0 后约 4.61 GB：

[5] Snapdragon 8 Elite 产品简介与第五代简介：支持 LPDDR5X 最高 5,300 MHz、容量最大 24 GB，未给总线宽度。Micron LPDDR5X 产品页：最高速率档 10.7 Gbit/s，JEDEC 标准为 x16 单通道器件，1γ 工艺新增 6 / 12 / 24 GB 容量选项，另有面向旗舰手机的 16 GB 器件；Samsung LPDDR5X 产品页同为最高 10.7 Gbit/s。Apple iPhone 16 Pro 规格页不公布内存容量与速率。

[6] 端侧总线与能耗分账：通道数按四条 x16 计（`declared_phone_channels_x16=4`；SoC 简介未给总线宽度），每引脚取 SoC 支持的 10.6 Gbit/s，合计 84.8 GB/s；15,136,819,200 bytes 权重与 1,207,959,552 bytes KV 直接读取单请求 8K decode 结果。

[7] 端侧总线与能耗分账：通道数按四条 x16 计（`declared_phone_channels_x16=4`；SoC 简介未给总线宽度），每引脚取 SoC 支持的 10.6 Gbit/s，合计 84.8 GB/s；15,136,819,200 bytes 权重与 1,207,959,552 bytes KV 直接读取单请求 8K decode 结果。

[8] MELTing Point: Mobile Evaluation of Language Transformers，arXiv 2403.12844，§5.2.2 与附录表 5：每 token 0.16 / 0.20 / 0.21 mWh；iPhone 14 Pro 持续功率最高 13.8 W、瞬时超过 18 W，Galaxy S23 持续低于 8.5 W；iPhone 14 Pro 上 Zephyr-3B q4_k 为 14.8 token/s、Llama-2 7B q3_k 为 6.0 token/s；一次充电可运行 490.05–590.93 个提示（S23 为 542.78）。提取值逐行核对见数值提取表。

[9] Snapdragon 8 Elite 产品简介与第五代简介：支持 LPDDR5X 最高 5,300 MHz、容量最大 24 GB，未给总线宽度。Micron LPDDR5X 产品页：最高速率档 10.7 Gbit/s，JEDEC 标准为 x16 单通道器件，1γ 工艺新增 6 / 12 / 24 GB 容量选项，另有面向旗舰手机的 16 GB 器件；Samsung LPDDR5X 产品页同为最高 10.7 Gbit/s。Apple iPhone 16 Pro 规格页不公布内存容量与速率。

| 手机内存 | 扣除预留与 4.61 GB 权重后 | 可容纳的请求与上下文 |
|---|---|---|
| 6 GB | 约 −2.61 GB | 无法容纳 8B；3B 级 q4 模型约 1.7 GB，可以容纳 |
| 12 GB | 约 3.39 GB | 两条 8K 请求，或单条约 2.3 万 token |
| 16 GB | 约 7.39 GB | 六条 8K 请求，或单条约 5.0 万 token |
| 24 GB | 约 15.39 GB | 十二条 8K 请求，或单条约 10.4 万 token |

一条 8K 请求的 BF16 KV 为 1.21 GB，每 token 占 147,456 bytes，随上下文长度线性增长；容纳模型之后，剩余内存直接决定上下文长度。16 GB 一档也是 8-bit 量化的分界：q8_0 权重约 8.70 GB，扣除预留后剩约 3.30 GB，可放两条 8K 请求；更小内存的手机无法容纳 8-bit 的 8B 模型。

**能耗与续航是端侧特有的代价。**第 4.1.3 节的能耗分账给出 H100 上单请求 8K decode 每 token 约 0.534 J 的量级估计，只含数据搬移与矩阵计算。手机上的实测则包含整机功耗：MELTing Point 测得每 token 0.16–0.21 mWh，即 0.576–0.756 J；持续功率最高 13.8 W，瞬时超过 18 W。吞吐与能耗的实测值可以互相核对：14.8 token/s 乘以每 token 0.576 J 约为 8.5 W，在 13.8 W 的持续功率之内。按这组测量推算，一次充电可完成约 490–590 个提示的推理。[10] 因此端侧的代价不只是时间和费用：持续生成的速率受功率上限约束，累计的工作量受电池容量约束。

**本地部署的三档设备。**比手机高的两档可以直接取自第 4.8.2 节，那里已用同一模型列出 11 款候选设备的 decode / prefill 时间下界：M3 Ultra 的统一内存为 819 GB/s、256 / 512 GB，每步读取下界 19.96 ms，约 50.1 token/s，容量足以容纳 4-bit 的 235B MoE（第 4.6.3 节）；RTX PRO 6000 为 1,792 GB/s、96 GB，每步 9.12 ms，约 109.6 token/s。图 12-9 把三档设备放进同一条时间轴。

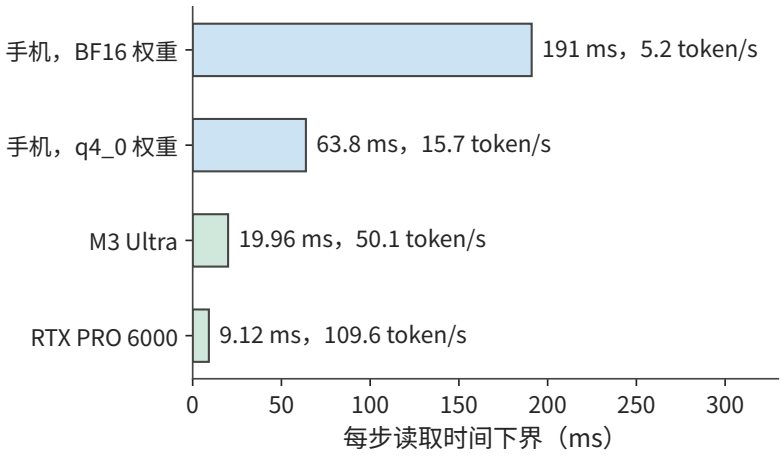



图 12-9：三档本地设备读一遍 Qwen3-8B 单请求 8K decode 主要载荷（16.345 GB）的时间下界。手机按四条 x16 LPDDR5X 通道计，合计 84.8 GB/s，BF16 与 q4_0 权重分别成行；M3 Ultra 与 RTX PRO 6000 取自第 4.8.2 节的设备表。柱端标注对应的 token/s 上限。

**运行时各解决一层问题。**设备给出带宽与容量下界，实际执行还取决于运行时。llama.cpp 解决权重如何存储、如何逐块执行：GGUF（llama.cpp 使用的模型文件格式）的分组量化布局决定加载后的容量与每步读取量，第 8.4.1 节已算过 Q2_K 每组 84 bytes、平均每值 2.625 bit。Ollama 解决模型的分发与本地服务封装：对应

[10] MELTing Point: Mobile Evaluation of Language Transformers，arXiv 2403.12844，§5.2.2 与附录表 5：每 token 0.16 / 0.20 / 0.21 mWh；iPhone 14 Pro 持续功率最高 13.8 W、瞬时超过 18 W，Galaxy S23 持续低于 8.5 W；iPhone 14 Pro 上 Zephyr-3B q4_k 为 14.8 token/s、Llama-2 7B q3_k 为 6.0 token/s；一次充电可运行 490.05–590.93 个提示（S23 为 542.78）。提取值逐行核对见数值提取表。

第 12.2.3 节“一次准备、多次复用”的权衡，Ollama 节省的是部署准备时间，而不是每步执行时间。MLX 面向 Apple 统一内存（第 4.6.3 节）：CPU 与 GPU 共用同一内存池，免去两者之间的数据拷贝，模型容量上限即整机内存。Unsloth 面向端侧与单卡微调：把训练状态的容量预算压进一张卡或一台整机，量化格式仍来自第 8.4 节。这四个运行时都不改变以上推导的带宽与容量下界，只决定实际执行能在多大程度上接近下界。衡量这段距离的统一尺度，是实测耗时与下界的倍数。第 12.5.2 节将用实验 8-1 的记录说明，RTX PRO 6000 上一步 decode 的实测时间是读取下界的 2.83 倍，多出的主要是每步的固定开销；按第 1.3.4 节的判据，这是可以减少的系统开销，不是设备能力不足。

其中，Unsloth 节省的容量可以用第 10.1.2 节的每参数字节数直接算出。全参数混合精度 Adam 每参数 16 bytes：BF16 权重与梯度各 2 bytes，FP32 主权重与两份矩各 4 bytes。LoRA 冻结基础权重，只为新增的低秩 adapter 保留梯度与优化器状态：Qwen3-8B 在 Q、V 投影上的 rank-16 BF16 adapter 为 14.6 MiB（第 8 章末尾的多 LoRA 算例），约 765 万参数。adapter 的权重、梯度、主权重与两份矩合计每参数 16 bytes，约 0.12 GB；再加上 16.38 GB 的 BF16 基础权重，共约 16.5 GB。只按常驻状态比较，激活与临时缓冲另计：

| 显存 | 全参数混合精度 Adam，16 bytes / 参数 | BF16 基础权重 + rank-16 Q / V LoRA |
|---|---|---|
| 24 GB（RTX 4090） | 约 1.5B 参数 | 约 12B 参数；Qwen3-8B 需约 16.5 GB |
| 96 GB（RTX PRO 6000） | 约 6B 参数 | 约 48B 参数 |

一张 RTX 4090 无法容纳 Qwen3-8B 全参数训练的 131.1 GB 状态，却能容纳其 LoRA 训练；Unsloth 节省的是这份状态，而非每步读取权重的带宽下界。

**翻转条件：何时端侧足够，何时必须上云。**三个下界各给出一个翻转条件。带宽：每轮生成 $G$ 个 token 时，端侧模型时间下界为 $G$ 乘以每步时间。第 12.5 节的“端侧每轮模型计算 2.9 秒”就是这样得到的：按 q4_0 的 15.5 token/s，2.9 秒对应每轮生成约 45 个 token；按 BF16 的 5.2 token/s 则只对应约 15 个。反过来从期限推预算：45 秒期限摊到 20 轮，每轮 2.25 秒，扣除 0.3 秒终端工作后剩约 1.95 秒，手机 q4_0 下每轮最多生成约 30 个 token；需求超过该数量，就要换带宽更高的设备或上云。每轮 45 个 token 的需求已超出预算，所以第 12.5.2 节的端侧方案需要 64.0 秒，超过期限。容量：权重加所需上下文超过本档可用内存时，本档不可行——12 GB 手机无法容纳 BF16 的 8B 模型，手机一档无法容纳 235B 模型。能耗：连续生成把电池变成约束，一次充电对应约 490–590 个提示，批量或长时间生成应交给接通电源的设备。这三个条件补全第 12.1.4 节的初步比较：那里比较各方案的完成时间，这里先排除端侧根本不可行的情形。练习 12-9 用同一组公式复算手机一档的带宽、容量与功率上限。

这里使用的是硬件下界，因此可行性判断并不对称：若时间下界已经超过期限，该方案一定不可行；若时间下界低于期限，只能说明该方案尚未被排除。软件开销、未达到峰值的有效带宽和持续负载下的降频都会使实测时间更长，最终是否达标仍需用目标设备上的测量结果校准。

## 12.2 跨设备分工

### 12.2.1 完整模型调用与分阶段执行

同一张截图，可以先发给服务器，由服务器转成模型使用的特征；也可以先在终端生成特征，再把特征发送到服务器。两条路径中，跨网络的对象已经不同。第 12.1.1 节的图片算例说明，文件传输慢会使计算等待；把一部分计算移到端侧以后，真正需要发送的对象也随之改变。

将图片或音频转换为模型所用数值特征的子网络，称为编码器。把整个模型放到远端，需要上传输入并下载结果；把编码器留在端侧，需要上传编码器的输出。图片文件和特征张量的数据量由各自的表示方式决定，端

侧先做了计算，并不保证上传量随之减少。

以截图 Agent 为例，图片压缩用较少的字节表示纹理和重复区域；视觉编码器却要把图片展开成每个视觉 token 的通道值，供语言模型读取。前者为了紧凑存储，后者为了后续计算。

比较两种方案：一种先上传图片，再在远端编码；另一种先在本地编码，再上传特征。两者后续的语言模型计算相同，求两种方案的总耗时之差时，这部分正好抵消。因此，只需比较本地与远端的编码时间，以及图片与特征的发送时间。

### 12.2.2 端侧编码的传输量与计算开销

先沿图 12-10 和图 12-11 中的箭头确认两种方案分别传什么，再比较传输量和节省的远端计算时间。

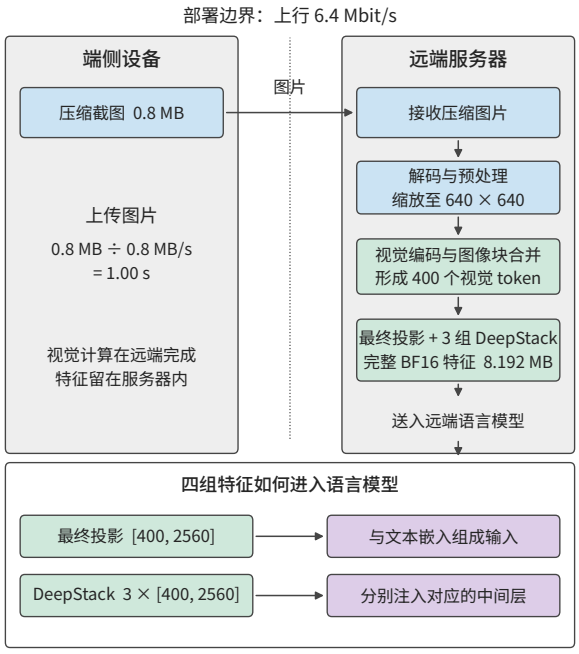



图 12-10：远端编码的执行路径。端侧上传 0.8 MB 压缩截图，远端依次完成解码与预处理、视觉编码和特征接入；最终投影与三组 DeepStack 特征均留在服务器内。6.4 Mbit/s 上行的图片发送时间为 1 秒，编码和排队时间另计。

**例：上传视觉特征为何可能比上传原图更慢？**沿用 0.8 MB 截图与 6.4 Mbit/s 上行。视觉编码采用 Qwen3-VL-4B 的固定配置：预处理后的图像尺寸为 $640 \times 640$，图像块合并后得到 400 个视觉 token；这里的视觉 token 是表示图片内容的特征向量，不是文本词元，也不是原图的单个像素。最终投影的每个特征向量有 2560 个分量，另有三组同宽 DeepStack 特征（第 3.3.1 节），这些分支输出要一起传送。每个视觉 token 共需保存 $4 \times 2560 = 10240$ 个 BF16 数，完整输出为

$$S_E = 400 \times 10240 \times 2 = 8\,192\,000 \text{ bytes} \approx 7.8 \text{ MiB}.$$

四组特征分别输入语言模型的不同层。如果只传最终投影，就会漏掉另外三组特征。完整特征约 8.2 MB，是压缩图片的约十倍；在原有上行链路上，发送时间从 1 秒增加到约 10.2 秒（图 12-12）。[11]

把这一比较写成一般形式：将本地编码时间记为 $T_{E,l}$，远端编码时间记为 $T_{E,r}$，本地编码方案额外的序列化和转换时间记为 $T_x$。两种方案的时间差为

[11] 固定视觉形状与完整 EC / KV 计算、多模态阶段放置案例。固定视觉变体、DeepStack 及语言状态分别计量。

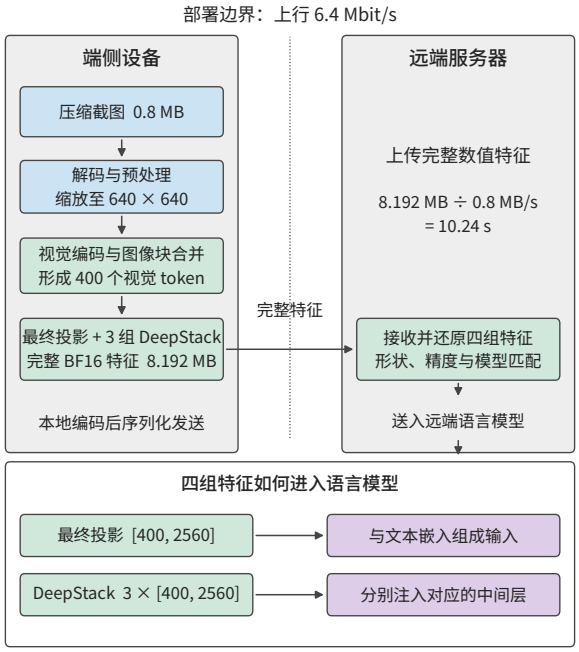



图 12-11：端侧编码的执行路径。预处理和视觉编码移到终端，跨网络的对象变为最终投影与三组 DeepStack 的完整 BF16 特征，共 8.192 MB；远端接收后仍按相同方式接入语言模型。相同上行的特征发送时间为 10.24 秒，本地编码、序列化和转换时间另计。图中数值采用下文的 Qwen3-VL-4B 固定配置。

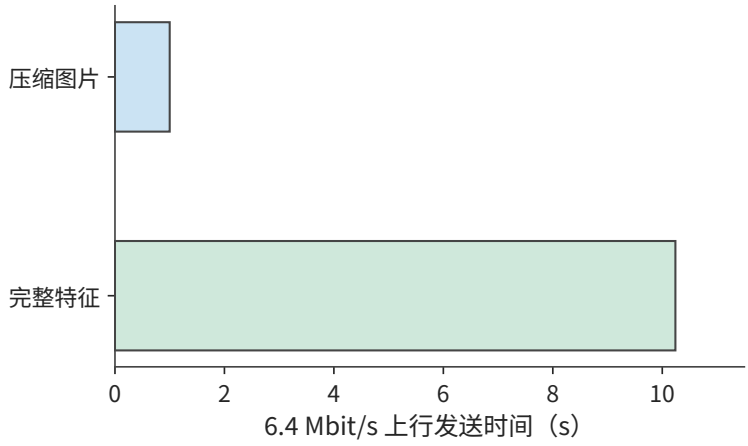



图 12-12：同一图片的完整视觉特征比压缩图片需要更多传输时间。压缩图片为 0.8 MB；Qwen3-VL-4B 在预处理后 640×640 输入上产生 400 个视觉 token，最终投影及三组 DeepStack 合计 [400,10240] BF16，共 8,192,000 bytes。上行速率为 6.4 Mbit/s，柱长表示发送时间；编码、排队和转换时间另计。两种方案处理同一张图片，完成相同的视觉理解任务。

$$\Delta T = T_{E,l} - T_{E,r} + T_x + \frac{S_E - S_I}{B_u}.$$

$\Delta T < 0$ 时端侧编码更快。设终端是一台装 RTX 4090 的台式机，远端是一张 H100 SXM。编码一张图需要 1.31 TFLOPs 矩阵运算，按两者的 BF16 稠密峰值 165.2 与 989.4 TFLOP/s 计，编码时间下界分别约为 7.9 ms 和 1.3 ms。取 $T_x = 0$，本地编码方案约需 10.25 秒，远端编码方案约需 1.00 秒。本地方案在计算和传输上都更慢：本地计算多约 6.6 ms，特征发送多约 9.2 秒。[12]

[12]编码位置与端边云三档的设备计算：视觉编码的矩阵工作量取自单图视觉编码结果，峰值取自硬件参数表；ConnectX-7 数据手册：单端口速率至 400 Gbit/s。

链路变快后，发送特征比发送图片多出的时间会减少。若编码器与语言模型分处同一机房的两台服务器，经一个 400 Gbit/s 的 ConnectX-7 端口相连（每方向 50 GB/s），图片与特征的发送时间只相差约 0.15 ms，已小于上面两张卡的编码时间之差。这时，编码与排队合计只要减少 0.15 ms 以上，另一台机器上的编码方案就更快；在原有的慢速上行链路上，则要节省超过 9 秒。决定编码器放在哪里时，慢链路上先比传输量，快链路上则更要比编码和排队时间。

多轮任务中，重复出现的图片可以复用 EC，进一步减少编码次数。相同图片再次出现时，缓存命中就省去一次编码；界面内容改变后则要重新编码。一组使用相同尺寸图片的 CPU 实验观察到了同样的行为。[13] 计算多轮任务的编码总时间时，可以用单次编码时间乘以缓存未命中的次数。此前轮次留下的缓存由此也计入比较。

视觉特征只能省去视觉编码；如果希望复用语言模型已经算出的结果，就要保存语言模型处理前缀时逐层产生的 KV。该配置每个语言模型输入 token 需要 144 KiB 的逻辑 KV 存储空间，400 个视觉 token 约为 56.3 MiB。图片、约 7.8 MiB 的视觉特征、约 56.3 MiB 的视觉 token KV，分别对应不同重算起点（图 12-13）。缓存越接近模型的最终输出，再次使用时节省的计算越多，需要传送的数据也可能越大。第 12.2.3 节比较传送一次状态需要多久，以及后续使用这些状态能节省多少时间。

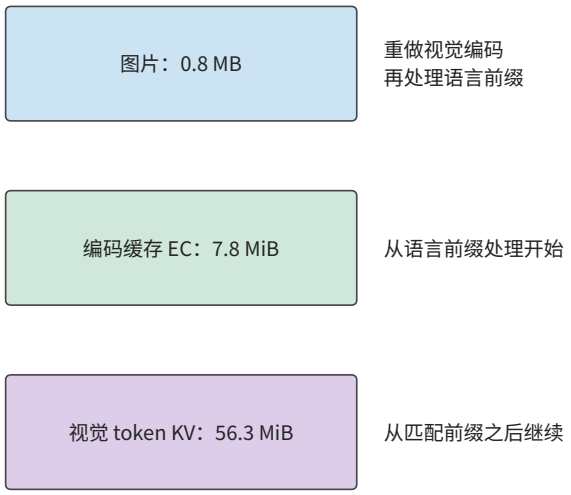



图 12-13：三类缓存内容对应三个恢复计算的起点。EC 省去视觉编码；匹配模型权重、前缀与 token 位置索引的 KV 进一步省去已处理前缀的语言模型计算。容量对应正文固定视觉配置。

### 12.2.3 会话迁移的准备、复用与恢复

会话迁移将缓存和任务进度传到另一台设备，使接收设备可以接着执行。目的设备已有兼容权重时，需要传送的是编码缓存、语言 KV、任务记录和恢复信息。确认两端的状态格式兼容后，就可以比较迁移前的准备时间与迁移后每轮节省的时间。

设迁移所需的准备时间为 $M$，新设备每轮节省的时间为 $\Delta t > 0$，任务还剩 $N$ 轮。继续在原设备执行与迁移到新设备执行的时间差为

[13] Qwen3-VL-8B CPU 编码与本机 TCP 传输实验。输入为 256×256 的合成测试图像，完整 EC 约 2 MiB，PNG 约 3.4 KiB；8 次传输前后的数据逐位一致。编码在传输计时开始前完成，所列时间对应特征传输阶段。

$$G(N) = N\Delta t - M.$$

$G(N) > 0$ 表示迁移获益。每增加一轮，收益增加 $\Delta t$；准备时间 $M$ 则使整条收益曲线向下移动。所以同一迁移对短任务不划算，对长会话却值得。

**例：至少还需交互多少轮，状态迁移才能节省时间？** 迁移需通过 80 Mbit/s 的链路传输 64 MiB 状态，再花 1 秒恢复。迁移准备的总耗时为

$$M = \frac{64 \times 2^{20} \times 8}{80 \times 10^{6}} + 1 \approx 7.7 \text{ s}.$$

每轮快 0.4 秒时，10 轮净损失约 3.7 秒，20 轮净省约 0.3 秒，40 轮净省约 8.3 秒。用未经四舍五入的 $M$ 求 $N > M/0.4$，得到最少 20 轮。这一轮数说明何时开始抵消迁移开销，净收益大小则说明值得为这次切换承担多少额外成本。

图 12-14 中，迁移刚开始时净节省为负，之后每完成一轮就上升 0.4 秒。回本轮数对应曲线越过零线的位置。

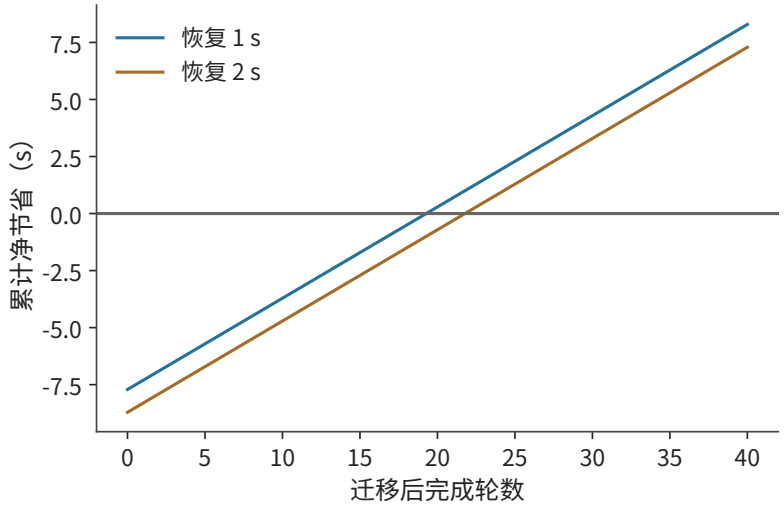



图 12-14：迁移 64 MiB 状态，链路 80 Mbit/s，迁移后每轮节省 0.4 秒。两条曲线分别取恢复时间 1 秒和 2 秒，净节省为 N×0.4 减去传输和恢复时间。圆点标出首次获益的整数轮数；零线以上表示迁移更快，首次获益分别在第 20、22 轮。

例如，恢复时间再增加 1 秒，20 轮的净收益立即变成约负 0.7 秒，回本轮数增至 22。若预计后续还有 40 轮，新增这 1 秒之后仍可节省约 7.3 秒。部署系统因此需要预测剩余会话长度，并为额外的传输和恢复开销留出时间。

曲线中的“恢复时间”包括新设备继续执行前所需的准备工作。新设备除了要将缓存数据加载到内存，还要确定任务已经执行到哪一步。语言 KV 绑定模型、前缀和 token 位置索引；截图 Agent 的进度由已经执行的操作决定。一次点击已经改变了界面，即使应答丢失，也应恢复到点击后的状态。操作编号和提交记录让目的设备可以查到既有结果，从下一轮继续。保存这些记录可以避免重复执行操作，就像复用缓存可以避免重复计算。

连接预热、图编译和模型加载也可用 $N\Delta t - M$ 分析：将准备成本放入 $M$，将后续每次减少的等待放入 $\Delta t$。[14] 这些准备是否值得，取决于后续调用节省的时间能否抵消一次性的准备时间。

[14] FlexSP / llm.npu 论文与实现资料。

### 12.2.4 模型内部切分的通信边界

会话迁移只在更换设备时传送一次状态。另一种分工方式是让多台设备一直共同执行模型，这时通信会在运行中反复发生。按阶段切分模型时，每个输入通常只需在相邻阶段之间传输一次；采用张量并行时，每层计算都要通信。链路从卡间互联换成局域网后，频繁同步会把微小的单次延迟累积成主要成本。

以 36 层 Qwen3-8B 在两台设备上做张量并行（TP=2）为例，考察每层注意力输出和 FFN 输出的两次归约；归约把两台设备的局部结果相加，使后续计算得到完整输出。采用两阶段环算法（第 7 章先做 ReduceScatter、再做 AllGather 的环形实现）时，每次归约有两次启动，每个阶段的启动耗时为 $\alpha$，所以生成一个 token 时，这些归约的启动时间合计为

$$T_{\text{start}} = 36 \times 2 \times 2\alpha = 144\alpha.$$

当 $\alpha = 2\ \mu\text{s}$ 时，这部分约为 0.29 ms。若两台设备改经第 12.4.1 节那种无聚合的 54 Mbit/s Wi-Fi 相连，每个阶段至少要完成一次短帧交换；按该节承载 ACK 的短帧交换 134 µs 计，144 次启动就要约 19.3 ms，是 RTX PRO 6000 一步 decode 读取下界 9.12 ms 的两倍多。即使载荷极小，通信启动仍需时间；而下一层需要本层归约结果，自回归的下一个 token 又需要当前 token 的输出。由于这些步骤必须依次执行，每次通信的启动时间都会累加到单个 token 的完成时间中。[15]

图 12-15 和图 12-16 依次展开一层的注意力输出与前馈输出，两者沿执行方向各经过一次归约后，才能开始下一层。单次等待虽短，重复的次数却由模型结构决定。

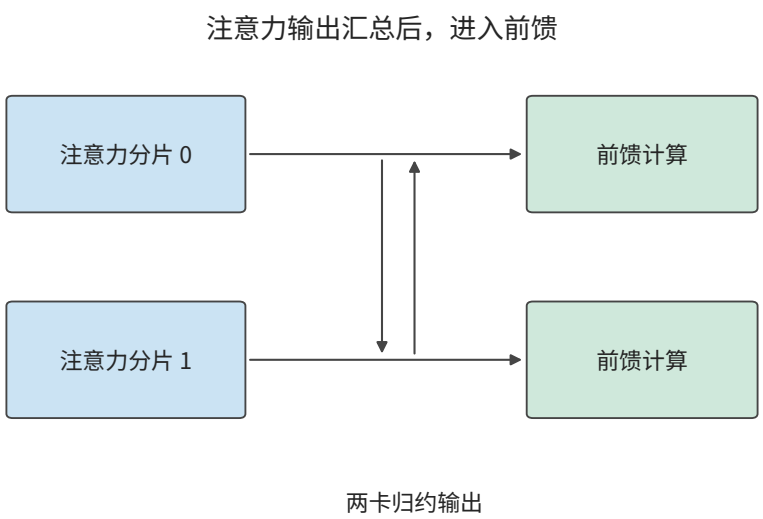



图 12-15：张量并行的一次注意力输出归约：两卡先分别计算，再交换并求和，取得完整输出后进入前馈。横向箭头表示执行顺序，中间纵向箭头表示两卡通信。

与逐层同步的张量并行不同，流水线并行把层分成连续的阶段，通信集中在相邻阶段之间的激活传输。对单个请求，一个 token 仍需依次经过各个阶段；对多个独立请求，不同阶段可以同时处理不同请求。由此得到两种收益：减少通信次数可以缩短单个请求的等待，让不同阶段同时处理请求可以提高总吞吐量。选择张量并行还是流水线并行时，先画出一个 token 的依赖，再安排多个请求占用各段资源，就能分别计算这两项变化。

[15] Qwen3-8B TP=2、PP=4 的逐操作通信量计算。36 层各有两次输出归约，共产生 72 次通信启动；嵌入、阶段激活、logits 与 token 返回还会增加通信。

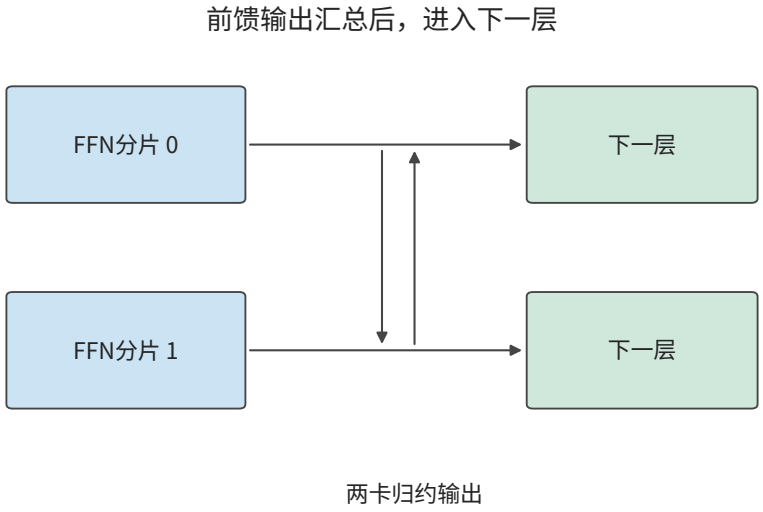



图 12-16：前馈网络也分别计算局部输出，再做两卡归约。完整前馈结果就绪后才能进入下一层，这一结构在 36 层中重复。

确定模型分工后，传什么、传多少次以及哪些计算要等待传输都已明确。第 12.3 节考察实际发送过程：数据准备好之后，连接是否建立、窗口是否允许发送、确认是否及时返回。

## 12.3 广域网传输中的额外开销与等待

### 12.3.1 连接建立与复用

第 12.1 节假设连接已经建立，链路以恒定速率传输数据。实际请求首先要建立连接和安全上下文，即通信双方为认证与加密维护的密钥、身份和会话状态；随后取得发送额度，最后等待接收方把数据交给应用。将这些等待加入第 12.1 节的时间模型，就能分析实际执行过程：数据虽然已经准备好，却未必能立即计算或发送。

这些等待涉及以下协议。HTTP 是组织请求和响应的应用层协议。HTTP/1.1 和 HTTP/2 通常使用 TCP（提供可靠有序字节流的传输协议）与 TLS（建立认证与加密上下文的安全协议），HTTP/3 使用 QUIC（在 UDP 数据报上提供可靠多流传输和连接管理的协议）。UDP 向应用提供独立的数据报，不自行负责重传与排序。QUIC 将传输握手与 TLS 1.3 结合，减少分别建立传输连接和安全上下文所需的往返次数。会话恢复时，客户端还可以根据保存的安全状态发送早期数据，即 0-RTT 数据。服务端接受后继续处理，拒绝后由客户端重发；服务端根据操作编号识别重试，并返回已保存的结果，同一操作就不会重复执行。[16]

把建链开销代入第 12.1 节的截图任务：沿用 30 轮截图任务的 200 ms RTT，设建链额外需要两次往返。每轮重建累计需 12 秒，只在第一轮建立则需 0.4 秒，节省 11.6 秒。连接复用之所以有效，是因为后续 29 轮共享了第一轮建立的状态。第 12.1 节的 105 秒是在连接已经建立的条件下计算的；加上首次建链后为 105.4 秒，每轮重建则为 117 秒。

连接建立是一次性开销，传输则每轮都要发生。请求频繁时，保持连接可以省去下一轮的建链时间；两次请求间隔较长时，空闲连接仍会占用内存等资源。关闭连接可以释放这些资源，但下次调用需要重新建链。与会话迁移一样，这里也要比较一次准备与后续多次使用的开销。

[16]RFC 9000：QUIC 传输，§2、§7、§9、§13；RFC 9001：TLS；RFC 9114：HTTP/3；RFC 9221：不可靠 Datagram。HTTP/3 规定了如何在 QUIC 上传输 HTTP 请求和响应。

### 12.3.2 窗口、反馈与有效吞吐

复用连接省去了建链等待，连续发送数据还需要足够的发送窗口。连接建立以后，发送方也必须限制在途数据。拥塞窗口限制已经发送但尚未确认的数据量；接收方的流量控制根据自己的缓冲空间，限制发送方还能发送多少数据。ACK 到达后，发送方可以继续发送；接收方从缓冲区读取数据后，也会通知发送方新增的可用空间。达到发送上限时，即使链路空闲，发送方仍要等待这些通知。

在途量是已经发出、尚未得到确认的数据字节数。若允许在途量为 $W$，往返时间为 $R$，每个往返周期最多传送约一个窗口的数据，因此

$$B_{\text{useful}} \le \min\left(B_{\text{path}}, \frac{W}{R}\right).$$

式中 $B_{\text{path}}$ 是路径可提供的载荷速率，$B_{\text{useful}}$ 是实际有效吞吐。第 12.1.1 节带宽 20 Mbit/s、RTT 100 ms 的图片传输路径，要持续填满链路，需要约 250 KB 在途数据。若只有 64 KB 额度，速率上限为 $64000/0.1 = 640000$ bytes/s，约 5.1 Mbit/s，仅发送这 30 MB 数据就至少需要约 47 秒。链路每秒本可以传输更多数据，发送方却要等下一批确认到来才能继续发送；只有扩大可用窗口，才能提高这时的实际传输速率。

单独考虑整批确认的模型：接收方收齐 64 KB 后才确认，从这批发完到确认返回固定为 100 ms。因此完整周期包含 25.6 ms 发送和 100 ms 反馈，共 125.6 ms。图 12-17 画出了这段等待。蓝色时段确实在发送，浅色时段链路虽然空闲，发送方却因为窗口已满而不能继续。物理带宽提高，只能缩短蓝色部分。

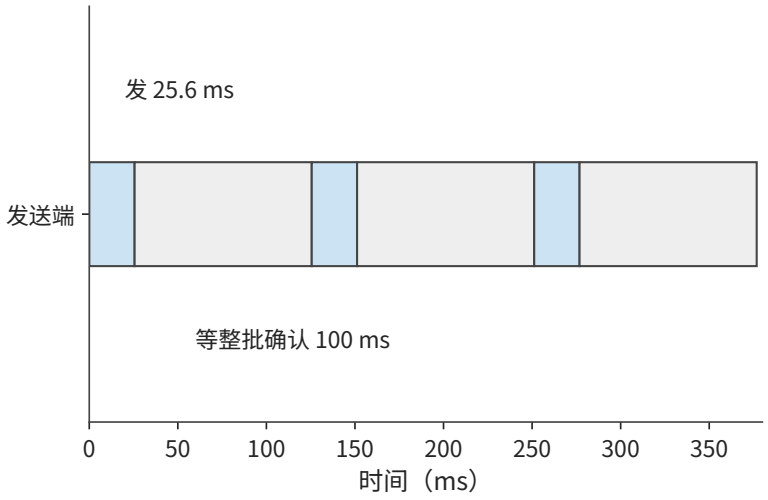



图 12-17：固定窗口 64 KB，链路 20 Mbit/s，一批数据发送需 25.6 ms。为单独显示停等，图设接收方收齐一批后统一确认，从这批数据全部发出到收到整批确认再需 100 ms；每批完整周期为 125.6 ms。

图 12-17 把窗口固定在 64 KB，以便观察停等过程。新连接通常从更小的窗口开始，随着确认到达逐步增加；短请求可能在窗口充分增大以前就已结束。设初始窗口为十个 1460-byte 段，每轮反馈后翻倍。前四轮累计可发 $14600(1+2+4+8) = 219000$ bytes，第五轮累计可达 452,600 bytes。约 355 KB 的语音请求因此要到第五轮才能发完。RTT 为 200 ms 时，几轮反馈就是数百毫秒；短请求的大部分时间可能都用于等待确认。第 12.3.5 节将用这一机制解释一次跨地域语音调用的性能观察。

窗口解释了发送为什么会暂停。请求总耗时还取决于究竟需要发送多少字节：网络上传送的除了图片，还有包头和确认。

**例：协议头与 ACK 如何增加图片传输时间？** 沿用上传 30 MB、下载 5 MB 的条件，取每包最多 1168 bytes

载荷，协议与网络头 60 bytes，每个数据包触发一个 92-byte ACK。合计约三万个数据包，头部与确认增加约 $30000 \times (60+92)/10^6 \approx 4.6$ MB，实际传输量从 35 MB 增至约 39.6 MB。逐包计算发送、确认和后续发送的时刻，完整请求约需 14.4 秒。[17]

与 12.8 秒相比，多出的约 1.6 秒来自模型中刚加入的工作与等待。只看实际传输的字节数仍不够：ACK 一面占用链路，一面释放后续发送额度，减少 ACK 数量会同时改变链路占用和发送额度释放的时机。

实际的确认行为由拥塞控制算法决定。NewReno 是根据丢包与确认调整拥塞窗口的算法，CUBIC 在拥塞避免阶段按时间的立方函数调节窗口；慢启动是连接初期随确认快速增加窗口的阶段。在相同的数据填充方式与 NewReno 配置下，将立即 ACK 改为每两包确认或最多等待 10 ms，实际传输的数据从约 39.6 MB 降至 38.2 MB，完整请求却从约 14.52 秒增至 14.56 秒，晚约 43 ms。[18] 这组对照揭示了反馈的双重作用：少发确认缩短传输时间，确认晚到则延长窗口等待。最终能否更快完成，要比较节省的传输时间与增加的等待时间。

ACK 决定发送方何时能够继续使用窗口发送数据，拥塞控制则决定窗口本身如何变化。相同路径的大图比较中，NewReno 与 CUBIC 的完整响应时间均约为 14.5 秒；窗口轨迹显示 CUBIC 一直在慢启动。[19] 本次请求在进入 CUBIC 的拥塞避免阶段之前就已结束，因此观察到的是启动阶段的行为。

### 12.3.3 丢包是不是拥塞

第 12.3.2 节的窗口随确认增长，也要随丢包收缩。传统 TCP 拥塞控制不直接测量路径带宽，而是把丢包当作拥塞的信号，用丢包反推可用速率。Mathis 模型是按丢包率估算 TCP 稳态吞吐的公式，它给出这种反推能得到的速率上界：[20]

$$B_{\mathrm{TCP}} \le \frac{\mathrm{MSS}}{R} \cdot \frac{C}{\sqrt{p}},$$

其中 MSS（maximum segment size）为每个报文的最大载荷，$R$ 为往返时间，$p$ 为丢包率；周期性丢包、每包确认时 $C = \sqrt{3/2} \approx 1.22$，延迟确认（接收方每两个报文才确认一次）时 $C < 1$，因此论文的式 (4) 去掉常数，给出简化上界 $\mathrm{MSS}/(R\sqrt{p})$。按该式，吞吐由丢包率决定：丢包率增至四倍，速率减半。该式的前提是丢包意味着拥塞；无线链路误码和跨地域链路上的随机丢包都不满足这一前提。

跨地域链路上的实测记录正属于后一种情形。笔者开发的广域网传输系统 Queqiao（鹊桥，第 12.3.5 节介绍）的路径记录给出了一条丢包不含拥塞信息的路径。Irvine 发往贵阳的下载方向，发送速率从 1 提到 300 Mbit/s，丢包率始终约为 14%，前后丢包相互独立；反方向贵阳发往 Irvine 的 41,663 个报文一个未丢，同一条路径的两个方向表现得像两条路径。往返时间的最小值与最大值只差几毫秒，路径上没有排队。速率提到 600 Mbit/s 时丢包才升至 44%——越过了约 333 Mbit/s 的容量膝点（再提速丢包率就陡增的位置），只有这里的丢包才来自拥塞。[21] 代入 MSS 1,448 bytes、$R = 0.2$ s、$p = 0.14$，按式 (4) 的简化上界（$C = 1$，路径记录也按此计算）得

[17] 精确分包为上传 25,685 包、下载 4,281 包；共 29,966 个数据包和同数 ACK，网络实际传输 39,554,832 bytes，完整响应 14.36339008 s。来源：完整图片参考闭环。

[18] 同一填充条件下，立即 ACK 为 39,555,120 bytes / 14.52103724 s，聚合 ACK 为 38,177,328 bytes / 14.564459796 s，差约 1.38 MB 和 43.4 ms。来源：立即 ACK 的 NewReno 对照、聚合 ACK 对照及CUBIC 控制阶段记录。

[19] 同一填充条件下，立即 ACK 为 39,555,120 bytes / 14.52103724 s，聚合 ACK 为 38,177,328 bytes / 14.564459796 s，差约 1.38 MB 和 43.4 ms。来源：立即 ACK 的 NewReno 对照、聚合 ACK 对照及CUBIC 控制阶段记录。

[20] Mathis 等，The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm，ACM CCR 1997。式 (3)：$BW = (MSS/RTT)\cdot C/\sqrt{p}$；周期性丢包、每包确认时 $C = \sqrt{3/2} \approx 1.22$，随机丢包时 1.31；式 (4) 给出更简的上界。

[21] 固定归档的路径记录与设计记录，来自提交 `168ff4b`。文件、调优设置和计时起止点均采用原记录中的定义。

$$\frac{1448 \times 8}{0.2 \times \sqrt{0.14}} \approx 0.155 \text{ Mbit/s},$$

取 $C = 1.22$ 则为 0.189 Mbit/s；两者都与同一路径实测的 0.13–0.47 Mbit/s 一致。第 12.3.2 节那份约 355 KB 的语音请求（354,640 bytes）按此速率仅发送就需要约 18.3 秒。TCP 并没有出错：它按规则响应了丢包，只是这条路径上的丢包不含拥塞信息。[22] 这是第 1.3.4 节所说第一类差距的极端例子：模型前提不成立，算出的上界比路径实际能承载的速率低了三个数量级。要修正的是模型假设，而不是传输实现。

**BBR：直接测量，而不是从丢包反推。** BBR（bottleneck bandwidth and round-trip propagation time，一种基于测量的拥塞控制算法）不以丢包为信号，而是持续测量两个量：瓶颈带宽估计 BBR.max_bw，取近期交付速率样本的窗口最大值；往返传播估计 BBR.min_rtt，取 RTT 样本的窗口最小值。发送节奏（pacing）按测得的带宽设定；在途量按带宽时延积（bandwidth-delay product，BDP）BDP = max_bw × min_rtt 设定：cwnd_gain 为 2，即拥塞窗口（cwnd）取两倍 BDP，为 ACK 聚合与重传留出余量。启动阶段（Startup）以 $4\ln 2 \approx 2.77$ 的 pacing 增益逐轮探测带宽上限，ProbeBW_UP 阶段以 1.25 的增益周期性探高。[23] 代入同一路径：BDP = 333 Mbit/s × 0.2 s = 8.325 MB，拥塞窗口 16.65 MB；理想有效吞吐 $(1-p) \times 333 \approx 286$ Mbit/s，是 Mathis 上界的约 1,850 倍。在丢包不含拥塞信息的路径上，直接测量带宽得到的速率，比从丢包反推的上限高出三个数量级。

**测准带宽之后，重传的尾部还在。** BBR 回答“发多快”，不回答“丢了怎么办”。354,640 bytes 分为 245 个报文，各以 14% 的概率独立丢失，期望丢失 34.3 个。下面按理想化的逐轮选择性重传计算：每轮用一个 RTT 重传丢失的报文，重传的报文仍可能再丢；不计窗口收缩与超时重传，所以结果是有利于 TCP 的下界。无丢包时的串行预算为一次 RTT 加模型 30 ms，再加按膝点速率发送的 8.5 ms，合计约 238.5 ms；有丢包时期望完成时间为 0.756 s，约为串行预算的 3.2 倍；p99 为 1.24 s，对应 6 轮。[24] 即使 BBR 把带宽测准，高随机丢包率下重传造成的尾部延迟依然存在，这不是拥塞控制能解决的问题。

**编码冗余：让接收方不等重传。**既然重传的等待难以接受，就多发冗余数据，由接收方自行修复缺包。前向纠错（forward error correction，FEC）在 $k$ 个数据符号之外多发 $r$ 个修复符号，本例一个符号就是一个 MSS 大小的报文；一块中丢失不超过 $r$ 个符号，接收方就能直接恢复，不需要任何重传轮次。按二项分布精确求 99.9% 成功率所需的最小冗余：丢包率 14% 时，245 个数据包需要 63 个修复包，冗余 25.7%；发送量增至 308 个符号，发送约 10.7 ms，完成时间约 241 ms，贴近 238.5 ms 的串行预算（图 12-18）。同一路径另一时段丢包率为 3.6% 时，逐轮重传的 p99 降为 0.84 s，FEC 只需 20 个修复包，冗余 8.2%。[25] 所以冗余比例应随测得的丢包率调整，而不是固定不变。这组数字也是第 12.3.5 节 Queqiao 案例的定量动机：它的前向纠错与发送调度，正是按路径测量结果安排冗余量与发送节奏。

---

[22] 14% 丢包与3.6% 丢包两档计算。路径参数取自鹊桥路径记录：往返 199–207 ms、容量膝点约 333 Mbit/s、Irvine 发往贵阳的下载方向丢包约 14%（另一时段 3.6%），贵阳发往 Irvine 的 41,663 个报文零丢包、MSS 1,448 bytes、固定文件 354,640 bytes；模型时间取固定文件复测一节的约 30 ms，与实测相符：调优后的中位数 236.5 ms 减去同期最短往返 197 ms 和发送 8.5 ms，模型时间不超过约 31 ms，记录前文的 38 ms 来自轮换八个文件的早期测试；FEC 符号大小取 MSS，块恢复成功率目标取 99.9%。重传模型每轮一次理想选择性重传，不计窗口收缩与超时重传，是有利于 TCP 的下界。

[23] BBR Congestion Control，IETF 草案 draft-cardwell-iccrg-bbr-congestion-control-02：BBR.max_bw 为近期带宽样本的窗口最大值，BBR.min_rtt 为窗口最小 RTT 样本，BBR.bdp 为两者之积；Startup 的 pacing 增益为 $4\ln 2 \approx 2.77$、cwnd_gain 为 2，ProbeBW_UP 的 pacing 增益为 1.25。

[24] 14% 丢包与3.6% 丢包两档计算。路径参数取自鹊桥路径记录：往返 199–207 ms、容量膝点约 333 Mbit/s、Irvine 发往贵阳的下载方向丢包约 14%（另一时段 3.6%），贵阳发往 Irvine 的 41,663 个报文零丢包、MSS 1,448 bytes、固定文件 354,640 bytes；模型时间取固定文件复测一节的约 30 ms，与实测相符：调优后的中位数 236.5 ms 减去同期最短往返 197 ms 和发送 8.5 ms，模型时间不超过约 31 ms，记录前文的 38 ms 来自轮换八个文件的早期测试；FEC 符号大小取 MSS，块恢复成功率目标取 99.9%。重传模型每轮一次理想选择性重传，不计窗口收缩与超时重传，是有利于 TCP 的下界。

[25] 14% 丢包与3.6% 丢包两档计算。路径参数取自鹊桥路径记录：往返 199–207 ms、容量膝点约 333 Mbit/s、Irvine 发往贵阳的下载方向丢包约 14%（另一时段 3.6%），贵阳发往 Irvine 的 41,663 个报文零丢包、MSS 1,448 bytes、固定文件 354,640 bytes；模型时间取固定文件复测一节的约 30 ms，与实测相符：调优后的中位数 236.5 ms 减去同期最短往返 197 ms 和发送 8.5 ms，模型时间不超过约 31 ms，记录前文的 38 ms 来自轮换八个文件的早期测试；FEC 符号大小取 MSS，块恢复成功率目标取 99.9%。重传模型每轮一次理想选择性重传，不计窗口收缩与超时重传，是有利于 TCP 的下界。

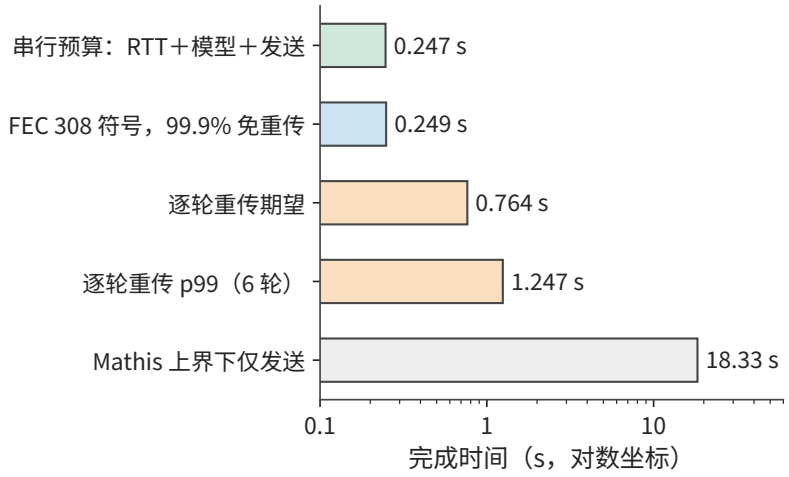



图 12-18：同一路径（RTT 0.2 s、容量膝点 333 Mbit/s、丢包率 14%）传送 354,640 bytes 的完成时间，横轴为对数坐标。串行预算是 RTT、模型 30 ms 与发送 8.5 ms 之和，约 238.5 ms；FEC 多发 63 个修复符号（冗余 25.7%）后 99.9% 不需重传；逐轮重传两行是不计窗口收缩与超时的下界；Mathis 上界对应把丢包当作拥塞的 TCP，仅发送就需约 18.3 s。

### 12.3.4 多流传输与优先级

前几节讨论单条数据的传输；本节考察多条数据共用出口时的相互影响。图片、音频和控制消息共用出口时，有两个顺序可以改变：发送调度决定链路先为谁发送数据，接收方对数据顺序的要求决定已到达的字节何时交给应用。

先分析资源分配。在一组共享链路的计算中，FIFO 按入队顺序发送，图片第 5 秒完成、音频第 6 秒完成；音频优先时，音频第 3 秒完成、图片第 6 秒完成。音频早了 3 秒，图片晚了 1 秒，原因是音频数据提前占用了原先用于发送图片的时段。[26]

再分析接收依赖。保持发送和丢包恢复的时刻不变，音频数据在第 3 秒已经齐备，图片直到第 7 秒才齐备。这里把接收数据交给应用称为交付。整个连接共用一个接收顺序时，前方缺口会阻塞后续数据，形成队头阻塞。整体有序交付时，音频也要等到第 7 秒才能交给应用（图 12-19）；逐流交付在第 3 秒就能将音频交给应用，图片仍为第 7 秒（图 12-20）。这里节省的 4 秒，是因为音频不必再等图片的接收进度，发送的数据量和链路速率都没有改变。

整体有序与逐流交付的区别，正对应两种实际协议的做法。HTTP/2 将多个流放入一个有序 TCP 字节流，前面的缺口会阻挡后续字节。QUIC 以流为单位保持有序，其他流已经连续到达的字节可独立交付。[27] 这与音频播放的递推关系属于同一种分析方法：把一个覆盖全部数据的等待条件拆成各流自己的等待条件，已经满足条件的流就能向应用交付数据。

这两种机制分别回答“何时发送”和“何时交给应用”。第 12.3.5 节用跨地域语音案例说明这些机制如何影响实际请求；第 12.4 节再结合音频的预定播放时段和截图版本，讨论数据交给应用时是否仍有价值。[28]

### 12.3.5 案例：Queqiao 如何加速跨地域语音服务

第 8、9 章介绍了模型服务的批处理与资源池化。把各地请求汇聚到少数数据中心，可以让更多请求共享模型副本与加速器，减少分散部署时各地分别预留却未充分使用的容量，也更容易集满 batch。集中部署因此有机

[26]FIFO与音频优先；固定发送记录的整体有序与逐流交付。分别考察发送调度和接收方交付依赖的影响。

[27]RFC 9000：QUIC 传输，§2、§7、§9、§13；RFC 9001：TLS；RFC 9114：HTTP/3；RFC 9221：不可靠 Datagram。HTTP/3 规定了如何在 QUIC 上传输 HTTP 请求和响应。

[28]本地 TCP / HTTP3 传输与HTTP3 单连接多流。

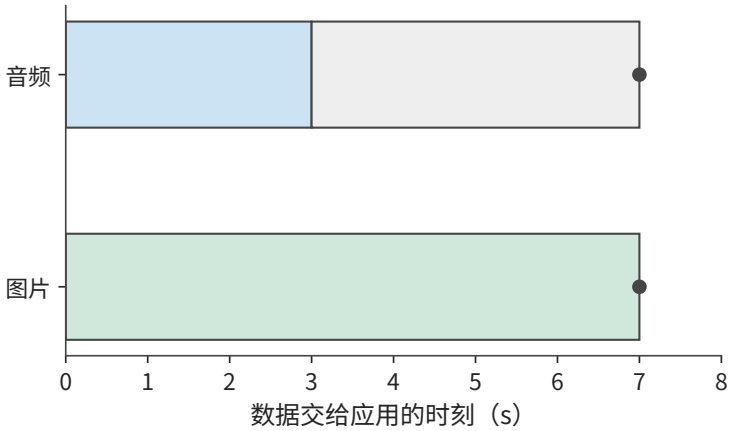



图 12-19：整体有序交付：音频第 3 s 已收齐，却因图片缺口继续等到第 7 s。灰色段表示数据已齐后的等待，圆点表示数据交给应用的时刻。

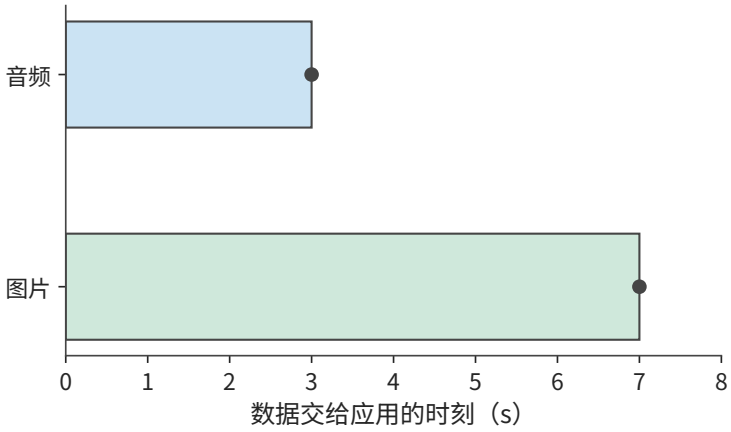



图 12-20：逐流有序交付：音频在第 3 s 收齐后立即交付，图片仍于第 7 s 交付。两图中的发送、到达与恢复时刻相同；区别在于不同流之间是否需要相互等待。

会提高加速器利用率、降低单位请求成本，代价是远方用户要经过更长的网络路径。例如，将语音模型集中部署在美东，美西的用户或接入服务都需要跨地域调用。资源池中的模型即使迅速完成计算，用户仍要等待请求上传和结果返回。

**Queqiao 是什么，解决什么问题？** Queqiao 开源、可自托管，在可控制的客户端与可信网关之间承载 TCP 和 UDP 流量。应用通过本地的 SOCKS5 代理（一种转发任意 TCP / UDP 连接的通用代理协议）接入客户端，客户端跨广域网连接网关，网关再将请求转交目标服务。在这样的数据中心间部署中，客户端在调用方的服务器上，网关靠近远端的推理服务，需要优化的长距离链路就集中在客户端与网关之间。应用仍调用原有的 ASR、TTS 接口，Queqiao 负责传送请求与结果。[29]

项目 README 描述了这一实际需求：模型运行在美国，客户端分布在不同地方。ASR 上传几百 KB 音频，返回一句识别文本；TTS 提交一段文本，返回几百 KB 音频。README 中的 TTS 场景在模型完成后集中返回音频，属于一次突发传输；第 12.1 节的流式场景则在生成首块后开始传输，接收与播放随生成过程持续推进。这类短请求即使能在几毫秒内发完全部字节，连接建立、窗口增长和丢包恢复仍可能各自引入一次长距离的反馈等待。

[29] 系统定位与跨数据中心动机见 Queqiao README 固定归档，对应 GitHub 提交 `496ca627`，读取于 2026-09-10。系统介绍依据此版本，性能对照数据来自提交 `168ff4b`。

Queqiao 的设计目标是减少这些额外等待，使短请求尽量接近必要传播、数据发送和模型处理所需的时间，同时提高长流对可用带宽的利用。Queqiao 让多条应用流共享客户端—网关路径的测量结果与传输状态，复用连接，并根据路径情况安排发送和恢复；前向纠错则多发一些冗余数据，让接收方不必等下一轮重传就有机会恢复缺包。交互流与大文件共存时，还要靠发送调度控制交互流的等待。第 12.3.1 至 12.3.4 节讨论的连接、窗口和多流机制，在此共同服务于跨地域调用。传播距离本身仍然决定必须付出的时间；优化后的完整请求能否满足业务期限，决定了集中部署是否适合这项任务。

**跨地域语音请求为何远慢于约 240 毫秒的理想耗时？**本节使用笔者归档的语音测试检验这些机制。实际测量路径是贵阳至美国 Irvine，并非上面的美东—美西部署示例所用路径。固定音频文件约 355 KB，链路 333 Mbit/s，往返约 200 ms，模型处理约需 30 ms；计时从发起请求开始，到收到结果结束。理想发送时间约为 8.5 ms，加上传播与处理约为 238.5 ms。最初的直接调用却超过一秒，而 Queqiao 约需三百毫秒。

这一差距有两种可能的解释。一种把优势归于传输机制本身；另一种认为直接调用的大部分时间用于连接准备、窗口增长和等待发送。两种解释给出的预测不同：若连接准备与发送等待是主要原因，充分调优直接路径后，请求时间应当接近前面按数据量、计算时间和必要往返算出的最短时间；若协议机制本身仍带来较大优势，差距会继续存在。

检验预测先要固定输入。早期测试轮换使用八个 146–405 KB 的文件，报告却只标注了最后一次请求的大小（355 KB）；标注该大小的中位数实际涵盖整个区间。改为反复发送同一份 354,640 bytes 的文件，并交替运行两条路径之后，每次请求都发送相同数量的字节。[30] 固定数据量后，再改变连接和发送配置，就可以比较两者如何影响请求时间。

**复用连接并调优后，两条路径的耗时差距还剩多少？**

固定文件的两组结果如下。

| 连接条件 | 直接路径的请求中位数 | Queqiao 的请求中位数 | 观察到的差距 |
|---|---|---|---|
| 新建连接 | 约 1.19 s | 约 0.30 s | 直接路径约为 3.9 倍 |
| 保持连接并调优 | 约 0.24 s | 约 0.24 s | 两者接近 |

新建连接时两条路径相差约 0.89 秒，复用连接并调优后都约为 0.24 秒（图 12-21、图 12-22）。[31] 原来的大部分差距随着基线配置的改变消失了。这支持第二种解释：连接准备和发送方式在最初的差距中起了主要作用。即使使用同一种协议，配置不同，请求完成时间也可能相差甚远。这个案例的步骤与第 1.3.4 节一致：先由传播、发送和模型处理算出约 240 ms 的下界，再与实测比较，最后用只改变连接条件的实验分辨差距来自模型漏项还是实现开销。这里的差距几乎全部来自实现开销，下界本身没有错。

**单因素实验如何区分连接、窗口与发送节奏的影响？**连接复用改变请求开始时是否需要握手，窗口大小改变确认返回前允许在途的数据量，发送节奏改变链路的空闲区间。图 12-23 将这三类设置与对应事件配对：固定文件、路径和其余设置，每次改变一个因素，记录受其影响的事件发生时刻与请求总时间。连接实验比较握手结束和首字节发送时刻，窗口实验比较在途量与 ACK 到达后的发送，节奏实验比较两次发送之间的空闲时间。

窗口耗尽时，发送一批数据后在途量达到上限，出口空闲，ACK 返回后才继续发送。扩大窗口可以缩短这类空闲区间；若请求也相应提前结束，发送与确认的时间线便说明，总时间的缩短来自窗口变化。

实验的运行顺序也会影响比较结果。交替测试两条路径，可以让一组对比中的两次请求在相近时间运行，减

[30] 固定归档的路径记录与设计记录，来自提交 `168ff4b`。文件、调优设置和计时起止点均采用原记录中的定义。

[31] 新建连接的直接 / Queqiao p50 为 1185.3 / 301.6 ms，调优保持连接为 240.9 / 236.5 ms；两个中位数之比分别约 3.93 / 1.019。逐轮计算耗时比再取中位数，结果分别为 3.96 / 1.03。来源：Queqiao 同条件统计。

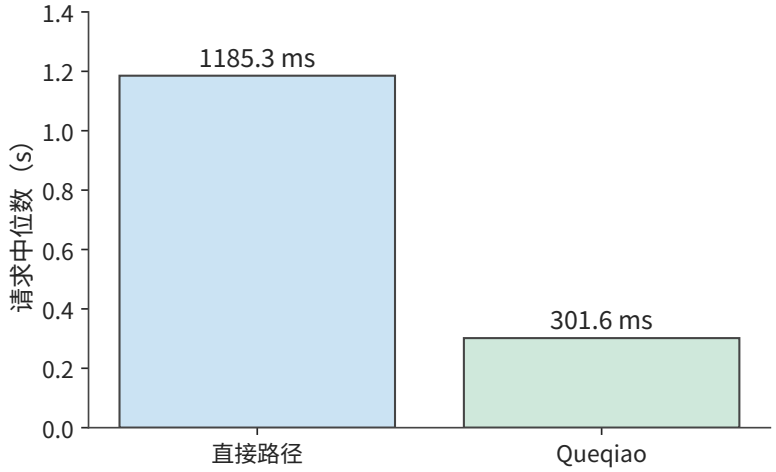



图 12-21：新建连接时，354,640 bytes 固定音频的直接路径与 Queqiao 请求中位数分别为 1185.3、301.6 ms。计时从发起请求到收齐结果，输入文件相同，两条路径交替运行。

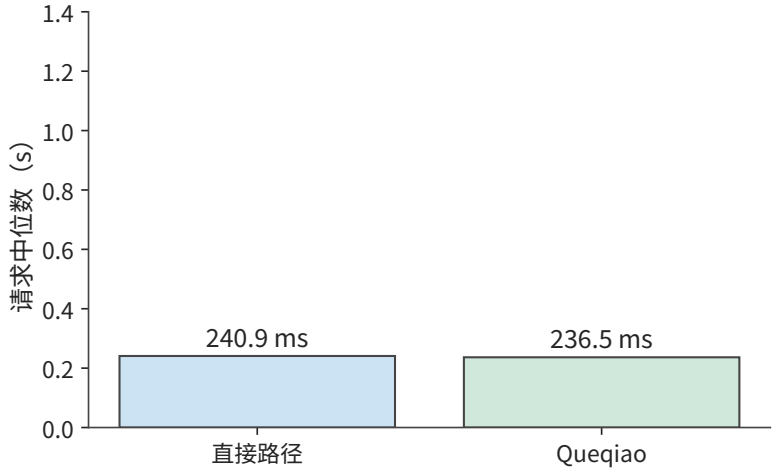



图 12-22：保持连接并调优后，同一固定音频的直接路径与 Queqiao 中位数分别为 240.9、236.5 ms。与前图采用相同纵轴，两条路径均接近数据发送、必要往返和处理所需的总时间。

小网络变化对比较的影响；同期 RTT 约为 197–205 ms，跨度约 8 ms。调优后，两条路径的请求耗时中位数相差约 4 ms，小于同期 RTT 的波动幅度。在这组实验中，两条路径的请求时间都已接近 240 ms。

**降低网络往返延迟与加速模型计算，哪个收益更大？**调优对照方案后，下一步优化的重点也会改变。模型从 30 ms 加速到 3 ms，最多节省 27 ms；将 RTT 从 200 ms 降至 20 ms，可节省约 180 ms。只要改在附近执行多出的准备和计算时间少于这 180 ms，就比原来的远端路径快。这与第 12.1 节的比较方法相同：先算出更换服务器可以少等多久，再扣除新增的计算和准备时间。

请求时间之外，还要检验播放是否连续：音频到达后要由播放器连续播出。另一组采用静音输出设备回调的实验中，完整 PCM 最终全部收到，播放回调却累计有约 0.36–1.44 秒取不到足够的音频样本。[32] 请求结束时刻只反映最后一批数据何时到达，播放器则在此之前就要持续读取音频。请求总时间与播放停顿时间描述的是

[32] 8 次完整接收测试中，收到的 PCM 与播放回调读取的 PCM 均逐位一致，播放回调累计有 360–1440 ms 无法取得足够音频样本；实验使用静音输出设备，记录包含接收与播放回调事件。另有 3 次请求在首个播放回调之前取消，接收字节、连接关闭事件和远端 EOF 记录了取消过程。完整四轮数据见配套证据说明。来源：显式物理接口绑定的音频实验，固定提交 `496ca627`，基线含 socket 绑定适配。

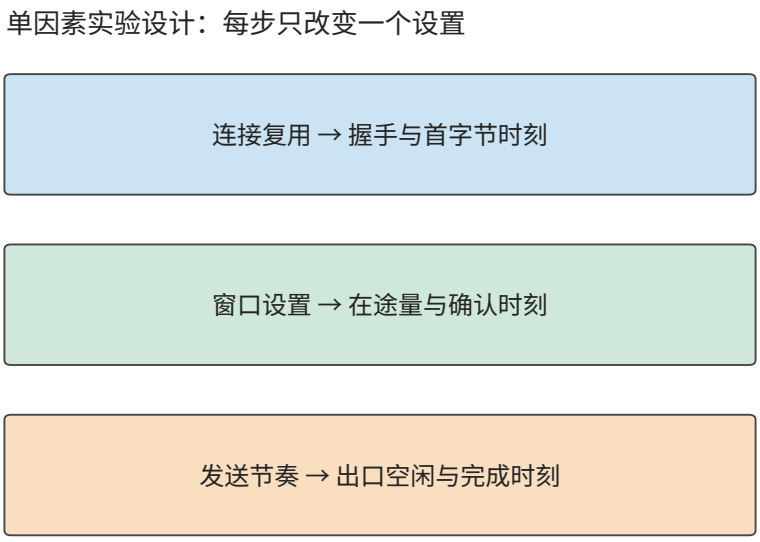



图 12-23：单因素实验方法：保持相同文件、路径和其他设置，每次只改变一个因素，对照中间事件与总完成时间。

两个问题。

音频块的生成、到达、进入播放缓冲和被播放回调读取，构成连续播放的时间线。第 12.1 节的递推式给出缓冲第一次耗尽的时刻：此时若下一块尚未生成，等待出在模型处理；若已生成但未到达，等待出在传输；若已到达却没被读取，等待出在本地缓冲与调度。

这一案例走完了从估算到部署决定的全过程：先算出理想完成时间，再从执行记录中找出等待，用单因素实验解释原因，最后根据还能节省多少时间决定继续调优还是更换服务器。第 12.4 节把无线接入中的争用、波动与恢复加入分析，第 12.5 节汇总完整部署方案。

## 12.4 无线变化下的交互与恢复

### 12.4.1 共享空口与上下行争用

第 12.3.4 节区分了“何时发送”和“何时交给应用”。**空口（air interface）**是设备通过无线信号通信的接口。无线网络还改变了发送本身的代价：上行数据和下行反馈要轮流占用同一段空口时间。每次交换除了发送字节，还要等信道空闲、等帧间间隔，并接收本跳的确认。因此除了应用字节数，还要计算交换次数。

端到端 ACK 发给远端发送方；介质访问控制（MAC）层组织单条无线链路的发送，MAC ACK 确认本跳无线帧。承载端到端 ACK 的无线帧，也要做自己的 MAC 确认。因此，即使端到端 ACK 很短，也需要等待信道空闲，并完成本跳确认。

**例：短 ACK 帧的无线接入与确认开销有多大？**正交频分复用（OFDM）把数据分到多个相互正交的子载波上传送；PPDU 是物理层协议数据单元，包含物理层前导、头部和承载的数据，代表一次无线发送的完整物理层内容；SIFS 是连续交换之间的短帧间间隔。取无聚合的 OFDM 参考交换，数据速率 54 Mbit/s，MAC ACK 速率 6 Mbit/s。数据包交换包含 34 µs 接入等待、208 µs 数据帧、16 µs SIFS 和 44 µs MAC ACK，共 302 µs。承载端到端 ACK 时，数据帧缩至 40 µs，其余三项保持不变，共 134 µs。[33]

[33]共享空口图片计算。数据与 MAC ACK 分别采用 OFDM 54 / 6 Mbit/s，不使用帧聚合；接入等待取 SIFS 16 µs 加两个 9 µs 时隙，即

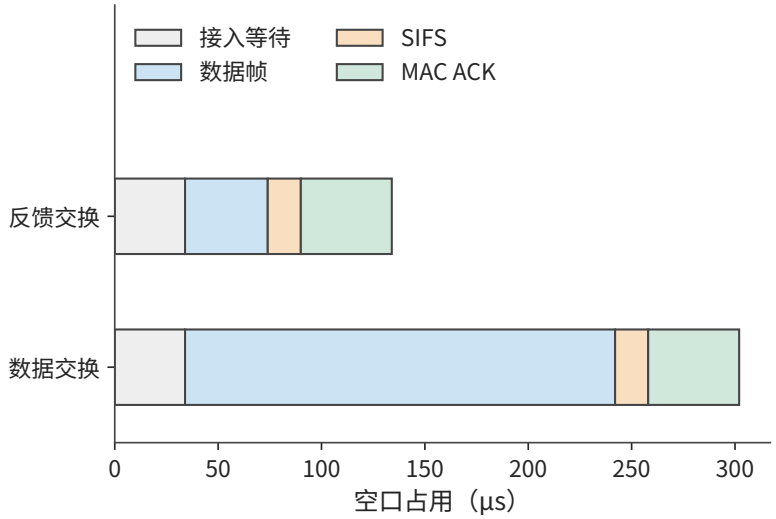



图 12-24：报文变短后，固定交换开销仍然存在，因此空口占用时间不会按相同比例缩短。两种成功交换采用相同的 34 μs 接入等待、16 μs SIFS、44 μs MAC ACK，区别只在数据 PPDU：208 μs 或 40 μs。条件为 OFDM54 ／ 6、无聚合、无重传的参考模型。

图 12-24 中，两种交换都包含 $34+16+44=94$ μs 固定开销。数据帧的发送时间从 208 μs 缩到 40 μs，缩短约八成，整次交换的耗时却只缩短约 56%。固定项所占比例越大，继续缩短载荷的收益越小；把多个小报文合并发送、减少交换次数，降低开销更直接。

仍发送 30 MB 图片与 5 MB 成片，按该无线配置需要的数据交换和反馈交换各为 30,800 次，累计占用空口的时间约为

$$30800 \times (302+134)\ \mu\text{s} \approx 13.4\ \text{s}.$$

保持数据与恢复轨迹不变，将端到端 ACK 数减半，能释放约 $15400 \times 134\ \mu\text{s} \approx 2.1$ 秒空口。第 12.3.2 节已经说明，反馈延迟又会增加窗口停等。因此，需要重新计算各包的发送与确认时刻：减少 ACK 释放了空口时间，但等待确认又可能推迟后续发送。两者共同决定完整请求何时结束。

实际协议中已有按这一思路设计的机制。TACK 是减少无线确认开销的一种反馈机制。TACK 将常规的累积确认与需要即时反馈的事件分开：前者合并后发送以减少交换次数，后者立即返回，让发送方尽快继续发送或重传丢失的数据。[34] 这种分工同时减少确认报文的发送开销和等待反馈的时间。

### 12.4.2 业务时限、过期数据与取消

第 12.1 节的播放器等待缺块，以保持全部内容；另一种播放器按固定时钟播放，晚到的音频块不再用于原定的播放时段。前者把网络延迟变成停顿，后者则会使部分声音缺失。任务的质量要求决定采用哪一种播放规则。

例如，八块音频各长 20 ms，计划从第 0.4 秒开始连续播放，共 0.16 秒。这些音频与大图共用链路时，FIFO 下这些块最终全部到达，却都错过了各自预定的播放时刻；媒体优先时八块都按时到达。[35] 两种情况下最终收

802.11a OFDM 的 DIFS 34 μs，不计随机退避；MAC 层与端到端确认分别计量。

[34] 计算机网络的新黄金时代（三），介绍 TACK、Link Turbo 与代理部署的历史背景。

[35] 媒体优先与聚合 ACK 的混合反馈计算及同目录 FIFO ／ 立即 ACK 对照。模型处理时间和预定播放时刻由算例设定。

到的字节数相同，按时可用的声音却分别为零与 0.16 秒。调度时统计按时播放的内容，才能反映用户实际听到了多少声音。

晚到的数据错过原定播放时段，取消则让尚未播放的数据立即失去价值。播放器的待播队列中若已有三块 20 ms 音频，即使远端停止生成，本地仍可继续播出 60 ms。若控制消息需要 100 ms 才能到达服务端，那么在消息到达前，远端还会产生新块。取消过程因此分成两条并行路径（图 12-25）：本地清空待播缓冲，远端停止生成和发送后续音频。前者决定用户何时不再听到声音，后者决定何时停止消耗资源。

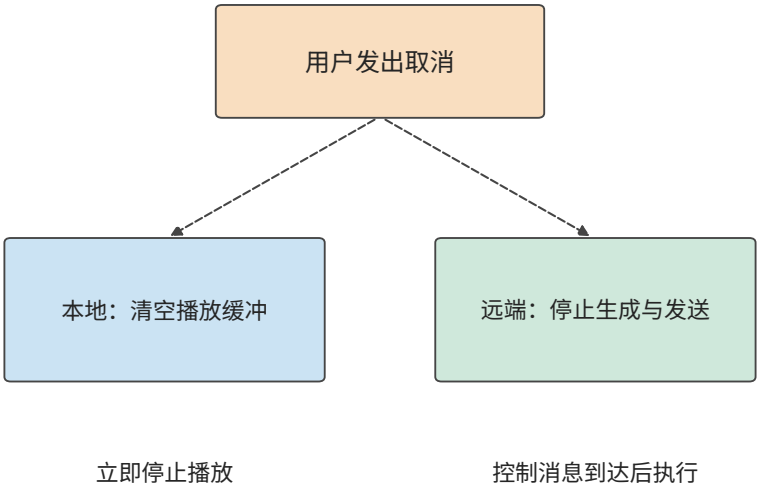



图 12-25：一次取消同时触发两条路径：本地清空待播音频，远端在控制消息到达后停止生成和发送。虚线表示控制流；本地停止播放与远端资源释放有各自的完成时刻。

音频取消后，丢弃的只是尚未播放的数据；截图 Agent 的动作则在提交后改变环境。应答丢失时，重试应查询原操作编号对应的结果；如果再次点击，就把通信重试变成第二次业务操作。保存操作结果后，网络恢复时就可以从原有进度继续，避免增加重复操作。

### 12.4.3 双路径选择、分流与故障切换

此前的分析只考虑单条接入路径；终端往往同时连接两条。Wi-Fi 与蜂窝网络提供两条接入路径。分流把不同部分的数据交给不同路径，目标是同时使用资源；复制把同一份数据交给两条路径，目标是更早取得一份有效结果。这两种方法的数据量和完成条件不同，需要分别计算。

**例：双路径如何分配图片数据以缩短传输时间？**两条独立恒速路径分别为 20 和 10 Mbit/s，将比例 $x$ 分给快路，其余分给慢路。两部分齐备才完成，因此

$$T_{\text{split}} = \max\left(\frac{xS}{B_1}, \frac{(1-x)S}{B_2}\right).$$

如果一条路径先结束，可以把另一条上的部分字节移给它，让原本较晚完成的部分提前结束。因此，最优比例应让两条路径同时完成，即 $x = B_1/(B_1 + B_2)$。快路传 20 MB、慢路传 10 MB，各需 8 秒，比只用快路的 12 秒少 4 秒（图 12-26）。

若两条路径汇入同一个 24 Mbit/s 的出口，两条接入路径仍可同时发送、各需 8 秒，但所有数据都要经过该出口，出口传完 30 MB 至少需要 10 秒。瓶颈于是转移到了共同出口。继续提高接入速率，只会让更多数据

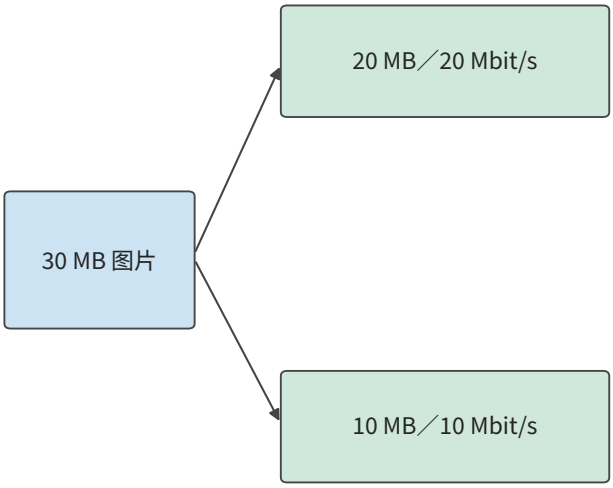



图 12-26：两条独立路径按带宽比例分配输入，分别发送 20 MB 和 10 MB，均需 8 s。任务等两部分都齐备后完成。

排队等待通过该出口。

图 12-27 中，两条线在出口重新汇合。这一汇合点既解释了两条接入路径的速率为什么不能直接相加，也引出另一个问题：出口所在的设备故障时，另一条接入能否继续传输。

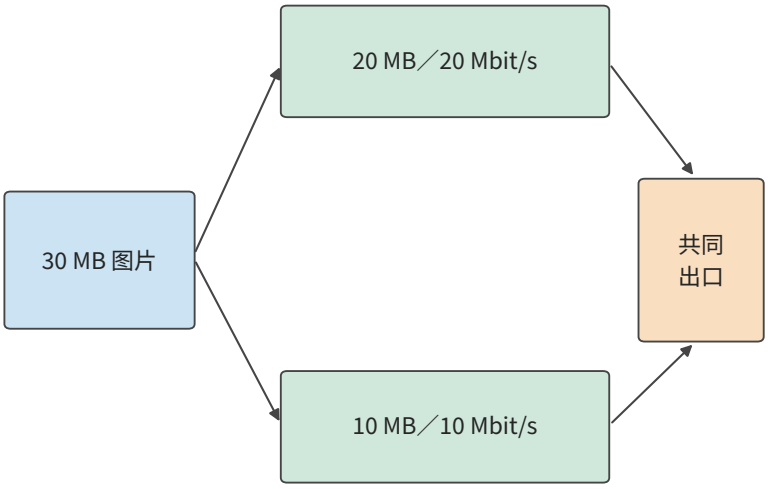



图 12-27：30 MB 图片按 20 ／ 10 MB 分到 20 ／ 10 Mbit/s 两条独立接入，各需 8 秒。两路再经过同一 24 Mbit/s 出口，全部数据通过该出口至少需 10 秒。箭头表示数据路径，不表示传播距离；容量取恒定值。

如果希望一条路径断连后仍能收到同一份数据，可以采用复制，并检查两份数据会在哪些地方同时失败。设共同端点失败概率为 $q$；端点正常时，两条接入的失败相互独立，概率为 $p_1, p_2$。失败包括端点失败，以及端点正常但两条接入均失败，两类互斥事件相加得

$$p_{\text{fail}} = q + (1-q)p_1p_2.$$

取 $p_1 = 0.1, p_2 = 0.05$，两条接入同时失败的概率为 0.5%；加入 $q = 0.02$，总体约为 2.5%。其中 2 个百分点直接来自共同端点。即使把两条接入做得完全可靠，总体失败概率仍为 2%；把服务副本放到不同故障域，才会降低这一项。

避开共同故障后，复制还要支付额外带宽。以八块 640-byte 音频为例，一份为 5120 bytes，全量复制为 10,240 bytes。接收方用数据块编号去重，先到的完整副本进入播放器，后到副本被丢弃。多发送一份数据，就能在两条路径中使用先到的结果；如果两路共用同一个出口，重复数据也会占用其他任务需要的带宽。[36]

分流、复制和汇合点在实际系统中常由协议或接入代理实现。多路径 TCP（MPTCP）在一个逻辑连接中组织经过不同路径的 TCP 子流，接入代理可在终端与普通服务器之间汇合这些路径。华为 Link Turbo 的多网络协同实践也利用不同接入的状态选择发送方式。[37] 代理的位置同时影响共享容量、传播与故障域，因而成为部署模型中的一个计算和传输节点。

## 12.5 从端边云分工到完整部署

### 12.5.1 地域、数据与服务位置

继续分析第 12.1 节的 30 轮截图任务。前十轮已完成，要为剩余二十轮选择执行位置；从此刻起，任务必须在 45 秒内完成。无论选择哪种方案，此前已经花掉的时间都相同，比较的是剩余工作。原服务每轮 3.5 秒，继续执行需要 70 秒，因此必须换一种执行方案。

仍使用每轮 0.8 MB 截图、0.3 秒终端工作，原云路径上行 6.4 Mbit/s、RTT 0.2 秒。比较三个执行位置：端侧是第 12.1.5 节的手机，附近工作站装一张 RTX PRO 6000 Blackwell 工作站版，云地域使用一张 H100 SXM。三处运行同一份 Qwen3-8B q4_0 权重和 8K 上下文，每轮都生成第 12.1.5 节的 45 个 token，任务质量因而相同。每轮模型时间沿用第 12.1.5 节的口径，取 decode 读取下界：45 步，每步读取 5.47 GB，除以设备的内存带宽。新服务接手时，要先对前十轮留下的 8K 上下文做一次 prefill，即表中的 “一次准备”，按 133.6 TFLOPs 的矩阵工作量除以 BF16 稠密峰值计算；手机没有可引用的矩阵峰值，准备记为 0，这一取值只会让端侧显得更快。先采用无排队、无故障、控制回复发送时间为零的条件。[38]

[36] 双进程双 TCP 连接实验：实验通过双进程、双 TCP 连接模拟两条传输路径，30 次正式尝试包含共同端点失败的情况。

[37] 计算机网络的新黄金时代（三），介绍 TACK、Link Turbo 与代理部署的历史背景。

[38] 端边云三档的设备计算。每步读取量取单请求 8K decode 结果的权重与 KV，权重按 q4_0 每 32 个值 18 bytes 重排；手机总线取端侧总线与能耗分账；带宽、BF16 稠密峰值与额定功率取自硬件参数表；实测倍数取自实验 8-1 的效率表中 batch 1、8K 一行。结果文件同时列出十轮、收紧期限与断连恢复的全部情形。

| 执行位置 | 设备 | 一次准备 | 每轮模型 | 上行 | 每轮往返传播 | 截图去向 | 20 轮能耗 |
|---|---|---|---|---|---|---|---|
| 端侧 | 手机，84.8 GB/s | 0 s | 2.90 s | 无需上传 | 0 s | 留在设备 | 518–680 J |
| 附近工作站 | RTX PRO 6000，1,792 GB/s，600 W | 0.27 s | 0.137 s | 80 Mbit/s | 0.02 s | 传到附近工作站 | 约 1.81 kJ |
| 云地域 | H100 SXM，3,350 GB/s，700 W | 0.14 s | 0.073 s | 6.4 Mbit/s | 0.20 s | 传到云地域 | 约 1.12 kJ |

成本用能耗衡量。两张 GPU 按板卡额定功率乘以忙碌时间计算，忙碌时间为一次准备加二十轮模型计算；额定功率是上限，所以这两行也是上限。手机取 MELTing Point 实测的每 token 0.576–0.756 J，二十轮共 900 个 token。[39] 上传与传播消耗的网络能量不计。

比较时间与成本之前，先要满足数据驻留（data residency）的约束，即数据只能在规定的设备或地域内存储和处理。表中端侧一行的原始截图始终留在设备上；附近工作站与云地域两行每轮都要把 0.8 MB 的原始截图传出设备。若截图含有不允许离开设备或不允许出境的内容，这两行在比较开始前就被排除；下面的时限与成本比较以允许上传为前提。

将云端进一步分为不同地域后，仍然需要比较这些时间与成本。采用 Regionless（由平台为请求选择云地域，而非应用固定绑定地域）服务时，应用指定任务、质量要求和完成期限，由平台选择部署地域。[40] 平台仍需比较各方案的实际开销：服务器距离影响传播时间，缓存影响重新计算的工作量，跨地域传输影响数据传输成本。

### 12.5.2 时限、成本与恢复的联合比较

**例：状态迁移与剩余交互轮数如何改变端边云部署选择？**剩余二十轮的时间为

$$
\begin{aligned}
T_{\text{local}} &= 20(0.3 + 2.90) \approx 64.0 \text{ s},\\
T_{\text{near}} &= 0.27 + 20(0.3 + 0.137 + 0.02 + 6.4/80) \approx 11.0 \text{ s},\\
T_{\text{cloud}} &= 0.14 + 20(0.3 + 0.073 + 0.20 + 6.4/6.4) \approx 31.6 \text{ s}.
\end{aligned}
$$

H100 的内存带宽是 RTX PRO 6000 的 1.87 倍，云端每轮的模型计算比附近工作站少用约 0.064 秒，二十轮省约 1.3 秒，准备再省约 0.13 秒；但上传每轮多 0.92 秒，传播多 0.18 秒，二十轮多等 22 秒。合计下来，云端反而慢约 20.6 秒。把时间差拆开，便能看到更快的模型如何因每轮通信而失去优势：每轮 decode 只要几十到一百多毫秒，远少于每轮的通信时间。

图 12-28 至图 12-30 汇总了三种部署方式下完整任务的各阶段耗时。云端的绿色计算段更短，蓝色上传段却更长；两段与其余工作依次累加，决定横条终点所示的任务完成时间。

45 秒期限先排除端侧：端侧能耗最低，却需要 64.0 秒。附近工作站和云端都能按时完成，接着比较能耗：

[39] MELTing Point: Mobile Evaluation of Language Transformers，arXiv 2403.12844，§5.2.2 与附录表 5：每 token 0.16 / 0.20 / 0.21 mWh；iPhone 14 Pro 持续功率最高 13.8 W、瞬时超过 18 W，Galaxy S23 持续低于 8.5 W；iPhone 14 Pro 上 Zephyr-3B q4_k 为 14.8 token/s、Llama-2 7B q3_k 为 6.0 token/s；一次充电可运行 490.05–590.93 个提示（S23 为 542.78）。提取值逐行核对见数值提取表。

[40] 计算机网络的新黄金时代（二）的云际网络与 Regionless 讨论。本章的地域方案用设备能耗和 GPU 时间衡量成本。

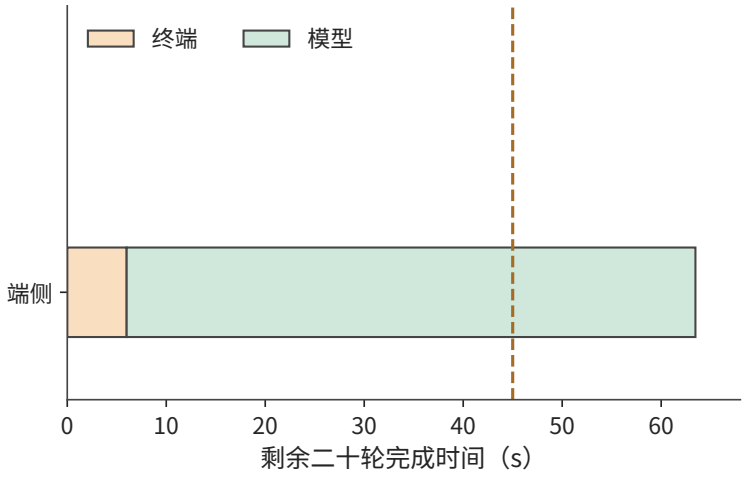



图 12-28：端侧：终端工作 6 s，模型计算 58.0 s，共 64.0 s。三图共用 45 s 期限线和同一横轴，颜色分别汇总二十轮同类工作的耗时。

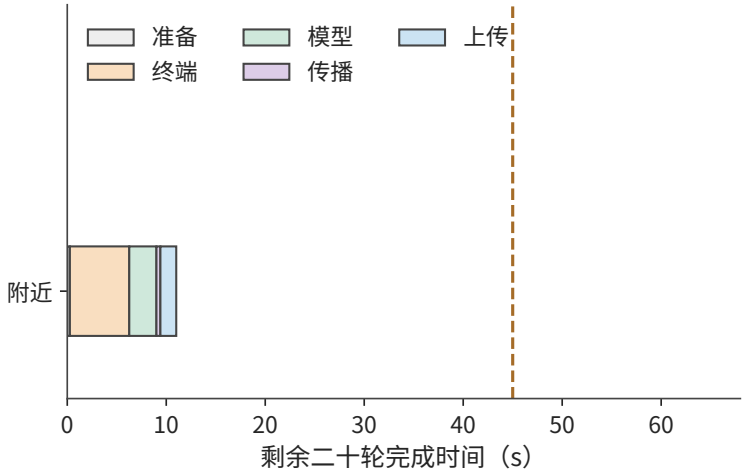



图 12-29：附近工作站（RTX PRO 6000）：准备 0.27 s，二十轮终端 6 s、模型 2.7 s、传播 0.4 s、上传 1.6 s，共 11.0 s，满足 45 s 期限。虚线表示 45 s 完成期限。

云端约 1.12 kJ，附近工作站约 1.81 kJ。H100 的额定功率比 RTX PRO 6000 高，每轮的忙碌时间却只有后者的一半多，能耗反而更低。因此，在能按时完成的方案中，能耗最低的是云端方案。

端侧一行还要按第 12.1.5 节核对电池。表中二十轮的 518–680 J 占一次充电的比例需要折算。iPhone 16 Pro 的规格页只给出最长 27 小时视频播放，没有电池的瓦时数，所以改用 MELTing Point 的实测折算：一次充电可完成约 490–590 个提示，20 轮相当于 20 个提示，约占一次充电的 3.4%–4.1%。[41] 电池不排除端侧方案，排除端侧的是 64.0 秒的完成时间。

云端方案对上行速率的依赖可以定量表示。只改变云端路径的上行速率 $b$，单位 Mbit/s，其他参数保持不变，云端完成时间为

[41] MELTing Point: Mobile Evaluation of Language Transformers，arXiv 2403.12844，§5.2.2 与附录表 5：每 token 0.16 / 0.20 / 0.21 mWh；iPhone 14 Pro 持续功率最高 13.8 W、瞬时超过 18 W，Galaxy S23 持续低于 8.5 W；iPhone 14 Pro 上 Zephyr-3B q4_k 为 14.8 token/s、Llama-2 7B q3_k 为 6.0 token/s；一次充电可运行 490.05–590.93 个提示（S23 为 542.78）。提取值逐行核对见数值提取表。

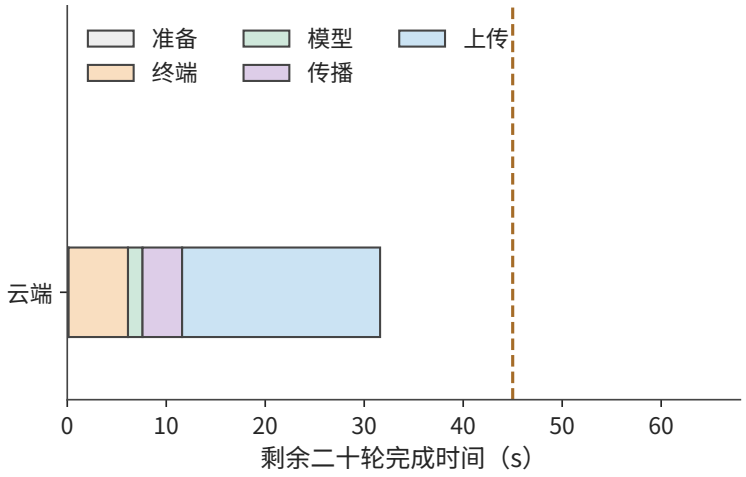



图 12-30：云端（H100 SXM）：准备 0.14 s，二十轮终端 6 s、模型 1.5 s、传播 4 s、上传 20 s，共 31.6 s。计算更快而传输更久，比附近工作站慢约 20.6 s，仍在期限之内。虚线表示 45 s 完成期限。

$$T_{\text{cloud}}(b) = 11.6 + \frac{128}{b}\ \text{s}.$$

11.6 秒来自一次准备和二十轮非上传工作，128 Mbit 是二十张截图的总上传量。满足期限需要 $b \geq 128/33.4 \approx 3.8$ Mbit/s（图 12-31）。但无法比附近工作站更快：即使上传瞬间完成，云端也要 11.6 秒，仍比附近的 11.0 秒慢。原因在于每轮 0.2 秒的往返传播：它比附近工作站每轮的传播与上传之和（0.1 秒）多出 0.1 秒，超过了 H100 每轮节省的约 0.064 秒模型时间。

反解带宽门槛前必须先检查固定时间。若期限为 $D$，这里的门槛为 $b_{\min} = 128/(D - 11.6)$ Mbit/s，只在 $D > 11.6$ 秒时有意义。当 $D \leq 11.6$ 秒时，即使上传带宽无限大，固定时间也已经用完期限，不存在有限的带宽门槛；不能把公式算出的零、负数或除零错误解释为可行带宽。

“追不上”这一结论依赖读取下界的口径。实验 8-1 在 RTX PRO 6000 上测得 batch 1、8K decode 每步 25.83 ms，是读取下界 9.12 ms 的约 2.83 倍，多出的主要是每步的固定开销。把两张 GPU 的模型时间都按该倍数放大，附近工作站为 16.04 秒，云端不含上传为 14.29 秒，两者相差 1.75 秒；上行超过 $128/1.75 \approx 73$ Mbit/s 后云端才更快。[42]

接近期限时，还要计算能为额外开销留出多少时间。原上行 6.4 Mbit/s 时，云端任务比期限提前约 13.4 秒完成；上行降到 4 Mbit/s，时间约为 43.6 秒，只剩约 1.4 秒余量；降到 3.5 Mbit/s，约为 48.2 秒，已经超期。若要求 99% 的任务按时，就需要分析任务完成时间的概率分布。由于每轮速率 $b_i$ 可以不同，包含上传在内的总时间应写为

$$T = 11.6 + \sum_{i=1}^{20} \frac{6.4}{b_i}.$$

设每轮速率在其上传期间恒定，十轮 6 Mbit/s、十轮 10 Mbit/s 的平均速率也是 8 Mbit/s，任务却约需 28.7 秒，比全程 8 Mbit/s 的 27.6 秒慢。发送时间与速率成反比，慢的几轮多出的等待大于快的几轮节省的等待。平

[42] 端边云三档的设备计算。每步读取量取单请求 8K decode 结果的权重与 KV，权重按 q4_0 每 32 个值 18 bytes 重排；手机总线取端侧总线与能耗分账；带宽、BF16 稠密峰值与额定功率取自硬件参数表；实测倍数取自实验 8-1 的效率表中 batch 1、8K 一行。结果文件同时列出十轮、收紧期限与断连恢复的全部情形。

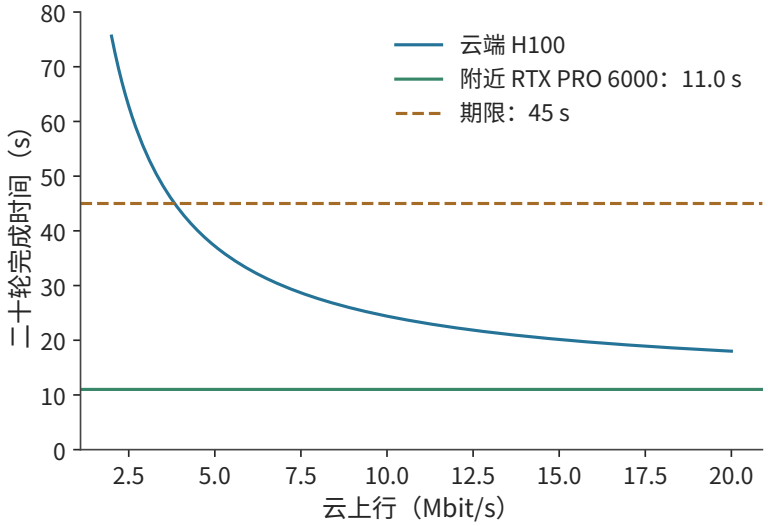



图 12-31：云路径上行改变完整任务的部署选择。剩余 20 轮每轮上传 0.8 MB，云端准备 0.14 秒，每轮其余工作合计约 0.57 秒，因此总时间为 11.6+128/b 秒；附近工作站为 11.0 秒，期限为 45 秒。本例保持模型、处理时间和其他网络参数不变，忽略排队与故障。约 3.8 Mbit/s 起云方案可满足期限；上行再高，云端也不低于 11.6 秒，始终慢于附近工作站。

台应根据整个任务的上传时间分布，判断是否满足 $P(T \le 45\ \text{s}) \ge 0.99$，还要考虑网络持续变慢时，相邻多轮会一起受到影响。

以上部署选择主要受截图上传时间影响。换成输入较小、前缀重复较多的文本任务，优化重点会转向计算复用。保留会话状态就能减少这些重复计算。考虑一条文本 Agent 会话：逐轮重送前缀合计约 66 KB，保留状态后只送约 10 KB，回复均约 2.5 KB。在 333 Mbit/s 上，少传约 56 KB 只节省 1.3 ms。[43] 与截图的 16 MB 上传相比，这里的主要收益来自少做前缀计算。

同一会话的矩阵计算量从约 $3.0 \times 10^{14}$ 降至 $6.0 \times 10^{13}$ FLOPs。在云端 H100 SXM 上按 989.4 TFLOP/s 计，保留状态节省约 0.239 GPU 秒。保存的约 465 MB 状态持续占用显存；按其在 H100 80 GB 显存中所占的比例折算，每保存 1 秒相当于占用 $0.465/80 \approx 0.0058$ GPU 秒。两者相除：状态保存约 41 秒之后，占用的显存就会抵消本次节省的计算。这种盈亏平衡的计算方法与第 12.2.3 节的迁移时间比较相同：复用收益必须覆盖保存或搬移状态的成本。

保存状态既能减少正常执行中的重复计算，也能减少故障后的重做。以选定的云端方案为例，比较同一次断连在两种进度保存方式下的开销。云端完成一轮任务约需 1.57 秒。设第十轮已执行完，但确认尚未返回时连接中断，恢复连接需要 1 秒。若前九轮进度已经提交，只重做未确认的第十轮，额外约需 2.57 秒，任务由 31.6 秒增至 34.2 秒；若十轮都要重做，额外约需 16.7 秒，任务变为 48.3 秒，超过期限。仍然使用同一台服务器，能否保留进度却决定了任务能否按时完成。附近工作站每轮只需约 0.54 秒，即使丢失十轮进度，也只要 17.4 秒；期限余量更大的方案，对故障也更宽容。

图 12-32 和图 12-33 标出了两种恢复方式的差异所在：相同的断连事件之后，一种只重复最后一轮，另一种重复此前十轮。恢复开销有多大，首先取决于哪些结果已经可靠保存。

以上比较的是一次已发生的断连。若要估计大量任务的平均成本，还要把失败发生的概率计入：重试也会增加成本。每次尝试独立、成功率为 $p$，失败后全部重试且每次收费 $C$，期望成本满足 $E = C + (1-p)E$，所

[43] 同一模型、token 序列与轮次的重放，传入服务端的数据量分别为 66,207 / 10,193 bytes，返回客户端的数据量均为 2,527 bytes。矩阵工作量分别为 296,505,803,538,432 与 60,380,764,176,384 FLOPs，除以 H100 SXM 的 BF16 稠密峰值得到 GPU 时间下界；状态 465,371,136 bytes，按其占 80 GB 显存的比例折算驻留时间，约 41.0 s 后抵消省下的 0.239 GPU 秒。来源：固定 Agent 轨迹的地域放置，字节与 token 来自原轨迹。

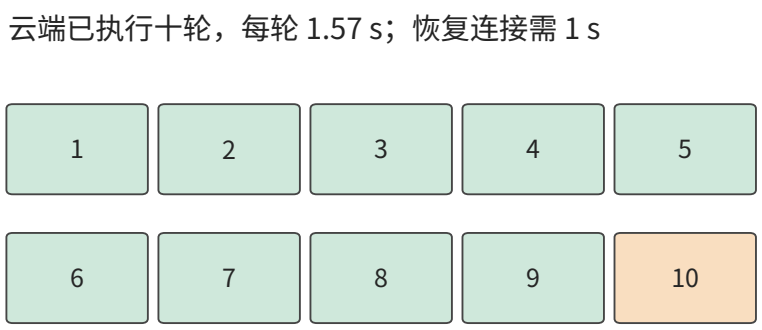



图 12-32：保留前九轮提交记录（绿色），只重做未确认的第十轮（橙色）。云端每轮 1.57 s，恢复连接 1 s，增加 2.57 s，总时间为 34.2 s。此例采用可安全重放的操作；已提交的外部操作则先查询其结果。




图 12-33：丢失十轮进度时，恢复连接后重做十轮，额外 1 + 10×1.57≈16.7 s，总任务从 31.6 s 延至 48.3 s，超过 45 s 期限。

以 $E = C/p$。99% 成功率对应约 $1.01C$，80% 对应 $1.25C$。保存进度可以把整项重试变成局部重做，直接降低每次失败追加的时间和成本。

### 12.5.3 何时需要更换部署方案

综合第 12.5.2 节的时间、能耗与恢复比较，本例应把剩余二十轮放到云端：31.6 秒完成，能耗约 1.12 kJ。与继续使用原服务的 70 秒相比，节省约 38.4 秒。终端继续采集并执行操作，云端的 H100 负责模型判断，每轮只传输截图和控制结果。

这一决定来自对三种部署方案的逐项比较：端侧先因超期被排除，附近工作站和云端都能按时完成，其中云端的能耗更低。第 12.1 至 12.2 节的压缩、特征与状态分析则给出了进一步改进的方法：压缩截图可以减少上传时间，上传完整特征会增加传输时间，保留已提交的进度则可以减少恢复后的重复工作。

| 条件变化 | 结果 | 决策 |
|---|---|---|
| 原云端路径的上行速率为 6.4 Mbit/s | 云端 31.6 s、约 1.12 kJ；附近 11.0 s、约 1.81 kJ | 两者都按时，使用能耗更低的云端 |
| 云端路径的上行速率降至 3.5 Mbit/s | 云端约 48.2 s | 云端超期，改用附近工作站 |
| 云端路径的上行速率无论多高 | 云端不少于 11.6 s | 始终慢于附近工作站；更紧的期限只能由附近工作站满足 |
| 云端发生一次断连，保留前九轮进度并重做第十轮 | 34.2 s | 仍在期限内 |
| 同次断连丢失十轮进度 | 48.3 s | 超期；需要改进进度保存机制，或改用附近工作站 |

任务执行期间更换设备，还需要迁移会话状态。第 12.2 节的 64 MiB 会话需要约 7.7 秒准备，每轮快 0.4 秒，20 轮只净节省约 0.3 秒。若带宽改善只持续十轮就恢复原状，这次迁移会净损失约 3.7 秒。因此，平台应把预计的持续收益与切换成本放在一起比较：预计节省的时间足以抵消迁移准备并留有余量应对额外开销时，才迁移状态。在一段时间内继续使用同一台设备，可以避免在临界带宽附近反复迁移状态。

第 1 章通过容量、运算和读取时间判断加速器能否承载模型；本章进一步分析通信、执行顺序和恢复过程，判断部署方案能否按时完成任务。选择部署方案时，需要依次回答：任务包含哪些工作，哪些步骤必须等待，实际执行还会增加哪些开销，以及条件变化后哪种方案更合适。

### 12.5.4 资源条件变化后的系统调整

以上部署比较固定了模型，只改变执行位置；反过来，固定任务流程、只加速模型，完整任务的收益同样有限。设一次固定轨迹中，模型执行合计 8 秒，工具与网络等不可重叠部分合计 2 秒。即使模型整体加速 10 倍，任务也只能从 10 秒降至 2.8 秒，加速比约为 3.57；[44] 模型耗时趋近于零时，下界仍是 2 秒（图 12-34）。

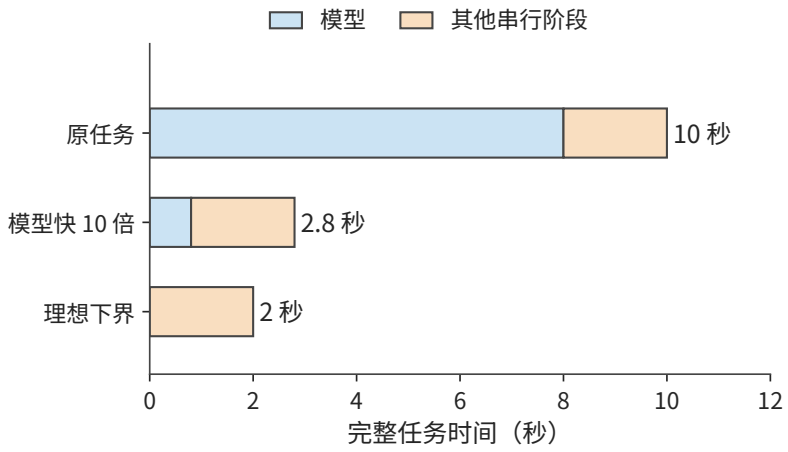



图 12-34：固定任务轨迹中的加速上限。模型从 8 秒缩短至 0.8 秒，其他串行阶段保持 2 秒；第三行表示模型时间趋近于零的理想下界。

要继续缩短剩余两秒，可以改变系统组织。把执行移到本地可减少网络往返，保留环境可减少创建等待；相应增加的本地设备与驻留空间，则进入新的成本预算。第 12.5.2 节的部署算例已经给出比较方法：先求满足质量与期限的方案，再比较每个完成任务的总成本。

[44] 本章条件比较的逐项复算与计算程序。

同样的思路也出现在模型设计中：DeepSeek V4 到 V4.1 就是模型侧的同类调整。跨层共享减少全局 KV 的重复保存，使模型可以用更细的粒度保存历史信息；CED 减少输入阶段反复经过解码器的工作，缓存恢复则缩短多轮会话重新开始的路径。[45] 这些改变分别作用于容量、输入计算和恢复等待，可以沿同一任务逐项计入预算。

从片上存储到端边云，设计选择始终依赖对完整执行过程的了解。注意力的归约关系使中间矩阵可以按块处理，重复的模型结构使执行计划可以复用，前缀与分支关系使上下文可以共享，工具等待则为状态换出和环境准备提供了时间。每一项改进都利用模型或任务的信息，指导其他层次的设计，重新安排数据存放位置、计算划分和交接方式。

## 练习

练习按计算、改变条件和解释机制递进。12-2、12-3、12-7 为核心练习。未另说明的数值沿用正文教学条件。

**12-1　图片精修的传输与计算预算〔延伸，★〕。**

(a) 上行速率分别取 10、20、100 Mbit/s，计算三种方案的完成时间：保持原方案、将图像处理速度提高到原来的十倍，以及将输入大小减半。输入压缩所增加的编解码耗时仍为 0.15 秒。

(b) 求压缩节省的传输时间恰好等于新增编解码时间时的上行速率。再以原图大小和 20 Mbit/s 上行速率为条件：若原有的 0.3 秒处理时间中只有 0.2 秒可以加速，将这部分的执行速度提高到原来的十倍后，任务总耗时是多少？

(c) 把图像分为三个可独立处理的块，画出上传、处理与回传的重叠。标出限制任务耗时进一步缩短的步骤。

**12-2　截图压缩节省的传输时间能否抵消额外交互〔核心，★〕。**

(a) 将每轮截图从 0.8 MB 压缩到 0.2 MB，每轮新增 30 ms 编码时间。复算完成 30 轮截图任务总共能节省多少时间。

(b) 压缩后需要 38 轮时，任务是否更快？求压缩方案仍比原图 30 轮方案更快时，最多允许执行多少轮。

(c) 将每轮 RTT 增加 100 ms，分别计算原图 30 轮和压缩图 38 轮的总时间。解释为什么同一个 RTT 变化会产生不同的任务耗时增量。

(d) 为截图版本与操作编号设计记录字段，说明怎样识别压缩导致的纠错轮次与旧状态导致的无效轮次。

**12-3　视觉编码位置与状态迁移如何影响交互耗时〔核心，★〕。**

(a) 一张图片占 0.8 MB，编码后得到形状为 $[400, 10240]$ 的 BF16 特征张量，上行带宽为 6.4 Mbit/s。视觉编码需要 1.31 TFLOPs 矩阵运算；本地在 RTX 4090 上编码，远端在 H100 SXM 上编码，均按 BF16 稠密峰值（165.2、989.4 TFLOP/s）求编码时间下界。比较本地编码后上传特征与上传原图后在远端编码的总耗时。

(b) 将链路改为一个 400 Gbit/s 的 ConnectX-7 端口（每方向 50 GB/s），以直接上传原图为基准，求上传视觉特征多花多少传输时间；与 (a) 中两张卡的编码时间之差比较，判断此时哪一项决定编码器的位置。

(c) 迁移需要以 80 Mbit/s 传输 64 MiB 状态，再用 1 秒恢复；迁移后每轮执行时间缩短 0.4 秒。分别求剩余 10、20、40 轮时净节省的时间。若恢复时间再增加 1 秒，至少还需执行多少轮，才能收回迁移耗时？

(d) 分别说明保存图片、视觉特征和语言 KV 后，从何处开始重算；说明已提交的操作结果怎样进入恢复过程。

**12-4　连接复用与多流传输〔延伸，★〕。**

(a) RTT 为 200 ms，任务共 30 轮，每次建链需额外两次往返。计算每轮新建连接与全程复用连接的时间差。

[45] DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

(b) 分别计算立即确认与聚合 ACK 两种方案中，图片、包头和 ACK 合计传输的字节数，解释为什么实际传输的字节更少，请求完成时间反而可能更长。
(c) 比较发送优先级与图 12-20 的逐流交付：分别指出资源使用时段和等待依赖发生了什么变化。
(d) 设计一组只改变连接复用方式、其他条件相同的对照实验，列出起止事件及用于解释时间变化的中间事件。

**12-5　ACK 频率与空口效率〔延伸，★〕。**

(a) 一次数据帧交换耗时 302 µs，一次确认帧交换耗时 134 µs。共发生 30,800 次数据帧交换，原先每次都单独确认。将 ACK 数量减半后，能释放多少空口时间?
(b) 画出发送方因等待聚合 ACK 而暂停发送的时间线，解释释放空口与提前交付之间的关系。
(c) PCM 播放所需速率为 256 kbit/s、实际接收速率为 130 kbit/s、初始缓冲 60 ms，求耗尽时刻。保持接收速率和 60 ms 初始音频时长不变，将采样率从 16 kHz 改为 24 kHz，求新的耗尽时刻。

**12-6　双路径的分流与备份〔延伸，★〕。**

(a) 30 MB 经 20 与 10 Mbit/s 两路分流，推导使两路同时完成的比例与时间。
(b) 加入 24 Mbit/s 共同出口，求新的传输时间下界，并指出提高哪一段链路的带宽能够降低这一下界。
(c) 两条接入路径的失败概率分别为 0.1、0.05，共同端点的失败概率为 0.02。按正文的独立故障假设，求向两路各发送一份完整副本后，任务仍然失败的概率；两条接入路径完全可靠后，该概率是多少?
(d) 将八块音频各复制一份，每块为 640 bytes，求新增的传输字节数，并说明接收方怎样避免重复播放。

**12-7　用单因素实验区分连接准备、窗口与传输延迟〔核心，★★〕。**

(a) 分别计算每次新建连接和调优后保持连接这两组实验中，直接路径与 Queqiao 路径耗时中位数的比值，说明“主要差距来自连接准备与发送等待”这一解释给出了什么预测。
(b) 设数据量约为 355 KB，传输速率为 333 Mbit/s，RTT 为 200 ms，处理耗时为 30 ms，求理想完成时间。与最初约一秒的实测耗时比较，估算还有多少等待时间需要解释。
(c) 设计窗口大小的单因素实验，说明如何交替运行不同窗口配置，以及应观察到怎样的在途数据量、ACK 与发送时序，才能支持“窗口限制了传输速度”的解释。
(d) 使用配套音频记录，画出一个块从生成到播放的各个时刻，说明如何区分生成不及时、传输延迟与本地调度等待。

**12-8　上行带宽与故障恢复如何改变跨地域部署的时间和能耗〔延伸，★★〕。**

(a) 复算剩余二十轮任务在三种部署方案下的时间与能耗，再求云端按期完成所需的上行带宽，并说明为什么上行再高，云端也追不上附近工作站。
(b) 改为剩余十轮、期限 23 秒，选择按时且能耗最低的方案；再把期限收紧到 15 秒，重新选择。
(c) 二十轮交互中，十轮的上行速率为 6 Mbit/s，另外十轮为 10 Mbit/s，求云端的总时间，与恒定 8 Mbit/s 比较并解释差额。
(d) 对十轮任务、期限 23 秒，设第八轮刚执行完就发生连接中断，恢复连接需要 2 秒。分别考虑两种恢复情况：已保留前七轮的进度，只需重做第八轮；前八轮的进度全部丢失，需要全部重做。计算两种情况下完成剩余工作所需的时间，以及整项任务的总耗时。

本小题仅假设远程服务发生断连，端侧方案仍正常执行。远程服务在断连期间保留运行状态，连接恢复后继续使用，因此不必重新准备；任务进度则按上述两种情况分别恢复。重做的轮次要再计一次模型能耗。重新选择满足期限且能耗最低的方案。

**12-9　端侧设备的带宽、容量与能耗预算〔延伸，★〕。**

(a) 四条 x16 LPDDR5X 通道、每引脚 10.6 Gbit/s，求合计带宽；再求 Qwen3-8B BF16 单请求 8K decode（每步读取权重 15.14 GB、KV 1.21 GB）的每步时间下界与 token/s 上限。

(b) 权重按 q4_0 量化（每 32 个值从 64 bytes 变为 18 bytes）后，重算每步下界与 token/s 上限；在 12 GB 内存、预留 4 GB 的手机上，求扣除权重后剩余的空间、可容纳的 8K 请求条数，以及单条请求上下文的最大 token 数。

(c) 每 token 能耗取 0.576–0.756 J、持续功率上限 13.8 W，求仅由功率约束给出的 token/s 上限范围；分别与 BF16 和 q4_0 的带宽上限比较，指出哪个约束先起作用。

**12-10　丢包、重传与编码冗余〔延伸，★★〕。**

(a) MSS 1,448 bytes、RTT 0.2 s，分别求丢包率 14% 与 3.6% 下的 Mathis 上界，以及 354,640 bytes 在两个上界下仅发送所需的时间。

(b) 245 个报文各以 14% 独立丢失，求期望丢失数。每轮用一个 RTT 做理想的选择性重传，重传的报文仍以概率 $p$ 再丢，于是每个报文需要的发送轮数服从成功率 $1-p$ 的几何分布，全部 245 个报文在 $n$ 轮内送达的概率为 $(1-p^n)^{245}$。串行预算为 238.5 ms，每多一轮加一个 RTT；求完成时间的期望值，再取轮数分布的第 99 百分位得到 p99。

(c) 99.9% 成功率下，两种丢包率各需多少修复包、冗余比例是多少？求加上冗余后的完成时间，并与对应的重传 p99 比较。

(d) 说明冗余比例为什么应随测得的丢包率调整，而不是固定取 25.7%：从达不到成功率目标与浪费发送带宽两个方向说明理由。

## 本章小结

完整交互应从三个层次分析。先根据数据量、计算量和传播距离求出各步骤所需时间，再根据执行依赖确定哪些耗时需要相加、哪些步骤可以重叠，最后加入连接、窗口、竞争和恢复来解释实际等待。图片的 12.8 秒完成时间、语音的逐块递推和 Agent 的逐轮累积，分别展示了这三个层次如何配合。端侧本身也用同一套方法展开：内存带宽给出每秒 token 的上限，容量决定能容纳的模型与上下文，能耗与电池决定能持续多久。

模型分工改变了跨设备传输的数据形式和次数。把计算移到端侧，可能把紧凑图片换成更大的特征；把模型拆到多个设备，可能把一次传输换成逐层同步。比较时先扣除两种方案中相同的耗时，再比较节省的计算时间与增加的通信时间，便能找到收益来自哪里。

复用把一次准备的开销分摊到后续调用。迁移、连接预热、图编译与状态保存，都可以根据准备成本和每次节省的时间，求出至少需要使用多少次才划算。预期节省的时间或成本，还要留出一部分来应对波动和恢复带来的开销。

设计实验时，也可以沿用同一套时间模型。固定输入和计时的起止点，记录各阶段的等待，再逐项改变设置，检查总时间的变化能否由具体阶段的变化解释。Queqiao 的基线修正展示了这一过程：调优消除大部分额外等待后，进一步改善性能就需要缩短传播时间。丢包也不一定意味着拥塞：在随机丢包的路径上，用丢包反推带宽会低估可用速率约三个数量级；直接测量带宽，再按测得的丢包率安排编码冗余，才能把完成时间压回串行预算附近。

最终选择先满足质量和期限，再比较成本。即使平均计算速度和传输速率足够，任务仍可能因慢速轮次或丢失进度超期；按时完成的概率与恢复后的重做量，因而是部署模型的组成部分。

回到第 1 章提出的模型应用：开发者通过上下文表达任务，执行系统据此安排计算、保存状态并传输数据。现在，保留一段历史、增加一个候选分支、提前准备工具环境或迁移一次会话，都有了可以追踪的资源代价。理解这些代价，也就能够识别哪些通用执行开销可以利用应用信息消除，并根据任务质量、完成时间和总成本选择实现方式。这是学习 AI Infra 带来的系统设计能力。